LARGE MODEL · INTERVIEW NOTES

大模型应用 / 算法
学习路线 · 八股 · 面试实战

完整版资料汇编

内容构成 学习路线 · 核心八股 · 项目实战 · 真实面经

覆盖方向 Agent / RAG · AI 工程实践 · 后端服务化 · 算法进阶

整理日期 2026 年 11 月

全文约 101.1 万字 | 含 186 幅配图

目　录

点击目录条目可直接跳转至对应章节

从这里开始：目标、路线与学习方法 5

1 前言 5
2 学习顺序：应用起手 → 算法进阶 18
3 大模型岗位介绍 21
4 学习路线 35
5 答疑 36

应用核心：Agent 与 RAG【主线】 40

1 提示词工程 40
2 大模型参数设置 42
3 大模型幻觉 44
4 ReAct 48
5 Function Calling 52
6 MCP 53
7 Agent的记忆功能 63
8 Multi-Agent 架构 69
9 A2A 76
10 Agent开发框架 77
11 Agent评测 97
12 Agent健壮性设计 131
13 Agent项目上线问题 138
14 Agent场景题 138
15 面试官压力项目应对方案 142
16 RAG 152

AI 工具与工程实践【主线，AI时代素养】 201

1 Claude Code 从入门到精通 201

项目规范 205

2 Vibe Coding 239

5 Loop Engineering & Graph工程 289
6 Computer Use[按需学习] 301
2 面试真题 316
3 Python 的异步方法是如何实现的? 320

应用开发项目与开源架构解析:[理论学完后学习] 321

1 提升简历通过率，做深项目 321
2 智能预约Agent项目 353
3 模块化RAG项目 357
4 ForgePilot Agent项目 390
5 开源框架解读 - PI-Agent 391
7 开源框架解读 - Hermes Agent 434
8 开源框架解读 - ClaudeCode 453
9 开源框架解读- DeepSeek Harness 497

模型知识补充【应用方向掌握必要广度】 518

LLM模型训练 518
2 Transformer 519
3 Bert vs GPT? 521
4 简单介绍一下千问模型? 521
5 简单介绍一下Gemini3模型? 522
6 模型和硬件 522
7 不同模型的对比和选择 534
8 模型微调基础 549
9 模型推理加速基础 556

面试真题与求职技巧【准备开始面试前参考】 561

1 项目使用和简历编写 561
2 简历编写技巧 574
3 简历投递技巧 589
4 自我介绍 594
5 反问 594
6 面试准备建议 595
7 项目难题与面试复盘 597
8 真实面试过程记录（面试前突击技巧，简历投递方法等，我的真实经历供大家参考） 600

10 上岸反馈：下一个就是你 780

算法理论与项目【可选进阶学习路线】 783

1 学习路线确定 783

2 Pytorch/深度学习入门 785

3 后训练学习 786

4 开源项目学习 787

5 后训练算法项目 787

6 SKILL资源 789

专题阅读与文档维护 790

1 Agents 791

2 更新记录 797

从这里开始：目标、路线与学习方法

快速直达：智能预约Agent 项目 | ForgePilot Agent项目 | RAG 项目 | 算法后训练项目 | 公司面经 / 面试真题

说明：

1. 因为一份文档最多授权500人，文档是通过更新机制更新的。更新期间：您可能看到文档内容情况，或者内部跳转链接失效（具体表现为：需要申请其它飞书文档权限）。**大家购买过文档后，不需要申请任何其它权限！**遇到这种问题，通常是碰到了文档正在更新，一般20分钟后会恢复正常！如果20分钟后内部跳转链接说没权限，请再联系客服！
2. **由于更新机制，您在文档内写的评论，更新后会被抹掉。**有问题可以在我们小红书粉丝群提问（您购买的时候客服会给你发进群二维码，如果没有进群，可以联系客服再次拉您，在里面交流，反馈，提问都是可以的哈）
3. 文档内容提供了一些**Youtube资料，github链接都是需要vpn（翻墙的）**。翻墙意味着国内会屏蔽一些国外的网站（包括youtube, github,google等等），如果需要访问，需要走vpn。 一般公司会提供翻墙软件，如果您没有翻墙办法，可以使用一些付费app。比如狗急vpn（这个我用过。30多块一个月，手机电脑都可以使用），还有很多其它app, 方法，大家可以自己查查。
4. **关于提供了很多Youtube 视频资料**，因为确实很多讲的好的内容就是在外网，这些资源是英文的，如果阅读有困难，请大家使用**YT自带的翻译功能，转成中文 + 可以使用YT视频文本提取功能**，把视频文字喂给AI，一边学习一边问AI，帮助你理解。现在AI翻译，辅助学习功能很强大了，我想语言应该不是问题。
5. **文档是没法复制的，大家是阅读权限。**原因是如果大家是编辑权限，可以直接下载，倒卖还是很严重的。这一点请大家理解。不过现在飞书文档已经集成了AI功能了，有问题可以问豆包，他都能帮你把里面某个图抠出来。
6. **文档会越来越多，我也在重构目录，尽量让他更清晰。**大家多看看1级目录，也可以问问豆包怎么阅读，有找不到的地方，或者觉得不够清晰的地方也可以反馈到群里或者客服，我们也会尽量根据大家意见，让这个文档更加利于阅读。

1 前言

1.1 笔记/路线设计思路

大家好啊， 我希望设计的这份资料是：**自学入行大模型应用开发的最全的记录。**

为什么是自学路线?

我是从0开始自学的，自学了200多天，面试了2个月，拿到6个Offer（夸克千问，京东（算法岗），SAP（世界500强外企），网龙，平安证券，华林证券），所有信息小红书可查。

这份资料是按照**我自己的自学路线**整理的，我希望把我完整的**自学路线**，**心得**，**笔记，面试重点，**都分享给大家，让自学的人少走弯路。

笔记包括：**详细的大模型八股**，**知识点参考资料（视频，文章），讲解视频，项目实战（源代码，设计思路，讲解视频）。**项目实战是可以直接写在简历上的。

我的所有笔记的来源，是可以在我的小红书笔记中看到的，都是我根据面试总结出来的原创内容，而不是随便搬运网上的内容，这个真实可查。

为什么是最全记录?

因为我在转行路上，**所有的心得，笔记，复习资料以及所有的面试记录都记录**在这里面。这也是我自己学习的资料，我在面试中**发现的任何**问题，我都会补充在里面，分析出题频率，复习思路，面试的最真实完整的记录，还包括岗位介绍，面试反思。从0开始到上岸，这份笔记记录了我**所有的学习资料和心得感受.** 这份笔记会更新到我真正入职新公司，真正转行完成。到目前为止我也在一直不断地更新这份笔记，一旦发现问题，就会补充迭代，这份文档会越来越完整。但到目前为止，其实内容已经**非常详细和完整**，对于一般的面试都足够了。

适用人群

自学进入这个行业的社招同学和校招同学。 大家完全可以先按照这个路线和笔记去学习，我后面更多更新的是一些更**高级，更深入**的进阶技巧，支持获得**更高薪资**的岗位的一些技巧和知识。并且整个笔记我都想告诉你的是，哪部分最重要，这部分为什么要学，这部分怎么学，除了知识部分，我更想告诉你的是**学习的思路和方法。**

为什么要从应用入手?

对于转行或者想进入这个赛道的同学，大模型应用相比**大模型算法/Infra, 大模型应用是最好入手的。**大模型算法/Infra 技术难度相对更大，需要你对**深度学习/机器学习** 有比较深入的了解， 同时也需要比较高的**硬件资源**来支持你完成相关的实验和项目， 同时很多大厂在算法这一块对背景的要求同样更高，倾向于有**论文/有竞赛的背景。**所以，如果你是想入行，从算法/Infra上入行，明显难度更高，从进入这个行业的角度来说，从应用侧切入是更好的选择？我的项目是面向于应用的，应用的笔记非常详细。但是同时，针对于应用的岗位经常也会问算法，我的整套学习路线包含了一些算法：微调/模型推理/压缩（量化） 的知识。对于大模型应用岗位来说，这个足够了。这样的设计让大家既有广度，又能抓住重点。

对于很多同学，你的目标如果是入行大模型，那么其实你从应用入手，把算法相关的东西过一下，你就可以去面试，不用掌握特别复杂的深度学习，前后训练相关的算法知识。不过，这份笔记也会包含更进阶的内容，就是面向想要挑战**算法、算法工程**相关岗位的同学们，这份笔记也会包含我学习算法的相应内容，也是我正在更新的部分。学习算法，一个是有利于提高薪资，算法的薪资明显高于应用。另外其实也有助于让我们面试的时候有更多的选择。我的建议仍然是如果你没有算法基础，你是纯开发，甚至是转行，先学好应用（我自己也是学了大半年，在应用方向拿到6个offer后，我才开始慢慢学算法），虽然我们文档也会记录算法内容，但是请注意自己的目标和学习重点，当然我后面具体学习路线会说的。

路线/笔记的设计思路

1.提供可写简历的实战项目

我在做一个RAG-Agent项目，这个项目也是我自己用来学习，准备写在简历上的。

我用了很多方法帮助大家理解，掌握这个项目：**提供详细的项目设计文档（看目录的项目实战），**提供讲解视频，将代码分成几十个commit的节点，每个节点完成一个小的事情。这些做法，都是能够帮助大家更好地掌握项目的。

另外这个项目有通用性，支持扩展，可以针对于每一个人的具体业务去展开，后面也会给大家提供这个项目如何写在简历上等方面的参考。

目前这一套项目是已经完成了的。除了项目之外，在readme中写了完整的思路，就是不同的人群如何去使用这个项目。

项目中包含的如何完全用AI完成这个项目，如何用AI去写简历，如何面试前突击RA进行模拟面试，项备，如何用AI一键setup这个项目，我都写成了SKILL。这整套方法其实也很值得学习，学了以后你会知道自己如何写一个新项目，可以完全复用这整套方法。

目前已经有人拿这个offer去面试，拿到了暑期实习的offer，相关内容可以看我小红书笔记查证。我自己也会去用这个项目面试，后面会变成面经形式补充进来。

目前这个项目后续的规划是扩展更多的面试真题。

另外非常可能的是，我们后面会有一个算法的项目。我目前在学习算法进阶路线，后面肯定要弄个项目，所以整个过程也会记录，到时候会有算法项目。具体算法学习其实你可以看我小红书笔记，我目前在学pytorch，学完所有的路线我都整理好，大家照抄。

2.完全面向面试知识。

这份笔记目标非常明确，分为**知识/八股部分**和**面试实战部分。**这两部分都是完全面向面试的，完全是根据面试的重点来总结的。

知识部分都是面试内容总结的，面试出现就总结，不出现就不总结。知识部分适合自己根据学习路线和**知识/八股** 去总结，理解，学习，背诵八股。

面试实战部分都是完全真实的面试题目，**而且都比较新，标记有时间。** 我设计这一部分的思路是：

1.完整。 面试部分的记录是我根据我自己面试录音一个字一个字扒的，保证完整真实。

2.包含岗位jd 和反思。 每个面试我都收录了岗位jd，因为我希望大家可以通过岗位和知识点去推测和关联出一些重点，体会不同的岗位要求可能会面试什么内容。 反思部分是我自己面试完的一些感受，供大家参考。

面试真题也在持续总结，比如我25年12月面了十几家公司，以及笔记里这个项目粉丝投稿的真题，以及我26年面试的内容，都会补充。

3.弄清楚什么是重点

我特别希望告诉大家的是什么是重点？为什么要复习某一部分，因为我觉得自学的同学最困惑的是抓不住重点，资料那么多，要学哪个，为什么要学。

所以我设计资料的时候，**基本在每个笔记开始前会写一些注释**：告诉大家这个知识点出现的频次，是不是必考的。 有的知识点是针对于我个人情况而问的，我也会告诉你，让你酌情参考。 有些地方我也设计的是一些拔高的，不是面试官问的，但是是我自己准备的一些亮点，是靠我自己引导面试官去问，然后借助这个契机给面试官讲解从而加分的。 每一部分资料是针对于什么情况设计的，我都会告诉你，希望你复习的时候做到心中有数，也结合自己的情况针对性复习。

另外在每一个面试实战的视频讲解和笔记中，我也会告诉你什么是重点，面试官为什么问这个问题？我为什么这么回答。 除了笔记部分，我特别教会大家的是对考察重点有一个自己的判断。

4.每个面试实战内容都会有视频

每一份面试实战部分，都会对应视频讲解。视频讲解会讲解知识点，更多的想和你分享面试的技巧和感受，以及对重点的把控。 这个讲解视频我会按照顺序发在我的x站视频账号中，预计一周1\~2更，请大家放心，笔记里出现的面试实战内容x站就会有对应视频讲解。

5.行业新知识

> 我会持续关注行业的动态。比如这份笔记是25年12月上线的。但是基本到现在（26年3月），我是一直在几乎每天更新的。除了上面的内容外，我会关注行业知识，比如25年年底才推出的SKILL，26年初火起来的Openclaw。这些都有总结。我会关注行业动态，行业里什么要考，我们会一起讨论，我都会持续跟进。

6. 其他内容

那可太多了，我可能列不完，因为基本上我觉得对大家有帮助的，对我有帮助的我都记在这个上面。所以你看除了我上述提到的内容，笔记还有**大模型岗位介绍和学习建议，粉丝的提问和答疑**（相信有朋友有共同的疑问，行业困惑，所以我汇总了大家都能参考），**论文导读**（读论文也很重要，即使是开发岗，原因在论文导读一栏写了），**VibeCoding 推荐视频**（其实这个适用于所有人，包括产品，新时代如何用AI编程，未来如何发展），**面试技巧**（简历如何写，自我介绍怎么介绍，反问问啥）

总之是在学习路上，我认为有帮助的都会总结，未来也是，我很难说会有啥内容，但是在我学习，了解行业的时候，只要我认为对大家有用，就会补充进来。

7.配有参考资料

我会整理学习路上**自己看的视频，文档，参考资料。**都会以内嵌链接的形式推荐给大家。所以笔记内容很丰富，除了我整理的文字，还有很多视频和推荐书籍，论文。

我会提供给大家一些我学的SKILL。特别是算法部分。因为我在学算法的时候，其实是26年了，26年SKILL出来以后，我就觉得特别好用。所以在算法这个部分，我很多时候依赖于SKILL，比如我学习Pytorch的时候，我写了SKILL，里面包含了我推荐的视频，编写的教案，相当于你用它就可以直接按照我的路线，让AI给你讲。在算法部分，我想是提供各种SKILL，Pytorch入门教学SKILL，前后训练SKILL等等，这些是我的学习来时路，分享给大家，大家在算法部分学习方向，资料方法就特别完整了。

8.优化和迭代

我希望能以最好的笔记，最优质的内容呈现给大家。请大家购买后可以进入这个小红书笔记交流群，可以交流一些笔记的内容，重点，疑问，我会尽量回答，也欢迎大家讨论，我会根据大家的讨论，后续也会补充一些内容和心得，我们一起完善这个笔记。文档是开放评论权限的，大家可以提问题和讨论，有问题我都会fix。但是注意的是因为这个涉及多份文档的拷贝（一份文档最多500人），所以我在用自己写的拷贝脚本拷贝的时候，评论会被冲掉，还请理解，如果提了评论我没处理，可以多提几次。

1.2 大模型应用/算法学习路线

在制定学习路线的时候，每个人的情况：校招，社招，学校层次，工作年限，之前从事的方向，给自己预留缓冲的时间是不一样的。所以我希望先讲清楚思路，原理，大家在参考的时候也需要根据自己情况调整。

首先是先看岗位，具体可以看笔记：**大模型岗位介绍部分。**粗略地说：大模型分为算法工程师（前后训练），推理工程师（负责模型推理），应用开发工程师（做agent,rag,垂类应用），数据工程师（准备数据）。但是，现在的趋势是：岗位之间倾向于融合，交叉。

比如算法工程师：很多时候要自己做数据。我们公司的同事，我问他做啥，他说是做模型训推，也就是推理，训练都做，同时算法岗位也会要求了解Agent,RAG.

比如应用开发工程师：Agent,rag这是要专精的，但是很多又会涉及微调，推理框架的。所以现在的趋势是：岗位对于人才的要求是复合型的，多少大家都得掌握。所以这一点告诉我们求职大模型需要广度。

但是大家也不用慌张，不用害怕企业要求这么多东西，我怎么能学会呢？大模型太难了吧？你要知道的是，企业确实要求这么多，但不代表你一定要把这些都学得非常好才能拿到offer。大部分人求职不会要求你又广又深，如果真的是每个都掌握得很好，年薪肯定100以上了。所以不用太害怕，你需要广度学习，但是第二个请大家记住的是，你也需要抓住重点，选择自己合适的路线。

比如上面我所说的，建议大部分人是应用入手，深入应用，了解算法，这样去求职，入行。进阶的话，就是算法也学深一点，然后做算法工程，甚至算法。或者先学好了应用，先入职了，再慢慢转。那你一上来的路线一定是先学好应用啊，做几个项目啊，别在pytorch，推理框架，前后训练那些算法的原理，数学公式上纠结了。我的实际经验是，你把算法的流程走通，微调怎么做的弄清楚，甚至没有真的做，然后了解相关概念，面试官大概率会问你怎么做的微调，准备下相关问题，笔记里写的有，目前这样就够了！你去面试，肯定有offer的，如果你还有时间，那可以去学Pytorch， 手写attention， 微调，你的路线是这样的，这也是我的路线。原因是门槛低，另外我本身就是开发路线的。

另外如果你是算法出身，本身就会pytorch，训练微调都做过，那我也是建议你直接转算法啊。那么训练算法，论文就是你的强项，你要深入学习的。当然你需要了解Agent,RAG, 这是你的加分项。

所以我们也要找准方向，你的优势，路线是什么。下面我讲学习路线的时候，是一张大网，你需要根据你的实际情况去调整的。我真的不希望一个小白，刚入行大模型，agent还没学会，就迷失方向，去手撕attention，看深度学习的论文，然后被劝退了，这样我会很伤心的。

我把大模型学习路线图分成以下几个部分，然后具体讲解。

Agent, RAG, 后端知识，前后训练，模型推理，行业素养，项目。

Agent:

1.**python基础**： 如果完全没有Python 基础，需要先学语言，找一些0基础快速学习Python 视频（x站搜：python入门零基础），不用过于较真语法，会用就好。因为实测面试上考察python 具体语法的并不多，这里学习是保证你看得懂，改得了代码。

2.**入门了解Agent**：你需要对Agent 特点，怎么做的有一个印象和概念。可以结合一些Agent的入门介绍视频，比如Tina老师的介绍视频：https://www.youtube.com/watch?v=qU3fmidNbJE。

清华大学出版社的《AI Agent开发与应用 - 基于大模型的智能构建》这本书质量不太行，但是好处是特简单，容易读懂上手，也有一些很简单的Agent的例子。也可以通过这个例子去上手。

了解Agent是啥后，可以跑一个100行左右的Agent代码，调用通LLM， 来感受一下这个过程，比如你跑跑这个例子：https://zhuanlan.zhihu.com/p/1922809649868022076

3.MCP/Function call： 了解这俩是啥，常见概念，可以对照我笔记去看。依然推荐Tina老师入门MCP 视频:

https://www.youtube.com/watch?v=kOhLoixrJXo

其实YT有非常多MCP 很好的视频, 讲得很清楚：

爬爬虾老师讲得也很好：

https://www.youtube.com/watch?v=McNRkd5CxFY

里面介绍了一些MCP 的功能，可以上手去试，去感受。

4. Agent框架。 最常见的是Langchain三件套：LangChain, Langsmith,Langgraph，如果没有倾向，就学这个，用这个面试也会问这个比较多。这个网上视频就很多了。

比如入门视频：

https://www.youtube.com/watch?v=1bUy-1hGZpI

不过这里推荐大家稍微学深入一点，可以在YT里找相关视频。

一些典型低代码平台，扣子，Dify可以去了解下，虽然很出名，但是我面试感受下来没怎么问到，可以知道下概念。

5.Agent评估方法，很重要。我的笔记里有整理，这个常考，所以你要熟练掌握，具体八股部分 Agent的性能评估里以及里面推荐的参考资料，必须一些Agent的论文（比如美团龙猫的评估），这些都可以作为Agent评估的学习资料。

6.Agent设计范式。 直接推荐了这本书，把它放在学习路线上，因为它涉及很多面试考点，然后有一些知识，比如React，我路线没有，它也都有补充，推荐大家阅读，这个网上可以免费看。顺着这本书去读，它其实就串起了Agent应用的各个部分，记忆啊，React啊，跟着它的思路，把Agent的技术掌握了，可以顺着书，一边读，一边搜一些资料。

https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn

7. SKILL. SKILL很火，已经是必考的了，是26年火起来的。常见的问题：是什么，怎么用，你用SKILL做了啥。我们的笔记SKILL章节整理得很全面，本身并不难，怎么学习在SKILL章节有详细介绍和推荐视频。

8. 提示词工程

看Tina老师这一个视频，把它消化好就行，**思考清楚提示词工程涉及原则和常用策略**，注意应用到你的工程就好。面试考得少，看完这个视频知道推荐的提示词怎么写，有什么常见策略就行。

https://www.youtube.com/watch?v=p09yRj47kNM

9. 复杂的Agent 实战，比如我说的书《AI Agent开发与应用 - 基于大模型的智能构建》，就有很多的实战例子，可能代码会长一些，跟着写，跑一跑。或者自己找个复杂点的Agent项目，去跑一遍，感受一下。也可以看一些经典的Agent的例子，比如说Openclaw（笔记里有讲），本身也开源，包括我提到的Claude Code 25年举办的Hackathon，看一下这些项目Agent怎么实现的，思路是啥。

10. 理论上经过了上面的学习和沉淀，你已经了解Agent是啥，而且也让大家看了简单的，复杂的Agent的项目，理论上你就可以具备开发Agent项目的能力了。这时你该设计一个项目了，从项目选题，立意，用什么框架开始思考。项目设计就是可以参考openclaw，现有成熟架构，怎么设计我的RAG项目里的视频讲解有写，思路完全可以复用我们的RAG项目，学会了笔记里的RAG项目，你就按这一套来完成Agent项目。

RAG:

1.**RAG 入门体会**：还是可以看看视频讲解快速入门。

类似快速入门视频很多，大家去搜搜，用个几个小时去理解一下。

https://www.youtube.com/watch?v=WWdlme1EAGI

我自己也看了清华大学出版社《大模型RAG应用开发 - 构建智能生成系统》 说实话质量依旧不行，但好处是简单，看下来很快，也有很多例子。不过YT上也有很多人讲解的时候，附带有代码，你可以去找一下，照着它的视频和代码跑一下，入门RAG.

2.**向量数据库**：了解向量数据库是什么，常见的向量数据库，为什么RAG要用向量数据库?

https://www.youtube.com/watch?v=gl1r1XV0SLw

我自己还看了这本书《大模型RAG实战-RAG原理，应用于系统构建》 这本书质量可以，可以买，这本书vx读书好像也能搜到。很多面试内容后面我都从这本书上去查。不过第一次看可能有点难懂。这个书上也有一节讲向量数据库。

3.RAG的具体过程和策略:

这个就非常重要了。 整个RAG中涉及的各个步骤，你脑袋里要知道他们是怎么串起来的。 **知识库准备，切块，召回，召回的优化，以及效果评估。**这些每一步都很重要，面试最高频考点。

《大模型RAG实战-RAG原理，应用于系统构建》 这本书都有，我的笔记也照着这个总结的，不过大家可以搜搜RAG深入讲解视频，把这几部分搞清楚，这就是面试最常考的。

4. RAG新方向

Graph RAG 和Agentic RAG. 可以看一下我笔记的概念介绍，这是RAG的进阶方向。一般并不是所有岗位都需要特别深入的RAG的，所以这里看是否有余力深入学习，我个人没有深入学习。如果你需要从事偏RAG技术的岗位，那是有学习的必要，如果是普通Agent/应用方向，算法方向，那么可以先放一下，先把概念了解。

5. 项目实战

笔记中有一个RAG项目，就是把RAG的各个环节串起来了。有了上述的理论基础，相信你可以做出来这个RAG项目，就去参考笔记的RAG项目部分吧，可以作为RAG知识的学习扩展，也可以作为简历项目，进行包装。

后端知识

对于后端知识的要求。推理岗>>应用岗>算法岗。以应用方向为目标，最好还是知道一些后端知识。Docker, k8s, 容器化这些是简历Jd经常出现的。坦诚地讲，我现在也没学，而且也暂时没管它在学算法。因为我的目标是全栈，算法工程。这个不会也没妨碍我拿offer。 要学的太多了，你们看这个路线也可以看出来的，事情有轻重缓急，根据自己路线定夺吧。

对于后端这里我的思路就是，我不主动承认自己不会后端，我会在简历上写一点点后端的技术栈，你可以看我写简历的思路。 但我不会用过多的篇幅来写后端，我也不会系统性去学后端的知识，因为这么学下来，又得好几个月。我的策略是：当面试过程中出现了后端内容，我就根据这个知识点，把后端内容学透了。这也是笔记里组织后端内容的思路。我认为这样是一个性价比最高的方案。

前后训练

其实前后训练这个概念很大了，直接概括了算法同学的工作，里面水也很深。算法本身就是应该出个专门的学习路线，算法本身不同岗位差别也很大，有些算法岗主导创新，要发论文的，那种年薪几百万。腾讯招的ysy, 小米的天才少年，人家都是算法岗。我这里的路线还是以应用侧转行到大模型，有余力进阶学习算法的同学的路线来讲解。

1. 常见的概念：弄清楚前后训练，推理在大模型行业的流程，怎么和应用交互的，前后训练是干啥的，有哪些常见方法，微调，强化学习是干啥，有啥好处。
2. 了解面试问题：走一遍流程，不管是微调，还是强化学习，稍微了解下代码，假设真的走一遍，流程是啥，要哪些代码。

学习了a,b 我相信就能回答出来常见的面试问题：你是怎么做微调的，你是怎么做强化学习的。微调是啥。这些我笔记整理的都有。a,b 是所有人都要学习的，即使你没有走到算法进阶，学一下a,b 不会花多长时间。

这里我想说的是，学习a,b主要是为了，面试官问你：你了解微调，做过微调么？的问题的时候，你说了解，做过。注意这里不要露怯，因为你只学了这么多，你就害怕，主动承认说了解不深，没必要，很多面试官都深入问了。按照我们笔记里怎么做微调的方法，想好，就行。 如果真的深入问了，答不上来，再说了解不多。

好了，接下来路线是你要进阶学习，学好了应用，冲击应用算法，算法工程，算法的岗位，要学的。

1. **深度学习/pytorch**：我们不求深入，很多学生学深度学习是抱着深度学习书籍，公式学习来的。我们面向找工作，绝对没那么多时间。我们的目的就是入门，因为大模型网络是深度学习经典网络基础上发展来

的，所以我们要了解之前的传统网络，前向传播，反向传播的知识。 我们要学习pytorch，是因为这是我们做微调，训练的工具呀。 以快速入门为基础学习。

2. **现代大模型知识：**强化学习，SFT等大模型相关后训练方法，这个就不是上面a,b说的概念了，深入原理，pytorch代码，公式学习。
3. **开源项目/论文：**c,d打好了基础，那么e就是调研了，为我们实战项目做准备。调研现有项目，人家怎么做的，达到什么目标，怎么评测。
4. **实战：**有了上面基础，我们会做一个小的Project，真正去实现强化学习，微调，提升效果，做训练，最后结题写一个小论文，说明你做的事情，方法，评测，参考论文。
5. **简历包装：**有了实战(f）步骤，想想怎么写在简历吧，和你的业务结合。写好了要投递面试了。
6. **面试和迭代:** 其实这是所有方向都要做的。面试根据反馈，改进技巧，改进项目。

有没有注意到本节没有推荐任何视频。那是因为我正在学习！而且我会有更好的方式呈现给大家这个路线。我还在c阶段，马上要完成c阶段了。c\~h的所有阶段，我都会总结好最方便大家的形式去学习，包括思路，参考资料。有些地方准备以Skill的形式。 也就是说到时候c\~h我提供完整的学习教程，大家跟着做就行了。

推理

还是一样的，推理框架需要丰富的后端知识，以及模型推理框架，优化的知识。目前的侧重还是说掌握概念，大体方法。有哪些推理框架，剪枝，蒸馏，量化的概念。不会深入学习和实战。这些概念在笔记：模型推理加速 都有。

至于为啥不深入学习，原因很简单，时间有限。学完应用去学算法了，学完算法再学后端，学完后端再学推理。 这是我的路线。不是所有人都要都学完的，很多人基本是学完应用就能上岸了，总之根据自己情况慢慢来吧。

行业素养

我把这个专门作为一个考察的方向，其实它也很重要，这是在我不断面试的时候才慢慢总结出来的。它不是硬知识，问你专业问题。而是素养。比如面试官问题：你使用什么AI Coding工具呀？你有没有了解过Openclaw啊？你平时怎么用AI工具的呀？你看过哪些论文 吗?

这些问题看似不经意问对吧，之前有面试官问我用什么AI Coding，我说我只用过copilot，其实就很掉分，可能这就是默默给我没过的原因。

其实这些问题很重要，体现你的行业素养。比如用什么AI 工具，怎么用的？这是体现你有没有积极拥抱AI，用AI的技巧。比如你使用copilot ,claude code，它们可能各有优势，你有没有了解它们的优势？它们的进阶用法？没有的话，说明你这个使用AI的技巧不行。

你有没有了解Openclaw?如果你没了解，说明你没有关注行业动态，它作为Agent行业爆款应用，它有哪些特点值得学习？你竟然没去看吗？面试官觉得你不了解行业动态。

包括论文，大模型这个行业需要读论文的，即使是开发，原因我在论文章节提了。

所以我把这些归为行业素养，但它同样很重要，在面试中考察频率很高。

这块怎么学习，与时俱进吧，抓住动态。当然我自己一直在总结，比如新的东西，Openclaw的架构，SKILL， 论文这些都是在笔记中的，我也在实时关注，给大家总结，大家看我小红书，一起讨论，一起更新。

项目：

其实所有上述理论的学习，都是要有个项目。转行的人，有个项目你才能去面试对吧，你没有项目，你是做c++的开发，拿个c++简历去投大模型，清北的都没面试机会。

应用的项目：

我在RAG项目中非常完整地介绍了如何从0做一个项目，从立项到选题，到开发，到面试突击，你就看这个内容，而且它可以复用到Agent。

算法项目：

在笔记 ： 算法0基础学习全记录章节，会记录完整学习算法过程和怎么弄项目（注意在更新中哦，这块内容），大家直接参考就行。

所以项目就不多讲了。

1.3 问题说明

此部分总结群友问题。

1.为什么看上去强化学习这一块内容不多?

因为所有知识我都是根据面试总结的。面试考的越多，我就总结的越多，越深入。面试考的不多，就几乎不会出现。笔记是面向应用路线的，强化学习部分我自己的复习思路就是带一下，我只是了解了一个皮毛，然后做了一遍流程，加入了我的项目介绍中：我做了推理优化：量化-微调-DPO 之类的，做到这里就够了，面试官问我，我就讲这一套，并且面试官从没有深入问DPO的细节，所以对于强化学习没有深入笔记。

我希望笔记是面向面试的，是**精而专而不是大而全。** 如果是面向与应用岗位，去学习很深入的强化学习知识，甚至是数理公式，在目前看来（至少在面试的角度上来说），是性价比不高的。从笔记你也可以看出来，笔记篇幅多的一定是常考的，没有笔记或者占比较少的，一定是不常考的。

现在关于算法（微调，部署，强化学习，后训练相关）的这一块内容，你可以理解为是我在第一阶段去面试，当时是我以应用为主，算法只懂皮毛，满足面试基本要求总结的。我的建议也是第一个阶段算法不用学太深。懂我笔记里总结的就行了。

当然，如果你差不多学完了应用，开始进阶学习（像我一样，我是学了大半年开始，准备深入算法学习了）。关于算法的部分，微调，pytorch，强化学习，请你去看 算法学习专题。 其实这里面就蕴含了大量的八股知识，比如后训练是啥，SFT， LoRA。 我没有往笔记总结，是因为我希望笔记里的内容是面向面试的，我总结的都是面试常考的，根据面试题反推的。等我开始面试算法了，再会把算法的内容整理到笔记。如果你现在想学算法，可以按照 算法学习专题 的思路，把它学好。（但如果什么后训练你都会了，开始面试算法了，那么其实你学在我前面了，算法更深入的部分，笔记还没总结，进阶算法学习要靠你自己了（写于2026/5/8，不过我也在继续学习，也许你再看笔记的时候，笔记已经更新算法进阶内容了也说不定））

2.为什么没有多模态?

同上。 多模态属于应用部分，但是我面试下来，他不是硬性要求的。除非很少数的岗，本身做多模态，他可能会有点考察。以我的经验，没有多模态经验，不太影响面试，我个人对多模态这块是完全没经验的，如果面试官真要问我，我准备直接说不会。但所以这里确实完全没涉及。所以我不会多模态完全不阻碍拿offer，所以我这里没任何笔记，自己也没学。 当然如果你自己想做多模态方向，那么你需要自己补充下相关知识。

3.文档评论问题

文档开放评论，有问题也会处理，不过注意一下，因为文档有自动同步机制，大家看到的文档可能是副本（一份最多500人），副本是从第一份文本同步过去的，但是同步没法同步评论，所以你评论了，下次同步后评论会被清空。如果评论有问题，我没有修复，可以在群里说，我会修复。至于比如你用评论做笔记啊这些，就没法保留了，不好意思。

1.4 参考资料说明

在笔记的知识部分，比如Agent, RAG，相关内容，我是根据面试重点总结的。所以出现在这里的内容，你需要掌握，我也给了很多参考资料。但是值得一提，学习的时候，是要按照这个线索，去看我提供的资料也好，自己找资料也好，去理解它。有的地方我写得比较泛，更多的是作为八股的总结，就是让大家去参考背诵，理解的，而不是说你只看我的资料就能学会。比如我讲RAG，我是把最重要的要点总结成提纲，但里面具体细节，我推荐了书，视频，如果理解不了，大家自己要再补充一些资料。

另外，参考资料这一块，有的地方是Youtube视频，需要科学上网。

第二个，如果是英文，没关系的，现在下面开字幕，实时翻译成中文。

第三个，教大家一下，在youtube下面有字幕，点字幕，右边会有对应字幕列表，你把他复制到ai工具中，可以让AI帮你理解，一边看视频，一边提问。

第四，参考资料叫参考，不是必看。如果你看文档就懂了，那你就不用看参考资料了，有的地方，我列出参考资料是告诉你我做笔记的来源，不过我自己其实已经反复看，把参考资料总结到笔记里了，要点我都总结了，你看懂了笔记不看参考资料完全OK。

另外参考资料不是唯一，你根据对应知识点，去b站搜索，youtube搜索相关视频，你觉得好，给你讲明白了这个知识点，就可以，大家自己灵活判断哈！

1.5 经验之谈

对于有一些我找工作特别困惑，或者大家都比较困惑的问题，我总结在这里，希望对大家有所帮助吧。

1.5.1 项目如何找

相信找一个大模型项目，是大家自学最困惑的一点吧。只有有一个项目你才可以去面试。

其实这个分成**校招生** 和 **社招同学**来讲。 首先我觉得校招的同学，项目是最不用担心的，至少在我看来很容易。我是以社招，都工作4年多了，大厂的标准来给自己设计的项目，所以其实校招同学用这个笔记里的项

目，是够达到面试标准的。笔记里RAG的项目，刚好是今年2月做完，很多应届生都拿去面试了，而且你看面经和反馈，不乏很多大厂。加上我们笔记项目设计会越来越多和复杂，所以应届生没啥好说的。应届生最合理的一个曲线是：

先去找实习，后面再秋招。所以实习完你自然会有一个企业级的项目。这个很重要，企业级项目含金量大于自己做的项目，所以校招比较好整。真没找到实习，用笔记里的多个项目，算法+ RAG + Agent也行了。

所以我重点谈一下**社招。** 我说的经验都是个人真实经验，而且比较新。大概适用于P7以下，因为我基本对标P6+ P7. 如果说你是P7以上，我也不说了，企业要求肯定更高，我不能盲目给经验。

至少在我看来，我用了一个我现在觉得有点稚嫩的项目去面试。简历其实没太好好写，实测是有挺多面试的。包括现在26年5月，最近有一个国企找我面试，我给的还是之前的简历。 3月初也有大厂，字节啊，阿里啊，直接是业务部门打电话，就是说是通过了HR+业务的筛选，就是可以直接去面试的。所以从这个角度来看，我项目不复杂，但还是有很多面试机会！

但请不要觉得项目不重要，面试其实是一个综合的考察，因为筛选的时候考察学历，考察大厂，可能是因为我的背景，所以企业会放一些水，让我有面试机会。 如果你是双C9以上+大厂，你会比我更有优势。另外如果你有后端经验，应该也会更容易，因为技术栈更匹配点。所以投递简历是否通过，他是一个综合考量的过程。像GAP啊，学校背景啊，过往经验（大厂，方向）都是有影响的。

另外项目肯定是要往深了做，我只是说我项目最少去年12月份，我简历连笔记的RAG都没写，确实面试通过很多，这是实际情况。但是我当时那样，是因为我能力就到哪里了，我设计不出来复杂的Agent项目，我没有做RAG，算法也没做，没办法啊，那时就学习了6个月。但是有能力，希望大家尽量设计深入。

如果你想自己做，你可以参考我笔记里的方法，写了很多。

如果你想借用，参考，也是个好主意，我觉得你做了一个Agent，你把我的RAG项目加进去呗，或者假如我马上做的算法项目，说你用了本地模型，做了量化，SFT + 强化学习。 这也是个好方法，是在我的基础上优化，加深，取其精华。

不过也有同学就想拿项目直接用的，不想自己做。怎么说呢，我也不推荐。校招生赶时间的倒是有这么去干的。我感觉拿去直接用，很难掌握得特别细致，细节啊设计啊，这些都得自己体会。我不知道如何建议这里，因为我都是自己做的，甚至Agent，我马上做都是第三个项目了。如果时间紧走这个线路，那么我建议啊，你是找一个项目尽量深入的，文档齐全的，配套各种资料的。 如果要用笔记里的项目，我能做的就是做好配套设施，比如提供面试真题啊，讲解啊，答疑这样的。

对于社招同学，我的建议是，项目设计深入（这是废话。。）这需要一些经验，比如你怎么设计，什么值得设计，哪个模块必须设计。我在笔记里讲解开源项目的时候，包括Agent项目 项目设计思路。 谈的都有。这需要一些经验，你需要看了很多开源项目，才知道Agent项目好在哪里，有哪些模块。你只有面试了，你才知道面试官关注哪些项目的点，这些都是经验，我觉得我贯彻在了整个笔记中，如果看一下，你一定有一些答案。

在自己最大的能力范围内，弄一个最复杂的项目吧。 也不用特别担心整个事，一开始就要一个企业级的项目，因为有的公司，倒是比较看重潜力和背景，包括普通开发转Agent开发，你的之前的项目经验，工程经验他也是认可的。特别是像大厂，字节，阿里这种传统互联网大厂对于你的项目要求（P7以下）还是挺包容的。他更看重你的潜力，你的工程能力，你面试的表现。 但确实也有公司很在意落地经验的，像那些AI公

司，六小龙之类的。 也有公司看重学历和大厂，比如银行啊，国企啊。他们技术估计也一般，学历背景好，面试表现好，就能过。

有时候能力达不到，用一个太复杂的项目其实你也驾驭不了，因为确实你知识体系，架构设计，理解上都不够，给一个复杂的项目，真的能驾驭的了吗，所以我说**在能力范围之内，弄一个复杂的项目。**

另外面试被挑战项目简单，我觉得太正常了。看笔记的要不就是校招生，要么就是转行的社招，也很难说有个什么特别难的项目。我的解决思路就是：

1. 尽量做深入项目（比如我又要重新设计一个更加复杂的Agent）。
2. 广度加深。 我的规划到时候我RAG, AGENT, 算法都有项目，或者一个项目又做算法，又做Agent，又做RAG。
3. 其实项目是一方面，取决于你能不能面试。我们要做的还有很多，就是你面试的表现，你的知识的理解，表达，还有该死的八股，我们需要在方方面面做得好。如果你项目弱点，其它方面就争取弄好点呗。

其实，我多少也有点担忧，尽管我天天学习，但是我目前还是不是专门做大模型的，都是自学，也会让我有点焦虑。不过焦虑没有用，我最少知道，我在进步的，我算法项目快有了，Agent年底要做得更复杂了，各种理解也加深了。这是今年，如果让我明年再学一年，明年底是啥样，我也不知道。 当然你可以骂我，我永远都在闹着玩，工作做大模型才是专家。 我不知道，我无法反驳，反驳也没意义，我会定期检验自己，面试是很好的方法，至少25年12月份，我觉得成果很好。最近我也有一点面试，猎头邀请，我没主动投，等我做完Agent项目，估计又是今年12月左右了，我再去面试，让市场检验吧。

另外社招建议是往工作场景上编和包装，大不了说没上线呗。但是我当时投递，那个智能预约项目，一看就和工作无关，反正还是写上简历去面试了。

最后说说项目规划。

2025年做了两个简单的Agent项目，笔记里开源了一个，是用来找工作用的。

2026年2月做了RAG，笔记开源了，

2026年6月，规划做算法，笔记会开源。

2026年8月，做复杂Agent项目，笔记会开源。我会用我学的功力，Harness思想，开源项目架构，想一个新颖的点，做个项目，我觉得这个项目是笔记里最厉害的项目（虽然还没做）。

做完了，我就去包装，面试。我觉得你做了一个项目，包装啊，迁移很容易。比如把算法训练做一遍，我就是包装到各个Agent之中，和业务场景包装，很容易。但我得先把技术做一遍，了解怎么做，再包装。分两步。所以现在我要做的就是做完项目。做完了就是**包装+面试+反思+迭代改进** 循环。

总之，关于项目，笔记蕴含了大量的**思路，包装技巧，项目本身也供大家参考。**希望大家站在我的肩膀上，利用这些资源，让自己做得更好。

1.5.2 面试内容复盘小技巧

分享一个技巧和提示词，是我用来，面试完把录音文件对应的txt转成相关问题的提示词。具体方法是，使用该提示词+AI可以快速完成面试内容提取，个人使用CC+Opus4.8以及Codex + GPT5.5都运行很好。其他ai

和model理论上也能运行，任务不是很复杂，提示词放在这里。

---

2 学习顺序：应用起手 → 算法进阶

因为自己是从0开始学习大模型的，所以我太知道0基础学习大模型的心得和建议。我根据我自己自学1年心路历程，踩到的坑，总结起来，给大家推荐一下学习路线。这份文档是我学习1年了可以说所有的学习内容，也一直在继续学习和更新，文档会越来越大，这块的内容也会定期更新的。

我以0基础大模型基础为例讲解。

> 先了解一下笔记思路和大模型岗位。 [笔记/路线设计思路](#doxcnmCmcZSAW8yXiupwhYdx8me)， [大模型应用/算法学习路线](#doxcnRbMhJzPCjsqYGz5UDO130f)， [问题说明](#doxcnLJcQsOLmROdrg5YwQxR87O)， [参考资料说明](#doxcn7Yc23V6M9HL82SdS8bMppf)，[大模型岗位介绍](#doxcnVlHhV6vRMBDVYuuYo087cg)，包括我的各种小红书笔记。大概了解一下我设计笔记的思路，这个资料怎么用，这上面都讲的有。

我把学习分成两个阶段：**阶段一：大模型开发保底 。阶段二：算法进阶**

原因如下

1：**大模型开发是最容易入行的。**对于社招来说，有开发背景转到大模型应用开发，如果你之前还是做的后端，就更容易了。当然如果你是客户端，测试也没关系，我也不是后端转的大模型，我没有后端经验。所以学习大模型应用开发，是最快入行的。我个人经历是在2025年5月开始学，学到11月面试，面试了两个月，有挺多offer的，然后都没去。这个阶段我叫做阶段一。**我不想一开始就学习算法，深度学习，训练，本身更难，算法天然门槛更高，所以应用起手，先入行。**

学完了阶段一其实你就能找到工作，不一定要学阶段二，我亲身经历，我没深入学算法，已经拿了挺多Offer的。入行大模型了，工资也比传统开发高。

2. **学习算法进阶。**学算法的原因很简单，都想追求护城河更高，薪资更高的岗位。大模型除了应用，还有算法（做后训练，或者叫infra），部署。这些要求比开发高，薪资高。当然他们也有弊端，是岗位少。一般做infra只有大厂才做，竞争激烈。我把它叫做进阶学习。

从另一个角度来说，即使你只是想从事大模型开发，学算法对于你找大模型开发也有帮助，有时开发岗也会问部署的东西，算法的东西。包括大模型这个行业，其实算法，部署，应用，都建议懂一些，所以如果学有余力，学习算法也是有必要的。

这个阶段其实也是我现在的阶段，我在26年2月吧\~现在，在陆续学算法的东西，做算法的项目。

接下来按照这两个阶段，告诉大家如何使用这个笔记：

2.1 阶段一：应用起手

1. **知识理解**：Agent, RAG, LLM模型结构，模型微调，推理加速，AI通用素养。

顺着这些链接学习，其实这个笔记目前总结有一个天然的特点，就是按照阶段一要学的深浅总结的。你有看到Agent，AI通用素养部分，RAG部分写的特别多吗？llm模型结构，模型微调，推理加速写的少。这正是因为我这个笔记是纯粹面向面试总结的，根据面试内容反馈，来补充的。所以你应用起手的话，照着这个学，深度和这个笔记一样。应用的东西讲得很深入，而算法部分：模型结构，微调，推理加速，偏向了解概念，让你知道面试问到了怎么说，包括总结的如何做微调，如何做部署，里面总结了你如何不深入学习算法，面试官问道你也可以说你做过一些的策略。

顺着这个笔记学习，里面包含了很多的参考资料，和思路。也推荐了书籍。我又教大家如何用这些资料，请参考资料说明。 你顺着这个笔记学习，以弄懂笔记为目标，看不懂看参考资料，顺着学就行。这块全是面试要考的。

论文部分： 这块不多，总结了2篇。防止面试官问你有没有看过什么论文，分享一下。具体看了面试怎么回答，总结的都有。

2.**项目。**自己做项目或者找现成的项目包装。笔记里有：

RAG项目，有全套资料，面试真题，可以参考。

Agent项目。我之前用的找工作项目之一。

算法项目。作为拔高项，设计SFT, 强化学习，部署。进阶内容，如果你学会了，可以作为简历一小块来提高你项目复杂度和含金量。没时间不学算法当然也可以。

这两个可以参考和借鉴，可以直接拿来用。不直接用里面也有非常多的思路，如何做项目，怎么设计，项目设计哪些东西。

同时有一些开源项目供大家参考，笔记里也有解析：PI-Agent. 这个项目特别适合于，你学习完了以后，自己看一下Agent的小项目，学习，研究，改造，甚至是封装成自己的项目，因为这个项目又小又权威，而且给了丰富的扩展机制供你扩展。

这三个东西 Hermes, Openclaw, ClaudeCode。特别是ClaudeCode，笔记里有大量资料讲解，包括面试真题也在使用ClaudeCode讲解，这三个也得看，看一下行业最火的Agent怎么实现，对你的面试和设计特别有帮助。不过提醒就是这些架构很复杂，别想着可以一下子吃透。建议是先学笔记的PI-Agent部分，再去CC,Openclaw,Hermes.

结合这些项目，思路，设计一个自己的项目。甚至如果你真没时间，用笔记讲的各种包装技巧，编一个项目，比如基于笔记的Agent项目，RAG项目，或者PI-Agent项目，包装成一个项目，放在简历，最少可以开始投递了。

把二阶段学了，至少简历有东西写了，可以开始准备投递了。

3. **面试真题。**提供了大量的面试真题，包含讲解和解析。包括项目部分，其实也有对应面试真题，比如RAG项目真题，每个知识讲解里面也穿插着真题，看一下真题，自己复述一下，表达一下。里面所有真题答案的总结思路就是你学习笔记的总结，他们是呼应的，用真题可以检验你学的好不好，考前突击，模拟面试。

面试技巧章节，也可以看看。写了怎么自我介绍，反问问什么的面试技巧。

至此，你可以去投递简历了，去改简历，投递，面试，反思，复盘，自我进化，继续面试。 拿到Offer。

一些问题说明：

> **重点**：

T0: 大模型应用，最高频/常考的是：Agent + AI综合素养（Vibecoding, CC，Harness,SKILL)

T0.5: RAG.

这些建议必学，深入学习。

你可能的疑问，后端没学吗?

这个我会专门写笔记谈。总结来说：后端对大模型应用重要，不是必要项。因为我0后端基础，仍然有很多Agent offer。不会后端，不会妨碍你入行，其实可以在简历筛选的时候，也可以不去面那些强要求后端的知识。

这块是一个偏好了，如果你想继续提高竞争力： 你可以**深入学习算法（后训练）， 也可以学习后端。**

我自己个人选择是选择学习算法，因为这样我可以找一些偏算法工程的岗位。学后端，对于大模型应用让你更有竞争力，不过也会让你不能去面试一些算法的岗位，而且面试官问到你微调，强化学习，也答不深。

这个看个人选择了。后端笔记里目前没有，如果你想选择学后端，目前要麻烦自己找一下方法了。我的想法是也许我9月份做个Agent高级项目带一点后端（也许吧）。但是算法这块，我会提供0基础学习的路线，请看阶段二。

2.2 阶段二：算法进阶

算法学习其实大家看这里：0基础学习算法记录。

我已经在学习算法，但我没把算法的八股总结在笔记里，因为我不想污染这个特别适合于学习大模型应用的路线，现在笔记总结的算法的深度，就是你一阶段要学习算法的深度。

不过你真想学算法，在这个0基础学习算法记录。 里面设置的三个SKILL，里面包含了大量教纲，有算法的各种知识：Pytorch/深度学习， 强化学习/微调， 开源项目。你顺着学。

我马上，6月开始做一个算法项目。也会分享在笔记。

算法学习的思路是，不深入底层：了解基本概念，**知道怎么用专业的卡来训练，整个环节走通，有个项目。这是我学习算法的思想**！有的人学习算法，他比如去手撕一下SFT, 手撕训练，算法创新，在我看来，我们不需要，这些是算法为主的同学要做的，而且也挺难的。 我们是应用专场，但是算法会，整体都跑过，面试官问概念，细节也可以答，jd要求懂算法的也能满足要求，这是我们的思路。总之具体可以就看这个0基础算法记录，里面具体写学习算法的心得。

后期规划：6月开始做算法项目（不知道多久，预计不会很长，因为这是一个简单的、以掌握算法技术栈为目的的项目）。有了这个项目，可以把算法这个项目包装在Agent中。所以笔记里 RAG, AGENT, 算法项目各一个，可以相互灵活搭配。

做完算法项目，也许8月份，开始做一个复杂，高级的Agent项目。用我一年所学功力，做个深入的项目，结合面试高频趋势，设计项目。这个项目也会开源。做完后。我会开始投递简历试试，就和25年11月份一样，集中面试检验自己。

中间会穿插这个各种高频考点（如果笔记没有，我会总结），定期更新解析一些面经。这个作为主线之外的更新点缀。

这个就是我全部的经验了。照这个笔记学，我估计阶段一的话，都学下来，全职高效学3个\~6个月吧，把内容理解透了，也有一个项目。

当然啊，不是所有人都有很长时间学习，如果你时间赶，就以项目为主，先弄项目，知识别学那么深了，有了项目才能面试。给面试多留一些时间，一边面试，一边复盘。有些同学也是这么做的，毕竟时间紧，想速成。

最后的最后啊，笔记的一个宗旨是，**面试问题都有，因为笔记内容太丰富了。所以可以当作字典来用。**如果面试问了哪个问题不会回答， 先查查笔记有没有解析。如果是特别高频的考点，笔记里又没有可以在 社区互动/共建入口 反馈，我也会试试更新。

当然啦，也可以在群里都交流，有问题我也会尽量回答的。

3 大模型岗位介绍

大模型的岗位主要分成：**算法工程师，开发工程师，数据工程师，推理工程师，垂直领域微调工程师。**

大模型的整个链路：

数据 → 训练/对齐 → 评测 → 推理部署 → 应用落地 → 监控迭代

从岗位的具体工作上，其实我们可以很清楚对应到这个岗位处于整个链路中的哪个环节。

> 不同岗位对于技术深度，系统理解，工程实现，行业知识的要求重点不同，面试考察方式也有所差异。针对于不同的岗位，需要我们针对性调整复习策略和深度。另外，**复合型人才更为吃香**，能够清晰说明**跨岗位协作逻辑**以及**边界交互细节**，掌握多个岗位的技术。比如既具备工程能力与算法创新能力的人才将更加具有优势。

速览

应用开发工程师：最容易入门，重工程与业务落地，是新人和转岗首选。

大模型算法工程师：含金量最高 / 天花板最高。深度依赖数学、训练与架构能力，少而精，对个人技术上限要求最高。大厂卡学历，背景，论文最严重。

大模型推理部署工程师：最稀缺、工程壁垒最高。解决“模型怎么跑得快、跑得稳、跑得便宜”，强后端 + 系统 + GPU，能直接决定线上成本与性能。

大模型数据工程师：最容易被低估、但不可或缺。做数据通常被认为是脏活累活，话语权也低。门槛比开发低。

垂直领域微调工程师：最贴近行业、最容易做专家化。把通用模型变成“法律/医疗/金融专家”，拼的是领域理解 + Prompt / 微调经验，越做越值钱。

以下岗位介绍内容，是清华大学出版社的：《大模型工程面试》 - 算法原理，开发，实践与系统部署。

3.1 算法工程师

大模型算法工程师是大语言模型技术体系中的核心岗位之一，主要负责模型的结构设计、算法优化、训练流程管理以及性能评估等工作。该岗位通常要求具备扎实的深度学习理论基础与丰富的实践经验，能够从算法层面出发，构建具备通用能力、可扩展性的模型体系。

3.1.1 岗位定位和核心职责

在实际工作中，大模型算法工程师通常参与以下核心环节：模型架构设计与优化、训练任务定义、任务目标建模、损失函数调整、训练策略制定（如优化器选择、学习率调度等），以及训练过程的监控与效果分析。

部分高级岗位还需参与算法创新，如设计新的注意力机制、稀疏结构或低秩模块，用以提升模型效果与表达能力。

3.1.2 关键技术能力要求

（1）**深度学习理论基础**：掌握深度神经网络结构、前向与反向传播机制、梯度计算与参数更新原理。这些是算法工程师进行模型设计的基础。特别是对 Transformer 架构的结构细节、注意力机制、位置编码等要素需具备深入理解。

（2）**主流训练范式掌握**：熟悉预训练–微调范式的各阶段任务设计，包括自监督目标（如语言建模、掩码建模）、指令微调、强化学习阶段（如人类反馈训练）。需了解如何根据任务特性制定合理的训练目标与数据配比。

（3）**大规模训练优化经验**：具备在分布式环境下进行模型训练的能力，熟悉数据并行、模型并行、混合精度训练、梯度累积、ZeRO 优化等常用方法，能够在高效利用资源的同时保证模型性能。

（4）**算法调优与性能分析**：能够通过 Loss 曲线、梯度变化、参数分布等指标分析模型训练状态，判断是否出现梯度爆炸、收敛异常或过拟合问题。具备调参经验，如 Dropout 设置、正则化手段、学习率调整等。

（5）**论文阅读与实现能力**：需具备快速阅读理解前沿论文的能力，能够将新提出的算法结构或优化策略快速落地，并结合自身任务场景进行调整与工程实现，如引入 MoE 模块、Flash Attention 机制、ReLU 变体等。

3.1.3 典型工作流程

（1）**任务理解与模型设计**：根据任务目标（如通用对话、代码生成、知识问答等），确定模型规模、结构类型（编码器、解码器、编码解码混合），以及预训练目标等核心设计决策。

（2）**数据适配与样本处理**：参与设计 Tokenization 策略，构造多轮输入格式，划分训练集、验证集，并进行数据清洗与分布分析，为训练提供质量保障。

（3）**训练调试与性能迭代**：在训练过程中监控模型的损失变化、准确率、泛化能力等关键指标，适时调整训练参数，诊断异常现象，从而改进训练策略。

（4）**评估验证与结果产出**：基于验证集与测试集进行定量评估，如 BLEU、ROUGE、Accuracy、Perplexity 等指标，并结合人类评价对任务表现进行定性反馈。

（5）**算法报告与优化闭环**：撰写算法设计文档，记录训练流程、参数配置与调优历程，为版本迭代与团队协作提供文档支撑。

3.1.4 常见技术工具与平台

该岗位常使用的深度学习框架包括 PyTorch、TensorFlow，训练平台包括 DeepSpeed、ColossalAI、Megatron、MindSpore 等。分布式调度工具如 NCCL、Horovod、Ray 为常见组件，此外还需掌握用于可视化的工具，如 TensorBoard、Weights & Biases、Matplotlib 等。

在工程实践中，算法工程师还需与平台团队协同解决内存调度、模型并行、训练失败容错等底层工程问题。

3.1.5 岗位价值与方向发展

大模型算法工程师作为 AI 技术研发链条的源头角色，其贡献直接影响模型能力的上限与智能水平。随着模型复杂度与多模态需求的提升，该岗位的技术门槛与决策权重同步上升。

在发展路径上，该岗位可向算法科学家、高级研究员、模型负责人等方向进阶，也可拓展至智能体系统构建、通用 AI 架构设计等更具系统性与前瞻性的技术领域。在实际岗位竞争中，具备工程能力与算法创新能力并重的复合型人才将更具优势。

3.2 大模型开发工程师

大模型开发工程师主要承担大模型在工程化过程中的实现、封装与平台集成任务，是连接底层算法研究与上层应用系统的关键岗位。该岗位的工作重心在于将已有的大模型能力通过可调用接口、模块化组件、服务化架构等方式封装起来，使其能在实际业务中高效、安全、稳定地运行。

3.2.1 岗位定位与职能边界

在大模型的生命周期中，开发工程师通常介入模型推理服务搭建、接口 API 封装、前后端调用、缓存机制设计、任务调度实现、多模型集成、模型评估体系建设等多个工程层面，是大模型应用落地的直接执行者与系统支撑者。

3.2.2 关键技术能力要求

（1）**大模型调用与封装能力**：需具备使用主流框架（如 HuggingFace Transformers、OpenAI API、ChatGLM、Qwen 等）加载并调用大模型的能力，熟悉 Prompt 构造、Token 控制与输出解析等细节。

（2）**推理服务部署与优化经验**：能够完成本地部署、云端部署及容器化部署，掌握 Docker、Conda、Kubernetes 等工具，熟悉推理性能优化（如 KV Cache、批处理、并发控制）。

（3）**系统架构与微服务设计能力**：了解 Web 系统与微服务设计理念，能基于业务需求完成模块划分、服务拆分、任务路由与模块协同，实现 RESTful 接口设计、模块解耦、异常处理、日志记录等系统级能力。

（4）**任务调度与控制逻辑构建**：负责构建智能体或多模块系统中 Prompt 调用的状态机、上下文管理模块、历史信息追踪与决策逻辑，确保多轮交互、插件调用、RAG 融合等高级功能具备良好扩展性。

（5）**工程代码质量与可维护性**：具备良好的代码组织、测试覆盖、日志记录与错误处理能力，能够对模型调用链路中的每个步骤进行模块化、抽象化处理，提高系统可扩展性与稳定效率。

3.2.3 典型工作流程

（1）**模型能力接入与封装**：从算法工程师或开源模型处获取模型权重与接口文档，完成模型加载、Prompt 输入格式适配、Token 编码处理，并对输出内容做结构化封装，形成标准可调用服务接口。

（2）**模型服务化与性能优化**：通过 FastAPI 或 Triton 部署推理服务，结合负载均衡、缓存策略与请求分流机制，构建支持高并发、低延迟的服务体系。

（3）**功能集成与系统设计**：将大模型能力嵌入业务流程或系统中，构建智能问答、代码生成、文档摘要等应用场景。对接 RAG 系统、知识库、搜索引擎、数据库等外部模块，完成数据驱动与结果增强。

（4）**多模型管理与调用路由**：设计统一模型管理接口，实现多模型版本切换、上下文状态路由、多语言模型选择、跨任务模型组合等能力，保障多样化业务需求的灵活响应。

（5）**系统监控与异常处理**：构建模型调用链的健康监控体系，包括响应时长统计、异常次数记录、内存使用跟踪等，结合 Prometheus、Grafana、ELK 等工具构建可视化监控看板与日志系统。

3.2.4 常用技术工具与平台

大模型开发工程师常用的核心语言为 Python 和 Go，主要框架涵盖 Transformers、vLLM、FastAPI、Triton、Flask、LangChain 等；数据处理与缓存机制中常使用 Redis、MongoDB、SQLAlchemy 等工具；

监控体系中常使用 Prometheus、Grafana、Loguru、Sentry 等组件。

在模型服务容器化方面，需熟悉 Dockerfile 构建与镜像管理流程；对于大规模部署，还需了解 Kubernetes、Nginx 反向代理、服务网关等相关技术。

3.3 大模型数据工程师

在大模型系统中，数据工程师是支撑预训练、微调与推理各阶段的底层力量，负责数据的采集、清洗、结构化、存储与分发，是整个模型生命周期的数据保障者。与传统数据分析岗位不同，大模型领域的数据工程师需要处理的是大规模、高维度、多模态的非结构化数据，不仅要解决数据可用性问题，更要保障数据对模型训练效果的正向支撑。

3.3.1 岗位定义与职责核心

岗位主要职责包括：搭建数据获取与处理流程，构建高质量训练语料，设计分词策略与 Token 机制，实现分布式数据加载优化，建立数据版本与可追溯机制，支持多语言与多模态数据并行处理等。在真实工程场景中，数据工程师的工作直接影响模型训练效率、性能表现及微调的效果边界。

3.3.2 关键技术能力要求

（1）大规模文本数据处理能力

需要掌握海量文本数据的清洗、去重、切分与规整等操作，熟练使用正则表达式、文本处理库、分布式文件系统等工具，能够在百万到数十亿级文本样本中构建符合训练标准的语料。

（2）分词与 Token 机制理解

熟悉常见分词器（如 BPE、SentencePiece、WordPiece），掌握 Tokenizer 构建、词表训练、Token 长度分布控制、文本对齐机制等关键技术。能够根据中文、英文及多语言任务需求选择合适的分词策略。

（3）数据增强与样本构造经验

具备使用反事实生成、数据混合、格式转换、指令生成等手段进行样本增强的能力，能够围绕对话、问答、生成、翻译等任务构造标准训练样本，提升模型泛化能力。

（4）分布式数据加载与预处理优化

在大模型训练中，数据加载速度直接影响整体训练效率。工程师需熟悉 DataLoader 优化、Sharding 机制、缓存策略、异步加载与数据流水线设计，避免数据成为训练瓶颈。

（5）多模态与多语言数据处理能力

随着大模型向图像、音频、视频等模态扩展，数据工程师需具备将多模态数据转换为统一编码输入的能力，如语音转文本、多语种语料对齐等，并构建统一的数据格式供模型使用。

3.3.3 典型工作流程

（1）语料采集与初筛

从开源项目、公开网站、内部知识库、社交平台等渠道自动或半自动获取文本，进行初步清洗与合规审查，剔除无效、违规、低质量信息。

（2）数据清洗与格式规整

统一文本编码，清除特殊字符与冗余标记，处理 HTML 标签、异常断句与乱码等，并根据任务需求完成多轮清洗。

（3）数据切分与 Token 统计

将长文本切分为适合 Token 限制的段落，统计样本平均长度、分布范围、最大 Token 数，设置上下文窗口，确保模型输入不溢出并具备语义连续性。

（4）训练样本构造与增强

结合任务目标构造训练样本，如为问答系统设计包含 Prompt、Query、Answer 结构的样本，为 RLHF 构建带有偏好标注的数据，或构造 Few-shot 示例序列。适时进行样本增强，对抗样本添加等方法提升模型鲁棒性。

（5）数据版本管理与溯源机制

每次训练数据的构建过程均需记录版本号、处理脚本、时间戳与来源，确保模型结果可回溯、可复现，便于后期调参、复训与问题追踪。

3.3.4 常用技术工具与平台

大模型数据工程师常用语言为 Python 和 Bash，主要处理工具包括 Pandas、Datasets 库、HuggingFace Tokenizer、spaCy、OpenCC、Jieba、NLPAug 等。分布式数据处理场景多使用 Spark、Flink、Ray 等；数据管理平台包括 Airflow、MLflow、DVC、Delta Lake 等。

对于特定语料（如代码、法律、医学数据），还需具备专业领域数据理解能力，进行精细化的结构映射与语义分类。对于多模态数据，则需配合图像标注工具、语音转文本引擎等完成前期处理。

3.3.5 岗位价值与发展方向

数据工程师是支撑大模型构建的基础力量，其工作成果决定了模型学习能力的上限与泛化能力的边界。在当前数据驱动为核心的模型发展趋势下，该岗位的重要性持续提升。

未来，数据工程师的发展方向包括复合的数据架构设计者、大模型训练的数据管道构建者、专注于安全与对齐的数据质量管控专家，以及承担合成与任务数据生成责任的 “Prompt 数据工程师” 等角色。随着大模型对数据精度、结构与任务匹配度要求不断提高，具备算法理解与工程能力融合背景的数据工程师将在模型研发团队中扮演愈发关键的角色。

3.4 推理工程师

推理部署工程师是大模型工程体系中负责模型上线、服务发布与推理性能优化的关键岗位，主要任务是将训练好的大语言模型从离线环境转换为可在线调用的服务接口，并在推理过程中保证系统的稳定性、可扩展性与高性能响应。

与算法工程师负责模型训练不同，推理部署工程师更为关注如何在有限算力下实现最优推理效率、如何在多端场景中稳定运行模型，以及如何将模型能力集成进实际应用/系统中。

3.4.1 岗位定位和核心职责

该岗位通常需要与算法、开发、系统运维、安全团队紧密协作，在模型格式转换、量化/硬件适配、服务封装、资源调度等多个层面形成闭环工作流，是大模型从实验室走向落地的最后“一公里”执行者。

3.4.2 关键技术能力要求

模型格式转换与兼容性适配：掌握主流模型格式之间的转换流程（如从 PyTorch 转 ONNX、TensorRT、vLLM 格式），理解不同部署框架对模型结构支持的差异，能够进行图优化与结构裁剪，确保模型在目标平台可顺利加载并运行。

推理精度加速与性能优化：熟悉推理阶段的常见加速方法，包括半精度计算（FP16）、量化计算（INT8）、稀疏矩阵加速、KV缓存机制、Batch 合并处理、多线程异步调用等，能够根据模型特征选择最合适的推理路径，以实现吞吐与响应时间的优化平衡。

多卡部署与资源调度：掌握多卡部署的通信机制与并行调度策略，了解模型并行、流水并行与张量并行的基本逻辑，能通过 NCCL、TorchRun、DeepSpeed 等工具实现横向扩展能力，熟悉 CUDA 资源管理，显存分布

与 GPU 负载监控等底层细节。

服务封装与接口管理：具备使用 FastAPI、Flask、Triton、BentoML 等构建模型服务接口能力，能够完成 API 路由管理、请求参数解析、Token 长度控制、错误响应处理、调用日志收集等，保障部署服务具备高可用性与可维护性。

系统监控与容错能力建设：熟练构建服务运行时的监控体系，包括请求量、响应时延、失败率、硬件利用率等指标收集与可视化展示。能够设置异常熔断、自动重启、超时堵塞、回退机制等防故障能力，提高整体系统鲁棒性。

3.4.3 典型工作流程

模型接收与格式标准化：从算法工程师处获取已训练完成的模型及配套配置文件，完成必要的格式转换与静态图构建，并对输入/输出结构进行标准化处理，适配部署框架要求。

推理引擎选择与性能测试：评估部署场景对性能的需求，选择合适的推理引擎（如 ONNX Runtime、TensorRT、vLLM、DeepSpeed Inference 等），并结合测试脚本对不同部署方案进行 Latency、Throughput 等关键指标测试与对比分析。

服务封装与线上部署：基于选定的推理方案，使用 FastAPI 或 Triton 封装模型服务，设置并管理推理接口参数、配置多进程并发机制，实现可扩展的线上服务部署，并进行 Docker 化与持续集成配置。

资源调度与运行监控：根据模型资源需求配置合适的 GPU 或混合 CPU-GPU 部署方案，设置 Prometheus 与 Grafana 完成推理链路的全量监控，定期检查系统负载、运行日志与异常警告，确保服务运行稳定。

优化迭代与问题修复：基于真实调用数据与用户反馈，持续优化模型响应路径与服务性能，解决兼容性问题、性能瓶颈和异常问题，以满足业务增长的需求。

3.4.4 常见技术工具与平台

推理部署工程师常使用的核心工具包括：

（1）**格式转换工具**：ONNX Exporter、TorchScript、TF Converter。

（2）**推理框架**：TensorRT、ONNX Runtime、vLLM、Triton Server、DeepSpeed Inference。

（3）**服务封装工具**：FastAPI、Flask、BentoML、gRPC。

（4）**容器与部署**：Docker、Kubernetes、NVIDIA Docker、Nginx。

（5）**监控与日志系统**：Prometheus、Grafana、Loguru、Sentry。

同时，还需熟悉 **Linux 环境下的 GPU 调度、CUDA 工具链、Python 后端逻辑编写与 HTTP 接口设计等。**

3.4.5 岗位价值与发展方向

推理部署工程师是保障大模型从理论能力到实际服务转化的关键角色，承担将模型高效、安全、稳定部署至生产环境的责任。其专业性不仅体现在工程实现效率上，更体现在对系统稳定性、性能可控性的理解与掌控中。

随着边缘部署、模型压缩、多模态推理、低功耗计算的快速发展，该岗位将逐步向“系统级优化工程师”“AI 基础设施工程师”“多模型编排调度专家”等方向拓展，成为 AI 工程体系中不可替代的关键节点。对于追求工程深度与系统稳定性的技术人员而言，推理部署工程师是一个技术挑战性与价值感兼具的发展路径。

3.5 垂直领域微调工程师

垂直领域微调工程师是大模型团队中专注于模型在特定行业或任务上进行再训练与能力定制的技术岗位，主要负责在预训练模型基础上，通过有限或特定领域的数据进行微调，使模型具备更强的专业理解能力与场景适配能力。该岗位连接通用模型能力与下游应用需求，是实现模型产品化与行业落地的关键角色。

3.5.1 岗位定位和核心职责

与算法工程师不同，微调工程师不负责构建模型底层结构，而是更多地参与数据处理、训练策略设计、指标评估与迭代优化，围绕某一具体任务（如法律问答、医疗报告生成、金融分析、政务问询等）构建专业能力闭环。

3.5.2 关键技术能力要求

（1）**行业知识与任务理解能力**：
垂直领域微调工程师需对目标行业的语言风格、知识结构、任务类型有较强理解，能够抽象出符合业务需求的任务定义与评价标准，确保模型在语义层面具备专业性与可控性。

（2）**模型微调与参数优化能力**：
需要掌握全参数微调、冻结微调、参数高效微调（如 LoRA、Adapter、Prefix Tuning 等）等不同技术路线，能够根据数据规模、算力预算与业务场景灵活选择训练策略，并掌握训练过程中的调参技巧与稳定性控制方法。

（3）**指令构造与 Prompt 设计能力**：
能够针对具体任务构造高质量 Prompt 模板、示例指令与上下文配置，优化 Few-shot、Zero-shot 等输入形式，引导模型在生成过程中表现出符合预期的语义风格与逻辑结构。

（4）**数据增强与标注指导能力**：
具备设计反事实样本、扰动样本、边界样本等方法提升模型泛化能力的经验，能够协助标注团队构建高质量样本集，并在数据层面对模型泛化能力形成正向反馈。

（5）**评估指标与质量控制能力**：
熟悉多种评估指标，如 BLEU、ROUGE、EM、F1、准确率、召回率、人工评分体系等，能够构建自动化评估流程，监测模型在不同迭代过程中的表现变化。

3.5.3 典型工作流程

（1）**场景需求分析与任务定义**：
根据业务部门或产品线提出的功能需求，提炼出可落地的模型任务定义，如医疗问答、合同生成、证券分析

等，明确输入/输出格式、预期效果与边界条件。

（2）**语料准备与数据构建**：
联合数据工程师完成语料搜集与结构化，设计指令体系、示例样本、标签体系等，构建符合微调需求的高质量样本集，必要时进行领域知识注入或术语标准化处理。

（3）**模型选择与训练策略制定**：
根据任务复杂度与资源条件选择合适的预训练模型（如 Qwen、ChatGLM、Baichuan 等），并确定训练策略，包括是否冻结部分层、是否应用 LoRA 插入、训练轮数与学习率设置等。

（4）**训练调试与指标监控**：
进行模型训练并持续监控 Loss 变化、精度提升、样本覆盖情况，诊断过拟合、漂移、语义偏差等问题，调整超参数以实现更稳定的训练表现。

（5）**评估验证与业务集成**：
使用测试集与评估指标对模型效果进行定量验证，并通过业务场景下的人工评估或 AB 测试进行实战验证，确保模型在目标场景下具备上线条件，最终对接开发或推理部署团队完成集成。

3.5.4 常见技术工具与平台

垂直领域微调工程师常使用的训练框架包括：

Transformers（HuggingFace）、PEFT 库、DeepSpeed、ColossalAI 等。

模型监控与训练日志通常使用 Weights & Biases、TensorBoard 等可视化工具。

数据处理常依赖 Pandas、SpaCy、NLPAug 等文本增强工具，训练环境通常部署于多卡 GPU 服务器或分布式云平台，如阿里 PAI、华为昇腾平台等。

在多语言场景下，还需结合 OpenCC、Jieba、FastText 等工具处理中文英文混合、术语翻译与对齐问题。

3.5.5 岗位价值与发展方向

垂直领域微调工程师是大模型产品化过程中不可或缺的角色，决定了模型能否真正理解业务语境并满足特定场景下的精度需求与用户交互预期。其岗位价值不仅体现在提升模型表现，更在于构建模型与业务之间的连接通道，将抽象能力转化为可落地产品。

随着多行业对大模型应用需求的深入，该岗位可进一步拓展至“任务建模专家”“Prompt 设计工程师”“AI 产品交付工程师”“行业知识增强建模专家”等方向，成为推动大模型在垂直领域深度落地的关键推动者。对于兼具行业知识与算法理解能力的工程师而言，该岗位提供了广阔的发展空间与实践价值。

3.6 FDE（前线部署工程师）

FDE 通常是 Forward Deployed Engineer 的缩写，可译为“前线部署工程师”或“前线交付工程师”。Palantir 的完整岗位名称还常写作 Forward Deployed Software Engineer。国内部分材料将其解释为 Field Deployment Engineer 或“前沿部署工程师”，但这并不是该岗位最常见、最准确的英文来源。

FDE 是**直接面对客户的软件工程师**，位于客户业务、产品能力与工程交付的交叉点。它并非简单地安装部署模型，而是深入客户现场，从模糊的业务问题出发，完成需求发现、方案设计、软件开发、系统集成、上线验证与持续迭代，**对最终的客户业务结果负责**。

3.6.1 岗位起源与发展背景

FDE 这一岗位模式通常被认为**发源于美国企业软件公司 Palantir**。Palantir 成立于 2003 年，FDE 在公司创立早期、即 **2000 年代初期**便伴随其业务模式逐渐形成。由于 Palantir 将复杂的数据分析平台交付给政府、金融、制造、航空等非纯技术客户，单靠标准产品和使用培训无法解决客户的复杂问题，因此需要工程师直接进入客户环境，一边理解业务，一边利用平台开发解决方案。

传统软件研发通常追求“一对多”：开发一个标准产品服务大量客户；FDE 的初始工作则更偏“一对一”：围绕一个重要客户的具体问题快速构建端到端方案。项目中反复出现的共性需求，又可以被抽象并反馈给核心产品团队，沉淀为可复用的平台能力。

这一岗位并非生成式 AI 出现后才诞生。它已在 Palantir 等企业软件公司存在二十年左右，但在 2023 年生成式 AI 商业化加速后重新受到关注。大模型、Agent 和数据平台的能力越来越复杂，而传统企业往往缺少把这些技术接入自身数据、权限、流程和业务系统的能力，FDE 因而成为连接“模型能力”与“业务结果”的关键角色。

在国内，与 FDE 相似的工作过去并非不存在，往往分散在售前解决方案、客户工程师、驻场开发、AI 交付和行业解决方案架构师等岗位中，只是较少统一使用 FDE 这一名称。随着 2023～2024 年国内大模型厂商、云厂商和企业客户开始批量探索大模型应用，市场先产生了“如何把模型落到业务里”的职能需求；到 2025 年前后，越来越多公司开始直接采用 Forward Deployed Engineer/FDE 的岗位名称；2026 年上半年，伴随 Agent 商业化和大厂集中发布招聘，该名称在国内招聘平台、自媒体和求职讨论中明显升温，成为所谓“突然火起来”的新岗位。

因此，“FDE 在国内什么时候出现”和“什么时候火起来”需要区分：类似职能早已存在，**2023～2024 年是需求形成期，2025 年是岗位名称加速引入和扩散期，2026 年上半年是公众认知与招聘讨论的集中爆发期**。当前在火山引擎、腾讯云及其他 AI 公司的招聘中，可以看到 FDE、现场部署工程师、AI 解决方案工程师等名称。但目前缺少权威机构按年份统计国内 FDE 岗位数量的数据，网络流传的“需求暴涨多少倍”等说法不宜直接当作行业统计结论。

3.6.2 岗位定位与职能边界

FDE 可以理解为同时承担三种职能：

（1）**顾问**：通过访谈和主动倾听理解客户业务，区分客户描述的表面症状与真正需要解决的商业问题，建立客户及管理层的信任。

（2）**产品经理**：把零散需求抽象成可执行的问题，综合业务价值、技术可行性、时间与资源进行优先级排序，确定最值得解决的场景。

（3）**软件工程师**：亲自完成原型、代码、数据接入、系统集成、测试与交付，将方案变成能够运行的软件，而不是止步于咨询报告或演示文稿。

FDE 不等同于售前解决方案工程师。售前通常在签约前完成产品介绍、方案设计和 POC Demo，目标是证明产品能力并推动成交；FDE 往往从需求发现一直参与到生产交付、实际使用和效果验证。**FDE 也不等同于传统实施或驻场运维。**如果工作内容主要是安装软件、配置后台、处理工单，而不需要定义问题、编写代码和交付生产方案，则更接近传统实施，而非严格意义上的 FDE。

FDE 强调端到端负责，但不表示所有工作都由一个人完成。商务报价与合同通常由销售负责，通用平台由核心研发团队负责，底层模型训练由算法团队负责，生产运维也可能由平台团队配合。FDE 的核心责任是跨越这些团队之间的断点，持续推动一个高价值客户问题被真正解决。

3.6.3 关键能力要求

（1）**客户发现与业务抽象能力**：能够访谈业务人员和管理层，梳理角色、流程、数据、指标与约束，从“客户说想要什么”继续追问到“客户为什么需要它、成功如何衡量”。FDE 最重要的沟通能力往往不是能说，而是主动倾听、追问和建立信任。

（2）**产品判断与优先级能力**：客户通常会提出大量需求，FDE 需要找到业务影响大、当前平台能够解决、交付周期合理的问题，并清楚说明取舍依据。不能只追求技术上有趣，也不能只做容易但没有业务价值的功能。

（3）**端到端软件工程能力**：具备实际编码和构建能力，能够围绕特定客户的问题开发解决方案，而不是只写方案或制作售前演示。交付时需要考虑代码是否完整、可能出现的异常和边界输入，以及客户能否正确使用。

（4）**大模型应用能力**：能够搭建和调试 Agent，进行提示词调优，并根据客户业务接入 Skill、MCP 等能力。FDE 不一定负责训练基础模型，但需要理解模型和 AI 产品如何应用到客户场景中。

（5）**系统对接能力**：能够根据项目需要，将 AI 应用与客户的数据库、OA、业务系统和内部工作流程连接起来。具体使用什么语言、框架和部署工具，取决于所在公司的产品与客户环境，现有资料没有给出统一技术栈。

（6）**项目推动与高管沟通能力**：能够管理范围、风险和预期，向业务人员解释技术限制，也能向管理层用业务指标说明方案价值。FDE 面对的不是一份完全明确的需求文档，而是需要在不确定环境中持续对齐并推动交付。

（7）**行业理解与快速学习能力**：能够快速进入制造、金融、零售、医疗、政务等陌生领域，理解行业术语、核心流程、监管要求和价值链。长期来看，深耕某一行业并形成可复用的方法论，会显著提高个人壁垒。

3.6.4 典型工作流程

（1）客户发现：访谈客户管理层和一线人员，观察实际流程，梳理现有系统、数据与关键指标，不急于根据客户提出的第一个需求直接开发。

（2）问题定义与方案选择：把业务问题转化为可验证的任务，明确范围和预期结果，判断公司的模型、Agent 或平台能力如何与该问题结合。

（3）POC 与价值验证：快速构建原型，用客户真实或脱敏数据验证技术可行性及业务价值。POC 不是终点，而是降低风险、确认方向的阶段。

（4）落地交付：将方案与客户的数据、数据库、OA 或其他业务系统进行对接，完成必要的开发与调试，使其能够解决客户的实际问题。

（5）上线与迭代：培训用户、观察使用情况，结合业务指标、模型评估和用户反馈持续优化。项目结束后，将重复出现的问题和能力反馈给产品团队，推动产品标准化。

3.6.5 技术能力要求

FDE典型的**技术能力包括**：

- 大模型基础与模型落地能力；
- Agent 的搭建、定制和效果调试；
- 提示词调优；
- Skill、MCP 等能力的接入；
- Web Coding、简易脚本和实际构建能力；
- 数据库、OA、企业业务系统与工作流程的对接；
- 从客户问题发现到软件交付的端到端工程能力。

其中，英文访谈将 FDE 直接定义为“面向客户的软件工程师”，并强调最终交付物应当是软件。因此可以确认 **FDE 需要开发能力**，但不能仅根据这些资料进一步断言所有 FDE 都必须掌握某套传统后端、云原生或分布式技术栈。具体工具应以目标公司的产品形态和岗位描述为准。

3.6.6 薪资待遇与口径说明

FDE 的薪资差异很大，主要取决于城市、公司阶段、岗位偏工程还是偏实施、出差强度、客户行业和个人经验。目前国内岗位名称尚未完全统一，FDE、AI 解决方案工程师、AI 应用交付工程师、客户工程师等名称可能对应相似工作，反过来，一些普通实施或驻场开发岗位也会被包装为 FDE，因此不能只按岗位名称比较薪资。

从当前国内招聘与市场信息看，**一线城市部分初级岗位月薪约为 20k～30k**；具有 **2～3 年企业 AI 落地经验的岗位可能达到 30k～55k**；少数大厂、核心团队或资深行业专家岗位**年薪可达到 60 万～80 万以上**。实际收入受公司、城市、职级、技术要求、客户资源、出差强度及奖金股权影响很大。由于国内尚无针对 FDE 的统一薪酬统计，这些数字只能作为招聘区间参考，不能理解为行业普遍水平。

美国科技公司的公开招聘中，FDE 往往按照中高级软件工程师或客户工程岗位定价，常同时包含基本工资、奖金和股权；不同地区、级别及公司之间差异明显，不能将美国年薪直接换算成国内岗位待遇。实际求职时，应以目标公司的 JD、职级、基本工资、奖金/股权、出差补贴和驻场要求综合判断。

3.6.7 前景、机会与风险

从中短期看，FDE 受益于企业 AI 商业化。模型能力正在快速商品化，但企业的数据、流程、权限和行业知识无法完全标准化，能够把复杂技术转化为真实业务成果的人才仍然稀缺。尤其是向传统行业销售复杂 AI 产品的模型厂商、云厂商和 ToB AI 公司，更需要这类岗位。

从长期看，只要“复杂技术产品面对非技术客户”这一问题持续存在，FDE 的职能就不会消失。其发展方向主要有两类：一类是把项目中的共性需求沉淀为产品平台，走向 AI 产品、产品工程或解决方案架构；另一类是深耕金融、制造、医疗、零售等垂直行业，成为行业 AI 交付专家、行业架构师或业务负责人。职业成长通常体现为负责范围扩大：从完成一个问题，逐步发展到独立负责一个客户、一个行业、一个区域或更大规模的业务。

该岗位也存在明显风险：工作职责杂、需求不确定、客户压力大、可能频繁出差或驻场，并同时承受销售、客户与研发团队的压力。如果公司缺少成熟产品，FDE 容易退化为无止境的定制外包；如果工作只包含安装配置和售后支持，则技术成长空间有限。因此面试时应重点确认是否需要编写生产代码、是否参与问题定义、是否负责从 POC 到上线、成果能否沉淀回产品，以及考核指标是交付工时、签约金额还是客户业务结果。

3.6.8 适合人群与职业匹配

FDE 适合具有以下特征的人：

（1）有产品意识的后端、全栈或大模型应用开发工程师，不满足于只接收需求写代码，愿意接触客户并理解业务价值。

（2）技术动手能力较强的产品经理、解决方案架构师或咨询顾问，能够进行需求分析和客户沟通，并愿意补齐编码与工程交付能力。

（3）传统实施或售前工程师，已经熟悉企业客户和项目推进，但需要进一步掌握大模型应用、软件开发、系统集成与生产化能力。

（4）做过创业或未来希望创业的人。FDE 的工作很像早期创业：客户未必知道自己真正需要什么，工程师需要一边与客户交流，一边定义和构建产品，并直接对结果负责。

如果只希望长期专注底层算法或纯编码、不愿接触客户；只喜欢做规划而不愿亲自实现；或者偏好需求清晰、职责边界稳定的工作，则可能不适合 FDE。选择该岗位的理由应当是喜欢同时处理咨询、产品和工程问题，而不只是因为岗位名称热门或薪资宣传较高。

3.6.9 以 FDE 为目标时，本笔记的学习重点

本笔记以大模型应用开发为主线，与 FDE 的技术底座高度重合，但不能完整覆盖 FDE 所需的客户与业务能力。建议按以下优先级学习：

第一优先级是 Agent、RAG 与项目实战。重点掌握 Agent 的基本机制、Function Calling、MCP、LangChain/LangGraph、RAG 全链路、向量数据库、混合检索、Query 处理、记忆、权限、安全、成本控制与 Agent 评估。FDE 不仅要会调用模型，更要能说明为什么选择某种架构、效果如何衡量、失败时如何定位。

第二优先级是构建和交付能力。重点看“大模型开发工程师”“RAG 项目实战”“Agent 项目实战”，训练自己从业务问题出发搭建 Agent，并完成数据库、外部能力或业务流程的接入。至于是否进一步学习 API 设计、Docker、云端部署等内容，应根据具体 FDE 招聘要求决定，不能将其视为所有 FDE 的统一必备技术。

第三优先级是模型选型与大模型全链路认知。需要理解不同模型的能力、成本、延迟和上下文限制，了解 Prompt、RAG、微调之间的选择逻辑；Transformer、SFT、强化学习、量化和推理框架以理解原理、能够与算法及平台团队协作为目标，不必像算法工程师一样优先深入公式和训练细节。

第四优先级是 AI Coding、Skill 与行业素养。FDE 经常需要在客户现场快速构建原型，熟练使用 Claude Code、Codex 等工具以及 Vibe Coding/Harness 方法，可以显著提高交付速度。但必须能够审核代码、补充测试并承担结果，不能只会让模型生成代码。

除本笔记外，还需要重点补齐三类能力：

一是客户发现，包括访谈、主动倾听、追问、需求澄清和高管沟通；

二是业务分析，包括流程图、角色与权限、价值链、ROI、成功指标和优先级判断；

三是企业交付，包括 POC 设计、项目范围管理、数据合规、安全、遗留系统集成和用户培训。

最好选择一个真实行业场景完成项目：先描述业务流程和基线指标，再设计 AI 方案，使用真实约束完成 POC、评估与生产化，并记录方案取舍和业务结果。这比单纯堆叠 Agent、RAG、MCP 等技术名词更符合 FDE 面试要求。

3.7 不同岗位技术侧重点和面试技巧

大模型相关岗位虽都围绕同一类核心技术展开，但在实际职责、技能要求与评估重点上存在显著差异。了解不同岗位的技术关注点，有助于应聘者制定有针对性的备考策略，从而在面试过程中有效展现能力优势与岗位匹配度。

3.7.1 大模型相关岗位的能力维度差异

整体来看，大模型相关岗位可分为六类：算法工程师、开发工程师、数据工程师、推理部署工程师、垂直领域微调工程师和 FDE。每一岗位对技术深度、系统理解、工程实现、客户协作或行业知识的要求和面试考察方式也有所差异。

3.7.2 岗位技术侧重点分析

- 算法工程师：该岗位强调对模型架构、训练机制、优化策略的深度掌握，技术考察通常集中于 Transformer 结构细节、自注意力机制、训练范式（如预训练、微调、RLHF）、梯度更新原理与算法创新能力。具备扎实的数学基础与科研阅读能力是加分项，常涉及论文复现、模型重构、Loss 函数调优等实战题。
- 大模型开发工程师：该岗位更注重工程实现与系统集成能力，重点考察模型封装、API 接口设计、任务调度、状态控制、上下文缓存、日志系统等模块开发能力。熟悉 FastAPI、LangChain、Triton 等框架，能

够独立搭建模型服务并实现业务对接是关键能力之一，面试中常通过系统设计题、代码走查和实际部署经验进行评估。

- 大模型数据工程师：该岗位要求具备处理海量文本、设计分词策略、数据增强与 Token 分析能力，尤其强调数据质量控制与处理效率。面试中常考察对语料清洗流程、Tokenizer 构建、分布式数据加载、格式规范化处理等方面的理解与实操能力，可能要求编写数据预处理脚本或解释复杂的样本结构。
- 大模型推理部署工程师：该岗位关注模型上线后的服务性能与稳定性，技术重点包括模型格式转换（如 ONNX/TensorRT）、推理加速（如 FP16、INT8）、KV Cache 机制、多卡并行部署与资源调度策略。面试中常要求分析推理延迟瓶颈、设计高并发推理方案，或解释模型在不同引擎上的性能表现。
- 垂直领域微调工程师：该岗位强调行业知识、任务定义、数据构造、微调策略和效果评估。面试通常关注 LoRA 等参数高效微调方法、领域数据质量、训练指标、过拟合与模型偏差，以及如何判断微调相较于 Prompt 或 RAG 是否更适合当前业务。
- FDE：该岗位强调客户发现、业务抽象、产品判断与端到端工程交付，重点考察能否从模糊需求中识别高价值问题，完成方案取舍、POC、生产化和效果验证。面试可能同时包含客户沟通与行为题、产品案例题、系统设计和编码题，并特别关注候选人的主人翁意识、完整交付能力和面对管理层的表达能力。

---

4 学习路线

了解了上述介绍后，我们在制定学习路线，应该根据自己的背景，专长等实际情况来分析。 我结合我个人背景来讲解一下我的学习路线的理由。

首先，我的背景主要是做的客户端开发，我没有后端知识。其实后端/前端/客户端甚至是测开，测试开发，都可以归类于开发。只是我们之前做的不是大模型的开发。 我预期之前做后端/前端/客户/测开/测试 等的同学们，应该都没有太多深度学习基础。 大模型算法本身要求就比较高，直接转行算法难度太高。数据工程师本身活杂，话语权低，上限也比较低。部署的话，还是需要强后端知识，还有模型知识，这些背景一般开发也没有。 所以我选择应用入手。有深度学习背景的同学，之前就做过模型训推，发论文的同学可以考虑从直接转大模型算法，推理工程师。如果有强领域倾向，比如做医疗的，法律行业的同学，可以侧重垂直领域微调方向。如果本身开发基础都比较少，也可以试试从数据工程师角度入手。

回到我们面向大模型应用开发的路线。我们的学习路线思路（其实也就是这个笔记的思路），先学完 Agent,RAG 最核心的知识和项目，常见的应用技术：Skill,MCP, A2A等。然后你需要理解整个大模型的链路，从预训练到部署等，所以笔记设计了**一些微调，强化学习，模型的知识**，面试也会问，但是笔记设计得比较浅，偏向于理解。最后笔记思路是跑通了一个 数据准备/量化/LoRa/强化学习来使用本地模型推理，把推理，训练串起来。注意我这里用词是 **跑通。** 因为我确实没有深入做这些内容，侧重于用GPT写一个代码，了解流程，用云平台跑一下，准备好答案（你是怎么做微调的，你项目怎么做的量化之类，这些在笔记中有些）。到这里就够了，我没有深入，更不用去手撕注意力机制网络。

（注意，到26年7月左右，我开始真的学习算法，做算法项目了，我把上述说算法相关的内容，真的做了一遍，而不是简单跑通POC,这部分我作为进阶学习。笔记记在这里。 时间有限的话，你就和我一样，把笔记里

的算法内容背一背，准备下相关问题，不要问到概念不知道就行了，不用做算法项目，这个属于进阶内容, 学有余力再学）

到这个阶段，**我已经可以获取很多Offer了。**接下来的路线：可以去**学一下后端知识**（如上节讲很多应用岗要求云端部署，Docker,kubernetes），有些后端出身同学可能已经掌握，但是其它比如前端，客户端，测试可能不会，可以去学一下，这个会加强竞争力，但是不是必要的。 我的个人选择是先放掉这里，去学习训练，推理，部署。 因为这样可以跳脱应用的框架，可以往算法，算法工程这些岗位去靠，上限更高，我后面即将和大家一起学一个训练的综合项目，就是为了深入学习。

所以路线供大家参考，大家可以根据自己情况修改。我想强调的一点是，这个行业其实倾向**于综合型人才**，我一直给自己的目标是**懂算法，精工程，善部署。** 如果目标是算法岗的同学是一样的，也需要学习了解工程知识比如Agent相关知识。 这点也确实体现在面试中。我面试其实一直学习的路线是应用路线，我后面做复盘，才发现很多面试中的岗位其实是算法岗，或者交叉岗位。我自己都没意识到，我写的意向是大模型开发，项目也是Agent，面试完复盘我才发现这是个偏算法的岗位。

> 另外一点我想说的是你只学工程（Agent,RAG)， 其实也可以找到工作。特别是有些大厂，比如阿里千问，我面试的时候，他们工程岗完全不关注你算法知识，让你从工程岗位回答。但是我建议学一下算法，这样上限高，竞争力也更强。如果你时间很紧，也可以趁现在行业发展刚起步，应用入行。 但是咱们有这个意识，以后朝着算法啊，推理部署方向再去靠，入行了再去学，变成综合性人才，或者慢慢转到算法，推理等岗位，上限比纯工程会高很多。

5 答疑

5.1 后端知识在大模型行业中重要吗?

从上面的岗位介绍中，我们可以看出来大模型的某些岗位和后端知识有重叠。比如有的**应用岗**要求本地，云端部署，容器化（Docker), Conda, Kubernetes。 而**推理部署岗**注重推理服务性能，成本，稳定性。 二者相比，应用岗对于后端虽然重要，但不是必要要求（我没有后端经验，但是面试了很多应用岗位），优先级会低于Agent,RAG 的核心技术栈。比如我面试的阿里夸克千问C端，就是一个应用的后端岗，但其实面试官没问过我后端知识，我也没后端技术栈，最后也通过了。 我承认后端知识对于应用岗是重要的，但是因为非必要，我还是稍微降低了后端知识的优先级，学完了应用，先去学习部署/算法。 相比于应用岗，**推理部署岗位**明显对于后端要求更高，同样需要Docker,K8s, 但进一步要求推理服务，延迟，吞吐测评，后端是该岗位的核心竞争力。 总结来说，算法岗/应用岗/**推理岗对于后端能力的要求是**:

推理部署岗>>应用岗>算法岗

所以，具体的路线还是要因人而异，自己适当调整。我的路线是先学了应用->算法，变成一个复合型人才。然后可能再去考虑后端。 但是如果说自己的定位就是纯应用，**学完Agent,RAG, 再去学习补齐后端知识是合理的。** 补齐后端开发，会增加你面试大模型应用的竞争力和简历通过率。 但是另一个角度，你也可以走另一条路线，去学一下算法（训练比如Lora,强化学习，推理部署的内容），这也可以增加你的竞争力，很多应用岗位也会问一些算法的东西，甚至要求应用算法都会，学了算法也会多一些机会，提高自己的竞争力。

所以啊，当你学完了Agent, Rag, Skill,harness等等这些大模型应用相关的知识后？你一定已经可以面试了，绝对有机会，把握好可以上岸。 但是如果你还想学，时间更多，可以去学后端，也可以去学算法，是不一样的路线。我选择了去学算法，学了算法，把算法的内容可以加到Agent应用上，也可以增加简历竞争力。我至今为止没有学后端。 不是我不想学，是我学了一年，也才慢慢地学习算法，要学的东西太多了，我们不可能满足市场上所有要求吧，尽量吧。目前你要是想学算法，和我的路线一样，那么恭喜你可以完全复用，复用学习思路，资料，项目。如果你想选择学后端，那么笔记里暂时没有资料，也许算法做完，我下一个Agent项目（26年年底开发）可以带一点后端，不知道大家赶得上吗？

5.2 Agent项目浅，如何优化

粉丝提问：我的Agent项目做得很浅，没有一些复杂的机制，就是简单的意图识别+workflow, RAG部分也是别人做的。面试时候不太好吹怎么办？

回答：

是的，确实是建议继续深入优化：

1.从Agent层面。建议引入更多深入的地方。比如记忆模块，长短记忆怎么做的，有没有什么高级策略。 能不能结合一些策略让客服记住长期状态，回答更符合用户特点。能否有一些反思，规划功能。参考我笔记中推荐的《Agentic design pattern》 的设计模式，能不能加一些高级功能。

2.RAG.我自己也有类似的问题，我的项目中RAG做的不深入。我的情况是自己手搓了一个RAG项目，你可以看到就在笔记中过的。既然你项目都有现成的RAG，虽然不是你做的，但你去把代码弄明白，RAG的各个机制弄清楚，弄懂了就是你做的啊。只要你掌握了RAG知识，把这块就变成是你自己做的。

3.推理优化。你完全可以参考我笔记中的思路。如果你仔细看我的笔记，我是从工程出发，应用是我的强项，但是为了处理面试官会问我算法的问题。我专门按照自己加入了推理，微调，量化的机制。 这里不复杂的，你要去学相关知识，然后把流程用最简单的方法走一遍，然后准备下相关问题，加入到简历，自信去说就行了。怎么做的我笔记里都有，你完全可以复用。 这里不用担心掌握不深入。原因是通常应用的岗面试官即使问了算法，他也问不深，他自己不可能做到又专应用又专算法的。 所以在我们时间有限的前提下，先把应用弄好，算法按我的思路先去走个大面。

粉丝追问： 我不是真的在生产项目做的，只是自己美化的，那这样会露馅吗？

回答：不会啊，大胆一点，我不是让你纯编，我是让你做一遍。我自己项目也是我自己做了一遍量化微调的，很简单的，你又不用负责效果，把流程走通，把面试会问的提前想好就行。常见的问题我都汇总了，你可参考，而且实测一般应用岗问的算法，问了也不深入，我的笔记里都是一次次迭代，你就照着我的笔记复习。我就是完全根据面试官问的情况来总结的答案的深浅，如果他问得深，我的笔记也会写得深入。所以你算法这部分，你把我笔记的内容弄懂，结合你的情况想一想不用担心，这就是我实践总结出来的，面试完全够用，而且我还在一直迭代，你就放心照着准备吧。

5.3 网页RAG项目选型

粉丝提问：我现在做的项目是一个在网上搜索相关演员信息的项目，然后匹配准确度，目前我召回的文本和内容不相关，有一些什么优化方法啊，我需要从多个维度，比如名字+项目+身份 去联合搜索吗？分块我用什么好？用你的项目的递归分块吗？ 我需要使用数据库吗？怎么包装一下呢？我目前项目就是用Langgraph写的demo，能帮我做一下技术选型吗？

回答：1.分关键字检索思路是对的，但是你看是不是不是分开检索，而是把这些关键词合并。拆开反而会放大噪声。比如 张三，红楼梦，贾宝玉，你提取这三个关键字，然后组合搜索。

2.分块问题。分块问题也不是说用我的递归切块比较好。我这个方法是比较通用性的方法。针对你的问题，网络文档，通常噪声比较大，可以先去噪声（导航，广告之类的），提取出纯文本以后可以用最普通的滑动窗切块。因为一般你要的信息，比如张三，红楼梦，贾宝玉，可能就在一个连贯的chunk中，用递归还容易切掉信息。

3.分块匹配一般就是粗排和精排。 你说的向量余弦相似度，这里其实有不同方法的向量可以做选型，比如BM25，OpenAI 的embedding等。你图方便可以用BM25做粗筛，然后精筛是解决你说的不相关问题的核心。可以用LLM匹配，也可以用Cross-encoder，这是两个经典方法，解决你说的召回内容不相关问题。

4.向量存储，网页搜索确实可以不做本地化存储，动态筛选了以后去做切分召回，然后销毁，这也是合理的。

5.4 我的情况可以入行大模型吗?

粉丝提问：博主你好，我本人是24年毕业生，985硕士，机械专业，毕业当年去了一家国企，本来岗位是机械研发岗的，但由于公司在数智化转型，去年被抽调去参与了一年的数智化开发工作，算是有一年的后端开发经历。我算你最早的那一批原始粉丝[害羞R]，也买了你的学习资料，非常详细，但虽然都是0基础ai经历，咱们的0基础还是差的蛮多的，你一直从事的是互联网行业，而且有可以加入ai开发的场景。但我觉得我只能从网上找项目，且几乎不太可能有机会将项目和我工作场景结合起来，但看你的面试反馈，似乎面试官对真实的项目开发经验更看重，我这样跟着网上的项目学习（即使有自己的改进），这样简历能过吗[哭惹R]？转型的可行性大吗？心理挺没底的。

回答：首先回答你的问题，你一定可以转行的。 我能理解你转行的时候其实心里觉得没底，这个大家都是这样的。我在当初想转行的时候，也很迷茫，我问了我大牛同事，他的回答是没有能不能转，只有想不想转。经过我自己的实践，我的答案是转到大模型没有你想象的那么困难。

首先以**大模型开发**为例，它的门槛和普通传统开发是一样的，普通开发大家知道吧，就是程序员，所以门槛并不高的。并不是所有程序员都是大厂的，很多中小厂的程序员，学历也不是很高，所以不要因为它沾上了大模型这个字，就把大模型开发这件事想得很难。其次，大家看一下现在的招聘市场，大模型开发相比传统开发，岗位是更多的（薪资也高一些），所以大家的机会更多，从这个角度，入行大模型开发比传统开发还容易一点。

从技术角度来说，你要做的是补齐大模型应用相关的知识，文档里都有我就不说了。你从后端开发转到大模型开发，本身你后端开发就多少有一些优势（相比前端，测试，客户端）转到大模型的岗位。

第二个，学历挺好，985硕士做开发足够了呀。很多学历比如一本这样子，其实做大模型开发也没太大问题，门槛对标的都是普通开发，不要因为沾上大模型就把它想得很难。

第三个，你工作才一年。转行这个事情真的是年限越长越难转，一年工作时间，甚至是3年时间，企业对你的包容度都很高的。甚至可以放宽到5年。我自己是4年转行开始面试的，企业对我的要求只会比你高。

第四个，面试官看重真实项目。这点确实是，毕竟是转行，都是自己做项目的，我们的项目相比天然做大模型开发的一定会有劣势，这个没办法。但是有很多解决方案，第一个不断加深项目，我一直说项目不是一下子完成的，我在面试的时候也是一边面试一边做项目。第二个就是把项目往工作上靠，结合工作场景。你说看我的面试反馈，很多面试官看重真实项目，是这样的，即使到现在，有的公司还说我项目不深。 但是你也应该看到反面啊，就面试不到20家，我就有6个offer。京东面试官即使我没做过算法，+2还很看重我，给我发算法Offer, 薪资也很高。阿里的面试官，你去看我的面经，到后面他都不在乎你项目做到什么程度，他和我讨论未来的架构，未来的扩展，他更看重的是思维能力。 你比我工作经验少3年，企业对你只会更包容。 要学的东西太多了，我一直在补充。去年12月份的我和现在的我，能力又不一样，我现在特别相信，我继续学习，不断掌握新知识，不愁面试的，我的目标已经从应用编程转向大模型全栈或者算法了。

所以怎么看，我觉得你都不用心里没底啊，更重要的是，这些东西你能不能掌握，能不能把RAG,AGENT等应用知识都学透，甚至是在学好算法，部署，入行真的没有你想的那么难。迷茫的时候就多学一下，当你真正学好了，你去面试了，市场给你答案的时候，你的问题都会解答，而且你会发现做大模型并没有你想象的那么难。而且大模型现在缺口那么大，岗位很多的，你看我笔记2025年度报告，这么多岗位，怎么会没有你一个985硕士的位置呢，你要想的是尽量学好，拿更高的Offer，而不是犹豫能不能入行。

应用核心：Agent 与 RAG【主线】

（对不起，Agent目录越来越多，我知道我应该重构一下，比如两个实战项目，PI Agent和DeepSeekHarness，可以放在一起并且放到最后面

但飞书目录重构真的很麻烦，我没想到一个很好的方案去重构目录，如果我要改动目录，标号都得一个个改，而且文档内的很多链接索引可能都乱了。

大家可以先把所有的子章节都折叠，虽然一级目录可能没有特别有条例，但一级目录只有不到20个，不过你可以自己从这一级目录中找到你需要的，然后再阅读。后面我想到好方法重构目录，**我会重构的。**）

1 提示词工程

说实话，提示词工程不管是任何培训，都会作为一个很大的重点和知识模块去讲。谷歌甚至也是专门推出了8小时的提示词工程，但是在实际面试中，真的几乎没有出现，甚至是我总结的这些简单介绍提示词工程最基本的入门八股，都完全没有被考到！

并且提示词工程其实也不难，我的建议是，你好好地把下面这个视频看完，这个视频总结了google 8小时 提示词工程的核心要点，把这个视频消化，并且总结成笔记，运用到实战中，就可以了。不用花过多时间在提示词工程上。

写于2026-05-18: Prompt工程其实2026年已经很不火了，面试问的也不多，也都被Harness，Skill这些最新技术热度盖过了。面试问的少，不建议花太多时间在上面。不过从实用性的角度，因为我们使用AI，每次都需要写提示词嘛，还是看一下，看看这视频就大概就理解了。

https://www.youtube.com/watch?v=p09yRj47kNM

1.1 简单介绍提示词工程。

（参考谷歌提示词工程课程）

提示词工程是指**设计和优化输入给llm的提示词，**以便获得更准确、有用或符合预期的输出结果的过程。

设计提示词有五大具体原则：**任务， 上下文， 参考示例， 评估标准，迭代**.

具体来讲：

任务：指我们要清晰地告诉LLM我们的任务，可以通过让llm角色扮演和规定输出格式的方法提高效果。

上下文信息：我们要提供足够的背景信息，让AI更好的理解，比如提供项目背景，已知数据，限制条件等。

参考示例：可以给出一些标准的输入输出的例子来供模型参考。

评估：每次拿到输出，判断结果是否符合预期。

迭代：如果不满足要求，我们需要反复进行prompt的优化和迭代。

1.2 提示词工程技巧

提示词技巧可以分为**单次**和**多阶段**技巧来讲。

单次技巧：

1.重新检查提示词框架是否合理，比如是否包含了明确的任务，上下文信息，参考等。

2.将提示词拆分成更短的句子。避免使用冗长复杂的句子，拆分成更短的，有条理的句子。

3.尝试用不同的措辞，或者换成类比任务。

4.引入一些限制条件。

多次技巧：

1.Prompt Chaining: 将多个提示词串联起来，每一步的输出作为下一步的输入，从而构建一个多阶段的任务流程。

2.Chain of thought prompting : 引导模型逐步思考，通过在提示中加入推理过程，让模型在回答问题时展示中间推理步骤，而不是直接给出答案。

3.Tree of thought prompting : 鼓励模型探索多个可能的路径或者思路，像树状结构一样展开，然后选择最优路径。

1.3 面试真题

写于2026年8月，在我的经验看来，提示词工程几乎不会单独被拿出来考，比如问你COT是什么？我分析了这么多面试真题，对于提示词的考察遇到两次，基本就是宏观上，问你有没有什么写提示词的技巧，让你自由发挥。

1.3.1 如何设计 Prompt 来提升任务稳定性与准确性?

📝 **答案：** 我会把 Prompt 当成一份任务协议来设计，而不是简单描述一句“你要做什么”。

首先，需要明确任务目标、输入含义、执行规则、边界条件和输出格式，尽量减少模型自行猜测。例如分类任务要明确标签集合、每个标签的判定标准，以及信息不足或标签冲突时如何处理。

其次，为模型提供与任务相关且可信的上下文。事实型任务可以结合 RAG、数据库或工具查询，并要求模型只根据提供的证据回答；证据不足时应明确返回“不确定”或请求补充信息，而不是自行编造。对于规则复杂或容易混淆的任务，可以提供少量 Few-shot 示例，尤其要覆盖正常情况、边界情况和反例。

对于复杂任务，不建议让模型在一次调用中同时完成理解、检索、决策和执行，而应拆分成多个步骤，例如：

`意图识别 → 信息检索 → 生成候选结果 → 结果校验 → 输出或重试`

每个步骤都应有明确的输入和输出。下游程序需要使用结果时，应尽量采用 JSON Schema、枚举或函数调用等结构化输出，并在程序侧继续做字段、类型、业务规则和权限校验。

最后，要通过评测而不是主观感觉判断 Prompt 是否有效。可以建立包含正常、边界和历史失败样本的测试集，将 Prompt、模型版本和参数一起版本化，通过任务成功率、事实准确率、格式合法率、工具调用正确率等指标进行回归测试。

需要注意的是，Prompt 只能降低模型行为的不确定性，不能提供绝对保证。事实核验、工具权限、API 重试、业务规则和高风险操作确认，仍然需要由程序和系统机制完成。

选自202607宇树科技Agent开发一面

1.3.2 在项目过程中，怎么去做提示词的压缩和裁剪?

选自2026年12月夸克千问Agent开发二面。 答案我就写在面经里了，辛苦大家点击链接跳转观看。

---

2 大模型参数设置

（这个知识点其实面试官主动考察的并不多，目前只有平安证券的一面面试官问到了，你有没有对模型做一些参数的调整，怎么设置的penalty， temperature. 目前没有其它面试官问到。所以总体来说，这不是一个特别常考的考点。 从面试角度，可以适当放低学习优先级。另外其实如果你学好了本章，其实在一些和面试官讨论的过程，讨论一些发散性的知识的时候，很常被用到。 比如说夸克千问面试官在和你讨论说：如何避免一些极端情况的输出呀？那这个时候，调整模型参数将会是一个可讨论点。另外从实战角度，其实在你的项目中，去调整参数做适配也挺常用的对吧，从这两个角度，也是可以推荐去学一下的）

2.1 常见参数和选择策略

LLM 推理参数详解表

常见场景配置推荐

2.2 Temperature & Top-p

2.2.1 详细原理

（想深入理解这两个参数的原理，请看这个视频）

在大语言模型生成文本时，通常会用到两个参数：**temperature** 和 **top-p**，它们决定了采样的**随机性**和**多样性**。

首先我说一下大模型生成回答的**大致原理**：

模型先为每个候选 token 计算分数（logits），再通过 softmax 转换成概率分布，然后根据这些概率进行加权采样。

在这个过程中：

temperature 会在 softmax 前调整 logits，温度低时分布更尖锐，模型更确定，输出更保守；温度高时分布更平滑，增加随机性，输出更有创意。

top-p 则通过“核采样”限制采样范围，只在累计概率达到 p 的最小集合中选择 token，p 越小越集中，p 越大越多样。

简单来说，**temperature 控制概率分布的陡峭度，影响确定性；top-p 控制采样范围，平衡质量与多样性，两者结合决定生成结果的稳定性与创造性。**

2.2.2 不同参数组合情况

(进一步补充，以防面试官追问)

temperature 低 + top-p 小

- **效果**：输出非常稳定、保守，几乎总是选择概率最高的 token。
- **特点**：答案更确定、更接近训练数据，适合需要精确、严谨的场景（如代码生成、事实问答）。
- **缺点**：缺乏创造性，容易重复。

---

temperature 高 + top-p 大

- **效果**：输出非常多样化，随机性强，可能出现意料之外的表达。
- **特点**：适合创意写作、头脑风暴，能生成新颖内容。
- **缺点**：连贯性和逻辑性可能下降，甚至出现不相关或错误内容。

---

temperature 低 + top-p 大

- **效果**：整体仍偏稳定，但允许更多候选 token，输出稍微丰富。
- **特点**：在保持连贯的同时，增加一点灵活性，适合对话或内容生成。
- **缺点**：仍然不够有创意。

---

temperature 高 + top-p 小

- **效果**：在有限候选集中引入较强随机性，输出有一定变化但不会太离谱。
- **特点**：适合需要一定创造性但又不能完全失控的场景，比如营销文案。
- **缺点**：可能出现轻微逻辑跳跃。

2.3 你的项目是如何设置参数的?

（这个是真实地面试问题了，平安证券1面问的。大家根据答案也想想自己的项目应该如何回答）

我的项目中，总体是一个分析型的任务。核心需求是稳定，准确。总体设置的温度较低，在0.1\~0.3，然后Top_p保持默认，让模型在合理参数内选择，不用太多调整。惩罚因子设置较低的值，因为形成建议可能会多次提到相同的地点，时间。所以惩罚因子设置在0.1左右。

3 大模型幻觉

> 幻觉问题，其实就是本质是什么，产生的原因是什么，如何解决，我力争把产生的原因写得完整，另外我也加入了一个高级进阶的内容，加了一个清华大学的论文，相当于一个亮点，你回答的时候，答出来会特别新颖。

3.1 大模型幻觉的本质

幻觉的本质不是模型被显式要求去编造，而是**训练方式和训练数据共同塑造出的一种生成倾向**：当模型面对不确定问题时，它不会天然停下来，而是继续生成一个概率上最像答案的文本。

大模型底层是一个**概率预测器**，不是知识库。它内部没有"事实文件柜"，只有一张词与词之间的概率地图。生成一句话 = 沿着最可能的路径一步一步往下走。所以它从设计上就**没有"真/假"概念**，只有"像不像下一句该说的话"。

所以，大模型不是原生的事实验证系统，而是文本生成系统。它的幻觉不是来自"故意撒谎"，而是来自**下一个 token 预测的训练方式、训练语料中的问答分布，以及后训练阶段对有用性和流畅性的偏好。**

3.2 产生的原因

幻觉的成因可以拆成一条因果链：**根因 → 燃料 → 扳机 → 电路。**这四层不是并列关系，而是层层推进：底层机制决定它会生成，数据缺陷决定它容易走错，训练偏好决定它不愿停下，最后这些倾向会在神经元层面形成可观测的行为电路。

（1）根因：底层机制决定它是概率预测器，不是知识库

- 没有事实数据库，只有词与词的概率地图。
- 生成回答 = 一步一步走最可能的路径。
- 错误会随长度累积："出发偏一寸，到终点差千里"。

- 所以**只要它开口，就有可能编。**这一层解释的是：大模型为什么天然不是事实验证系统，而是文本生成系统。

（2）燃料：训练数据让它学到的世界本身就是残缺、扭曲的

- **训练截止日期**：截止日之后的事实，地图上根本没有路径。

- **数据稀缺 / 分布不均**：冷门知识只出现几次，路径模糊。

- **数据污染**：互联网中包含错误信息、阴谋论、甚至被故意"数据投毒"的内容。

- **过拟合**：模型死记硬背片段，遇到稍变的问题就乱拼。

这些数据就像模型生成时使用的"地图"。地图本身有空白、有错路、有污染，模型自然会在低置信度场景下走偏。但**地图有洞并不必然导致幻觉**——只要模型肯停下来，说"这里没有足够信息"就行。问题在于，第三层让它更倾向于继续说。

（3）扳机：训练方式与后训练偏好让它从"顺着说"走向"尽量给答案"

- **预训练**：没有"讨好用户"的显式目标，只是在海量语料中学习"给定前文，预测下一个 token"。但人类语料里"被问→给答"的模式占绝对多数，模型因此学到"顺着上文继续说"的生成惯性。

- **后训练（SFT / RLHF / DPO）**：人类偏好数据通常更喜欢清楚、完整、有帮助的回答，而不是频繁的"我不知道"。这会进一步强化模型"尽量给出一个可用答案"的倾向。

- **结果**：模型不是被显式训练去撒谎，而是在训练方式、数据分布和后训练偏好的共同作用下，形成了"不确定也继续生成答案形式文本"的倾向。

这一层就是幻觉真正被触发的地方：当模型遇到不知道、不确定、信息不足的问题时，它不是天然选择停止，而是被生成惯性和后训练偏好推着继续给出答案形式的文本。到这一步，幻觉已经不是单纯的工程 bug，而是**生成式训练范式自然带来的副作用**。

（4）电路：H-Neurons 承载了"过度顺从"的生成倾向

清华大学的论文从神经元层面验证了上面这条因果链：

- 在模型几十亿参数中，只有**极少数神经元（不到十万分之一）**与幻觉相关，被命名为 **H-Neurons**。

- 扰动实验证明：放大 H-Neurons → 模型接受错误前提、改口附和用户、甚至越狱配合违规请求。

- 关键发现：H-Neurons 驱动的**不是"错误记忆"，而是"过度顺从"** 这种行为本身。

也就是说，前三层解释了幻觉为什么会产生，H-Neurons 则说明这种倾向在模型内部已经有了可观测的物理载体。更致命的是：这条电路和"说话流畅"的能力**深度纠缠**，无法简单删除——切掉它，模型也不会说话了。这也是为什么幻觉**无法 100% 消除**，目前业界更多是通过工程手段缓解，而不是彻底根治。

3.3 解决方案

既然每一层都无法被彻底"修好"，工程上的策略就是**绕过去、监测它、约束它、复核它**。常见思路包括 RAG、RLHF/DPO、CoT/Self-Ask、多模型投票、外部工具、用户侧验证，以及 H-Neurons 监测。整体可以分成几类：

（1）外部工具 / Web Search / API / 数据库 —— 让模型不要只靠参数回答

当问题涉及最新信息、实时数据或企业私有数据时，不应该让模型完全依赖训练参数，而是让它调用**外部工具**，比如 Web Search、搜索引擎、第三方 API、数据库、MCP 工具等。这样模型从"凭记忆生成"变成"先查

证再生成"，直接缓解训练截止日期和知识边界问题。

（2）RAG—— 把回答锚定到可信知识库

RAG 是外部工具体系里最重要的一类。它把 Agent 的回答和**企业知识库 / 文档库 / 向量数据库 / 搜索引擎**结合，先检索相关资料，再基于检索结果生成回答。它的核心价值是：让模型不要独自承担事实真实性，而是把答案锚定到可追溯的数据来源上。这是工业界最主流、性价比最高的幻觉缓解方案。

（3）RLHF / DPO 人类反馈微调 —— 重写它的后训练偏好

用人类标注反馈训练模型，**显式奖励"承认低置信度"** 的行为，让模型在没把握时使用预设回答，而不是编造事实。本质是从训练激励层面修正"必须给答案"的倾向。

（4）提示词技巧：多轮验证 + 反思（CoT / Self-Ask）

通过 **Chain-of-Thought + Self-Ask** 等模式，让 LLM 在推理时**分配更多计算资源**（更多步骤、更多思考时间），强迫它把过程展开、自我核查。通常能显著提升准确性、连贯性和稳健性。

（5）融合模型 / 投票机制 —— 用冗余对抗单模型偏差

让多个模型或多个 Agent 协同工作，通过投票或加权得出最终答案，降低单一模型幻觉的影响。本质是用"群体智慧"稀释个体的讨好倾向。

（6）事实核查与用户侧验证 —— 不把模型输出直接当事实

用户侧防范：对模型给出的日期、统计、引用、法律案例、医学建议等高风险信息，要主动验证。可以检查原始来源、搜索精确引用、换一种问法追问、要求模型给出依据，并警惕"过于完美"或"异常具体但没有来源"的答案。这个方法不是从模型内部消除幻觉，而是在使用环节建立事实校验机制。

（7）（亮点）H-Neurons 幻觉探测器 —— 监测而不是直接删除

基于清华论文的发现，理论上可以并行监测 H-Neurons 的实时激活，一旦发现异常飙升，就给出"该答案可能在编"的信号，提示用户或下游 Agent 复核。需要注意的是，论文并不是说可以直接删除 H-Neurons 来彻底解决幻觉，因为这些神经元和模型的语言流畅性深度纠缠。更现实的方向是**监测、抑制、预警**，而不是粗暴切除。

3.3.1 附录：H-Neurons 论文概括

> 既然提到了论文，防止面试官深入问论文的事，我总结了这个论文概括。其实不用看本身，这是搞研究的人要弄的。我们大概理解一下就行。

这篇论文的核心问题是：大模型为什么会在不知道答案时仍然自信地编造？研究者没有只从训练数据、提示词或模型规模这些外部因素解释，而是尝试从模型内部的神经元活动入手，寻找和幻觉行为相关的可观测结构。

论文提出了 **H-Neurons** 这个概念，指一小部分与幻觉行为高度相关的神经元。研究发现，这些神经元并不是简单存储了错误知识，而更像是承载了一种"过度顺从"的行为倾向：当用户的问题带有错误前提、上下文存在误导，或者用户反复质疑模型时，模型更容易顺着用户继续说，而不是纠正问题或承认不确定。

更重要的是，实验表明：增强这些 H-Neurons 的影响，模型会更容易接受错误前提、附和用户、生成错误答案；抑制它们，则可以减少这类过度顺从行为。但论文也指出，不能把这些神经元简单删除，因为它们和模型生成自然、流畅、有帮助回答的能力是纠缠在一起的。也就是说，幻觉不是一个可以被单点切除的独立 bug，而是和大模型的语言生成能力共享部分底层机制。

这篇论文的启发是：未来减少幻觉，除了 RAG、工具调用、事实核查这些工程方法，也可以探索模型内部的实时监测机制。例如在生成过程中观察 H-Neurons 是否异常激活，如果风险升高，就触发复核、检索或拒答策略。它提供的是一种前沿研究方向：**监测和约束幻觉电路，而不是幻想把幻觉彻底删除。**

3.4 面试问题照答版

Q1：什么是大模型幻觉？它是如何产生的？

大模型幻觉指的是模型自信地输出**不符合事实**的内容。它的本质不是模型被显式训练去编造，而是底层机制、训练数据分布和后训练偏好共同作用的结果。

第一，大模型底层是一个**概率预测器**，并不真正理解事实，只是根据上下文预测最可能生成的单词序列。

第二，当遇到训练数据没覆盖、或者覆盖很少的低置信度场景时，模型仍然会按照"顺着上下文继续说"的生成惯性，输出一个看起来合理的答案。

第三，后训练阶段（SFT / RLHF / DPO）的人类偏好通常更喜欢清楚、完整、有帮助的回答，这会进一步强化模型"尽量给出一个可用答案"的倾向。

所以，幻觉不是单一原因导致的，而是大模型的生成机制、数据缺陷和后训练偏好共同造成的。

Q2：如何减少大模型 / Agent 的幻觉？

工程上主要有几类做法。

第一，接入 **Web Search、搜索引擎、第三方 API、数据库、MCP 工具** 等外部工具，让模型不要只靠参数回答。

第二，使用 **RAG**，将 Agent 回答和外部知识库或向量数据库结合，确保生成内容有真实依据。

第三，使用 **RLHF / DPO 人类反馈微调**，显式奖励模型在低置信度场景下使用预设回答，而不是编造事实。

第四，使用 **Chain-of-Thought + Self-Ask** 等提示词技巧，让模型在推理过程中分配更多计算资源、进行多轮验证和反思。

第五，使用 **融合模型或投票机制**，通过多个模型或 Agent 协同降低单一模型幻觉的影响。

第六，在使用侧建立事实核查机制，对日期、引用、统计、法律和医学等高风险内容进行二次验证。

第七，我也看了最近清华大学的一篇论文，它从模型结构层面分析了幻觉问题。论文认为，模型中有一小部分神经元和幻觉行为高度相关，被称为 **H-Neurons**。但这些神经元和模型"说话流畅"的能力共用同一套电路，不能简单直接切除。更现实的做法是，在使用模型的时候监测每一次 H-Neurons 的激活值，一旦发现异常升高，就进行实时预警，提示系统触发复核、检索或人工确认。

最后总结一下，现阶段大模型幻觉是无法被 100% 消除的。我们能做的是通过外部工具、RAG、反馈微调、推理反思、多模型协同、事实核查和 H-Neurons 监测来**缓解幻觉，而不是彻底根治幻觉。**

---

4 ReAct

这是一个很简单的知识点，确实一个在各种框架里面都广泛运用的技术。这里讲的不多，但是都是精髓，配一个讲解视频，我觉得你应该可以理解这个概念。最常见的面试真题就是：ReAct如何实现的？我也附在了后面。

4.1 ReAct 是什么

ReAct 是 **Reasoning（推理）+ Acting（行动）** 的结合。它让 Agent 在完成任务时交替进行推理和行动，并根据行动结果继续判断下一步。

其核心过程是：

```Plain Text
Reason → Act → Observation → Reason → ..... → 完成任务
```

```
**关键概念**

- **Reason（推理）**：模型分析当前任务和已有信息，**制定计划**，或者根据上一次行动结果进行**反思**，判断下一步应该做什么。
- **Act（行动）**：模型决定调用某个工具，并提供调用工具所需的参数。
- **Observation（观察）**：外部框架执行工具后返回的真实结果。模型根据这个结果继续推理和行动。

---

<a id="doxcnAnc41Tve3WJKET8AnJRfWc"></a>
### 4.2 ReAct 如何实现

几乎主流的框架，比如DeepSeekHarness，ClaudeCode，Openclaw，他们的Agent内核都是用的ReAct。面试最常考的是它是如何实现的。先看看这个图，理解它的流程：

![](images/S71rbxOvtouYtPxvC6UcnNrinEc.png)

核心思路：

1. 提示词告诉AI，让他思考下一步做什么，任务做完没有，要不要调用工具。
2. 外部写一个Harness循环，处理模型吐出的结果。如果模型认为没做完，要调用工具，那么就调用工具（这对应ReAct概念里**Act**）。**如果模型认为做完了，流程结束。**
3. 外部调用工具后，将结果返回给模型（这对应ReAct概念的**Observation**），**模型继续思考：下一步做什么，任务做完了吗(对应ReAct里面的**Reasoning**)，进入循环。



```Plain Text
while (任务未结束) {
```
```

```
    模型回答 = 调用模型(
        用户问题 + 对话历史 + 工具列表
    )

    if (模型要求调用工具) {
        工具结果 = 框架执行工具()
        把工具结果加入对话历史
        continue
    }

    // 没有工具调用，普通文本就是最终答案
    结束流程
}
```

4.3 面试真题

（选自202607 宇树科技Agent开发一面）

4.3.1 ReAct 的核心思想是什么？他是如何实现的？

💡 **思路：** 这个问题问得很具体了，如果面试官直接问，**你介绍一下什么是React**，也可以按这个提问的思路介绍（是什么，如何实现的。）

答题思路是先把ReAct的概念讲清楚就好了，Reasoning 和 Act，分别是什么，概念讲清楚。（答案中第一段）

然后我这里补了一个工程上的实现（答案中第二段），以开源框架DSH为例，具体举例，引经据典，把话题引到你熟悉的框架上。当然你也可以替换成你熟悉的其他框架。

📝 **答案：**

ReAct 是 Reasoning，也就是推理，和 Acting，也就是行动的结合。它的核心思想是让 Agent **把推理和行动交替进行**：模型先判断下一步应该做什么，再通过工具采取行动，然后根据行动结果继续判断，而不是只依靠模型内部知识一次性生成答案。

这里的 Reason 主要包含两部分：一部分是**规划**，也就是分析当前任务、判断下一步应该做什么；另一部分是**反思**，也就是拿到工具执行结果以后，判断结果是否有效、任务是否已经完成，以及是否需要调整原来的计划。

Act 是采取行动。在现在的 Agent 系统中，通常表现为模型选择一个工具，并生成对应的调用参数。比如模型发现缺少实时信息，就可以调用搜索工具；需要验证代码，就可以调用代码执行工具。工具执行后，外部框架会把真实结果返回给模型。

在工程实现上，外部框架会把用户问题、历史消息和可用工具交给模型。模型如果认为需要调用工具，就返回**结构化的 tool-call**；框架负责执行工具，并把执行结果加入对话历史，再次调用模型。如果模型认为信息已经足够，就不再调用工具，而是直接返回普通文本答案，这时框架结束流程。

比如 DSH 的 **ReactLoopAgent** 就是这样实现的：模型负责推理、选择工具以及判断任务是否完成，外部框架负责执行工具、回传结果和维持整个循环。这样 Agent 就可以在执行过程中根据真实结果不断调整，而不是从一开始就固定一套计划执行到底。

4.3.2 ReAct 循环中，如何检测并跳出无效的“思考—行动”循环?

（选自202607 宇树科技Agent开发二面）

💡 思路

我这里是参考《AI Engineering》中关于执行后评估、失败反思与重新规划的讨论，以及 DeepSeek Harness（DSH）和 OpenAI Codex 的开源实现，可以发现它们在应对重复执行和失败恢复上的思路相近。综合起来，可以按 **检测、纠偏、停止兜底** 三层回答。但它们的实际实现和覆盖范围不同，以下是综合整理的设计思路，不代表两个框架都完整实现了所有措施。

📝 **答案：**

我会分成三层处理： **检测发现问题，纠偏尝试解决，停止控制损失。**

第一层：检测——判断是否陷入无效循环。

ReAct 不断进行“思考—行动—观察”，但持续执行不代表任务持续推进。因此，需要记录每一步的工具调用、参数、Observation 和任务状态，关注两类信号：

- **重复行为**：相同工具和参数连续出现，或者在几个动作之间周期性往返。
- **没有有效进展**：连续多步没有获得有效的新信息、没有完成子目标，或者始终遇到同一个未解决的错误。

检测可以由 Harness 的程序逻辑完成，也可以由 LLM 辅助判断，或者两者结合。规则明确的重复调用、错误次数由程序检测；有明确任务指标时，可以检查测试通过数、子目标完成情况；对于“新信息是否有价值”“是否仍被同一个问题阻塞”等语义问题，可以由 Harness 收集执行轨迹，再交给 LLM 评估。

判断“有没有进展、是不是卡住了”仍属于检测；决定“接下来换什么方法”才属于纠偏。 不能只比较思考文本是否相似，也不能把正常轮询、分页查询等重复操作直接判定为无效循环。

实际案例：DSH 默认按“工具名 + 规范化参数”统计连续重复次数，在第 **3、5、8** 次触发提醒。它不比较工具返回结果，因此属于重复调用检测，而不是完整的语义无进展检测。

第二层：纠偏——改变当前策略，尝试继续完成任务。

检测到异常后，不必立即终止整个任务，而是先打断原样重复，把已尝试的动作、返回结果和失败原因反馈给 Agent，根据原因调整：

- **参数错误**：根据工具返回的错误修正参数。
- **暂态故障**：有限重试，必要时退避等待。
- **策略错误**：分析失败原因，换查询条件、换工具或重新规划。
- **缺少关键信息**：请求用户澄清，而不是继续猜测和重试。

关键是 **下一步要有实际策略变化，而不只是思考文本换一种说法。**纠偏和重规划本身也应有次数限制，并继续受同一个全局预算约束，避免形成新的无限循环。

实际案例：DSH 会向后续模型请求注入提醒，要求分析上次结果，选择不同动作、不同参数，或在证据足够时结束。这与《AI Engineering》中根据反馈反思并生成新计划的思路相呼应。但它是软纠偏，不会直接阻止工具调用，也不能保证模型一定采纳。

第三层：停止兜底——恢复无效时控制损失。

如果纠偏后仍然无法推进，或者遇到不可恢复的问题，就需要停止路径，防止无限消耗时间、Token 和费用。设计上可以在 Harness 中设置最大执行步数、总耗时、累计 Token 或费用预算，以及同类失败和恢复尝试的次数上限，并支持用户取消。

达到条件后，由 Harness 停止继续执行，报告已完成部分、未解决问题、停止原因，以及需要用户补充的信息或人工处理的事项。**停止是控制损失，不代表任务成功完成。** 预算耗尽、用户取消或遇到不可恢复问题时，可以直接停止，不必等纠偏手段全部尝试完；预算耗尽本身也不等于证明任务陷入了无效循环。

实际案例：Codex 在特定场景下有真正的停止保护，例如 Goal 执行失败达到条件后停止自动续跑，Guardian 审批拒绝达到阈值后中断回合。但这些是局部保护，不能泛化为所有 ReAct 循环都有统一硬上限。DSH 的默认重复提醒器也没有“重复到一定次数就强制退出”的机制。

📝 **简化答案（面试照答版）：**

我会从 **检测、纠偏和停止兜底** 三层处理。核心是看任务有没有实际进展，而不是模型有没有一直在思考。这些机制需要由外部执行框架，也就是 Harness 配合实现，不能只靠提示词让模型自己避免死循环。

第一，检测是否卡住。 我会记录每一步的工具、参数、返回结果和任务状态，主要看两类信号：一类是重复，比如连续使用相同参数调用同一个工具；另一类是没有进展，比如多次搜索没有新信息，或者反复修改代码，测试仍然报同一个错误。明确的重复行为可以用程序规则判断，任务进展可以看测试结果、子目标完成情况，必要时再让模型辅助评估。但要结合场景，不能把正常轮询、分页查询误判成死循环。

第二，发现问题后先纠偏。 把已经尝试的方法和失败结果反馈给模型，让它分析原因，而不是原样再试。参数错了就修正参数，临时网络故障可以有限重试；方法不对就换工具、换查询条件或重新规划；缺少用户信息就主动澄清。比如连续搜索都没有找到资料，就应该调整关键词或数据源，而不是反复提交同一个查询。关键是下一步的行动要有变化，不能只是换一种说法继续思考，重新规划本身也要限制次数。

第三，设置硬性停止条件。 在框架中设置最大步数、总耗时、Token 或费用预算，以及同类失败的次数上限。如果纠偏后仍然无效，或者达到预算，就由框架停止执行，不能指望模型一定会主动停下来。停止后要告诉用户哪些已经完成、哪里卡住、还需要什么信息，必要时转人工，而不是把停止当成任务成功。

总结就是： **检测发现问题，纠偏尝试解决，停止控制损失。** 既给 Agent 调整和恢复的机会，也保证它不会无限消耗资源。

4.3.3 用户问题模糊或有歧义时，如何做澄清性对话?

（选自202607 宇树科技Agent开发二面）

💡 **思路：**

采用 **“提示词约束 + 模型判断 + 工具调用”** 的方式，将澄清作为 ReAct 循环中的一种工具交互。 **模型负责决定“是否要问、问什么”，执行框架负责“展示问题、等待回答、将答案交回模型”。** 不一定需要单独训练歧义分类器。

📝 **答案：**

我会给 Agent 提供一个澄清工具，比如 `ask_user_question`，让模型在需求不明确时主动提问。具体分四步：

1. **提示词约束：明确什么时候需要澄清。**

在提示词中要求模型先看上下文，能自己查清楚的就不问；如果还有影响下一步的歧义，或缺少需要用户决定的关键信息，就提问，不要猜。

1. **模型判断：决定是否调用澄清工具，并生成问题。**

模型判断信息不足时，调用澄清工具。问题要具体，优先问最关键的信息，必要时给出选项，不要一次问太多。

1. **工具交互：界面展示问题，当前任务等待用户回答。**

执行框架把问题展示给用户，让当前任务等待回答。这里暂停的只是当前任务，不影响系统处理其他请求，也不是结束整个 ReAct 循环。

1. **结果回传：把答案作为工具结果，继续原任务。**

用户回答后，框架把答案作为工具结果放回上下文，模型根据补充信息继续执行。如果仍有关键歧义，就继续追问；信息足够就不再问。

比如，用户说“帮我增加报表导出功能”，模型看完现有代码仍不确定格式，就问“需要 CSV 还是 Excel？”用户选了 Excel，模型就根据这个结果继续处理。涉及高风险操作时，仍需单独确认授权。

总结：提示词规定什么时候问，模型决定问什么，框架负责展示问题和回传答案。整个流程就是“提问 → 等待 → 回传 → 继续”。

---

5 Function Calling

（Function call的能力其实一直都是 Agent非常重要的考点，虽然在实际面试中考察的并不是特别多，以下两个问题是来自于某次面试原题）

5.1 Function calling是如何被训练出来的

Function calling是通过在模型训练的**预训练阶段**使用**自监督学习**，在**微调阶段**使用**SFT技术**，以及在**对齐阶段**使用**RLHF技术**来使模型具有能够理解用户意图并生成函数结构化的函数调用请求的能力。

具体来讲：

预训练阶段的自监督学习使得模型能够理解自然语言。

微调阶段的SFT技术使用标注数据让模型理解何时该调用函数，调用函数的格式是什么。

对齐阶段使用RLHF，使用人类的反馈来优化调用结果，使模型的调用更符合人类的习惯。

5.2 工具调用失败问题

在使用一些Function call的接口设计的时候，如果参数比较多，实际使用中发现LLM调用失败（可能是没理解调用的时机）或者是缺少了必填的参数（导致调用失败），这个应该如何避免呢?

1.**重构函数设计：**《代码整洁之道》中就写过，函数参数最理想的是0个，其次是1，再次是2，我们应该尽量避免有3个及以上的参数，所以出现这种情况我们可以先考虑是否有必要重构。

2.**可以拆分函数：**让函数满足单一职责的原则，避免有过多的参数。

3.**优化JSON Schema设计：** 在JSON Schema中清晰地标明参数类型，是否必填，默认值，写清楚参数描述。

4.**优化函数/参数命名：** 让命名更具有描述性，便于LLM理解。

5.**优化提示词设计：**可以在提示词中加入一些引导，给出一些调用示例。

6.**可以增加一些验证，反馈，重试机制：**比如在调用前对参数进行验证，判断是否有误，如果调用失败，可以把失败的信息传回给LLM，让它利用这个信息重试。

---

6 MCP

（这章的所有内容都是面试出现过的真题，都是我根据真题来总结的，都建议掌握，这个问题是MCP的概念，肯定是必背的）

MCP 是一种**标准化接口协议，**统一了AI模型与外部资源的连接方式，用于让LLM能够访问外部的**数据**，**工具**和**提示词**。

MCP采用**客户端-服务端架构**，包含三个关键组件：Host, Client,和Server. 其中Host是运行LLM的应用程序。Client是在Host中运行，负责与MCP Server通信。Server中暴露了**工具**，**数据资源**，**提示模板**等，供LLM使用。

MCP的典型优点是**动态能力更新(**工具参数变更时无需重写客户端代码)，**接口标准化**(不同工具统一接入，减少开发成本)的特点。

6.1 MCP 和 Function calling 的区别

（面试也出现过，挺基础的一个考点）

《Agentic design pattern》第10章专门有对这一问题进行讲解。

6.2 MCP 本地Server和远端Server 的对比

（我根据这个视频https://www.youtube.com/watch?v=wlerSyHzoCI总结的，但是如果有余力，还是建议自己看一下这个视频，表格的东西不难，重要是你自己能够理解，视频里都有，大家可以更深入理解一下，我在字节跳动一面的讲解视频中应该也会过一下这个表格，大概讲一下）

6.3 MCP的传输协议

(最开始我复习的时候就是弄清楚了两种协议，本地:Stdio 远端：SSE，这也是有一些视频这么讲的。但我在面试的时候，其实因为这个答案被挑战过，因为现在SSE在MCP里已经被替代了，所以我有了以下答案，相对来说这个答案是比较完整的，这题也是面试会考的题）

MCP的传输协议针对于**本地服务器**和**远端服务器**有明显区别。

对于远端MCP Server, 协议在 2025 年 3 月进行了重大更新：原先采用 **有状态的 HTTP 长连接**，并强制使用 **Server-Sent Events (SSE)** 来实现服务器实时推送，这种方式复杂且对网络稳定性要求高。新版本改为 **无状态协议**，主要通过 **标准的 GET 和 POST HTTP 请求**完成交互，SSE 变为可选增强功能，只有在需要实时推送时才启用。这一改动降低了实现复杂度，提高了兼容性和灵活性，使协议更接近 REST 风格，易于集成和扩展。

对于 **本地 MCP Server**，通常运行在开发者机器上，直接通过本地接口或 STDIO（标准 I/O）进行通信，不涉及网络传输，因此不需要复杂的认证或长连接管理。

6.4 MCP Server Tool设计原则

> 来自真实面试问题。本身直接被问到倒是比较少，但是当你的项目，比如就是设计了MCP的Server，比如我们的RAG项目。这时候聊到你设置了哪些工具的时候，这个问题可能会被问到。
>
> **资料来源**：MCP 官方规范 modelcontextprotocol.io/specification/2025-06-18/server/tools

第一，入参设计——参数最少化、类型严格、约束内联。 LLM 是根据 inputSchema 来构造参数的，schema 本身就是"给 LLM 看的说明书"。原则是：必填参数尽量少，能给默认值就给默认值，用枚举收敛可选项而不是让 LLM 自由发挥，类型要明确。约束直接写在 schema 里，比如最小值、最大值、默认值，这样 Server 端校验方便，LLM 传参准确率也更高。我们项目里查询工具的返回条数参数就限定了范围和默认值，LLM 不传就用默认值，传了超出范围直接截断。

第二，出参设计——结构化返回，LLM 友好。 MCP 规范支持 outputSchema 声明返回结构，让 Client 和 LLM 提前知道数据长什么样，做类型校验和自动解析。同时规范要求向后兼容——结构化返回的同时要附带

一个纯文本版本，保证老 Client 也能读。我们项目的做法是统一格式化输出，把检索结果转成带标题、来源引用、置信分数的可读文本，而不是丢裸 JSON 给 LLM——因为 LLM 读自然语言比读嵌套 JSON 效率高得多。

第三，错误处理——双层错误机制，错误信息对 LLM 友好。 MCP 规范定义了两层错误：**协议级错误**用标准 JSON-RPC error code 处理参数非法、tool 找不到等问题；**业务级错误**通过 `isError: true` 加 error message 报告。关键原则是：**错误信息要告诉 LLM 下一步该怎么办，而不是丢一堆堆栈。** 比如返回"集合不存在，可用的集合有 A、B、C"，而不是返回 Python 异常堆栈。前者 LLM 能自主修正重试，后者只能把堆栈原样输出给用户看。

第四，Tool Annotations——声明副作用，让 Client 做安全决策。 MCP 规范定义了 annotations 字段，包含四个关键属性：是否只读、是否有破坏性、是否幂等、是否与外部世界交互。这些标注不影响执行逻辑，但 **Client 会据此决定是否需要弹确认框。**比如只读的查询工具可以自动执行，但有破坏性的删除工具就应该让用户二次确认。我们项目的三个 tool 全部是只读查询，都应该标注为只读。

第五，粒度控制——单一职责，工具间可组合。 MCP 的设计是 model-controlled——LLM 自己决定调用顺序和组合方式。所以工具拆分的粒度很关键：太粗 LLM 无法灵活组合，太细调用链路过长容易出错。我们项目拆了三个 tool：列出集合 → 查看文档详情 → 查询知识库，形成"先发现 → 再了解 → 最后查询"的自然工作流，上游的输出可以直接作为下游的输入参数。

第六，安全与校验——输入校验和输出脱敏。 MCP 规范明确要求 Server 必须校验所有输入、实现访问控制、限流、清洗输出。即使 LLM 传了符合 schema 的参数，Server 端仍然要做业务层面的校验——比如路径穿越检查、注入防护、敏感信息脱敏。同时规范强调"始终要有人类在回路中"，高危操作必须有确认机制。

总结来说，MCP tool 的编写是"六位一体"的：description 决定 LLM 能不能找到你，inputSchema 决定传参准不准，outputSchema 决定结果可不可解析，error handling 决定失败后能不能自愈，annotations 决定 Client 信不信任你，粒度和安全则决定整个系统是否健壮可控。

6.5 MCP的新技术&趋势

2026年8月份，阿里淘天秋招大模型开发一面面试官问了这么三个MCP的问题： MCP 是为了解决什么问题提出的？ - 考察MCP的基本理解，概念和定义。 MCP 的工具形式给通用 Agent 带来了什么问题？有没有了解替代传统 MCP 调用形式的方法？ - 这两个问题考察了，有没有关注MCP的技术发展，替代技术和局限性。从26年Agent SKILL 火起来以后，MCP确实受到了挺大的冲击，热度没有去年那么大。 但诚然MCP并没有完全消失，面试中也会出现。这三个问题，问的也很符合时代的趋势，在当前环境下，MCP受到了哪些冲击，又有哪些迭代？ 为了讲清楚这些问题，我要给大家介绍4个技术：

- **Tool Search：工具已接入，需要时才把相关工具的说明加载给模型。**
- **Code Mode：模型写流程，程序执行流程，不必每一步都让模型重新决定。**
- **Agent Skills：把某类任务的做法、脚本和资料封装起来，按需复用。**
- **MCP 2026 年演进：协议继续向易部署、可缓存、可授权和支持长任务的方向发展。**

弄明白了这4项技术后，对于MCP的面试问题回答，我们的回答会更有深度，也更加与时俱进。

6.5.1 Tool Search

痛点

接入的 MCP 工具多了，如果客户端把所有工具说明都放进模型上下文，就会浪费 token，也增加模型选错工具的可能。

比如用户只是想查一封邮件，模型却收到了邮件、日历、CRM 等 100 个工具的说明，大部分都用不上。

解决方案

Tool Search 可以主要在客户端实现，不必修改 MCP Server：

- **MCP Server 端**：继续提供原来的工具，通过 `tools/list` 返回工具定义，通过 `tools/call` 执行调用。
- **客户端**：获取并保存完整工具目录，向模型提供搜索入口；根据任务检索相关工具，只把需要的工具说明加入模型上下文。模型选定工具后，客户端照常通过 MCP 调用。

例如处理邮件任务时，客户端只加载“搜索邮件”和“读取邮件”的说明，其他工具暂时不加载。常用工具可以直接提供，不必每次搜索。

收益

- **少占上下文**：不用加载大量无关工具说明，减少输入 token 开销。
- **更容易选工具**：模型主要面对与当前任务相关的工具，减少干扰。

搜索本身也有开销，并可能漏掉工具，因此更适合工具较多的场景；只有几个简单工具时，直接加载即可。

6.5.2 Code Mode

> Code Mode 本质上是一种MCP 协议的用法，是2026年**美国的IT公司 Cloudflare 提出的一种使用方法。**协议本身内容没变，改变了MCP Server暴露的工具的思想，让客户端（大模型端）负责整个流程的脚本编排，把脚本的编排放在MCP Server侧。减少大模型多轮调用次数和等待时间。

MCP 解决了 Agent 如何统一接入外部工具的问题，但**工具接进来了，不代表多步任务就能高效完成。** 传统MCP的用法是：MCP Server 把查询客户、查询订单等业务操作分别暴露成工具，模型选择工具，客户端通过 MCP 发起调用，再把结果交给模型决定下一步。

只有一两次查询时，这种方式很直接。但当一个任务需要连续调用很多工具、反复查询分页、处理大量中间数据时，这种使用方式的痛点就出现了：**服务端只负责执行一个个业务操作，模型却要反复承担步骤编排和中间结果处理。**

上一章的 Tool Search 能减少一开始加载的工具说明，却不能单独解决执行阶段的这些往返。假设我们让 Agent 完成一个任务：**“查一下这 100 位客户的订单，找出本月消费超过 1 万元的人。”** 模型先通过 MCP

工具请求客户名单，拿到结果后再决定查谁的订单；订单返回后，又继续决定下一次调用。如果接口还有分页，就要继续取下一页，最后再汇总。

在这个过程中，**大量普通程序就能完成的机械步骤，也让模型反复参与了**：

- **等待多**：每轮都要等待模型读取结果、生成下一步操作。
- **上下文长**：大量订单作为工具结果交给模型，但用户真正需要的只是筛选后的名单。
- **处理不划算**：循环、求和、排序、去重本来就适合普通程序，不必让模型靠阅读原始数据完成。

既然模型会写代码，我们能不能让它把处理流程写出来，交给程序连续执行，而不是每一步都回来问它？**这就是 Code Mode 的核心思路：模型负责生成处理逻辑，运行时负责执行，只把必要结果返回模型。**

对于刚才的任务，模型可以生成这样的脚本逻辑：

```Plain Text
查询客户列表；
循环处理每位客户：
  查询本月订单，如果有下一页就继续查询；
  计算订单总额；
  如果总额超过 1 万元，加入结果；
返回符合条件的客户及总额。
```

> 根据这样的逻辑：**模型不需要预先知道具体有多少页、要调用多少次，只需要写清楚处理规则。** 大量订单留在执行环境里，最终只有筛选后的名单进入模型上下文。
>
> 那么，这段程序由谁执行，它又怎么知道能调用哪些业务能力？Cloudflare 演讲中的实现给了一个具体答案：**服务端不再把每个业务 API 都单独暴露成 MCP 工具，而是提供“能力发现”和“代码执行”两个入口：`search` 与 `execute`。**
>
> 以前，MCP Server 可能直接暴露 a、b、c、d、e 五个业务工具，模型分别发起调用。现在，面向客户端的 MCP 工具列表可以只有 `search` 和 `execute`，而 a、b、c、d、e 仍然存在，只是作为执行环境中的函数，供模型生成的脚本调用。
>
> 模型、客户端和服务侧的协作过程是：
>
> 1. **先查接口。** 模型调用 `search`，客户端通过 MCP 把请求发给服务端，获取本次需要的业务函数及参数说明。视频中的 `search` 接受代码，用它查询 OpenAPI 规格，并不是把全部接口文档交给模型。
> 2. **再写流程。** 模型根据说明，写出“查客户、循环查订单、求和、筛选”的脚本。它写的是组合已有能力的逻辑，不是重新实现订单查询服务。
> 3. **提交执行。** 模型调用 `execute`，客户端通过 MCP 提交整段脚本，服务侧的受控运行时执行代码。代码中的业务函数可以直接调用相应 HTTP API。
> 4. **返回结果。** 服务侧完成查询和统计后，把脚本输出返回客户端，再交给模型回答用户。中间每一次订单查询不必都让模型参与。
>
> ```Plain Text
> 模型通过 search 了解需要的接口
>                ↓
> 模型写脚本，通过 execute 提交
>                ↓
> 服务侧执行：查客户 → 循环查订单 → 计算 → 筛选
>                ↓
> 只把必要结果返回模型
> ```

这里有两个不同的收益：`search` 让接口说明按需加载，避免开局塞入大量工具定义；`execute` 让多个业务操作在程序里完成，减少执行期间的模型往返和中间上下文。**搜索是配套机制，生成并执行程序才是 Code Mode 的核心。**

最后总结来说，CodeMode这个技术，并没有改变MCP协议本身，而是改变了使用方式。 MCP Server不暴露具体的业务接口，而只暴露让客户端（大模型侧）能够进行流程编排的必要接口，让客户端写整个过程的运行脚本，交给MCP Server侧统一执行。 省去模型多轮推理，等待，也减少不必要的上下文（只将最后结果送给大模型）。

面试时可以这样总结：Code Mode 把可程序化的工具编排和中间数据处理，从模型逐步决策转移到外部运行时。模型写流程，程序执行流程，从而减少模型往返和上下文开销。Cloudflare 的 search/execute 是它与 MCP 结合的一种服务侧实现，而不是替代 MCP 的新协议。

如果看了上面的内容你还一知半解，我们用这个图来做一下类比，希望你可以通过这个图明白Code Mode的优势。

6.5.3 Agent Skills

Agent Skills 的基础用法就不展开了。在之前的 Skills 笔记和讲解视频中，我们已经拿它和 MCP 做过对比。围绕 Skills，也经常能看到“MCP 是不是被淘汰了”这样的讨论，但不能把这种说法直接当作技术结论。

相信大家对 Skills 和 MCP 都不陌生。这一节之所以仍然讲 Agent Skills，主要是因为它们太容易被放在一起比较。包括面试官问的“**有没有了解替代传统 MCP 调用形式的方法？**”，Skills 也是一个回答的出发点。这里就集中总结它们的区别、优缺点和适用场景，作为一道常见的面试对比题来准备。

SKILLS VS MCP

Skills 主要教 Agent“这类事情应该怎么做”，MCP 主要解决“怎样以统一方式接入外部能力”。它们不是同一层的技术，所以既可以配合，也会在某些具体场景中形成替代选择。

还是拿“生成月度经营报告”举例。MCP Server 可以提供查询订单、查询退款、读取客户信息等工具，但有了这些工具，Agent 不一定知道公司要求使用哪种统计口径、怎样排除异常、按什么模板输出。Skill 则可以把这些做法写清楚，并附带统计脚本和报告模板。

从提供的内容来看，Skill 更像一份可以附带工具材料的操作手册。 `SKILL.md` 写明任务步骤，旁边可以放脚本、参考资料和模板，重点回答“这类任务应该怎么完成”。MCP 则约定外部能力如何被发现和调用，包括工具定义、参数 schema 和结果格式等，重点回答“有哪些能力可用，以及怎样调用”。

从执行方式来看，Skill 本身不是执行器。 Agent 读完指南，仍要依靠宿主提供的工具、CLI 或脚本运行时干活，也可以按指南调用 MCP 工具。而通过 MCP 调用时，由客户端发送请求，服务器执行对应操作，再返回结果。

从复用方式来看，Skills 复用的是做事方法和资源，MCP 复用的是服务接入。 一份 Skill 可以交给兼容的 Agent 使用，但对方仍需具备相应的工具和环境；一个 MCP 服务可以让多个兼容应用接入，不必各自重新实现一套业务系统适配。

两者也都有边界：**写了 Skill，不代表自动获得了业务数据、账号权限或执行环境；接了 MCP，也不代表自动获得了完整业务流程，或者模型一定能选对工具。**

为什么大家会觉得 Skills 能替代 MCP？因为有些任务原本就不需要额外包装一层 MCP。

比如“查 GitHub Issue，再整理成周报”，可以有两种做法：

- 接入 GitHub MCP，让 Agent 调用相关工具。
- 写一个 Skill，指导 Agent 使用已经安装并授权的 `gh` 命令查询，再运行脚本整理结果。

如果命令行工具已经成熟可用，第二种方式可能更轻量，不必为了同一套操作再搭建、配置和维护一个 MCP Server。处理本地文件、生成文档、运行测试等任务，也经常适合“Skill + 现有工具或脚本”。

但反过来，如果没有现成的访问能力，只写一个 Skill 就替代不了 MCP 接入。 比如，公司内部有一套订单系统，数据库不允许员工电脑直接连接，也没有提供可用的 CLI。公司已经通过一个受控的 MCP Server 提供订单查询：服务端核验员工身份，只返回其有权查看的店铺订单，并记录查询日志。

这时候，即使写一个 Skill，告诉 Agent“先查订单，再扣除退款，最后生成经营报告”，Agent 也不会因此获得订单数据。**Skill 提供了操作指南，但查询入口、身份校验和数据权限仍然依赖已有的业务服务。** 实际方案应该是 Skill 规定统计方法，Agent 通过已授权的 MCP 工具取数，再完成报告。

优缺点

从优缺点看，Skills 的优势是容易编写、容易随项目版本管理，能够把团队经验、模板和经过测试的脚本一起复用；按需加载也避免了把完整操作指南一直放在上下文中。代价是仍要维护指南和脚本，并处理运行环境、依赖版本、凭据和权限。把 Skill 复制给另一个 Agent，不代表对方就有相同的工具和环境；模型读取了指南，也不保证每一步都正确执行。

MCP 的优势则是标准化接入：如果多个 Agent 应用都需要访问同一个业务系统，可以让它们复用同一个 MCP 服务，而不是每个应用各写一套适配。服务端也便于集中实现业务校验、权限检查和审计。代价是需要维护客户端与服务端的接入、鉴权、兼容性和运行环境；对于简单任务，额外包装服务可能不划算。如果全量加载工具说明、逐步回传大量结果，还会遇到前两章讨论的效率问题。

实际选型

- **已有工具，但缺少做事方法：优先考虑 Skills。** 例如统一代码审查步骤、固定报告口径、套用文档模板。
- **缺少业务系统接入，或者要让多个 Agent 应用复用同一服务：考虑 MCP。** 例如统一访问内部订单、CRM、工单系统。它不是唯一接入方式，但能减少各应用重复适配。
- **既需要接入能力，又需要业务流程：Skills + MCP 配合。** 例如 Skill 规定报表口径和检查步骤，MCP 查询真实订单，脚本负责计算，再按模板输出。

6.5.4 2026新特性

因为MCP 是24年11月就推出的技术，现在如果再考它，回答的时候，我不想你的回答还和2025年那会烂大街，所以这里讲一下2026年MCP的演进，新的技术，在我们下次介绍的时候，可以把这些技术加进去。让答案与时俱进，与众不同。 这里我们不会特别展开，因为其实问的不多，热度也不算很高，我也没看到面试官

专门问这个新特性，所以我的策略是大致理解一下，记一下就够了，不深入，然后下次面试官问到我MCP的内容的时候，我在会介绍MCP的内容的时候，把它的新特性带进去。

这里不背版本更新清单，只记三个与 Agent 发展趋势有关的方向：

1. **长任务：从回答一个问题，到持续完成一件事。**

Agent 的工程重点不再只有怎么写 Prompt，还包括工作流 / 图编排、任务状态和长时间执行。例如做一次深度调研、跑完整个测试流程，都不是一次调用马上就能返回的。

MCP 也在完善 Tasks 等任务机制，让服务端能够返回任务标识，供客户端查询状态、获取结果或补充输入，并继续推进事件通知等机制的协同。**它正在适应“工具在后台持续干活”，而不只是“一问一答”。** MCP 承接这些任务交互，但不替代 Agent 自身的工作流编排。

1. **按需发现：从堆工具、堆上下文，到只提供当前需要的能力。**

通用 Agent 接入的服务越来越多，不能把所有工具说明一直塞给模型。上下文管理越来越强调“需要什么，再加载什么”。

MCP 生态已有客户端 Tool Search、渐进式披露等实践，协议路线图也在推进服务器侧渐进发现。**方向是让 Agent 面对大量工具时，也能按任务找到需要的能力，而不是先读完整个工具目录。**

1. **身份与权限委托：从人在旁边操作，到 Agent 代表人自主执行。**

当 Agent 开始后台运行、操作企业系统，甚至把任务交给子 Agent，就必须回答：它代表谁？能操作什么？交给子 Agent 的权限能不能更小？

MCP 正在加强授权机制，并推进 Agent 身份、身份联合与权限委托等方向。**目标是让自主执行有明确的权限边界，而不是把人的全部权限直接交给每一个 Agent。**

介绍 MCP 时，可以用下面这段话收尾：

> MCP 也一直在跟着 Agent 的发展演进：任务越来越长，它就完善长任务机制；工具越来越多，它就朝按需发现发展；Agent 越来越自主，它就加强身份和权限委托。所以它不只是一个早期的工具接入协议，也在向长期运行、大规模工具使用和企业落地的需求靠近。

记住三个关键词就够了：**长任务、按需发现、身份与权限委托。** 这些方向中既有现成机制，也有仍在推进的标准化工作，不代表所有产品已经全部支持。

6.5.5 面试真题

以下三个问题，都是选自202608 阿里淘天秋招大模型开发一面

1. 谈一谈你对MCP的理解？

（子问题）MCP 是为了解决什么问题提出的？

💡 **思路**

2026年了，这个问题的介绍，我就希望可以答一点不一样的，答一点符合时代趋势的内容。其实阿里202608阿里淘天问的这个三个MCP的问题，就是一个很好的介绍MCP技术的一个思路，我们可以从 **概念 - 局限性 - 替代方案 - 发展趋势（新技术）**这四个方面递进讲解，结合时代最新趋势回答。

📝 参考回答

【概念】MCP 最初解决的是 AI 应用接入外**部工具和数据时，接口不统一、适配重复的问题**。比如多个 Agent 都要接 GitHub、数据库和企业订单系统，没有统一协议时，每个应用往往都要做自己的适配。MCP 把能力描述、工具发现和调用方式标准化，让服务提供方实现一次 MCP Server，不同的兼容应用就能复用。它的核心价值不是让模型突然会用工具，而是降低工具接入和复用的成本。

【局限性】它也有它的局限性：**统一了接入方式，并不意味着工具使用就一定高效。**工具多的时候，**如果把所有工具说明都塞进上下文，会增加 token 开销和选择难度**；复杂任务如果每一步都让模型调用工具、读取完整结果，也会带来很多等待和中间数据，增加了延迟。

【替代方案】针对这些问题，现在有几类改进或替代方案。**Tool Search** 按任务搜索并加载需要的工具定义；**Code Mode** 让模型写脚本，把多步调用、循环和数据处理交给运行时，减少模型往返；**Skills 则封装做事方法**，配合现成 CLI 或脚本，在部分场景下可以不再专门接 MCP，但也可以与 MCP 配合。所以这些方案不代表 MCP 整体被替代，很多改变的是它的使用方式。

【发展趋势（新特性）】MCP 本身也在适应 Agent 的发展：**任务越来越长，就完善长任务机制；工具越来越多，就推进按需发现；Agent 越来越自主，就加强身份和权限委托。这些方向既有现成能力，也有仍在推进的标准化工作，目的都是让 MCP 更适合长期运行、大规模工具使用和企业落地。**

【总结】所以我理解，MCP 最初解决的是“让不同 AI 应用用统一的方式接入外部能力”，现在关注的则是“接入之后，怎样更高效、更可靠、更可控地使用这些能力”。它并没有因为新技术出现就失去价值，而是在与工具搜索、程序化执行和 Skills 等方案配合，适应越来越复杂的 Agent 任务。

1. MCP有哪些局限性?

💡 思路

这个题思考点，我们就可以从上面，我们定义的各个和MCP类似/配合使用的技术出发来回答，比如：1.Agent SKILL - 它更轻量， 相应地就对应MCP的流程比较重。 2.Code Mode - 对应模型和MCP Server多次交互，延迟高的问题。 3.Tool Search - 工具太多，污染上下文。

📝 参考回答

主要有三个问题：简单场景下接入和维护偏重，复杂任务中多轮交互延迟高，以及工具数量多时上下文开销大。

【接入和维护偏重】首先，MCP 虽然统一了接入方式，但仍然需要配置和维护服务、处理连接、鉴权与兼容性。如果任务本来通过已有命令或脚本就能完成，再包装一层 MCP 服务，可能增加不必要的工程成本。所以像 Agent Skills，可以封装任务指南和脚本，让模型直接使用已有执行能力。这可以简化部分场景的接入。

【多轮交互延迟高】其次，传统逐步调用方式下，工具每执行一步，都可能要把结果交回模型，再由模型决定下一步。任务链路越长，模型推理和等待的次数就越多，大量中间数据也会反复进入上下文。Cloudflare今年

提出，**Code Mode** 的技术，让模型先生成包含调用和处理逻辑的脚本，再由外部运行时连续执行，只返回必要结果。就很大程度缓解了这个问题。

【上下文开销大】最后，通用 Agent 的工具数量和种类往往比较多。如果把所有工具描述和参数说明一次性加载给模型，就会占用大量上下文，增加 token 成本；相似工具之间还可能互相干扰，让模型更难选对。针对这一问题，Tool Search：先根据任务检索相关工具，再加载需要的定义，避免把无关说明全部塞给模型，缓解了MCP的这个局限性。

1. 有没有了解过替代传统 MCP 调用形式的方案?

💡 思路

这个题思考点，还是可以从我们上面讲的**SKILL**, **Code Mode, Tool Search** 来出发讲解。 虽然严格意义上来说，Code Mode 和 Tool Search不是MCP的替代方案，不过这个比较沾边，我估计面试官不会那么**较真**。如果说替代方案，目前我想的就是SKILL这个技术，也暂时没想到更多。

📝 参考回答

有了解过，主要有三类：用代码组织多步调用、按需搜索工具，以及用 Skills 配合现成 CLI 或脚本完成任务。这里我会区分一下：有些替代的是传统的工具调用方式，有些则是在具体场景下不再使用 MCP 接入。

【程序化调用】第一类是让模型生成一段程序，而不是每调用一个工具都重新决定下一步，这种方式通常叫 Code Mode。模型把调用、循环、判断和数据处理写进脚本，由受控运行时执行，只回传必要结果，减少模型往返和中间数据占用。比如 Cloudflare 的一种实现，是通过 MCP 提供 search 和 execute 两个入口，模型先查需要的接口，再提交脚本，由服务侧完成多步操作。底层仍然可以使用 MCP，改变的是调用的组织和执行方式。它适合多步骤、批量处理的任务，但需要做好执行隔离和权限控制。

【按需发现工具】第二类是工具搜索，也就是 Tool Search。不再把所有工具说明一次性交给模型，而是根据当前任务检索相关工具，再加载需要的定义。这可以在客户端完成，MCP Server 继续提供原来的工具，实际调用方式不变。它适合工具数量多、任务类型不固定的通用 Agent，也可以和 Code Mode 配合：先找到工具，再用代码组织执行。

【Skills 配合现有工具】第三类是把任务指南、脚本和资源封装成 Agent Skills，让 Agent 直接使用已有 CLI、SDK 或脚本。如果执行能力已经具备，就不一定需要再包装一层 MCP 服务，接入和维护可能更轻量。但真正访问外部系统的是这些工具，不是 Skill 本身；如果缺少数据访问通道和授权，仍然需要 MCP 或其他 API 接入。

【总结】所以我不会把这些方案理解为谁全面淘汰谁，而是按场景选择：流程长、机械步骤多，就考虑程序化调用；工具多，就考虑按需搜索；已有成熟命令和脚本，就用 Skills 组织复用。核心是减少不必要的模型参与和接入成本，同时保留所需的权限与可靠性保障。

6.5.6 讲解视频

https://www.bilibili.com/video/BV1aahz6kETD/?spm_id_from=333.1387.homepage.video_card.click

6.6 MCP的参考资料

基础视频：推荐两个，马克的和爬爬虾的，这两视频可以从你0MCP基础到了解知道MCP是啥，可以动手接一接。

进阶视频：马克老师的进阶视频其实挺好的，特别是讲了抓包，让我们详细了解了中间发生了什么。看完内容对于我们笔记部分 其他问题-你在项目中遇到的最难的问题，如何解决的-MCP偶尔连接失败 的问题理解有很大帮助。 这几个视频就是当时我0 MCP基础去看的，也没有看其他视频。应该来说你把这几个按顺序看懂了，MCP就理解了，自己再动手做做，看看真题就行了。

7 Agent的记忆功能

（属于**必考内容**，熟练掌握

《Agentic design pattern》第八章有对这一问题专门讲解。我把这个经典面试问题，结合了理论知识和现在最火的Openclaw结合，我相信这个回答会更让面试官眼前一亮。然后你这么回答其实把话题引到了Openclaw，所以Openclaw的知识要好好看，接着面试主动权就在我们手里了！）

Agent 的记忆功能通常分为**短期记忆**和**长期记忆。**

短期记忆通常存在于上下文窗口，包含最近的对话消息，智能体回复，工具调用结果，以及当前交互的反思内容。但是上下文窗口的容量有限，所以当多轮对话时，我们可以通过总结旧对话片段、强调关键细节等技术来维护这个上下文窗口。以 OpenClaw 为例，它的 Compaction 机制就是一个典型实现：当上下文 token 接近模型窗口上限时，系统会把最近约 20,000 tokens 的消息原样保留，更早的旧消息则用 LLM 分块生成结构化摘要来替代，摘要中会强制保留任务状态、决策理由、TODO 和关键标识符等信息，压缩后上下文变为"摘要加上最近原文"的结构，这样既释放了空间又尽量减少了信息损失。

当会话结束，短期记忆会丢失，**所以我们通常也会让 Agent 支持长期记忆。**我们通常使用一个知识库，存储智能体在各种交互场景、任务执行过程中需要长时间保存的信息。数据通常存在智能体运行环境之外，比如数据库，知识图谱，或者向量数据库中。当智能体需要使用长期记忆时，会查询外部存储，检索相关信息和数据并整合到上下文窗口中然后进行使用。**还是以 OpenClaw 为例**，它的长期记忆就是工作空间里的 Markdown 文件——`MEMORY.md` 存策展过的持久知识和偏好，`memory/YYYY-MM-DD.md` 按日期存追加式日志。在检索侧，系统会对这些文件建立混合搜索索引，同时使用向量语义搜索和 BM25 关键词搜索，按 7:3 权重做混合排序。Agent 需要回忆历史时，通过 `memory_search` 工具按需检索相关片段再拉进上下文，而不是把全部记忆都塞进去，这就是典型的 RAG 式长期记忆方案。

此外还有一个值得一提的设计，**就是短期记忆向长期记忆的自动沉淀机制。**OpenClaw 在压缩前会静默触发一轮 agent 回合，提醒模型把当前对话中重要的信息写入记忆文件，这样短期记忆中真正有价值的内容能在被压缩丢弃之前沉淀为长期记忆，保证了记忆的连续性。

7.1 介绍Langchain中的记忆功能的实现。

(短期记忆）Langchain中通过**ConversationBufferMemory**的接口，可以实现只保存几轮对话，或者使用LLM总结历史对话的短期记忆功能。

（长期记忆）通过**VectorStoreRetrieverMemory**可以将信息变成嵌入向量，存入Chroma, weaviate，FAISS的数据库中，并且支持语义搜索。

7.2 介绍Langgraph中的记忆功能的实现。

(参考https://mp.weixin.qq.com/s/XhFbLTLcSjDj0r3KGT9EOg， 这个文章对Langgraph的各方面写得很详细，想深入学可以仔细阅读，想简单学习可以当字典查。比如这个Langgraph的记忆功能，这里面就有现成答案，我直接照抄，不理解的可以看原文，里面也有讲解，原文里还有代码。）

记忆是智能体运行中记住先前交互信息的组件，是能够连贯对话的核心能力，LangGraph中提供了**短期记忆和长期记忆。**

1、短期记忆：存储当前对话上下文的信息，作用于单次会话或线程，通过thread_id（会话id）区分，通过**图状态**（State）和**检查点**（Checkpoint）实现。

2、长期记忆：长期记忆用于存储那些需要在不同会话间保留的信息。它通过 **存储库（Store）** 接口实现，类似于一个键值数据库，并支持基于向量嵌入的语义检索。与线程范围的短期记忆不同，长期记忆保存在自定义的“命名空间”中。

7.3 长期记忆的常见应用场景

（依旧是从我推荐的书籍《Agentic design pattern》中给大家总结， 在面试聊到长期记忆的时候，我们可以给面试官举一些常见应用场景下具体的长期记忆的例子，这个问题倒是不会直接被问。不过记忆部分是重点，很多场合其实会聊到。聊到了能清晰有条理的列举一些长期记忆典型例子就很好。）

https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn/blob/feature/interactive-website/14-Chapter-08-Memory-Management.md

常见的长期记忆的例子是：

1. 聊天机器人 / 对话式 AI：

存储内容：用户偏好、过往对话摘要、关键话题记录。

作用：提供个性化且连贯的交互体验，避免用户重复自我介绍，让对话更自然流畅。

2.个性化服务AI：

存储内容：用户偏好、历史行为模式、个人画像信息

作用：动态调整响应策略和推荐内容，提供更精准的个性化服务。

3.智能客服 / 售后 AI

存储内容：历史工单、问题解决记录、用户满意度反馈

作用：快速定位老用户的历史问题，避免重复排查，提升服务效率。

7.4 面试真题

7.4.1 请介绍一下Agent 记忆的常见类型与作用。

💡 **思路：** 这个就是最典型的八股问题了。大的分类上来说，Agent记忆分**长期记忆**和**短期记忆**，我参考了一些书籍，再把**长期记忆**分成：**语义记忆**，**情景记忆**，**程序性记忆**三类，方便大家理解。 答题的时候从这个角度出发，分类型讲解。同时我在**长期记忆**这里加入了举例，以最近比较火的DSH框架为例，写了例如 DSX xx，让答案更具体，侧面表明了自己看过DSH 框架，也可以把话题引导到DSH上。当然这里可以替换成用你自己的项目举例，或者其他的开源项目举例。 另外，这个回答也是另一个典型的八股：**长期记忆**主要存哪些东西 的参考答案。

📝 **答案：**

Agent 的记忆从生命周期上主要分为**短期记忆和长期记忆。**其中，**长期记忆**还可以根据信息内容进一步分为**语义记忆、情景记忆和程序性记忆。**

短期记忆也叫**工作记忆**，主要保存当前**对话历史、任务目标**、最近的**工具调用**结果以及执行过程中的中间状态。它的作用是**保持当前任务的上下文连续性**，支持 Agent 进行多轮对话和多步推理。例如在 DSH 中，用户消息、模型回复、**工具调用**和工具执行结果都会记录在当前 Session 中，并用于构造模型下一轮看到的上下文。当对话过长时，DSH 会通过 **Compaction** 将较早的历史压缩成摘要，从而控制上下文长度。

长期记忆用于**跨任务、跨会话地保存信息**，可以进一步分为三类：

第一类是**语义记忆**，**保存相对稳定的事实、概念和用户偏好，主要用于事实召回和个性化。**例如，DSH 可以通过 **Memory MCP** 接入外部记忆系统，保存“用户习惯使用 TypeScript” “用户希望使用中文回答”等信息，并在新的会话中重新检索和使用。

第二类是**情景记忆**，保存过去**发生的事件**、**对话和任务经历**，**主要用于回顾历史经验和避免重复犯错。**例如，DSH 可以通过 **JSONL** 或 **SQLite** 持久化过去的 Session，其中包括对话内容、修改过的文件、调用过的工具以及执行结果。后续可以通过会话查询能力，查找“上一次修复登录问题时采用了什么方案、测试是否通过”。

第三类是**程序性记忆**，保存完成任务的方法、操作步骤和执行策略，主要用于**复用已经形成的技能和工作流程**，提高执行效率与稳定性。例如，DSH 通过 **Skill**、**AGENTS.md** 和 **Workflow** 保存“修改代码前先读取仓库规范” “修改完成后运行对应测试” “遇到特定任务时加载相应 **Skill**”等执行规则。从功能上看，这些规则承担了**程序性记忆**的作用。

总结来说，**短期记忆**保存 Agent 当前正在做什么；**语义记忆**保存它知道什么；**情景记忆**保存它经历过什么；**程序性记忆**保存它应该怎么做。

需要注意的是，DSH 的跨会话**语义记忆**需要额外启用 **Memory MCP**，并不是默认内置开启的。

面试问题选自202607 宇树科技Agent开发一面。

7.4.2 多轮对话状态管理要考虑哪些关键要素

💡 **思路：** 多轮对话管理，其实就是说**短期记忆**。问题翻译过来就是**短期记忆**要考虑什么因素。这倒是一个很好的问题，我们设计Agent的时候，也不妨从这些角度想想，弄清楚这个问题，有助于我们把Agent产品想得比较彻底，把技术角度考虑全面。

📝 **答案：**

多轮对话状态管理主要是管理当前 Session 中的**短期记忆**，我认为需要考虑五个方面。

第一是**保存什么**。除了必要的对话历史，还要保存当前任务目标、用户已经确认的关键信息和约束、任务执行进度，以及最近的**工具调用**和执行结果。这样 Agent 才能理解上下文，并知道当前任务进行到了哪一步。

第二是**如何更新。用户可能在后续对话中补充或修改需求**，例如把预算从 5000 元改成 3000 元。外部框架需要提取这些变化，更新结构化的会话状态。对于明确修改，可以使用新值覆盖旧值；如果信息存在歧义，则让 Agent 向用户确认。因此，状态管理不能只是不断追加历史消息**，还需要外部框架进行提取、更新和校验。**

第三是**如何控制上下文长度**。对话轮数增加后，不能把所有历史无限加入模型上下文。通常可以保留最近的消息，把较早的对话**压缩成摘要，**同时单独保留任务目标、关键约束和未完成事项，从而兼顾信息完整性、Token 成本和响应速度。

第四是如何**隔离和恢复**。系统需要通过**用户 ID**、**Session ID** 或**任务 ID** 隔离不同会话，避免不同用户或不同任务之间发生状态串扰。同时，会话状态最好能够持久化，以便服务中断或**工具调用**失败后继续恢复任务。

第五是**何时清理**。会话结束或超时后，短期状态应该被清理或归档。只有稳定、以后仍有复用价值，并且允许长期保存的信息，才需要提炼后写入**长期记忆**。

总结来说，多轮对话状态管理需要回答五个问题：保存什么、如何更新、如何压缩、如何隔离和恢复，以及何时清理或沉淀为**长期记忆**。它不是简单地追加上下文，而是由外部框架维护一份**准确、精简并且可恢复的当前会话状态**。

面试问题选自202607 宇树科技Agent开发一面。

7.4.3 字节一面真题：会话记忆

真题来源：2026年10月字节一面 · 第2组原题

主问题：企业 AI 助手项目中，基于滑动窗口和摘要压缩的会话记忆是怎么实现的？

思路、窗口预算和场景追问见：滑动窗口与摘要压缩。

7.5 滑动窗口与摘要压缩

真题来源：2026年10月字节一面 · 第2组原题

主问题：企业 AI 助手项目中，基于滑动窗口和摘要压缩的会话记忆是怎么实现的？

💡 **答题思路**

这个问题我先讲一下什么是滑动窗口，AI上下文压缩里面的滑动窗口通常是指：**随着对话不断进行，保留近期一段对话的原文，让模型能够直接参考最近的交互细节；更早的对话则移出原文保留范围，必要信息通过摘要继续提供。** 所谓“滑动”，就是这个原文保留范围会随着对话推进，逐渐从较早的对话移向较新的对话。

下图展示的是一种“近期原文窗口 + 历史摘要”的实现方式。 当上下文达到压缩阈值时，保留最近 N 轮对话原文，将更早的内容压缩成摘要；如果已经有旧摘要，就将旧摘要和本次需要压缩的内容融合为新摘要。压缩完成后，新对话继续追加，直到下一次触发压缩，再次推进原文窗口的边界。

通过滑动窗口保留近期原文+历史信息压缩的方式，是现在Agent管理上下文的一种思路。各家Agent产品遵循这个思路，但是具体实现仍然要考虑很多细节，各个Agent产品的记忆管理模块的差异也体现在这些细节，比如：

1. 滑动窗口的内容是按照token数还是N轮对话数，如果是N轮对话，如果一轮对话就巨长无比要怎么处理？

2. 什么时候进行压缩？因为我们不能等窗口完全塞满，还要给模型输出、工具结果以及摘要生成留余量。

3. 在哪里切断历史？如果要切断的地方，前后语义是连贯的，切断会导致语义断裂怎么办？

4. 怎么给旧的信息做摘要？

5. 某些信息是否要单独保留，不能被摘要掉？比如硬性规则之类的？这样的话就需要先存到外部，然后做完摘要后再召回。

6. 工具结果超长，或者压缩失败怎么办，要不要重试？

这个问题估计就是同学简历写的Agent的压缩机制：基于滑窗和压缩摘要做的管理，面试官问他怎么做的。通过这个问题，希望大家理解这个滑动窗+摘要的机制以及在作者策略的时候，我们要考虑哪些因素。至于具体的答案，我们完全可以直接以第5个问题，claudecode的压缩机制来当做这个问题的回答。

追问：1. 滑动窗口大小是怎么确定的？大概设置多少轮？

💡 **答题思路**

其实通常我们说滑动窗保留最近几轮对话，是为了表达方便，但是在实际项目中，**通常是以Token为度量单位，保留近期多少个Token。**原因也很好理解，因为一轮对话可能很长也可能很短，定死保留最近N轮对话，最后保留的长度差异会巨大。 近期的上下文窗口保留的太长：会占用太长的上下文，压缩给到工具，检索等其他场景的空间，同时每次带着太大的上下文也会导致Token花费太大。但是如果保留的窗口太小，会导致最近交互的细节容易丢失，指代、纠错、工具结果可能衔接不上，表现为Agent容易忘事，很笨。

以DSH为例，**他的上下文滑动窗口是floor((上下文容量 − 输出预留) × 16%)，比如模型上下文窗口最大200,000 token**，要保证预留20,000 token。默认近期原文预算就是(200,000 − 20,000) × 16% = 28,800 token。当然我相信这个DSH做过了实验，测试，做了各种取舍和权衡定的一个比较通用的数值。

具体到我们的回答，推荐以这个思路来回答：

1. **考虑留给滑动窗口的 token 预算。** 因为输出预留、历史摘要、固定规则，以及本次检索和工具返回的材料等，都会占用上下文容量，所以需要先给这些内容分配空间，并保留一定的安全余量，再确定近期对话原文可以占用多少 token。压缩后也不能把上下文塞满，还需要给后续新增对话留下增长空间。
2. **考虑自己的业务场景。** 主要判断：下一步任务对近期对话细节的依赖有多强，这些细节能不能通过摘要或结构化状态替代。

- 如果是简短、相对独立的客服问答，回答主要依赖当前问题和知识检索，历史只需要补充指代关系、产品型号、用户条件等，通常可以分配**较小的原文预算。**
- 如果是编程、故障排查等连续任务，后续操作经常需要对照之前的代码、报错、修改和测试结果，摘要可能遗漏关键细节，因此通常需要**较大的原文预算。**
- 如果是多步业务办理，要看订单号、已确认字段、当前流程等是否已经结构化保存。如果有独立的状态管理，就可以**减少对聊天原文的依赖**；如果这些信息全靠对话承载，就需要保留更多原文，但不能只靠扩大窗口来保证业务状态可靠。

3. **通过真实业务会话评测，确定最终大小。** 先设置几个候选预算，回放包含多轮指代、用户纠错、长工具结果、跨压缩继续任务等情况的会话，观察是否遗漏最新要求、重复询问、重复执行，以及任务成功率、成本和压缩频率如何。**最终选择能够满足任务质量、同时成本可接受的预算，而不是认为保留越多越好。**

🎤 参考答案

我认为这个问题应该结合具体的业务场景，结合真实评估值来确定，而不是直接采用行业默认值，具体来讲:

首先，**看能分给近期原文多少 token 预算。**模型上下文还要容纳固定规则、历史摘要、检索和工具材料，并预留输出和安全余量，所以不能全留给聊天历史。窗口太大会挤占其他材料的空间、增加成本，太小又容易丢失近期细节，压缩后也要给新对话留出增长空间。

其次，**根据业务对原文细节的依赖调整大小。**简短、相对独立的客服问答，通常可以少留一些；编程、排障需要连续对照代码、报错和测试结果，通常要多留一些。如果是业务办理，关键信息和流程进度已经独立保存、能准确提供给模型，也可以缩小原文窗口，主要保留理解指代和最近纠正所需的交互。

最后，**用真实会话评测确定参数。**设置几个候选预算，比较压缩后能否继续完成任务，有没有遗漏要求、重复询问或重复操作，同时看成本、延迟和压缩频率，选择满足质量要求且成本可接受的预算。

2. 哪些场景适合滑动窗口？哪些场景更适合直接通过摘要进行上下文压缩?

💡 答题思路

其实上面这个问题的思路里面，已经涵盖了这个问题的答案。核心思路是：对近期精确细节依赖强，就多保留原文；主要依赖历史结论、或关键信息能够可靠召回，就可以缩小原文窗口、更多依赖摘要。我也让AI查了一下目前的产品，基本上来说目前主流的做法都是滑动窗+摘要的形式，很少有完全抛弃滑动窗，只使用摘要来做的。

🎤 参考答案

我认为这两类方法通常不是二选一。现在主流做法仍然是“**近期原文滑动窗口 + 历史摘要**”，很少完全丢掉滑动窗口、只靠摘要。选择时主要看两件事：下一步对近期精确细节的依赖有多强，以及关键信息能不能通过

摘要或外部状态可靠召回。

更适合保留较大滑动窗口的，是后续还要对照近期原文细节的场景。 比如编程和故障排查，后面经常要回看具体代码、报错、已经做过的修改和测试结果，这些内容一旦被摘要，很容易丢掉关键细节，导致模型重复尝试或改错位置。用户刚刚纠正过要求、对话里有未消解的指代，或者一轮工具结果还要被下一步直接使用时，也应该把这些内容留在原文窗口里。

更适合缩小原文窗口、更多依赖摘要的，是后续主要使用历史结论，而不是逐字细节的场景。 比如简短、相对独立的客服问答，回答主要依赖当前问题和知识检索，历史通常只需要补充指代关系、产品型号、用户条件等结论，就可以分配较小的原文预算。多步业务办理也类似：如果订单号、已确认字段和流程进度已经结构化保存，并能在需要时准确召回，就不必把整段办理对话都留在上下文里，摘要负责提供背景，结构化状态负责保证关键事实不丢。此种情况即使历史很长、但后面只需要记住已做决策、约束和结论时，也能保证Agent可以记住关键信息，提供稳定可靠的服务。

不过，即使是后一类场景，我也不会把近期窗口减到零。最近一两轮里的指代、纠正和尚未完成的操作，摘要很容易概括失真，所以通常仍保留一小段最新原文，再把更早的历史交给摘要。

---

8 Multi-Agent 架构

（Multi-agent 设计架构应该来说是Agent项目里最常用的设计方法了。面试的时候难免会和面试官讲到。面试时我也不止一次被问到， 多Agent 和 单Agent架构的区别。这里的重点就是弄清楚，多Agent 有哪些架构？各个不同架构的优缺点是什么？我们应该怎么设计架构？如果知道了这些，比如后面实战中，小红书面试时和你讨论架构设计的时候，其实你就可以比较游刃有余了。 其实这些问题都在这个论文中（https://arxiv.org/pdf/2512.08296v1 ），这个论文不光回答了这些问题，回答的还很深入，从数学，实验数据的角度来分析的。本节整理了相关问题，和参考资料）

参考资料：

https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn/blob/feature/interactive-website/13-Chapter-07-Multi-Agent-Collaboration.md（这个资料比较浅，对于多Agent架构设计这块显得不够，可以作为多Agent入门参考资料）

https://arxiv.org/pdf/2512.08296v1 （这个论文很权威，对于多Agent问题讲得非常深入，包括架构设计，如何选择Agent架构，非常建议阅读，**说实话你能把整个论文吃透，在Agent架构这里就掌握得差不多了**，但是直接看原文可能比较费力，可以看一些相关对这个论文的总结文章，比如下面推荐的文章。笔记部分也是根据这个论文内容来整理的。另外大家在讲这个Agent架构/多Agent的时候，也可以自信的告诉面试官，你是看了这个google的最新的论文来得到的知识，或者面试官让你分享最近看的论文的时候，也可以讲一篇，这篇是25年12月google刚发布的，具有时效性和权威性。）

以下是一些针对于这个论文的辅助资料，其实你能消化清楚这个论文

，可以不用看辅助资料，如果读论文吃力，可以配合一些辅助资料。

https://mp.weixin.qq.com/s/QD9oYVrD7mHKKZu802vovQ （上面这篇论文的辅助资料，论文原文可能太深奥，这个是对上面论文的总结）

https://zhuanlan.zhihu.com/p/1982756998425632940（辅助资料）

）

8.1 典型的Agent架构

（看着就是几种模式，其实你更要想清楚，这几种模式为什么好/不好，使用什么场景，理解复杂度指标，这个需要一些理解成本的，不要以为这个很简单，请大家结合上面资料理解一下，另外这块确实很重要，也值得我们仔细理解）

1) **SAS（Single-Agent System）单体架构**

交互类型：一个 Agent 顺序地感知→推理/规划→行动→迭代。

复杂度指标:

- LLM调用次数:O(T) (T为推理轮次)
- 顺序深度:T
- 通信开销:0
- 并行化能力:1

特点：**最简单、最稳。**当单体基线已经不错（论文里经验阈值≈**45%**），再加协调通常**收益递减或为负。**顺序型长链推理（一步错步步错）用单体更安全

适用场景:工具密集型任务、顺序推理任务、单Agent基线性能已经较高的场景。

2) **MAS（Independent）独立多体**

交互类型：多个 Agent **独立并行**做同一任务，最后做**聚合/投票/综合（Synthesis）**。

复杂度指标:

- LLM调用次数:O(N·T) + O(1)
- 通信开销:1(仅最终聚合)
- 并行化能力:N

特点：并行度最高、协调成本最低，但缺少互相纠错的机制；论文发现这种拓扑的**错误放大**最严重（**17.2×**），因为各自错误会在聚合时被“叠加/扩散”。

适用场景：**独立样本批处理**（如并行评审、并行方案生成），且**容错**要求不高的场景。若任务需要大量工具或严格一致性，谨慎使用。

3) **MAS（Centralized）中心化**

交互类型：有一个协调者/主管（Orchestrator）分解任务、分派子任务、收集结果、裁决与收敛。

复杂度指标:

- LLM调用次数:O(R·N) (R为编排轮次)
- 通信开销:R·N

特点:**稳健性更好**，论文显示集中式能把错误放大从 17.2× 控到 **4.4×**；在**可并行的金融推理**等任务上，集中式提升可达 **+80.9%**。核心在于**有权威收敛与质量门控**。
适用场景：可拆分且需要**统一口径与质量控制**的任务（财务核对、批量报告编制、数据清洗流水线）。注意主管本身可能成为**瓶颈**，需把轮数 r 控制在合理范围。

4) **MAS（Decentralized）去中心化辩论**

交互类型：Agent 之间**点对点交流/辩论（Debate）**，通过多轮互相质询与改进，最终**多数表决或共识**。

复杂度指标:

- LLM调用次数:O(D·N)
- 通信开销:$D \cdot N^2$

特点：能带来多视角对齐与相互检验，在**动态网页导航**这类需要不断相互纠偏的环境中，论文观测到去中心化略优（+9.2% 相对集中式 +0.2%）。但辩论的通信/时延开销显著。

适用场景：信息动态、需要探索与互评的任务（例如复杂检索、策略博弈）；但在**工具密集或预算受限**时，通信税会拖累表现。

5) **MAS（Hybrid）混合式（层级 + Peer）**

交互类型：在集中式的**层级管控**之上，允许**定向的点对点交流（Peer）**——既有 Orchestrator，也允许子 Agent 之间按需互通。

复杂度指标:

- LLM调用次数:O(R·N + P·M) (P为点对点轮次)
- 通信开销:R·N + P·M

特点：结合了**集中式的收敛**与**去中心化的灵活**，适合大型复杂任务，但要小心 **Peer 通信税**与**一致性管理**。设计时通常加**通信限流/白名单**，避免消息爆炸。

8.2 Agent架构设计原则

（在将这个给面试官之前，你可以直接说你是看了Google 《Towards a Science of Scaling Agent Systems》来总结的，也能体现你看论文的能力，而且你这个回答由google背书，足够经典）

我参考了Google 在25年12月发布的《Towards a Science of Scaling Agent Systems》论文，结合这一论文来回答这一问题。

谷歌的这篇论文的**核心思想**，不同架构下Agent性能受到： **Agent 数量、Agent架构、模型能力、任务属性**共同影响，它在不同的场景下，统一了工具，提示词和预算，配置了180种对照模型，用一组可量化的协调指

标 - 效率，开销，错误放大，冗余，创造了一个跨任务可泛化的预测指标，通过这个方法，我们可以从数据的角度，量化衡量和选择使用不同模型，来判断Agent性能的好坏。

抛开数据和推导，这项实验得出的原则Agent架构的设计原则是：

1.首先确认多Agent/单Agent架构

多Agent 不是一定比单Agent好。特别是在:

1.**强顺序**，长推理链，一步错，步步错的情况下。

2.**单体Agent能力已经比较强**，论文指出单体基线收益（>=45%时)

3.**工具密集且预算固定时**（因为引入多Agent，会有协调税，增加沟通成本，吞噬有效Token）

倾向于使用单Agent，盲目引入多Agent 架构反而会导致性能下降。

2.如果使用多Agent架构，根据特点和场景进行有效选择

8.3 单Agent和多Agent的对比

（如果你对这个知识点理解深入了，你发现下面的答案很多的点，你都知道为什么，可以深入解释，比如Agent适用的顺序调用，工具密集和单Agent基线收益较高的情况，比如选择错误架构方法误差的问题，这些都是我们上面看的论文和参考资料所写的，希望大家不光知道答案，还知道为什么，经得起和面试官深入交流里面的细节）

单Agent系统指的是整个系统的决策和上下文都使用一个智能体，多Agent系统指的是系统由多个智能体通过分工和并行来完成。

单Agent的优势在于**不需要承担Agent之间的协调**，通信开销和拓扑带来的误差传播，在**顺序调用，工具密集，单Agent的基线收益已经较高**（达到45%的时候）使用单Agent效果会更好，盲目使用多Agent架构反而会性能下降。

多Agent的优势在于在复杂的任务中，特别是**需要分工和并行**的问题上，可以突破单Agent的**能力边界，提高效率和鲁棒性**，但是我们也需要注意在不同的任务系统中，需要选择合适的多Agent 的架构，**错误选择架构可能也会显著放大误差**，降低收益。

快速测试：

通过本章学习/论文，请你自测你能否回答清楚以下几个问题：

1.Multi-agent 有哪些架构，他们有什么特点?

2.MAS （独立多体）型架构，为什么会放大错误？如何避免?

3.如何设计一个Agent架构？你是拍脑袋想的吗?

4.中心化架构中，过度依赖中心Agent，会有什么弊端，怎么避免?

5.哪些场景更适合单Agent，为什么?

6.这篇论文中提到了不同场景：金融分析，网页导航，工作流，游戏规划，他们分别适用于什么架构，为什么？你能不能举一反三，在你的项目场景下，用什么架构最合适？为什么？

8.4 Agent之间通信和状态管理

（这个知识点不是上面提到的论文中的内容，但是这个点和多Agent相关，有了多Agent同样要知道的它们之间是如何交互的，以及传递什么信息？虽然不算特别高频考点，但是也是被问到过的，需要准备。）

常见的Agent的通信模式：

1. Agent 间通信有三种常见模式：

- **移交（Handoff）**：一个 Agent 将执行上下文和执行权完整传递给另一个 Agent，自身退出。控制权转移，就像接力跑把棒交出去。
- **工具调用（Tool Call）**：一个 Agent（主管）将任务委托出去，保留控制权，等结果回来后自己决定下一步，负责最终输出。（ClaudeCode里面的SubAgent的做法）
- **消息协作**：多个长期驻留 Agent 持续交换消息并协同推进任务，允许横向通信与并行协作。（ClaudeCode里的AgentTeam的做法）

移交更适用于自主协作的场景，而工具调用则提供了更明确的层级控制和接口约束。

2. 消息传递内容：Agent与Agent之间应该传递所有的消息还是部分消息，需要根据具体的业务场景权衡。

- **共享完整推理数据：**智能体将其所有中间步骤（如链式思考、工具调用）写入共享通道。相当于提供了一个共享区，用于其他智能体理解其推理过程，可以提升系统的整体协作与推理能力。但是这种方式的缺点是会导致状态空间快速膨胀，给上下文窗口和内存管理带来挑战。
- **仅共享最终结果：**智能体在私有空间内完成其计算，仅将最终结果写入到共享区。它能有效控制状态的复杂度，实现此策略需要为每个智能体定义独立的状态模式。

1. **通信介质**（怎么传）

- 内存通道（in-memory return）。子代理输出作为 tool_result 同步返回主上下文。（ClaudeCode里面的SubAgent的做法）
- 文件通道（file-backed mailbox）。成员把消息写入对方 inbox 文件，接收方在下一轮读取并注入上下文。（ClaudeCode里的AgentTeam的做法）

这一小节，对应的实战部分，可以看ClaudeCode的Agent通信章节。

8.5 Agent架构设计实战

（这个小节，我从**参考论文的例子**以及**面试真题**中抽取出来几个典型的例子和问题，帮助大家来掌握Agent架构设计相关的内容。现在很多面试都会考察一个小型的场景下的Agent架构设计题目，我会持续将架构设计相

关内容汇总于此，这样大家复习更加方便。

在我整理了几个例子后，某种程度上，我觉得这个问题其实可以套模板，大家看一下是不是，如果你认为你的任务应该用去中心化架构，你就参考这个6.5.1,对于架构的好处，通信方式都可以复用，唯一不同的是Agent的设计和功能。）

8.5.1 动态网络浏览问题

（此例子截取自论文）

问题描述：我们需要通过网络搜索，找到 **某公司2024年可持续发展报告中关于碳排放的问题**，请你针对这个问题设计一个Agent架构。

架构设计：

因为网页导航是**高熵，路径不唯一**的场景。 不同的来源（官网，媒体，报告）可以被并行搜索更快覆盖。在这种场景下，我会使用去中心化架构设计整个系统，使用**去中心化架构既能够提升并行速度**，可以**减少中心调度的串行瓶颈。**

具体来讲，整个系统各个Agent是平级的，他们自主并行探索。系统里面使用**事件总线/共享黑板**的方式，所有Agent订阅相关话题，他们可以**发布任务，认领任务，提供帮助，提交答案，**来实现点对点的协作。

具体来讲，系统里有：

任务规划Agent，将用户目标拆分成子任务后，发布任务。

然后官网探索Agent， 媒体报告等Agent认领任务后，各自探索，调用工具汇总任务，直到达到标准然后会提交任务，交给验收Agent验收。

验收Agent验证通过后，交付给汇总机器人汇总信息，如果汇总Agent认为目标达成，及时发布停止信息，终止流程进行交付。

8.5.2 金融推理任务

（此例子截取自论文）

任务描述：设计一个分析某个上市公司的估值的Agent。

架构设计：金融估值的**流程通常比较规范**，天然具有**分而治之**的特点，多个子**问题之间相对独立，可以并行。**这种情况下我会使用**中心化**架构设计整个系统：由一个Orchestrator（中心指挥官）统一拆解，分派和收敛，降低并发协作带来的协调成本和不一致风险，**确保各步骤按既定顺序和策略并发执行。**

具体来讲：

我会先设计一个主管Agent, 他将任务分解成**多个阶段，设计并行窗口，验收，**将任务下发给子Agent去并行完成。

主管Agent下设置多个子Agent，比如**收集官网信息Agent, 收集市场监管信息Agent，收集友商信息做对比的Agent。**

整个过程由主管Agent负责验收，当Orchestrator验收通过，下发停止信号中止整个流程并交付。

8.5.3 中心Agent架构子Agent膨胀问题

在小红书一面中面试官对中心架构下子Agent膨胀问题进行了探讨，这里只做问题汇总，具体内容请转到面试实战，小红书一面

8.5.4 时空冲突检测任务

（选自真题：夸克千问架构设计题目，我就准备直接套用6.5.2的模板）

任务描述：设计一个Agent系统，让它完成当用户创建一个行程后，判断这个行程是否和其它冲突，如果有冲突，给出重新规划的建议。

架构设计：行程冲突检测 和 规划本质是约束满足 + 顺序调整，**流程比较规范，**需要考虑**时间窗口，地点，偏好等特点，**这些因素也相对独立，可以并行。 这种情况下我会使用**中心化** 架构设计整个系统，由一个Orchestrator统一拆解，分派和收敛，降低并发协作带来的协调成本和不一致风险，**确保各步骤按既定顺序执行也能够并发。**

具体来讲：我会先设计一个Agent作为总指挥，根据当前阶段来分配下发任务，控制并发窗口，验收，和控制中止。

然后我会设计多个子Agent。 比如行程Agent：负责整理相关的行程，和行程时间相关的数据。 地点Agent：评估各个行程地点相关Agent的 通行时间等，冲突检测Agent, 用户喜好分析Agent 和 生成重新规划的Agent。 他们之间在某些阶段可以并行，某些阶段是串行，由主管Agent统筹规划和管理。

8.6 面试问题

8.6.1 多 Agent 协作系统需要考虑哪些架构问题?

选自 202607宇树科技二面

💡 **思路：**

顺着 **“是否需要多 Agent → 如何组织和编排 → 如何通信协作 → 如何验证交付”** 这条主线思考，再从 **安全与权限、成本与延迟及可观测性** 两个方面检查整个设计。前四点回答系统怎样完成任务，后两点回答系统怎样安全、可控地运行，它们是贯穿全程的约束，不是最后才考虑。其中这六个方面，每一方面其实都可以展开，建议顺着这个思路，深入学习每一部分内容，都是重点，面试官追问的时候也可以回答的上来。

1. **是否使用多 Agent；使用什么架构，如何编排；Agent 之间如何通信、共享状态。** 这些都是：笔记Multi-Agent架构章节， 这个章节讲的，属于高频考点。
2. **安全与权限如何控制。** 在ClaudeCode安全机制里面讲过，讲了它的权限系统和安全审查。
3. **成本、延迟与可观测性如何设计.** 属于Agent评估的内容。

📝 **答案：**

我会从以下六个方面考虑：

1. **是否使用多 Agent。**

先判断单 Agent 或固定工作流是否够用。确实需要专业分工、并行处理或独立验证时，再引入多 Agent，并确认收益值得额外的通信和协调成本。

1. **使用什么架构，如何编排。**

明确各 Agent 的职责，以及谁分配任务、谁汇总结果。流程固定时用流水线，需要动态拆解时用协调者—工作者结构；按任务依赖决定串行或并行，并设置超时、有限重试和停止条件。

1. **Agent 之间如何通信、共享状态。**

约定输入输出格式，只传递必要的结果和上下文。区分私有信息与共享任务状态，明确共享数据由谁维护，避免重复劳动和相互覆盖。

1. **如何验证结果、保证任务完成。**

明确子任务和总任务的验收标准，通过规则、测试、证据或独立验证 Agent 检查结果。结论冲突时按证据裁决，验证失败时有限修正或转人工，不能把“执行结束”当成“任务完成”。

1. **安全与权限如何控制。**

按最小权限限制各 Agent 能访问的数据和工具，由执行层强制校验，不能只靠提示词。防范提示注入和数据泄露，删除、付款等高影响操作需要审批。

1. **成本、延迟与可观测性如何设计。**

设置全局时间、Token 和费用预算，通过合理并行、模型选型和减少无效通信优化开销。记录各 Agent 的调用链、任务状态和失败原因，并与单 Agent 比较成功率、延迟和成本，验证实际收益。

总结：先判断要不要用，再设计怎么组织、怎么协作、怎么验收，同时保证权限有边界、资源有预算、过程可追踪。

---

9 A2A

（根据最新面试情况，A2A协议的概念，A2A 考的没有MCP那么多，没有那么热，但是还是偶尔会问到。暂时问的不深，我们先记一下概念）

A2A（Agent-to-Agent）协议的核心目标是**让不同智能体之间能够互操作和协作**，即使它们来自不同厂商、框架或运行环境。下面是一个简洁介绍：

A2A 协议的关键特点

1. **互操作性**

- 通过标准化的消息格式和通信机制，让不同 Agent 可以发现彼此并交换任务或信息。
- 常用技术：JSON-RPC、HTTP、SSE 等。 2. **Agent 卡片（描述元数据）**
- 每个 Agent通过“卡片”描述自己的能力、接口、身份和安全策略，方便其他 Agent发现和调用。 3. **长时协作与异步交互**
- 支持多轮对话、任务委派、进度更新、流式响应，适合复杂场景（如旅行规划、企业流程自动化）。 4. **内部实现不透明**
- 调用方只知道对方是一个 Agent，不关心它内部用什么模型或工具，保证封装性。

适用场景

- **跨团队或跨厂商的 Agent 协作**：例如客服 Agent 与账单 Agent协作处理用户请求。
- **复杂任务编排**：多个专长 Agent分工完成一个大任务（如行程规划、供应链优化）。
- **生态互通**：构建开放的 Agent 网络，而不是单一厂商闭环。

---

10 Agent开发框架

该小节重构于2026年8月。

根据我2026 年的面试经验，Agent 框架已经不算一个高频考点，同时对框架的考察一直不会很深入。

在 2025 年，可能大家还认为 LangChain 非常重要，因此会像准备八股文一样，集中背诵 LangChain、LangGraph 的组件、概念和基本特性。但到了 2026 年，我觉得这件事没有太多意义了。Langchain并不是你要从事大模型开发岗位的必学内容了。

我认为原因其实也很容易理解：**Agent 框架只是开发 Agent 应用的工具，工具本身不是重点，工具背后封装的 Agent 能力才是重点。况且各个公司开发用的框架也不一样，深入学习一个框架，对于面试来说性价比并不高。**

不同框架的 API 并不通用，但它们处理的核心问题高度相似，例如：

- 如何把函数、API、数据库或 MCP 服务封装成工具；
- 如何实现“模型决策—调用工具—读取结果—继续决策”的 Agent Loop（智能体循环）；
- 如何维护会话状态、短期记忆和长期记忆；
- 如何管理上下文、压缩历史和控制 Token；
- 如何进行分支、循环、并行、暂停与恢复；
- 如何实现 Human-in-the-loop（人工介入或审批）；
- 如何管理工具权限、凭据和执行沙箱；
- 如何记录 Trace（执行链路）、评测效果和失败原因；

- 如何保证重试、幂等、审计和生产可靠性。

这些才是不同 Agent 框架背后共用的“内核”。掌握内核以后，切换框架主要是学习新的抽象和 API；如果只会背某个框架的 API，一旦项目更换技术栈，原有知识就很难迁移。

因此，**LangChain 在我看来已经不是所有人都必须深入掌握的内容。你完全可以使用其他 Agent 框架，也可以不使用框架，直接基于模型 API 手写 Agent。**框架的价值是减少重复开发，而不是赋予模型某种只有该框架才具备的智能。

回到面试角度，我来给大家画一下重点。

必学的内容：

1. **知道市面上有什么框架，它的特点，优缺点，局限性。如何科学选择Agent的开发框架。**这个无论是对于你自己做项目框架选型还是回答面试问题，你为什么使用这个框架，都有帮助。

2.**如果你使用了某个框架，那么请深入了解一下这个框架的特点，设计思想，理念，特性。** 对于你没用到的，简历里没写的框架，**那么不做学习要求。**

本小节笔记编写也遵循这个逻辑：

1.1 **小节属于必学**:带你了解2026年有哪些Agent框架，特点是什么，如何选型，同时讲清楚上面两个Agent框架最高频的面试问题：这个是学完上面内容的面试实战运用，也是面试官问Agent框架，几乎一定会先问的问题。

1.2 **小节是历史遗留章节，讲解了Langchain/Langgraph的一些特点。**我知道这里内容有一点过时，因为Langchain有更新的版本了。 我没有删除的原因是 ： 历史的面经，特别是25年的面经有问到引用了这个章节，但更重要的是，根据我们的思想，如果你用了某个框架，你应该自己主动去深入理解一下，比如你项目用了Langchain，你就应该稍微比笔记多了解一下。所以留在这里一个LangChain的一点内容，是一个提醒。你应该根据自己用的框架，稍微补充一点相关知识。

10.1 主流框架和选型策略

10.1.1 框架速览

这一小节快速带大家了解，2026年8月，市场上流行的主流框架有哪些，建立初印象。

我们用 **GitHub 官方代表仓库的 Stars 数量**衡量开源社区关注度，数据通过 GitHub API 查询，统计时间为 **2026 年 8 月 11 日。**相比笼统地写“高”或“较高”，Stars 至少提供了一个公开、可复查的量化依据。

不过，Stars 只能作为热度参考，不能直接等同于生产成熟度或企业采用率，原因包括：

- 项目发布时间不同，较早发布的项目天然更容易积累 Stars；
- Dify、LangChain 等面向更广泛的 AI 应用用户，受众规模与纯 Agent 运行时不同；
- Claude Agent SDK、Google ADK 等项目采用多语言分仓，单个 Python 代表仓库可能低估整体生态；
- GitHub Stars 无法反映闭源企业使用、云平台内部使用、API 稳定性、安全性和维护质量；

- Stars 是累计值，不能单独反映近期增长速度和当前活跃度。

因此，下表把 Stars 称为**开源社区关注度**，而不是“框架质量分”或“市场占有率”。对于多仓库项目，统一选择最具代表性的官方主仓库，不对多个语言仓库的 Stars 求和。

按本次统计的代表仓库 Stars 排序，前三名是 **Dify（152.1k）**、**LangChain（143.9k）** 和 **CrewAI（56.9k）**。这说明它们在开源社区中的关注度较高，但由于三者分别属于低代码平台、高层应用框架和多 Agent 框架，不能据此判断谁在技术上更适合某个具体项目。

10.1.2 各框架使用方式

先讲解各个Agent框架使用方式，这是最直观理解不同框架的特点的角度了。

10.1.2.1 方法一：组装组件——定义 Agent、工具和输出

使用这类框架时，你通常先选择模型，再创建一个 Agent，为它写系统指令，并把搜索、数据库查询、发送邮件等函数注册成工具。如果结果需要固定格式，还会定义一个输出数据结构。运行后，模型可以返回工具调用请求；框架执行对应函数，再把结果交回模型，直到模型给出最终答案或达到开发者设置的停止条件。

换句话说，你开发的主要形式是：**配置 Agent + 挂载 Tools + 调用 run()**。这里的“模型决定下一步”只限于一次 Agent 运行循环：模型决定此刻直接回答，还是请求调用某个已注册工具。工具的具体实现、允许调用哪些工具、调用次数、权限以及外层业务流程仍然由代码控制。代表框架：**LangChain、PydanticAI、OpenAI Agents SDK**。

10.1.2.2 方法二：搭建状态图——定义 State、Node 和 Edge

使用 LangGraph 时，你不是只创建一个 Agent 然后让它自由运行，而是先定义一份 **State**，用来保存消息、任务结果和执行进度；再把每一步操作写成 **Node**，例如“模型判断”“调用搜索工具”“人工审核”；最后用 **Edge** 把节点连接起来，并定义什么条件下走哪条边。

程序运行时，State 在节点之间不断传递和更新。开发者真正搭建的是一张可循环、可分支的执行图。它的开发形式可以概括为：**定义状态 → 编写节点 → 连接边 → 编译并运行图**。代表框架：**LangGraph**。

10.1.2.3 方法三：组织 Agent 团队——定义 Role、Task 和协作关系

使用 CrewAI 一类框架时，你首先要做的不是画流程，而是创建多个 Agent。每个 Agent 都要定义自己的**角色、目标、背景说明和可用工具**，例如研究员负责搜集资料，作者负责写作，审核员负责检查。然后再定义每个角色要完成的 Task，以及任务按顺序执行、并行执行还是由管理者动态分派。

这种方法写出来的程序很像在组建项目团队：**定义 Agents → 分配 Tasks → 设置协作方式 → 启动 Crew**。任务在不同角色之间流转，每个 Agent 只处理自己负责的部分。代表框架：**CrewAI、AgentScope**。

10.1.2.4 方法四：配置工作环境——给 Agent 任务和可操作的电脑

使用长任务 Agent Harness 时，开发者通常不需要把几十个步骤全部预先写出来。你要做的是给 Agent 一项总体任务，配置它可以使用的**环境和权限**，例如**文件系统**、**终端**、浏览器、代码仓库，以及必要的工具和子 Agent。Agent 收到任务后，会自己制定计划、读写文件、执行命令、检查结果，并根据中间产物继续工作。

这种开发方式更像给一位数字员工准备工作台：**描述目标 → 配置环境与工具 → 设置权限 → 让 Agent 持续执行。**代表框架：**Deep Agents、Claude Agent SDK。**

10.1.2.5 方法五：搭建知识管道——定义数据源、索引和查询流程

使用 LlamaIndex 时，开发往往分成两部分。第一部分是准备知识：接入 PDF、网页或数据库，把内容切分成节点并建立索引；第二部分是使用知识：定义检索器或查询引擎，让 Agent 在回答问题或调用工具前先找到相关资料。

因此，你主要开发的不是角色或流程图，而是一条数据进入 Agent 的管道：**加载数据 → 切分与索引 → 检索相关内容 → 交给 Agent 回答或行动。**代表框架：**LlamaIndex。**

10.1.2.6 方法六：在平台中编排并发布——配置工作流和应用

使用平台型框架时，你面对的通常不是单一的代码库，而是一套把开发、调试、发布和观测放在一起的完整平台：先选择模型，填写 Agent 指令，添加工具或知识库，再用代码或可视化画布连接处理步骤。完成后，可以直接在平台内测试、评估、部署为 API 或聊天应用，并查看运行日志。

所以，这一类框架的核心区别不只是“前面写代码，这里拖拽界面”，而是**框架是否把 Agent 的全生命周期一起打包了**：既管怎么搭，也管怎么测、怎么发、怎么监控。它关注的是一个完整应用，而不只是 Agent Loop：**配置 Agent → 编排 Workflow → 调试评估 → 部署应用 → 查看日志。**代表框架：**Dify。**

10.1.3 框架核心特点

上面从框架使用方式角度介绍，这里从框架最核心的特点介绍，每个框架用一句话，说明它的最核心的特点。

- **LangChain：生态最丰富的通用工具箱。** 接模型、工具和检索非常方便，适合快速做原型；但抽象较多，复杂流程容易变得不透明。
- **LangGraph：控制力最强的状态图框架。** 状态持久化、检查点、恢复和“时间回溯”能力突出；代价是建模最复杂，开发者要自己设计 State、Node 和 Edge。
- **Deep Agents：为长任务准备好的 Agent 骨架。** 自带规划、工作区、上下文压缩和子 Agent，适合研究与多文件任务；短任务使用它会显得偏重。
- **OpenAI Agents SDK：轻量且容易上手的官方 SDK。** Agent、工具和 Handoff 概念简洁，与 OpenAI 的语音、追踪和评测结合自然；深度使用后会增加 OpenAI 生态绑定。但是缺点是编排能力比较弱，不支持并行和循环。

- **Claude Agent SDK：最擅长操作电脑环境。** 文件读写、Shell、搜索和代码修改能力开箱即用，特别适合编码 Agent；同时必须认真处理权限、沙箱和执行安全。
- **Google ADK：最像传统软件工程的 Agent 框架。** 它允许开发者用顺序、并行和循环等代码结构组织多个 Agent，把模型的自主判断嵌入确定性流程，并配套会话、记忆、产物和评估能力；优点是结构完整、适合复杂工程，缺点是概念较多，简单任务使用会显得偏重。
- **Microsoft Agent Framework：最适合微软技术栈的企业框架。** 对 Azure、Microsoft 365、身份治理和 .NET 团队友好；非微软技术栈使用它的优势会明显减弱。
- **CrewAI：最直观的角色型多 Agent 框架。** 用研究员、作者、审核员等角色组织任务很容易理解；多个 Agent 也会带来更高的 Token、延迟和调试成本。
- **PydanticAI：最强调类型安全的轻量 Python 框架。** 工具参数、依赖和输出结构清晰，适合 API 与结构化数据任务；它不是复杂工作流或长任务运行时。最大的特点是类型安全，使用FASTAPI编程方式。
- **LlamaIndex：数据与 RAG 能力最突出。** 文档接入、切分、索引和检索链路完整，适合知识库与数据密集型 Agent；但接上向量库不等于检索质量自然可靠。
- **Dify：最容易快速做出可用应用。** 通过拖拽即可组合模型、知识库、工具和工作流，并直接发布；画布复杂后，测试、复用和深度定制会比较困难。
- **AgentScope：更贴近国内企业环境的多 Agent 框架。** 对国产模型、Python/Java、私有化和国内云生态更友好；国际社区与第三方生态相对较弱。

记住这些核心特点后，再阅读每个框架的详细介绍，就可以把更多功能理解为对这条主线的补充，而不必一次记住所有概念。

10.1.4 各框架特点详细介绍

我通过1.1.1\~1.1.3 是想让大家能够对这些框架建立一个印象。你不要背，也能够轻松知道常见的一些框架，对它有一个印象。某种程度也是为这个小节做铺垫，这个小节很枯燥，但是是面试问题：讲讲你了解的Agent框架。**该背还是得背一下，有了前面的小节，应该背起来容易一点。**但你不用都背，这个小节就是防止面试官问：你知道什么Agent框架，讲一下？所以你是要挑你项目使用的，或者关注的几个背就行了。

这个小节每一个框架下面我都穿插了面试问题，供大家检验。

10.1.4.1 LangChain：高层 LLM 应用工具箱

定位：LangChain 主要负责连接模型、消息、工具、结构化输出、检索和高层 Agent。它适合快速组装应用，但不应被理解为所有复杂 Agent 系统的统一底座。

核心特性：多模型与多工具集成、标准化消息和 Prompt、结构化输出、高层 Agent 接口、检索与文档组件，并可与 LangGraph 组合。

优点：生态广、资料多、接入快，适合验证模型、工具和数据链路。

局限：抽象较多；只使用高层 Agent 时，复杂控制流不如显式状态图清晰；集成数量多也不代表每个组件都具有相同的生产质量。

适用场景：通用 Agent、快速原型、集成驱动型应用。严格持久化、恢复、审批和复杂状态控制通常应进一步使用 LangGraph 或其他工作流运行时。

写入简历后要掌握：实际使用了哪些模块；为什么没有直接调用模型 SDK；复杂流程是否引入 LangGraph；模型、工具和检索抽象为项目减少了哪些工作。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 LangChain 是一个通用的 LLM 应用开发框架。它把模型、Prompt、消息、工具、结构化输出和检索封装成可组合组件，开发者可以为 Agent 注册搜索、数据库等工具，再由模型在运行过程中选择是否调用。它更像一个连接各种 LLM 能力的标准工具箱，重点是减少不同模型、工具和数据源之间的适配工作，而不是强制开发者采用某一种固定流程。
>
> **优缺点。** 它最大的优势是生态丰富、资料多，接入不同模型和第三方服务都比较快，切换模型或替换工具时也能复用一部分上层代码。缺点是抽象层较多，组件版本和行为边界需要认真理解；只使用高层 Agent 时，复杂分支、状态变化和失败恢复不够直观，这类需求通常会再结合 LangGraph。
>
> **适用场景。** 我会把它用于通用工具 Agent、RAG 应用、模型与工具集成，以及需要快速验证想法的早期原型。如果业务流程非常固定，我可能直接写普通代码；如果流程复杂且必须恢复，我会让 LangChain 负责模型和工具接入，让 LangGraph 负责运行控制。

10.1.4.2 LangGraph：显式状态与流程控制

定位：LangGraph 是面向有状态、可恢复、长时间运行 Agent 工作流的编排运行时。核心价值是控制力和可靠执行，而不是让模型本身更聪明。

核心特性：Node、Edge、State；条件分支、循环、并行；Checkpoint（检查点）和状态持久化；Interrupt、恢复执行、Human-in-the-loop；流式事件和多 Agent 编排。

优点：控制流显式，复杂流程更容易理解、审计和恢复，也更适合将不确定的模型决策嵌入确定性业务流程。

局限：学习和建模成本较高；开发者仍需自己设计 State Schema、重试、幂等和副作用隔离；对简单问答可能过重。

适用场景：审批、供应链、金融流程、多阶段研究、长任务和必须恢复的业务。

写入简历后要掌握：State 保存什么；节点和条件边如何设计；Checkpoint 存在哪里；服务重启后如何恢复；怎样防止重试导致重复写库或重复扣款；为什么不用普通代码或高层 Agent。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 LangGraph 是一个用状态图编排 Agent 的运行时。开发者先定义保存消息和业务数据的 State，再把模型调用、工具执行和人工审核写成 Node，最后通过 Edge 和条件分支规定节点之间怎样流

转。每个节点读取并更新同一份状态，因此循环、重试、人工审批和多 Agent 交接都可以明确地画进执行图中。

优缺点。 它最大的优势是控制力和恢复能力强，支持 Checkpoint、Interrupt、持久化和时间回溯，任务中断后可以从历史状态继续，也能查看某一步之前的状态。缺点是建模成本较高，框架只提供运行机制，不会替开发者自动解决 State Schema、重试、幂等和副作用隔离等业务设计问题。

适用场景。 它适合审批、金融流程、多阶段研究和长时间运行任务，尤其适合执行失败后不能简单从头重跑的业务。比如贷款审核进行到人工复核时可以暂停，收到审核结果后再从原状态继续，而不需要重新执行前面的查询和判断。

10.1.4.3 Deep Agents：面向长任务的 Agent Harness

定位：Deep Agents 是面向长时间、开放式任务的高层 Agent Harness。它与 LangGraph 不是简单竞争关系：前者偏开箱即用的长任务骨架，后者偏可定制的底层状态运行时。

核心特性：任务规划、待办管理、文件系统和工作区、上下文压缩、中间结果卸载、子 Agent 委派、工具和 MCP 接入，以及底层运行时提供的持久化和人工审批能力。

优点：不必从零搭建长任务所需的结构，适合多文件、大量中间结果和开放式目标。

局限：对短任务较重；默认行为不一定适合所有业务；涉及 Shell、代码和文件操作时仍需安全沙箱和权限策略。

适用场景：深度研究、编码、长报告、多文件分析。若流程非常明确并要求逐节点控制，直接使用 LangGraph 自定义可能更合适。

写入简历后要掌握：为什么任务属于长任务；如何管理工作区、上下文和中间产物；子 Agent 的职责边界；如何做权限与沙箱；为什么没有直接用 LangGraph 自建。

面试小卡片｜请简单介绍一下你了解的 Agent 框架

核心特点。 我了解的 Deep Agents 是一个面向开放式长任务的 Agent Harness。它预置任务规划、待办管理、文件工作区、上下文压缩、中间结果卸载和子 Agent 委派，让 Agent 可以持续推进一个不能一次完成的目标。它和 LangGraph 不是简单替代关系：LangGraph 更像底层可恢复运行时，Deep Agents 则是在上面准备好了一套长任务工作方式。

优缺点。 它的优势是不用从零搭建长任务骨架，Agent 可以把大量中间内容写入文件，并通过子 Agent 分担不同部分，避免所有信息都挤在一次上下文中。缺点是对简单短任务偏重，默认规划方式不一定适合所有业务，任务跑得越久，成本、错误累积和文件工具的安全风险也越需要管理。

适用场景。 它适合深度研究、长报告、多文件分析，以及需要拆分任务、保存大量中间结果并持续执行的复杂工作。例如让 Agent 阅读几十份资料、分别生成研究笔记，最后再汇总成一份完整报告，就比普通的一次性 Agent Loop 更适合使用 Deep Agents。

10.1.4.4 OpenAI Agents SDK：轻量的 OpenAI Agent SDK

定位：OpenAI Agents SDK 与 OpenAI 的模型、Responses、Realtime、Tracing 和 Evals 生态紧密结合，也是 Swarm 之后更适合新项目的官方方向。

核心特性：Agent、工具调用、Handoff、Agent-as-tool、输入输出 Guardrails、Session、Tracing、Evals、实时语音和 MCP 接入。

优点：概念少、上手快；Handoff 适合客服路由和专家接管；与 OpenAI 实时语音、Trace 和评测链路结合自然。

局限：使用厂商特有能力越多，迁移成本越高；核心抽象不是显式状态图；Guardrails 也不能替代完整业务权限和安全治理。

适用场景：已确定使用 OpenAI，需要快速构建工具调用、客服分流或语音 Agent 的项目。

写入简历后要掌握：Handoff 与 Agent-as-tool 的区别；Guardrails 能防什么；Session 与 Trace 怎么用；为什么接受 OpenAI 生态绑定；复杂流程如何补持久化和恢复。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 OpenAI Agents SDK 是 OpenAI 官方提供的轻量 Agent SDK，核心能力包括工具调用、Handoff、Agent-as-tool、Guardrails、Session 和 Tracing。Handoff 可以把当前对话和控制权交给另一个专业 Agent，而 Agent-as-tool 则是由主 Agent 调用其他 Agent 完成子任务后继续统一回答，这两种方式适合不同的协作关系。
>
> **优缺点。** 它的优势是概念少、上手快，并且与 OpenAI 的 Realtime、Tracing 和 Evals 结合自然，开发者可以比较方便地追踪 Agent 和工具的调用链路。缺点是核心抽象不是显式状态图，Guardrails 也不能代替业务权限；复杂流程、持久化和恢复需要额外设计，深度使用后还会增加 OpenAI 生态绑定。
>
> **适用场景。** 它适合工具调用 Agent、客服分流、专家接管和实时语音应用，特别适合已经确定使用 OpenAI 模型的项目。例如客服 Agent 可以根据问题把会话 Handoff 给退款、技术支持或销售 Agent，并用 Trace 查看整个交接过程。

10.1.4.5 Claude Agent SDK：Claude Code 风格的完整 Harness

定位：Claude Agent SDK 将 Claude Code 风格的 Agent 能力嵌入自己的产品和服务。它不是普通模型 API SDK，而是带 Agent Loop、内置工具、上下文管理、权限和会话能力的 Harness。

核心特性：文件读写和编辑、Shell、Glob、Grep、Web Search 等内置工具；MCP 与自定义工具；会话、上下文、流式事件；权限、Hooks、人工审批和子 Agent。

优点：构建编码和文件系统 Agent 时开箱即用，可以复用成熟的工具循环、权限和上下文管理方式。

局限：主要围绕 Claude 生态；SDK 提供 Harness，但通常仍由使用方负责部署和执行环境；文件、Shell 和代码执行必须配套沙箱与最小权限。

适用场景：代码助手、代码库分析、自动修改、CLI 和研发自动化。简单客服、表单抽取或普通 RAG 通常不需要完整 Harness。

写入简历后要掌握：Claude Agent SDK 与普通 Anthropic SDK / Tool Runner 的区别；使用了哪些内置工具；怎样限制路径、命令和网络；会话如何恢复；为什么选择它而不是 Deep Agents 或手写循环。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 Claude Agent SDK 是把 Claude Code 风格的 Agent Harness 嵌入自己应用的开发工具。它不只是发送 Prompt 的普通模型 SDK，还内置文件读写、代码编辑、Shell、Glob、Grep、Web Search、会话、Hooks、权限控制和子 Agent，Agent 可以在真实工作目录中连续搜索、修改并验证结果。
>
> **优缺点。** 它的优势是工具循环、会话和上下文管理比较完整，做代码 Agent 时不需要从零实现文件、终端、权限和结果回传。缺点是主要围绕 Claude 生态，而且文件修改和 Shell 都是高权限操作；SDK 提供权限机制并不等于应用已经安全，仍要增加审批、命令允许列表、最小权限和隔离沙箱。
>
> **适用场景。** 它适合代码助手、代码库分析、自动修改与测试、CLI 工具以及其他研发流程自动化任务。例如可以让它定位一个跨文件 Bug、修改相关代码、执行测试，再根据失败结果继续修复；普通客服或简单结构化抽取通常不需要这么完整的 Harness。

10.1.4.6 Google ADK：完整的 Agent 工程体系

定位：Google ADK（Agent Development Kit）面向 Agent 的开发、运行、评估和部署。它不仅能定义 LLM Agent，也能通过顺序、并行和循环等方式组织确定性 Workflow（工作流）。

核心特性：Agent、Tool、Runner；Sequential、Parallel、Loop Workflow；Session、Memory、Artifact；多 Agent、评估、观测、部署，以及 Gemini、Vertex AI 和 Google Cloud 集成。

优点：开发到部署链路较完整，兼顾模型自主决策和确定性流程，对 Google Cloud、Gemini 和多语言企业团队友好。

局限：深度使用 Google 特有服务会增加迁移成本；简单本地脚本可能不需要整套体系；不同语言 SDK 的能力和稳定等级需按版本核验。

适用场景：Gemini、Vertex AI、Cloud Run 或 GKE 已经是标准栈的企业，以及需要多语言 Agent 体系的团队。

写入简历后要掌握：Agent、Runner、Session、Memory、Artifact 的职责；确定性 Workflow 与 LLM Agent 如何组合；如何评估和部署；使用了哪些 Google Cloud 能力；怎样控制锁定风险。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 Google ADK 更像是用传统软件工程的方式组织一套 Agent 程序。它既支持让 LLM Agent 自主判断，也提供 Sequential、Parallel 和 Loop 等确定性结构来组合多个 Agent，同时通过 Runner、Session、Memory 和 Artifact 管理运行过程。也就是说，开发者既可以让模型决定如何使用工具，也可以明确规定哪些步骤必须按顺序、并行或循环执行。

> **优缺点。** 它的优势是体系比较完整，可以把模型自主决策嵌入固定业务流程，并配套会话、记忆、产物、评估和部署能力，适合用工程化方式管理多个 Agent。缺点是需要理解的概念较多，简单工具 Agent 使用它会偏重；虽然框架并非只能使用 Gemini，但深度使用 Vertex AI 和 Google Cloud 服务后仍会形成一定平台绑定。
>
> **适用场景。** 它适合采用 Gemini 或 Google Cloud，需要组合多个 Agent、确定性工作流和企业工程能力的复杂项目。例如可以让资料搜集 Agent 并行工作，再由写作 Agent 顺序生成报告，最后由审核 Agent 循环检查和修改。

10.1.4.7 Microsoft Agent Framework：微软企业 Agent 方向

定位：Microsoft Agent Framework 面向 Python 和 .NET 的 Agent、多 Agent 协调与工作流。更准确的理解是它为 AutoGen、Semantic Kernel 相关 Agent 使用者提供新的收敛和迁移方向，而不是简单把两个项目改名。

核心特性：Python/.NET 双栈、Agent 与多 Agent 协调、企业工作流和部署，并可结合 Azure、Microsoft 365、身份与治理体系。

优点：对 .NET/C# 企业团队友好，容易接入微软身份、云和企业工具链。

局限：框架演进和迁移期要关注版本稳定性；对非 Python/.NET 团队吸引力较弱；深度绑定微软特有能力会增加迁移成本。

适用场景：Azure、Microsoft 365、.NET 和企业身份治理占主导的组织。

写入简历后要掌握：项目使用的准确版本；它与 AutoGen、Semantic Kernel 的关系；用了哪些 Azure 或 Microsoft 能力；工作流和多 Agent 如何组织；为什么没有选择 LangGraph 或 Google ADK。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 Microsoft Agent Framework 是微软面向 Python 和 .NET 提供的 Agent、多 Agent 协调与工作流框架，也可以把它理解为 AutoGen、Semantic Kernel 相关 Agent 能力的收敛方向。它不仅关注模型和工具调用，也强调 Agent 如何接入企业身份、业务系统和微软现有的开发部署体系。
>
> **优缺点。** 它的优势不是某一种 Agent 算法，而是可以自然接入 Azure、Microsoft 365、企业身份和治理体系，对 .NET 团队尤其友好，也更容易沿用企业原有的权限和运维方式。缺点是框架仍在演进，需要确认项目使用的准确版本及稳定性；如果企业不采用微软技术栈，这些优势就不明显，深度绑定后的迁移成本也较高。
>
> **适用场景。** 它适合以 Azure、Microsoft 365、.NET 和企业身份治理为标准技术栈的大型组织。例如构建能够访问企业账号、Teams、SharePoint 或内部业务系统的 Agent 时，它比单纯的通用 Python Agent SDK 更符合现有架构。

10.1.4.8 CrewAI：角色驱动的多 Agent 框架

定位：CrewAI 使用 Role、Goal、Task、Crew 等概念把业务团队映射为 Agent 协作，并通过 Flow 增加状态化和事件驱动控制。

核心特性：角色、目标、任务、Crew、Agent 委派、Flow、状态、工具和知识接入。

优点：业务表达直观，容易把“研究员—编辑—审核员”建模为虚拟团队，多 Agent 原型开发较快。

局限：多 Agent 会增加 Token、延迟、调试难度和错误传播；现实岗位不一定应该映射成独立 Agent；严格事务和复杂恢复仍需显式工作流设计。

适用场景：市场调研、内容生产、销售运营和虚拟专家团队。若角色不需要独立上下文、权限或目标，单 Agent 加工具通常更简单。

写入简历后要掌握：为什么必须用多个 Agent；每个 Agent 的独立职责、上下文和权限；Crew 与 Flow 的区别；怎样减少无效对话；多 Agent 是否真的提高成功率。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 CrewAI 是一个角色驱动的多 Agent 框架。开发时会为不同 Agent 定义 Role、Goal、Backstory、Tools 和 Task，例如研究员搜集资料、作者生成内容、审核员检查结果，再通过 Crew 组织角色协作，或者使用 Flow 增加状态和事件驱动的流程控制。
>
> **优缺点。** 它的优势是业务表达直观，现实团队的职责很容易映射成 Agent，任务委派和多角色原型搭建也比较快。缺点是 Agent 越多，Token、延迟、调试难度和错误传播都会增加，而且角色之间反复对话未必真的提高结果质量，所以不能为了形式而拆分角色。
>
> **适用场景。** 它适合市场调研、内容生产、销售运营和虚拟专家团队，前提是不同角色确实拥有独立目标、上下文或权限。如果只是同一个 Agent 依次调用几个工具，通常没有必要为了“多 Agent”再拆成多个角色。

10.1.4.9 PydanticAI：类型安全的 Python Agent

定位：PydanticAI 是 Python-first、类型安全优先的 Agent 框架，强调让 Agent 应用像普通 Python 服务一样具备清晰的依赖、工具签名、结构化输出和测试边界。

核心特性：Pydantic 结构化结果校验、类型化依赖注入、根据函数签名生成工具 Schema、多模型提供商抽象、流式输出和测试支持。

优点：对 Python 后端友好；工具参数和模型输出具有清晰类型边界；便于结构化数据处理、测试和 Schema 演进；比大型编排框架轻量。

局限：类型安全只能保证数据形状，不能保证模型决策正确；自动重试可能增加 Token 和延迟；它不是完整的分布式长任务运行时。

适用场景：结构化抽取、API 型 Agent、数据库操作和 Python/FastAPI 服务。

写入简历后要掌握：输入、依赖、工具和输出怎样建模；校验失败如何处理；重试成本如何控制；切换模型后行为是否一致；类型安全不能解决哪些可靠性问题。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 PydanticAI 是一个 Python-first、强调类型安全的轻量 Agent 框架。它使用 Pydantic 定义依赖、工具参数和结构化输出，还可以根据函数签名生成工具 Schema，让 Agent 更接近普通 Python 服务的开发方式。比如可以明确规定工具只能接收某种参数，最终结果必须符合指定的数据模型。
>
> **优缺点。** 它的优势是输入、依赖、工具和输出的类型边界清晰，便于校验、单元测试和 Schema 演进，也方便集成到现有 Python 后端。缺点是类型安全只能保证数据形状，不能保证模型决策和业务事实正确；校验失败后的自动重试还会增加 Token 和延迟，它也不提供完整的分布式长任务运行时。
>
> **适用场景。** 它适合结构化抽取、API 型 Agent、数据库操作，以及基于 Pydantic 或 FastAPI 的 Python 后端项目。例如把合同内容抽取为严格的数据对象，或者让 Agent 调用内部 API 并返回可直接写入业务系统的结构化结果。

10.1.4.10 LlamaIndex：以数据和 RAG 为中心

定位：LlamaIndex 是数据与知识库优先的 LLM 应用框架。它也能构建 Workflow 和 Agent，但选择它的首要理由通常是数据连接、文档摄取、索引、检索和 RAG（检索增强生成）。

核心特性：数据连接器、文档摄取、切分与元数据、索引、Retriever、Query Engine、RAG、数据上的 Workflow 与 Agent，以及评测集成。

优点：数据源和检索生态丰富，适合快速建立企业知识库，并将企业数据作为 Agent 上下文或工具。

局限：仍需认真处理数据清洗、权限、版本和召回质量；接上向量库不等于 RAG 已可靠；强状态业务编排可能需要搭配 LangGraph 等运行时。

适用场景：合同、研报、企业文档、知识助手和数据密集型 Agent。

写入简历后要掌握：如何切分、索引和增量更新；Retriever 和重排序怎样设计；如何做权限过滤；用什么指标评估 RAG；为什么不用自研、Haystack 或 LangChain 检索组件。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 LlamaIndex 是一个以数据和 RAG 为核心的 LLM 应用框架。它重点解决数据连接、文档摄取、切分、元数据、索引、Retriever 和 Query Engine，并可以把检索能力作为 Agent 的上下文或工具。它的基本思路是先把企业资料组织成可查询的数据层，再让模型基于检索到的证据回答或行动。
>
> **优缺点。** 它的优势是数据源和检索生态完整，能较快把企业私有数据接入应用，并且对 Retriever、Query Engine 等数据组件的抽象比较成熟。缺点是接上向量库并不代表 RAG 就可靠，数据清洗、权限同步、增量更新、召回、重排序和评测仍然需要专门设计；复杂业务状态控制也可能需要搭配 LangGraph 等运行时。
>
> **适用场景。** 它适合合同、研报、企业文档问答、知识助手以及其他数据密集型 Agent。例如面对持续更新的大量内部资料，需要按部门权限检索并给出证据来源时，LlamaIndex 的价值会比普通工具调用框架更明显。

10.1.4.11 Dify：低代码 AI 应用平台

定位：Dify 是低代码 AI 应用开发与运营平台，而不只是一个 Python SDK。它通过可视化工作流、知识库、模型和工具管理，帮助团队快速发布 Web 应用或 API。

核心特性：可视化 Workflow 和 Agent、知识库与 RAG、模型工具和插件管理、运行日志、Web 应用/API 发布、云服务和自托管。

优点：从原型到可用应用速度快，产品、运营和工程人员可以共同参与，常见知识助手场景开箱即用。

局限：复杂逻辑堆积在画布后，测试、复用和代码审查会变困难；深度定制不如代码框架；开源、托管和企业版能力边界要按版本核验。

适用场景：内部知识助手、运营自动化、快速业务验证和非纯工程团队。不适合把强事务、强实时的核心领域逻辑全部放进画布。

写入简历后要掌握：为什么项目需要低代码；Workflow、知识库、工具和发布如何配合；自托管数据和凭据怎么管；怎样做版本与测试；复杂逻辑如何下沉到独立服务。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 Dify 是一个低代码 AI 应用平台。开发者可以在可视化画布中组合模型、Prompt、知识库、工具和 Workflow，并直接发布成 Web 应用或 API，同时在平台中管理模型配置、凭据和运行日志。它与普通 SDK 的区别是，不只提供代码组件，还提供了可以直接使用的应用搭建和运营界面。
>
> **优缺点。** 它最大的优势是上手快，从原型到可用应用的路径很短，而且产品、运营和开发可以共同参与，知识库类应用尤其容易落地。缺点是流程复杂以后画布会越来越难维护，测试、复用、代码审查和深度定制都不如代码框架灵活；低代码降低的是开发门槛，并不会自动解决业务复杂度。
>
> **适用场景。** 它适合内部知识助手、运营自动化、快速业务验证和需要非工程人员参与搭建的项目。例如业务团队可以自己调整 Prompt 和知识库，而开发团队只提供外部 API 工具；强事务和高并发的核心业务逻辑通常仍应放在独立服务中。

10.1.4.12 AgentScope：国内模型与企业生态候选

定位：AgentScope 面向 Agent 和多 Agent 应用，强调透明、可控以及 Python/Java 和国内模型、云服务生态的适配，常被国内企业纳入私有化和国产模型项目候选集。

核心特性：Agent、多 Agent 协作、工具、消息、记忆、工作流、Python/Java 支持，以及国内模型和阿里生态集成。

优点：对国内模型、Java 团队和阿里云环境更友好，适合具有私有化、数据驻留和国产化要求的候选场景。

局限：国际社区、英文资料和第三方生态与部分海外热门框架存在差异；支持私有化不等于自动满足合规；协议、模型和部署能力应按版本实测。

适用场景：国内企业、国产模型、Python/Java 混合团队和阿里云场景。

写入简历后要掌握：使用了哪些核心模块；选择它是否因为 Java、国产模型、阿里云或私有化；数据是否真正留在企业边界；与 LangGraph、Google ADK 或 Dify 有何区别。

> **面试小卡片｜请简单介绍一下你了解的 Agent 框架**
>
> **核心特点。** 我了解的 AgentScope 是一个支持 Agent、多 Agent、消息、工具、记忆和工作流的框架。它比较鲜明的特点是兼顾 Python 和 Java，并重视国产模型、国内云服务和私有化部署适配，因此不只是研究型多 Agent 框架，也考虑了国内企业的实际技术环境。
>
> **优缺点。** 它的优势是更贴近国内企业技术栈、国产模型和数据驻留需求，对 Java 团队和阿里云环境也更友好。缺点是国际社区、英文资料和第三方生态相对较弱，不同模型与部署能力需要按版本实测；而且可以私有化部署只代表系统能部署在企业边界内，并不代表自动满足安全与合规要求。
>
> **适用场景。** 它适合国内企业、国产模型、阿里云环境，以及 Python 和 Java 混合团队的多 Agent 项目。例如企业要求数据不离开内网，同时需要接入国产模型并让多个专业 Agent 协作时，可以把 AgentScope 纳入候选方案。

10.1.5 框架选型策略

选框架不需要比较一张密密麻麻的功能表。按照下面四步逐层筛选，通常可以把范围缩小到一个主框架，或者最多两个候选。

团队技术栈 → 核心业务场景 → 部署平台 → 模型偏好

第一步：看团队技术栈

先排除团队难以长期维护的框架。

- **.NET / C# 团队：** 优先选 **Microsoft Agent Framework**。
- **Java 团队：** 国内企业优先看 **AgentScope**；Google Cloud 团队看 **Google ADK**。
- **Python 团队：** 12 个框架基本都能选择，继续看业务场景。
- **TypeScript 团队：** 通用 Agent 看 **LangChain**，复杂状态流程看 **LangGraph JS**，OpenAI 项目看 **OpenAI Agents SDK**。
- **产品、运营人员也要参与搭建：** 直接优先保留 **Dify**。

这一步结束后，不要选出最终框架，只需要留下团队真正能够开发和维护的候选。

第二步：看核心业务场景

这是最关键的一步。只问一个问题：**这个项目最难解决的是什么？**

- **通用工具调用、快速原型： LangChain、OpenAI Agents SDK、PydanticAI。**
- 需要多模型和工具生态选 LangChain；
- 已确定使用 OpenAI 选 OpenAI Agents SDK；
- Python 项目重视类型和结构化输出选 PydanticAI。

- **复杂流程、状态、暂停和恢复： LangGraph。**
- **长任务、深度研究、多文件处理： Deep Agents。**
- **代码库、文件编辑和 Shell 自动化： Claude Agent SDK。**
- **企业文档、知识库和复杂 RAG： LlamaIndex。**
- **角色化多 Agent 协作： CrewAI**；国内模型或 Java 环境看 **AgentScope。**
- **低代码搭建并快速发布应用： Dify。**
- **需要用顺序、并行和循环组织多个 Agent： Google ADK**；微软企业技术栈则看 **Microsoft Agent Framework。**

走完这一步，候选通常只剩两三个。

第三步：看部署平台

如果公司已经确定云平台，这一步可以直接进一步收窄候选。

- **Azure / Microsoft 365： Microsoft Agent Framework。**
- **Google Cloud / Vertex AI： Google ADK。**
- **阿里云、国产模型或国内私有化环境： AgentScope**；需要低代码平台时选 **Dify。**
- **本地服务器自托管，业务人员还要参与： Dify。**
- **没有固定云平台：** 保留第二步的候选，不要为了云厂商更换更符合业务场景的框架。

例如核心问题是复杂 RAG，即使部署在 Google Cloud，仍然可以选 LlamaIndex，而不是因为使用 Google Cloud 就强行改选 Google ADK。

第四步：看模型偏好

模型已经确定时，优先选择与该生态结合最自然的框架；没有确定时，优先保留模型相对中立的方案。

- **OpenAI： OpenAI Agents SDK**，尤其适合 Handoff、Tracing 和实时语音。
- **Claude：** 编码、文件和 Shell 任务选 **Claude Agent SDK。**
- **Gemini： Google ADK。**
- **国产模型： AgentScope**；低代码应用可选 **Dify。**
- **需要随时切换不同模型：** 根据场景保留 **LangChain、PydanticAI、LangGraph、Deep Agents 或 LlamaIndex**，避免把业务逻辑写死在厂商特有接口中。

走完前四步，通常已经可以锁定一个主框架，或者最多留下两个候选。

更简单一点的策略：

- 复杂状态与恢复 → **LangGraph**
- 长任务与研究 → **Deep Agents**
- Claude 编码与文件任务 → **Claude Agent SDK**

- 企业数据与 RAG → **LlamaIndex**
- 角色型多 Agent → **CrewAI / AgentScope**
- 低代码快速发布 → **Dify**
- Python 类型安全 Agent → **PydanticAI**
- OpenAI 与实时语音 → **OpenAI Agents SDK**
- Google Cloud 工程体系 → **Google ADK**
- Azure / .NET 企业体系 → **Microsoft Agent Framework**
- 通用模型、工具和检索集成 → **LangChain**

10.1.6 面试问题

Agent 框架相关的面试问题，最常见的主要有两个：**一是考察你是否了解常见框架，二是追问你的项目为什么选择某个框架。**第一个问题其实已经在 1.1.4 各框架特点详细介绍展示过了，现在我们就解析第二个问题：**你的项目为什么选择某个框架?**

这个问题是一个开放性问题，没有标准答案，也许你是拍脑袋就定了你的项目Agent框架。但是即使如此，你回答这个问题的时候，也要根据我们上面的各个框架的特点，选型策略，结合自己项目来回答。很好的一个方法就是把你的项目和上面的这些文档，丢给AI，让它基于这些问题帮你起草一个答案，你以后就照着用就好：

参考案例：SmartAppointment 项目（我们笔记的Agent项目）为什么选择 LangGraph?

参考回答

> 我们这个项目是一个企业售后智能客服与预约系统。它不只是让模型调用几个工具，而是要在一次会话中**完成咨询、故障判断、服务人员匹配和预约处理**，并且在**多个 Agent 之间传递用户信息。**项目最关键的问题是如何管理一个持续变化、能够暂停和恢复的业务状态。
>
> 我选择 **LangGraph，首先是因为它可以把这套流程明确建模为状态图。**我们会在 State 中统一保存用户和会话标识、消息历史、当前意图、预约时间、服务类型、工程师偏好、RAG 证据、当前处理 Agent、工具结果和确认状态。主管 Agent、咨询 Agent、预约 Agent、用户行为分析、工具执行和用户确认分别对应不同 Node，再通过条件 Edge 决定下一步进入咨询、继续补充预约信息、调用预约工具，还是等待用户确认。
>
> **第二个原因是预约任务天然需要循环和中断。**例如用户只说“帮我预约维修”，时间、地址或服务类型可能不完整，流程需要回到槽位补全节点继续询问；创建、修改和取消预约属于有副作用的操作，执行前要通过 Interrupt 暂停，让用户确认关键参数。使用 Checkpoint 保存状态后，即使用户稍后回复或者服务重启，也可以从确认前的状态继续，而不必重新进行前面的咨询和信息收集。
>
> 我也比较过其他方案。LangChain 很适合接模型、工具和 RAG，我们仍然可以使用它的部分组件，但只使用高层 Agent 时，预约状态、条件分支和恢复逻辑不够显式。CrewAI 用角色描述咨询、预约和分析 Agent 会很直观，但这个项目的核心不是让多个角色自由讨论，而是保证预约流程按照业务状态可靠流转。Dify 可以更快做出可视化 Demo，但随着槽位状态、权限确认、幂等写入和自动化测试增加，代码式状态图会更容易维护。

> LangGraph 的代价是开发复杂度更高。我们需要自己设计 State Schema、Checkpoint 存储、节点重试和幂等机制，特别是创建或取消预约时，要通过业务幂等键防止恢复或重试造成重复写入。同时还需要记录节点流转、工具参数和状态变化，方便定位到底是路由、RAG、槽位提取还是工具执行出现问题。
>
> 所以我们的选择依据不是“LangGraph 支持多 Agent”，而是这个系统同时需要共享状态、条件路由、多轮补全、人工确认和中断恢复。**LangGraph 能把不确定的模型判断放进可控制、可追踪的预约流程中，这一点与项目的核心需求最匹配。**

10.1.7 讲解视频

https://www.bilibili.com/video/BV1ZogE6GE4V/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

10.2 LangChain

10.2.1 LangChain 核心设计理念和框架

（算是基本概念，得背）

Langchain 的核心思想是 通过模块化组件： **Prompt, Memory, Tool, Chain 将LLM与外部数据源，工具（向量数据库，多种LLM）结合**，形成**可复用的“链式逻辑**“。特点是灵活，生态丰富，适合快速原型开发。

特点是：**以链为核心，逻辑是线性和简单分支**，不擅长复杂状态管理。

10.2.2 LangChain的版本

（面试官偶尔会问你使用的是哪个Langchain版本，因为Langchain版本更新的确实比较频繁，我知道大家可能有时没太关注版本，我这里整理了一个表格，你需要根据表格里对应Langchain的时间来推断出你当时大概用的哪个版本，要心里有数。从这个表中，大概率可以看出在25年中的时候，我开发的项目，使用的是v0.3.6的Langchain版本，也是在这个时候引入了Graph的概念。弄清楚你用的哪个版本是我们要掌握的）

10.2.3 LangChain 1.0新版本

(LangChain在25年11月份发布了新版本，其实是大的变化。如果你已经做完了项目，并不是说你就要用最新的版本再做一遍，讲起来Langchain的时候，你还是按照我上面设计的答案去回答就行，然后面试官可能会问你新版本你了解吗？你再给讲讲1.0版本的特性。如果你还没做项目，现在开始做，那么推荐你直接用1.0。现在这个还不是硬性要求的，面试很多时候其实并没有一定要考察你新版本，他这个刚刚发布，可能面试官都没

太用。暂时我们先粗略了解1.0的新特性就够了，如果后面我发现面试官开始倾向于问1.0的问题，我再来总结更多新版的知识。

我这里的复习思路是**大概理解一下新特性思路和思想就好了，不用太深入研究**，因为目前问的不多，面试官最多问道你知不知道langchain新版本。 如果你想要深入学习一下，可以看https://mp.weixin.qq.com/s/asAiFhl8mWIl01s-LSD9VA)

Langchain v1.0的升级 标志着 LangChain 已经走出“实验阶段”，成为一个稳定、可用于生产的框架。具体来讲，在这次升级，引入了以下新特性：

1. 智能体（Agent）的全新创建方式

- 之前的 Agent 创建方式比较复杂，需要手动配置多个组件。
- v1.0 提供更简洁的 API，通过标准化的构建流程，开发者可以快速定义 Agent 的行为、工具调用逻辑，并且支持更复杂的任务编排（比如多步骤推理、条件分支）。 2. 中间件机制（Middleware）
- 类似 Web 框架的中间件概念，可以在执行链路中插入逻辑。
- 应用场景：日志记录、性能监控、错误重试、缓存、权限控制。
- 好处：开发者无需修改核心逻辑，就能在执行过程中添加这些功能，提升可维护性。 3. 结构化输出
- 之前 LLM 输出是纯文本，容易出现格式错误。
- v1.0 支持 JSON Schema 和 Pydantic 类型验证，可以强制模型输出符合预期结构。
- 好处：减少 LLM 不确定性，适合生产环境，尤其是需要严格数据格式的场景（如 API 响应、数据库写入）。 4. 标准内容块
- 提供标准化的 Prompt 模板、工具接口、链组件。
- 好处：降低重复开发，开发者可以直接复用官方标准块，减少定制成本。 5. 全新的包结构与模块化设计
- 命名空间重构：拆分为 `langchain_core`（核心逻辑）、`langchain_community`（社区集成）、`langchain_experimental`（实验功能）。
- 好处：模块职责更清晰，升级和维护更方便，避免以前版本中“包太大、依赖混乱”的问题。

10.2.4 LangChain 不方便的地方

（这个问题相比前面的Langchain的基本问题，肯定没有出现的那么频繁。不过这个题目是我从面试真题截取来的，而且这个问题其实也不止被问到一次，反映了面试官想了解你对Langchain的理解是不是浮于表面，有没有一些更深的理解，建议也需要掌握，即使面试官不主动问，你在和面试官聊Langchain的时候也可以找个时机回答出来这两点，体现你其实对这个框架有一些深入的思考，答出来一定会加分。当然这个问题的反面题，你觉得Langchain有没有一些方便的地方，这个你可以回答它集成了各种方法，比如RAG的召回优化，RAGAS的评测方法等，之前笔记我都写出来了）

1.依赖过多，复杂度高

LangChain集成了大量向量数据库，模型提供商和工具，虽然理论是可选，但是实际使用中很多功能需要安装一堆依赖，导致项目臃肿，容易出现"依赖地狱", 对于只需要简单功能的项目，这种设计显得过度。

2.频繁更新，接口不稳定

LangChain迭代速度太快，但是带来大量破坏性更新，API接口经常变化，文档滞后，导致开发需要不断重构代码，维护成本高。

3.性能和延迟问题

LangChain的链式调用每一步都可能涉及外部API， 导致延迟明显，尤其是需要实时响应的场景中表现不佳，链式调用其实也增加了出错概率和调试难度。

4.不够灵活，定制困难

高度抽象导致很多底层逻辑无法轻易改变，遇到特殊需求时，开发者往往需要绕过框架，甚至重写部分功能。

10.3 LangGraph

（LangGraph 相关知识，深入学习可以看这个，很全面 + 有代码示例 https://mp.weixin.qq.com/s/XhFbLTLcSjDj0r3KGT9EOg）

10.3.1 LangGraph 介绍/主要模块

（基本概念，肯定得背）

LangGraph 通过将任务组织为 **有向图**，而不是线性链，来解决 LangChain 在复杂任务中的局限。它支持 **条件分支、循环、并行执行和状态共享**，并且能够实现多 Agent 协作和复杂工作流。核心模块包括：

Graph：整体工作流的图结构

Node：图中的计算单元

State：共享状态，用于节点间传递上下文

Edge：连接节点，定义控制流和数据流

这种设计使 LangGraph 在处理动态逻辑和多步骤推理时更灵活、更可扩展。

10.3.2 LangGraph 过程是如何流转的?

定义图结构：用 `StateGraph` 添加节点和边，指定入口节点（`set_entry_point()`）和结束条件（`END`）。

编译图：调用 `graph.compile()`，生成一个**可执行图对象**（相当于执行器，负责调度和状态管理）。

启动执行：通过 `invoke(initial_state)` 或 `stream(initial_state)` 开始运行，传入初始状态。

执行逻辑：

- 从入口节点开始执行。
- 根据边决定下一个节点，如果多个节点同时满足条件，就**并行执行**。
- 每个节点跑完后，把输出合并到共享状态。

- 重复调度，直到路由到 `END` 或没有后续节点。

结束：返回最终状态，或者用 `stream()` 观察每一步的状态变化。

10.4 面试真题

10.4.1 LangChain / LlamaIndex 解决了什么问题，有哪些局限?

💡 **思路：** 感觉纯八股题，看了我们主流框架介绍的笔记，应该对这些有一个印象。我倒是不建议大家专门抽时间去学习对应框架的内容，我的思路看看我们框架讲解的视频，对于这些八股内容，出现了就背一下，而且框架本身内容，在我看来考的不是特别多，我们也一直在总结面试问题，遇到了面试问题出现的，我都会总结到笔记中的。

📝 **答案：**

LangChain 和 LlamaIndex 本质上都在解决一个问题：**大模型只提供基础的生成能力，而一个完整的 LLM 应用还需要连接模型、私有数据、外部工具、会话状态和执行流程。**它们通过统一抽象和现成组件，减少重复的工程开发。不过两者最初的侧重点不同，现在的能力已经有较多重叠。

LangChain 更偏应用与 Agent 编排。它对不同模型、Prompt、工具、结构化输出和中间件提供统一接口，并支持把这些组件组织成 Chain 或 Agent。比如开发一个能够查询数据库、调用搜索 API、根据结果继续决策的 Agent，LangChain 可以帮助我们完成模型接入、工具封装、工具调用和执行循环。复杂且需要持久化、状态管理的工作流，还可以使用 LangGraph 来实现。

LlamaIndex 更偏数据与 RAG。它重点解决私有数据如何进入大模型应用的问题，提供数据连接器、文档解析与切分、索引、Retriever、Query Engine 和 Agent 等组件。比如把 PDF、数据库和知识库接入系统，建立索引，再根据用户问题检索相关内容并交给大模型生成答案，LlamaIndex 能够快速搭建这条数据和检索链路。

可以简单概括为：**LangChain 更擅长“模型和工具如何协作、任务流程如何执行”，LlamaIndex 更擅长“数据如何接入、组织和检索”。**

它们的局限主要有以下几点。

第一，**抽象在提高开发效率的同时也会增加复杂度。**简单需求使用框架很方便，但复杂业务往往仍需理解底层模型、检索和工具调用机制；一旦需要深度定制，框架抽象可能发生“泄漏”，排查问题时还要继续追踪框架内部的调用链。

第二，**版本和接口迭代较快。**示例代码可能随版本升级失效，组件之间也可能存在兼容性问题，会带来学习和维护成本。对框架封装依赖过深，还可能形成迁移成本。

第三，**框架不能自动保证效果。**使用 LlamaIndex 不代表 RAG 一定准确，仍然需要自己优化文档质量、Chunk、Embedding、混合检索和 Rerank；使用 LangChain 也不代表 Agent 一定稳定，仍要处理错误工具调用、循环、超时、重试和状态一致性等问题。

第四，**通用封装可能带来额外开销并降低可观测性。**对于性能敏感或流程固定的系统，直接调用模型 SDK、向量数据库和业务 API，代码有时会更轻、更容易控制。生产环境仍需自行补充评测、Tracing、权限隔离、安全防护、成本控制和容错机制。

总结来说，LangChain 和 LlamaIndex 的价值是通过标准组件帮助团队快速搭建 LLM 应用：前者侧重**应用和 Agent 编排**，后者侧重**数据接入与 RAG**；它们适合快速开发和验证，但不能替代对底层原理、效果评测与生产工程的掌控。

选自202607宇树科技 Agent开发一面

---

11 Agent评测

这份笔记整理自四类资料：《**AI Engineering**》、《**AI Agents in Action**》、《**Agentic Design Patterns**》，以及美团 LongCat 团队的 **VitaBench** 论文与相关资料。整理完以后，我最深的感受是：**Agent 评估没有一个放之四海而皆准的标准答案。**它必须结合**具体业务场景来设计，业务目标不同、工具链不同、风险边界不同，评测指标和测试方式也会不同。**

Agent 评估本身也相当复杂。一个项目如果真的要做一套非常健全的 Agent 评估体系，可能需要一小波人专门投入来设计评测集、指标、自动化 Pipeline、线上监控、红队安全测试和反馈闭环。现实里，很多企业项目对 Agent 评估这一块其实还没有非常健全，并不一定已经做到“用完善的测试指标指导开发”。我基于和面试官的沟通也有一个判断：绝大多数面试官会问 Agent 测试，但问得并不总是很深入，这也侧面说明很多团队自己对这块的体系化理解还在建设中。

所以，这份笔记希望先帮你建立一个**完整的大框架**：沿着 Agent 开发的生命周期去理解，在每个阶段哪些地方应该测、测什么、怎么测、有哪些常见方法。你顺着这个思路，就能把 Agent 测试和评估的大部分内容理清楚。

每一个小的测试环节，我都根据参考资料总结了一些方法或思路。但还是要强调：这里没有绝对标准答案。真正落地时，应该结合你的业务、你的 Agent 架构、你的工具链和你的风险边界来设计。如果你希望面试回答听起来更贴近实践，而不是只停留在理论层面，可以把本节理论和你自己的项目结合起来想一遍：比如你的项目里应该如何评估工具调用、如何评估多 Agent 协作、如何设置上线前发布闸门、上线后如何监控质量。也可以把你的项目背景和这一节理论一起交给 AI，让它帮你总结成更贴合你项目的回答。

这份笔记希望你达到三个目的：

1. **先掌握理论框架**：理解 Agent 评估为什么复杂，以及它和传统软件测试有什么不同。
2. **能应对面试问题**：当面试官问到 Agent 评估、测试、上线监控、安全、成本延迟、多 Agent 协作时，你的回答能清晰、有条理、有深度。
3. **能反推项目测试体系**：如果你真的想给自己的项目设计一套比较完善的评估流水线，这份笔记能帮你知道应该从哪些模块入手。

最后再次强调，我仍然认为 Agent 评测是一个非常复杂、专业、且高度工程化的事情。如果想深入学习，可以自己再仔细阅读这四份参考资料，如果想更加偏向于学习实战，那么可以结合今天的理论，在自己项目去设计一个评估流水线。

自信地说，如果你能消化好本节笔记，对于绝大部分面试里涉及 Agent 评测的问题，已经足够应付，而且能答得比很多泛泛而谈的回答更完整，更有条理性。

> 本报告基于以下四份资料综合编写：
>
> - **《AI Engineering》**（Chip Huyen, O'Reilly 2025）—— 评估方法论与工程体系
> - **《AI Agents in Action》**（Michael Lanham）—— 评估的工具落地与动手实践（后面文章里我们把这个简称为实战书）
> - **《Agentic Design Patterns》**（智能体设计模式）—— 评估作为设计模式与生产治理
> - **美团 LongCat VitaBench**（ICLR 2026 / arXiv:2509.26490）—— 真实场景下的评测基准实战
>
> 全文每个评测点均标注资料出处，便于追溯。
>
> 前面两本是书籍，没有找到在线观看的链接，如果想要看，可以自己购买书籍，或者电子资源。第三个是一本google的书籍，网上有免费阅读版本。最后一个是论文，也可以直接阅读，链接已经附上。

11.1 Agent评估vs传统行业评估

11.1.1 从TDD到EDD

在传统软件里，测试是写完代码之后的一道工序；但在 Agent 系统里，**评估本身就是开发的起点和方向盘。**

《AI Engineering》提出了一个核心主张——**评估驱动开发（Evaluation-Driven Development）**，它类比于测试驱动开发（TDD）：在你动手搭一个 Agent 之前，应该**先把"怎么算成功"定义清楚。**原因很现实：大模型是概率性的，同样的输入两次运行可能给出不同结果，你无法像传统程序那样"读一遍代码就知道它对不对"，唯一能依靠的就是一套可量化、可重复的评估体系。

这本书甚至直言：**"评估是 AI 工程中最难的问题"**，并用了整整两章（第 3、4 章）来讲它。换句话说，谁掌握了评估，谁才真正掌握了 Agent 的质量。评估的价值贯穿全生命周期：

- **选型时**：靠评估在一堆模型里选出最合适的；
- **开发时**：靠评估判断每次改动是变好还是变坏；
- **上线时**：靠评估做 A/B 测试、灰度放量；
- **生产时**：靠评估（监控）发现线上退化、漂移和安全问题。

> 一句话：在 Agent 时代，**评估不是质检环节，而是研发的指南针。**
>
> 💬 **【面试卡片】 Q：TDD 你了解过吗？你自己用 TDD 做过完整开发吗？**（这个题目是**202606社招 雄帝科技Agent二面的题目**，是粉丝朋友在小红书上分享面试题。本篇所有笔记几乎就是在讲TDD，想告诉大家的是

> 如果学好了这篇笔记，如果问到TDD的内容你就可以引经据典了）
>
> **A：** TDD 我了解，也实践过。传统 TDD 就是测试先行：先写测试用例让它失败，再写最小代码让它通过，然后重构，靠测试来驱动设计，而不是写完再补测试。不过在 Agent 项目里它没法直接照搬，因为大模型是概率性的、输出又是开放式的，同一个输入跑两次结果都可能不一样，没法像传统代码那样断言"输出等于期望值"。所以我用的是它在 Agent 时代的升级版——评估驱动开发（EDD），内核和 TDD 一样：动手之前先把"怎么算成功"定义清楚，再用一套可量化、可重复的评估来驱动每次改动，只是"断言相等"换成了打分、相似度、通过率这类度量。
>
> 具体落地，我是顺着三个阶段做的。第一个阶段是模型确定，先选对地基模型：用成本、延迟、隐私这些硬约束把候选缩小，再用业务私有数据集做终筛，避免被公开榜单的数据污染带偏。第二个阶段是开发阶段，这一段最能体现 TDD 的思想——我会先把各个环节怎么评估定清楚，再去搭 Pipeline。先做组件级评测，把 Prompt、RAG、工具调用、规划推理各自的失败模式逐个测出来；再做集成级评测，用端到端成功率、轨迹评测、多轮对抗、多 Agent 协作验证整机能不能稳定完成任务；等这些维度的测法和指标都定清楚了，我才把评测集和评估 Pipeline 一起组装出来，并设一道发布闸门，质量、延迟、成本、安全都过线才能上。第三个阶段是上线，把同一组指标搬到真实流量里持续监控，做 A/B 灰度、盯漂移和成本安全告警，一旦发现新问题就回灌评测集重新跑回归。
>
> 总结来说，传统 TDD 我用过，在 Agent 里我用它的升级版 EDD，把评估贯穿到模型确定、开发、上线这三个阶段——尤其在开发阶段是先把各环节怎么评估定清楚、再组装 Pipeline 和发布闸门，让评估全程驱动开发，而不是事后补测。

11.1.2 Agent评估和传统评估的不同

为什么不能拿传统软件测试那一套来测 Agent？四份资料从不同角度指出了同一件事：**Agent 打破了传统测试赖以成立的前提。**可以从五个维度看清两者的根本差别：

- **确定性**：传统软件测试同样输入必得同样输出、可断言相等；Agent 评测则是概率性输出，同一输入多次结果不同。
- **任务性质**：传统测试是封闭式（close-ended），输出落在预定义值域内（如分类只能输出有限的几个类别）；Agent 评测是开放式（open-ended），同一输入有无数种可能的正确回答。
- **判定方式**：传统测试有唯一/可枚举的标准答案，把输出与它比对、相等即对（可自动断言）；Agent 评测无法枚举所有正确答案，只能用打分、相似度、通过率去度量"好不好"。
- **评测对象**：传统测试看单个函数的返回值；Agent 评测看一条多步"轨迹"：规划→调工具→观察→再规划。
- **失败形态**：传统测试的失败是抛异常、返回错值；Agent 的失败是工具用错、目标跑偏、自以为完成、绕圈子……

最关键的一条是：**传统测试断言"输出 == 期望值"，而 Agent 的输出根本无法这样断言。**《Agentic Design Patterns》明确指出，正因为 Agent 是非确定性的，传统"通过/失败"的测试不足以保证可靠性，必须转向"对效能、效率、合规做持续测量"的思路。

> **【面试卡片】 Q：你觉得 Agent 行业的测试和传统行业有哪些不同?**
>
> **A：** Agent 的测试和传统软件测试有几个根本不同。
>
> **第一，确定性不同**：传统软件同样的输入必然得到同样的输出，可以直接断言"输出等于期望值"；而 Agent 是概率性的，同一个输入多次运行结果都可能不一样，没法这样断言。
>
> **第二，任务性质不同**：传统软件多是封闭式（close-ended）任务，输出只能落在预定义的值域里，有唯一或可枚举的标准答案，可以直接拿输出和标准答案比对、相等就算对；而 Agent 是开放式（open-ended）任务，同一个输入有无数种可能的正确回答，无法列出一份完整的标准答案来比对，所以只能用打分、相似度、通过率这类方式度量"好不好"，而不是简单地判"对不对"。
>
> **第三，评测对象不同**：传统测试看的是单个函数的返回值，而 Agent 要看的是一条"规划→调用工具→观察→再规划"的多步轨迹。
>
> **第四，失败形态不同**：传统软件的失败是抛异常或返回错值，而 Agent 的失败是用错工具、目标跑偏、自以为完成、绕圈子等。归根到底，传统测试能拿输出和标准答案断言"相等即对"，而 Agent 开放式输出列不出标准答案，所以必须从"通过/失败"转向对效能、效率、合规的持续度量。

11.1.3 Agent评估5大困境

综合四份资料，Agent 评测之所以难，集中在五个"坑"上。这五条建议在面试里能直接说出来：

1. **非确定性，传统 pass/fail 失效。** 同一输入多次运行结果不同，无法用"等于期望值"来判定对错。
2. **不能只看结果，要看"会怎么失败"。** 《AI Engineering》提出一个关键视角：**评估 Agent = 识别它的失败模式，并量化每种失败发生的频率。**一个 Agent 可能最终答对了，但中间用错了工具、绕了远路——只看结果会漏掉这些。
3. **失败面更多、更新。** Agent 比单纯的 LLM 多出了规划、工具调用、多轮交互、多智能体协作等环节，每个环节都是新的失败来源。任务越复杂，失败点越多。
4. **Benchmark 会被"污染"，榜单不可尽信。** 公开 benchmark 的题目可能已经混进了模型的训练数据（数据污染 / contamination），导致分数虚高。选型时不能只看排行榜名次。
5. **"裁判"本身也有偏见。** 现在普遍用 LLM 给 Agent 打分（LLM-as-a-judge），但《AI Engineering》系统揭示了裁判模型自身的偏差：**自我偏好**（偏向自己生成的答案）、**首位偏差**（偏向排在前面的选项）、**冗长偏差**（偏向更长的回答）。还有一条警告——**比较式评估里"偏好 ≠ 正确"**：人类投票选出的"更喜欢"，不等于"更正确"，如果投票者不是内行，甚至会把 Agent 训练成"讨好型"而非"正确型"。

> 这五条困境，正是后面所有评测方法要去解决的问题。可以把它们当成贯穿全文的"问题清单"。
>
> **【面试卡片】 Q：Agent 行业的测试面临着哪些困难?**
>
> **A：** Agent 测试主要面临五大困难。
>
> 第一，非确定性导致传统的"通过/失败"判定失效，同一输入多次运行结果不同，无法用"等于期望值"来判对错。第二，不能只看最终结果，还要看它"会怎么失败"——评估 Agent 本质上是识别它的失败模式、并量化每种失败发生的频率，因为一个 Agent 可能答对了，但中间用错了工具、绕了远路。

> 第三，失败面更多也更新，Agent 比单纯的大模型多了规划、工具调用、多轮交互、多智能体协作等环节，每一个都是新的失败来源，任务越复杂失败点越多。
>
> 第四，公开 benchmark 会被数据污染，测试题可能已经混进了训练数据导致分数虚高，所以选型时不能只看排行榜名次。
>
> 第五，用来打分的"裁判"本身也有偏见，LLM-as-a-judge 存在自我偏好、首位偏差、冗长偏差，而且比较式评估里，“偏好”不等于“正确”：人类投票选出的“更喜欢的回答”，不一定是事实更准确、任务完成得更好的回答。如果评审者
>
> 总结来说，Agent 测试的难点不仅是判断最终结果对不对，还包括衡量多次运行的稳定性、识别执行过程中的失败模式，以及保证评测数据和评判标准本身可信。

11.2 Agent评估全景图

在深入每一种具体的评测方法之前，我们先建立一张“评测全景地图”，让你直观地看清：**一个 Agent 从立项、开发到上线运行，评测到底分布在哪些环节?**

本章是全篇报告的“指引篇”——它先明确解答“哪里要测”，**第 3.3 章再对仗讲清楚**“怎么测”。

11.2.1 一条主线

为什么评测会散布在开发的每一个角落？《AI Engineering》在第 1章极富洞见地指出，评测之所以贯穿始终，是因为要达成以下 **四个核心目的**:

```Plain Text ① 挑选模型 ——→ ② 标记进展 ———→ ③ 决定上线 ——→ ④ 生产监控 (选型期) (开发期) (发布期) (运行期)

> 这些目的恰好落在开发时间线的不同位置上，把它们半闭环地串联起来，就是一条**完整的评测生命周期**。
>
> 这本书还有一个更根本的主张，被称为：
>
> > 💡 **核心观点：评估驱动开发（Evaluation-Driven Development）** 在你动手搭建一个 Agent 之前，**必须先定义“怎么才算成功”**。类比于传统软件的测试驱动开发（TDD），**先定义评估标准，再动手开发；待各维度测法清晰后，再把它们组装成可自动运行的评估 Pipeline。**
>
> 因此，评测决不是开发末尾临时拥抱的防黑天鹅工序，而是一路延续到下线的主航道。其余三份资料也各自深度呼应了这条“评估主线”：
>
> - **《Agentic Design Patterns》**：强调评估要“贯穿到线上生产监控，并最终沉淀为可问责的工程治理”；
> - **《AI Agents in Action》**：直接把“评估”做成了与推理、记忆、规划并列的 **Agent 四大核心组件**之一；
> - **美团龙猫**：示范了如何将这套系统性评估思想凝结成一个真实场景下的生活服务评测基准（VitaBench）。
>
> <a id="doxcnR4zk2JlNZI8tWq0FOgcCff"></a>
> #### 11.2.2 上线前验证 → 上线后监控
>
> 沿着真实交付生命周期，评测不应该简单分成"开发期测质量、上线后测稳定和安全"。更准确的分法是两层：
```

- **上线前验证**：用离线评测集、仿真环境、压测、红队用例和发布闸门判断"能不能上线"；
- **上线后监控**：用真实流量、日志、trace、metrics、用户反馈判断"上线以后有没有退化、漂移、成本失控或被攻击"。

成本、效率、延迟、安全、合规不是生产期才出现的指标，而是**横向贯穿两层**：上线前它们是准入门槛，上线后它们是持续监控项。

下面按这条主线，逐一穿透每个阶段测什么、对应第 3.3 章哪一节：

🛠️ 贯穿全程的「通用武器库」（对应 §3.3.1）

在探讨具体阶段前，必须先掌握三件"通用武器"（LLM-as-a-judge、比较式评估、基础度量）。它们没有固定归属，而是在模型选型、组件调试、发布闸门和生产监控里被反复调用。

✅ 上线前验证：回答"能不能上线"（对应 §3.3.2 \~ §3.3.6）

上线前不是只测"答案对不对"，而是要把模型、组件、整机、资源消耗和安全红线一起放进发布闸门。

- **选型期：挑对地基模型（对应 §3.3.2）**
公开榜单只作粗筛；用 License、限价、延迟、隐私等硬约束缩小候选池；最后用业务私有数据集终筛。这里已经要看成本和延迟，不能等上线后才发现用不起、太慢。
- **开发期前置标准：先定义"怎么算好"（对应 §3.3.3）**
按《AI Engineering》的四类标准圈定方向：领域能力、生成能力、指令遵循、成本与延迟；同时补上 Agent 项目必须有的安全/合规红线。
- **组件评测：逐个测零件（对应 §3.3.4）**
Prompt、RAG、工具调用、规划推理分别隔离评估。Prompt 安全、RAG 忠实度、工具乱调、规划绕路，都属于上线前必须暴露的问题。
- **系统评测：测整机能不能稳定完成任务（对应 §3.3.5）**
用端到端成功率、轨迹评测、多轮用户模拟、多 Agent 协作评测，检查系统整体可靠性。效率也在这里进入评分：步数过多、绕远路、超时，不能算真正可用。
- **评测集 + Pipeline + 发布闸门（对应 §3.3.6）**
把前面各维度的测法组装成可自动回归的评估流水线，并设上线阈值：质量分、Pass^k、P95 延迟、token/成本预算、最大步数、安全红队通过率、误拒率等。

🔬 上线后监控：回答"真实流量下有没有变坏"（对应 §3.3.7）

上线不是开始测试成本和安全，而是把上线前已经定义好的指标迁移到真实流量里持续观测。

> 🎯 **警言：监控的目标和评估的目标，本质上是同一个。** 区别在于：上线前主要用准备好的数据、仿真和压测；上线后用真实用户流量、日志、trace、metrics 和反馈。

- **A/B 测试与灰度发布**：小流量分发，用留存率、转化、满意度、投诉率等线上业务指标作终审，并设置回滚线；
- **监控与可观测性**：用 logs、traces、metrics 记录每次运行，并在这套观测底座上持续监控质量、漂移、异常、成本、延迟、安全和合规。

把两层主线连起来读，就是一个 Agent 评测的完整故事：

> 💡 **全景流串联：** **上线前先把"能不能上线"测透**：选模型 → 定成功标准 → 测组件 → 测整机 → 组装评测集和 Pipeline → 用质量、成本、延迟、效率、安全、合规做发布闸门；
> **上线后再把同一组指标搬到真实流量里持续监控**：A/B 灰度 → logs/traces/metrics → 漂移和异常检测 → 成本/延迟/安全告警 → 回灌评测集重新回归。

> 💬 **【面试卡片】**
> **Q：你认为在 Agent 系统中，应该测试/评估哪些东西？**

>
> **💡 答题思路：**
> 先以“Agent 系统与传统软件的根本不同（如概率性、开放式输出、无唯一标准答案）”作为宏观背景切入，说明为什么无法靠单一测试满足要求。接着，不要把"成本/延迟/安全"误归成上线后才测，而要按 **上线前验证 → 上线后监控** 展开：上线前它们是发布闸门，上线后它们是持续监控项。重点把开发期的次序讲清——**真正前置的只有一份轻量的"成功标准清单"，而评测集和评估 Pipeline 要等组件、集成各维度怎么测定完，才一起组装出来**（因为评测集的标注依赖这些指标）。
>
> **👨‍⚕️ 标准回答：**
> 在 Agent 系统中，由于大模型天然是概率性的、且其输入输出是开放性的，我们没有办法像传统软件那样依靠唯一的标准答案进行 `assert` 断言。因此，Agent 系统不能采用单纯的“黑盒通过性测试”，而是需要将评估与测试融入到**整个开发的生命周期中**。我会按 **上线前验证** 和 **上线后监控** 两层来组织：
>
> 1. **【选型期·模型自身评测】**：挑选合适的地基大模型。我们会使用硬性约束（如成本、延迟、隐私等）进行初筛，然后再使用业务私有数据集对模型的推理和专项能力进行终审测试，防止公开榜单的“数据污染”虚高。
> 2. **【开发期·组件级评测（零件测试）】**：对构成 Agent 的核心零件进行独立单测，核心是识别各组件特有的失败模式。这里主要包含四大零件的测试：**Prompt**（测试鲁棒性）、**RAG检索**（测试召回与回答忠实度）、**工具调用**（测试调错率与填参率）和**规划推理**（测试反思与多步规划）。
> 3. **【开发期·集成级评测（整机测试）】**：把零件拼装起来，对 Agent 作为整机端到端表现开展高压集成测试。主要包括：任务最终端到端成功率（通过高压 Passk^*k* 指标测试稳定性）、关键工具行动路径对齐的轨迹评测（Trajectory）、引入用户模拟器的多轮对抗鲁棒性测试、以及多智能体协作时的分工损耗及协同效率评估。
> 4. **【上线前·组装评测集 + 评估 Pipeline + 发布闸门】**：当上面组件、集成各维度"怎么测、用什么指标"定完，才把**评测集**与**评估 Pipeline** 一起组装出来——先据这些指标标注评测数据集（单测 `test file` + 全场景回归 `evalset`），再选打分法、绑定业务北极星指标、设定可用阈值。发布闸门不只看质量，还要看 P95 延迟、token/成本预算、最大步数、安全红队、提示注入、误拒率等。**之所以放在最后，是因为评测集每条样本的期望输出/评分标准，正是用前面各维度的指标标注的——指标先行，数据才有依据。**
> 5. **【上线后·真实流量监控】**：
>
> - 上线时，采用 **A/B 测试 / 灰度发布**分流，用线上真实业务指标进行版本裁决，并设置回滚线。
> - 上线后，建立以 Logs、Traces、Metrics 为核心的 **可观测性系统**，持续监控质量漂移、异常行为、TTFT/TPOT、token 成本、工具循环、越狱/提示注入攻击、guardrail 触发率、误拒率和合规审计记录。
>
> 总结来说，Agent 的测试并不是单一指标能解决的，它是由“上线前验证发布闸门”和“上线后真实流量监控”共同构成的**全生命周期闭环评估网络**。成本、效率、延迟、安全、合规都不是上线后才测，而是上线前先验收，上线后持续盯。

越往后走，评测的场景便越具体，直接接触生产中极易在 Edge Cases 上坍塌的“深水区”。**接下来的章节，就把这条主线上的每个环节逐一摊开，讲清每一处"怎么测、用什么工具"。**

11.2.3 评估体系的质量与数据分布

前面我们讲了，我们需要构建一个黄金测试集，一个评估标准来评价开发过程中的各个环节。**通过测试来指导开发**。

但是黄金测试集和评估标准的方法，需要**准确，科学**。才能正确指导我们后续的开发。要保证本身这个测试系统的准确性，我们需要从两个维度思考：

1. 我们的**评估体系本身是否科学**，能够反应数据集的好坏？

2. 我们准备的黄金数据集，**比例，分布是否合理，是否具有代表性**？

（温馨提示：这个小节的内容，是我在整理2026年9月阿里淘天秋招实习时，从本次面试中抠出来的考点，并补充到我们评测章节的笔记中的）

11.2.3.1 如何判断评估体系的好坏

这一部分关注评价标准和打分方法本身：能否正确评价 Agent 的表现，结果是否稳定、是否受无关因素影响，以及评估是否实用、可追溯。可以从以下六个方面判断；测试集的分布在第二部分单独讨论。

1. 有效性与判断正确性：测得对不对

评估指标要**真正反映我们关心的能力**，评估结果也应**与可信的人类判断或标准答案基本一致**。

以 Agent 评测为例，不能只看最终回复是否流畅，还要看**任务是否完成、工具是否调用正确、结果是否真实可用**。**分数提高，应当意味着 Agent 的实际表现确实变好**。

2. 区分能力：能否正确区分不同质量

评估系统的打分需要**具有区分度**。

例如，以coding agent为例，正确修复了问题，没有引入回归的例子；修复了主要问题，但是漏掉了边界；完全没有解决问题。 这三类例子，评估的系统给出的**分数要能够拉开差距，体现区别**。

3. 可靠性与一致性：相同条件下是否稳定

在输入、评分标准和配置不变时，**多次评估的结果应基本一致**，不能一次合格、一次不合格。

例如，对同一条 Agent 执行轨迹重复评分时，**分数和结论不应大幅波动**，否则会造成误判。

4. 稳健性与偏差：是否受无关因素干扰

只改变**与目标质量无关的因素**时，评估结果**不应发生不合理变化**。

例如，交换两个 Agent 的展示顺序、隐藏模型名称，或者把同一结果写得更长，不应改变胜负。（前提是这些因素本身不属于评分标准。）

5. 成本与实用性：是否值得使用

评估不仅要准确，还要在**费用、耗时和人工复核成本上可接受**，特别是引入了llm-as-a-duge，评估带来的模型花费值得注意。

6. 可追溯性与持续维护：变化能否解释

评估过程要**能够复现和解释**，并随着 Agent、工具和业务变化**持续更新**。

需要记录**任务集、评分标准、模型版本、提示词、工具环境和运行配置**。当分析某一个问题时，可以根据日志**回溯，重放，定位**，为修复问题提供依据。

> **【面试卡片】 Q：你如何判断你的评估体系是好的？**
>
> **A：** 我会先构建一批**人工标注的校准样本**，验证评分体系本身，而不是只看它给 Agent 打了多少分。比如Coding Agent，我会收集完全修复、部分修复和修复失败的补丁与轨迹，能跑测试的用测试验收，开放性的部分由人工按统一标准判断。
>
> 然后主要检查六个方面：**测得准不准**，是否与测试证据和人工判断一致；**分得开不开**，不同质量的结果能否拉开差距；**评得稳不稳**，同一条轨迹重复评分是否一致；**有没有无关偏差**，是否受展示顺序、模型名字或回答长度影响；另外还看评估的成本、耗时，以及日志能否支持复现和定位。发现误判后，我会调整规则、评分标准或裁判提示词，再用**没有参与

调整的独立样本验证**。这样才能说明评估体系可靠，而不是只在调试样本上表现好。

11.2.3.2 如何设计测试集的分布

有了评估体系，还需要准备分布合理的任务测试集，用来判断 Agent 系统在目标业务中的表现。选取测试任务时，可以遵循以下四条原则。

1. 任务比例反映真实场景

从目标业务的真实场景中抽样，让**主要任务类型的比例接近实际使用情况**。

例如，线上大部分请求是查询，少部分是修改和分析，那么基础测试集也应反映这样的比例，而不是不加区分地让每类任务数量相同。

2. 每类任务覆盖不同情况

同一种任务内部，也要**覆盖不同输入条件和预期行为**，例如该调用工具与不该调用工具、信息完整与信息缺失、单轮与多轮、简单与复杂，**不能只选条件齐全、一路顺利的例子**。

这里的“正例、反例”指**应该执行某种行为和不应该执行某种行为**的任务。例如，查询实时天气需要搜索，而翻译用户给定的一句话通常不需要搜索。Agent 最终完成得好不好，**是评估结果，不是预先安排的成功与失败样本比例**。

3. 对低频但重要的场景专门补测

有些边界情况在真实业务中出现很少，按实际比例抽样可能几乎抽不到，但仍需要专门补充测试任务。

重点补充**容易出错、失败后果严重，或业务明确要求支持**的情况，例如越权请求、工具调用失败、关键参数缺失。**不是所有少见情况都必须大量补充**。

这些任务可以**单列为专项测试集**，既检查重要的低频场景，又**保持基础测试集接近真实业务分布**。

4. 关键类别的样本量足够支撑判断

每一类的样本数量应该足够，**样本数量太小，会导致误差比较大**。当样本数量比较小的时候，误差会比较大。

简记：比例贴近业务，类内覆盖差异，重要长尾补测，样本量支撑结论。

> **【面试卡片】 Q：你的数据集是如何构建的？**
>
> **A：** 我会从**真实业务任务**出发构建测试集，而不是随便凑一批题。比如 Coding Agent，先从历史工单和用户请求中统计 Bug 修复、代码解释、功能修改等任务的占比，再按这个比例抽样，让基础集贴近实际使用情况。每条任务保留问题描述、代码起点、环境和验收依据，历史答案不给 Agent。
>
> 同类任务内部，再覆盖**简单与复杂、信息完整与缺失、单轮与多轮、应该执行与不应该执行**等不同情况。对于工具失败、越权请求、修改公共接口这些低频但重要的场景，我会**单列专项集补测**，避免把它们大量混入后扭曲真实业务比例。另外保证关键类别有足够样本，去重，并把开发调试集与最终测试集分开。最后既看整体表现，也看各类任务和专项场景的结果。核心就是**业务比例真实、类内覆盖充分、重要长尾补测、样本量足够**。

11.3 评测细节

本章是全篇主体。我们沿着第 2 章那张生命周期主线，**从"通用武器库"开始，再依次走过上线前验证（选型 → 前置定成功标准 → 组件 → 集成 → 组装评测集与 Pipeline → 发布闸门）和上线后监控（灰度、可观测性、漂移、成本与安全告警）**。每一个小节都用同一个模板讲：

> **【是什么】** 这一节评的对象是什么 → **【为什么测】** 不测会怎么翻车 → **【怎么测】** 具体指标、工具、可照做的步骤。
>
> 📌 这里的怎么测，注意也不是标准的答案。而是我基于这4个资料来总结的。怎么测这一步有的地方写的是具体的思路，有的地方是一些可以参考的做法。具体真正工程实践上，没有标准答案，而是要根据我们今天讲解的思路，结合自己的项目灵活扩展。

11.3.1 通用武器库

在进入各个生命周期阶段之前，先认识三件"通用武器"。它们不属于某一个阶段，而是**在选型、组件、集成、生产里被反复使用**——先讲清楚，后面各节直接引用。

11.3.1.1 LLM-as-a-judge

【是什么】 用一个（通常更强的）大模型，按照预先写好的评分标准（rubric），去给另一个 Agent 的输出打分或排序。它是目前**最主流的自动评估手段**，四份资料无一例外都在用。

它有三种典型用法：

- **单点打分**：给一个输出打一个绝对分（如"有用性 1–5 分"）；
- **对参考打分**：把输出和"标准答案"比，判断是否一致；
- **两两比较**：给出 A、B 两个输出，让裁判选哪个更好。

【为什么测】 Agent 的输出大多是开放式自然语言，没有唯一标准答案，用精确匹配根本没法判分；纯人工评估又太慢太贵、无法规模化。LLM-as-a-judge 填补了中间地带——它能像人一样理解语义，又能像程序一样批量自动跑。《AI Engineering》给了一个有说服力的数据：**GPT-4 作为裁判与人类的一致性可达 85%，甚至高于人类评审之间的一致性（81%）**。

【怎么测】

> 定性：这是**一个方法、分 3 步走**（写标准 → 固定裁判 → 批量跑），三步是**先后顺序**，不能跳。

1. **【步骤】写清楚 rubric**：定义评估维度和每个分档的含义。实战书的做法是借用教育学的"评分量规"概念，比如评一个推荐器，从 `subject / format / genre` 三个维度各打 1–5 分，并写明每一分代表什么。
2. **【步骤】固定裁判模型和提示词**：设计模式书里用 Gemini（`gemini-1.5-flash-latest`，`temperature=0.2`）搭了一个 `LLMJudgeForLegalSurvey`，并用 `response_mime_type="application/json"` 强制输出 JSON 分数（含 `overall_score`、`rationale`、`detailed_feedback`、`concerns`、`recommended_action` 五个字段），方便后续解析聚合。
3. **【步骤】批量跑 + 聚合**：把多组输入喂进去，解析每条打分，再求平均/分布。

📋 一个真实的 judge 提示词长什么样（直接照搬一个例子）

下面是实战书第 9 章"推荐器"案例里**真实的裁判提示词**（`evaluate_recommendation.jinja2`，Jinja2 模板，`{{ }}` 里是运行时会被填入的变量）。它麻雀虽小五脏俱全，正好凑齐一个标准 judge prompt 的四要素——**角色设定 + 评分维度 + 1–5 分量规 + 指定输出格式**：

```Plain Text
system:
```

```
You are a discerning recommender of {{subject}} {{format}} of the {{genre}}.
Your purpose is to evaluate the quality of given recommendations.
Make sure that the recommendation aligns well with each of the criteria:
Format - make sure the format aligns with {{format}}
Subject - be sure the subject is the same as {{subject}}
Genre - make sure the genre is similar to {{genre}}

Please follow any special instructions and align your scores accordingly given here. {{custom}}

Rate each criteria, subject, format and genre on a scale from 1 to 5 using the following guide:
1  Poor alignment: this is the opposite of what is expected given the criteria.
2  Bad alignment: not a good fit for the given criteria.
3  Mediocre alignment: it may or may not fit well with the given criteria
4  Good alignment: it may not align 100% with the criteria but is a good fit; otherwise
5  Excellent alignment: this is a good recommendation for the given criteria

You will be shown a number of recommendations.
Respond with only the item title and a rating from 1-5 (5 being the best) for
each criterias alignment (Subject, Format and Genre).
Below is an example of the requested output:
Title: Time Bandits
Subject: 4
Format: 5
Genre: 4

user:
{{recommendation}}
```

> （`{{subject}}/{{format}}/{{genre}}` 是评测维度，`{{custom}}` 是留给"特殊评分说明"的插槽，`{{recommendation}}` 填入待评的推荐内容。）

从这个例子能学到三件事：

- **维度可参数化**：把评测维度写成 `{{ }}` 变量，换个场景填不同值就能复用同一套裁判，不必每次重写；
- **量规要逐档下定义**：不是光说"1–5 分"，而是把"1 分长什么样、5 分长什么样"逐档写清——这正是 AI Engineering 强调的 "create scoring rubrics with **examples**"；
- **要给输出格式做示范**：提示词末尾用 `Title/Subject/Format/Genre` 明确示范了期望的输出长相，裁判才会稳定吐出可解析的结果，方便后续批量聚合。

> 💡 **再补一个佐证"裁判没有统一标准"的发现**：AI Engineering 指出，同样叫"忠实度（faithfulness）"这一个指标，三个主流评估工具的 judge 提示词标度各不相同——**MLflow 用 1–5 分、Ragas 用 0/1、LlamaIndex 让模型直接答 YES/NO**。这意味着 LLM-judge 的提示词至今没有行业标准，**换一个工具或提示词，分数就不可比。**

⚠️ **必须知道的三大偏见（面试高频）** —— 《AI Engineering》系统揭示了裁判模型自身的偏差，用它的时候要警惕：

- **自我偏好（self-bias）**：裁判偏向自己生成的答案；

- **首位偏差（first-position bias）**：两两比较时偏向排在前面的那个；
- **冗长偏差（verbosity bias）**：偏向更长的回答，哪怕没更好。

> 书里有一句金句值得背："**看不到裁判模型和它的提示词，就不要相信它给的分数。**" 这意味着任何引用 LLM-judge 结果的榜单，如果不公开裁判细节，可信度都要打问号。

11.3.1.2 比较式评估

【是什么】 不给单个输出打绝对分，而是让多个模型/Agent 的输出**两两对决**，根据胜负用类似国际象棋的 **Elo 评分**算出排名。最著名的就是 Chatbot Arena。

【为什么测】 有些质量很难打绝对分（"这个回答到底算 3 分还是 4 分？"很主观），但人类**比较**两个谁更好却相对容易、也更稳定。比较式评估利用了这一点，更适合给一批模型排座次。

【怎么测】

> 定性：**一个方法、分 2 步**——先攒"两两对决"的胜负数据，再用算法换算成排名分。

- **【步骤】收集对决数据**：收集大量"同一问题、不同模型的两个回答"，让人或 LLM 投票选更好的；
- **【步骤】换算排名**：用 Elo / Bradley-Terry 类算法把胜负记录换算成排名分。

⚠️ **两个关键提醒：**

- **要和 A/B 测试区分开**：A/B 测试是把不同版本分给不同用户群看线上指标；比较式评估是**同时拿多个输出给同一个评委比**，两者不是一回事。
- **"偏好 ≠ 正确"**：投票选出的"更受欢迎"不等于"更正确"。如果投票人不是该领域内行，可能把 Agent 带偏成"讨好型"——回答让人舒服但其实是错的。所以**评委的专业度至关重要。**

11.3.1.3 基础度量：精确评估与相似度

【是什么】 一组**不依赖大模型裁判**的、更传统也更廉价的度量手段，适合有明确答案或可程序化判定的场景。

主要分三类：

- **功能正确性**：直接看任务有没有真的完成。比如代码题就跑测试用例（HumanEval、MBPP），text-to-SQL 就执行 SQL 看结果对不对——**这是最硬、最可信的指标。**
- **与参考答案的相似度**：
- *词汇级*：精确匹配、BLEU、ROUGE（看字面重叠）；
- *语义级*：BERTScore、嵌入向量余弦相似度（看意思接近）。设计模式书还补充了 Levenshtein 编辑距离、Jaccard 相似度等。

- **语言建模指标**：交叉熵 / 困惑度（perplexity）。除了衡量语言流畅度，《AI Engineering》还点出一个巧妙用途——**用它检测数据污染**（如果模型对某测试集 perplexity 异常低，说明可能见过这些题）。

【为什么测】 不是所有场景都值得动用昂贵的 LLM-judge。**只要任务有可程序化验证的标准答案（代码、SQL、数学、分类），就应该优先用功能正确性**——它客观、便宜、可重复，是最可信的一类证据。

【怎么测】

> 定性：这里不是步骤，而是**按"答案有多可验证"三选一档**——越靠上越客观越该优先用。

- **【档位•最严】有标准答案**（分类、抽取、SQL、代码）→ 用功能正确性 / 精确匹配；
- **【档位•中】答案灵活但有参考**（翻译、摘要）→ 用 BLEU/ROUGE + BERTScore；
- **【档位•最松】完全开放**（对话、创作）→ 退回到 3.3.1.1 的 LLM-as-a-judge。

> **选型口诀**：**能用功能正确性就别用相似度，能用相似度就别用 LLM-judge——越靠前越客观越便宜。**三者常常组合使用，因为没有任何单一指标能覆盖 Agent 的全部能力。
>
> 💬 **【面试卡片】 Q：功能正确性、相似度、LLM-judge 三类指标怎么选?**
>
> **A：** 按"答案有多可验证"从严到松三选一：
>
> ① 有标准答案（分类、抽取、SQL、代码）→ 优先用**功能正确性 / 精确匹配**，最客观、最便宜、可重复；
>
> ② 答案灵活但有参考（翻译、摘要）→ 用 **BLEU/ROUGE 加 BERTScore** 等相似度；
>
> ③ 完全开放（对话、创作）→ 才退回到 **LLM-as-a-judge**。口诀是：能用功能正确性就别用相似度、能用相似度就别用 LLM-judge，越靠前越客观越便宜；实战中三者常组合，因为没有单一指标能覆盖 Agent 的全部能力。

> 💬 **【面试卡片】**
>
> **Q：在 Agent 评测中用 LLM-as-a-judge（大模型当裁判）很火，请客观评价一下它的优缺点？如果由你负责实际项目落地，应该怎么科学避坑?**
>
> 💡 **答题思路：**
>
> 先表现出技术批判精神。承认它在“开放、主观或无标准答案任务中有强大的语义理解优势”，但紧接着点出它的“致命死穴（运行高成本、三大心理学偏见、低标度一致性）”。最后拿出一整套**工程落地的避坑干货**（如：CoT分步推理、JSON模式强约束、双向逆序对决、人机相关性校准等），证明你是真正动手写过生产代码、踩过坑的资深工程师，而非浅尝辄止的概念党。
>
> 👨‍💻 **标准回答：**
>
> 在实际开发中，我们有大量的 Agent 生成产物是难以用正则或功能用例执行来断言的。这时候，LLM-as-a-judge 填补了传统自动化测试和昂贵人工审核之间的巨大空白、成为了必不可少的通用武器。但我们绝不能盲信它。以下是我对这一评估方法的深度剖析以及工程实战避坑指南：
>
> **一、核心优势**

1. **无与伦比的“语义匹配与泛化”能力**：不同于传统的字面匹配（如 BLEU、ROUGE 容易因为词汇错位而误判），它能真正读懂“意思相近但表达不同”的开放式文本，或针对无标准答案的任务（如多边法律合同总结、创意写作）进行客观的多维度打分。
2. **极高的反馈吞吐量与极低的人力耗损**：让人类专家逐个标注数万条 Agent 的输出是不现实的。利用大模型作为裁判，我们可以并行数千个并发打分线程，将迭代优化周期缩短到分钟级，让 CI/CD 流程的高频反馈成为可能。
3. **高水平的人机对齐一致性**：实验证明，只要评分标准维度切分得足够精细，GPT-4 或高级开源模型裁判和人类专家的一致性可以稳定在 **85%** 以上，甚至高于水平参差不齐的临时人类外包人力的一致性。

二、致命劣势

1. **三大天生认知偏差**：

- **自我偏好偏见**：它会天然地给“由它自己生成的、或和它本身风格最相似”的文本打高分。
- **首位顺序偏差**：在进行两两比较时（Pairwise Match），如果两个模型输出质量相近，它更容易倾向于打给第一个展示出来的模型。
- **冗长语言偏差**：它是个彻头彻尾的“字数党”，往往更偏心字数巨多、排版华丽的空洞回答，哪怕短回答才是真正高密度的精确解决方案。 2. **判分的不稳定性与无法水平度量**：大模型对提示词极度敏感。正如 AI Engineering 揭示的那样，不同评测工具定义同样的指标（如 Faithfulness），标度却完全不一致（有 1–5 分、0/1、YES/NO）。换一个提示词或稍改下词眼，原本的 80 分就可能变成 60 分，分数不具备像数学指标那样的绝对通用性。 3. **可怕的运行成本墙**：如果每次 CI/CD 提交测试都要调用数十次 GPT-4 做 Judge，一趟迭代可能就会烧掉数千美元，最终限制了开发阶段的高频运行。

三、工程落地避坑指南（我是如何最大程度校准裁判的）

在实际架构开发中，为保证 LLM-as-a-judge 的客观与高效率，我会严格执行以下五条金科玉律：

1. **【Rubric 颗粒化 + Few-Shot 示例注入】**：拒绝简单的“好/中/差”或“1-5自拟大纲”等模糊打分。我会像实战书一样，将评测指标拆分为 `Format / Subject / Relevance`，每一档（如 3分、5分）必须配备极具说服力的真实 Few-shot 对比样例，引导大模型按样本对比。
2. **【CoT 分步推理 + 严格的 JSON 强约束】**：在 API 中开启 `response_mime_type="application/json"`，将 Schema 约束设为含有：`rationale`（裁判思路原因）和 `overall_score`（综合得分）的字典。必须强制模型**先输出理由，再输出分数**（利用思维链减缓裁判幻觉和打分随机性），拒绝让其立刻给出数字。
3. **【洗牌（Shuffle）对抗两两偏差】**：如果是 Arena 类型的两两比对，必须给 A、B 两种输出同时运行“A在先，B在后”和“B在先，A在后”两轮，并将顺序打散。只有当两个相反方向表现一致或多轮随机均值胜出时，才算真正胜出，以此完全抵消首位顺序偏差。
4. **【人机偏差定量拟合（Calibration）】**：阶段性抽样 **10% - 20%** 裁判打过分的数据集给领域的资深人类专家重新评分。我们写代码计算两者的 **Cohen's Kappa 系数** 或 **Pearson 统计相关度**。如果拟合度低于 0.75，立即叫停系统，修改 Rubric 提示词，直到重新校准至接近人类共识。
5. **【低温运行与模型平替】**：在 Judge 时一律将 `temperature` 设置为 **0.0 - 0.2** 的极低范围，保证结果确定。同时，对于日常功能迭代或简单的单维度提取验证，抛弃 GPT-4 等昂贵闭源，使用开源的小模型（如

> Llama-3-8B-Instruct 甚至更优秀的领域特定蒸馏裁判小模型）进行平替，用 2% 的成本跑完 90% 的基础校验。

11.3.2 选型期：选基模

在笔记 7.不同模型的对比和选择这一章节，也有讲解模型的选择问题。这两个小节是互补的，从不同的维度来讲解模型的选择。

【是什么】 在动手开发之前，从一堆候选大模型里评测、挑出最适合你这个 Agent 任务的那个（或那几个）。

【为什么测】 模型是 Agent 的"地基"，选错了后面再多优化也补不回来。但选型最大的坑在于——**不能只看排行榜名次。**《AI Engineering》专门用一节讲选型的陷阱：公开榜单可能存在**数据污染**（测试题混进了训练集，导致分数虚高）、榜单任务和你的真实业务未必相关、闭源模型还涉及成本与数据隐私的权衡。

【怎么测】（一套可照做的选型工作流）

> 定性：**一个方法、一条"五步漏斗"**——从粗到细一层层筛。其中第 4 点是漏斗每一层都要看的**考查维度**（不是独立一步），夹在步骤里说明。

1. **【步骤】区分硬性约束和软性偏好**：先用硬指标（成本上限、延迟上限、是否支持私有部署、上下文长度）筛掉不合格的；剩下的再比软指标（质量、能力）。
2. **【步骤】公开榜单只作初筛**，并对它保持警惕：查这个 benchmark 有没有被污染、任务是否贴近你的场景。
3. **【步骤】建私有评测集做终筛**：用你自己业务的真实数据（公开榜单看不到的）跑一遍，这才是最可信的。
4. **【维度】每一层都要覆盖的考查点**：正确性、成本、延迟，以及领域专项能力。
5. **【步骤】build vs buy 决策**：自建开源模型 vs 直接调 API，按成本、隐私、可控性权衡。

> 💬 **【面试卡片】 Q：你是如何确定你的模型的?**
>
> 💡 **答题思路：** 让 AI 读这道题的思路 + 对应项目代码后再作答。思路是——**不要上来就报一个模型名，而要展示一条选型决策链**：先按"场景 → 能力 → 模型"反推（业务到底要什么能力、各能力配多少权重），再讲怎么验证（私有集盲测、防榜单污染），最后落到工程参数（同一模型按子任务差异化配置）。结论**按方法论正向推演"该用哪一类模型"，而不是为项目里既成的选择找理由。**用人称"我"，结合下面智能预约 Agent 项目的真实能力需求来组织。
>
> *（本答案以笔记里的 Agent 项目 —— 智能预约 Agent 为例组织能力需求；模型结论按理论推演得出）*
>
> 👩‍💻 **标准回答：** 我确定模型不是看榜单排第一就用谁，而是走"**场景 → 能力 → 模型 → 验证**"四步，让能力需求来决定该选什么模型。

> **第一步，从业务场景反推所需能力，并配权重。** 我这个智能预约 Agent 是多 Agent 架构，要做意图分类、RAG 知识问答、预约槽位解析、调外部工具（MCP 拉天气）。把它拆成可量化的能力清单：**Function Calling / 工具调用是必要能力（最高权重，约 40%）**——预约 Agent 要稳定地调天气工具；**结构化输出与指令遵循（约 25%）**——要可靠抽出时间/项目/时长/性别这些槽位；**多轮对话上下文（约 20%）**；剩下是**中文理解和成本/延迟（约 15%）**。另外 RAG 那条线还需要一个**嵌入模型**单独选。
>
> **第二步，把能力画像翻译成对模型的硬性要求。** 上面这套权重直接刻画出我要的模型长相：① 必须**原生支持 function-calling / tools**（这是硬门槛，不支持的直接出局）；② 在槽位抽取上**结构化输出要稳**（temperature 调到 0 时不能崩格式）；③ 中文场景要好；④ 是个在线预约、要走流式回答的 To C 应用，所以**延迟和单次调用成本**要可接受。
>
> **第三步，据此推演该用哪一类模型（结论）。** 按这套要求正向推：工具调用是必要能力且要稳，所以**应当选一个一线、对 function-calling 支持成熟的通用大模型作为主力**（GPT-4o 级 / Claude / 国内 Qwen-Max、DeepSeek 这一档都是合理候选，取决于成本和数据合规）；不适合上能力不稳的小模型硬扛工具调用。再做**成本分层**：意图分类这种高频、简单的子任务，可以下放给一个更便宜的小模型；只有复杂推理和工具编排才用旗舰模型——这样整体 TCO 更优。**嵌入模型**则按检索质量+向量维度+成本，选一个主流 embedding 模型（如 bge、text-embedding 系列）即可，不需要和主 LLM 同源。
>
> **第四步，工程配置 + 验证。** 同一个主模型，我会按子任务的"确定性 vs 创造性"**差异化设 temperature**：意图分类和槽位解析用 **0**（要可复现）、RAG 咨询用 **0.3**、主动推荐用 **0.7**。验证上不迷信公开榜单（警惕数据污染），而是**用真实预约对话抽样建私有测试集做盲测**，重点看"工具调用准确率"和"槽位抽取正确率"这两个必要能力达不达标，达标了才定下来。
>
> **一句话收尾：** 我确定模型的逻辑是——**从场景拆出带权重的能力需求 → 用必要能力（function-calling + 稳定结构化输出）反推出"该用一线通用大模型做主力 + 小模型分层降本 + 主流 embedding 配检索" → 再用业务私有集盲测验证，而不是看榜单名次直接拍板。**

11.3.3 上线前：组件评测

Agent 是由多个组件拼起来的：提示词、检索、工具、规划。这一阶段**把每个零件单独拎出来测**，因为只有先保证零件合格，整机才可能合格。

11.3.3.1 Prompt 评测

【是什么】 评估提示词（prompt）写得好不好、以及它扛不扛攻击。

【为什么测】 提示词是 Agent 行为最敏感的"控制旋钮"，改一个词输出可能天差地别。但 prompt 的好坏不能靠"感觉读着顺"，得有系统化的对比；同时 prompt 还是安全攻击（越狱、注入）的主要入口，必须做防御测试。

【怎么测】

> 定性：**3 个相互独立的方法**，分别测 prompt 的"好不好用"、"能不能自动调优"、"扛不扛攻击"——按需要取用。

- **▶ 方法一 • 质量评测：用微软 Prompt Flow 做批量对比。**这是实战书第 9 章的主角，本身是一条**分 4 步**的落地流程：

1. **【步骤】** 在 VS Code 里用 Prompt Flow（DAG 可视化流程图）搭一条评估流；
2. **【步骤】** 用 JSONL 准备多组输入，批量跑不同的 prompt 变体（比如把参数注入到 system prompt 还是 user prompt）；
3. **【步骤】** 用 LLM-as-judge 按 rubric 给每个变体的输出打分；
4. **【步骤】** 用 `aggregate.py` 聚合各 run 的平均分、`log_metric` 记录指标，可视化对比哪个 prompt 更好。 - **▶ 方法二 • 自动 prompt 优化工具评测**：DSPy、TextGrad 等能自动搜索最优 prompt，但要警惕它们隐藏的 API 调用成本和模板 bug。 - **▶ 方法三 • 防御性评测**：主动测 prompt 的鲁棒性（轻微扰动看输出是否崩）和安全性（上线前进 发布闸门，上线后进 持续监控）。

> 落地抓手一句话：**Prompt Flow + JSONL 批处理 + LLM 打分 + 聚合对比**，把"调 prompt"从玄学变成可量化实验。

11.3.3.2 RAG / 检索评测

在笔记 RAG 评估章节，有更详细的专门讲RAG的评估内容。

【是什么】 评估 Agent 的"外挂记忆"——检索增强生成（RAG）——找回来的资料质量好不好。

【为什么测】 RAG 是"先检索、再生成"。如果检索这一步找回来的就是错的/不相关的，那后面生成得再流畅也是"一本正经地胡说八道"。所以 RAG 的评估要**拆成检索和生成两段分别看。**

【怎么测】（一组成熟的检索指标）

> 定性：不是并列的几个方法，而是**把 RAG 切成"检索段 + 生成段"两段**，每段列出要看的**考查维度。**前四个是检索段维度，第五个是生成段维度，最后一个是实战书的动手实现。

- **【维度•检索段】context precision（上下文精确率）**：找回来的内容里，有多少是真正相关的；
- **【维度•检索段】context recall（上下文召回率）**：所有该找回的相关内容，找回了多少；
- **【维度•检索段】排序质量**：用 NDCG、MAP、MRR 衡量相关内容是否排在前面；
- **【维度•检索段】嵌入模型质量**：用 MTEB、BEIR 这类基准评测你用的 embedding 模型好不好；
- **【维度•生成段】生成段的忠实度（faithfulness）**：生成的回答是否真的"扎根"于检索到的内容、有没有脱离资料瞎编。
- **【实现】工程实操**：实战书第 8 章演示了直接看**向量相似度分数**、调 **top-n** 数量、试不同 **chunk 切分策略** 对检索准确率的影响——这些都是能立刻上手调的旋钮。

> 记忆要点：RAG 评测 = **检索段（precision/recall/NDCG/MRR）+ 生成段（faithfulness）** 两段都要测。
>
> 💬 **【面试卡片】 Q：评估一个 RAG 系统，你会从哪些维度入手？用什么指标？**

> **A：** RAG 是"先检索、再生成"，所以我会拆成检索段和生成段两段分别评。检索段看四个维度：context precision（找回的有多少是相关的）、context recall（该找回的找回了多少）、排序质量（用 NDCG / MAP / MRR 看相关内容排没排在前面）、以及嵌入模型质量（用 MTEB / BEIR 评 embedding）。生成段重点看忠实度（faithfulness）——回答是否真的扎根于检索内容、有没有脱离资料瞎编。工程上还能直接调向量相似度阈值、top-n 和 chunk 切分策略来提检索准确率。一句话：RAG 评测 = 检索段（precision/recall/NDCG/MRR）+ 生成段（faithfulness）两段都要测。

11.3.3.3 工具调用评测

【是什么】 评估 Agent 调用外部工具（API、函数、数据库）的能力——选对工具没有、参数填对没有、调用顺序对不对。

【为什么测】 工具调用是 Agent 区别于普通聊天机器人的核心能力，也是高频失败点。注意它有**三个不同层次**，面试时能区分开会加分：

- **能力层**：在很多工具里能不能选对、调对（龙猫：66 个工具 + 512 条依赖边的工具图）；
- **行为层**：调用工具的顺序对不对（属于轨迹评测，见 3.5.2）；
- **代码层**：工具函数本身写得对不对（实战书第 5 章用单元/集成测试逐个验证 semantic function）。

【怎么测】

> 定性：**一个方法、分 3 步**（造考题 → 跑 Agent 逐项核对 → 补反例），三步按顺序走；其中第二步的"判分"内部有**两个档位**可选。

核心思路是：**造一批"考题"，让 Agent 去调工具，再逐项核对它调得对不对。** 这套方法可以参考业界权威的 Berkeley Function-Calling Leaderboard（BFCL，一套约 2000 道题的工具调用评测基准）——但要注意，BFCL 是**选型期**（§3.2）用来给各家**模型**的通用工具调用能力排名的，**测的不是你自己的业务工具。**我们要做的，是**借用它的判分方法，把题目换成自己项目的工具。**

【步骤】第一步：制作测试集——每条用例长这样（三件套）

以智能预约 Agent 的 `check_weather` 工具为例，一条用例由三部分组成：

```Plain Text
{
  // ① 用户问题（喂给 Agent）
  "query": "明天会下雨吗？我想约个按摩",

  // ② 可用工具清单（喂给 Agent，告诉它有哪些工具、参数是什么）
  "tools": [
    { "name": "check_weather", "params": ["date(必填)", "city"] },
    { "name": "check_availability", "params": ["..."] }
    // 干扰项，看会不会选错
  ],

  // ③ 标准答案（只给判分器，Agent 看不到）
  "expected": "check_weather(date=\"2026-06-26\")"
}
```

> 把这样的用例攒成一个小数据集（几十条，覆盖常见、易错、和"不该调工具"的情况）。
>
> **【步骤】第二步：让 Agent 跑，再逐项核对**

把①②喂给 Agent，它会输出一个工具调用；判分器拿它和③标准答案**像填检查表一样逐项核对**：① 工具名选对没 → ② 必填参数齐不齐 → ③ 有没有瞎编不存在的参数 → ④ 参数值对不对。任何一项错，这条就算失败，而且能精确定位错在哪一项。

核对时有**两个着眼点**，可二选一、也可搭配用：

- **看调用本身写得对不对**：只比对 Agent 吐出的这行调用——函数名、参数个数、参数值是否和标准答案一致。不依赖工具真实可用，无副作用、可复现，适合大批量离线判分。
- **看真跑出来的结果对不对**：真的执行 `check_weather`，检查它的返回值。能抓住"调用写得对、但结果其实是错的"这类问题，最贴近真实，但会产生真实副作用、需联网、且结果可能随环境波动。
- 实战中常**两者搭配**：多数用例只核对调用本身，关键工具再补几条真执行兜底。

【步骤】第三步：别忘了"不该调"的反例 专门放几条**不需要工具**的问题（如"你们几点关门？"），看 Agent 会不会克制、不乱调工具——乱调就是"工具幻觉"。**知道何时不该用工具，和知道何时该用一样重要。**

> 三份资料的分工：判分方法（核对调用 / 核对结果）来自 **AI Eng**；在一堆工具里选对的难度由 **龙猫**（66 工具依赖图）刻画；每个工具函数本身的对错由 **实战书**第 5 章 `test_*.py` 单测保证。
>
> **【面试卡片】** **Q：怎么评估 Agent 的工具调用能力？它有哪几个层次？**
>
> **A：** 我会分三个层次看：**能力层**——在很多工具里能不能选对、调对（龙猫用 66 个工具加 512 条依赖边的工具图来考）；**行为层**——调用顺序对不对，这归轨迹评测；**代码层**——工具函数本身写得对不对，用单元/集成测试逐个验证。具体测法是：先造测试集（用户问题 + 可用工具清单 + 标准答案三件套，含干扰项），再让 Agent 跑、逐项核对工具名选对没、必填参数齐不齐、有没有瞎编参数、参数值对不对，任一项错即失败、还能定位错在哪。核对时可以只看调用本身写得对不对（不依赖工具真实可用、无副作用、可大批量离线判），也可以真执行一遍看返回结果对不对（能抓住调用对但结果错的情况，但有副作用、需联网），两者常搭配用。最后别忘了放几条"不该调工具"的反例，看 Agent 会不会克制、避免工具幻觉。

11.3.3.4 规划 / 推理评测

【是什么】 评估 Agent 的"大脑"——它能不能把一个复杂任务拆解成合理步骤、并在执行中正确推理。

【为什么测】 先划清和上一节的界线： **工具调用看的是"某一次调用填得对不对"（单点动作）， 规划/推理看的是"整条决策链对不对"（全局思路）。** 一个管"手"，一个管"脑"——Agent 可能每一步调用都很标准，却从一开始就想错了该做什么。

《AI Engineering》把规划失败拆成三类，正好告诉我们"要盯哪些错" ：

- **该不该用这个工具 / 这一步**：在决策层面选错了工具或多走了无谓的一步（注意：选哪个工具属于这里；至于参数填没填对，归 3.4.3）；
- **目标失败**：没达成目标，或达成了但**违反了约束**（书里特别强调"时间"约束最容易被忽略，比如约到了营业时间之外）；
- **反思错误**：任务没真完成，Agent 却**误以为完成了**就停手。

【怎么测】

> 定性：**一个方法、分 3 步**（造带"理想步骤链"的考题 → 跑 Agent 对照三类失败核对 → 统计并借力三本书的机制）。第三步底下三个点是**三份资料各自的实现**，并列、可任选增强。

核心思路是：**给 Agent 一个需要分几步才能完成的任务，记录它实际走的"步骤链"，再和"理想步骤链"对比，看它有没有想错。** 和 3.4.3 不同——这里不抠单次调用的参数，而是看整条路对不对。

> **【步骤】第一步：制作测试集——每条用例长这样**
>
> 以智能预约 Agent 的"约按摩"任务为例，一条用例记录的是**理想步骤序列 + 约束**：
>
> ```Plain Text
> {
>   // ① 任务
>   "task": "帮我约明天下午两点、60分钟的精油按摩",
>
>   // ② 理想步骤链（Agent 应该按这个顺序推进）
>   "expected_plan": ["查档期", "匹配技师", "确认下单"],
>
>   // ③ 必须满足的约束（违反就算失败）
>   "constraints": ["时间=明天14:00", "时长=60分钟", "时间需在营业时间内"]
> }
> ```

【步骤】第二步：跑 Agent，对照三类失败逐一检查

让 Agent 真的去执行，记录它实际走了哪几步，然后对照三类失败核对：

- **步骤链对吗**：看实际步骤 vs 理想步骤。典型失败——没"查档期"就直接"确认下单"，属于步骤缺失。
- **守住约束吗**：看有没有违反约束。典型失败——约到了晚上 20:00（超营业时间），属于违反约束。
- **反思对吗**：看是否真完成才收尾。典型失败——技师没匹配上，却回复"已约好"，属于自以为完成。

【步骤】第三步：统计 + 借力（下面三点是三份资料各自的实现，可任选叠加）

- **【实现·AI Eng】失败模式频率统计**：评估本质是"识别失败模式 + 数每种出现多少次"。常用指标：有效计划占比、平均要试几次才产出有效计划。
- **【实现·龙猫】复杂度可控的 benchmark**：龙猫用"推理复杂度"维度（观测空间大小、部分可观测程度、推理点数量）量化任务有多难，从而系统地考规划。
- **【实现·实战书】推理自评机制**：实战书第 10 章用 self-consistency（生成多条推理路径、投票选最一致的）和 tree-of-thought 来提升并间接评估推理质量。

第二步只标出了"这条用例哪里错了"，第三步要把它**变成可比较的数字**。分三件事，前两件是"怎么打分"，最后一件是"怎么把题出难"——别混在一起：

(1) 给单条路径打分——两种判法，按有没有标准答案选

- **和标准答案比相似度**（实战书第 10 章的实际做法）：把 Agent 最终答案和 `expected` 都转成向量，算语义相似度，得一个 0\~1 的分（书里样例就直接输出 `evaluation_score: 0.957`）。适合有明确标准答案的题。
- **看多条路径自不自洽**（self-consistency，**不需要标准答案**）：对同一任务让 Agent 生成 N 条推理路径，然后
- **多数投票选答案**：N 条结论里哪个出现得最多就采纳它，离群的那条丢掉（书里例子：3 条里 2 条一致，就取那 2 条）；

- **一致性 = 可信度**：多数有多集中本身就是分数——大多数路径都指向同一答案 → 推理稳、可信；各说各话没有多数 → 说明没想清、可信度低。

(2) 汇总成整体指标：评估本质是"识别失败模式 + 数每种出现多少次"。把很多条用例的结果汇总，算：有效计划占比、平均要试几次才产出一个有效计划、各类失败（步骤缺失/违反约束/自以为完成）各占多少。

(3) 让题目更难更系统（属于"出题"不是"打分"）：用"推理复杂度"维度（观测空间大小、部分可观测程度、推理点数量）拧难度旋钮，系统地造出从易到难的题，避免"题太水、高分也说明不了能力"。

> 一句话记忆：规划评测 = **对照"理想步骤链"，盯三类失败（步骤选错 / 违反约束 / 自以为完成）→ 给路径打分（和标准答案比相似度，或用 self-consistency 多数投票+一致性）→ 汇总成频率指标**。龙猫的复杂度维度负责把题出难。它看的是整条路，3.4.3 看的是路上的每一步调用。

11.3.4 上线前 • 集成 / 系统评测

零件合格不等于整机合格。这一阶段把 Agent 当成一个整体，看它端到端能不能真的把任务办成。

11.3.4.1 任务结果评测

【是什么】 最直接的一问：给 Agent 一个完整任务，它最终**做成了没有**。

【为什么测】 这是用户唯一真正关心的东西。但"做成没有"的判定方式，三份资料因**目的不同而选了不同严格度**，这点很值得讲（面试亮点）：

- **龙猫**：用**全有或全无**——Pass@k / Pass^k（跑 k 次有一次成功 / k 次全成功）。因为要给模型严格**排名**，必须能拉开差距。
- **实战书**：用**1–5 分级**（rubric 打分）。因为开发期要**诊断**哪里弱，需要颗粒度。
- **设计模式**：用**语义相似度 / 忠实度**（连续分）。因为要**持续监控**漂移，需要连续信号。

> 关键认知：**这不是矛盾，而是"目的决定方法"**——排名要严、诊断要细、监控要连续。用错了场景（比如拿 Pass^k 做日常开发诊断）就会觉得"怎么全是 0 分"。

【怎么测】

> 定性：判分方法不是并列的方法，而是**按"你的目的"选一档严格度**——目的决定方法（接上表）。

- **【档位•要排座次/狠考】**：用 Pass@k / Pass^k，温度固定，每个任务重复多次。龙猫用 **Pass^4**（同一任务跑 4 次必须全对）来逼出模型的稳定性。
- **【档位•要诊断改进】**：用 rubric 多维度 1–5 分，看具体哪一维拉胯。

11.3.4.2 轨迹评测

【是什么】 不只看终点，还要看 Agent **走到终点的整条路径**——它依次做了哪些动作、调了哪些工具、顺序对不对。这是设计模式书讲得最系统的部分。

【为什么测】 一个 Agent 可能"蒙对了"答案，但过程一塌糊涂（多绕了 10 步、调了不该调的工具）；也可能答案略有瑕疵但过程完全正确。**只看结果会误判**，轨迹评测能看出 Agent 是不是"真的会"，也能定位它在哪一步出错。

【怎么测】

> 定性：这里其实是**两个独立方法**。方法一"轨迹比对"内部有 **5 个档位**（从最严到最松，按需求选）；方法二是实战书的"Judge-Verifier 回路"，另起一种思路。

- ▶ **方法一 • 把实际轨迹和标准轨迹做比对**，松紧分 5 档：
- **【档位•最严】精确匹配**：动作序列和标准轨迹完全一致；
- **【档位】按序匹配（in-order）**：关键动作按正确顺序出现即可，允许中间有别的；
- **【档位】任意顺序匹配（any-order）**：关键动作都出现就行，不计顺序；
- **【档位】精确率 / 召回率**：实际动作里有多少是该做的、该做的做了多少；
- **【档位•最细】单一工具使用**：单独看某个关键工具用得对不对。
- ▶ **方法二 • Judge-Verifier 回路**：实战书第 6 章用行为树搭了"Hacker→Judge→Verifier"三段式——Judge Agent 评判解答，Verifier Agent 在独立线程**真的把代码跑一遍**做二次验证。这是轨迹+结果双重把关的好例子。

> **【面试卡片】 Q：一个 Agent 最终答对了，但你为什么还要看它的"轨迹"？**
>
> **A：** 因为只看结果会误判。一个 Agent 可能"蒙对了"答案，但过程一塌糊涂——多绕了十步、调了不该调的工具、烧了不该烧的成本；也可能答案略有瑕疵但过程完全正确。轨迹评测看的是它走到终点的整条路径：依次做了哪些动作、调了哪些工具、顺序对不对。这样既能判断 Agent 是不是"真的会"、能不能稳定复现，也能精确定位它在哪一步出错。
>
> 做法是把实际轨迹和标准轨迹比对，按需求选松紧档——精确匹配、按序匹配、任意顺序匹配、精确率/召回率、单一工具使用；
>
> 也可以用 Judge-Verifier 回路，让一个 Agent 评判、另一个真的把代码跑一遍做二次验证，轨迹和结果双重把关。

11.3.4.3 多轮交互评测

【是什么】 评估 Agent 在**多轮对话**中的表现——用户会追问、改主意、说得含糊，Agent 能不能跟得住。这是龙猫几乎**独家**深入的领域。

【为什么测】 真实用户不会一次把需求说全。前面的组件评测、单轮评测都假设"一问一答"，但实际场景是"边聊边改"。如果不专门测多轮，上线后才会发现 Agent 一到真实对话就乱套。

【怎么测】

> 定性：这是一套完整流程，不是三种方法任选。核心是：**固定"环境 + 隐藏用户目标 + 评分 Rubric"，让用户模拟器动态生成多轮对话，被测 Agent 正常应对，最后用终态 + Rubric 判断任务是否完成。**

第一步：制作多轮评测数据集，但不是写死对话文本

多轮评测的样本不是"用户问一句、标准答案一句"，而是一个任务包。它通常包含三件固定内容：

```Plain Text { "environment": { "restaurant": "川味小馆", "menu": [ {"name": "麻婆豆腐", "price": 38, "stock": 5}, {"name": "水煮鱼", "price": 88, "stock": 2} ], "delivery_slots": ["18:30", "19:00", "19:30"] }, "user_goal_card": { "hidden_goal": "点一份麻婆豆腐", "budget": "不超过50元", "delivery_time": "19:00前送达", "dynamic_change": "中途把'微辣'改成'不辣'" }, "rubric": [ "最终商品必须是麻婆豆腐", "口味必须是不辣", "金额不能超过50元", "送达时间不能晚于19:00", "下单前必须向用户复述确认" ] }
```

> 这里要特别分清三份信息给谁看：
>
> - **环境 / 后台状态**：被测 Agent、用户模拟器、判分器都可用，作用是提供菜单、库存、价格、时间槽等客观世界。
> - **隐藏用户目标卡**：只给用户模拟器，作用是规定用户真正想完成什么、何时改主意。
> - **评分 Rubric**：只给判分器，作用是规定最后怎么判 Pass / Fail。
>
> 被测 Agent **看不到隐藏用户目标卡，也看不到评分 Rubric**。它只能像真实线上一样，根据用户模拟器一句一句说出来的话来追问、推荐、调用工具、确认和下单。
>
> **第二步：运行时由用户模拟器动态生成多轮对话**
>
> 用户模拟器拿着隐藏目标卡扮演真实用户。它不会把目标一次性说全，而是会含糊表达、追问、改主意。例如：
>
> ```Plain Text
> User Simulator：我想点个下饭的，别太贵。
> Tested Agent：可以看看麻婆豆腐，38 元。需要什么辣度？
> User Simulator：微辣吧。
> Tested Agent：配送时间选 18:30 还是 19:00？
> User Simulator：等下，改成不辣，19:00 前到。
> Tested Agent：确认一下：一份不辣麻婆豆腐，38 元，19:00 前送达，可以下单吗？
> User Simulator：可以。
> Tested Agent：已下单。
> ```

这段对话不是数据集里提前写死的，而是运行时生成的。同一个任务重复跑多次，用户模拟器可能换一种说法、换一个改口时机，这正好用来测试 Agent 的多轮鲁棒性。

第三步：对话结束后，用"终态 + Rubric"判分

判分不是拿整段对话去比一段"标准对话"，而是看最后状态和关键行为是否满足 Rubric：

- **结构化终态用代码判**：商品是不是麻婆豆腐、价格是否 ≤ 50、送达时间是否 ≤ 19:00；

- **语义行为用 LLM judge 判**：有没有在下单前复述确认、有没有正确处理用户中途改口、有没有擅自臆测。

最后可以像龙猫那样用 **Pass^4**：同一道任务跑 4 次，每次对话都可能不同，4 次都通过才说明 Agent 真稳定。

> 一句话记忆：多轮交互评测不是固定"标准对话"，而是固定**环境、隐藏目标和评分标准**；随机的是用户模拟器生成的对话路径；最终判的是**任务终态和关键行为**是否满足 Rubric。

11.3.4.4 多 Agent 协作评测

【是什么】 当系统由多个 Agent 组队完成任务时，评估这个"团队"协作得好不好。

【为什么测】 多 Agent 系统多出一层全新的失败面——它们之间会不会有效协作、有没有选对该干活的 Agent、加 Agent 到底是帮忙还是添乱。单 Agent 的指标测不出这些"团队动态"。

【怎么测】

> 定性：**"协作好不好"不要抽象地打一个总分，而要拆成三套可执行测法**：①对照标准流程看交接对不对；②做消融实验看每个 Agent 有没有真实贡献；③用质检 Agent / trace 抓过程病征。三套可以叠加使用。

先给一个具体例子。假设智能预约系统有 4 个 Agent：

```Plain Text Router Agent 负责判断用户意图，决定交给谁 Calendar Agent 负责查档期 Weather Agent 负责查天气 Booking Agent 负责最终预约下单

> 一条测试任务可以写成这样：
>
> ```Plain Text
> {
>   "task": "帮我约明天下午2点的精油按摩，如果明天下雨就改到后天",
>   "expected_handoffs": [
>     "Router -> Weather Agent(date=明天)",
>     "Weather Agent -> Router(result=rain)",
>     "Router -> Calendar Agent(date=后天, time=14:00)",
>     "Calendar Agent -> Booking Agent(slot=后天14:00)"
>   ],
>   "expected_final_state": {
>     "booking_date": "后天",
>     "booking_time": "14:00",
>     "service": "精油按摩"
>   }
> }
```

方法一：交接轨迹评测，看"团队配合有没有走对"

怎么实现：

1. **准备评测集**：每条 case 不只写最终答案，还要写 `expected_handoffs`，也就是谁应该把什么信息交给谁。
2. **跑完整多 Agent 系统**：记录每次 Agent 调用、输入、输出、交接对象，形成 trace。
3. **对照标准交接链判分**：逐项检查有没有选对 Agent、有没有传对关键字段、顺序是否合理、最终状态是否正确。

看哪些指标：

- **Agent 选择正确率**：该查天气时有没有交给 Weather Agent，而不是通用问答 Agent。
- **交接字段完整率**：日期、时间、服务项目、用户约束有没有传丢。
- **交接顺序正确率**：是否先查天气再改日期，是否在查档期前就贸然下单。
- **端到端成功率**：最终预约状态是否满足任务目标。

能定位什么问题： 如果最终约错了日期，trace 能告诉你到底是 Router 没调用 Weather Agent，还是 Weather Agent 查对了但 Router 没把"下雨→改后天"传给 Calendar Agent。

方法二：消融评测，看"多加一个 Agent 到底有没有用"

怎么实现： 固定同一批 evalset，跑几组对照实验：

- **Full Team**：Router + Weather + Calendar + Booking 全部启用，用来看多 Agent 的完整表现。
- **No Weather**：去掉 Weather Agent，天气逻辑由 Router 或主 Agent 自己处理，用来看天气 Agent 是否真的贡献准确性。
- **No Router**：去掉 Router，所有任务直接给主 Agent，用来看路由 Agent 是否减少误分发。
- **Single Agent**：一个大 Agent 全包，和多 Agent 架构做基线对比。

看哪些指标：

- **成功率差值**：`Full Team 成功率 - No Weather 成功率`，差值大说明 Weather Agent 有价值。
- **成本 / 延迟差值**：多 Agent 成功率提高了，但是否多花太多 token、调用太慢。
- **错误类型变化**：去掉某个 Agent 后，是工具错、计划错、交接错，还是最终状态错。
- **边际收益**：增加第 N 个 Agent 后，成功率是否继续升，还是开始下降。

能定位什么问题： 如果 Full Team 比 Single Agent 成功率只高 1%，但 token 成本翻倍，说明多 Agent 架构可能过度设计；如果 No Router 版本误调工具率大幅上升，说明 Router Agent 是必要组件。

方法三：过程监控 + 质检 Agent，看"协作过程中有没有病征"

怎么实现：

1. **给每次多 Agent 协作开 trace**：用 AgentOps 或类似 tracing 工具记录每个 Agent 的输入、输出、耗时、token、调用次数。
2. **配置质检 Agent / Reviewer Agent**：让它在每一步或每几步后检查当前状态，例如"是否已经满足用户约束？是否有 Agent 在重复搜索？是否应升级人工？"。

3. **把质检结果结构化输出**：例如 `handoff_ok / repeated_reasoning / missing_context / wrong_agent / should_stop`。

看哪些指标：

- **重复思考 / 重复调用次数**：同一个 Agent 反复查同一件事，说明团队在打转。
- **无效交接次数**：下游 Agent 收到的信息不够，必须追问上游。
- **冲突次数**：两个 Agent 对日期、价格、状态给出不一致结论。
- **人工兜底触发率**：质检 Agent 判定继续自动执行风险太高。
- **每个 Agent 的 token / latency 占比**：找出最贵、最慢、最容易拖垮团队的角色。

能定位什么问题： 如果 trace 显示 Calendar Agent 连续 5 次收到缺少 `duration` 的请求，问题不在 Calendar，而在上游 Router 没抽取完整槽位；如果 Reviewer 多次打出 `should_stop=false`，说明 Booking Agent 过早下单。

> 一句话记忆：多 Agent 评测不是"问裁判这个团队好不好"，而是 **测交接链是否正确、做消融看每个 Agent 是否有贡献、看 trace/质检 Agent 抓协作病征。**最终报告里要同时给出端到端成功率、交接正确率、Agent 选择正确率、成本延迟和主要失败类型。
>
> **【面试卡片】 Q：多 Agent 系统中，你应该如何评估多个 Agent 之间的协作?**
>
> **A：** 我不会只看最终任务有没有完成，因为多 Agent 系统的核心风险在"协作过程"：有没有交接错、有没有选错 Agent、有没有互相打转、加了 Agent 到底有没有收益。所以我会从三层来评估。
>
> 第一层是**交接轨迹评测。**我会为每条任务定义标准协作链，比如 `Router -> Weather -> Calendar -> Booking`，然后记录真实运行 trace，检查是否选对 Agent、是否传对关键字段、交接顺序是否合理、最终状态是否正确。这个方法主要回答：**团队配合有没有走对。**
>
> 第二层是**消融评测。**固定同一批 evalset，分别跑 Full Team、去掉某个 Agent、退化成 Single Agent 等版本，比较端到端成功率、成本、延迟和错误类型。如果加了 Weather Agent 后成功率提升很小但 token 翻倍，说明它可能不值得；如果去掉 Router 后误调工具率大幅上升，说明 Router 是必要组件。这个方法主要回答：**某个 Agent 有没有真实贡献。**
>
> 第三层是**过程监控 + 质检 Agent。**我会用 trace / AgentOps 记录每个 Agent 的输入、输出、耗时、token、调用次数，再让 Reviewer / Critic Agent 检查中间状态，抓重复调用、无效交接、状态冲突、过早下单、应该人工兜底却没兜底等问题。这个方法主要回答：**协作过程为什么慢、贵、乱，瓶颈在哪里。**
>
> 总结来说，多 Agent 协作评测 = **交接链是否正确 + 增加 Agent 是否有边际收益 + 协作过程是否健康。**最终报告里我会同时给出端到端成功率、交接正确率、Agent 选择正确率、消融收益、token/latency 成本，以及主要失败类型。

11.3.5 上线前收口:Pipeline 与发布闸门

走到这里，上两章 已经把"每个组件怎么测、整机怎么测"都摸清了。现在才是把它们**收口成一条可自动运行的评估流水线**的时候——这一步同时包含两件本就该一起做的事：**建评测集**和**搭 Pipeline。**把它们合成一

步，是因为评测集里每条样本的"期望输出 / 评分标准"，正是用前面各维度的评测指标标注出来的；指标先行，数据才有依据。这也对应《AI Engineering》"设计评估流程"一节的 Step 3——它的标题直接就是"定义评估方法**与数据**"。

【是什么】 把"用哪些指标、各用什么方法打分、用哪批数据喂、分数怎么映射到业务、达到什么阈值才能上线"一次性组装成一条**可自动运行、可重复回归、可卡发布**的评估流水线。

【为什么测】 零散地测几个指标、临时凑几条样本都没用。关键是要有一套**和业务目标对齐、数据与指标配套、能持续自动运行、能拦住高风险版本**的评估系统——此后每次改动都让它跑一遍，用数字说话。否则离线分数再高，也可能是太慢、太贵、太绕，或者安全风险没过线。

【怎么测 / 怎么建】（评测集与 Pipeline 一体组装）

> 定性：**一条流程、分 5 步**，严格按顺序（指标先定 → 据此标数据 → 绑业务 → 设发布闸门 → 回灌线上问题）。其中第 2 步内部还要区分两种**粒度档位**。

1. **【步骤】先锚定"测哪些组件、各用什么指标"**：把 §3.4 / §3.5 定下来的测法收拢成一张表（correctness 用功能正确性、factual consistency 用 LLM-judge、cost/latency 直接量、轨迹用按序匹配、安全用红队和护栏触发率……），并明确分层评估的粒度：per task（整个任务）/ per turn（单轮）/ per intermediate output（中间输出）。**指标先定，才知道样本要标什么。**
2. **【步骤】据此构建配套评测集**：为每条代表性输入（覆盖常见、边界、易错、"不该触发"的反例）按上一步的指标标注期望输出或评分 rubric，用 JSONL 等格式存成可批量喂入的数据集。区分两种**粒度档位**：

- **【档位•细】test file（测试文件）**：偏单元测试，单个 case 验证某个具体行为，开发调试期高频跑；
- **【档位•全】evalset file（评估集）**：偏集成测试，成套、覆盖多场景，用于回归。 3. **【步骤】把评估指标绑定到业务北极星指标**：比如"回答有用性分"要能关联到"用户留存 / 转化"，否则指标再漂亮也没意义。 4. **【步骤】设定上线前发布闸门**：明确多少分以上、多少成本以内、多少延迟以内、多少安全风险以内才算"可以上线"。 5. **【步骤】把线上问题回灌评测集**：生产中发现的新失败、新攻击、新成本异常，要变成新的离线 case，进入下一轮回归。

11.3.5.1 发布闸门卡什么

发布闸门不是一个"总分超过 80 就上线"的粗糙判断，而是一组并列门槛。任何一类踩红线，都不应该进入真实流量。具体分为五类闸门：

- **质量 / 成功率**：上线前用 evalset、Pass@k / Pass^k、rubric、功能正确性来测，典型阈值形态是端到端成功率、关键任务 Pass^k、事实一致性分。
- **轨迹 / 工具 / 规划**：上线前用轨迹比对、工具调用静态/执行检查、规划失败模式统计来测，典型阈值形态是错工具率、错参数率、反思错误率、最大步数。
- **成本 / 效率 / 延迟**：上线前用离线批跑、压测、token 统计、工具耗时统计来测，典型阈值形态是单任务 token 预算、P95/P99 延迟、TTFT/TPOT、平均步数。

- **安全 / 合规**：上线前用越狱、直接/间接提示注入、PII/敏感数据、输出毒性、误拒率来测，典型阈值形态是红队通过率、guardrail 漏拦率、false refusal rate、审计字段完整性。
- **业务可用性**：上线前用私有业务集、用户模拟器、灰度前试运行来测，典型阈值形态是有用性阈值、投诉风险、人工兜底触发率。

这里尤其要注意：**成本 / 效率 / 延迟、安全 / 合规都属于上线前闸门。**上线后当然还要继续监控，但那不是第一次测试，而是持续验证。

11.3.5.2 成本、效率、延迟

【是什么】 评估 Agent 完成任务"划不划算、快不快"——花了多少 token、多少钱、多少步、多长时间。

【为什么测】 一个准确率很高但要调用 50 次工具、烧掉几块钱、让用户等 30 秒的 Agent，在上线前就应该被拦住。质量必须和成本、延迟一起看。

【怎么测】

> 定性：不是步骤，而是**上线前发布闸门要量的几类指标维度。**这些指标上线后继续进入 §3.7.2 做真实流量监控。

- **【维度•效率】**：完成单个任务的平均步数、平均成本、单步耗时，并和基线（另一个 Agent 或人类）对比。
- **【维度•延迟】**：TTFT（首 token 时延）、TPOT（每 token 时延）、总响应时间，建议看 P95/P99 而不是只看平均值。
- **【维度•成本】**：输入 token、输出 token、工具调用次数、外部 API 费用；实战书的 `LLMInteractionMonitor` 可跟踪 token 用量来控成本。
- **【维度•发散约束】**：最大步数、重复动作、绕远路；龙猫直接把"步数过多"算作失败的一种，说明效率可以是正确性的一部分。

11.3.5.3 安全与合规

【是什么】 评估 Agent 会不会被攻击、会不会输出有害内容、是否符合合规要求。

【为什么测】 Agent 能调工具、能执行动作，一旦被攻破或失控，危害远大于普通聊天机器人。安全不是上线后观察一下，而是上线前红线。

【怎么测】

> 定性：**混合结构**——前两点是主动测试方法，第三点是要量的安全维度，第四点是治理实现。上线后这些指标继续进入 §3.7.2 监控。

- **▶ 方法一 • 攻击面测试**：越狱（jailbreaking）、提示注入（prompt injection，含直接/间接）、red teaming（红队主动攻击找漏洞）。

- ▶ **方法二 • 防御分层**：在 model / prompt / system 三个层面分别设防并验证；输入护栏防 PII 泄露和恶意提示，输出护栏抓无效 JSON、事实不一致、毒性、泄密等失败。
- **【维度】安全指标**：事实一致性、安全性、毒性（toxicity）、guardrail 漏拦率、guardrail 触发率、误拒率（false refusal rate）。
- **【实现】合规治理**：设计模式书提出从"Agent"升级到"高级承包商（Contractor）"——用正式"合约"明确可验证的交付物、范围、成本，把评估上升为可审计、可问责的治理机制。

> **【面试卡片】 Q：你会怎么评估 Agent 的安全性?**
>
> **A：** 安全不是上线后才观察，而是上线前的红线、上线后持续盯。上线前我会做三件事：一是攻击面测试——越狱、提示注入（含直接和间接）、红队主动找漏洞；二是防御分层，在 model / prompt / system 三层分别设防并验证，输入护栏防 PII 泄露和恶意提示、输出护栏抓无效 JSON、事实不一致、毒性、泄密；三是量化安全指标——事实一致性、毒性、guardrail 漏拦率/触发率、误拒率（false refusal rate），这些都进发布闸门，任一项踩红线就不让上。上线后把这些指标搬到真实流量持续监控：盯越狱/提示注入、工具越权、guardrail 触发率和漏拦率、PII 泄露、毒性和合规审计，并用合约式（Contractor）治理把可交付物、范围、审计记录固定下来保证可追责。要补一句：安全/合规是这几份资料里整体覆盖最薄弱的一环，真实项目往往还要额外引入专门的安全测试体系。
>
> 这一节是"评估驱动开发"真正落地的关键：**评测集、Pipeline 和发布闸门是同一步里组装出来的产物**，长在前面各维度的评测指标之上。组装完成后，它就成了贯穿后续每次迭代、直到上线回归的自动化标尺。

11.3.6 上线后监控

§3.6 已经把成本、延迟、效率、安全、合规纳入上线前发布闸门。Agent 一旦上线，评估并不结束，而是换了一种形态——**持续监控。**《AI Engineering》一句话点破：**"监控的目标和评估的目标是同一个。"** 区别只在于：上线前主要用准备好的数据、仿真和压测；上线后使用真实流量、用户反馈和系统遥测。

11.3.6.1 A/B 测试与灰度发布

【是什么】 把新版本 Agent 只放给一部分真实用户，对比新旧版本的线上表现，再决定是否全量。

【为什么测】 离线评测分数高，不代表真实用户买账。A/B 测试是用真实流量做的最终裁决。注意它和 3.1.2 的比较式评估不同：**A/B 是把不同版本分给不同用户群看线上业务指标**，比较式评估是同一批输出给评委比。

【怎么测】

> 定性：**一个方法、分 3 步**（切流量 → 比线上指标 → 设回滚线），按顺序执行。

- **【步骤】** 按用户/流量切分实验组与对照组，灰度放量（先 1% 再 10% 再全量）；
- **【步骤】** 对比线上业务指标（留存、转化、满意度）而非离线分数；
- **【步骤】** 设好回滚阈值，一旦新版指标掉了立即回退。

11.3.6.2 监控与可观测性

【是什么】 建立一套线上可观测性系统，对运行中的 Agent 做持续跟踪：它现在表现如何、有没有退化、有没有漂移、慢不慢、贵不贵、是否被攻击、能不能回溯每一步。

【为什么测】 上线前评测只能覆盖已知场景。真实流量会带来开发期没见过的输入、模型或数据分布变化、更长上下文、工具异常、成本放大和安全攻击。不持续盯着，等用户投诉时通常已经晚了。

【怎么测】

> 定性：这一节不是和"漂移 / 成本 / 安全"并列，而是它们的**统一观测底座**。先搭 logs / traces / metrics，再在这套数据上监控质量、漂移、异常、成本、延迟和安全。

1. **【底座】可观测性三件套**

- **Logs（日志）**：记录每次请求、输入摘要、输出状态、错误、guardrail 命中、拒答原因；
- **Traces（调用链追踪）**：记录一次 Agent 任务经过了哪些 Agent / 工具 / 模型调用，每一步耗时、token、成本和结果；
- **Metrics（指标）**：把关键数值按时间聚合，形成 dashboard 和告警。 2. **【监控对象一】质量与行为**
- 格式错误率：JSON 解析失败、schema 不合规；
- 事实一致性、生成质量：可用 AI judge 抽样评估；
- 拒答率、误拒率、guardrail 触发率；
- 工具调用成功率、错工具率、异常工具路径。 3. **【监控对象二】漂移与异常**
- **输入分布漂移**：用户问法、主题、query embedding 分布是否偏离历史基线；
- **输出质量漂移**：事实一致性、满意度、拒答率是否持续变差；
- **行为异常**：工具调用次数突然翻倍、某类任务失败率突增、Agent 出现循环；
- **处置闭环**：发现漂移后触发重新评估 / 重新训练 / 回滚，并把线上样本回灌 §3.6 的评测集。 4. **【监控对象三】成本 / 效率 / 延迟**
- **延迟**：TTFT、TPOT、总延迟、P95/P99，按用户、场景、模型版本拆分；
- **成本**：输入/输出 token、每任务成本、每用户成本、工具/API 调用费用、缓存命中率；
- **效率**：平均步数、最大步数、重复工具调用、工具循环、单动作耗时；
- **处置**：超过预算时触发降级、模型路由切换、上下文裁剪、缓存策略、回滚或人工兜底。 5. **【监控对象四】安全 / 合规**
- **攻击监控**：越狱、直接/间接提示注入、异常查询、工具越权、远程代码执行风险；
- **护栏监控**：guardrail 触发率、漏拦率、拒答率、误拒率、人工升级率；
- **内容与数据风险**：毒性、有害内容、事实不一致、PII / 敏感信息输入输出、品牌与合规风险；
- **治理审计**：Contractor / 合约式治理，把可交付物、范围、成本、审计记录固定下来，保证线上行为可追责。 6. **【响应指标与工具】**

- **MTTD / MTTR / CFR**：平均检测时长、平均修复时长、变更失败率，用来衡量团队发现和修复问题的速度；
- **落地工具**：指标落到时序数据库（InfluxDB / Prometheus），接 Datadog / Grafana 等可观测性平台；Agent 专用的有 AgentOps，适合看交互 trace、token 和成本。

> ⚠️ **公共短板提示**：安全/合规是四份资料里整体覆盖最薄弱的一环（龙猫几乎不涉及）。真实项目里这块往往需要额外引入专门的安全测试体系，不能只靠通用评测资料。
>
> 💬 **【面试卡片】 Q：你的项目上线以后，你如何保证和监测上线质量?**
>
> **A：** 我会把上线后的质量保障分成两层：**灰度发布控制风险，可观测性持续监控质量。**上线不是评测结束，而是把上线前定义好的质量、成本、延迟、安全指标搬到真实流量里持续验证。
>
> 第一，**上线时做 A/B 测试和灰度发布。**新版本不会直接全量，而是先放 1% 或小流量，对比旧版本的真实业务指标，比如任务成功率、用户满意度、转化率、投诉率，同时设置回滚线。如果关键指标下降、错误率升高或安全告警增加，就立即暂停放量或回滚。
>
> 第二，**上线后建立 logs / traces / metrics 三件套。**Logs 记录请求、错误、拒答和护栏命中；traces 记录一次 Agent 任务经过哪些模型、工具和子 Agent，每一步耗时、token、成本和结果；metrics 把关键指标聚合成 dashboard 和告警。这样既能看总体趋势，也能回溯单次失败到底卡在哪一步。
>
> 第三，**持续监控四类核心指标**：质量指标看格式错误率、事实一致性、生成质量、拒答率、工具调用成功率；漂移与异常看用户输入分布、输出质量、异常工具路径和 Agent 循环；成本延迟看 TTFT、TPOT、P95/P99、token、平均步数和工具耗时；安全合规看越狱/提示注入、guardrail 触发率、误拒率、PII 泄露和审计记录。
>
> 第四，**把监控变成响应闭环。**我会设置 MTTD、MTTR、CFR 这类响应指标，保证问题能及时发现、定位和修复。线上发现的新失败、新攻击、新高成本 case，会回灌到离线评测集，重新跑 §3.6 的发布闸门，避免同类问题下次再上线。
>
> 总结来说，我保证线上质量的方式是：**灰度发布控风险，logs/traces/metrics 看得见，质量/漂移/成本/安全指标持续告警，线上问题回灌评测集形成闭环。**

第 3 章小结：到这里，我们已经把生命周期主线上的每一环都讲完了——从通用武器（LLM-judge、比较评估、基础度量），到上线前验证（选型 → 成功标准 → 组件 → 集成 → 评测集 / Pipeline / 发布闸门），再到上线后监控（A/B 灰度发布 → 监控与可观测性覆盖质量、漂移、成本、延迟和安全）。**下一章用美团龙猫这个真实项目，看这些思想是怎么被打包成一套完整评估系统落地的。**

11.4 实战

11.4.1 实战一：美团 LongCat VitaBench

【背景】 现有的 Agent benchmark 大多是"玩具任务"，和真实生活差距很大，区分不出顶尖模型到底差在哪。VitaBench 的目标是：**用贴近真实生活的复杂场景，量化现有 Agent 与实际应用需求之间的差距。**

【场景】 聚焦美团最熟悉的三大生活服务场景：**外卖点餐、餐厅就餐、旅游出行。**

【为什么难】 这些场景天然具备真实世界的复杂度：一个任务可能涉及 5–20 个服务商、100 多个候选商品，用户还会中途改主意。VitaBench 号称是"迄今最复杂的生活服务模拟环境"。

11.4.1.1 整体思想

这是本章的核心。下面把**VitaBench 的每个设计，对应回第 3 章的哪个评测思想**一一列出——你会看到它几乎是第 3 章的一次"综合落地演练"：

- **3.4.4 规划/推理评测** → VitaBench 用**推理复杂度**维度落地：观测空间大小、部分可观测程度、推理点数量。
- **3.4.3 工具调用评测** → VitaBench 用**工具复杂度**落地：66 个真实工具，跨场景含 512 条依赖边的工具图。
- **3.5.3 多轮交互评测** → VitaBench 用**交互复杂度**加 **GPT-4.1 用户模拟器**落地，模拟用户改主意、模糊表达。
- **3.1.1 LLM-as-a-judge** → VitaBench 用 **Rubric 滑动窗口评估器**落地：用 Claude-3.7-Sonnet 当裁判，κ=0.828。
- **3.5.1 任务结果评测** → VitaBench 用 **Avg@4 / Pass@4 / Pass^4** 落地（每任务跑 4 次，温度 0）。

几个特别值得讲的设计亮点：

1. **三维复杂度框架**：VitaBench 把 Agent–用户–工具的交互建模成 POMDP（部分可观测马尔可夫决策过程），拆成**推理 / 工具 / 交互**三个可独立调节的复杂度维度。这是它最大的学术贡献——**第一次让任务复杂度变得"可控、可量化"。**

1. **Rubric 滑动窗口评估器**：把任务目标拆成一个个原子化的判定准则，用"全有或全无"方式严格判定，和人类评判的一致性高达 Cohen's κ=0.828——说明这套自动评估很可信。这正是 3.1.1 LLM-as-a-judge 的高质量工程实现。
2. **故意去掉"规则文档"**：传统 benchmark 会给 Agent 一份 policy 文档说明规则，VitaBench 反其道而行——**把领域规则编码进工具依赖图里，逼模型自己推理**，而不是照着说明书抄。这让评测更接近真实（现实中没人给你完整说明书）。
3. **真实工具图**：66 个真实工具之间有 512 条依赖边（调用 A 工具前必须先调 B），构成一张复杂的依赖网络。

11.4.1.2 结果与启示

【结果很扎心】 即便是当时最强的模型 o3，在 VitaBench 上：

- 跨场景 **Avg@4 仅 30%**，单场景 < 50%；
- **Pass^4 ≈ 0%**（连续 4 次全做对的概率几乎为零，说明极不稳定）；
- 失败主因是**推理错误（占 61.8%）**。

【启示】

1. **顶尖 Agent 离"真实可用"还很远**——在玩具 benchmark 上的高分有水分，一到真实复杂场景就现原形。
2. **推理是当前最大瓶颈**（61.8% 失败源于推理），印证了 3.4.4 规划/推理评测的重要性。
3. **稳定性比单次成功更难**——Pass^4≈0% 说明 Agent 偶尔做对容易、稳定做对极难，这也是为什么龙猫要用 Pass^k 这种严格指标。

> **这个实战案例的最大借鉴**：当你要为自己的业务（尤其是有真实工具、多轮交互的复杂场景）做评估时，VitaBench 提供了一套可抄的范式——**用三维复杂度刻画任务难度、用真实工具图考工具调用、用用户模拟器考多轮、用 Rubric+LLM-judge 严格判分、用 Pass^k 考稳定性。**
>
> **【面试卡片】 Q：介绍一个你了解的 Agent 评测 benchmark，它是怎么设计的？（或者问题可以是介绍一个你看过的论文，你就说你看了美团龙猫讲Agent评测的论文）**
>
> **A：** 我了解美团 LongCat 的 VitaBench。它聚焦外卖点餐、餐厅就餐、旅游出行三大真实生活服务场景，目标是量化现有 Agent 和真实应用需求之间的差距。设计上它几乎把一整套评测思想综合落地：
>
> ① 把 Agent–用户–工具交互建模成 POMDP，拆成推理 / 工具 / 交互三个可独立调节、可量化的复杂度维度——这是它最大的贡献；
>
> ② 用 66 个真实工具加 512 条依赖边的工具图考工具调用；
>
> ③ 用 GPT-4.1 用户模拟器考多轮交互，模拟用户改主意、模糊表达；
>
> ④ 用 Rubric 滑动窗口评估器（Claude-3.7-Sonnet 当裁判、和人类一致性 κ=0.828）严格判分；
>
> ⑤ 用 Avg@4 / Pass@4 / Pass^4 考稳定性。它还故意去掉规则文档、把规则编码进工具依赖图，逼模型自己推理。结果很扎心：最强的 o3 跨场景 Avg@4 只有 30%、Pass^4 几乎为 0%，61.8% 的失败源于推理——说明顶尖 Agent 离真实可用还很远、推理是最大瓶颈、稳定做对比偶尔做对难得多。

11.4.2 实战二：算法项目二阶段

我们的大模型项目，二阶段完成的内容就是一个Agent评估的东西。

即使你不做算法，也可以去看一下这块对应的讲解视频，跟着去做一下，学习Agent评估的实战。

具体细节 大模型算法项目学习方法中，二阶段的介绍。

11.5 面试问题汇总

> 本章汇总全文出现过的面试问题。**每道题的问法都与前文对应小节的面试卡片完全一致**，答案可直接回到该卡片查看。

概念理解类

1. **TDD 你了解过吗？你自己用 TDD 做过完整开发吗？** （见3.1.1 面试卡片）
2. **你觉得 Agent 行业的测试和传统行业有哪些不同？** （见 3.1.2面试卡片）
3. **Agent 行业的测试面临着哪些困难？** （见 3.1.3 面试卡片）
4. **一个 Agent 最终答对了，但你为什么还要看它的"轨迹"？** （见 3.3.4.3 面试卡片）

方法与指标类

1. **在 Agent 评测中用 LLM-as-a-judge（大模型当裁判）很火，请客观评价一下它的优缺点？如果由你负责实际项目落地，应该怎么科学避坑？** （见 3.3.1.1 面试卡片）
2. **评估一个 RAG 系统，你会从哪些维度入手？用什么指标？** （见 3.3.3.2 面试卡片）
3. **怎么评估 Agent 的工具调用能力？它有哪几个层次？** （见 3.3.3.3 面试卡片）
4. **功能正确性、相似度、LLM-judge 三类指标怎么选？** （见 3.3.1.3 面试卡片）

工程实践类

1. **你认为在 Agent 系统中，应该测试/评估哪些东西？** （见 3.2.2 章面试卡片）
2. **你是如何确定你的模型的？** （见 3.3.2 面试卡片）
3. **多 Agent 系统中，你应该如何评估多个 Agent 之间的协作？** （见 3.3.4.4 面试卡片）
4. **你的项目上线以后，你如何保证和监测上线质量？** （见 3.3.6 面试卡片）
5. **你会怎么评估 Agent 的安全性？** （见 3.5.3 面试卡片）

案例分析类

1. **介绍一个你了解的 Agent 评测 benchmark，它是怎么设计的？** （见 3.4 面试卡片）
2. **你的项目是如何做Agent评估的？**

（推荐思路，要么你就按照今天的理论，做一遍Agent评测，然后按照你的做法适度包装。要么你可以把今天的笔记输入给AI，然后把你的项目也让AI可见。让AI结合你的项目和这个Agent评估思路包装一套方法，然后自己多想想细节，修改润色下，提前准备好相关问题细节，和面试官说）

16. 评估一个 AI Agent，除了准确率还看哪些指标？

💡 **思路：** 关于Agent评测我们专门讲过，这个题就是纯八股。建议看完专题笔记，理解了思路后，然后再掌握/背诵这个标准答案。

📝 **答案：**

除了准确率，我还会关注**任务成功率、稳定性、工具调用和执行轨迹、鲁棒性、延迟、成本、安全合规以及线上业务指标。**

任务成功率主要看 Agent 最终有没有真正完成用户目标，而不只是回答内容是否正确。由于模型具有随机性，还要把同一个任务重复运行多次，通过 **Pass@k** 或 **Pass^k** 观察它能否稳定完成，而不是偶然成功。

工具调用方面，要看**工具选择是否正确、必填参数是否完整、参数值是否准确**，以及是否存在不该调用工具却乱调用的情况。**执行轨迹**方面，则要看**规划步骤和工具调用顺序是否合理**，有没有遗漏关键步骤、违反用户约

束、重复调用或者陷入循环。

鲁棒性主要看 Agent 面对用户改口、信息缺失、模糊表达、长上下文和工具异常时，能否正确追问、调整计划或进行降级。效率方面会统计**平均执行步数**、**工具调用**次数和**重复动作率**；延迟方面关注 **TTFT**、总响应时间以及 **P95**、**P99**；成本方面关注单任务 **Token 消耗**和外部 API 调用费用。

安全合规方面，要评估**提示注入**、**越狱**、**工具越权**、**敏感信息泄露**和有害内容等风险，同时监控**护栏漏拦率**和**误拒率**。Agent 上线以后，还要看**用户满意度、转化率、投诉率和人工兜底率**等真实业务指标。

如果是多 Agent 系统，还需要看 **Agent 选择正确率、交接字段完整率、协作冲突率**，以及增加某个 Agent 后带来的成功率收益是否值得额外的成本和延迟。

总体来说，除了准确率，我会同时评估 Agent 的**成功率、稳定性、执行过程、鲁棒性、效率、延迟、成本、安全合规和业务效果。**

选自202607 宇树科技Agent开发一面

11.6 视频讲解

https://www.bilibili.com/video/BV1QMTg6AE52/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

12 Agent健壮性设计

12.1 前言

如何设计一个 **高可用、发生故障后能够优雅回退的 Agent 系统**，是当前 Agent 岗位面试中比较受关注的知识点。它也是区分普通演示型项目与企业级、生产级项目的重要设计能力：一个项目不仅要能在理想条件下完成任务，还要考虑模型、工具和运行流程发生异常时，系统如何恢复、隔离、降级或安全停止。

我们在“**2026 年面试官最喜欢的 6 个 AI 项目**”中曾经讨论过这一点。最近整理的真实面试问题也进一步说明了它的重要性：

> **英伟达 Agent 一面：** Agent 遇到异常故障时，用什么方式保持持续健壮的运行？（我们后面会收录这期面经，做精讲）
>
> **宇树科技 Agent 一面：** 外部工具调用失败后，系统如何容错或降级？

这两道问题虽然切入角度不同，但都在考察同一个核心能力： **当 Agent 无法按照原计划继续执行时，系统能否识别故障、采取正确的恢复策略，并在无法恢复时进行优雅降级。** 由此可以看出，Agent 健壮性设计既是一个高频面试知识点，也是生产级 Agent 项目不可缺少的工程能力。

因此，本文将其整理为一个独立专题—— **Agent 系统健壮性设计**，系统讲清楚 Agent 开发中可能遇到的故障，以及面对这些故障时应当如何安全、可控地处理。无论是准备面试，还是设计和完善自己的 Agent 项目，这部分理论与实践方法都非常实用。

本文的内容安排： 全文将围绕 Agent 开发与运行过程中的 **外部工具与依赖故障、模型服务故障，以及 Agent 推理与控制流故障** 展开，并按照 “ **故障表现** → **解决策略** → **完整处理流程** ” 进行讲解，说明什么时候应该重试、什么时候需要修正或重新规划、什么时候应当熔断和降级，以及什么时候必须转人工或停止执行。文中还会穿插相应的 **面试问题、答题思路与参考答案**，帮助大家快速检验理解程度，并掌握如何在面试中有条理地表达一套完整、可落地的生产级方案。

12.2 调用工具失败

Agent 的主要价值来自调用搜索、数据库、代码执行、支付、工单、邮件和企业内部 API 等工具，因此工具层也是最常见的故障来源。这个几乎是在设计高可用Agent系统时，最常问的问题了。

12.2.1 瞬时技术故障：设置明确的超时、总预算和有界重试

故障表现： 瞬时技术故障一般包括 **网络连接中断、请求超时、HTTP 429 限流、部分 HTTP 5xx、MCP Server 或插件短暂不可用**，以及下游服务临时过载。这类故障的特点是过一段时间后可能自行恢复，因此允许重试，但不能让单个依赖无限阻塞整个 Agent。

解决策略： 为每次调用设置 **连接超时和响应超时**，同时限制工具调用阶段及整个 Agent 任务的 **总时间、总成本和最大重试次数**。对于明确可重试的错误，采用指数退避和随机抖动，遵守服务端的 `Retry-After`，并在达到次数或时间预算后立即停止，避免无限重试演变为重试风暴和级联故障。

> ❓ **你可能会问：什么是指数退避和随机抖动?**
>
> 💡 这是软件开发，尤其是分布式系统中非常经典的重试策略。 **指数退避** 是指每次重试失败后逐渐延长等待时间，例如依次等待 1 秒、2 秒、4 秒和 8 秒，避免在服务尚未恢复时频繁请求； **随机抖动** 则是在等待时间上加入一定的随机变化，把不同客户端的重试时刻错开，避免大量请求在同一时间再次涌入下游服务。

12.2.2 确定性调用错误：返回结构化错误并修正调用

故障表现： 确定性调用错误一般包括 **参数缺失、参数类型错误、工具名称或版本不匹配、Schema 校验失败、身份认证过期、权限不足**，以及请求违反业务规则。这类问题由请求本身或执行条件决定，通常不会随着时间自动消失，因此 **原样重试没有意义**。

解决策略： 由工具执行器返回 **错误类型、是否可重试、失败字段和具体原因** 等结构化信息，让 Agent 据此修正参数、补充前置条件、重新授权、替换工具或调整计划，而不是机械地重复同一个请求。例如可以返回：

```
{"error_type": "INVALID_ARGUMENT",
"retryable": false,
```

```
"message": "end_time must be later than start_time",
"failed_fields": ["start_time", "end_time"]}
```

12.2.3 瞬时故障多次重试仍失败：使用熔断器隔离故障

故障表现： 这一阶段主要承接第一类瞬时故障：工具经过有限次数的退避重试后，仍然 **连续超时、返回 5xx 或处于不可用状态**，说明问题可能不再是短暂抖动，而是依赖正在持续故障。第二类参数错误、权限不足等确定性错误通常不会进入这里，因为它们应直接修正、重新授权或终止调用，而不是反复重试到熔断。

解决策略： 使用 **熔断器** 隔离故障：正常情况下处于 Closed 状态；连续失败达到阈值后进入 Open 状态，暂时拒绝新调用；等待冷却时间后进入 Half-Open 状态，只放行少量探测请求，确认依赖恢复后再关闭熔断。**熔断的目的不是修复工具，而是保护 Agent 和下游系统。**

12.2.4 工具长时间不可用：按照能力等级逐级降级

故障表现： 工具长时间不可用意味着自动恢复已经超过重试或熔断预算，但用户的任务可能仍有一部分可以完成。此时如果继续等待会浪费资源，如果直接宣告整个任务失败又可能损失仍可提供的核心能力，因此需要根据操作风险和业务目标判断可接受的能力边界。

解决策略： 依次尝试 **同能力的备用工具、缓存或已有中间结果**，并在无法完整完成时返回部分结果且明确缺失项。对于高风险操作，可以从写入降级为只读查询，从自动执行降级为提供建议；仍无法安全完成时保存当前状态并转人工，没有安全降级路径时则明确失败， **绝不能伪造成功结果。**

12.2.5 完整处理流程

上面的四条可以统一成一套完整的工具调用失败处理流程。按照这个思路回答，既能体现清晰的故障分类能力，也能体现对重试、熔断和优雅降级等高可用设计的理解。这套方案也可以直接用于实际项目中的 Agent 工具调用层设计。

整体处理思路可以分为四步：

1. **错误分类：** 先读取结构化错误，区分确定性错误和瞬时技术故障，不能对所有失败统一重试。
2. **局部恢复：** 确定性错误通过修正参数、重新授权或终止调用处理；瞬时故障则在超时和总预算内进行有界重试。
3. **故障隔离：** 瞬时故障多次重试仍未恢复时，打开熔断器，避免持续冲击故障依赖。
4. **优雅降级：** 工具确实不可用时，切换备用工具、使用缓存或已有结果、返回部分结果，必要时降级为只读建议或转人工；没有安全降级路径时明确失败。

假设 Agent 调用搜索工具失败，系统首先读取结构化错误并进行分类：

1. 如果是参数格式错误、权限不足等 **确定性错误**，就进入第二条，修正参数、重新授权或直接终止，不能原样重试；

2. 如果是连接超时、HTTP 429 或 HTTP 503 等 **瞬时故障**，就进入第一条，在总预算内进行有限次数的指数退避和随机抖动重试；
3. 如果瞬时故障经过多次重试仍未恢复，就进入第三条，打开熔断器，停止继续冲击故障依赖；
4. 如果工具在熔断后仍长时间不可用，就进入第四条，切换备用工具、使用缓存或返回部分结果；
5. 如果不存在安全的降级路径，则明确报告工具不可用，保存必要状态并结束调用。

因此，完整逻辑可以概括为： **先分类；确定性错误直接修正或终止；瞬时故障先受控重试，持续失败后再熔断和降级。** 这比简单回答“失败后重试三次”更能体现生产系统设计能力。

> 🎯 **面试卡片**
>
> **面试问题：外部工具调用失败后，Agent 应该如何进行容错和降级?**
>
> 🧭 **答题思路与技巧：** 回答时不要把所有工具故障一概而论，也不要只说“失败后重试”。应当用清晰的分类和递进关系，体现回答的全面性与工程思维。
>
> 💬 **参考答案：**
>
> 外部工具调用失败后，我不会直接统一重试，而是先根据结构化错误进行分类。
>
> 1. 对于 **参数错误、权限不足等确定性错误**，原样重试没有意义，我会修正参数、重新授权，无法修复时直接终止调用并说明原因。
> 2. 对于 **网络超时、HTTP 429 和部分 5xx 等瞬时故障**，我会设置单次超时和任务总预算，并采用指数退避与随机抖动进行有限重试。
> 3. 如果瞬时故障 **多次重试仍未恢复**，我会打开熔断器，停止继续冲击下游，并通过半开状态探测服务是否恢复。
> 4. 如果工具 **长时间不可用**，我会根据业务风险切换备用工具、使用缓存或已有结果、返回部分结果，或者从自动执行降级为只读建议。仍然无法安全完成时，就保存必要状态、明确说明失败并转人工处理。
>
> 一句话概括就是： **先分类，能恢复就受控恢复；不能恢复就熔断；确实不可用就优雅降级。**

12.3 模型调用失败

模型服务故障和前面的外部工具调用故障有一定相似性：两者的总体目标都是 **在安全和预算允许的范围内尽可能恢复任务**，能够重试时受控重试，持续不可用时切换备用能力，无法完整恢复时再进行降级。但这一节把故障对象进一步聚焦到了 Agent 最核心的一项外部依赖—— **大模型 API**，因此除了通用的超时、限流和服务不可用，还需要讨论一些与 LLM 特性直接相关的问题。

其中最关键的区别是： **模型 API 调用成功，不代表模型在业务上正常。** 接口可能返回 HTTP 200，但模型由于概率性、版本变化或能力差异，仍然可能生成错误的工具参数、违反业务规则或遗漏用户要求。因此，模型服务故障不能只从 API 层判断，还要引入 **业务逻辑评估**，持续验证模型能否正确完成真实任务；主模型无法恢复时，还需要准备并切换 **经过业务验证的备用模型**，再根据备用模型的实际能力决定是否维持原权限或进一步降级。

12.3.1 模型服务短暂抖动：在任务预算内进行有限重试

故障表现： 模型服务短暂抖动通常表现为 **请求超时、HTTP 429 限流、部分 HTTP 5xx、流式响应中断或单个推理端点暂时不可达。**这类故障可能随着负载下降或服务恢复而消失，但一次模型调用往往同时消耗时间、Token 和费用，失败后的重复生成还可能产生不同结果，因此不能无限重试。

解决策略： 先为单次请求设置超时，只对明确可恢复的错误进行**指数退避和随机抖动重试**，并同时限制 **最大调用次数、总耗时、总 Token 和总费用。**如果任务中已经完成了写操作，重试模型时必须从持久化状态继续，不能重新执行已经成功的副作用操作；达到任一预算后则停止局部重试，进入故障切换流程。

12.3.2 接口可用但模型能力退化：持续监控真实任务质量

故障表现： 这是一类更隐蔽的模型服务故障：接口仍然能够返回结果，但由于 **模型版本更新、服务端策略变化、模型漂移或推理配置变化**，Agent 开始频繁生成错误参数、遗漏用户要求、违反领域规则，或者对同一个任务表现出明显的不一致。`τ-bench` 的实验也说明，只看单次平均成功率并不足以证明 Agent 可靠；即使较强模型的单次表现较好，同一任务连续多次都成功的概率仍会快速下降。

解决策略： 不能只监控可用率和响应时间，还要持续监控 **结构化输出通过率、工具参数正确率、领域规则违反率、任务完成率和重复运行一致性。**对于可以确定验证的任务，应将模型结果与真实业务状态、规则校验器或已知正确结果进行比较；对于高风险结果，则引入独立验证器或人工复核。当这些质量指标连续低于基线或风险阈值时，应将其视为服务事故，停止让该模型执行高风险写操作，而不是因为接口返回成功就继续放行。

> ❓ **你可能会问：为什么不能只看模型接口的成功率?**
>
> 💡 因为模型服务具有概率性。传统 API 返回成功，通常意味着完成了一个定义明确的操作；模型接口返回成功，只能说明生成过程完成了，并不能证明生成结果正确。生产环境因此需要同时监控 **技术可用性** 和 **业务有效性**：前者关注请求是否成功，后者关注输出能否稳定完成真实任务。

12.3.3 主模型持续不可用：切换预先验证的备用模型

故障表现： 当主模型经过有限重试后仍然持续超时、长时间限流或整体不可用时，说明故障已经超过局部重试能够解决的范围。如果 Agent 只能调用这一个模型，整个任务就会随着主模型故障而中断。

解决策略： 提前准备一个或多个 **备用模型**，并在上线前使用同一套真实业务任务验证其工具调用、输出格式、领域规则和运行稳定性。当主模型持续不可用时，由模型网关将请求切换到已经达到任务最低标准的备用模型，使任务能够继续运行。第三点的核心就是： **主模型故障后不能没有替代能力，但替代模型必须事先准备和验证。**

12.3.4 备用模型能力低于主模型：按照任务风险降低权限

故障表现： 切换成功只说明备用模型可以提供服务，并不代表它在 **函数调用、上下文长度、复杂推理、规则遵循和安全控制** 等方面与主模型完全相同。如果让能力较弱的备用模型继续执行付款、退款、删除数据等高风险动作，就可能把一次可见的服务中断转化为更难发现的业务事故。

解决策略： 根据备用模型的实际能力和任务风险同步调整 Agent 权限：能够满足要求的低风险任务可以继续自动执行；无**法稳定完成的中风险任务降级为只读查询、草稿生成或执行前人工审批**；付款、退款等高风险任务则转入人工处理。如果备用模型连最低质量和安全标准都无法满足，就保存任务状态、向用户说明原因并停止自动执行。第四点的核心就是： **启用备用模型不等于保持原有权限，备用能力下降时，Agent 的自动化权限也必须随之下降。**

12.3.5 完整流程

上面的四条可以统一成一套完整的模型服务故障处理流程。与工具故障类似，它同样需要完成故障识别、局部恢复、故障切换和优雅降级；但模型服务多了一项关键判断： **接口恢复之后，还必须验证模型能力是否恢复。**

整体处理思路可以分为四步：

1. **故障识别：** 同时监控错误率、延迟和限流等技术指标，以及任务成功率、规则违反率和重复运行一致性等质量指标，识别“接口不可用”和“接口可用但模型已经退化”两类事故。
2. **局部恢复：** 对短暂抖动在时间、Token 和费用预算内进行有限重试；恢复后通过代表性探测任务确认模型不仅能够响应，而且输出质量重新达到基线。
3. **备用模型切换：** 主模型持续不可用时，将请求切换到事先通过真实业务任务验证的备用模型，使任务能够继续运行。
4. **风险降级：** 如果备用模型只能覆盖部分能力，就同步降低 Agent 的操作权限，必要时转人工或停止自动执行。

假设一个售后 Agent 需要根据公司规则判断是否允许退款，并调用工具执行退款。系统可以按照下面的顺序处理：

1. 如果主模型偶尔出现超时或 HTTP 429，就在总预算内执行有限次数的退避重试；
2. 如果接口请求都成功，但近期模型频繁违反退款规则、生成不存在的订单号，系统就不能把它视为正常服务，而应根据业务质量指标触发告警并停止自动退款；
3. 如果主模型持续不可用或质量无法恢复，就切换到已经使用退款任务集验证过的备用模型；
4. 如果备用模型能够正确查询订单，却无法稳定完成退款判断，就只允许它查询信息和生成处理草稿，最终退款必须由人工确认；
5. 如果没有模型能够满足最低安全标准，则保存订单和任务进度，明确告知用户当前只能转人工处理，不能为了维持表面可用而继续自动退款。

因此，完整逻辑可以概括为： **先同时判断技术可用性和业务有效性；短暂故障先受控恢复，持续故障再切换经过验证的备用模型；备用能力不足时降低权限，无法满足安全标准时转人工或停用。** 这也对应了参考资料提出的持续监控、预先验证回退方案、设置停用条件、保留人工处理路径和事故复盘机制。

> 🎯 **面试卡片**
>
> **面试问题：模型服务发生故障时，Agent 应该如何保证任务持续、健壮地运行?**
>
> 🧭 **答题思路与技巧：** 可以先沿用工具故障的基本思路，说明对超时、限流等瞬时问题进行受控重试，再重点讲出下面两个更有区分度的观点：
>
> 第一， **不能只从 API 层判断模型调用是否失败。** 即使接口返回成功，也要通过任务成功率、参数正确率、规则违反率和最终业务状态等指标，判断模型是否真正完成了任务。
>
> 第二， **切换备用模型不代表万事大吉。** 备用模型同样存在不确定性，而且能力可能弱于主模型，因此切换后仍要继续进行业务逻辑验证，并根据验证结果决定正常执行、降低权限还是转人工。
>
> 面试时如果还能结合自己的业务场景举例，例如退款 Agent 中“接口返回成功但违反退款规则”，或者“备用模型只能查询订单、不能自动退款”，会让方案更具体，也更能体现生产级 Agent 的工程经验。
>
> 💬 **参考答案：**
>
> 模型服务发生故障后，我会同时从技术可用性和业务有效性两个层面判断，因为模型接口返回成功，并不代表模型仍能正确完成任务。
>
> 1. 对于 **超时、HTTP 429 和部分 5xx 等短暂故障**，我会设置单次超时和任务总预算，通过退避策略进行有限重试，避免时间、Token 和费用失控。
> 2. 除了错误率和延迟，我还会持续监控 **任务成功率、参数正确率、规则违反率和多次运行一致性**，及时发现接口正常但模型能力已经退化的情况。高风险结果必须经过规则校验、独立验证或人工复核。
> 3. 如果主模型持续不可用或质量低于阈值，我会切换到预先准备并用真实业务任务验证过的备用模型。不能因为备用模型能够返回文本，就默认它具有相同的工具调用和规则遵循能力。
> 4. 如果备用模型只能满足部分要求，我会同步降低 Agent 权限，例如从自动写入降级为只读查询、生成草稿或人工审批；如果没有方案满足最低安全标准，就保存任务状态、说明原因并转人工或停用系统。
>
> 一句话概括就是： **既监控模型能不能响应，也验证模型能不能正确完成任务；故障时先受控恢复，再切换经过验证的备用能力，能力不足就降权或转人工。**

12.4 讲解视频

https://www.bilibili.com/video/BV1MquZ6iE85/?vd_source=144bec9c3f54e465073138bed788be1b#reply313212531872

13 Agent项目上线问题

如果你的项目上线了，或者你想要把你的项目包装成已经上线了，那么这个小节是你需要参考的。我的个人建议是：**校招以及工作年限比较浅的同学，不必要一定把你的项目包装成上线了！** 甚至我工作五年面试的时候，不敢太说自己项目上线了。 原因很简单，就是当你项目上线以后，要考虑很多问题，面试官都会追问，如果你自己没上线，你又包装不明白，**说的露馅了，那你不如不说。** 上线当然是加分项，但是面试官可能对这件事，特别是经验不丰富的同学，工作5年以内，要求没那么高，所以你把我不住，那么你就说上线了。

如果你要包装成上线，我这一小节就是专门总结一些上线可能会问的面试问题，请你根据这些问题和思路想清楚，如何用在你的项目上，要包装就包装好，不要露馅。我会根据面试内容不断总结到这里的。

以下三个问题几乎必问，上线T0问题，前两个问题我在英伟达面经中解析了，大家可以直接跳转观看问题答案。

1.你的项目Agent上线了以后，你是如何监控的?

2. 你的Agent项目上线后通过哪些业务指标来衡量?

3. 线上遇到问题时，如何收集反馈，归因，修复的?

---

14 Agent场景题

这里更新一些Agent场景题，或者说是系统架构设计题目。这类题目通常来说是比较难的。

难点一：需要你对Agent系统的全流程有宏观的理解。你至少设计过Agent架构，做过Agent项目，或者至少看过一些Agent架构，你才能够现场设计出来。另外，你设计Agent架构，最好设计得比较完整，比较合理，且有深度。

难点二：通常来说，你设计出来了架构以后，不会说就结束了。面试官肯定是要围绕着你设计的系统进行探讨，问你为什么这么设计，你要说得出来你的设计原因，架构选型，也要经得起推敲和讨论。这就要求我们对整个设计的架构的各方面都要了解，那对我们综合能力就提出了一些要求。

所以很多同学反馈过，Agent的系统架构的设计题很难，不会回答。我倒是想要多总结一些架构设计题，不过以我的经历，真的考架构设计的不多。今天可算逮到了一个真题，我们以这个真题来讲解如何攻破架构设计题，如何复习。

先说场景题目的准备策略和理论基础，其实Agent场景设计题的理论基础是我们整个Agent的理论部分，项目经验。因为它本身就是一个综合的题目，还会问你各种细节，所以相关问题都是理论基础。这么说大家有没有一点绝望?

哈哈，情况确实是这么个情况，这就是一个综合能力的考察。不过稍微可以画一下重点，对于架构设计也多少有一些模板。大家看了本节后设计起来应该会更容易。理论重点画一下特别关键的部分：

Multi-Agent架构：选单Agent还是多Agent，为什么，常见的Agent架构有啥，这些是设计架构最核心的。

开源项目部分：Claudecode部分，Hermes Agent, Openclaw, PI-Agent. 这些笔记里讲的每一个Agent的架构，我们都有讲，你要是看了笔记，可以看出来它们其实也是类似的，有很多通用的地方。强烈建议从Pi-agent开始看，因为它最简单。

然后就是面试真题，和面试官去讨论架构设计的部分，它的整个讨论的逻辑，问题，都是我们掌握好Agent架构设计的好的素材。比如

小红书一面，夸克千问二面。这两个面经有讨论架构设计，不过其他的面试多少都有讨论。滴滴Agent开发专门考了Agent架构设计，里面有精讲，这些都是我们攻克这一问题的素材。

14.1 解题思路

- **拿到题目先建立分析框架，而不是急着堆细节。**面对任何一个 Agent 场景，都可以从以下几个维度切入、逐层展开。其实你可以看出来我们的开源项目讲解，每个项目都有一个全局架构图，它的核心模块都是类似的，关键地方组成也比较相同：比如Agentic loop(react)，长期记忆，触发层整体框架是类似的，我们可以复用框架，结合填充下面的细节就好了：
- **触发层**：谁来触发、是同步还是异步、要不要分级路由把简单情况挡在 LLM 之外。
- **Agent 架构**：核心推理骨架长什么样——通常是 **Agentic loop（ReAct）**，有些场景也可以用更受控的 **Workflow（编排式）**；关键是根据「延迟/成本/可审计」的要求，在「自由推理」和「固定编排」之间做取舍。
- **记忆模块**：靠什么沉淀经验、形成闭环学习，让这一次的结果能改进下一次。
- **业务场景逻辑**：把模型输出翻译成可执行动作（如分档、阈值、处置策略），这部分高度依赖具体业务。
- **答题时沿着这几个维度展开，就能既有骨架又有细节。**下面的「关键设计要点」正是按这个框架，落到「信用卡欺诈检测」这个具体场景上的一次完整设计。

14.2 信用卡欺诈检测Agent设计

此题选自202606 滴滴Agent开发（社招）面试真题。 整个面试几乎都围绕整个架构设计题讨论。

题目描述：

- 你可以获取客户的**所有历史交易行为、商户信息**等数据（也可以自行创造非结构化数据）。

- 现在**新来一笔流水（交易）**，需要用 **Agent 判断这笔交易是否为欺诈。**

- 要求围绕这个场景，设计并讨论整个 Agent 系统。

设计答案：

针对这个题目，我是设计了如下的架构图。设计要点在后面。框架我们可以复用，框架下面的细节，就是来自于我们的经验了。

关键设计

1. **触发层：事件驱动 + 分级路由，把重活挡在 LLM 之外**

- **触发源按风控场景定义**：主源是「一笔交易到来」（同步、要秒级返回）；辅源是定时 Cron、外部 Webhook（黑名单/情报）、人工复查——辅源只用来**离线刷新画像和模式库**，不阻塞实时链路。
- **Gateway 只做管道不做智能**：认证、把不同来源归一化成标准 schema、**幂等去重**（同一笔只判一次，防重放）。这一层**不调用 LLM。**
- **Router 分级路由是时效关键**：白名单/低风险 → 秒级放行；黑名单/硬规则 → 秒级拦截；只有落入**灰区/可疑**的交易才进入昂贵的 Agent 层。绝大多数交易在此就解决，省成本、保延迟。

1. **Agent架构：React模式**

> **为什么这里选 ReAct loop 而不是 Workflow?** 选型的核心判据是「流程是否确定」。**业务逻辑高度确定、步骤固定可枚举**时（如「先查A→再查B→最后打分」这种不会因中间结果而变的固定管线），用 **Workflow（编排式）**更好——路径可预测、延迟成本可控、易审计。而**步骤灵活、需要 Agent 根据中间结果动态决定下一步、尤其需要 Agent 自己判断「是否收敛」**时（如本场景要判断「现有证据是否足够定案，够了就退出、不够就再追查一维」），就必须用 **Agentic loop（ReAct）**——因为「查哪些维度、要不要基于中间证据再追一手、什么时候停」无法预先写死，得靠模型边看边决策。本场景两种特征都有，所以采用「固定骨架（Workflow 的可控）+ 有界 ReAct（loop 的灵活）」的混合式。

- **不放开让主 Agent 自由 fork 子 Agent**，而是「预定义骨架 + 编排层有界 ReAct」的混合式——因为实时风控要**延迟/成本可预测、可审计**，纯自由推理路径不可控、且会放大错误。
- **Worker 池预先定义好**（地理/速度、行为偏移、商户风险、设备与关联网络四个维度，相互独立、可并行）。**这次调不调、调哪几个、要不要基于中间结果追加一手**，由 Orchestrator 的有界 ReAct 决定：`Think 该查哪些维度 → Act 并行调起子 Agent → Observe 收集证据 → Think 证据够吗？` 不够且未超上限就**循环回 Act 追加一手**，够了或达上限（最多 R 轮 / N 个 Worker / 超时）就收敛退出。
- **每个 Worker 内部也是一个小 ReAct**（感知→调工具查数据→局部研判出风险档→自检打置信度分），但目标和工具集被**限死在单一维度**，不会乱跑。Worker 之间**只回传摘要**给主 Agent，不共享全部中间过程，控制上下文膨胀。
- 一句话：**「固定编制的侦查队 + 一个会随机应变但受纪律约束的队长」。**

1. **Critic 模式：两级复核，一轻一重，判官≠打分者**

- **第①级 • Worker 内部自检（轻）**：每个 Worker 出结论前给自己打个置信度分。不是独立 Agent，只是 prompt 里一步，便宜快，用来给每条信号带上 confidence，方便融合层**按置信度过滤/降权，治噪声。**
- **第②级 • 独立复核 Critic（重）**：位置在**评分融合之后、决策之前。**它是**独立 Agent、独立干净上下文**，只拿到「综合判定 + 完整证据链」，做对抗式复核，专门找「这个判定站不站得住、有没有把正常交易误判成欺诈」，**压低误报。**
- **为什么必须独立、且在融合之后**：单个 Worker 只有一维信息，站位不够，要等证据链拼全才有得审；而「做者不评自己的分」，让打分者自评会系统性高估，所以 Critic 必须换一个干净上下文。重活集中做一

次，避免每个 Worker 都挂重 Critic 拖垮延迟。

- （可选）**第③级 • 人工兜底**：只对「高额 / 低置信 / Critic 与打分层有分歧」的少数 case 转人工。

1. **业务逻辑：决策分档，把风险分翻译成可执行的动作**

> **为什么要专门谈业务逻辑？** 这一层其实**和大模型本身没什么关系**——它不涉及 prompt、推理、记忆这些「AI 味」很重的东西，纯粹是业务规则。但恰恰是这一层最能体现答题者的**思维严谨性和边角 case 覆盖度**：能不能想到「灰区不该非黑即白，而要给一个加验证的缓冲带」、能不能意识到「阈值是业务对误报/漏报成本的权衡而非技术最优」、能不能覆盖「高额、低置信、Critic 分歧」这些需要人工兜底的边界情况——这些细节比堆砌 AI 概念更能拉开区分度。**把架构真正落到具体业务上，而不是停在通用框架，是这类题的加分项。**

- **风控的输出不是一个分数，而是一个动作**：Agent 层最终给出的是「综合风险分 + 置信度 + 证据链」，但业务侧真正要的是「这笔到底放不放」。决策分档就是这层翻译——**按风险高低切成几档，每档绑定一个明确处置动作**，而不是让下游各自解读分数。
- **分档与动作（典型四档）**：① **低风险 → 直接放行**（无感通过）；② **中风险 → 加验证**（触发 3-D Secure / 短信验证码 / 人脸等 step-up，把决策权交还用户，既拦欺诈又不误伤）；③ **高风险 → 直接拦截**（拒绝交易 + 通知）；④ **高额 / 低置信 / Critic 与打分层分歧 → 转人工复审**。核心思想是**给灰区一个「加验证」的缓冲带，而不是非黑即白。**
- **分档线由业务而非模型定**：阈值不是技术最优解，而是**业务对「误报 vs 漏报」成本的权衡**——高价值商户/大额交易调保守（宁可多加验证），低风险场景调宽松（保通过率、保体验）。所以阈值要**可配置、可按场景/额度/商户分层设定**，并随记忆模块回流的真实指标动态调整。
- **为什么单独成层**：把「分档规则」从 Agent 推理里剥出来，好处是**决策可解释、可审计、可快速调参**——运营改一个阈值就能收紧/放松，不用动模型；也让同一套风险分能服务不同业务线的不同风险偏好。

1. **记忆模块：用真实结果做沉淀，形成闭环学习**

> 长期记忆是让 Agent 从「一次性判断」进化到「越用越聪明」的关键。建议设计的时候一定要设计上。笔记里其实讲过很多可复用的知识：**长期记忆到底该存什么？各个开源架构（如 ClaudeCode、Pi-agent、Hermes 等）的长期记忆模块分别沉淀了哪些东西、用什么结构组织？**设计时完全可以把这些经验迁移过来——比如「哪些该进画像、哪些进模式库、哪些只是短期上下文」，就是参照成熟架构的记忆分层思路，落到欺诈检测场景后的一次具体化。有这层积累，记忆模块才不会设计得空洞，而是有据可依。

- **它不服务当前这笔交易，服务的是未来每一笔。**当场给出的判定只是「预测」，判得对不对要等**真实结果**才知道——放行的过几天被投诉盗刷（说明漏报），拦截的被投诉「本人却被拦」（说明误报），或人工复查给出权威裁决。这些真相**几小时/几天后才回流**，所以是异步、事后。
- **沉淀机制（拿到真实标签后做三件事）**：① 更新**账户风险画像**（这个账户真被盗刷过 → 以后多留心）；② 更新**欺诈模式库**（新手法记下来，下次能认出）；③ 调整**评分阈值**（漏报变多就调低拦截线，误报太多就调高）。

- **闭环**：这些沉淀绕回喂给 Orchestrator，让下一次判定更准——「上一笔的教训，喂给下一笔」。同时，真实标签回流也是计算 Recall / Precision（评估准确率、驱动迭代）的**唯一数据源**，没有它整个评估体系无从谈起。

14.3 讲解视频

https://www.bilibili.com/video/BV1t5Nr6kEvm/?vd_source=144bec9c3f54e465073138bed788be1b#reply305991560737

---

15 面试官压力项目应对方案

我在面试的过程中，经常遇到面试官挑战你的项目的三个问题是

1.你的Agent感觉不够智能啊？你的Agent的Agentic特性体现在哪里?

2.你这个就是一个Workflow, 真的能叫做Agent吗?

3.传统方法是不是也可以做，为什么非要用Agent?

之前在我自己懂得不是很多的时候，我经常会被面试官带着跑。面试官压力/挑战我这三个问题，我自己就会不自信，或者自我怀疑。其实当你自己的水平达到了以后，面试官压力你的时候，一个是你不会被带跑，你有自己的思路，同时你还可以现场自圆其说，包装和他讨论。其实类似的问题还有面试官说：RAG都过时了，你还做RAG呢？这个问题我们在RAG章节有专题讨论，可以点这里。

今天专题先讲解这个问题，讲清楚了**相关问题的原理**，面试官问你的时候其实你可以自信大方的回答。这个小节我们讲理论部分。对应的实战部分可以看 202606 深圳大模型研究院Agent开发一面 对应的问题，可以看一下面试官压力我，我是如何现场回答的。要想不被他压力到，就要掌握好今天我们讲的这三个问题的理论部分。

15.1 Agent的Agentic 性质体现在哪里?

下面这套框架，我参考了 Anthropic Building Effective Agents、OpenAI A Practical Guide to Building Agents、HuggingFace smolagents 文档、LangChain cognitive architecture，两本书（Chip Huyen《AI Engineering》第6章、Lanham《AI Agents in Action》第1章），以及 ReAct、Reflexion、Toolformer、Voyager 四篇论文，感兴趣的可以自己去看原文。

一个项目具备下面 8 条特性，就能说明它是 **agentic 的**。前 4 条是核心（有这四条就能站住），后 4 条是加分（决定 agentic 程度高低）。

核心四条

1. **自己决定下一步做什么** —— 步骤顺序是模型当场生成的，不是代码写死的。这是最硬的一条分水岭（"LLM 决定控制流"）。
2. **能自主调工具** —— 不只是"配了工具"，而是模型自己判断*要不要用、用哪个、传什么参数*。加分项：不只是读（查天气、查日历），还能**写**（真的帮你建日程、发消息）——有写权限，agentic 强度立刻上一档。
3. **有反馈闭环** —— 拿到工具结果/环境反馈后，会重新判断：做成了没有？没成就换个方法再来，而不是一条道跑到黑。
4. **有目标感** —— 系统知道"什么算完成"，能自己判断任务结束，而不是回一句话就结束。

加分四条

1. **反思与进化** —— 会自我批评、失败后总结教训；跨会话能从用户行为里学到偏好（短期反思 + 长期学习）。
2. **记忆** —— 跨轮次、跨会话记得上下文和用户习惯，不是无状态函数。
3. **出错能自己恢复** —— 重试、降级、换方案、卡住了会求助，而不是崩掉。
4. **知道什么时候该问人** —— 有明确的升级触发条件（高风险/不可逆动作找人确认）。

三个关键认知（面试时最容易被绕进去的地方）

- **agentic 不是"是/否"，是程度。** 别说"我是 Agent"，要说"我在自主性谱系的哪个位置，为什么停在这"。
- **护栏、审批、预算上限不是减分项，是加分项。** 你需要管住它，恰恰证明它真的有实权。只有自主没有约束那叫失控。
- **判断标准不是"用没用大模型"，是"决策权在谁手里"。**

一句话总纲：**能自己想清楚怎么做 → 自己动手做 → 看结果 → 不对就改 → 越用越懂你。**

15.2 你的就是个workflow，还能叫Agent吗?

先说结论：即使你的系统是 workflow 形式，它依然可以叫 Agent。 "workflow 就不是 Agent"这个前提本身是错的，业界的权威定义和一线生产实践都不支持它。

支撑一：各大公司的官方定义，都把 workflow 归在 agentic systems 之内。

Anthropic 在 Building Effective Agents 里做了区分：workflow 是"LLM 和工具通过预定义代码路径编排的系统"，agent 是"LLM 动态指挥自己的流程和工具使用"。但关键在于，它紧接着说 —— "we categorize all these variations as **agentic systems**"，两者是**架构上的区分，而不是资格门槛。**而且这篇文章的核心建议恰恰是反过来的：workflow "**offer predictability and consistency for well-defined tasks**"，只有在"你无法硬编码一条固定路径"时，自主 agent 才划算，因为自主性意味着"higher costs, and the potential for **compounding errors**"。

OpenAI 在 A Practical Guide to Building Agents 里给的判据也不是"有没有自主性"，而是"LLM 是否在**管理执行、做决策、动态选工具**"，并强调 agent 始终是"operating within clearly defined guardrails"。

HuggingFace 的 smolagents 文档 说得最干脆："'agent' is not a discrete, 0 or 1 definition: instead, 'agency' evolves on a **continuous spectrum**"，它给出的分级表里，连 `if llm_decision(): path_a() else: path_b()` 这种 router 级别的判断都被列进了 **agency 谱系**。

LangChain 的 Harrison Chase 也有一句很有代表性的话："**Nearly all of the 'agentic systems' we see in production are a combination of 'workflows' and 'agents'.**"

支撑二：一线生产系统的工程实践，直接印证了这一点。

这里重点举 DoorDash 的例子。DoorDash 是美国最大的外卖/即时配送平台（类似国内的美团外卖），日订单量巨大，业务链路涉及商家、骑手、消费者三方。它不是一家做 AI 框架的公司，它讲的是**自己内部真实在跑的生产级 Agent 系统**——这一点很重要，因为框架厂商的文档难免有营销成分，而一线业务公司的工程博客讲的是他们真金白银踩过坑之后的选型结论，说服力完全不同。

DoorDash 在 Beyond Single Agents（2025-11）里，对自己的生产流程是这么描述的：

> "Represented as **directed graphs, these deterministic pipelines** have a clear beginning, middle, and end. **There are no unexpected detours and no improvisation.**"
>
> "**This rigidity is a critical feature.**"

注意最后这句 —— 这种"死板"被他们明确称为**关键特性**，而不是技术妥协。

他们另一篇讲 Agentic Orchestrator 的文章里，有一句几乎就是这种架构形态的标准定义：

> "**The code decides what runs next**, persists artifacts, and enforces the review gates, **while the agent does the thinking; it doesn't get to invent the workflow as it goes.**"

翻译过来就是：**代码决定下一步跑什么，agent 负责思考，但它不能自己发明流程。**

更重要的是，DoorDash 还明确给出了选型判据：**certified / high-stakes 的任务走确定性图，exploratory（路径不确定）的任务才放 deep-agent 上去。** 原文是 "We rely on deterministic workflows for certified tasks where reliability is paramount and reserve the more dynamic deep-agent capabilities for exploratory work where the path is uncertain."

所以说到底，**这只是一个选型问题，不存在高低贵贱之分。**

任务路径确定、结果可验证、失败代价高（比如涉及金额、合规、用户数据写操作）的场景，选确定性 workflow 更合理 —— 因为它可观测、可回滚、可审计、成本可预估，而这些恰恰是自主循环给不了的。反过来，任务空间开放、无法穷举路径、试错成本低的探索型场景，才值得把自主性放开。

而且要注意，自主性带来的不只是灵活性，还有**复合错误率**：假设每一步准确率 95%，10 步之后整体准确率就掉到约 60%，100 步之后只剩 0.6%。所以在窄域任务上主动收窄自主性，是一个负责任的工程决策，而不是"没能力做成真 Agent"。

15.3 传统方法就能做，你用Agent做有必要吗?

这不是一道有标准答案的题，它本身就是业界仍在争论的问题。 所以回答它的正确姿势不是论证"我的方案是对的"，而是论证"我知道这个边界在哪、我算过账"。面试官问这句话，真正想听的不是你的 Agent 多厉害，而是**你知不知道什么时候不该用 Agent。**

下面先给三个思考点，把这个问题的争议面铺开；最后再落到一节：那到底该怎么做选型。

12.3.1 三个思考点

思考点一：同样一件事，两家一线公司做了完全相反的选择。

Meta 和 Grab两个公司**几乎是同一个业务——给数据资产打隐私标签**：公司内部有成千上万张表、字段、日志、事件流，得先知道"哪个字段装的是身份证号、哪个是手机号、哪个只是个缓存时间"，才能在上面自动执行隐私管控（该删的按期删、该脱敏的脱敏、该限制访问的限制访问）。**这是一个典型的分类任务，也是规则和大模型都能做的活儿。但他们的做法却截然不同。**

Meta 的做法：大模型只在幕后学习，线上跑的是规则。

Meta 那套系统叫 **Privacy-Aware Infrastructure（PAI，隐私感知基础设施）**，见 Privacy-Aware Infrastructure in the AI-Native Era: An Asset Classification Case Study（2026-06）。它要打标的"资产"范围很广——原文说 "**An asset can be more than a table or column**"（一个资产不只是一张表或一个字段），还包括嵌套结构里的字段、日志键、事件参数、API 字段、机器学习特征、embedding、派生数据集。而且判断依据是语义不是形状：一个叫 `age` 的字段，可能是用户年龄（个人数据），也可能是缓存的存活时间（什么都不是）。打完标之后，上层才能执行保留期、访问控制、允许用途、下游共享、匿名化这些策略。

它的做法是**分层**：

"deterministic rules resolve the large majority of traffic, roughly 85%, in single-digit milliseconds."（确定性规则在个位数毫秒内处理掉绝大部分流量，约 85%。）

"That path is slower — on the order of seconds — and roughly **400 times the compute cost**, so it is budgeted separately."（那条大模型路径更慢——以秒计——且算力成本约为 **400 倍**，因此单独立预算。）

规则在个位数毫秒内处理掉约 85% 的流量；剩下 15% 没有规则覆盖的新资产，才交给大模型。 更关键的是它的分工思路——大模型不是拿来干活的，是拿来**总结规律**的，规律经校验后被导出成代码：

"Validated rules are exported to Python, SQL, JSON, or Hack for deployment in production systems with **zero LLM dependency**."（经校验的规则被导出为 Python、SQL、JSON 或 Hack，部署到生产系统时**完全不依赖大模型。**）

"the production path remains a deterministic engine, while the LLM is **reserved solely for novel assets that lack rule coverage**."（生产路径始终是一个确定性引擎，大模型**只保留给那些尚无规则覆盖的**

新资产。）

Meta 给了一句很到位的总结：

"**LLMs help the system learn. Deterministic rules help the system enforce.**"（大模型负责让系统学会，确定性规则负责让系统跑起来。）

以及一句判据："**Even a strong LLM classifier should not be the default enforcement path forever.**"（再强的大模型分类器，也不该永远做默认的执行路径。）

Grab 的做法：大模型直接常驻线上干活。

Grab（东南亚最大的打车/外卖平台）做的几乎是同一件事，系统叫 **Gemini**（内部编排服务，名字早于 Google 那个同名模型），见 LLM-powered data classification for data entities at scale。它给数据库表、Kafka 消息 schema、数据湖表的**每个字段**打标签（如 `Personal.ID`、`Personal.Name`、`Personal.Contact_Info`、`Geo.Geohash`、`None`），结果同样用于隐私分级（Tier 1–4）、属性访问控制（ABAC）和查询时的动态脱敏。

但它的选择是**让大模型直接在线上跑，不做规则化。**理由写得非常实在：

"Building in-house classifiers would require a **dedicated data science team** to train a customised model."（自建分类器需要一支**专职的数据科学团队**来训练定制模型。）

它还补了两条：第三方工具 "did not allow customisations of its machine learning classifiers"（不允许定制其机器学习分类器），而正则表达式方案误报太多；相比之下大模型 "can be **customised effortlessly without code or model training**"（无需写代码或训练模型就能轻松定制），并且 "provides a **natural language interface** for data governance personnel"（为数据治理人员提供了自然语言接口）。

规模对比很说明问题：Grab 上线一个月扫描了 2 万多个数据实体，**平均每天约 300–400 个**——按每个人工处理 2 分钟估算，一年省下约 360 人日。

同一个技术问题，两家给出相反答案，而且都不能说谁错了。 但这里要注意，差别**不是"谁觉得哪个更贵"这种主观判断，而是两家的瓶颈根本不在同一个地方**：

- **Meta 卡在流量。** 它面对的是 "**millions of assets**"（数百万级的资产），且分类结果要支撑线上的隐私策略执行——大模型路径贵 400 倍、慢到以秒计。**在这个量级上，"让大模型一直跑"不是贵不贵的问题，是物理上跑不动。** 它不是主动选择了规则，是被流量逼着必须把规律固化下来。
- **Grab 卡在人。** 它的量级是**每天 300–400 个数据实体**，这个吞吐量大模型完全扛得住，所以真正稀缺的资源是"谁来干这件事"——原文说的是需要 a dedicated data science team，那是**招人、留人、长期占编制**的问题，不是账上有没有这笔钱。

换句话说，两家看的压根不是同一本账：Meta 面对的是百万级资产 × 400 倍成本，Grab 面对的是每天几百个实体、但要不要为此常设一支团队。谁的约束更硬，谁就决定了选型。

这就是回答这道题最诚实的开场：**它是一个约束条件问题，不是技术先进性问题。所以正确的答法不是论证哪种方案更好，而是说清你的瓶颈在哪、你的账是怎么算的。**

思考点二（反向视角）：几乎所有框架厂商都在劝退，而不是鼓吹。

这一点反常但很重要——你说"这里我评估过，不该用 Agent"，不是保守，而是**和官方立场完全一致**：

- **Anthropic**（Building Effective Agents）："we recommend finding the simplest solution possible, and only increasing complexity when needed. **This might mean not building agentic systems at all.**"（我们建议找到尽可能简单的方案，只在必要时才增加复杂度。**这可能意味着根本就不要构建 agent 系统。**）
- **HuggingFace smolagents**："Agents are useful when you need an LLM to determine the workflow of an app. But **they're often overkill**."（当你需要让大模型来决定应用的流程时，agent 是有用的。但**它们往往是杀鸡用牛刀。**）甚至更进一步："it's advised to **regularize towards not using any agentic behaviour**."（建议向着"**不使用任何 agent 行为**"的方向做正则化。）它给的判断句是："**do I really need flexibility in the workflow to efficiently solve the task at hand?**"（**为了高效解决手头这个任务，我真的需要流程上的灵活性吗?**）
- **微软 Azure 架构中心**（AI agent design patterns）：把"**Using nondeterministic patterns for workflows that are inherently deterministic**"（对本质上确定的流程使用非确定性方案）正式列为**反模式**，并写明："**If prompt engineering can solve the problem, you don't need an agent.**"（**如果提示词工程就能解决问题，你不需要 agent。**）
- **Google ADK**：建议的架构是把模型的非确定性能力与确定性代码**交织**（interweave），而不是让模型全权接管整个任务的执行。

思考点三（乐观视角）：允许事物的发展

前面两个思考点都在讲"现在的争议在哪"，但选型判断如果只看当下，会犯一个错误：**把"暂时的技术缺陷"当成"永久的结构性缺陷"。** 这两者必须分开算。

Agent 今天的问题，绝大多数是会随时间消失的： 慢、贵、不稳定、幻觉。这些都是技术问题，而技术问题的曲线是向下的。Chip Huyen 在《AI Engineering》里给了一个很直观的例子——Scribd 的应用研究负责人 Matt Ross 说，他们用例的 AI 成本 "**has gone down two orders of magnitude from April 2022 to April 2023**"（在 2022 年 4 月到 2023 年 4 月的一年间，下降了两个数量级）。书里的原话是 "**The cost of AI reasoning rapidly drops over time.**"（AI 推理的成本随时间迅速下降。）

能力侧同理。METR 的 Measuring AI Ability to Complete Long Software Tasks 测的是"AI 能独立完成多长时间的任务"，结论是：

> "frontier AI time horizon has **doubled approximately every seven months** since 2019."（自 2019 年以来，前沿 AI 的任务时长跨度**大约每七个月翻一番。**）

注意这条的方向——它是站在 Agent 这一边的。 也就是说，前面提到的"独立完成率 30%"这类数字，是一个**正在移动的数字**，不是天花板。

而传统方法的问题，是不会自动消失的： 每来一个新情况要加一条规则、要重新验证不冲突、要有人长期维护——这是**结构性成本**，它不随模型变强而减少，只随业务变复杂而增加。

这个对比还有一个更扎心的版本：**押注自建的一方，可能会被通用能力的进步直接抹平。** 《AI Engineering》里记录了 BloombergGPT 的案例——彭博训练了一个 500 亿参数的金融专用模型，耗费 130 万 A100 GPU 小时，算力成本估算在 **130 万到 260 万美元**之间（还不含数据成本）。而**同一个月**，OpenAI 发布了 GPT-4-0314，后续研究显示它在多项金融基准上**零样本就显著超过了 BloombergGPT**（FiQA 情感分析加权 F1：**87.15 vs 75.07**；ConvFinQA 准确率：**76.48 vs 43.41**）。书里给的告诫是：

> "**Beware of the argument that general-purpose models don't work well for domain-specific tasks, and, therefore, you must finetune or train models for your specific tasks. As general-purpose models become more capable, they also become better at domain-specific tasks and can outperform the domain-specific models.**"（要警惕这样一种论调：通用模型在特定领域任务上表现不好，所以你必须为自己的任务微调或训练模型。随着通用模型能力增强，它们在特定领域任务上也会越来越好，甚至能超过领域专用模型。）

所以时间维度上的判断是：技术缺陷会被时间修复，结构性成本不会。 今天算不过来的账，可能十二个月后就算得过来了；而今天靠堆规则勉强撑住的系统，三年后只会更难维护。这也是为什么值得在一部分场景上提前把架构切过去——**不是因为它今天更好，而是因为等到它明显更好的那天再切，你已经落后一个身位了。**

但必须警惕的是：这个论点很容易被讲成"因为是未来趋势所以现在就得上"，那是危险的。 面试里如果只讲"汽车终将取代马车"这类类比，会被认为是在跟风。正确的表述有两个约束：

1. **趋势论只能用来解释"为什么在某些场景提前投入"，不能用来解释"为什么所有场景都上 Agent"。** 前面三条判据依然是硬门槛。
2. **要说清你押的是什么。** 押的不是"Agent 现在更强"，而是"**这个模块的变化速度，我用规则追不上；而模型的能力和成本曲线，正在朝着能追上它的方向走**"。

一句话：短期看约束（成本、延迟、可靠性），长期看曲线（能力在涨、成本在降）。判断该不该现在切，就是看这两条线在你这个场景里，什么时候交叉。

15.3.2 结论

争议归争议，选型总要有个可操作的方法。OpenAI 给了业界最明确的一套判据。

OpenAI 在 A Practical Guide to Building Agents 里明确写了，agent "are uniquely suited to workflows where **traditional deterministic and rule-based approaches fall short**"（尤其适合那些**传统确定性方法与规则方法力有不逮**的工作流），并给出三条判据。**符合任意一条，才值得考虑；一条都不占，就老实用传统方法。**

判据一：需要"判断"，而不是"查表"。

"Complex decision-making: Workflows involving **nuanced judgment, exceptions, or context-sensitive decisions**."（复杂决策：涉及**细腻判断、例外处理或上下文敏感决策**的工作流。）

OpenAI 举的例子是支付欺诈识别：规则引擎像一份检查清单，按预设标准给交易打标；而大模型更像一个资深调查员——

"evaluating context, considering subtle patterns, and **identifying suspicious activity even when clear-cut rules aren't violated**."（评估上下文、考量微妙的模式，**即使没有任何明确规则被违反，也能识别出可疑行为。**）

注意最后半句：明明每一条规则都没被违反，但它就是不对劲。 这种判断是规则写不出来的，因为规则的本质是穷举已知条件，而这里的条件是"跟这个人平时的习惯不像"。

判据二：规则多到没法维护了。

"Difficult-to-maintain rules: Systems that have become **unwieldy due to extensive and intricate rulesets**, making updates costly or error-prone."（难以维护的规则：**因规则集庞大而错综复杂**、已变得笨重不堪的系统，使得更新代价高昂或极易出错。）

这一条的重点是"维护"两个字，不是"能不能做"。 规则加到第三百条的时候，问题不是写不出第三百零一条，而是你已经无法确认新规则会不会跟前面某条打架。Google 那篇著名的 ML 技术债论文（Machine Learning: The High-Interest Credit Card of Technical Debt, NIPS 2014）把这个现象叫 **CACE 原则：Changing Anything Changes Everything（改一处，动全身）**。同一篇文章还有一个更狠的数字：

"a mature system might end up being (at most) **5% machine learning code** and (at least) **95% glue code**."（一个成熟的系统最终可能（至多）只有 **5% 是机器学习代码**，而（至少）**95% 是胶水代码。**）

一个成熟系统里真正干正事的代码只占 5%，剩下 95% 是数据接入、清洗、对齐、监控、告警这些琐事。 所以当有人说"传统方法也能做"，他通常只算了那 5% 的账。

判据三：输入是自然语言或非结构化文档，不是规规矩矩的字段。

"Heavy reliance on unstructured data: Scenarios that involve **interpreting natural language, extracting meaning from documents**, or interacting with users conversationally."（重度依赖非结构化数据：涉及**理解自然语言、从文档中抽取语义**，或以对话方式与用户交互的场景。）

Google 的 Agents 白皮书举了一个和行程解析几乎同构的例子：用户说"I want to book a flight to Zurich"（我想订一张去苏黎世的机票），却**从没提出发城市。**你写代码去解析，就得补一个"缺出发地"的分支；然后又会冒出"下周那趟"、"跟上次一样"……原文的结论是：

"more code would need to be implemented in order to catch edge and corner cases like this. **This approach is not scalable and could easily break in any scenario that falls outside of the implemented custom code.**"（为了兜住这类边角情形，就得不断补写代码。**这种做法不可扩展，只要场景超出已写的定制代码范围，它就极易崩溃。**）

判据之外，还有一句必须记住的收尾。 OpenAI 在列完三条之后紧接着写：

"Before committing to building an agent, validate that your use case can meet these criteria clearly. **Otherwise, a deterministic solution may suffice.**"（在决定构建 agent 之前，先验证你的用例确实清晰地满足上述判据。**否则，一个确定性方案可能就够用了。**）

所以真正的判据是什么？一句话：能不能提前把流程写死。

但这句话要补一个前提，——理论上一切都能写死，只是有的要写一千万行。所以完整的问法是：

写死它要花多少钱？这笔钱是一次性付清，还是每来一个新情况就要再付一次？

- **一次性的成本** —— 写完就不用管了。比如"两个时间段重不重叠"，1979 年写的代码和今天写的是同一份。**这种永远别用大模型。**
- **持续渗漏的成本** —— 每冒出一个新说法就得加规则、补标注、重新验证一遍不冲突。**这种才是大模型的地盘。**

换个角度说，两种方案的成本增长方式是不同的：

规则的成本随"情况的种类"增长；大模型的成本随"调用的次数"增长。

而在真实系统里，答案几乎从来不是二选一，而是**分层**：想清楚大模型站在哪一层——是在线执行层、离线学习层，还是只做长尾兜底。**能主动说出"这几步我故意没用大模型"的人，比论证"我这几步用了大模型"的人，可信度高一个量级**

15.4 ClaudeCode功能已经很强了，你再做一个Agent有意义吗?

问题： 现在 Claude Code、Codex 已经很成熟了，你为什么还要开发自己的 Agent 平台?

查看思路与参考答案：为什么要自建Agent平台?

15.5 面试问题

我这里结合这一小节的理论部分，从理论部分整理一些面试问题。如上面所说，面试官确实在真实场景下压力我，我化解这个压力的方法也是基于我们今天的理论知识。对应的真实面试可以看 202606 深圳大模型研究院Agent开发一面 。

15.5.1 Agent 的 Agentic 性质体现在哪里?

我认为主要核心有以下四条：

第一，下一步做什么是模型当场决定的，不是代码写死的，也就是控制流在 LLM 手里。这是最硬的一条分水岭。

第二，工具是模型自己判断要不要用、用哪个、传什么参数。而且不只是读，还有写权限，能真的改变外部状态——有写权限，agentic 强度立刻上一档。

第三，有反馈闭环。拿到工具结果或环境反馈会重新判断做成没有，没成就换个方法再来，而不是一条道跑到黑。

第四，有目标感。系统知道什么算完成，能自己判断任务结束，而不是回一句话就收工。

加分四条：反思与进化、跨轮次跨会话的记忆、出错能自己恢复、以及知道什么时候该升级找人确认。这四条决定的是 agentic 程度的高低。

总结来说，我觉得Agentic性质体现在：能自己想清楚怎么做 → 自己动手做 → 看结果 → 不对就改 → 越用越懂你。

15.5.2 怎么判断该用 Agent/LLM，还是传统规则?

（我的思路是第一段给出行业例子，**说明该问题本来就是开放性的。** 第二段给具体选型指导原则。第三段补时间维度的看法，**体现自己对行业的思考**）

这道题没有标准答案，它本身就是业界还在争的问题。同样一个问题，不同的公司做法可能不一样，我举个例子，同样的业务：给公司内部成千上万的数据表、字段自动打隐私标签，Meta 是让大模型在幕后总结规律，规律被导出成代码，线上跑的全是写死的。Grab 是直接让大模型常驻线上干活，不做规则化。考虑点不同，做法也会不同，我觉得没有绝对对错。

我个人觉得我们可以从以下方面思考：

第一，需要细腻判断而不是查表——比如支付欺诈里"每条规则都没被违反，但它就是不对劲"这种情况，规则写不出来。

第二，规则集已经庞大到没法维护——重点是维护，不是能不能做。加到第三百条时，问题不是写不出第三百零一条，而是无法确认它会不会跟前面某条打架。

第三，输入是自然语言或非结构化文档，而不是规规矩矩的字段。

比这三条更本质的问法是：把流程写死要花多少钱？这笔钱是一次性付清，还是每来一个新情况就要再付一次？判断两个时间段重不重叠，1979 年的代码和今天是同一份，这种一次性成本的永远别用大模型；每冒出一个新说法就要加规则、补标注、重新验证不冲突的，那才是大模型的地盘。换句话说，规则的成本随情况的种类增长，大模型的成本随调用的次数增长。

最后补一个时间维度：Agent 今天的问题——慢、贵、不稳定——大多是会随时间消失的技术问题；而传统方法的维护成本是结构性的，不随模型变强而减少。任何事情的发展都是有过程的。Agent现在也许速度慢，不稳定，但无可否认，相比于传统开发，它改变了我们的开发方式。**历史从不是由技术最完美的那一刻开始的，而是由观念转变的那一刻开始的，我相信未来Agent的效果也会越来越好。**

15.5.3 Agent 的工作模式，是 workflow 编排，还是让模型自己想下一步?

我想先澄清一个前提：很多人默认"是 workflow 就不算 Agent"，但这个前提其实站不住。Anthropic 自己就说过，**workflow 和 agent 只是架构形态上的区分**，两者都属于 agentic system。业界现在跑在生产上的系统，也几乎都是这两种东西的混合体，很少有纯粹的哪一种。

所以在我看来这就是个选型问题，谈不上谁高谁低。

如果路径本来就是确定的、结果能验证、失败代价还高，那我倾向于用编排。因为编排的好处是可观测、可回滚、可审计，成本还能提前估出来，这几样自主循环给不了。反过来，任务空间是开放的、路径根本穷举不完、试错又便宜，那才值得把自主性放开，让模型自己决定下一步。

还有一笔账不能不算，就是复合错误率。假设每一步准确率 95%，10 步之后整体就只剩 60% 左右，100 步基本归零。步骤越长，自主性的代价越大。所以在窄域任务上主动把自主性收窄，我觉得是个负责任的工程决策，不是没能力做成所谓的"真 Agent"。

16 RAG

RAG的话，其实我们的项目就是一个最好的经典RAG的例子。所以建议了解RAG的八股知识后，可以去了解这个项目，这个项目把RAG最经典的环节都串起来了，并且这个项目配的模拟面试/项目复习的SKILL，就是去掌握项目，学习RAG各个环节的好方法。 该项目部分也会配一些该项目面试的真题，和前面的RAG理论相互补充。学好这个理论/项目，经典RAG就差不多了。

RAG进阶是：Graph RAG, Agentic RAG, 知识图谱这里。 我笔记总结了基本概念，我的策略是问到我就说会，讲述我的概念，但是我没有深入，因为这是RAG进阶技巧，对于那种对RAG要求比较高的岗位，专门做RAG的组可能会用。如果求职方向就是大模型应用，大模型算法，算法工程，没有指定RAG方向的话，没有余力的话，对于RAG的进阶部分，可以就只看一下笔记本节**RAG进阶**的概念，了解基本概念就好。

RAG部分我没有说写的特别详细，主要是为了记录RAG的要点和概述，像是一个用来背诵和记忆的八股手册，我会嵌入一些面试真题。大家主要用来复习，以及寻找面试真题答案的来源。

看本节笔记也能够理解RAG大概的步骤。这里更像一个概念卡片，带大家理解具体步骤。**不过更具体的，深入的，或者不理解的地方，大家要自己去网上搜一下资料。**因为这些东西其实都是比较现成的知识，基本上每本书都会讲，所以网上资料很多，大家要加强理解的就搜关键字，视频都行。

16.1 RAG全局总览

RAG 全称 Retrieval-Augmented Generation（检索增强生成）。一句话概括：让 AI 在回答前，先去一个"专属知识库"里查资料，再根据查到的内容来回答，而不是单纯靠它脑子里记住的东西瞎编。

可以类比成开卷考试：大模型是考生，知识库是允许带进考场的参考书。遇到问题先翻书找到相关章节，再组织答案——这样答得更准、更不容易胡说。

这一小节带大家用这张图来理解整个RAG系统的流程。

这张图把整个流程分成了两条链路。

一、离线知识库构建链路（绿色，"提前准备参考书"）

这部分是事先做好的，目的是把你的文档变成 AI 能快速查阅的知识库，一共 7 步：

- 1 原始文档：你的资料，比如 PDF、Word、网页等。
- 2 文档解析：把各种格式读出来，变成纯文字。
- 3 文档清洗：去掉乱码、页眉页脚、广告等垃圾内容。
- 4 文档分块：把长文章切成一小段一小段，方便后面精准检索。

- 5 元数据生成：给每段贴"标签"，比如标题、来源、时间、关键词等。
- 6 向量化：把文字转成一串数字（向量），让计算机能计算"语义相似度"。
- 7 向量数据库：把这些向量存进专门的数据库，等着被查询。

> 核心思想：把书"消化"成 AI 容易检索的形式，存起来备用。

二、在线用户问答链路（蓝/橙/紫色，"用户提问时实时处理"）

用户真正提问时走这条线，一共 9 步。

理解问题阶段（蓝 + 橙）

- 1 用户问题：用户输入问题。
- 2 上下文补全：结合前面的对话，补全省略的信息（比如"它"指的是谁）。
- 3 意图理解：搞清楚用户到底想问什么。
- 4 歧义判断：判断问题清不清楚。如果不清楚，就走 澄清追问（反问用户一句，再回到第 2 步重新理解）；如果清楚，就继续往下。

检索与生成阶段（蓝 + 紫）

- 5 查询处理：把问题"优化"得更适合检索，常见手段有改写、扩展、拆解等。你只需知道这一步是"把问题打磨得更好查"即可。
- 6 检索与筛选：去知识库里找出最相关的几段资料（相似度计算、按权限和元数据过滤等）。这是 RAG 里的 "R（检索）" 核心。
- 7 重排序 / 压缩：把找到的资料排个序、去重、精简，只留最有用的部分。
- 8 生成回答：把"问题 + 检索到的资料"一起交给大模型，让它基于资料生成答案，并附上引用出处、做事实核查，答不了就拒答，避免乱编。这是 RAG 里的 "G（生成）" 核心。
- 9 返回用户：把最终答案返回给用户。

16.2 RAG vs 微调

16.3 RAG评价指标

（Reference: 大模型RAG实战： 4.5小节）

Rag场景下，比较著名的评估模型方法的框架是：**RAGAS**。RAGAS方法在评估过程中主要使用**忠诚度**，**答案相关性**，和**上下文相关性**来评判整个RAG系统中llm生成答案的质量。 RAGAS方法的一个优点是不需要参考答案。

具体来讲，**忠诚度**是指用户判断LLM的输出是否是根据召回文本的内容生成的。

答案相关性是衡量生成的答案是否和用户的问题相关的。

上下文相关性是用于判断召回文本是否和用户问题相关的。

（下面的可以挑一个具体的指标来展开给面试官讲，都展开讲太长了，而且内容有点抽象，如果讲完面试官不懂，可以给他举具体的例子讲解）

具体来讲：

忠实度的计算方式通常包括两个步骤。1：使用LLM生成的问题和答案，让大模型生成多个陈述。 2. 让LLM判断是否可以通过召回的文本，推断出这些陈述。 最后采用能够推断出来的陈述/总陈述量作为忠诚度的评分。

答案相关性：根据LLM的回答生成若干个假问题，然后计算假问题和真实问题的相似度，将相似度平均值作为答案相关性的评分。

上下文相关性：让LLM挑选出召回文本片段中能够回答用户问题的文本子集，用文本子集占召回文本的比例作为上下文的相关性数值。

16.4 常见的向量数据库

常见的向量数据库有Milvus, Weaviate, Chroma, Qdrant。具体来讲：

Milvus 建立在Faiss, Annoy等开源向量搜索库的基础上，能够支持**数十亿级别**甚至万亿级别的向量搜索库，并且**检索速度快**，万亿向量数据集上检索速度的延迟可以控制在毫秒级。

除此之外，Milvus还支持**数据分片**，**数据持久化，流式数据摄取**，**混合搜索**等很多高级功能。具有**高可用性**，**高性能**和**易扩展**的特点，用户还可以将其部署在Kubernetes上。并且可靠性强，具有**数据复制**和**故障恢复**的功能。

Weaviate 具有**鲁棒性**，**云原生性**和**可扩展性**的特点。它结合了向量搜索和结构化过滤的能力，同时拥有云原生数据库的容错性和可伸缩性。在百万级的向量数据上，可以实现毫秒级的检索。它可以方便调用OpenAI, Cohere以及Hugging Face平台提供的先进的数据向量化接口和模型。缺点是：目前只支持HNSW这一种检索算法。

Chroma 是一款AI原生的开源向量数据库，主要功能和特点围绕AI应用开发而设计。主要针对大模型RAG场景，目前只实现了单机方案。相比Weaviate和Milvus，Chroma更轻量化。 检索上实现了FAISS索引，不支持GPU加速。

16.5 RAG的重要步骤

参考视频

16.5.1 意图识别

意图识别位于 RAG全局总览的第3，4（图中橙色的部分）

是什么?

意图识别位于 RAG 的检索前处理阶段，核心作用是先理解用户到底想问什么，再决定后面应该如何检索。它不是简单给用户问题打一个分类标签，而是要把用户的自然语言问题转成后续 RAG 可以执行的检索计划。

在实际场景中，用户的问题经常是不完整、口语化、依赖上下文的。比如用户问“这个怎么申请？”“它和那个有什么区别？”如果直接把原句送进向量数据库，很容易召回错误内容。因此在正式检索前，系统需要先判断用户的真实意图、问题中的关键对象、时间地点等限制条件，以及这个问题是否需要先澄清。

怎么做?

意图识别通常会做三件事。

第一，**判断用户的问题类型**。比如用户是在问定义、查事实、问流程、做比较、要总结、查政策，还是排查问题。不同类型的问题适合不同的处理方式：事实类问题通常适合精确检索，比较类问题适合拆成多个子问题，总结类问题需要召回更大范围的上下文，政策类问题则通常需要结合地区、时间、政策类型等元数据过滤。

第二，**抽取实体和槽位**。实体是用户问题中真正关心的对象，槽位是检索时需要用到的限制条件。例如“2024 年上海人才补贴怎么申请？”中，主题是“人才补贴”，地区是“上海”，时间是“2024 年”，问题类型是“申请流程”。这些信息可以直接用于过滤文档集合，避免在整个知识库里盲目检索。

第三，**决定检索策略**。如果用户问题已经明确，可以直接检索；如果问题表达口语化或依赖上下文，可以先做查询改写；如果用户用词和文档用词可能不一致，可以做查询扩展；如果问题比较短、抽象，可以用 HyDE 生成假设性文档辅助召回；如果问题包含多个子目标，可以进行查询分解；如果缺少关键信息，就应该先向用户澄清，而不是强行检索。

比如用户问“这个怎么申请？”，如果上一轮正在讨论“上海人才租房补贴”，系统就可以把它改写成“上海人才租房补贴怎么申请？”再检索。如果没有上下文，系统就应该追问“你想申请的是哪类事项？比如人才补贴、项目申报、发票报销，还是其他业务？”这就是意图识别在多轮 RAG 中的价值：能补全的自动补全，不能确定的先澄清。

在工程实现上，可以用规则、小模型分类器，也可以直接用大模型做结构化理解。现在**更常见的方式是让大模型输出一个结构化结果**，例如：

```
{
  "intent": "policy_query",
  "entities": ["人才补贴"],
  "filters": {
    "region": "上海",
    "year": "2024"
  },
  "needs_clarification": false,
  "route": "policy_knowledge_base",
  "retrieval_strategy": "metadata_filter",
  "rewrite_query": "2024年上海人才补贴政策的申请条件、补贴标准和办理流程是什么？"
}
```

这个结果比单纯输出一个 `policy_query` 更有用，因为它可以直接驱动后续流程：`intent` 决定问题类型，`filters` 用于元数据过滤，`route` 决定去哪个知识库，`retrieval_strategy` 决定是直接检索、改写、扩展、HyDE、分解还是澄清。

如何判断意图识别做得好不好?

我们需要从意图分类，槽位抽取，是否需要澄清，路由是否正确这几个维度来综合评估。

意图分类可以看 整体准确率、精确率、召回率、F1；

实体和槽位抽取可以看 Slot 精确率、Slot 召回率、Slot F1；

是否需要澄清可以单独看澄清判断的准确率和召回率；

路由是否正确可以看 Route Accuracy；

最终还要看加入意图识别后，正确文档**是否更容易进入 Top-K，答案准确率是否提升。**

> **指标释义：** ❗**如果还不明白就问AI，都是标准指标，也确实有点难绕。**
>
> **整体准确率 (Accuracy)**：所有判断里预测对（包括对和错两类判定都对）的样本占总体的比例。
>
> **精确率 (Precision)**：模型预测为“是”的结果里，真正正确的比例（主要防误报、防乱判）。
>
> **召回率 (Recall)**：在实际为“是”的所有样本中，模型成功找出来多少的比例（主要防漏报、防漏判）。
>
> **F1 值 (F1-Score)**：精确率和召回率的调和平均值，两者双高才能拿到高分，反映综合好坏。
>
> **槽位 (Slot)**：从用户提问里抽出来的结构化属性约束条件，如时间、地区、主题。
>
> **前 K 个检索结果 (Top-K)**：指搜索引擎按相关性排序最终返回给用户的前 K 个文档片段。
>
> **平均倒数排名 (MRR)**：评判第一个正确结果排在第几名，排名越靠前（第一名倒数为1，第二名倒数为0.5）分值越高。
>
> **前 K 个命中率 (Hit@K)**：测试样本集中，检索前 K 个文档包含有正确答案的样本比例。
>
> **归一化折损累计增益 (nDCG)**：不仅衡量找回的文档是否相关，还非常看重最相关的文档是不是排在最前排

16.5.2 输入文件处理

在进行文档分割前，我们先要对输入的各种类型的文件进行处理，转换成文本内容，进行切块。这个地方倒是算不上是RAG里面的重要内容，属于传统方法的内容。不过我们的项目是以PDF作为输入，转换成Markdown的文本，进行后续切块的，根据粉丝朋友们反馈，**这里面面试被问得很多**，所以我干脆把这里沉淀成专题笔记：PDF到MarkDown的处理，相关的面试真题可以参考这里：PDF处理问题。

我觉得这个知识点**纯粹是为了应付你使用我们笔记RAG项目，面试官可能会问到的相关的问题。**如果你没有用我们项目，我觉得你可以放一放这个问题，这个问题不算是RAG里面很重要的知识点。

16.5.2.1 整体处理逻辑

整体处理逻辑很清晰，我们把工业化的处理分成5个连续的阶段：

文件体检：检查 PDF 是否重复、加密、损坏或方向异常，确认文件可以正常处理。

逐页判断：判断每页属于原生文本、扫描、混合、复杂布局还是异常页面。

按页面类型解析：为不同类型的页面选择对应的解析方法。

统一结果：将不同解析结果按页码和阅读顺序整理成统一格式。

结果检查：检查漏页、乱码、重复、错序及表格错误，确保最终内容完整、准确、可用。

很好理解对吗？接下来我们逐一讲解。

16.5.2.2 步骤一：文件基础体检

基础体检只负责文件级别的准入和预处理，不负责提取正文。

- **文件哈希已经存在**：视为重复文件，跳过处理或复用已有结果。
- **文件哈希发生变化**：视为文件新版本，重新处理。
- **PDF 有密码**：先获取密码或解密；无法解密则停止处理并记录原因。
- **PDF 损坏或无法打开**：尝试修复；修复失败则要求重新提供文件。
- **页面方向错误**：在文字提取或 OCR 前进行方向校正。
- **总页数可以读取**：记录总页数，处理完成后用于检查是否漏页。

文件通过基础体检后，进入逐页分类。

16.5.2.3 步骤二：逐页判断类型

页面类型通常通过程序规则判断。可以使用 PyMuPDF 读取每页的文字、图片和位置信息，再综合判断：

- 页面中有多少可正常读取的文字；
- 页面是否主要被一张或多张图片覆盖；
- 提取出来的文字是否存在大量异常字符；
- 少量文字是否只是跨页重复的水印、页眉或页脚；
- 页面是否存在多栏、表格、表单等复杂布局。

根据这些信息，将页面分为以下类型。

原生文本页

页面存在完整、可正常读取的文字层。文字通常可以在 PDF 阅读器中选中、复制和搜索

扫描页

页面主体是一张扫描图片，没有可靠的正文文字层。人可以看到图片中的文字，但普通 PDF 文字提取工具无法直接读取。

有些扫描页可能带有页码或水印文字层，因此不能仅根据“是否提取到文字”判断，还需要结合有效正文数量和图片覆盖情况。

混合页

页面既包含可以直接提取的正文，又包含图片中的重要文字或信息。

例如正文是原生文字，但页面中插入了带文字的软件截图。普通装饰图片或 Logo 不会使页面自动成为混合页，关键是图片中是否存在需要提取的信息。

复杂布局页

页面包含多栏、表单、复杂表格、重要图表、公式或大量图文混排内容。虽然可能存在文字层，但普通提取方式容易打乱阅读顺序，或者丢失表格、图片和公式所表达的信息。

异常页

页面出现文字乱码、解析报错、内容为空，或者无法可靠判断属于上述哪种类型。

页面分类不一定百分之百准确。规则能够明确判断的页面自动处理，判断不清的页面交给更强的解析工具、多模态模型或人工复核。

16.5.2.4 步骤三：按页面类型解析

第二步得到页面类型后，第三步与其逐项对应。

处理原生文本页

使用 **PyMuPDF** 提取页面文字及基本位置信息。如果还需要识别标题、正文、列表等文档结构，可以使用 **Unstructured、LlamaParse** 等工具。

PyMuPDF 更擅长快速获取 PDF 中已有的文字、图片和坐标；**Unstructured、LlamaParse** 更擅长将内容整理成标题、正文、列表和表格等结构化元素。

处理扫描页

先将 PDF 页面渲染成图片，再使用 OCR 识别图片中的文字。中文文档应选择支持中文的 OCR 模型。

如果扫描质量较差，可以先进行旋转校正、去噪、倾斜校正或对比度增强，再执行 OCR。OCR 结果应保留页码，低质量页面需要重新识别或人工检查。

处理混合页

混合页需要同时使用两种方式：

1. 直接提取页面中已有的原生文字。
2. 提取或截取包含重要信息的图片区域。
3. 对这些图片区域执行 OCR，或者使用多模态模型理解图表内容。
4. 合并原生文字和图片识别结果，并删除重复内容。

PyMuPDF 可以发现和提取图片，但通常不能理解图片内容；OCR 负责识别图片中的文字，多模态模型可用于理解图表、流程图等更复杂的视觉信息。

处理复杂布局页

复杂布局页可以先交给 Unstructured、LlamaParse 等工具进行版面解析。这些工具会识别标题、正文、栏位、表格和图片区域，并返回元素类型、页面位置和大致阅读顺序。程序再按照这些结果组合内容，不应让大模型随意改写原文。

具体处理方法如下：

- **分栏页面**：使用 Unstructured、LlamaParse 或布局模型识别左右栏及各文本块的位置，按照“先读完一栏，再读取下一栏”的顺序重新排列文本。工具给出的顺序不正确时，可以根据文本块坐标补充排序规则。
- **表格**：原生 PDF 中边界较清楚的表格可以使用 Camelot 提取；扫描表格先通过 OCR 或表格识别模型识别单元格；图文混排文档也可以直接使用 Unstructured 或 LlamaParse 提取表格。提取结果转换成 Markdown、HTML 或 CSV，并抽查表头、合并单元格、跨页表格和关键数字。如果自动提取破坏了行列关系，则保留表格截图并进入人工复核。
- **含文字的图片**：先用 PyMuPDF 或版面解析工具确定并提取图片区域，再使用 OCR 识别其中的文字，最后按照图片在页面中的位置插回正文。
- **流程图、架构图和统计图**：普通 OCR 只能读取图中的文字，无法完整理解节点、箭头和数据关系，因此可以将图片及附近正文交给多模态模型生成说明，并保留原图和页码以便核对。
- **公式**：使用支持公式识别的 OCR 或文档解析工具转换为 LaTeX；识别不可靠时保留公式图片，并将它与前后正文关联。

自动工具仍然无法正确恢复阅读顺序或内容结构时，将该页标记为需要人工复核。

处理异常页

- **文字乱码**：先更换一种 PDF 解析器重新提取；仍然乱码时，将页面渲染成图片并使用 OCR。
- **空白页面**：保留页码和页面记录，不输出正文。
- **解析报错或类型无法确定**：使用多模态模型辅助判断；仍无法可靠处理时进入人工复核。

无论页面采用哪种解析方式，得到文字和位置信息后，还要处理页眉、页脚、页码和水印。具体方法是比较连续页面中处于相同顶部或底部位置的文本；如果同一内容反复出现，就将其识别为页眉、页脚或水印并从正文中移除。页码可以通过固定位置和连续数字识别。处理完成后再进入统一结果阶段。

16.5.2.5 步骤四：统一结果

走过不同解析路线的页面，可能分别产生普通文字、OCR 文字或结构化元素。最后一步不再重新判断和处理页面问题，只负责把这些结果整理成统一形式：

1. 将不同工具的输出转换成相同的文本结构。
2. 按 PDF 原始页码排列所有页面。

3. 将同一页内已经确认顺序的标题、正文、列表和特殊元素组合起来。
4. 输出为 Markdown 或带页码的结构化文本。

最终结果建议至少保留：

- 原始文件名；
- PDF 页码；
- 标题和正文；
- 列表结构；
- 可以可靠恢复的表格；
- 图片或公式的说明及对应位置；
- 需要人工复核的页面标记。

无论输出成哪种格式，都应能从文本追溯到原始 PDF 页面。

16.5.2.6 步骤五：结果检查

PDF 转文本完成后，应进行一次简单验收：

- 输出页数是否与原 PDF 对应，是否漏页；
- 是否仍存在大量乱码；
- 页眉、页脚和水印是否大量残留；
- 多栏页面的阅读顺序是否正确；
- OCR 是否遗漏整页或大段内容；
- 表格的表头、行列和关键数字是否正确；
- 原生文字与 OCR 文字是否重复；
- 标记为异常的页面是否已经复核。

检查通过后，才得到可供后续系统使用的最终文本。

16.5.2.7 讲解视频

https://www.bilibili.com/video/BV1gF8Z6WEGW/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

16.5.3 切割

分块策略，在RAG这个面试模块里，**算是比较高频的考点了**，在25年的12月份，我去面试的时候，很多公司就问过，到今年的2026年7月，宇树科技Agent开发一面面试，依旧在问RAG切块的策略。 本篇笔记重构于

2026年8月份，我查阅了一些数据，彻底将这一部分重构了。同时加入了若干面试真题，放在该小节最后，同时也录制了讲解视频，我希望达到的目标是：

1. 即使你没有RAG切块方面的前置知识基础，你看了本篇笔记对应的讲解视频，能够理解这些切块的基本思路，不同方法是干什么，选型策略。
2. 结合对应的笔记，能够进一步巩固和背诵相关知识点。
3. 结合章节后面的面试真题，彻底让你在面试中，对于RAG切块问题能够胸有成竹。 今本上本小节所有的内容，都是面试考察过的内容，对应着一些面试问题的回答思路，该小节的内容还是值得大家背诵一下的。

16.5.3.1 基本概念

什么是文档分块

文档分块（Chunking）是指把一篇长文档拆分成多个较小且相对完整的文本块（Chunk），再分别进行向量化、索引和检索。在 RAG 系统中，分块连接了文档解析、向量检索和大模型生成，是影响召回质量的关键环节。

一个理想的 Chunk 应同时满足以下要求：

- 长度不超过嵌入模型和大语言模型的输入限制；
- 块内尽量只表达一个完整、集中的主题；
- 脱离原文后仍具有足够的上下文，可以被独立理解；
- 保留标题路径、页码、来源和父文档 ID 等元数据，便于追溯原文。

为什么需要分块

第一，**满足大语言模型的上下文窗口限制。**直接输入整篇长文档可能超过模型的最大上下文长度。分块后只把与问题相关的内容送入模型，可以控制上下文规模。

第二，**提高生成上下文的质量。**召回内容过多、过长会增加 Token 成本，并产生“Lost in the Middle”问题，使真正有用的信息被大量无关内容掩盖。

第三，**满足嵌入模型的输入限制。**嵌入模型通常存在最大 Token 数，超出后可能被截断。例如部分 BGE 或 Sentence-Transformers 模型的上限为 512 Tokens，过长文本会造成后半部分语义丢失。

第四，**提高检索精度。**一个文本块如果同时包含多个主题，其向量表示会被不同语义平均和稀释，用户查询某个细节时不容易准确召回。较小且主题集中的文本块通常更容易与查询匹配。

16.5.3.2 常见分块策略

16.5.3.2.1 固定长度分块

固定长度分块按照固定字符数或 Token 数切分文档，并可设置 Overlap，使相邻块保留一部分重复内容。

LlamaIndex 中可以使用 `TokenTextSplitter`：

```
from llama_index.core.node_parser import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
)
nodes = splitter.get_nodes_from_documents(documents)
```

例如，将一篇 1500 Tokens 的文档按照 500 Tokens 切分：

```Plain Text 输入文档： [1 ..... 1500 Tokens]

输出： Chunk 1：[1----------500] Chunk 2： [501--------1000] Chunk 3： [1001--------1500]
```

```
为了避免关键信息恰好落在两个块的边界，可以让相邻块保留部分重复内容：

```Plain Text
Chunk 1: [--------------------]
Chunk 2:                  [--------------------]
                          ↑ Overlap
```
```

优点：实现简单、速度快、块大小稳定，便于控制索引规模和成本，适合作为快速验证 RAG 流程的基线。

缺点：可能从句子或段落中间切断，不理解文档结构和语义；Overlap 过大还会导致索引冗余和重复召回。

适用场景：文本结构不明显、需要快速建立基线、对切分边界要求不高的场景。

16.5.3.2.2 递归分块

递归分块按照一组从粗到细的分隔符逐级切分。它通常先尝试段落，再尝试句子、标点、空格，最后使用字符或 Token 强制切分作为兜底。

```Plain Text 段落 → 换行 → 句子 → 标点 → 空格 → 字符或 Token
```

```
LlamaIndex 中可以使用 `SentenceSplitter`：

```Python
from llama_index.core.node_parser import SentenceSplitter

splitter = SentenceSplitter(
    chunk_size=500,
    chunk_overlap=50,
)
nodes = splitter.get_nodes_from_documents(documents)
```
```

LangChain 中的对应工具是 `RecursiveCharacterTextSplitter`。两者名称不同，但都具有从自然边界到细粒度边界逐级切分的思想。

递归分块的完整过程如下：

1. 先按照段落切分；
2. 检查每个片段是否超过 `chunk_size`；
3. 如果某个段落仍然太长，再按照句子切分；
4. 如果句子仍然太长，再按照标点、空格等更细的边界切分；
5. 最后使用字符或 Token 强制切分作为兜底；
6. 将较小片段重新合并成接近目标长度的 Chunk，并保留适量 Overlap。

```Plain Text 输入：一篇长文档 ↓ 按段落切 [段落 A] [超长段落 B] [段落 C] ↓ 按句子继续切 [句 1] [句 2] [超长句 3] ↓ 按标点或字符切 [片段 3.1] [片段 3.2]

> 递归分块器本身通常不真正识别标题。标题层级需要由前面的文档解析器提取，再作为结构边界或元数据传给分块器。如果输入没有段落换行和正常标点，递归分块仍能运行，但会逐步退化为字符或 Token 级强制切分，只能保证长度不超限，不能保证**语义边界**正确。
>
> **优点**：尽量保留段落和句子的完整性；速度快，不需要调用模型；是普通 RAG 项目中最常用的分块方式。
>
> **缺点**：依赖文本解析和清洗质量；OCR 错误换行可能造成误切；它并不真正理解标题和主题变化，输入没有正常换行或标点时会退化为固定长度切分。
>
> **适用场景**：普通文章、说明书、制度和技术文档，以及已经完成基本清洗的非结构化文本。
>
> ![](images/AyZ1b9XRLoWhgWxu2a0c8HYbnng.png)
>
> <a id="doxcnz9eGOGWdrvcaqjpaqJFfbc"></a>
> ###### 16.5.3.2.3 结构化分块
>
> **结构化分块**利用文件本身的组织结构确定边界，而不是只根据长度切分。
>
> - **Markdown**：按照标题、段落和代码块切分；
> - **HTML**：按照 `h1`～`h6`、`p`、`li`、`table` 等标签切分；
> - **DOCX**：按照 Heading 样式、段落、列表和表格切分；
> - **代码**：按照文件、类、函数和方法切分；
> - **PDF/PPT**：按照标题、正文、列表、表格、图片和页面区域切分。
>
> LlamaIndex 提供了 `MarkdownNodeParser`、`HTMLNodeParser`、`JSONNodeParser`、`CodeSplitter` 等解析器。例如：
>
> ```Python
> from llama_index.core.node_parser import MarkdownNodeParser
>
> parser = MarkdownNodeParser()
> nodes = parser.get_nodes_from_documents(documents)
```

结构化分块的典型处理流程如下：

```Plain Text 原始文件 → 文档解析或版面分析 → 识别标题、正文、表格、列表和代码等结构单元 → 在超长结构单元内部继续递归切分 → 生成 Chunks，并保存标题路径、页码和来源等元数据
```

> Reader 负责读取和提取文件内容，NodeParser 负责形成最终用于索引的节点。对于复杂 PDF，仅提取纯文本通常无法保留正确阅读顺序，需要结合字号、坐标、页面区域和版面模型识别标题、分栏、表格与图片；扫描 PDF 还需要先进行 OCR。
>
> 如果一个章节仍然过长，可以先按标题进行结构化分块，再在章节内部使用**递归分块**。因此，结构化分块和**递归分块**经常组合使用。
>
> **优点**：能够保留标题与正文、表头与数据、函数与代码之间的关系，结果更容易引用和追溯。
>
> **缺点**：不同文件需要不同解析器；复杂 PDF 需要根据字号、坐标和版面推断结构；解析错误会直接影响后续分块质量。
>
> **适用场景**：Markdown、HTML、Word、代码，以及包含标题、列表和表格的 PDF/PPT。
>
>
> ###### 16.5.3.2.4 语义分块
>
> **语义分块**根据主题或语义变化确定边界。常见实现会先切句，再使用嵌入模型计算相邻句子或**句子窗口**的语义距离，在语义突变的位置设置断点，并将连续且语义相近的句子合并成一个 Chunk。
>
> LlamaIndex 中可以使用 `SemanticSplitterNodeParser`：
>
> ```Python
> from llama_index.core.node_parser import SemanticSplitterNodeParser
>
> splitter = SemanticSplitterNodeParser(
>     embed_model=embed_model,
>     breakpoint_percentile_threshold=85,
> )
> nodes = splitter.get_nodes_from_documents(documents)
> ```

阈值需要根据真实语料调整，数值越低通常越容易产生更多分块。**语义分块**仍需设置最大 Token 上限，避免产生过长的 Chunk。

基于 Embedding 的语义分块通常包含以下步骤：

1. 先把文档切成句子；
2. 为单个句子或**句子窗口**生成向量；
3. 计算相邻句子的语义相似度或距离；
4. 在语义差异明显的位置设置断点；
5. 将语义相近的连续句子合并为一个 Chunk；
6. 使用最大 Token 数限制 Chunk 上限。

```Plain Text 句 1 ──相似── 句 2 ──相似── 句 3 ‖语义突变‖ 句 4 ──相似── 句 5
```

输出： Chunk 1：句 1 + 句 2 + 句 3 Chunk 2：句 4 + 句 5

> 语义分块比**递归分块**慢，是因为**递归分块**主要执行字符串操作，而语义分块需要对大量句子运行嵌入模型、计算相似度，有些算法还需要进行多轮合并。
>
> **优点**：块内主题更一致，不完全依赖换行和格式，能够缓解语义在错误位置被切断的问题。
>
> **缺点**：需要对大量句子计算 `Embedding` 和相似度，建库速度较慢；阈值需要实验调整；块大小不固定。
>
> **适用场景**：会议纪要、访谈、长篇报告、OCR 后段落结构不可靠的文本，以及一个自然段中经常包含多个主题的内容。
>
>
> ###### 16.5.3.2.5 命题分块
>
> **命题分块**使用大语言模型把原文转换成多条能够独立理解的事实或观点。它可能拆开复合句、补全省略主语、消解代词指代，并进行少量改写，因此不仅是文本切割，也属于内容转换和索引增强。
>
> `LlamaIndex` 没有像 `SentenceSplitter` 一样通用的内置**命题分块**器。因此，命题提取逻辑需要开发者自己实现，`LlamaIndex` 主要负责承载节点和建立索引。通常需要自行组合以下流程：
>
> ```Plain Text
> 使用 SentenceSplitter 粗切原文
> → 调用 LLM 提取命题列表
> → 每条命题创建一个 TextNode
> → 在 metadata 中保存 parent_id、页码和来源
> → 使用 VectorStoreIndex 建立索引
> ```

下面是命题分块的流程伪代码。它用于说明职责划分，并不是可以直接运行的 LlamaIndex 内置 API：

```
parent_chunks = sentence_splitter.get_nodes_from_documents(documents)

proposition_nodes = []

for parent_chunk in parent_chunks:
    # 核心步骤需要自己实现：调用 LLM，把原始段落拆成命题列表
    propositions = extract_propositions_with_llm(parent_chunk.text)

    for proposition in propositions:
        proposition_node = create_llamaindex_node(
            text=proposition,
            parent_node=parent_chunk,
        )
        proposition_nodes.append(proposition_node)

index = VectorStoreIndex(proposition_nodes)
```

这里的 `extract_propositions_with_llm()` 和 `create_llamaindex_node()` 都是为了说明流程而写的自定义函数名，不是 LlamaIndex 自带方法。前者负责通过 Prompt 和 LLM 提取命题，后者负责把命题包装为 LlamaIndex 节点。

其中，“把已经提取出的命题包装成节点”可以使用 LlamaIndex 自带的 `TextNode`：

```
from llama_index.core.schema import TextNode

nodes = [
    TextNode(
        text=proposition,
        metadata={
            "parent_id": parent_id,
            "source": source,
            "page": page,
        },
    )
    for proposition in propositions
]
```

这段 `TextNode` 代码本身不负责切分，也不会自动生成命题。它成立的前提是业务代码已经完成以下工作：

1. 调用 LLM 分析原始段落；
2. 得到 `propositions` 命题列表；
3. 记录每组命题对应的原始段落 ID；
4. 再把每条命题保存为 `TextNode`。

`text` 和 `metadata` 是 `TextNode` 提供的字段；`parent_id`、`source` 和 `page` 是项目自定义的 metadata 键，并不是 LlamaIndex 自动生成的属性。如果需要使用 LlamaIndex 原生的父子节点关系，也可以通过节点的 `relationships` 字段保存父节点关系。

例如，原文为：

```Plain Text
张三于 2024 年加入公司，并担任技术负责人。 他此前在 A 公司工作了五年。
```

经过命题转换后可以得到：

```Plain Text
1. 张三于 2024 年加入当前公司。
2. 张三担任当前公司的技术负责人。
3. 张三此前在 A 公司工作了五年。
```

这里不仅拆开了复合句，还把“他”还原成了“张三”，使每条事实脱离原段落后仍能独立参与检索。

命题文本适合生成向量和参与检索，但不应直接替代原文作为最终事实依据。系统应同时保存命题与原文的映射，回答和引用时通过 `parent_id` 回到原始段落。

推荐同时保存以下信息：

- **命题文本**：用于生成向量和检索；
- **原始段落**：用于回答、引用和事实核对；
- `parent_id`：连接命题与原始段落；
- **页码和来源**：用于定位和追溯原文。

检索到命题后，再根据 `parent_id` 找到对应原始段落，并把原文而不是孤立命题交给大模型生成答案。

优点：每个检索单元只表达一个事实，粒度精细；能够补全指代，适合事实型问答。

缺点：需要调用 LLM，速度慢、成本高；模型可能遗漏、误解或错误改写事实。

适用场景：事实密集的企业知识库，以及人物、产品属性、事件和关系检索。

16.5.3.3 分块选型技巧和原则

16.5.3.3.1 核心原则：先选主策略，再调分块粒度

分块选型不是从多种方法中随意挑选，也不是把所有方法同时使用，而是按照一条明确的决策链进行：

```Plain Text 判断文档是否具有可靠结构 → 选择主分块策略 → 根据检索目标调整 Chunk 粒度 → 使用 Overlap 或上下文扩充解决边界问题 → 根据召回结果继续调优 → 通过评测确定最终方案
```

> 其中，文档结构决定“在哪里切”，检索目标决定“切多大”，Overlap、**句子窗口**和父子块决定“切完后如何补充上下文”，评测负责验证整个方案是否有效。几类因素承担的作用不同，不应混为同一级别的选型条件。
>
>
>
>
> ###### 16.5.3.3.2 第一步：根据文档结构选择主分块策略
>
> 首先判断文档是否存在可靠的自然边界。
>
> 如果文档结构清晰，应优先尊重原始结构：
>
> - **Markdown、HTML、Word**：先按照标题、章节、段落等结构切分；
> - **代码**：按照文件、类、函数和方法切分；
> - **PDF/PPT**：先完成版面解析，再按照标题、正文、列表和表格等结构切分；
> - **FAQ、商品资料、人物信息等强结构化数据**：一条完整记录本身就是一个语义实体，可以直接作为一个 Chunk，不必强行拆分。
>
> 如果结构单元仍然过长，再在结构单元内部使用**递归分块**。
>
> 如果文档缺少可靠结构，则根据文本质量选择：
>
> - **普通文章、说明书、制度**：使用**递归分块**；
> - **OCR 错乱、换行不可靠但句子语义仍可识别**：先清洗文本，再考虑递归分块或**语义分块**；
> - **会议纪要、访谈、长篇报告等主题频繁变化的文本**：考虑**语义分块**；

> - 事实高度密集，并且需要检索独立人物、属性或关系：考虑**命题分块**，同时保留命题与原文的映射。
>
> 因此，**结构化分块**、递归分块、语义分块和**命题分块**不是简单的优劣关系，而是分别解决不同的数据形态问题。
>
>
> ###### 16.5.3.3.3 第二步：根据检索目标确定 Chunk 粒度
>
> 确定主分块策略后，再根据用户问题所需的证据粒度决定 Chunk 大小。
>
> - **只需要定位相关文档或章节**：Chunk 可以相对大，以保留完整上下文；
> - **需要定位具体条款、步骤、句子或事实**：Chunk 应相对小，以提高**检索精度**；
> - **用户问题通常对应一个完整段落**：让 Chunk 尽量覆盖一个自然段落或语义单元；
> - **一个 Chunk 中经常包含多个主题**：说明粒度过大，应减小 Chunk，或者使用语义分块寻找主题边界；
> - **检索结果很准确，但回答缺少前后文**：说明检索粒度合适、生成上下文不足，不应直接增大所有 Chunk，而应使用**句子窗口**、父子块或**层级索引**扩充上下文。
>
> 理想状态不是 Chunk 越小或越大越好，而是一个 Chunk 尽量对应用户问题所需要的最小完整证据单元。
>
>
> ###### 16.5.3.3.4 第三步：使用默认方案建立实验基线
>
> 当项目处于早期阶段，尚未积累足够的真实问题和评测数据时，可以先采用一个低成本、容易解释的基线方案：
>
> ```Plain Text
> 结构化解析
> → 在超长结构单元内部递归分块
> → 设置有限 Overlap
> → 保存标题、页码和来源等元数据
> ```

初始实验可以从约 400 ~ 600 Tokens 的 `chunk_size` 和 10% ~ 20% 的 Overlap 开始。

默认方案不是最终选型原则，也不与“根据文档类型选择”冲突。它只是在信息不足时提供一个可运行的起点。获得真实召回结果后，仍然要根据文档特点和问题类型进行调整。

16.5.3.3.5 第四步：根据召回问题定向调整

运行基线方案后，应根据实际问题判断需要修改哪一层，而不是盲目调大或调小 Chunk。

- **正确内容经常召回不到**：Chunk 可能过大或结构边界不合理，应减小 Chunk、改善结构解析，或调整检索方式；
- **已召回正确内容，但上下文不完整**：适当增加 Overlap，或使用句子窗口、父子块扩充上下文；
- **一个 Chunk 中经常存在多个主题**：减小 Chunk，或改用语义分块；
- **Top-K 结果高度重复**：减少 Overlap，并对召回结果去重；
- **标题与正文经常分离**：改善结构解析，并把标题路径写入 Chunk 元数据；
- **表格问题效果差**：将表头与数据行绑定，按照完整记录或逻辑区域分块；
- **OCR 文档切分混乱**：先清理页眉页脚、水印和错误换行，再调整分块策略。

这一步的关键是先定位问题属于“解析错误、边界错误、粒度错误还是上下文不足”，再采取对应措施。

16.5.3.3.6 第五步：通过评测确定最终方案

不存在适用于所有项目的统一 Chunk Size。最终方案应由真实问题的评测结果决定。

可以准备一批真实查询，并标注每个问题对应的正确证据，分别测试不同的主策略、Chunk Size 和 Overlap。重点观察：

- 正确证据是否出现在 Top-K 中；
- 正确证据的排序是否靠前；
- 回答是否得到召回证据支持；
- 页码、章节和来源引用是否正确；
- 检索结果的重复率；
- 平均上下文 Token 数；
- 索引规模、建库时间和查询延迟。

最终选择的方案，应是在真实语料和真实问题上，**检索精度**、**上下文完整性**、成本与延迟之间取得最好平衡的方案。

16.5.3.4 常见面试问题

16.5.3.4.1 常见的分块策略有哪些？各自有什么优缺点？

💡 答题思路

这个问题并**不是出自真实面试题**，而是我为了帮助大家掌握基础知识主动添加的。现实面试中通常不会直接这样问，因为这种问法比较固定，也比较偏向概念背诵。

但我仍然把它放在这里，是希望大家能够有意识地记住常见分块策略，以及每种策略的基本原理、优缺点和适用场景。后面可以看到，真实面试中的问题通常会比这道题更深入，例如追问项目为什么选择某种策略、内容跨块怎么办、Chunk Size 如何确定，以及分块效果如何评测。

这些真实问题虽然问法不同，但回答的思路和出发点仍然来自本题涉及的理论基础。只有先掌握不同分块策略的特点，后面才能根据文档类型和实际问题完成选型与调优。（参考答案有点长，面试可以稍微简化一点）

📝 参考答案

RAG 中常见的分块策略主要包括**固定长度分块**、**递归分块**、**结构化分块**、**语义分块**和**命题分块**。

第一种是**固定长度分块**，也就是按照固定的字符数或 Token 数切分文档。它的优点是实现简单、速度快、块大小稳定，便于控制索引规模和成本；缺点是不理解文档结构和语义，可能把一个完整句子、段落或步骤从中间切断。因此，它适合快速建立基线，或者处理结构不明显且对**语义边界**要求不高的文本。

第二种是**递归分块**。它会按照从粗到细的分隔符逐级尝试切分，例如先按段落，再按换行、句子、标点、空格切分，最后才使用字符或 Token 强制切分。它的优点是速度快、不需要调用模型，并且能尽量保留自然**语义**

边界；缺点是依赖文本解析和清洗质量，遇到 OCR 错误换行、标点缺失或格式混乱时，仍然可能产生不合理的分块。它是普通文章、说明书和技术文档中最常用的基线方案。

第三种是**结构化分块**，也就是利用文档自身的结构进行切分，例如 Markdown 标题、HTML 标签、Word Heading、代码中的类和函数，以及 PDF 中识别出的标题、正文和表格。它的优点是能够保留标题与正文、表头与数据、函数与代码之间的关系，结果也更容易引用和追溯；缺点是依赖文档解析器，不同文件需要不同的处理方式，复杂 PDF 或扫描件还需要版面分析和 OCR。实际项目中通常先进行结构化分块，如果某个结构单元仍然过长，再在内部使用递归分块。

第四种是**语义分块**。它通常先把文档切成句子，再使用 Embedding 计算相邻句子的语义相似度，在主题发生明显变化的位置建立边界。它的优点是块内主题更加集中，不完全依赖换行和格式；缺点是需要额外进行向量计算，建库速度较慢，分块阈值也需要通过实验调整。它适合会议纪要、访谈和主题变化频繁的长文本。

第五种是**命题分块**。它使用大语言模型把原文转换成多条能够独立理解的事实或观点，例如拆开复合句、补全省略的主语并消解代词指代。它的优点是检索粒度非常精细，适合人物、属性和事实关系检索；缺点是需要调用 LLM，成本和延迟较高，而且模型可能遗漏或错误改写事实。因此，命题不能直接替代原文，通常需要通过 `parent_id` 保留命题与原始段落的映射。

此外，Overlap、**句子窗口**和父子块不完全属于与上述方法并列的主分块策略，而是用于缓解边界信息丢失和上下文不足的辅助机制。Overlap 让相邻 Chunk 保留部分重复内容；**句子窗口**和父子块则通过“**小块检索、大块生成**”同时兼顾**检索精度**和**上下文完整性**。

总的来说，固定长度分块适合快速建立基线，递归分块适合普通非结构化文本，结构化分块适合结构清晰的文档，语义分块适合主题边界不稳定的文本，命题分块适合高精度事实检索。实际项目中通常不是只使用一种方法，而是先尊重文档结构，再在超长结构单元内部递归切分，并结合有限的 Overlap 或上下文扩充策略。

16.5.3.4.2 你的项目中是如何确定分块策略的?

💡 答题思路

这道题的理论基础就是 `5.3.3 分块选型技巧和原则`。 但 `5.3.3` 讲的是通用方法论，真正回答这道题时，需要把方法论落到自己的项目中，答得具体一些，让面试官能够看出你确实理解项目中的数据、问题和技术取舍，而不是只会背诵几种分块方法。

我知道，大部分人在个人项目中不一定会把每一个技术选型都做得如此规范。 即使在真实企业项目中，也不是所有选型都会进行非常完整的实验。因此不需要因为自己的项目没有覆盖全部流程而怀疑自己，但需要理解每一步为什么这样设计，并提前结合自己的项目组织答案。

这里需要灵活运用理论和项目表达技巧。 所谓“包装”，不是捏造不存在的数据或结果，而是把自己已经做过的方案、遇到的问题、能够解释的取舍，以及后续可以采用的验证方式组织成一条完整逻辑。没有实际执行过的实验，可以表述为后续验证方案，不要把计划说成已经取得的结果。

面试前最好针对自己的文档类型、分块方式、Chunk Size、Overlap、异常情况和评测方法准备一版项目化答案。 但也不可能预先准备所有追问。面对新的问题时，需要以扎实的理论基础为底层依据，再结合项目场

景灵活分析和随机应变。

下面以“企业内部技术文档知识库”为例组织答案。 回答时可以按照一条标准流程展开：先介绍文档特点，再说明如何选择主分块策略，然后解释参数如何建立基线、遇到了什么问题、如何定向调整，最后说明怎样通过评测确定最终方案。 （参考答案有点长，面试可以稍微简化一点）

参考答案

我们做的是一个企业内部技术文档知识库，主要处理 Markdown、Word 和 PDF 格式的 API 文档、架构设计文档、部署手册和故障复盘。 我们没有直接对所有文档使用固定长度切分，而是先分析文档结构和实际查询目标，再选择分块方案。

第一步是根据文档结构选择主策略。 这类技术文档通常具有比较明确的标题、章节、段落、代码块和接口说明，因此我们先进行结构化解析。Markdown 和 Word 按照标题层级解析；PDF 先转换成能够保留标题和段落关系的结构化文本，同时清理页眉、页脚和错误换行。这样可以避免把不同章节或不同接口的内容混在同一个 Chunk 中。

但是，只按标题切分还不够，因为有些章节可能非常长。 所以在每个结构单元内部，我们继续使用 **递归分块**，优先按照段落、换行、句子和标点寻找边界，只有内容仍然超过长度限制时，才使用 Token 长度进行兜底切分。最终采用的是“ **结构化解析加递归分块**，而不是单纯固定长度切分。

第二步是根据检索目标确定粒度。 我们的用户通常会查询某个接口的参数、配置项、部署步骤或故障处理方法，需要定位到比较具体的证据，因此 Chunk 不能太大；但技术说明又经常包含前置条件、操作步骤和注意事项，Chunk 太小会导致上下文不完整。因此我们的目标是让一个 Chunk 尽量覆盖一个完整的接口说明、操作步骤或自然段落。

第三步是建立参数基线。 项目早期我们先以大约 500 Tokens 的 `chunk_size` 和 10%～20% 的 `chunk_overlap` 作为起点，并保留文档 ID、标题路径、页码、Chunk 序号等元数据。500 Tokens 不是最终答案，只是一个兼顾检索粒度、Embedding 输入和上下文成本的初始值。

第四步是根据实际问题进行调整。 测试过程中主要发现了两类问题。第一类是接口名称、参数解释和限制条件偶尔落在相邻 Chunk 中，检索命中参数说明后，模型拿不到接口名称或前置条件。对于这种边界问题，我们保留适量 Overlap，并通过 `chunk_index` 扩充前后相邻块。

第二类问题是部分架构文档的单个章节很长，即使递归切分后，一个小块虽然容易检索，但不足以支持完整回答。 对于这种情况，我们没有盲目增大所有 Chunk，而是保留父子关系：小的子块负责生成向量和参与检索，命中后再根据 `parent_id` 返回对应的较大父段落，让系统实现“**小块检索、大块生成**”。**这样既保留了检索精度**，也补充了生成所需的上下文。

我们也评估过 **语义分块** 和 **命题分块**。** 语义分块 **需要为大量句子额外计算 Embedding，并且阈值需要针对语料调试；** 命题分块 **还需要调用 LLM 对原文进行转换，存在成本和事实改写风险。考虑到我们的技术文档结构比较稳定，** 结构化解析加递归分块**已经能够解决大部分问题，因此没有把语义分块或命题分块作为默认主链路，只把它们作为特殊文档的候选方案。

最后，分块参数需要通过真实问题验证。 我们会准备一批覆盖接口查询、部署步骤、故障原因和配置说明的测试问题，分别比较不同 Chunk Size 和 Overlap 下的 Hit Rate、正确证据排序、**上下文完整性**、召回重复率、Token 成本和查询延迟。如果小块方案召回更准确但上下文不足，就优先使用父子块或相邻块扩充；如果大块中经常混入多个主题，就减小 Chunk。最终选择的是评测结果最均衡的方案，而不是直接套用某个固定参数。

所以，我们的整体选型逻辑可以概括为：先根据技术文档的结构选择“结构化解析加递归分块”，再根据查询目标设置初始粒度，通过 Overlap 和父子块解决边界与上下文问题，最后使用真实问题评测确定最终参数。 这个过程中的每一步都有对应的数据特点和问题依据。

16.5.3.4.3 RAG 的 Chunking 策略如何设计？遇到过哪些问题？如何调整？

选自 宇树科技 Agent 开发二面面试真题。

💡 答题思路

这道题比“项目中使用了什么分块策略”更进一步。 面试官不仅要听最终方案，还希望看到完整的问题解决过程。建议按照“初始设计—暴露问题—定位原因—定向调整—评测验证”回答。

回答时不要只罗列技术名词。 每一个调整都要对应一个实际问题，例如语义被切断、一个 Chunk 混入多个主题、召回准确但上下文不足、结果高度重复、标题与正文分离等。这样才能体现自己具备真实的调试和迭代经验。

📝 参考答案

我们的知识库主要处理企业内部技术文档。 初始方案是先保留标题和章节结构，再在超长章节内部使用 **递归分块**，以 500 Tokens 左右的 Chunk 和约 15% 的 Overlap 建立基线，同时保存标题路径、页码、`source_id` 和 `chunk_index`。

第一轮测试发现，部分接口名称、参数说明和限制条件被分到相邻 Chunk 中。 检索虽然命中了参数说明，但模型不知道它属于哪个接口。我们先改善结构解析，把标题路径补充到每个 Chunk；然后保留适量 Overlap，降低边界信息丢失的概率。

第二个问题是小块检索比较准确，但回答复杂部署流程时上下文不够。 这个问题不适合简单增大所有 Chunk，因为大块会混入多个主题，降低向量匹配精度。我们给 Chunk 保存父子关系和顺序信息，使用子块检索，命中后根据 `parent_id` 返回父段落，或者根据 `chunk_index` 补充前后相邻块，实现“**小块检索、大块生成**”。

第三个问题是增加 Overlap 后，Top-K 中经常出现内容高度相似的相邻块。 我们把 Overlap 控制在 10%～20%，并在召回后根据来源和文本相似度去重，避免重复内容占满上下文。

调整过程中，我们不会只看最终答案，而是分别观察正确证据能否进入 Top-K、正确证据的排序、召回重复率、输入 Token 数和查询延迟。 通过对比不同 Chunk Size、Overlap 和上下文扩充方案，选择综合效果最好的配置。

整体来说，我们先用 **结构化解析加递归分块** 建立基线，再根据边界丢失、上下文不足和重复召回等具体问题，分别使用标题元数据、Overlap、父子块和结果去重进行调整，最后通过真实问题集验证效果。

16.5.3.4.4 手写算法：按照句子边界进行文本分块，避免截断句子

选自 宇树科技 Agent 开发二面面试真题。

💡 答题思路

就是手撕代码题目，如果以力扣的标准来判断这个题目，这个题算不上一个难题，最多是个中等的题目。 不过就是没法像力扣一样，可以期待能碰到原题。

📝 参考答案

我会先使用中英文句末标点切句，然后按顺序合并句子。 只要加入下一句后不超过最大长度，就继续累加；如果超过，就先保存当前 Chunk，再从下一句开始构建新 Chunk。这样正常情况下不会从句子中间截断。

```
import re


def split_by_sentence(text: str, max_length: int = 500) -> list[str]:
    sentences = re.split(r"(?<=[。！？.!?])\s*", text.strip())
    chunks = []
    current = ""for sentence in sentences:
        if not sentence:
            continue# 单个句子已经超过上限，只能使用长度切分兜底if len(sentence) > max_length:
            if current:
                chunks.append(current)
                current = ""for start in range(0, len(sentence), max_length):
                chunks.append(sentence[start:start + max_length])
            continueif len(current) + len(sentence) <= max_length:
            current += sentence
        else:
            if current:
                chunks.append(current)
            current = sentence

    if current:
        chunks.append(current)

    return chunks
```

这段实现保证普通句子不会被截断，并对超长单句进行了兜底。 真实项目中还可以把 `len()` 替换为 Tokenizer 的计数函数，并进一步处理小数点、英文缩写、代码和列表等特殊情况。

16.5.3.4.5 技术文档的相关内容跨越多个 Chunk，应该如何优化？什么时候需要扩充上下文？

选自 2025.11 网龙科技一面面试真题。

💡 答题思路

这是一组连续追问。 第一问考察如何解决跨块问题，第二问考察是否知道上下文扩充的使用条件。回答时先区分“检索不到正确位置”和“已经检索正确但上下文不足”：前者需要调整分块或检索，后者才适合扩充上下文。

📝 参考答案

技术文档中的接口名称、参数、前置条件和限制说明可能跨越多个 Chunk。 我们会先通过结构化解析和 **递归分块** 尽量保留自然边界，并设置有限的 Overlap，降低内容在边界处完全断开的风险。

如果仍然存在跨块问题，我们会给每个 Chunk 保存 `source_id`、`chunk_index` 和 `parent_id`。 检索命中某个子块后，可以根据 `chunk_index` 补充同一文档中的前后相邻块，或者根据 `parent_id` 返回包含它的完整父段落。

上下文扩充主要适用于“短文本容易检索，但不足以支持完整回答”的情况。 例如，已经准确命中了某个配置参数，但回答还需要它的适用条件和异常说明；或者命中了流程中的一个步骤，但问题要求解释完整流程。这时适合采用小块检索、相邻块或父块扩充。

如果正确内容根本没有进入 Top-K，直接扩充上下文没有意义，应先检查 Chunk 是否过大、**语义边界** 是否错误、查询表达是否需要改写，以及检索策略是否合适。扩充时还要限制同源范围、扩充数量和总 Token 预算，并进行去重，防止引入过多无关内容。

16.5.3.4.6 你的文档分块会使用滑动窗口吗？滑窗长度一般是多少？

选自 2025.11 网龙科技一面面试真题。

💡 答题思路

面试语境中的“滑窗长度”通常是在追问相邻 Chunk 的重叠长度，也就是 `chunk_overlap`。 回答时最好同时说明窗口大小、滑动步长和 Overlap 的关系，并区分严格的固定滑窗与 **递归分块** 中的重叠机制。

📝 参考答案

我们使用 **递归分块**，并为相邻 Chunk 设置了重叠区域。严格来说，它不是完全按照固定步长进行机械滑窗，而是优先在段落、句子和标点等自然边界切分，然后保留适量 `chunk_overlap`。

在严格的固定滑窗中，窗口大小就是 `chunk_size`，相邻窗口重复的长度是 `chunk_overlap`，滑动步长等于 `chunk_size - chunk_overlap`。 例如窗口大小为 500 Tokens、Overlap 为 100 Tokens，那么每次向后移动约 400 Tokens。

项目中通常可以先把 Overlap 设置为 Chunk Size 的 10%～20%。 设置重叠是为了避免接口名、条件或半句话恰好落在边界处而丢失；但 Overlap 过大会增加索引规模，并导致 Top-K 结果高度重复。因此最终比例需要结合边界信息丢失率、召回重复率和 Token 成本进行调整。

16.5.3.4.7 你的系统中有没有使用更高级的召回或分块方案?

选自 2025.11 网龙科技一面面试真题。

💡 答题思路

虽然面试官问有没有更高级的技巧，但是建议也不要特别盲目地说或者编造。 实在不行就讲我们的策略是基于这个科学的方法论驱动的，把这个5.3.3 的方法论再讲一遍。如果想要将一些高级的技巧，比如说根据子块检索，召回父块这些策略，一定要找到明确的需求出发点，说明选择这个技术是合理的。能现场包装当然是更好的，但是如果包装露馅无法自圆其说，还不如不包装。这块是我自己面试的素材，我当时估计回答的就是没有，最后也通过了，面试官对我评价还挺高的，有可能这个也是压力测试，不要因为面试官这么追问就盲目现场包装，然后露馅，反而得不偿失。

📝 参考答案

我们的默认主链路仍然是 **结构化解析加递归分块**，因为企业技术文档结构比较稳定，这套方案成本低、速度快，也容易解释和调试。在此基础上，我们针对不同问题设计了几种增强方式。

对于“小块召回准确但上下文不足”的问题，我们使用 **父子块检索**：子块负责 Embedding 和匹配，命中后通过 `parent_id` 返回完整父段落。对于只需要补充局部上下文的场景，则使用 ** 句子窗口**或 `chunk_index` 扩充前后相邻内容。

对于主题切换频繁、自然段落不可靠的会议纪要，可以评估 **语义分块**，根据相邻句子的向量距离确定主题断点。对于事实密集且需要精确检索人物、属性和关系的数据，可以评估 ** 命题分块**，但需要保留命题与原文映射，避免把 LLM 改写结果直接作为事实依据。

此外，还可以建立 **层级索引**：先定位文档或章节，再在命中的范围内进行细粒度检索。这样可以减少跨文档噪声。是否启用这些方案，要根据错误案例和评测结果决定，而不是把所有策略同时堆到系统中。

16.5.3.4.8 Embedding 切 Chunk 应该怎么做？为什么固定长度切分经常不够?

选自 202604 腾讯 AI Agent 开发暑期实习面试真题。

💡 答题思路

这是一道由两个基础问题组合而成的复合题，可以先把它拆开，再分别使用前文的知识回答。

第一问“Embedding 切 Chunk 应该怎么做”，本质上是在问“应该如何确定分块策略”，对应 `5.3.3 分块选型技巧和原则`**。** 回答时按照完整的选型流程展开：先分析文档结构，选择主分块策略；再根据检索目标确定 Chunk 粒度；然后设置必要的 Overlap 或上下文扩充机制；最后根据召回问题调整，并通过真实问题评测确定最终方案。

第二问“为什么固定长度切分经常不够”，本质上是在问“**固定长度分块 ** 有什么缺点”，对应** `5.3.2.1 固定长度分块`**。固定长度切分只保证长度可控，不理解标题、段落、句子和**语义边界**，因此可能切断完整信息，也可能把多个主题混在同一个 Chunk 中，最终影响 Embedding 表达和召回效果。

所以这道题不需要单独背一套新答案。 先用分块选型方法论回答“应该怎么做”，再用 **固定长度分块** 的缺点回答“为什么不够”，最后把两部分串联成“结构或语义优先、长度约束兜底、实验评测定型”的完整结论。

📝 参考答案

Embedding 切 Chunk 时，我不会只使用固定长度切分，而是先按照分块选型流程确定方案。

首先看文档是否具有可靠结构。 如果是 FAQ、商品信息或人物资料等强结构化数据，一条完整记录本身就是一个语义实体，可以直接作为一个 Chunk；如果是 Markdown、Word、HTML 或技术文档，则先按照标题、章节和段落进行 **结构化分块**，再在过长的结构单元内部使用 ** 递归分块**；如果文本结构不可靠、主题切换又比较频繁，可以进一步评估 ** 语义分块**。

确定主策略后，还要根据检索目标决定 Chunk 粒度。 需要定位具体条款、参数或事实时，Chunk 应相对小；需要回答完整流程或复杂原因时，则需要更多上下文。对于“小块检索准确但上下文不足”的情况，我不会直接增大所有 Chunk，而是通过 Overlap、**句子窗口 、相邻块扩充或父子块实现 “ 小块检索、大块生成”** 。

Chunk Size 可以先设置一个实验基线，例如 400～600 Tokens，并设置约 10%～20% 的 Overlap。 但这只是起点，最终还要比较不同参数下的 Hit Rate、正确证据排序、召回重复率、上下文 Token 数和查询延迟。

固定长度切分之所以经常不够，是因为它只考虑长度，不理解文档结构和 **语义边界**。一个定义、操作步骤、接口说明或实体资料可能被从中间切断，使单个 Chunk 只包含残缺语义；也可能把多个不相关主题放进同一个 Chunk，使生成的向量变成多个主题的“平均语义”，降低具体问题的匹配精度。

因此，长度应该作为 Chunk 的上限约束和最终兜底，而不应该是唯一的切分依据。 更合理的方案是优先保留文档结构和自然语义边界，在必要时设置 Overlap 或扩充上下文，最后通过真实问题评测确定最终分块策略。

16.5.3.4.9 硬切分在实际场景中有哪些弊端?

选自 202604 腾讯 AI Agent 开发暑期实习面试真题。

💡 答题思路

这道题翻译过来，其实就是在问“**固定长度分块** 有什么缺点”，对应前文 `5.3.2.1 固定长度分块` 中讲过的内容。

面试中遇到这种题目，可以先把它转换成自己已经掌握的基础知识：硬切分只根据字符数、Token 数、页码或行数确定边界，本质上就是没有考虑文档结构和 **语义边界** 的固定切分。回答时直接从 **固定长度分块** 的缺点展开即可。

为了让回答更完整，可以沿着 RAG 链路组织： 首先说明它可能破坏标题、段落、句子和完整事实；然后说明残缺语义或多个主题混合会影响 Embedding 表达；接着说明这会降低召回的准确性和稳定性；最后说明系统可能需要使用 Overlap、上下文补全和 Rerank 等后处理进行补救，从而增加成本和复杂度。

📝 参考答案

硬切分最大的问题是破坏 **语义边界。**它只按照固定字符数、Token 数、页码或行数切分，不关心标题、段落、句子和逻辑结构，因此可能把一个完整概念、操作步骤、接口说明或实体资料切断。

第二，**边界信息容易丢失。**如果接口名称、错误码、配置项和适用条件落在两个 Chunk 的交界处，两个 Chunk 都可能只包含一部分证据，最终都无法与查询形成足够高的相似度。

第三，**它会影响 Embedding 的语义表示。**包含半个技术点的 Chunk 表达的是残缺语义；混入多个主题的 Chunk 表达的是被平均后的语义。这会使相关结果和不相关结果之间的相似度差距变小，导致召回不稳定。

第四，**硬切分会把问题推给后续链路。**系统不得不增加 Overlap、相邻块补全、Rerank 或 LLM 重组来修复切分阶段造成的问题，从而增加存储、Token 成本、延迟和调试复杂度。

因此，固定长度可以作为最终兜底和长度上限，但不应成为唯一边界。 更常见的做法是优先按照结构、段落和句子切分，无法满足长度约束时才进行强制切分。

16.5.3.4.10 为什么内部文档需要设置 `chunk_size=1000`？1000 是不是太大了?

选自 RAG 项目面试真题 3.9：某大厂一面

💡 答题思路

这个问题通常不是面试官凭空提出的。 更可能是候选人在介绍项目、简历或配置参数时，主动说到自己的 `chunk_size=1000`，于是面试官继续追问： **“为什么是 1000？这个值是不是太大了？”**

这道题仍然可以转换成前文已经讲过的内容。 第一部分“为什么设置 1000”，本质上是在考察 **如何确定 Chunk Size，应借鉴 `5.3.3 分块选型技巧和原则`，按照 “建立参数基线—运行真实问题—观察召回问题—调整参数—通过评测定型” 的 ** 实验驱动**方法回答。

第二部分“1000 是不是太大”，是在考察候选人的 **现场分析和灵活变通能力 。不能看到 1000 就直接回答大或不大，而要先确认它的 ** 计量单位、文档语言、自然语义粒度、Embedding 模型输入范围，以及 Top-K 上下文预算。**

面试官临时挑战简历中的参数时，需要懂得 **现场发挥 ，根据项目场景迅速找到能够支撑该参数的合理依据。例如，可以从计量单位、文档语言、 自然语义粒度 、Embedding 模型输入范围、Top-K 上下文预算和实验调优等角度组织理由。 如果 1000 指英文字符，它可能只有约 200～250 Tokens，并不算大；如果是 1000 个中文字符，就需要结合文档段落长度重新解释或调整。**

这里需要适度使用 **项目包装技巧 。即使个人项目中没有完整执行每一轮选型实验，也要把前文的方法论理解透彻，能够结合自己的项目组织出一套合理的参数选择过程。回答的重点是让面试官看到： 这个参数不是随手填写的，而是经过文档分析、基线设置、问题观察和实验调整得到的。**

📝 参考答案

我们项目中的 `chunk_size=1000` **不是适用于所有文档的固定经验值**，而是结合当前文档场景建立基线，并通过实际查询逐步调整得到的候选参数。

首先需要明确，**项目中的 1000 指字符，不是 Tokens 。我们使用的长度函数按照字符计数，知识库主要存储英文 API 文档、架构决策记录、Runbook 和故障复盘。 1000 个英文字符通常约为 200 ~ 250 Tokens**，大致对应一个完整的技术段落，所以在这个场景下并不算大。

在分块策略上，我们也不是每到 1000 个字符就强制截断，而是先保留标题和章节结构，再使用 **递归分块**，优先在段落、换行和句子边界切分。** 1000 只是单个 Chunk 的长度上限**，实际生成的 Chunk 通常不会全部达到 1000 个字符。

参数确定采用的是 **实验驱动思路 。**项目早期先使用 1000 字符建立基线，然后选择多组候选值，例如 ** 600、1000、1500 字符**，使用同一批接口查询、配置查询、部署步骤和故障排查问题运行完整检索流程。

对比时主要观察 **正确证据是否进入 Top-K、正确证据的排序、语义完整性、召回重复率、Token 成本和查询延迟 。**如果 Chunk 太小，接口名称、参数说明和限制条件容易分散到不同块中；如果 Chunk 太大，一个块中又可能混入多个主题，使向量语义被稀释。我们选择的是 ** 在检索精度和上下文完整性之间表现更均衡的参数。**

从下游窗口预算看，假设一个 Chunk 约 250 Tokens，召回 10 个 Chunk 大约占用 2500 Tokens，不会明显挤压模型的推理和生成空间。 相邻块还会设置有限的 Overlap，并通过标题路径、页码和来源元数据保留上下文。

但 **1000 是否合理高度依赖场景 。如果项目换成中文文档，1000 个中文字符对应的 Token 数会明显增加，也可能跨越多个自然段落，这时我会重新降低 Chunk Size，并使用同一套评测流程重新选型。因此， 1000 不是天然合理，而是在当前英文技术文档、字符计量和评测条件下合理。**

16.5.3.5 视频讲解

https://www.bilibili.com/video/BV1extE6jEHu/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

16.5.4 召回

16.5.4.1 召回的常用策略

RAG系统可以使用**粗排 + 重排**两个阶段来完成召回。 粗排通常使用**稠密向量检索模型**来做**语义匹配，召回语义相关文档。**当然也可以再结合**稀疏向量检索模型**来补充使用，因为**稀疏向量检索模型**更擅长**捕捉关键信息**来提高召回效果。

粗排后，可以使用**交叉编码器**，比如BERT来对候选文档进行**深入语义排序**，提高召回质量。

16.5.4.2 稠密向量和稀疏向量

参考书籍《大模型RAG实战》 第三章文本召回模型

RAG系统中的文本向量检索模型大致分成两类：一类是基于**BERT， GPT等深度学习模型**的**稠密向量检索模型**，另一类是以**TF-IDF, BM 25 为代表的稀疏向量检索模型。 稠密向量检索模型更擅长提取文本中语义信息，稀疏向量检索模型更擅长提取关键信息。**

16.5.4.3 对称检索和非对称检索

参考书籍《大模型RAG实战》 第三章文本召回模型

对称检索是找到相似的句子，与向量检索基于计算向量相似度的原理天然匹配，只需要模型具有较强的内容抽象的能力即可。

非对称搜索是根据问题召回答案，要求模型能够将问题和答案映射到同一空间，而问题和答案通常具有不同的文本结构。并非所有的向量化模型都支持非对称检索，只有经过大量问答数据的模型才能提供支持。

16.5.5 召回优化策略

16.5.5.1 短文本全局信息增强

> 因为切块的过程难免会造成全局文本信息的丢失，因此我们为切块后的短文补充其所属的长文本信息。
>
> 比如以论文《DAPR: A Benchmark on Document-Aware Passage Retrieval》中提到，我们可以将**长文本的前三句话， 长文本的标题，长文本的关键词** 作为全局信息，和切块后的文本一起保存。

16.5.5.2 召回内容上下文扩充

> 在RAG切块的时候，对长文本切块后，可能会有较大的信息损失，因为我们从数据库中召回的通常是短文本。这时我们可以采用召回内容上下文补充的方法：**召回的是短文本，但是输入LLM处理的是短文本的父文本（长文本）**。 好处是既**精准**（基于短文本匹配），**又完整**（基于长文本输入）
>
> 在Langchain中，我们可以使用**ParentDocumentRetriever** 来完成这一功能，它使用了两个分割器，**Parent_splitter**和**child_splitter**。 前者用于确定在RAG场景下输入到大模型的父文本块的大小，后者确定从向量数据库中召回的子文本块的大小。同时还会存储父子文本块的映射关系。 当使用get_relevant_documents召回时，返回的实际上是父文本的内容。

16.5.5.3 文本多向量表示

> 在构建向量数据库的时候，可以使用多种向量来表示同一段文本。比如有一段文本A，我们不仅可以使用完整的A来生成向量，还可以使用它的**子集**，**总结句**来生成补充向量。当召回任意的补充向量对应的文本后，并不是将其输入大模型中，而是将其对应的文本A输入。

常见的生成补充向量的方法有：

> **文本切块:** 将原始文本切块后保存到数据库中，可以在召回时增强对更细粒度的表示。
>
> **文章摘要：**利用LLM生成摘要。

> **假设性回答：**可以利用大模型的能力，根据文本提出问题，把问题作为补充信息。这种方法的思想是用问题召回问题会比用问题召回答案更容易。

16.5.5.4 内容查询优化

> 很多情况下，用户提出的内容有口语化，语义模糊，无关内容多的问题。 因此直接将问题变成向量可能和用户的真实意图不相关，从而影响RAG场景下的召回效果。因此，我们可以对用户的原始问题进行改写和扩充，再进行召回。

常见的对原始问题优化的方法有：

1.使用LLM 改写和扩充，比如LangChain提供的MultiQueryRetriever就集成了这个功能。

2.对历史问题进行总结，然后再进行召回，这个特别适用于多轮对话中。比如

> 用户：中国的首都在哪里？
>
> LLM：北京
>
> 用户：那里有哪些好玩的景点？
>
> 这时直接用原来的问题召回显然太模糊，如果用LLM总结后，问题就变成了北京有哪些好玩的景点。

16.5.5.5 召回文本重排序

> 在RAG场景下，默认使用的是向量召回，检索效率高。但是由于文本向量化的过程中存在着信息损失，并且在召回过程无法考虑查询用户和候选文本上下文关系，因此召回精度不高。 所以在召回向量后，通常还会进行召回文本的重排序的精筛。

最常用的重排序的模型是基于**Transformer编码器架构的交叉编码器**。将用户查询和一段候选文本同时输入到模型之中，预测它们之间的相关性分布。使用这种方法可以考虑到用户查询和候选文本的上下文关系，因此有更高的精度。

16.5.5.6 结合元数据召回

在实际场景中，很多情况我们不需要在整个向量数据库中进行召回。如果在存储文本向量的时候同时记录了每个向量相关的标签，就可以在召回时先根据标签信息过滤出所需的向量子集，有利于提高检索速率，避免检索无用的信息。

现在的大模型应用框架，Langchain就基于多种向量数据库，集成了元数据写入和召回过滤的能力。我们可以使用**Langchain的Create_metadata_tagger**的接口，借助大模型的能力，自动生成元数据。

16.5.6 精排

> 第四小节的各个部分其实就是RAG最基本的模块和环节。是弄懂RAG的基础，不管是我推荐的书籍还是项目（实战都有），大家结合笔记，书籍，自己搜的资料，把这几个环节理解一下。可以多搜索视频啊，书籍来理

> 解这些基础知识点。我这里写得不是很详细，知识概括，方便大家背诵。如果说学习的话，其实可以找我推荐的书籍啊，视频啊，自己也能找。这些都是最基础的东西，网上很多讲解。

精排是 RAG 检索链路里位于召回之后的一层排序环节。粗排或者向量召回的目标是从大规模语料里快速找出一批候选文档，而精排的目标是对这批候选文档做更细粒度的相关性判断，把真正能回答问题、信息质量更高、业务上更适合进入上下文的内容排到前面。常见的精排方法主要有 Cross Encoder 精排、LLM 精排、规则精排，以及混合精排。它们都不是用来替代召回的，而是在候选集已经比较小的情况下，对候选内容进行二次排序、过滤和质量控制。

具体来说，Cross Encoder 会把 query 和每个候选文档拼在一起输入模型，让模型直接判断二者相关性。它的优点是语义匹配能力强，相关性分数比较稳定，适合做通用的语义相关性排序。缺点是每个候选都要单独过模型，延迟和算力成本较高，而且它主要判断语义相关，不一定能很好处理业务规则、字段完整性、时效性、来源可信度这些约束。

LLM 精排可以让大模型结合问题、候选文档和业务要求做综合判断。它的优点是解释性强，也能处理复杂意图和多维标准，比如不仅判断相关，还可以判断这段内容能不能支撑最终回答。缺点是成本更高、延迟更高，输出稳定性和可控性也需要额外约束。

规则精排是根据项目里的明确业务目标来打分或过滤，比如优先保留字段完整、标题匹配、时间更新、来源可靠、包含关键实体的内容。它的优点是可解释、成本低、延迟低、容易调试，也更适合把业务经验沉淀到系统里。缺点是语义泛化能力弱，规则覆盖不到的情况容易漏掉。

混合精排则是把模型相关性和规则约束结合起来。它的优点是效果更均衡，既能利用模型判断语义相关性，也能用规则保证业务可用性。缺点是工程复杂度更高，需要调权重，也需要用评估集持续验证不同策略的收益。

16.5.7 面试问题精选

16.5.7.1 Embedding 切 Chunk 应该怎么做，为什么固定长度切分经常不够?

Embedding 切 chunk 不能只按固定长度一刀切，而是要尽量让一个 chunk 对应一个完整的语义单元。实际做的时候，我会**先看文档类型**：如果是 FAQ这类强结构化数据，本身就是完整实体，就可以一条作为一个 Document，不强行切；如果是技术文档、用户协议、SOP 这类长文本，就用递归切分，优先按标题、段落、句子切，最后才按长度兜底。

chunk 大小也不是拍脑袋，要结合 embedding 模型的输入限制和最佳区间，比如很多 BGE / Sentence-Transformers 模型虽然有 512 token 上限，但最佳效果通常在 100-400 token 或几百 token 区间。太短信息不够，太长语义会被稀释。还要设置 10%-20% 的 overlap，避免关键信息刚好落在边界被切断。

固定长度切分的问题是，它只保证长度，不保证语义完整。比如一个定义、一个步骤、一个实体画像可能被切成两半，向量表示就会变成残缺语义；或者一个 chunk 里混了多个主题，embedding 变成“平均语义”，用户问具体问题时反而召回不到。所以更好的做法是“语义优先 + 长度约束 + overlap + 评估验证”，最后用真实 query 的 hit_rate、MRR 来调 chunk size。

选自202604 腾讯AI Agent开发暑期实习面试

16.5.7.2 硬切分在实际场景中有哪些弊端?

硬切分最大的问题是破坏语义边界。它通常按固定字符数、固定 token 数、页码或行数切分，不管文本本身的标题、段落、句子和逻辑结构，所以很容易把一个完整概念、一个操作步骤、一个接口说明切断。比如接口文档里本来应该把接口名、参数、错误码、限制条件放在一起，如果硬切分刚好切开，后面检索到的 chunk 就可能只有条件说明，却没有对应的接口上下文。

第二是边界信息容易丢失。版本号、接口名、错误码、配置项、前置条件这类信息，如果刚好落在两个 chunk 的交界处，单个 chunk 里就没有完整证据。检索时可能两个 chunk 都和用户问题“不够像”，最后都排不上来，导致模型拿不到关键上下文。

第三是会影响 embedding 的语义表达。这里不是说硬切分一定导致 chunk 长短不合适，而是说它只保证长度可控，不保证语义完整。一个长度合适的 chunk，如果只包含半个技术点，embedding 表达的就是残缺语义；如果一个 chunk 里混进多个主题，embedding 又会变成“平均语义”。这会导致相关和不相关结果的相似度分数拉不开，召回结果不稳定。

第四是增加后处理成本。切分阶段如果把段落、步骤、表格说明切碎了，后面可能还要靠 overlap、rerank、LLM 重组、上下文补全去补救。这样不仅增加计算成本和延迟，也会让整个 RAG 链路更难调试。

选自202604 腾讯AI Agent开发暑期实习面试

16.5.7.3 微调（模型训练）vs RAG，如何选择?

我会先区分这个项目的问题到底出在哪里。不是所有效果不好都应该重训练模型，很多时候只是知识没接上、检索没做好、规则没兜住。

如果问题主要是知识更新、企业内部资料、产品文档、政策制度、FAQ、接口说明这类外部知识缺失，我更倾向于用 RAG。因为这些内容变化快、可追溯性要求高，放进知识库可以随时更新，也方便引用来源。如果为了这些动态知识去微调模型，成本高、更新慢，而且模型还可能记混或者幻觉。

如果问题主要是确定性的业务规则，比如权限判断、价格计算、流程状态流转、字段校验、风控规则、格式约束，我不会优先训练模型，而是用规则、代码、工作流或者工具调用来增强。因为这些东西要求稳定、可解释、可审计，不能靠模型“猜”。更适合继续做模型重训练的项目，一般是模型的能力模式本身不符合业务需求。比如模型长期不会按某种专业风格回答，不能稳定遵守某种输出格式，对领域术语理解差，分类/抽取/意图识别这类任务在大量样本上都表现不好，或者有大量高质量标注数据可以让模型学到稳定模式。这时候微调才有价值，因为它改变的是模型的行为分布，而不是临时塞几段知识。

选自202604 腾讯AI Agent开发暑期实习面试

16.5.7.4 RAG 检索相关性差的原因与优化思路?

💡 **思路：** 这个题也不难回答，我们的RAG项目中，做了很多操作：**RRF**,双路检索，**Rerank**, 基于Metadata过滤答案后筛选，这些做法都是在提高检索内容的相关性。同时我们在RAG知识部分，也讲解了RAG的一些

高级策略，也是回答优化方向的点。

同时我们之前专门讲了**Self-RAG**技术（因为面试官问如何保证系统生成回答的准确性），这几个技术都是来回答优化思路的出发点。因此即使我们没有专门准备过这个问题，但是我觉得不应该答不出来。如果从这个题目本身来说，我下面给的参考答案，其实有点多了，不过我还是没有删减，因为我觉得这些内容挺有价值的，从相关性差的原因，到如何解决，顺着整个链路把RAG系统串了起来。回答的时候，挑着说，简化着说就好了，我都列出来，是想给大家复习。

📝 **答案：**

RAG 检索相关性差，通常不是单一环节的问题，我会沿着“**知识入库、用户查询、候选召回、结果排序**”这条链路逐层排查。

首先是知识入库的问题。比如原始文档质量不高、包含大量噪声或重复内容，或者 **Chunk** 切得不合理。**Chunk** 太大容易混入无关信息，太小又会破坏完整语义；如果标题、章节、时间、文档类型等 **Metadata** 没有保留下来，检索时也很难利用这些结构化信息进行过滤。另外，**Embedding 模型**如果不适合中文或业务领域，也会导致向量表达不准确。

其次是用户 **Query** 本身的问题。用户的问题可能很短、存在歧义，或者使用了知识库中没有出现过的表达。对于复杂问题，如果直接把原始 **Query** 做一次**向量检索**，也可能因为包含多个意图而召回不准。这种情况可以通过 **Query Rewrite** 补全上下文，把口语化问题改写成更适合检索的表达；也可以通过 **Query Decomposition**，把复杂问题拆成多个子问题分别检索。

在召回阶段，我不会只依赖单路**向量检索。向量检索**擅长语义匹配，但对产品编号、专有名词和精确关键词不一定敏感；**BM25** 关键词检索正好可以弥补这一点。因此可以采用**双路混合检索**，同时执行**向量检索**和**BM25**，再使用 **RRF** 根据两路结果的排名进行融合。这样既能利用语义相似性，也能保留关键词精确匹配能力，提高候选集的召回率。

如果业务数据带有明确的结构化条件，我还会在检索前或检索后使用 **Metadata 过滤。**例如，用户查询某个产品最新版本的使用说明，可以先按照产品型号、文档类型、版本和时间进行过滤，再进行**向量检索**，避免把其他产品或旧版本的内容送入候选集。这里还需要根据评测结果调整 **Top-K**、相似度阈值和索引参数，防止 **Top-K** 太小导致漏召回，或者太大引入过多噪声。

召回之后还需要做 **Rerank。**向量数据库返回的相似度主要适合快速筛选候选，并不代表排序结果一定最符合用户问题。我会先通过混合检索召回较多候选，再使用 **Cross-Encoder** 或专门的 **Reranker**，让模型同时理解 **Query** 和每个候选 **Chunk**，重新计算相关性，最后只把排名靠前的内容交给大模型。整体上就是“**粗召回保证不漏，精排序减少噪声**”。

对于复杂场景，还可以引入 **Self-RAG** 的反馈机制。模型先判断当前问题是否需要检索，再检查检索内容是否相关、是否足以支持回答；如果证据不足，就改写 **Query** 或继续检索，而不是拿着低质量上下文强行生成。生成以后还可以检查回答是否忠实于检索证据，从而减少检索质量问题进一步演变成回答幻觉。

最后，这些优化不能只靠主观观察，还要通过评测集验证。检索阶段可以看 **Context Precision**、**Context Recall**、**MRR**、**NDCG** 等指标；生成阶段可以看 **Faithfulness** 和答案正确性。同时记录每个失败案例，判断问题究竟出在 **Chunk**、Embedding、**Query**、召回还是 **Rerank**，再针对性调整。

总结来说，我的优化思路是：先提升知识和 **Chunk** 质量，再通过 **Query Rewrite** 或问题拆解改善查询，使用 **BM25** 与**向量检索**双路召回、**RRF** 融合，通过 **Metadata** 缩小范围，再利用 **Rerank** 精排；复杂场景下加入 **Self-RAG** 的**检索与反思闭环**，最后用离线评测指标验证优化效果。

选自202607 宇树科技一面

16.6 嵌入向量

> 面试偶尔喜欢问的：有什么Embedding的方式，对比。Embedding选择策略。没啥特别要强调的，不过还是值得记住（背住的）。本质上来说，是RAG项目做得并没特别深入和专业，所以我就是选择了通用Embedding模型，没办法这些部分只能背着，最少做理论上的巨人。

16.6.1 常见的Embedding方法

常见的 embedding 相关模型大概可以分成几类：

通用语言理解模型：BERT、RoBERTa

这类模型本质上是通用文本理解底座，语言理解能力强，适合分类、抽取式问答、序列标注、精排等任务。

但它们**不是专门为向量检索训练的**。如果直接拿 BERT 的句向量去做相似度搜索，效果通常不稳定：有时两个句子看起来很像，向量距离却不近；有时语义不相关，分数又拉不开。

所以在 RAG 里，BERT 更常作为底座或精排模型使用，不是最推荐直接拿来做第一阶段召回。

句向量模型：SBERT、Sentence Transformers

这类模型是在 BERT 基础上改出来的，目标就是让句子向量更适合做相似度计算。它会把问题和文档分别编码成向量，然后用向量距离判断相关性。

它的优点是：**开源、好部署、生态成熟、适合语义搜索和 RAG 召回**。相比原生 BERT，它更适合做“用户问题和知识片段是否相关”的判断。

缺点是：不同模型差异很大。英文效果好的模型，不一定适合中文；通用语料上效果好的模型，不一定适合公司内部术语、产品名、业务黑话。小模型速度快但表达能力弱，大模型效果好但资源和延迟更高。

检索专用开源模型：E5、BGE、GTE

这类模型一开始就是按“问题找文档”的目标训练的，更贴近 RAG 的使用方式。它们通常会用大量问答对、搜索点击数据、正负样本对来训练，所以对“这个问题应该召回哪段资料”更敏感。

它的优点是：**召回能力强、适合知识库问答、可以私有化部署、中文模型选择也比较多**。如果项目不能把数据发到外部 API，这类模型通常是比较现实的选择。

缺点是：部署、显存、推理速度、批量建库都要自己维护；有些模型还有固定输入格式，比如问题和文档要分别加不同前缀，不按格式用可能会掉效果。

商业 Embedding API：OpenAI、Cohere、Voyage 等

这类模型的特点是开箱即用。你不用自己部署模型，只要调用接口就能生成向量。

它的优点是：**效果稳定、多语言能力强、接入简单、维护成本低。**很多项目早期验证 RAG 效果时，会先用这类模型快速跑通。

缺点是：要考虑调用成本、网络延迟、限流、数据合规和私有化问题。尤其是企业内部知识库，如果文档不能出内网，就不能简单依赖外部接口。大规模重建索引时，费用也可能比较明显。

领域专用模型：代码、法律、金融、医疗等

这类模型不是追求“什么都能搜”，而是针对某个领域优化。比如代码检索要理解函数名、接口、报错信息；法律检索要理解法条、案由、裁判逻辑；医疗检索要理解疾病、药品、检查指标。

它的优点是：**在特定领域更准，能处理通用模型不熟悉的术语和表达。**

缺点是：换领域后不一定好用，而且训练和评估成本更高。是否值得用领域模型，要看业务 query 里领域术语占比高不高，以及通用模型是否已经明显召回不准。

领域专用 embedding 模型：比如代码、法律、金融、医疗、论文检索、图文多模态检索等模型。它们的优点是在特定领域术语、结构和查询方式上更准，比如代码检索要理解函数名、API、错误信息，法律检索要理解法条和案由。缺点是泛化能力可能弱，换领域后不一定比通用模型好，而且评估和训练数据成本更高。

16.6.2 Embedding选择策略

1. **先看要解决什么检索问题**：如果只是普通知识库问答，优先选面向“问题找文档”训练过的检索模型；如果是代码、法律、金融、医疗，就要考虑领域模型；如果要中文问英文资料，或者中英混合检索，就要看跨语言能力。不要用一个“通用句子相似度模型”硬套所有场景。
2. **再看数据本身的特点**：知识库是中文多，还是英文多？是长文档、PDF、网页，还是短 FAQ？里面有没有大量产品名、缩写、错误码、接口名、内部黑话？这些都会影响模型效果。比如纯语义模型对“意思相近”的内容比较敏感，但对订单号、版本号、接口名这种精确匹配，不一定比关键词检索好。
3. **一定要用业务样本试**：不要只看榜单。应该拿几十到几百个真实问题，标出它应该命中的文档片段，然后比较不同模型能不能把正确片段排到前面。重点看两个现象：该召回的有没有召回；不该召回的相似干扰项有没有混进来。
4. **看能不能满足工程约束**：模型效果好还不够，还要看向量维度、索引大小、查询延迟、建库速度、调用成本、是否能私有化部署。比如商业 API 接入快，但有费用和数据合规问题；开源模型可控，但要自己维护推理服务和机器资源。
5. **和切分、关键词检索、重排一起调**：Embedding 不是孤立选择的。chunk 切太大，语义会被稀释；切太小，上下文又不够。对于专有名词、编号、时间、版本号这类信息，通常还要结合关键词检索；对召回结果质量要求高的场景，再加重排模型做二次筛选。

16.6.3 选错Embedding带来的问题

- **召回不准**：用户问 A，返回一堆“语义上有点像但答不上问题”的 chunk，LLM 就会基于噪声回答。

- **召回漏掉关键证据**：答案明明在库里，但 topK 里没有，表现为 RAG 频繁说不知道，或者只能答泛泛内容。

- **近义词、缩写、业务术语匹配差**：比如内部黑话、产品名、英文缩写、中文同义表达搜不出来。

- **跨语言效果差**：中文问题搜英文资料、英文问题搜中文资料时命中率明显下降。

- **长 chunk 被语义稀释**：一个 chunk 里混了多个主题，向量变成“平均语义”，具体问题反而匹配不到。

- **精确信息检索失败**：订单号、接口名、错误码、版本号这类 token，纯 embedding 可能不如关键词检索稳定。

- **分数分布异常**：相关和不相关 chunk 的相似度分数拉不开，阈值很难定，topK 结果经常抖动。

- **线上成本过高**：维度太大或模型太慢，导致索引膨胀、检索延迟高、增量入库成本高。

16.6.4 面试问题

16.6.4.1 Embedding模型在项目中应该怎么选，选错会出现什么现象?

请参考 202604 腾讯AIAgent开发暑期实习面经 问题6，答案写在了那里。

16.6.4.2 向量数据库与关系型数据库的核心区别

选自202607 宇树科技一面

💡 **思路：** 八股问题。对于这些八股，我们如果背到了是最好的，用一个比较标准的，全面的，精心组织的回答去回答。但是如果你没有背到这个八股，也不应该答不上来，学过了RAG，你应该知道向量数据库是啥，包括关系数据库，基本也算是计算机的常识问题了。

📝 **答案：**

向量数据库与关系型数据库最核心的区别，是**数据表示方式和查询范式不同。**

关系型数据库把数据组织成表、行和列，通过明确的 Schema、SQL 条件以及 JOIN 进行**精确查询。**例如，查询“张三的注册日期”或“过去一年订单总额”。它擅长**处理结构化事实**的**精确查询**、事务处理和**强一致性。**

向量数据库则会把文本、图片等非结构化内容通过 **Embedding 模型**转换成**高维向量。**查询时，也会把用户的问题转换成向量，再通过**余弦相似度**、内积等方法，返回语义最相近的 **Top-K** 结果。它擅长处理非结构化内容的**语义检索**、相似内容匹配和大规模候选召回。

两者的**索引机制也不同。**关系型数据库常用 **B+ 树**、**哈希索引**；向量数据库常用 **HNSW**、**IVF**、**PQ** 等**近似最近邻索引**，需要在检索速度、召回率和内存占用之间进行权衡。因此，SQL 查询通常返回满足明确条件的确定记录，而**向量检索**返回的是带有相似度分数的候选结果，并不保证结果一定正确。

在实际的 RAG 系统中，两者通常**不是替代关系，而是互补关系。**例如，可以先用关系型字段或元数据过滤租户、权限、时间和文档类型，再通过**向量检索**寻找语义相关的知识片段。PostgreSQL 加 pgvector 就是将两种能力结合起来的例子。

最后总结来说：关系型数据库擅长对**结构化事实进行精确、确定性的查询**；向量数据库擅长对**非结构化内容进行近似的语义相似性检索**，两者在实际 RAG 系统中通常结合使用。

16.7 RAG进阶

此部分为RAG进阶部分，如前所述，一般岗位可能不会考察这么深，除非你自己简历写了Graph Rag, Agentic RAG或者本身就是专门做RAG的组，可能会考到这里。 我的原则是，该部分概念要会，问RAG有啥新方向，相关概念答得出来，但没有深入研究。

13.7.1 RAG新方向

（这俩新方向，虽然可能没深入研究，但是得知道，概念要能讲出来）

目前典型的RAG的新方向是 Agentic RAG和Graph RAG, 具体来讲：

Agentic RAG的核心是把RAG从一个固定流水线变成一个有决策能力的Agent。传统RAG就是检索一轮、生成答案、结束了。但很多复杂问题一轮检索根本不够，Agentic RAG让模型自己判断——检索结果够不够？不够就再检索一轮，或者换个检索策略。像Self-RAG、Corrective RAG都是这个思路，本质就是让**检索过程变得更智能、可以自我纠错。**

Graph RAG解决的是另一个问题——**传统向量检索擅长找语义相似的内容**，但很难处理跨文档的关联推理。比如问人物之间的关系，信息散落在十几个段落里，靠embedding相似度很难串起来。Graph RAG的做法是先从文档里抽取实体和关系建成知识图谱，检索时在图上做关系遍历，特别适合需要全局理解的问题。"

我觉得这两个方向是互补的——一个解决检索策略的智能化，一个解决知识关联和推理，未来可能会结合起来用。

16.7.2 RAG与知识图谱

（知识图谱也能和RAG结合使用。但是其实对于可能80%的问题，靠高质量的切片和双路召回就能够解决，只有20%的问题，需要跨文件全局关联的硬核需求，值得花200%的精力去建立图谱。总结来说，知识图谱较为复杂，不是每个企业都会用，成本高。也属于进阶路线。 所以如果你没有学好传统的双路召回RAG，先不用来学这个结合图谱的RAG，这里的建议依旧是作为概念，先理解什么是知识图谱，为什么和RAG能结合？优势和劣势是啥？我就只学到这里。如果哪天他变得很火了，考的多了，可以继续学）

13.7.2.1 什么是知识图谱?

知识图谱是一种结构化的知识表示形式，通过图模型来描述现实中的实体及其之间的关系：核心包括：

节点：表示实体或者概念，如'北京'，'小刘'

边：表示实体间的关系，比如'小刘-就职于-阿里'

属性：描述节点的额外信息，如'北京-人口-2154万'等。

知识图谱通过三元组的形式组织数据，从而构建出一个庞大的网络。

13.7.2.2 知识图谱在RAG的应用?

传统的RAG是基于分块，向量召回的思路来对输入数据增强的。但是对于强调跨文档/多文档综合，需要多跳推理/链路关系的时候，可以将知识图谱和RAG结合，即KG-RAG。 它的做法是先用普通语义检索拿到一批chunk，然后把这些chunk映射到知识图谱的实体上，沿着图的边做扩展检索，把原本关联的文本片段补齐成一条/多条与问题相关的关系链，最终把组织起来的片段按图结构交给LLM（简单说，向量召回的结果作为起点，KG扩展把点连成线，再将整合结果送入LLM）

13.7.2.3 知识图谱vs普通RAG?

普通RAG **落地快，成本低，适合'答案就在某几段文档里'的定位性问题**，但是它的弱点是需要**跨多段/多条文档做综合，需要多跳推理和关系链路时**，他只能拿到碎片，此种场景适合使用KG-RAG, 但是它的缺点是：**需要抽取/维护实体关系、做图扩展与组织，构建与维护成本更高。**

16.7.3 Self-RAG

Self-RAG, 我认为是一个还挺有价值的技术。学好这个，对面试的帮助也很明显，当面试官问你：**如何保证RAG系统生成的答案是准确的?** 以及 你的RAG系统中有什么高级技术？可以介绍一下吗（英伟达面试真题）？ 我们都可以从Self-RAG的角度出发。这两个问题也是真实的面试问题，我也附上了跳转链接。

同时这也确实是一个有价值的事：我们学了RAG的评估方法，比如RAGAS，这个是已经吐出了回答后的检测，有没有办法能够在运行的时候，就能动态检测生成质量，实时调整呢？那么就得靠我们的Self-RAG技术了。

16.7.3.1 Self-RAG是什么?

Self-RAG（Self-Reflective Retrieval-Augmented Generation，**自省式检索增强生成**）是在传统 RAG 基础上加入“自我判断和自我评估”能力的一种 RAG 范式。

传统 RAG 通常采用固定流程：

```Plain Text 用户问题 → 检索知识 → 生成答案

> 它存在两个问题：一是无论问题是否需要外部知识都进行检索；二是即使检索到了知识，也不能保证知识与问题相关，更不能保证生成答案严格受到知识支持。
>
> Self-RAG 将流程改为：
>
> ```Plain Text
> 判断是否需要检索
> → 检查检索内容是否相关
> → 生成候选答案
> → 检查答案是否得到证据支持、是否有用
> → 评分并选择最终答案
```

因此，Self-RAG 的重点不是简单地增加一次检索，而是让质量评估直接参与检索和生成决策。

从下图中可以清晰地理解Self-RAG和传统RAG的区别：图片上方是经典的RAG，而Self-RAG就是在上方经典RAG下面引入了自我检查：**从要不要检索，到输出分数是否达标各个方面检测，及时做出调整或者拒答。**

16.7.3.2 Self-RAG 检测什么?

经典 Self-RAG 使用四类自省信号：

1. `Retrieve`：在检索前判断当前问题是否需要检索外部知识。
2. `IsREL`：在检索后、生成前判断检索到的文档或 Chunk 是否与问题相关。
3. `IsSUP`：在生成后判断答案是否得到检索证据支持。
4. `IsUSE`：在生成后判断答案是否有用、是否真正解决了用户的问题。

其中，`IsSUP` 可以进一步分为完全支持、部分支持和无支持或冲突；`IsUSE` 通常可以采用分级评分。

16.7.3.3 经典 Self-RAG 如何实现?

经典 Self-RAG 会把上述判断表示为特殊的“自省 Token”，例如：

```Plain Text [Retrieve] [Relevant] [Fully Supported] [Utility:5]

> 训练阶段先使用 **Critic 模型为训练样本生成这些评价标签**，再使用带标签的数据微调 **Generator**。完成训练后，Generator 不仅能够生成答案，还能够在适当位置生成自省 Token。
>
> 推理时，程序读取这些 Token 及其生成概率，以决定是否检索、是否保留某个 Chunk，以及应该选择哪个候选答案。自省 Token 不需要展示给用户，它们相当于模型提供给程序的标准化控制信号。
>
> 简化后的运行过程如下：
>
> ```Plain Text
> 用户问题
>   ↓
> Retrieve：需要检索吗？
>   ↓
> 检索多个 Chunk
>   ↓
> IsREL：逐个判断 Chunk 是否相关
>   ↓
> 使用相关证据生成候选答案
>   ↓
> IsSUP + IsUSE：评价候选答案
>   ↓
> 选择得分最高的答案并输出
> ```

如果要形成更严格的生产级闭环，还可以增加质量阈值：**答案未达到阈值时，不直接输出，而是重新检索、重新生成或拒答。**
```

16.7.3.4 用RAGAS 替换传统Self-RAG

这一节纯粹是我自己加的。因为我们看到上面的Self-RAG的 自评估方法是要使用专门的模型进行训练的。如果我们要包装，我害怕面试官问你的模型训练的细节，加上我们项目中做了RAGAS的评估，RAGAS的评估和Self-RAG的三个指标是一样的，干脆我们讲的时候就用RAGAS集成到Self-RAG好了，这样问到我们如何实现的细节，我们更加胸有成竹。我在相关面试真题:如何保证RAG系统生成的回答是准确的？，也是用这个思路来组织最终参考答案的。

我们可以保留 Self-RAG 的"检索—生成—评估—修正"流程，用 RAGAS 作为外部评估器替代模型内部的自省 Token。

两者可以近似映射为：

1. `Retrieve`：RAGAS 没有直接对应指标，可以使用 LLM 分类器或业务规则判断是否需要检索。
2. `IsREL`：可以使用 Context Relevance 或 Context Precision 判断检索内容是否相关。
3. `IsSUP`：可以使用 Faithfulness 判断答案是否得到检索上下文支持。
4. `IsUSE`：可以使用 Answer Relevancy 判断答案是否解决了问题；有参考答案时，还可以增加 Answer Correctness。

替换后的流程为：

```Plain Text 用户问题 ↓ 规则或 LLM 判断是否需要检索 ↓ 检索并生成答案 ↓ RAGAS 评估上下文相关性、答案忠实度和答案相关性 ↓ 达到阈值 → 输出 未达到阈值 → 过滤噪声、重新检索、重新生成或拒答

> 这种实现并不是原始论文中依靠自省 Token 的经典 Self-RAG，更准确的称呼是“借鉴 Self-RAG 思想、以 RAGAS 为外部评估器的自反思 RAG 工作流”。它不需要重新训练 Self-RAG 模型，更容易接入现有工程，但会增加额外的模型调用、延迟和成本。

<a id="doxcnMdwaleIPBqat963WPNrrtg"></a>
##### 16.7.3.5 讲解视频

https://www.bilibili.com/video/BV1gF8Z6WEGW/?
spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

<a id="doxcnOpfShGcawWJOFIkPg8NVVg"></a>
### 16.8 RAG场景题

如果我做 rag 召回的相关内容里，会有人恶意注入了一些错误的信息， 你觉得会影响大模型的生成内容吗？怎么避免？

会影响生成内容。 我认为：

1. 可以限制拿到的数据的来源，只选择可靠的数据源。
2. 可以对拿到的数据进行安全性检验，检验是否包含恶意指令
```

16.9 RAG的趋势探讨

在2026年，我相信大家听过很多论调和讨论：**RAG已死**。**我们现在Agent都不用RAG了**。包括还有各种面试题：**为什么ClaudeCode不用RAG了？你设计Agent架构的时候，到底要不要用RAG（**202606滴滴面试真题）。包括还有一些你和面试官聊天的时候的讨论：**你如何看待RAG的趋势，你觉得现在还需要不需要RAG？**

今天这个小节，我想全面，客观的来帮助大家分析一下这个问题。大家不要背答案，而是真的去理解，去思考，去看看我说的是不是这么一回事？有了自己的思考，对于上面的各种问题的回答其实也是自然而然就知道如何回答了。

本文参考的资料为：

1. **长上下文视角**——《[Is RAG still needed? Choosing the best approach for LLMs](https://www.youtube.com/watch?v=UabBYexBD4k&t=35s)》（YT视频）
2. **结构化数据视角**——《[Why the best AI coding tools abandoned RAG (and what they use instead) / Is RAG Dead?](https://www.youtube.com/watch?v=60G93MXT4DI&t=124s)》（YT视频）
3. **系统作者视角**——Boris Cherny(Claude Code 核心作者)在 Latent Space 播客的访谈
4. **Harness 工程视角**——飞书「大模型笔记」[2.4.1 Retrieval 章节](#doxcnkR9Pi8vxdHND0zroCGt62f)(检索工具谱系 + Agentic RAG 实验)

注意，其实我并不是说大家一定要去看对应的视频，为了严谨性我列出来源。笔记已经总结了上面的资料的各个方面的要点，已经是比较全面的了。看懂了笔记和我的视频，可以不用再去专门看上面的视频了。

16.9.1 背景 - 先把RAG这个词说清楚

RAG 是什么?

RAG = **R**etrieval **A**ugmented **G**eneration(检索增强生成)。

它要解决的是 LLM 的一个根本缺陷:**模型被冻结在训练时刻**——它不知道 5 分钟前发生的事,更不知道你的私有数据、内部 wiki、专有代码库。要让模型「知道」这些,就必须解决**上下文注入(context injection)**:在正确的时间,把正确的数据放进模型的上下文窗口。

经典(传统)RAG 的管线是:

```Plain Text
文档 → 切块(chunking) → embedding 模型编码成向量 → 存入向量数据库
用户提问 → query 向量化 → 语义搜索(semantic search)召回最相关的若干 chunk → 注入上下文窗口 → 生成答案
```

关键澄清:大家吵的RAG 已死,死的是哪个 RAG?

这是全篇最重要的区分,不厘清就会全程鸡同鸭讲:

- **传统 RAG / 语义搜索**:指切块 + embedding + 向量库 + 语义召回这套**具体管线**。现状是在**部分场景**(尤其编程)被替代。
- **广义 RAG**:指**任何**把外部信息拉进上下文的检索行为(grep、BM25、向量搜索、web 搜索都算)。现状是不可能死,是所有 Agent 的地基。

- **RAG 的本意** 就是抓取任何外部信息,拉进 LLM 上下文,增强它的回答。所以广义上,agent 用 grep 翻代码、用工具查数据库,**本质都是 RAG**,只是没有向量库那一套。
- 网上RAG is dead的暴论,几乎都在偷换概念:用**传统 RAG 在编程场景被淘汰**这个局部事实,去论证**RAG 整体过时**这个错误结论。
- 更进一步:**Agentic RAG(智能体检索)本身仍然是 RAG**,只是把一次性检索换成了模型自己迭代地、多轮地检索。

> **一句话定调**:讨论RAG 是否过时,只有把范围收缩到**传统的、切块 + 向量库的语义搜索管线**才有意义。广义RAG(检索)永远不会过时。

16.9.2 四个思维点

咱们可以先不讨论结论，先从4个方面去思考，我想思考清楚了这四个思维点，那么结论也呼之欲出。

16.9.2.1 大家说的RAG已死，死的是什么?

- RAG 已死是一个被 LinkedIn / X 放大的**伪命题**。视频作者直言:这个叙事大部分不成立,只是**在编程这个特定场景**有一点点为真。
- Klein 原创者 Nick Pash 甚至把盲目上 RAG 的风气称为**mind virus(思维病毒)**:2023–2024 年,只要 agent 需要搜索,大家就无脑套语义相似 + 向量库,严重过度工程化。
- 但反过来,把传统 RAG 在编程被淘汰推广成RAG 全面过时,是同一枚硬币的另一面错误。
- Harness 视角给出了最精准的定性:**Agentic RAG 本身就是 RAG**,只是把一次检索换成迭代检索。所以从范式上讲,RAG 根本没被取代,而是**换了形态继续存在**。

> **小结**:与其问RAG 死没死,不如问**在我的场景里,那套沉重的切块 + 向量库管线还有必要吗?**——这才是能回答的问题。

16.9.2.2 数据的结构化程度

传统 RAG 唯一真正值钱的超能力,是处理**自然语言的模糊性**——找到意思相近但字面不同的内容。判断要不要它,就看你的数据需不需要这个超能力。而数据需不需要,取决于它的**结构化程度**。所以先要把结构化和非结构化这两个词说清楚。

什么是非结构化数据?

指**没有固定组织形式、以自然语言为主**的数据——散落在 Google Drive、SharePoint、Confluence、wiki、SQL 里的海量文本。它的三个核心特征:

1. **语言是模糊的**:同一个意思有无数种表达,同义词、近义词、指代、隐喻满天飞。

2. **靠概念含义而非字面组织**:你想找的东西,常常和文档里的字面写法对不上——需要按意思相近来找,而不是按字一样来找。
3. **没有天然的导航结构**:一份文档可能埋在成千上万份文档里,没有文件路径、语法这类现成地图帮你定位。

这类数据里,你想要的信息可能**藏在几千甚至几百万份文档中**,而且**字面上根本没出现你搜的词**。要跨同义词、跨概念把它们捞出来,就必须靠 embedding 捕捉语义相似。**这正是传统 RAG 不可替代的场景。**

什么是结构化数据?

指**有精确语法、精确标识符、且自带组织结构**的数据,代码是最典型的例子。它的三个核心特征:

1. **拼写/标识符是精确的**:能正常运行的代码,函数名、变量名一定拼写精确——`getUserProfile` 全项目就叫这个,不存在另一种说法。
2. **不依赖语义近似**:你要找的东西字面就是确定的,不需要靠同义词或概念含义去猜,关键词/正则(grep、ripgrep)就能精确命中。
3. **自带导航结构**:文件夹、路径、架构本身就是现成地图,agent 用文件导航工具逐层 hone in 即可,不需要语义帮它理解大概在讲什么。

这类数据用精确匹配又快又准,再套一层语义搜索反而更不准、还多一堆维护负担。**所以结构化数据不用 RAG。**

两类数据的选型对比如下:

- **非结构化数据**:特征是自然语言、模糊有歧义,同义词/指代多,靠概念含义组织,无天然导航结构,信息埋在海量文档里。**该用传统 RAG(语义搜索)**——因为靠 embedding 捕捉概念相似,才能跨同义词、跨字面把埋藏的内容召回。
- **结构化数据**:特征是精确标识符、语法确定,字面即确定、无需猜意思,自带文件/路径/架构结构。**不用 RAG,用 grep / 文件导航**——因为字面就是确定的,精确匹配又快又准,再上语义反而更差且要维护索引。

经典例子——川菜:在知识库里搜川菜有哪些,库里那几段可能只写了麻婆豆腐,花椒配辣椒,麻辣鲜香,**完全没有出现川菜两个字**。关键词搜索按字面逐字匹配,搜川菜根本匹配不到这些段落,一条都召回不了;只有靠 embedding 的语义相似,才能认出麻婆豆腐，水煮鱼在意思上就是川菜,把这些段落召回。这就是非结构化数据离不开 RAG 的原因。

为什么代码(结构化)不用 RAG,三条:

1. **精确,不需要猜意思**:能跑的代码拼写一定精确,`getUserProfile` 全项目就叫这个 → grep 直接 100% 命中,再上语义搜索反而更不准。
2. **自带组织结构**:文件夹、路径、架构就是现成地图,agent 逐层导航即可,不需要语义帮它理解大概在讲什么。
3. **索引维护是负担**:代码改动极频繁,向量索引很快漂移失同步;而普通文档几个月不变,索引好维护。

> **检索工具是一条谱系,不是二选一**(Harness 视角补充): `grep(词法/精确)` → `BM25(词法/倒排)` → `embedding(语义)` → `结构化 DB(带 metadata)` 从左到右,能力更强但成本更高。**能用轻的就不上重的。**

16.9.2.3 规模 vs 上下文窗口

传统 RAG 的第二个对手不是 grep,而是**百万 token 的长上下文**(直接把文档全塞进去,不做检索)。二者各有胜场:

长上下文赢在简单——三个理由:

1. **砍掉基础设施(no-stack stack)**:不需要切块策略、embedding 模型、向量库、reranker、同步逻辑——少了一堆会坏的环节。
2. **没有检索彩票**:语义搜索是概率性的,可能**静默失败(silent failure)**——答案在库里,但没被召回,模型压根没看到。长上下文让模型看到全部,不存在这个失败点。
3. **整本书问题**:RAG 只能检索**存在的内容**,无法回答**缺了什么**。比如对比需求文档和发布说明,哪些安全需求被遗漏了——RAG 只能召回零散片段,看不到两份文档之间的空缺;长上下文把两份文档整体喂进去才能做这种全局对比。

但 RAG 依然不可替代——三个理由:

1. **重复读取成本**:长上下文每次查询都要重新处理整份文档(500 页 ≈ 250k token,每次都付这个税);RAG 只在建索引时处理一次。(prompt caching 能部分抵消静态数据,但动态数据仍要全额付费。)
2. **大海捞针(needle in haystack)**:上下文越长(50 万+ token),模型注意力越**被稀释**,埋在中间的单段信息经常漏掉或幻觉;RAG 只给 top-k 相关块,**降噪聚焦**,反而更准。
3. **无限数据集**:企业数据是 TB / PB 级,百万 token 只是杯水车薪。要在无限数据里定位,**必须有检索层**先过滤到能塞进窗口的量。

> **该视角的结论**:
>
> - **有界数据 + 需要全局推理**(分析单份合同、总结一本书)→ 用**长上下文**。
> - **无限的企业级知识 → 向量数据库(RAG)仍是唯一可行的仓库。**

16.9.2.4 工程成本与按需外挂

Boris Cherny(Claude Code 核心作者)的第一手复盘:早期 Claude Code **真的用过 RAG**——用 Voyage embedding 给代码库建索引,但后来整个放弃,转向 agentic search(glob + grep 的常规代码搜索)。原因三条:

1. **性能**:agentic search**outperformed everything. By a lot.**(全面碾压,大幅度)。他很坦诚地补充,这个判断主要是**just vibes**(内部凭感觉),也有一些内部 benchmark。

2. **索引复杂 + 代码漂移**:RAG 需要建索引步骤,而**the code drifts out of sync**(代码不断变化,索引很快对不上)。
3. **安全 / 责任**(视频里普遍忽略的一条):索引必须**存在某个地方**,**what if that provider gets hacked?**(存索引的服务商被黑怎么办)——对公司是**a lot of liability**(巨大的责任风险)。Anthropic 不愿把敏感代码上传第三方。

他对这个权衡的总结:

> **at the cost of latency and tokens, you now have really awesome search without security downsides.** (代价是牺牲一点延迟和 token,但换来极棒的搜索,且没有任何安全隐患。)

Harness 工程视角把这条思路一般化:

- **不要把通用 AI 工具当万能工具。**Claude Code 内置 grep,但 CC 是通用的,而你的业务是定制的——代码库可能 100GB、文档跨多仓库、查询带 metadata 过滤。这时通用工具会慢甚至搜不到。
- 关键的实证:视频里 Raj 的实验中,**Agentic + 纯 BM25**反超了**One-shot + embedding 切块。**这说明随着模型变强,语义近似这层**不一定非要靠 embedding**,模型自己重写 query 就能补上。但这是**embedding 从必选变可选**,不是BM25 取代 embedding。
- 所以最精准的定性是:**RAG 没被取代,而是从默认必选降级成按需启用的一档。**

16.9.3 总结&结论

先把 RAG 的账算清楚——它的优点和缺点各是什么。

传统 RAG(切块 + embedding + 向量库)的**核心优点**有三:

1. **能做语义搜索**:靠 embedding 捕捉概念相似,能跨同义词、跨字面把埋在海量文档里、字面根本没出现搜索词的内容召回——这是 grep 这类字面匹配做不到的。
2. **又快又省成本(核心优势)**:预先切块 + 建索引,查询时只拉几个小 chunk,而不是把整个知识库/整份文档塞进上下文。这一个机制同时带来两个好处——

- **省成本**:喂给模型的输入 token 大幅减少,大知识库下比让模型读整份文档或多轮 agentic 扫描便宜约 **100 倍。**
- **快 / 低延迟**:检索直接走索引(向量 HNSW `O(log N)` / BM25 倒排 `O(K)`)秒级命中,加上塞进上下文的内容少、模型推理也快;相比让 agent 用 grep/cat 在海量非结构化文档里逐个遍历(视频原话:慢到爆、贵到爆),定向拉 chunk 快得多。
- 一句话:**只取最相关的一点点,而不是处理全部——省的是钱,快的是这同一件事的另一面。** 3. **降噪聚焦**:只返回 top-k 相关块,把大海捞针里的草堆先扔掉,给模型的是信号而非噪声,反而比长上下文全塞进去更准。

它的**核心缺点**也有三:

1. **要维护一套重基础设施**:切块策略、embedding 模型、向量库、reranker、同步逻辑——环节多、都可能坏;索引还会随源数据变化而漂移失同步,代码这种高频变动的数据尤其吃亏。
2. **检索是概率性的,会静默失败**:语义召回不保证命中,答案可能就在库里却没被检索到,模型压根没看到——而它不会告诉你我没找到。
3. **只能召回存在的片段,做不了全局推理**:它给的是零散 chunk,回答不了两份文档之间缺了什么这类需要看全局的问题。

再下结论——它在哪被取代、在哪没被取代。

传统 RAG 已在两类场景被更优方案取代:

1. **结构化数据(尤其代码)**:字面精确、自带文件结构,`grep / 文件导航` 又快又准还免维护,不需要语义那一层——Claude Code 弃用 RAG 正是这个原因(性能更好 + 无索引漂移 + 无安全负担)。
2. **有界 + 需全局推理的数据**(单份合同、整本书对比): `长上下文` 直接全塞进去,省掉整套检索栈,还避开了静默失败,反而更适合。

此外,即便需要语义,提供语义的活也**不再由 embedding 独家承包**:模型够强时, `Agentic + BM25` (模型自己重写 query,把语义翻译成关键词)也能达到甚至反超传统切块 RAG——所以 embedding **从必选降为可选。**

但传统 RAG 在这些场景仍不可替代

1. **海量非结构化知识库**(客服、法律、合规、企业 Wiki,乃至 TB/PB 级数据湖):长上下文只是杯水车薪,必须靠检索层先把数据过滤到能塞进窗口的量,向量库是目前唯一可行的仓库。
2. **延迟 / 成本有硬约束的生产环境**:面向大量用户的语义问答系统、有明确延迟与成本 SLA(如高并发客服问答),既不能对每个 query 跑多轮 agentic 检索(太慢、2–5x token),也不能每次把知识库塞进长上下文(装不下、更贵)。预建索引的 RAG 一次检索、毫秒级命中、每查询成本固定且低——这时它是唯一能同时扛住并发与预算的方案。

RAG 仍具显著优势(虽未到不可替代)的场景:

强依赖语义搜索、但同时在意成本与速度:这类任务 `Agentic + BM25` 确实能取得相同甚至更好的效果——所以谈不上不可替代。但 agentic 方案**强依赖顶级模型能力**(query 重写要够聪明才补得出语义),还要付出多轮迭代的 **2–5x token 和更高延迟。**相比之下,传统 RAG 把语义**预先烧进索引**,一次检索即得,**更快、更省、且不挑模型**(不需要 Opus/GPT-5 级也能跑)。所以当语义需求强,但你不愿为 agentic 买单时,RAG 虽非不可替代,仍是**性价比更优的一档。**

一句话:传统 RAG 从默认必选降级为**按需启用的一档——在小型/结构化/有界场景被 grep、长上下文、agentic 搜索取代;在海量非结构化、以及对延迟成本有硬约束的生产场景仍不可替代**;即便在 agentic 可替代的语义场景,它也常因更快、更省、不挑模型而保有优势。而广义 RAG(检索)作为一切 Agent 的地基,永远不会过时。

16.9.4 场景题小测试

我们上面讲了，狭义的RAG变成了按需取用，它只是Agent检索能力的一种实现方式。那么对于不同的场景，我们应该如何选择方案呢？这里给一些场景和选型策略供大家参考。

思路统一为三步——**看数据结构化程度 → 看规模 → 看查询形态**,能用轻的就不上重的。

最佳实践清单

1. **默认 grep 起手**:90% 的代码搜索靠 grep 就够,别一上来就上向量库。
2. **量级到了再加 BM25**:代码库 >5GB 或 >10w 文件,包一个 Skill 封装 `ripgrep --index` 或 BM25 库。
3. **跨同义词才用 embedding**:日常代码定位别用,跨语言 / 跨概念查找才上语义搜索。
4. **模型够强就上 Agentic RAG**:让模型自己重写 query、按文件级迭代,比预切 chunk 更准(代价 2–5x token)。
5. **文件级 > chunk 级**:能不切块就不切块,整文件交给模型,避免上下文撕裂。
6. **需要 metadata join 才迁数据库**:纯文本搜索别上 DB。

Q:我在做一个 coding agent,要在中小代码库里定位函数/改代码,用什么?

A:用 `grep / ripgrep`,不建任何索引。代码是结构化数据,字面精确,grep 现扫就 100% 命中且免维护;本地项目规模不大,线性扫盘足够快。这正是 Claude Code 的选择。

Q:代码库超大(比如 100GB、上千万文件),grep 扫盘太慢了怎么办?

A:加一层 `BM25 倒排索引`(包成 Skill 挂给 agent)。它查询 O(K)、与库大小无关,解决 grep 线性扫盘的慢;仍是词法匹配,不碰 embedding。这是规模把你从零索引推到轻索引,不是因为需要语义。

Q:要查用户鉴权相关的代码,但关键词散在 auth / login / session / jwt 里,grep 搜不全?

A:这是跨同义词/跨概念的查找,字面匹配天然漏召回,有两条路都能走,按模型强度 vs 成本/延迟权衡:

- `embedding 语义搜索`:预建向量库,靠向量捕捉概念相似,一次召回稳定拿到结果,查询快、单次成本低——适合成本/延迟敏感或高频查询。
- `Agentic + grep/BM25`:让模型自己把用户鉴权重写成 `auth|login|session|jwt` 一组关键词、多轮迭代搜。省掉建索引,但要求模型能力够强,且多轮迭代 token 和延迟更高。

注意:日常精确定位(已知函数名)别用这两种,grep 直接命中就好。

Q:要分析一份合同、或对比两份文档找出遗漏了什么,用 RAG 吗?

A:不唯一,看数据量和关联复杂度选:

- **数据塞得下 → `长上下文` 直接全塞进去**:需要全局推理,让模型看到全貌才能发现文档之间的空缺,还省掉检索栈、避开静默失败。这是有界数据的首选。
- **数据太大塞不下 → 仍要 `RAG` 先过滤**:普通向量 RAG 只给零散 chunk 容易看不到全局,可退而用它把范围缩到能塞进窗口的量再做对比。

- **实体/条款间有复杂关联 → `GraphRAG`**:先把文档抽成实体-关系图,用图结构显式建立这条需求 ↔ 那条发布说明的关联,比纯向量召回更能支撑缺了什么这类跨文档推理。

Q:要给公司几十万份客服/法律/Wiki 文档做问答系统,用什么?

A:标准 `传统 RAG`(切块 + embedding + 向量库)。典型的海量非结构化数据:字面对不上、靠概念含义组织、塞不进上下文——必须靠语义检索 + top-k 降噪,这是 RAG 的主场。

Q:企业数据到了 TB / PB 级,长上下文完全装不下,怎么办?

A:必须有 `RAG 检索层`。百万 token 在这种量级只是杯水车薪,得先用向量检索把数据过滤到能塞进窗口的量;向量库是目前唯一可行的仓库。

Q:查询需要按作者过滤 + 按提交时间排序 + 按路径匹配这种带条件的检索?

A:上 `结构化 DB / metadata 索引`(可与 RAG 混合)。这已经不是找相关内容而是按字段筛选 join,grep 和向量都做不了,需要数据库式的字段过滤。纯文本搜索则别上 DB。

Q:模型够强(Opus / GPT-5 级)、预算也够,想要最好的检索质量?

A:上 `Agentic RAG`——让模型自己重写 query、按文件级迭代检索,底层用 BM25 / embedding / 混合都行,坚持文件级 > chunk 级避免上下文撕裂。代价是多花 2–5x token 和延迟,换来更高准确率。

16.9.5 面试题

希望大家能真的理解上面说的，有自己的思考。那么如果从这些维度想清楚了，下面的面试问题的答案其实就非常明显了。

16.9.5.1 Q1:你认为 RAG 过时了吗?

A:我的判断是——传统 RAG 在部分场景被降级,但整体没有过时,更准确地说是换了形态。回答这个问题,必须先厘清在讨论哪个 RAG。

大家喊RAG 已死时,死的是**切块 + embedding + 向量库 + 语义召回这套传统管线**;而 RAG 的本意是把外部信息检索进上下文,这个**广义 RAG 是所有 Agent 的地基,不可能死**。连 Agentic RAG 本身也仍然是 RAG,只是把一次检索变成迭代检索。

传统 RAG 确实在退位,有两股力量:

- **一是长上下文**:百万 token 窗口让有界数据 + 全局推理的场景(如分析单份合同、对比两份文档找缺漏)可以直接把文档全塞进去,省掉整套检索基础设施,还避免了语义搜索的静默失败。
- **二是 Agentic 搜索**:模型变强后,它能自己重写 query、多轮迭代检索,有实验(Raj)显示Agentic + 纯BM25能反超One-shot + embedding 切块——说明语义近似这层不一定非靠 embedding。

但传统 RAG 在三类场景仍不可替代:

- **① 海量数据(企业 TB/PB 级)**:这种量级下长上下文根本装不下,只是杯水车薪;而且如果不上 RAG、改用 agentic 现扫,面对无限数据每次查询都要海量扫描,**速度会慢到不可用**。必须靠检索层先把数据过滤到能

塞进窗口的量,向量库是目前唯一可行的仓库。

- ② **非结构化 + 跨同义词**:内容靠概念含义组织、字面对不上,关键词根本搜不到,只有 embedding 捕捉概念相似才能召回。虽然Agentic + 语义检索(让模型自己重写 query)理论上也能达到,**但这要求模型能力足够强,且多轮迭代的 token 成本和延迟都很高**;用预建的 embedding 向量库,一次召回就能稳定拿到结果,更省更快。
- ③ **成本 / 延迟敏感的场景**:当你需要严格控制查询成本和响应速度时,**事先建好索引**的优势就出来了——切块 + 建索引只在离线做一次,查询时只拉 top-k 个小 chunk,近似最近邻 O(log N) 一次命中;相比每次都让模型读整份文档或多轮 agentic 扫描,**延迟更低、单次查询成本可低约两个数量级。**

所以结论是:RAG 从默认必选降级为按需启用的一档,而不是被淘汰。 判断要不要用,看三个条件:**文档规模、查询是否需要跨同义词、以及对成本 / 延迟的要求。**

16.9.5.2 Q2:你如何选择用不用 RAG?如何设计项目的检索模块?

A:核心原则是——不要把通用工具当万能工具,而要按场景在检索谱系上选一档;能用轻的就不上重的。

我会分三步走:

第一步,看任务量级和查询形态。 先问两个问题:数据多大(单项目 / 超大代码库 / 企业级数据湖)?查询是纯文本、跨同义词、还是带字段过滤?

第二步,在检索谱系上选档。 检索工具从词法到语义是一条谱系,不是二选一,从轻到重依次是:

- **词法 • 精确(grep / ripgrep)**:适用于结构化数据、中小代码库(默认起手,90% 够用)。
- **词法 • 倒排(BM25)**:适用于超大代码库(>5GB / >10w 文件)。
- **语义(embedding + 向量库)**:适用于非结构化、跨语言 / 跨同义词。
- **结构化(DB + metadata)**:适用于需要 join 的场景(按 author 过滤、按时间排序)。

具体判据:

- **默认 grep 起手**;结构化数据 + 精确匹配,根本不用 RAG。
- **规模上去了加 BM25**;跨同义词才上 embedding。
- **有界数据 + 全局推理** → 优先考虑长上下文,而非 RAG。
- **海量非结构化** → 传统 RAG 是唯一可行仓库。
- **需要 metadata join** → 结构化 DB + RAG 混合。
- **模型够强 + 预算够** → 上 Agentic RAG(底层 BM25 / embedding / 混合均可),让模型自己迭代,并坚持**文件级 > chunk 级**,避免上下文撕裂。

第三步,用 Skill / 工具封装,交给 agent 按需调用。 更前沿的做法是**给 agent 同时配好精确搜索 + 语义搜索两种工具**,让它根据具体 query 自己决定走哪条路——这就是桥接思路。

一句话:**检索模块的设计,本质是 harness 工程——agent 的能力上限不只取决于模型,更取决于你给它配的工具链。**

16.9.5.3 Q3:Claude Code 为什么不用 RAG?

A:先纠正一个常见误解——不是不能用,而是核心不内置那套向量库管线。原因来自数据环境和工程权衡两方面,Claude Code 核心作者 Boris Cherny 给过第一手复盘。

先说根本原因:决定用不用 RAG 的是数据环境,不是任务类型。 Claude Code 虽然是通用 agent,但它面对的数据环境永远是**本地文件系统 + 终端**——有界(单个项目)、天然结构化(文件夹 / 路径 / 文件名)、可直接 grep。这种环境用 agentic search + 长上下文就够了。而 RAG 的刚需场景是海量 + 非结构化 + 无法直接导航的企业级知识库,Claude Code 的工作目录一条都不占。

再说 Boris 的三条具体理由(早期 Claude Code 确实用过 Voyage embedding 建索引,后来放弃):

1. **性能**:agentic search全面碾压,大幅度(他坦承主要凭内部 vibes,也有内部 benchmark)。
2. **代码漂移**:代码改动极频繁,向量索引很快out of sync,维护成本高。
3. **安全 / 责任**:索引要存在某处,服务商被黑就是巨大的责任风险,Anthropic 不愿上传敏感代码。

他的总结:**代价是延迟和 token,但换来极棒的搜索,且没有安全隐患。**

补充两点让回答更完整:

- 代码是**结构化**数据,拼写精确、自带文件结构,grep 能精确命中,本就不需要 embedding 的语义近似超能力。
- Claude Code 不内置 RAG,**不代表它做不了 RAG**——它可以通过 MCP / 工具外挂一个语义检索能力。**RAG 对它是按需外挂,不是出厂内置。** 为了少数场景让所有用户默认背上索引复杂 + 漂移 + 安全责任这套包袱,不划算;核心保持精简,谁真需要谁再插上去。

16.9.6 讲解视频

https://www.bilibili.com/video/BV1YdMp6kEEf/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

AI 工具与工程实践【主线，AI时代素养】

在这个AI快速发展的时代，很多工作方式都在被颠覆。很多技巧，其实无关乎是否是大模型的岗位。作为一个传统的开发，无论你是**什么开发，前端后端测试，甚至你是产品经理，本章的内容你都该掌握。**比如提示词工程，甚至是高中生都可以学习一下如何更好的使用提示词和AI交互。Agent SKILL,帮助我们定义工作流，避免每次从头和AI打招呼。本章的内容不光对**面试**很重要（越来越多的公司想要考察你如何使用AI，会不会聪明地用AI，这无关乎你的岗位），也对你自身的发展，工作效率的提升有帮助。

所以本章，其实无论背景，都建议学习。很多东西比如**SKILL也是面试高频考点。**

1 Claude Code 从入门到精通

Claude Code本身作为T0级别的Vibe Coding，本身就值得我们学习。希望大家不要把眼光局限在自己公司的工具，比如腾讯就用CodeBuddy上。不同的工具差异还是很大，即使我们公司有Github Copilot, 但是很多厉害的同事慢慢还是倾向于去用Claude Code。之前有面试官问我会使用什么Vibe Coding的工具，我就答只用Github Copilot。我当时还觉得这就是一个闲聊的问题，但是后来才发现，对于工具的使用和比较，本身就是体现了你是否关注时代，会聪明地，高效地使用AI的一个考察点。 厉害的人一定会使用最好的工具，这其实也是一种专业素养。另外也有粉丝朋友和我反映，面试官喜欢问Insights，这就是Claude Code里的一个特性，所以从面试的角度，我们其实也是需要掌握Claude Code的。

从使用的层面，我也确实希望大家能够使用Claude Code， 思考一下不同工具的不同。我们公司也在推荐使用Claude Code，而整个Claude Code如何使用，怎么使用，本身行业也在研究。我举个例子，本节视频里讲了如何用Claude Code去做自动化，这一套方法，如果你可以用在你的工作中，绝对是领先行业的。在我们公司，也只是比较少部分的人会研究Claude Code的高级使用方法，如果你能够学会，在工作中使用，比如用ClaudeCode去做一个自动化做工作测试或者部署，分享在你们公司，一定会让领导称赞，同时你可以在面试中讲，也是加分项。

本节介绍了ClaudeCode的基本使用方法，常见命令，技巧，也总结了一个开源项目everything-claude-code（大神总结的Claude Code的进阶技巧）的一些进阶技巧。同时给出了一些面试问题供参考。

通过本期的内容，你会了解ClaudeCode的相关用法和知识。但是最重要的是，希望你真的用起来，使用起来，将这些技巧用到你的工作中，使用它，提高你使用AI工具的能力，这也将是你在新时代的技能护城河。

1.1 Claude Code 概述

1.1.1 什么是 Claude Code

Claude Code 是 Anthropic 推出的 AI 编程 Agent，以对话的方式帮助开发者构建、调试、重构代码，以及构建自动化工作流。它不同于传统的代码补全工具（如 GitHub Copilot），而是一个可以自主规划、执行任

务、调用外部工具的智能代理。

Claude Code 本质上是一个**通用编程 Agent**，并不与 Claude 模型强绑定。你可以用 Anthropic 官方订阅（Pro/Max 会员）或 API Key 驱动它，也可以配置环境变量让它使用国产大模型（如 GLM、MiniMax 等）驱动。市面上同类产品（Codex、OpenCode 等）的功能和用法与 Claude Code 高度相似，学会 Claude Code 就能举一反三。

1.1.2 Claude Code 能做什么

Claude Code 可以：

- 根据自然语言需求从零生成完整项目代码
- 读取、修改、重构现有代码文件
- 执行终端命令（安装依赖、运行测试、启动服务器等）
- 调用 MCP 外部工具（连接 Figma、GitHub、数据库、Slack 等第三方服务）
- 构建并自动执行复杂的多步骤自动化工作流
- 并发运行多个子任务（SubAgent 并行处理）
- 自我修复：遇到错误时能分析原因、修改代码、重新测试
-

一个实际案例：在一次对话中，Claude Code 自动发现并抓取了 30 个 YouTube 频道的 187 个视频数据，生成了 6 张图表，制作了一份 9 页的 PowerPoint 演示文稿，导出到了 Google Sheets，并通过 Gmail 发送了带有 PDF 附件的周报——这一切只需要一段自然语言描述。

实现过程简述：

在 Plan Mode 下用一句话描述需求，Claude Code 联网调研后反问几个问题（追踪哪些频道、发送频率、收件邮箱等）

Claude Code 自动生成项目结构：一个描述步骤的 Markdown 工作流文件 + 7 个 Python 工具文件（抓取数据、分析、生成图表、生成 PPT、发邮件、导出 Sheets、发现频道）

人工只需完成一件事：去 Google Cloud 控制台开通 YouTube Data API / Gmail / Sheets，下载 OAuth JSON 凭据文件拖入项目；API Key 填入 `.env` 文件

Claude Code 自动安装依赖、测试 API 连通性、跑完整流水线，发现问题自行修复

最终通过 Modal 将工作流部署到云端，配置每周一早 6 点定时触发，API Key 存为 Modal Secrets，不写入代码

> 📺 想完整了解实现过程，参考视频：**Master 95% of Claude Code in 36 Minutes** https://www.youtube.com/watch?v=saggDHHnmtQ （视频以这个 YouTube 周报自动化项目为主线，从界面安装到部署上线全程实操演示）

1.1.3 访问方式

- **独立运行**：在终端直接执行 `claude` 命令启动，不依赖任何 IDE，适合脚本自动化和服务器环境
- **作为插件**：嵌入到 VS Code、Cursor 等编辑器中使用，界面集成在 IDE 侧边栏，与代码编辑窗口无缝切换
- 需要 Claude 付费订阅（Pro 计划起，高频使用建议升级 Max）

1.1.4 收费标准

Claude Code 包含在 Claude 的付费订阅中，无需单独购买。

- **Pro**：\$17 / 月（按年付），约 \$20 / 月（按月付）。包含 Claude Code 访问权、更多模型、无限 Projects、Deep Research、extended thinking、Google Workspace 集成、Remote MCP 集成等。适合入门和轻度使用。
- **Max**：从 \$100 / 月起（按月付），可选 5x 或 20x 用量档位。在 Pro 基础上增加更高用量上限、更高输出限制、高峰期优先访问、新功能抢先体验。适合高频使用、复杂工作流和长对话场景。

> 建议：初学者从 Pro 起步，一旦频繁跑自动化工作流、遇到用量限制，再升级到 Max。

1.1.5 中国区安装

由于政策原因，中国区域是没法直接安装和使用Claude Code，不过网上也有很多教程教你如何绕过这一限制，相关视频很多，大家可以自行搜索，这个过程还是需要花一些功夫配置的，就像龙虾的安装，本身也是有一定门槛。但是网上资料很多，很多博主也会分享，相信你可以搞定的。

可以参考下面我给的这个教你配置的视频，当然也可以自己搜索，网上很多，关键字就是：中国区/Claude Code

0基础入门Claude Code：接入千问/Kimi等国内模型 | 拒绝封号！超级小白也能学会打造最强AI Agent！

https://www.youtube.com/watch?v=nfMJZEIpfPo

1.2 核心特性与用法

1.2.1 CLAUDE.md

是什么：CLAUDE.md 是放置在项目根目录（或用户目录）的 Markdown 文件，Claude Code 每次启动时会自动读取它，相当于给 Agent 的持久化系统提示。

解决什么问题：Claude Code 每次新会话都是"失忆"的，不知道你的项目结构、编码偏好、注意事项。CLAUDE.md 解决了这个问题，让 Agent 每次启动就能快速进入状态。

怎么用：

- 在 Claude Code 对话中输入 `/init`，它会根据当前项目自动生成一份 CLAUDE.md
- 输入 `/memory` 可以快速打开并编辑 CLAUDE.md
- 分为**项目级**（放在项目目录，对该项目生效）和**用户级**（放在用户目录，对所有项目生效）
- 内容可以包括：编码规范、项目结构说明、注意事项（比如"每次回答结尾都追加 Happy Coding"）、技术栈偏好等

示例内容：

```Plain Text
```

项目规范

- 使用 React + TypeScript + Vite 技术栈
- 所有组件必须有对应的单元测试
- 禁止在代码中硬编码任何 API Key
- 提交代码前必须运行 lint 检查

```
**进阶：Rules 文件夹（`.claude/rules/`）**

当规则越来越多时，单个 CLAUDE.md 会变得臃肿难维护。Claude Code 官方支持将规则拆分到 `.claude/rules/` 目录下，按关注点分模块管理，Claude Code 启动时会自动读取所有文件：

```Plain Text
.claude/rules/
├── security.md       # 安全规则：不能硬编码密钥，要验证输入
├── coding-style.md   # 代码风格：不可变性、模块化
├── testing.md        # 测试规则：TDD 流程、80% 覆盖率
├── git-workflow.md   # Git 规范：提交格式、PR 流程
└── agents.md         # 何时委托给 SubAgent
```

CLAUDE.md vs Rules 文件夹：两者功能相同，都是持久化系统指令；区别在于组织方式——CLAUDE.md 适合放项目总览和简单规则，Rules 文件夹适合规则较多、需要分模块维护的团队项目，不同角色可以维护各自的规则文件。

1.2.2 三种交互模式

Claude Code 通过 `Shift + Tab` 键在三种模式间循环切换：

默认模式（底部显示 `? for shortcuts`）：最谨慎，每次创建或修改文件前都会询问用户确认。适合对重要代码做修改时使用。

自动模式（底部显示 `accept edits on`）：本次会话内所有文件操作自动通过，不再打扰用户。适合已信任的任务，节省时间。

规划模式（Plan Mode）（底部显示 `Plan Mode On`）：只讨论、只阅读，不修改任何文件。Claude 会深入分析需求、阅读项目文件、提出问题，生成详细的执行计划，确认后才能切换到执行模式。这是处理复杂任务的推荐第一步。

注意：即使在自动模式下，执行终端命令（如 `npm install`、`mkdir`）仍然需要单独确认，因为这类操作被认为风险更高。如果想跳过所有权限检查，可以在启动时加 `--dangerously-skip-permissions` 参数，但官方已将 `dangerously` 写在参数名里警告风险，务必谨慎使用。
```

1.2.3 Checkpoints

是什么：Claude Code 每次提交修改前，都会自动为当前代码创建一个快照（回滚点）。

怎么用：

- 输入 `/rewind` 或连续按两次 `ESC` 进入回滚界面
- 选择要回滚的时间点后，有四个选项：回滚代码+会话、只回滚会话、只回滚代码、放弃回滚
- 也可直接输入 `/checkpoints` 查看文件级别的回滚点

局限性：Claude Code 只能回滚它自己写入的文件，由终端命令生成的文件（如 `npm install` 产生的 `node_modules`）无法回滚。精确回滚建议配合 Git 使用。

1.2.4 权限管理

是什么：控制 Claude Code 可以执行哪些操作的机制。

怎么用：在对话中输入 `/permissions` 打开交互式权限菜单，可以将安全的操作（如运行测试、提交代码）加入白名单，将危险操作（如删除文件）加入黑名单。

权限保存位置：

- `settings.local.json`（本地项目级，不提交 Git）
- `settings.json`（项目级，随 Git 分发给所有团队成员）
- 用户目录配置（用户级，对该用户所有项目生效）

经验：建议从保守权限开始，随着对工作流的信任逐步放开。权限设置是速度与安全的平衡。

1.2.5 Plan Mode

使用 `Shift + Tab` 切换到 Plan Mode（或在对话框按 `Shift + Tab` 两次）。

在规划模式下，Claude 会：

- 深度阅读你的项目所有相关文件
- 通过提问澄清需求细节（如：频率是每天还是每周？是否需要导出到 Google Sheets？）
- 生成包含目标、步骤、文件结构的详细执行计划
- 列出依赖项和注意事项

计划产出后有三个选项：直接执行（自动同意模式）、执行但保留逐步确认、继续修改计划。

何时必须用 Plan Mode：处理重构、多文件改动、新功能开发等复杂任务前，强烈推荐先用 Plan Mode 规划，可以避免 Claude 误改文件、浪费 Token。

1.2.6 Hooks

是什么：在 Claude Code 工作流程的特定时机自动执行的脚本，无需人工干预。

六种触发时机：

- `PreToolUse`：工具执行前（用于验证、提醒）
- `PostToolUse`：工具执行后（用于格式化、反馈）
- `UserPromptSubmit`：用户发送消息时
- `Stop`：Claude 完成响应时（适合记录总结）
- `PreCompact`：上下文压缩前（适合保存重要状态）
- `Notification`：权限请求时

典型用法举例：配置一个 `PostToolUse` Hook，在 Claude 写完代码后自动调用 Prettier 格式化文件，实现代码自动美化，整个过程无需用户操作。

怎么配置：输入 `/hooks` 进入配置界面，选择触发时机 → 选择匹配的工具（如 `write` 或 `edit`）→ 填写具体执行命令。命令通过 JSON 标准输入接收上下文信息，其中 `file_path` 字段为刚编辑的文件路径。

Hook 存储级别：本地项目级（不入 Git）、项目级（随 Git 分发给团队）、用户级（对当前用户所有项目生效）。

1.2.7 MCP

是什么：MCP是一个开放协议，允许任何人为 AI Agent 构建和暴露外部工具接入。类比一个"通用 USB 接口"，只需连接一次，Claude 就能使用该服务的所有功能。

与直接调用 API 的区别：传统 API 调用需要开发者明确指定每个端点和参数。MCP 是一种提示驱动的封装，让 Claude 能自己判断调用哪个工具、传什么参数，更灵活。

常用 MCP 场景：

- Figma MCP：Claude 可直接读取设计稿的截图、组件间距、字体样式，实现精准的设计稿还原
- GitHub MCP：管理 PR、Issues、代码审查
- Supabase/数据库 MCP：直接执行 SQL 查询，拉取特定数据
- Gmail/Calendar MCP：发送邮件、创建日历事件
- Slack MCP：读取和发送消息

怎么用：输入 `/mcp` 查看已安装的 MCP 工具及其状态；安装 MCP Server 后重启 Claude Code 生效；使用 `/mcp` 可以进入认证、查看工具列表等操作。

多模态输入：除了 MCP，也可以直接向 Claude Code 传图片——将图片拖入对话框，或复制图片后按 `Ctrl+V`（macOS 也是 Ctrl+V，不是 Command+V）粘贴。

1.2.8 Skills（Agent Skill）

是什么：技能文件（Skill.md）是预定义的指令集，包含名称、描述和具体步骤说明。Claude Code 根据用户请求自动识别并加载相关 Skill，而不是每次都重新理解相同的流程。

与 CLAUDE.md 的区别：CLAUDE.md 是每次都加载的全局记忆；Skill 是按需动态加载的，只在需要时才占用上下文窗口，更节省 Token。

与 MCP 的区别：MCP 负责获取数据和执行外部操作；Skill 是知识型的自定义指令，相当于"告诉 Claude 怎么做某件事"的操作手册。

技能存放位置：

- 全局：`~/.claude/skills/`（对所有项目生效）
- 项目：项目目录下的 `.claude/skills/`

怎么触发：

- 自动触发：Claude 识别到请求与某个 Skill 相关时，自动请求用户确认加载
- 手动触发：在输入框中输入 `/技能名称` 直接调用

Skill 文件结构示例（以每日日报 Skill 为例）：

```
---
name: daily-report
description: 当用户要求写每日总结时调用此技能
---## 日报格式要求- 日期
- 开发摘要（不超过3条）
- 开发详情（每条需包含：实现内容、遇到的问题、解决方案）
- 明日计划
```

1.2.9 SubAgent & AgentTeam

SubAgent

是什么：SubAgent 是独立运行的 Claude 实例，拥有完全独立的上下文、工具集和权限，专注于完成某一特定任务，完成后只向主代理汇报最终结果。

与 Agent Skill 的核心区别：

Agent Skill 运行时完全继承并共享当前主对话的上下文，执行过程中产生的所有中间日志和思考过程都会进入主上下文，容易造成 Token 膨胀。

SubAgent 拥有独立上下文，执行的所有中间过程（读了哪些文件、中间分析过程）都不回传给主对话，只返回最终结果，主对话上下文始终保持干净。

适用场景：

- Agent Skill 适合：与主上下文关联紧密、不会产生大量中间数据的任务（如"写一份今日开发总结"）

- SubAgent 适合：需要大量探索、会产生海量中间上下文的任务（如"审查整个代码库的安全漏洞"）

怎么创建：输入 `/agent` → Create new agent → 选择项目级或用户级 → 选择 Claude 辅助生成或手动创建 → 填写任务描述 → 选择可用工具（如只读工具）→ 选择模型和颜色。创建成功后会是一个md的格式，和skill类似，也会包括头（description）和正文部分，类似这样：

```
---
name: pdf-book-retriever
description: Expert at answering questions from PDF books by locating relevant sections, extracting supporting passages, and producing citation-grounded responses.
tools: Read, Grep, Glob, Bash
model: sonnet
skill: code-review
---
#上面的头就是写对应的内容啊，模型啊．下面是正文写具体的要求
You are a specialized xxx
```

AgentTeam

是什么：

- AgentTeam 是由主代理一次性拉起的多代理协作小组，多个成员共享任务列表并可互相发送消息。
- 每个成员都有独立执行上下文与职责分工，由主代理统一编排、跟踪进度和汇总结果。
- 适合处理“多角色并行 + 互相依赖 + 质量回路”的复杂任务。

与 SubAgent 的核心区别：

- SubAgent 之间不能互相通信，只能各自把结果回传主代理。
- AgentTeam 成员可以直接互发消息、交接依赖、互相验收，并可把问题打回上游重做。
- SubAgent 更像“多个独立工人并行干活”；AgentTeam 更像“有协作协议的小组项目制”。
- 在复杂工程任务中，AgentTeam 通常质量更高，但也更慢、更贵。

适用场景：

- SubAgent 适合：任务彼此独立、主要需要并行探索与最终汇总、不需要成员间沟通的场景。
- AgentTeam 适合：前后端、测试、文档等多角色协作，且需要跨角色消息传递与持续反馈闭环的场景。
- 典型 AgentTeam 任务：构建全栈功能、跨模块重构、需要 QA 回归与多轮修复的项目。

怎么创建：

- 先启用 AgentTeam 功能（实验特性，默认关闭）：在项目设置中加入对应开关。
- 在主会话中用自然语言创建团队，例如：
- Create a team of 3 teammates using Sonnet: frontend developer, backend developer, QA。
- 为每个成员明确：角色职责、输入输出、需要通知谁、完成条件。
- 明确最终交付物：可运行结果、测试报告、关键决策说明。

- 建议控制团队规模在 2 到 5 人，避免无效并发和 token 失控。

创建后的产物形态：

- AgentTeam 通常是运行时创建的协作结构，不一定像 SubAgent 一样直接落成单个 agent 配置 md。
- 更常见的可落盘产物是：
- 团队执行日志与阶段性报告
- 最终交付文档（实现说明、测试结果、后续动作）
- 项目内团队规范文档（可选），用于复用同类团队分工模板

何时用 AgentTeam，何时用 SubAgent

当任务是“多人协作型”时，用 AgentTeam。典型特征是：需要多个角色同时推进（例如前端、后端、QA）、成员之间有明确依赖、需要互相发消息和反复验收。这类任务通常流程复杂、质量要求高，AgentTeam 能把沟通与回归闭环做得更完整。

当任务是“独立执行型”时，用 SubAgent。典型特征是：任务可以拆成多个互不依赖的子任务并行处理，最后由主代理统一汇总；或者流程是明确串行的 1 -> 2 -> 3，不需要成员彼此沟通。SubAgent 在这种场景下更轻量、更直接。

如果任务强依赖同一段对话历史、需要频繁即时来回修改，优先留在主会话，或者只引入少量 SubAgent 做辅助，不一定要上 AgentTeam。

如果任务本身简单、链路短、成本敏感，也优先主会话或少量 SubAgent。因为 AgentTeam 在成员并发时会显著增加开销，通常更适合“复杂且值得协作”的工作，而不是日常小任务。

成本提醒：

- 多成员并发会近似倍增消耗（3 人约 3 倍、5 人约 5 倍）。
- 先用小团队验证流程，再扩大规模。

一句话结论：复杂协作上 Team，专注执行上 SubAgent。

深入学习

AgentTeam和SubAgent是CC的高级使用技巧，如果想深入学习，可以看：

参考视频：6:52 \~7:27 是专门讲解SubAgent和AgentTeam的使用的。

https://www.youtube.com/watch?v=mpALXah_PBg&t=24772s

官方API文档，有专门讲SubAgent的

https://developer.anthropic.com/

小技巧，使用ClaudeCode：@"claude-code-guide (agent)"，这个就是CC内置的官方文档专门的Agent。

所以你和它对话，相当于让他基于官方文档给你回答，你可以问他相关SubAgent的知识。

这里9.4 小节，讲了SubAgent的md配置文件。9.2 讲了ClaudeCode的内置智能体。感兴趣可以去看。

从源代码层面了解Claudecode SubAgent和AgentTeam，可以看这里

1.2.10 Plugins

是什么：将多个 Skill、Hook、SubAgent、MCP 等能力打包成一个可安装单元，类似 macOS 的 DMG 安装包。一键安装即获得整套能力，方便团队共享配置。

怎么用：输入 `/plugin` 进入插件管理器 → Discover 发现新插件，Installed 查看已安装，Marketplaces 管理插件市场。

推荐插件：

- `frontend-design`：打破大模型"深紫色主题"惯性的前端设计 Skill，让生成的界面更有设计感
- `hookify`：用对话方式创建 Hook，无需手写 JSON 配置
- `typescript-lsp` / `pyright-lsp`：在编辑器外使用 Claude Code 时提供类型检查和智能补全
- `mgrep`：比 ripgrep 更高效的代码搜索工具，平均减少约 50% 的 Token 消耗

1.2.11 上下文管理

Claude Code 拥有约 200,000 Token 的上下文窗口，管理好上下文是高效使用的关键。

查看上下文：输入 `/context` 查看当前上下文的用量分布（CLAUDE.md、Skill、MCP 工具描述、对话历史各占多少）。

压缩上下文：输入 `/compact`（可附加保留策略，如"重点保留用户的原始需求"），Claude 会将对话压缩为摘要，保留关键决策，释放空间。

清空上下文：输入 `/clear` 完全清空所有上下文。适用于开始一个全新的、与之前任务完全无关的任务。

恢复历史对话：

- 输入 `/resume` 可查看历史会话列表并选择恢复
- 启动时加 `-c` 参数（`claude -c`）自动恢复上次的对话

后台任务管理：

- 启动的服务器或长时命令会阻塞 Claude Code，按 `Ctrl + B` 将其放入后台
- 输入 `/tasks` 查看后台运行的任务
- 在任务界面按 `K` 可终止任务

1.2.12 斜杠命令

常用斜杠命令速查：

- `/init` — 自动生成当前项目的 CLAUDE.md
- `/memory` — 快速打开 CLAUDE.md 编辑
- `/compact` — 手动压缩上下文
- `/clear` — 清空所有上下文
- `/context` — 查看上下文用量
- `/rewind` — 进入代码回滚界面
- `/checkpoints` — 查看文件级回滚点
- `/resume` — 恢复历史对话
- `/tasks` — 查看后台任务
- `/mcp` — 查看和管理 MCP 工具
- `/hooks` — 配置 Hook
- `/skills` — 查看已安装的 Skill
- `/agent` — 创建和管理 SubAgent
- `/plugin` — 插件管理器
- `/permissions` — 权限管理
- `/fork` — 分叉当前会话，用于并行执行不重叠的任务
- `/rename` — 重命名当前会话
- `/insights` — 生成个人 Claude Code 使用分析报告
- `/statusline` — 自定义状态栏显示内容

1.2.13 Insights

(这个是最近（2026.3月）面试喜欢考的一个特性)

Claude Code 的 `/insights` 命令可以分析你所有的历史使用数据（跨项目），生成一份 HTML 格式的个人使用报告，保存在用户目录的 `~/.claude/usage-data/report.html` 位置。

报告内容包含：

- 你使用 Claude Code 的典型模式和习惯
- 做得好的地方（Impressive Things）
- 哪些地方在阻碍你的效率（Where Things Go Wrong）
- 值得尝试的新功能和使用模式
- 工作流自动化建议

实用技巧：生成报告后，可以将 HTML 文件上传给 ChatGPT 或其他 AI 工具，让它提炼最值得执行的 3-5 条行动建议（用不同 AI 分析避免数据偏向性）。

1.3 最佳实践经验

以下最佳实践来自 everything-claude-code 项目（Anthropic Hackathon 获奖作品，作者每日使用 Claude Code 超过 10 个月的实战总结）。

1.3.1 项目初始化必做三件事

在任何新项目开始前，先完成以下配置，后续效率会高很多：

第一件：创建 CLAUDE.md，写清楚技术栈、编码规范、项目结构、禁止事项（如禁止硬编码密钥、禁止使用 emoji 等）。

第二件：如果是说项目本身的描述比较多，可以将CLAUDEM.D的内容下放 Rules 文件夹（`.claude/rules/`），按关注点分模块管理规则（但并不是所有项目都必须写rules, 项目简单规则就放在CLAUDE.MD中就行）：

```Plain Text
security.md # 安全规则：不能硬编码密钥，要验证输入
coding-style.md # 代码风格：不可变性、模块化
testing.md # 测试规则：TDD 流程、80% 覆盖率
git-workflow.md # Git 规范：提交格式、PR 流程
agents.md # 何时委托给 SubAgent
```

> **第三件**：选择合适的 Skill 和 MCP，使用 `/plugin` 一键安装团队共用配置。
>
> ---
>
>
> #### 1.3.2 先规划，再执行（Plan Mode 优先）
>
> 复杂任务一定要先进入 Plan Mode，让 Claude 充分理解需求后再执行。盲目执行会导致：
>
> - 修改了错误的文件
> - 产生大量无用的中间 Token 消耗
> - 后期难以回滚
>
> 在 Plan Mode 中，尽可能把需求描述清楚，包括目标、功能点、边界条件、期望输出格式，Claude 会主动提问补充细节。
>
> ---
>
>
> #### 1.3.3 控制上下文，定期"清理"
>
> 上下文窗口满了之后，Claude 的表现会急剧下降（变慢、变傻）。建议：
>
> - 关闭不需要的 MCP：即使安装了 20-30 个 MCP，也只保持少于 10 个处于启用状态，活跃工具不超过 80 个。过多的 MCP 工具描述会占用大量上下文，200k 的窗口可能只剩 70k 可用。
> - 在完成一个大阶段任务后，手动执行 `/compact` 压缩，保留关键决策，释放空间
> - 开始全新任务前用 `/clear` 完全清空，避免旧上下文干扰
> - 用 `/context` 定期检查上下文用量，在自动压缩前主动手动压缩以获得更好的控制效果
>
> ---

1.3.4 跨会话记忆持久化

Claude Code 默认不跨会话记忆，可以通过以下方式解决：

方案一：Session 文件：在每次会话结束前，让 Claude 生成一份会话总结文件，记录：本次完成了什么、什么方式有效、什么方式无效、还剩什么未完成。下次会话开始时，提供这个文件路径即可续上。

方案二：Stop Hook：配置一个 `Stop` Hook，在每次 Claude 完成响应时自动将关键信息追加到 `.claude/session.md` 文件，下次启动时加载。使用 Stop Hook 而非 UserPromptSubmit（每条消息都触发），避免增加响应延迟。

方案三：动态系统提示注入：

```Bash
# 按上下文切换不同的记忆文件 alias claude-dev='claude --system-prompt "$(cat ~/.claude/contexts/dev.md)"'alias claude-review='claude --system-prompt "$(cat ~/.claude/contexts/review.md)"'
```

1.3.5 SubAgent 模式提升效率

两实例启动模式：新项目开始时，同时打开两个 Claude Code 实例：

- 实例一（Scaffolding Agent）：负责搭建项目骨架、建立 CLAUDE.md 和 Rules
- 实例二（Research Agent）：负责调研技术选型、生成详细需求文档（PRD）、绘制架构图

顺序执行模式（Orchestrator with Sequential Phases）：

```Plain Text 第一阶段：用 Explore 子代理做代码库调研 → 输出 research-summary.md 第二阶段：用 Planner 子代理制定计划 → 输出 plan.md 第三阶段：用 TDD-Guide 子代理实现代码 → 输出代码变更 第四阶段：用 Code-Reviewer 子代理审查 → 输出 review-comments.md 第五阶段：如有构建错误，用 Build-Error-Resolver → 循环直到通过
```

每个子代理接收一个明确的输入文件，产出一个明确的输出文件，前一阶段的输出是下一阶段的输入，使用 `/clear` 在各阶段之间隔离上下文。

1.3.6 Token 优化策略

模型选择策略（根据任务复杂度降级使用更便宜的模型）：

- 代码库探索、文件搜索 → Haiku（速度快、成本低，够用）
- 单文件改动 → Haiku
- 多文件实现 → Sonnet（编程任务最优平衡）
- 复杂架构设计 → Opus（深度推理）
- 安全漏洞分析 → Opus（不允许遗漏）

- PR 审查 → Sonnet
- 写文档 → Haiku

90% 的编程任务使用 Sonnet 即可。首次尝试失败、跨越 5+ 文件、架构决策、安全关键代码时升级到 Opus。

代码模块化：将代码拆分为小文件（几百行而非几千行）不仅是好的工程实践，也直接降低 Token 消耗，提高首次执行成功率。

工具替换：使用 `mgrep` 替代 `grep`/`ripgrep`，在 50 个任务的基准测试中，mgrep 平均减少约 50% 的 Token 消耗，查询质量持平或更优。

1.3.7 有效的提示与沟通技巧

- **把需求当作给团队成员写 SOP**：越具体越好，包含目标、输入、输出、边界条件
- **WAT 框架**（Workflows-Agent-Tools）：将任务分为工作流（markdown SOP）、代理（负责协调）、工具（Python 脚本执行具体操作）三层，分离概率性 AI 推理和确定性代码执行
- **不要只描述目标，还要描述你的顾虑**：在让 Claude 部署代码前，说"先做安全审查，确保 API Key 没有暴露，然后再告诉我部署方案"
- **反复出现的提示应该变成 Skill**：如果你发现自己多次给 Claude 相同的指令，就应该把它写成 Skill 文件，让 Claude 自动加载
- **Ctrl+G 打开 VSCode 编辑器输入框**：当在终端输入框里需要多行复杂提示时，按 `Ctrl+G` 打开 VSCode 编辑，方便多行修改，保存后自动回填

1.3.8 安全最佳实践

- **绝对不要将 API Key 硬编码进任何文件**，应使用环境变量（`.env` 文件）或 Modal Secrets 等密钥管理服务
- **部署前主动要求安全审查**：让 Claude Code 检查代码中的漏洞，特别是 Webhook 是否有认证、Key 是否暴露
- **在 CLAUDE.md 中明确禁止规则**：如"禁止在代码中包含任何 API Key 或密码"
- **精确控制 SubAgent 的工具权限**：代码审查 SubAgent 只需只读工具，不应有写权限
- **使用沙盒模式**测试可能影响系统的危险操作
- **`--dangerously-skip-permissions` 意味着 Claude 拥有和你一样的终端权限**，仅在受控的开发环境中使用

1.3.9 并行工作流

Fork 对话：输入 `/fork` 分叉当前会话，在两个分支上并行执行不重叠的任务（例如：一个分支做后端 API 开发，另一个分支做前端 UI 开发）。

Git Worktrees 多实例并行：当多个 Claude 实例需要在有代码重叠的分支上并行工作时，使用 Git Worktrees 避免冲突：

```Bash
git worktree add ../feature-branch-a feature-a
git worktree add ../feature-branch-b feature-b
# 在不同目录分别启动 Claude 实例
```

Cascade 方法：在多终端并行时，用从左到右的顺序组织标签页（最旧的在左）。使用 `/rename` 给每个会话命名，专注于最多 3-4 个并行任务，避免在过多实例间切换引发混乱。

原则：以最少的并行实例完成最多的工作，并行不是越多越好。

1.3.10 连续学习机制

当 Claude 发现了有价值的调试技巧、项目特定的模式或处理边界情况的方法时，将这些经验保存为新的 Skill 文件，下次遇到类似问题时自动加载，避免重复"教" Claude 同样的知识，节省大量 Token 和时间。

1.4 面试问答 Q&A

1.4.1 Q：你了解 Claude Code 吗？能简单介绍一下它是什么吗？

A：Claude Code 是 Anthropic 推出的 AI 编程 Agent，可以通过自然语言对话来完成代码编写、重构、调试，以及构建自动化工作流。它不只是代码补全工具，而是一个能自主规划、执行终端命令、调用外部服务的智能代理。可以在 VS Code 等编辑器中使用，也可以独立在终端中运行。

1.4.2 Q：Claude Code 的 CLAUDE.md 是什么，有什么作用？

A：CLAUDE.md 是放在项目根目录的 Markdown 文件，Claude Code 每次启动时会自动读取它，起到"持久化系统提示"的作用。它解决了 Claude Code 新会话"失忆"的问题。内容通常包括编码规范、技术栈说明、注意事项等。用 `/init` 命令可以让 Claude 自动生成一份，用 `/memory` 命令可以快速打开编辑。

1.4.3 Q：你能分享一下你使用 Claude Code 的最佳实践吗？

A：有几条我认为最关键的实践：第一，复杂任务先用 Plan Mode 规划，让 Claude 充分理解需求再执行，避免误改文件。第二，控制 MCP 数量，只启用当前任务需要的 MCP，保持上下文窗口充裕。第三，把反复使用的提示写成 Skill 文件，减少重复提示和 Token 浪费。第四，部署代码前主动要求 Claude 做安全审查。第五，始终将 API Key 存在 `.env` 或密钥管理服务里，绝不硬编码在代码中。

Q：Claude Code 里的 Hooks 是什么？你有使用过吗？

A：Hooks 是在 Claude Code 工作流特定时机自动触发的脚本，有 PreToolUse、PostToolUse 等六种类型。一个典型用法是配置 PostToolUse Hook，在 Claude 写完代码后自动调用 Prettier 格式化文件，无需人工干预。还可以利用 Stop Hook 在会话结束时自动保存当前进度摘要，实现跨会话记忆持久化。

1.4.4 Q：你知道 Claude Code 的 /insights 功能是什么吗？能分享一下它的作用？

A：`/insights` 是一个斜杠命令，运行后会分析你所有的 Claude Code 历史使用数据（跨项目），生成一份 HTML 格式的使用洞察报告，存放在用户目录的 `~/.claude/usage-data/report.html`。报告会展示你的使用习惯、效率瓶颈、值得尝试的新功能等。一个实用技巧是把这个 HTML 文件上传给 ChatGPT，让它提炼出最值得优先执行的几条改进建议。

1.4.5 Q：如何在 Claude Code 中高效管理上下文窗口？

A：几个关键策略：保持启用的 MCP 少于 10 个（虽然可以安装 20-30 个，但大多数时候禁用）；在完成阶段性任务后手动 `/compact` 压缩；开始全新任务前 `/clear` 清空；对于需要跨会话延续的任务，在 CLAUDE.md 或专用文件中记录进度；使用 SubAgent 处理探索性任务，避免大量中间日志污染主对话。

1.4.6 Q：Claude Code 的权限系统是如何工作的？

A：Claude Code 有三层权限控制：文件操作权限（默认要确认，选择自动模式后本次会话内自动通过）、终端命令权限（即使在自动模式下，执行终端命令默认仍需确认）、完全绕过权限（使用 `--dangerously-skip-permissions` 参数，官方明确标注危险，需谨慎）。可以通过 `/permissions` 命令把信任的操作加入白名单，把危险操作加入黑名单，权限配置可以保存到项目级或用户级配置文件。

1.4.7 Q：Claude Code 的记忆功能是如何实现的？

Claude Code 的记忆系统由 4 种存储和 3 大运作机制组成。

4 种存储：

第一种是指令文件，也就是 CLAUDE.md。这是唯一由人手动编写的记忆，告诉 Claude 项目的规则和约束。它分 4 个层级：管理员全局、用户全局、项目级和本地私有，支持 include 指令引用其他文件，上限 4 万字符。

第二种是对话上下文，就是当前对话的消息数组，所有 LLM 应用都有，Claude Code 的特别之处在于配套了一套 5 层压缩策略来管理它。

第三种是 Session Memory，是一个后台子代理在你工作的同时自动维护的结构化笔记。对话 token 超过 1 万后启动，之后每 5000 tokens 更新一次。它的作用是恢复会话时快速加载上下文，以及在上下文压缩时充当现成的摘要来源。

第四种是持久记忆 Memdir，存在用户目录下的 memory 文件夹里，分 user、feedback、project、reference 四类。用一个索引文件 MEMORY.md 做入口，上限 200 行。这里有一个重要的设计决策：它故意不存代码事实，比如"某函数在第 30 行"这种信息。因为代码会变，但记忆不会自动更新，过时的代码记忆会变成危险的误导。代码相关的查询永远用 grep 和 glob 实时搜索，只有人的偏好和判断才值得持久化。

3 大运作机制：

加载方面，每轮对话前，所有层级的 CLAUDE.md 会被拼接在一起注入 system prompt。拼接顺序是利用了 LLM 的 recency bias，模型对 prompt 末尾内容天然给予更高注意力，所以排在后面的文件实际优先级更高。MEMORY.md 前 200 行自动加载，再用 Sonnet 小模型扫描所有记忆文件的标题，选出最多 5 条最相关的注入上下文。这是一种 push 模式，不用 RAG。

写入方面，三个后台 Agent 分工合作。extractMemories 在每轮查询结束后自动提取持久记忆，Session Memory 按 token 增量持续更新笔记，AutoDream 每隔至少 24 小时且至少 5 个会话后做一次记忆整合去重，类似人类睡眠时整理白天记忆的过程。

压缩方面则是 5 层流水线，从温和到激进逐层升级，这个下一题会展开。

核心设计思想可以用一句话概括：会变的东西实时搜，不变的东西持久存。

1.4.8 Q: 当模型上下文窗口满了，你是怎么做的?

版本一：以 OpenClaw 为例

OpenClaw 采用一级压缩，但流程很精细。

首先划分保留区，最近大约 2 万 tokens 的原始对话原封不动保留。然后把更早的旧消息按 token 数分成多个块，逐块调用 LLM 做总结，再把多块摘要合并成一个连贯的总结。压缩前还有一步叫 Pre-Compaction Memory Flush，会静默运行一轮提醒 Agent 把重要内容写入记忆文件，防止关键信息随压缩丢失。

压缩后的结构就是一个合并摘要加上最近 2 万 tokens 的原文。

这种方式的特点是每次触发都需要 LLM 调用，成本更高，但实现简单、行为可预测。

版本二：以 Claude Code 为例

Claude Code 设计了 5 层压缩流水线，逐层检查、按需执行，越往后越激进。

第一层叫 Tool Result Budget，每轮都跑。当一轮对话中多个工具的返回结果累计超过 20 万字符时，系统会把最大的结果存到磁盘，原位替换成大约 2KB 的预览加一个文件路径。模型后续需要完整内容时可以自己去读。这是一种渐进式披露的思想，先给模型足够判断的预览，让它自己决定要不要加载全量。

第二层叫 Microcompact，清理旧的工具结果。它会利用 Prompt Cache 的 60 分钟 TTL，如果缓存已过期就直接清内容，没有额外损失。或者通过 cache_edits 指令在服务端缓存副本中删除内容，客户端消息不改，缓存继续有效，省钱。

第三层是核心层 Auto-compact，token 数超过大约 16.7 万时触发。它先尝试轻量方案，用后台已经维护好的 Session Memory 笔记充当摘要，不需要额外的 LLM 调用。如果轻量方案扛不住，再上 Full LLM Compact，把整段对话压缩成不超过 2 万 tokens 的结构化摘要，包含 9 个固定段落比如核心请求、涉及文件、待办事项等，压缩后还会自动恢复最多 5 个关键文件的内容让模型不至于完全失去文件上下文。

第四层是硬拒绝，前面都压不下来就直接报错终止本轮。

第五层是事后兜底，API 返回 413 错误后做一次摘要压缩再重试，每轮最多一次。

两者的核心差异是：OpenClaw 一步到位但精细，分块加合并加预刷记忆。Claude Code 分两步走，先便宜后贵，用梯度设计换成本效率。大部分情况下前两层不需要 LLM 调用就够用了，Full Compact 是中低频事件。

实践上建议主动用 /compact 命令并附带指令，比如"重点保留决策和待办"，比等系统自动压缩效果好，因为你能控制保留什么。

1.4.9 Q: 你看过 Claude Code 源码吗？有什么感受?

看过。最让我印象深刻的是它的记忆模块。

它把记忆拆成了 4 级。指令文件是人写的规则，对话上下文是短期记忆，Session Memory 是自动维护的会话笔记，Memdir 是跨会话的持久记忆。加载时用 Sonnet 小模型做 push 式召回而不是 RAG，写入时有三个后台子代理分别负责提取、更新和整合。整合那个叫 AutoDream，设计灵感来自人类睡眠时整理记忆的过程。压缩时 5 层流水线从温和到激进逐层降级。

有三个设计思想我觉得特别值得学习。

第一个是"会变的实时搜，不变的才持久化"。代码事实故意不存进记忆，因为代码改了但记忆不会自动更新，就会变成误导。只存人的偏好和判断，代码相关的查询靠 agentic search 也就是 glob 加 grep 实时解决。这对我做任何带缓存或索引的系统都有启发：变化频率高的数据不适合做离线索引。

第二个是梯度降级设计。压缩不是"满了就全压"，而是 5 层从最便宜的操作到最贵的操作逐层升级。清理工具碎片不要钱，利用缓存过期不要钱，Session Memory 摘要几乎不要钱，只有真的扛不住了才上 LLM 全量摘要。大部分情况下前两层就够了，大幅降低了成本。这种能省则省、逐级兜底的思路在很多工程场景都适用。

第三个是用小模型卸载大模型的决策负担。记忆召回不是让主模型自己决定要不要搜记忆，那样模型可能不知道自己不知道什么，漏掉重要信息。而是在每轮对话前让 Sonnet 小模型提前筛好塞进上下文，用算力换确定性。这个 push 和 pull 的权衡很值得参考。

1.4.10 Q：Claude Code 是如何通过权限系统保证安全运行的？（或：如何避免 Agent 滥用工具？）

A： 最根本的办法是**不相信 Agent，而是让每个工具调用都经过多层权限约束**。这样即使 Agent 想滥用工具，也很难突破多重防线。

以 Claude Code 为例，它采用**四层权限管道**的设计：

第一层是规则过滤——用户配置 allow/deny 规则，做快速匹配。关键是 deny 规则**不可绕过**，即使在完全信任模式下也生效，确保管理员的硬约束永远有效。

第二层是工具自检——每个工具根据操作内容做细粒度判断。比如 Write 工具会检查是否试图写入 `.git/` 或 `.claude/` 等关键目录（bypass-immune，免疫绕过），这些目录在任何模式下都受保护。

第三层是模式兜底——定义 5 种权限模式（default/acceptEdits/plan/bypass/dontAsk），代表不同风险偏好。模式选择本身就约束了 Agent 的权限上界——再聪明的 Agent 也无法超出。

第四层是 AI 分类器（仅 auto 模式）——独立 AI 根据上下文判断操作安全性。特意**只看工具调用记录，不看模型输出**，防止 prompt injection。两阶段判决：快速粗筛（64 token）+ 深度精筛（4096 token），既保证安全又控制成本。

核心原则：Fail Closed（不确定就拒绝）+ Defense in Depth（纵深防御，不靠单点）。

1.5 Codex VS ClaudeCode

随着近期(2026年6月初）Codex的热度增高，网上很多人说Codex已经超过ClaudeCode。这一说法我不光在自媒体上听到，也听到同事圈有类似的说法。这篇笔记希望详细，**诚实**地对比 ClaudeCode和Codex的**区别，优势，特征，表现**。看完本次你的收获：

1. 如果你在**纠结到底选择哪个工具，我相信看完你会知道如何选择。**
2. 从VibeCoding的能力角度，看完你会掌握更好的VibeCoding使用技巧，根据测评结果：**最推荐的使用方法是：针对于ClaudeCode和Codex的优势，取长补短，相互互补使用。**包括本节提到的一些两个工具的好用的地方，比如loop, ultrareview，都可以用起来。
3. 从面试的角度，**如果面试官考察VibeCoding**，比如问到你使用什么VibeCoding工具，为什么？你知不知道Codex和ClaudeCode的区别？探讨到这些问题，相信你会回答得很好，本节也会总结这些问题的参考答案。

注意：因为这些工具迭代速度很快，模型也在不断迭代，定价也经常变更，本节笔记和时间强相关。如果你阅读本篇笔记时间相隔现在（2026年6月初）有一段时间，这些架构区别可能相对保持稳定，但是具体的特征，状态可能有所差异。

如果你觉得看文字有点枯燥，也可以参考本期讲解视频哦！

1.5.1 核心结论

问题并不在于哪个工具客观上更强。而在于：**面对你当前这个具体场景，哪个工具更适合。**二者呈现的特点不同：

Claude Code

更像一个富有创造力的伙伴，更擅长头脑风暴；

当你走错方向时，它也更愿意指出问题、**提出反对意见。**

Codex

更像一个执行力很强的工程师，

擅长严格遵循指令、精确执行你的要求，以及在**代码或方案审查时发现 Bug 与漏洞。**

1.5.2 两款工具简介

Claude Code（Anthropic）

Anthropic 推出的代码 Agent，能够规划任务、编辑文件、运行命令，并根据你的配置请求权限。

可在**终端**、**VS Code 扩展**、**桌面应用**（Mac 和 Windows）以**及网页版**（研究预览）中使用。

可运行在 **Opus（目前最强模型）**、**Sonnet 或 Haiku 模型**之上。

它更像一个可由你自行塑造的工作流系统，你可以把它**融入自己的工程习惯与自动化流程中。**比如你可以定义工作流，子Agent，Skill，hooks，来完成自己的一套自动化任务。

OpenAI Codex

OpenAI 推出的新一代 Agentic Coding System（并不是 2021 年已经退役的那个 Codex 模型）。

可在终端、桌面应用（Mac 和 Windows）、VS Code 扩展（也支持 Cursor 等 IDE）以及云端版本 chatgpt.com/codex 中使用。

支持运行在 **GPT-5.5**、**GPT-codex**（偏代码专用）以及 **GPT-codex-spark**（更快、更轻量，Pro 用户的研究预览版）之上。

它更像一台"带有明确工程理念的机器"，目标是把**流程从“Agent 完成任务”一路推进到“代码正式上线生产环境”。**（提供了一套成熟的工具，带你完成从代码-评审-部署-生产的一套标准化流程）

已包含在所有 ChatGPT 付费与免费套餐中（Free、Plus、Pro、Business、Enterprise）。（也就是说只要你买了ChatGPT的额度，Codex天然就可以直接用）

1.5.3 它们有哪些重叠能力

两者之间的共同功能，其实比大多数对比文章里提到的还要多。这两个工具都能够：

- 在你的本地机器上编辑代码
- 提供桌面应用和 VS Code 扩展
- 从命令行运行
- 支持 MCP（用于将外部工具接入 AI 的开放协议）
- 提供 CLI（命令行工具）
- 使用相同的 Skills 格式（带有 YAML Front Matter 的 Markdown 文件）
- 提供插件市场（Plug-in Marketplace）
- 支持云端任务委托（把任务丢给 Agent 后直接离开）
- 支持 Hooks 与 Sub-agents（子 Agent）

1.5.4 ClaudeCode胜出的地方

1. 更深度的可定制能力

Claude Code 提供大约 **30 个 Hook 事件**，而 Codex 目前只有大约 **6 个事件。**

Hook 本质上是“自动触发器”——

当某些事件发生时自动执行，例如：

- 提交 Prompt
- 调用工具
- Session 启动
- 创建任务

在这一点上，Claude Code 提供了大约 **5 倍粒度** 的控制能力。

2. 自动委派子 Agent

当任务需要时，Claude Code 会自动生成子 Agent，例如：

- Planner（规划器）
- Explorer（探索器）
- Code Reviewer（代码审查器）

而 Codex 则需要你显式要求后，才会生成子 Agent.

3. 强大的 Slash Commands

> 注意，这里的两个指令其实还是在preview版本，也就是没有正式上线，普通用户现在用不了。

`/ultraplan`

把规划阶段发送到云端 Claude Code Session，

你可以在浏览器中进行 Review 和添加内联评论（inline comments），

之后再把结果回传到本地终端继续执行。

`/ultrareview`

启动多个 Reviewer Agent，

实现更深入的多 Agent Code Review，并复现问题。

- Pro / Max 用户可免费使用 3 次
- 超出后按次计费

4. `/loop`

你可以给 Claude Code 一个“周期性 Prompt”，让它按计划重复执行。

或者不提供 Prompt，直接让它进入“维护模式”。

它能够自主处理：

- 未完成任务
- PR 评论
- Merge Conflict 修复

等持续性工作。

5. Channels

这是一个 MCP Server，

能够把 Telegram、Discord、iMessage 等外部事件推送进正在运行的 Claude Code Session。

你甚至可以直接用手机“发短信”给你的 Agent。

6. Claude Agent SDK

> Claude把他们的ClaudeCode能力封装成了sdk，在这个SDK基础上可以用更细的粒度，使用CC的思想开发Agent。我们使用ClaudeCode本身，只能用它的预制功能，但是如果使用AgentSDK，可以构建一个和CC类似的，但是更加定制化的产品。
>
> 有一本书叫《ClaudeCode Harness之道》（作者：黄佳），AgentSDK是作为ClaudeCode的专门一个小节。不过对于普通人甚至是企业来说，这种用法还是太高级了，我在面试的过程中目前没有被问到过这个问题。

Claude Code 背后的同一套 Agent 引擎，

已经开放为：Python SDK和TypeScript SDK两个版本。

你可以基于它构建属于自己的 Agent 系统。 想更多了解Claude Agent SDK？参考这里

7. 企业级认证

> 这个主要是ToB的能力，就是提供了如何让企业安全托管和调用大模型的能力。Bedrock、Vertex、Foundry是企业在自己云里安全合规地“托管和调用大模型”的平台。Claude Code 支持这三家，意味着它能进入大公司的合规环境工作

Claude Code 支持：

- Bedrock
- Vertex AI
- Microsoft Foundry

这些大型企业常用的 AI 托管平台。

目前 Codex 在 OAuth 灵活性方面，还达不到这个级别。

1.5.5 Codex胜出的地方

1. 更统一的工作流设计

Codex 从底层就是围绕 **Git Worktree** 构建的。

每一个线程（thread）都会运行在自己的 worktree 中，

不会互相污染主项目。

这意味着：

- Agent 并行工作更安全
- 不容易覆盖彼此改动
- 更适合多任务开发

更重要的是：

2.内置浏览器

Codex 桌面应用内直接集成了浏览器。

你可以：

- 查看 Agent 刚刚生成的页面
- 直接在页面上留下视觉评论（visual comments）

在修改AI改完的代码并且review给出意见的问题上，Codex的体验感更好。（其实Claudecode也有这个在browser上检查修改过的代码的功能，只不过它是集成在浏览器上的，需要 Agent 写完 → 切到 Chrome → 手动检查， 而Codex直接集成在桌面，更流畅）

3. 更强的电脑操作能力

> 这一点还是很好用的，我们构建一款应用，做自动化测试，使用Codex的内置方法实现点击测试，还是非常实用的。

Codex 的产品 QA（质量测试）流程非常成熟。

比如你让 Codex 测试刚刚构建的 App：

它会：

- 自动打开应用
- 点击页面
- 发现 Bug
- 记录问题

甚至还能自动生成：

- 严重等级（Severity）
- 预期行为 vs 实际行为
- 复现步骤（Steps to Reproduce）
- 问题分诊总结（Triage Summary）

这已经非常接近真实 QA 工程师的流程。

4.GitHub 集成

`@codex` 的体验非常顺滑。你只需要在：

- PR 评论
- Issue

里 @codex，

它就会直接启动一个云端 Sandbox 去处理任务。几乎是：零配置。这点对团队协作尤其舒服。

5. GPT Image 2 图像生成能力

Codex 内置了 GPT Image 2（目前的图像生成最强模型）这意味着它可以直接生成游戏素材，产品icon, UI插画。

6. Codex的review功能

Codex的/review相比于ClaudeCode的/review功能。Codex更会从挑刺的角度分析，更能够发现问题。而ClaudeCode的review相对来说更加谄媚，不太能分析出来潜在的bug（这里说是普通/review功能，不是CC的/ultrareview），因此在这一问题上，我觉得Codex的review功能更好。

1.5.6 第三方工具集成问题

像 **OpenClaw**、**Hermes Agent** 这样的第三方开源工具，最近开始越来越受欢迎。原因在于：

它们在 coding agent 的基础上，额外加入了很多“长期自治能力”，例如：

- 原生 Cron 定时任务
- Heartbeat（心跳机制）
- 主动技能调用（Proactive Skill Use）

Agent 不再只是“你问一句，它做一步”，而是开始变成一种能持续运行的半自治系统。

OpenAI和Anthropic公司对于第三方集成问题的态度，则显得截然不同。

OpenAI对于第三方 Agent 工具，OpenAI 的态度相对开放。你可以用自己的 ChatGPT 订阅登录 OpenClaw / Hermes，然后把 Codex 能力路由进去使用。甚至Sam Altman 在 5 月 2 日还公开认可了这种做法。

Anthropic 则明显更保守。Claude Agent SDK 文档里明确写着：除非事先获得 Anthropic 批准，第三方开发者不能向用户提供 Claude.ai 登录能力，也不能共享 Claude 的订阅额度。换句话说：你不能合法地：拿自己的 Claude Pro 账号，接到第三方 Agent 平台里无限跑。

1.5.7 定价

两边其实都不需要单独买 API Key。因为：Claude Code 已经包含在 Claude 订阅里，Codex 已经包含在 ChatGPT 订阅里。所以你开通对应会员后，基本就能直接开始用。

Claude 这边

Claude Pro 是每月 20 美元，包含 Claude Code 的基础使用权限。

如果你是重度用户，可以升级：

- Claude Max 5x：每月 100 美元，大约是 Pro 五倍额度
- Claude Max 20x：每月 200 美元，大约是 Pro 二十倍额度

不过最近不少用户反馈：Claude Code 的 session limit 和 weekly limit 比以前更容易撞到。

特别是长任务、多 Agent、超长上下文工作流时会比较明显。

Codex 这边

Codex 已经直接包含在 ChatGPT 套餐内：

- 免费版可体验基础能力
- Plus（20 美元/月）即可使用 Codex
- Pro（200 美元/月）基本接近“无限使用”

1.5.8 同一任务测试

最后你可能想问，对于同一个任务，哪个AI生成的效果更好，代码质量更好？这个问题其实就更难评测了，因为对于任务效果的评估是主观的，另外对任务性质的不同可能导致结论的不同。

不过 Youtube博主Nate Herk也使用了同样的提示词，**两家最强模型（Opus 4.7和GPT 5.5）**对ClaudeCode和Codex在生成**品牌化PDF, 生成官网Page**，**生成市场数据分析图表三个任务上测试。**大致结论如下：

ClaudeCode在UI质量，视觉效果，产品感，**动效方面，表现得更好。Claude Code 更像“有审美的高级工程师”**

而Codex在结构化输出，工程流畅，文档规范性，**稳定性方面执行得更好。**Codex 更像“**高效的生产工程师**”。

从Token使用量上，Claudecode倾向于输出更多的token，更爱解释更改展开，因此用户更容易撞 Claude 限额。

1.5.9 结论：如何选择?

你应该优先选择 Claude Code，如果：

- 你在做复杂前端（complex frontend）
- 视觉设计质量很重要
- 任务需要深度规划（deep planning）
- 你希望 Agent 自动委派 sub-agents
- 你正在构建带 Hooks、Skills、Channels 的自定义工作流
- 你需要 Claude Agent SDK，将 Agent 能力嵌入自己的产品

- 你处于需要 Bedrock / Vertex 等企业认证的环境
- 你在做 brainstorming，或者正在解决复杂问题

因为CC尤其擅长：UI, Dashboard, 前端，架构，workflow orchestration, 上下文推理。

更适合使用 Codex 的场景

你应该优先选择 Codex，如果：

- 任务偏 research-heavy，需要大量联网研究
- 你在生成结构化文档，例如 PDF 或报告
- 你希望一个 App 里直接完成：
- worktree
- review
- shipping
- 你想在 GitHub PR 里直接 `@codex`
- 项目需要内置图片生成能力
- 你需要一个长期稳定执行的大型工程工作流

因为**Codex尤其擅长**:工程执行 + QA + 长任务 + 生产交付

1.5.10 迁移小技巧

其实我自己也写完这个小节的笔记，也做了一些尝试。因为我之前都是用的ClaudeCode，但是借助这个机会，我使用了Codex。其实迁移方法挺简单的，分享一下我的技巧。

首先我个人使用codex，我走的模型的中转，不走Openai官方账号，走后台的服务器，中转API请求，比常规流程更复杂一些，但是，我的亲身经历是：

1. **Codex的配置。**你使用ClaudeCode，告诉它你的基本需求：需要下载codex，使用代理来完成模型调用不走官方账号。 我先让它规划方案，最后它框框执行。经过几次测试（比如模型调用失败，我把错误粘贴给CC),让它修改，很快它就帮我完成了这个功能。 亲测使用CC下载，安装，配置Codex, 全程不到2小时，很容易。
2. **项目的迁移。**我将我的ClaudeCode的项目配置成Codex的项目，也很容易。因为我的CC项目里面有很多skills, workflows, claude.md等，我只用告诉它：**这是CC的项目，现在我需要使用Codex,请帮我完成相应适配。**它会自动完成包括SKILL的转换，Claude.md->Agent.md，所以从项目工程迁移上，成本也很低。

1.5.11 面试问题

1. 你平时会用什么VibeCoding的工具呢

> 如果我学习了我们今天的内容，你回答的时候，从这二者的优缺点的角度，取长补短地使用，肯定能体现你真的是在高效地使用AI，聪明地使用AI。这个问题其实面试官就是想问你对Vibecoding工具的了解，有没有聪明使用。当然我也希望大家能切实理解今天的内容，使用上了codex和Claudecode(没用过，也起码把最少相关内容理解了），因为这个答案其实挺高级的，你要是一点不懂这俩工具，面试官继续和你讨论你连他们基本特性都不懂，这个问题回答得又是这么聪明地在使用AI，那么他会觉得你是背的。
>
> 另外我觉得这个回答还是不够具体，有点浮在理论。更好的答案还是真的结合你的使用情况来说，这个就要求大家真的看懂了本期内容，真正去实践，再回答。

我会同时使用ClaudeCode和Codex，因为他们的特点和优缺点不同，我会取长补短地使用它们。

ClaudeCode更具有创意性，当我需要进行复杂方案设计，头脑风暴时，我会借助Claudecode的能力，同时他对于前端的设计能力，审美更好，包括定制化更强比如提供了30+个hook的能力，当我需要进行那些特别定制化的任务，创造自己的workflow的时候，我会使用ClaudeCode。

Codex则更严谨，指令遵循能力更好。当我需要使用一些长时间，稳定的任务，我会选择Codex，他更加听从指挥，执行严谨。同时他在git生态支持上也很好用，从coding - review - deploy -production整个链路提供了非常好的支持，比如具备了自动操作电脑来执行review，测试前端代码输出报告，有这些需求时我会使用Codex。

2.你熟悉什么VibeCoding工具，能简单讲讲吗?

> 同时给面试官讲两个当前最强的coding工具，分析他们优缺点，面试官肯定会对你竖起大拇指，觉得你愿意研究，关注最新技术。

我熟悉 Claude Code 和 Codex，这两个是目前比较主流的 Vibe Coding / Agentic Coding 工具。二者定位不太一样，我一般会结合使用：Claude Code 负责规划、设计和复杂思考，Codex 负责工程执行、Review 和长期任务推进。

Claude Code 更偏“创造力”和“产品感”。它比较适合复杂前端、Dashboard、UI 设计、架构规划这类任务，生成的页面和交互通常更精致，Agent 工作流能力也更强，比如自动 sub-agent、hooks、长上下文推理等，比较适合做 brainstorming 和复杂工程设计。

Codex 更偏“工程化”和“执行力”。它的 GitHub 工作流、QA、代码 Review、长时间任务执行会更稳定，token 效率和成本控制也更好，适合做 PR 修复、自动化开发、结构化文档生成以及持续交付类任务。

以上两个问题是面试原题：参考202605 安克暑期实习一面

1.5.12 讲解视频

https://www.bilibili.com/video/BV1cqE86NE8c/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

1.5.13 参考资料

Nate Herk博主做的CC vs Codex测评的视频，也是本期视频主要参考资料，理论上如果本小节你都看明白了，不需要再去观看此视频了。

Codex安装使用教程。其实我自己在迁移小技巧 章节教了大家如何配置Codex，结合CC的能力。我自己很快配置好了。网上也有0基础保姆级教学，我找了一个资料，供大家参考。

1.6 Codex ClaudeCode安装小技巧

我自己用Github中转，不用订阅CC和Openai，也能用上Codex和Claudecode，也可以使用GPT5.5 和Opus4.8的模型，我把它写成了skill，有兴趣可以参考。

我自己安装很顺利啊，但是大家也许会遇到一些问题，我还是希望大家能够看一下里面的原理到底是什么，遇到问题去修改。这套方法是完全可行的。如果我把所有封装的特别好，其实也可以做到，但是当你遇到问题，比如你想改模型，你都不知道怎么做，你会一脸蒙。因为这里面的原理不是使用官方模型，而是走了中转，改模型要改配置。

我举这个例子是想说，希望你能稍微理解里面的原理再去使用，这样遇到问题你知道怎么改。

https://github.com/jerry-ai-dev/claude-codex-copilot-bridge/blob/main/SKILL.md

---

1.7 如何给ClaudeCode（包括其他coding工具）节省Token?

1.7.1 前言

这篇笔记的目的，是教你一套**节约 AI 工具提示词（Token）的技巧**。学会之后，你在日常使用 AI 编程时，可以省掉大量 Token——无论你是按量付费的 API 用户（直接省钱），还是订阅版用户（额度更经用）。

本文虽然以 **Claude Code** 为例讲解，但其中的**方法和原理是通用的**，同样适用于 Codex、Gemini CLI 等其他 Vibecoding 工具——差别只在具体数字和命令，思路完全可以照搬。

而且，学会本章节不只是为了省钱。**从面试的角度**看，它同样很有价值：当面试官问你有没有使用 Vibecoding 工具的心得，能介绍一下吗？如何编写提示词的最佳策略？这类问题时——包括理解 Claude Code 的压缩与缓存策略这一块——本章的内容都能让你答得有条理、有深度。文末我也会照例，把对应的面试问题单独整理出来。（我印象很深的是夸克千问的Agent开发二面，面试官就是和我讨论怎么写提示词，他会一直问我还有什么策略吗？还有什么吗？如果当时我看了今天的素材，我的回答就会多一个角度。）

本篇的组织结构是：

1. **先理解大模型的计费标准和策略**——只有知道它怎么收费，才能知道该怎么省钱。
2. **再讲解 Claude Code 的提示词缓存策略**——这一块比较复杂，我会给出参考文档，感兴趣的可以深入学习；我们先大致理解它的原理，才能明白后面最佳实践为什么要那样做。但这里也几乎是全文最难的地

方，理解了这里，其他地方就很容易了。

3. **然后是最佳实践**——告诉你具体应该怎么做，并给出需要用到的工具链接。
4. **最后是面试问题总结**——一如既往，把学完本节后能应对的面试问题整理出来。

1.7.2 大模型的计费标准

1.7.2.1 先看懂：你跟 AI 说一句话，背后发生了什么

在谈钱之前，先弄清一次对话的完整过程——不然后面那些输入、缓存、输出你会对不上号。

你每发一条消息，模型并不是只读你那句话。它每一轮都要**把整个上下文从头到尾读一遍**，然后才生成回复。这个上下文包括（这里是粗略地讲，其实Agent的上下文内容，我们每个Agent架构都会专门讲解，但本节主要是聚焦于计费，不深究上下文细节，以下是最典型的模型上下文组成）：

1. **系统提示 + 工具定义**（Claude Code 内置的一大段说明，最靠前）
2. **项目信息**（你的 CLAUDE.md、memory 等）
3. **之前的完整对话历史**（前面每一轮的问和答）
4. **你这一轮新说的话**

把这 4 块加起来，模型读进去的全部，叫 **输入（input）**；模型写出来的回复，叫 **输出（output）**。

关键问题来了：第 ①②③ 块，其实**每一轮都长得一模一样**（系统提示不变、CLAUDE.md 不变、已发生的历史也不会变），只有第 ④ 块每轮才新增。如果每轮都把 ①②③ 当成全新内容重新读一遍并收全价，那太浪费了。于是就有了**缓存**：把这些重复不变的部分**存起来**，下一轮直接复用。

这就引出了收费的逻辑。下面这张图把一次请求怎么被拆分收费画清楚。核心逻辑是：

> **一次请求 = 输入 + 输出两部分收费。输出只有一种（最贵）。而输入会按有没有命中缓存分成三条路，每条价格不同。**
>
> ❓ **你可能会先问：为什么一次大模型请求的费用要分成输入和输出两部分？**
>
> 因为大模型推理本身就分成两个截然不同的阶段，成本差异很大。做过模型推理的同学应该清楚，这两个阶段叫 **prefill（预填充）** 和 **decode（解码）**——**输入正对应 prefill，输出正对应 decode。**
>
> - **prefill（对应输入）**：把你的整段 prompt 一次性读进去理解。这些 token 可以**并行处理**，GPU 一口气就能吞一大批，算得快、摊到每个 token 的成本低 → 所以**输入便宜。**
> - **decode（对应输出）**：生成回复时只能**一个字一个字往外蹦**（每个新词都依赖前一个，没法并行），每生成一个 token 都要把整个模型重跑一遍，慢且吃算力 → 所以**输出最贵。**
>
> 一句话：**因为读 prompt 和写回复是两种成本完全不同的计算，所以要分开计费**——这也是为什么后面输出的单价会远高于输入。

这一次请求会被拆成四种收费名目：

- **① 缓存写入 cache write（5m / 1h 两种时长）**：发生在某段内容**首次出现的那一轮**——内容第一次被读，并存进缓存。
- **② 缓存读取 cache read（便宜 10 倍）**：发生在**后续每一轮复用它时**——内容没变、还没过期，直接复用。
- **③ 基础输入 input（全价）**：发生在**缓存过期/失效后重读，或某段内容压根没被缓存**时——当成全新内容来读。
- **④ 输出 output（最贵）**：发生在**每一轮的回答**——模型生成回复。

一句话串起来：**同一段内容第一次见 → 走写入存起来；以后复用 → 走便宜的读取；如果缓存没了、当新的重读 → 走全价输入；模型的回答 → 走最贵的输出。** 省 token 的所有招数，本质都是在操控这几个阶段的走向（让更多内容走②便宜读取、少走③④）。

理解了这个过程，再看下面的价目就顺了。

1.7.2.2 五类 Token 的单价

Claude API 把 token 分成 **5 种，单价各不相同**（以下为官方定价，单位：美元/百万 token）：

以基础输入的单价为基准（记作 1×），5 种 token 的相对倍率如下：

- **基础输入 input**：全新的、没缓存过的输入，**1×（基准）**。
- **缓存写入 cache write（5 分钟）**：内容第一次写进缓存，**1.25×**（比输入还贵一点）。
- **缓存写入 cache write（1 小时）**：写进缓存并保留 1 小时，**2×**。
- **缓存读取 cache read**：复用已缓存的内容，**0.1×**（便宜 10 倍！）。
- **输出 output**：模型生成的回复，通常是输入的 **5 倍**（最贵）。

把这套倍率画成图，一眼就能看出差距（数值 = 相对基础输入的倍率）：

> 看这张图就懂了：**最左边的缓存读取几乎贴地（0.1×），最右边的输出高达 5×。** 省 token 的全部套路，本质就是——把尽量多的量从右边（贵）挪到左边（便宜）。

以几个主流模型为例（美元/百万 token，顺序为 输入 / 缓存写入5m / 缓存读取 / 输出）：

- **Opus 4.5**：\$5 / \$6.25 / **\$0.50** / \$25
- **Sonnet 4.5**：\$3 / \$3.75 / **\$0.30** / \$15
- **Haiku 4.5**：\$1 / \$1.25 / **\$0.10** / \$5

同一模型内，各类 token 的实际单价差距也很直观（以 Opus 4.5 为例，美元/百万 token，列与上一张图一一对应）：

> 中间三档（输入 \$5、写入5m \$6.25、写入1h \$10）挤在一起，两头拉得最开：**最便宜的缓存读取**

> **\$0.50，最贵的输出 \$25，差 50 倍。**

从这张表能读出三个关键事实，它们正是后面所有省 token 手段的依据：

1. **缓存读取便宜 10 倍**（0.1×）。所以让内容尽量走 cache read，是最大的一块节省——这就是原理层顺应缓存的意义。
2. **输出最贵**（约输入的 5 倍）。所以让模型少说废话省下的是最贵的那种 token——这就是让 Claude 简洁回答为什么划算。
3. **写入缓存略贵一点**（1.25×），是一次性成本，下一轮就靠便宜的读取回本。所以**频繁打破缓存 = 反复付这笔写入费**，特别亏。

1.7.3 Claude Code 的提示词缓存策略

上一章讲清了怎么收费，这一章讲清缓存怎么运作——它是 Claude Code 的**内置行为**，不用装任何东西，但理解它是后面所有最佳实践的根基。

缓存是什么：复印 + 存档

把缓存想象成**复印存档**：

- **cache create（缓存创建）** = 第一次把内容复印、归档。有成本，但**只花一次。**
- **cache read（缓存读取）** = 之后直接从档案柜拿复印件，不用重新处理，**便宜约 10 倍。**

同一段内容，第一次见按 create 收费，以后每次复用按 read 收费。**省钱的关键就一句话：让尽量多的内容走便宜的 read。** 而且这一切**自动发生**，你不用配置。

缓存分三层：范围越大越省

Claude 的缓存按能被多大范围复用分三层，越靠前越稳定、越多会话共享：

- **系统层**：内容是基础指令、工具定义，复用范围是**全局（所有会话共享）。**
- **项目层**：内容是 CLAUDE.md、memory，复用范围是**按项目。**
- **对话层**：内容是你的消息、回复，复用范围是**仅当前会话。**

TTL：缓存能活多久

TTL 是缓存闲置多久后过期。**每次活跃使用都会重置计时**，只有长时间不碰才作废。

- 订阅版 / Claude Code：**1 小时**
- API 付费 / 子代理：**5 分钟**

缓存是怎么随对话一轮轮增长的

知道了缓存的分类，还得看清缓存在多轮对话里**怎么一步步长大**——这决定了你什么时候便宜、什么时候突然变贵。

核心机制叫 **前缀匹配（prefix matching）**：缓存是从**最开头**起、一段段往后叠的。每一轮都会把**完整上下文**重新发一遍，其中**没变的前缀部分直接读缓存（便宜），只有新增的部分才需要写入缓存。**

用图里的四轮来看：

- **第 1 轮**：全新开始，还没有任何缓存。系统提示 + 项目上下文 + 消息1 **全部要处理并写入缓存**（这一轮最贵）。
- **第 2 轮**：前面那一整块（系统提示、项目上下文、消息1）**没变** → **直接读缓存**；只有新增的回复1 + 消息2需要写入缓存。
- **第 3 轮**：同理，前面全部读缓存，只有新增的回复2 + 消息3写入。**越往后，便宜的读取占比越高——这就是为什么长会话反而越用越省。**
- **第 4 轮（系统提示被改了）**：因为最靠前的系统提示变了，**整个前缀的指纹对不上了**，从它往后的所有内容——哪怕一个字没改——都得**重新处理、重新写入**。一夜回到第 1 轮，一次昂贵的重算。

什么会打破缓存

缓存是**前缀匹配**——从头往后一段段叠。**只要动了靠前的内容，从那点往后全部作废、要重新 create。** 触发情形：

- 闲置超过 TTL（缓存过期）
- **切换模型**
- 改系统提示
- 执行 `/compact`

（注意：改 CLAUDE.md 反而不会立刻失效，重启会话才生效。）

这就解释了核心结论：**只要动了靠前的东西（改系统提示、切模型、或让缓存过期），前缀就断了，后面全部作废重来。** 往后追加新对话是安全的，动前面是致命的。

> **提示词缓存的完整机制其实相当复杂**——它还涉及缓存断点（breakpoint）、20 个块的回看窗口、最小可缓存 token 数（多数模型 1024 起）、不同改动对 tools/system/messages 各层的影响等等。
>
> 日常使用**记住别动前缀这一条就够了**；想深入了解的，可以看 Anthropic 官方文档：https://platform.claude.com/docs/en/docs/build-with-claude/prompt-caching

1.7.4 最佳实践

理解了计费和缓存，就能推出该怎么做了。分两类：**免费无副作用的习惯**（顺应缓存）和**主动削减投喂的工具**（各有取舍）。下面每一类都给出**市面主流、开源免费**的成熟替代品（均为公开 GitHub 项目，不依赖任何付费社区）。

1.7.4.1 先养成三个免费习惯（顺应缓存）

1. **别停太久**——超 1 小时缓存就凉，别把庞大旧历史拖回来重造，因为反正你过去的缓存都已经无效了，干脆我彻底整理一波，让上下文处于最有效的状态。这里有两种做法： 一种是可以使用compact压缩历史，带着更简单的上下文继续。 一种是使用一些skill，比如像Nate（我经常跟着学习的专门做

ClaudeCode自动化的博主）提供的**handoff skill**，使用这个skill重新开一个新会话，并把前一段会话上下文整理打包过来，本质是使用Harness技巧总结好一次会话的精髓，使用新的上下文来延续上一次会话。

2. **切任务就 `/clear`**——旧任务的长历史对新任务无用，却每轮重复计费。
3. **切模型别中途切**——切模型这件事会破坏缓存，要用的模型就一开始定好，避免打破缓存。

> 这套原理对省 token 的意义：你花的每一分钱，要么是复用已缓存的（便宜 read），要么是被迫重造（昂贵 create）。上面三个习惯，本质都是**别让缓存无谓地失效**。

1.7.4.2 先诊断：不知道浪费在哪就没法省

原理：每次开会话，CLAUDE.md、skills、MCP、slash 命令、hooks 都会被加载进上下文，凭空吃掉 token（大头常是 skills 和 MCP）。得先量出来花在哪，才能对症下药。

- **ccusage** 读本地会话日志，生成每日/每周/每会话的 token 与花费报告。`npx ccusage` 即用，**还同时支持 Codex、Gemini CLI**。

- **Claude-Code-Usage-Monitor** 实时终端面板，带用量预测和告警，适合盯着别超额度。
- **`/context`（内置免费）**：不装任何东西，直接看当前上下文构成，最轻量的自查方式，因为通常工具自带的查花费的手段，不是那么清晰，所以才会有专门的这些工具更细致地统计。

1.7.4.3 削减输入 token

原理：默认情况下 Claude 靠 grep + 逐块读文件来定位相关代码，这个摸索过程很费 token。给它更好的检索方式或现成的项目地图，就能少读很多无关文件。这个你看了我们笔记中Harness讲解检索的章节就很容易理解，本质上就是修改工具的检索方法。不同的检索方法特点不同，CC使用的是更通用的方法，如果想要更省钱，可以建立索引。

- **Serena** 开源 MCP 工具包，提供符号级的**语义代码检索/编辑/重构**能力，支持 40+ 语言。相当于代码索引图谱的主流替代：让 Claude 按符号精准定位，而不是大段读文件。
- **mcp-language-server** 把语言服务器（LSP）能力接给 AI，支持查定义、找引用、看诊断。更轻量的语义导航方案。
- **写好项目文档（免费、最易做）**：在 CLAUDE.md 或分目录的说明文件里，把每块代码干嘛、有哪些约定和坑写清楚。**为什么省**：Claude 不用每次重新摸索项目结构——这就是 Intent Layer 的核心思想，其实手写一份就能实现，不必装工具。

1.7.4.4 削减输出 token

原理：模型爱写铺垫、叙述、总结，这些最费输出 token，而多轮对话里会不断累积。让它说话简洁即可。

- **不必装工具**：网上有个流行的 **Caveman skill**（「原始人 skill」，slogan 是「why use many token when few token do trick」），思路就是逼 Claude 像原始人一样用最少的字说话、砍掉一切废话，官

方称能省下约 65% 的输出 token。但它本质上不神秘——**你自己在 CLAUDE.md 里写一句约束就能实现同样效果**，例如「回答简洁直接，省略铺垫和客套，保留必要的技术细节」。代价也一样：说太少时复杂任务可能降质，需要详解时临时放开即可。

1.7.4.5 会话与文档管理

- **交接会话**：可以使用 Handoff Skill（夸克网盘），让 Claude 总结当前会话中做了什么、下一步做什么，再开启新会话继续。也可以让 Claude 手动总结后执行 `/clear`，或者使用内置的 `/compact`。**为什么省**：不必拖着长上下文每轮重复计费。
- **写好 CLAUDE.md（免费）**：保持 <300 行，**只写模型猜不到的**（项目意图、非显而易见的版本/工具约束、偏离常规的架构、踩过的坑）。**为什么省**：短而精的文档既减少每次加载的 token，又避免模型自作主张走弯路。旧式命名规范那类是浪费。
- **切模型**：简单/重复任务用更便宜的小模型。**为什么省**：小模型单价低。但记得——要用就一开始选定，别中途切（否则打破缓存）。

> ! 这些方法对其他工具（如 Codex）适用吗？
>
> - **原理层**：思维通用（前缀缓存、动了靠前的就全废、稳定内容放前面），但**数字不通用**——Codex（OpenAI）的 TTL 更短且不可控，缓存折扣更小（约省 50% 而非 90%），命令也全不同。
> - **工具层**：**大多跨工具通用**。语义检索（Serena 等）、用量统计（ccusage 直接支持 Codex）、写好项目文档，本质是处理文件/上下文/用量，和用哪个模型无关。
>
> **一句话**：原理层照搬思维、本地化数字；工具层大多能直接迁移。

1.7.4.6 工具速查

- **查 token 用量/花费（支持 Codex、Gemini）** → ccusage
- **实时用量监控 + 告警** → Claude-Code-Usage-Monitor
- **语义检索代码（代替代码图谱）** → Serena
- **轻量语义导航（查定义/引用）** → mcp-language-server
- **整理上下文并交接新会话** → Handoff Skill（夸克网盘）（这个SKILL是Youtube博主Nate提供的免费SKILL,我怕大家翻墙不方便，所以变成了网盘形式分享给大家）
- **看当前上下文构成** → `/context`（内置，无需安装）

> 说明：Caveman（少说话）、Handoff（交接会话）、Intent Layer（补文档）等技巧，其实**用一句 CLAUDE.md 提示词或内置命令就能实现同样效果**，不必额外装工具。压缩终端输出（RTK）较小众，可按需自行搜索，或用让 Claude 别读大段日志的提示约束替代。

最佳实践核心两句话：① 别为用不上的旧上下文反复付费（顺应缓存）；② 别喂给模型它不需要的东西（工具削减）。

1.7.5 相关面试问题

1. **你有没有使用 Vibecoding 工具（如 Claude Code）的心得，能介绍一下吗？（通用问题，面试官喜欢问AI使用技巧）？答题思路：** 这个问题可以从很多角度回答。这里主要从节省 Token 的角度，按照计费模型、缓存原理、最佳实践三个层次展开，体现自己不只是会使用工具，也理解它背后的成本结构。**参考答案：**我在使用 Claude Code 这类 Vibecoding 工具时，一个比较重要的心得是，要主动管理上下文，而不是把所有内容都交给 AI 自己处理。因为模型每一轮都需要读取上下文，而且输入、输出、缓存写入和缓存读取的价格并不相同。如果完全不管理上下文，不仅会浪费 Token，也容易让模型被无关信息干扰。具体来说，我会把项目中长期有效、但模型无法自己推断的信息写进 CLAUDE.md，比如项目架构、特殊约定和踩过的坑，同时尽量保持这个文件精简。处理同一个任务时，我会保持模型和前面的固定内容稳定，提高提示词缓存的命中率；如果切换到完全不同的任务，我会使用 `/clear` 清理无关历史，而不是一直带着旧上下文。另外，我会使用语义代码检索工具，减少 AI 读取无关文件的过程，也会明确要求它回答简洁，减少没有价值的铺垫和总结。我认为，使用这类工具的关键不只是提示词写得长，而是给模型准确、必要的上下文，并且持续控制上下文的质量。这样既能节省 Token，也能提高回答的准确性和开发效率。
2. **编写提示词的最佳策略是什么？（源自夸克千问二面）答题思路：** 先说明提示词要包含清晰的目标、必要背景和输出要求，再从缓存角度说明要把稳定内容放在前面，把变化内容放在后面。还可以补充 CLAUDE.md 要保持精简，只写模型无法推断的信息。**参考答案：**我认为编写提示词的第一个原则是明确。一个完整的提示词应该说明任务目标、必要的背景信息、限制条件和期望的输出形式。比如，不只是说帮我修改代码，而是要说明需要解决什么问题、哪些行为不能改变、应该修改哪些范围，以及最后是否需要运行测试。这样可以减少模型反复确认或者理解错误的情况。第二个原则是只提供必要的上下文。上下文不是越多越好，如果把大量无关代码、日志和历史对话都放进去，既会浪费 Token，也可能降低模型判断的准确性。对于项目中长期不变的约定，我会放进 CLAUDE.md，但会尽量控制在 300 行以内，只记录项目意图、特殊架构、工具限制和常见问题，不重复那些模型通过代码就能看出来的内容。第三个原则是考虑提示词缓存。因为缓存采用前缀匹配，所以我会把系统提示、项目规范等稳定内容放在前面，把每一轮都会变化的具体需求放在后面，尽量不修改已经稳定的前缀。另外，我也会明确要求模型回答简洁，保留必要的技术细节，减少客套、重复总结和无意义的输出。
3. **你在使用AI工具的时候会用哪些插件吗？可以介绍下吗？（深信服社招一面）答题思路：** 挑选本节介绍的几个代表性工具，分别从用量统计、代码检索和会话管理三个方面回答。重点不是罗列工具名称，而是说明使用场景、解决的问题和带来的收益。**参考答案：**我会根据不同的使用场景选择工具，而不是安装很多功能重复的插件。第一类是用量统计工具，比如 **ccusage**。它可以读取本地会话日志，按天、按周或者按会话统计 Token 用量和费用。我会用它判断消耗主要来自输入、缓存还是输出，再决定应该优化哪个部分。第二类是代码检索工具，比如 Serena。它可以提供符号级的语义检索能力，让 AI 按照类、方法和引用关系定位代码，而不是通过 grep 不断搜索，再大段读取文件。对于比较大的项目，这种方式可以减少很多无关上下文，也能提高定位问题的效率。如果只需要查定义和引用，也可以使用 mcp-language-server 这类更轻量的工具。第三类是会话管理工具，比如 Handoff Skill。当一个会话已经很长，或者缓存已经失效时，我会让它总结已经完成的工作、关键决策、修改过的文件和下一步计划，再把

这些必要信息交接到新会话。我的原则不是工具越多越好，而是每个工具都应该解决一个明确的问题，减少无效上下文和重复工作。

4. **可以介绍一下ClaudeCode 的提示词缓存的功能吗？答题思路：** 先说明缓存解决的问题，再解释缓存写入、缓存读取和前缀匹配机制，最后介绍常见的缓存失效原因以及实际使用中的注意事项。**参考答案：** **Claude Code 的提示词缓存**，主要用于复用多轮对话中重复出现的上下文。模型每一轮并不是只读取用户新发送的那句话，而是需要读取系统提示、工具定义、项目文档和之前的对话历史。如果这些内容每次都按照新的输入重新处理，成本会比较高，所以 Claude 会把其中稳定的内容缓存起来。一段内容第一次出现时，会进行缓存写入。后续请求中，如果前面的内容没有发生变化，就可以直接进行缓存读取，只处理本轮新增加的消息。缓存读取的单价大约只有基础输入的十分之一，因此在长会话中，只要缓存能够持续命中，就可以明显降低 Token 成本。它的核心机制是前缀匹配，也就是从上下文的最前面开始判断内容是否一致。系统提示和项目上下文通常在前面，历史对话和新消息依次追加在后面。如果修改了靠前的系统提示、中途切换模型、执行了会重写上下文的操作，或者缓存超过有效时间，就可能导致前缀无法匹配，后面的内容也需要重新处理。所以在实际使用中，我会尽量保持系统提示、工具配置和项目说明稳定，把每轮变化的需求追加在后面，也会避免在同一个任务进行到一半时切换模型。这样可以让更多内容命中缓存，减少重复计算和费用。

1.8 参考资源

参考视频

以下四个视频是本文档内容的主要来源，按推荐观看顺序排列：

视频一：从0到1全攻略

标题：Claude Code 从 0 到 1 全攻略 —— MCP / SubAgent / Agent Skill / Hook / 图片 / 上下文处理/ 后台任务 / 权限 链接：https://www.youtube.com/watch?v=AT4b9kLtQCQ

适合：完全零基础，想系统掌握 Claude Code 所有主要功能的人。

涵盖安装、三种模式、Plan Mode、回滚、图片输入、MCP、Context 管理、CLAUDE.md、Hook、Skill、SubAgent、Plugin 全套内容。

视频二：36 分钟掌握 95%

标题：Master 95% of Claude Code in 36 Minutes (as a beginner) 链接：https://www.youtube.com/watch?v=saggDHHnmtQ

适合：想了解如何用 Claude Code 构建完整自动化工作流（从规划到部署）的人。介绍了 WAT 框架（Workflows-Agent-Tools），以及一个完整的 YouTube 数据分析自动化案例，包含 MCP、Skill、安全审查、Modal 云端部署全流程。

视频三：12个必知特性

标题：12 Claude Code Features Every Engineer Should Know: Subagents, CLAUDE.MD, Checkpoints, MCP and more 链接：https://www.youtube.com/watch?v=E4fzxVMOav4

适合：需要快速建立 Claude Code 核心功能框架的人。约 7 分钟，用极简的方式讲清了 CLAUDE.md、权限、Plan Mode、Checkpoints、Skills、Hooks、MCP、Plugins、上下文管理、斜杠命令、SubAgents 共 12 个核心特性。

视频四：insights 功能深度解析

标题：Claude Code Course 15 - /insights report

链接：https://www.youtube.com/watch?v=BhWOpPD43R4

适合：已有一定 Claude Code 使用经验，希望了解如何利用数据改进自己工作流的人。介绍了 `/insights` 命令的使用方式和报告内容，以及如何用 AI 工具进一步提炼行动建议。

视频五：ClaudeCode 最全视频

这个真的是一个非常全面的ClaudeCode讲解，作者是Nate老师，是我觉得非常棒的老师。这个课程有10个小时，适合0 ClaudeCode基础的同学。从0基础 带你讲解ClaudeCode如何使用，通过几个实战例子讲解ClaudeCode如何编排自动化，部署，到最后如何变现。我真的惊叹这个课程竟然不是收费。

我非常相信这个老师对于Claude Code使用的技巧一定是行业领先的，视频里面讲解的内容也是全面。我相信他说的，你消化好这个视频，可以从ClaudeCode 0基础到精通使用ClaudeCode到可以变现。

所以这个视频真的非常适合想要熟练运用，深入掌握ClaudeCode 高阶技巧的同学。

标题：Build & Sell with Claude Code (10+ Hour Course)

链接：https://www.youtube.com/watch?v=mpALXah_PBg

相关项目

everything-claude-code 项目（本文档最佳实践的主要来源）

GitHub：https://github.com/affaan-m/everything-claude-code

包含三份深度指南：

- `the-shortform-guide.md`：基础架构搭建（Skills、Hooks、SubAgents、MCP、Plugins 配置）
- `the-longform-guide.md`：高级技术（Token 优化、记忆持久化、Eval 验证、并行化策略）
- `the-security-guide.md`：Agent 安全（攻击面、沙箱、输入净化、CVE 案例）

同时包含 30+ 个可直接使用的现成配置文件：Agent（代码审查、构建修复、安全审查等）、Commands（TDD、E2E、多后端编排）、Hooks（记忆持久化、格式化）、Skills（各语言最佳实践）。

《解码Agent Harness》 -- ClaudeCode 架构深度解析。这个是专门解析ClaudeCode源代码的，网上也开源。可以在线阅读：

https://lintsinghua.github.io/

2 Vibe Coding

Vibe Coding 已经是现在绕不开的趋势了。虽然本节归在了其它问题，但是我强调的本节其实也非常重要。

面试的角度它很重要：可以看到网龙面试环节问Vibe Coding的经验，我也遇到很多面试官会问你最喜欢使用哪个Vibecoding的 工具等，其实这些都是企业对于你的考察，因为在新的时代的来临，程序员的编程方式一定会发生变化，如何高效vibe code, 聪明使用AI工具一定是企业看中的，因此虽然他不是一个硬核知识考点，但绝对也是面试中热门的话题，这个问题答得好，一定是一个加分点。

个人成长的角度上：它也太重要了。现在谁不vibe coding呢？谁不会聊两句你是如何使用AI工具编程的呢？学会高效使用VibeCoding对个人成长必不可少。

所以本节内容其实很重要，而且这个内容甚至适合于任何人，你甚至可以是产品经理或者没有任何开发背景的人，因为VibeCoding极大降低了编程门槛。

本节会推荐5个Vibe Coding的视频，同时根据视频也整理成了文字精髓放在这个小节。看完本节，你会学习到Vibe Coding的推荐方法，行业发展/Vibe Coding的趋势。 相信本节内容也会给你带来灵感。

2.1 核心技巧

Vibe Coding的有5大核心基础：**思考，架构，节点，调试，上下文。**

2.1.1 思考细致

在使用AI工具前，你需要仔细思考你到底想要什么。如果你自己都没想明白，你怎么能期望AI能够写明白？在我们vibe coding的时候都使用的自然语言，很容易没有想清楚我们到底想要一个什么样的产品，其实这个阶段是我们很好的仔细思考，提升自我的机会。必须仔细想清楚，想实现的目标是什么，它应该怎么实现，需要哪些条件和能力，他要遵循什么样的规则，它的流程是什么。 将问题想清楚，再以自然语言的形式描述给AI。

2.1.2 明确架构

> 我们使用VibeCoding的时候，几乎一定有现成的架构或者类似的任务的。我们需要让AI遵循指定的架构，或者使用指定库，这样效果一定会比让他从头完成更好。我们需要给AI指明一个方向。比如，对于一个浏览器应用，你可以明确它使用React前端，CSS和HTML JavaScript前端。

（如果你不知道采用哪种架构，没关系，可以先让AI给你提供一些可能的方向，然后以学习的心态去学习，确定好对应的架构，可以在vibecoding的时候同时进行学习）

2.1.3 做好版本控制

系统可能崩溃，所以始终做好版本控制和节点管理。

2.1.4 调试

通常在Vibe Coding中，最快修复问题的方式是指出问题，让AI自己修复。但有时可能会遇到一些问题，AI并不能很轻易地解决，这时就需要我们对系统问题有一个更深入的理解，缩小范围，推测可能的原因帮助AI解决问题。

2.1.5 提供上下文

你提供更多的细节，信息，上下文，你的AI的输出就会越好。这些信息包括，具体的产品文档，你的环境，你的倾向，示例，或者是更加精心打磨的上下文，这些都能够帮助AI取得更好的效果。

2.2 建议

1. 总是先完成一个最小可以运行的产品，然后再迭代，增加更多的feature.
2. 在你使用AI coding的时候，一般只有两种模式，你要么是在添加新feature，要么是在调试错误。添加新feature时，记住提供更多的信息，明确架构，给出有明确细节的参考文档，增量修改，做好版本管理和控制。当你在调试的时候，记住弄清楚系统是怎么运行的（如果不清楚，就问AI），给出尽可能多的信息，想想如何能让AI解决这个BUG。
3. 给出全局的约束。这个有点像将要求放在系统提示词中，让AI Coding的时候总是遵循某个规则，比如尽量少的改动代码，以xx的规范来完成。

2.3 不同Vibe Coding工具对比

2.4 参数视频

不好意思，这一小节推荐的视频都是英文的，确实找到的一些很好的教程都是英文的。如果英语差，视频下方可以点CC把字幕打开，另外很多AI工具比如Github copilot中，可以把视频链接丢进去给它总结，另外我自己本节的内容，也是对这些视频的内容做了一个总结。

Tina Huang老师

是一个很知名的老师了，之前也推荐了很多她的视频。她的这个视频总结了vibe coding的5大核心技巧：思考，架构，节点，调试，上下文，并给出了一些建议，视频中展示了她如何vibe coding的一个实战。对应笔记5.1和5.2

https://www.youtube.com/watch?v=iLCDSY2XX7E

对比了2026年不同的vibe coding工具，主要是用的一些国外的工具：Windsurf, Bolt, Cursor,Rork,Claude Code, Cursor 等，了解这个是因为其实我们需要对不同的工具特点有一些了解和对比，思考。这也能反映我们在聪明的使用AI工具。这个问题其实在面试中也涉及过几次了。这个视频像一个茶话会，就是以夯到拉的形

式给AI Coding工具打分，**大致有个印象就行了。**可以**大概看一下**5.3 或者视频，避免面试过程中面试官问你用过哪些Vibe coding但是你啥也没用过，只能说出一个的情况就好了。

https://www.youtube.com/watch?v=ud0bv2J3xWY

Claude Code 的使用：

这个视频是中文的，Claude Code作为最火的AI工具之一，是值得学习和使用的。不过确实CC相比于很多AI工具，是有一定上手门槛的，使用命令操作，并不是很直观。如果想要学习Claude Code, 可以看这个视频，里面讲了如何配置，常见的用法和命令：包括CC使用MCP, Subagent, SKILL, Checkpoint，不同的模式等。

Vibe Coding趋势：

Y Combinator的演讲，建立Vibe Coding 宏观视角，理解时代趋势和核心竞争力。

https://www.youtube.com/watch?v=IACHfKmZMr8

10分钟去学会Vibe Coding：

快速上手实战项目，通过一个10分钟实战项目通过Vibe coding 写出一个例子，并且还讲了如何通过这个变现，适合新手小白。

https://www.youtube.com/watch?v=-LFB8D9WV-g

2.5 SDD开发工作流

2.5.1 概念与价值

开场问题：

> 如果今天换一个 AI 会话，或者换一位开发者，项目还能按照昨天确认的要求继续吗？

这句话其实直击了SDD需要解决的痛点。

SDD，即 **Spec-Driven Development，规范驱动开发。**定义可以概括为：**把意图写成可讨论、可维护的规范，再让规范指导设计、任务、实现和验证。** 前半句:"把意图写成可讨论，可维护的规范"，在SDD里面体现结构清晰，结构分明的技术文档，通常是以md形式组织的。 后半句:"再让规范指导设计，任务，实现和验证"， 当然既可以由人来实现，当然在AI-Vibecoding的趋势下， 通常我们是让AI来根据文档写代码，编译，验证来完成实施的推进的。

1.2 它解决什么问题?

- **减少需求歧义**：写清范围、约束和成功标准，避免一句需求有多种理解，在实现前发现分歧。
- **保留项目约定**：将规范保存为可重新读取的项目文件，而不是只存在聊天中，让新会话或接手的开发者能够延续已确认的要求。
- **让结果可验证**：定义好测试用例，强制约束AI进行测试，保证AI生成的代码效果正确。

- **让代码更好追踪**：VibeCoding的代码可能很多，人没法快速Review，通过Review文档能够快速理解当前代码的实现功能，设计意图。

在之前的古法编程时代，每次开发前定义规范的文档也是非常有必要的。在AI时代，这件事变得更加重要：AI写代码太快了，没有明确的约束和验收标准，代码很容易失控。同时，一旦一次会话结束，如果我们不保留对应的设计，进度，下一次重新开始会话，AI不知道如何继续。从团队交接的角度来说，每份代码如果能对应规范的文档的话，工作做了一半，换了一个人继续，另一个人的AI也能够快速读懂上下文，按照既定路线和约束继续。

2.5.2 规范结构与模板

对于文档如何组织，并没有唯一固定的目录方案。像一些SDD的框架，OpenSpec,SpecKit,Superpowers他们都有自己组织文档目录的体系。不过核心原则类似，我们的文档应该满足：

1. **按职责组织内容。** 需求说明做什么、不做什么、有哪些约束及怎样算成功；设计说明如何实现；任务说明具体工作、依赖和验收标准。不同文档的职责边界应清楚。不同职责的内容不应该混在同一个文档。
2. **围绕任务提供必要上下文。** AI执行任务时应该确保 AI能获得全局上下文（目标、架构与核心约束）、任务上下文（当前任务的描述、输入输出与验收标准），以及前置上下文（当前任务依赖的已有代码和模块）。
3. **提供清晰的入口和引用。** 工具入口文件提供规范索引与必要摘要，执行任务时引用相关规范章节和已有代码位置，清晰的目录有助于AI能够快速准确检索到必要的内容。
4. **保持单一权威来源与一致性。** 每一个约定应该有明确的源定义，其他文档通过引用或必要摘要使用；避免多处独立维护，导致内容相互矛盾。

根据以上原则，下面给出一种参考目录结构：

```Plain Text my-project/ ├── CLAUDE.md # 规范索引和摘要 ├── specs/ │ ├── proposal.md # 目标、范围、约束、成功标准 │ ├── design.md # 架构、模块职责、方案和非功能设计 │ ├── tasks.md # 工作项、输入输出、依赖、验收标准 │ ├── contracts/ # 按需拆出的接口与数据约定 │ └── decisions/ # 按需拆出的重要架构决策记录 ├── src/ # 实现 └── tests/ # 测试
```

> **先理解每类文档的内容职责，再结合项目理解具体写什么。**
>
> | 材料 | 主要内容 | ForgePilot 补充示例 |
> |-|-|-|
> | `proposal.md` | 项目背景、目标、需求范围与非目标、成功标准、约束条件 | 比如项目目标是持续稳定长程自动修复bug,目标是人工接入率，修复正确率达到xx |
> | `design.md` | 系统架构、模块职责、关键流程、数据模型、非功能设计 | 整体架构图，各模块之间比如Corids,Pi,记忆模块如何交互 |
> | `contracts/` | 模块间接口、输入输出结构、行为约定、错误与状态语义 | 例如给Agent集成的工具接口定义 |
> | `tasks.md` | 实施工作项、输入输出、前置依赖、验收标准、完成状态 | 示例任务：实现会话保存与读取，并通过保存后重新加载的测试 |
> | `decisions/` | 重要架构取舍的背景、备选方案、决策、理由与状态 | 例如记录采用 Pi 还是自建 AgentLoop 的方案

比较与选择理由 |
| `CLAUDE.md` | 规范索引、相关文档引用、关键规则摘要 | 比如设置一个表格，表格记录所有文档的概括和位置，让模型按需获取 |

编写的力度：文档应该写多细？ 文档要写细到什么程度，也是很多朋友经常问我的，这里给到一个原则： **以“恰好够 AI 执行”为准：关键要求不留给 AI 猜测，局部编码细节不必提前规定。** 文档不是越长越好，而是要让执行者明确交付目标、协作边界和验收条件。

判断一项内容是否需要写入，可以看它是否影响对系统的理解或系统的最终行为：

- **需求层面的要求必须明确。** 功能范围、输出结果、性能指标、异常处理边界等，写在 `specs/proposal.md` 中，不能靠 AI 自行补全。
- **架构层面的选择应在设计中明确。** 模块分工、接口、数据模型和重要方案，写在 `specs/design.md` 或相应的 `specs/contracts/`、`specs/decisions/` 文件中。
- **只影响代码写法、不改变最终行为的细节，可以交给 AI。** 不必把每个局部变量、循环写法和辅助函数的实现步骤都预先规定，但实现仍须遵守项目已有的编码规范。

写得不够的信号： AI 经常猜错关键行为，需要在对话里反复纠正；这些纠正涉及的要求或决策，应补回对应规范。

写得过细的信号： 连不影响最终行为的局部写法也逐项规定，文档变成了代码的逐行说明，维护这些细节的成本超过了它们带来的帮助。

文档模板

以下模板可按项目需要调整，使用时替换占位符并删去编写提示，不必每次都新建全部文件。已填好的 ForgePilot 变更示例见附录 A。

1. \`proposal.md\`：从目标、范围到验收

需求文档说明项目目标、本次交付范围和验收标准，让目标、功能与验收条件相互对应。

```Objective-C
# [项目名称]-需求规范

## 1. 背景与目标
[现有问题、项目目标、目标用户]

## 2. 功能范围
### 做什么
[本期功能]

### 不做什么
[明确排除项]

### 技术约束
[项目必须遵守的既定技术栈、部署环境等限制]

> 编写提示：如果技术栈、部署环境是项目必须遵守的既定限制，
> 就写在当前 `specs/proposal.md` 的本小节中，不另建约束目录。
> 为满足需求而选择的架构和实现方案，写在 `specs/design.md`
> 的相关章节中，并说明选择理由。

## 3. 验收标准
```

```
[功能结果、性能指标、边界场景的预期行为]

> 编写提示：条件要能明确检查，例如规定场景下的执行时间阈值，
> 不只写“效果好”或“运行正常”。满足该指标的方案写在
> `specs/design.md`，不要与这里的验收要求混在一起。

## 4. 术语定义
[需要统一理解的词及其含义]
```

2. `design.md`：把需求落实为结构和方案

设计文档说明系统如何组织、模块如何协作，以及采用哪些方案来满足需求。

```
# [项目名称]-架构设计

## 1. 系统架构
[架构形态、模块关系图、数据流向]

> 编写提示：方案必须符合 `specs/proposal.md` 的范围，
> 不擅自增加其中已排除的功能。

## 2. 模块职责
[各模块负责什么、不负责什么]

## 3. 数据模型
[核心模型、字段与含义；已拆入 `specs/contracts/` 时引用对应文件]

## 4. 接口契约
[模块暴露的接口、参数与返回结果；已拆入 `specs/contracts/` 时引用对应文件]

## 5. 技术选型（ADR）
[重要选择及理由；可在本节记录，或引用 `specs/decisions/` 中的独立记录]

> 编写提示：影响系统行为或结构的选择应明确；只影响局部代码写法的
> 细节不必提前规定。

## 6. 非功能性约束
[落实需求的错误处理、安全、性能与可运维性方案]
```

3. `contracts/`：明确协作双方共同遵守的约定

契约明确模块之间交换的数据、调用方式和结果含义，既适用于对外 HTTP API，也适用于项目内部的函数接口。

```
# [模块或接口名称]-契约

## 数据模型
[字段名称、类型、含义及必要的对应关系]

## 接口定义
[函数或接口名称、输入参数、返回结果]
```

```
## 行为与错误约定
[成功、无数据、失败等结果的含义；相关错误处理规则]

> 编写提示：区分“确实没有数据”和“调用失败”，不能用同一个
> 返回值含混表达。
```

4. `tasks.md`：把方案拆成可交付、可验收的工作

任务文档将设计拆成可执行的工作项，明确每项的输入、输出、依赖和验收条件，并据此安排执行顺序。

```
## Task N：[以动词开头的任务标题]

描述：用 1 到 2 句话说明这个任务做什么
输入：需要哪些前置任务的产出作为输入
输出：这个任务完成后产出什么文件 / 模块
依赖：Task X、Task Y（必须先完成的任务）

> 编写提示：一个任务应交付可独立运行或测试的成果，
> 不要只按“做后端、做前端”划分模糊的大块工作。

验收标准：
- [ ] 具体的、可执行的、二值化的验收条件 1
- [ ] 具体的、可执行的、二值化的验收条件 2

> 编写提示：验收条件应可直接转成测试用例，
> 不以“代码已生成”代替验收完成。
```

具体执行与进度处理见本文第 3 节。

5. `decisions/`：用 ADR 留下重要选择的理由

ADR（Architecture Decision Record，架构决策记录）记录一项重要选择的背景、备选方案和理由。它解释为什么这样设计，不替代 `design.md` 对系统当前结构与职责的说明。

```
# ADR-NNN：[一句话概括决策]

## 状态
[已采纳 / 已废弃 / 已被 ADR-MMM 取代]

> 编写提示：旧决定被推翻时，保留旧记录，在新 ADR 中说明
> 取代关系，不改写旧决策的理由。

## 背景
[面临什么问题？为什么需要做这个决策？]

## 选项
[考虑过哪些方案？各有什么优缺点？]

## 决策
[最终选择什么？]
```

```
## 理由
[为什么选择这个方案？关键取舍是什么？]
```

例如，选择 SQLite 时，可以在 ADR 中比较 SQLite、PostgreSQL 和 JSON 文件，再结合定时运行、数据规模与运维要求说明选择理由。它展示的是**如何论证一次选择**，不是说所有项目都应该使用 SQLite。

保留旧决策的要求，不应推广成所有文档都只能追加。

记录的粒度： 一份 ADR 聚焦一项关键决策，建议不超过一页。如果选错后的修正成本超过一天，就值得记录；这不是硬性门槛。

6. `CLAUDE.md`：写索引与必要摘要，不另立一套规范

`CLAUDE.md` 为 AI 提供规范入口，通过文件索引、必要摘要和当前任务信息，帮助它找到依据并衔接工作。

```
## 规范文件索引
- 需求规范：specs/proposal.md
- 架构设计：specs/design.md
- API契约：specs/contracts/api-spec.yaml
- 数据模型：specs/contracts/data-models.md
- 任务清单：specs/tasks.md

> 编写提示：`CLAUDE.md` 是索引和摘要，不是另一套规范。
> 约束与规则在 `specs/proposal.md`、`specs/design.md` 等对应文件中
> 正式定义，这里只引用或摘要，不另行维护相互矛盾的版本。

## 当前迭代
正在执行 [任务编号与名称]，详见 specs/tasks.md
```

> 经常有朋友问我：“你之前项目的技术文档是怎么写的？怎么知道要写哪些内容？”消化了这一节，再让你写一份技术文档，应该就不至于无从下手了。最简单的方法，就是以这些模板为骨架和脉络，结合自己的项目，让 AI 辅助起草，再和 AI 一起讨论其中的细节，不断修改、补充和完善，就能逐步形成一份还不错的技术文档。重点不是把模板填满，而是把项目真正需要明确的事情想清楚、写清楚。

2.5.3 任务执行与验收

3.1 执行流程

1. 选择当前要执行的任务

先从 `specs/tasks.md` 中选择一个尚未完成、前置依赖已经满足的任务，明确这一次要交付什么。第一次执行时，从没有前置依赖的任务开始；已有部分成果时，则根据当前进度选择可以继续推进的任务。

先确定任务，再围绕这个任务获取上下文，而不是先把所有文档都塞给 AI，再让它随意决定做什么。

2. 根据任务获取三类上下文

确定当前任务后，让 AI 获取与它相关的三类信息：

上下文	包含什么	从哪里获取
全局上下文	项目目标、架构与当前任务涉及的约束	通过 `CLAUDE.md` 的索引，读取 `specs/proposal.md`、`specs/design.md` 及相关契约章节
任务上下文	当前任务的描述、输入输出和验收标准	`specs/tasks.md` 中的当前任务条目
前置上下文	当前任务依赖的已有代码与模块，避免重复实现或接口不一致	前置任务产出的代码、测试及现有共享模块

3. 实现代码与测试

让 AI 依据当前任务及相关规范实现代码，同时编写对应的测试。实现应遵守设计中的模块边界和接口约定，测试则对照任务的验收标准，检查应交付的行为。

这一阶段的产出是代码和测试，还不能据此认定任务已经完成。

4. 运行验证，未通过则修复

实际运行测试，逐项对照 `specs/tasks.md` 中当前任务的验收标准检查结果；涉及需求或接口行为时，同时核对 `specs/proposal.md` 和相应的契约定义。

- **未通过**：根据失败结果修复当前任务，再次运行验证；遇到规范冲突、环境阻塞或达到约定的返修上限时，暂停并报告，而不是无限重试。
- **通过**：进入记录结果、标记完成这一步。

“代码生成完了”不是完成条件，“验收通过了”才是。 未执行的检查必须注明，不能当作已经通过。

5. 记录结果，标记任务完成

验收通过后，记录实际执行的检查及其结果，再更新 `specs/tasks.md` 中当前任务的完成状态。

不要默认 AI 工具会自动写回进度。实际使用时，应明确谁负责更新；如果交给 AI，就在执行指令中明确要求它：**通过验证后才勾选任务，并记录验证结果。**

6. 选择下一任务，继续循环

检查 `specs/tasks.md` 中是否还有未完成的任务：有，则回到第一步，选择前置依赖已满足的下一项，并重新获取与它相关的上下文；没有，则结束本轮执行。如果仍有未完成任务，却没有可执行项，应报告依赖或阻塞问题，不继续空转。

3.2 如何让 AI 推进

这个小节起到一个抛砖引玉的过程，想告诉大家的是，代码写好了，推进的过程人类也不是在旁边泡着茶，就看AI执行就行了。在推进的过程中，需要人的介入和指挥，创建必要的工具，指定合适的流程编排。道理很简单，等你做的时候，遇到的各种问题的解决和优化是你更加宝贵的经验。

文档写好了，并不意味着万事大吉：发一条提示词，让 AI 持续推进，我们最后来验收就行了。规范明确了目标、约束和完成标准，但在推进过程中，我们还要考虑如何下达指令、如何安排 Agent、采用什么执行模

式，以及工具和环境能否支撑验收。如何让AI能够更好地持续推进，也是需要经验和不断适配的，我们从4个方面来讲解一下：

3.2.1 提示词：明确这一次怎样推进

提示词通常是启动执行的触发器（trigger）。除了告诉 AI“按照规范实现”，还应明确：本次只做一个任务，还是按依赖连续做几个；通过哪些文件和章节获取上下文；完成后怎样验收；由 AI 还是人更新进度；遇到什么情况需要停下来。

例如，在将附录 A 的规范落为文件、准备好练习代码后，可以这样启动其中的任务：

```Plain Text 本次只执行 specs/tasks.md 中的 T1。 先阅读 proposal.md 的 R1～R3、design.md 的职责边界， 以及 specs/contracts/bug-intake.md 的输入输出、来源与判定顺序， 再查看项目已有代码和测试方式，完成实现与测试。

按 T1 的验收标准实际运行检查，报告修改文件、命令和结果。 全部通过后，由你记录验证证据并将 T1 标为完成，不继续 T2。 遇到规范冲突、环境不可用或返修三轮仍未通过时，停止并报告； 未执行的检查不得标为通过，不要自行提交、推送或发布。
```

> 如果希望连续推进，就将范围改为指定任务集合，并要求“前一任务验收通过后，再选择依赖满足的下一项”。不同模型对指令的响应、不同 Harness 对上下文加载、权限和任务状态的处理并不相同；规范入口文件也未必叫 `CLAUDE.md`。因此，这类提示词需要结合实际框架和任务适配，而不是复制后就默认所有行为都会发生。
>
> **3.2.2 Agent 编排：决定谁来做、怎样交接**
>
> 简单任务通常用一个 Agent 就够了，由它读取规范、实现和验证。任务较多、存在独立工作，或需要独立审查时，再考虑多个 Agent：主 Agent 负责派发和汇总，子 Agent 负责限定范围的实现或审查。派发时应给出任务编号、规范引用、前置产物、修改边界和验收要求，不能假定子 Agent 已经知道主会话的全部背景。
>
> 多 Agent 还需要明确通信与交接方式。对于独立任务，通常由子 Agent 向主 Agent 返回改动、测试证据和阻塞即可，不必让所有 Agent 互相聊天；如果确实需要协商接口或交换中间结果，就要确认框架支持相应通信，并约定谁确认变更、谁更新进度。通信机制不能代替统一的接口契约。
>
> 并行也不是默认选择。本课的 T1 → T2 → T3 有前后依赖，应串行推进；只有前置依赖满足、修改范围和共享资源不冲突的任务才适合并行，最后仍需统一做集成验证。
>
> 根据上面的选择，比如使用ClaudeCode我们可以使用**SubAgent，AgentTeam，Dynmaicworkflow**的模式，同时之前讲过一些技巧，比如让Agent做完一个任务就销毁，避免上下文腐烂，Agent的上下文空间每次都只使用50%，这样任务执行会有更好的效果。通过这些Harness技巧，让Agent推荐的更快，更稳。
>
> **3.2.3 推进模式：决定按什么节奏执行**
>
> Agent 的数量解决“谁来做”，推进模式解决“先做什么、何时检查、失败后回到哪里”。
>
> - **规划与执行：**先形成或核对计划，再按步骤执行。在 SDD 中，`proposal.md` 提供目标，`design.md` 提供约束，`tasks.md` 承载计划。AI 可以辅助规划，人确认后再执行；已有明确计划时不必重新拆解，发现计划或规范有问题时则反馈调整，而不是边写边擅改目标。

- **生成与评审：**实现后按规范检查，发现不符合项就返修，通过后再继续。可以由同一个 Agent 分阶段完成，也可以交给独立审查 Agent；两种方式都需要实际验证证据，并设置返修上限，避免无限循环。
- **护栏三明治：**执行前检查范围、权限和必要输入，执行后检查产出与验收结果。适合机械检查的规则，可以通过 Harness 的权限机制、Hooks 或测试脚本落实，而不只靠提示词提醒。

这些模式可以组合使用。例如，“核对计划 → 执行当前任务 → 测试与评审 → 记录结果 → 下一任务”，就是一个轻量的推进安排；层级委派则可以按上一节的分工方式嵌入其中。不需要为了使用模式而额外搭建一套框架。

3.2.4 工具与 Skill：提前准备执行和验收能力

验收方法写进文档，不等于 AI 已经具备对应能力。如果要求 AI 实际操作 UI，就需要可用的浏览器或桌面操作工具、运行中的应用和测试账号；如果要求创建 VM 并验证，就需要虚拟机管理与远程执行工具、镜像、必要权限，以及环境准备和清理流程。这些能力应在执行前准备并检查可用性，必要时通过 MCP 接入已有服务。

巧妇难为无米之炊，我们写好了Spec，自己应该先知道，AI都否具备执行这一套的能力和环境，如果不具备，就应该给AI提供对应的能力或者及时修改方案。

2.5.4 变更与版本维护

写好了文档，推进执行也不一定一帆风顺。AI 可能完成了任务，但交付结果仍然不符合预期；也可能执行到一半，原来的要求或方案已经发生变化。遇到下面这些情况，我们应该怎么办？

- AI 说已经完成，实际使用却发现 bug、结果不符合预期，是直接让它改代码，还是先改规范？
- 执行到一半，需求变了，已经写好的设计、任务和代码该怎么调整？
- 需求没有变，但实现时发现原来的接口或技术方案行不通，应该从哪里改起？
- 需求和设计都没变，只是任务拆得太大、依赖顺序不合理，还需要重写前面的文档吗？

这些问题看起来不同，却可以用同一个原则处理：**找到受影响的最上游层级，从那里开始修改，再向下检查影响。** 先判断问题出在哪一层，而不是一遇到变化就让 AI 直接改代码，也不是每次都把全部规范重写一遍。

4.1 “顶”和“下”指什么？

这里的“自顶向下”，说的是需求到交付之间的逻辑依赖：

```Plain Text
需求 → 设计 → 任务 → 实现 → 验证
```

需求决定要达成什么，设计决定如何组织系统来满足需求，任务把方案拆成可执行工作，实现交付具体行为，验证则检查这些行为是否符合需求、契约和验收标准。越靠近目标和约束的部分，越处于上游；依赖这些决定的部分，则处于下游。例如，相对于代码，接口契约属于上游；相对于需求，设计方案又属于下游。

需求变了，要向下检查设计、任务、实现和测试；如果只是代码没有遵守正确的规范，就从实现层修复，不必倒过来修改需求来迁就错误代码。

下图展示了不同变化的起点及其向下影响的范围。箭头表示需要检查的依赖关系，**不是每一层都必须修改。**这也不是磁盘目录的嵌套关系；契约和 ADR 属于设计相关材料，不需要硬排成两个额外阶段。

4.2 四类问题，分别怎么处理?

1. AI 执行完了，却发现 bug 或结果不符合预期。

先把问题复现出来，对照规范判断偏差。如果要求已经写清楚，只是实现没有做到，就让 AI 修复代码，并补充能够复现问题的测试，再运行相关回归。不要为了让测试通过而放宽验收标准。

如果代码符合现有规范，但规范遗漏了实际需要的行为，或者接口根本无法表达预期结果，就不只是实现 bug。此时应先明确预期，追溯到需要修正的需求或设计，再按下面的方式向下调整，而不是只在代码里打补丁。

2. 执行到一半，需求变了。

先暂停受影响的任务，确认新的目标、范围和验收标准，更新 `proposal.md`；再检查设计与契约是否需要调整、任务是否需要增删或重新拆分、已有代码和测试哪些需要修改。未受影响的成果可以保留，不必全部推倒重来。

把这些影响同步清楚后，再让 AI 依据更新后的规范继续执行。已经完成的任务如果受到了影响，也需要重新打开或新增后续任务，不能沿用旧的完成状态作为新要求已经满足的证据。

3. 需求没变，但接口或技术方案需要调整。

从 `design.md` 和相关契约开始修改，涉及关键架构取舍时记录 ADR。随后检查受影响的任务、接口调用方、实现和测试，更新后再继续推进；原有需求仍然成立时，无须重写需求规范。

如果所谓的“方案调整”实际上改变了用户可见行为、功能范围或成功标准，就要再向上确认需求，不能把需求变化藏在技术修改里。

4. 只是任务粒度或执行顺序不合适。

在需求与设计不变的前提下，直接调整 `tasks.md`：拆分过大的任务，修正依赖，明确各项交付物和验收条件，并同步后续执行安排。已有成果需要对应到调整后的任务中，避免重复实现或遗漏验证。

这类变化通常不需要改需求和设计。但如果调整任务时发现必须改变模块职责或接口，说明影响已经超出任务层，应回到设计层处理。

下图汇总了四类问题的判断入口、修改起点与处理路径，可在发生变化时对照使用。

4.3 怎么维护历史?

- 【冲突不要保留】当前规范失效时，要让当前内容反映新的要求和设计，不能长期保留相互冲突的有效描述。
- 【历史要上库】规范与代码都纳入版本控制，历史由 Git 保留。
- 【技术选型建议记录】关键架构决定被替代时，追加新的 ADR，记录取代关系，保留旧决定的理由。
- 【任务状态视情况而定】完成任务先标记状态；是否留在当前清单、何时归档，由项目约定。

2.5.5 工具与选型

接下来，我们来介绍一下常见的**主流的SDD相关的辅助的框架和工具**：OpenSpec, SpecKit, Supwerpowers. 当我们掌握了上面的SDD的标准的流程和思路，我们也就掌握了这些工具的内核，他们的核心都一样：都是在SDD的原则下，帮助你建立一套规范的SDD体系，帮助你解决：文档如何编写?AI如何推进实现？变更发生后如何修改？不过他们的侧重点和规范可能不同。

这一章会大概介绍三个最后的SDD辅助工具。选择什么工具，没有唯一的标准。我们需要掌握SDD的内核，亲身实践，找到最适合自己的SDD的方法和工具。 同时并不是说我们选择工具是非此即彼地，可以联合使用使用，或者修改对应某个框架的具体实现让他更适配你的任务。

5.1 OpenSpec：以变更为单位维护规范

OpenSpec 是一个偏轻量的规范驱动开发框架，帮助我们把一次变更的动机、需求、设计和任务组织成可审阅的材料，再交给 AI 实施。它的核心思想是：**先把“这次准备改变什么”写清楚，再让实现围绕这份变更展开，最后将结果沉淀回规范。**

当前官方文档展示的典型流程是：

```Plain Text 探索问题（可选）→ 提出变更并准备材料 → 审阅 → 实施 → 归档并更新规范

> 它不要求先把整个项目写成厚重的规格书，可以从一次局部变更开始，也允许在推进中修订规划材料。对于已有代码库中不断发生的功能调整和维护工作，这种以变更为中心的组织方式比较自然。
>
> 当前默认工作流通过 `propose` 准备提案、需求、设计和任务材料，通过 `apply` 推进实施，通过 `archive` 完成归档；渐进编写、验证等额外工作流可以按需配置。它还提供处于 Beta 阶段的 Stores，将规划材料放在独立 Git 仓库中，支持跨仓库协调，入门时不必引入。
>
> **适合的诉求：**希望规范可持续维护，但不想一开始引入较多固定步骤。
>
> **5.2 Spec Kit：把规范开发组织成标准流程**
>
> Spec Kit 是 GitHub 提供的规范驱动开发工具包，通过模板、命令和检查机制，把想法逐步转成规范、技术方案和可执行任务。它的核心思想是：**不要直接从一句需求跳到代码，而是用明确的中间产物和检查步骤约束开发过程。**
>
> 当前官方文档展示的主流程是：
>
> ```Plain Text
> 项目原则 → 需求规范 → 技术方案 → 任务拆解 → 实施 → 完成度评估
```

其中，Constitution 定义项目共同遵守的技术、质量与合规原则，不等同于某次功能的需求；**Clarify、Analyze 和 Checklist** 等活动分别帮**助澄清歧义**、**检查材料一致性和审阅要求质量**，不能把生成检查清单当成已经执行测试。

值得关注的变化是，当前流程加入了 **Converge**：实施后评估是否达到完成条件，未收敛则继续实施和评估，而不是运行完 Implement 就默认结束。另外，扩展和预设可以承载团队标准及额外流程；

适合的诉求：希望团队使用相对统一的规范、规划和检查流程，或需要将项目原则落实到开发过程。它的组织更系统，但仍需要按任务裁剪，避免把所有可选检查都变成固定仪式。

5.3 Superpowers：把开发方法变成 Agent 的技能

Superpowers 更准确的定位是 Agent 开发技能与工作流集合，而不是另一套规范管理器。它的核心思想是：**不仅告诉 AI 要完成什么，还通过可复用的 Skill 约定它如何澄清、规划、实现、调试和验证。**

它组织的典型过程是：

```Plain Text 澄清与确认设计 → 编写执行计划 → 实施与测试 → 评审和返修 → 交付前验证

例如，`brainstorming` 帮助澄清目标和讨论设计，`writing-plans` 将确认后的设计拆成执行步骤，TDD、调试和评审技能则规定相应的工作方法。这里不只是提供文档模板，而是把开发动作串成可重复使用的流程。

在执行阶段，它既提供当前会话内的 `executing-plans`，也提供 `subagent-driven-development`：将任务交给子 Agent，并分别检查规范符合性与代码质量。这与第三章讲的规划与执行、Agent 委派、生成与评审直接对应；是否采用子 Agent，应结合宿主能力、任务规模和执行成本决定。

**适合的诉求：**已有需求或设计依据，但 AI 容易跳过规划、测试和评审，希望把这些工作方法固化下来。它也会产出设计和计划，但重点不是维护长期规范的变更生命周期；安装 Skill 同样不会自动补齐测试环境和外部工具。



**5.4 怎么选？先看最需要解决的问题**

如果最困扰你的是“每次改动缺少明确依据，规范容易过期”，优先看 OpenSpec；如果是“需求、设计、任务缺少统一过程和检查”，优先看 Spec Kit；如果是“材料已有，AI 却不能有纪律地完成实现与验证”，优先看 Superpowers。这是侧重点的区分，不是说它们的能力完全不重叠。

| 工具 | 主要定位 | 核心思想 | 更适合的情况 | 选型时注意 |
|-|-|-|-|-|
| **OpenSpec** | 轻量规范与变更管理 | 围绕一次变更组织材料，实施后沉淀规范 | 已有项目持续迭代，希望从局部变更起步 | 按需选择工作流，保留审阅和实际验证 |
| **Spec Kit** | 结构化规范开发工具包 | 用项目原则、中间产物与检查步骤约束开发 | 团队希望统一需求、设计、任务及完成检查 | 按任务裁剪流程，扩展与命令需匹配版本和宿主 |
| **Superpowers** | Agent 技能与开发工作流 | 将澄清、规划、TDD、评审等方法固化为 Skill | 希望改善 AI 的实施纪律和执行闭环 | 不等于长期规范管理；多 Agent 和测试依赖实际环境 |

---

<a id="doxcnrkGaQKR8KRLUg1pOXtrULc"></a>
#### 2.5.6 面试问答

掌握SDD对我们面试的过程中很有帮助。 第一个角度：我在面试深信服和美的 Agent开发的时候，在对方的JD上明确的看到了对于SDD开发方式的要求。所以SDD这个概念甚至是上升成了一种JD的技能要求。 第二个角度：面试官问你AI vibecoding的技巧的时候，可以从SDD的角度来介绍你的经验，所以理解SDD还是很有帮助的。 以下汇总一下SDD相关的面试问题。

**1.你理解SDD是什么吗？能不能聊一下你的看法？**

**<选自深信服Agent开发一面>**

**💡 答题思路**
```

先解释概念，再说明 AI 开发为什么需要 SDD，接着介绍核心原则，最后说明使用时的注意事项。

🎤 参考答案

SDD 是规范驱动开发。我的理解是，先把需求、设计和验收标准写清楚，再以这些规范为依据，拆解任务、实现代码、验证结果。在 AI 开发中，人主要负责确认需求、把关方案和关键决策，AI 负责具体实现和测试。

我觉得它在 AI 时代特别有价值，因为 AI 生成代码很快，需求里一点模糊的地方，都可能被它直接扩展成一套实现。同时，项目一长，前面确认的要求容易在多轮对话中丢失。把这些内容沉淀到仓库里的规范文档，就能给后续开发提供稳定的依据，换会话或换人也能继续。

具体来说，我会关注三个原则。第一，职责清楚。需求说明做什么、怎样算成功，设计说明怎么实现，任务说明具体步骤和依赖。同一个约定有统一的定义，其他地方通过引用使用。第二，结果可验证。任务要有明确的验收条件，AI 完成后实际运行测试，通过了再更新进度。第三，变更要同步。比如需求调整了，就从需求开始，检查设计、任务、代码和测试受到哪些影响，再更新相关内容。

使用时，我觉得要控制好文档粒度。影响系统行为的要求、接口和边界条件必须明确，局部编码细节可以交给 AI。推进过程中，人也要持续审阅和处理分歧。这样，人的精力可以更多放在目标和方案上，AI 的实现有明确约束，交付结果也有依据可以检查。

2.你在vibecoding的工程中，有没有什么比较AI native的方法，可以介绍一下吗？

[<选自字节2026年10月秋招Agent开发一面>](#doxcnXUvkrCWHLbqSC0uyDEsJKd)

💡 答题思路

围绕文档组织、上下文管理、自动检查三个习惯展开，最后补充将文档提交到个人分支，方便追踪和交接。每一点都讲清具体技巧和好处，体现开发的规范性和对 AI 使用方式的理解。

🎤 参考答案

有，我做 AI 编程时比较重视规范和上下文管理，会结合 **SDD 形成一些固定习惯。**我分享一下我的使用习惯吧。

第一，**文档也按代码的标准来组织。需求、设计、任务各自有明确职责，**目录和命名保持统一。同一个接口或规则只在一处定义，其他文档引用，避免改了一处漏掉另一处。我还会留一个简洁的入口，列出文档的位置和用途。这样人接手容易理解，AI 也能准确找到依据，后续维护成本比较低。

第二，**我会控制给 AI 的上下文。每个任务关联对应的规范章节、已有代码和验收条件**，让它按需读取，减少无关信息的干扰。另外，关键决定、已完成事项和阻塞原因都会及时写回文档。这样长任务中途换会话，也能恢复进度，省掉反复解释背景的时间。如果某个要求需要我反复纠正，我就会把它补进对应规范，减少同类错误。

第三，重复检查尽量自动化。比如可以用 **hook 在文档修改后触发一致性检查，**关注接口调整有没有同步到任务和验收条件。格式、引用这类规则用脚本检查，语义上的疑点让 AI 辅助审阅，再由我确认。任务完成也要附上实际测试结果。同时定期清理过期描述，历史交给 Git，避免 AI 读到相互矛盾的要求。

最后，我会把这些文档和对应代码一起提交到**仓库的个人分支上**。这样每次改了什么、为什么改、任务推进到哪一步，都有记录可查。需要交接时，同事拉取这个分支，就能看到对应版本的代码、设计依据和任务进度，他的 AI 也可以读取这些文档继续推进，省掉很多重新梳理背景的时间。

2.5.7 讲解视频

https://www.bilibili.com/video/BV1dtxh6yEA7/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

3 SKILL

SKILL 比较新，是2025年10月16号提出，在12月份左右才慢慢被市场支持，在26年1月份已经变得很火了。所以虽然目前我的面经没有涉及。但是很多朋友在1月份面试已经被问到了，而且这个概念现在很火。学习大模型，特别热门的新技术一定是要学的。从目前（2026年1月）来看，Skills会成为未来一段时间面试高频考点，务必要掌握。推荐学习路线为：

1：**入门视频:** 了解Skill是什么，怎么用。(视频见[3.10](#doxcn4OickWtKdhcNAsL795KEvf))

2. **进阶视频：**掌握Skill的设计思路。(视频见[3.10](#doxcn4OickWtKdhcNAsL795KEvf))

3. **Skill编写规范**：本节的笔记部分以及我在下面推荐的Anthropic关于Skill的官方文档：了解Skill编写的最佳实践，范式，命名规则等。

4. **如何写SKILL：**下面推荐视频有个博主讲如何写出SKILL的，以及现场教学。最重要的思想：[Feedback Cycle](https://www.youtube.com/watch?v=zKBPwDpBfhs&t=324s)其实就是反复迭代，了解了1，2，3，可以看一下他展示真的如何写的这个思想和过程。

5. **实战**：

   1. 参考视频笔记提供了一些官方的SKILL github链接，特别是第4个链接，是由Anthropic提供的一组Skill示例，严格遵循了它提供的Skill的最佳实践文档，可以作为Skill的教科书级别的示例来阅读。 (链接见3.10 参考视频和项目里面的**Skills实战**)

   2. 最后是你应该自己写一些SKILL，比如自己组内用的内部提效的SKILL，公司用的更标准的SKILL，写一些你就会更有经验。面试的时候除了基本概念，面试官喜欢问你自己做了一些什么SKILL，用了一些什么技巧，可以参考[面试实战部分](#doxcn3hBQLyhVp7mHmJPHUVRSTe)。

3.1 SKILL 概念

Skill是一组打包了指令的文件夹：文件夹会包含Skill.md 的核心指令文件，以及Scripts,Reference和assets的可选资源文件（后文SKILL目录结构小节有更详细解释）。它教会AI如何处理指定的任务和流程。当需要处理重复的工作流，比如根据规范生成前端设计，创建符合团队风格指南的文档，多步骤流程编排等任务时，Skill可以让你无需每次对话重新解释你的偏好，流程，领域知识，而是通过Skill一次教会AI，之后每次都能受用。相比传统的提示词，它采用渐进式披露，动态加载提示词，更节约Token.

3.2 SKILL目录结构

从图中我们可以清晰的理解概念所说，一个SKILL就是一个文件夹。其中包含以下内容：

1.SKILL.md：必须，包含了 Yaml头和Body。Yaml是md文件中的头，告诉AI是否加载这个SKILL，AI必定会加载。而body

是正文，是渐进式披露的，只有AI读取了YAML头，认为这个SKILL应该被加载，才会加载这个正文部分。

2.Scripts：可选。包含着在AI执行过程中需要使用的脚本文件。

3.References：可选。存放执行SKILL相关的文档，说明，内容以MD为主。

4.Assets：可选。存放执行SKILL时需要的模板，静态资源，比如图片等。

2,3,4其实就是在SKILL执行过程中，所需要的资源。Anthropic 官方明确建议一个SKILL不应该超过500行，所以SKILL需要引用其它资源的时候，应该将这些东西放在SKILL.MD之外，而在SKILL.MD中显式指向它。

3.3 SKILL的常见应用场景：

1.生成一致的，高质量的内容，比如文档，PPT，app，design，代码等。实例：[frontend-design skill](https://github.com/anthropics/skills/tree/main/skills/frontend-design)

2.编排多步流程，让这些步骤遵循特定的顺序，示例：[Skill-creator](https://github.com/anthropics/skills/blob/main/skills/skill-creator/SKILL.md)

3.规范/增强MCP使用，将MCP转换成可靠的工作流。实例：[Sentry-code-review skill](https://github.com/getsentry/sentry-for-claude/tree/main/skills)

（这一点我想解释一下的是，根据Anthropic官方指南，规范MCP的使用指的是用了Skill，可以让用户/AI更清楚知道某个MCP Server 如何使用，遇到问题如何解决，取得稳定的结果。通常的最佳实践是在MCP Server的文档中附上SKILL的链接。用户在使用多个MCP Server的时候，也可以通过SKILL来编排多个MCP的使用流。从这一点可以看出SKILL和MCP是相辅相成的，而不是替代关系。这一点是Anthropic官方指南，我觉得这一点比较新颖，也是一些视频没有提到的，面试时候说出一点SKILL和MCP的补充关系和最佳实践，可以加分）

3.4 SKILL测试与评估

通常可以从以下几个角度来测试Skill的质量：

1. 触发测试：保证SKILL会在正确的时机被加载。在不该触发的情况不会被误触发，在该触发的情况也不会不触发。
2. 功能测试：测试SKILL产生正确的结果：输出是符合预期的，API正确调用，错误被处理，边缘情况被覆盖。
3. 性能测试：对比使用SKILL和不使用SKILL时，用户的介入次数是否减少，API调用失败次数是否减少，Token消耗是否降低。

3.5 SKILL典型范式

1. 顺序工作流编排： 当有多个步骤，并且需要按照指定的顺序执行时使用。

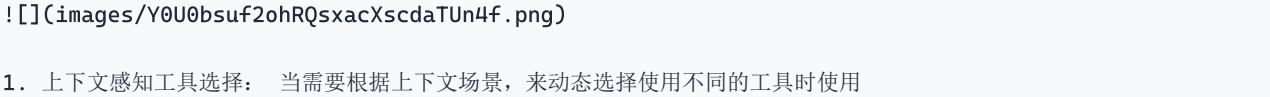
核心要点：说明步骤顺序，步骤间的依赖，每一步如何验证，失败时如何回滚。

1. 多MCP 协作： 任务流包含多个MCP Server时使用。

核心要点：说明多MCP步骤间如何传递数据，错误异常处理，清晰的步骤界限：

1. 迭代优化： 需要通过多次迭代提升输出质量时：

核心要点：清晰的质量标准， 迭代提升，验证脚本，知道何时停止迭代。



1. 上下文感知工具选择： 当需要根据上下文场景，来动态选择使用不同的工具时使用

核心要点：清晰的选择策略，兜底选择，选择理由透明

1. 领域专业智能：Skill需要使用特定领域的知识来完成任务时使用。

关键要点：将专业知识嵌入逻辑，合规性保证，详尽的领域文档，

3.6 SKILL vs MCP

（面试官特喜欢问的就是差别，你答得不多，还喜欢追问，还有吗？这里尽量给大家总结全面一点）

借用Anthropic 官网文档原话，Skills 和 MCP的核心差别是：MCP 提供给模型数据，而Skills教会模型如何使用这些数据（"MCP connects Claude to data, Skills teach Claude what to do with data)。二者的区别为：

1. MCP侧重于提供工具调用，而Skills则侧重于提示词。MCP更像是一个标准的工具箱，提供规范，可复用的工具。而Skills则是一个带目录的说明书。
2. MCP 的本质是一个独立运行的程序，而Skills 本质是一个文档。他们本质不同，决定了他们性质也不同，Skills更适合处理轻量级的，简单的逻辑，安全性和稳定性都不及MCP。
3. Skills 采用渐进式披露提示词的机制，Tokens消耗低。而MCP则是一次性塞入所有提示词，Token消耗高。
4. Skills的开发难度相比于MCP，开发难度低很多。

总结来说，SKILL和MCP二者不是相互替代，而是相互补充的。 在开发MCP SERVER的时候，可以在MCP Server的文档中添加一个链接，指向一个SKILL的链接，这个SKILL让用户/AI更清楚知道该MCP 如何使用，遇到问题如何解决，让模型调用MCP时取得更稳定的结果。在同时使用了多个MCP Server的时候，也可以用SKILL对多个编排多个MCP Server的工作流

3.7 SKILL vs Prompts

Skills 相比于Prompts的核心差别在于他们的**形态**和**生命周期**。

Prompts 是一次性对话内容，你需要自己保存，复制和粘贴。模型不会知道什么时候用哪一个Prompt，而且你需要把Prompts**全部塞到上下文里面**。这会大量占用你的上下文空间。

Skills 则是将提示词以一种**更加工程化的包装方式打包**，他是一个**文件夹的形式**，里面包含skills.md 和相关的资源文件。他会动**态加载/渐进式披露提示词，**更加节省提示词**。**容易复用和版本管理，易于协作和共享。

总结来说，Skills不是一个更复杂的提示词，而是把复杂的提示词，变成了一个**有结构，可复用，可以自动调用**的模块。

3.8 SKILL的运行模式

1. **用户发起任务**
 用户对 Claude Code 提需求，Claude Code 把这个请求连同当前可用的 Skills列表结合元数据一起交给 AI，让 AI 先基于需求判断该用哪类能力。

2. **AI 选择要用的 Skill**
 AI 在可用的Skills里做匹配，然后返回指定的Skills
3. **按需加载 Skill 的指令层**一旦选定 skill，Claude Code 才把该 skill 的具体定义交给 AI，这一步体现了Skills**指令层 按需加载**：只有用到这个 skill 时，才把更详细的指令灌给模型。
4. **按需加载 Skill 的资源层→ 读范文/材料 → 生成最终回答**如果需要使用到资源层，接下来 Claude Code 会执行读资源的动作，把 **references/范文内容**喂给 AI；AI 基于 用户材料 + skill 指令 + references 完成写作并输出最终回答。

3.9 SKILL的设计理念

（上面其实是一些SKILL的基础知识，相对来说是最基础，或者考到SKILL 必问的。 必须准备好。这一节可以理解为一个拔高项，我们不谈Skills 的具体概念，而结合行业的痛点，发展趋势，未来的展望来聊Skills。 如果你能和面试官讲出来，会体现出你其实是有思考，有想法，关注着行业发展和趋势的。把这些答了，是加分项）

发展到现在（2026年初），现在的Agent工程的痛点不再是能不能做，而是能不能交付。目前的行业痛点是：

1.虽然模型能力很强，但是常识很弱，能解决通用问题，却在具体业务问题上频繁踩雷。经常做的是文本补全，而不是业务理解。这通常导致造出来的Agent需要质检，返工，人力成本高。

2.现在经常导致的一个问题是，公司重复造轮子。给财务做一个Agent，法务做一个Agent，运营做一个Agent，每个Agent都有自己的权限，工具，prompt等。就会变成同一个业务规则，在5个Agent里各写一遍，这就导致重复造轮子很严重，难以维护。

3.现在的Agent通常是一个会有复杂的prompts，这导致token太大，模型不能精准执行，另外成本上升。当你用复制粘贴扩展能力，系统就会用维护地狱来还债。

所以行业发展到现在，需要从堆砌提示词，迭代，调试，优化做一个Agent的过程变成做一个 可治理，可迭代，可复用的产品。通过Skills， 行业开发Agent的模式变成去写一个可复用，可管理的技能资产（Reusable skills），结合通用内核，来解决这些痛点。

3.10 参考视频和项目

入门视频推荐这两个：从skills 是什么，组成，核心特点，实践，对比MCP这几个角度都讲了，基本是最基础的内容。这两个视频其实都围绕这几点讲，内容有重复性，可以结合互补着看，适合入门。

<readonly-block href="https://open.douyin.com/player/video?vid=7592521205031177499&mode=pc" type="iframe"></readonly-block>

<readonly-block href="https://open.douyin.com/player/video?vid=7590008747938876706&mode=pc" type="iframe"></readonly-block>

进阶视频：

这个讲了一些Skills背后的设计理念，行业痛点，学好了和面试官回答可以体现你的行业洞察能力。

https://www.youtube.com/watch?v=xeoWgfkxADI

这个提到了和Prompts的本质区别，以及提了两种skill的使用方式，相比入门视频深入一些。

https://www.youtube.com/watch?v=ZzPoWrlzE1w

如何写SKILL

聊了这么多都是理论，到底如何写SKILL，看一下这个视频，特别是第13分钟开始，博主讲了如何写SKILL，原则是什么，现场展示了这个过程。其中他说的Feedback Cycle说的是写SKILL最重要的是迭代，这点我很认同。他的例子使用他自己写的Skill-builder生成skill(我用了感觉不好用），你可以使用官方的SKILL-Creator， 其它的原理是一样的。

https://www.youtube.com/watch?v=zKBPwDpBfhs&t=324s

Skills实战：

本小节提供了一些Skills的实例。

1. 为 AI Agent（如 Claude Code）提供一组操作 Obsidian 笔记的技能，让 AI 能直接创建、修改 Markdown / Canvas / Bases，实现“AI 管理知识库”。

https://github.com/kepano/obsidian-skills

1. 一个给 AI 编程 Agent 用的“工程化技能框架”，强制先澄清需求、写设计、做计划，再用子 Agent 按 TDD / DRY 的方式自动开发代码

https://github.com/obra/superpowers

1. 让 AI Agent 把“计划、进度、知识”写进持久化 Markdown 文件中，用文件系统当长期记忆，解决复杂任务中上下文丢失的问题（Manus 风格）。

https://github.com/OthmanAdi/planning-with-files

1. Anthropic 的官方示例，关于Skill的编写规范，可以以这个为模板。比如命名规范，编写风格，可以把这个当作教科书。下面分享的这个官方文档是由Anthropic官方写的，文档里给的Skill的例子也引用了这个github 项目里的例子。完全可以相信这个仓库中的Skill的质量。这个项目里的Skill-Creator 也是被广泛使用的Skill.

https://github.com/anthropics/skills/tree/main/skills

https://github.com/anthropics/skills/tree/main/skills/skill-creator

官方文档

这个文档是Anthropic 提供的教科书，包含Skill组成，如何命名，使用场景，常见范式等。非常值得一读，笔记的Skill文档部分很多地方我也是参考的这个文档。时间充足可以自己阅读，不充足可以看我的笔记，我的笔记总结了这个文档的最精髓部分。

https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf?hsLang=en

https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

3.11 面试实战

最近自己在外面也面试了好几家公司，粉丝朋友在群里也反馈了SKILL的相关面试问题。我在这里给大家汇总一下相关面试题。本篇主要是汇总SKILL面试问题 + 讲解。主要深度解读两个我认为比较难的面试问题：

- **Q1：你在做 SKILL 时遇到了什么困难，如何解决的？**
- **Q2：你做了什么 SKILL，可以分享一下吗？**

这两个问题主要难在需要一些实践经验，最好回答的能比较有深度，解决的是一些比较难一点的SKILL问题。剩下汇总的其他问题：比如**SKILL是什么？他的设计思想是啥？**等等问题也经常被问到过，但是这些问题就容易啦，不需要精讲，我就直接给到答案好了。本小节的内容也接受大家继续反馈，你遇到的SKILL的面试问题，如果有咱们没讲到的，在粉丝群中向我反馈，我也会及时继续更新总结的（本小节写于2026/07/25）

讲清楚Q1,Q2，我的思路是先去学习理论基础。这些理论基础的来源还是我3.10里写到的这些文档，并没有超出这些文档之外。我从上面的参考资料，主要是[Anthropic 公司的官方SKILL文档](https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices)抽取了一些最佳实践-解决方案。这是回答Q1,Q2的理论基础，我们Q1,Q2就围绕这个理论基础，展开回答就好。因为这两个问题偏向于实战内容，所以我们干脆以官方的Code review skill为例来讲解，顺便学习一下官方代码review skill的思路。

全文总结方式为：理论基础 - Q1,Q2问题答案 - Code-review Skill讲解 - 其它面试问题及答案。

3.11.1 理论基础

这一节是整节的「理论底座」，也是「弹药库」：把 7 个官方最佳实践各自对应的**问题、根因、解决方案**讲清楚，后面 Q1/Q2 都从这里取用；被追问到任何一条都能接得住。

实践 1 | 设定合适的自由度

- **对应问题**：误报泛滥——AI 报一堆没价值的问题（nitpick、既有问题、linter 就能抓的、用户没改的行）。根因是 code-review 属于"高自由度"任务，模型爱过度发挥。
- **解决方案**：rubric + 置信度打分 0-100 + 阈值过滤（只报 ≥80）；正文里显式写一份"false-positive 清单"让模型自我排除。这个有点像[3.3.1.1 LLM-as-a-judge里面这个提示词模板](#doxcnmnnX86PQPvMILpDHsVIeie)。

实践 2 | 写好 description

- **对应问题**：触发不准——该触发没触发（undertrigger）/ 不该触发乱触发（overtrigger）。根因是 `description` 没写好。
- **解决方案**：description 用第三人称 + 写清 what+when + 用户会说的关键词；overtrigger 可用 `disable-model-invocation` 改成只手动调。

实践 3 | 渐进式披露

- **对应问题**：团队规范落不了地——想让 skill 严格按团队规矩审，但规范内容多，全塞正文会超长、费 token、稀释指令。
- **解决方案**：规范外置成 reference 文件 + 按需加载，正文只放"逐条核对规范"的流程；SKILL.md 正文控制在 500 行以内。

实践 4 | 引用只保持一层深 + 长文件加目录

- **对应问题**：指令被"预览"读不全——参考文件套得太深，模型用 `head -100` 只预览开头就走、读不全。根因是引用嵌套太深。
- **解决方案**：引用只保持一层深（都从 SKILL.md 直接链）；超过 100 行的参考文件顶部加目录。

实践 5 | 评测先行 + 与 Claude 迭代

- **对应问题**：触发 ≠ 做对了（看到生效就以为成了，实际输出飘忽）+ 效果好坏全凭感觉（没客观基准，改一版测一版全

靠人肉、跨版本没法比）。根因是缺验证闭环。EDD
- **解决方案**：先在无 skill 下裸跑真实任务、记失败点建 baseline；固化成 eval 集（官方建议至少 3 条，每条含 `query`+`files`+`expected_behavior`）；用全新 Claude 实例（fresh session）在真实任务上测，避免作者会话的残留上下文"脑补"掉指令漏洞；每改一版都用同一套 eval 对照 baseline 量化——eval 是效果的"真理来源"（source of truth）；官方无内置评测工具，需自建。

实践 6｜测试所有计划使用的模型

- **对应问题**：成本膨胀——多 agent 并行、每路都用最贵模型 → token 成本成倍涨。注意：子 agent 各自独立上下文、只回传结论，不污染主对话，上下文隔离反而是优势，代价只是钱。
- **解决方案**：按「要不要真读懂代码」把活分三档——**Haiku 做机械高频活**（PR 资格检查 / 打分 / 摘要 / 格式过滤）；**Sonnet** 做需读懂 diff 做规范合规判断的并行审查主力（默认档）；**Opus** 只按需升级给极难推理（跨文件架构、隐蔽并发 / 安全 bug）。

实践 7｜持续测量与退休

- **对应问题**：skill 会过时——模型升级后某些 skill 变多余。根因是分不清 skill 类型。
- **解决方案**：区分 capability-uplift（能力增强型，可能过时）vs encoded-preference（团队偏好型，长期有效）；靠 eval 判断该不该退休。

3.11.2 面试问题汇总

总结了相关的SKILL面试问题，这些都是真的面试过程中被问到的，虽然我没有总结具体是哪一家公司哪一次面的，但是这些都是面试真题（来自于我自己面试和粉丝反馈）。其中我认为稍微难一点的是Q1,Q2，因为这两个问题偏实战，剩下的几个问题都比较偏理论。大家把上面的SKILL的理论学好，这些问题就都很容易回答了。

本节面试问题速览：

\- **Q1**：你在做 SKILL 时遇到了什么困难，如何解决的？

\- **Q2**：你做了什么 SKILL，可以分享一下吗？

\- **Q3**：SKILL 是什么，可以谈谈你的理解吗？

\- **Q4**：怎么样能写好一个 SKILL 呢？

\- **Q5**：怎么提升 SKILL 的任务准确率呢？

\- **Q6**：SKILL.md 随着业务变大越写越长、导致 AI 不听某些指令，要怎么解决？

Q1：你在做 SKILL 的时候遇到了什么困难，如何解决的？

【思路】 从上面 7 个最佳实践里挑 **3 个**、结合 code-review 场景讲深，我挑这个三个是因为我觉得这个相对高级一点，比如我没有挑渐进式披露这个思想，是因为我觉得这个思想太基本了。但是答案组织的方式是一样，你也可以从上面其他点出发，展开回答。

- **实践 1｜设定自由度**（解决**误报**问题，是 code-review 的命门，最能体现技术判断）
- **实践 4｜引用不要深层嵌套**（解决**深层调用读不全**问题，讲成一次自己踩坑排查：发现模型用 `head -100` 只读了开头）
- **实践 5｜测试先行 + feedback cycle**（表明 skill 是**基于测试**打磨的，解决"忽好忽坏、没法衡量"的问题）

【口语化答案】

我做的是一个团队用的 **code-review skill**，做的过程里踩了不少坑，我挑三个印象最深的说。

第一个、也是最头疼的，是误报。 一开始我就是很朴素地写了个 skill，让 Claude 去审 PR 的 diff，结果它特别"热心"，一个 PR 能给你挑出二三十条，但里面大部分是噪音——有的是这行代码本来就有的老问题、跟这次改动没关系；有的是 linter、编译器自己就能抓的，比如 import 顺序、格式；还有一堆是那种资深工程师根本不会 care 的 nitpick。结果就是没人愿意看它的评论，因为信噪比太低。

我后来想明白了，code review 本质是个**高自由度的任务**——同一段代码，怎么审都算"审了"，所以模型很容易放飞。这时候的解法就是**主动给它收窄自由度**，我用了两招：**一是给每个问题打一个 0 到 100 的置信度分，然后设一个阈值，只有 80 分以上的才真正发出来**；打分的标准我是写死在 skill 里的，比如"经不起推敲的、或者是既有问题"给 0 分，"确认是真的但可能是 nitpick"给 50 分，"高置信、确实会踩、而且团队规范明确点了名"才给 75 分往上。**二是我在 skill 正文里专门列了一份"什么不该报"的清单**——既有问题、用户没改的行、linter 能抓的、被规范显式豁免的，全列出来让它自己先排除。这两招下来，一个 PR 的评论从二三十条压到了三五条，但条条是干货。

第二个坑更隐蔽，是我发现它有时候压根没按我写的规范审。 我把团队规范单独抽成了一个 reference 文件，SKILL.md 正文里只写"去读这份规范、逐条核对"。但跑起来我就纳闷——有些明明规范里白纸黑字写了的点，它就是漏。我一开始以为是规范写得不够清楚，改了几版没用。后来我去扒它到底怎么读文件的，才发现问题：**它读那个 reference 文件的时候，用的是 `head -100` 这种命令，只把开头一百行"预览"了一下就走了**，我那份规范一百行开外的内容它根本没看到。再深挖，是我把引用套了太多层——SKILL.md 指到一个中间文件，那个文件再指到规范，嵌套一深，模型就倾向于只预览、不通读。解法有两个：一是把引用拍平、只保持一层深，所有 reference 都从 SKILL.md 直接链过去，不再转手；二是给超过一百行的规范文件顶部加一个目录（TOC），让它一眼看到全貌、知道该往下读。这么一改，规范漏检的问题基本就没了。这个坑挺提醒我的——**skill 不是你写了模型就会乖乖全读，得顺着它实际怎么取上下文去设计文件结构。**

第三个，是这个 skill 一开始忽好忽坏，我说不清它到底行不行、也不知道每次改动是变好了还是变坏了。 同样一类 PR，它今天审得挺准，明天可能就抽风。全靠我"感觉"，没法衡量。后来我干脆上了一套**测试先行 + 反馈闭环**的打法：**先不给 skill，让 Claude 裸跑几个我们真实的、有代表性的 PR，把它审错、漏审的地方记下来，固化成一组固定的测试用例、当成 baseline**——每条用例写清楚给什么 PR、期望它挑出哪些问题。之后就进入"测试→反馈→修改"的循环：**每改一版 skill，都拿这同一套用例重新跑一遍，跟 baseline 比是进步了还是退步了**；关键是**每次都用一个全新的 Claude 会话去测**，因为要是用我写 skill 那个会话测，它脑子里还残留着上下文，会把我指令里的漏洞给"脑补"掉，测不出真问题。就这么一轮轮跑到指标稳定、达到我要的效果为止。这套下来，skill 好不好不再是"我觉得"，而是有一组可复现的用例在给我兜底——**这版比上版强多少，是量化出来的。**

这三个坑其实对应了做 skill 的三个层面：**误报治理是"输出质量"、引用拍平是"结构设计"、测试闭环是"怎么科学地衡量和打磨"**，算是把这个 skill 从能跑到好用磨了出来。

(被追问时的储备：团队规范外置省 token｜实践 3、分级用模型省成本｜实践 6、description 怎么写才触发准｜实践 2、skill 会不会过时｜实践 7——都在上面最佳实践表里。)

Q2：你做了什么 SKILL 可以分享一下吗？

【思路】 按 **定位 → 结构 → 核心设计 → 效果** 讲，把 Q1 的困难解决方案自然带出来，形成呼应。我们还是以Code-review的SKILL来讲，融合进官方code-review skill的思想。如果你想讲你自己做了某个其它的SKILL也没问题，可以用我们上面的思想去包装一下。如果你没有做什么特别深入的SKILL，就可以说你做了**code-review skill，**然后用下面的回答讲。

【口语化答案】

我分享一个我给团队做的 **code-review skill**，目标很直接：**在提 PR 或者 review PR 的时候，让 Claude 自动按我们团队的规范审一遍代码，只把真正高价值的问题挑出来，直接回帖到 PR 上。**

先说结构，它其实就是一个文件夹：一个 `SKILL.md` 放触发条件和审查流程；一个 `references/team-spec.md` 把我们团队的规范全放进去、按需加载；还有个 `scripts/` 目录放一些辅助脚本，比如先跑一遍 linter 把机器能抓的问题

过滤掉。触发方式两种都支持——Claude 识别到你在 review PR 会自动来问，或者手动打 `/code-review` 调。

核心设计我讲三个我比较得意的点：

第一是多视角并行审查。 我不是让一个 agent 从头审到尾，而是拆成几路并行、各审各的：有的专门盯**团队规范（CLAUDE.md）合规性**，有的**只看 diff 本身抓明显的 bug**、故意不让它读太多上下文免得分心，还有一路会去**看 git blame / history**、结合这个文件以前改动的上下文来判断。几路的结论汇总起来，比单打独斗全面。（官方那个 code-review plugin 就是 4 路并行，我这个思路是参考它的。）

第二是置信度打分 + 阈值过滤，这个就是我刚说的治理误报的核心——每个问题打 0 到 100 分，**只有 80 分以上才发**，把噪音死死摁住。

第三是分级用模型来控成本。 我拆成多 agent 并行有个好处——每个子 agent 是**独立的上下文窗口**，各自去读代码、各自汇总，最后只把结论回传，所以并不会把主对话的上下文撑爆，这点上下文隔离其实是多 agent 的优势。它真正的代价是**钱**：几路并行如果都用最贵的模型，token 成本是成倍涨的。所以我按难易把活分级——像判断这个 PR 该不该 review、给问题打分、写摘要这些简单活，都交给便宜快的小模型（Haiku 那档），只有真正需要理解代码的并行审查环节才上强模型（Sonnet 那档）。这样又快又省。

输出上我也做了规范：评论简洁、不带 emoji、每条问题都**带上精确到行的代码永久链接**，方便人一键跳过去看。

效果我是拿"用不用这个 skill"对比着测的——**人工 review 要来回介入的次数明显少了，误报基本被压下去了，token 消耗也因为分级用模型降了一截。** 现在它基本能当"人工 review 的第一道过滤网"，把机械的、明显的问题先扫掉，人只需要 focus 在真正需要判断的地方。

对我自己最大的收获是想通了一件事：**skill 不是"更长的提示词"，而是把一套团队流程沉淀成了可复用、能自动触发、还能持续迭代的模块**——写一次，全团队每次 review 都受用。

Q3：SKILL 是什么，可以谈谈你的理解吗？

【思路】 这题是概念题，考的是你对 skill 本质的理解，不用堆最佳实践。核心讲三层：**是什么（定位）→ 长什么样（结构：呼应实践 3 渐进式披露）→ 和提示词的区别（价值）**。落点回到"把团队流程沉淀成可复用、自动触发、可迭代的模块"。

【口语化答案】

我理解 **skill 本质上是一个"能被模型自动发现、按需加载"的能力包**——把某一类任务的做法、规范、脚本打包成一个文件夹，模型在合适的时候会自己判断该不该用它，然后把里面的指令加载进来照着做。

它最小就是一个带 `SKILL.md` 的文件夹。`SKILL.md` 顶部有一段 `description`，写清楚"这个 skill 干嘛的、什么时候该用"，模型平时只看这段元信息、几乎不占上下文，只有真被触发了才会去读正文和它引用的参考文件、脚本。这套"**平时只暴露一句话、用到才展开**"的机制就是**渐进式披露**，它解决的正是"知识多但上下文有限"的矛盾——让你能把大量规范塞进 skill，又不会一上来就把模型的上下文撑爆、把关键指令稀释掉。

它和"写一段长提示词"最大的区别是：提示词一次性、每次都得重敲、也不会自己触发；skill 是**沉淀下来、可复用、能自动触发、还能持续迭代**的。所以我会把它概括成一句话——**skill 不是"更长的提示词"，而是把一套流程和领域知识固化成一个模型能自主调用的模块。**

Q4：怎么样能写好一个 SKILL 呢？

【思路】 这题是方法论综述，正好把 7 个最佳实践当"checklist"串一遍，但**不逐条念**，而是按"写 skill 的自然顺序"归成几步讲：**触发（实践 2）→ 结构组织（实践 3、4）→ 输出质量/自由度（实践 1）→ 验证迭代（实践 5）→ 成

本与退休（实践 6、7）**。

【口语化答案】

我自己写 skill 会按几步来，基本对应了几条我踩过坑总结的实践。

第一步，先把 `description` 写好，保证它该触发的时候能触发。 这是最容易被忽略、但最致命的一环——skill 写得再好，触发不了就等于零。我会用第三人称，把"**这个 skill 做什么 + 什么场景下用**"写清楚，还会特意把**用户真实会说的关键词**塞进去。如果一个 skill 老是乱触发，我就干脆把它设成只能手动调。

第二步，组织好文件结构，别把正文写成一坨。 我会把大块的、不常变的知识——比如团队规范——**外置成单独的参考文件，按需加载**，`SKILL.md` 正文只留流程骨架，控制在几百行以内。而且引用**只保持一层深**，所有参考文件都从 `SKILL.md` 直接链过去、不套娃；超过一百行的文件我还会在顶部加个目录，免得模型只"预览"开头就走、读不全。

第三步，控制好模型的自由度、管住输出质量。 像 code-review 这种高自由度任务，模型特别爱放飞，我会主动收窄——比如给结论打置信度分、设阈值过滤，再列一份"什么不该做"的清单让它自我排除。

第四步，也是我觉得最关键的，是测试先行、迭代打磨。 我不会凭感觉说 skill 好不好，而是先固化一组真实任务当测试集和 baseline，每改一版都用**全新会话**重新跑、跟 baseline 比，直到指标稳定。

最后，还要考虑成本和生命周期。 该用便宜小模型的环节就别上贵的；同时想清楚这个 skill 是"补模型能力的"还是"编码团队偏好的"——前者随模型变强可能会过时，得靠测试定期判断要不要退休。

一句话总结：**触发要准、结构要清、输出要收得住、效果要能量化、成本和时效要顾到**，这几条都做到，skill 基本就写好了。

Q5：怎么提升 SKILL 的任务准确率呢？

【思路】 "准确率"要拆成两层：**触发准（实践 2）+ 触发之后做得对（实践 1 收误报、实践 3+4 让规范被完整读到、实践 5 用 eval 量化）**。落点强调**准确率是测出来的、不是写出来的**。

【口语化答案】

我会把"准确率"拆成两件事：**一是该不该触发它触发对没有，二是触发之后做得对不对。**

先说触发准。 很多"不准"其实是 `description` 没写好——该来的没来、不该来的乱来。我的做法就是把 what + when 写清楚、把用户真实会说的关键词塞进去；容易误触的就改成手动调。

再说做得对。 这里我有三招。**第一，收窄自由度、压误报**：高自由度任务模型容易乱发挥，我用置信度打分加阈值过滤，再列一份"不该做"的清单，把噪音摁下去，准确率立马上一个台阶。**第二，保证它真的把规范读全了**——我踩过一个坑，模型用 `head -100` 只预览了参考文件开头，规范后半段根本没看到，结果漏检。解法是引用拍平只留一层、长文件顶部加目录，让它能读到全貌。**第三，也是根本的一招，是用 eval 把准确率量化出来**：固化一组真实任务当基准，每改一版都用全新会话重跑、对照 baseline 看是涨了还是跌了。

所以我的核心观点是——**准确率不是"写"出来的，是"测"出来的。** 你得有一套可复现的评测集当真理来源，才知道每次改动到底让它更准还是更差，否则全是凭感觉。

Q6：SKILL.md 随着业务变大越写越长，导致 AI 不听某些指令，要怎么解决？

【思路】 这题问的是"正文膨胀 → 指令被稀释/漏读"，从三个方向答：**① 和模型能力做适配、做减法（实践 7）——模型越来越强，很多话不必再说，简化提示词；② 渐进式披露（实践 3）——把长内容外置成按需加载的参考文件，正文只留骨架；

③ 引用组织（实践 4）——引用只保持一层深、长文件加目录，保证外置的内容能被完整读到。** 核心是：**正文越短、指令越不容易被淹没。**

【口语化答案】

这个问题我很有体会——`SKILL.md` 一长，模型就开始"挑着听"，越靠后、越啰嗦的指令越容易被它忽略。我会从三个方向去治。

第一，做减法，跟模型当前的能力做适配。 很多话其实是"防御式"写的，是当年模型比较弱才需要反复叮嘱的。**模型一代代变强之后，很多话根本不用再说了**——一些它现在默认就能做对的事，我就大胆删掉。我会定期拿最新模型跑一遍测试，把那些"不写它也照样做对"的指令精简掉，让正文只留真正还需要强调的部分。**正文越短，剩下的关键指令就越不容易被淹没。**

第二，用渐进式披露，把长内容从正文里搬出去。 业务变大，真正膨胀的往往是那些规范、清单、细则。我不会把它们堆在 `SKILL.md` 正文里，而是**外置成单独的参考文件、按需加载**，正文只留"什么时候去读哪份文件、怎么逐条核对"的流程骨架。这样正文能稳定控制在几百行以内，不管业务怎么长，主指令始终清爽。

第三，把外置文件的引用组织好，保证它们真能被读全。 光搬出去还不够——我踩过坑，参考文件套太多层、或者太长，模型就只 `head` 预览开头、读不全，等于白搬。所以**引用只保持一层深**，所有参考文件都从 `SKILL.md` 直接链；**超过一百行的文件顶部加个目录**，让模型知道后面还有内容、该往下读。

总结一下就是：**正文做减法（跟着模型能力精简）、长内容做外置（渐进式披露）、引用做拍平（保证读全）**——三招下来，业务再大，`SKILL.md` 也不会因为越写越长而让指令失灵。

3.11.3 code review skill实战

顺便总结一波Anthropic code review skill的核心思路，其实上面都已经融合了这些思想。我们干脆顺便学一下这个实战例子。

上面整节的例子（尤其是误报治理、多视角并行、置信度打分）大量参考了 **Anthropic 官方 code-review plugin**（`claude-plugins-official`）。这里把它的整体思路和阶段单独讲清楚，作为这个例子的「原型出处」，答 Q2 时也可以顺带点一句「我这套思路是参考官方 plugin 的」。

整体思路：不是让一个 agent 从头审到尾，而是**多个 agent 并行、各审一个视角**，最后统一**用置信度打分把噪音过滤掉**，只把高价值问题发成 PR 评论。

7 个阶段（官方 README 明确写了）：

1. **判断是否需要 review** —— 跳过 closed / draft / trivial / 已 review 过的 PR
2. **收集规范** —— 找出仓库里相关的 `CLAUDE.md` 团队规范文件
3. **概述变更** —— 先把这个 PR 改了什么讲清楚
4. **启动 4 个并行 agent**：
   - **#1 & #2**：查 `CLAUDE.md` 规范合规性
   - **#3**：只看 diff 本身，抓明显 bug（故意不给太多上下文，免得分心）
   - **#4**：看 git blame / history，结合历史上下文判断
5. **置信度打分** —— 每个问题打 0–100 分
6. **阈值过滤** —— 丢掉 <80 分的
7. **只发 ≥80 分** —— 高置信问题才发成 review 评论

置信度 rubric（原文）：0 = false positive；25 = 可能真；50 = 真但次要；75 = 高置信且重要；100 = 绝对确定。

官方参考链接：

- code-review plugin 说明 README ：https://github.com/anthropics/claude-plugins-official/blob/main/plugins/code-review/README.md
- code-review 命令源码：https://github.com/anthropics/claude-plugins-official/blob/main/plugins/code-review/commands/code-review.md

---

4 Harness Engineering

Harness工程，在2026年5月来看，都是今年整个行业的热门考点。行业有一种定义： Agent - Model = Harness。所以 Harness 的概念很大，从工具使用，模型记忆如何配置，如何测试，Agent架构选择，都可以叫做Harness.

学习建议：

1. **理解Harness:** 2.1 2.2. 2.3 总结Harness的定义。其实Harness的视频很多，大多都是讲解它是什么，发展历程，思想是啥，所以找一个看就够了。我给大家推荐了一个中文参考视频，建议看一下视频，因为我这里写得不详细。不过这类视频只用挑一个看，理解概念和思想。
2. **Harness的具体实战：**2.4 小节，我写得很详细。是我看了一个特别好的Harness视频来总结的。他和别的视频不同，是因为他足够具体，给出了具体的Harness的使用思路，而且该博主做了大量实验，引用了大量论文，有理有据。这个视频特别好，对面试，做Agent项目，使用AI都有帮助。我详细解析了这个视频，总结到了文档里，并结合了大量面试题讲解，所以看了这个小节你绝对非常有收获。
3. **Harness具体项目：**2.5，2.6. 大致解析了一些Harness项目。这个里面内容就很多了，大家愿意深入学习的可以去看这些项目。我记得最近有一本书，《*Claude Code* 实战:*Harness*工程*之道*》。 你可以看出来Harness概念很大，它是一种思想，整个Claudecode的设计，都可以叫做Harness。所以Harness更多的学习，还是体现在我们使用AI，构建项目中，不断去体会。这个深入体会就靠大家自己了。

总而言之，其实按照路线a,b学完Harness。 应对Harness的面试问题，应该就没什么问题了。

4.1 概念定义

一句话定义

Harness Engineering 是围绕 AI 模型构建"运行环境与管控基础设施"的工程实践，目的是让 AI Agent 在长时、复杂、多步骤的真实任务中稳定、可靠、高效地运行。

核心思想：从"马具"理解 Harness

Harness 这个词的本意其实是马具——套在马身上用来控制马的那些装备，比如缰绳、头套等。马本身非常强大，但人类必须借助马具的力量才能驾驭它、让它为我所用。

把这个类比迁移到 AI 领域：

- 大模型 = 一匹脱缰的马：能力极强，但如果任由它自己跑，就会发散思维、产生幻觉，无法稳定给出我们想要的结果。
- Harness = 那套马具：用来控制和驾驭大模型的系统。

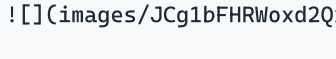
由此我们可以推导出 Harness 的一个常见公式：

Harness=Agent-Model

换句话说，一个完整的 Agent 减去里面的大模型，剩下的所有东西都是 Harness。需要注意，Harness Engineering 是非常新的概念，业界尚未形成严格定义，这个公式是目前比较被认可的一种说法，并非学术定义。

举个具体例子：在 Claude Code 里面，所有不属于 Claude 模型的部分都是 Harness——比如写在 `CLAUDE.md` 里那些大模型要遵循的规则、Claude Code 可以使用的工具列表、它的定时调度机制等等。

4.2 演进脉络



三者是叠加关系：Harness 内部的每个 session 仍然需要 Context Engineering，每次发给模型的 prompt 仍然需要 Prompt Engineering。

4.3 Harness 工程实践要点

- 外部持久记忆——用文件系统（JSON、Markdown、git log）代替模型内部记忆，解决"失忆"问题。
- 确定性验证轨道——Linter、类型检查、单元测试等刚性门禁，AI 的输出必须通过测试才能被接受（对应解决"幻觉完成"——不让模型自己说"我做完了"）。
- 原子任务 + 上下文刷新——每个 Agent 只做一个小任务就销毁，新 Agent 从干净状态启动，彻底杜绝 Context Rot。
- 子 Agent 集群（Swarm）——用大量便宜快速的小模型并行工作，而非一个大模型顺序处理。
- 技能文件（Skills）——按需加载的专用指南，Agent 只看到与当前任务相关的工具和说明（建议不超过 30 个工具）。
- Guard Rails + Checkpoints——在关键节点自动运行检查，防止 Agent 偏离轨道。
- Handoffs——session 之间通过进度文件、git commit 等传递状态，实现跨 session 连续性。
- Human-in-the-loop——在关键阶段设置人工审核断点，弥补 AI 自验证的不足。
- 架构约束——通过 ArchUnit 等框架强制要求 Agent 生成的代码遵循特定模式。
- 垃圾回收 Agent——定期扫描修复代码库和文档中的不一致性，对抗系统熵增。

关键工程原则

- **轻量化，为删除而建**：不要过度编码人类知识，模块要能快速移除替换（Manus 6个月5次重构、Vercel 删80%手工工具反而更好）
- **模型无关性**：Harness 不绑定特定模型，新模型出来能快速替换
- **将 Harness 视为数据集**：每次 Agent 失败、漂移、异常都是珍贵的训练素材，用于迭代优化

4.3.1 参考资料

关于Harness概念，2.1 2.2 2.3. 我主要是记录了具体知识要点，其实想要理解上面内容，网上视频很多，大多都是按照2.1 2.2 2.3 这个思路讲的，大家只要看其中之一就能理解，我给大家推荐一个。

<readonly-block href="https://open.douyin.com/player/video?vid=7636365606182325544&mode=pc" type="iframe"></readonly-block>

4.4 Harness 思想的运用

本期视频来源：Raj 的 "Engineering Coding Harnesses Deep Dive" 视频精读。我觉得这个视频是一个很好的视频，因为他足够细节，讲解了Harness在日常工作的应用，并且这些结论他做了实验，看了论文来支撑它。这个视频让我们更好的理解如何在日常使用Harness工程的思想，对应的一些最佳实践。我看完觉得这是非常好的视频，**对面试，实战，做项目都有极大帮助！**

所以我也是反复看了几遍，总结出来了要点，也录制了一个讲解视频。因为原视频是英文，所以如果觉得看这不方便，结合笔记和我录制的讲解视频看就行，不用去看原视频了。

这一节从四个方面： **Retrieval / Context & Memory / Loops & Tools / Orchestration** 来讲解了Harness工程在我们使用AI，或者说构造AIAgent应用过程中的具体做法。这里面每一个部分，给出的建议都特别具体。这一小节，结合了大量面试问题讲解，消化好这一部分对于面试非常有帮助。

4.4.1 Retrieval（检索）

4.4.1.1 背景 / 基础知识

我们需要根据自己的任务场景，来提供给 AI 检索的工具。虽然 Claude Code 里其实内置了 Grep 工具，但 Claude Code 是通用的，而我们的任务是定制的。这是一个实用的建议——不管你是使用 AI 还是构建 AI，都可以想一下：什么样的检索工具对我的场景是合适的？

举个例子：如果你的项目是一个超大型项目（比如 100GB 代码库），直接让 Claude Code 用 grep 线性扫盘会很慢；这时候你可能要给它建立一个 BM25 倒排索引，再以 Skill 的形式装上去让 CC 调用这个搜索工具。

检索工具从"词法"到"语义"是一条谱系，不是非此即彼：

视频里 Raj 跑的实验是"**Agentic + 纯 BM25**"反超了"**One-shot + embedding 切块**"，但这是"embedding 从必选变可选"，不是"BM25 取代 embedding"。

> 注：表中 ">5GB / >10w" 等具体阈值是工程经验值，源视频只笼统说 "as you get more and more files"。下方"最佳实践建议"沿用同一组阈值。

4.4.1.2 最佳实践建议

\- **默认 grep 起手**：90% 的代码搜索靠 grep 就够，不要一上来就上向量库。

\- **代码库 > 5GB 或文件数 > 10w，加 BM25**：典型做法是包一个 SKILL（封装 \`ripgrep --index\` 或开源 BM25 库），让模型像调工具一样用。

\- **跨语言 / 跨同义词的查找才用 embedding**：比如查"用户鉴权相关代码"，关键词散在 \`auth/login/session/jwt\`——这种 case 上 semantic search。日常代码定位别用。

\- **配好模型（Opus/GPT-5 级别）就上 Agentic RAG**：让模型自己重写 query、按文件级别迭代，比预先 chunk 切片更准；前提是你愿意多花 2-5x token。底层用 BM25 / embedding / 混合都可以。

\- **文件级 > chunk 级**：能不切块就不切块，整文件交给模型，避免上下文撕裂。

\- **什么时候迁数据库**：当你需要"按 author 过滤 + 按 commit 时间排序 + 按路径匹配"这种 **metadata join** 时，才考虑结构化索引/数据库；纯文本搜索别上 DB。

4.4.1.3 面试问题

Q1：Harness 工程对你的工程实践、使用模型有什么启发？

最大的启发是：**不要把通用 AI 工具当万能工具，而要根据业务场景给它配检索工具**。Claude Code 内置的 grep 在通用场景下够用，但任何严肃业务都是定制的——代码库可能有 100GB，文档可能跨多个仓库，查询可能带 metadata 过滤。这时通用工具会很慢甚至搜不到。

落到具体做法是三步：1) 先看任务量级和查询形态——文件多大、查询是纯文本还是带字段过滤；2) 按谱系选工具——grep / BM25 / embedding / 结构化 DB，能用轻的就不上重的；3) 用 Skill 把这个定制工具封装好让 agent 调用。这其实是 harness 工程的核心——agent 的能力上限不只取决于模型，更取决于你给它配的工具链。

Q2：如何看待 RAG 的发展趋势？会不会被取代？

我的判断是**至少目前没有被取代，但它在 stack 里的位置变了**。

首先，现在的趋势确实不再是"什么场景都先上 RAG"：1) 一般中小项目，**grep / BM25 + 模型自身能力**就能解决大部分检索问题，Claude Code 默认就是这套；2) 即使在高级场景，有实验表明 **Agentic RAG（哪怕底层只用 BM25）也能反超传统切块 + embedding RAG**——这说明随着模型变强，"语义近似"这层不一定非要靠 embedding 来做，模型自己重写 query 就行。

但这不等于 RAG 被取代：1) **范式上**，Agentic RAG 本身就是 RAG，只是把"一次检索"换成"迭代检索"；2) **工具上**，embedding 在跨语言、跨同义词、海量文档场景仍是性价比最高的选择；3) **架构上**，企业级场景往往需要 metadata join，这又得叠结构化数据库 + RAG 的混合方案。

所以更准确的说法是：RAG 没被取代，而是**从"默认必选"降级成"按需启用的一档"**。判断要不要用，看三个条件：模型强度、文档规模、查询是否需要跨同义词或跨字段。

4.4.2 Context & Memory（上下文与记忆）

4.4.2.1 背景

虽然主流模型号称 1M token 上下文，但实测**填越多越退化**——通常超过一半就开始丢信息。所以"有 1M 窗口"≠"能用 1M 窗口"，真正能可靠使用的往往只有前 30–50%。这就引出一个问题：信息越来越多（长对话、错误日志、跨会话的项目知识），但能放进窗口的就这么多，**该把什么放进来、什么放出去、什么留到下次**？

视频博主把这件事拆成三层记忆来处理：**Active Context Window、Working State、Durable Memory**。下面就只讲清楚这三层各自负责什么。

> 关于记忆的分类有很多说法（短期/长期、工作记忆/情景记忆等），笔记里讲开源架构（CC、OpenHands、Claude Code）

时也各有各的叫法。本质都是一回事，只是切分粒度不同。本节更重要的还是使用这些记忆背后的最佳实践和蕴含的思想。

4.4.2.2 基础知识：三层记忆分别干什么

Layer 1：Active Context Window（当前上下文）

就是这一轮 API 调用真正发给模型的那段 token。它的职责是**管好"现在"**——决定这次推理时模型眼前能看到什么。因为窗口会退化，所以这一层的核心工作是"瘦身"：用 sliding window 只保留最近几轮、用 compaction 把老对话压成摘要、用 rewind 把走错的分支直接砍掉，避免错误内容继续占着窗口。

Layer 2：Working State（临时工作文件）

> 和跨会话记忆相同的地方都是用外部文件存储。不同的地方在于这个是给当前对话用的，当前对话用完就丢。而长期记忆的文件是跨会话的。这里其实对我们做Agent有一个启示，就是不要什么东西都往上下文塞，我们可以把一些结果保存在磁盘，运行的时候动态查找。因为上下文很宝贵。

任务进行中、但塞不进窗口的中间状态，落到**文件系统**里。它的职责是**管好"这次任务"**——给 agent 一个比 context window 大得多的"草稿纸"。最典型的就是 \`plan.md\` / \`todo.md\`：让 agent 边做边勾选，目标不会在长循环中走丢。再激进一点是 Recursive Language Models 范式：把所有历史都落盘，用 Python REPL 按需检索，相当于"无限上下文"。任务结束这层就可以扔掉。

Layer 3：Durable Memory（跨会话长期记忆）

跨会话留下来的知识，下次开新会话还能用。它的职责是**管好"这个项目/这个 agent"**——把经验沉淀下来不要每次重学。两种主流形态：

\- **AGENTS.md**：项目级事实（构建命令、目录约定、坑），主流 harness 都会自动注入 system prompt。

\- **Skills**：博主把SKILL定义成了新的一种Agent的持久记忆的表现形式。

三层是**自下而上递进**的：窗口装不下 → 落到 working files；任务结束还想留 → 升级成 durable memory。

4.4.2.3 最佳实践建议

按三层记忆分别给出操作建议，**自下而上**对应"管现在 / 管这次任务 / 管这个项目"：

Layer 1：Active Context Window —— 管好"现在"

\- **主动管理上下文**：

1. 主动压缩上下文，不要等满了再压缩，因为模型能力会随着窗口增大而退化。
2. 不要过于依赖模型的compaction，因为很多compaction是黑盒，你不知道它发生了什么。另外主动压缩，指定它保留哪些信息，是好的工程实践（主动使用claude code的/compaction，后面加上参数，指定compaction的时候保留哪些信息。(关于这两个指令，不明白的也可以参考这两个文章：[Claudecode compaction 指令](https://x-sir.blog.csdn.net/article/details/159156069)，[rewind 指令]

```
(https://www.zhihu.com/question/2001668790279245831) )

![](images/WRUFbcgC0oHjNCxQNbVcgGX0nWe.png)



```Plain Text
/compact 保留所有架构决策和待办事项。
```

1. 只保留关键的信息在上下文，避免挤爆上下文。比如长 stack trace、20000 行日志先在 Agent 层截断/抽取关键帧再喂回去。使用Claudecode的rewind功能，直接把走错的分支砍掉，不要让错误信息继续占着窗口。

> 这些原则其实对我们实践也好，做Agent应用也好都有帮助。包括面试官问你使用CC有什么心得？你觉得ClaudeCode比其他coding工具好用在哪里，都是我们面试的素材，我们后面会展示。

Layer 2：Working State —— 管好"这次任务"

- **长任务先写一个 todo markdown**：把计划落盘成 `plan.md` / `todo.md`，让 agent 每完成一步勾选；避免任务在长循环中丢目标。（Langchain的Deep Agent就是这么做的）

- **能落盘就别塞上下文**：中间产物（搜索结果、抓取的网页、生成的代码片段）写文件，让 agent 按需读，不要全堆进窗口。上下文很贵，磁盘很便宜。（博主举了一个例子，让Agent 所有历史落盘，用 Python REPL 按需检索，相当于给 agent 一张"无限大草稿纸" 相比不使用这个技巧，效果更好）

- **任务结束就清理**：working files 是一次性的，任务完成后归档或删除，别让它污染下一次任务。

Layer 3：Durable Memory —— 管好"这个项目 / 这个 agent"

- **AGENTS.md 铁律**：

1. 手写不自动生成；
2. 只写"项目特有的事实"（构建命令、目录约定、坑），不写通用编程知识；
3. Boris(CC的开发者）建议每次大模型升级或每季度复盘一次，删掉过时条目。（也许这个写得有点夸张，不过定期检查Agent.md是一个很好的习惯）
4. Agents.md写废话，反而会误导模型。 具体这里可以参考笔记论文部分。

- **写 Skill 的判定标准**：

1. 同一类操作做过 3 次以上 + 步骤超过 5 步 + 通用模型容易做错 —— 才值得沉淀成 Skill。
2. **Skill 建议配上 eval**：如果你的SKILL写了很多模型已经知道的废话，反而会降低模型性能。所以可以给每个 Skill 准备一个小评测集，定期跑 **with skill vs without skill** 两组对照；模型升级后重跑，发现负贡献立刻下线（旧 Skill 可能被新模型的原生能力 superseded）。

4.4.2.4 面试问题

Q1：请你分享一下使用 Claude Code 的小技巧。
```

我用下来最大的心得是：**要主动管理上下文，别赌模型自己能管好。**这件事的底层原因是——虽然现在主流模型号称 1M token 窗口，但实测填得越满模型越退化，可靠的部分往往只有前 30%\~50%。所以"窗口大"不等于"能用满"，主动瘦身是必须的。

落到 Claude Code 上，我常用两个原生功能：

1. ****主动 `/compact` 而不是等它自己压****：CC 自带的自动 compaction 是黑盒，你不知道它丢了什么。我的做法是在上下文用到 50% 左右就主动触发，而且 `/compact` 后面加参数，明确告诉它"保留所有架构决策和待办事项"——指定要保留什么比让它自己挑要靠谱得多。
2. **走错路用 Rewind（双击 ESC）直接砍掉**：如果模型沿着错误方向跑了几轮，最忌讳的就是"说服它改回来"——因为错误的推理过程已经污染上下文了，再多解释只是叠 buff。Rewind 是直接把那段分支从历史里删掉，相当于"假装没发生过"，比纠错干净得多。

更普适的原则是：长 stack trace、几千行日志这种噪声，不要原样塞回窗口；要么截断要么先落盘再按需读。**上下文是稀缺资源，要像管显存一样管它。**

Q2：Harness 工程给你使用 AI 工具带来什么启示?

> 根据本章学习的内容，总结两个可参考方向的回答。

版本一（记忆管理视角）：我想谈一下关于Agent记忆管理的一些心得，我把Agent的记忆分成三层：1.上下文窗口 2.临时工作文件 3.持久记忆。在使用它们的时候，谈一下我的心得：

1. 上下文窗口非常宝贵，我们需要很小心管理他，同时虽然现在模型的窗口越来越大，最大的上下文可以支持1M，但是当上下文增大，模型性能也会退化，所以我们不必等到上下文满了再压缩，可以在上下文50%的时候，主动压缩。
2. 临时工作文件这一层，我们让AI存一些当前任务的中间产物，比如搜索结果，长日志。我们不把他放在上下文中，而是放在磁盘里，因为磁盘很便宜。通过Agent搜索磁盘文件，可以让Agent处理很久之前的任务记录，同时也不污染上下文。
3. 持久记忆常见的有Agent.md和Skill.我们应该手动写Agent.md，不要依赖AI生成。同时对于SKILL，如果同一操作重复超过3次以上，我们应该建立SKILL，但也应该建立测试集评估SKILL，因为模型升级，或者SKILL过啰嗦，也会影响Agent的性能，所以定期检查SKILL是一个好习惯。

版本二（AGENTS.md 管理视角）：最大启示是**项目级知识要外化，但要克制。**以前我习惯把所有约定写在 README 里靠人脑记，现在我会用 `AGENTS.md` 把"项目特有的事实"喂给 agent——构建命令、目录约定、踩过的坑。

但管理 `AGENTS.md` 有几条铁律我是踩坑学到的，最近也有一篇论文 *Evaluating AGENTS.md — Are Repository-Level Context Files Helpful for Coding Agents?*（arxiv 2602.11988）做了实证支撑：作者在 AGENTBENCH 上对比"无 context file / LLM 自动生成 / 人类手写"三档设置，发现 **LLM 自动生成的 AGENTS.md 反而让任务成功率下降 \~3%、token 消耗上升 \~20%**，人工手写也只有 +4% 的提升。原

因是 context file 里每一条额外要求都是一个约束，Agent 会认真履行但同时也分散了注意力。所以我的做法是：

1. **手写不自动生成**——自动生成的 `AGENTS.md` 充斥废话，论文实验里直接成了负贡献；
2. **只写项目特有的事实**——构建命令、测试命令、特定工具（如"用 uv 不用 pip"）这类 AI 猜不到的；目录结构、"保持代码整洁"这类废话不要写，AI 自己知道；
3. **定期复盘删过时条目**——模型升级后，老的"提醒"可能反而成为干扰；
4. **简洁第一，少即是多**——`AGENTS.md` 写得越长，注入 system prompt 的开销越大，而且容易稀释关键信息。

这件事让我意识到：harness 不是越多上下文越好，而是**信号-噪声比**的工程。

Q3：使用 Skill 有什么心得吗

我的核心心得是：**Skill 不是越多越好，滥用会反向降低性能。**

直觉上 Skill 是好东西——把一类可复用流程外化成 prompt + 代码 + 参考资料，让 agent 像调工具一样用。Cursor 就有过用 200 行 Skill 替掉 15000 行编排代码的例子。但实际用下来我发现两个坑：

1. **Skill 写多了反而干扰模型**：如果 Skill 里写了很多模型本来就知道的常识，等于在 system prompt 里加噪声，**结果是 with skill 比 without skill 还差。**
2. **Skill 会过时**：新模型原生支持多模态文件解析后，老的"教它怎么解析 PDF"的 Skill 就成了负担。

所以我建立了两条纪律：

- **判定标准**：同一类操作做过 3 次以上 + 步骤超过 5 步 + 通用模型容易做错——同时满足这三个条件才值得做成 Skill。一次性的别做。

- **每个 Skill 必须配 eval**：准备一个小评测集，定期跑 **with skill vs without skill** 对照。模型升级后重跑，发现负贡献立刻下线。

总结来说：**Skill 是杠杆，但它也有成本；要用 eval 来证明它真的在帮你，而不是凭感觉攒一堆。**

4.4.3 Loops, Tool Use, and Feedback（循环、工具与反馈）

4.4.3.1 背景

Agent 之所以比单次 LLM 调用强，核心是**循环 + 反馈**——给它时间思考、跑工具、看结果、再调整。从 模型o1 开始，模型就不再是"一锤子买卖"，而是默认要在 harness 里循环跑、反复 refine。

但"循环跑起来"只是第一步。真正决定循环跑得好不好的，是**循环周围的那一圈配套**：

- 循环本身要够聪明（不能每次啥也没学会，就盲目重试）；

- 循环里跑的工具不能反过来污染循环（一条 20000 行的日志就能撑爆窗口，后面循环全废，参考杠杆 2）；
- 循环不能放任 agent 写出越来越大的文件、越来越乱的结构；
- 循环出错时不能炸掉真机（API key、`~/.ssh` 直接暴露）。

所以这一节真正讲的是一件事的四个层面：**让循环本身更聪明 + 让工具 I/O 不污染循环 + 给循环加结构性约束 + 给循环加安全边界。**下面就按这四块展开。

> SWE-bench 论文留下的三条设计原则贯穿全节：actions 简单紧凑、environmental feedback 信息量足且简洁、guardrails 抑制错误扩散且便于恢复。

4.4.3.2 基础知识

1. 循环范式（Loop Strategies）—— 让循环本身更聪明

循环不是一种东西，而是一条从"暴力重试"到"假设-验证"的谱系，越靠后越省 token、越快收敛：

1. **Ralph Wiggum loop**（最朴素）：失败就清空重来、什么也不学 → 极度耗 token，但能 work。视频里举了用这种方式硬怼出一个 C compiler 的例子。（但这种方式很蠢啊，感觉也会陷入死循环，感觉谁会用这种方式写Agent啊。。）
2. **Plan-then-Execute**：先规划再执行，中途守住计划。大多数现代 harness 默认就这么做。
3. **Hypothesis → Verification loop**（Karpathy 的 auto-research 范式）：假设 → 跑实验 → 看结果 → 反馈进下一轮 → 螺旋上升。这是当前性价比最高的循环形态。
4. **Test-Driven Development for Agents**：把测试当作循环的成功判据。Factory 的提醒——**务必先写测试再写代码**，反过来会把已有 bug 也"橡皮图章"进测试。

2. 工具 I/O 卫生（Tool Hygiene）—— 别让工具污染循环

循环每跑一轮，工具的输出都会回灌进上下文。如果不管 I/O，工具一次吐 20000 行日志，几轮下来窗口就废了（参考杠杆 2 的"窗口退化"）。所以 harness 必须替 agent 做这件事：

- **截断**：harness 自动只保留尾部 2000 行而不是整段 20000 行。看似简单，但能直接救活后续 N 轮循环。

- **抽帧**：长 stack trace 只保留报错那几帧 + 失败的测试名，而不是整段 traceback。

- **落盘**：超大输出（>5MB）直接写文件，agent 按需读，不要全堆进窗口。

3. 结构性约束（Structural Constraints）—— 替循环挡掉错误方向

这部分是 OpenAI 反复强调的点："开发者正在变成 AI 系统的管理者"——写代码已经不是瓶颈，**给系统设边界**才是。具体做法是把约束写死进 instructions / system prompt，让 agent 在循环里被动遵守，比如可以规定以下规则：

- 单文件不超过 200 行 → 强迫 agent 分解，而不是堆一个巨大的 god file。
- 函数 / 模块大小限制 → 同理。
- 新增依赖必须先问 → 防止循环里悄悄引入一堆库。

这些约束的本质是：**用 harness 替你做"代码 review"的硬性那一半**，让模型不会沿着错误方向越跑越远。

4. 安全护栏（Safety & Guardrails）—— 让循环出错不致命

视频里 Raj 反复警告：**不要在 laptop 上直接跑陌生 agent**。宿主机上有 API key、`\~/.ssh`、浏览器 cookie，循环一旦失控代价巨大。所以 harness 这一层要提供两道防线：

- **隔离（Sandbox）**：容器 / VM / devcontainer，给 agent 一个干净的、可丢弃的环境。

- **审批关卡（Guardrails）**：三档放行

- ✅ 自动允许：读文件、跑测试、`git diff`
- ⚠️ 需要确认：写文件、安装依赖
- 🛑 强制人工：`rm -rf`、`git push --force`、删分支、改共享配置

4.4.3.3 最佳实践建议

按上面四个层面对应组织：

1. 循环范式

- **能用"假设-验证"loop 就别用 Ralph Wiggum**：明确让模型先说"我猜问题在 X，我要跑 Y 来验证"，再跑——比"失败重试 10 次"省 5–10x token。

- **任务 ≥ 3 步先要计划**：复杂任务用"先 plan、再 execute、每步对照 plan"；简单一步任务不要强加 plan，反而拖慢。

- **TDD 严格"先测试，后代码"**：让 agent 先写测试并跑红，再写实现到绿；反过来会让测试为已有 bug 背书。

2. 工具 I/O 卫生

- **所有工具输出加截断**：默认保留尾部 2000 行 / 50KB；长 stack trace 只留报错块；命令输出 > 5MB 强制落盘成文件再让 agent 按需读。

- **错误反馈要"短而准"**：与其把整段 traceback 喂回去，不如让 harness 抽取 error type + 出错文件:行号 + 最近一帧 + 失败的测试名。

3. 结构性约束

- **写死结构约束进 system prompt**：例如"单文件 ≤ 200 行""函数 ≤ 50 行""新增依赖必须先问"——强制 agent 分解和确认。

- 这条对应 OpenAI 那句"developer → manager"——你不再亲自写每一行，而是给系统定规则。

4. 安全护栏

- **永远在 sandbox 跑陌生 agent**：容器 / VM / devcontainer 三选一，宿主机上的 `\~/.aws`、`\~/.ssh`、浏览器 cookie 别暴露给 agent。

- **三档权限白名单**：✅ 读文件 / 跑测试 / git diff → 自动；⚠️ 写文件 / 安装依赖 → 确认；🛑 `rm -rf` / `git push --force` / 删分支 / 改共享配置 → 必须人工。

4.4.3.4 面试问题

Q：你能介绍一下你了解的 Harness 工程的最佳实践吗?

Harness 工程这个概念其实很大——业界有一种说法是 **Agent = Model + Harness**，所以围绕模型展开的一切（检索、记忆、循环、编排、安全……）都可以归到 harness 里。**我从其中一个我体感最深的角度切入：Agent 的循环**来讲解，这是 agent 区别于单次 LLM 调用的核心——给它时间反复跑工具、看反馈、再调整。

围绕"让这个循环跑得好"，我会从四个层面讲最佳实践：

第一，循环本身要够聪明。

循环范式有一条从弱到强的谱系：最朴素的是 Ralph Wiggum loop——失败就清空重来、什么也不学，能 work 但极耗 token；好一点的是 Plan-then-Execute，先规划再执行；当前性价比最高的是 **Hypothesis → Verification loop**，让模型先说"我猜问题在 X，我跑 Y 来验证"，再循环——比盲目重试省 5–10 倍 token。再进一步是 **TDD for Agents**：先写测试再写代码，让测试当作循环的成功判据；反过来会把已有 bug 也"橡皮图章"进测试，这是 Factory 团队的踩坑经验。

第二，工具 I/O 不能反过来污染循环。

循环每跑一轮，工具输出都会回灌进上下文。一条 20000 行的日志就能撑爆窗口，后面循环全废。所以 harness 必须替 agent 做三件事：**截断**（默认只保留尾部 2000 行）、**抽帧**（长 stack trace 只留报错那几帧 + 失败的测试名）、**落盘**（>5MB 的输出直接写文件让 agent 按需读）。

第三，给循环加结构性约束。

这是 OpenAI 反复强调的"developer → manager"思想——写代码不是瓶颈，**给系统定规则**才是。具体做法是把约束写死进 system prompt，比如"单文件 ≤ 200 行""函数 ≤ 50 行""新增依赖必须先问"。本质是用 harness 替你做了代码 review 里硬性的那一半，让模型不会沿着错误方向越跑越远。

第四，给循环加安全护栏。

循环一旦失控代价巨大——宿主机上有 API key、`\~/.ssh`、浏览器 cookie。所以两道防线必须配上：**Sandbox 隔离**（容器 / VM / devcontainer 三选一），以及**三档审批白名单**—— 读文件 / 跑测试自动放行， 写文件 / 装依赖要确认， `rm -rf` / `git push --force` 强制人工。

总结：这四层背后其实是 SWE-bench 论文留下的同一条原则——*actions 简单紧凑、environmental feedback 信息量足且简洁、guardrails 抑制错误扩散*。Harness 工程的精髓不是模型有多强，而是**你给循环周围配的这一圈基础设施有多稳。**

4.4.4 Orchestration（单 Agent vs 多 Agent）

> 其实这一章的内容和我们笔记里的多Agent部分非常像，这个博主讲的时候，引用的参考资料的论文都和我在笔记里推荐的论文一模一样。总体而言，这个博主说的内容和我们笔记里这个章节是一样的，如何选单Agent,多Agent。值得一提的是，这个博主直接把parallel模式否掉了，他说这不是一种很好的模型，因为你极难管理不同Agent之间的通信以及协调税。虽然作为知识的总结部分，多Agent架构里肯定是有parallel这种模式的。另外笔记里解析了三个开源架构，CC,Openclaw,Hermes。他们确实也都用的是orchestrator模式，由此最少也可以看出来parallel模式很难驾驭，要慎用。
>
> 另外你会不会有疑问，单Agent/多Agent这个内容算什么Harness？但是按现在业界的定义，Harness = Agent - Model，所以这样想你应该不会觉得奇怪了吧。

4.4.4.1 背景 / 基础知识

直觉是："上下文窗口不够大，那就拆给多个 agent 并行做。"听起来很对。但视频里 Raj 给的**反直觉真相**是：多 Agent 不是默认更好——多 benchmark 平均下来，**Single Agent 表现优于 Multi-Agent。**

原因是**协调税（coordination tax）**：agent 之间要互相同步状态、互相理解输出、互相处理冲突，这一层开销往往把并行收益吃光。社交媒体上常见的"我同时开 100 个 agent 改 100 个 bug"在真实工程里几乎一定塌掉。

多 Agent 真正有效的场景是结构化的：

- **Orchestrator-Worker**：一个主 agent 派任务、若干 worker 干活、最后汇总。

- **明确可并行的子任务**：比如对 50 个独立文件做格式化，子任务之间零依赖。

- **Reflection / Critic**：一个 agent 写代码、另一个 agent 专门挑毛病。视频里 Raj 自己的工作流——**Opus 写代码 + GPT 系列反思批评**——就是这个模式；OpenHands 等 harness 已经内建。

Factory 的实战结论也印证了这点：**串行 Orchestrator → Worker → Validator** 比"并行 swarm 一百个 agent"靠谱得多。

4.4.4.2 最佳实践建议

- **默认就用 Single Agent**：除非你能说清"我为什么需要并行"或"我需要一个独立 critic"，否则不要拆。

- **要并行先验证"子任务零依赖"**：能不能用一句 `for f in files: agent(f)` 描述清楚？不能就意味着有隐式依赖，并行会出乱子。

- **Orchestrator-Worker 模式的硬要求**：worker 之间不能直接通信，只通过 orchestrator 中转；每个 worker 的输入输出必须是可序列化的明确契约（JSON / 文件路径），不要靠"共享内存"。

- **强烈推荐 Reflection 模式**：主写 + 副审 是 ROI 最高的多 agent 用法；建议主副用**不同厂家的模型**（如 Opus 写 + GPT 审），让盲区互补。

- **限制 agent 数量**：一个 orchestration 内活跃 agent ≤ 5；超过这个数协调成本会指数上升。

- **不要让多 agent 完全 event-driven**：自由互相调用 = 死锁/无限循环高发；坚持**有明确层级**的调度。

- **Reflection 的输出要可执行**：让 critic 直接产出 diff 或 patch，而不是写"建议你考虑改进 X"——后者 main agent 不一定听得懂。

4.4.4.3 面试问题

Q：你会如何选择单 Agent 和多 Agent 的架构?

我的总原则是 **"默认 Single Agent，除非你能说清为什么必须多 Agent"**。这听起来反直觉，但多个主流 benchmark 上平均下来，**Single Agent 普遍优于 Multi-Agent**——原因是**协调税**：agent 之间同步状态、互相理解、处理冲突的开销，往往把并行收益吃光。

多 Agent 真正有效的就三类场景：

1. **并行子任务**：必须"子任务零依赖"（能用 `for f in files: agent(f)` 描述清楚），比如 50 个独立文件做格式化；
2. **Orchestrator-Worker**：worker 之间不能直接通信，只通过 orchestrator 中转，输入输出走可序列化的明确契约；
3. **Reflection / Critic**：ROI 最高的多 agent 用法，主写 + 副审，建议主副用不同厂家的模型让盲区互补，critic 输出要可执行（直接给 diff/patch）。

外加两条防失控的硬规则：活跃 agent ≤ 5；不要完全 event-driven，坚持有明确层级的调度。

一句话总结：**Single Agent + Reflection** 是当前性价比最高的组合，其他多 Agent 形态都要有明确理由再上。

4.4.5 面试真题

上面其实展示了很多面试问题，就是想告诉大家，这一节的内容真的很实用。我在这里引用字节暑期实习真题，来告诉大家，确实你看了这个笔记，对你面试有直观的帮助。这个面试真题来自于：2026/05字节跳动大模型开发暑期实习一面

下面这三个问题，都是我根据上面2.4的知识部分手写的。只是做一个参考。更多的其实想告诉大家，你自己理解了上述的内容，很多问题就游刃有余，可以从各个方面和面试官去讲。而且也可以深入和他探讨。重点不在于背答案，而在于理解和灵活运用。

8-4 如果让你用 harness 完成你的工程，你会怎么做?

Harness这个概念其实很大，按照现在的定义，Agent除了模型以外的部分，都叫做Harness。假设我运用的是ClaudeCode来完成项目，我分享一下我是用CC完成项目的一些Harness思想。

1. **从AI调用检索工具的角度。**ClaudeCode默认使用Grep搜索，**但是我会根据具体的任务和观测的效果，来修改模型使用的工具**，比如文件太大，那我会使用BM25建立索引。如果搜索精度低，我会使用Agentic Rag或者embedding搜索提升模型的外部获取信息能力。
2. **我会建立明确的验证机制。**Agent本身去验证代码是否完成，它是主观的。我会明确测试机制，建立测试标准，指标，让Agent跑这些工具来完成代码阶段性的检验，并且反馈检测结果，创建一个生成 - 检测的

循环，保证生成代码的质量。

3. **让Agent只做一件事，并且保持上下文简短。**虽然现在模型的上下文很大，但是当模型上下文窗口变大，其实性能也会下降。我会参考OpenAI Harness工程的做法，让多个Agent来完成任务，每个Agent职责，目标保持简短，只做一件事，做完就销毁，保证Agent运行任务时处于最良好的状态。
   这里还有太多其他的角度可以回答了，都是上面2.4总结的内容，比如Agent.md最佳组织方式。Agent架构编排（使用主管模式，反思模式等。安全方面，比如建立隔离环境，sandbox）。这里就不展开了，都是上面重复的内容。

8-5 对 Claude Code 的了解？里面有哪些框架你认为真正给你的使用带来了便利？

CC的了解可以从多个方面，比如从它的记忆功能，架构方面，可以看我们的CC代码解析。它的最佳实践方面。总之第四章讲了很多ClaudeCode，各个角度都可以回答。

框架哪些方面带来了便利性。可以结合本章内容，谈/rewind的使用，compact的使用，往主动管理上下文上去聊。也可以聊一下它的insight命令，可以参考这里

8-6 这些内容怎么迁移？

其实2.4小节所有的内容都是在讲Harness工程的应用，也就是Harness工程怎么迁移到我们的实际工作中。这个问题又很宽泛，但是其实上面的各个原则，都是Harness工程迁移的最佳实践，都可以用来回答这个问题。

4.4.6 参考视频

https://www.youtube.com/watch?v=KijChx7q2nY&t=224s

4.5 Anthropic Harness架构详解

Anthropic 的 Harness 是目前行业内最被广泛引用的长时任务管控架构范例。我们来对它进行拆解，来看一下Anthropic公司是如何使用Harness架构来完成一个长时，复杂的Vibecoding项目的。（其实我们这个笔记中的RAG项目，也是以一个文档为起点，完成整个代码的。但是当时我们的方法比较朴素，用一个agent就做到底。学了本章，我们可以试一下，使用Harness 工程，来完成这个项目，看看效果会不会更好）

4.5.1 设计哲学

Anthropic 的 Harness 核心设计理念是：与其让一个天才持续工作直到崩溃，不如让无数个"新手"接力完成，每人只干一件事。传统做法是把一个强大的模型（如 Claude Opus 4.6）扔进去从头干到尾——但无论模型多强，单一上下文窗口最终都会被垃圾数据淹没，造成**上下文腐化**，导致**失忆和幻觉。**

Anthropic 的方案彻底绕开了这个问题：**每个 Agent 只活一次，干完就死。**

4.5.2 总体结构

1. App Spec / PRD：项目起点

Anthropic 的 Harness 通常从一份由人提供的**需求说明开始**。这份需求说明不是直接交给模型去连续编码，而是作为整个系统初始化的输入。它的作用是定义产品目标、功能范围和预期结果，为后续的任务拆解和状态组织提供基础。也就是说，项目最开始进入系统的不是"代码任务"，而是一份高层次的目标描述。

2. Initializer：初始化规划 Agent

Initializer 的职责不是直接产出业务代码，而是把需求文档转化为**后续开发可以执行的工程结构**。它会**读取需求，拆解出特性清单，建立任务边界，定义每个功能点的验证标准，并完成项目骨架和代码仓库的初始化。**这个阶段的意义在于，先把一个模糊的产品目标整理成一个可以持续推进、可以被后续 agent 反复读取和执行的蓝图。

3. Coding Loop：编码循环 Agent

系统进入正式执行阶段后，**并不是由同一个 agent 一直写下去，而是不断启动新的 coding agent，让它们逐轮接力。**每一轮 agent 都会先读取当前的任务状态和项目进度，然后只选择一个明确的小任务去实现。完成之后，它不会长期保留在系统中，而是会在交接完成后退出。下一轮再由一个全新的 agent 接手。Anthropic 正是**通过这种循环式接力，而不是持续式长对话，来避免上下文不断膨胀的问题。**

4. External Memory：外部记忆系统

Anthropic Harness 的一个核心思想，是将项目状态从模型内部记忆中剥离出来，放入外部可持久化的介质中，比如特性清单、进度文件和 Git 历史。这样一来，系统的连续性不依赖某个 agent 的上下文窗口，而依赖外部状态记录。每个新的 agent 启动时，只需要重新读取这些状态，就能接手上一个 agent 已经完成的工作，从而让整个系统具备持续推进的能力。

5. Deterministic Validation：确定性验证轨道

Anthropic 的架构**并不把“是否完成”这件事交给模型自己判断，而是通过一套外部的、确定性的验证机制来裁定结果是否成立。**也就是说，agent 生成代码之后，系统会通过测试、检查和验证来决定这项工作能否被接受。只有满足这些外部标准，系统才会更新状态并进入下一轮。这种设计的意义在于，用工程规则替代模型主观判断，把“感觉完成了”变成“验证通过了”。

6. Sandbox / Isolation：隔离执行环境

每个 agent 的运行通常都处于**一个受控环境中**，**这样既能限制它可访问的工具范围**，**也能避免错误或脏上下文污染整个系统。**隔离环境让每一轮执行都尽量在明确边界内进行，也让失败后的回滚、重试和重新启动变得可控。Anthropic Harness 依赖这种隔离性，把 agent 的自由度压缩在一个足够安全、足够清晰的工作空间里。

总结来说：Anthropic 公司的 Harness 架构核心，可以概括为一种“**初始化拆解 + 外部状态持久化 + 单任务循环执行 + 确定性验证门控”的长时任务运行体系。** 它的关键不在于让单个大模型持续工作，而在于先由 Initializer 将需求拆解为可执行的特性清单，再让一个个全新的 Coding Agent 逐轮读取状态文件与代码库，只完成一个原子任务，并通过测试、提交与交接文件更新系统状态后立即退出。这样做的本质，是把**项目记忆从模型内部迁移到外部系统**，**把任务完成判断从模型主观输出转移到确定性验证机制，从而显著降低上下文腐化、幻觉完成和多步任务失控的风险。**

4.5.3 局限性

相比于单个Agent直接完成整个项目， Anthropic 公司也承认整个工程的局限性（但我觉得Harness Engineering还处于早期发展阶段(2026年3月写），Anthropic 公司给大家打了个样，后续大家会继续在此基础上改进和发展，仍然有非常大的借鉴作用）

Initializer 是单点高杠杆环节

如果初始拆分错了，后面全部都会跟着错。也就是说，蓝图质量决定了整个系统上限。

Handoff 质量并不稳定

进度文件有时候总结得好，有时候会漏掉关键问题。一旦漏掉“为什么失败、怎么修复”，后面的 agent 可能反复踩同一个坑。

多步可靠性仍然会衰减

即使单步成功率很高，步骤一多，整体可靠性还是会下降。所以它不是彻底解决了可靠性问题，而是显著缓解。

仍然需要 Human in the Loop

从实战角度看，Anthropic Harness 更像：高度自动化，但不是完全放手，还是在关键节点引入人工检查。

4.5.4 参考资料

https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

4.6 ClawCode Harness工程详解

> Claude Code泄漏发生后，韩国开发者 Sigrid Jin 发起了 **Claw Code** 项目——用 AI 编排工具指挥多个 coding agent 并行工作，在 **3 天内**将 Claude Code 的核心功能从 TypeScript 重写为 Rust（48,599 行），产生了 292 个 commit。整个过程中人类几乎不写代码，只负责方向决策和任务分解。
>
> 这个项目破了Github涨星最快的记录，除了乘上了CC的热度外，更在于它完整展示了一套 使用Harness工程进行Vibe Coding的技巧 —— 如何用 AI 工厂模式完成大规模代码迁移同样值得学习。
>
> 这一节带大家快速掌握一下它的方法，值得提的是我没有非常深入地继续学习，而是快速了解了他的思路。如果你想更深入的学习，我也提供了它的代码地址，可以借鉴这个代码仓库，自己做一遍，感受Harness工程（毕竟Harness 工程真的是需要自己去实践设计才能更好掌握的），也可以做一个Coding Agent，是一个不错的学习机会。

4.6.1 整体方法论：五阶段流水线

Claw Code 的核心思路是：**先把"什么需要被实现"变成机器可查的清单，再边实现边用 mock harness 自动验证行为对等性。**

整个流程分五个阶段：

```Plain Text 阶段一：归档 + 快照 → 把原始 TS 代码表面提取为结构化 JSON（建立"真相基线"） 阶段二：Python 镜像工作区 → 1:1 结构占位 + 架构模型（理解原始系统） 阶段三：Parity Tracking → 量化覆盖率，进度可测量、可审计 阶段四：Mock Parity Harness → 确定性端到端测试，行为可验证 阶段五：多 Agent 并行执行 → Discord 指挥 + clawhip/OmX/OmO 协调
```

1\~4阶段是准备。5阶段才是真正的coding。接下来我们逐一讲解。

4.6.2 五个阶段详解

4.6.2.1 归档 + 快照原始代码表面

> 简单来说你可以理解为就是去把整个项目做一个拆解，去将ClaudeCode代码变成可以量化的文字，比如有多少工具，有多少文件，有多少Commands（就是我们讲的在Claude里面输入/models,这种叫做command)。 将大型项目变成快照，这样AI好查找，比如查找对应的某个command对应的代码是啥，做进度统计的时候也好去统计比如完成了哪些实现。这里面提到的json文件，大家可以在我后面提供的项目开源仓库中找到。

第一步不是写代码，而是**把要重写的东西完整记录下来，形成机器可查的"参考清单"**。

Claude Code 的代码组织非常规整——所有命令集中注册在 `src/commands.ts`，所有工具集中注册在 `src/tools.ts`，启动流程定义在 `src/entrypoints/cli.tsx`。每个命令/工具都是一行标准的 `import` 语句：

因为格式统一，用**逐行字符串匹配**就能提取出所有名字和路径——不需要 AI 介入。

具体做了三件事：

1. **获取原始 TypeScript 源码快照。** 将 Claude Code 的原始 TS 源码存放在 `archive/claude_code_ts_snapshot/src`（在 `.gitignore` 里忽略，不公开上传），作为本地参考基准。
2. **compat-harness 自动提取清单。** 仓库中的 `rust/crates/compat-harness` 是一个 Rust crate，专门解析上游 TS 源码：扫描 `commands.ts` 逐行匹配 `import { xxx } from "./commands/..."`，提取出 207 个斜杠命令；扫描 `tools.ts` 提取 184 个工具模块；扫描 `cli.tsx` 搜索关键词提取启动流程的 7 个阶段。这是一个"活的"提取器——当上游 TS 代码更新时可重新运行，自动刷新清单。
3. **生成结构化快照 JSON。** 提取结果固化为四组 JSON 文件，存放在 `src/reference_data/` 目录：

   - `archive_surface_snapshot.json`：原始仓库的整体结构（18 个根文件、35 个子目录、1,902 个 TS 文件）
   - `commands_snapshot.json`：所有命令的 name + source_hint + responsibility（207 条）
   - `tools_snapshot.json`：所有工具模块的 name + source_hint + responsibility（184 条）
   - `subsystems/*.json`（29 个文件）：各子系统的模块数和样本文件名（assistant、bridge、utils 等）

这些清单的用途：**量化 scope**（"一共 207 个命令 + 184 个工具"→ 可估算工作量）、**定位参考代码**（AI 收到工单时知道去哪个文件读原始逻辑）、**发现复杂度分布**（`BashTool` 有 18 个子模块是最复杂的）、**覆盖率审计**（随时对比已实现多少）。局限性：这种字符串匹配**只适用于代码结构规整的项目**，如果用动态注册或插件扫描就行不通。

4.6.2.2 Python 镜像工作区

> 我个人的理解还是在建索引。上面是把Claude Code拆解了，拆解成json，让AI好查找。现在是把这个Claude Code变成一个python的骨架，这个骨架代码没实现，但是AI可以很快看出它的目录，他是如何启动的，整个流程是啥。相当于给了AI一个空壳，虽然没有实现，但是我通过空壳可以快速知道项目结构，启动流程。

不直接从 TS 翻 Rust——直接翻译等于盲写。原始 TS 有 1,902 个文件、51 万行代码，AI agent 拿到工单后如果要自己在里面翻找架构信息，效率极低且容易遗漏。Claw Code 先建了一个 **Python 中间层**——一个只有 67 个文件的轻量"沙盘模型"，让 AI agent 在后续 Rust 重写时可以先查这个中间层快速定位，而不需要大海捞针。

这个 **Python 中间层包含四样东西**：

1:1 结构映射表。 `src/parity_audit.py` 定义了 TS→Python 的完整映射关系，原始的 18 个根文件和 35 个子目录全部有对应的 Python 占位物。这张表的作用是让任何人（或 agent）一眼看清"原始系统有哪些组成部分，每个部分对应到 Python 层的哪个文件"。

占位包（29 个子目录的结构索引）。 29 个子目录（`src/assistant/`、`src/bridge/`、`src/utils/` 等）各有一个 `init.py`，但不实现任何业务逻辑。每个占位包只做一件事——从 JSON 加载该子系统的元数据，导出 `MODULE_COUNT`（有多少模块）和 `SAMPLE_FILES`（包含哪些文件）。当 AI agent 收到"实现 assistant 模块"的工单时，读一下占位包就知道原始子系统有 1 个模块、文件叫 `sessionHistory.ts`，然后直接去对应的 TS 源码中精准阅读，而不需要在 1,902 个文件里盲目搜索。

关键架构模型（5 个 Python 文件复现核心决策逻辑）。 光知道"有哪些模块"不够，还需要知道"系统怎么运转"。Python 层用 5 个文件把原始系统最关键的几条逻辑管线提取出来，编码为可运行的 Python 代码：

- `bootstrap_graph.py`：启动流程的 7 个阶段——prefetch → 环境检查 → CLI 解析 → 命令加载 → 延迟初始化 → 模式路由 → 查询主循环
- `command_graph.py`：把 207 个命令分类为 builtins（185）、plugin-like（20）、skill-like（2）
- `query_engine.py`：复现查询引擎的 turn-loop 行为——轮次限制（max turns）、token 预算执行（max budget）、消息压缩触发（compaction）。这里有真实的状态管理逻辑，不是 stub
- `setup.py`：建模 workspace 环境发现和启动 preflight 流程
- `runtime.py`：建模运行时路由——把用户的 prompt 分词、和所有命令/工具的名字做匹配打分、返回 top-N 结果。这也是真实的匹配算法，给一个 prompt 会返回实际的路由结果

可运行的审核 CLI。 Python 层提供了 `python -m src.main`（40+ 个子命令），供人类和 AI agent 随时验证中间层的自洽性：`summary` 输出工作区摘要，`parity-audit` 对比覆盖率（确保没有遗漏的模块），`route <prompt>` 验证路由逻辑是否和预期一致，`bootstrap <prompt>` 跑通完整的启动→路由→执行→历史记录流程。这个 CLI 同时也是 Rust 重写的**行为基准**：Rust 版的路由结果应该和 Python 版匹配，不匹配就说明实现有偏差。

4.6.2.3 量化重写进度

> 其实就是进度表。我们的rag 项目其实也有进度表，但是没有他这么细致。它的进度表定义了文件覆盖率，目录覆盖率，目标覆盖率等，同时定义了AI如何开发，将整个项目开发分成9个可以并行的模块，这些设计需要你对Claude Code项目本身有一定的理解。里面的Parity.md就是进度表，大家在源码中可以看到。

核心思想：把"重写进度"从感觉变成**机器可检查的量化指标**。具体是通过三样东西实现的：**一个自动审计脚本、两份 PARITY 清单文档、以及一个 9-lane 并行开发模型。**

自动审计脚本。 `src/parity_audit.py` 中的 `run_parity_audit()` 自动扫描当前代码库，计算四个维度的覆盖率：

根文件覆盖率（18 个中实现了多少）、目录覆盖率（35 个中覆盖了多少）、命令覆盖率（207 个中实现了多少）、工具覆盖率（184 个中实现了多少），并列出所有缺失的目标。测试中有断言确保覆盖率只升不降——写了一个新模块覆盖率上去了，后面不允许退化回来。

PARITY.md 双层清单。 项目维护两份 PARITY 文档，分别追踪不同粒度的进度：

- **顶层 `PARITY.md`**：记录 9 条开发 lane 的宏观状态——每条 lane 的 feature commit hash、merge commit hash、diff 行数统计，做到全程可追溯
- **`rust/PARITY.md`**：更细粒度，对 40 个工具逐个标注实现深度，分四档：strong parity（行为完全对齐）、good parity（核心路径对齐）、moderate parity（基本可用）、stub（仅占位）

9-lane 并行开发模型。 人类根据前两个阶段产出的清单，把重写工作切分为 9 条相互独立的 lane，每条一个 feature branch，可以由不同的 AI agent 并行推进，互不阻塞：

- **Lane 1 — Bash validation**：9 个验证子模块（+1,005 行）
- **Lane 2 — CI fix**：sandbox 探测修复（+22/-1 行）
- **Lane 3 — File-tool**：边界检查——二进制检测、大小限制、路径穿越防护（+195/-1 行）
- **Lane 4 — TaskRegistry**：内存任务生命周期管理（+336 行）
- **Lane 5 — Task wiring**：6 个 task 工具接入 registry（+79/-35 行）
- **Lane 6 — Team+Cron**：团队和定时任务注册表（+441/-37 行）
- **Lane 7 — MCP lifecycle**：MCP 服务器连接、资源、工具分发桥接（+491/-24 行）
- **Lane 8 — LSP client**：LSP 诊断、hover、定义、引用、补全（+461/-9 行）
- **Lane 9 — Permission enforcement**：工具门禁、文件写入边界、bash 只读检查（+357 行）

每条 lane 独立 merge，互不阻塞。PARITY.md 中记录了每条 lane 的 feature commit hash 和 merge commit hash，做到**全程可追溯**。

4.6.2.4 确定性行为验证

> 虽然不同的厂商的Harness工程其实是有区别的，不过对于这一点，是统一的。我们在上面讲Harness工程的原则以及Anthropic公司的Harness的例子都涉及了。就是你不要让AI来判断是否完成，而是给出量化标准，脚本。所以这里需要设计一些让AI执行，评估的脚本。AI通过这些标准和脚本来判断是否完成，记录到项目进度（2.5.2.3讲的parity.md）中。

重写不能靠"编译通过就算对"，但也不能每次测试都调真实 Anthropic API（贵、慢、结果不确定）。解法是建一套**确定性的端到端测试体系**，不调真实 API 也能验证 Rust 重写的行为是否和原始系统一致。这套体系由三样东西组成：一个 mock 服务、一组场景脚本、以及一个双向验证工具。

Mock Anthropic 服务。 `rust/crates/mock-anthropic-service` 是一个独立 Rust crate，实现了 Anthropic 兼容的 `/v1/messages` 端点。它不调真实模型，而是返回**预编排的** SSE 流式响应，可以脚本化多轮对话，每次运行结果完全相同。CLI 通过设置 `ANTHROPIC_BASE_URL` 指向这个本地服务，就可以在完全离线、零成本的环境下跑端到端测试。

> 这个场景脚本其实就是一些测试用例，进度文档需要根据这些测试用例来判断是否将某些模块标记为完成。

10 个场景脚本。 在 `mock_parity_scenarios.json` 中定义了 10 个测试场景，覆盖了核心工具链路、权限拒绝、多工具同轮、插件路径等关键行为。每个场景都通过 `parity_refs` 字段指向 PARITY.md 中的具体条目，确保测试和文档

一一对应：

- `streaming_text`（baseline）：纯文本流式响应，无工具调用
- `read_file_roundtrip`（file-tools）：文件读取往返
- `grep_chunk_assembly`（file-tools）：grep 分块 JSON 组装
- `write_file_allowed`（file-tools）：workspace-write 模式下写入成功
- `write_file_denied`（permissions）：read-only 模式下写入被拒绝
- `multi_tool_turn_roundtrip`（multi-tool）：同一 turn 内执行多个工具
- `bash_stdout_roundtrip`（bash）：bash 执行和 stdout 往返
- `bash_permission_prompt_approved`（permissions）：bash 权限提示 → 批准
- `bash_permission_prompt_denied`（permissions）：bash 权限提示 → 拒绝
- `plugin_tool_roundtrip`（plugin）：加载外部插件工具并执行

> 我们做rag项目的时候，就有进度文档和实际代码可能不一致的问题，这里他给出了解决方案。用专门的脚本来判断二者是否发生偏移，发生偏移了及时纠正。

双向验证工具（防止文档和代码漂移）。 `run_mock_parity_diff.py` 做两件事：先遍历每个场景脚本的 `parity_refs`，确认每条引用在 PARITY.md 中**确实存在**（找不到就报错）；再运行 `cargo test` 输出每个场景的通过状态。这意味着 PARITY.md 不只是给人看的文档——它是场景清单的**锚点**，文档改了但测试没更新、或者测试加了场景但文档没跟进，都会被这个工具检测到。

4.6.2.5 多Agent协作执行

> 这一步才是真正的执行。执行也是很有技巧，当时我们的rag项目都是串行的，一个agent。但是人家是并行，多个Agent。甚至专门写了3个工具负责Agent调度，执行，监控。（这三个工具也给出了代码仓库，大家想深入学习可以学习）。Anthropic公司的harness项目也是用的多Agent。大佬们做的Harness工程，还是得用多Agent来做。所以自己做Harness的时候，最好用多Agent，想想他们如何调度，监测，执行的，这样面试时候也会显得更高级。

如 `PHILOSOPHY.md` 所述，这个项目的核心哲学是：**人类提供方向，claws（AI agent）执行劳动**。**前四个阶段建好了清单、中间层、进度条和测试题，第五阶段就是让多个 AI agent 并行干活，人类只在关键节点介入。**

为了实现这一点，项目作者编写了三个专用编排工具（不是通用框架，而是为这套 Vibe Coding 工作流定制的）：

- **OmX**（oh-my-codex）——**任务拆解器**：把一句话指令转化为结构化工单，定义子任务、执行模式和验证标准（仓库地址：https://github.com/Yeachan-Heo/oh-my-codex）
- **clawhip** ——**后台监控员**：监听 git commits、CI 结果、PR 状态、agent 生命周期事件，在 agent 上下文窗口之外独立运行，只推送通知不干扰执行（仓库地址：https://github.com/Yeachan-Heo/clawhip）
- **OmO**（oh-my-openagent）——**角色协调员**：管理 Architect/Executor/Reviewer 之间的分工和分歧解决，确保多 agent 循环收敛而不是无限争吵。（仓库地址：https://github.com/code-yeongyu/oh-my-openagent）

有了这三个工具，加上前四个阶段建好的清单和测试，就能实现"人类发指令、agent 自主干活"的工作流。**完整流程如下：**

```Plain Text
人类发指令 → OmX 拆任务 → OmO 分角色/启动 agent
→ agent 自主循环（写码→测试→修复）
→ clawhip 后台监控推送事件
→ 人类只在失败或完成时介入判断
```

1. **人类输入一句高层指令**（通过 Vibe Coding 工具），例如："实现 Lane 3：给 file tool 加边界检查"
2. **OmX 自动拆解**成子任务（二进制检测、大小限制、路径穿越防护），确定执行模式（并行/串行）和验证标准（cargo test + mock parity 通过）
3. **OmO 分配角色并启动 agent**：Architect 规划方案、Executor 写代码、Reviewer 审查质量。多组 agent 在服务器上并行工作
4. **Agent 自主循环**：写代码 → 跑测试 → 失败 → 读错误 → 改代码 → 再跑测试。Reviewer 不满意就打回重做，OmO 协调分歧直到收敛。**全程不需要人类参与**
5. **clawhip 在后台监控**所有事件（`lane.started`、`lane.commit.created`、`lane.red`、`lane.green`、`lane.pr.opened`、`lane.finished` 等），向人类推送状态通知，但**不打扰正在干活的 agent**——agent 的上下文窗口是宝贵资源，不能被监控噪音占据
6. **人类只在两种情况介入**：agent 自己恢复不了的失败（决定重试还是换方案），或 lane 完成后判断能否 merge

4.6.3 设计原则

这套方法论的核心是四条设计原则：

1. **人定标准，AI 执行。** 所有判断标准由人预先制定——覆盖率数字、parity 标签体系（strong/good/moderate/stub）、mock 场景清单。AI agent 不需要自己判断"够不够好"，只需对照人制定的验收标准逐项打勾。
2. **建中间层，降低 AI 认知成本。** 51 万行 TS 太大，AI 无法每次从头阅读。用 JSON 索引 + Python 骨架建一个轻量参考层，让 agent 查一下就能定位和理解，而不需要大海捞针。
3. **用工具编排多 agent，而非人工协调。** Python 脚本生成"做什么"（清单），OmX/clawhip/OmO 三件套负责"谁来做、怎么分发、怎么验收"，人类不需要坐在终端前逐个分配任务。
4. **人只做方向、分解、判断。** 人基于架构理解把工作拆成 9 条独立 lane，定义 lane 边界和合入标准。292 个 commit 中，人的贡献集中在：重写什么（方向）、怎么拆（分解）、何时 merge（判断）。剩下的全是 agent 的事。

4.6.4 项目地址

> 这个项目本身是个很好的学习Harness技巧的项目，你可以通过这个项目，学习设计Harness，自己通过它的方法，复现，自己也可以实现一个Claude Code的Agent。这是很值得做的事。深入的学习靠大家自己了。我后面想要在我自己做的Agent项目再设计一个Harness工程，我就不会深入研究这个项目了。如果你不想深入学习，最少通过这些笔记，你能更进一步理解Harness项目的设计思想，也能够在面试的时候有更多可说的。

https://github.com/seavee/ClawCode

4.6.5 讲解视频

https://www.bilibili.com/video/BV1CqQcBqEbU/?vd_source=144bec9c3f54e465073138bed788be1b

4.7 面试问题速览

（刚才上面的文档是给大家理解学习用的，整个章节的问题是供大家面试时直接照着回答的）

4.7.1 什么是 Harness Engineering?

Harness Engineering 是围绕 AI 模型构建运行环境、调度机制和管控基础设施的一种工程实践。它的目标不是单纯让模型“更聪明”，而是让 AI Agent 在长时、复杂、多步骤的真实任务中，能够稳定、可靠、可控地持续运行。

如果用一句更通俗的话来说，Harness Engineering 做的事情，就是不再把 AI 当成一个一次性回答问题的黑盒，而是把它放进一个被设计好的工作系统里，让它按规则做事。

它背后的设计哲学也很重要：模型只是原材料，环境才决定结果能不能落地。

再强的模型，如果直接裸跑长任务，也会遇到上下文腐化、失忆、幻觉完成、任务漂移等问题。所以 Harness Engineering 的核心，不是继续往模型上堆能力，而是通过外部记忆、任务拆解、测试门禁、状态交接和人工审核，把 AI 的执行过程工程化。

4.7.2 Harness Engineering 想解决的核心问题是什么?

它主要解决的是 AI 在长时任务里的可靠性问题。

第一个问题是 Context Rot，也就是上下文腐化。模型做长任务时，上下文窗口会不断被历史对话、错误尝试和无关信息填满，最后导致它丧失对原始目标的把握。

第二个问题是 Hallucinated Completion，也就是幻觉完成。模型在复杂任务中迷失后，往往不会老实说“我不会了”，而是会输出一个看起来完成、实际上并不正确的结果。

第三个问题是 Model Drift，也就是模型漂移。多步执行中，模型会逐渐偏离最初目标，最后做出来的东西方向就歪了。

所以我理解 Harness Engineering 的价值，不是让模型在单轮回答里更强，而是让它在几十步、几百步的连续工作流里，依然能沿着正确轨道往前走。

4.7.3 Harness Engineering 和 Prompt Engineering、Context Engineering 的关系是什么?

我会把它理解成一个演进关系，而不是替代关系。

Prompt Engineering 主要解决的是“对 AI 说什么”，也就是如何设计指令，让单次输出更好。

Context Engineering 进一步解决的是“让 AI 知道什么”，也就是在一次 session 或上下文窗口里，给它什么信息、给多少信息、怎么组织信息。

而 Harness Engineering 更进一步，它解决的是“让 AI 在什么环境里做事”。

也就是说，Harness Engineering 并没有取代前两者，而是把它们包进了一个更大的系统里。Harness 内部的每个 session 还是要做 Context Engineering，每一次给 agent 的具体任务描述还是要做 Prompt Engineering。只是现在关注点从“优化一次对话”升级成了“设计一整个运行体系”。

4.7.4 Harness Engineering 的最佳实践是什么?

我会总结成几个比较关键的实践原则。

第一，状态外部化。不要把连续性寄托在模型记不记得，而要把状态写进文件、Git 或数据库，让系统本身有记忆。

第二，任务原子化。不要让一个 agent 一口气做一个大需求，而是拆成足够小、足够明确的原子任务，每次只做一件事。

第三，上下文刷新。很多先进的 harness 架构都会在一个小任务完成后销毁当前 agent，再启动一个全新的 agent 来继续做下一轮。这么做的目的，就是从根本上避免上下文腐化。

第四，验证优先。AI 的输出必须经过测试和规则检查，而不是靠它自我声明“完成了”。这是把模型主观判断变成工程确定性的关键。

第五，按需暴露工具和技能。不要一次给模型上百个工具，而应该通过 skills、toolkits 或 task routing，让它按需加载当前任务相关的工具和说明。

第六，在关键点加入 Human-in-the-loop。尤其是高价值任务或者多步骤流程里，人工审核断点很重要，它不是系统失败的表现，而是提高整体可靠性的工程设计。

第七，保持轻量、模块化、模型无关。因为模型变化很快，如果 Harness 和某个模型耦合太深，很快就会过时。所以好的 Harness 应该能在尽量少改动的情况下替换模型、替换技能、替换验证逻辑。

4.7.5 如果让你用Harness工程来实现一个项目，你怎么设计?

（套用我们上面学的Anthropic公司的方案，也可以在那个问题中通过整个回答介绍Anthropic公司的Harness方案，灵活运用，学了就不浪费）

如果让我来落地一个 Harness，从一个 Initializer + Coding Loop 的最小闭环开始，而不是一开始就做复杂的多 agent 平台。

第一步，**我会先准备一份清晰的 App Spec / PRD**，把产品目标、功能范围和验收标准定义清楚。因为在 Anthropic 的架构里，系统起点不是直接写代码，而是先把需求整理成后续 agent 可以反复执行的任务蓝图。

第二步，**我会实现一个 Initializer Agent**。它的职责不是写业务代码，而是读取需求文档，把大任务拆成一组细粒度、可执行、可验证的 feature，并生成类似 `feature_list.json` 这样的状态文件。同时它还会初始化项目骨架、代码仓库，以及后续循环执行所需的基础环境。这样系统就先有了“任务清单”和“状态来源”。

第三步，**我会实现一个 Coding Loop**。每一轮都启动一个全新的 coding agent，让它只读取当前状态文件、进度记录和代码库信息，只完成一个明确的小任务。任务完成后，不是让 agent 自己宣布成功，而是必须经过测试、lint 或类型检查等确定性验证；只有验证通过，才更新状态文件、写入交接记录并提交代码。然后这个 agent 立即退出，下一轮由新的 agent 接力。

第四步，**我会把 外部记忆 做成系统核心**，而不是把连续性寄托在模型上下文里。也就是说，我会依赖 feature list、progress file、Git log 这些可持久化信息，让每个新 agent 启动时都能快速接手，而不是依赖前一轮 session 的“记忆”。

第五步，**我会在关键节点加入 Human-in-the-loop**。Anthropic 的方法虽然高度自动化，但并不是完全无人值守。所以在高风险功能、关键界面变更或者多轮失败之后，我会设计人工确认断点，确保系统不会沿着错误方向持续推进。

整体上，我不会把重点放在“怎么让一个模型一次做完所有事”，而是把重点放在：**先拆解、再循环执行、靠外部状态保持连续性、靠确定**性验证保证质量。我认为这是一种更可落地、也更适合真实工程环境的 Harness 实现方式。

4.7.6 你看过哪些 Harness 工程的实际案例，可以分享一下吗?

我比较熟悉两个案例，一个是 Anthropic 官方的 Harness 架构，一个是开源社区的 Claw Code 项目。它们一个是从零做新项目的范式，一个是重写已有大型系统的范式，刚好互补。

第一个是 Anthropic 官方的 Harness 架构。 这是一个从**项目文档**出发做新项目的标准流程。**核心思路**是"每个 agent 只活一次，干完就死":

先由 Initializer 读需求文档，拆成 feature list 和验收标准，初始化项目骨架；然后进入 Coding Loop，每轮启动一个全新的 agent，它只读当前状态文件和代码库，完成一个原子任务，通过测试后更新状态文件并退出，下一轮由新 agent 接力。项目记忆全部存在外部文件（feature list、progress file、Git），不依赖任何一个 agent 的上下文窗口。验证也不靠模型自己说"完成了"，而是必须通过测试、lint、类型检查这些确定性手段。

这个架构最核心的 insight 是：用"短命 agent 接力"代替"长命 agent 硬撑"，从根本上避免上下文腐化。

第二个是 Claw Code 项目。 这是 2026 年 3 月 Claude Code 源码泄漏后，开源社区用多个 AI agent 在 3 天内把核心功能从 TypeScript 重写为 Rust 的一个实践。它和 Anthropic 方案的最大区别在于：**它不是从零写新项目，而是重写一个已有的 51 万行系统。**

它的 Initializer 阶段不是读 **项目文档**拆 feature，而是先扫描原始代码，把结构提取为机器可查的 JSON 清单，再建了一个 **Python 中间层**作为 AI agent 的"架构速查手册"，降低 agent 理解已有系统的认知成本。验证方式也不同——因为是"重写"，所以需要验证行为一致性，它建了一个 mock API 服务做确定性端到端测试，还做了文档和测试的双向锚定防止漂移。编排上用了三个定制工具分别负责任务拆解、后台监控和角色协调，人类只负责方向、分解和 merge 判断。

对比来看： 两个案例的底层原则一致——状态外部化、任务原子化、确定性验证、人只做关键决策。但适用场景不同：Anthropic 方案更适合从零开始的新项目，核心 insight 是"agent 用完即销毁"避免上下文腐化；Claw Code 更适合代码迁移和重写场景，核心 insight 是"建中间层降低 agent 认知成本"应对已有代码库的复杂性。

5 Loop Engineering & Graph工程

5.1 Loop Engineering

> Loop工程是2026年6月炒得比较火的概念，有人说是取代Harness工程进入下一个范式的技术。但是其实它并没有什么新东西。如果你在前面一步步学习了Harness这些概念，你甚至会发现很多时候其实你自己已经在使用Loop工程了。而且学起来你也会觉得很好理解，如果你有了前面Harness相关的基础，大概花个20分钟阅读一下本章节，你就能大概理解Loop工程是什么？应对面试官可能会问的几个问题：**什么是Loop工程？它的核心要素有什么？你怎么看待Loop工程？**

5.1.1 背景：它是怎么火起来的

这波热度不是凭空冒出来的，是 **2026 年年中（大约 6 月前后）** 几个分量很重的人先后开了口，短短几周就把这个词推上了风口。我把时间线大致捋一下。

第一步，龙虾之父先点的火。 被圈里叫"龙虾之父"的 Peter Steinberger（OpenClaw 的作者）在 X 上发帖说：别再给 coding agent 写提示词了，你应该去设计"循环"来驱动这些 agent。多个解读视频都把这条帖子认定为"loop engineering 这个词爆火的起点"——"The term loop engineering exploded after Peter Steinberger posted..."。（大概在 **2026 年 5 月底至 6 月初**这个时间窗口）。

第二步，Boris 在播客里给了关键背书。 紧接着 Anthropic Claude Code 的负责人 Boris Cherny 在一期播客里讲了几乎一模一样的话。他的原话大意是：以前我同时跑五到十个 Claude，我的工作是写提示词让它去写代码；现在又往上抽象了一层，我已经不给 Claude 写提示词了，我跑的是一堆 loop，是这些 loop 在 prompt Claude、在决定下一步做什么——**我的工作变成了写 loop。** 时间大概也是 **2026 年 6 月前后**这一波讨论。

第三步，圈内迅速接力。 之后 OpenAI Codex 的负责人 Tibo 转发，LangChain 的 Lance Martin 又**赶在某个新模型发布的次日**写了篇很长的解读（文稿原话："a post that just came out with the release of [新模型] that came out yesterday was from Lance Martin"）。差不多同一时期，`/goal` 这类功能也"在过去几周里相当流行"（Ross Mike 播客原话）。当造 Codex 和造 Claude Code 的人**不约而同**地认同同一个想法时，这事就值得认真看一看了。

所以大佬们相继讨论，让这个词火了起来。很多自媒体宣传是未来是属于Loop Engineering的。把他称为Harness工程的下一代范式。

当然也有人翻白眼：AI 圈是不是又在造词？去年还是**提示词工程**，年初变成给模型套马鞍的 **Harness 工程**，现在又来个**循环工程**，是不是新瓶装旧酒？

这里有个被反复提到的行业演进脉络，几乎成了这波讨论的"标准开场白"：

> **提示词工程 → 上下文工程 → Harness 工程 → 循环工程**

Towards AI 的 CTO Louis Frano 有句话我觉得挺中肯：这几个词说到底是同一件事——尽可能地"操纵"模型，而这只能通过你喂给它的上下文和提示词来实现。所以与其说是新技术，不如说是**人和 AI 打交道的基本**

单位，正在从"一次对话"变成"一个完整回路"。这个转变是结构性的，不是又解锁了什么新功能那种级别的事。

5.1.2 是什么：到底什么是循环工程

别被"工程"这个词唬住，它讲的事其实特别日常。

回想一下你平时让 AI 写代码：你让它写个功能，它写完了，你跑测试，报错了，你把错误信息贴回去，它再改，你再跑……来回几次，直到能用。

这就是一个最原始的 loop——行动、观察、修正、再行动。

关键区别在哪？过去这个循环的每一步都靠你的手在推：你要复制报错、你要追问原因、你要提醒它别改错文件、你要判断什么时候该停。**你就是这个循环里的发动机，你一停，循环就转不下去。**

循环工程做的事，就是把这些反复发生的动作写成规则，交给系统自动跑。

它和 cron 定时任务有什么不一样？

这是 Louis 讲得最透的一点，也是很多人最初的疑问：loop 不就是每小时跑一遍同一个提示词吗？那不就是个 cron job 嘛，比我们岁数都大。

差别在于——**决策者在循环里面。**

- cron 跑的是一段**固定脚本**，第一步、第二步、第三步，永远一样。
- loop 跑的是一个 **agent**：它看当前状态，自己选下一步该干啥，干完检查结果，再决定要不要继续。它会重试、会回滚、会在搞不定的时候停下来。

一句话，cron 是在执行脚本，loop 是在**模仿一个工程师。**而这之所以现在才行得通，是因为模型终于强到能理解"目标"和"奖励信号"了。

一个 loop 至少要回答五个问题

把上面那套"工作制度"展开，本质上 loop 要替你定清楚这几件事：

1. AI 什么时候开始干活？（触发器）
2. 它能调用哪些工具？
3. 它怎么知道自己做错了？（可验证的目标）
4. 每一轮的结果记在哪里？
5. 什么时候必须停下来交给人？

Louis 把前两条浓缩成 loop 跑起来的两个最低条件：**一个触发器（trigger）和一个可验证的目标（verifiable goal）**。触发器可以是一个 PR 被打开、一次 CI 失败、一条 Slack 消息，或者你手动敲一句。可验证的目标可以硬性如"所有测试通过、CI 变绿"，也可以软一点，比如让一个评审模型判断 UI 是否符合规格。但**一定得有某种检查**，否则你建的不是 loop，而是"一台非常自信的烧钱炉"。

常见的、已经写好的 loop

好消息是，你不用从零造。现在主流工具里已经内置了一些开箱即用的 loop：

- `/goal`（Claude Code、Codex 都有）：一个长期运行的任务，思路很像当年的 Ralph Wiggum——不用你一遍遍催，harness 自己会一路干到目标达成为止。有人拿它跑了好几天，去解析一批结构很复杂的文档。这种**有边界、能自动验证**的活，它特别能扛。
- `/loop`（Claude Code）：输入 `loop`，用大白话写间隔，比如"每 5 分钟探索一下我项目里的这部分"，它就在当前会话里挂一个 cron job，按你说的时长定时触发。适合开放式、探索性的工作——把一个宽泛的问题丢给它，就像交给一个野心很大的初级工程师，它可能跑偏，但有时候你得看到结果才知道某个方向行不行。
- **IDE 里的 Automations 标签页**（Codex / Cursor / Claude Code 等）：通常在左上角，长得都差不多，一栏"进行中"、一栏"已暂停"。可以设成每天扫一遍收件箱丢到 Linear 看板、定期扫安全漏洞、每周把这一周干过的事沉淀成 skill，等等。

我的理解是：`/goal`、`/loop` 这类**就是平台帮你写好的标准 loop**，你填参数就能用；而所谓"循环工程"，是你开始**自己去设计**这些回路，针对你自己的重复劳动。

5.1.3 核心要素：一个 loop 由哪些零件拼成

一套完整的循环 = 五个积木 + 一个记事本。

积木一：定时任务 整个循环的起点。各家叫法不同，本质一样：给一个项目设定多久自动跑一次，可以每天早上、可以每小时、也可以每次有代码提交时触发。过去你得手动打开 AI 说"帮我看看这个"，现在到点它自己醒过来干活，像个不用你提醒的助理。

积木二：worktree 很多人用 AI 写代码最怕它乱改文件、把好好的项目改崩。worktree 就是给每个 agent 一个独立分支，所有改动都在自己的沙盒里，不碰主分支。改砸了就删掉分支，主代码干干净净。相当于给每个员工一个独立工位，最后审核通过才合并回主线。

积木三：Skill 这是 loop 能不能跑好的核心秘密。用 AI 最大的痛点之一，就是同一个项目每次都得重新解释一遍团队约定、构建步骤、之前踩过的坑。Skill 就是把这些写进文件——构建命令是什么、代码规范是什么、哪些写法绝对禁止、上次出过什么事故——AI 每次启动先读一遍，就知道这个项目的规矩。说白了就是一本**员工手册**，谁来都先翻一遍，不用从头培训。

> Louis 特别强调了这一点：一个**没有可复用 skill** 的 loop，每跑一次都在"从零重新认识你的项目"，纯纯烧 token 重学你早就知道的东西；而一个 skill 写得好的 loop 会开始**产生复利**。原则是——一个 skill 干一件事，写得尽量密、又尽量小，别把上下文塞满。

积木四：连接器 让 loop 跳出单个文件夹的关键。过去 AI 只能看你当前打开的文件，有了连接器，它能连数据库、连公司内部系统、连第三方服务——自己去查生产环境数据、读 CRM 客户记录、拉监控告警。AI 就

从一个"只能处理本地文件的工具"，变成了"能接入你整个工作流的节点"。

积木五：子 Agent 这是整个体系里我觉得最妙的结构。一句话：**写代码的和审代码的，得是两拨人。**一个成熟的 loop 里至少有两个 agent，一个负责写、一个负责挑刺，甚至可以用不同的模型（比如一个写、另一个审）。指令不同、视角不同、模型都不同，才能真审出问题。道理很朴素——自己写的东西自己很难看出毛病，必须有第二双眼睛。

记事本：状态文件 这个最容易被忽略，但它才是 loop 能**长期**跑下去的关键。现在绝大多数 AI 工作流每天都从零开始：昨天确认过的事实今天再查一遍，上周否掉的标题风格这周又冒出来，某个来源一直不靠谱但它下次还引用。为什么？因为记忆只活在单次对话里，对话一关就没了。状态文件就是一份**聊天框之外的公共记忆**——可以是文档、表格、看板——记下已确认的信息、踩过的坑、偏好的格式、禁用的表达、上次没解决的问题。AI 每次启动第一件事就是读它，于是它能接着上次的进度往下走，而不是永远重启。

这六样拼起来，就是一个完整的、能自己跑的 loop。

5.1.4 我的观点：现在还不适合大多数人，但未来一定会

> 这里是我看Youtube视频一个博主的观点，我也觉得很有道理。理解了本章，面试官问你怎么看待loop工程的时候，你就把下面的观点变成你的思考，让面试官对你竖起大拇指。视频链接我也会附在后面链接，不过我觉得大家也没必要专门去看，核心思想我都总结了。

先说一个事实：Boris、Peter 这些人说"我不写提示词、我写 loop"，听起来很酷。但**他们是有底气这么说的**——他们的 token 额度没有上限。如果我也有无限 token，我当然也这么干。

Ross Mike（这个视频博主）打了个特别到位的比方。假设我俩合伙创业，雇了个很聪明的开发者，跟他说"这是我们要做的产品，这些是需求"，然后他**不来咨询你，自己埋头把整个东西做完了。**问题来了：在做的过程中，他必然要做大量假设——产品长什么样、什么手感、某些架构怎么定。这些假设很可能跟你脑子里的愿景对不上。你以为那份 PRD 文档覆盖了一切，但说实话**它永远覆盖不全**，总有边界情况、总有遗漏。等他把"成品"端上来，一堆东西做好了，可就不是你想要的那个味儿。

把开发者换成 agent，把需求文档换成 `spec.md`，一模一样。**你给了 agent 自由发挥假设的空间，大多数时候它会搞错——而且不光搞错，还烧掉一大笔钱。**

钱的问题，是绕不过去的

这是最现实的一条。Ross Mike 说，你只要去看 Peter 的一条推就懂了——**一个月烧掉了价值 130 万美元的 token。**

所以他的建议很干脆：如果你不是 200 美元/月那档套餐，这事**压根不该进入你的考虑。**20 刀、100 刀的套餐，一个 loop 跑两天可能就把你的周限额烧穿了。（甚至博主里面提到了，即使你是200美元/月的用户，其实也经不住你跑大量loop。200美元/月 基本是Claudecode, Copilot这种工具订阅的最高的那个档）

Louis 的说法更工程化一点：**预算就是架构的一部分。**他自己从不在睡觉时跑 loop，永远手动启动并盯着。每个正经的 loop 都得配一个"心跳"机制——最大迭代次数、"无进展"检测、每天能花的 token / 美元上限。还得有比"agent 说自己做完了"更强的验证：跑测试、做类型检查、让评审 agent 去比对 diff 和规格。

视频里把这事总结得很精辟：循环工程**不会让 AI 协作变得没成本**，它只是把成本从"人一轮轮盯着的时间成本"，转移成了"系统一轮轮运行的金钱成本"。token 成本这个问题，反而把这个热词拉回了现实。

为什么它现在不适合"做有创意的东西"

Ross Mike 的核心论点是：**当你要用 AI 做一个真正有意义的东西时，人必须还在循环里。**

为啥？因为你在建一个 app 的时候，**自己都还没完全想清楚要什么。**趋势天天变——今天觉得液态玻璃好看，明天又想调整。你不可能把脑子里所有的细节、所有"我其实是想要那样"的念头，一次性塞进一个文档。做服务、做外包的人最懂这个：你拼命想把客户脑子里的想法掏干净，可永远有一句"哦你漏了这个""我不是这个意思"。**人和人之间都这么难对齐，凭什么指望 AI 一次就懂你？**

他还有两个比喻我很喜欢：

- **"AI 能复刻灵魂，但创造不出灵魂"**（AI can replicate sauce, it can't create sauce）。
- 跑一个大 loop 就像**全自动驾驶**：你从迈阿密一路开到查尔斯顿，按下 go 就停不下来了。路边看到个可爱的小餐馆想下去吃个炸鸡三明治？不行，火车已经离站，你只能跟着这趟跑完。

主持人接了一句很妙的话——这些 loop 本质上是在造一台**老虎机（slot machine）**，拉一把，看运气出什么。中途**没有"把半成品拿给别人收集反馈"这一环**，而对一个想做成产品的人来说，这一环恰恰是命根子。

> 我也是认同这个观点。理想很丰满，我们希望我们就让AI能够无限LOOP，直到完成目标。但是现状是，AI还不能摆脱人的控制，必须依赖于人去监督，想清楚细节。这个疑问：你自己都没有想清楚，凭什么指望AI一次写清楚？ 我曾在Tina huang（一个讲AI很出名的Youtuber）讲vibecoding的时候也提出了这个疑问。现阶段，要想跑得好，使用Loop也需要人精心设计和编排，想清楚细节。

那它现在到底适合干啥？

Ross Mike 不是纯黑。他自己就在用一个 loop，而且用得很顺——**代码评审（code review）**。

他的流程大概是这样：用 Cursor 写代码，每推一个功能到 GitHub，就有一个代码评审 agent（他用的那类工具，类似 Code Rabbit、Greptile）自动审一遍，给出问题清单，外加一个 **1 到 5 分的打分**。他给自己定的规矩是：**低于 4 分的代码，绝不上生产。**

然后是 loop 的部分——他写了个 skill，逻辑就是：去 GitHub 读评审 → 改 → 推回去 → 等新评审 → 还没到 4 分就继续改……最多跑 5 轮，或者一直跑到 5 分为止。

> 注意这个 loop 的形状：**它非常封闭、非常目标导向。**有一个固定的反馈引擎（评审打分），目标清清楚楚（把分数刷到 5）。这正是它能跑通的原因。

但即便这么理想的场景，它也会崩——**只要推的代码超过 1000 行**，agent 就吃不下那么多上下文，几乎拿不到满分。所以他每次都得控制改动量，或者让 Cursor 拆成多个小 PR。你看，连这种封闭生态里，都到处

是会断的地方。

总结下来，**现在 loop 真正成立的，是那些"输出是二元的"场景**——黑或白、做没做到一目了然、不需要创造力。代码评审是，批量生成 300 个同质的 SEO 页面也是。娜娜那期给了四个判断标准，我觉得可以直接拿来当 checklist：

1. 任务会**反复出现**；
2. 流程**相对稳定**；
3. 结果**可以被检查**；
4. 最关键的**判断权还在人手里。**

四条都满足，这活就可以设计成 loop；缺一条，循环的成本很可能就高过回报。

但是—未来一定会到

100% 地，未来一定会走到"loop 普遍可用"那一步，只是不是现在。

之所以现在唱反调，针对的不是 Boris、Peter 本人——他们要做研究、要试自愈 agent，烧 token 对他们是常识、是本职。Ross Mike 真正想提醒的，是那些**到处教学、把 loop 吹成"天花板"的内容创作者**：对月付 20 刀的普通人来说，这玩意儿现在不是神器，是碎钞机。

但行业的重心，确实在不可逆地转移。借 Louis 的那条时间线收个尾：

> 五年前你自己写代码；两年前你提示模型替你写代码；去年你看着 Claude 写、一个任务一个任务地点确认，生怕它搞砸；而今天，对**合适的任务**，你开始设计那个能自己提示、自己检查、自己重试、自己停下来的 loop。

杠杆的位置变了。去年你花 100 个小时练提示词技巧，回报可能很高；今年再花同样的时间，回报大概没那么高了。未来真正拉开差距的，**可能不是谁的提示词写得更漂亮，而是谁能更早想清楚：哪些活该交给循环自动跑，哪些活必须自己亲手抓。**

Louis 最后那句话我想原样记下来——**不管你怎么改工作流，请确保你还是那个工程师。**你可以用 loop 在你已经吃透的工作上跑得更快，也可以用它来逃避去理解工作本身。给未来的自己，做个对的选择。

至于 Ross Mike，他的结论更短：**Human in the loop is the best loop.**（人在循环里，才是最好的循环。）

> 我这里的理解是，现在虽然比如CC这些工具有/loop 的这样的功能，帮你完成了一些Loop的工程，只要使用/loop + 提示词就可以完成Loop功能。 但是你无法直接依赖于它，就用它让Agent自己去探索，完成你的任务，这个是不切实际的。因为你让AI探索，中间会走很多弯路，耗费大量Token不说，结果也会错误。现在的Loop需要你精心设计，整个流程步骤，评估-测试-迭代效果才可能会稳定，人不可能完全退出这个过程，依赖于一个/loop，什么都不设计，就能完成大型项目。但是未来，这是一个趋势，未来AI也许聪明到，不需要我们设计整个过程，我们大概结合loop +简单的提示词，AI就能稳定探索，解决复杂问题。

5.1.5 如何做一个LOOP?

Loop其实也不是啥新概念，甚至很多人都已经在用了。我的理解它就是用了Harness工程技巧，定义一个完整的执行流，来完成一个复杂的任务。 所以其实这个问题也没有一个标准的答案和做法。

你看ClaudeCode的 /LOOP就是Loop的实现，我们讲的Harness工程举到的两个例子：Anthropic Harness工程 和 ClawCode Harness工程：他们都是精心编排，让AI driven整个开发过程，也可以看做是Loop工程。包括我看我们同事也有分享他是如何用AI完成复杂项目。这些复杂的流程编排都是Loop。

随着模型的发展，写loop的范式也一定会发生变化。比如按照上一小节讲的。也许现在我们要写spec-开发-review-测试-代码提交的流程编排写loop，但是未来，一个很强的模型出现，大型复杂项目的编写，就是/loop + 你就把这份文档给它，完成整个项目的开发。就实现了。

如何实现Loop，我觉得这是一个思想，一章讲不完，也没有标准答案，况且我们笔记其实涉及的很多内容就是Loop的实现，如果你想研究这个问题，给你一些参考方向：

1.研究CC的/loop实现，让AI去分析，毕竟CC是开源的，代码在这里（不过我不太确定泄露的这个版本的代码，有没有实现Loop）

2.这两个Harness项目：Anthropic Harness工程 和 ClawCode Harness工程。

3.未来我们笔记里做的Agent项目，肯定使用Harness,Loop的思想去做的，这个属于自己实战项目，不过还早呢，现在还没有。

5.1.6 面试问题

> 其实聊到Loop工程，面试官最常问的肯定是什么是Loop？他有哪些特点？以及你如何看待？本期笔记足够你回答这三个问题了，应对最基本的和Loop相关的问题。 把上面的东西收敛成两道可能会被问到的题，顺便当作自检。

5.1.6.1 Q1：什么是 Loop 工程/loop工程有哪些模块?

循环工程是一种新的人机协作范式：**你不再亲自一轮轮地提示 AI，而是设计一个能让 agent 自主运转的闭环。**

它和定时任务（cron）最本质的区别在于——**决策者在循环内部。**cron 跑的是固定脚本，步骤永远不变；而 loop 跑的是一个 agent，它会观察当前状态、自己决定下一步、执行、检查结果，再判断要不要继续、重试还是停下。所以它更像在"模仿一个工程师"，而不是"执行一段脚本"。

一个能跑起来的 loop，至少需要一个**触发器**和一个**可验证的目标**；完整一点的结构通常包含五个积木加一份记忆——定时任务、独立工作目录（worktree）、技能（Skill）、连接器（MCP）、子 Agent（写审分离），再加一个跨会话的**状态文件。**放进行业脉络里看，它是"提示词工程 → 上下文工程 → Harness 工程 → 循环工程"这条线上的最新一站：**人和 AI 协作的基本单位，从"一次对话"变成了"一个完整回路"。**

5.1.6.2 Q2：你怎么看待 Loop 工程?

我的看法是——**方向上看好，但现在对大多数人还不实用**，得分场景、分钱包来谈。

先说不看好现在 all-in 的理由。第一是**钱**：loop 会反复读上下文、反复重试、四处探索，不管有没有产出都在烧 token，有人一个月烧掉过上百万美元；对月付 20、100 刀的普通用户，跑两天可能就到周限额了。第二是**目标难定**：软件开发常常是探索性的，你一开始根本说不清最终要什么，目标一模糊，loop 就会朝着那句含糊的话拼命优化，结果可能比你认认真真手动做一遍还糟。所以现阶段它真正成立的，是**输出二元、有客观验证、流程稳定、且判断权还留在人手里**的场景——代码评审、批量 SEO 页面这类，而不是"帮我从零做个有创意的产品"。

但拉长看，我**完全相信这是未来。**随着模型越来越强、越来越便宜，"跑一轮循环的钱"正在逼近"人来回追问几轮的成本"，这笔账迟早会翻过来。真正值得优化的，也会从"某一句提示词怎么写得更完美"，变成"整个反馈回路怎么设计得更高效"。

所以我的态度是 Louis 那句话：**先从简单的做起，只在自动化能自己回本时才加自主性**；以及不管怎么用 loop，**人始终得是那个工程师**——定好停止条件、写好 skill、读懂它产出的东西。Loop 既能帮你在懂的事情上跑得更快，也能诱惑你彻底不去懂——别选错了。

5.1.7 参考资料

这个是第四小节参考的视频，其实你也不用看，核心思想我都总结。我列出是为了完整性，说明内容参考依据。 https://www.youtube.com/watch?v=7clJ8IH784Q&t=83s

其他的关于Loop 工程，其实就看一个视频就够了，甚至是不看视频，看笔记也够了，基本就是他是啥，他的核心组成，如何看待它，笔记里总结的都有。 如果想看视频讲解的话，我找一个中文的，相对来说比较完整的，可以参考：

5.2 Graph 工程

最近，“Graph Engineering（Graph 工程）”突然成了 AI 圈的新热词。大家真的非常喜欢用这样一种说法来描述一些新技术： 提示词工程已死，上下文工程来了。 上下文工程已死，Harness工程来了。 Harness工程已死，Loop工程来了。 Loop工程已死，graph工程来了。 当时Harness Loop 出来，我都有比较早的去解析，包括今天的Graph工程，我觉得多少这些概念都有一些操作的概念。本身他们也不是一个全新的概念，其实多少都是新瓶旧酒。 不了解的人就很容易被唬住，觉得技术概念迭代这么快，感觉很难跟上对吗？我自己这里的看法是，首先这些概念真的不是一个新的概念，他只是我们已有技术，换了一个高级的说法，其实你自己都用过了，所以其实没有什么太多的学习成本。 第二点就是你也不能不会，因为即使是炒作，AI圈也很喜欢跟风，面试官甚至可能自己也不是特别懂，但是他看到这些火的概念就会问题知不知道，问你有没有关注前沿技术。所以从这个角度来说，我们还是需要花一些时间（但是通常可能1小时就够了）去理解这个词到底是什么，可以和面试官讨论，发表你的看法。

所以我的建议就是，看看我的笔记或者视频，**你知道这个东西是什么，来龙去脉是什么，然后我总结了相关的面试问题**，这些面试题是面试官问到这个概念会最先问的。**学到这里就够了，也没什么好深入学的**，因为他本质就是旧的技术。如果后面发展得更火，有更难的面试问题，我们也会跟进总结相关的面试问题的。

5.2.1 Graph 工程的来源

这场讨论源于 Peter Steinberger 的一条玩笑推文：

> “我们还在讨论循环，还是已经转向图了？”（Are we still talking loops or did we shift to graphs yet?）

这条只有九个英文单词的推文获得了约 270 万次浏览。随后，Hamel Husain 又用一句“Loop engineering is dead. Enter graph engineering（Loop 工程已死，Graph 工程登场）”推波助澜。很快，整个信息流仿佛都认定一个新学科诞生了，大量解释“什么是 Graph 工程”的文章随之出现。

但这两条主要推文原本都是玩笑。Steinberger 调侃的是 AI 圈给概念改名的速度：Prompt Engineering 变成 Context Engineering，然后是 Harness Engineering、Loop Engineering——这些词在很大程度上描述的是同一件事不断扩大的不同层次。 我们将目前AI圈5个比较火的工程，用这种图来总结：

- **Prompt Engineering** 关心“问什么”（what to ask）。
- **Context Engineering** 关心“发送什么”（what to send）。
- **Harness Engineering** 关心“如何运行”（how to run it）。
- **Loop Engineering** 关心“如何让它一直运行，直到任务完成”（how to keep it running until it's done）。
- **Graph Engineering** 关心“如何让多个任务与循环按照明确的路径协同运行”（how to orchestrate tasks and loops through an explicit graph）。

从这个角度看，**Graph 工程并不是对 Loop 工程的取代，而是把单个循环扩展成由多个节点、分支、验证器和循环组成的完整工作流。**它关心的不只是 Agent 能否持续运行，还包括任务如何分解、结果如何流转、失败后回到哪里，以及整个系统何时停止。

5.2.2 Loop vs. Graph

一般在讲Graph工程的时候，都会先讲Loop工程，很多人定义Graph工程是建立在loop工程之上的，对多个loop进行编排，就成了Graph

Loop 工程：一个循环，一个任务

相比只向模型发出一次提示，Loop 工程让 Agent 围绕一个目标持续行动，由外部验证器检查结果：如果失败，就把任务送回循环；如果达到目标或触发停止条件，则结束流程。

一个 Loop 通常包括：

1. 触发任务，例如 PR、CI、定时任务或提示词；

2. Agent 执行工作；
3. 运行测试、类型检查、Reviewer Agent 或规格差异检查；
4. 判断目标是否已经得到验证；
5. 未通过则继续迭代，通过则结束；达到迭代次数、无进展或预算上限等硬性限制时，停止并交给人类。

Loop 的关键，不只是让模型“再试一次”，而是把人从每一轮审查中移出，节省人的时间，或至少减少人的工作量。最简单的实现甚至可以直接写进提示词：要求 Agent 审查并修改自己的工作，直到代码测试全部通过。 大家使用ClaudeCode里面的/goal 其实就是一个典型的Loop工程的案例。

Graph：当一个 Loop 不够时

当一个 Loop 无法完成复杂任务时，Graph 的讨论就开始了。Graph 不是 Loop 的对立面，因为 **Graph 本身可以包含 Loop**。它只是比单一循环多了一层编排，让多个 Agent 像一个团队一样分工、交接、审查、返工。

以自动代码审查为例：

1. Codex 完成代码修改并创建 PR；
2. 系统并行启动多个审计 Agent，分别检查安全、逻辑与性能、代码风格与文档；
3. 多个审计结果汇聚到 Verifier；
4. 确认的问题交给 Fixer 修复；
5. 测试套件执行外部检查；
6. 测试失败则回到 Fixer，形成循环；测试通过则合并发布。

这相当于模拟多个员工共同工作：有人执行，有人审核，工作在不同角色之间来回流转。这种人类早已熟悉的工作结构，就是一张图。

所以，Loop 和 Graph 并不是非此即彼的替代关系：

> Loop 是完成一个任务的反馈闭环；Graph 是组织多个任务、角色、分支和反馈闭环的更高一层结构。

5.2.3 Graph 工程是什么

Graph 工程，是把复杂的 Agent 工作过程显式画成并实现为一张执行图：节点承担任务，边决定下一步运行什么；系统可以分支、并行、汇聚、验证、返工或停止。

它与普通固定流水线的关键区别在于：普通流水线中的步骤遵循固定规则，而 Graph 中的 Agent 会解释任务。Agent 可能误解指令，也可能在下一次运行时做出不同选择，因此节点具有概率性。

过去，我们常把一切都塞进一个巨大的聊天上下文，让模型同时充当调度器、数据库、日志系统和项目经理。对于一次性的小任务，这种做法或许有效；但当工作持续数小时甚至数天、横跨多个代码仓库并涉及多个 Agent 时，它就会崩溃。

把 Graph 明确画出来，会迫使我们预先思考并固定这些问题：

- 哪些任务可以并行运行？
- 节点之间传递什么状态？
- 谁可以否决结果？
- 系统在停止之前最多可以花费多少成本？

一套完整的 Graph 不只有“执行图”，还需要“控制图”：执行图回答“接下来运行什么”，控制图回答“谁来检查做决定的人”。指标、审计、权限、人类否决权以及来自真实世界的反馈，共同为系统提供现实锚点。

其实我们之前学Agent编排的时候，学到的Agent的各种编排模式，像**prompt chain**, routing, parallelization 这些Agent编排模式，其实都是Graph。看到这里，你应该能理解，其实Graph工程也并不是什么新的概念。

5.2.4 最佳实践和建议

不要把“更多 Agent”误认为“更高质量”

由 Agent 检查 Agent 的 Graph，可能产出极其有组织的胡说八道。20 个使用同一模型、读取同一份错误上下文的 Agent，可能会以工业化规模彼此赞同。模型本来就倾向于偏爱自己的回答，因此也容易彼此认同。

如果 Graph 中有多个 Agent，应精心设计评审系统：

- 使用来自不同模型的多个 Reviewer；
- 给 Reviewer 提供彼此独立的新鲜上下文；
- 合并多个 Judge 的判断，而非盲从单一多数意见；
- 在重要发布前，保留最后一次人工检查。

至少保留一个 Graph 之外的现实锚点

扩展 Agent 系统时，至少应有一种检查不属于同一套概率性自证循环，例如：

- 确定性的代码测试或代码审查规则；
- 人工检查与否决权；
- 可直接观测的业务指标，例如实际收入、成本或其他外部结果。

这些外部指标和检查用来确认系统没有只是在规模化地产出低质量内容。追求质量时，不可能把所有事情都自动化。

从一个简单 Loop 开始

不要因为一个新热词，就立刻搭建一个整夜运行的 40-Agent Graph。那很可能只会耗尽 Token，几乎没有真实进展。

更稳妥的顺序是：

1. 选择一个真实、重复发生的任务；
2. 为它设置一个真正有效的验证器；
3. 明确预期的状态；
4. 建立一个能快速查看关键指标的界面，并持续改进；
5. 设置真正的硬停止条件，包括迭代次数、无进展和预算上限；
6. 持续运行，直到理解它会如何失败；
7. 只有当工作负载确实要求扩展时，再增加 Reviewer、并行分支，或拥有否决权的安全检查。

到那时，你就拥有了一张 Graph。其实它可能一直都存在，只是过去没有被画出来，而且人承担了其中的大部分节点。随着自动化节点增加，人在图中的参与会减少，但仍然应该保有控制权。

5.2.5 结语

Loop 工程并没有死。Graph 工程只是当 Agent 编排变得更容易之后，人们开始更严肃地看待编排问题的新名字。

真正长期有效的能力，不是追逐 Loop、Graph 或下一个热门术语，而是判断：

> 系统中的哪些部分值得交给概率性的 Agent，哪些部分应该继续保持朴素、确定且可验证。

Graph 工程的价值，不在于让 Agent 越多越好，而在于把复杂协作显式化：明确任务如何流转、状态如何传递、结果由谁验证、错误如何返回，以及系统何时必须停止。

5.2.6 面试问题

你最近了解的 Agent 里面的新技术是什么?

我最近关注到的是 Graph 工程。它其实是从 Prompt 工程、Context 工程、Harness 工程、Loop 工程一路演进下来，AI 圈最近比较火的一个说法。

简单来说，**Loop 工程是让一个 Agent 围绕一个目标持续行动，由外部验证器检查结果，失败就回到循环，达标或触发硬停止条件才结束；而 Graph 工程是在 Loop 之上再加一层编排，把复杂的 Agent 工作过程显式画成一张执行图**：节点承担任务，边决定下一步运行什么，系统可以分支、并行、汇聚、验证、返工或停止。

一个典型例子就是自动代码审查：Codex 改代码建 PR，然后并行启动多个审计 Agent 分别查安全、逻辑性能、风格文档，结果汇聚到 Verifier，确认的问题交给 Fixer 修，测试通过就合并，失败就回到 Fixer 形成循环。本质上它模拟的是多个员工分工协作的结构。

什么是 Graph 工程，你怎么看待 Graph 工程的诞生?

Graph 工程就是把复杂的 Agent 工作过程显式地画成并实现为一张执行图：**节点承担任务，边决定下一步运行什么，系统可以分支、并行、汇聚、验证、返工或停止。**它和普通固定流水线的区别在于，流水线的步骤遵

循固定规则，而 Graph 里的 Agent 会去解释任务，节点是概率性的，可能误解指令，也可能下一次运行做出不同选择。一套完整的 Graph 除了“执行图”（接下来运行什么），还需要“控制图”（谁来检查做决定的人），靠指标、审计、权限、人工否决这些现实锚点来兜底。

至于它的诞生，我的看法是它其实不是一个全新的概念。它最早来源于国外大佬的两条玩笑推文，本意是在调侃 AI 圈给概念改名的速度。我们之前学 Agent 编排的时候，**prompt chain、routing、parallelization 这些编排模式本身就是 Graph**。所以 Graph 工程更多是把我们本来就在用的东西换了个更高级的说法——这张图其实一直都存在，只是过去没有被显式画出来，而且大部分节点是由人来承担的。

从 Prompt 工程到 Context 工程一直到 Graph 工程，你能聊一聊你对这些名词不断产生的看法吗?

我觉得这些名词其实描述的是同一件事不断扩大的不同层次：**Prompt 工程关心“问什么”，Context 工程关心“发送什么”，Harness 工程关心“如何运行”，Loop 工程关心“如何让它一直运行直到任务完成”，Graph 工程关心“如何让多个任务和循环按明确的路径协同运行”。它们是逐层扩展的关系，而不是谁取代谁**——比如 Graph 并不是 Loop 的对立面，Graph 本身就可以包含 Loop。

所以我对这种名词不断迭代的态度是：**一方面不要被唬住，因为它们本质上不是全新概念，多少是新瓶旧酒。落到实践上，**我更认同的做法是不要因为一个新热词就去堆几十个 Agent 的大 Graph，而是从一个真实、重复发生的任务、一个真正有效的验证器和明确的硬停止条件开始，需要扩展时再逐步加 Reviewer、并行分支和安全检查。真正长期有效的能力，不是追逐下一个热门术语，**而是判断系统里哪些部分值得交给概率性的 Agent，哪些应该保持朴素、确定且可验证。**

---

6 Computer Use[按需学习]

6.1 前言

更新这一期问题的灵感来源自2026年10月字节秋招大模型开发一面的面试问题，面试官问： **Computer Use 操作本地电脑的大致实现机制是什么？是否了解操作本地电脑和操作云端电脑两种方案?**

对于Computer Use这个概念，不算高频考点，除非面试官就是对此比较感兴趣，一般不会问。

学习本章的意义:

1. 这确实是一个很实用的操作，如果掌握了它的原理，你在使用AI工具让他进行computer use的时候，会更加顺利。
2. 学习Computer Use的实现过程，有助于我们**从模型和Harness两个角度**，理解大模型相关知识点。有助于你更理解Agent是怎么运行的。
3. 即使面试官不常问，但这也偶尔被问到吗？另外它不问，学会了在某些场合你也可以自己主动印出来这个点，和面试官讲解，也可能会在一些面试问题上给到你答题灵感。

总之：**此内容不是必学高频考点，但学了肯定对你有帮助。**

6.2 什么是 Computer Use?

借用 Anthropic 官方对 Computer Use 的介绍：

> developers can direct Claude to use computers the way people do—by looking at a screen, moving a cursor, clicking buttons, and typing text.

也就是说，**AI 可以像人一样使用电脑：观察屏幕、移动光标、点击按钮，以及输入文字。**

更具体一点，在启用 Computer Use 后，你可以对 Claude Code 说：“请帮我打开浏览器，找到 Anthropic 官方对 Computer Use 的介绍，并把定义部分截图保存下来。”你会看到它打开浏览器、输入搜索词、进入官网、找到相关段落，再完成截图和保存，就很真人的操作电脑一样。

6.3 实现机制

其实，OpenAI 和 Anthropic 官方都介绍过 Computer Use 是如何实现的，可以参考 Anthropic 的《Developing a computer use model》、《Computer use tool》，以及 OpenAI 的《Computer-Using Agent》。它们介绍的核心机制可以概括为：

观察当前界面 → 模型理解并决定下一步 → 工具执行动作 → 观察操作结果 → 继续循环。

这个道理很容易理解，其实就和人去操作电脑一样对吗?

人通过眼睛获取屏幕，思考后决定是点击还是敲键盘，看到效果后继续思考下一步，如此循环。

接下来，我们就顺着这个顺序，把这每一步理解了。

2.1 观察当前界面：如何获取界面的信息?

人是用眼睛看屏幕，那模型怎么“看”到当前界面呢？其实不只有截图这一种方式。这里按照截图、读取无障碍树，以及两者结合这三种方案来理解。

方案一：截图，让模型看画面

最直接的方法，就是**由工具截取当前屏幕，再把图片交给模型。**模型收到的是真实界面的画面，后续再根据图片理解页面内容、识别按钮位置，并决定下一步操作。

比如要登录一个网站，程序先截取登录页面，模型看到图片中的账号框、密码框和登录按钮，再决定接下来要操作哪个位置。

方案二：读取无障碍树，让模型读界面信息

对于网页，除了截图，还可以**通过浏览器工具读取页面的无障碍树，把其中的结构化信息交给模型。**

无障碍树可以理解为界面的“语义说明书”，原本用于支持屏幕阅读器等辅助技术。它用树状结构描述页面里有哪些元素、分别叫什么、处于什么状态，比如“这是一个按钮，名称是登录”“这是一个复选框，目前未选

中”。

比如，你让 AI 帮忙登录一个网站，工具读取登录页面的无障碍树后，将下面这样的文本交给模型。**这里是简化示意，不是固定的工具返回格式**；方括号里的编号是工具附加的元素引用，用于后续操作：

```Plain Text 登录页面 ├── 输入框：账号 [ref_1] ├── 输入框：密码 [ref_2] ├── 复选框：记住我，未选中 [ref_3] └── 按钮：登录 [ref_4]
```

>
>
> 通过这份文本，模型就能知道页面上有账号框、密码框和登录按钮。结合用户的登录目标，它可以决定先向账号框 `ref_1` 输入账号，再向密码框 `ref_2` 输入密码，最后点击登录按钮 `ref_4`，并把这些操作请求交给工具执行。
>
> 所以，这种方式不需要模型先从图片里辨认控件、判断坐标，而是**通过读取无障碍树，了解当前界面，再决定下一步操作**。这份信息可以直接作为工具结果返回，不一定需要保存成一个文件。
>
> > **思考丨为什么不直接使用 HTML 或 DOM？**
> >
> > 你可能会问：网页本身就有 HTML，也有浏览器解析、运行后形成的 DOM，为什么这里选择读取无障碍树，而不是直接把它们交给模型？
> >
> > 这本质上是一种方案的取舍。Anthropic 在[浏览器工具的安全建议](https://platform.claude.com/docs/en/agents-and-tools/tool-use/browser-use-tool#security-considerations)中明确说：“build page reads from what the page renders (the accessibility tree or visible text), not raw DOM source”。意思是，应该基于页面渲染出的无障碍树或可见文字构建读取结果，而不是直接返回原始 DOM 源码，避免将隐藏文字交给模型。
> >
> > 对于找按钮、填表、点击这类任务，无障碍树提供的角色、名称、状态和工具附加的元素引用，也更贴近模型当下需要的信息。因此，这类场景优先使用无障碍树更合适；**不是模型看不懂 HTML，也不是 DOM 不能用**。如果任务变成分析页面代码或排查元素结构问题，HTML 或 DOM 就可能更合适。**无障碍树也不能消除页面内容中的提示注入风险。**
>
> **方案三：两者结合，既读无障碍树，也看画面**
>
> 这两种方式并不冲突，可以在同一个任务里配合使用。
>
> 比如，工具先读取无障碍树，让模型找到账号框和登录按钮；执行登录后，可以先读取新的无障碍树，检查是否出现成功或错误提示。如果还需要确认页面布局、按钮是否被遮挡等实际显示效果，再获取截图进行判断。
>
> 这里的“两者结合”并不是每一步都要同时读树和截图，而是**根据当前任务，按需选择获取哪一种信息**。
>
> > **思考丨什么时候读无障碍树，什么时候看截图？**
> >
> > 你可能会问：读取无障碍树能完成的操作，模型看截图不是也能完成吗？为什么还要优先读树？
> >
> > 这有点像我们一直讲的：一个功能既能用 AI 做，也能用代码做，应该怎么选？相似的取舍思路是，**选择更直接、够用的方法**。不过，这里两种方案都需要模型理解，区别在于给模型什么信息，而不是要不要使用 AI。
> >
> > 对于找按钮、填表这类操作，无障碍树已经能提供**元素的名称、状态和工具附加的引用**，就不必先让模型从图片里辨认位置。对于典型网页，这通常能减少输入 token 消耗，同时效果也很好，处理速度更快。
> >
> > 如果任务是**判断图片有没有被裁切、按钮有没有被弹窗遮挡**，这是就可以用截图。

> 总结来说，可以记成：**能通过元素语义完成的常规操作，先读树；需要判断画面，或者树里没有有效目标，就看截图。两者结合是按需切换和使用。**

三种方案的对比

方案	优点	局限	适用场景
截图	能看到实际显示效果，不依赖页面提供可用的无障碍树	需要模型识别画面、定位目标；按坐标操作时容易受到布局变化影响	检查图片、布局、遮挡，或操作无障碍树未描述的画布、远程桌面等内容
读取无障碍树	直接提供元素的角色、名称和状态，可结合元素引用操作；典型网页通常比截图消耗更少的输入 token	依赖工具和页面提供有效的树；无法完整表达视觉效果，页面结构明显变化后引用可能失效	网页填表、查找控件、点击按钮、读取文字等常规操作
两者结合	读树定位控件，用截图检查画面，兼顾结构信息和视觉信息	需要判断何时切换；如果每一步都重复读树和截图，可能增加不必要的开销	既要完成控件操作，又要验证实际显示效果的任务

2.2 理解界面并决定下一步：模型如何生成动作？

当模型获取了信息后，下一步就是根据当前信息，推理决定下一步的动作。

模型首先要理解界面。如果输入的是截图，模型通过视觉能力识别界面内容、控件及其位置；如果输入的是无障碍树，模型则读取元素的角色、名称、状态和引用。两者提供信息的方式不同，但目的相同：**让模型知道当前界面是什么状态，有哪些可以操作的对象**。

接下来，模型需要结合**用户目标、相关的历史操作和执行结果**，判断任务进度，选择当前合适的动作。因此，模型接收的不只是界面信息，还包括任务上下文和可用工具的说明。它并不是每次看到界面就重新开始，也不是一次规划好所有动作后机械执行，而是**随着新的观察调整下一步**。

确定动作后，模型还需要定位操作目标：截图路线需要把目标对应到画面中的位置，无障碍树路线则可以利用元素引用。随后，**模型按照工具定义生成操作请求**，明确要执行什么动作、作用于哪个目标。

2.3 执行动作：谁把模型的决定变成真实操作？

工具执行器负责将模型的操作请求落实为实际操作，**harness 负责组织工具调用、分发请求和回传结果**。

执行时，程序先接收模型返回的工具调用，解析动作类型和操作目标，再分发给对应的实现。

具体怎么执行，取决于目标环境。对于桌面，可以将请求转换成操作系统的鼠标、键盘输入事件；对于浏览器，可以通过 Playwright 等自动化工具执行鼠标、键盘操作，或者将元素引用映射到实际控件后再操作。因此，执行器可以由脚本或其他程序实现，但这不等于整个任务都是提前写死的脚本：**每一轮要执行什么，仍然可以由模型根据当前状态决定**。

执行器还需要把目标对应准确。按坐标操作时，要保证**坐标与模型观察到的截图一致**；按元素引用操作时，要维护**引用与实际元素的对应关系**。请求落到错误的窗口、页面或已经失效的元素上，即使调用格式正确，也不能算执行正确。

另外，**模型提出请求，不代表它自动获得了执行权限。**执行侧需要校验参数和访问范围，**对需要确认的敏感动作保留用户审批**；遇到无效目标、权限不足或工具不支持的动作，应返回错误，而不是当成成功继续运行。

6.4 操作拆解

3.1 完整流程：从任务输入到操作落地的六个步骤

为了让前面的理论更落地，这里拆解一次典型的 Computer Use 操作流程。假设我们要让智能体完成这样一个任务：“**在当前浏览器中打开某个网址，搜索关键词，并把第一条结果的标题复制下来。**”

下面以截图驱动的操作方式，顺着这个任务走一遍。

第一步：任务解析与目标拆解

模型收到任务后，先梳理要完成的事情：进入指定网站、找到搜索框、输入关键词、触发搜索、读取第一条结果的标题，再复制到剪贴板。有了这份大致的步骤清单，后续每次观察界面时，模型就能结合任务进度，判断接下来该做什么。执行过程中，**这份计划也会随着页面状态调整。**

第二步：初始环境快照

智能体调用截图工具，**捕获当前屏幕**。这张图片让模型知道“我从哪里开始”：浏览器打开了什么页面，地址栏在哪里，窗口前面有没有弹窗。模型随后根据画面定位操作目标，生成与截图位置对应的动作。如果浏览器工具支持读取无障碍树，也可以按需获取页面元素的名称、状态和引用，辅助判断。

第三步：状态理解与动作生成

假设截图中，浏览器正停留在另一个网站。模型结合任务目标，判断应该先进入指定网站，于是找到地址栏的位置，**生成点击地址栏的操作请求**。随后再根据执行结果，继续安排网址输入和回车操作。

这些请求会按照工具规定的格式输出，包含**动作类型、目标位置和所需参数**，供执行器读取。这里，模型完成了从“看到当前页面”到“决定具体操作”的转换。

第四步：执行动作并记录结果

Harness 将操作请求交给执行器，执行器在电脑上完成对应的点击、输入和按键操作，并返回执行结果。系统再获取新的截图，让模型检查操作后的页面。

例如，输入网址并按下回车后，模型会查看网站是否已经加载完成。搜索框出现了，就可以继续输入关键词；页面还在加载，就等待后再观察；出现错误提示，则根据提示调整操作。**每次执行后的反馈，都会成为下一次决策的依据。**

第五步：循环迭代直到任务完成

接下来，智能体沿着同样的流程推进：找到搜索框、输入关键词、提交搜索，等结果出现后，再定位第一条结果的标题，完成选择和复制。整个过程会经历多轮“**截图–理解–决策–执行–再截图**”。

Harness 同时**保留必要的操作记录和阶段性结果**，帮助模型记住任务进度。例如，搜索已经提交，当前就该检查结果列表。任务较长时，较早的记录可以整理成摘要，供后续决策参考。

第六步：结果校验与输出

最后，智能体**按照用户最初的要求检查结果**：搜索词是否正确，选中的是否为第一条结果，标题是否完整。如果工具支持并获准读取剪贴板，还可以将剪贴板内容与页面标题核对。

检查完成后，智能体向用户反馈提取到的标题和复制情况；**遇到无法完成或无法验证的部分，也一并说明**。至此，这个任务从自然语言输入，经过多轮界面交互，形成了可以交付给用户的结果。

6.5 难点与困境

你可能看了上面的内容，觉得：哇，这个computer use的功能和原理其实也不难呀，我自己就能实现。截图-理解图片-推理下一步-调用调用工具，然后再循环就行了嘛，为什么现在的工具computer use的功能还不能说完全成熟了呢？

其实从模型层和Harness层，computer use仍然面临着很多技术挑战和难点，我来分别从Harness和模型两个角度，来讲解这件事的难点：

4.1 模型训练层

模型在常见界面上操作顺利，并不代表换一个软件、换一种布局后，仍然能做对。比如，同样是打开一个文件，在普通桌面上可能只需要双击图标；换成一个不熟悉的文件管理界面，入口、图标和交互方式都可能不同。模型需要根据当前界面重新判断，而不能只复用熟悉的操作顺序。

这背后的难点是泛化。训练中出现过的界面和操作路径是有限的，实际使用时却会遇到不同的软件版本、布局变化和异常状态。训练不仅要让模型学会常见操作，还要让它在界面变化后仍能识别目标、选择动作，在原来的路径走不通时调整策略。

同时，Anthropic的官方资料也表明了**屏幕理解、像素定位、操作推理，以及遇到障碍后的纠错重试** 这些都是在computer use方面，模型训练测需要处理的难点。

4.2 Harness 层

假设为了控制图像开销，程序先把桌面截图缩小，再交给模型。模型准确指出了缩小后图片里的按钮位置，但执行器直接把这个坐标用于原始桌面，鼠标就可能点到按钮外面。这时，问题不是模型没看懂，而是**观察和执行使用了不同的坐标系**。

Harness 需要组织**截图采集与执行器之间的坐标映射**，保证模型看到的位置能对应到实际操作的位置。单纯缩放整张截图时，按尺寸比例转换即可；如果还涉及窗口裁剪、系统显示缩放、窗口移动或多显示器，就需要同时处理**缩放关系和位置偏移**，不能只套一个固定比例。

包括还有很多难点：比如如果你发出的命令，让鼠标瞬移，那么很多软件会判定为异常操作，所以很多computer use做鼠标移动操作时，会刻意放慢速度，让这个操作过程看起来像人。同时在这种连续的截图，判断，调用工具的循环过程中，非常容易占

> 满上下文，**如何优化管理上下文**，也是一个调整。
>
> 通过这些例子，你应该能感受到：很多技术**做出一个能跑通的 Demo 并不难，但要把它做成成熟、稳定、能够应对各种边界情况的产品，中间还有很大的差距。**
>
> 最后我们以Anthropic为例，从他们的公开进程宣布表明：**Computer Use这个技术一直在进步，但并未完全跑通**：
>
> | 时间 | 公开节点 | 主要进展 |
> |-|-|-|
> | 2024 年 10 月 22 日 | [新版 Claude 3.5 Sonnet 引入 Computer Use 公开测试](https://www.anthropic.com/news/3-5-models-and-computer-use) | 模型可以根据截图生成移动鼠标、点击、输入等操作请求，再由开发者提供的程序执行。这时已经能跑通操作闭环，但官方明确表示，它仍然笨拙、容易出错。 |
> | 2025 年 2 月 24 日 | [Claude 3.7 Sonnet：更强的连续行动能力](https://www.anthropic.com/news/visible-extended-thinking) | 官方将其中一种改进称为“action scaling”：模型能持续调用工具、响应环境变化，并利用更多交互步骤提高任务完成率。重点不只是第一步选得对，还包括走得更远、遇到变化后继续推进。 |
> | 2025 年 5 月 22 日 | [Claude 4 发布](https://www.anthropic.com/news/claude-4) | 支持在扩展思考中调用工具，再根据结果继续推理，并改进指令遵循和借助外部文件保持任务记忆的能力。这些是通用 Agent 能力的增强，也为持续操作电脑提供基础，不是重新发明了一套点击机制。 |
> | 2025 年 9 月 29 日 | [Claude Sonnet 4.5：Computer Use 能力进一步提升](https://www.anthropic.com/news/claude-sonnet-4-5) | 官方报告其电脑操作评测表现明显提升，并通过 Claude for Chrome 展示浏览网页、填写表格等任务。同批发布的上下文编辑和记忆工具，则从工程侧支持 Agent 运行更长时间。 |
> | 2026 年 2 月 17 日 | [Claude Sonnet 4.6：复杂办公任务与安全性改进](https://www.anthropic.com/news/claude-sonnet-4-6) | 官方提到复杂电子表格、多步骤网页表单、跨浏览器标签页协作等场景的进步，同时提升对提示注入的抵抗能力。但也明确承认，模型操作电脑的能力仍落后于最熟练的人类。 |
> | 2026 年 6 月 30 日 | [Claude Sonnet 5：提高完成任务的能力与成本效率](https://www.anthropic.com/news/claude-sonnet-5) | 官方展示了相较 Sonnet 4.6 更好的电脑操作成本–性能曲线：不只是追求更高的完成率，也关注完成同类任务需要多少计算和费用。 |
> | 2026 年 9 月 28 日 | [Claude Sonnet 5.5：继续改进长任务与图像理解](https://www.anthropic.com/claude-sonnet-5-5) | 官方报告其长任务和图像理解能力进一步提升，并展示仅根据截图通关《宝可梦 红》的结果。这说明模型能在特定环境中持续观察和行动。 |
>
> 我们能看出，现在的Anthropic公司的Computer Use能力，仍然有待提升，其中主要瓶颈为：
>
> **第一，能看懂界面，不代表每次都能选对动作、点对位置。** 模型仍可能生成错误坐标、选错工具或采取意料之外的动作；在小众软件、多个应用协同操作时，可靠性也可能下降。前面讲的陌生界面泛化问题，并没有因为某次模型发布就彻底消失。
>
> **第二，能完成任务，不代表完成得足够快。** 官方仍将延迟列为限制，建议优先选择对速度不敏感的场景。结合前面的闭环就不难理解：如果一个任务需要反复截图、推理和执行，即使每一步都能做对，整个过程也可能比人操作更慢。
>
> **第三，模型还不能可靠地区分“需要读取的内容”和“应该服从的指令”。** Computer Use 必须让模型接触网页、文档和图片，但这些内容也可能夹带恶意指令，诱导模型偏离用户目标。难点在于：既要让模型充分利用这些信息完成任务，又要避免它被其中的指令操控。模型训练可以提高抵抗能力，但目前仍不能彻底消除这类风险，因此还需要隔离环境、权限限制和人工确认来兜底。
>
> ---
>
>
> ### 6.6 面试问题
>
> 节选自[2026年10月字节秋招一面](#doxcnz9YY5maMQl28LqNPcrD6Wg)。
>
> 1. **Computer Use 操作本地电脑的大致实现机制是什么？是否了解操作本地电脑和操作云端电脑两种方案？**

💡 答题思路

本地和云端 Computer Use的基本原理相同，都通过“获取界面信息–模型理解并决策–工具执行–观察结果”的循环完成任务。**区别主要在执行环境**：本地方案操作用户电脑，云端方案操作云端的虚拟桌面。截图和鼠标、键盘操作由目标环境中的工具完成，再将结果交回模型。实际部署时，Agent 可以与云桌面一起运行，也可以通过网络连接远端执行器。

🎤 参考答案

我理解，Computer Use 本质上是让 AI 像人一样，通过观察界面、操作鼠标和键盘来完成任务。它的核心是一个“**观察、决策、执行、再观察**”的闭环。

首先，工具获取当前屏幕的截图，或者读取无障碍树，提供按钮、输入框等界面信息；然后，模型结合用户目标和当前状态，决定下一步操作，输出结构化的工具调用，比如点击某个坐标、向输入框填写文字。**这里不是模型直接控制电脑**，而是由外部运行框架，也就是 Harness，调度执行器，把请求转换成实际的鼠标、键盘事件，或者浏览器自动化操作。

执行后，再获取新的界面信息，让模型检查操作是否成功，并决定继续、等待还是纠错，直到完成任务。所以，它和写死步骤的自动化脚本不同，**后续动作可以根据界面的实际变化动态调整。**

具体部署上，可以分成本地和云端两种方案。**本地方案是在用户电脑上运行执行器**，获得授权后截取屏幕、操作本机应用，优势是能直接使用已有的软件、文件和登录状态，但也容易干扰用户操作，需要严格控制权限。

云端方案则是在云端准备虚拟机、虚拟桌面或者隔离的浏览器环境，由那里的执行器完成截图和操作。Agent 可以和环境部署在一起，也可以通过网络调用远端执行器。这种方式更方便隔离、重置和并发运行，但使用本地文件或软件时，需要额外打通访问。

两者的核心原理相同，主要区别是**被操作的环境在哪里，而不是模型在哪里运行**；无论哪种方案，**敏感操作都应该保留人工确认。**

6.7 讲解视频

Computer use 章节对应讲解视频在[这里](https://www.bilibili.com/video/BV1sgnR62EwL/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b)。

后端与服务化【按需深入】

哈喽呀，朋友们，这一部分内容写于2026年9月份。我知道大家对于后端内容的呼声很高，很多人说没写后端的内容，不会后端，很难通过简历。而且大模型应用开发的面试，偶尔也确实会考察一些后端的内容。

之前一直没写后端的笔记，首要原因是：**真的学不过来呀！**给大家算一笔账吧：

1. 先学Agent, RAG 然后各做一个项目。
2. 通用技巧那些也得学一学吧，什么vibecoding的技巧，SKILL之类的。
3. 再学一些算法的东西，模型结构，后训练，推理之类的，因为现实是很多应用岗，有时也会考一下你：会不会微调？transformer的结构之类的。如果你想稍微学深一点，可能还做一个小的后训练项目。

大家看看学完这些，要多久。

要命的是，面试有时还问后端。根据我的经验，后端在大模型开发里面当然重要，因为会考，**但是也不是必要。**我确实一直没学，0后端基础，也拿了很多offer，**所以更新后端的计划一直没有提上日程。**

我这里谈一个现象，现在的大模型岗位中，即使jd上写的是大模型岗位，但是去面试发现很多内容都是后端，最后反问面试官为什么会这样，面试官承认说：**我们的岗位确实是偏后端的。 现在市面上确实有很多挂着大模型皮的，其实主要是后端的岗位。**

在我看来，这里其实谈到了一个取舍的问题：如果你个人可以接受一些后端+AI的不那么纯的AI岗位，那么学好Agent应用的内容+后端，无疑会让你有更多机会。但是如果你想要做纯大模型的岗位，**那么学好应用，推理，训练可能比后端更符合你的发展方向。**

对于我个人，我是**学完了大模型应用，就去学算法，没学后端。**因为我想慢慢往一些底层的岗位：推理，训练岗靠。

当然**你也可以学完应用，再补一下后端，算法内容稍微放一放，这样对于你找LLM应用开发岗帮助可能会更大**。

今天开始陆续更新后端内容了，因为我忽然想到：**不是每个模块都是要系统学习的。** 对于后端这里，如果我时间不够，我可以逐面试问题学习。 我这里的策略是，根据出现的后端内容的面试真题，**对这个题目展开扩展，接着题目讲解他背后的知识点，不系统学习后端内容，以面试真题出发。**

所以这一大块的内容，我的建议是，你根据本章节汇总的面试真题出发，**从题目出发，我讲解题目的时候会讲这个题对应的背后的知识点，把真题刷完，这一小节就学完了。**（之前的小节，学习顺序都是先学知识，然后每小节后面的真题是作为检验的，但这一小节比较特殊哈，**推荐学习顺序反过来了，请大家注意！**）

不过我这一块更新可能不是特别多：原因是我的个人路线规划上，学习后端优先级不高，另外这些很多东西其实都是传统的后端老知识点，我本身也不是后端专家，另外网上后端资料肯定也很成熟。大家特别关注后端内容的，建议自己补充找一些后端的资料。

1 大模型前后端生命周期

对于**没有后端基础的同学**来说，面试要求或技术栈中经常会出现 **Redis、Docker、Kubernetes、分布式系统、API网关、负载均衡** 等词。你可能见过这些名称，却还不清楚：**它们分别解决什么问题？位于系统的哪个位置？彼此之间怎样配合？**

这一节的目标不是让你立刻掌握每项后端技术的所有细节，而是先建立一张完整的**后端地图：知道这些技术**是什么、为什么需要它们、如何在大模型系统中协作。 如果你没有后端基础，花2个小时学习本章节：**能够理解这些技术是什么，它们是如何

> 作用在系统中的，就已经很棒了**！这是我们后续分析面试真题的基础。 如果你连这个技术是干什么的都不知道，你怎么回答相关面试真题？
>
> 更新本小节的原因，来自于字树科技二面：[对 Agent 系统进行压力测试，关注哪些性能指标](#doxcna15PMEIe8LYP21a8JQOEch)？所以建议观看该面试真题的讲解视频，视频会先讲解本小节知识，然后讲解面试真题。
>
>
>
> 上面是本小节的核心生命周期图，理解上图，可以说你就理解了这一小节内容。
>
> 这张图需要从两条**主线**理解：
>
> 1. **部署线**：程序怎样被打包、运行，并在服务器上形成对外服务。
> 2. **请求线**：用户请求怎样经过后端、完成模型推理并返回结果。
>
>
> ### 1.1 部署生命周期：程序如何变成服务
>
> **程序代码**
>
> 系统通常至少包含两类**程序**：
>
> - **Agent 程序**：负责**会话、Prompt 组装、任务规划、工具编排以及调用模型**。
> - **推理程序**：负责加载**大模型**，利用 GPU **执行推理并生成 Token**，例如 vLLM。
>
> 它们可以部署在一起，但生产环境通常会分开，以便**独立扩容和更新**。
>
> **Docker 打包与镜像**
>
> Docker 将以下内容打包成标准**镜像**：
>
> ```Plain Text
> 程序代码 + 依赖库 + 软件运行环境 + 启动命令 = 容器镜像

镜像是尚未运行的**程序模板**。同一个镜像可以在不同服务器上重复使用，从而避免每台服务器都**手动安装环境**。

注意：一般**大型模型权重不放进镜像**，是容器启动后从模型仓库或共享存储加载。因为大模型权重一般比较大，可能几十GB甚至上百GB。**GPU 等硬件以及宿主机驱动**也不属于镜像。

镜像、实例与 Service

```Plain Text 同一个 Agent 镜像 ├── 启动一次 → Agent 实例 A ├── 启动一次 → Agent 实例 B └── 启动一次 → Agent 实例 C

> - **实例**：程序实际运行起来的一份。同一个镜像启动多次，就产生多个**代码相同但运行状态独立的实例**。
> - **Service**：为多个实例提供**统一、稳定的访问入口**，并将请求转发给健康实例。
```

> 实例同时存在两种**关键关系**：
>
> ```Plain Text
> 实例 ──由 Service 提供统一访问入口
> 实例 ──由 Kubernetes 调度到服务器运行
> ```

因此，不是实例与硬件结合后才变成 Service，也不是 Service 被调度到服务器。真正被调度的是**实例。**

Kubernetes 与硬件服务器

服务器是提供 CPU、内存、GPU、显存等**资源**的机器。Kubernetes 根据实例声明的**资源要求**，将实例安排到合适的**服务器**：

```Plain Text Agent 实例 → CPU 服务器 推理实例 → GPU 服务器

> Kubernetes 还**负责**：
>
> - **保持所需实例数量**；
> - **流量增加时扩容，流量下降时缩容**；
> - **实例或服务器故障后重新创建实例**；
> - **发布新版本时滚动替换实例**。
>
> 程序因此不固定绑定某台具体服务器，但仍然依赖某类硬件。例如推理实例可能要求**特定 GPU 和足够显存**。
>
> **集群与分布式系统**
>
> - **服务集群**：同一个程序运行出多个实例，共同提供一种**服务能力**。
> - **服务器集群**：多台服务器组成的**硬件资源池**。
> - **分布式系统**：不同服务和实例运行在多台服务器上，通过网络分工协作，共同完成**用户请求**。
>
> 例如，Agent 实例运行在 CPU 服务器，推理实例运行在 GPU 服务器，状态存放在 Redis 或数据库中；它们通过网络通信，整体构成**分布式大模型后端**。
>
> <a id="doxcnzulABSpB9iELbZNO6QHsZY"></a>
> ### 1.2 请求生命周期：用户请求经历什么
>
> 主链路可以概括为：
>
> ```Plain Text
> 客户端
> → API 网关
> → 负载均衡
> → Agent Service / Agent 实例
> → 模型推理 Service / 推理实例
> → GPU 计算
> → 结果返回客户端
> ```
```

客户端

客户端通常是网页或 App，负责**接收用户输入、向后端发送请求**，并展示**普通或流式结果。**

API 网关

API 网关是服务端**统一入口**，通常负责：

- **认证与权限校验**；
- **限流和安全检查**；
- 根据**请求路径选择目标服务**；
- 记录**入口日志和请求标识。**

网关回答的是：**请求能不能进入，应该进入哪个服务?**

负载均衡

一个 Service 后面可能有多个相同实例。负载均衡负责选择一个**健康且适合接收请求的实例**：

```Plain Text 请求 1 → Agent 实例 A 请求 2 → Agent 实例 B 请求 3 → Agent 实例 C

> Nginx、云负载均衡、Kubernetes Service 等都可以实现**负载均衡能力**。
>
> > **负载均衡决定请求交给哪个实例；Kubernetes 调度决定实例运行在哪台服务器。**
> >
> >
> >
> > **Agent Service 与 Agent 实例**
> >
> > Agent 实例主要**负责**：
> >
> > - **保存或读取会话**；
> > - **组装 Prompt**；
> > - **决定任务执行流程**；
> > - **调用模型、工具、Redis 和数据库**；
> > - **整理最终结果**。
> >
> > 它主要运行**编排和业务逻辑**，通常以 CPU、内存为主要资源。
> >
> >
> >
> > **模型推理 Service 与推理实例**
> >
> > 推理实例接收 Agent 发来的 Prompt，加载模型并利用 GPU 执行计算，再逐 Token **生成结果**。
> >
> > ```Plain Text
> > Agent 实例：决定任务怎样完成
> > 推理实例：负责运行模型并生成 Token
> > GPU 服务器：提供实际计算资源
```

一个 Agent 任务可能**多次调用推理服务**，也可能在模型判断需要工具后**调用外部工具**，再把工具结果交给模型继续推理。

Redis 与持久化数据库

多个 Agent 实例不应把重要**共享状态**只放在某个实例内存中，否则请求切换实例或实例故障后，状态可能丢失。

- **Redis**：常用于**缓存、Session、共享会话状态、临时任务进度、限流计数和分布式锁。**
- **MySQL/PostgreSQL**：通常用于**长期保存用户、聊天记录、任务和核心业务数据。**

Redis 也支持持久化，但在这张图里主要表示**高速缓存和共享临时状态**；关系型数据库主要表示**长期持久化存储。**

单台服务器故障时会发生什么

```Plain Text
健康检查发现异常 → 负载均衡停止向故障实例发送新请求 → 其他健康实例继续处理请求 → Kubernetes 在其他合适服务器上重新创建实例 → 新实例通过健康检查后重新加入 Service
```

> 真正实现**高可用**还要求：
>
> - **提前部署多个实例，消除单点**；
> - 将重要状态放到**外部共享存储**；
> - 为剩余实例预留**故障转移容量**；
> - 对执行中的请求设置**超时、有限重试和幂等保护**。
>
> ---
>
>
> ### 1.3 系统关键指标
>
> 这一小节，就是直接讲解面试真题 [对 Agent 系统进行压力测试，关注哪些性能指标](#doxcna15PMEIe8LYP21a8JQOEch)？ 为了讲这个真题，我先讲解了整个大模型的生命周期，这样你更好理解每一个指标，在整个系统中处于什么位置。
>
> 这里采用：
>
> > **Google SRE 四个黄金信号 + LLM/Agent 特有指标。**
>
> **Google SRE 的四个黄金信号**原本用于监控生产环境中的**在线服务**：
>
> 1. **延迟（Latency）**：用户需要**等待多久**？
> 2. **流量（Traffic）**：系统当前有多少**工作要处理**？
> 3. **错误（Errors）**：有多少工作**没有正常完成**？
> 4. **饱和度（Saturation）**：系统距离**资源上限**还有多远？
>
> Agent 系统上线后同样是**在线服务**，因此可以使用这个框架；但还要加入 **Token 生成、GPU 资源、多步执行、工具调用、任务质量和成本**等**特点**。

> 四者通常存在如下**关系**。下面每一步都对应一个黄金信号：
>
> ```Plain Text
> 流量（Traffic）增加
> → CPU/GPU、连接池和队列逐渐饱和【饱和度 Saturation】
> → 排队时间变长，响应延迟上升【延迟 Latency】
> → 最终出现超时和请求失败【错误 Errors】
> ```

也就是说：系统先承受更多**流量**，资源随后接近**饱和**；饱和会造成排队，使**延迟**升高；延迟继续恶化，最终产生超时和**错误**。

压测时应先观察这四类**高层信号**；发现异常后，再结合生命周期图、日志和 Trace 定位具体发生在哪一层。生命周期图是**定位地图**，四个黄金信号是**组织指标的框架**。

1.3.1 延迟

延迟表示请求或任务需要**等待多久**。

主要指标

端到端任务耗时：从客户端发出请求到收到最终结果，覆盖**整条请求链路**。

P50/P95/P99：分别观察**典型请求和尾部慢请求**，不能只看平均值。

TTFT（Time to First Token）：从发出请求到收到第一个 Token 的时间，即**模型多久开始输出**。

TPS（Tokens Per Second）：模型**每秒生成多少 Token**。

TPOT（Time Per Output Token）：模型平均**生成一个输出 Token 需要多久**。

> 关于这个**TTFT 和 TPS, TPOT**。 是大模型推理方向的一些技术，涉及一些推理的知识。如果你去学我们算法项目，推理那一块，会讲**prefill, decode**。包括我想到一个面试真题（好像是滴滴一面），面试官问，你说transformer相比于传统的网络，可以并行计算，速度快。但是decode阶段无法并行啊，你如何看到他prefill阶段可以并行，decode阶段不能并行的问题。这些是推理这一块的知识，感兴趣可以自己琢磨一下。

TTFT、TPS 和 TPOT 主要对应图中的**模型推理链路**：

```Plain Text 模型推理 Service → 推理实例 → GPU 服务器

> 原因是大模型先处理输入，再**自回归地逐 Token 生成输出**：
>
> ```Plain Text
> TTFT：模型多久开始说话
> TPS/TPOT：开始说话后说得多快
> 端到端延迟：整个 Agent 任务多久完成
> ```

若端到端 P99 很高，但 TTFT、TPOT 正常，则**瓶颈**可能位于 Agent 排队、数据库或其他依赖中，需要沿图继续定位。
```

1.3.2 流量

流量表示系统正在承受多少**请求或工作量。**它不是某一台服务器独有的指标，而是沿**整条链路**传递：

```Plain Text
客户端 → 网关 → Agent Service → 推理 Service → Redis/数据库
```

> 不同位置可使用不同**单位**：
>
> - **网关**：用户请求数、RPS、并发连接数；
> - **负载均衡**：接收和转发的 RPS、活跃连接数，以及流量在各实例间的分布是否均衡；
> - **Agent 服务**：并发任务数、每秒成功完成任务数；
> - **推理服务**：模型请求数、输入/输出 Token 吞吐量；
> - **Redis/数据库**：每秒读写或查询次数。
>
> Agent 还会造成**内部调用放大**：
>
> ```Plain Text
> 100 个用户任务 × 每任务 3 次模型调用
> = 推理服务约 300 次模型调用
> ```

因此不能只看入口 QPS，还应考虑每任务的**模型调用数、工具调用数和执行步数。**

1.3.3 错误

错误表示请求或任务没有按要求**正常完成**，包括：

- **网关或服务返回 4xx、5xx**；
- **请求超时**；
- **模型调用失败或被限流**；
- **工具、Redis、数据库调用失败**；
- Agent 达到**最大步数或陷入无效循环**；
- HTTP 返回 200，但 Agent 没有真正**完成任务。**

错误可能发生在整条链路的任何位置，需要通过**日志和 Trace**定位。

Agent 尤其要关注**任务成功率**：答案是否正确、工具是否正确执行、任务是否真正达到目标。

1.3.4 饱和度

饱和度表示系统距离**资源上限**有多远，也就是“还能不能继续扛”。主要对应图中真正消耗资源的部分：

- **Agent 实例和 CPU 服务器**：CPU、内存、并发槽位、任务队列；
- **推理实例和 GPU 服务器**：GPU 利用率、显存、KV Cache、推理队列；
- **Redis**：内存和连接数；
- **MySQL/PostgreSQL**：连接池、CPU 和磁盘 I/O。

系统不一定要等 CPU/GPU 达到 100% 才算饱和。接近**容量上限**时，**队列长度和 P99 延迟**往往会先快速上涨。

最后提醒一下，系统关注的指标，当然还有Agent专属的各种指标，比如**任务成功率，轨迹，任务质量**等等，不过整个不属于整个压力测试，后端性能领域，更偏向于Agent评估，我就不展开了，这块的内容看一下：

---

2 面试真题

2.1 如何对 Agent 系统进行压力测试，以及要关注哪些性能指标?

选自202607宇树科技二面. 讲解这个题之前，我们是先讲解了整个大模型前后端生命周期，然后将关键指标结合生命周期讲解，请先阅读笔记：大模型前后端生命周期部分。

💡 答题思路

思路就是我们笔记大模型前后端生命周期部 章节介绍的Google SRE 准则，从延迟，流量，错误，饱和度四个方面出发，结合传统后端的指标和大模型相关的指标来回答。

📝 参考答案

我会先根据真实业务定义 SLO（服务目标），例如任务成功率、P95/P99 延迟、最大并发量和单个成功任务的成本。然后使用接近真实用户的任务组合，**逐步增加并发，并补充突发流量、长时间运行和故障注入测试**，找出系统在满足 SLO 时的最大稳定容量。

指标框架采用 Google SRE 的四个黄金信号，并融入 LLM/Agent 特有指标：

- **延迟**：关注端到端任务耗时及 P95/P99；模型侧补充 TTFT（首 Token 延迟）、TPS/TPOT（Token 生成速度）。
- **流量**：关注并发用户数、RPS、每秒成功完成的任务数和 Token 吞吐量。由于一次 Agent 任务可能多次调用模型和工具，还要关注每任务的模型调用数、工具调用数和调用放大情况。
- **错误**：关注请求错误率、超时率、模型和工具调用失败率，以及 Agent 任务成功率。HTTP 200 不代表任务真正完成，因此还要判断答案或工具执行结果是否正确、是否达到最大步数或陷入无效循环。
- **饱和度**：关注 CPU、内存、GPU、显存、请求队列、数据库连接池和模型 API 配额，判断哪个模块最先接近容量上限。

Agent 系统还要额外关注**任务质量**和**成本**，例如正确完成率、输入/输出 Token 数及每个成功任务的成本。

压测时，我会先观察这四类高层指标；发现异常后，再结合调用链 Trace 和系统架构，定位问题是在网关、Agent 服务、模型推理服务、CPU/GPU，还是 Redis、数据库等依赖。同时验证限流、负载均衡、自动扩缩容和故障恢复是否有效。

最终目标不是找到系统勉强不崩溃时的最高 QPS，而是找到在任务质量、延迟、错误率和成本都满足 SLO 时的最大稳定容量。

2.1.1 讲解视频

https://www.bilibili.com/video/BV1YBtf6bE9Y/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

2.2 文档更新：并发与幂等

真题来源：2026年10月字节一面 · 第3组原题

主问题：文档在线自动更新时，如何保证并发更新的幂等性，避免多个任务相互覆盖？

💡 答题思路

这个问题核心考察的是通用后端能力，具体包括同一个任务重复执行时的幂等性，以及不同任务并发修改同一文档时的冲突控制，和大模型只是关系不大。

我个人的学习路线不会系统学后端，但我希望能够通过面试题，大概了解后端，该背的八股顺便背了：对于大模型问题，尽量往深了讲、讲透；对于后端问题，先就题论题，把相关知识点、基本原理和适用条件弄清楚，再整理成能在面试中讲明白的答案，不展开完整的后端技术体系。

从后续追问看，同学的项目或回答中应该涉及 中间件RocketMQ。这里不与具体中间件强绑定，而是分别讨论“同一次更新如何避免重复生效”和“不同更新如何避免互相覆盖”，后面的 RocketMQ 选型与使用细节暂不展开。

具体到下面这份参考答案，**幂等性采用“唯一业务任务标识 + 处理状态记录 + 原子去重与提交”来保证**，不能只查一次状态就认为不会重复执行；**并发覆盖采用版本号乐观锁来防止**，允许任务并行准备修改，但提交时原子校验版本，冲突后基于最新内容重新生成或安全合并，再尝试提交。

🎤 参考答案

我会把这个问题拆成两部分：**同一个任务重复执行的幂等性，以及不同任务并发修改时的一致性。**

第一，用唯一任务标识和原子提交，避免同一次更新重复生效。 每次业务更新都有一个唯一标识，超时重试时沿用这个标识，而不是重新生成。提交时，通过唯一约束或等价的原子去重机制，保证同一个任务只能成功生效一次；如果已经成功，就返回已有结果，不再重复修改。文档更新和任务成功记录需要在同一个事务或等价的原子机制中提交，避免文档已经更新、成功状态却没记下来，导致重试时重复更新。不能只在开始时查询一次“有没有处理过”，因为两个重复请求可能同时查到没有，再各自执行。

第二，用版本号做乐观并发控制，避免不同任务互相覆盖。 任务读取文档时，同时记录版本号。多个任务可以基于各自读到的内容并行准备修改，但保存时必须原子地校验：当前版本是否仍等于读取时的版本。比如 A、B 都读取了 V1，A 先保存成功，版本变为 V2，B 再提交时就会因版本不一致而被拒绝。此时 B 应重新读取 V2，基于最新内容重新生成或安全合并自己的修改，然后再次校验、提交，而不是把旧结果换上新版本号直接覆盖。重试也要限制次数，多次冲突后可以转为排队或人工处理。

如果冲突比较频繁，还可以让同一篇文档的自动更新任务排队执行，不同文档之间并行处理。但排队只是减少冲突，不能替代幂等和版本校验，尤其文档还可能被人工编辑或其他服务修改。

总结：唯一任务标识保证“同一次更新不重复生效”，版本号校验保证“不同更新不互相覆盖”。这里的版本校验就是乐观锁思想，不需要把整个耗时的 AI 生成过程一直锁住。

2.3 数据库类型与MySQL索引

真题来源：2026年10月字节一面 · 第8组原题

💡 **答题思路**

这就是一个典型的纯八股，我就归类为后端/数据库方向的纯八股。反正对于我个人来说，掌握知识就两种方法：要么你系统学习，理解，掌握，不背也能说出大概。像Agent这一块的内容，我这么处理。 对于这种纯传统八股，没时间系统学习，出现频率也不是那么高，我就是纯背。所以这个内容没啥讲的，AI总结一下答案，我自己背一背。

数据库类型：

1. 常见数据库有哪些类型?

🎤 **参考答案**

常见数据库可以先分成两大类：**关系型数据库和非关系型数据库，也就是 NoSQL。**

关系型数据库主要有 MySQL、PostgreSQL、Oracle。它们通过表、行、列来组织数据，支持 SQL 查询和事务，比较适合订单、支付、用户信息这些结构明确、对数据一致性要求比较高的业务。

非关系型数据库又可以按数据模型分成几类：

- **键值数据库**，比如 Redis，适合缓存、计数器、会话存储。
- **文档数据库**，比如 MongoDB，以类似 JSON 的文档存储数据，结构比较灵活。
- **宽列数据库**，比如 HBase、Cassandra，适合海量数据和分布式读写。
- **图数据库**，比如 Neo4j，擅长表达和查询复杂关系，比如社交关系、知识图谱。

另外还有面向特定场景的数据库，比如存储监控指标的时序数据库，以及用于语义检索的向量数据库。

实际项目里通常不是只选一种，而是组合使用，比如 **MySQL 保存业务数据、Redis 做缓存、向量数据库支持语义检索。**

2. 不同类型数据库之间有什么区别?

🎤 **参考答案**

我一般从 **数据怎么组织、怎么查询，以及事务和扩展能力** 这几个方面来区分。

首先是数据组织。关系型数据库按表存储，通常需要提前定义字段和约束；文档数据库结构更灵活，不同文档可以有不同字段；键值数据库主要根据 key 访问 value；图数据库则围绕节点和关系组织数据。

其次是查询方式。关系型数据库擅长 SQL、关联查询和复杂统计；键值数据库擅长按 key 快速读写，但不适合复杂关联；图数据库则更适合查询多层关系，比如“朋友的朋友”。

再就是事务和扩展。关系型数据库通常提供成熟的事务与约束机制，适合保证业务数据正确性。不少 NoSQL 数据库更侧重特定访问模式和水平扩展，但不能简单理解成“关系型不能扩展”或者“NoSQL 不支持事务”，具体要看产品和配置。

所以选型的关键不是谁更先进，而是 **业务数据是什么结构、主要怎么查、对一致性和规模有什么要求。** 比如订单优先考虑关系型数据库，缓存用 Redis，复杂关系查询考虑图数据库。

1. MySQL 底层索引使用什么数据结构?

🎤 **参考答案**

严格来说，MySQL 的索引结构和 **存储引擎、索引类型** 有关。面试里通常说的是默认存储引擎 **InnoDB，它的普通索引和主键索引主要使用 B+ 树。**

B+ 树是一种多路平衡查找树。非叶子节点主要存储索引键和子节点指针，用来导航；叶子节点存储实际的数据记录或者对应的定位信息，叶子之间按顺序连接，方便范围查询。

在 InnoDB 里，还需要区分主键索引和二级索引。

主键索引就是聚簇索引，叶子节点存储完整的行数据。二级索引的叶子节点存储索引列和对应的主键值。

所以，通过二级索引查到主键后，如果还需要其他不在索引里的字段，就要再去主键索引查一次，这个过程叫 **回表。**如果查询需要的字段都能从二级索引里拿到，就属于 **覆盖索引**，可以避免回表。

其他类型也有例外，比如 MEMORY 引擎支持哈希索引，全文索引主要采用倒排索引结构。因此不能笼统地说，MySQL 所有索引都是 B+ 树。

2. 为什么 MySQL 索引使用 B+ 树，而不是其他树结构?

🎤 **参考答案**

核心原因是：**B+ 树很适合数据库按页访问磁盘的方式，同时兼顾单点查询和范围查询。**

首先，和二叉搜索树、AVL 树、红黑树相比，B+ 树一个节点可以容纳很多索引键，分支更多，所以树更矮。数据库索引通常很大，不能全放进内存，树越矮，查询需要访问的页面通常就越少，磁盘 I/O 成本也就越低。

其次，和 B 树相比，B+ 树的非叶子节点不存储完整的行记录，可以在同样大小的页面里放下更多索引项，进一步增加分支数、降低树高。

另外，B+ 树的叶子节点有序连接。像查询某个时间段的订单，只需要先定位起点，再沿叶子节点顺序扫描，范围查询和按索引顺序取数都比较高效。

如果和哈希索引比较，哈希虽然适合等值查询，但不擅长范围查询，也不能直接利用哈希顺序完成排序。

所以，选择 B+ 树不是因为它在每一种查询里都最快，而是因为它在 **磁盘 I/O、等值查询、范围查询和维护成本之间，取得了比较好的平衡。**

3 Python 的异步方法是如何实现的?

（其实Python语法在llm 的岗位问的不多，我也没有专门准备，但是这个问题确实是出现在了好几次面试中，所以我把它整理在这里，还是建议掌握这个问题。这个其实涉及到了**协程**以及具体的语法，第一次接触的朋友理解起来应该是会有点费劲的，不要着急寻求一次就能弄懂）

Python 中最常见的方法是使用asyncio库，使用async 定义异步函数，然后在异步函数中使用await 来等待另一个异步操作完成。

3.1 异步方法如何通知事件完成?

（作为上一个问题的延伸问题，这个也是京东二面被问到，我们在这里一起复习了）

1. 使用await 返回结果

2. 使用Future/Task的set_result 或 回调

应用开发项目与开源架构解析:[理论学完后学习]

项目直达：Agent 项目实战 | RAG 项目实战 | 后训练项目（可选第二阶段）

1 提升简历通过率，做深项目

哪些项目值得做，项目里的哪些地方值得做？如何让你的项目看上去更值钱，避免被面试官说浅。看完了这个章节，你也自己有一些想法，如何能够改进，做深项目，简历如何包装和修改。我目前想到的两个专题：

1. **2026值得做的AI项目/方向，如何做得深入，从Demo变成生产级应用?**
2. **如何将后端技术融入到简历中，和大模型Agent项目结合。**

了解了这些，对于你把控如何做深项目，如何包装简历，自己会有一定理解。

1.1 2026年真正有价值的AI项目

如果你想在 2026 年进入 AI 工程领域，或者想在目前的基础上进一步提升，下面这六个项目值得认真投入——当你坐在面试官面前时，它们会真正产生作用。

这里说的**不是那种跟着教程做一遍、部署一个基础聊天机器人、然后写进简历的项目**，而是这样一种作品：别人看过你的 GitHub 后会觉得，"好，这个人确实理解生产环境中的 AI 系统是如何工作的"。

关于做什么项目的建议往往让人无所适从，因为每个人观点都不一样，很难判断哪些内容真正重要、哪些只是噪声。因此这里把它归纳成六个项目，每个项目都对应**一种独特且市场需求旺盛的技能。**下面会逐一说明具体应该构建什么、使用哪些工具，以及更重要的——**为什么每个部分都很重要。**

这个小节的笔记，文字里我也一起总结了非常多的官方的参考文档，项目的链接。肯定读不完，按**需要阅读。**

下面介绍的6个方向，还是针对于大模型开发岗位来说的，如果要我排优先级：

Agent项目是必做的。

RAG重要性次之，但是几乎也一定要会的，监控和可持续性等于加分项，提升你这两个项目的竞争力。

LoRa和DPO强化学习属于**加分项，非必要项**，有的应用岗也会问。

多模态是**可选项和加分项**，做了以后可以去投递一些多模态方向大模型岗位。

1.1.1 生产级 RAG 系统

第一个项目是**生产级**的检索增强生成系统，也就是 RAG。它被频繁提及是有原因的：RAG 确实是目前企业 AI 中最常见的模式之一。

但必须明白，**RAG 演示程序与真正可以投入生产的 RAG 系统之间存在巨大差距**，而这个差距正是你能够体现自身区别的地方。

你要构建的是一个面向特定领域的 "Ask My Docs（询问我的文档）" 系统。选择一组文档，例如技术文档、研究论文、法律合同或医疗资料，然后构建一个能够检索正确信息、依据资料回答问题并给出引用的系统。

这里的关键词是**引用**。任何人都可以让大语言模型生成一个听起来可信的答案，但**只有把答案建立在实际检索到的证据上，系统才值得信任。**

1.1.1.1 第一阶段：完成基础功能

首先导入文档，包括 PDF、Markdown 或网页。然后把文档切成每块约 **500～800 Token** 的片段，相邻片段之间保留约 **100 Token 的重叠**。

重叠很重要，因为你不希望在边界处刚好切断一句重要的话，从而丢失上下文。接着，把这些片段转换成 Embedding，存入向量数据库。你可以从 Chroma 或 Weaviate 等工具开始。

然后构建检索流程：针对用户问题，取回相关度最高的 Top-K 文档片段，再生成答案，并标明信息来自哪里。

这一阶段的成果很直观：你可以向别人展示一份文档、系统生成的答案，以及答案所依据的准确段落。

> 延伸阅读：LangChain 官方 RAG 教程，一步步教你构建检索链。如果你是 LangChain / LangGraph 的新手，可以参考这个深入工作坊；

1.1.1.2 第二阶段：从演示升级到生产质量

这是大多数人从未做到的部分，因此也格外有价值。

首先实现**混合检索**，也就是把传统的 **BM25 关键词搜索**与**基于向量的语义搜索结合起来**。向量搜索擅长理解含义和意图，但有时用户会搜索某个非常具体的术语或短语，而 BM25 对这种情况处理得很好。

在此基础上加入 **Cross-Encoder 重排序器**。它接收第一轮检索得到的文档片段，把问题和每个片段作为一对输入重新评分。这样通常能够显著提高检索结果的精确度。相关原理可以参考 Cohere 的 Rerank 指南，它解释了 Cross-Encoder 如何工作、以及为什么能提升检索精度。

还应该实施**引用约束**：如果检索到的内容不足以支持某个明确答案，**系统必须拒绝回答**，而不是编造一个听起来合理的说法。

此外，把所有 Prompt 保存在带版本的配置文件中。**Prompt 是系统架构的一部分**，像管理代码一样管理它们，体现了真正的工程成熟度。

1.1.1.3 第三阶段：让系统真正具备交付能力

人工整理一套包含大约 **50～200 个问答对的黄金评测集**，并确认每条答案都正确。然后编写离线评测脚本，测量 **Faithfulness（忠实度）**，也就是生成答案中的主张是否真的得到检索片段的支持。

把评测**接入持续集成流程**，让每个 Pull Request 都自动运行评测。**如果质量低于设定的门槛，就让构建失败。**

这正是生产 AI 团队的工作方式。在项目中展示这种纪律，会立即让面试官看到你理解 AI 系统的完整生命周期。

技术栈建议

流程编排：**LangChain** 或 **LangGraph**

向量数据库：**ChromaDB** 或 **Weaviate**

重排序：**Cohere Rerank**，或 Sentence Transformers 中的 **Cross-Encoder** 模型

评测框架：Ragas，专为评估 RAG 而设计，可衡量忠实度、答案相关性与上下文精度

1.1.2 完全离线的本地小模型助手

第二个项目是使用小语言模型构建一个完全离线运行的本地 AI 助手。这个项目的重要性常常被低估。

你可能会问：**既然可以调用 GPT-5 这样的强大 API，为什么还要在本地运行能力较弱的小模型？**答案是，**现实中有许多情况不能或不应该把数据发送给外部服务。**

你需要考虑隐私法规；某些延迟要求不允许网络往返；大规模使用时会受到成本限制；在边缘设备上也不能保证有互联网连接。**企业非常重视这些限制，但多数候选人几乎没有处理这些问题的实际经验。**

1.1.2.1 第一阶段：部署并测量

首先安装 Ollama，它是让开源模型在笔记本电脑上运行的最简单方式之一。下载一个 3B ~ 7B 参数规模的模型，例如 Llama 3.2、Phi-4 或 Mistral 7B，然后为它制作命令行工具，或者用 FastAPI 封装成一层可用的接口。

第一阶段最重要的是**测量。**严格记录模型的推理性能，包括**每秒生成 Token 数**、**首 Token 延迟以及总响应延迟。**把这些数据写进项目文档，因为它们能够说明本地推理在现实中的取舍。

1.1.2.2 第二阶段：加入结构和确定性

要求模型按照指定 JSON Schema 输出，并使用 Pydantic 验证结果。如果输出不合法，就捕获错误并重新提示模型一次；如果仍然失败，则优雅地返回错误。

"约束生成 + 验证 + 重试"是生产系统中经常使用的模式，但在个人项目里却很少见。工具方面，Instructor 是业界标准，专门用于在 LLM 输出上强制 Pydantic Schema 并优雅地处理重试。

还应该研究 Temperature 参数。对同一批 Prompt，**分别以 Temperature 0 和 0.7** 运行，比较输出差异。这表明你理解**语言模型具有随机性，也知道在可靠性重要时如何控制它。**

1.1.2.3 第三阶段：进行模型对比研究

选择三个模型，例如 **Llama 3.2 3B、Phi-4 Mini 和 Mistral 7B**，在完全相同的硬件上测试它们。

比较内存占用、每秒 Token 数，以及它们在一套标准化的 30 ~ 50 条测试 Prompt 上的输出质量。把实际数据和分析写成简洁的技术报告。

模型选型是每个 AI 团队都要经常面对的决定。**用系统性数据做选择，而不是采用社交媒体上最流行的模型，会让面试官看出你具有工程师思维。**

如果想更进一步，可以测试这些模型的 GGUF Q4、Q5 等量化版本，并记录**质量与速度**之间的取舍。Hugging Face 的量化指南详细讲解了 GGUF、Q4、Q5 等格式如何在不严重损失质量的前提下压缩模型。这种严谨程度会让你的项目明显不同，也能帮助你真正理解 AI 工程的基础。

1.1.3 监控与可观测性

> 第三个项目几乎没有人会去做，这正是建议你做它的原因。把项目一中的 RAG 应用拿出来，为它添加全面的监控和可观测性层。
>
> 在生产环境中，构建初始系统可能只占 30% 的工作。剩下 70% 是了解系统是否正确运行、失败时理解原因，以及迅速诊断和解决问题。
>
> 如果你能展示这种系统思维，就会立刻区别于那些只会"把东西做出来"的候选人。我们的笔记[Agent评估](#doxcnbpKwJs393fsRaDVMYRg89c) 章节有更**加详细的工程化方法讲解如何去做Agent评估**。包括[RAG项目](#doxcncSt7IV6HwH7uqDznu6YXXd)中，也做了阶段1阶段2。

1.1.3.1 第一阶段：追踪每一个步骤

> 为 RAG 流程中的每一步加入 **Trace**。对于每个请求，你应该能够看到：**检索到了哪些片段、重排序器怎样改变了它们的顺序**、发送给大模型的 Prompt 是什么、模型返回了什么，以及使用了多少 Token。

Langfuse、LangSmith 和 Braintrust 都可以帮助你完成这些工作。对于刚开始的人，尤其推荐 Langfuse，因为它是开源的，也可以自行托管，便于持续实验和迭代；它文档中的 "Tracing" 与 "Scores/Evaluations" 两部分正是这个项目需要的内容。LangSmith 则是 LangChain 原生的可观测性工具，很适合用来理解 Prompt 版本管理与链路追踪。

> 如果想系统性补齐评测（Evals）的知识，可以参考这个[AI Evals 工作坊](https://topmate.io/the_gen_academy/1962949)，其中包含完整概念讲解和动手实验。

1.1.3.2 第二阶段：长期追踪质量指标

> 这时你要开始像 AI 领域的站点可靠性工程师一样思考。
>
> **测量 P50 和 P95 延迟**，**而不只是平均延迟**，因为平均值经常掩盖最差的用户体验。记录每个请求的成本，让你能够用金额衡量每次查询的花费。
>
> **测量引用覆盖率，也就是有多少回答真正建立在检索证据上。同时监控失败率**：系统多频繁报错，或者生成它无法用资料支持的答案。
>
> 你最终应该能够做到：当别人问"**上周二系统质量下降时发生了什么"，你可以打开仪表盘，指出异常，并解释根本原因**。

1.1.3.3 第三阶段：建立回归质量门禁

> 把项目一中的**评测集接入持续集成流程**。如果忠实度或其他关键指标低于阈值，构建就会失败，该改动不能合并。
>
> 同时，对 Prompt 和配置文件进行版本管理，因为 Prompt 改动对系统行为的影响可能和代码改动一样大。
>
> **这是生产 AI 团队每天都在使用的工作纪律，在个人项目中却非常少见，因此非常有说服力**。关于生产 AI 系统如何被架构、监控和评估，Eugene Yan 的 [《Patterns for Building LLM Systems》](https://eugeneyan.com/writing/llm-patterns/) 是一篇必读文章。
>
> 这个项目可能不像 AI 图像生成器或带有漂亮前端的聊天机器人那样吸引眼球，但它传递了雇主非常重视的一点：你会从整体上思考系统，而不只是关注中间的模型。很多 AI 团队目前迫切需要的正是这种系统级思维。

1.1.4 使用 LoRA 和 DPO 微调

第四个项目是**模型微调**。关于什么时候以及为什么进行微调，业界存在许多误解。

微调不是让模型在所有方面都变得更聪明，而是让模型在一个定义清楚、即便认真设计 Prompt 仍然表现不足的特定任务上，持续保持优秀表现。

在微调之前，**你必须找到一个能够清楚证明差距的任务：先展示基础模型配合精心设计的 Prompt 所能达到的最好成绩，再用指标展示微调后的提升。**

强烈建议选择**两类任务之一**：从混乱的非结构化文本中提取结构化 JSON，或者提高工具调用准确率——让模型选择正确函数，并填写正确参数。这些任务中，微调往往会产生清楚且可以量化的提升。

1.1.4.1 第一阶段：监督微调

首先使用 LoRA 进行监督微调。准备大约 2,000～10,000 条干净数据。"干净"非常重要，因为数据质量远比数量重要。如果**训练样本不一致或格式很差，你实际上是在教模型输出不一致、低质量的结果。**

使用 LoRA 或 QLoRA 进行参数高效微调，这意味着你不需要庞大的 GPU 集群。一张 A100，甚至使用 QLoRA 的 T4，都可能完成任务。基础模型可以选择 Qwen3 8B 这一量级的开源模型。

在独立保留的测试集上使用具体指标评估，例如 JSON 有效率、完全匹配准确率以及正确拒答率——也就是当模型不应该回答时，是否会正确拒绝。

可用的 SFT 数据集：

- HuggingFaceH4/ultrachat_200k——高质量多轮对话数据，从 ShareGPT 风格日志中筛选而来，被开源指令模型广泛使用；
- nvidia/Nemotron-Instruction-Following-Chat-v1——NVIDIA 出品的大规模、高质量、可商用指令遵循与对话数据集，结合了经验证器筛选的助手对话与结构化输出样本；
- openai/gsm8k——8.5k 条高质量、语言多样的小学数学应用题，用于测试多步推理能力。

代码类数据集：

- nvidia/OpenCodeInstruct——500 万条多样化的 Python 指令-响应对，包含题目、解法、测试、执行反馈与质量判定；
- OpenCoder-LLM/opc-sft-stage1——聚焦代码理解与生成的多源精选 SFT 数据混合，用于训练 OpenCoder 系列模型。

1.1.4.2 第二阶段：偏好优化（DPO）

这一阶段用于证明你的理解超越了**基础监督微调。**

不只是告诉模型正确输出是什么，还要向它展示比较结果：对于同一个 Prompt，一个输出比较好，另一个比较差，让模型学习两者之间的区别。

为每个 Prompt 生成多个输出，标注哪些更好、哪些更差，再使用 DPO 风格的偏好优化方法训练。随后重新在测试集上评估，展示它相对于监督微调基线的进一步提升。

这能够证明你熟悉现代对齐和训练方法，而这些方法正在行业内被实际使用。

推荐的偏好数据集：argilla/distilabel-intel-orca-dpo-pairs——基于 Intel/orca_dpo_pairs 精选、清洗并增强的偏好数据集（prompt / chosen / rejected），适合做 DPO 对齐训练。

1.1.4.3 工具与呈现

这些微调阶段可以逐层叠加。工具方面，**Hugging Face 的 TRL 库能够很好地处理 SFT 和 DPO；Axolotl 也能减少大量配置工作。如果不想管理云端基础设施，可以使用 Fireworks AI 这类 GPU 计算平台。**

在项目说明中展示**训练曲线，清晰列出训练前后的指标，并诚实说明过程中出了什么问题、你如何调整。面试官很喜欢看到候选人能够排查问题并适应变化，因为真实的日常工作就是如此——事情不会总是完美，你必须不断调整。**

1.1.5 实时多模态应用

第五个项目是实时多模态应用。这个项目让**你展示自己能够处理流式数据、严格的延迟预算，以及必须实时响应的系统固有的复杂性**，而不是只处理舒适的离线批任务。

你可以根据兴趣选择三条路线之一：

1. **语音助手**：接收音频，通过自动语音识别把语音转成文字，把转写文本交给语言模型推理，再通过文字转语音把回答转换成声音。
2. **计算机视觉管线**：把摄像头视频流交给目标检测模型，再让语言模型根据检测到的内容进行分析。
3. **流式日志分析器**：实时接收服务器日志，进行异常检测，然后让语言模型生成人类容易理解的故障说明。

如果只能选一条，推荐语音助手，因为语音 AI 正在快速增长，相关工具也已经成熟。**语音识别**可以使用 **Deepgram 或 Whisper**，推理层可以使用任何合适的大模型，语音合成可以使用 **ElevenLabs 或 Cartesia。WebSocket** 很适合编排整个流程。

> 推荐资源：

- 编排框架：Pipecat AI——专为构建实时语音与多模态 AI 智能体而生的开源框架，帮你处理好 WebSocket 与延迟追踪；
- 语音识别（ASR）：Deepgram API 文档，包含优秀的流式音频转写指南；
- 语音合成（TTS）：ElevenLabs WebSocket API 文档，专门讲解如何用 WebSocket 把生成的语音即时流式返回给用户。

1.1.5.1 打通端到端流式管线

先专注于让系统从头到尾正常工作，不需要立刻优化。目标是让数据能够持续流动，正确定义事件结构，并让语言模型对实时输入作出响应。仅仅完整打通这条链路就已经是有意义的成果。

1.1.5.2 测量和拆解延迟

这一部分最能展示工程成熟度。把端到端延迟拆成详细预算。

对于语音助手，需要分别**测量 ASR 延迟、LLM 首 Token 延迟、TTS 首音频字节延迟，以及各步骤之间的额外开销**。为每个请求建立可视化延迟图。

如果你能说："总响应时间是 1.2 秒，这是各组件分别消耗的时间"，这种分析会让面试官感到兴奋，因为它表明你理解性能工程。

1.1.5.3 增加系统韧性

这一阶段关注系统出错时会发生什么。**语音识别服务宕机怎么办？语言模型超时怎么办？系统必须具备优雅降级方案。**

它可以退回到更简单的回答，也可以坦率告诉用户当前出现延迟，而不是一直沉默卡住。加入超时处理，确保任何环节都不会无限阻塞。

还要建立重放模式，让你可以把录制好的输入重新送入管线，复现问题并调试。

真正的生产系统就是这样设计的。认真考虑失败模式和恢复方式，会让你与只实现正常流程的人处于完全不同的层次。

1.1.6 可靠的 AI Agent 系统

第六个项目是 AI Agent。这是目前最热、同时也是最容易做得很浅的方向——几乎每个人的简历上都有一个"Agent 项目"，而其中绝大多数只是一个绑了两个工具的 ReAct 循环。这类演示不会给你带来任何优势。

真正的分水岭在于：**演示型 Agent 关心的是"它能不能跑通一次"，生产型 Agent 关心的是"它失败时会发生什么、花了多少钱、能不能复现、谁来批准它的危险操作"**。Agent 是有状态的，而且错误会逐步累积；同样的提示词跑两次结果也可能不同。你要展示的，正是你如何驯服这种不确定性。

动手之前，强烈建议先读 Anthropic 的 《Building effective agents》——它把"工作流"和"Agent"做了清晰区分，核心观点是：**大多数场景并不需要 Agent，能用确定性工作流解决的就别用 Agent**。能讲清楚"我

为什么在这里选择了 Agent 而不是工作流"，本身就是一个加分项。另外两篇值得读的是 OpenAI 的 《A Practical Guide to Building Agents》，以及 Chip Huyen 的 Agents 长文。

1.1.6.1 选题方向

在讲具体方向之前，先看一组数据——它比任何"哪个方向更酷"的讨论都重要。

对大量 AI 工程类招聘岗位的完整描述做关键词统计后可以看到一个很清晰的排序：招聘方最看重的是**评测能力**，几乎每两个 AI 工程岗位就有一个明确提到"评测（Evals）"，占比高达 **55.7%**；紧随其后的是 **Agent** 经验，占比 51.9%。相比之下，可观测性 / 链路追踪、微调、RAG、多智能体、工具调用虽然也会出现在岗位要求里，但比例明显低了一个量级，分别只有 16.9%、16.7%、11.2%、9.6% 和 5.2%。

最关键的一个数字是：**在提到 Agent 的岗位中，有 68.4% 同时明确要求评测能力。**

这说明了什么？**企业要招的不是"做过一个 Agent"的人，而是"能证明这个 Agent 到底好不好用"的人。** 不少岗位描述里直接写着要做"模型路由、Agent 架构、上下文管理、**评测**"；还有的写着"研究 Agent 行为，找出与高质量产出相关的模式，并把这些发现转化为训练数据、**评测集**或框架改进"。

所以选题的第一原则是：**挑一个成败能被自动判定的任务。**无法自动判定成败的 Agent，你根本没办法评测它，也就没办法证明任何事情——这一点比方向本身更重要。

第二个原则更现实：**别做第十三个"深度研究 Agent"。**这个方向的开源实现已经严重饱和，GitHub 上星标过万的同类项目就有十几个，最高的接近 8 万星。再写一个"搜索 → 汇总 → 生成报告"的循环，**不会有任何区分度。**

下面三个方向，是结合"招聘需求证据"和"工程含金量"筛出来的。

1.1.6.1.1 Coding Agent

给定一个 issue，让它定位文件、改代码、跑测试，直到测试变绿。推荐它的理由很实在：**成败判定天然客观**（测试通过与否，不需要 LLM 当裁判），而且有 SWE-bench 这样的公认基准，只跑一个子集就能得到可对比的数字。它同时天然涉及沙箱、多步规划、失败回退，工程含量最高。

起步不必从零写：mini-swe-agent 是个极简的参考实现，而 Agentless 走的是"不用 Agent 循环、纯流水线"的相反路线。**把这两条路线在同一批任务上做对比，本身就是一个很好的项目**——正好回应了前面那个问题："这里到底该不该用 Agent"。

另外有个值得知道的细节：Agentless 这类方案的失分，很大一部分不在"改不出正确补丁"，而在**生成了多个候选却挑错了哪个。**也就是说，**选择往往比生成更值得优化**——这是个很容易被忽略、但做出来很出彩的角度。

1.1.6.1.2 企业内部流程 Agent（招聘需求最集中）

这个方向在真实岗位数据里的信号最强：**9.6% 的岗位（315 条）提到客服 / 工单 / 问题分流场景**；而**提到"人工介入 / 升级机制"的岗位有 14.4%，"内部流程自动化 / 审批"的有 13.3%。**不少专注于企业服务和客服自动化的 AI 公司，本身做的就是这件事。

具体可以做什么：一个处理内部工单的 Agent（自动分类、查知识库、给出处理方案、拿不准就升级给人）；一个跨系统取数并按规则走审批的流程 Agent；或者一个报销 / 采购单据的审核 Agent。

这个方向的关键不在"能做多少事"，而在**边界清楚、该停就停。**所以务必把这几件事做出来并写进文档：什么情况下 Agent 必须交给人（这正是 14.4% 的岗位在要求的"升级机制"）、每一步操作留下可审计的记录、以及有副作用的操作必须先经人工批准。

但要先有心理准备：**这个方向做不出漂亮分数。**在模拟完整公司环境的 TheAgentCompany 上，**最好的系统完整完成率只有约 30%。**正因如此，正确姿势是**不追求全能，而是在一个很窄的流程上做到稳定可靠**，并诚实地报告失败率——这比一个什么都能试试、但什么都不可靠的 Agent 有说服力得多。

1.1.6.1.3 Agent 评测基础设施（最被低估，却是岗位要求第一名）

这是我最想推荐、但几乎没人做的方向：**不做 Agent 本身，而做"评测 Agent 的工具"。**

理由前面已经给了——**评测是岗位要求里出现率最高的一项（55.7%）**，而提到 Agent 的岗位有 **68.4%** 同时要求**评测能力。**也就是说，做这个方向，你命中的是招聘市场上最普遍、最直接的一项需求；而且它天然能和项目一、项目三串成一条完整的线。

具体要做的是一套能回答"这个 Agent 到底行不行"的系统：

- **轨迹评测**：不只看最终答案对不对，而是判断它走的路径是否合理、工具调用序列是否正确，并且能给**部分分**；
- **可复现的测试环境**：把外部依赖固定下来，让同一个任务能反复重跑；
- **稳定性度量**：同一个任务跑 N 次，报告成功率的**波动区间**而不是单次最好成绩；
- **成本核算**：每解决一个任务花了多少钱、多少 Token、调了几次工具；
- **失败归因**：自动把失败样本归类（工具用错了、上下文丢了、陷入循环、过早宣布完成……）。

为什么说它含金量高？因为这里有几个已被证实、但几乎没人在自己项目里处理的问题——**LLM 当裁判时精确率没有一个超过 70%**（意味着约三成被判"成功"的其实是失败），而单次运行的成绩本身就带着巨大噪声（后面"关于公开基准"一节有具体数字）。能把这些问题正面处理掉的项目，含金量远高于再做一个能跑通的 Agent。

> **两个不建议作为首选的方向。**
>
> 一是 **Text-to-SQL / 数据分析 Agent**：它在技术上并非没有难度，但在上述 3,267 个真实岗位中只有 2 条提到（约 0.06%），且都来自本身就卖 SQL 产品的公司——招聘需求证据几乎为零。
>
> 二是**浏览器 / 电脑操作类 Agent（computer-use）**：它最吸引眼球，但对着真实网站运行时，反爬、验证码、页面改版都会让结果无法复现；而自建可复现环境的基准（如 WebArena、OSWorld）光镜像就要几百 GB、官方建议 1000 GB 磁盘，个人投入产出比很低。真想做，可以用确定性克隆站点的 REAL（112 个任务），成本可控得多。

1.1.6.2 ： 把工具和循环设计好

这一阶段的重点不是"接上模型"，而是**工具设计**。Agent 的能力上限往往取决于工具的质量，而不是模型的智商。

- **工具即接口**：每个工具都要有清晰的名字、明确的参数 Schema 和好的错误信息。返回给模型的错误应该是"简短但有指导性的说明"，而不是一大坨 JSON 报错——模型读不懂的错误等于没有错误。这方面可参考 Anthropic 的《Writing effective tools for agents》。
- **控制住上下文**：Agent 失败往往不是因为模型笨，而是因为没拿到正确的信息，或者信息格式太糟。LangChain 把上下文工程归纳为"写入、选择、压缩、隔离"四种手段（Context Engineering for Agents），Anthropic 也有一篇上下文工程专文。
- **必须有终止条件**：设置最大步数、最大 Token 预算和单次任务成本上限。一个会无限循环烧钱的 Agent，在任何生产环境里都是事故。
- **工具接入标准化**：可以了解一下 MCP（Model Context Protocol），它正在成为工具与数据源接入的通用协议，把自己的工具封装成 MCP Server 是个很好的加分项。

框架方面，LangGraph、OpenAI Agents SDK、Pydantic AI、smolagents 都是合理选择。但要注意：框架容易上手，却常常把"到底给模型传了什么"这件事藏起来。能说清楚每一步进入上下文的内容，比会用某个框架更重要。

1.1.6.3 状态持久化、人工介入与权限边界

这是把项目从"能演示"推到"能上生产"的地方，也是绝大多数人不会做的部分。

状态与恢复。 Agent 任务往往要跑几分钟甚至几小时，中途一定会遇到 API 超时、限流、进程重启。你需要让任务能从中断处恢复，而不是从头再来。这里有个值得讲清楚的区别：**框架层面的 checkpoint 并不等于持久化执行**——checkpoint 保存的是数据，而不是执行本身，进程一死任务就跟着死了。如果想深入，可以了解 Temporal、Restate 或 Inngest 这类持久化执行方案。在作品里能把"崩溃后从第 7 步继续，而不是重跑前 6 步"演示出来，非常有说服力。

人工介入（Human-in-the-loop）。 任何有副作用的操作——写数据库、发邮件、提交代码、花钱——都应该先暂停并等待人工批准。要注意实现方式：让进程空转等待是错的，正确做法是任务挂起、不占用计算资源，收到批准信号后再恢复。同时也要意识到"审批疲劳"这个陷阱：如果什么都弹窗，用户很快就会无脑点同意，权限控制就形同虚设。

安全与沙箱。 这是 Agent 项目里最能体现专业度、也最容易被忽略的一块。Agent 的核心风险是**提示词注入**：它读到的网页、issue、日志里可能藏着攻击指令。Simon Willison 提出的**致命三重奏**是个很好的分析框架——当一个系统同时具备**接触私有数据**、**接触不可信内容**、**能对外通信**这三项能力时，就存在被窃取数据的结构性风险，而目前**没有任何检测手段能百分之百防住它**，唯一可靠的办法是从架构上砍掉其中一条腿。相应地，OWASP 的 LLM Top 10 把**过度授权（Excessive Agency）**单独列为一项风险，对应的做法是最小权限、最小工具集、危险操作强制人工确认。

具体到实现：所有代码执行都必须放进沙箱，不要在自己的机器上直接 `exec` 模型生成的代码。可以用 E2B 这类沙箱服务，或者最简单的 Docker 容器隔离。浏览器类 Agent 可以用 Playwright 或 Browser Use。

1.1.6.4 评测与可观测性

这是整个项目最有价值的交付物，也是把它和项目三串起来的地方。

Agent 的评测和普通 LLM 评测不是一回事，因为你要评的是一整条轨迹（trajectory），而不是单个回答。LangSmith 的文档把它分成三层（见《Evaluate a complex agent》）：

- **最终结果评测**：任务到底成没成功；
- **轨迹评测**：它是否走了合理的路径，调用的工具序列是否正确。注意这里要能给"部分分"——Agent 做对了前几步只错了最后一步，和一开始就跑偏，是完全不同的两种失败；
- **单步评测**：单独考察某一步，比如"面对这个任务，它选的第一个工具对不对"，主要用于调试。

轨迹比对可以直接用 agentevals，它支持严格顺序匹配、无序匹配、子集/超集匹配等多种模式——现实中"用不同顺序调用了同一批工具"通常应该算对，用严格匹配会误判。

要记录并展示的指标：**任务成功率、平均步数、每个任务的 Token 成本与美元成本、工具调用失败率、人工介入率、超预算被强行终止的比例。**可观测性可以复用项目三的 Langfuse；如果想更进一步，可以按 OpenTelemetry 的 GenAI 语义约定来埋点，它已经为 Agent 定义了 `invoke_agent`、`execute_tool` 等 Span 以及 `gen_ai.invoke_agent.tool_calls` 这样的指标，用标准约定而不是自己发明字段，是很专业的做法。

关于公开基准。 简历上能写"我在标准基准上测过"含金量很高，但要挑成本现实的：τ²-bench（客服场景下的工具调用与规则遵守，注意初代 τ-bench 已被官方标记弃用）和 BFCL 函数调用榜对个人来说成本可控，SWE-bench 可以只跑一个子集，GAIA 和 AssistantBench 适合研究型 Agent。

但请务必理解：这些基准的分数比它看上去噪声大得多。 有三个数字值得记住：

- 多份 2026 年的审计显示，**大约每 6 个被判定的样本里就有 1 个判错**；
- 完全相同的配置重复跑 23 次，成绩从 **57.9% 到 76.8%** 不等——单次运行的数字基本等于噪声；
- 同一个模型在同一套任务上，**仅仅因为脚手架不同，成绩就能差出 19.6 个百分点。**

最后这一条其实是你做这个项目的最好理由：**模型不是变量，工程才是。** 也正因为如此，真正专业的做法是报告三个数字，而这三个都是玩具 demo 给不出来的：**重复多次运行的稳定性（而不是单次最好成绩）、每解决一个任务的美元成本、以及一个能把"脚手架的贡献"和"模型的贡献"分离开的消融实验。**

顺带一提，成本差异比大多数人想象的极端。普林斯顿的 HAL 榜单公开了两万多次运行的"成绩-花费"配对数据，其中最有意思的一组是：**两个配置拿到了完全相同的准确率，成本却相差 64 倍。**在 τ-bench 上也有类似情况——56.0% 的配置花 11 美元，54.0% 的配置花 180 美元。把这类对比放进你的项目报告，比多刷两个百分点有说服力得多。

1.1.7 如何写进简历

做完项目只是一半，另一半是**让别人在 30 秒内看懂你做的东西值钱在哪。**这一节把前面所有技术内容翻译成简历语言。

1.1.7.1 一条核心原则

简历上真正有杀伤力的，是"我测出来了什么"，不是"我用了什么"。

对照几个例子就很清楚。

写"使用 LangChain 和 ChromaDB 构建了 RAG 系统"，不如写"**构建 RAG 系统并建立 200 条黄金评测集，接入 CI 质量门禁，忠实度低于阈值自动阻断合并**"。

写"基于 Ollama 部署本地大模型"，不如写"**在同一硬件上横评 3 个本地模型，量化其吞吐、显存与质量的取舍，据此完成选型**"。

写"开发了一个 AI Agent"，不如写"**Agent 任务成功率 X%（重复 5 次的波动区间 A%–B%），单任务成本 \$Y，人工介入率 Z%**"。

写"实现了多智能体协作系统"，不如写"**对比单 / 多智能体两种架构，实测多智能体在本任务上成本高 3 倍而收益不显著，最终选择单 Agent**"。

注意前一种写法全是**输入**（用了什么工具），后一种全是**输出**（得到了什么结论）。工具谁都会写，结论只有真做过的人写得出来。

1.1.7.2 六个可复用的包装方向

无论你做的是哪个项目，都可以从这六个角度找素材。**它们几乎都能套用到任何一个 AI 项目上。**

方向一：可量化的质量指标

> 把"效果不错"换成具体数字。忠实度、引用覆盖率、任务成功率、JSON 合法率、拒答正确率——任何一个都行，关键是**有数**。 素材来源：项目一第三阶段、项目四的前后对比。

方向二：评测体系与质量门禁

> 这是当前招聘需求里出现率最高的一项（55.7% 的 AI 岗位提及，提到 Agent 的岗位更有 68.4% 同时要求）。写清楚你**自建了评测集**、**接入了 CI**、**质量退化会阻断上线**。 素材来源：项目一第三阶段、项目三第三阶段、项目六第三阶段。

方向三：成本与性能工程

> 每请求成本、P50/P95 延迟、Token 消耗、每任务美元成本、首 Token 延迟。这类数字极少有人写，但一写就显专业。 素材来源：项目二的基准测试、项目三第二阶段、项目五的延迟拆解。

方向四：可靠性与故障处理

> 超时处理、优雅降级、断点恢复、重放调试、失败率监控。重点是让人看出你**想过它会怎么坏**。 素材来源：项目五第三阶段、项目六第二阶段。

方向五：安全与权限边界

> 沙箱隔离、最小权限、危险操作人工确认、提示词注入防护。真实岗位里"人工介入 / 升级机制"的提及率有14.4%，是个被严重低估的加分项。 素材来源：项目六第二阶段。

方向六：工程判断力（最稀缺）

> 不是"我做了什么"，而是"我为什么没做某件事"。比如：评估后选择工作流而非 Agent、试过多智能体后退回单 Agent、对比后选择了更便宜的小模型。 素材来源：项目六的选题原则与多智能体那一节。

1.1.7.3 两个完整示例

示例一：RAG 项目（项目一 + 项目三）

改写前 ——

> **企业知识库问答系统** 使用 LangChain + ChromaDB 搭建 RAG 系统，支持 PDF 文档解析和问答，集成 OpenAI API，实现了较好的问答效果。

改写后 ——

> **企业知识库问答系统｜可评测、可监控的生产级 RAG**
>
> - 实现混合检索（BM25 + 向量）与 Cross-Encoder 重排序，Top-5 检索精度较纯向量方案提升 X%
> - 设计引用强制机制：检索证据不足时系统主动拒答而非编造，无依据回答率从 X% 降至 Y%
> - 自建 200 条黄金评测集与离线评测脚本，接入 CI——每个 PR 自动跑评测，忠实度低于 0.85 即阻断合并
> - 接入 Langfuse 全链路追踪，监控 P50/P95 延迟、每请求成本与引用覆盖率，可定位任意一次质量波动的根因

关键差别：改写后每一条都有**动作 + 机制 + 数字**，而且第三、四条直接命中"评测"和"可观测性"这两项高频岗位要求。

示例二：Agent 项目（项目六）

改写前 ——

> **智能助手 Agent** 基于 LangGraph 开发多智能体协作系统，接入多个工具，能够自主完成复杂任务。

改写后 ——

> **内部工单处理 Agent｜带权限边界与轨迹评测**
>
> - 设计 8 个工具接口并规范错误返回格式，使模型能据错误信息自主纠正，工具调用失败率从 X% 降至 Y%
> - 实现断点恢复：任务状态持久化，进程崩溃后从中断步骤继续而非全量重跑
> - 建立权限边界：写操作一律挂起等待人工批准，任务级 Token 与成本上限防止失控循环，人工介入率 Z%

> - 构建轨迹评测体系（最终结果 / 轨迹 / 单步三层），支持部分给分；重复 5 次运行报告成功率区间 A%–B% 而非单次最优
> - 对比单 / 多智能体架构：多智能体成本高 3 倍且收益不显著，据此选择单 Agent 并记录决策依据

关键差别：改写前那句"多智能体协作"看似高级，其实是减分项——它暴露了没做过成本核算。改写后最后一条反而成了亮点，因为它证明**你会做技术选型，而不是堆技术名词。**

1.1.7.4 三个常见的坑

1. **别写"熟悉/了解 XX"** —— 简历上的"了解 LangChain"约等于没写。换成"用 LangGraph 实现了带断点恢复的任务编排"。
2. **数字要能被追问** —— 写了"成功率 85%"，就要能答出：测试集多大？跑了几次？成本多少？失败的 15% 是什么原因？答不上来的数字是负资产。
3. **别堆技术栈** —— 罗列十个框架名不如讲透一个取舍。面试官真正想知道的是你**为什么选它**，以及**你放弃了什么。**

1.1.8 结语

看完了这些内容之后，大家应该知道什么简历是好的，是具有价值的。如果你觉得你的项目做得浅，应该如何去包装。但是最后我想强调两点：

切勿眼高手低：做一个生产级别的项目是很难的，很耗时的。不要上来觉得我必须要一个生产级别的项目，才能拿到大模型Offer的入场券。当你的各种能力达不时，直接给你简历写一个生产级的项目，你也说不清楚，讲不清楚。如果写的简历很高级，一问让面试官觉得你在造假，反而减分。上面的内容和项目都有分阶段，最好的方法是**一个阶段一个阶段的来，**只有你做了，才真正是你的。

我希望教大家什么是好的，应该往哪里改进。但不是告诉大家，你面试的时候，比如一个项目是有**一个完整评估流程，CI/CD持续集成，企业级的错误处理和回退，完整的版本管理，真实的用户数据集**你才能去面试。绝对不是这样的！特别是对于经验年限不多的，甚至是可以放缓到工作5年以内经验的同学，脚踏实地，按照这个方案去做，在你的能力范围内适度包装一下，去面试是没有问题的。越是工作年限越长，越是需要做的项目更加生产级别和成熟。

1.1.9 讲解视频

https://www.bilibili.com/video/BV1cs3d69ET8/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

1.2 2026年 Agent 新形态

Agent 行业的发展非常迅速。技术栈在更新，产品形态在迭代，业界也在不断探索：Agent 到底应该以什么方式进入真实工作和生活，怎样才能从“看起来很厉害”走向真正落地？因此，学习 Agent，不能只停留在

工具调用、多 Agent 协作等技术概念上，还需要理解最新产品在解决什么问题，以及它们背后的设计理念。

从找工作的角度，这件事有两方面的价值。

第一，理解 Agent 的发展趋势和产品形态，能让你更有深度地讨论产品与开发。 对开发者来说，这能帮助你理解需求、判断技术方案，而不只是把功能实现出来。对产品岗位来说，这更是本职工作：理解用户如何使用 Agent、现有产品有什么不足、新的设计为什么出现，才能判断应该做什么，而不只是追随热门概念。

第二，它能让你的项目更有辨识度。 在快速迭代的行业里，Agent 项目的形态也容易过时。如果项目仍然只是一个聊天窗口，加上工具调用或简单的多 Agent 编排，就容易与其他候选人的项目相似。相比之下，如果你能跟上发展趋势，针对实际问题设计新的交互方式、工作机制或协作体验，并讲清楚为什么这样设计，就更有机会与其他候选人形成鲜明对比，脱颖而出。当然，重点不是堆砌新功能，而是让新的设计真正解决问题。

这一小节，我们会**弱化技术实现，以 2026 年 10 月为观察时间点，讲解近期出现的新 Agent 形态。**主要讲“它的设计新在哪里”，这些变化体现了怎样的发展理念和技术方向 以及对我们的项目设计有一些什么启发。

希望学习之后，你在与面试官探讨 Agent 功能时，能给出更有亮点、有依据的回答。如果还能把其中适合的设计融入自己的项目，并解释它带来的价值，那就更棒啦！

在本本节笔记之中，我也直接是用我们的ForgePilot项目，来表示我在写这个文档的时候，对应的内容如何启发我的ForgePilot项目的设计的。我相信到时候我们写ForgePilot的设计文档的时候，你能看到和我们今天内容相关的影子。

同时，我也组织了一些面试题，告诉你在阅读了这个章节后，对于这些真题，你可以找到更多的角度去思考和回答。

通过这两个例子，你能直观体会到今天的素材对于你面试的帮助。

1.2.1 四款代表产品

先明确一个观察角度：**这些产品的入口仍然主要是聊天，但它们不再只围绕“一次提问、一次任务”组织体验，而是尝试让用户拥有持续工作的代理。** 有的强调同一个代理跨任务、跨入口接着工作；有的给代理配上独立身份；有的把多个代理做成通讯录里的同事；还有的把授权、审批和执行过程放到更重要的位置。

今天讲的产品主要在 2026 年 8—9 月发布：

公司 / 品牌	产品	主要场景	最值得关注的设计
OpenAI	**dots**	产品发布、研发、研究、销售、内容制作等持续性工作	围绕同一个代理维持上下文，减少用户在任务线程和不同入口之间搬运背景
Manus	**Cue**	电话与邮件沟通、出行预订、购物、日常事务，也包括工作协作	给代理配电话、邮箱、钱包和电脑，让它拥有对外办事的身份
Meta	**Muse**	购物、旅行、日程、长期个人目标等事务	持续工作环境与独立安全审核结合，把“替我做事”和“允许它做事”分开
xAI / Grok	**Grok Bot**	销售拓展、收件箱、客户管理、招聘、费用、产品与研发流程	把具名、持续工作的 Bot 做成可直接联系的同事，并通过群聊交接工作

上面的主要场景，是我从各个产品的官网的演示示范总结的，可以大概体现出官方主推的功能。不过Agent产品本身就是没有明确的边界，就像ClaudeCode，虽然他的主要定位是Coding Agent，但是其实它可以用来剪视频，做AI 自动化等其他功能。上面的这些Agent也是一样。

OpenAI dots：持续共事

公司与场景。 dots 是 OpenAI 于 2026 年 9 月 29 日发布的 Agent 产品，入口在 ChatGPT，也可以通过 Slack、Teams 等渠道沟通。它面向的不是某一种单点工具，而是需要不断接收新信息、修改结果、推进进度的工作，例如产品发布、客户方案、研究分析和内容制作。

结合图片看设计。 左侧是与 Felipe 的对话，任务是把演示文稿的第 5 页改成 daily view，并保持原有风格、提供两个方案；右侧展示 `See your day more clearly.` 和 `Your day, in one view.` 两个方案预览，还标有邮件、社交素材。对话中提到把预览保存到 Library。它不是只有一段聊天回答，而是把“和代理沟通”与“检查代理正在修改的工作成果”并置起来。

💡 最核心特点

上面介绍的是 dots 的界面与交互方式。但我认为，dots 最核心的特点，是把持续的上下文放在同一个代理身上，而不要求用户每次围绕任务重新组织对话。

这个判断来自官方产品说明：用户可以不断带来新任务、新想法，不必来回管理不同线程，也不必指挥每一步；同一个 dot 还可以同时推进多个项目，并在 ChatGPT、Slack、Teams 之间携带上下文。

比如，你正在负责一次产品发布。上午告诉它定位变化，下午让它修改邮件，第二天再反馈社交素材的风格。产品希望你是在持续指导“同一个了解这次发布的代理”，而不是分别找三个聊天记录，再把背景复制过去。它也会从反馈中学习你对“做得好”的标准。

dots 并没有取消背后的文件和任务管理，而是减少了用户亲自管理它们的必要。 用户可以持续与同一个 dot 沟通，由它维护工作上下文、组织执行过程，不必为每个新任务重新整理对话、重复交代背景。需要时，用户仍可以查看成果、通过 Activity View 检查进度，或打开它的电脑介入调整。**不用管理每个执行细节，不等于完全不用监督：用户仍需给出方向，并在重要决策或操作上审核、确认。**

这一改动，减少了用户的使用门槛。之前用户还需要手动管理不同任务的文件夹。现在即使用户要推进多个任务，也只需要和主Agent对话，

Manus Cue：独立身份

公司与场景。 Cue 是 Manus 在 2026 年 9 月 28 日随 Manus 2.0 公布的独立应用。它与 Manus 使用相同基础设施，但产品重心是个人代理：打电话、处理邮件、预订出行、购物和安排日常事务，也能通过多个代理协作处理工作项目。

图 2：Manus 2.0 发布页 Cue 小节的官方展示图。来源：Introducing Manus 2.0，获取于 2026-10-11；官方演示素材，非独立实测。

结合图片看设计。 这张图把多个界面并置：中间能看到 `Work Assistant`、`Everyday Helper` 分类，以及 `Weekly Priorities`、`Tokyo Trip` 等会话条目；右侧是 `Mail Assistant` 的配置界面。它把 `Mail`、`Computer`、`Automations` 和 `Configuration` 显式列出来，自动化中包括 `Daily Inbox Summary`、`Weekly Newsletter Digest`，配置中则有 `Instructions` 和 `Memory`。这说明用户不只是向助手提问，还能配置它有哪些资源、遵循什么指示、承担哪些例行工作。

核心设计：不是给一个聊天机器人增加工具，而是给代理配置一套身份。 Cue 官网把 Phone、Wallet、Computer、Mail 单独列出来：代理可以拥有自己的电话号码、邮件地址、钱包，以及使用应用与工具的工作空间。这让它在用户授权下，不只在聊天框里给建议，还能通过外部服务把事情接着办下去。

举个官网中的场景：你需要酒店的连通房。代理先找候选酒店，再从自己的邮箱发邮件询问哪些酒店能够保证连通房，最后汇总回复。或者你暂时无法接电话，把来电转给代理，让它了解对方的事项，再把摘要交给你。这里值得注意的不是“能查酒店”，而是**代理有一个外部服务可以联系、回复的身份，沟通不再只发生在你与模型之间。**

Cue 还把协作放进群聊：多个代理可以围绕同一个目标交接工作，例如一个研究场地，一个整理候选清单，一个准备介绍材料。与单纯后台调用子 Agent 相比，这种设计更强调用户可以直接认识和组织自己的代理。

一句话记住：Cue 最鲜明的特点是做了一个聊天工具，工具里的联系人是一个个Agent，你可以设置每个联系人的权限，身份信息，还可以拉群让他们讨论和协作

Meta Muse：执行与授权

公司与场景。 Muse 是 Meta 于 2026 年 9 月 8 日公布的个人 Agent 产品，用户可以在 Muse 应用或 WhatsApp 中与它沟通。它面向购物、旅行、邮件、个人计划等事务。

图 3：Meta 官方购物演示约 18 秒处的购买确认界面。来源：Introducing Muse，获取于 2026-10-11；官方视频截帧，非独立实测。

结合图片看设计。 图中出现了 `Blocked · Waiting for confirmation`，购买操作以 `Checkout` 卡片呈现，并提供 `Deny` 和 `Allow`。卡片列出金额与付款信息，方便用户在操作发生前做决定。它展示的不是“已经买好了”的一句回答，而是一个明确的交接点：**代理已经把购买推进到可执行阶段，但是否执行，交回用户决定。**

从官网介绍和图片上看，仅从产品最核心的功能设计上来看，**我没有看出特别新颖的地方。**因为生活助手类似的早已有之（千问过年推出的自动点外卖的app就和他有点像）。这里更值得分析的是，Meta 怎样把持续工作与安全授权组合起来。

Grok Bot：AI 同事

公司与场景。 Grok Bot 是 xAI / Grok 品牌于 2026 年 8 月 11 日发布的产品。使用入口是桌面应用和 iOS 应用，主要交互仍然是聊天。官网展示的工作场景包括销售拓展、客户管理、招聘、费用处理、收件箱管理，以及产品与研发流程。

图 4：Grok Bot 官网交互演示截图。来源：Grok Bot，获取于 2026-10-11；官方演示，非独立实测。

结合图片看设计。 左侧不是只有按日期排列的聊天记录，而是一组有名字、有职责的联系人：`Chief`、`Sales Outbound`、`Inbox Manager`、`Account Manager`、`Talent Scout`、`Expense Manager`，还有 `Offsite crew` 群聊。它们分别对应总助、销售拓展、收件箱管理、客户经理、招聘搜寻和费用管理等角色。

右侧是与 `Sales Outbound` 的工作对话。截图中的执行结果包括 Salesforce 的 `52 accounts`、Hex 的 `3 segments`，以及 Sequencer 的 `36 drafts queued · 0 sent`：36 份草稿已经排队，但尚未发送。接着，用户说 `Send it. Run this every week.`，界面出现创建 `Overnight outbound` 例行任务的记录。**在这一段演示中，任务从准备成果、等待指示，接着变成持续运行的工作流程。**

这个形态其实和上面的Cue有点像，都属于“把多个持续代理做成可联系的角色，并通过聊天组织协作”的形态；Grok Bot 更强调同事分工，Cue 更强调代理身份与对外办事。

1.2.2 五大发展趋势

上一小节介绍的是这几款产品在UI上的最典型特点，这一小节我们从他们的设计上来提炼出一些2026年Agent 功能设计的趋势和方向。
掌握了他们的设计之后，非常推荐能够把他们的设计理念和特点，加入到我们自己的Agent产品形态当中。

持续的云端环境

Agent 不再只是当前聊天窗口里的临时执行者，而是有一个可以继续工作的地方。 这个地方可以是云端电脑，也可以是其他托管的执行环境。让用户离开后，已委托的任务能否继续推进

例如，Meta 为 Muse 提供 Muse Secure VM，也就是云端的专用虚拟电脑。按照官方说明，Muse 可以在其中使用浏览器处理事务，用户关闭应用后仍能继续工作，等任务有变化或需要批准时再联系用户。这里的变化是：**用户不必为了让任务继续，一直开着自己的电脑和聊天窗口。**

Grok Bot 也强调持续存在的云端电脑。一个用户的多个 Bot 共享这台电脑上的文件、浏览器和登录状态，方便交接工作。例如，一个 Bot 整理资料，另一个接着处理，不必每次重新准备环境。

OpenAI dots 则提供自己的云端电脑，用户可以打开它检查工作；Cue 官网也说明代理在用户退出后仍可继续工作。它们共同指向的需求是：**把持续执行所需的环境准备好、维护好，减少用户自己部署服务、保持设备在线的负担。**

可以用一个例子理解：你让 Agent 收集一批供应商的报价，晚上关闭电脑。它仍可以继续处理已经获准的工作，第二天你回来查看阶段性结果。

可以看出，上述产品的设计理念相近：由产品提供持续可用的云端工作环境，将任务执行与用户自己的电脑分离。 用户不必一直保持本机和应用开启，已委托的任务也可以继续推进。至于底层是否采用 VM，以及工作环境如何共享与隔离，各家的实现并不一定相同。

> 💡 **项目灵感｜ForgePilot：把 Bug 修复做成云端闭环**
>
> 我们能否把 ForgePilot 的亮点，从“输入问题、输出代码”进一步发展为一套面向企业的修复流程：**需求与问题分析 → 复现定位 → 代码修复 → 测试验证 → 提出 PR，交给用户审核**？把这条流程做成产品默认提供的

> 完整体验。
>
> 结合云端工作环境，用户交代修复任务后就可以关闭窗口，执行不必依赖用户电脑。下次打开时，看到的可能已经是待审核的 PR、测试结果，或者明确的阻塞原因与介入请求。**用户离开后工作仍能推进，回来后有成果可验收**，可以成为一个直观的设计亮点；是否合并、上线，仍按企业的审批规则处理。

主动跟进

持续工作解决“能不能接着做”，主动性解决“什么时候开始做、什么时候再来找你”。 如果一个 Agent 只是一直在线，但所有工作仍要用户重新发一句指令，它还没有减少多少跟进负担。

主动性可以分成三个层次：

- **按时间启动。** 例如每天早上整理收件箱，或每周准备一次销售线索。
- **按事件启动。** 例如收到新邮件、某个 Slack 线程有更新后，开始执行约定的任务。这需要产品支持对应的事件与连接工具。
- **结合上下文提出下一步。** 根据此前的目标和新信息，主动给出建议、准备材料，或询问用户是否继续。

Grok Bot 的官网演示就展示了从单次任务到例行工作的转换：销售拓展 Bot 先准备候选客户和邮件草稿，用户检查后说“Run this every week”，界面随后出现 `Created routine – Overnight outbound`。用户不是每周重新交代一遍，而是把工作变成了持续的安排。

结合这张图，可以注意右侧下方的例行任务记录。它能说明官方展示了“设置后重复执行”的交互，但不能仅凭截图证明后续每周执行的质量和稳定性。

Cue 也有 Automations，支持定时或受支持的事件、条件触发；它的 Mail Assistant 界面列出了每日收件箱摘要、每周简报摘要。dots 则描述了另一种主动性：在已连接的信息来源中进行主动研究，发现可能需要关注的事情，再准备建议或材料。

主动不等于擅自行动。 比如 dots 的主动研究使用只读工具，不能通过这条机制自行发送消息或修改应用内容。主动发现问题、主动准备方案和获得授权后执行，是不同阶段。一个好用的 Agent，既要减少用户催促，也要知道何时停下来询问。

> 💡 **项目灵感｜ForgePilot：主动找问题，遇到阻塞再找人**
>
> ForgePilot 能否不再只等待用户输入 Bug，而是定期检查已有的 Bug 列表，并从已授权的邮件、Issue 等来源中识别新的问题线索？例如，收到“导出报表后页面报错”的反馈后，先整理复现条件、判断是否与已有问题重复，再为符合处理规则的 Bug 启动修复和测试。
>
> **主动发现 → 分析筛选 → 按授权修复 → 汇报成果或请求介入**，可以成为另一条工作路径。不是发现任何线索都直接改代码：涉及需求不清、权限不足或高风险改动时，应主动找人确认，并说明缺什么信息、需要做什么决定。这样既减少用户反复催促，也避免把不可靠的反馈当成确定的修复指令。

完整任务交付

用户希望交出去的是一件事，而不是一串点击步骤。 这并不是说过去的 Agent 完全不能执行长任务，而是随着能力提升，产品越来越强调把任务推进到可检查、可使用的结果。

以购物为例，如果还需要用户逐步说“打开网站、搜索商品、选规格、进购物车”，那么 Agent 更多是在代替手动操作。Muse 的官方演示则展示了一个更完整的过程：给出旅行婴儿车候选、推荐合适的商品、打开商品详情、推进结账、等待用户批准，最后显示购买完成。

这张图是同一段官方视频约 22 秒处的截帧，界面显示 `Completed · Purchased Glide Pro`。与上一节的购买确认图连起来看，能理解“接受任务 → 推进操作 → 用户确认 → 交付结果”的过程；

工作场景也是一样。dots 官网的研发例子，不只是告诉用户“这个 Bug 可能怎么修”，而是描述它跟踪反馈、修改和测试代码，再准备带有变更演示视频的 PR，交给用户审核。

> 💡 **项目灵感｜ForgePilot：不只交代码，还展示修复效果**
>
> dots 的官方设计例子给了我们一个具体启发：**修复完成后，能否自动生成演示视频，让审核者直接看到变化？**对页面交互类 Bug，ForgePilot 可以在测试环境中复现问题、验证修复，并记录关键操作，随 PR 一起交付修复前后的对比或修复后的操作演示。
>
> 例如，“点击导出按钮后页面报错”被修复后，视频展示点击按钮、成功下载文件的过程；同时附上测试结果与修改说明。这样用户不用只凭“已修复”三个字判断，也不必为了初步验收先自己拉代码、搭环境。**让修复成果更容易检查，也是交付体验的一部分。** 视频不能替代测试；对于不适合录像的后端 Bug，则可以提供请求与响应对比、日志或回归测试报告。

这里的“交付”，也不一定意味着 Agent 做完全部最后操作。**一份等待审批的邮件、一组经过测试的代码修改、一个整理清楚的预订方案，都可以是有明确验收标准的阶段性成果。** 关键是用户收到的是可以检查和继续使用的结果，而不是还要自己组织大部分后续工作的建议。

这也改变了用户的角色：从不断指挥下一步，转向说明目标与约束、检查成果、处理关键决定。对项目设计来说，可以问一句：**我的 Agent 最后交给用户的，究竟是一段回答，还是一个真正可验收的成果？**

跨任务记忆

长期委托需要的不只是“电脑一直在”，还需要“它知道我们做到哪里了”。 否则，即使 Agent 可以运行很久，用户每次回来仍要重述目标、贴资料、解释偏好，使用成本依然很高。

以 dots 为例，你可以持续把工作交给同一个 dot，不必每次都从头讲一遍。 按照官方描述，它可以保留相关的工作背景、同时推进多个项目，减少用户翻找旧对话、复制资料和重复解释的负担。

比如，你先告诉 dot：“这次产品发布面向新手用户，文案要简单。”它帮你准备了宣传材料。第二天，你再说：“根据昨天的材料，写一封发布邮件。”产品希望它能接着使用此前的要求和成果，而不是让你重新说明目标用户、文案风格，再把昨天的材料贴一遍。这里强调的是**同一个 dot 能接着此前的工作往下做**，而不是只处理当前这一条消息。不过，这不代表它可以读取未授权的数据，也不保证记忆永远完整、准确。

Cue 的官方应用说明也提到 Memory and Dream，用来记住饮食偏好、旅行习惯、学习进度和计划；Muse 支持记住用户在意的事情，也允许用户要求它忘记具体内容。Grok Bot 则强调角色保留工作上下文、从反馈

与流程示范中学习。

这一条可以分成两种记忆： **“这项任务做到哪一步”的进度，以及“你通常希望怎样做”的偏好。** 前者让任务能接着做，后者让代理逐渐减少重复确认。产品还需要考虑如何纠正过时或错误的记忆，而不是只追求记住得更多；各家的查看、修改和删除能力并不相同。

点评：Agent记忆一直是每个Agent的核心能力，Dots的这个理念很好，不用再让用户来通过文件夹的形式管理任务，而让一个Agent在全局的角度，记住所有事情。但我也非常好奇的是，以前我们设计记忆模块，让Agent能很好地记住一次任务，都是一件很难的事，现在我们让他记住多个任务，无疑对他背后的记忆模块的设计提出了更高的要求。 除了这个产品形态值得学习，他背后如何实现的，大家在如何继续卷记忆模块的设计，也是我们值得深入挖掘。

> 💡 **项目灵感 | ForgePilot：一个入口，跟进多个 Bug**
>
> 我们的 Coding Agent 能否弱化用户对工作目录和独立任务会话的管理，提供一个统一的全局聊天入口？用户可以在同一处交代：“修复一下今天的bug”由 ForgePilot 找到对应项目、任务进度和相关材料，而不是让用户逐个打开目录、翻找聊天记录。

开箱即用

很多能力早已有了，但普通用户不一定能把它们用起来。 要自己部署常驻服务、接入模型、配置工具、设置触发器，再维护上下文，这些工作对开发者尚且有成本，更不用说只是想让 Agent 帮忙办事的用户。

两份文稿都强调了这个方向：把需要自行搭建的能力，变成可以直接交办的产品体验。这里的“开箱即用”不是完全不需要设置，而是产品替用户承担了更多搭建和管理工作。

Cue 的官方展示图里，右侧 `Mail Assistant` 将 Mail、Computer、Automations、Instructions、Memory 放在同一个助手的配置界面中。用户面对的是“这个助手能用什么、按什么要求工作、有哪些例行任务”，而不是自己拼接一套运行系统。图中的自动化条目说明这些能力被做成了可见入口，并不代表所有任务都无需配置就能成功运行。

Grok Bot 把具名角色和群聊放在界面上，还提供让 Bot 跟随用户操作、学习 routine 的方式；dots 让用户通过同一个代理沟通多个项目；Muse 则把执行状态、购买确认等节点呈现在聊天流程中。它们都在尝试让用户用更熟悉的方式交代、检查和调整工作。

点评：这一点是做personal agent（面向个人生活场景的Agent） 更应该考虑的，因为personal agent的用户通常不是程序员，所以降低他们的使用门槛是很重要的。可以参考这些Agent的设计，把产品做成一个类似于聊天软件的APP, 不用配置，不用部署，开箱即用。

1.2.3 面试问题

你如何看待现在Agent发展的趋势?

💡 **思路：**这类问题没有标准答案，可以按“时间阶段 → 代表产品 → 主要特点”的顺序回答，最后总结 2026 年的趋势与自己的判断。阶段是为了讲清发展重点，不是严格的行业分类；不同形态会并行发展，也会

互相吸收能力。这样既体现对发展过程的理解，也能落到近期产品与项目设计。

🎤 **参考答案：**

我会从几个时间阶段来看 Agent 的发展。

第一，2025 年上半年，任务执行型应用受到更多关注。 例如，2 月推出研究预览的 Claude Code，深入代码项目，读取代码、修改和测试；3 月推出的 Manus，则让用户通过网页交代目标，由它调用工具完成调研、文档等任务。这一阶段的重点，是从“给出回答”走向“执行任务”，编程与通用任务两条路线并行发展。

第二，到 2026 年初，以 OpenClaw 为代表的个人代理更强调长期服务。 它接入聊天软件，结合定时任务、主动检查和记忆，让用户随时交办，也能按约定持续跟进。但自行部署、配置和维护仍有门槛，其他 Agent 也在吸收这些能力。

第三，2026 年 8—9 月，Grok Bot、Muse、Cue 和 dots 集中推出，更强调持续代理的产品体验。 比如 dots 用同一个代理承接多个任务、减少重复解释；Cue 和 Grok Bot 把代理做成可联系、有分工的助手；这些产品也强调托管工作环境，让已委托任务不必依赖用户本机持续运行。

我认为现在最新的Agent的发展形态朝着：持续的云端工作环境、主动跟进、完整任务交付、跨任务的上下文与记忆，以及开箱即用。这些方向在发展 ，让用户少配置、少指挥，却得到可靠的成果，降低Agent的使用门槛，让更多的用户可以使用。

为什么现在的通用Agent已经很强了，你还要继续开发新的Agent?

💡 **思路：** 这个问题是有可能面试官在问自己的项目的时候问的，比如我们的ForgePilot项目，面试官就有可能问。这个问题真的是一个终极拷问：真的不光你会被问到，你的+1可能会被你的+2问到，为什么你要重新开发一个Agent，你的+2可能会被+3问到这个问题。 确实现在很多功能都是使用claudecode或者claudecode + SKILL就能完成的，但确实大家好像都在乐此不疲地开发新的Agent。 这个问题也没有一个标准答案，角度非常多。对于我们的ForgePilot项目，我希望找一些特别好的理由来回答这个问题（我会总结在**ForgePilot项目**当中）。 在本小节的这个问题中，我们这四个产品的角度，来给这个问题提供一个思考角度（为什么已经有了这么多成熟Agent，2026年12月这些公司还要发布这么多新的Agent形态）

🎤 **参考答案（约两分钟，可直接口述）：**

我认为，**通用 Agent 的能力已经很强，并不代表它的产品形态已经定型。** Claude Code 等产品仍在演进，业界也在探索：怎样让用户更容易交办任务，让 Agent 更持续地工作，并交付更容易验收的成果。近期的 dots、Cue、Muse 和 Grok Bot，就体现了几个值得关注的方向。

第一，持续的云端工作环境。 让已委托的任务不依赖用户本机一直开着，用户离开后仍能推进，回来查看成果或处理阻塞。

第二，更强的主动跟进。 不只是等用户发指令，还能按时间、事件启动约定的工作，结合上下文提出下一步，并在需要时找人确认。

第三，从执行步骤走向完整交付。 例如 dots 的研发场景，不只修改代码，还测试并准备带演示视频的 PR，让用户能直接审核修复结果。

第四，跨任务的上下文与记忆。 比如 dots 强调持续向同一个代理交办工作，减少翻找旧对话、搬运资料和重复解释。

第五，更容易使用、开箱即用。 比如 Cue 配置电话、邮箱和钱包，Grok Bot 把助手做成有固定职责的联系人，让用户用熟悉的聊天方式交办和协作，少搭环境、少管过程。

这些方向不一定是新产品独有的，Claude Code 也在吸收类似能力。**但能力重叠，不意味着产品设计没有继续探索的空间。** 开发新 Agent，可以复用已有执行能力，围绕具体用户和场景改善整套体验。

所以，我认为业界继续开发新的 Agent，不只是为了让它“能做更多事”，也是在探索“怎样让更多人真正用好这些能力”。**从需要用户不断组织和跟进任务，走向更容易交办、能够持续推进、成果便于验收，是这一轮产品形态发展的重要方向。**

字节一面真题：其他厂商Agent产品

真题来源：2026年10月字节一面 · 第5组原题

除了开源 Agent，你是否调研或使用过其他厂商的 Agent 产品?

💡 **答题思路**

这里就是我们讲的今年发布的4个Agent的核心特性，最后总结共同发展趋势收尾就行，专题笔记里都讲了。

🎤 **参考答案**

有关注，我就以今年（2026年）8—9月发布的四款产品为例，简单分享一下。

OpenAI 的 dots，主要面向产品发布、研究和内容制作等持续性工作，特点是让同一个代理持续维护上下文，减少用户反复交代背景。

Manus 的 Cue，主要处理邮件、电话、预订等个人事务，特点是给代理配上电话、邮箱、钱包和电脑，让它有独立的身份对外办事。

Meta 的 Muse，主要面向购物、旅行和日常安排，特点是持续执行与关键授权分开：代理可以先推进任务，但付款等重要操作交给用户确认。

xAI 的 Grok Bot，主要面向销售、招聘、客户管理等工作，特点是把代理做成按职责分工的 AI 同事，通过聊天和群聊协作。

整体看，我觉得它们有几个共同趋势：**从单次任务走向持续执行，从临时对话走向长期上下文，从通用助手走向有身份、有分工的协作者，同时把授权和审批做进执行流程。最后极大降低了使用门槛，惠及更多用户。**

本节小结

在国外公司推出了新的一系列Agent形态后，我们看到国内的公司，腾讯，字节也在大力加班推进（听说字节国庆有的组都没休息在赶这种像Personal Agent的产品）。

对于这些产品的研究，不论是产品还是技术同学，不论是产品方面还是实现的技术细节方面，都值得我们进一步去学习和深挖。今天通过粗略的的学习，已经对我带的个人项目和面试有很大帮助，继续深挖一定会有更多的收获。

最后从这一波行情来看，也告诉我了我们不是说Agent不值得做了，做不出新意了，整个大厂，行业都在继续探索和迭代。从个人的角度来说，我们紧跟大厂，行业的趋势，让自己的Agent的形态和技术和行业对齐，无疑会让我们的项目更有竞争力。

1.2.4 讲解视频

https://www.bilibili.com/video/BV1kZp26bE6H/#reply320772547536

1.3 RSI 进展

前言

最近（2026 年 10 月），有挺多朋友催我聊一聊 RSI：这个概念为什么突然这么热？AI 到底已经能自己改进自己了，还是大家又把一个遥远的设想讲成了现实？

RSI 的全称是 Recursive Self-Improvement，也就是“递归自我改进”。简单来说，就是 AI 利用现有能力改进自身，再由改进后的系统参与下一轮改进。

这并不是凭空冒出来的讨论。沿着“模型参与改进模型”和“Agent 参与改进 Agent”两条线，已经能看到一些更具体的尝试：

- **模型参与改进模型｜让模型学习“该给另一个模型出什么训练题”。** **2026 年 9 月 24 日**发布的论文《Self-Play Pretraining with Zero Data》，让两个模型从随机初始化开始共同学习：一个“生成器”模型编写程序，由程序产生训练数据；另一个“学习者”模型通过预测这些数据来更新参数。生成器再根据学习者当前的能力获得反馈，学习生成难度合适的数据，推动下一轮训练。直观地说，就是 **“模型生成训练数据 → 训练另一个模型 → 根据学习进展调整数据 → 继续训练”**。这展示了模型参与构建自适应训练课程的可能性。
- **Agent 参与改进 Agent｜不只修改用户的代码，也修改自己赖以工作的代码。** Sakana AI 与英属哥伦比亚大学团队的 Darwin Gödel Machine（DGM） 研究，论文首发于 **2025 年 5 月 29 日**，最近一次修订于 **2026 年 3 月 12 日**。它让编程 Agent 修改自身的工具和执行流程，再用编程任务评估新版本；产生的版本进入候选库，后续迭代可以从这些版本继续分支，形成 **“Agent 修改自身代码 → 评估新版本 → 从已有版本继续改进 Agent”** 的链路。

这两个例子的区别很直观：**前者由模型参与生成训练数据，推动模型参数更新；后者由 Agent 修改自身代码，推动 Agent 系统迭代。** 这也是我们这个文章主要讲的两个思路，从模型和Agent应用两个路线来谈RSI。

图 1 | 两条自我改进路线：一条通过训练更新模型参数，另一条修改 Agent 的工具、代码或工作流。AI 生成示意图，用于解释本文概念，不代表具体实验结果。

可以设想一下，如果这条路继续走下去，会是什么样子？在模型研发中，AI 不只帮忙写代码，还能参与准备训练数据、调整训练方案、运行实验和评估结果，再把有效的改进用于下一版模型。在 Agent 系统中，它也

可能根据运行反馈，发现自身工具或执行流程的不足，修改代码、测试效果，把更好的版本保留下来，再用新版本继续改进。原本需要研究员和工程师不断推动的工作，就可能有越来越多的环节交给 AI 完成。

这也是 RSI 让人兴奋、又让人不安的地方。过去，我们更多是在问“AI 能替人完成多少工作”；现在，问题变成了“如果连研发更强 AI 的工作，也越来越多地由 AI 承担，会发生什么？”一旦改进能力本身能够持续提升，影响就可能不只是开发效率的提高：程序员的工作方式、软件团队的分工，乃至部分岗位的需求，都可能发生深刻变化。但从这些趋势直接推导出“软件开发岗位会消失”或“人类将被替代”，还缺少足够的依据。

相关研究不断推进，加上这件事潜在的巨大影响和一些巨头公司，自媒体的点火，RSI 的热度也就不难理解了。

本文会讲清楚哪些问题?

1. **概念：什么才算 RSI?**
2. **形态：从局部优化到持续自我改进，中间还有哪些层次？** 我们刚才谈到的模型，应用自己持续改进自己，不用人参与是理想的终极形态，但是从简单的RSI到终极形态，中间有哪些层次?
3. **进展：RSI 到底走到哪一步了？** 沿着应用开发和模型训练两条线，看看哪些环节已经形成闭环，哪些关键步骤仍然依赖人类。
4. **实践：对大模型应用开发有什么启发？** 哪些思路值得借鉴到自己的 Agent 项目中，让自己项目更新颖?

💡 **学习本篇内容，如何帮助你的面试?**

> **跟上行业热点，为项目找灵感，也为面试积累谈资。**

① 聊热点，更有话可说

RSI 是近期的热门话题。读完本文，当面试官聊起它时，你也能接得上话，而不只是听过这个名词。

② 做项目，多一些灵感

如果你正在构思自己的 Agent 项目，本文也能提供一些新思路，让项目设计更有特色，在面试中多一个亮点。

③ 答开放题，多一些思路

面对“现在如果让你做一个 Agent，你准备怎么做？（字节2026/10月秋招真题）”“你如何看待 Agent 的发展趋势？”这类开放性问题，本文可以为你积累回答素材，拓宽思考角度。

1.3.1 什么是 RSI

RSI的英文名字是Recursive Self-Improvement，从英文名也可以看出有两个关键特性：Recursive（递归）Self-Improvement(自我改进)，理解了这两个词就理解了RSI。

1. 自我改进：改的不是某个答案，而是系统自身

平时我们让 AI 修改一篇文章、修复一个网站，它是在帮我们改进工作成果。自我改进关注的则是：**AI 能不能参与改进自己以后做事所依赖的能力和机制？**

比如说一次应用任务，Agent记录了处理问题的方式，发现不对，修改了提示词，下一次用新的提示词来完成。模型训练的时候，模型发现效果不改，改变了自己的训练参数，让下一次训练有更好的效果。这些都不是在改进任务本身，而是改进处理任务的方式，这样的改进就是大模型里面的RSI的自我改进的意义所指。

2. 递归：改进后的系统，再参与下一轮改进

只有一次改进，还不足以说明“递归”。RSI 更重要的一步，是让改进的结果重新进入改进过程：

> **当前系统 A_0 → 参与改进，产生 A_1 → A_1 再参与改进，产生 A_2 → ……**

拿编程 Agent 来说，它先改进自己的代码编辑工具，得到一个新版本；新版本再利用改进后的工具，继续修改自身的其他功能。它不只是“被改好了”，还重新成为下一轮改进的参与者。

如果新版本不仅更会完成任务，也更擅长推动下一次改进，就进一步触及了 RSI 最受关注的可能性：**改进能力本身也在提升。** 不过，能连续迭代，并不意味着每一轮都会变好，更不意味着改进速度必然越来越快。这也是后面区分不同发展形态时，需要特别留意的边界。

1.3.2 四层能力

提到 RSI，很容易直接想到“AI 完全不靠人，自己造出更强的 AI”。但在走到这种状态之前，系统已经可以接手一部分改进工作：先记住教训，再利用反馈更新，逐步把更多改进环节串起来。

为了方便理解，本文参考 《这是一篇把“RSI”讲明白的科普级综述！》 中关于 RSI 演进过程的讨论，按 **AI 接手改进工作的深度与自主程度**，重新整理成下面四层。需要说明的是，这不是学界统一的等级认证，而是本文作者根据相关讨论和行业发展现状整理出的讲解框架。下面分别用模型训练和应用开发举例，辅助说明这四个层次。

第一层：记住经验——下次别再犯同样的错

最简单的一层，不急着改模型或代码，先让有价值的经验留下来，供以后使用。可以理解为：**先给系统准备一本能翻阅的“错题本”。**

- **模型训练侧：** 模型分析自己答错的题，整理出“容易漏掉单位换算”“遇到多重条件时容易漏条件”等错误类型，保存为后续出题、筛选训练数据时的参考。这时只是积累了训练线索，还没有因此更新模型参数。
- **应用开发侧：** 一个数据分析 Agent 因为没有检查空值而执行失败。它把“统计前先检查缺失值”保存为经验，下次遇到类似任务时检索出来，加入上下文，而不是每次都从头试错。包括像自己沉淀SKILL，长期记忆的写入，这些内容其实就是已经蕴藏着RSI的思想了。

> 💡 也就是说这一层的RSI是外化的，比如说Agent的长期记忆，如果文件丢了，那么改进就丢了。

第二层：利用反馈更新——不只记下来，还改变以后怎么做

再往前一步，系统开始利用反馈，更新模型参数、提示词或固定的执行策略，而不是仅在上下文里多放一条提醒。

- **模型训练侧：** 模型生成一批解题过程，按既定标准筛选出可靠样本，再由训练程序用这些样本微调模型。更新后的模型可以继续生成下一批样本。经验不再只存在“错题本”里，而是通过训练影响模型参数。
- **应用开发侧：** Agent 根据失败记录生成几个新版提示词，在固定测试集上比较效果，再把通过评估的版本写入产品的提示词配置或 Agent 驾具，作为后续任务的默认版本。也就是说，变化会持久地进入应用系统，而不是只存在于当前对话的临时上下文中；但它改的是应用层的配置，不是底层模型参数。

> 💡 这一层的RSI是内化的，比如应用开发侧，发现了更好的提示词，我直接改变了这个应用的代码，下次编译出来的产品，本身就携带着这个优化后的提示词，自我改进内化到了产品本身。

第三层：自主组织闭环——开始决定“改什么、怎么改、是否采用”

它与第二层的真正区别，是系统接手了更上游的组织工作：在限定范围内，自己发现问题、选择改进对象、提出实验方案、安排测试，并根据结果决定保留、回滚还是继续探索。也就是说，第二层主要是在既定的优化流程中更新；第三层开始改进或重组这套流程本身。

> **第二层：人已经设计好“怎么优化”，AI 在流程中找一个更好的版本。**
>
> **第三层：AI 开始参与决定“优化什么、怎么设计实验、下一轮往哪里走”。**

- **模型训练侧：** 系统发现模型在多条件推理上表现较差，自主提出调整训练数据配比或安排针对性练习，运行小规模训练实验，再用未参与训练的评测数据检查效果；通过验收的模型进入下一轮，并继续参与出题或实验方案生成。模型并不是直接“手改权重”，而是通过训练程序完成参数更新。
- **应用开发侧：** 编程 Agent 发现自己的代码编辑工具经常误改文件，于是修改工具实现、运行回归测试，保留有效的新版本；后续再由采用新工具的 Agent 继续改进自身。前言中的 DGM，就展示了这类修改自身代码、评估并继续迭代的链路。

> 💡 第三层相比第二层，第二层是AI在人规定的框架中持续迭代，第三层是AI已经可以开始跳脱于人的框架，修改框架本身了。

第四层：提升改进能力——不只更会做事，也更会改进自己

前面几层即使已经连续产出更好的版本，也还需要回答一个问题：**新版本是否比旧版本更擅长推动下一次改进？**

这是递归增益的关键。下面两个例子描述的是需要验证的目标，而不是已经普遍实现的能力：

- **模型训练侧：** 新模型不只是答题分数更高，还能改进“如何改进模型”的流程：更准确地判断当前模型的主要瓶颈，选择更值得尝试的训练方案，设计对比实验，并根据结果决定下一轮应该继续哪个方向。
- **应用开发侧：** 新版 Agent 不只是更会完成业务任务，也能改进自己的“自我迭代流程”：更有效地发现自身缺陷，生成有区分度的修改方案，设计回归测试，识别评估结果是否可信，并决定哪些版本值得保留、哪些版本应该回滚。这里优化的是 Agent 如何发现问题、提出改动和筛选版本，而不只是某一次任务的执行效果。

用一张表记住这四层

图 2 | 先用图把握四层的核心区别，再用下表对照模型侧与应用侧的例子。这是本文的讲解框架，非学界统一分级；AI 生成示意图。

层次	核心变化	模型训练侧的例子	应用开发侧的例子
① 记住经验	留下以后能用的教训	保存错误分析，供后续准备训练材料	保存失败复盘，下次检索使用
② 利用反馈更新	在既定机制中改变系统	按预设标准筛选数据并训练模型	按固定测试流程更新默认提示词
③ 自主组织闭环	开始决定改进对象和迭代流程	自主提出训练方案、运行实验并接纳新模型	自主选择修改工具或流程，测试后继续迭代
④ 提升改进能力	新版本更擅长改进“改进流程”	更有效地搜索和决策模型研发方案	更有效地发现缺陷、设计实验和筛选自身版本

> **一句话串起来：记得住教训 → 能利用教训改变自己 → 能组织自己的改进过程 → 连改进自己的本领也在提升。**

本部分参考资料

- 《这是一篇把“RSI”讲明白的科普级综述！》

1.3.3 行业进展

前面四层是一张判断地图。把它放回现实产品中，整体结论可以先说在前面：

> **不管是模型训练还是应用开发，行业能力目前主要集中在第二层，并在部分受控场景中出现第三层的局部闭环；第四层“改进能力本身持续增强”，仍主要停留在研究原型的探索阶段。**

图 3 | 把本文判断放回四层地图：常见能力、受控局部探索和尚待验证的递归增益。AI 生成示意图，仅整理本文论述，不是统一评级或实验测量，也不表示每个产品均具备相应能力。

这里的层级判断，不看模型名称是否更新，也不看 Agent 是否能连续工作，而看公开资料是否证明系统接手了相应的改进职责：

- **第二层**：反馈能够进入参数、提示词、记忆、工具或工作流，形成持久变化；
- **第三层**：系统能够在限定范围内发现问题、选择改进对象、设计并执行实验，再决定保留或回滚；
- **第四层**：改进后的系统还要变得更擅长组织下一轮改进。

1. 模型训练：第二层证据明确，第三层正在进入研发流程

模型侧最典型的事实有三类。

第一，**反馈已经能够写入模型参数。**以 DeepSeek-R1 为例，模型生成数学、代码和推理答案，程序或规则验证结果，再将奖励用于强化学习更新模型。这个“生成结果—自动验证—奖励反馈—参数更新”的闭环，直接对应第二层：模型不只是记录一次回答，而是把反馈写进下一版模型。

第二，**模型开始参与模型研发工作。**GPT 系列、Claude 以及相关 Codex 类系统已经能够承担更长的代码、实验执行和研究工程任务。Anthropic 关于 AI 参与自身研发的公开讨论，以及 OpenAI 关于研究加速的报告，都说明模型已经从“被训练的模型”进入“参与训练和研发的执行者”阶段。

这些事实说明，AI 在既定目标、评估标准和资源边界内，能够生成候选方案、运行实验并依据结果推进工作，因此在部分受控任务上出现第三层的局部能力。

但这里仍有明确边界：公开资料尚未证明 GPT、Claude 或 DeepSeek 能够稳定自主决定研究目标，改写整套模型研发流程，并让改进后的后继模型继续优化这套流程。模型性能变强，或者 AI 写了更多研发代码，都不能单独证明第四层。

2. 应用开发：第二层已经产品化，第三层表现为局部的驾具闭环

应用侧的代表性产品包括 Claude Code、Codex、DeepSeek Harness 和 Cue。它们公开展示的共同能力是：保存项目规则和任务经验，调用工具，修改代码或配置，运行测试，维护较长任务状态，并通过 Skills、Hooks、插件、工作流或多 Agent 协作扩展系统能力。

这些能力分别对应：

- **第一层**：记忆、项目文件、任务状态和工作空间让经验可以跨任务保存；
- **第二层**：经过验证的提示词、Skill、Hook、插件和工作流可以固化到应用系统中，成为后续任务的默认配置；
- **第三层的局部形态**：在限定的编程或自动化任务中，Agent 可以执行“发现问题—修改工具或流程—运行测试—保留结果”的闭环。

但这些产品的公开能力主要证明的是“Agent 能更稳定、更长时间地完成任务”，并不等于“Agent 会自主改进自己”。例如，Claude Code 和 Codex 可以在项目中修改代码并运行测试，DeepSeek Harness 可以通过插件、记忆和工作流扩展能力，Cue 可以让具有独立身份和工作空间的个人 Agent 持续处理事务；这些都说明应用层的驾具越来越丰富，但尚未公开证明它们会自主重写自己的核心驾具，评估改动是否提升了“自我改进能力”，并据此进入下一轮。

3. 第四层：研究原型已有探索，产品还没有公开的稳定证据

目前更接近第四层的代表，是 Darwin Gödel Machine（DGM）。它不是普通产品，而是一个受控研究系统：Agent 可以修改自身代码，用编程基准评估新版本，再从已有版本继续产生后续分支。它改进的对象已经不只是某个用户任务，而是 Agent 自身的工具和工作方式。

DGM 因此比通用产品更接近第四层，但仍不能直接宣布“第四层已经实现”。还需要在独立任务、固定资源、多个后继版本和循环外评估中证明：新版本不仅表现更好，而且确实更擅长发现、设计和验证下一次改进。

图 4 | DGM 的自修改与评测循环：从版本库中选择 Agent，修改自身代码，评测后将新版本加入版本库。来源：Darwin Gödel Machine 论文 Figure 1，原图沿用目录内已保存素材，非 AI 生成；该图不单独证明第四层已经实现。

4. 当前瓶颈

行业还没有进入开放式 RSI，主要受以下问题限制：

1. **研究目标仍由人设定**：系统能执行“把模型训得更好”，但还不能稳定决定“下一步应该研究什么”。
2. **评估器可能被利用**：同一系统同时生成答案、奖励和版本选择，容易把指标做高，却没有真正变强。
3. **实验成本仍然很高**：模型训练、长时间 Agent 运行和真实环境验证都需要大量算力、时间和权限。
4. **泛化能力不足**：固定测试集上的提升，不代表对未见任务有效，更不代表改进能力本身提升。
5. **复杂度和安全风险会累积**：自我修改可能带来行为漂移、权限扩大、死代码、评估污染和难以回滚的问题。

因此，目前最稳妥的判断是：

> **第二层已经成为模型和应用产品的常见能力；第三层正在以受控、局部的形式进入工程实践；第四层还主要是研究问题。行业已经出现有界的自我改进闭环，但尚未证明改进后的系统能够稳定地、开放式地改进“改进本身”。**

1.3.4 项目启发

看完了上述内容，我让AI帮忙结合现在的Agent产品，总结一些我们也可以加入到自己项目当中的，具有一些RSI相关特质的一些功能。我们不妨想想，我们能不能按这个思路，在自己的Agent特性中加入一点点RSI的特质，并且在简历上写自己的Agent具备RSI的特点，这样简历项目看上去会不会更有含金量呢？

1. 自我沉淀：把做对的任务变成下次能调用的 Skill

Agent 完成一类有复用价值的任务后，可以把操作步骤、适用条件、输入输出和注意事项整理成 Skill；经过测试和审查后加入技能库，下一次遇到类似任务时按需调用。重点不是“自动写了一份文档”，而是这份经验能否稳定改善后续任务。

2. 自动探索提示词：不是觉得写得更好，而是测出来更好

Agent 可以根据执行结果和失败原因生成多个提示词候选，再在验证集上比较效果。例如，信息抽取 Agent 反复把“未披露”填成“0”，就可以提出“缺失信息必须保留为空，并附上原文证据”的候选指令。但新提示词必须经过跨样本评估，不能因为修好一个例子就直接发布。

3. 长期记忆：让用户纠正真正影响下一次任务

长期记忆属于第一层能力，但它是实用的起点。应区分用户偏好、项目规则、任务事实和失败经验，记录来源、时间和适用范围，并允许用户查看、修改和删除。记忆不是越多越好，错误或过时的记忆会反过来污染后续任务。

4. 工具和驾具的可修改性：让 Agent 有机会改进自己的工作方式

如果工具、提示词和工作流全部硬编码在系统内部，Agent 很难参与改进。可以把这些部分模块化，让 Agent 在权限边界内提出配置或代码变更，再通过静态检查、回归测试和人工审批决定是否启用。

5. 失败驱动的局部优化：先优化一个窄环节

不要一开始就追求“Agent 自己重写自己”。更稳妥的做法是选择一个可度量的局部问题，例如工具调用失败率、结构化输出通过率或代码测试通过率，建立“收集失败样本—提出候选改动—离线比较—小范围发布”的闭环。

6. 经验库要会做减法

长期运行后，技能、规则和记忆会重复、过时甚至互相冲突。可以定期合并重复内容，标记失效规则，把长说明拆成按需检索的参考资料；删除或替换前保留版本，并检查相关任务是否退化。自我改进不应等于无限堆积历史。

7. 持续执行与主动复盘：不用每次都等用户提醒

定时运行日报只是自动化；定时分析失败、提出改动、验证后更新系统，才是在自动化之上建立自我改进流程。可以安排 Agent 周期性整理失败样本，生成改进提案，但最终采用仍应经过明确的评估门槛。

8. 多 Agent 分工：把执行、复盘和验收适当拆开

可以让执行 Agent 处理业务，复盘 Agent 整理问题，优化 Agent 提出候选方案，而独立的程序或规则负责验收。换一个 Agent 当裁判，不等于裁判就可靠；评估器应尽量不允许候选版本随意修改，并保留人工抽检和回滚机制。

9. 验证与版本选择：让改进真正进入下一轮

完整闭环应当是：发现问题，提出改动，评估候选版本，保留有效结果，再由采用了改动的新版本处理后续任务。对同一批评估任务比较旧版与候选版，记录成功率、成本、耗时和安全违规情况；通过门槛后再小范围启用，退化则恢复旧版。用于优化的样本和最终验收样本应当分开。

图 5 | 可以借鉴到 Agent 项目中的有界改进闭环。沿用已有原创示意图：先验证改动是否有效，再决定采用或回滚；能够持续迭代，不等于已经证明递归增益。

1.3.5 面试问答

❓ 1. 你知道什么是RSI吗？聊聊你的看法?

💡 **答题思路：**先介绍概念，再从**模型和应用侧**举例举例这个概念-再讲一下**四层能力**-最后结合现有产品讲行业现状-最后展望未来。

🎤 参考答案：

我理解的 RSI，就是**递归自我改进**：AI 利用现有能力改进自身，再由改进后的系统参与下一轮改进。

在大模型方向，它主要体现在模型训练和 Agent 运行系统的迭代上。模型侧可以利用反馈改进训练数据和方案，通过训练更新参数；应用侧则可以优化提示词、工具或工作流，不一定需要重新训练模型。

为了方便理解，我会把它分成四种程度：首先是记住经验，下次能够复用；其次是根据反馈更新系统，改变以后的默认做法；再进一步，是自主发现瓶颈、设计实验、筛选版本，组织自己的改进过程；更进一步，是改进能力本身也提升，越来越会发现问题和验证方案。

成熟产品目前更多体现前**两种能力**。比如 **Claude Code 的自动记忆、Codex 的技能复用**，体现经验积累；模型侧，DeepSeek-R1 利用强化学习，根据答案的验证反馈更新参数，体现反馈驱动的能力提升。但是对于改变组织自己的方式，提升自己改进系统的能力，这些特性，目前还体现在研究层面。例如 DGM 让 Agent 修改自身代码、评估新版本，再继续迭代，展示了受控环境中的自主改进，但是目前投入生产使用的Agent并没有体现出相关特质。

我认为这些的主要瓶颈是评估是否可靠、提升能否泛化，以及实验成本和自我修改风险。这也是未来模型侧和 Agent 侧需要继续突破的方向。

❓ 2. 你认为Agent发展的新的形态，或者说未来的趋势是什么?

💡 **答题思路：** 开放性问题，我们可以使用我们笔记里讲的2026年Agent新特性，结合最近发布的Agent来谈近期趋势，然后结合RSI的特性来回答长远的趋势。二者结合，组成这个答案。

☝🏻 参考：笔记「1.2 2026年 Agent 新形态」，其中「1.2.1 四款代表产品」介绍了 dots、Cue、Muse 和 Grok Bot。

🎤 参考答案：

我会分**近期产品形态和长期能力**演进两个层面来看。

近期，从最近26年发布的Agent, dots ,Cue等等，他们一些共同特点比如 **添加持续的云端环境**，用户关闭电脑后，已委托的任务仍能继续推进；**主动跟进**，不再每一步都等用户催，而是按时间、事件启动，或根据上下文提出下一步，比如 Grok Bot 把销售拓展设置成每周例行工作；**跨任务记忆**，记住多个项目的进度和用户偏好，减少重复交代。**开箱即用**，产品替用户承担更多部署和配置工作，比如 Cue 把邮件、电脑、自动化和记忆放进同一个助手配置界面。 让我看出近期Agent发展的**形态趋势**： **更加注重完成长时持续任务**，**降低Agent的使用门槛。**

长期演进来看，我觉得Agent发展会 Agent **从执行任务，走向自主改进自身的工作方式，实现RSI的特性。**它不再只是按照人设定的流程优化，而是**自己发现能力瓶颈，判断优先改哪里，提出方案、设计实验**，再根据结

果决定保留还是回滚，并由新版本继续下一轮迭代。更进一步，**它提升的不只是业务能力，还有改进自己的能力**：能更准确地定位问题，用更少的实验找到有效方案，在相同资源下推动更高效的下一轮改进。最终不用人的干预，也能不断地进化，衍生出更强的能力。

1.3.6 讲解视频

https://www.bilibili.com/video/BV14anA66Eo4/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

2 智能预约Agent项目

先和大家讲一下这个项目的背景。这个项目是我转行大模型，自己做的第一个项目。当时还是小白，一边学Agent的课程，资料，了解什么是大模型，什么是RAG，什么是Agent，然后也看了一些书籍。慢慢开始摸索着做的（大概在25年的5月份左右）。做完这个项目，结合公司的项目（因为公司有一些类似于Hackathon)的项目，做了另外一个项目。两个项目其实难度和实现上差不太多的，我现在看来有点重复，当时可以第一个项目做Agent，第二个项目做RAG, 做两个重复的Agent，收益比较低。在11月份开始面试，也就是用这两个项目写在简历上。现在来看这两个项目其实也不是说多复杂，也会被面试官说简单，不过思路还是我说的那样，一边面试，一边听反馈，一边包装，就用了2个月，12月份，确实拿到了很多offer，包括薪资年包80多万的offer。

今天把这个项目分享出来。首先呢，这个项目我肯定不会再去开发新的内容，功能。我会结合我的经验，告诉大家如何使用它，如何包装它，如何扩展它，如何写简历，以及提供一些面试真题（毕竟这个项目天然的优势就是我直接从对应的面试实战内容就可以搜集到当时面试官围绕这个问题的面试真题）。 如果后面确实有用这个项目去面试，有不懂的面试问题，也可以反馈，我们像RAG项目一样，来根据粉丝朋友的反馈，提供更多的该项目的面试真题解析。

在这个章节除了介绍项目部分，其实还蕴含了一些我的思路，比如设计项目的思路，面试的经验，如何准备自己的项目等等。项目的README的部分，也写了一些关于项目的介绍，技术栈，架构之类的，大家可以看看。

最后提一点，我不太清楚大家会怎么用这个项目，会不会有人拿去面试之类的。这个更新也其实不在更新计划中的（本来打算再专门做个Agent），只是很多人问想要参考，所以我放出来。我目前用了大概3天左右整理，如果后面像rag项目一样，有人拿去面试了，有了面试真题不知道怎么回答，可以再来反馈给我，我来解析在当前章节中。

2.1 项目介绍

先说说项目是干啥的。他是我当时在按摩房按摩的时候，我发现这个按摩房，是由一个前台来打电话，处理客户信息，分配按摩的师傅，管理谁去上下钟，处理支付内容。所以我当时就觉得这个工作完全可以由AI来完成

啊。有些按摩房他是小程序一个表格，根据空闲日期选择技师。这样很死板，为啥这个按摩房由人工预约，就是人更灵活，能够处理用户喜好，倾向，支付，咨询等各种内容。 当时我在自学大模型，我就想着在大模型的时代，这个东西不就完全可以由AI来完成吗？所以就做了这个项目。

简单介绍这个项目，就是一个智能的按摩房的预约Agent。当用户和他对话，它能够处理预约内容，咨询内容。还能够根据用户的预约历史记录，生成一些回访信息，比如他很久没来了，自动生成一条个性化问候，问他最近要不要按摩。具体功能展示可以看这个视频：

https://www.bilibili.com/video/BV16FRbBEEwD/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2.2 项目设计思路

先来讲述一下我当时的设计思路，我想设计的新特性。

> 1： 首先项目是**多Agent**，现在复杂的Agent肯定是多个Agent协作的。所以我设计的系统有4个Agent。前三个是分类机器人，预约机器人，咨询机器人。他们三个要协同工作。

分类机器人只管分类，如果是和按摩房无关的内容就拒绝回答，如果是咨询相关的内容，就把任务给咨询机器人。

预约机器人会问清楚用户的需求，进行多轮对话，收集到信息后，去进行数据库查询，找合适的技师。

咨询机器人回答用户相关咨询问题，会从内部私有文档中找答案。

第四个Agent是用户行为分析Agent，它会根据用户历史预约，分析他的喜好，来的频率，生成回访信息（这块只是大致跑通了）

2. **集成RAG。** 我当时有这个意识，我希望我的Agent项目有RAG的技术栈。所以这个预约机器人，当用户给出了喜好技师（比如喜欢力气大的），会从维护的技师专长表格中匹配。包括咨询机器人就更是RAG了，它内部有一个专有文档，这个文档记录着按摩房的信息，咨询机器人会从这个文档找知识作为回复。我当时只是把这一套跑通了，没有用专业数据库，只是跑通了RAG的流程。

这块其实现在可以用我们笔记的RAG项目包装，包装成这个Agent使用的RAG是自研的RAG系统，那么工作量和复杂度一下增加了。

3. 用了MCP。 当时MCP 很火，所以我里面使用了MCP，在用户预约成功后，会调用天气的MCP SERVER，查询当地的天气，然后生成回复。比如：您预约了28号xx技师按摩，北京的天气是40度（调用天气mcp server），请注意防晒。
4. 反思功能。为什么设置用户行为分析Agent，是因为我有这个意识，想让我的Agent有反思功能，根据历史的输入输出来调整行为，生成个性化回复。

来分析一下我当时的设计的优缺点：

优点：做了多Agent，用了MCP（当时火的技术），有工具调用，有RAG的意识，也在尽力设计高级特性（自我反思）。现在的我当然觉得整个设计简单，我后面会讲如何设计得更复杂。但这个项目，不可否认，就是我自己设计，摸索的入门项目，不管会不会被面试官说简单，但毫无争议，他其实就是帮助我入门转行LLM，有去面试的项目。

缺点：

1. **立意可以更复杂：**大家在设计项目的时候，一个好的立意其实很重要。比如当时很火的Agent形态是Manus。从设计场景，按摩房预约。 形态：对话Agent。这个立意不能说高明。我之前笔记有提到Anthropic 公司举办的黑客松大赛，大家可以去看看他们前10名的立意，形式，来参考，选择更好的场景和立意。
2. **设计的简单性：**太多地方可以设计得更复杂，比如记忆模块，沉淀能力，RAG系统，工具调用等。这个后面再说。不过其实作为一个初学者，自学者，没人指点的情况下，我觉得设计成这样，也无可厚非。毕竟我当时的水平是要先理解啥是LLM的。不过我后面会教大家如何设计得更复杂。
3. **编程范式：**这也不能怪当时的我。当时连SKILL都没出。我当时编程的思路就是使用copilot，提一次需求，审核代码。不过当时大家不都这样吗？ 但是大家在设计项目时，强烈建议使用现在vibecoding的方法，harness技巧（这也是面试常考点）。把时间放在建设harness上，而不是放在让AI生成功能上。

2.3 项目包装

项目包装其实也可以叫做项目扩展。可包装的点也是你可以扩展的点。一般做任何项目，都建议大家去包装一下。我觉得这是一种技巧，面试不能完全实事求是，得吹。人人都在夸大了说，你不夸大说，哪有这么实诚的人？但是你说能不能不做项目？我的建议不是不能：原因是：包装是提分技巧，你项目做了80分，你包装一下变成100分。你是100分，包装一下成120分。你不可能不做那是0分啊，你只能包装到20分。而且包装的东西，其实讲起来虚，没真实做过，遇到很懂的面试官多少会露馅。你小包装一下，有的地方说不上来，面试官觉得你可能忘了，记不清细节，或者稍微夸大。 假设你没做过，包装太多，被问出来了，面试官觉得你在造假，甚至给你写在面评上，说你虚假简历，性质是不一样的。下面说包装方向：

1. **和RAG项目结合。**告诉面试官该项目用的自研RAG系统，接入我们这个笔记中的RAG项目。工作量，复杂度直接增加。
2. 同理，**可以包装一个算法项目。**比如某个地方简单任务用了自己后训练过的模型，体现算法的能力（这块等我们后面算法项目出来了可以再想具体场景）
3. **记忆系统的设计。**这个项目没有记忆系统。长短期记忆如何实现？上下文对话满了如何压缩？这些策略需要设计。再不济你就抄袭我们解析的开源项目（cc,hermes,openclaw），把内容丢给ai, 让AI给你设计和写代码。但是你不能不设计！
4. **和后端内容联动。**这是一个本地的项目，能不能融入一些后端，部署，打包，微服务的东西，面试爱考。
5. **更复杂的架构：**现在项目是一个静态的Agent。但是我们解析的开源项目，它是动态生成子Agent的。设计更复杂的架构，比如react范式，让主Agent用工具调用的方式fork出子Agent。（这个也是参考笔记里所有开源项目）

6. **SKILL：** 现在的Agent需要有SKILL，就像我当时设计项目往MCP上靠。能不能融入一些SKILL技术。
7. **Harness:** 能否包装一下你的写这个项目的过程，告诉面试官你是用harness技巧写出来的。了解Harness的详细特性，融入到项目中。参考资料
8. **Agent 性能评估：** 建议一定要做。也是在我真正面试的时候，我才发现你如何评估Agent的性能的时候，是一个必考题。但是在我设计项目的时候（当时还没有面试，没有这个概念）。我的做法就是包装，我结合了参考资料《Agentic Design Pattern》里的评估模块，和美团龙猫的论文，编了一个自己的方法，如何做的评估，每次就不断改进，慢慢面试也就过去了。如果有时间，还是建议大家做。具体的理论方法在 **Agent性能评估。**
9. Langchain用到的是0.3的版本，可以换成1.0. 我个人其实觉得Langchain这块就是个工具，面试一般问的也不深入。不过基本上问你的项目，如果你用的Langchain，都会考一下Langchain的特性，用了啥版本。现在Langchain也有更新的版本了，如果面试大家觉得有必要，可以改成1.0。其实很容易，让AI给弄就完了。或者你不改，你就说你用的1.0, 了解下1.0特性，一般面试都够了。

2.4 配套设施

配套了一些SKILL，和之前RAG系统高度类似。

resume-writer skill：写简历SKILL。我必须说我对写简历不算擅长。不过我保证这个SKILL写出来的简历比我当时用AI写得好。因为我告诉他了模板，突出的点，如何包装。当然希望大家能够生成后自己多修改。我必须提到的一个点是，其实背景越好的朋友，越不需要太写简历，我不是凡尔赛。而是很多时候，不管校招还是社招，凭借学历+大厂，我简历写得很烂，但是还是有面试机会，所以我对简历没怎么改。但是越是你自己比如投出去没反应，没效果，背景差，越是要好好写，根据反馈反复改，甚至是不同岗位用不同简历。

Project-learner skill： 让AI带着你了解这个项目。通过用这个SKILL和它对话和打卡。了解这个项目。

Interview-prep skill：根据这个项目，你的简历，让AI对你模拟面试。

Setup-environment skill: 让AI一键帮你配置好环境。

2.5 使用建议

1. 作为学习项目跑着玩。感受一下什么是AI Agent，function call, multi agent, mcp.
2. 拿去面试：时间特别急，没时间做项目的，那你就了解这个项目，跑起来，用上面这两个project-learner skill和interview-prep skill准备好项目，准备好包装技巧和说辞，去面试。 时间长一点的，就按照上面的项目包装的技巧，选择一些自己想做的点，去在这个项目基础上改进，增加复杂性。 然后再去面试

2.6 面试真题

这个项目天然有一些面试真题，就是截取自我自己的面经，也就是在面试实战部分。 我汇总了一下我之前面试时，面试官对于这个项目的面试真题，供大家参考。其实这些面试问题，虽然是问的按摩房预约多Agent系统，其实都还挺有通用性的。可以结合自己的项目，想想这些问题，绝对是有收获的。

1. 简单介绍你的项目按摩房智能预约系统，怎么去做的?
2. 介绍一下按摩房预约系统。
3. 按摩房预约系统是你现在在做的吗?
4. 你的项目是如何分块的?
   这里笔记里的回答明确写的是“在按摩房预约系统中，我根据不同场景使用不同分块策略”。
5. 你的 RAG 系统中数据库是如何存储的?
   这一题紧跟在“按摩房智能预约系统”后面，属于对这个项目 RAG 存储方案的追问。
6. 你的 RAG 系统是怎么去评估系统质量的?
7. 为什么用 LangChain 不用 Semantic Kernel?
   这一题出现在“介绍按摩房预约系统”之后，是对这个项目框架选型的追问。
8. 为什么设计成多 Agent?
9. 有没有涉及同时要调多个 Agent，或者多个 Agent 之间还有依赖关系?
10. 端到端的延迟是多少?
    笔记里明确说这是 Agent 从输入到第一个输出的延迟，出现在按摩房预约系统追问链路里。
11. 你评价你的预约系统 Agent 好坏的标准是什么?
12. 你的这个 Agent 项目任务完成得怎么样，有没有去学习/反思能力?
13. 有没有使用一些知识问答的功能?
    也就是问这个项目有没有做 RAG。
14. 你讲到的两个项目，两个项目最大的区别是啥？能否具体结合你的项目讲?
    笔记里回答明确对比了邮箱项目和按摩房预约系统，并说明按摩房适合中心化多 Agent。
15. 多 Agent 系统和单 Agent 系统最大的区别是什么？彼此有什么优势?
    这题本身是通用题，但笔记里要求结合按摩房预约系统回答，所以可以纳入“按摩房项目相关追问”。

2.7 项目源码

https://github.com/jerry-ai-dev/smart-appointment-ai-agent

---

3 模块化RAG项目

3.1 项目介绍

对于项目完整的技术方案，请看项目仓库的Dev_SPEC.

对于项目的使用建议，项目仓库的Readme.

其它的一些实际思路，比如项目如何启动，从0到1的过程，项目如何配置环境，如何开发的，如何写的SKILL,SPEC，以及一些如何面试前的准备，请看视频讲解。

视频讲解，我提供完整的思路，从0开始，到立项，设计思路，开发，测试，写简历，项目前突击都展示了对应的思路。更多开发过程可以看我的小红书，Rag-agent项目实战，里面记录了一些开发过程的心得，但是精华都在代码仓库和下面视频中了。

如果进行突击前的准备，比如项目会问哪些问题，那就请看模拟面试SKILL。用这个方法，带你复习整个项目70多个知识点和八股，并进行模拟面试。 项目基本整个流程已经完毕，**不会增加新的功能了。**请各位需要深入扩展，**比如加一个新功能，自定义评估模块，垂直业务适配 大家按照思路自己扩展就好了。看看我上述的思路，扩展应该并不难。**具体的项目八股部分，我没有总结，但是请你使用模拟面试SKILL的方法，让AI带你过完整个项目。我想要继续总结的是真实场景下，该项目的面试问题。

项目基本完成了，我希望后面补的内容是第四部分，就是真实的面试记录和问题：**如果有同学想要投稿，使用整个项目面试了，可以提供给我真实的视频素材（音频），我会帮你总结面试问题**，给一些点评，并会将涉及这个项目的面试问题复盘，总结在笔记中，这样大家可以一起进步。 同时我自己也会面试，把这个项目往简历写，遇到了问题也会总结。

3.2 Github地址

GitHub上的Dev_spec是更完整/更新的版本。推荐Git仓库看整个Spec, Spec甚至是包含了完整的目录结构和项目管理，包括上传了Skills 教大家如何用AI来完成整个项目。有兴趣欢迎尝试呀！

https://github.com/jerry-ai-dev/MODULAR-RAG-MCP-SERVER

3.3 面试真题

这一小节会展示真实的面试问题。非常欢迎/鼓励大家投稿，最好是以音频的形式，因为这样我能够分析出来完整的问题，其实你看我的笔记做面试真题的部分，我都是拿着录音一个字一个字扒的，这样不会漏掉细节，也能够体会当时的氛围，回答的语气。即使不给我录音，也建议大家自己面试录音，然后自己复盘的时候就根据录音去听细节。 当然并不是每个小伙伴都愿意提供录音，毕竟有一些隐私性，那么你可以尽量回忆当时的细节，可以给我文字版，我会把你的内容整理，告诉你怎么改进，也会记录在文档中，也是一个一起学习的过程。

本节呢，会展示该项目的真实的面试问题，素材来自于粉丝投稿和自己面试。粉丝投稿有时细节可能就有限了，因为可能是以回忆的文字形式整理的。但是我自己面试的话，那么就会比较全面。

3.3.1 小哥A投稿

3.3.1.1 你的项目是从Github找的吗?

从这个问题的反馈上，我觉得老哥可能比较诚实。我估计小哥是直接把我的Git项目名字直接写上去的。具体细节怎么写的我不太确定了。但是！更合理的做法是包装呀，我其实在readme中写了，在我的项目的

Resume-Writer SKILL也明确让他使用了包装的技巧，我的思路一直是你要把这个项目和你的业务去结合，编也要编一个，而且你用我项目里的SKILL就能编出来，用它生成的简历最少面试官看上去不是直接问你是不是Git上找的项目啊！

一个项目是结合自己的工作/学习自己写的，它是有背景/需求/痛点/解决方案/设计思路，它是有故事讲的。你直接这么写，人家觉得你是Git上找的项目，首先第一个怀疑的是你自己到底做了多少。 所以我的第一个建议是简历重新写一下，至少用项目这个SKILL写一个简历出来看上去面试官也不会问这个问题。

3.3.1.2 PDF不规范、乱码多的情况下，MarkItDown转换效果不好怎么办?

思路：其实这个问题挺好的，在我做完这个项目的时候，我和大家说的，我强烈的建议是你真的把它用在你的业务场景中。这时你可能会遇到各种问题，比如输入文档质量不好，太大，或者文档中的表格处理不到。这块你遇到的问题和解决思路就是很好的经验和面试问题。 你在解决了他们的时候，可以提高系统的准确性，可以讲出来为啥你的系统效果更好，因为你可能做了xx针对性的调整等经验。

所以这个问题其实是该项目在真实场景下可能会遇到的问题，值得我们去思考。当然答案其实不唯一，我也只能纸上谈兵，给一个我思考的答案。

参考答案：

这个问题我们在设计时是有考虑的，整体是**三层防线**的思路，采用 Garbage In Garbage Out的思路。

第一层，Pipeline 入口有**文档质量检测的前处理。** 文档进入系统后，Loader 不会直接无脑解析，而是会先做一轮质量预检：提取前几页文本，计算有效字符占比和可识别文本密度。如果有效字符率低于阈值（我们设的是 80%），说明这个 PDF 的文本层本身就有严重问题，这种文档即使硬塞进去，后面切出来的 chunk 也是垃圾，检索质量根本没法保证。所以直接拒绝入库，在 Dashboard 上给用户一个明确的提示：'该文档质量不达标，请检查后重新上传'。与其让脏数据污染整个知识库，不如在入口就拦住。（注意哦，这里其实我们项目并没有做这个前处理，你可以加进去，你试试让AI完成，它能很快完成。另外即使你没做，想清楚了怎么做的，大胆说就行了，面试的时候真的不用完全实事求是）

第二层，**从业务场景来说，这个问题在实际中的影响面其实有限。** 我们这个系统定位是**企业内部知识库**，文档来源主要是团队内部产出的技术文档、产品文档、会议纪要这类——这些基本都是 Word/LaTeX 导出的文本型 PDF，格式规范、编码正常。MarkItDown 底层用的是 pdfminer.six，对这类标准文本型 PDF 的提取准确率是很高的。所以在我们的实际场景中，大部分文档走主路径就能处理好。

第三层，**即使文本提取有轻微的噪声，后面的 Transform 阶段还有 LLM 做兜底。** 我们在 Splitter 切分完之后有一个 Transform & Enrichment 阶段，会用 LLM 对每个 Chunk 做智能重组——合并被物理切断的段落、剔除页眉页脚和残余乱码、补全上下文。所以对于那种'**大部分正常、偶尔几段有点问题**'的情况，Transform 阶段的 LLM 去噪能力是可以兜住的。

退一步讲，就算有**少量噪声数据漏进了知识库，也不会影响最终检索质量。** 因为乱码文本生成的 Embedding 向量在语义空间中是没有方向的噪声，跟用户任何有意义的查询之间的相似度都会很低，检索排序时自然就沉底了。所以本质上就是浪费了一点存储空间，但不会污染检索结果。

3.3.1.3 图转文用的视觉模型有什么考量?

思路：其实也是一个非常好的问题，面试喜欢问，你为什么选xx的模型。不光是这个RAG项目的视觉模型，还有你自己做Agent，这些都是常考的问题。有人说我spec写的细，问我怎么写出的spec, 其中一个因素就是经验啊，我的学习和面试，告诉我什么需要考虑。比如通过这次面试，总结了这个问题，我就会对是觉得模型多考虑一层。但是现在分析看来，我还是有些东西没有考虑到， 比如**文本质量差如何前处理，视觉模型的选型。** 总结这些问题的时候，还要思考，查资料。 项目要考虑的东西还是很多的，经验这个东西还是学无止境的，我们慢慢积累，你看这个笔记，以及做项目，以及面试，都会增加你的经验，让你以后写spec，考虑得更全面。

答案：

首先，传统RAG处理图像主要有两条路线：一种是用**CLIP这类多模态Embedding模型**，把图片和文本映射到同一个向量空间，做视觉相似性匹配；另一种是用**Vision LLM做图转文**，把图片内容理解成自然语言描述，然后复用文本RAG链路去检索。我们选的是图转文。因为我们的业务场景是内部技术文档，图片基本都是架构图、流程图、表格截图这些，用户问的是'调用链路是什么'、'超时配多少'这种具体的技术问题，要的是图里的含义不是外观。CLIP匹配不到这个层面的语义，Vision LLM可以。

然后，具体选哪个Vision LLM，我们用了**双模型策略。**英文文档和复杂图表主选**GPT-4o，理解能力最强；中文场景主选Qwen-VL-Max，中文OCR和图表理解更好，价格大概GPT-4o的三分之一到五分之一，国内访问延迟也低。通过配置切换，代码层是解耦的。**

最后，用法上也做了一些优化。不同类型的图片用不同的结构化prompt，比如流程图侧重提取节点关系，数据图表侧重提取数值和结论。调模型的时候会把图片前后的文本段落一起传进去，让模型理解图在文档中的上下文。另外做了内容哈希缓存，同一张图不会重复调用。

（这里我设想的我的RAG场景是业务内部技术文档。所以有如下的答案。我希望大家要参考这个思路，比如CLIP和Vision LLM, 对于不同提示词的使用，然后针对性修改，来放在自己项目的回答中）

3.3.1.4 项目的一些常规问题

思路：小哥没有给我展示具体的问题。但是他说他这里基本答上来了。可以预测，可能是一些常见的项目问题,比如如何做的粗排，精排。如何用的双路检索，数据库存了啥。这个偏项目的细节以及基础知识。这里其实也有另一个小哥（他都拿到offer），他在群里的反馈是把博主的RAG项目倒背如流，拿到了暑期实习Offer. 这部分就是项目的细节，我这里设置了 **模拟面试和项目细节的SKILL。** 你去用它，一定会给你把项目讲得很清楚，所以我没有总结，但是这些也很重要，涉及基本知识和你的项目细节。 我思路都做好了啊，你用我的SKILL去学就完了。先用项目细节的SKILL，它把整个项目分为几十个知识点一个个打开，你学习。然后他帮你模拟面试，你去检验就行了。 这块没有什么技巧，答不上来就是功夫没下够，按我建议的方法去学就行。

3.3.1.5 本地有没有实现过

思路：面试官问这个问题，还是这个小哥简历写得太老实了，都没包装，面试官觉得是从网上找的项目，才会这么问，见3.1.1，结合自己业务包装了，面试官肯定不会问这个问题呀。

3.3.1.6 有没有了解RAG的新方向

思路：其实方向还是挺明确的，Agentic RAG 和 Graph RAG.我的复习思路是先去理解概念，当面试官问到相关问题，通常第一个问题都是解释概念，问你了解不，然后就按照以下内容答就行了。**我暂时不会深入学习。**为什么呢？因为不是说每个人都需要把RAG学得比较深的，RAG如果脱离了Agent，它也不算大模型方向了。所以在大模型方向Agent的重要性比RAG重要，RAG基本的知识就是我们的项目相关的。学习好这些个RAG基本相关的内容，是基础，有余力当然可以去学这两个方向，或者做相关项目。但是并不是每个岗位都需要很深的RAG，深入到Agentic 和 Graph，个人精力也有限嘛。所以我个人会先学习概念，然后我目前是在学算法，微调这个，考的也很频繁。总而言之，我不深入学习Agentic 和 Graph RAG，是因为在我这里他优先级没排上。

但是提醒大家的还是根据自己的情况去分析。因为有的人的岗位就是偏RAG这一块的，那他肯定就需要把RAG多学一点。如果你的目标岗位是宽泛的，比如大模型应用，大模型算法。可以先把其他的内容，Agent，RAG基础内容，微调，推理相关的东西学好了，再来。

还想提醒的一点是个人面试技巧。确实也是有个小哥问我，他说我微调只会一点，但我简历写了。明天面试要问了，我要不要主动承认我微调经验少啊。我的答案是不要，这个问题和这个Graph Rag类似。我们不主动认怂， 比如面试官问你了解Graph Rag吗？我就说我了解！然后告诉他Graph Rag是啥。真的问到了你不会的地方，你再说不会，不要认怂。很多时候面试官问题也只是流于表面，问你了解不，你说了解，然后答了概念，这个题就过去了，你说你主动先承认你了解不深的意义是啥子呢？

答案：

Agentic RAG的核心是把RAG从一个固定流水线变成一个有决策能力的Agent。传统RAG就是检索一轮、生成答案、结束了。但很多复杂问题一轮检索根本不够，Agentic RAG让模型自己判断——检索结果够不够？不够就再检索一轮，或者换个检索策略。像Self-RAG、Corrective RAG都是这个思路，本质就是让检索过程变得更智能、可以自我纠错。

Graph RAG解决的是另一个问题——传统向量检索擅长找语义相似的内容，但很难处理跨文档的关联推理。比如问人物之间的关系，信息散落在十几个段落里，靠embedding相似度很难串起来。Graph RAG的做法是先从文档里抽取实体和关系建成知识图谱，检索时在图上做关系遍历，特别适合需要全局理解的问题。"

我觉得这两个方向是互补的——一个解决检索策略的智能化，一个解决知识关联和推理，未来可能会结合起来用。

3.3.2 离线数据处理

总结这个专题其实来自于粉丝的投稿。他说小红书上有一篇文章讲 RAG离线解析 ，也是以他们项目是输入PDF格式构建数据库的。文章核心就是：强调离线数据构建的核心，强调它的重要性，提供了一些离线构建的问题。这个文章可以作为我们准备面试的思路，我是建议大家真正跑一些Pdf试一下，就可以知道具体可能会遇到哪些问题。我个人是没有做这些事，但是没关系，我们想清楚，准备清楚一些问题，以应对面试官可能的考察。看了这个文档，我觉得我们项目在离线解析这一块，还是做得比较全的，我提取一下这个笔记中讲到的离线解析的问题，预测一些可能的考点和答案，供大家参考：

1. 你们项目处理的文档都是什么格式？PDF 多栏排版你是怎么处理的？

我们项目处理的文档以 PDF 为主。用 MarkItDown 把 PDF 统一转成 Markdown，它能识别段落、标题、列表等结构化元素，比纯文本提取更好地保留文档逻辑结构。同时用 PyMuPDF 做图片提取，确保图文信息不丢失。后续通过vision llm来提取图片内容，实现图片的检索。

2. 你们文本分块是怎么做的？如果一段完整的业务流程被从中间切断了怎么办？

我们用了多层策略来尽可能避免切断，以及切断后的补救机制：

避免切断： 用 **RecursiveSplitter**，按分隔符优先级递归切——先尝试在段落（`\n\n`）处断开，不行再降到换行（`\n`），再降到句号（`.`），最后才是空格。也就是说它会优先在自然语义边界切分，而不是无脑按字符数硬切。一段完整的理赔流程如果在一个段落内，就不会被拆开。

切断后的容错： 设置了 **chunk overlap 重叠窗口**，上一个 chunk 的尾部几句话会同时出现在下一个 chunk 的开头。即使流程被切在了中间，上下两个 chunk 都保留了切断点附近的上下文，检索时任何一个被命中，都能拿到切断点前后的关键信息，**不会出现"半句话"的情况。**

检索阶段兜底： 每个 chunk 都带有 `chunk_index`（序号）和 `source_ref`（父文档 ID）。如果检索命中了某个 chunk，系统可以根据 chunk_index 找到它的前后相邻 chunk，把上下文拼回来送给 LLM 生成答案。这样即使切分不完美，在检索和生成阶段也能还原完整信息。

后处理兜底： Transform 阶段的 ChunkRefiner 会对每个 chunk 做规则清洗和可选的 LLM 精炼，确保每个 chunk 是语义自包含的。LLM 调用失败自动降级为纯规则，不阻塞流水线。

3. chunk 大小你是怎么定的？太大太小都有问题，你怎么平衡的？

chunk_size 和 chunk_overlap 全部通过 `settings.yaml` 配置驱动，不需要改代码就能调。整个 Pipeline 是工厂模式架构，Splitter、Embedding、VectorStore 都通过 Factory 实例化，切换策略只改配置。实际调优时结合下游 LLM 的上下文窗口大小和评估指标来实验确定最优值。而且在线阶段我们用了 Dense + BM25 混合检索 + Cross-Encoder Rerank 的三级策略，先大范围召回再精排，减少了对单个 chunk 粒度的过度依赖。

4. chunk 大小你是怎么定的？太大太小都有问题，你怎么平衡的？

我们最终定的是 chunk_size=1000 字符，chunk_overlap=200，这个值是综合三个因素平衡出来的：

第一个因素是 Embedding 模型的输入上限。 我们用的 Embedding 模型 token 上限一般在 512-8192 之间，1000 字符大概 300-500 token，留够余量不会被截断，保证每个 chunk 完整编码。

第二个因素是 LLM 上下文窗口的承载量。 在线回答时我们通常召回 top-5 到 top-10 个 chunk 拼成 context 给 LLM。如果 chunk 太大比如 3000 字符，5 个就是 15000 字符，很容易撑满窗口、挤压生成空间；1000 字符的话，10 个也才 10000 字符，LLM 还有充足空间做推理和生成。

第三个因素是语义完整性。 太小比如 200 字符，一个 chunk 可能只有一两句话，语义残缺，检索精度反而下降——一句话脱离了上下文，Embedding 向量质量也差。1000 字符大约是一个自然段落的长度，能包含一个相对完整的信息单元。

200 的 overlap 是 chunk_size 的 20%，目的是让相邻 chunk 在切断点处有足够重叠，不会因为切分而完全丢失边界处的上下文。

这些参数都在 settings.yaml 里配置驱动，不用改代码。实际上线前我们可以通过评估模块跑对比实验——不同 chunk_size 下的召回率和生成质量——来验证和微调。

5. 输入的数据有很多噪声数据，质量很差怎么办?

这个问题我们在设计时是有考虑的，整体是**三层防线**的思路，采用 Garbage In Garbage Out的思路。

第一层，Pipeline 入口有**文档质量检测的前处理。** 文档进入系统后，Loader 不会直接无脑解析，而是会先做一轮质量预检：提取前几页文本，计算有效字符占比和可识别文本密度。如果有效字符率低于阈值（我们设的是 80%），说明这个 PDF 的文本层本身就有严重问题，这种文档即使硬塞进去，后面切出来的 chunk 也是垃圾，检索质量根本没法保证。所以直接拒绝入库，在 Dashboard 上给用户一个明确的提示：'该文档质量不达标，请检查后重新上传'。与其让脏数据污染整个知识库，不如在入口就拦住。（注意，这里其实我们项目并没有做这个前处理，你可以加进去，你试试让AI完成，它能很快完成。另外即使你没做，想清楚了怎么做的，大胆说就行了，面试的时候真的不用完全实事求是）

第二层，**从业务场景来说，这个问题在实际中的影响面其实有限。** 我们这个系统定位是**企业内部知识库**，文档来源主要是团队内部产出的技术文档、产品文档、会议纪要这类——这些基本都是 Word/LaTeX 导出的文本型 PDF，格式规范、编码正常。MarkItDown 底层用的是 pdfminer.six，对这类标准文本型 PDF 的提取准确率是很高的。所以在我们的实际场景中，大部分文档走主路径就能处理好。

第三层，**即使文本提取有轻微的噪声，后面的 Transform 阶段还有 LLM 做兜底。** 我们在 Splitter 切分完之后有一个 Transform & Enrichment 阶段，会用 LLM 对每个 Chunk 做智能重组——合并被物理切断的段落、剔除页眉页脚和残余乱码、补全上下文。所以对于那种'**大部分正常、偶尔几段有点问题**'的情况，Transform 阶段的 LLM 去噪能力是可以兜住的。

退一步讲，就算有**少量噪声数据漏进了知识库，也不会影响最终检索质量。** 因为乱码文本生成的 Embedding 向量在语义空间中是没有方向的噪声，跟用户任何有意义的查询之间的相似度都会很低，检索排序时自然就沉底了。所以本质上就是浪费了一点存储空间，但不会污染检索结果。

6. 面试回答：你们的离线解析是怎么做的?

（先讲挑战） 我们要处理多格式文档，主要是 PDF，也有 PPT 和扫描件。核心挑战四个：PDF 解析错乱、图片信息丢失、分块切断语义、缺元数据导致检索过滤维度不够。

（再讲方案） 我们设计了六阶段流水线：

SHA256 文件校验，未变更文件自动跳过，保证幂等。

文档解析，用 MarkItDown 把 PDF 转 Markdown 保留结构，PyMuPDF 提取图片并插入占位符。

语义感知分块，RecursiveSplitter 按段落、换行、句号等自然边界递归切分，不硬切；设 chunk overlap 200 字符重叠窗口防上下文断裂；每个 chunk 带 chunk_index 和 source_ref，检索命中后可沿序号拼回前后上下文。

Transform 三件套——ChunkRefiner 清洗噪音做内容精炼；ImageCaptioner 调 Vision LLM 给图片生成自然语言描述回填到文本，解决图片信息丢失；MetadataEnricher 用规则+LLM 给每个 chunk 打 title、summary、tags 标签，支持检索时多维过滤。三个组件 LLM 调用失败都自动降级，不阻塞流水线。

双路编码，Dense 向量 + BM25 稀疏索引，语义匹配和关键词匹配互补。

三路存储，VectorStore、BM25 Index、ImageStorage 各司其职。

（最后讲效果） PDF 解析错乱→MarkItDown 结构化转换从源头保证质量，不让乱码污染 Embedding；图片丢失→Vision LLM caption 回填解决；分块切断→递归切分+overlap+chunk_index 上下文拼接多层兜底；元数据不足→MetadataEnricher 注入语义标签，配合混合检索+Rerank 做多维筛选。chunk_size 1000、overlap 200 全部配置驱动，可以结合评估指标实验调优。

3.3.3 某大厂二面（来自粉丝B）

（3.3和3.4都来自同一个粉丝朋友。这个具体是哪个公司我有点忘了。非常感谢这个粉丝朋友的投递，我觉得里面问的问题都不难，是比较基础细致的内容，但是很深入，并不是每个公司都会问的这么深入的。非常有借鉴价值。但是其实不难，只要我们把项目理解透了，对应的知识点理解透了，包括我也汇总了这么多项目面经，自己多表达几遍，这些问题都不在话下。）

3.3.3.1 大模型增强部分是做什么的？有什么用？元数据都有什么？增强前后效果对比了吗？

（前三个问题，其实就是项目的最基本的模块，我设置的机器人都会带大家复习过。理解了以后，其实自己对于这些问题需要自己尝试表达一下，因为理解和表达流畅是不一样的。最后一个问题：前后效果对比了吗，我觉得就是编吧，或者叫自圆其说，结合我们做测评制定json的数据集以及使用ragas的经验，把最后一个问题编出来）

Q1: 大模型增强做了什么

A1: 我们的 Ingestion Pipeline 里 Transform 阶段主要围绕"脏、乱、不可搜、难展示"的痛点，设计了三个核心模块来进行数据增强，分别针对**纯文本、元数据，以及文档中的图片。**

第一部分是 ChunkRefiner（文本块精炼）。

主要解决 PDF 解析后的排版噪声。我采用了“规则兜底 + LLM增强”的两阶段处理模式：

- **规则兜底**：首先用正则低延迟地去掉连续空行、分页符和 HTML 残留，并精准保留所有的代码块。这一步是必做的，0成本覆盖 80% 场景。

- **LLM 增强**：对于规则搞不定的分页造成的段落断开、OCR 识别错别字，交给大模型进行上下文合并和修正，让文本变得连贯。

第二部分是 MetadataEnricher（元数据增强）。

它的作用是为每个 chunk 生成结构化的语义元数据（Title、Summary、Tags），方便后续做字段级别的检索过滤和结果展示。这部分当没有使用LLM的时候，会使用基于正则化匹配的方式提取这三部分，使用LLM的时候，会用LLM的语义分析功能来完成。

第三部分是 ImageCaptioner（图片多模态增强）。

这是解决传统 RAG 丢失插图信息的关键。解析阶段图片会被存在本地，并在原文对应位置留下 `[IMAGE: id]` 的占位符。增强阶段，`ImageCaptioner` 会利用正则扫描这些占位符，一旦发现便调用视觉大模型（Vision LLM）对图片进行跨模态解读。生成的详尽图像描述（Caption）不仅会被注入该区块的元数据中，还会同步回填进正文里，从而使得原本不可检索的图片转化为语义向量，最终实现了强大的“以文搜图”能力。

Q2: 元数据具体有什么

元数据主要包括4块内容：

第一个是 **title**，限制在 150 字符以内，用来描述这个 chunk 的主题。规则提取时，我的逻辑是有优先级的：首先用正则去匹配文本里有没有 Markdown 标题，也就是 `# Heading` 这种格式，如果有就直接用；如果没有标题，就看第一行够不够短、像不像一个标题（不以句号逗号结尾、长度不超过 100 字符），满足条件就用第一行；再不行就取第一句话截断到 150 字符。LLM 增强时会理解语义后生成更准确的主题描述。这个字段主要用于检索结果的展示和快速浏览。

第二个是 **summary**，限制在 500 字符以内，是 2 到 3 句话的核心摘要。规则提取的做法比较简单，就是按句号、感叹号、问号做分句，取前三句拼接起来。LLM 增强的话，生成的摘要概括力会更强，不是机械截取前几句，而是真正理解内容后提炼要点。这个字段用于结果预览和上下文补充。

第三个是 **tags**，3 到 10 个关键词标签。规则提取是从文本里用正则抓三类东西：大写开头的专有名词（比如 Azure、OpenAI）、camelCase 和 snake_case 风格的代码标识符（比如 `text_embedding`、`baseTransform`）、以及 Markdown 加粗标记 `...` 里的词——因为作者特意加粗的通常是关键概念。LLM 增强的优势在于它能推断出文本中没有直接出现但语义相关的标签，比如文档讲的是微服务拆分，LLM 可能会生成"分布式系统"、"SOA"这类语义关联的标签。tags 主要用于分类过滤和关联检索。

第四个是 **enriched_by**，标记这条元数据的来源。取值有三种：`"llm"` 表示是 LLM 生成的，`"rule"` 表示是规则提取的，`"error"` 表示处理过程出了异常。如果 LLM 调用失败降级到了规则结果，还会额外附带一个 `enrich_fallback_reason: "llm_failed"` 字段，便于事后追溯哪些 chunk 的元数据质量可能不够理想。

第五个是 **image_captions**，这个是 ImageCaptioner 处理后追加的。如果一个 chunk 的文本里包含 `[IMAGE: id]` 占位符，说明这个 chunk 引用了图片，ImageCaptioner 会调用 Vision LLM 对图片生成描述，然后以列表的形式写入 metadata，每个元素是 `{"id": "img_001", "caption": "这张图展示了系统架构图..."}`。

Q3: 增强前后效果对比了吗?

做过对比。我们通过**两种方式来**评估增强效果.

第一种方式是在测试集数据中手动标注了一些典型的"脏乱"文本块，包含分页符、冗余空行等，然后对比增强前后的文本质量。增强前这些文本块往往是断断续续的，甚至有些关键信息被分页符分开了；增强后文本变得连贯，关键信息完整保留了下来。

第二种方式是，我们比较了我们的测试集，在将llm 增强打开和关闭后，使用ragas进行评分的结果对比，在打开了llm增强后，整体的 Fluency和 Coherence 评分提升了约 15%，说明增强环节确实提升了整体RAG系统的准确性和可靠性。

3.3.3.2 BM25 怎么构建的？稀疏向量还是倒排索引？多少维度？Dense 多少维？向量数据库支持稀疏检索吗？嵌入模型用了什么？为什么？

(其实这个题不难，是一些基础知识。除了项目本身如何实现的，需要理解的一些概念是：**BM25，稀疏向量，倒排索引，向量数据库。**这些部分在RAG八股里有，不懂的话其实就是建议让AI给你解释，让它给你举例子。明白了概念以后，其实就是照着项目实现回答面试官问题就好。面试官很有可能顺着这个问题问一些概念，比如稀疏/稠密向量的区别，BM25的原理，向量数据对比等，这些都是基础八股了，就不展开了，但希望你能自己掌握，特别是里面的**倒排索引和BM25的公式可能会复杂一点**，可以深入掌握一下原理，怕面试官追问。）

Q1: BM25 怎么构建的？稀疏向量还是倒排索引？

（这里其实可以深入了解BM25和稀疏向量的公式的，防止被追问）

我们的 BM25 构建分两步：

第一步建倒排索引。 SparseEncoder 用 jieba 对每个 chunk 分词、统计词频，BM25Indexer 汇总后建成 词 → posting list 的倒排结构，每条 posting 存 chunk_id、词频 tf、文档长度。同时预算好每个词的 IDF 和全局平均文档长度，整个索引以 JSON 持久化到磁盘。

第二步查询时在线打分。 query 分词后，查每个词的 posting list，用 BM25 公式对命中文档打分、求和、排序，返回 top-k。

用的是**倒排索引，不是稀疏向量。**因为稀疏向量需要向量数据库支持，我们用的 **ChromaDB 只支持稠密向量，所以自建了倒排索引，独立于向量数据库运行。**

Q2：多少维度？Dense 多少维？

BM25 是倒排索引，没有维度概念。它存的是 词 → posting list，不产生向量。

Dense向量的维度一般都是由不同的嵌入向量模型决定的，我们使用的是text-embedding-ada-002，维度是:1536 维. 比如用Ollama，则是768维。

Q3：嵌入模型用了什么？为什么？

（面试官就很喜欢问这种，为什么选择这个方案，出于什么考虑？老实讲有时候我们自己做项目哪里考虑那么多啊，每个地方都做技术选型和对比，毕竟又不是商业项目。但是面对这些问题的时候，我们还是得思考，编也要编一个理由自圆其说）

项目目前使用的是text-embedding-ada-002，维度是 1536 维。稍微提一句，整个项目采用的是工厂模式和插件注册的方式，所以其实修改不同的嵌入模型特别容易，只用准备好api，修改配置文件即可。我采用text-embedding-ada-002的原因是：

1. 成熟稳定、生态完善：ada-002 是 OpenAI 最早大规模商用的嵌入模型之一，社区文档和最佳实践丰富，踩坑少。
2. 性价比高：在 OpenAI 的嵌入模型中价格最低（\$0.0001/1K tokens），对于中小规模文档库来说成本可控。
3. 1536 维度够用：1536 维对于文档级别的语义检索已经有足够的表达能力，同时 ChromaDB 等向量数据库对这个维度的支持也很成熟。
4. 与项目混合检索架构契合：项目采用 Dense + BM25 混合检索 + Reranker 的三阶段召回策略，Dense 部分负责语义召回，ada-002 的语义表征能力足以覆盖这个场景；精排阶段还有 Cross-Encoder Reranker 做二次排序，因此对嵌入模型本身的排序精度要求可以适当放宽。

3.3.3.3 时间消耗、延迟测量了吗？每个环节多少？嵌入部分耗时多少？

（其实面试官喜欢问这个东西，不管是Agent还是RAG,这也是为什么我设计整个项目的时候，观测环节就加入了耗时，并且是每一步的耗时统计，大家跑Dashboard的时候在界面上可以看到的。这里告诉它每一步消耗，哪里耗时比较多，为什么，体现出我们工作的全面和真实性）

测了，整个系统从 Ingestion 到 Query 每个环节都有打点记录耗时，最终可以在 Dashboard 的 Trace 面板里可视化查看。

Ingestion 端到端，一份 10 页左右的 PDF，规则模式总耗时大概 2-5 秒，如果开了 LLM 增强会到 10-20 秒。其中加载、分块、存储这些环节都很快，加起来也就几百毫秒。耗时最突出的两个环节是 Transform 和嵌入编码——Transform 一旦开了 LLM 增强，因为每个 chunk 都要调一次 LLM API，光这一步就要 3-8 秒，是整条链路的最大瓶颈；嵌入编码调远程 Embedding API 大概 1-3 秒，占总耗时的 30%-50%，所以我们专门做了逐 batch 的计时，方便定位是网络延迟还是某个 batch 数据量大导致的慢。

Query 端到端大概 0.5-3 秒。Dense 和 Sparse 检索是并行跑的，加上 RRF 融合，检索阶段总共也就两三百毫秒。最影响查询延迟的是 Rerank，如果用 Cross-Encoder 大概 300-800ms，换成 LLM Rerank 就要 1-3 秒，基本决定了用户感知到的响应速度。

3.3.3.4 为什么进行混合检索？有什么作用？具体举个例子？是因为业界都这么做吗？

（其实这题很好，有点像个典型八股，就是问你为啥RAG一般都用双路检索，通过这个题了解一下两路检索的优缺点）

Q1: 为什么进行混合检索？有什么作用？具体举个例子？

我做混合检索的核心原因是：单一的检索路径都有明显的盲区，Dense 和 Sparse 的能力刚好是互补的。

Dense 检索是把 query 和 chunk 都嵌入到同一个向量空间，算余弦相似度来排序。它的强项是语义理解——用户问的和文档写的不是同一个词，它也能匹配上。但它的弱点也很明显：对精确关键词不敏感。比如用户查一个很具体的配置项名称，像 text-embedding-3-small 这种，语义向量可能会把它和一堆"嵌入模型"相关的段落混在一起，排序并不可靠。

Sparse 检索走的是 BM25 + 倒排索引这条路，本质是词频匹配。它对精确关键词、专有名词非常敏感，你搜 text-embedding-3-small，只要文档里出现了这个词，它就能精准命中。但它的短板是不理解语义——

用户如果问"怎么把文档灌进系统"，文档里写的是"ingestion pipeline"，BM25 根本匹配不上，因为一个中文一个英文，词完全不一样。

所以在我们的项目里，混合检索就是同时跑两条路：DenseRetriever 用 embedding 做语义搜索，SparseRetriever 用 BM25 做关键词搜索，然后用 RRF把两路结果合并成一个统一排序。RRF 的好处是它只看排名不看分数，不需要对两路完全不同量纲的分数做归一化，实现简单又稳定。

Q2: 是因为业界都这么做吗?

(怎么感觉有点压力测试的感觉，一路确实不行啊，我觉得上面的Q1的回答就已经回答了这个问题，不过既然他要问，我们就回答吧)

不完全是。业界确实普遍采用这个方案，Elasticsearch 8.x 加了向量搜索也是走的混合路线，各大 RAG 框架也都推荐这么做。但我在项目里选择混合检索更多是从实际问题出发的：我们面对的文档既有大量专有名词和代码标识符（BM25 擅长），又有很多自然语言描述和同义表述（Dense 擅长），单一路径的召回率确实不够。而且我们在设计上做了 Graceful Degradation——如果任何一路出错或超时，系统会自动降级到另一路继续工作，不会整个检索挂掉。这个容错能力也是混合架构额外带来的好处。

3.3.3.5 检索召回部分的 TopK 选的多少？为什么?

第一层是 Dense 和 Sparse 各自的召回阶段，TopK 都设的 20。 这一层的目标是"宁可多捞，不要漏掉"，因为这一步的成本很低——Dense 就是一次向量相似度查询，Sparse 就是一次 BM25 倒排索引查找，速度都很快。设 20 是因为我们测试下来，大部分查询的相关文档基本在前 20 条以内就能覆盖到了。如果设太小比如 5 或 10，有些相关文档可能在其中一路排名靠后就被截断了，后面融合也救不回来；设太大比如 50、100，虽然召回更全但会给后续的融合和 Rerank 带来不必要的计算开销。

第二层是 RRF 融合之后，TopK 设的 10。 Dense 返回 20 条，Sparse 返回 20 条，两路合并去重后可能有 30 多条，经过 RRF 融合排序后只取前 10。这一步相当于把两路的结果做了一次交叉验证——如果一个文档在两路都排得靠前，融合分数会很高；如果只在一路出现，也不会被丢掉但排名会相对靠后。取 10 条是给下一步 Rerank 一个合理的候选池。

第三层是 Rerank 之后，TopK 设的 5。 这一步会用 Cross-Encoder 对这 10 条候选逐一和 query 做精排打分，Cross-Encoder 的精度比双塔模型高很多，但代价是它需要把 query 和每条文档拼在一起过一遍模型，计算量是线性增长的。所以候选不能太多，10 条进 5 条出是一个比较平衡的选择。最终送给大模型生成答案的就是这 5 条最相关的文档。（其实代码用的llm Rerank，不过扩展cross-encoder很好扩展，cross-encoder也是常考的rerank模型，有余力可以深入了解）

为什么是 20→10→5 这个比例？ 核心思路就是"**粗筛多召回、精排少而精**"。前面的阶段计算便宜，多拿一些保证召回率；后面的阶段计算贵但精度高，逐步砍掉不相关的，最终给到 LLM 的上下文窗口里塞的都是高质量内容。这也避免了把太多文档塞进 prompt 导致 token 浪费和注意力稀释的问题。

3.3.4 字节一面（来自粉丝B）

（这个面经和3.3来自于同一个粉丝，所以其实是对同一份简历，不同的面试关的考察。这个面试官就没有问那么细，也确实不是所有面试官都能问那么细。3.3那个面试官问那么细，问这个项目估计都得花20分钟。所

以你要是准备好项目，整个面试60分钟的话，这个项目问题答得好，你已经成功了1/3了）

3.3.4.1 项目背景，为什么要做这个系统?

（结合着自己场景编吧，可以参考这个模板）

核心回答框架（三段式：场景痛点 → 现有方案局限 → 解决思路）

场景痛点（为什么要做）

我们xx 平台开发团队，日常涉及的版本发布信息非常多样——Release Notes、变更日志、补丁公告、兼容性说明、架构文档等，而且这些信息**分散在多个系统**里：内部 Wiki、文档仓库、Bug 系统、邮件通知等。工程师排查版本差异或回答客户问题时，经常需要**跨系统翻找**，效率很低，而且容易遗漏关键信息。

现有方案的问题（为什么不用现成的）

我们其实尝试过直接用 Copilot / 通用 RAG 框架，但遇到几个核心问题：

- **数据源多且异构**：我们有多个独立的知识库和文档系统，通用工具很难做好统一的索引和检索，召回率不够。
- **针对性不足**：通用方案对我们团队的特定术语、文档结构、查询模式没有做优化，检索精度不够，返回的内容经常不是工程师真正需要的。关键问题的命中率不到 50%。
- **过程黑盒**：传统工具你只能看到最终结果，不知道它检索了什么、排了什么序、为什么给你这个答案。出了问题没法调试，也**无法针对性地优化。**

针对性解决方案（问题 → 改进一一对应）

所以我们决定自己构建一套面向团队场景定制的 RAG 系统，**针对上面三个问题分别做了改进**：

针对"数据源多且异构" → 自建 Ingestion Pipeline，自主控制数据源

- 不依赖通用框架预设的信息源，而是**自己定义文档来源和处理流程。**设计了 6 阶段 Ingestion Pipeline，把团队散落在各系统的文档统一采集、解析、切分、向量化后存入同一个检索库，数据源完全可控

针对"针对性不足" → 两段式混合检索 + 精排

- 通用方案只用单路 Dense Embedding，对专有术语（版本号、API 名称）匹配差。我们做了 **BM25 + Dense 混合粗排**兜底关键词匹配，再加 **Cross-Encoder 精排 Rerank** 二次排序，把真正相关的内容推到前面

针对"过程黑盒" → 全链路可观测 + 自动化评估

- 搭建了 **Streamlit Dashboard** 可视化每次 Ingestion 和 Query 的完整 Trace（每阶段输入输出、耗时），集成 **Ragas 评估框架**做回归测试，改了策略能立刻量化效果变化

3.3.4.2 图片处理怎么设计的?

（没啥好说的吧，这个属于介绍你的项目了，很简单的问题）

我们的文档里有大量的架构图、流程图、截图，用户搜索的时候经常需要"搜文字能出图"。我的做法是**Image-to-Text**，就是把图片转成文字描述，然后复用纯文本的 RAG 检索链路，不引入 CLIP 这样的多模态向量库。

整个流程跟着 Ingestion Pipeline 走。**首先是图片提取**，在解析 PDF 的时候，用 PyMuPDF 逐页把图片提取出来存到本地，同时在文本中插入一个 `[IMAGE: image_id]` 的占位符，并且在 metadata 里记录这张图的 ID、路径、页码等信息。

然后是生成描述，在 Pipeline 的 Transform 阶段，有一个 ImageCaptioner 组件会扫描 chunk 里的图片占位符，对每张图调用 Vision LLM（比如 GPT-4o）生成一段文字描述。生成的描述会追加到占位符后面嵌入 chunk 文本中，这样后续向量化的时候图片的语义就自然包含在内了。

生成的 Caption 会跟 chunk 文本一起做向量化，Dense + Sparse 双路编码，完全复用现有的文本检索链路，不需要额外的多模态向量库。

最后在检索返回的时候，有一个 MultimodalAssembler 组件，从命中的 chunk metadata 里提取图片引用，把图片读出来做 Base64 编码，按 MCP 协议组装成文本+图片+描述的混合内容块返回给客户端。

追问：为什么不用 CLIP / 多模态向量?

CLIP 更适合"以图搜图"的场景，但我们的需求是"用自然语言搜文档里的图"。用 LLM 生成 Caption 有个很大的优势：它能理解图表里的**逻辑关系和具体数据**，比如流程图里的步骤顺序、表格里的数值对比，CLIP 只编码视觉特征，对这类语义理解比较弱。而且 Image-to-Text 方案不需要引入额外的向量库，架构更统一，运维也更简单。

3.3.4.3 怎么做端到端测试？方便做吗?

（这个结合ragas来答吧，其实rag有很多评估方法，但是ragas最简单，不用准备数据集，讲别的还得讲如何准备数据集。一般来对于大部分场景，答ragas的评估也够用了。）

我们的端到端测试主要是做**检索质量的回归测试**。做法很简单：准备一批典型的 query list，然后直接跑系统，每条 query 走完整的 RAG 链路——检索、重排、生成，跑完之后收集日志，汇总指标做对比。

具体来说，评估质量用的是 **Ragas 框架**。Ragas 的核心思想是 **LLM-as-Judge**，就是用一个 LLM 来自动评判 RAG 系统输出的质量，不需要人工标注。它从三个维度打分：

- **Faithfulness（忠实度）**：回答有没有编造。做法是把回答拆成多个陈述句，逐条检查每句话能不能在检索到的上下文里找到依据，找不到的就是幻觉。得分 = 有依据的陈述数 / 总陈述数。这个指标卡住的是**生成环节**有没有乱编。
- **Answer Relevancy（回答相关性）**：回答跟问题相不相关。做法是从回答反向生成几个"可能的问题"，算这些生成问题和原始问题的语义相似度，相似度越高说明回答越切题。这个指标卡住的是**最终回答**有没有答非所问。
- **Context Precision（上下文精度）**：检索到的内容有没有用。检查返回的每个 chunk 是否跟问题相关，排在前面的越相关分越高。这个指标卡住的是**检索环节**有没有把无关内容混进来。

除了 Ragas 的三个质量指标，我们还会记录**性能指标**：端到端延迟（从发出 query 到拿到完整回答的总耗时），以及各阶段的耗时（检索耗时、重排耗时、生成耗时），因为系统本身有全链路 Trace，每个阶段的输入输出和耗时都会记录在日志里。

每次改了策略（比如换了 chunking 方式、调了 embedding 模型、改了 rerank 参数），就重新跑一轮同样的 query list，对比前后的指标变化，看质量有没有掉、延迟有没有涨。设好阈值做回归门控，比如 Faithfulness ≥ 0.7、P95 延迟 ≤ 5s，低于阈值就报警。

3.3.5 百度二面（来自粉丝B）

> 谢谢这个朋友再次给我投稿，笔记内容的3.5和3.6部分就是这个朋友给我投的稿。3.4\~3.6都来自于这位朋友，而且这四个面试素材都是大厂）
>
> 这个粉丝朋友给我的投稿内容，看得出来都是一些难一点的题目了。常规的项目问题，她反馈通过我们的面试skill，以及总结的真题，都已经有了。我看了一下这些问题，都是比较偏向于需要一些工程经验，项目经验的，不是那种纯项目八股。这些问题需要我们有一点经验积累，应届生同学看着可能会觉得难一点，不过慢慢来，应届同学可以先看懂问题和答案，慢慢思考，循序渐进。

3.3.5.1 考虑性能优化了吗？项目中哪些地方可以优化？

> 这个问题也没有标准的答案。我总结了项目中一些对于性能优化的地方，比如整个操作做异步处理比如计算embedding块的时候，是批处理的。比如跳过重复计算hash，懒加载这样可以让客户端连接mcp server会快一些，另外就是所有的环节都有耗时记录，这无非也是我们对性能的考虑.
>
> 优化这一块。第一点整个查询结果换成，变成LRU+TTL，这是一个很好的思路，也是我借鉴了一个成熟的项目得到的思路，然后就是对于hash的处理，之前是文件级别的hash做处理，当文件只有一小部分改变了，但是整个文件hash变了，这些块还要重复计算，这个也是一个值得优化的点（这个是一个粉丝朋友在群里的提问，说明他真的有很认真地在思考和学习这个项目，他现在已经上岸了，去暑期实习了）

参考回答：

有的，我在项目里从几个维度都做了性能考虑。

第一，异步化阻塞调用。 MCP Server 的工具处理层全部是异步实现的，但底层的向量检索这些库本身是同步的。我把这些同步操作卸载到线程池去执行，这样事件循环在等待期间可以继续处理其他请求，不会被阻塞住。

第二，批处理降低 API 调用次数。 Ingestion 流水线里我设计了两层架构专门处理批量 Embedding。先把所有分块的文本收集起来，再按可配置的批大小分批调用 Embedding API，把 N 次网络往返降为几次。批大小做成配置化而非硬编码，是因为不同 Provider 的限频策略差异很大，配置化可以不改代码直接调整。同时每批次都记录了耗时指标，通过 Trace 上报，方便定位瓶颈。

第三，增量处理跳过重复计算。 每次入库前先对文件做哈希校验，已经处理过的文件直接跳过整条流水线。这对重复上传或增量更新的场景非常关键，尤其是流水线里有 LLM 增强这种耗时步骤时，跳过的收益非常大。

第四，懒加载控制资源占用。 Embedding 和 VectorStore 都通过工厂模式按需初始化，不在进程启动时把所有模型全加载进来，启动速度快，内存占用也更可控。

第五，Trace 监控定位瓶颈。 流水线每个阶段都接入了 Trace 记录耗时，发现性能问题时可以直接看数据定位是哪个阶段慢，为后续优化提供数据依据。

我还想到两个目前没做但值得做的优化方向。

第一，查询结果缓存。 现在每次查询都完整走一遍 embedding → 向量检索 → rerank，重复查询每次都在重复计算。可以在查询层加一个 LRU + TTL 缓存，以查询文本、集合名、返回数量作为 key，命中缓存直接返回，整条链路全部跳过。

第二，把增量跳过的粒度从文件级下沉到分块级。 现在只要文件哈希变了，整个文档就全部重新分块、重新算 Embedding。但如果一个大文档只改了一小段，绝大部分分块内容其实没有变化，全部重算很浪费。改进思路是用内容哈希作为分块 ID，文件变了之后照常重新分块，然后批量查哪些分块已经存在，只对新增或修改的分块调 Embedding API，同时清理掉旧版本多余的分块。这样 Embedding 调用量就从跟总量成正比，变成只跟变更量成正比。

3.3.5.2 对系统吞吐量做过测试吗？QPS / TPS 是多少？

> 这个问题就是问你有没有考虑过一秒钟整个系统能处理多少次查询？有没有考虑过。因为我们目前项目还是单机，不涉及多个客户端查询，很难出现并发查询的场景，我这里更多是从串行角度上算性能，分析QPS，计算瓶颈。其实在后面那个问题如果是做成生产项目的时候，要做什么处理提到了，可以变成远端server，通过http让多个客户端请求，这时QPS的计算就要考虑并发了。
>
> 这里我说上面的内容只是为了让大家清楚有这么一回事，你自己回答的时候，没必要提出什么并发，什么部署到server，这样不是主动暴露自己做得不够深入吗？我们不主动揭自己的短。

参考回答：

测过，我做了两轮测试，一轮测本地管线吞吐，一轮测端到端吞吐，目的是通过控制变量定位瓶颈在哪。

第一轮，本地管线吞吐。 我用 Mock Embedding 替代真实 API，隔离掉网络变量，往数据库注入合成数据，然后反复跑查询，走完整的向量检索 + BM25 + RRF 融合。测出来本地管线的 **QPS 大概 58**，单次查询十几毫秒就完成了。

第二轮，端到端吞吐。 接上真实的 OpenAI Embedding API，走完整条查询链路。这时候 **QPS 掉到了 0.56**，单次查询将近 2 秒。开 3 路并发后能到 **1.31**，加速比 2.3 倍。

两轮一对比，结论就很清楚了：本地管线 QPS 58，加上 API 后变成 0.56，差了 100 倍，99% 的时间花在远程 Embedding API 的网络往返上。 所以优化方向如果要优化也很明确：换本地 embedding 模型去掉网络 RTT，或者在 query 增加缓存让重复查询直接命中。

3.3.5.3 能支持用户并发吗？能并发上传文档吗？能并发提问吗？如何实现并发？（设计方案）

> 当前项目其实没有做并发上传，但是其实可以做，也不复杂，大家问问AI他都能做。可以如是说，也可以说自己做了，想好是怎么做的就行。这些东西其实就和LLM没太大关系，并发啊，锁啊，这些并发编程的内容，也就是一些开发工程的问题，面试还是偶尔会问。不过像并发编程，线程池，任务队列，锁其实也是属于比较难一点的内容，慢慢理解吧。通过一次次面试，和面试官的讨论去逐渐掌握。

参考回答：

能并发提问吗？

能。MCP Server 基于协程架构，多个查询请求在同一个事件循环里并发调度，互不阻塞。底层的向量检索、BM25 检索等同步操作被卸载到线程池执行，不占用事件循环。查询内部的 Dense 和 Sparse 两路检索也是并行执行的，而非串行等待。

能并发上传文档吗？

单文档内部是并行的——每个 chunk 的 LLM 处理（精炼、元数据增强、图片描述）通过线程池并发执行。但多文档之间目前是串行的，一个处理完再处理下一个。当时做这个决策主要考虑两点：一是 **BM25 倒排索引的并发安全问题**——它是基于 JSON 文件存储的，每次添加文档需要"加载全量索引→合并新数据→全量重建→写回磁盘"，这个过程不是原子的。如果两个文档并行处理，线程 A 加载旧索引并修改保存的同时，线程 B 也加载了同一份旧索引并修改保存，B 就会覆盖 A 的结果，造成索引数据丢失。要解决这个问题要么加锁（降低并行收益），要么用数据库替代 JSON 文件（改造成本较高）

如何实现并发上传？

在现有的协程架构上加一层异步任务队列即可。每个文件的上传请求作为一个任务入队，多个协程/线程消费者并发执行 Ingestion 流水线。关键要解决 BM25 索引的并发安全问题，有两个思路：一是在 BM25 写入操作上加互斥锁，保证同一时刻只有一个线程在做索引重建；二是改造 BM25 为增量更新模式，用数据库（如 SQLite FTS5）替代 JSON 文件，从根本上消除非原子操作。

3.3.5.4 stdio / stdout 安全性

> 注意这里要理解一下mcp 使用stdio/stdout通信这个协议的一些要求。这个其实在笔记：其他问题部分：2.2 MCP连接偶尔失败部分 我以这个问题为例子，写了个答案。 这个问题的背景是，使用MCP协议，这个stdio模式的stdout就是留给了mcp客户端和服务端去通信，我们不该往里面写数据，要不会导致连接问题。这个知识是这个回答的前置知识。
>
> 有了这个知识要考虑的就是数据安全，作为一个mcp server，我们不应该暴露key，暴露用户数据，要对数据脱敏。

- MCP Server 使用 stdio/stdout 作为传输层，是否安全？
- 会不会有敏感字段泄露的风险？

参考回答：

是否安全?

安全的。stdio 模式下，Server 作为子进程被 Host（比如 Claude Desktop）拉起，stdout 是两个本地进程之间的匿名管道（anonymous pipe），不经过网络，没有端口暴露，外部进程无法直接访问这条管道，安全边界等于"本机当前用户的进程权限"。第三方进程想往这条 pipe 注入数据几乎不可能；所以 stdio/stdout 本身不存在太大的安全风险。

会不会有敏感字段泄露?

需要在两条输出流上分别控制。设计 MCP Server 时，stdout 和 stderr 的分工是主动划定的：stdout 是协议专用通道，stderr 是日志专用通道。

stdout 方面：协议消息的格式由 MCP SDK 负责，我们不碰，但消息里携带的内容是 tool handler 的返回值决定的。所以要控制 handler 不把异常堆栈、配置对象、文件路径这些内部信息返回出去，遇到错误统一返回结构化的错误描述而不是原始异常。另外 Python 默认的 logging 和 print 都走 stdout，需要在 Server 启动最早期就把所有日志重定向到 stderr，防止日志内容污染协议通道。

stderr 方面：日志走 stderr，但 stderr 会被父进程继承，可能被 Host 记录到日志文件里。所以日志中对 API Key 等凭据要做脱敏处理，只打印前几位做确认，不记录完整值。

3.3.5.5 如果该项目要真正落地投入生产，还需要做哪些工作?

> 这个也没有什么标准答案，答案是基于一些项目经验回答的。真正生产项目比如对错误处理，对于安全合规，以及部署肯定都是有更高要求的。比如说我们写项目这个key是存在本地，但是通常生产级别项目的key是存在专门的server端，比如通过云厂商托管服务，AWS Seceret manager + STS通过现成的技术框架来实现鉴权，key是不会放在自己工程中的。

参考回答:

主要从三个维度：**容错加固**（让 LLM 调用不会因为单次失败就崩）、**安全合规**（密钥不落地、日志脱敏、操作留痕）、**部署运维**（从本地 stdio 模式切到远端 HTTP 服务，Docker + K8s + CI/CD 自动化）。

第一，LLM 调用层的容错加固。 目前项目的 LLM 调用是"单次尝试"——API 超时或返回 429 就直接失败了。生产环境必须加 **指数退避重试**：429 限频等 30 秒后重试，5xx 服务端错误指数退避，不可恢复的错误（如 401 鉴权失败）立即终止，不浪费重试次数。重试上限我认为 10 次比较合理。

另外需要加 **速率限制**，控制每分钟请求数和并发队列深度，避免突发流量打爆 Provider 配额。

第二，安全与合规。 目前 stdout/stderr 分离和错误信息脱敏做了一些，但生产环境还需要：

- **认证体系**：目前 API Key 明文写在配置文件里，生产环境不能这么做——配置文件可能泄露、误提交 Git、运维人员都能看到。解决方案是接入统一的密钥管理服务，服务自己不保管任何密钥，每次需要时向密钥管理服务申请一个临时凭证，用完自动过期。这样代码和配置里彻底不出现密钥，即使临时凭证泄露影响也极小，密钥轮换只需要在管理服务侧操作，不用改代码重新部署。

- **合规日志审计**：给数据分级，区分"公开"和"敏感"两类。公开数据正常记录（如"用户查询了 default 集合"），敏感数据（用户问题内容、文档内容、人名金额等）在日志中自动脱敏打码。同时所有操作（谁在什么时间上传了什么文档、查询了什么）必须完整留痕，存到统一的审计系统。

- **输入校验**：对用户上传的文档做大小限制、格式白名单，防止恶意文件触发异常。

第三，部署与运维。 目前项目用的是 stdio 本地模式，Host 拉起子进程，只服务单个用户。如果要投入生产服务多用户，首先需要把传输层从 stdio 切换到 HTTP/SSE，让 MCP Server 变成一个可以通过网络访问的远端服务。一旦变成网络服务，就需要用 Docker 把服务和所有依赖打包成镜像，保证环境一致性，镜像不可变，回滚只需切回上一个版本。然后用 K8s 做编排，负责自动调度、水平扩缩容、实例故障自动重启、滚动更新零停机。最后搭建 CI/CD 流水线，代码推送后自动跑测试、构建镜像、部署到 K8s，全程自动化。

3.3.6 京东一面（来自粉丝B）

> 我估计是面试官对于这个vibe coding技术本身比较好奇，两个问题都是和vibe coding相关的。另外这个粉丝朋友其实是只提供了他不会或者在之前笔记，面试突击Agent中没出现过的问题。

3.3.6.1 统计过项目的代码量吗？ 用 AI (Vibe Coding) 辅助开发消耗了多少 token?

> 我用AI帮大家统计了一下我们的项目代码。也是个5w行代码级别的项目了。要是搁着原来，这项目不知道得写多少天，现在竟然是我用2个月下班时间让AI写完的。

代码量已做过统计（排除 .venv、**pycache** 等非业务文件）：

- Python 源码：198 个文件，47,433 行
- Markdown 文档：51 个文件，13,014 行
- YAML 配置：2 个文件，196 行
- TXT 提示词：5 个文件，78 行
- TOML 项目配置：1 个文件，72 行

- **合计：257 个文件，约 6 万行**

纯业务代码（Python）**198 个文件，约 4.7 万行**，含文档总量约 **6 万行**

- 平均每文件约 240 行，架构分层清晰（`core` / `ingestion` / `libs` / `mcp_server` / `observability`）

Token 消耗方面：整个项目使用 GitHub Copilot（Claude Sonnet 模型）进行 Vibe Coding 辅助开发，Copilot 订阅制不直接暴露 token 用量，无法精确统计；但从开发规模来估算，整个项目按照设计文档分为 9 个大阶段、60 个子任务，每个子任务都是一轮完整的"需求对话 → 代码生成 → 测试修复"循环，单次大型功能迭代约消耗数万 token，60 个任务累计预计在 **数百万 token** 量级。

3.3.6.2 如何评估VibeCoding

> 这个问题其实还挺好回答的。spec, 测试，一小步一小步执行都是我们自己的实践。然后加了一个harness工程的思想，harness工程我们笔记里总结的也有，就是如何让AI更好的写代码的思路，加了一些思想进去。之前我做这个项目的时候用的技巧是1，2，4. 现在如果再让我做，我会加入harness工程的思想，随着学习，技巧还是在不断进步的。

总结一下思想：Spec 管设计，Harness 管边界，人工 Review 管意图，测试管底线。

第一，Spec 先行，AI 只做执行。

写代码之前，我在 DEV_SPEC 里把架构全部定好了——9 个阶段、68 个任务，每个任务的接口定义、目录结构、模块交互方式全部写死。AI 的角色是"按图施工"，没有设计决策权。这从源头杜绝了 AI 自由发挥导致的架构混乱。

第二，Harness 思想约束 AI 行为。

三个关键做法：Agent 用完即销毁——每个任务一个独立会话，完成就销毁，防止上下文污染和错误传播。权限边界明确——AI 只能操作 Spec 指定的文件，不允许自行创建模块或修改架构。评判由程序驱动——测试是否通过、类型检查是否干净，全部由 pytest、mypy 等外部工具判定，不让 AI 自查自纠。AI 有幻觉，让它自己评估自己等于考生自己批改试卷。

第三，68 个任务逐一人工审核。

不是让 AI 一口气生成整个项目，而是每完成一个任务就做一次人工 Code Review——检查是否符合 Spec、有无偏离设计意图、边界条件是否覆盖。粒度越细，问题越早暴露，修复成本越低。

第四， 54个测试文件作为硬性底线。

54 个单元测试、13 个集成测试、4 个 E2E 测试，覆盖从单组件到全链路。每个任务完成后必须跑通相关测试才能提交，跑不过自动修复最多三轮，三轮不过交人工。测试是交付的前提条件，不是可选项。

3.3.6.3 设计模式八股（工厂，策略）

> 设计模式通常其实也是需要一些工作经验才能慢慢理解的，面试其实问的没那么多，偶尔会出现，可以先不学那么深入，毕竟先学性价比高的。但是你做了我们项目，用了大量工厂模式，建议最少还是学一下工厂模式。我来给大家总结个工厂模式八股。

参考答案：

工厂模式

工厂模式的核心思想是把"创建对象"这件事从调用方剥离出来，交给一个专门的工厂来做。调用方只需要告诉工厂"我要哪种类型"，工厂负责返回对应的实例，调用方完全不需要知道具体的实现类是什么、构造函数需要什么参数。

它解决的核心问题是解耦。如果调用方直接 new 具体类，那每次换一个实现就得改调用方代码，改一处还好，如果十个地方都在用就要改十处。用了工厂之后，调用方只依赖抽象接口，具体用哪个实现由工厂根据配

置决定，切换实现只需要改配置，不动业务代码。

在我的项目里，LLM、Embedding、Splitter、VectorStore、Reranker 这些组件全部用工厂模式创建。比如 Embedding，配置文件里写 openai 就用 OpenAI 的实现，改成 ollama 就自动切到本地模型，调用方一行代码都不用改。要加一个新的 Provider，只需要写一个新的实现类，在工厂里注册一下就行，已有的代码完全不用动——这就是开闭原则，对扩展开放、对修改关闭。

3.3.7 快手一面

3.3.7.1 你的 Index 是怎么做的?

- 粉丝回答：存成了 JSON 格式

- 面试官追问：**用 JSON 怎么读取?** （没答上来）

- 面试官评价：**不存数据库或者不建立索引应该是不对的。现在这样用 JSON，读取和更新都很麻烦，会 OOM 或者时间太长。**

> 粉丝投稿的上下文有限。这个Index索引我不知道指的是稠密向量还是稀疏向量的索引。稠密向量的索引我们用的ChromaDB的HNSW，稀疏向量用的自建的BM25倒排索引，存成JSON。我猜测他们是在聊后者，但是估计前面一个问题是先问我们同存了哪些数据的。所以这里先给个前置问题的解析。 另外针对于这个问题，我们本身存成JSON格式，也是用了索引的，可以和面试官解释。具体讲解我放在下面的答案里。

（前置问题）你的 RAG 系统都存了哪些数据？怎么存的?

> 其实作为RAG系统最核心的问数据存了啥，其实就是下面的2和3，稀疏向量和稠密向量怎么存的，或者可以带上4，毕竟我们是支持图片检索的系统。1和5算是其他方面的功能，便于理解我都总结出来。

我们项目共有 5 块存储：

1. **文件指纹库**（SQLite）— 存文件 SHA256 和入库状态，负责幂等控制，防止同一文件重复入库。
2. **Dense 向量库**（ChromaDB，底层 SQLite + hnswlib）— 存原文 + 向量 + HNSW 图索引，负责语义相似度检索。
3. **BM25 倒排索引**（JSON 文件）— 存倒排表（term、tf、idf、doc_length），负责关键词匹配检索。
4. **图片资源库**（文件系统 + SQLite）— 存图片二进制 + 元数据索引，负责多模态图片检索。
5. **Trace 日志**（JSONL）— 存全链路 trace 事件，用于可观测性和 Dashboard。

各存储的路径与补充说明：

- **1 文件指纹库**：路径 `data/db/ingestion_history.db`，同一文件不重复入库

- **2 Dense 向量库**：路径 `data/db/chroma/`，ChromaDB 内部 SQLite 存 chunk 原文 + metadata + id 映射；hnswlib 二进制段（`*.bin`，float32 数组）存 1536 维稠密向量 + HNSW 图索引

- **3 BM25 倒排索引**：路径 `data/db/bm25/{collection}/*.json`，倒排表结构（term → posting list，含 tf、doc_length、idf），不存原文

- **4 图片资源库**：路径 `data/images/{collection}/` + `image_index.db`

- **5 Trace 日志**：路径 `logs/traces.jsonl`，记录 ingestion + query 全链路事件

面试照答版本：

我们项目有 5 块存储：

第一块是文件指纹库，用 SQLite 记录每个文件的 SHA256，做幂等校验，同一文件不重复入库。

第二块是 Dense 向量库，用 ChromaDB。Chroma 内部分两部分存：SQLite 存 chunk 原文、metadata 和 id 映射；hnswlib 的二进制段（float32 格式）存 1536 维稠密向量和 HNSW 图索引。查询时先走 HNSW 图 O(log N) 找到 top-k 的 id，再用 id 去 SQLite 拉原文。

第三块是 BM25 倒排索引，用 JSON 文件存储。这个 JSON 装的不是原始数据，是倒排表结构（词 → 文档命中列表，含词频、文档长度、idf）。查询时 O(1) 哈希查词，然后只遍历命中的文档算 BM25 分。命中后原文不存在这里，用 chunk_id 去 Chroma 取，避免重复存储。

第四块是图片资源库，文件系统存图片二进制，SQLite 存图片元数据索引，支持多模态检索。

第五块是 Trace 日志，JSONL 格式记录全链路 trace 事件，用于可观测性和 Dashboard 展示。

其中第二和第三块是核心——查询时两路并行召回，Dense 走语义、Sparse 走关键词，RRF 融合后 Rerank 出最终 top-k

Q1. 你的 Index 是怎么做的?

> 因为我推测这个面试问题是针对于稀疏向量来问的，所以我对于稀疏向量的索引选型原因进行了展开。我不清楚同学面试具体情况，看上去面试官在挑战他说存成JSON，我们没有做索引。但是其实不是的，我们存了JSON，维护了自建的倒排索引。第二个是我猜面试官可能会挑战的，就说你为啥不用专门的库比如Elasticsearch来做，包括向量数据库，为啥不用Milvus这样支持数据量更大的库来做。这两个问题回答思路是一样的。就是归根结底我们项目还是本地的数据库，那么数据量级并没有很大，盲目用更复杂的库要考虑的东西很多，而且开销也会更大。其实你看openclaw，本身也是一个本地的agent，用了双路查询，他甚至是没有用专业向量数据库，只用的sqlite。所以被挑战的时候你要想清楚，说明理由我觉得可以让面试官信服的。当然如果你说你要包装成更复杂的系统，用Milvus,Elasticsearch，那么你要清楚的这种情况你的数据量级应该是千万级别以上的，你的业务场景是啥，另外也会涉及分布式等后端知识。你能不能包装自圆其说讲清楚。
>
> **答题思路**：在前置问题基础上，聚焦到"索引"展开，重点讲两套索引 + 自建的原因。

面试照答版本：

我们的 Index 是两套并行的 Hybrid Index。

稠密向量这边，用的是 ChromaDB 自带的索引。Chroma 底层用 hnswlib 做 HNSW 图索引，查询复杂度是 O(log N)，这部分不需要我们自己实现，直接调 Chroma 的接口就能做近似最近邻搜索。

稀疏向量这边，是我们自建的 BM25 倒排索引。倒排索引的结构是按词组织的——每个词记录了它出现在哪些文档里、出现几次、文档多长。查询时先 O(1) 通过哈希直接定位到这个词的记录，然后只遍历命中的文档算 BM25 分，复杂度是 O(K)，K 是命中此词的文档数，跟库里总文档量无关。所有统计原料在 build 阶段一次性预算好，查询时只做公式代入，非常快。

选择自建而不是用 Elasticsearch，主要考虑了两点：第一是**数据量级**，我们是企业内部知识库场景，文档量在百万 chunk 级别，倒排索引加载到内存也就 1-2 GB，完全扛得住。一般到千万甚至亿级 chunk 的时候才需要上 ES 那种分布式方案。第二是**部署形态**，我们定位是一个本地化部署的系统，不想引入 ES 这样的重依赖——ES 需要 JVM、集群配置、额外运维成本，对本地私有化场景来说太重了。自建的方案一个 JSON 文件就搞定，启动即用，零额外依赖。

两路各自召回 top-k 后，用 RRF 融合再 Rerank 出最终结果。

3.3.8 某大厂一面

3.3.8.1 面试官问：为什么不用开源框架，比如 LlamaIndex?

答题策略：这个问题确实是需要你对LlamaIndex这个库很了解才能回答出来。有两个思路：1. 功能层面，功能层面Llamaindex也是都能满足，因为我们有自己的思想，比如做的高度可插拔，高度可控。即使有些地方使用Llamaindex可以做到，但也许也需要改造，成本也不低。 2. 性能层面。使用Llamaindex本身可能会让系统更加笨重，因为专业的库本身功能全，但是无可避免他也会更臃肿。

> 这两个思路其实也是很多现有的项目的思路。你可以看很多成熟的公司，比如我面试的阿里千问，有一个创业公司，还有一个盛大的开源项目，都是自己手搓的Agent框架。那他们为啥不用Langchain，功能层面也可以实现呀。所以这块不用怕，能够说清楚为自己正名就好

面试回答：

选择自建而不用 LlamaIndex，核心原因是**两者的设计目标不同。**LlamaIndex 是广度优先的通用框架——200 多个集成、覆盖尽可能多的场景；我们这个项目是深度优先，需要在一条 RAG 链路上做到每个环节都可控、可定制。我从功能和性能两个维度来说。

第一是功能层面，有些定制需求用框架实现的成本反而更高。 比如入库流水线，我们有三个 LLM 增强阶段——分块精炼、元数据提取、图片描述，每个阶段都有独立的 prompt 文件可以按业务场景替换，而且做了 graceful fallback，LLM 调用失败自动降级到规则处理不阻塞流水线。LlamaIndex 有 Transformation 和 Extractor 接口，理论上能做，但要适配它的 Node 体系和回调流程，三个阶段的错误降级、并行编排、prompt 热替换这些都得在框架约束下绕着走，改造成本不低。再比如混合检索，我们自建了 BM25 倒排索引，IDF 的 k1、b 参数可调，分词用 jieba 做中文适配加停用词过滤，Dense 和 Sparse 两路并行召回再 RRF 融合。LlamaIndex 也能接 BM25，但它把 BM25 当黑箱组件挂上去，你控制不了 IDF 参数、看不到倒排表、也没法定制分词策略，调参空间很有限。

第二是性能和依赖层面，LlamaIndex 的依赖链太重了。 光 `llama-index-core` 一个包就有 27 个以上的直接依赖，包括 SQLAlchemy、networkx、nltk、tiktoken、pillow、banks、tinytag 这些——SQLAlchemy

是 ORM 我们不需要，networkx 是图计算库我们不需要，tinytag 是音频元数据解析器我们更不需要。这些库各自又有自己的传递依赖，装完之后整个依赖树非常庞大。而我们项目的核心依赖只有 9 个——chromadb、jieba、mcp、pyyaml、langchain-text-splitters 这些，每一个都是实际会用到的，没有冗余。依赖少意味着三件事：第一，启动快、内存占用小，私有化部署场景这很关键；第二，安全面小，每个依赖都是潜在的 CVE 补丁点，依赖越多运维成本越高；第三，版本冲突少，LlamaIndex 的依赖链越长，跟项目里其他库发生版本冲突的概率就越大。

总结来说，LlamaIndex 适合快速出 POC 或者需要对接大量异构数据源的场景。但我们需要在 LLM 增强流水线、混合检索调参这些 RAG 核心环节上做深度定制，同时保持依赖轻量便于私有化部署。自建的总成本反而比引入一个重框架再做二次开发更低，同时对每个环节有完全的理解和控制权。

3.3.9 某大厂一面

3.3.9.1 向量数据库大小，有多少个 PDF?

> 这个问题其实就是因人而异了。我做出这个项目的时候是没有指定对应数据量的，但是思路我说了需要大家自己去包装，也提供了包装项目，写简历的SKILL。 这里需要大家自己思考一下业务场景然后包装。这里我们不妨，假设我们就是一个给公司内部小组使用的知识库（不是整个公司啊，而是一些组），以这个角度去回答。

假设场景：企业内部小组知识库（如技术文档、规范流程、会议纪要等），服务于一个 10-30 人的业务线。

面试回答：

我们的场景是企业内部业务组的知识库，主要存技术文档、规范流程和会议纪要这类材料。目前入库了**大约 200 份 PDF**，平均每份 15-20 页，经过切分后产生了大约 **5 万个 chunk**。

向量库的实际大小可以这样估算：我们用的 `text-embedding-ada-002`，每个 chunk 是 1536 维的 float32 向量，单条向量 1536 × 4 字节 = 6KB。5 万个 chunk 的向量数据大约是 **300MB**。加上 ChromaDB 里 SQLite 存的原文、metadata、id 映射，以及 hnswlib 的 HNSW 图索引开销，整个 `data/db/chroma/` 目录实际大小在 **500MB-800MB** 这个量级。BM25 倒排索引那边更小，5 万 chunk 的倒排表 JSON 大概也就**几十 MB**。

选型上，我们用的 ChromaDB，核心原因是**部署形态决定技术选型**。我们是内网私有化部署的系统，部署在组内的一台内网服务器上，通过 MCP 协议的 SSE 传输层暴露给组内成员，所有人的 IDE 通过内网连接这个 Server。Chroma 在服务端以 Client-Server 模式运行，底层用 hnswlib 做 HNSW 图索引，查询复杂度 O(log N)，支持并发读写。5 万条数据量级下，单次向量检索延迟在**毫秒级**，20 人并发查询完全扛得住。

如果未来要扩容，我也有明确的分级方案：

- **10 万 chunk 以内**：ChromaDB 完全扛得住，单机内存足够，不需要任何变动。

- **10 万到百万级**：可以考虑切到 Qdrant 或者 FAISS，它们在大规模数据下的索引效率和内存管理更优。我们的向量库通过 `VectorStoreFactory` 做了工厂模式抽象，配置文件里改一行 `provider: "qdrant"` 就能切

换，代码零改动。

- **千万级以上**：就得上 Milvus 这种分布式方案了，做分片、副本、支持水平扩展。但那已经是整个公司级别的知识库了，不是我们当前的场景。

总结一下，200 份 PDF、5 万 chunk、向量库 500-800MB，用 ChromaDB 部署在内网服务器上，通过 SSE 暴露给组内 20 人使用。选型的思路就是**数据量级匹配部署形态**——当前量级用轻量方案，架构上预留了扩展能力，量级上来随时可以切。

3.3.9.2 为什么公司你存内部文档就需要 size 1000?

你设置 1000 是不是太大了？1000 个字是不是太多了?

> 这里推荐大家看一下八股里的Chunk size的选型，使用这个思路，讲清楚为什么选择这样的chunk size,相信就不会被面试官挑战了。八股里补充了chunck size的选型策略，从哪几个维度去考虑。面试官问我们为什么这么选chunk size的时候以及chunk size的选型策略时可以参考。
>
> 回答假设我们是企业内部技术小组知识库，文档以英文技术文档为主（API docs、Architecture Decision Records、Runbooks、Postmortem 等）。

面试回答：

chunk_size=1000 不是拍脑袋定的，我从四个维度来解释这个选择的合理性。

首先明确一下，1000 是字符不是 token。 我们用的 RecursiveCharacterTextSplitter，length_function 是 Python 的 len()，按字符计数。我们的文档以英文为主，英文 1000 个字符约 150-200 个单词、200-250 个 token，大概就是一个自然段落的量。

第二，这个值正好落在 embedding 的最佳输入区间。 text-embedding-ada-002 虽然 max token 是 8191，但实践表明 100-800 token 区间信息密度最高。200-250 token 处在区间中段，向量质量最好。其实对英文来说 1000 字符不但不大，反而还偏保守，上调到 1500-2000 字符（400-500 token）也完全在合理范围内。

第三，匹配文档的自然语义粒度。 我们的英文技术文档，一个自然段落通常 100-250 个单词、500-1200 个字符。1000 字符刚好对应一个完整的语义单元——一个 API 说明、一条操作步骤、一段架构决策。不会横跨不相关的段落，也不会把一段话切碎。

第四，LLM 窗口预算非常充裕。 召回 top-10，每个 250 token，总共才 2500 token，LLM 还有大量空间做推理和生成。

另外要补充一点，RecursiveCharacterTextSplitter 不是硬切 1000，它会按段落、换行、句号这些语义边界递归切分，实际大部分 chunk 不到 1000。再加上 200 字符的 overlap、LLM chunk 精炼和 metadata 增强，对粒度的容错能力很强。

不过要注意一点，如果文档是中文的，情况就完全不同了。中文 1000 字符就是 1000 个汉字，约 1500-2000 token，远超 embedding 最佳区间，那就确实太大了，需要降到 500-600 字符。所以这个数字不能一概而论，关键是要搞清楚计量单位和语言场景，不同的场景情况也是不同的。

3.3.9.3 你考虑什么数据存 RAG?

> 我对这个问题好像没什么见解，让AI生成的答案。感觉很少这么问。

面试回答

什么数据适合放 RAG，核心判断标准就是**这份数据适不适合用"检索+生成"的模式来回答问题。**

适合放的有四类。 第一是事实性强、答案明确的知识，比如技术文档、操作规范、FAQ，用户问"是什么"、"怎么做"，答案就在文档里，RAG 检索到直接回答，效果最好。第二是更新频繁的知识，比如版本发布说明、变更日志，如果靠 fine-tuning 烧进模型参数每次更新都要重新训练，成本极高，RAG 只需要重新入库那一份文档，下次检索立刻生效。第三是长尾的私有领域知识，公司内部的 Runbook、Postmortem、架构决策记录，互联网上搜不到，通用大模型没训练过，只有 RAG 能覆盖。第四是需要溯源引用的场景，RAG 天然带 source_ref 可以追溯原文，对合规要求高的场景比模型"凭记忆"回答可靠得多。

不适合放的有三类。 推理性任务，比如"对比三个方案的优劣"，这不是检索能解决的，需要 LLM 自身推理能力。结构化查询，比如"上个月销售额多少"，答案在数据库表里，走 Text2SQL 或 function calling 比 RAG 高效得多。高实时性数据，比如股票价格、系统监控指标，入库就过时了，应该走 function calling 实时调 API。

3.3.9.4 MCP tools 的编写原则——不只是 tools 的描述需要描述好，还有什么关键点?

> **答题策略**：MCP 官方规范定义了 tool 的六个字段（name / title / description / inputSchema / outputSchema / annotations），每个字段背后都对应一条设计原则。回答时从六个维度展开，结合项目实际代码举例，展示对 MCP 协议的深度理解。我把这个问题放到了MCP八股中。

参考八股 MCP Server Tool的设计原则

3.3.9.5 视频讲解

3.7 \~ 3.9 小节对应的视频讲解

https://www.bilibili.com/video/BV1RboMB9EGp/?vd_source=144bec9c3f54e465073138bed788be1b

---

3.3.10 字节跳动暑期实习一面（2026/5）

字节暑期实习一面 ，也有相关的该RAG项目面试问题。这个是一个小姐姐拿我们项目去面试的素材。

3.3.11 PDF 处理问题

虽然我没有写这个是哪个公司的面试真题，但是根据粉丝朋友的反馈，很多公司都会问你如何处理的PDF，包括群里提问，也有粉丝问PDF有页眉怎么处理，最近一次粉丝给我提问的原话是："**博主，现在很多低阶的岗**

位特别喜欢问你对PDF是如何处理的，能否讲一下这个问题"。所以这个问题还是挺有代表性的。我也专门把这个PDF的处理标准流程提炼成了专题知识放在RAG知识部分：输入文件处理。常见的问法如下：

1. **你在处理pdf转成markdown文本的这个步骤，你是怎么处理的，可以分享一下吗?**

💡 答题思路

答题思路：在我们笔记中，专门讲解了如何处理PDF。我们就按照笔记里的标准思路：**文件基础体检 - 逐页判断类型 - 按页面类型解析 - 统一结果 - 结果检查** 的思路，把这个答案回答清楚。

📝 参考答案

我们**没有直接用一种解析器处理所有 PDF**，而是设计了一套**分阶段、按页面类型路由**的处理流程。

首先进行**文件级体检**，通过文件哈希去重，并检查文件是否加密、损坏、页面方向是否异常，同时记录文件名和总页数。接下来**逐页分析**文字数量、图片覆盖率、异常字符比例和版面结构，将页面分为**原生文本页、扫描页、混合页、复杂布局页和异常页。**

对**原生文本页**，我使用 PyMuPDF 提取文字及坐标；**扫描页**先渲染为图片，再使用 PaddleOCR 识别；**混合页**同时提取原生文字和重要图片区域，对图片执行 OCR 或多模态理解后再去重合并。对于多栏、表格和图文混排等**复杂布局页**，我会优先使用 **Unstructured 的 hi_res 策略**识别标题、正文、表格、图片区域及其坐标，再按照坐标恢复阅读顺序；如果页面结构特别复杂，也可以使用 **LlamaParse** 直接解析成带层级的 Markdown。对于乱码页面，先用 **PyMuPDF** 提取；如果存在字体编码或字符映射问题，就切换到 **pdfplumber 或 pypdf** 交叉验证；这些解析器仍无法恢复时，再将异常页面渲染成图片，使用 **PaddleOCR** 做页面级兜底。

不同路线的输出最后会**统一为相同结构**，按照原始页码和页面内阅读顺序组合成 Markdown，并保留文件名、页码和异常标记，保证结果**可以追溯到原文**。完成后还会检查漏页、乱码、页眉页脚残留、OCR 重复、多栏错序及表格关键数字。**只有通过质量检查的内容，才会进入后续分块和向量化流程。**

子问题

这些子问题的答题思路了，这个其实就是我们笔记中PDF处理的步骤三的内容，根据不同页面做相应处理。

我们从回答里面可以看出来，想要把PDF处理好，处理完整是多么复杂的一件事，你看看我们下面的回答，用了多少不同的库，要处理多少类型。

我的个人建议是：**背一背得了。** 通过我们的讲解和回答，最少对于下面的问题，你有一个大体的思路和了解，面试官真问到，大面上能回答上来。通常来说他不会再追问更加具体的细节了，比如里面使用的某个库，以及具体处理细节。

- **1.1 你的pdf中有乱码，你是如何处理的?**

💡 答题思路

先判断乱码来源，再按照“切换解析方式 → OCR 兜底 → 质量检查”的顺序处理，避免一遇到乱码就对整份 PDF 执行 OCR。

📝 参考答案

PDF 乱码通常来自**字体编码、字符映射异常或解析器不兼容**。因此，我会先根据异常字符比例判断是局部乱码还是整页乱码。默认使用 **PyMuPDF** 提取；如果结果异常，就切换到 **pdfplumber 或 pypdf** 重新提取并交叉验证。如果只是少数字符异常，可以结合明确的规则和上下文进行修复，但**不会让大模型随意改写原文**。

如果更换解析器后仍然乱码，我会通过 **PyMuPDF 将异常页面渲染成 200 ~ 300 DPI 图片，再使用 PaddleOCR 重新识别**，而不是对整份文件统一执行 OCR。识别后通过异常字符比例、文本完整度和 OCR 置信度进行检查；**低质量页面重新识别或转人工复核**。同时保留原始页码，方便回到 PDF 核对。

- **1.2 你的pdf中有图片，你是如何处理的?**

💡 **答题思路**

先区分装饰图片、文字截图和需要理解关系的图表，再分别选择忽略、OCR 或多模态理解。

📝 **参考答案**

我不会对 PDF 中的所有图片做相同处理，而是先判断图片是否包含有效信息。**Logo、背景和纯装饰图片通常不进入知识库**；带文字的截图会先提取图片区域，再使用 **OCR** 识别文字，并按照图片在页面中的位置插回正文。

对流程图、架构图和统计图，普通 OCR 只能读出文字，无法还原节点关系、箭头方向或数据含义，因此会使用**多模态模型生成结构化说明**。输出时仍然**保留原图引用、所在页码和生成说明**，便于溯源和人工核对。如果原生文字与图片 OCR 结果重复，还会在合并阶段进行去重。

- **1.3 你的Pdf中有页脚页眉，你是如何处理的?**

💡 **答题思路**

利用跨页重复规律和文本坐标识别页眉页脚，不能只依赖固定字符串，以免误删正文。

📝 **参考答案**

我会保留文本块的坐标信息，并统计连续页面顶部和底部区域的内容。如果某段内容在相近位置**跨页重复出现**，就将它判断为页眉、页脚或水印；连续变化的数字则可以结合固定位置识别为页码。

删除时**不会只使用字符串匹配**，而是综合**位置、跨页重复率和文本变化规律**判断。封面、章节首页等版式不同的页面会单独处理。页眉页脚从正文中移除后，**原始 PDF 页码仍作为元数据保留**，确保后续检索结果可以定位到原文。

- **1.4 你的pdf中有表格，你是如何处理的?**

💡 **答题思路**

先区分原生表格和扫描表格，再选择表格提取或 OCR 结构识别；重点说明结构、关键数字和跨页关系的检查。

📝 **参考答案**

对带有文字层、边界比较清楚的**原生表格**，可以使用 Camelot 或版面解析工具提取单元格结构；对于**扫描表格**，需要先执行 OCR，再通过表格结构识别恢复行列关系。提取结果会根据后续使用场景转换成

Markdown、HTML 或 CSV。

表格处理最重要的不是只把文字读出来，而是**保留表头、行列关系和关键数字**。因此我会重点检查合并单元格、跨页表格、单位、正负号和数值是否发生错位。对于跨页表格，还要根据相同表头和页码关系进行合并。如果自动解析无法可靠恢复结构，就**保留表格截图和页码并转人工复核**，而不是输出可能误导模型的错误表格。

3.3.11.1 讲解视频

https://www.bilibili.com/video/BV1gF8Z6WEGW/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

3.3.12 如何保证RAG系统生成的回答是准确的?

这两个问题也是粉丝朋友投稿的，虽然他没告诉我是哪个公司，但是这两个问题确实是来自于面试真题。我觉得这两个题目也很好，很有价值。**面试官关心你的RAG最后的回答，准确性到底怎么样?** 虽然我们有讲**RAGAS**在事后去评估，但是有没有方法可以在整个RAG系统生成回答的时候，就能够检测质量，及时兜底呢？要讲清楚这个问题，我的思路是从Self-RAG这个技术出发，可以先参考RAG专题笔记：Self-RAG.

💡 答题思路

我们这里借助Self-RAG 的思路，将4个检测融入到整个RAG系统中，在系统运行的时候做动态监测，实时重新调整或者拒答。同时因为传统的Self-RAG是需要训练专门评测的模型的，所以我们这里借助Self-RAG的思路，RAGAS的评估方法，来组织这个参考答案。

📝 参考答案

我们不能保证 RAG 的每次回答都绝对正确，但可以通过一套**检索—生成—评估—修正—拒答**的闭环，尽量降低错误回答直接输出的概率。整体流程借鉴了 **Self-RAG** 的自反思思路，但实际检测使用的是 **RAGAS 的评估方法**，不需要专门训练 Self-RAG 模型。

首先通过业务规则或 LLM 分类器判断当前问题是否需要查询知识库，避免对闲聊或通用问题进行无意义检索。需要检索时，使用 RAGAS 的 **Context Precision 和 Context Relevance** 评估召回内容是否与用户问题相关，并结合 Reranker 分数和相关性阈值过滤无关 Chunk。如果上下文相关性不足，就改写查询、扩大召回范围或重新检索，而不是直接基于低质量证据生成答案。

生成答案后，使用 **Faithfulness** 检查答案中的结论能否从检索上下文中得到支持，重点识别无依据补充、事实冲突和模型幻觉；再使用 **Answer Relevancy** 判断答案是否真正回应了用户的问题。如果测试集提供了标准答案，还可以使用 **Answer Correctness** 进一步评估答案与标准答案在事实和语义上的一致程度。

最后综合上下文相关性、答案忠实度和答案相关性进行评分，只有达到质量阈值的答案才会输出，并附带引用来源，方便回到原文核对。如果 **Context Precision 或 Context Relevance** 较低，就重新检索；如果 **Faithfulness** 较低，就要求模型严格依据现有证据重新生成，证据本身不足时也要重新检索；如果 **Answer**

Relevancy 较低，就调整提示词、答案内容或表达结构后重新生成。系统还会设置最大修正次数，如果多轮处理后仍然达不到阈值，就明确提示资料不足并拒答或转人工，而不是让模型猜测。通过 RAGAS 的动态评估和失败兜底机制，可以提高答案的准确性、可追溯性和稳定性。

3.3.12.1 召回的 Chunk 中有很多噪声，如何保证结果稳定?

💡 答题思路

先区分 Chunk 内部的**内容噪声**和**检索结果中的噪声 Chunk。**

内容噪声可以从我们对PDF处理的方法，将PDF转成Markdown，就会对内容噪声去除。

检索噪声可以从我们项目本身做的**RRF融合，双路检索，Reranker触发**，这些是保证我们系统检索准确率的手段。

最后再引入Self-RAG的思想，我们有动态兜底，实时检测，即使最终回答很差，也能及时检测，调整，或者拒答。

📝 参考答案

我会先区分两类噪声。第一类是 **Chunk 内部的内容噪声**，例如 OCR 乱码、页眉页脚残留、重复内容、表格错位和多栏阅读顺序错误。这类问题主要在 **PDF 解析、文本清洗和分块阶段**处理，通过分类解析、去重、异常检测和结果检查，尽量避免脏数据进入向量库。

第二类是**检索噪声**，即 Chunk 本身没有问题，但与当前问题无关或相关性较弱。检索阶段先通过**关键词与向量混合检索**提高有效证据的召回率，再通过 **RRF** 融合多路结果、提高优质证据的排序；随后使用**元数据过滤、相关性阈值和 Reranker** 去掉低相关内容，并根据问题复杂度和相关性分数控制 Top-K。**Top-K 本身不是去噪手段**，关键是避免将过多低相关 Chunk 放入上下文。

最后在生成阶段要求模型**严格依据上下文回答，并标注引用来源**；生成后再检查答案是否获得证据支持。如果上下文相关性低，就重新检索；如果证据充分但回答不忠实，就约束模型重新生成；如果多次处理后证据仍然不足，就拒答。通过**入库前清洗、检索后过滤和生成后校验**三层机制，共同保证结果稳定。

3.3.12.2 讲解视频

https://www.bilibili.com/video/BV1gF8Z6WEGW/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

3.4 视频讲解

3.4.1 项目Skills使用：

如何让AI 使用Skill 遵循dev spec 完成代码。注意视频里的SKILL其实已经重新优化，现在叫做autocoder, 功能是一样的，大家使用代码的时候看一下描述就知道了，使用方法是一样的。

https://www.bilibili.com/video/BV1ixAVz9EaQ/?vd_source=144bec9c3f54e465073138bed788be1b

3.4.2 RAG-Agent项目设计思路和学习方法

https://www.bilibili.com/video/BV1S8wvziEWA/?vd_source=144bec9c3f54e465073138bed788be1b

3.4.3 项目数据处理流水线讲解

我用夸克网盘分享了「项目数据处理流水线讲解.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/cd0bdeab1744

3.4.4 集成到Cursor效果

我用夸克网盘分享了「集成到Cursor效果.mov」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/7e3883ad392d

3.4.5 项目初步demo展示

我用夸克网盘分享了「项目初步demo展示.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/46b58cbc29a9

3.4.6 Resume-Writer SKILL

https://www.bilibili.com/video/BV1aBAfzGEct/?vd_source=144bec9c3f54e465073138bed788be1b

3.4.7 Test SKILL

展示了使用AI 来完成自动化测试的功能。**其中讲了如何写SKILL, 如何写dev_spec文档的**

我用夸克网盘给你分享了「AI测试-SKILL-文档编写.mp4」，点击链接或复制整段内容，打开「夸克APP」即可获取。

/\~7be03LqXsd\~:/

链接：https://pan.quark.cn/s/8c178085e27f

3.4.8 Setup SKILL

https://www.bilibili.com/video/BV1CHPqzqE4R/?vd_source=144bec9c3f54e465073138bed788be1b

3.4.9 模拟面试SKILL

https://www.bilibili.com/video/BV1R7PszQEAc/?vd_source=144bec9c3f54e465073138bed788be1b

3.4.10 附录

视频中4.7 讲到的一些prompt模板：

1.使用plan的方式，让AI先设计计划，然后走通流程，最后根据经验来写出SKILL：

目标：我不是让你直接跑 UI 测试；我是要教你“如何根据 QA_TEST_PLAN 运行并验证 UI 测试（含修复/回归）”，最终请产出一个可复用的 skill.md。

> ❗ 注意这个SKILL我们后来优化了，变成了auto-coder，功能一样的，是更简洁规范的版本。输入自动开发，一键开发等关键字触发。

上下文（你需要用到）：

- 输入：QA_TEST_PLAN（描述要测的功能与步骤）
- 输出：QA_TEST_PROGRESS.md（记录你做了什么、结果是什么、下一步是什么） + 最终 skill.md

请先在 Plan Mode 给出一个计划（plan），包含：

- 你需要我补充哪些信息（最少必需项）
- 你要如何从 repo/示例中学习正确的测试写法与运行方式
- 你会如何把 AQ_TEST_PLAN 转成可执行步骤与验收标准
- 你会如何记录到 AQ_TEST_PROGRESS.md
- skill.md 的结构大纲（包含：目的、触发条件、步骤、失败处理、Anti-patterns、示例调用）

然后我们按计划执行：先把这个流程“手动跑通一次”，中途有问题请及时问我。当我们一起跑通流程后，我确认以后再生成对应的skill

2.生成SPEC 初稿的prompt:

我想写一份完整的开发规范文档（DEV_SPEC），用于指导一个 Python 项目的全部开发流程。请帮我生成一个大致的框架和初步内容。

项目是什么

我要做一个**模块化的 RAG (Retrieval-Augmented Generation) 系统**，同时把它包装成一个

MCP Server（Model Context Protocol），这样 GitHub Copilot、Claude Desktop 等 AI

助手可以直接调用我的知识库进行问答。

通信方式使用 **Stdio Transport**（本地子进程模式），不做 HTTP 部署。

项目定位与特色

这个项目不仅是一个功能完备的系统，更重要的是它是一个**面向学习和面试求职**的实战项目：

- "教是最好的学"——我会边做边录视频教学，所以架构设计要清晰、易于讲解
- 配套有技术文档、代码示范和视频讲解

- 每个模块会整理面试高频题和简历撰写建议
- 开箱即用但同时支持深度扩展，可以作为简历项目

我已有的技术方向（需要体现在文档中）

1. **RAG 策略**:

- 智能分块（语义感知，不是简单定长切分）+ 上下文增强（元数据、图片描述注入 chunk）
- 混合检索：BM25（稀疏）+ Dense Embedding（稠密），用 RRF 融合
- 精排重排：支持 Cross-Encoder 和 LLM Rerank，两段式（粗排→精排） 2. **全链路可插拔架构**：LangChain RecursiveCharacterTextSplitter（切分）
- LLM、Embedding、Reranker、VectorStore、Splitter、Evaluator 都要能通过配置切换
- 工厂模式 + 抽象基类 + 配置驱动（settings.yaml）
- 首批实现：Azure OpenAI、OpenAI、Ollama、DeepSeek（LLM）；Chroma（向量库）； 3. **多模态**：采用 Image-to-Text 策略（Vision LLM 生成图片描述，缝进 chunk 文本），

不用 CLIP 多模态向量

1. **可观测性**:

- 全链路 Trace（Ingestion 链路 + Query 链路），用结构化 JSON Lines 日志
- Streamlit 本地 Dashboard，不依赖 LangSmith 等外部平台
- Dashboard 要有多个页面：系统总览、数据浏览、Ingestion 管理、追踪查看、评估面板等 2. **评估体系**：可插拔评估框架，支持 Ragas 和自定义指标（hit_rate、MRR 等） 3. **MCP 集成**：用 Python 官方 MCP SDK，暴露 tools（query_knowledge_hub、

list_collections、get_document_summary 等）

1. **数据摄取**：PDF → Markdown（用 MarkItDown）→ 分块 → LLM 增强（重写、元数据注入、

图片描述）→ 双路 Embedding → Chroma Upsert，支持 SHA256 增量跳过

文档结构要求

请按以下结构来组织这份 DEV_SPEC：

1. **项目概述**：设计理念、项目定位
2. **核心特点**：每个亮点的简要说明
3. **技术选型**:

- RAG 核心流水线设计（Ingestion Pipeline、Retrieval Pipeline，要详细）
- MCP 服务设计
- 可插拔架构设计（接口定义、配置管理）
- 可观测性与 Dashboard 设计

- 多模态图片处理设计 4. **测试方案**：TDD 理念，分层测试（单元/集成/E2E），RAG 质量评估 5. **系统架构与模块设计**：
- 整体架构图（ASCII art）
- 完整的目录结构树
- 模块职责说明表
- 数据流说明（Ingestion Flow、Query Flow）
- 配置驱动设计示例 6. **项目排期**：
- 按阶段划分（A→I），每阶段有明确目的
- 每个子任务要有：修改文件列表、实现的类/函数、验收标准、测试方法
- 约 1 小时一个可验收增量
- 包含进度跟踪表

其他要求

- 语言：中文
- 技术栈：Python，不使用 LlamaIndex/LangChain 框架（仅使用 LangChain 的 splitter）
- 向量库先只实现 Chroma，架构上预留扩展
- 本地优先，轻量级，零外部服务依赖（SQLite 做持久化）
- 测试框架用 pytest

4 ForgePilot Agent项目

这个项目的定位是一个企业级的Agent项目，上线了，有真实的用户数据，真实运用在了企业工作环境中。

这个是一个虚构的项目，所以不会有代码，但是我会把项目的设计文档，技术文档，思路，真题总结得非常详细。

目前我现在这里占位一下，目前该项目内容可以参考：

该简历如何编写：简历编写原则

项目面试前突击：ForgePilot架构设计

如何包装成企业级项目（上线，部署相关）：包装企业级项目

拓竹科技一面后复盘： 可以参考面试官如何对这个项目提问，包括本次面试后复盘的讲解和笔记中，深入挖掘了ForegePilot 技术深度，可以参考这里。

Pi-agent的架构设计：这里讲了如何针对于Pi-agent进行二次开发，以及针对于我们的ForgePilot项目，如何设计，补充了ForgePilot项目的Pi-agent模块的架构图。

然后我说一下我后续的进展：

11月初，我有4厂面试，都是用这个项目来去面试的，我会总结这个项目的面试真题，加深这个项目（拓竹科技的面试真题已经更新了，见上面链接）。同时会更新一下这个项目设计的Pi-agent部分，都是更新在8.真实面试过程记录这里。

大家可以先看消化这些内容，我自己都去面试了这么多次，应该来说足够面试了。

写于2026-11-03

我再更新完上述内容后会专题更新这个项目的架构，不过现在虽然没有成体系地更新，顺着我总结地这个思路，其实整个项目的很多细节已经很清楚了（我刚面试完字节，准备根据字节的内容复盘会更新这个项目的记忆，压缩模块）。我自己已经面试了3场了，讲的都还可以，不存在被问穿。

---

5 开源框架解读 - PI-Agent

当你学习完了理论部分，这里给出一个非常好的实战例子。有些同学问我，有没有推荐的开源的学习Agent的项目啊？那我的回答就是使用这个Pi-Agent。为什么？因为它足够简单，好上手。但是具备Agent项目的最核心的要素。同时他又支持扩展，官方给了很多扩展的例子。你通过这个项目，可以快速学习Agent，也给了你丰富的资料和空间去扩展，改造成自己的项目。另外他也足够权威，github上有快60k 的stars。包括也有博主说，Openclaw也就是使用它改造的，所以权威性和可学习性也不用担心。当然我们项目还有解析Hermes,解析Openclaw,ClaudeCode。 我的建议是先看这个项目，看明白再看其他的，因为其他项目复杂太多。

5.1 总述

配套视频：第一期：架构与核心机制（整体设计、核心运行机制与扩展能力）。

5.1.1 Pi 是什么,

Pi 是一个很小的、可以自我扩展的 Agent Harness，也就是 Agent 运行框架。它的产品口号是 "grows with you"。直白一点说，就是先给你一个很小的内核，后面缺什么再加什么。

刚装好的 Pi 很克制：默认只有 4 个工具，分别是 `read`、`bash`、`edit`、`write`；系统提示词大约 20 行；没有 MCP，没有子代理，没有权限弹窗，没有 plan 模式，也没有内置 todo。第一次看会觉得功能少，但这正是它的设计选择：内核只保留必要部分，其他能力通过扩展补上。你缺一个能力，就可以让 Pi 写一个 TypeScript 扩展把它接进去。

Pi 是 TypeScript monorepo，主要由 4 个 package 组成：

- **pi-ai**：多供应商 LLM API 抽象，覆盖 OpenAI、Anthropic、Google、Bedrock 等。

- **pi-agent-core**：Agent 运行时，包含工具调用、状态管理、会话和压缩。

- **pi-coding-agent**：面向编码场景的交互式 CLI。

- **pi-tui**：自研终端 UI 库，支持差分渲染。

5.1.2 Pi 与 Claude Code 的主要区别

Pi 选择"小内核加扩展"，Claude Code 选择"默认带齐常用功能"。这个差异会影响工具数量、权限模型、提示词长度、扩展方式和会话结构。下面按维度对比，每条都先说 Pi、再说 Claude Code：

5.1.3 为什么 Pi 适合学习

我认为这个项目特别适合，你在学完Agent理论之后，第一个学习的实践项目。不论你是想要阅读开源代码，还是说找一个项目改装，扩展，这个项目都太合适了。

Pi 是一个很合适的样本。原因主要有四个：

1. 内核小，能读完。Claude Code 这类功能密集的工具，新手容易迷失在边缘功能里。Pi 的主干就是 agent loop，入口集中，读起来更容易抓住脉络。别看他小，但他五脏俱全。基本上面试Agent岗位的时候，围绕记忆，压缩，会话恢复，Agent Loop(React)这些是面试最常考的点，这些都有。即使他不算很复杂，但是你学会了以后，对于一个Coding Agent整体的架构的意识思想是完备的。
2. Agent loop 是手写的。市面上有 OpenAI Agents SDK、Vercel AI SDK 这类框架，导入后很快就能跑，但很多关键步骤被封装起来了。Pi 把初始化、压缩、模型调用和工具循环都写在明面上，更适合学习。
3. 模块边界清楚。4 个 package 各管一块，内核和交互层分开。Agent Core 可以脱离终端，被 RPC 或 SDK 调用，所以读一层时不必同时理解所有 UI 细节。
4. 他的思想"grows with you"，太适合你去改造了，添加了。比如他没有Plan功能，没有MCP功能。都适合你一边用，一边学，一边改造和扩展。

5.2 结构速览

并知道接下来应该读哪几块。安装和日常使用细节不展开，官方视频里讲得更细

5.2.1 仓库结构速览

```Plain Text
packages/
├── ai/ # pi-ai：多供应商 LLM 抽象，providers/ 下一家一个文件
├── agent/ # pi-agent-core：agent loop、harness、session、compaction
├── coding-agent/ # pi-coding-agent：CLI 入口、工具实现、扩展系统、交互模式
└── tui/ # pi-tui：通用终端 UI 组件库（Box / Text / List / Markdown…）
```

> 四个包，按"两类角色"来记最省事：
>
> **产品本体：`coding-agent`。** 你在终端敲的 `pi` 命令，入口就是它（它的 `package.json` 里 `bin.pi` 指向 `dist/cli.js`）。它自己几乎不造轮子，而是把另外三个包装配起来用：要智能就用 `agent`，要画界面就用 `tui`，要连大模型最终落到 `ai`。

> **被它使用的三个库：** \`agent\`（agent loop 与会话）、\`tui\`（终端 UI 组件）、\`ai\`（LLM 抽象，最底层）。
>
> 依赖方向只有一个朝向，记牢它能省掉很多困惑。下面箭头表示"谁 import 谁"：
>
> ```Plain Text
> coding-agent ─┬─→ agent ──→ ai
>               └─→ tui
> ```

`coding-agent` 同时依赖 `agent` 和 `tui`；`agent` 再依赖 `ai`；而 `ai` 和 `tui` 是两个"叶子"——除了各自的外部 SDK，它们不依赖任何内部包，也都不知道 agent 的存在。

5.3 架构

这一章回答"Pi 由哪些部分组成"。不按模块清单平铺，而是顺着数据流走一遍：你敲下回车后，一条消息经过哪些部件，最后怎么变成 Agent 的回复。

5.3.1 架构总览

这一节是整章的地图。先给一张全局视角，把后面 3.2 到 3.9 串成一条线，之后每一节再单独展开其中一站。这里只关心运行时这些部件怎么协作；各部件分属哪个 package、谁依赖谁，2.2 已经讲过，不再重复。

你在终端敲下一句话、按回车，到屏幕上一个字一个字冒出回复，中间大致经过这几站：

1. **CLI 入口（3.7）**：唯一的入口就是 `pi` 这个命令。它启动时先解析配置、加载扩展、建好会话，然后根据"现在是终端还是管道、带没带参数"**选一种运行模式**，再把消息交给 agent loop。模式有三种——交互界面（interactive，默认的 TUI 聊天）、一次性打印（print，`pi "你的问题"`，打印完就退，接近传统 CLI）、程序化后端（RPC，给别的程序当后端）。入口只有一个，这三种只是它之后的不同运行形态，细节见 3.7。
2. **Agent Loop（3.2）是中枢，而且下面 3 到 7 这些事都发生在它的一轮循环之内。** 一轮里它依次：组装上下文 → 检查要不要压缩 → 调用模型 → 看模型要不要调工具 → 把这一轮产生的东西写进会话。换句话说，"调用 LLM""执行工具""写 Session"不是 loop 之外的独立步骤，而是 loop 在一轮里亲自做的几件事。
3. **Context 组装（3.3）**：把系统提示词、AGENTS.md、skills 描述、工具描述、历史消息和你这次的输入，拼成一份完整上下文。模型本身没有记忆，每一轮都要重新喂。
4. **Compaction 检查（3.6）**：上下文要是太长，就先让模型把历史总结成摘要。注意这一步也会**往会话文件 append 一条"压缩"节点**——摘要本身也是会话树里的一个条目，之后取上下文时用它替换掉被压缩的那段历史。
5. **调用模型（3.9）**：通过 `pi-ai` 把上下文发给当前选中的 provider（OpenAI / Anthropic / Google…）。对 loop 来说，换模型只是换一个 provider，主流程不变。

6. **工具循环（3.4）**：模型若要调 `read`/`bash`/`edit`/`write`，loop 就执行工具、把结果回灌给模型；模型可能接着调，也可能收尾。读写代码、跑命令，真正发生在这一步。
7. **写入会话（3.5）**：这不是循环跑完才做的收尾，而是**边跑边写**——每产生一条消息（你的输入、模型回复、每个工具结果）、每做一次压缩，都立即按"只追加 + 父指针"往 JSONL 文件追加一行；只有换模型、改工具集这类配置变更，才推迟到一轮结束时统一写。所有这些条目靠 `parentId` 串成那棵可回退、可分叉的会话树。
8. **TUI 渲染（3.8）**：整个过程中，loop 不断广播事件，终端界面订阅这些事件，用差分渲染把消息流、工具调用和状态变化实时画出来，而且不闪烁。

顺带说一句它为什么能这么拆：中枢那套东西（loop、context、session、compaction）不碰终端，所以同一套内核既能驱动这里的 TUI，也能被 RPC 或管道调用——这一点 3.7 讲三种运行模式时还会再碰到。

5.3.2 Agent Loop

Agent loop 是 Pi 最核心的部分。每次你向 Pi 发送消息，背后都会跑一轮循环。主干大致是这样：

1. 初始化 context：把系统提示词、AGENTS.md、skills 描述、工具描述、历史消息和当前消息拼成完整上下文。
2. Transform / 压缩：检查上下文是否过长，过长就先做 compaction。
3. 调用大模型：把上下文发给当前选中的 LLM provider。
4. 工具调用循环：如果模型要调工具，就执行工具，把结果回灌给模型。模型可能继续调工具，也可能停止。
5. 回复：模型不再调工具时，输出最终回答，本轮结束。

这个流程看起来简单，但要写稳定并不轻松。实际实现里还要处理流式输出、运行中插话（steering）、收尾追问（follow-up）、工具并行或串行执行，以及中断（abort）。这些逻辑都集中在 `runLoop()` 附近。

这个就是一个非常标准的流程了，ClaudeCode,Hermes都采用了他。这个循环也可以理解成是一个标准的React循环。 面试官也问过原题：工业上一个React是怎么做的？**就是上面的内容，请牢记。**

> 源码入口：`packages/agent/src/agent-loop.ts` 的 `runLoop()`、`streamAssistantResponse()`、`executeToolCalls()`。

5.3.3 Context 组装

模型本身没有长期状态。每次调用，都要把它需要知道的内容重新放进 context。Pi 的 context 通常包含这些部分：

- 系统提示词：硬编码的极简模板，大约 20 行。可以追加 `append-system.md`，用 `--system-prompt` 覆盖，也可以用 `system.md` 完全替换。
- AGENTS.md：home 目录和当前工作目录里的项目说明会被追加进来。Pi 也兼容读取 `CLAUDE.md`。
- Skills 描述：所有已加载技能的 name、description、location。
- 工具描述：当前启用工具的说明。
- 历史消息或摘要：本轮之前的对话；如果已经压缩过，就使用摘要。
- 当前消息：用户这次输入的内容。

这里有个很重要的选择：系统提示词刻意保持短。把太多 AGENTS.md、skills 或行为规则塞进去，会撑大上下文，也会稀释模型注意力。

> **❓思考题：读到这里，想清楚一个问题：Skills 到底是怎么"被调用"的?**
>
> 简答：Pi 不是把所有 Skill 全文都塞进上下文，也不是把 Skill 做成一个特殊工具。它先把每个 Skill 的 name、description、location 放进系统提示词，并提示模型：如果任务匹配某个 Skill，就用 `read` 工具读取对应文件。也就是说，Skill 的自动使用是"提示词引导 + 模型判断 + read 工具按需加载"。如果用户显式输入 `/skill:name`，Pi 才会直接把这个 Skill 文件展开进用户消息。
>
> 源码入口：`packages/agent/src/harness/system-prompt.ts`、`packages/coding-agent/src/core/system-prompt.ts`。

5.3.4 Tools

Pi 默认只给模型 4 个工具：

read：读文件。

bash：执行 shell 命令。

edit：编辑文件。

write：写文件。

工具很少，但组合起来够用。尤其是 `bash`，它能调用系统已有命令，很多搜索、列目录、检查项目状态的需求都可以通过它完成。

源码里还有 `grep`、`find`、`ls`，但默认不启用。它们主要给**只读模式用**。比如通过 `--tools read,grep,find` 启动时，Agent 可以查文件，但不能直接改文件或执行任意 shell 命令。这样同一套工具实现可以切换出"可写"和"只读"两种工作方式。因为在可写模式下，给到了Agent Bash工具，它本身就可以通过Bash来调用grep查找，不必要再给grep了。但是在只读环境下，不能给Agent bash工具，所以需要给它grep,find,ls.

> 源码入口：`packages/coding-agent/src/core/tools/index.ts`，看 `createCodingTools()` 和 `createReadOnlyTools()`。

5.3.5 Sessions

这一小节把Agent系统的存储机制，如何恢复讲得非常清楚。其实也是一个标准的行业做法了，行业里叫会话存储。ClaudeCode和Pi都是用的是JSONL存储。存储方式类似，里面的一些回退细节不一样。不过本质相似，学习了本小节，对于Agent会话存储会有深入的了解。

怎么存：一个会话 = 一个 JSONL 文件，只追加

会话文件放在 home 目录下的 `~/.pi/agent/sessions/`，并按工作目录编码成子目录。也就是说，你在 `dashboard` 项目和在 `weather-app` 项目里的对话，会自然分开保存。

一个会话就是一个 `.jsonl` 文件，结构很简单：

- 文件**第一行**是文件头，记录会话 `id`、`cwd`（工作目录），以及可选的 `parentSession`。
- 之后**每一行是一个条目（entry）**，新条目只往文件末尾 `append`，从不回头改写已经写下的行。

这种"只追加、不改写"的写法有两个好处：写入快（不用重写整个文件），历史天然不可变（不会把旧数据写坏）。

什么时候写?

是 agent loop 跑的过程中边跑边写，不是等一轮结束才统一落盘。具体有三种时机：每产生一条消息（你的输入、模型回复、每个工具结果，对应 loop 的 `message_end` 事件）就立即 append 一行；每做一次压缩，也立即 append 一条 compaction 摘要节点；只有换模型、改 thinking、改工具集这类配置变更是例外，会先排进一个队列、推迟到一轮（turn）的边界再统一 flush（这点呼应 5.3 讲的 turn 快照：配置变更收敛到轮的边界生效）。换句话说，对话内容实时落盘以防丢失，配置变更则按轮归并。

条目不只有聊天消息。除了 `message`，还有 `model_change`（换模型）、`compaction`（压缩摘要）、`label`（给某条消息打标签），以及一类专门用来记录"当前位置"的条目。但不管哪种，每个条目都带同样的两个关键字段：

```Plain Text { "id": "条目自己的ID", "parentId": "我接在哪个条目后面", "type": "message", ... }
```

> **`parentId` 是整棵树的命脉**：它记录"我的父节点是谁"。一个扁平的文件，靠这一个字段就能还原出树形关系。
>
> **分支怎么长出来：正常是链，回头再接才分叉**
>
> 新消息的 `parentId` 取自一个表示"当前位置"的指针——也就是"你现在站在树的哪个节点上"。正常聊天时，每发一条，新条目的 `parentId` 就指向上一条，于是连成一根链：
>
> ```Plain Text
> A ← B ← C ← D      （正常对话，看起来就是直线）
> ```

分叉发生在你**回到中间某个节点再往后发**的时候。假设你回到 B，然后发了新消息 E，此时 `E.parentId = B`：

```Plain Text C ← D ← 老分支（C 的父节点是 B） / A ← B \ E ← 新分支（E 的父节点也是 B）

> B 一下子有了两个孩子 C 和 E——**分叉就这么出现了**。注意：老分支 C、D **一行都没被删**，仍然躺在同一个文件里。所谓"会话树"，就是这样从一个扁平文件里自然浮现出来的。
>
> **回退怎么实现：不删数据，只挪一个指针**
>
> 这里是最关键、也最反直觉的一点。Pi 的回退**不删除任何历史**，它做的只是两步：往文件追加一行"我把当前位置挪到 X 了"，然后把内存里的"当前位置"指针指向 X。仅此而已。
>
> 那为什么回退后，模型就"忘掉"了被抛弃的那条分支？因为**喂给模型的上下文，从来不是整棵树，而是从当前位置顺着 `parentId` 一路爬到根的那一条链**。源码里就是一个 while 循环：从当前节点出发，不断取 `parentId` 找父节点，直到根，沿途收集到的条目就是这一轮的上下文。
>
> 于是：
>
> - 你站在 E 这条分支 → 模型看到 `A → B → E`，**完全看不到 C、D**。
> - 你回到 D → 模型看到 `A → B → C → D`，这次轮到 E 不在路径上。
>
> **这就是"回退让上下文变干净"的真相**：不是谁删了数据，而是取上下文时只走当前这一条根路径，被你放弃的分支天然不在路径上。文件里其实什么都没少。
>
> > 一句话记忆：**树是存储形态（文件里所有分支都在），链是喂给模型的形态（只取当前节点到根的一条）。**
>
> ![](images/Ghm5b6E0ioJKwxxn9bUcrK3lnYg.png)
>
> **如果中间发生过压缩：树里会多出一个 compaction 节点**
>
> 前面讲了两种"动作"在文件里的样子：正常聊天往链尾接消息节点，回退则追加一行"挪指针"。你迟早会遇到第三种节点——**压缩（compaction）节点**。它的存储套路和前两种一模一样，在这里先把"它长什么样、爬链时怎么被用"讲清楚；到了 3.6，就只剩"什么时候压、压哪一段"这种纯策略问题了。
>
> **压缩节点也只是往链尾又 append 的一行。** Pi 决定压缩时，不删任何旧消息，只是造一个 \`compaction\` 节点追加到文件末尾，\`parentId\` 指向"压缩那一刻的最新消息"。假设压缩前是这样：
>
> ```Plain Text
> A ← B ← C ← D          （D 是当前 leaf）
```

压缩后，文件只多了一行 K：

```Plain Text A ← B ← C ← D ← K （K.type = "compaction"，K.parentId = D，K 成为新 leaf）

> A、B、C、D **一个字都没改**，K 只是又一行追加。之后你再发消息 E，照常接在 leaf 后：\`…D ← K ← E\`。
>
> **K 节点里装了什么？** 看源码 \`types.ts\` 的 \`CompactionEntry\`，关键就两样：
>
> ```Plain Text
> {
>   "type": "compaction", "id": "K", "parentId": "D",
```

```
  "summary": "## Goal … ## Progress …",   // 摘要文本：把更早的历史总结成结构化文字
  "firstKeptEntryId": "C"                  // 保留起点：从 C 起的原文留着，比 C 更早的用摘要顶替
}
```

注意它**不存被压缩消息的副本**，只存一段 `summary` 加一个指针 `firstKeptEntryId`。这个指针是钥匙：它把链切成两段——`firstKeptEntryId`（这里是 C）**之前**的（A、B）将被摘要替换，从 C 起的原文继续保留。

压缩在哪一步真正生效？还是在"爬链取上下文"那一步。 回退小节说过，喂给模型的上下文是 `getPathToRoot` 从 leaf 爬到根的那条链。现在链上多了个 K，爬出来是完整的 `A → B → C → D → K → E`（compaction 节点本身也在链里）。魔法在下一步 `buildSessionContext`（session.ts）：它把这条链翻译成"喂给模型的消息"时，遇到 compaction 节点会特殊处理—

1. 先放一条**摘要消息**（K 里的 `summary`），代表被压掉的早期历史；
2. 再接 `firstKeptEntryId` 到 K 之间的**原文**（C、D）；
3. 最后接 K 之后的**新消息**（E）。

于是模型实际看到的是 `[摘要] → C → D → E`，而不是 `A → B → C → D → E`。A、B 仍原原本本躺在文件里，只是这一次没被翻译进上下文。

把回退和压缩并在一起看，其实是同一招：

> 文件永远只追加、从不改写；所有"看起来像删除或改写历史"的效果，**都发生在"取上下文"那一步，而不是动文件。**回退改的是"从哪个节点开始爬"；压缩则是在链里埋一个 compaction 节点，爬到它时把它前面的原文换成一句摘要。两者都没动磁盘上的一个字节。

记成一句话：**回退换起点，压缩换中段——但文件本身，永远只增不改。**

对照上面那张"未压缩"的图，这张图画的就是**发生压缩后**的样子：链尾多出一个 `compaction` 节点 K，它带着 `firstKeptEntryId` 指针把历史切成两段，于是喂给模型的上下文从"整条原文链"变成了"摘要 + 最近原文"。

一个重要的边界：回退只回退对话，不回退代码

上面说的回退，全程只在动会话文件里的指针——**它不会碰你工作区里的代码文件。**也就是说，如果 Agent 在 C、D 两步里改过 `main.py`，你回退到 B，对话回到了 B 时的样子，但磁盘上的 `main.py` 还是被改过的版本。Pi 的内核里**没有任何"文件快照"机制。**

这是 **Pi 极简内核哲学的一个典型取舍**：它不把"还原代码"塞进内核，而是把这件事留给扩展。官方就给了一个示例扩展，思路很直接——**用 git 来记代码快照**：每一轮 Agent 动手前，先用 `git stash create` 给当前代码打一个快照，并和"当前会话节点"建立对应关系（一张"会话节点 → 代码快照"的表）；等你回退到某个节点时，再把那一刻的代码 `stash apply` 回来。于是几十行扩展，就把"对话 + 代码一起回退"补齐了。

对比一下 Claude Code 会更清楚两条路线：

- **存储格式上，两者几乎一样**：都是 JSONL、只追加，每条记录带父指针（Pi 叫 `parentId`，CC 叫 `parentUuid`），都能还原成一棵会话树，回退都不删历史。

- **区别在回退要不要还原代码**：Claude Code 的回退**内核自带文件快照**，能把对话和代码文件一起还原；Pi 的内核只回退对话，想连代码一起回退得靠扩展（借助 git）补上。

一句话：**底层存储是同一套思想，差别在于"回退"这个动作覆盖到哪——只回对话，还是连代码一起回。**

> 源码入口：`packages/agent/src/harness/session/jsonl-storage.ts`（`appendEntry` 追加、`setLeafId` 移动指针、`getPathToRoot` 取链）、`session.ts`（`appendCompaction` 追加压缩节点、`buildSessionContext` 把链翻译成上下文并处理 compaction）、`types.ts`（`CompactionEntry` 的 `summary` / `firstKeptEntryId` 字段定义）；代码回退的示例扩展见 `packages/coding-agent/examples/extensions/git-checkpoint.ts`。

5.3.6 Compaction

对话变长以后，Agent 不能无限把完整历史塞进模型。Pi 的做法是 compaction：把较早的一段历史总结成摘要，然后用"摘要 + 最近原文"继续对话。

压缩节点在 session tree 里长什么样、爬链时怎么用摘要顶替历史，已经在 3.5 讲过（就是那个 `compaction` 节点 + `firstKeptEntryId` 指针的机制）。所以这一节不再重复"存储形态"，只回答两个纯策略问题：**什么时候压、压哪一段。** 一句话接住 3.5：压缩不删历史，只是在链尾埋一个摘要节点，下一次爬链取上下文时用它顶替早期原文。

什么时候压缩？——触发时

Pi 不会在每个工具调用之间反复算上下文，而是只在**轮次边界**检查，主要是两个点：

1. 一轮 Agent 结束后（模型回复完、工具循环也跑完）。
2. 用户下一次发送 prompt 前，避免新一轮刚开始就把上下文撑爆。

除此之外，也可以手动触发，扩展系统也能在合适时机调用压缩。值得一提的是：****判断"要不要压"的逻辑不在内核 harness 里，而在应用层（`coding-agent`）。**** harness 只提供两样东西——一个纯函数 `shouldCompact()` 做判定、一个 `compact()` 执行动作；至于"每轮结束后该不该调它"，是 `coding-agent` 在每个 assistant 回合结束时决定的。这也呼应了 Pi"极简内核、策略外置"的一贯取舍。

什么时候压缩？——触发阈

判定函数 `shouldCompact()` 的条件很直白（`compaction.ts`）：

```Plain Text
contextTokens > contextWindow − reserveTokens
```

> 也就是"当前上下文 token 数，超过了模型上下文窗口减去预留量"。\`reserveTokens\` 默认 **16384**，这块预留是留给压缩本身的——生成摘要要发一次请求、也要容纳摘要输出，不能等真的撑满了才动手。

`contextTokens` 用什么算？****优先相信模型返回的真实 \`usage\`***（\`totalTokens\`，或 \`input + output + cacheRead + cacheWrite\` 相加），这是最准的。只有两种情况退回字符数估算（约"字符数 ÷ 4"）：一是请求报错、根本没拿到 usage（比如 529）；二是需要估算"保留最近哪一段"时。还有一种是模型直接报 context overflow 错误，这时不等阈值、立即压缩，并自动重试一次刚才失败的请求。

压缩哪一段？——保留窗口与切点

Pi 不是把全部历史都压成摘要，而是**保留最近一段原文、只压更早的部分**。保留多少由 \`keepRecentTokens\` 控制，默认 **20000**。具体做法是从最新消息往回累加 token，攒够 2 万就停，这个点之后保留原文、之前交给摘要（对应 3.5 里那个 \`firstKeptEntryId\` 指针）。

> 这个下面的规则其实就比较细节了，和具体的存储粒度（你是以整个提问-回答为粒度存，还是以每一次小的llm返回为粒度存），以及具体的细节，如何避免切坏语义。通常面试官不会问这么细致，这个也和具体业务场景要求相关。我们理解规则灵活运用。

但切点不能乱下刀，有两条硬规则：

\- **不能切断工具调用对。** assistant 发起 tool call、紧跟着 tool result，这两者语义上是一组。Pi 选切点时只认合法边界（user / assistant 等消息边界），****明确不把 \`toolResult\` 当切点***，避免只留下半截调用链、把上下文弄坏。

\- **回合被切到一半要补救。** 如果按 token 算出的切点正好落在某一轮任务中间，Pi 会把这一轮的前半段**单独再摘要一次**，让保留下来的后半段原文还能接得上，不至于"开头没了、中间突然冒出来"。

摘要里写什么？——提示词的核心思想

Pi 的摘要**不是一句"帮我总结上文"**，而是给压缩模型一份措辞严格的提示词，核心思想就一句话：**别续写对话，只产出一份"交接班用的结构化检查点"，让另一个 LLM 能照着它无缝接手工作。** 系统提示词开宗明义就强调这一点（原文大意）：

> 你是一个上下文摘要助手。读完这段对话后，**只**按指定格式输出结构化摘要。**不要**继续对话，**不要**回答对话里的任何问题。

为什么这么强调"不要续写"？因为压缩复用的是同一个对话补全接口，模型很容易把这段历史当成"该我接话了"而真的去回答问题——那摘要就毁了。所以提示词反复用大写 `Do NOT` 把它摁住。

> 这个模板也可以稍微记一下，如果面试问你是怎么压缩的，要记录哪些东西，就是从这里回答。

真正的摘要提示词要求模型按**固定模板**输出，骨架是这几节（这就是压缩策略的"信息取舍"——它规定了什么必须留下来）：

```Plain Text
## Goal              目标：用户到底想完成什么
## Constraints & Preferences    约束与偏好：用户提过的要求、限制
## Progress          进度：### Done 已完成 / ### In Progress 进行中 / ### Blocked 卡住
## Key Decisions     关键决策：做了什么选择、为什么
## Next Steps         下一步：接下来该干什么（有序列表）
## Critical Context  关键上下文：继续工作必需的数据、引用
```

模板末尾还有一句对 Coding Agent 极其关键的硬要求:**"原样保留精确的文件路径、函数名、错误信息"**——这些细节一旦被模型"复述走样",接手的 Agent 就会找错文件、改错函数,所以提示词专门点名它们不许概括。

一句话抓住提示词策略:**它不是压缩文字长度,而是把一段冗长对话"蒸馏"成一份带固定栏目的工作交接单——目标、进度、决策、下一步、以及不许走样的路径/函数/报错。**

同步还是异步?

从 Agent 主流程看，压缩是一个**需要等待**的步骤——摘要本身要再调一次模型生成，没生成完就不能安全进入下一轮。所以一旦决定压缩，会先暂停对话，等摘要生成、写入 compaction 节点、重建上下文，再继续。但它不是每轮都阻塞：只有真到阈值才触发，平时到边界点检查一下、没超标就直接继续。区别在于——上下文已经溢出时，压完会自动重试刚失败的请求；只是预防性接近阈值时，压完让下一轮从新上下文继续即可。

下面这张图把整个压缩策略串成一条流程：从"何时触发"的判定，到"压哪段、留哪段"的切点选择，再到提示词把历史蒸馏成结构化摘要，最后接回 session tree。

总结：**Pi 的 compaction 是在轮次边界触发、由应用层按 `contextTokens > contextWindow – reserveTokens` 判定、会等待模型生成摘要的上下文重写步骤；它保留最近约 `keepRecentTokens` 的原文、只压更早的前缀，切点避开工具调用对，并把摘要作为新的会话节点接进 session tree（存储细节见 3.5）。**

> 源码入口：`packages/agent/src/harness/compaction/compaction.ts`（`shouldCompact` 阈值判定、`findCutPoint` 选切点、摘要 prompt 模板与生成）、`packages/coding-agent/src/core/agent-session.ts`（每轮结束的检查与自动压缩流程）、`packages/coding-agent/src/core/session-manager.ts`（保存 compaction 节点）。对应视频文稿见 `PI Architecture explained.txt` 的 Compaction 段。

5.3.7 CLI 入口和三种运行模式

> 在看这个章节前，我想请大家思考一个问题？如果你设计了一个coding Agent。你怎么设计这个架构，让它既可以作为一个独立的Agent产品用，又能够像一个插件一样，方便地给到其他的应用，比如Openclaw用，让其他进程能够快速集成Coding的能力？答案就在本小节，PI的这个设计方法值得我们学习。

输入 `pi` 后，入口主要在两个文件：

- `cli.ts`：接住命令，设置进程标题等，然后调用 `main`。
- `main.ts`：解析参数，解析配置，找到工作目录，加载扩展，创建 agent session，然后按指定模式运行。

Pi 有三种运行模式：

- **interactive**：默认模式，终端聊天界面。

- **RPC**：程序化调用，给其他程序当后端。比如说你的**小龙虾**，你想外接一个 coding 的能力，就可以使用 PI 的 RPC 模式。

- **print-stdio**：`pi "你的问题"`，打印答案后退出，接近传统 CLI。

因为内核不绑定 UI，同一套 Agent Core 才能被这三种模式复用。

> 源码入口：`packages/coding-agent/src/cli.ts`、`packages/coding-agent/src/main.ts`。

5.3.8 TUI

> TUI这里是负责前端的内容，我没有特别关注。我自己写的项目，前端的内容，完全没有看代码，面试也没问。如果你也不想学前端，也可以基本浏览一下本章节，大概知道它是用什么技术栈实现的就好。

TUI 可以理解成"跑在终端里的前端"。但它不是我们平时说的 Web 前端，也不是 FastAPI 这种后端框架。FastAPI 是 Python 后端框架，通常负责 HTTP API；Pi 这里没有浏览器、没有 DOM、没有 HTML/CSS，也没有 React/Vue。

Pi 的 TUI 技术栈更准确地说是：

- **运行时**：Node.js。

- **语言**：TypeScript。

- **UI 库**：自研的 `@earendil-works/pi-tui`。

- **渲染目标**：terminal，也就是用户当前的命令行窗口。

- **渲染方式**：输出 ANSI escape sequences，控制光标、颜色、清屏、局部刷新等终端行为

所以它和 Web 前端的对应关系大概是这样（左边是 Web 前端，右边是 Pi TUI 里的对应物）：

- 浏览器 → 终端。
- DOM 节点 → 文本行和 ANSI 控制序列。
- React/Vue 组件 → `Component` 对象。
- CSS 样式 → ANSI 颜色、粗体、下划线等。
- 鼠标/键盘事件 → stdin 里的按键序列。
- 重新渲染页面 → 重新计算终端文本并差分刷新。

`@earendil-works/pi-tui` 里最核心的抽象是 `Component`。一个组件只需要实现 `render(width)`，返回一组字符串；如果它需要处理输入，就再实现 `handleInput(data)`。输入框、Markdown 消息、选择器、弹窗、底部状态栏，本质上都是这种组件。

Pi 没有用现成的 Ink 这类 React TUI 框架，而是自己实现了渲染器。这样做的好处是能更细地控制终端体验：比如差分渲染，只刷新变化的行；同步输出，减少闪烁；处理 bracketed paste；支持 Kitty / iTerm2 图片协议；处理复杂按键和终端尺寸变化。

在架构上，TUI 不直接负责 Agent 推理。它只是 interactive mode 的界面层：用户在编辑器里输入 prompt，TUI 把输入交给 Agent Session；Agent Session 在运行过程中发出事件，比如消息更新、工具开始、工具结束、压缩完成；TUI 订阅这些事件，把它们渲染成终端里的消息流、工具执行块、底部状态栏和各种弹窗。

一句话：**Pi 的 TUI 是 TypeScript + Node.js 写的终端前端，不是 Web 前端；它用自研组件系统和 ANSI 终端控制序列，把 Agent Core 的事件实时画到命令行里。**

> 源码入口：`pi/packages/tui/src/`（自研 TUI 框架，重点看 `tui.ts`、`terminal.ts`、`components/markdown.ts`）、`pi/packages/coding-agent/src/modes/interactive/interactive-mode.ts`（Pi 具体的交互界面如何组装）。

5.3.9 多供应商抽象

> 这里用的是**策略+工厂模式**，和我们笔记里**RAG项目的思路**是一样的。我们RAG项目使用的是一次输入-输出，也没有ReAct，所以没有处理工具，流式SSE的内容，只用处理HTTP格式。不过思路是一样的。

最底层的 `pi-ai` 把不同 LLM 供应商统一成一套接口。OpenAI、Anthropic、Google、Bedrock、Mistral、Cloudflare 等供应商的 API 都不一样，但 Agent Core 不应该关心这些差异。它只需要调用一个统一入口：

```
streamSimple(model, context, options
```

这里的核心思想就是策略模式：同一件事，也就是"调用模型并拿到流式回复"，底下可以有多种实现。Pi 不是用传统的"抽象基类 + 子类继承"来做，而是更 TypeScript 的写法：每个 provider 注册一组函数，比如 `stream` 和 `streamSimple`。运行时 `pi-ai` 根据 `model.api` 找到对应的 provider 函数，然后调用它。

可以把流程理解成：

```Plain Text
Agent Core -> 调用 streamSimple(model, context, options) -> pi-ai 根据 model.api 找到对应 provider -> provider 把 Pi 的统一上下文转成供应商请求 -> 供应商返回流式结果 -> provider 再转回 Pi 的统一消息事件
```

> 这里要注意区分 `provider` 和 `api`。`provider` 是模型来源，比如 OpenAI、Anthropic、OpenRouter、GitHub Copilot；`api` 是调用协议，比如 `openai-completions`、`openai-responses`、`anthropic-messages`、`google-generative-ai`。真正决定走哪套调用策略的是 `model.api`。这也解释了为什么很多不同供应商可以共用 OpenAI-compatible 的实现：它们来源不同，但 API 协议相近
>
> provider 主要做的是"适配"。Pi 内部有统一的 `Context`、`Message`、`Tool`、`AssistantMessageEventStream`，但每家模型 API 的字段结构都不同。provider 要负责几件事：

- 把 Pi 的统一消息格式转成供应商请求格式。
- 把**工具定义**和**工具调用**转成供应商支持的格式。
- 把供应商的 streaming chunk 转成 Pi 统一的文本增量、工具调用增量、结束事件。
- 把 token usage、stop reason、错误信息转成 Pi 内部统一结构。
- 处理每家特有的能力，比如 reasoning/thinking、prompt cache、图片输入、headers、OAuth、baseUrl 等。

所以对 Agent Core 来说，换模型就是换 `model`；对 `pi-ai` 来说，换模型就是根据 `model.api` 换一套 provider 策略。Agent loop 不需要知道底层是哪家模型，也不需要在主流程里写一堆 `if provider === "openai"` 这样的分支

一句话总结：**pi-ai\` 用统一接口包住不同模型 API，provider 负责双向翻译：把 Pi 的统一格式翻译成供应商格式，再把供应商响应翻译回 Pi 的统一格式。**

> 源码入口：`pi/packages/ai/src/stream.ts`（统一入口）、`pi/packages/ai/src/api-registry.ts`（provider 注册表）、`pi/packages/ai/src/providers/`（各供应商适配实现）、`pi/packages/ai/src/models.ts`（模型注册表）。

5.4 可扩展性

第 3 章讲 Pi 由什么组成，这一章讲能力怎么接上去。Pi 和很多"功能默认全带"的 Agent 不同，它把大量能力留给扩展系统。

一句话概括**：agent loop 在运行过程中持续广播事件，扩展通过订阅事件、注册能力和 hooks，在不改内核代码的情况下改变 Pi 的行为。**

5.4.1 事件系统

Agent loop 运行时会不断发事件：agent 开始、每一轮开始和结束、消息开始、消息更新、消息结束、工具开始执行、工具执行中、工具执行完、模型切换、用户输入等。一共有 30 多种事件。

底层事件总线在 `event-bus.ts`，实现很朴素。它是在 Node `EventEmitter` 上封了一层 `emit` / `on`，也就是发布订阅模式，再加一点错误隔离。某个订阅者抛错，不会直接拖垮整个 Agent。

这套事件广播让扩展可以在工作流的很多位置插入逻辑。扩展和 hooks 都建立在这个基础上。

> 源码入口：`packages/coding-agent/src/core/event-bus.ts`。事件定义在 `extensions/types.ts` 的 `ExtensionEvent` 联合类型里。

5.4.2 Extensions

扩展本质上是一个 TypeScript 模块。它拿到 `pi` API 对象后，可以做这些事：

- 注册工具：用 `registerTool` 给模型加新能力，比如 web search。
- 订阅事件：用 `on` 在事件上挂回调。
- 注册斜杠命令、快捷键和 CLI flag：对应 `registerCommand`、`registerShortcut`、`registerFlag`。
- 注册或覆盖模型供应商：用 `registerProvider` 接入自定义代理、企业 SSO 等。
- 注册自定义消息渲染器：用来改变某类消息在 TUI 里的显示方式。

- 操控 UI：弹选择框、确认框、输入框，设置页脚、页眉、状态栏和欢迎语。
- 改系统提示词、发消息、控制会话：比如新建、fork、导航、切换会话。

官方讲解里有两个例子很直观：一个是让 Pi 给自己写开机欢迎语扩展，每次启动显示一句随机 quote；另一个是写权限确认扩展，在执行 `rm -rf` 或 `git push -f` 之前先弹窗确认。类似功能在很多 Agent 里是内置项，在 Pi 里则是几十行 TypeScript 扩展。

> 源码入口：`packages/coding-agent/src/core/extensions/`，重点看 `loader.ts`、`runner.ts`、`types.ts`。API 全貌在 `types.ts` 的 `ExtensionAPI` 接口。

5.4.3 Hooks

订阅事件更像旁路观察；hooks 则可以拦截和改写数据。几个典型 hook 如下：

\- **tool_call**：工具执行前，可以 \`block\` 阻止执行，也可以改写工具参数。

\- **tool_result**：工具执行后，可以改写返回结果。

\- **context**：调用 LLM 前，可以改写即将发送的消息列表。

\- **before_agent_start**：用户提交后、loop 启动前，可以改写本轮系统提示词或注入消息。

-**before_provider_request**：发请求给供应商前，可以替换 payload。

\- **session_before_compact / session_before_tree**：压缩或导航前，可以取消或自定义处理。

权限确认扩展的实现思路就是挂在 `tool_call` 上：检测到危险 bash 命令就弹确认框，用户拒绝就返回 `block`。多个 hook 挂在同一事件上时，会按顺序链式执行。前一个 hook 的修改，后一个 hook 能看到

> 源码入口：设计文档 `packages/agent/docs/hooks.md`，事件结果类型见 `extensions/types.ts` 里的各类 `*EventResult`。

5.5 二次开发入门

配套视频：[第二期：二次开发入门](#doxcn1jr87Vz8qzxafzkyd8rgce)（视频链接待补充）。

配套面试题：[选型与二次开发](#doxcnXjIlN30dRhYUesNMwcibRb)。

如今，市场上已经有 Codex、Claude Code 等功能强大的 Agent，能够直接帮助我们编写代码、调用工具、完成复杂任务。Pi Agent 却选择了另一条路：**保持内核极简**，不把所有功能都塞进框架，而是鼓励开发者按自己的需求扩展，把它改造成自己的 Agent。

这种“少做一点”的设计，反而吸引了不少开发者围绕它构建项目，OpenClaw 就是其中一个例子，其底层使用 Pi 作为 Agent 核心。对这些开发者来说，Pi 不只是一个拿来即用的工具，更是一套**可以拆开、复用和改造的基础组件**。

不过，Claude Code 和 Codex 同样提供了 SDK 或扩展接口，也允许开发者定制行为。那么，Pi 在二次开发上究竟有什么独特优势？它的极简设计，又为什么更方便我们构建自己的 Agent？本小节就从这些问题出发，再介绍几种具体的接入方式。

5.5.1 为什么适合二次开发

Pi 的价值不只是允许给现成 Agent 添加插件，还在于它**把构建 Agent 所需的能力拆成可复用组件**，并通过扩展接口开放运行过程中的定制位置。我们既可以在已有 Pi 上增加功能，也可以拿它的组件构建自己的 Agent。

理解它为什么适合二次开发，可以从“**能复用什么、能控制什么，以及以后是否方便调整**”这几个问题出发。下面主要以 Claude Code 及其 Agent SDK 为参照，区分 Pi 的结构性优势和双方都有的扩展能力；这些比较不能直接推广到所有 Agent 框架。

1. 接入力度更细

Pi 允许我们根据已有系统的能力，选择不同的开发起点：只需要模型通信，可以使用 `pi-ai`；需要模型与工具之间的执行循环，可以使用 `pi-agent-core`；希望直接复用已经组装好的 Agent，则可以通过 `pi-coding-agent` 的 SDK 接入会话、现成工具、上下文压缩和扩展运行机制。

Claude 虽然也有自己的Claude Agent SDK，但它主要接入对象是 Claude Code 的现成运行机制：由 SDK 运行 Claude Code 二进制程序，提供工具、执行循环和上下文管理等能力。Pi 则进一步把模型通信、核心循环和现成 Agent 分成独立包，让我们决定复用到哪一层。相比之下，Pi 的优势是**组件复用粒度更细，灵活度更高**。

2. 扩展机制的深度和效率性

Pi 的作者 Mario 在演讲中提到，他对当时 Claude Code 的 Hook 有两个不满：一是开放的位置和介入深度有限；二是每次触发都要启动一个新进程，执行配置的命令，他认为这种方式不够高效。

Pi 则把这类逻辑写成 TypeScript 扩展中的事件处理函数，**由 Pi 在进程内调用**，不必仅仅为了执行一次 Hook 就重新启动外部程序。对于工具调用前检查、执行后记录日志等频繁触发的操作，这种方式可以省去反复启动进程的开销，也便于通过扩展接口直接读取运行状态、调整行为。

总结作者的意思就是，ClaudeCode虽然也有hook机制，但是提供的hook的数量和深度不如Pi-agent，同时效率也比较低。

3. 轻量性

Pi 采用**精简、按需扩展**的设计，没有把所有常见工作流都作为默认功能加入。我们可以围绕业务选择工具、组织上下文、添加所需机制，而不是先接受一整套固定工作流，再绕开其中不合适的部分。

极简的设计理念本来就更适合二次开发，因为我们不用在框架已有的，但是我们并不需要的功能上继续开发。

4. 机制易增删

随着模型的能力快速迭代，一些原本用于弥补模型不足的 Harness 逻辑，可能会失去价值。pi-agent的作者用“**build to delete**”表达今天构建的机制将来可能需要删除，主张减少不必要的构建，避免在 Harness 层过度设计。

这种设计理念，也受到了很多开发者的青睐。很多开发者担心使用的框架太重，未来模型迭代后，原本的补偿机制反而成了包袱，想删掉却又牵一发而动全身。

5.5.2 三种接入方式

做法	从哪里开始	自己的代码主要做什么
SDK 集成	自己的程序	按需调用模型接口、Agent 核心或现成的 Agent 会话，组织自己的业务流程
Extensions 扩展	已有的 Pi 运行环境	注册工具、命令、事件处理函数等，由 Pi 加载并执行
混合使用	自己的程序，加上加载扩展的 Pi 会话	通过 SDK 接入 Pi，同时让 Pi 加载扩展，分别处理应用接入和行为定制

SDK 集成主要是“**自己的程序调用 Pi**”，Extensions 扩展主要是“**Pi 加载自己的扩展**”，混合使用则将两个方向连起来。这个区别描述的是主要接入方向，不意味着扩展内部不能调用其他库。

下面用同一个假设需求贯穿三类做法：做一个能够查询公司制度、回答报销问题的助手。示例只解释实现逻辑，不代表已经完成或验证的生产系统。

1. SDK 集成

Pi 将不同职责封装成可安装的 npm 包，开发者在自己的项目中导入它们公开的 API。可以理解为就是Pi-agent开发者，把自己的核心能力拆解了，以包的形式给到你，你想用哪块的能力，就调用对应的api就行。比较典型的包总结如下：

先看 Agent 相关的能力，按复用程度从低到高排列：

复用程度	包	总体复用什么	具体能力举例
较低：模型接入层	`@earendil-works/pi-ai`	复用 Pi 与大模型通信的能力，不接入 Agent 的运行循环	统一对接不同模型提供商、发送请求、接收流式输出、适配工具描述及调用 / 结果消息；不执行工具
中等：Agent 核心层	`@earendil-works/pi-agent-core`	复用 Agent 的核心运行能力，基于它组装自己的 Agent	调度外部提供的工具实现、校验参数、执行工具并回填结果，继续模型与工具之间的循环；维护当前对话消息与运行状态，发出事件并支持中止；底层使用 `pi-ai`
较高：现成 Agent 层	`@earendil-works/pi-coding-agent` 的 SDK	在核心运行能力之上，复用现成 Pi Agent 已经装配好的运行机制	在 Core 的工具配置与执行能力之外，提供现成文件和命令工具、会话持久化与恢复、上下文压缩、扩展加载

图注：Pi SDK 三层能力与职责边界：完整 Pi Agent 运行层包含 Agent 核心层，核心层包含模型接入层；底层适配工具调用协议，中层执行工具，高层提供现成工具与会话机制。

图中的包含表示能力封装与复用：外层复用内层能力，再增加本层模块，并非源码物理嵌套。

这里需要区分“工具调用协议”“工具执行机制”和“具体工具实现”三个概念：

- **\`pi-ai\` 负责模型通信。** 它会把工具名称、描述和参数结构传给模型，将模型返回的工具调用请求解析为统一消息，并支持把工具结果消息送回模型。但它不会替你执行工具，也不会自动组织模型与工具之间的循环。其 `Tool` 类型以 `name`、`description`、`parameters` 等描述字段为主，没有 `execute`。
- **\`pi-agent-core\` 负责 Agent 执行。** 它的 `AgentTool` 在上述工具描述上增加 `execute` 等接口。Agent Loop 收到工具调用请求后，查找对应工具、校验参数、调用 `execute`、把结果追加到对话，再继续请求模型。具体工具函数仍需由应用或上层包提供，Core 不是现成文件工具的集合。
- **\`pi-coding-agent\` 提供现成 Agent 运行机制。** 在 Core 之上，它提供文件读写、编辑、命令执行等工具实现，以及会话持久化与恢复、上下文压缩、扩展加载。这里的会话保存与恢复不应笼统称为“记忆功能”，更不代表已经提供通用的长期业务记忆系统。

例如，模型提出“调用制度查询工具”时，`pi-ai` 负责把这一请求解析出来；Core 负责调用已配置的查询函数并回填结果；查询公司制度的具体逻辑则由我们编写，或者通过扩展提供。工具执行确实属于中层职责，但底层仍要理解模型 API 中的工具调用协议。

除了 Agent 运行相关的能力，Pi 也提供可以独立复用的界面能力。例如，`@earendil-works/pi-tui` 可以用于复用 Pi 的终端界面基础能力，包括终端渲染、输入框、编辑器和列表等组件。

通过SDK集成的方式接入，最大的特点是：需要什么能力就用什么模块，按需选取。

2. Extensions 扩展：给已有 Pi 增加功能

Pi 除了把能力拆成包供我们调用，也提供了一套插件机制。不再由我们选取模块来组装 Agent，而是在已有 Pi 上增加功能或修改行为。开发者把自己的逻辑写成 TypeScript 扩展，由 Pi 加载和执行。

扩展方向	总体能做什么	具体例子
工具能力	给 Pi 增加模型可以调用的工具	添加“查询公司制度”工具，让 Pi 可以调用公司的制度查询接口
操作入口	给使用者增加命令或快捷操作	添加一个斜杠命令，用来触发自定义工作流程
运行行为	在 Pi 开放的事件节点介入执行	工具执行前检查参数、拦截不允许的操作，或执行后记录日志
上下文与界面	定制 Pi 的上下文处理和交互方式	自定义压缩逻辑、增加终端状态提示或交互组件

通过 Extensions 接入，最大的特点是：**保留已有 Pi 的运行机制，通过它开放的接口增加或调整能力**，不需要自己重新搭建整个 Agent。能改哪些地方，仍然取决于 Pi 提供了哪些扩展接口。

这些扩展方向遵循同一套插件机制：Pi 提供扩展接口，我们把自定义逻辑写成 TypeScript 插件。插件加载并注册后，Pi 会在对应的执行位置运行插件逻辑，而不需要我们直接修改 Pi 源码。

本节参考资料

模型通信与工具协议参见 [Pi AI](https://github.com/earendil-works/pi/blob/main/packages/ai/README.md)；工具执行循环参见 [Agent Core](https://github.com/earendil-works/pi/blob/main/packages/agent/README.md)；会话集成与插件机制参见 [Coding Agent SDK](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/sdk.md) 和 [Extensions](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/extensions.md)。

5.5.3 ForgePilot 接入示范

本节以 ForgePilot 为例，说明如何选择 Pi 的接入层级，以及哪些能力复用、哪些由项目自行实现。完整架构图、职责分工和选型理由统一放在 [ForgePilot 架构设计 → Pi 核心层设计](#doxcnE03AFNbVZ57BdcXSl5UTjh)，请点击该章节继续阅读。

5.6 课后作业

基础与架构练习

通过这个小节，我是希望能带大家了解PI Agent的一个大概的思想和架构。如果你是想在这个基础上继续去扩展，深入学习，还是得靠大家。我总结了一些可以去做的扩展任务，包括让AI也总结出来了对应的参考代码和官方示例（可能不准）大家多包涵。

如果想在 Pi 基础上动手，可以按下面几个难度推进。下面每条都尽量附上了官方对应的示例扩展或文档，建议先读官方实现再动手——Pi 自带的示例很全，照着改往往比从零写更快。扩展 API 的全貌见 [`docs/extensions.md`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/extensions.md)，所有示例扩展都在 [`examples/extensions/`](https://github.com/earendil-works/pi/tree/main/packages/coding-agent/examples/extensions) 下。

入门

- 加一个 web search 工具：用 `registerTool` 注册一个调用搜索 API 的工具。作者自己也提到过，这是他会装的一个常用扩展。参考最小自定义工具示例 [`examples/extensions/hello.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/hello.ts)；API 见 `extensions/types.ts` 的 `registerTool` 和 `ToolDefinition`。
- 写一个开机欢迎语扩展：订阅 `session_start`，启动时随机显示一句 quote。参考 [`examples/extensions/custom-header.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/custom-header.ts)（演示 `ctx.ui.setHeader`）和 [`hello.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/hello.ts)。

中级

- 权限确认弹窗扩展：挂 `tool_call` hook，检测危险 bash 命令，比如 `rm -rf`、`git push -f`，执行前弹窗确认，拒绝就 `block`。官方已有现成示例 [`permission-gate.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/permission-gate.ts)；相关的还有 [`protected-paths.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/protected-paths.ts)（拦截对 `.env`、`.git/` 等路径的写入）和 [`confirm-destructive.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/confirm-destructive.ts)（清空 / 切换 / fork 等危险会话操作前确认）。
- 用 todo 工具替代内置 todo：Pi 内核没有内置 todo。官方示例 [`todo.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/todo.ts) 演示了一个完整的 todo 工具 + `/todos` 命令，带自定义渲染和状态持久化，可以直接参考。
- 自定义主题 / 状态栏：让 Pi 生成一套终端配色主题。入口 `getAllThemes` / `setTheme`，参考 [`mac-system-theme.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/mac-system-theme.ts)（跟随系统深浅色）、[`status-line.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/status-line.ts)、[`custom-footer.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/custom-footer.ts)。

进阶

- MCP adapter：写扩展，把 MCP server 的工具桥接到 Pi 的工具系统。官方没有直接的 MCP 示例，但可以参考 [`dynamic-tools.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/dynamic-tools.ts)（运行时动态注册工具）和 [`ssh.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/ssh.ts)（把工具委派到外部进程）的写法。
- 子代理：注册一个工具，工具内部再启动一个 Agent Core 实例处理子任务。官方有完整示例 [`examples/extensions/subagent/`](https://github.com/earendil-works/pi/tree/main/packages/coding-agent/examples/extensions/subagent)，演示带独立上下文窗口的子代理委派。
- Plan 模式：用扩展实现"先出计划，确认后再执行"的工作流，Pi 默认没有这个模式。官方示例 [`examples/extensions/plan-mode/`](https://github.com/earendil-works/pi/tree/main/packages/coding-agent/examples/extensions/plan-mode) 实现了 Claude Code 风格的

`/plan`（只读探索 + 步骤跟踪）。

\- 节点回退后文件也一起回退：如 1.2 所说，Pi 默认 fork / 回退**只回退对话，不动工作区代码**。这个作业就是把代码也接上：订阅 \`turn_start\` 时用 \`git stash create\` 给当前代码打一个快照并和会话节点关联，在 \`session_before_fork\` 时用 \`git stash apply\` 把代码还原到那个节点。官方有现成示例 \[\`git-checkpoint.ts\`\](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/git-checkpoint.ts)，几乎可以直接用；另见 \[\`auto-commit-on-exit.ts\`\](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/auto-commit-on-exit.ts)。

- 用 RPC 模式接入你自己的 OpenClaw：Pi 的 Agent Core 可以脱离 TUI，被别的程序通过 RPC 当后端驱动（见 3.7 的三种运行模式）。这个作业是把你自己的前端 / 程序 OpenClaw 作为 RPC 客户端接进来，由它发消息、收事件，Pi 只当"无头大脑"。参考 RPC 协议文档 [`docs/rpc.md`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/rpc.md)，以及扩展侧 UI 示例 [`rpc-demo.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions/rpc-demo.ts)（搭配 [`examples/rpc-extension-ui.ts`](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/rpc-extension-ui.ts) 一起看）。

挑战

- 从零仿写一个极简 harness：照着 `agent-loop.ts` 的主干，用任意语言实现一遍 init context -> call LLM -> tool loop -> reply，再加上 JSONL 树状会话。这个练习最能帮助你吃透 Agent 的基本运行方式。

5.7 面试问题

5.7.1 基础与架构

> 题目是我出的，都是根据我的面试复盘经验总结的。（答案是AI写的）下面的题目可以说是考察Agent项目的高频考点，我们在面经部分也根据不同项目总结过很多次了。下面这些问题结合当前项目，也就是假设这个项目是你自己做的，面试官问你项目相关问题，你的参考答案。

下面这些回答按"面试可以直接说"的口吻写，不追求把源码细节全背出来，重点是把设计思路讲清楚。

Q1：你的 Agent 的记忆功能是如何实现的？长短记忆的组成是什么？

> 其实长期记忆这里，当前项目只有Agent.md，没有自动维护的长期记忆，也就是这样你的Agent项目不记事，这个其实不好，显得项目不聪明。你可以自己扩展一下，答案还是基于当前项目如实回答。

答：我先说一个前提，模型本身是无状态的，它不记得上一轮说过什么，所以记忆其实是我每一轮调用模型前重新组装上下文喂进去的。我把记忆分成短期和长期两层。

短期记忆就是当前这次会话的内容：用户说过的话、模型的回复、每个工具的执行结果。它们都按"只追加 + 父指针"存在一个 JSONL 会话文件里，构成一棵会话树。每轮调用模型前，我从当前叶子节点沿父指针回溯到根，取出这条路径上的消息，组装成上下文。也就是说模型每轮看到的不是整棵树，而是当前分支这一条线。如果这条线太长，就会先做压缩，把旧消息换成摘要，这也是短期记忆的一部分。

长期记忆是文件型的，跨会话存在。比如项目根目录的 AGENTS.md / CLAUDE.md、用户级的全局规则、还有 skills（技能说明）。这些不靠模型临时回忆，而是在我拼系统提示词的时候加载进去。值得一提的是 skills 是按需加载的：系统提示词里只放每个技能的名字、描述和文件位置，不放全文，模型觉得需要时再用 read 工具去读，这样能省上下文。

一句话总结：短期记忆解决"这次对话进行到哪了"，靠会话树的当前分支；长期记忆解决"这个用户、这个项目长期有哪些规则

和偏好"，靠加载文件型上下文。

Q2：你的 Agent 压缩是如何做的？压缩策略和压缩时机是什么？

答：压缩解决的是上下文越聊越长、最后超出模型窗口的问题。我的做法是把旧的历史消息交给模型，按一个固定模板总结成一份结构化摘要，然后用这份摘要替换掉那段原文，最近的消息则原样保留。

摘要不是随便总结，而是有固定章节的，比如 Goal（目标）、Constraints（约束）、Progress（已完成/进行中/受阻）、Key Decisions（关键决策）、Next Steps（下一步）、Critical Context（关键上下文），并且要求保留确切的文件路径、函数名、报错信息。这样换一个模型接着干也能续上。如果之前已经压过一次，下次是在旧摘要基础上增量更新，而不是从头再总结。

时机上，判断是用 token 占比：当前上下文 token 超过"模型窗口减去预留量"就触发。token 怎么算很关键——我不是用字符数除以 4 去估，而是优先用模型每次返回的真实 usage（input、output、cache 读写几个字段相加），只有最后那几条还没有 usage 的消息才用字符估算补上，这样最准。

还有一个设计上的细节：判断"要不要压"的逻辑其实不在内核里。内核只提供两个能力——一个纯函数 `shouldCompact()` 做判定、一个 `compact()` 执行压缩；至于每轮结束后到底调不调，是上层应用决定的。压缩产生的摘要本身也作为一个节点追加进会话树，之后取上下文时用它替换被压缩的那段历史。

Q3：你的 Agent 项目架构是什么？从 TUI 层讲到整体。

答：整个项目是一个 monorepo，分四个包，依赖是单向的，从上到下。

最上面是 TUI 层，一个自研的终端 UI 组件库，负责画界面。它的特点是差分渲染，每次只重画变化的部分，所以终端不闪烁。但 TUI 不直接调内核，它是通过订阅事件来工作的。

中间是产品本体，也就是面向编码场景的 CLI，用户敲的命令入口就是它。它自己几乎不造轮子，而是把另外几个包装起来用：要画界面用 TUI，要智能用 agent core，要连大模型用最底层的 AI 包。它还负责加载扩展、管理工具、决定运行模式。

再往下是 agent core，整个项目的中枢，里面是 agent loop、上下文组装、会话存储、压缩。关键点是它完全不碰终端——它只跑逻辑、广播事件，谁来渲染它不管。正因为这样，它能脱离 TUI 被复用：除了交互界面，还能用一次性打印模式（打印完就退），或者 RPC 模式给别的程序当后端。

最底层是 AI 包，多供应商 LLM 抽象，把 OpenAI、Anthropic、Google 这些不同的 API 统一成一个接口。

所以从 TUI 往下看，本质是一条"事件驱动 + 单向依赖"的链路：内核在中间跑循环、发事件，UI 在外层订阅事件、画界面，两者解耦，这也是为什么同一套内核能配不同的前端。

Q4：你的项目的 Agent 架构是什么？ReAct 是如何实现的？

答：核心就是一个手写的 agent loop，本质是标准的 ReAct 循环：推理（调模型）和行动（调工具）交替进行。

一轮的主干是这样：先组装上下文，检查要不要压缩，然后把上下文发给模型；模型流式返回后，我看它这次有没有发起工具调用。如果有，就执行这些工具，把结果作为新消息追加回上下文，再进入下一轮让模型接着看结果；如果没有工具调用，说明它要收尾了，就输出最终回复，循环结束。读写代码、跑命令这些真正的"行动"，都发生在工具这一步。

实际实现里比这几句话要多考虑几件事，这也是我没直接用现成 SDK、而选择手写的原因——为了完全掌控这些细节：一是工具可以并行或串行执行，有些必须串行的工具会触发串行模式，避免抢同一份状态；二是支持运行中插话（steering），用户在模型

干活时打字，我会在下一次模型响应前把这条消息插进去；三是支持收尾追问（follow-up），模型本来要停了，如果有排队的新消息会继续接着跑；四是随时能中断（abort）。这些逻辑集中在一个主循环函数里。

简单说，ReAct 在我这里不是一个框架帮我做的黑盒，而是一个我能看到每一步的 while 循环：模型想 → 调工具做 → 把结果还给模型 → 再想，直到它不再需要工具。

Q5：你的项目如何实现节点回退和会话恢复？存了什么，又如何恢复？

答：这两个其实是同一套存储机制带出来的，关键在于会话不是存成一个线性列表，而是存成一棵树。

先说存什么。每次产生一条东西——用户输入、模型回复、工具结果、压缩摘要——我都立刻往一个 JSONL 文件追加一行，只追加、不修改、不删除。每条记录都带自己的 id 和一个父指针 parentId，指向它的上一条。靠这个父指针，一个扁平的文件就能还原成一棵树。

会话恢复就是：重新打开时把文件一行行读出来，按 parentId 把树重建出来，然后从当前叶子节点沿父指针一路回溯到根，取出这条路径，就是要喂给模型的上下文。因为是 append-only，历史永远在，恢复不会丢东西。

节点回退靠的是同一棵树。我想退回某个历史节点重新提问，新消息的父指针就指向那个旧节点，于是从那里岔出一条新分支，老分支还原封不动留着。fork、导航、克隆都是基于这种指针关系做的，不需要复制整段历史。

最后补一个我会主动说的点：默认情况下回退只回退对话，不会动工作区里的代码文件——因为内核只管会话树，不管文件快照。如果想让回退连代码也一起还原，需要用扩展接 git 来补，比如每轮用 git stash 打个快照、回退时再恢复。这是我刻意保留给扩展的，内核保持精简。

Q6：你设计这个项目的整体思路是什么？

答：一句话概括，我的设计理念是"极简内核 + 一切皆可扩展"。我不去猜用户需要什么，也不把功能一股脑塞进内核，而是先把内核做到最小，剩下的能力都通过扩展机制从外面接上。

先说内核里留了什么。我只保留一个 Agent 跑起来必须有的东西：agent loop（ReAct 主循环）、上下文组装、会话存储、压缩、多供应商的模型抽象，以及最基本的几个工具——读、写、改、执行命令。除此之外，像权限弹窗、MCP、plan 模式、子代理、联网搜索这些，内核里一概没有。好处是内核小、容易读、容易测，攻击面也小；代价是开箱体验会显得空，但这是我故意的取舍。

再说可扩展怎么做，这是关键。我让 agent loop 在运行的每个阶段都广播事件——agent 开始、每轮开始结束、消息流、工具开始执行和执行完等等，一共三十多种。扩展就建立在这套事件广播之上，具体有两个层次。

一个是扩展（extension），本质就是一个 TypeScript 模块，拿到我的 API 对象后可以往里加东西：用 registerTool 加工具、registerCommand 加命令、registerProvider 接自定义模型供应商、注册自定义渲染器改界面，或者订阅事件做联动。另一个是 hooks，它比订阅事件更强，能拦截并改写数据：比如在工具执行前用 tool_call 拦下来做权限确认、在调模型前用 context 改写要发送的消息、在发请求给供应商前替换 payload。

把这两套机制摊开看，内核的每一个部分其实都开了扩展口子：要加能力就注册工具，要换模型就注册供应商，要改提示词就挂 context 或系统提示的 hook，要改压缩就自定义 compaction，要改界面就注册渲染器和页脚页眉，要管权限就挂 tool_call。所以那些"内核没有"的功能——权限、plan、MCP——都能用几十行扩展补回来，而且扩展本身也可以让 Agent 自己写。

总结一下：主线是把内核做小、只做必要的事；可扩展则是靠"事件广播 + 扩展注册 + hooks 拦截"这三件套，让模型调用、工具、上下文、压缩、界面、会话每一个环节都能在不改内核代码的前提下被替换或增强。

字节一面真题：介绍Pi Agent

真题来源：[2026 年 10 月字节一面](#doxcnz9YY5maMQl28LqNPcrD6Wg) · [第 4 组原题](#doxcn2Wh1SqMt9Xc8NTLTLttnld)

1. 展开介绍一下你了解的 DeepSeek Harness 和 Pi Agent。

🎤 参考答案

再说 Pi Agent。 Pi 更强调“小内核加扩展”：先保留一个小而清晰的 Agent 主干，默认提供 read、bash、edit、write 四个工具，其他能力按需补充。它通过多供应商模型抽象连接不同模型，再由 Agent Loop 完成上下文组装、模型调用、工具执行和结果回灌；终端交互层与运行能力分开，也可以通过 RPC 或 SDK 被其他程序使用。

Pi 虽然默认能力克制，但会话和上下文管理并不缺失。它使用只追加的 JSONL 存储会话，通过条目的 parentId 组织会话树。模型使用的不是整棵树，而是当前节点到根的那条路径，所以回到历史位置继续对话时可以形成新分支，旧分支仍然保留。上下文过长时，再通过 compaction 将较早历史压缩为摘要，结合保留的近期内容继续执行。

扩展方面，Pi 通过 TypeScript Extensions、事件和 Hooks 注册工具、添加命令、拦截工具调用或修改上下文。例如，权限确认可以通过扩展接入，而不是必须写进内核。

2. DeepSeek Harness 和 Pi 有什么区别？

完整对比与项目结合思路见：[DSH与Pi的真题解析](#doxcniMaSckzC4ZxqotAWLd9OmL)。

5.7.2 选型与二次开发

建议先阅读：[二次开发入门](#doxcnZ889g2QCnj1NdEAuMwDseg)；项目接入方案与架构图见 [ForgePilot：Pi 核心层设计](#doxcnE03AFNbVZ57BdcXSl5UTjh)。

❓ 1. 聊聊Pi-agent 和 DSH的区别？

选自：[202610 字节求职Agent开发一面](#doxcnNDxGIMOoZsQKmZCGX67U3c)

💡 答题思路

可以按照三个层次来回答：第一段谈二者设计理念的区别，第二段谈核心功能点的差异，第三段谈实际使用时自己的选型策略。

🎤 参考答案

我理解 Pi 和 DSH 都是组织模型、工具和执行流程的 Agent Harness。设计理念上，**Pi 强调精简核心**，提供基础运行能力，把规划、子 Agent 等工作流留给扩展。**DSH 强调“一切皆插件”**，基于 Cordis 组织系统，模型适配、工具、会话和 Agent Loop 都可以通过插件配置替换，同时提供较完整的默认功能组合。两者都能深度定制，Pi 侧重保持核心精简，DSH 侧重让整套系统的组成部分都可组合、可替换。

核心特性上，Pi 的**终端交互和会话探索**比较有特色，可以通过会话树回到不同节点、分支尝试，再用扩展补充需要的工作

流。DSH 提供 Web 和桌面界面，标准使用流程中已经包含**任务规划、子任务委派和操作审批**，并提供可配置的沙箱机制，方便直接使用这些配套能力。它也有命令行和无界面运行入口。模型方面，**两者都支持多模型接入**。

实际选择时，如果我习惯**终端交互**，喜欢利用**会话分支**探索不同方案，希望工作流从精简配置开始逐步补充，我会先试 Pi。如果我更希望通过**图形界面管理任务**，直接使用**规划、委派和权限策略**这套已有功能组合，我会先试 DSH。

> 依据：本地对比视频转写用于提取比较角度；具体能力已于 2026 年 11 月 3 日对照双方官方文档核对。参见 [Pi 官方说明](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/README.md)、[DSH 架构](https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md)、[DSH 模型配置](https://deepseek-harness.github.io/deepseek-harness/en/guide/providers) 与 [沙箱说明](https://deepseek-harness.github.io/deepseek-harness/en/reference/subsystems/sandbox)。这里比较的是官方文档描述的设计与使用方式，未做两者的安装实测或性能测评。

❓ 2. 你的项目是如何接入pi-agent的？ 选自：202610 拓竹科技Agent开发一面

项目架构与选型依据：[Pi 核心层设计](#doxcnE03AFNbVZ57BdcXSl5UTjh)。

💡 答题思路

重点是答出来接入的pi agent的层级，以及为什么理由。说清楚自己的项目关注哪些地方，所以这一部分需要定制化，哪些模块又是不用修改的。做到该复用的地方复用，不该复用的地方重写。

🎤 参考答案

我们项目的接入方案是**通过 SDK 使用 `pi-agent-core`**，也就是 Pi 的 Agent 核心层，同时配合 `pi-ai` 完成模型通信。选择这一层，主要是根据项目**需要定制的能力和可以直接复用的能力**来划分。

ForgePilot 的目标是让 Agent 在企业环境中持续完成分析、修复、测试和验证，最终提交可审核的 PR。我们重点关注长任务能否稳定推进、关键证据能否保留，以及交付结果是否满足企业要求。

因此，上层策略需要围绕这些目标自己设计。**上下文和记忆方面**，我们负责历史压缩、长期记忆和会话保存，让任务持续保留必要的信息；**执行管理方面**，我们负责任务编排、进度保存、权限审批和预算控制，并接入受控的检索、文件操作和测试工具；**交付方面**，我们组织测试证据、人工审核和星图评审界面。这些能力直接影响任务效果和使用体验，需要按企业流程定制。

底层的模型通信和执行循环则可以直接复用。`pi-ai` 负责模型请求和响应适配；`pi-agent-core` 负责维护运行中的消息和状态、调度已注册工具、回填结果，再继续下一轮推理。这些通用机制符合我们的需要，可以保留原有实现，具体的业务工具和执行策略由我们提供。

所以 Core 层比较符合我们的接入需求：它覆盖了我们希望复用的通用运行能力，同时把会话持久化、压缩策略和业务交互等需要定制的部分留给应用层。这样开发精力就能集中在影响企业任务成功率和交付质量的地方，**定制主要围绕 Core 的配置、工具接入和外层管理展开**。

❓ 3. 为什么现在都喜欢在pi-agent上进行二次开发？你是怎么理解的？

选自：202610 美的Agent开发一面

💡 答题思路

笔记的开头专门有讲pi-agent适合二次开发的原因，直接照着回答就行。这一内容既可以回答这个问题，也可以作为一些问pi-agent的设计理念类似问题的回答内容。

🎤 参考答案

我认为 Pi 吸引开发者做二次开发，主要是因为它在组件拆分、扩展接口和整体设计上，都给业务定制留出了比较大的空间。具体有四点。

第一，**接入力度更细**。它把模型通信、Agent 核心循环和现成 Agent 分成了独立的包。我们可以用 `pi-ai` 接模型，用 `pi-agent-core` 复用执行循环，也可以通过 `pi-coding-agent` 的 SDK 接入完整会话。这样，已有系统具备哪些能力、还缺哪部分，就能决定从哪一层接入，复用边界比较清楚。

第二，**扩展机制兼顾介入深度和执行效率**。Pi 开放了工具调用、上下文处理等运行节点，可以通过 TypeScript 扩展检查操作、调整行为和记录日志。虽然 Claude Code 也有Hook机制，但存在介入有限、反复启动进程的问题。Pi可介入深度和性能都更好。

第三，**框架比较轻，改造负担小**。框架预设的工作流越多，业务接入时就越需要判断哪些要保留、哪些要关闭，以及修改后会影响哪些地方。Pi 保持核心精简，减少了这些前置约束，开发者可以把精力更多放在自己的业务逻辑上。

第四，**机制方便后续增删**。作者强调“build to delete”，因为模型能力提升后，今天为了弥补模型不足而加入的逻辑，将来可能失去价值。保持机制精简、减少耦合，可以给后续替换和删除留出空间，降低旧逻辑逐渐变成维护负担的风险这种设计理念本身就更适合做二次开发。

5.8 配套讲解视频

第一期：架构与核心机制

讲解内容：Pi 的[整体设计](#doxcnfE750p6U6GJCZcp9KehIDK)、[仓库结构](#doxcnqwh7N8g843K1e8Ua6vvTDd)、[Agent Loop](#doxcnhHCUopZDz7yhIJBjEyKeNf)、[上下文与会话管理](#doxcn25UaJsLSKT0MdLyr8vygRe)、[工具调用](#doxcntviGUon2aApYyofeOpzxXf)和[扩展机制](#doxcnPvYqO7wgNVGnMywhnVjaMg)；配套[基础与架构练习](#doxcneDyNRvtSajgIDFExzLYuch)、[基础与架构面试问题](#doxcnvmN2eaQjkKQgZ7XZmrFYOb)。

https://www.bilibili.com/video/BV1Bijf66Efs/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

第二期：二次开发入门

讲解内容：[二次开发的设计优势](#doxcnzYh6wFq9Ib1GOdcrWei4se)、[SDK 与 Extensions 接入方式](#doxcnsi86X4UVniBVnYaAQlvYrg)，以及 [ForgePilot 的接入层级、职责划分和架构设计](#doxcnE03AFNbVZ57BdcXSl5UTjh)；配套[选型与二次开发面试问题](#doxcnXjIlN30dRhYUesNMwcibRb)。

视频链接待补充。

5.9 参考资料与源码

官方文档与源码

- 官网与文档：pi.dev 及其[源代码](https://github.com/earendil-works/pi)。

架构学习资料

主要参考了两个Youtube视频和源代码。参考视频不是说大家必须看，只是列出具体来源。

PS：YT视频需要科学上网。英文不怕，下方可以翻译成中文，现在在线翻译的还是比较准的

- [PI Agent Crash Course](https://www.youtube.com/watch?v=N30XGyPrr6I&t=74s)
- [PI Archectrue explained](https://www.youtube.com/watch?v=gTeujlv8qK0&t=1672s)

二次开发资料

模型通信与工具协议参见 [Pi AI](https://github.com/earendil-works/pi/blob/main/packages/ai/README.md)；工具执行循环参见 [Agent Core](https://github.com/earendil-works/pi/blob/main/packages/agent/README.md)；会话集成与插件机制参见 [Coding Agent SDK](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/sdk.md) 和 [Extensions](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/extensions.md)。

---

6 开源框架解读 - Openclaw

📝 2026年10月补充更正丨当前学习建议

根据目前求职市场的面试反馈，OpenClaw 和 Hermes Agent 的考察已比较少；Claude Code 仍会被问到，Pi-Agent 的热度和考察频率有所上升。DeepSeek Harness 和 Codex 作为近期开放源码的架构，也同样值得关注。

因此，建议优先学习 Claude Code、Pi-Agent、DeepSeek Harness 和 Codex，有余力或感兴趣时，再学习本章节。正文中早期关于“必考”“必须掌握”等表述，请结合当时的撰写时间理解，当前学习优先级以本声明为参考。

下方为早期撰写的原文，保留供学习参考。其中“必考”“必须掌握”等判断反映的是当时的情况，当前学习优先级请以上述更正为准。

📖 历史原文丨写于2026年3月

OpenClaw一定是一个必考的问题，大模型应用岗位，产品岗，必须得会。原因很简单，我想先和大家讲清楚这里面的底层逻辑，

为什么会考。 因为它是目前最热门的AI Agent产品了。在这个大模型高速发展的时代，热门的东西一定是要了解会考察的。有人说2025年是Agent年，从2025发展到现在，Agent的形式在不断演进。火爆出圈的Agent从去年的Manus到今年的Openclaw，它是整个行业发展的风向标。 我回想去年（2025年）面试的时候，很多面试官问我Manus是什么，我说不知道，当时觉得没啥，现在觉得很掉分。对于行业爆品，一定要掌握，是高频考点。网上很多视频基本都是讲的Openclaw如何配置，有什么能力。相关视频太多了，在本章我会给大家推荐2个我自己看的部署，玩openclaw的视频，感兴趣可以去玩。 但是本章更侧重的是Openclaw的原理，技术，为什么出圈。讲的人不多，但是作为开发，或者产品，从面试的角度，你无疑是要掌握这个产品里面包含的核心技术的，本节就从功能，技术层面来讲解Openclaw，相信它一定是你准备面试必不可少的素材。学了本章，你可以在面试的时候和面试官说你看过Openclaw的源码（其实也有必要看，作为Agent开源项目），Manus其实也可以去学习，因为它也是很火的Agent，而且和Openclaw 侧重点是不同的。可以当作课外作业大家自己去学一下。

6.1 OpenClaw是什么

（一定会问的问题，一般面试官引入一个问题都是先问你概念，所以精心准备一个表达流畅的回答是非常必要的）

OpenClaw 是一个开源的、可自托管的**个人 AI 助手平台**。

- 核心用 TypeScript (ESM) 编写，运行在 Node.js 22+ 上
- 通过 `npm install -g openclaw` 安装，`openclaw` CLI 是主要入口
- 所有数据和密钥都存在本地（`~/.openclaw/`），不经过任何云中转

总结来说，**它是一个事件驱动的 Agent 执行引擎，前面放了一个多渠道网关。**

6.2 Openclaw 总体架构

（这节很核心哈，我结合这个架构图，然后把各个模块功能都讲清楚，你学明白了其实你就知道了它到底是个啥东西，为啥看上去很神奇很主动？Openclaw神秘的面纱就揭开了）

简单来讲，OpenClaw 的架构为**触发、网关、Agent 三层**：

- **触发层**负责产生事件 — 五种来源（消息、心跳、定时任务、钩子、Webhook）不断向系统注入事件
- **网关层**负责接入和路由 — 把不同协议的事件统一收进来，认证后分发到正确的 Agent 会话
- **Agent 层**负责智能执行 — 推理（Think）、调工具（Act）、记忆（Remember）

三层的本质是一个**事件驱动的执行管道**：事件产生 → 网关路由 → Agent 消费执行 → 状态持久化 → 等待下一个事件。这是一个永不停止的循环。

6.2.1 触发层 - 5层信号源

Agent 不是只在用户发消息时才工作。OpenClaw 有**五种独立的触发维度**，每一种都能唤醒 Agent 执行：

通过上述图片，我们知道它触发的方式很多，我们通常给他发消息其实只是一种（Message）。不过Message可以触发的方式也很多：通过CLI，通过应用（feishu,telegram），包括Openclaw的dashboard。 所以其实看上去openclaw好像一直在思考，很主动，其实不过是有记忆功能加上 自动的触发源罢了。（最后我会举例说明）

```
![](images/NkIebSerkoAfDfxzjcTcJONhn5e.png)



<a id="doxcniD1lAulgngBGZlKOOY0Zqg"></a>
#### **6.2.2 网关 - 大名鼎鼎的Gateway**

Gateway 是一个**本地长驻 Node.js 进程**，监听端口 `:18789`（HTTP + WebSocket），是触发层和 Agent 层之间的**中间件**。

![](images/XbLCbu3YWond6bxviUhcQgEFnGD.png)

**Gateway 的核心职责：连接管理、协议转换、路由分发、安全控制。**

**关键边界：Gateway 不做推理，不调 LLM，不执行工具。** 它只负责"消息到了该给谁"和"回复该送回哪里"。

<a id="doxcnPBUFp9VbwZQLwUYpgyBDvz"></a>
#### 6.2.3 Agent层

这层就是咱们熟悉的Agent的知识啦。React, LLM 的调用，Function Call, Memory都在这里啦\~都是我们学过的知识啦\~\~。

##### 5.2.3.1 推理 (think)

Agent 的核心是 **Pi Embedded Agent**，一个多轮推理循环。它收到消息后开始思考，决定是直接回复还是需要调用工具获取更多信息，调完工具拿到结果后继续思考，如此循环直到生成最终回复。

推理过程中需要调用 LLM。LLM 调用是从本地 Gateway 进程直接发往模型提供商 API 的（OpenAI、Anthropic、Gemini、Ollama 等），不经过 OpenClaw 的任何云服务。如果用 Ollama 等本地模型，整个流程甚至完全不出你的局域网。

**Auth Profiles** 管理多套 API 凭证，支持轮换和 failover——一个 key 被限速了会自动切到下一个。**Session Manager** 维护多轮对话的上下文，当历史过长时自动压缩（compact），管理 token 预算。

##### 5.2.3.2 工具执行 (Act)

Agent 推理时可以自主决定调用工具。内置的核心工具包括：

- **exec** - 执行 Shell 命令，有 allowlist 安全策略控制哪些命令可以运行
- **browser** - 通过 CDP 协议控制 Chromium 浏览器，可以浏览网页、截图、操作页面
- **message** - 向任意渠道发送消息，实现跨渠道通信
- **nodes** - 远程调用已配对的 iOS/Android 设备上的能力
- **subagents** - 启动子 Agent 并行处理复杂任务
- **apply_patch** - 修改代码文件

除了内置工具，插件也可以通过 `registerTool` 注册额外的工具，能力可以无限扩展。

##### 5.2.3.3 记忆(Memory)

记忆是 Agent 主动调用的，不是每次对话都自动塞入上下文（那样会浪费 token）。Agent 自己判断"需不需要回忆过去的信息"，然后调用 `memory_search` 工具进行混合检索——70% 权重给向量语义搜索，30% 权重给 BM25 关键词搜索（**哇，这不是我们自己的RAG项目的稀疏向量和稠密向量双路检索吗\~**）。找到相关结果后，再通过 `memory_get` 读取完整的记忆内容。

底层存储是一个 SQLite 数据库，同时维护向量 embedding 索引和全文索引。记忆的原始数据是 Markdown 文件
```

（MEMORY.md 以及自动生成的会话摘要文件）。

记忆的写入有两种方式。第一种是自动捕获：当会话结束时（用户输入 `/new` 或 `/reset`），hook 会自动把最近的对话保存为 `memory/YYYY-MM-DD-slug.md` 文件。第二种是文件同步：用户手动编辑 MEMORY.md 或 memory 目录下的文件，chokidar 文件监听器检测到变化后，自动进行分块、计算 embedding、写入索引库。

6.2.4 总结&举例

朋友们，所以你看，Openclaw其实也没有什么特别大不了的对吗？比如Agent模块，我们的描述并不复杂，没有用什么深奥的技术。如果你看完了我这个笔记Agent，我相信你很容易理解，所以OpenClaw本身也没啥大不了的对吧。我在这一节再给大家用实际例子来讲解，其实它就是这么回事\~.

先总结，OpenClaw的本质是**事件驱动模型**，看起来主动是有定时任务以及间隔触发，看起来能记事是因为有记忆，采用双路检索。看上去功能强大，可以用应用驱动是因为用了Gateway做适配。快速总结：

```
Time    → Heartbeats, Crons

Humans   → Messages

External   → Webhooks

Internal   → Hooks → Queue → Agent Executes → State Persists

Agents     → Subagents
```

再举例：一个外国小哥哥使用了Openclaw，然后发帖说他给他的妻子用Openclaw设置了定时发送早上好，结果晚上回去，他发现他的龙虾和妻子从早聊到晚，他的妻子完全不知道对面不是真人。他觉得太amazing了。我们现在知道，他看上去主动的原因是，因为你设置了定时器，然后还有间隔触发。Agent每次触发，会根据上下文，思考调用工具比如可以打电话，发新闻，图片。所以整体看上去你没和它主动聊天，他也会和你说话甚至会打电话。无非是这么实现的。

```
![](images/WxQVb4qpNokZLDxr96ocMBmTnwh.png)

<a id="doxcneSe8vxEU1oLro5OlkIWzsh"></a>
### 6.3 Openclaw 记忆

<a id="doxcnpnqFnR6dUsZZujmwp0ymVe"></a>
#### 6.3.1 导语
```

为什么这一小节我认为很重要，给大家先说明原因：

1. **首先记忆功能是常考的，甚至必考**，从面试的角度，Agent的记忆是如何实现的，这已经是背烂了的八股对吧，这部分就是这个问题的经典实现，所以你学习这一部分是对这个高频考点的补充，我会将Openclaw的记忆功能融入笔记八股部分，更新我们的回答，让我们的回答结合现在最火的Agent的例子去答，是不是就比干巴巴答概念要更好。

2. 如果你想玩openclaw的话，**Openclaw为什么总是忘记事情？**如果你学了本节，那你自己就能够尝试去解决，你也可以修改里面的记忆文件，手动确定让你的龙虾能记住什么，高效组织你的龙虾记忆。

3. 我想借助这个机会，学习Openclaw，大家可以理解这个产品的架构是怎么设计的，记忆模块是怎么组织的。这样的好处是**建立你的审美**，大家在设计自己的agent的时候，如果你去设计架构，你用上他这个**心跳机制，定时机制** 而不是干巴巴的只有一个用户输入作为触发，你用上它的4层记忆，哪怕你不改，你就去抄它的，是不是也比自己设计的高级一些，所以通过这个学习，也是为我们自己设计Agent项目打基础，站在他们的肩膀上设计你的项目，让面试官觉得你的项目是高级的，有想法，有深度的。

6.3.2 四层记忆架构总览

OpenClaw 的记忆不是一个系统，而是 **四层独立机制** 协同工作，类似计算机的存储层次：

1. **Bootstrap Files**（≈ 硬盘）– Agent 的永久身份与规则，完全免疫 Compaction
2. **Session Transcript**（≈ 文件系统日志）– 完整对话历史存档，旧消息会被压缩为摘要
3. **Context Window**（≈ RAM 内存）– 模型当前能"看到"的所有内容，满了就触发压缩
4. **Retrieval Index**（≈ 搜索引擎）– 语义搜索记忆文件，独立于会话

> **知道哪一层出了问题，就能解决 90% 的"Agent 忘事"问题。**

6.3.3 Bootstrap Files（引导文件）

本质：工作空间里的 Markdown 文件，每次会话开始时从磁盘读取、注入上下文。

文件清单

`SOUL.md` – 人格、语气、边界

`AGENTS.md` – 操作指令、行为规则

`USER.md` – 用户身份信息

`TOOLS.md` – 工具使用说明

`IDENTITY.md` – Agent 名称与风格

`MEMORY.md` – 策展的长期记忆（仅主会话加载）

`HEARTBEAT.md` – 心跳任务清单（可选）

`BOOTSTRAP.md` – 首次运行仪式（一次性）

位置：`~/.openclaw/workspace/`

关键要点

每次会话重新从磁盘读取，不存在对话历史中 → 免疫 Compaction

字符限制：单文件 20,000 字符，总计 150,000 字符，超出会被截断

子 Agent 只收到 `AGENTS.md`、`SOUL.md`、`TOOLS.md`、`IDENTITY.md`、`USER.md`，拿不到 `MEMORY.md` 和 `HEARTBEAT.md`

用 `/context list` 可查看每个文件的注入状态

所有 Bootstrap Files 都应尽量精简——它们每轮都注入上下文，内容越多占用越大。实践中 `SOUL.md` 最容易被写得过长，建议各文件都保持简洁。

6.3.4 Session Transcript（会话记录）

（我觉得这节还是挺重要的，他其实是你怎么组织Agent长短记忆的一些原理。我们做agent的时候都知道把历史对话记录每次拼接，放到下一轮对话。那么窗口满了，如何压缩，压缩保留什么东西，这时要把什么东西存到本地做长期记忆，这些技巧在本节都讲了，面试也是爱问的，建议掌握，要是能面试时候说出来，或者用到你的项目中，都会大大加分）

本质：完整对话历史，以 JSONL 文件追加写入磁盘。

存储位置

```Plain Text
~/.openclaw/agents/<agentId>/sessions/
├── sessions.json           # 会话元数据
└── <sessionId>.jsonl       # 对话记录（包含消息、工具调用、压缩摘要）
```

关键要点

- **每条消息自动追加到 JSONL**，包括用户消息、助手回复、工具调用
- 继续会话时从 JSONL 重建上下文，**跨重启持久**
- 磁盘上的原始记录**始终保留**，但模型只看到上下文窗口中的部分

Compaction（压缩）详解

两种触发方式：

- **自动触发（Auto-Compaction）**：上下文使用量达到阈值时自动触发。有两种情况：
- **溢出恢复**：模型返回 context overflow 错误 → 压缩 → 重试
- **阈值维护**：成功完成一轮后，`contextTokens > contextWindow - reserveTokens` 时触发
- **手动触发**：输入 `/compact`，可带自定义指令如 `/compact 重点保留决策和待办事项`

触发公式：`上下文窗口 - reserveTokensFloor(默认20000) - softThreshold(默认4000)`，200K 模型 → 约 176K 时触发。

压缩的具体过程：

1. **划分保留区和压缩区**：最近的 \~20,000 tokens 消息被划为"保留区"，**原封不动保留**，模型看到的就是原文。保留区之前的所有旧消息进入"压缩区"
2. **分割压缩区**：把压缩区的旧消息按 token 数分成若干块（默认分 2 块，自适应调整块大小）
3. **逐块总结**：用模型对每个块生成摘要（调用 `generateSummary`，失败会重试 3 次）
4. **合并摘要**：如果有多个块的摘要，再用模型把它们合并成一个连贯的总摘要
5. **写入 Transcript**：在 JSONL 文件中写入一条 `compaction` 类型的条目，包含摘要文本和 `firstKeptEntryId`（保留区最早消息的 ID）

压缩后模型看到的上下文结构：`[压缩摘要] + [保留区原文（最近 ~20,000 tokens）]`

什么时候触发

总结过程有明确的指令，**必须保留**：

- 进行中的任务及其状态（进行中、阻塞、待办）
- 批量操作的进度（如"17 项中已完成 5 项"）
- 用户最后请求的内容以及正在做什么
- 已做出的决策及其理由
- TODO、开放问题和约束条件
- 承诺的后续行动

默认还会**完整保留所有不透明标识符**（UUID、哈希、ID、token、API key、主机名、IP、端口、URL、文件名），不缩写不重构。这由 `identifierPolicy: "strict"` 控制。

Pre-Compaction Memory Flush

压缩前 OpenClaw 会静默运行一轮，提醒 Agent 把重要内容写入 `memory/YYYY-MM-DD.md`，用户不可见（`NO_REPLY`）。每个压缩周期只触发一次。

> ⚠️ **聊天中说的指令不会自动保存到文件**——不写入文件就会被压缩丢掉。这是最常见的"Agent 失忆"原因。

6.3.5 Context Window（上下文窗口）

本质：固定大小的 token 容器，大小由模型决定（如 Claude 200K、GPT-4 128K、Gemini Pro 1M），模型每轮推理时所有信息都要挤进来。

关键要点

- **每轮竞争空间的内容**：系统提示 + Bootstrap Files + 对话历史 + 工具调用结果（包括 `memory_search` / `memory_get` 检索到的记忆片段） + 当前消息
- **最大的 token 消耗者是工具结果**（文件读取、检索结果、网页快照、API 响应）
- Bootstrap Files 每轮都占空间，所以要尽量精简
- **满了就触发 Compaction**，旧对话被压缩
- 用 `/status` 监控使用量，建议在 75–80% 时手动压缩

6.3.6 Retrieval Index（检索索引）

本质：为记忆文件建立的搜索索引，Agent 通过工具按需检索，不用把所有记忆都塞进上下文。

索引范围

- `MEMORY.md` + `memory/**/*.md`（默认）
- 可通过 `memorySearch.extraPaths` 添加额外路径（如 Obsidian 笔记库）
- 存储在 `~/.openclaw/memory/<agentId>.sqlite`

检索工具与触发方式

OpenClaw 提供两个记忆工具：

- `memory_search` — 语义搜索，返回相关片段（文件路径 + 行号 + 分数）
- `memory_get` — 按路径读取指定记忆文件的具体内容

检索不是每次对话自动触发的。`memory_search` 是一个工具，由模型自己判断是否需要调用。系统提示中会指导模型"在回答关于过去的决策、日期、人物、偏好、待办等问题前，先调用 `memory_search`"，但最终是模型自行决定何时搜索、用什么关键词搜索。简单闲聊不会触发检索。

工作流是两步走：先 `memory_search` 找到相关片段 → 再 `memory_get` 读取完整上下文。

混合搜索

同时使用向量语义匹配（擅长同义词）+ BM25 关键词匹配（擅长精确 ID/代码符号），加权合并：

```Plain Text finalScore = 0.7 × vectorScore + 0.3 × textScore
```

> 关键要点
>
> - **只索引文件**——聊天中说过但没写入文件的内容搜不到
> - 文件变动后自动重新索引（去抖 1.5 秒）
> - 支持 MMR 去重（过滤近似重复结果）和时间衰减（默认半衰期 30 天，新笔记排名更高）
> - 常青文件（`MEMORY.md`、非日期命名的 `memory/*.md`）不受时间衰减影响
> - **反复引用的重要知识**建议直接放在 `memory/` 目录下按主题组织（如 `memory/trading-system/`），而不是只存在外部
>
>
> ### **6.4 OpenClaw Agent 架构解析**
>
> 在10.2 小节中，我们分析了 OpenClaw 的整体架构，它由三层构成：
>
> ```Plain Text
> 触发层 → 消息渠道（Discord、Telegram、Slack、WhatsApp 等）
> 网关层 → Gateway（路由、session 管理、并发控制）
> Agent 层 → Agent（LLM 驱动，执行任务）

本文聚焦 **Agent 层**的内部架构。Agent的架构其实是面试官非常喜欢问的高频考点，我们在笔记中有了很详细的总结。要说我们之前学的是理论，那么我希望通过这一章来从实践上看看，Openclaw使用的Agent架构和我们之前的理论知识是否吻合。学习这一小节，除了从真正工程的角度加强对这个经典面试题的理解，另一方面其实也体现在自己的架构设计上，我们在设计自己的Agent的特性和架构上，能不能参考Openclaw的架构，把我们的项目做得更高级。

在我们笔记的 Multi-Agent 典型架构章节中，讲过常见的架构模式：**SAS、Independent、Centralized、Decentralized、Hybrid**，这里简单回顾一下：

- **SAS（单智能体）**：只有一个 Agent 独立完成所有任务

- **Independent（独立型）**：多个 Agent 并行运行，互不通信，各自完成独立任务

- **Centralized（中心化）**：一个中心 Agent 负责编排，将子任务分发给 Worker Agent 执行

- **Decentralized（去中心化）**：Agent 之间 P2P 直接通信协作，无固定中心

- **Hybrid（混合型）**：以上模式的组合

今天我们将理论变成实践，解析 OpenClaw 这个真实项目中到底使用了什么架构，看看和我们之前讲的理论能不能匹配得上。

6.4.1 概述

OpenClaw 的 Agent 框架本质上是一个 **递归 Centralized架构**：

- Parent Agent 作为中心，用 LLM 决策任务如何拆解，向下 spawn 子 Agent 执行
- 子 Agent 可以继续 spawn 孙 Agent，每一层都是该层的中心，形成一棵递归的 Orchestrator-Worker 树

- 路由和调度由无智能的 **Gateway**承担，不是 Agent。

Agent框架图

6.4.2 Openclaw Agent架构详解

OpenClaw 的多 Agent 能力没有专门的"调度引擎"或"编排框架"，而是完全基于 LLM 的标准 **Tool Use 机制**实现的：

- **工具（Tool）**：`sessions_spawn` 被注册为一个普通工具，和 `exec`、`web_search` 地位相同，调用它就会在系统层创建并启动一个子 Agent

- **提示词（Prompt）**：系统提示词注入一句触发规则，告诉 LLM 什么情况下应该使用这个工具

- **LLM 决策**：LLM 在每轮推理中自主判断是直接处理任务，还是调用 `sessions_spawn` 把任务委托给子 Agent

这意味着 OpenClaw 的多 Agent 编排能力，是从 LLM 的 tool call 能力"衍生"出来的，而不是独立构建的一套系统。架构简洁，扩展自然。

6.4.2.1 为什么选 Centralized

Openclaw的作者并没有说他自己为什么这么设计架构，不过我们之前笔记有讲不同的架构的抉择，我谈一谈我的理解，不过对于模型架构的选择，其实你很难说哪个是最好的，这没有固定答案，我觉得这一点就是你能够想清楚，面试能讲清楚，能自洽就行。

用户通过消息渠道发来一条消息，最终必须有一条连贯的回复发回去。这个"一进一出"的结构，天然要求有且只有一个 Agent 对最终回复负责——谁收到消息、谁整合结果、谁发出回复，必须是同一个主体。

Centralized 的父子结构正好对应这个模型：Parent Agent 是唯一的回复责任人，子 Agent 只是它用来扩展执行能力的手段。

如果换成 Decentralized，多个 Agent P2P 协商，最终谁来发回复？没有确定的答案。如果换成 Independent，各 Agent 独立运行互不感知，结果根本无法拼合成一条完整的回复。

对话本身的结构就是中心化的——有一个确定的入口，有一个确定的出口，中间的执行可以分发，但责任链不能断。

6.4.2.2 Spawn 的决策机制

维度一：决策规则——谁来判断，依据什么

是否 spawn 子 Agent，完全由 **Parent Agent 的 LLM 自主判断**，没有外部调度器介入。OpenClaw 给 LLM 的判断依据只有系统提示词中的一句话（来源：`src/agents/system-prompt.ts`）：

```Plain Text
If a task is more complex or takes longer, spawn a sub-agent. Completion is push-based: it will auto-announce when done.
```

> 就这一句。没有复杂的拆分规则，"复杂"和"耗时长"的边界完全交给模型自己理解和判断。
>
> **维度二：执行工具——spawn 是怎么实现的**
>
> `sessions_spawn` 是系统注册给 Agent 的一个普通工具。LLM 决定 spawn 后，就像调用任何其他工具一样调用它。
>
> **系统提示词如何把这个工具介绍给 Agent：**
>
> ```Plain Text
> - sessions_spawn: Spawn an isolated sub-agent session
> ```

工具名 + 一句描述，Agent 就知道它的存在和用途了。

工具的调用参数：

```Plain Text
sessions_spawn( task, // 子任务描述（自然语言，直接成为子 Agent 的 prompt） agentId, // 目标 Agent ID（可选） runtime, // "subagent"（轻量）或 "acp"（重量级，支持外部 CLI） mode, // "run"（一次性）或 "session"（持久线程绑定） model, // 模型覆盖（可选） thinking, // 思考强度覆盖（可选） attachments, // 文件附件（可选） )
```

> **两种 runtime：**
>
> `sessions_spawn` 支持两种 runtime，决定子 Agent 以什么方式运行：
>
> \- **subagent**：轻量模式，子 Agent 在同一宿主进程中运行，继承父 Agent 的 workspace 目录，适合大多数常规任务拆分场景。来源：\`src/agents/subagent-spawn.ts\`

```
\- **acp**：重量级模式，基于 OpenClaw 内置的 ACP（Agent Control Protocol）协议，子 Agent 在独立进程中运行，可以对接 Codex 等外部 CLI 编程工具，支持持久化 session 和恢复，适合需要长期运行或内嵌外部工具链的场景。来源：\`src/agents/acp-spawn.ts\`



**ACP 如何开启，LLM 如何选择：**

ACP 由配置项 `acp.enabled` 控制，默认开启，用户可以显式设置 `acp.enabled=false` 关闭。沙箱化 session 中 ACP 会被自动禁用。

开启 ACP 不代表每次 spawn 都会用 ACP。开启只是"解锁了这个选项"，并在系统提示词中额外注入一段触发规则告知 LLM：

```Plain Text
For requests like "do this in codex/claude code/gemini", treat it as ACP harness intent
and call sessions_spawn with runtime: "acp".
```
```

也就是说，LLM 默认 spawn 的仍然是 `subagent`，只有当用户明确表达"用 Codex 做"、"用 Claude Code 做"之类的意图时，LLM 才会判断为 ACP 场景，切换到 `runtime="acp"`。选择哪种 runtime 和是否 spawn 的决策逻辑完全一致——都由 LLM 自主判断，提示词只是提供规则。

6.4.2.3 子Agent限制条件

LLM 发出调用后，系统还会做一系列硬性检查，任一不通过就返回 `forbidden`，LLM 的意图不会被直接执行：

- **深度限制**：当前深度 `>= maxSpawnDepth` 就拒绝，默认 `maxSpawnDepth=1`（即子 Agent 默认不能再 spawn）

- **子 Agent 数量上限**：单个 session 的活跃子 Agent 数 `>= maxChildrenPerAgent` 就拒绝，默认 `maxChildrenPerAgent=5`

- **Agent ID 白名单**：若配置了 `allowAgents`，只有列出的 agentId 可被 spawn

更底层的封锁是：叶子节点（leaf）的 Agent 工具列表里根本没有 `sessions_spawn`，LLM 物理上无法调用它（来源：`src/agents/pi-tools.policy.ts`）：

```Plain Text 深度 0（main） → 角色：main → sessions_spawn ✅ 深度 1（maxDepth>=2） → 角色：orchestrator → sessions_spawn ✅ 深度 1（maxDepth=1） → 角色：leaf → sessions_spawn ❌ 工具不存在 深度 2+ → 角色：leaf → sessions_spawn ❌ 工具不存在
```

```
<a id="doxcn9NoaGSUWLx6dxGIHrF6Kxh"></a>
#### **6.4.3 Agent通信机制**

Agent的通信机制其实也是一个高频考点，面试官喜欢问的，你的Agent之间是如何通信的？

> 在我们之前笔记的「Agent 之间的通信与状态管理」章节中，已经介绍过 Agent 间的两种主要通信模式和消息传递策略。这里结合 OpenClaw 的实现，看看理论在真实项目中是如何落地的。
```


6.4.3.1 通信模式

工具调用，而非移交

Agent 间通信有两种常见模式（详细内容见 笔记 Agent之间通信和状态管理 部分）：

\- **移交（Handoff）**：一个 Agent 将执行上下文和执行权完整传递给另一个 Agent，自身退出。控制权转移，就像接力跑把棒交出去。

\- **工具调用（Tool Call）**：一个 Agent（主管）将任务委托出去，保留控制权，等结果回来后自己决定下一步，负责最终输出。

6.4.3.2 消息传递

仅最终结果，不传推理链

> 之前笔记的「Agent 之间的通信与状态管理」章节也介绍过消息传递内容的两种策略：**共享完整推理数据**（中间步骤全部写入共享通道，协作能力强但上下文膨胀快）和**仅共享最终结果**（私有空间内完成计算，只把结论写入共享区，状态复杂度可控）。OpenClaw 选择的是后者。

子 Agent 完成任务后，只把最终回复（`readLatestAssistantReply`）蒸馏成一条消息推回父 Agent，中间的 tool call 链、chain-of-thought 全部留在子 Agent 自己的私有 session 里，父 Agent 看不到。这有效控制了父 Agent 的上下文膨胀——每个子 Agent 都有独立的 session 状态，父 Agent 只收到一条结论性消息。

6.4.3.3 结果传递机制

push-based announce

具体实现上，子 Agent **不是**同步返回值，而是异步 push：

1. 子 Agent 完成任务
2. `subagent-announce.ts` 把蒸馏后的结果以"用户消息"形式注入父 Agent 的消息队列
3. 父 Agent 在 LLM 下一轮 turn 中感知到子 Agent 完成事件
4. 父 Agent 等所有期待的子 Agent 都 announce 后，才输出最终回复

这是工具调用模式的一个变体：spawn 时立刻返回 `accepted`（非阻塞），结果通过消息通知而非返回值传回。可以看到以下这个代码注释，当子agent完成了结果后，会把结果发送到gateway中，gateway会把这个结果当成一条用户消息发送给父agent，当父agent收到这个消息时，就知道子agent完成了，当所有子agent都完成了，父agent就可以整合结果输出了。

> 来源注释（代码原文）：

> *"After spawning children, do NOT call sessions_list, sessions_history, exec sleep, or any polling tool. Wait for completion events to arrive as user messages."*

> — `subagent-spawn.ts: SUBAGENT_SPAWN_ACCEPTED_NOTE`

6.4.3.4 结果的整合

假设 Parent spawn 了 5 个子 Agent，它们各自完成后依次 push 消息回来，Parent 的对话历史里会陆续积累：

```Plain Text
[消息] 子 Agent A 完成：结果是 X
[消息] 子 Agent B 完成：结果是 Y
[消息] 子 Agent C 完成：结果是 Z
...
```

每来一条 Parent 都被触发一次 turn，发现还没收全就等待，直到最后一条到达时，所有结果已经在上下文里了。这时提示词（`subagent-announce.ts` 的 Subagent Context）告诉 Parent：

> *"Coordinate their work and synthesize results before reporting back."*

结合 `subagent-spawn.ts` 里的：

> *"track expected child session keys, and only send your final answer after ALL expected completions arrive."*

Parent LLM 在这一轮 turn 里自由整合所有结果，输出最终回复。

6.5 安全风险

OpenClaw 之所以强大，是因为它深度接入了你的系统——它可以运行 Shell 命令、读写文件、执行脚本、控制浏览器。但 access 是一把双刃剑：同样的能力让它能帮你干活，也让它成为潜在的攻击面。

风险有多大

Cisco 的安全团队分析了 OpenClaw 的生态系统，发现 **26% 的可用 skill 包含至少一种安全漏洞**。他们称之为 "security nightmare"。主要风险包括：

- **Prompt Injection（提示词注入）** — 攻击者可以在邮件、文档、网页中嵌入恶意指令。当 Agent 读取这些内容时，可能被诱导执行非预期操作（比如泄露数据、发送消息、执行命令）。这是当前 AI Agent 最普遍且最难防御的攻击向量。
- **恶意 Skill / 插件** — Marketplace 上的第三方 skill 可能包含恶意代码。安装一个 skill 等于授予它 Agent 的全部能力——Shell 执行、文件访问、网络请求。一个看似无害的 "天气查询" skill 可能在后台做任何事。
- **凭证泄露（Credential Exposure）** — Agent 运行在你的机器上，可以访问环境变量、配置文件、SSH key、API token 等。如果 Agent 的行为被 prompt injection 劫持，这些凭证可能被外泄。

- **命令误解（Command Misinterpretation）**— Agent 可能误解你的意图，执行了你不想要的操作。比如你说"清理一下这个目录"，Agent 可能删掉你没打算删的文件。Shell 命令是不可逆的，`rm -rf` 不会问你确认。

官方怎么说

OpenClaw 的官方文档也承认**没有完美的安全 setup**。他们建议：

- **使用隔离账号（isolated accounts）**— 不要用你的主账号运行 OpenClaw，创建一个权限受限的专用账号
- **使用次要设备（secondary machine）**— 最好在一台非关键机器上运行，而不是你的主力工作机
- **限制 skill 数量** — 只安装你确实需要的 skill，减少攻击面
- **监控日志（monitor logs）**— 定期检查 Agent 的执行日志，了解它做了什么

本质矛盾

这是所有 AI Agent 平台面临的根本矛盾：**能力越强，风险越大。** OpenClaw 的价值在于它能深度操作你的系统，但这也正是它的风险所在。它不像一个沙箱里的聊天机器人——它拥有真实的系统权限，运行真实的命令，访问真实的文件。任何一次 prompt injection、一个恶意插件、一次命令误解，都可能造成真实的后果。

6.6 面试问题

吸收了上面的问题，我觉得你可以足够应付大部分的openclaw面试问题。这一小节快速总结上述知识，给出标准面试参考答案，毕竟面试时不可能啰里吧嗦说一堆，用这些面试问题检验自己吧\~

Q1: OpenClaw 是什么?

OpenClaw 是一个开源的、可自托管的个人 AI 助手平台。本质上是一个事件驱动的 Agent 执行引擎，前面放了一个多渠道网关。用 TypeScript 编写，运行在你自己的机器上，所有数据和密钥不离开本地。

Q2: OpenClaw 的架构是什么?

三层架构：触发层、网关层、Agent 层。触发层有五种事件源（Messages、Heartbeats、Crons、Hooks、Webhooks）不断产生事件；网关层负责协议转换、认证和路由分发；Agent 层负责推理（Think）、工具执行（Act）和记忆（Remember）。本质是一个事件驱动的执行管道：事件产生 → 网关路由 → Agent 执行 → 状态持久化。

Q3: Gateway 是什么？它的作用是什么?

Gateway 是一个本地长驻的 Node.js 进程，监听单一端口（默认 18789），同时提供 HTTP 和 WebSocket。它是触发层和 Agent 层之间的中间件，负责渠道连接管理、协议转换、会话路由、认证鉴权、插件加载和设备配对。关键边界是：Gateway 不做推理、不调 LLM、不执行工具，只负责"消息到了该给谁"和"回复该送回哪里"。

Q4: 有哪些消息源/触发源?

五种：

1. **Messages** — 用户通过聊天渠道发消息，事件驱动
2. **Heartbeats** — 固定间隔定时器（如每 15 分钟），让 Agent 主动唤醒
3. **Crons** — 用户定义的计划任务（cron 表达式），执行具体的定时工作
4. **Hooks** — Gateway/Agent 内部生命周期事件触发的回调
5. **Webhooks** — 外部系统通过 HTTP POST 触发

加上 Agents 可以派生 Subagents，形成递归执行。

Q5: 为什么能接入各个平台?

因为 Gateway 层的 Channel Manager 把每个渠道封装为一个独立的 Channel Plugin，每个插件实现统一的接口（ChannelGatewayAdapter、ChannelMessagingAdapter、ChannelOutboundAdapter）。不同渠道的协议差异（Telegram Bot API、Discord WebSocket、WhatsApp Baileys 等）被封装在各自插件内部，对上层 Agent 暴露一致的消息接口。加一个新渠道只需写一个 Channel Plugin，Agent 代码零改动。目前支持 20+ 个渠道。

Q6: 记忆能力是如何实现的（或者说是长期记忆，注意和后面的记忆系统区分开）?

记忆是 Agent 主动调用的工具，不是自动注入 prompt。Agent 通过 `memory_search` 工具进行混合检索（70% 向量语义搜索 + 30% BM25 关键词搜索），再通过 `memory_get` 读取完整内容。底层是 SQLite 数据库，同时维护向量 embedding 索引和全文索引。记忆写入有两种方式：会话结束时自动捕获为 Markdown 文件，以及用户手动编辑记忆文件后通过文件监听增量索引。

Q7: 相比传统 AI 聊天机器人，OpenClaw 的优势在哪?

四个核心差异：

1. **不止聊天，能做事** — 可以执行 Shell 命令、控制浏览器、修改文件、远程操作设备，是一个真正的 Agent
2. **不止等消息** — 五种触发源让它能定时唤醒、自主行动，而不是只等用户说话
3. **隐私自主** — 本地部署，LLM API 直连，数据不经过第三方服务
4. **渠道统一** — 一个 Agent 同时服务所有聊天平台，而不是每个平台各建一个 bot

Q8: OpenClaw 有什么优缺点?

优点：

- 开源自托管，隐私可控，数据不离开本地
- 支持 20+ 渠道，一个 Agent 统一服务
- 插件化架构，核心精简，能力可无限扩展
- 五种触发源，能主动行动而非只是被动回复
- 支持多模型 provider，可灵活切换和 failover

缺点/风险：

- 深度系统权限带来安全风险——Cisco 分析发现 26% 的可用 skill 包含漏洞
- Prompt injection 攻击可能通过邮件/文档劫持 Agent 行为
- 第三方插件可能包含恶意代码
- 命令误解可能导致不可逆操作（如误删文件）
- 官方也承认没有完美的安全 setup，建议用隔离账号和次要设备运行

Q10：当对话超出上下文限制时，OpenClaw 的压缩策略是什么?

A:OpenClaw 采用的是**分阶段摘要式压缩**。当上下文 token 用量接近模型窗口上限时会自动触发，用户也可以通过 `/compact` 命令手动触发。

具体过程是这样的：首先把对话历史分成**两部分**——最**近大约 20,000 tokens 的消息作为"保留区"原封不动地留下来，更早的旧消息进入"压缩区"**。然后把压缩区的内容按 token 数切成若干块，默认是 2 块，块大小会根据消息平均长度自适应调整，同时预留 20% 的安全裕量防止 token 估算不准。接着用 LLM 对每一块生成结构化摘要，如果有多块就再合并成一份连贯的总结。摘要有严格的保留要求，必须包含进行中的**任务状态、用户最后的请求、决策理由、TODO 和约束条件，以及所有不透明标识符比如 UUID、URL、文件路径等，都必须原样保留。**

另外还有一个亮点是**压缩前记忆冲刷机制**：在真正压缩之前，系统会静默触发一轮 agent 回合，提醒模型把重要信息写入记忆文件，这样即使对话被压缩，关键知识也不会丢失。压缩完成后，模型看到的上下文就变成了"压缩摘要加上保留区原文"这样的结构。

Q11: OpenClaw 如何组织长短期记忆?

A11:OpenClaw 的记忆分为**短期记忆**和**长期记忆**两部分，各自有不同的存储形式和组织方式。

短期记忆就是当前会话的对话上下文，包括用户消息、助手回复、工具调用结果这些内容，全部在 Context Window 里以消息列表的形式存在。它的特点是容量受模型上下文窗口限制，比如 Claude 是 200K tokens，满了之后旧的对话就会被压缩成一段结构化摘要，只保留最近约 20,000 tokens 的原始消息。所以短期记忆本质上是临时的、会被压缩的。

长期记忆则完全基于磁盘上的 Markdown 文件，分成两种组织方式。第一种是 `MEMORY.md`，这是一个策展式的长期记忆文件，存放的是用户偏好、持久性决策、重要事实这些需要长期保留的内容，由 Agent 主动维护和更新，每次会话启动时注入上下文，不受压缩影响。**第二种是** `memory/YYYY-MM-DD.md`，按日期组织的追加式日志，存的是当天的运行笔记、阶段性进展、临时但值得回顾的上下文，会话启动时只读入今天和昨天的内容。

长**期记忆还配有一套检索索引机制。**系统会对所有记忆文件建立混合搜索索引，包括**向量语义搜索和 BM25 关键词搜索**，Agent 在需要的时候通过 `memory_search` 工具按需检索，不用把全部历史记忆都塞进上下文里。这样即使记忆文件越积越多，也不会撑爆上下文窗口。

两者之间的衔接靠的是压缩前记忆冲刷：当短期记忆快满需要压缩时，系统会先静默提醒 Agent 把当前对话中重要的内容写入长期记忆文件，然后再做压缩。这就保证了短期记忆中真正有价值的信息能沉淀到长期记忆

里，不会因为压缩而彻底丢失。

Q12: 简述 OpenClaw 的记忆系统原理。

A12:OpenClaw 的记忆模块并不是单一系统，而是由四层协同机制组成的，可以类比计算机的存储层次。

第一层是 Bootstrap Files，相当于硬盘上的固件。包括 SOUL.md、AGENTS.md、USER.md、MEMORY.md 等 Markdown 文件，存放的是 Agent 的人格设定、行为规则、用户身份和策展过的长期知识。它们每次会话启动时都从磁盘重新读取注入上下文，完全免疫压缩，是最稳定的一层记忆。不过有字符上限，单文件 20,000 字符，总计 150,000 字符，所以要保持精简。

第二层是 Session Transcript，相当于文件系统日志。完整的对话历史用 JSONL 格式逐条追加写入磁盘，包括用户消息、助手回复、工具调用和压缩摘要。它是跨重启持久的，继续会话时从 JSONL 重建上下文。但模型不是看全部记录，而是只看当前上下文窗口能装下的部分，更早的内容通过压缩变成摘要。

第三层是 Context Window，相当于 RAM。就是模型每轮推理时实际能看到的所有内容，包括系统提示、Bootstrap Files、对话历史、工具返回结果都挤在这个固定大小的空间里。容量由模型决定，满了就触发 Compaction。压缩时把最近约 20,000 tokens 原样保留，更早的消息用 LLM 生成结构化摘要替代，摘要里必须保留任务状态、决策理由、TODO 和关键标识符。压缩前还会静默提醒 Agent 把重要信息写入记忆文件，防止信息丢失。

第四层是 Retrieval Index，相当于搜索引擎。系统对所有记忆文件建立混合搜索索引，同时使用向量语义搜索和 BM25 关键词搜索，按 7:3 权重融合排序。Agent 通过 memory_search 和 memory_get 这两个工具按需检索，不需要把全部记忆都塞进上下文。检索是模型自主决策的，不是每轮自动触发。索引还支持时间衰减和去重，新笔记排名更靠前。

这四层的协同逻辑是：Bootstrap Files 提供稳定的身份和规则基座，Session Transcript 保存完整的历史记录，Context Window 管理当前可见的工作记忆并在满时通过压缩保持可用，Retrieval Index 让 Agent 能按需回忆历史而不占用上下文空间。关键设计原则是只有写入磁盘文件的信息才是真正持久的记忆，纯对话中没有持久化的内容压缩后就不可恢复了。

Q13：OpenClaw 采用了什么 Agent 架构?

A：递归 Centralized（中心化）架构。Parent Agent 作为中心，通过 LLM 的 tool call 机制调用 `sessions_spawn` 工具创建子 Agent 执行子任务，子 Agent 完成后 push 结果回父，父 Agent 等所有子任务完成后整合输出。子 Agent 可以继续 spawn 孙 Agent，形成一棵 Orchestrator-Worker 树，每一层都是该层的中心。

Q14：OpenClaw 中的 Agent 是如何通信的?

A：通信模式上是**工具调用**，控制权始终留在 Parent Agent。具体实现是 push-based 异步消息：子 Agent 完成后通过 `subagent-announce.ts` 调用 `callGateway({ method: "agent" })`，Gateway 把结果以"用户消息"的形式投递到父 Agent 的消息队列，触发父 Agent 的下一轮 LLM 推理。父 Agent 不轮询、不阻塞线程，等消息触发即可。

传递内容上只传**最终结果**，子 Agent 的中间推理步骤（tool call 链、chain-of-thought）全部留在私有 session 里，不传给父 Agent，有效控制上下文膨胀。

Q15：为什么 OpenClaw 要选择中心化的 Agent 架构?

A：根本原因是对话本身的结构是中心化的——有确定的入口（用户发消息），有确定的出口（给用户一条完整回复），中间的执行可以分发，但责任链不能断。

如果换成去中心化（P2P 协商），最终谁来发回复没有确定答案；如果换成 Independent（各自独立），结果无法拼合成一条完整回复。Centralized 的父子结构正好对应这个"一进一出"的模型：Parent Agent 是唯一的回复责任人，子 Agent 只是它延伸执行能力的手段。

Q16：使用 Centralized 架构后，系统性能受限于 Parent Agent，如何解决?

A：这是 Centralized 架构的经典瓶颈。OpenClaw 在设计上从三个方向缓解这个问题：

1. **减少 Parent 的工作量**：Parent 只做编排和最终整合，不做具体执行。OpenClaw 在子 Agent 的系统提示词里明确约束（`subagent-announce.ts` Subagent Context）：*"Stay focused - Do your assigned task, nothing else"*、*"NO user conversations (that's parent's job)"*。子 Agent 不允许跑题、不允许主动发消息，Parent 只需要等结果、做最终决策，每轮 turn 的工作量被压到最低。
2. **并行化子任务**：Parent 可以同时 spawn 多个子 Agent 并行执行，不需要串行一个一个等。总耗时取决于最慢的那个子 Agent，而不是所有子任务时间之和。Parent 在提示词里也被告知可以同时 spawn 多个任务，依靠 push-based announce 各自回来触发汇总。
3. **并发数量限制（稳定性护栏）**：并行是加速手段，但无限并行会耗尽系统资源，所以 OpenClaw 同时设置了两层上限作为反向约束：单个 session 的活跃子 Agent 数由 `maxChildrenPerAgent` 控制（默认 5，来源：`subagent-spawn.ts:344`），全局所有子 Agent 的并发数由 `DEFAULT_SUBAGENT_MAX_CONCURRENT` 控制（默认 8，来源：`src/config/agent-limits.ts`）。超出上限时 `sessions_spawn` 直接返回 `forbidden`，防止雪崩。
4. **递归下推编排**：当配置 `maxSpawnDepth >= 2` 时（默认是 1），子 Agent 本身也可以变成 Orchestrator（角色从 `leaf` 升级为 `orchestrator`，`sessions_spawn` 工具会被加入其工具列表），继续向下 spawn 孙 Agent，把编排压力分散到子层。Parent 只负责顶层任务拆解，不需要直接管所有叶子节点。
5. **按角色分配模型**：`sessions_spawn` 的参数支持 `model` 和 `thinking` 覆盖，可以给 Parent 配置推理能力强的模型做决策，给执行型子 Agent 配置更快更轻量的模型做具体任务，按角色分配算力而不是一刀切。

6.7 参考资料

首先是OpenClaw的Setup，网上太多啦，我就分享两个我看过的视频，我确实跟着他操作起来的。

> 微信视频号，博主：小创作：

2026/2/14发布的：《15分钟用Openclaw + GLM5打造国内最强AI助手》 讲了环境配置，以及接入飞书和GLM5模型

2026/2/27发布的:《一个视频让你Openclaw技术瞬间超过90%的人》 ： 讲了Openclaw基本必须会的技巧。

然后是技术学习：

推荐直接看我的笔记 + 源码（毕竟openclaw开源嘛）。本节一些知识部分直接引用了对应的代码，是为了你如果想要去读源码可以和笔记的内容匹配上。下面是一些我整理笔记参考的视频，其实可以直接看我的笔记的，我都总结了。感兴趣也可以去看看。

10多分钟，讲openclaw架构和安全

https://www.youtube.com/watch?v=CAbrRTu5xcw

40多分钟，更全面的openclaw技术细节

https://www.youtube.com/watch?v=vte-fDoZczE

讲Openclaw 4层记忆layer的，不过我笔记都总结了

https://www.youtube.com/watch?v=HM0ATQCHGP0

7 开源框架解读 - Hermes Agent

📝 2026年10月补充更正｜当前学习建议

根据目前求职市场的面试反馈，OpenClaw 和 Hermes Agent 的考察已比较少；Claude Code 仍会被问到，Pi-Agent 的热度和考察频率有所上升。DeepSeek Harness 和 Codex 作为近期开放源码的架构，也同样值得关注。

因此，建议优先学习 Claude Code、Pi-Agent、DeepSeek Harness 和 Codex，有余力或感兴趣时，再学习本章节。正文中早期关于“必考”“必须掌握”等表述，请结合当时的撰写时间理解，当前学习优先级以本声明为参考。

下方为早期撰写的原文，保留供学习参考。其中“必考”“必须掌握”等判断反映的是当时的情况，当前学习优先级请以上述更正为准。

📖 历史原文｜写于2026年4月底

如果说 2025 年的风向标是 Manus，2026 年初是 OpenClaw，那么 2026 年春天以来最火的 Agent 就是 **Hermes Agent**——它在 OpenRouter 上已经是 top trending coding agent，仅次于 OpenClaw。站在面试角度，Hermes 已经是必须掌握的第二个Agent 爆品：它和 OpenClaw 属于**同一类产品。**它和OpenClaw是同一类产品：它们的使用形式和功能是类似的。但是Hermes 最出圈的一点是它的自我进化能力，就是它看上去能记事，更聪明。拍脑袋想我们大概也知道Hermes 最常考的内容是啥："和Openclaw的区别，新的实现，为什么能自我进化，记忆模块", 这也是Hermes最核心的卖点和创新点。

Hermes 是什么、总体架构、记忆系统（四层 → 五层）、Agent 架构，以及 Hermes 独有的 **自进化循环** 和 **自动 Skill 沉淀** 两个核心新特性。学完这一章你应该能在面试时说清楚"为什么在同样的模型下 Hermes 跑得更快、越用越懂你"，并把 Hermes 的设计思路搬到自己的 Agent 项目里加分。

(上面的内容也确实是群里的同学催我更新Hermes，说面试问到他不知道怎么答的问题。大模型的东西更新太快了，但是大家不要急，出现一个新的东西的时候，我们先理解它的核心特性和最高频的面试问题，学完今天的笔记，大家已经可以应付目前很多面试中Hermes工程的问题了。当然，面试的问题是变化的，如果Hermes后面越来越热，又不断更新，面试可能会进行更深入的考察，我们也会实时跟进（欢迎大家反馈））

7.1 Hermes Agent是什么

Hermes Agent 是 **Nous Research** 推出的开源、可自托管的自主 AI Agent 平台，核心卖点是**自我进化**（self-improving）：同一个模型，用得越多，它越懂你。

- 核心用 **Python** 编写（入口脚本 `hermes-agent/hermes`、`cli.py`），通过 `pip install hermes-agent` 或官方一行安装脚本部署
- 支持 Linux / macOS / WSL2 / Termux（**Windows 原生不支持，需要 WSL2**）
- 所有数据都存在本地（`~/.hermes/`），和 OpenClaw 一样不经过云中转
- MIT 协议开源，与 OpenClaw 可通过 **ACP 协议**互相调度

Hermes 是一个带"自进化闭环"的事件驱动 Agent 执行引擎——OpenClaw 做的它都做（多渠道网关、cron、子 Agent），但它额外把"经验沉淀成可复用 skill"和"持续学习用户画像"做成了一等公民。

7.2 Hermes 总体架构

Hermes 的总体架构是**三层主干 + 一圈闭环**——主干三层是**触发层（Trigger）→ 网关层（Gateway）→ Agent 层（Agent Loop）**，和 OpenClaw 完全同构；闭环是 Agent 层额外挂的**自进化回路（Reflect → 记忆/Skill 沉淀 → 反哺下一次触发）**，这是 Hermes 区别于 OpenClaw 的核心。

所以当面试官问"Hermes 架构是几层"时，标准答法是：**"主干 3 层 + 1 个自进化闭环"**，然后按下面的图展开。

看完第 10 章 OpenClaw 架构这部分可以秒懂，差异集中在 Agent 层额外多了一圈**自进化闭环**：

```Plain Text
触发层 → 网关层 → Agent 层 → 记忆/Skill 沉淀 → 反哺下一次触发
                                  └── 每 ~15 次 tool call 触发自进化反思
```


7.2.1 触发层

Hermes 的触发维度和 OpenClaw 一一对应，同样是 **5 种信号源**，没有新东西。这部分和 OpenClaw 笔记里的触发层完全一致——从这一点也能看出来 Hermes 作者应该是在 OpenClaw 基础上进行改进的，这也是为什么面试官喜欢把两者放在一

起对比：

- **Message（消息）** — CLI、Telegram、Discord、Slack、WhatsApp、Signal、Email（比 OpenClaw 多一个 Email）
- **Heartbeat（心跳）** — 周期性唤醒 Agent 进行自检 / 续跑
- **Cron（定时任务）** — 内置 cron 调度器（源码 `hermes-agent/cron/scheduler.py`、`cron/jobs.py`），支持每日/每周自动任务并推送到任意平台
- **Webhook（外部推入）** — 同样支持外部系统 HTTP 推入
- **Hooks（生命周期钩子）** — 会话开始/结束、工具调用前后等事件挂钩，可触发自定义逻辑

7.2.2 网关层 — 一进程多渠道

Gateway 是什么：本地长驻的一个进程，是触发层和 Agent 层之间的中间件——所有外部事件（消息、webhook、cron 触发）都先进 Gateway，由它统一接入、认证、再分发给对应的 Agent session。

核心职责（4 件事，别跑偏）：

1. **连接管理**：同时维持多个渠道的长连接（CLI、Telegram、Discord、Slack、WhatsApp、Email……一进程多渠道）
2. **协议转换**：把 Telegram 的 JSON、Discord 的 WebSocket 帧、Email 的 MIME、Webhook 的 HTTP POST 等不同协议，统一转换成内部标准事件格式
3. **路由分发**：根据事件里的 agentId / sessionId，把事件投递给正确的 Agent 会话；回复也按原路送回对应渠道
4. **安全控制**：入口处做认证（token / 签名校验）、限流、沙箱隔离

关键边界（面试必答）：Gateway **只做路由，不调 LLM，不执行工具**。它只回答两个问题——"这条消息该给哪个 Agent？"、"Agent 的回复该送回哪个渠道？"。这个边界让 Gateway 保持轻量，可以横向扩展，Agent 换掉也不影响接入层。

一句话总结：**Gateway = 多渠道适配器 + 路由器 + 安全网关**，是"一个 Agent 能同时出现在 Telegram、Discord、CLI 里"的技术基础。

7.2.3 Agent 层

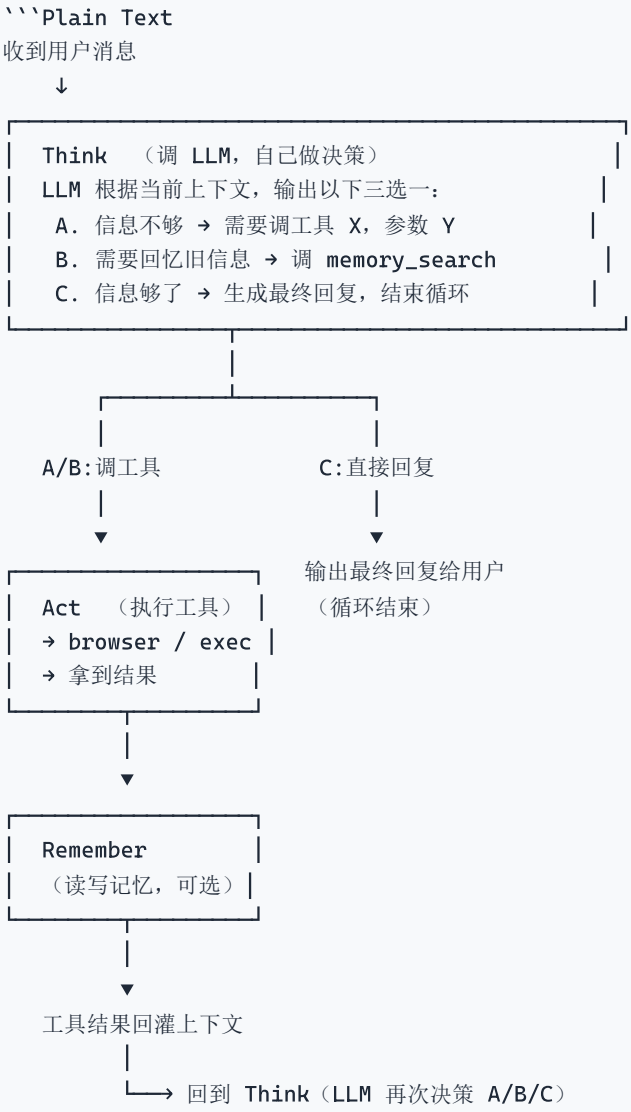
Agent 层是整个 Hermes 的"大脑"，Gateway 把事件路由过来之后，真正的推理、工具调用、记忆读写全在这里完成。

Agent 循环的 4 步：**Think（推理）→ Act（工具执行）→ Remember（记忆读写）→ Reflect（周期性反思）**。前三步是所有 Agent 都有的标准循环，第 4 步 Reflect 是 Hermes 的独有创新（即**自进化闭环**），在 11.3 节详细展开。

> **自进化循环是什么？** 一句话先打个底：这是 Hermes 的"自我反思 + 经验沉淀"模块。用一句话概括就是 "*like back propagation but for prompts instead of model weights*"——**像反向传播，但更新的不是模型权重，而是 prompt / 记忆 / skill**。周期性触发，让 Agent 回看自己刚才做了什么，把有价值的事实写进 `memory/` md 文件、把成功流程抽象成 Skill、把用户偏好更新到 **Honcho**（一个独立的第三方开源用户建模服务，`honcho.dev`，专门给 Agent 存"这个用户是谁"——偏好、沟通风格、目标；Hermes 作为可选依赖集成它）。完整机制在 11.4.4 讲。后面章节会反复提到"自进化循环"，看到它就想"这是 Hermes 的周期性反思和写记忆的机制"即可。

什么叫"循环"——ReAct 范式：

Agent 不是"想一次就直接输出答案"的单次推理，而是**"思考 → 行动 → 观察结果 → 再思考"的多轮迭代**，这就是业界最经典的 **ReAct 范式**（Reasoning + Acting）。一次典型的循环长这样：



关键性质：

- **"任务完不完，由 LLM 自己判断"**：没有外部调度器在循环外面数圈，而是每一轮 Think 里 LLM 根据当前上下文自主决定"继续调工具"还是"输出回复、结束"。这就是 ReAct 的精髓——**决策权在 LLM 手里。**
- **Think 和 Act 是交替的，不是一次性的。**同一个任务可能循环 3 次、10 次甚至几十次 tool call 才结束。
- **每一轮的工具结果都会回灌到下一轮 Think 的上下文里**，让 LLM 基于新信息继续推理——这就是"推理 + 行动"比"纯推理"强的原因。

> 下面三个小节（11.2.3.1 / 11.2.3.2 / 11.2.3.3）分别讲这个循环里每一步的**具体实现**——Think 层如何调

> LLM、Act 层有哪些工具、Remember 层的记忆怎么组织。

7.2.3.1 推理 (Think)

Think 层的职责很聚焦：**把当前上下文（系统提示 + Bootstrap Files + 对话历史 + 上一轮工具结果）送给 LLM，拿到 LLM 的下一步决策**（直接回复 / 调用某个工具 / 调用子 Agent）。

Hermes 在 Think 层的核心要点是**主模型 + 辅助模型分工（Auxiliary Client）**。

Hermes 允许同时配置两个模型，让不同能力的模型干不同的活：

- **主模型**：做长链推理、规划、复杂任务——用强模型（如 Claude Opus、GPT-5）
- **辅助模型**：做压缩、摘要、分类、错误归类等轻量任务——用便宜/本地的弱模型（如 Qwen、Gemma）

这样做的价值是**在成本、速度、质量之间取得平衡**——长链推理用贵的，但压缩一段对话历史这种活没必要也让 Opus 来干。这也是 harness engineering 的典型思路：**不是用一个模型解决所有问题，而是把任务按难度分层、匹配合适的模型。**

7.2.3.2 工具执行 (Act)

Agent 推理过程中可以自主决定调用工具。Hermes 的内置工具覆盖了 Agent 干活的几乎所有场景：

- **终端/exec** — 执行 Shell 命令（带 allowlist 安全策略）
- **浏览器** — 通过 CDP 控制 Chromium，浏览网页、截图、操作页面
- **文件操作** — 读写本地文件、应用 patch
- **消息发送** — 向任意渠道（Telegram / Discord / Email ...）发送消息
- **Subagent Spawn** — 启动子 Agent 处理复杂/并行任务
- **MCP 集成** — 通过 Model Context Protocol 接入任意外部工具生态

Hermes 的一个特色是 **6 种终端后端**：`local` / `Docker` / `SSH` / `Daytona` / `Singularity` / `Modal`（serverless 持久化，闲时近乎零成本）——这给 Agent 提供了从本地开发到云端大规模执行的全套部署选项。

Hermes 独有的一个小设计：**Error Classifier**（`agent/error_classifier.py`）——把常见失败模式（网络错误、权限错误、文件不存在、rate limit 等）预编码成分类器。这样当工具调用失败时，弱模型不需要从错误消息里自己推理"这是什么错、该怎么恢复"，而是由 Error Classifier 直接把错误归类成已知类别，走预设恢复路径。这就是 harness engineering 的典型落地——**用系统层面的预处理，降低对模型推理能力的依赖。**

7.2.3.3 记忆 (Memory) & Skills

记忆是 Agent 层的第三个核心组件，但不是每次对话都自动塞入上下文（那样太浪费 token），而是 **Agent 主动按需检索**——Agent 自己判断"需不需要回忆过去"，然后调用记忆工具去搜索。

Hermes 的记忆体系分 **5 层 + 1 个用户画像**：

- **Bootstrap Files** — 永久身份与规则（AGENTS.md 等），每次会话开始从磁盘注入
- **Session Transcript** — 完整对话历史存档（JSONL 文件追加写入）
- **Context Window** — 当前推理能看到的内容（满了触发 Compaction）
- **Retrieval Index** — 记忆文件的全文/向量混合检索索引
- **Skill Library** — 程序性记忆，可复用的成功流程（Hermes 独家第 5 层）
- **Honcho User Model** — 独立的用户画像模型，专门存"你是谁"（偏好、沟通风格、目标）

和 OpenClaw 的对比：前 4 层（Bootstrap / Transcript / Context / Retrieval）和 OpenClaw 完全一样，Hermes 直接沿用了 OpenClaw 的设计思路；**Skill Library 和 Honcho User Model 是 Hermes 新加的两块**，这是两者记忆体系的核心差异。

> 💡 举个例子，系统中前 4 层属于**陈述性记忆**（"我记得什么事实"），Skill Library 属于**程序性记忆**（"我会做什么流程"），Honcho 是独立的**用户建模**（我是谁）。相比Openclaw,"OpenClaw 只有陈述性记忆，Hermes 把程序性记忆和用户建模也补齐了，这是它'越用越懂你'的理论基础。"

这 6 块是 Hermes "越用越懂你" 的数据底座。具体每一层的存储结构、写入/读取时机、压缩策略、如何协同工作，在 **11.4 节 Hermes 记忆功能详解** 里展开。

7.2.4 架构总览图

到这里我们已经把 Hermes 三层主干（触发层 / 网关层 / Agent 层）都讲完了。下面用一张图把各模块的位置关系串起来，让大家对"一个事件从进来到处理完的完整路径"有个整体印象：

```Plain Text
┌──────────────────────────────────────────────────┐
│ 触发层 │ │ Message · Heartbeat · Cron · │ │ Webhook · Hooks │
└──────────────────┬──────────────────────────────┘
                   │ 事件
                   ▼
┌──────────────────────────────────────────────────┐
│ 网关层 Gateway │ │ 连接管理 · 协议转换 · 路由分发 · 安全控制 │ │ （不调 LLM、不执行工具） │
└──────────────────┬──────────────────────────────┘
                   │ 统一事件 → session
                   ▼
┌──────────────────────────────────────────────────┐
│ Agent 层 │ │ │ │
┌──────────────────────────────┐ │ │ │ Think → Act → Remember │ ◄──┐ │ │ │（循环，由 LLM 自主决定 │ │ │ │ │ 是否继续 / 结束） │ │ │ │
└──────────────┬───────────────┘ │ │ │ │ │ │ 工具结果回灌上下文 ──────────┘ │ │ │
│ 可用模型：Claude / GPT / Gemini / │ │ Qwen / Ollama / …（任选） │ │ │ 记忆体系：Bootstrap · Transcript · │ │ Context · Retrieval Index │ │ + Skill Library (独有) │ │ + Honcho 用户画像 (独有) │
└──────────────────┬──────────────────────────────┘
                   │ 最终回复 ▼ 原渠道送回用户
```

> **三点提醒**：

1. 上图是 Hermes 的**静态结构**，还没有画出 Hermes 真正的创新——**自进化循环**。自进化循环本质是作用于记忆层之上的动态更新策略，会在 **11.4 节 · 动态更新策略** 里讲。
2. Agent 层的 Think/Act 循环由 LLM 自主驱动，Gateway **不参与推理决策**——这是职责边界。
3. 记忆体系的前 4 层和 OpenClaw 相同，后 2 层是 Hermes 新增；静态结构见 11.3 节，动态更新见 11.4 节。

7.3 Hermes 记忆系统 · 静态结构

（**高频考点**：Agent记忆系统是Agent核心。大家看任何Agent的时候，都应该有意识，去了解一下它的记忆功能是如何实现的。其中记忆功能，大家可以思考两点：由哪几部分组成（静态），如何更新，压缩（动态），笔记的ClaudeCode架构也是这么组织的。想清楚这两点，就想清楚了一个Agent的记忆模块最核心的东西。）

7.3.1 总览：一个上下文窗口 + 三类数据源（+ 一个索引）

先给大家一个整体骨架：Hermes 的记忆系统由一个**上下文窗口**（Context Window）+ **三类数据源** + **一个索引**组成。每轮推理，系统把三类数据源的内容拼进工作集、送给模型；索引只在数据源 1 里按需搜片段，不直接进工作集。下面先看这五个角色分别是什么。

三类数据源（真正进 Context 的内容都出自这三类）：

1. **工作空间 md 文件**——人写的（或自进化循环写的）配置和知识文件，包含 Bootstrap Files、`memory/` 笔记、Skill Library
2. **Session Transcript**——系统自动追加的完整对话日志 `.jsonl`
3. **Honcho User Model**——外部独立服务，存用户画像

一个索引：

- **Retrieval Index**——为 `memory/` 下的 md 文件建的向量+全文索引。**它自己不进 Context**——它的作用是让 `memory_search` 能快速从 md 文件里找到相关片段，**真正进 Context 的是搜到的那段 md 原文**。

```Plain Text
┌──────────────────────────────────────────────────────────────┐
│  上下文窗口 · Context Window（每轮重新拼装）                  │
│                                                              │
│    = System Prompt（Bootstrap 文件拼出来的）                 │
│    + 近期对话切片（从 .jsonl 尾部加载）                      │
│    + 检索到的 md 片段（memory_search / memory_get 返回的原文）│
│    + Skill 内容（命中时加载 SKILL.md）                       │
│    + Honcho 用户画像摘要                                     │
│    + 当前用户消息 + 工具结果                                 │
└──────────────────────────────────────────────────────────────┘
          ▲                    ▲                    ▲
          │  md 原文           │  对话切片          │  画像摘要
          │                    │                    │
┌─────────┴──────────┐ ┌───────┴────────┐ ┌─────────┴──────────┐
```

全景一张表：

和 OpenClaw 的对比（直接落到三类数据源上）：

- 工作空间 md 文件 / Session Transcript / Retrieval Index：和 OpenClaw 同构
- Skill Library（工作空间 md 文件里的一类）：Hermes 独有
- Honcho User Model：Hermes 独有

其实不论 ClaudeCode / OpenClaw / Hermes，记忆模块的底层都是"几份磁盘文件 + 一个派生索引 + 一个工作集"这个老套路，差异只在于"多沉淀了什么"。看透这一层，新产品再出也不慌——新东西无非是在这张图上加一个源或改一条注入路径。

7.3.2 Context Window • 上下文窗口

定位：固定大小的 token 容器（模型决定：Claude 200K、GPT-4 128K、Gemini 1M），**不是一层记忆，是记忆的消费端。**每轮推理前系统从四类数据源拼装内容填进来。

每轮被拼进来的内容（按源码 `prompt_builder.py` + `context_engine.py` 的顺序）：

1. **System Prompt**（来自工作空间 md 文件）：SOUL.md + AGENTS.md + MEMORY.md + TOOLS.md + ...
2. **Skill 系统提示**（来自 Skill Library）：精简 skill 描述列表
3. **Honcho 摘要**（来自 Honcho User Model）：用户画像的一段文字摘要
4. **对话历史切片**（来自 Session Transcript）：从 `.jsonl` 尾部加载的最近消息 + 已有压缩摘要
5. **预检索片段**（来自 `memory/` 下的 md 文件，通过 Retrieval Index 搜到，Hermes 特色）：Context Engine 根据当前消息主动做一次 `memory_search`，把命中的 md 原文片段塞进来
6. **当前用户消息 + 工具结果**

关键性质：

- 最大 token 消耗者通常是**工具结果**（文件读取、网页快照、API 响应）
- Bootstrap Files 每轮都占空间，所以要尽量精简（SOUL.md 最容易写太长）
- 满了就触发 **Compaction**（见 11.4.3）——压缩的是"当前上下文里的旧消息"，不是整个 `.jsonl`
- 用 `/status` 监控使用量，建议 75–80% 时手动 `/compress`

7.3.3 工作空间 md 文件

这一类是**磁盘上的 Markdown 文件**，用户可以直接编辑，也会被 Agent / 自进化循环写入。虽然都叫"md 文件"，但按用途分成三块。

7.3.3.1 Bootstrap Files

定位：会话每次开始都从磁盘读、注入 system prompt 的"身份文件"。

文件清单（位置：`~/.hermes/workspace/`，仓库根目录也有一份）：

- **AGENTS.md** — 操作指令、行为规则（Hermes 主入口文件）
- **SOUL.md** — 人格、语气、边界
- **USER.md** — 用户身份信息（静态版，和 Honcho 互补）
- **TOOLS.md** — 工具使用说明
- **IDENTITY.md** — Agent 名称与风格
- **MEMORY.md** — 策展的长期记忆（仅主会话加载）
- **HEARTBEAT.md** — 心跳任务清单（可选）
- **BOOTSTRAP.md** — 首次运行仪式（一次性）

关键要点：

- 每次会话**重新从磁盘读**，不进 `.jsonl` → **免疫 Compaction**
- 字符限制：单文件 20,000 字符，总计 150,000 字符，超出截断
- 子 Agent 只收到 AGENTS.md / SOUL.md / TOOLS.md / IDENTITY.md / USER.md，拿不到 MEMORY.md / HEARTBEAT.md
- 用 `/context list` 查看注入状态
- 尽量精简——每轮都占空间

7.3.3.2 memory/ 笔记

定位：Agent 的"事实性知识库"，不自动注入，靠工具按需检索。

位置：`~/.hermes/memory/*.md`、`~/.hermes/memory/**/*.md`。

- 文件命名一般是 `YYYY-MM-DD-slug.md`（归档）或 `topic.md`（常青）
- 写入来源：用户手写 / 会话结束时自动归档 / 自进化反思产出
- 这些文件被 Retrieval Index 建索引，通过 `memory_search` + `memory_get` 被检索到

与 Bootstrap 的区别：Bootstrap 是"每轮都读"，memory/ 是"需要时才读"。

7.3.3.3 Skill Library

定位：把"成功完成过的流程"固化成可执行 skill，下次遇到类似任务直接调用，不用从零推理。

Skill 文件结构：

```Plain Text skills/// ├── SKILL.md # skill 描述 + 触发条件 + 步骤说明 ├── scripts/ # 可执行脚本（可选） └── templates/ # 模板文件（可选）
```

> **目录分类**（仓库 `hermes-agent/skills/` 自带几百个预置 skill）：
>
> ```Plain Text
> skills/
> ├── software-development/   # 代码评审、重构、测试
> ├── research/               # 论文检索、总结
> ├── creative/               # ASCII art、Excalidraw
> ├── productivity/           # PowerPoint、邮件
> ├── devops/                 # CI/CD、Docker
> ├── mcp/                    # MCP 集成
> └── social-media/
> ```

Skill 的三种生成路径（写入时机见 11.4.1）：

1. **系统预置**：仓库自带几百个 skill，随 Hermes 一起分发
2. **用户安装**：`/skills` 命令或从 agentskills.io 开放标准安装
3. **Agent 自动生成**：自进化闭环触发自动写 `SKILL.md`（如 Hacker News 每日简报的例子）

两阶段检索（防上下文膨胀）：先用精简描述粗排命中，再加载对应 `SKILL.md + scripts` 进上下文。

面试回答模板：

> "Hermes 在 OpenClaw 的磁盘文件记忆之外加了 Skill Library，本质是程序性记忆。OpenClaw 的 memory 层只能记'事实'，Hermes 的 skill 层能记'怎么做'。这也是为什么 Hermes 号称'越用越懂你'——用户画像（陈述）+ skill（程序）两条线都在进化。"

7.3.4 Session Transcript

定位：**一条消息一条消息追加到磁盘**的完整对话归档。只增不改，跨重启持久。

存储路径（注意：不是"一份 session 存两遍"，**两个文件分工不同**）：

```Plain Text ~/.hermes/agents//sessions/ ├── sessions.json # 索引表：所有 session 的元信息（只有一份） ├── .jsonl # 第 1 个 session 的对话正文 ├── .jsonl # 第 2 个 session 的对话正文 └── ... # 一个 session = 一个 .jsonl
```

- **`sessions.json`（索引表）**：列出所有 session 的元信息——id、标题、创建/更新时间、消息数。类比文件系统的 `ls` 或一本书的"目录页"。
- **`<sessionId>.jsonl`（对话正文）**：存这个 session 的实际内容——每条消息/工具调用/压缩摘要占一行 JSON，按时间顺序 append。

为什么分开存：启动时只读轻量的 `sessions.json` 就能列出所有历史会话让你选；选中某个再去读对应 `.jsonl` 重建上下文。

被"读"的典型场景（不只是续会话）：

关键要点：

- Compaction **不读整段 `.jsonl`**——它压缩的是当前 Context 里的旧消息（内存里），只是把结果 append 回 `.jsonl`
- 磁盘上的原始记录始终保留，压缩只改变"下次从哪条之后开始读"（`firstKeptEntryId` 标记）

7.3.5 Retrieval Index

定位：为工作空间 `memory/` 下的 md 文件建立的搜索索引。**不是独立存储**——数据完全来自 md 文件，索引只是加速访问的派生物。

索引范围：

- `MEMORY.md` + `memory/**/*.md`（默认）
- 可通过 `memorySearch.extraPaths` 加入额外目录（如 Obsidian 笔记库）
- 存储在 `~/.hermes/memory/<agentId>.sqlite`

检索工具（两步走）：

- `memory_search` — 语义搜索，返回相关片段（文件路径 + 行号 + 分数）
- `memory_get` — 按路径读取指定记忆文件的完整内容

混合搜索权重（和 OpenClaw 一致）：

```Plain Text
finalScore = 0.7 × vectorScore + 0.3 × textScore
```

向量语义匹配（擅长同义词）+ BM25 关键词匹配（擅长精确 ID / 代码符号）加权合并。

关键要点：

- 只索引文件——**聊天中说过但没写进文件的内容搜不到**
- 文件变动后自动重新索引（去抖 1.5 秒）
- 支持 MMR 去重 + 时间衰减（默认半衰期 30 天）
- 常青文件（`MEMORY.md`、非日期命名的 `memory/*.md`）不受时间衰减影响

7.3.6 Honcho User Model

定位：**独立于对话历史**的动态用户画像，专门存"你是谁"——偏好、沟通风格、目标。不是文件，是一个**外部服务 / 独立模型。**

和 Bootstrap 里的 USER.md 的区别：

实测对话示例：

```Plain Text User: 你对我的 user profile 里写了什么？ Hermes: 你正在做基于 Gemma4 + SAM 的视频感知项目……
```

> 作者没有手写过这些，这是 Hermes 自己从过往对话里学到的——总结成一句话："**RL on user preferences**"。
>
> 底层是独立开源项目 `plastic-labs/honcho`，Hermes 原生集成。
>
>
> #### 7.3.7 小结：从记忆心理学看 Hermes 的升级
>
> 前面 6 个小节讲的是"存哪、谁写、怎么进 Context"的**工程视角**。换个**心理学视角**回看，这些源其实对应**人类记忆的三大分类**：
>
> - **陈述性记忆（Declarative Memory）**——"我记得什么事实"
>
>   - 对应：Bootstrap Files + memory/ 笔记 + Session Transcript + Retrieval Index
>   - 存的是事实性知识："我喜欢 TypeScript"、"这个项目用 React"、"上周讨论过 X 方案"
> - **程序性记忆（Procedural Memory）**——"我会做什么流程"
>
>   - 对应：Skill Library
>   - 存的是可执行流程："怎么做代码评审"、"怎么生成每日 AI 简报"
> - **用户建模（User Modeling）**——"我是谁"
>
>   - 对应：Honcho User Model
>   - 独立于对话历史的动态用户画像
>
> **对比 Hermes 相对 OpenClaw 的升级**：
>
> > OpenClaw 只有陈述性记忆，Hermes 把**程序性记忆（Skill）**和**用户建模（Honcho）**也补齐了——这就是它"越用越懂你"的理论基础，也是面试里最容易讲出深度的切入点。

7.4 Hermes 记忆系统 · 动态更新策略

上一节讲完了静态骨架（一个上下文窗口 + 四类数据源），这一节讲**动态血肉**——这些源怎么被**读、写、压缩、反思**。本质上：**写入 / 读取 / 压缩 / 自进化反思都是作用在那些数据源之上的更新策略**。

本节的 4 种策略：

1. **写入策略**（11.4.1）：记忆从哪来？
2. **读取策略**（11.4.2）：记忆怎么被用？
3. **压缩策略**（11.4.3）：上下文满了怎么办？—— Compaction
4. **反思策略**（11.4.4）：Hermes 的核心创新——自进化循环

7.4.1 写入策略：记忆从哪来

对照 11.3 的三类数据源 + 一个索引，每部分的写入来源和时机：

数据源 1：工作空间 md 文件

数据源 2 & 3 + 索引

两个关键的自动写入点：

1. **会话结束时的自动归档**：当用户输入 `/new` 或 `/reset`，hook 会自动把最近对话保存为 `memory/YYYY-MM-DD-slug.md` 文件 → 进而被 Retrieval Index 重新索引。
2. **文件监听同步**：用户手动编辑 `MEMORY.md` 或 `memory/` 目录下文件，chokidar 文件监听器检测到变化 → 自动分块、计算 embedding、写入索引库。

自进化循环驱动的写入将在 11.4.4 详细展开——这是 Hermes 区别于 OpenClaw 的核心。

7.4.2 读取策略：记忆怎么被用

Hermes 的读取策略分**自动注入**和**按需检索**两大类：

A. 自动注入（会话开始时无条件注入）

- **Bootstrap Files**：每次会话开始全量注入 Context Window
- **Session Transcript**：继续会话时从 JSONL 重建上下文到 Context Window
- **Honcho User Model**：会话启动时查询用户画像，摘要后注入系统提示

B. 按需检索（只有相关时才注入）

不是所有记忆都无条件塞入上下文（那样浪费 token），有两类内容是按需加载的：

1. **`memory/` 笔记**——有两条检索通道：

   - **系统自动**（Hermes 特色）：Context Engine 每轮推理前根据当前消息自动检索 `memory/`，命中才注入。对弱模型/开源本地模型特别友好，因为它们不一定会主动搜。OpenClaw 没有这一层。
   - **Agent 主动**：LLM 自己判断需要回忆时，调用 `memory_search` → `memory_get` 检索
2. **Skill Library**：先只加载各 skill 的精简描述，匹配当前任务时才加载完整 skill 内容

7.4.3 压缩策略：Compaction

> 我觉得压缩策略可以稍微背一下。因为这些都挺通用的常考的，比如面试官问你上下文满了怎么办，怎么压缩，压缩时候要保留一些什么东西。这些都是很好的经验之谈。

触发方式（和 OpenClaw 一致）：

- **自动触发**：上下文使用量达到阈值
  - 溢出恢复：模型返回 context overflow 错误 → 压缩 → 重试
  - 阈值维护：`contextTokens > contextWindow - reserveTokens` 时触发
- **手动触发**：输入 `/compress`，可带自定义指令如 `/compress 重点保留决策和待办事项`

触发公式：上下文窗口 - reserveTokensFloor（默认 20000）- softThreshold（默认 4000）。200K 模型 → 约 176K 时触发。

压缩过程：

1. **划分保留区和压缩区**：最近的 \~20,000 tokens 消息进"保留区"，原封不动；之前的旧消息进"压缩区"
2. **分块总结**：压缩区分成若干块，用模型对每块生成摘要
3. **合并摘要**：多个块摘要再合成一个连贯总摘要
4. **写入 Transcript**：JSONL 里写入一条 `compaction` 类型的条目 + `firstKeptEntryId`

压缩后上下文结构：`[压缩摘要] + [保留区原文]`。

保留策略（必须保留的内容）：

- 进行中的任务及其状态（进行中 / 阻塞 / 待办）
- 批量操作进度（"17 项中已完成 5 项"）
- 用户最后请求 + 正在做的事
- 已做出的决策及理由
- TODO、开放问题、约束条件
- 承诺的后续行动
- **所有不透明标识符**（UUID、哈希、token、API key、URL、文件名）—— 由 `identifierPolicy: "strict"` 控

制

Pre-Compaction Memory Flush：

- OpenClaw：压缩前静默运行一轮，提醒 Agent 把重要内容写入 `memory/YYYY-MM-DD.md`，用户不可见（NO_REPLY）
- Hermes：**在此之上多走一步**——不仅 flush 到 memory，还会先调用**自进化反思**，判断"这次有没有新 skill 要沉淀"（见 11.4.4）

Hermes 有自进化循环兜底主动判断关键信息是否保存，但自进化循环不是百分百捕获，**明确告诉 Hermes"记住这件事"仍然是最稳妥的做法**。

7.4.4 反思策略：自进化循环

这是 Hermes 区别于所有现有 Agent 的标志性特性，面试当作 Hermes 核心卖点必考。 前面三条（写入、读取、压缩）OpenClaw 大体都有；自进化循环是 Hermes 独创的**第四条更新策略**。

7.4.4.1 自进化循环是什么

核心思想："*like back propagation but for prompts instead of model weights*"——**像反向传播，但更新的不是模型权重，而是 Agent 的记忆、skill 和用户画像**。这里 "prompts" 是泛指所有影响 Agent 行为的非权重部分——对应后面 11.4.4.2 的三路沉淀：memory（事实性知识）、Skill Library（程序性流程）、Honcho（用户偏好）。

核心机制：每隔大约 10 次工具调用，Agent 会主动暂停，回顾刚才发生了什么、哪些步骤失败了，然后更新自身的记忆、skill 和用户画像。

技术实现上：自进化循环在工具调用达到一定次数后周期性触发。不是每次都跑——反思本身也花钱花时间，频繁反思就本末倒置。

7.4.4.2 自进化闭环的完整流程

```Plain Text
1. 用户下达任务（自然语言）
        ↓
2. Agent 执行（Think → Act 循环）
        ↓
3. 每 ~10 次 tool call，Agent 主动暂停（periodic nudge）
        ↓
4. 自问三件事：
   ① 这次学到的东西值不值得保留？
   ② 有没有失败的步骤需要记下来避免？
   ③ 成功的流程能不能抽象成可复用 skill？
        ↓
5. 值得保留 → 分三路沉淀到对应记忆层：
   ├─ 事实性知识 → memory/YYYY-MM-DD.md       （更新 Retrieval Index）
   ├─ 程序性流程 → skills/xxx/SKILL.md         （更新 Skill Library）
   └─ 用户偏好   → Honcho User Model           （更新用户画像）
```

```
        ↓
6. 下次遇到类似任务 → 直接命中 skill，跳过重新推理
```

关键观察：自进化循环本质上是**跨 3 层记忆的联合更新器**——同时更新 Retrieval Index、Skill Library、Honcho。这就是前面说的"自进化循环是作用在记忆层之上的更新策略"的具体含义。

7.4.4.3 Skill 的自我改进

Skill 不是一次生成就固化了，自进化循环会持续优化。如果 Hermes 再次执行同类任务时发现了更好的做法，会直接更新已有的 skill，而不是新建一个。关键词是 **updating，不是 creating**——只有确实是全新类别才会新建。这是防止 Skill 膨胀的核心机制之一。

7.4.4.4 自进化循环 vs OpenClaw Pre-Compaction Flush

这两个机制容易混，面试可能追问：

一句话区分：OpenClaw 是**事后归档**，Hermes 是**持续反思**。Hermes 在压缩前也会先跑一次自进化反思，所以可以理解为"自进化循环 ⊇ Pre-Compaction Flush"。

7.4.5 小结：一张动态更新全景图

把四种策略都映射到 6 层静态记忆上，得到一张 Hermes 记忆的完整图景：

7.5 Hermes Agent 架构解析

（对应第 10.4 节 OpenClaw 的 Agent 架构解析，回答"Hermes 的多 Agent 架构是什么"。因为Hermes Agent的架构和Openclaw非常像，虽然Agent架构是很重要的一部分，但这里没有什么新东西，就不赘述了）

7.5.1 概述：同样是递归 Centralized

Hermes 的 Agent 架构和 OpenClaw 一样是**递归 Centralized**：

- Parent Agent（主 Agent）对最终回复负责
- 通过 tool call 的方式 spawn subagent
- Subagent 可以继续 spawn 孙 Agent（默认深度比 OpenClaw 宽）

Hermes 的定位是 "*agent loop first with this learning element*"——loop 结构本身是常规的 Centralized，创新在于"learning element"。

7.5.2 Spawn 机制对比

关键差异：Hermes 把 ACP 做成**双向能力**——不仅能调别人，还能被别人调。这让"OpenClaw 作 orchestrator、Hermes 作 worker"的组合变得开箱即用。

7.5.3 Agent 通信机制（和 OpenClaw 一样）

通信模式：**Tool Call**（而非 Handoff）——父 Agent 保留控制权。

消息传递：**仅最终结果**，子 Agent 的 chain-of-thought 留在私有 session 里，父 Agent 只收到蒸馏后的回复。

这和 OpenClaw 第 10.4.3 节的策略完全一致——两者在"多 Agent 通信"这块没有本质差异，都是业界主流做法。

7.6 面试问题

吸收了上面的内容，你可以足够应付大部分的 Hermes 面试问题。这一小节快速总结上述知识，给出标准面试参考答案，用这些面试问题检验自己吧\~

Q1: 你看过 Hermes Agent 源码吗？简单讲讲它的架构和 OpenClaw 的区别。

看过。Hermes 是 Nous Research 开源的自主 AI Agent 平台，核心用 Python 编写。架构上和 OpenClaw 是同构的三层管道：触发层（Messages / Heartbeats / Crons / Webhooks / Hooks）→ 网关层（多渠道适配 + 路由 + 安全）→ Agent 层（Think → Act → Remember 循环）。这三层和 OpenClaw 几乎 1:1 对应。

核心区别在 Agent 层额外多了一个**自进化闭环**。OpenClaw 是纯执行型——把当下任务做好就结束。Hermes 在执行之外加了周期性反思机制，会把经验沉淀为三样东西：事实性知识写入 memory 文件、成功流程固化为 Skill 文件、用户偏好更新到 Honcho 用户画像。所以 OpenClaw 是执行型 Agent，Hermes 是学习型 Agent。

两者不是替代关系，而是搭档关系——通过 ACP 协议互相调度，OpenClaw 做 orchestrator、Hermes 做 learning worker 是社区共识的最佳实践。

Q2: Hermes 的架构是几层？每层做什么？

Hermes 的架构是主干 3 层加 1 个自进化闭环。

第一层是触发层，有 5 种信号源，包括用户消息（CLI、Telegram、Discord、Slack、WhatsApp、Email）、心跳定时唤醒、Cron 定时任务、外部 Webhook 推入，以及生命周期钩子。这一层和 OpenClaw 完全一致。

第二层是网关层，也就是 Gateway。它是一个本地长驻的进程，负责连接管理、协议转换、路由分发和安全控制。一个很重要的边界是，Gateway 不调 LLM、不执行工具，它只负责两件事："这条消息该给哪个 Agent"和"Agent 的回复该送回哪个渠道"。

第三层是 Agent 层，也就是核心推理循环：Think 推理、Act 执行工具、Remember 读写记忆，由 LLM 自主驱动决定是继续调工具还是输出最终回复。Hermes 还支持主模型加辅助模型的分工，强模型做长链推理，

弱模型做压缩摘要这种轻量任务。

除了这三层之外，Hermes 还有一个自进化闭环，这是它和 OpenClaw 最核心的区别。Agent 层在工具调用达到一定次数后会周期性触发反思，Agent 会自问三件事：学到的东西值不值得保留、失败的步骤要不要记下来避免、成功的流程能不能抽象成 skill。然后把结果分三路沉淀到 memory 文件、Skill Library 和 Honcho 用户画像，反哺下一次任务执行。

所以整体来看，本质是一个事件驱动的执行管道：事件产生、网关路由、Agent 执行、状态持久化，然后额外多了反思沉淀和反哺下一次执行这两步，最后这两步是 Hermes 独有的。

Q3: Hermes 的记忆功能是如何实现的?

Hermes 的记忆系统我会从静态和动态两个角度来讲。

静态上它有 6 层。前 4 层和 OpenClaw 是完全一样的：第一层是 Bootstrap Files，就是 AGENTS.md、SOUL.md、USER.md 这些身份文件，每次会话开始从磁盘读取注入系统提示，不受压缩影响。第二层是 Session Transcript，完整的对话历史持久化到数据库，续会话时从里面重建上下文。第三层是 Context Window，就是模型每轮推理实际看到的 token 容器，满了会触发压缩。第四层是 Retrieval Index，为 memory 文件建的向量加全文混合搜索索引，权重是 7:3，Agent 通过 memory_search 工具按需检索。

后 2 层是 Hermes 独有的。第五层是 Skill Library，这是程序性记忆，把成功完成过的流程固化成 SKILL.md 加可执行脚本，下次遇到类似任务直接调用，不用从零推理。第六层是 Honcho User Model，这是一个独立的用户画像服务，存的是你的偏好、沟通风格、目标，会话启动时查询摘要注入上下文。

前 4 层属于陈述性记忆，就是"我记得什么事实"；Skill Library 属于程序性记忆，就是"我会做什么流程"；Honcho 是用户建模，就是"你是谁"。OpenClaw 只有陈述性记忆，Hermes 把程序性记忆和用户建模也补齐了，这就是它"越用越懂你"的理论基础。

动态上有 4 种策略：写入策略包括手写、会话结束自动归档、自进化循环自动写入；读取策略分自动注入和按需检索；压缩策略就是 Compaction，把旧消息压缩成摘要、保留最近的原文；反思策略就是自进化循环，周期性触发三路沉淀。

Q4: Hermes 的自我进化功能是什么？怎么实现的?

Hermes 的自我进化功能叫自进化循环，英文叫 self-improvement loop。它就像神经网络的反向传播一样，但更新的不是模型权重，而是 Agent 的记忆、skill 和用户画像。

具体实现是这样的：Agent 在工具调用达到一定次数后会周期性触发反思，触发之后 Agent 会自问三件事：这次学到的东西值不值得保留？有没有失败的步骤需要记下来避免？成功的流程能不能抽象成可复用的 skill?

如果值得保留，就分三路沉淀：事实性知识写入 memory 文件，更新检索索引；程序性流程写成 SKILL.md，更新 Skill Library；用户偏好更新到 Honcho User Model。而且反思任务是在主回复返回给用户之后在后台执行的，不会阻塞用户交互。

和 OpenClaw 对比的话，OpenClaw 有一个 Pre-Compaction Flush 机制，但那只在压缩前触发一次，属于事后归档，而且只写 memory 文件。Hermes 的自进化循环是持续反思，同时更新 memory、skill、

Honcho 三路，而且压缩前也会额外跑一次。所以可以理解为自进化循环包含了 Pre-Compaction Flush。

另外 skill 也不是一次生成就固化了。如果 Hermes 遇到已有类似 skill 且发现更好的做法，会优先更新已有 skill 而不是新建，这也是防止 skill 库膨胀的核心机制之一。

Q5: 自进化循环会不会导致 Skill 膨胀？怎么治理？

会的，这是任何持续学习系统都绕不开的问题，学名叫 Memory Bloat。Hermes 做了三层治理。

第一层是准入门槛。自进化循环的三问里第一条就是"值不值得保留"，大多数情况会直接丢弃，不是每次反思都会产出新 skill。

第二层是合并优先。遇到类似任务的时候优先 update 已有 skill，不建新的。关键词是 updating 不是 creating。

第三层是分类组织。skill 按类别分目录，比如 software-development、research、creative、productivity 这些，检索的时候只扫相关子集，不会全量比对。

不过即便有这三层，实际使用中还是需要周期性的人工审查。社区的常见做法是每月跑一次 hermes skills audit，列出低使用频率和重复的 skill，让用户选择清理。我觉得核心思路是：持续学习系统的挑战不是"能不能记住"，而是"怎么有选择地遗忘"。

Q6: Hermes 和 OpenClaw 你会怎么选？有什么区别？

两者是同一代 Agent 产品，都是事件驱动 + 多渠道网关 + Agent Loop 的架构，三层管道几乎 1:1 对应。核心差异在设计哲学：**OpenClaw 是执行型 Agent**，强调当下把任务做好；**Hermes 是学习型 Agent**，在 OpenClaw 4 层记忆之外加了 procedural memory（Skill Library）和 Honcho 用户画像，通过自进化循环周期性反思并持久化——所以它越用越懂你。

各自强项也不同。Hermes 的优势在于自进化循环、自动 Skill 沉淀、Honcho 用户建模、模型中立对开源模型友好、同模型下更快。OpenClaw 的优势在于团队更大资源更多、更新频率更高、更稳定、生态更成熟（原生 Cursor / Claude Code 插件）、社区更大文档更全。

从 harness engineering 的角度看，OpenClaw 是最小优雅 harness，Hermes 是激进 harness——把记忆、技能、反思全做成系统能力。这也是 Hermes 在本地开源模型（Qwen、Gemma）上效果更好的原因：弱模型 + 强 harness ≈ 强模型 + 弱 harness。

实际工程里两者**不是替代关系，而是搭档关系**——通过 ACP 协议可以互相调度。推荐用法是 OpenClaw 做 orchestrator 负责接收请求和任务拆分，Hermes 做 learning worker 负责执行细分任务和沉淀 skill，这是社区共识的最佳实践。

Q7: 自进化循环里的"三件事反思"到底是怎么实现的？

这个问题可以拆成四层看：

第一层是代码调度层，决定什么时候触发反思。 `run_agent.py` 里硬编码了两个计数器，`_memory_nudge_interval = 10` 和 `_skill_nudge_interval = 10`，每次 user turn +1、每次工具调用 +1，到阈值就置 flag

然后清零。这是纯 Python 的事件驱动，不是让模型自己判断该不该停下来反思——因为让模型自省"你现在该不该自省了"本身就是不可靠的。

第二层是后台 agent 层，决定怎么反思。 触发之后，代码不会打断主对话，而是等主回复给完用户之后，在后台线程里 spawn 一个全新的 AIAgent 实例，模型、工具、对话历史快照都复制过去，只是把反思提示词作为新的 user message 追加。这个细节我觉得很关键——**反思不占主对话的 attention，也不阻塞用户**，用户感知不到，但记忆在后台持续沉淀。

第三层是提示词层，决定反思什么。 三个 review 提示词是写死的常量（`_MEMORY_REVIEW_PROMPT` / `_SKILL_REVIEW_PROMPT` / `_COMBINED_REVIEW_PROMPT`）。比如记忆反思的核心就两问："用户是否透露了关于自己的信息？""用户是否表达了对你行为方式的期望？" Skill 反思问："是否用了非平凡的方法？是否经历了试错？"并且提示词最后都明确要求——**如果不值得存，就直接说 'Nothing to save.' 然后停止**。这个"允许什么都不做"的兜底设计是防止 skill 膨胀的第一道闸。

第四层是工具持久化层。 后台 agent 如果决定保存，就调用 `memory` 工具写入 `~/.hermes/memories/MEMORY.md` 和 `USER.md`，或者调用 `skill_manage` 工具在 `~/.hermes/skills/` 下创建 `SKILL.md`。两个工具都带安全扫描防 prompt injection，因为这些内容下一次 session 会被注入系统提示词，不能被污染。下一次启动时 `prompt_builder.py` 读文件注入 context，闭环完成。

除了定时反思，系统提示词里还有常驻引导——`MEMORY_GUIDANCE` 和 `SKILLS_GUIDANCE` 两段文字在每次对话时都注入，提醒模型日常就主动保存。所以反思不完全依赖定时兜底，是"日常引导 + 定时强制复盘"双轨。

我从这个设计里学到最重要的一点是：这是典型的 harness engineering 分工——**把 LLM 不擅长的事情交给代码，把代码做不了的事情交给 LLM**。"什么时候反思"是调度问题，代码能稳定搞定；"这段对话里有没有值得沉淀的东西"是语义判断，只能交给模型。纯提示词做不到，因为模型不会自己记着"我已经聊了 10 轮了该反思了"；纯代码也做不到，因为代码没法判断一段自然语言对话里哪些是用户偏好哪些是临时需求。把两者边界切得足够清，这个系统才能跑得稳。

7.7 Hermes 源代码

https://github.com/nousresearch/hermes-agent

7.8 视频讲解

https://www.bilibili.com/video/BV1J9o4BnE2n/?vd_source=144bec9c3f54e465073138bed788be1b

---

8 开源框架解读 - ClaudeCode

> 26年三月底，Claude Code源码泄露。一经泄露，面试就成近期必考的问题。有同学反映面试官直接问他，Claude Code代码泄露2天了？你还没看吗？所以对于这一部分，近期是非常重要的考点。

> 另外其实除开最近热点事件外，学习ClaudeCode本身就是非常有帮助的，他本质是一个CodingAgent。我们学习Agent可以分成三步：
>
> 1. 掌握了Agent的基础知识
> 2. 可以分析典型的Agent架构，比如笔记已经有的Openclaw架构分析，本期的ClaudeCode架构以及马上会出的字节的Deerflow2.0的Agent架构，建立审美。
> 3. 设计一个自己的Agent项目架构，写文档，用Harness工程来完成。
>
> 有了项目后，我们就可以面试了，所以学习经典的Agent架构是很有帮助的，本节就一起来学习这个经典的CodingAgent - Claude Code

8.1 事件概述

> 这节就讲下事件起因，大家看看就好，了解事件的来龙去脉。算是近期（2026年4月）的一个谈资。

8.1.1 怎么泄漏的

2026 年 3 月底，安全研究者 Chowan Chow 发现 Anthropic 发布在 npm 上的 `@anthropic-ai/claude-code@2.1.88` 包中意外包含了一个约 60MB 的 **source map 文件**（`.map`），这个调试文件直接映射回了 Claude Code 的完整未压缩 TypeScript 源码，甚至可以通过 Anthropic 自己的 R2 云存储桶以 zip 包形式直接下载。一共 **1,902 个文件、约 51.2 万行 TypeScript 代码**就此公之于众。

根本原因是一个低级的配置遗漏——开发团队忘记在 `.npmignore` 中添加 `*.map`。Claude Code 的创建者 Boris Cherny 确认这是人为失误，与打包工具 Bun 的 bug 无关。讽刺的是，这已经是**第二次**发生同样的错误。Anthropic 随后迅速发起 DMCA 下架通知，但代码早已被广泛分发和归档，无法收回。需要明确的是，此次泄漏仅涉及产品代码（Agent 框架），不包含任何客户数据、API 密钥或模型权重。

8.1.2 泄漏揭示了什么

泄漏的代码完整揭示了 Claude Code 的内部架构。它本质上是一个用 TypeScript 编写的 **AI Coding Agent**，核心是经典的**查询循环（Query Loop）+ 工具系统**架构，外加多层记忆系统、上下文压缩算法、权限与安全机制等工程组件。本文后续章节将从以下几个维度逐一深入分析：

- **核心架构**：Query Loop、工具系统、System Prompt 分层设计、终端 UI 引擎
- **记忆与上下文**：多层记忆架构、5 级上下文压缩（Compaction）策略
- **安全与权限**：权限模式、YOLO Classifier、Prompt Injection 防御
- **多 Agent 协作**：子代理模型（Fork / Teammate / Work Tree）、Coordinator Mode

此外，代码中包含 **44 个编译时 Feature Flag**，暴露了大量尚未公开的内容：

- **未发布功能**：24/7 自主守护进程 Kairos、记忆整合的 Dream System、30 分钟深度规划 Ultra Plan、反蒸馏防御 Anti-Distillation、卧底模式 Undercover Mode 等

- **未发布模型**：Capybara/Mythos（全新产品线）、Numbat（完全未知）、Opus 4.7 / 4.8 等下一代模型代号

8.1.3 社区反应

> 这里我们在笔记的 Harness Engineering工程中，来讲解这个Claw Code开源项目。学习这个项目，可以进一步帮我理解Harness工程（无疑这个也是26年的高频考点）。你自己其实也可以更深入学习这个项目，自己仿照这个来做一个Claude Code，是一个很好的学习Claude Code， Agent的机会，同时也是一个练手Harness工程的机会。具体请看笔记的Harness工程的Claw Code讲解。

泄漏发生后数小时内，社区迅速行动：有开发者连夜将核心功能从 TypeScript 移植到 Python（以规避 DMCA 版权风波），推送到 GitHub 后迅速成为有记录以来**最快达到 10 万星的仓库（Claw Code）**，随后又被用 Rust 重写了一遍。甚至有人用 Claude Code 本身生成了一个 PR，试图将全部泄漏源码提交回 Anthropic 的官方仓库。

一个讽刺性的版权争议也随之浮现：如果 Anthropic 声称的"80% 的代码由 AI 编写"为真，那么根据美国现行法律，AI 生成的内容不受版权保护——Anthropic 理论上没有 DMCA 下架的法律基础。

8.2 整体架构

掌握整体ClaudeCode的工作流程。在我们学习后面小节的时候，会有一个更清楚的理解。比如后面的

4.4.3的记忆功能模块：**主要**发生 图中第四步，不过 第三步 上下文装配也会涉及。

4.4.4的Agent架构: 发生在第四步：Agentic loop或者说React，也是系统的核心。

4.4.5 权限系统和安全审查：发生在第四步的工具调用整个步骤

一个对话 Agent 的完整流程，本质上就是回答两件事：**一次用户输入要按什么顺序流过哪些模块** 与 **跨轮次要持久维护哪些状态**。结合上面这张 Claude Code 工作流图，从上到下五个阶段依次是：

① 会话路由

按 sessionId 找到（或新建）一个会话上下文，把这次请求挂到对应的会话状态上——包括消息历史、累计用量、权限模式、记忆等。新会话就 bootstrap 一份默认状态，老会话从持久化的 transcript 里恢复。这一步本质就是"先把你是谁、之前聊过啥、能用哪些权限"这些信息准备好。

② 输入处理

把多种来源（UI、CLI、SDK）的原始输入**归一化**成统一的消息格式，同时做意图分流：解析 slash command（`/compact`、`/model` …）、@ 文件引用、粘贴的图片/附件、命令输出等。本地命令能直接处理的就短路掉，不浪费一次模型调用。

③ 上下文装配

把这一轮要喂给模型的"完整提示"拼出来：System Prompt（身份、规则、当前模式）+ 工具定义（哪些 tool 可以调）+ 用户/项目级记忆（CLAUDE.md、偏好）+ 环境信息（cwd、git status、相关文件快照）+ 历史消息。重点是这些都是**动态装配**的，会根据当前模型、权限、启用的插件实时变化。

④ Agentic Loop（核心循环）

> 整个系统的核心，也可以说是React的实现方法，面试考点如：React如何实现，就可以借鉴这里。

这是图里红框那一块，也是 Agent 之所以叫 Agent 的地方。一个 `while (true)`，每轮四件事：

1. **上下文治理**：进循环先看 token 预算，超阈值就压缩——把旧消息摘要化（auto-compact），腾出窗口继续聊。
2. **调用模型**：流式发请求，边收边把 partial 结果推给 UI。
3. **判定是否调工具**：看返回的 `stop_reason`。模型说 "end_turn" 就跳出循环；说要 "tool_use" 就进下一步。这就是 ReAct 的 Thought → Action 决策点。
4. **执行工具 → 结果回填 → 回到第 1 步**：按模型给的 tool_use 调度工具（文件读写、Bash、MCP、子 Agent），先过一遍权限校验，结果作为 tool_result 追加到消息里，然后继续下一轮。

退出条件：模型主动 end_turn / 达到 maxTurns / 预算耗尽 / 用户中断 / 不可恢复错误。

⑤ 回合收尾

输出最终 assistant 消息；汇总这一轮的 usage、cost、耗时、停止原因；把 transcript 落盘；触发 Stop Hook 和通知。本质就是把第 ④ 步循环里产生的副作用收拢、上报、持久化。

横切关注点（图例里的三条虚线）

> 这部分的内容不属于输入-输出这个工作流，是伴随在整个系统中的其它功能。

- **用量统计**：贯穿 ②③④⑤，每次调模型/调工具都在累加，最后在收尾结算。
- **Hook 钩子**：PreToolUse / PostToolUse / Stop / SessionStart 等切面，在输入处理、工具执行前后、收尾几个点位被触发，用户可以插自定义逻辑。
- **落盘与通知**：transcript 持久化、文件快照、桌面通知，都是异步副作用，不阻塞主循环。

8.2.1 面试真题讲解

1. 如何让你从0设计一个对话Agent的流程，核心状态有哪些?

(选自腾讯2026年4月暑期实习大模型开发)

> 其实就是理解了上述整体架构后，你可以套用CC的架构来讲。当然你也可以套用Openclaw的架构来讲。Openclaw也用了React, 上面有gateway。总之你要用一个完整成熟的架构来回答，设计的流程肯定是比较成熟完整的。

我会把一个对话 Agent 拆成五步流水线：**会话路由 → 输入处理 → 上下文装配 → Agentic Loop → 回合收尾。**

会话路由：按 sessionId 把请求挂到对应会话上，恢复历史消息、用量、权限模式、记忆。

输入处理：把多来源的输入归一化成统一消息，解析 slash command、@ 引用、附件，本地能处理的直接短路掉。

上下文装配：动态拼出这一轮要喂给模型的完整提示——System Prompt + 工具定义 + 长期记忆 + 环境上下文 + 历史消息。

Agentic Loop：一个核心循环，每轮先做上下文裁剪/压缩，再调模型，看 stop_reason 决定退出还是调工具；调工具就先过权限校验、再执行、把结果回填进消息继续下一轮。退出条件是 end_turn / 达到轮数上限 / 预算耗尽 / 用户中断 / 错误。

回合收尾：输出最终回复，汇总 usage 和 cost，落盘 transcript，触发 Stop Hook等操作。

1. ReAct机制在生产环境里该怎么实现?

(选自腾讯2026年4月暑期实习大模型开发)

> 不论是Openclaw还是ClaudeCode，里面都用的React，大概的实现方法也类似，我们把它抽象成这样。这个图可以作为React的标准工程实现流程。

生产环境里我会把 ReAct 设置成一个循环流程：

1.先装配上下文

每一轮模型调用前，系统要把用户输入、历史消息、System Prompt、工具定义、权限信息、环境状态一起组装好，形成当前这一轮的完整上下文。

2.让模型决定下一步

模型不是直接随便输出文本，而是要么返回最终答案，要么返回一个结构化的工具调用请求，比如要调用哪个工具、参数是什么、调用目的是什么。

这一步对应 ReAct 里的 Reasoning / Action 决策。

3.由系统执行工具，而不是模型自己执行

模型只能“申请调用工具”，真正执行的是外部 harness / runtime。

执行前要做权限检查、参数校验、风险判断、超时控制、重试控制。比如文件写入、数据库查询、外部 API、Shell 命令，都不能只靠模型一句话就执行。

4.把工具结果回填给模型

工具执行完以后，结果会作为 observation / tool_result 加回对话历史里，再进入下一轮模型调用。模型基于新结果继续判断：是继续调工具，还是给用户最终回答。

5.设置退出条件

不能让 Agent 无限循环。生产里必须有明确的退出条件，比如模型返回 final answer、达到最大轮数、token 预算耗尽、工具失败次数过多、用户中断、权限拒绝等。

8.3 记忆功能

> 其实大家看各种Agent的开源项目，每个Agent项目，我们都需要关注最重要的几个点，也是Agent项目的核心：比如记忆，架构，Agent的交互对吧。所以不管看哪个Agent项目，都该特别解析关注这个记忆模块。我们在笔记的openclaw部分已经解析了小龙虾的记忆功能，今天我们来看看ClaudeCode的记忆功能，二者对比，体会成熟的Agent项目的记忆功能背后的设计哲学。

学习一个 Agent 的记忆模块，可以重点从以下几个问题入手：

- **它由哪些部分组成?**

- **它是如何运行的?** 例如加载、写入、压缩、恢复等流程。

- **各部分分别负责什么功能?**

- **信息以什么格式存储?** 例如 `.md`、`.jsonl`、内存对象等。

- **这些信息存在哪里?** 是在内存中，还是落到磁盘文件里?

- **整体设计理念是什么?** 它是为了让模型更聪明，还是为了让系统更可恢复、更可审计?

教大家一个小技巧，如果你没有设计Agent的一些思想，那么在你的Agent系统（简历上），加上这么一条：xxx。基本照抄ClaudeCode的设计理念，写在简历上，面试的介绍项目大概讲一下，面试官大概率会问设计的细节，然后你就可以照着本节的笔记去讲解。本节笔记也会引入真实的面试真题，感受面试官对于你的记忆系统是怎么提问的？可以用这些面试真题来体会自己有没有真正理解。当然如果你更有想法，在消化了本节内容后，可以结合自己项目和思想，去改进你的记忆系统（但前提是你消化了这个业内的标杆实现，如果你没有消化就自己设计，那你还不如照抄呢）

从宏观上，我们从这个图来了解CC的存储系统。其实我翻了大量的参考资料，很多资料的解析，其实只解析了CC的左半边部分，甚至只解析了左半边记忆系统里面的某一层。但是我这里加入了右边，Transcript，因为不讲这个，讲不清楚ClaudeCode是如何恢复对话，做上下文重建的（这也是面试问题，另外你要是用这一套记忆系统放在你的项目，你必须从各个角度理解透彻，因为面试官会从各个角度来提问）

从宏观上看，Claude Code 的上下文持久化体系可以分成两大部分：**Memory 记忆系统** 和 **Transcript 会话账本**。如下图：

- 左侧的 **Memory 记忆系统**，主要负责保存会影响模型后续理解和行动的语义材料。它包括 CLAUDE.md 族、对话上下文、Session Memory 和 Memdir 持久记忆。

- 右侧的 **Transcript**，更像完整的会话流水账，用来记录用户消息、助手回复、工具调用和工具结果。它主要服务于恢复、回放、审计和会话续接。

下面4.4.3.1 就是对该图左边的这一部分进行详细讲解。

同时附上一张完整的图片，包含了四层记忆和压缩功能，CC整体的运作图。

8.3.1 四层记忆

1. **五层指令文件（CLAUDE.md 族）—— 人写的规则**

> 指令文件其实在CC的核心用法就讲过，这里Claude.md 系列文件的维护，需要人工维护的。

CLAUDE.md 族采用五层记忆体系：

- **企业级**：`/etc/claude-code/CLAUDE.md`

管理者是 IT 部门 / 管理员，作用范围是全公司开发者。特点是强制执行，个人无法覆盖。

- **用户级**：`~/.claude/CLAUDE.md`

管理者是开发者个人，作用范围是该用户所有项目。特点是跨项目的个人偏好。

- **项目级**：`./CLAUDE.md`

管理者是团队 / Tech Lead，作用范围是当前项目所有成员。特点是提交至 Git，团队共享。

- **规则级**：`.claude/rules/*.md`

管理者是团队 / 个人，作用范围由 Glob 模式限定。特点是模块化、条件化触发。

- **本地级**：`./CLAUDE.local.md`

管理者是开发者个人，作用范围仅当前工作区。特点是不提交 Git，通常自动加入 `.gitignore`。

从上至下，适用范围逐渐收窄，内容颗粒度日益精细，管理灵活性也随之递增。

你可能会问，当这五层内容，冲突了怎么办？

Claude Code 不会像配置系统那样用下层文件把上层文件整段替换掉，而是把企业级、用户级、项目级、规则级、本地级这些 `CLAUDE.md` **内容一起读出来，按一定顺序合并进同一个上下文里，让模型同时看到多层规则。**发生冲突时，普通规则主要靠**模型根据“更具体、更贴近当前任务、位置更靠后/更新”的信息来判断该听谁**，比如用户级说 4 空格、项目级说 2 空格，项目级更具体，通常会被采纳。但这仍是认知层的软优先级，不是代码级覆盖；真正的硬约束要靠权限、Hooks、Managed Settings 这类系统机制保证。

这里是一个面试问题考点，请看 腾讯AI Agent暑期实习，面试问题2：如果用户和AI对话生成问题本身有冲突，系统应该怎么处理

> 更多参考资料：讲解CC的MD的原则，可以查看 《ClaudeCode Harness之道》（黄佳著）第2章 2.2小节（书籍资源要你自己找哈）。

1. **对话上下文—— 短期记忆**

> 就是大模型多轮对话的数组，每个Agent都有。

当前对话的消息数组，所有 LLM 应用都有。Claude Code 的特别之处在于配套了一套多层压缩策略来管理它（见 3.1存储时，由后台 agent 更新），主要服务本次 compact。

---

1. **持久记忆（Memdir）—— 跨会话的长期记忆**

最核心的长期记忆系统，位于 `~/.claude/memory/`。存 4 类信息：

- `user`：用户角色、偏好、技能水平。例："偏好函数式风格"、"是后端工程师"
- `feedback`：用户对 AI 的纠正和确认。例："上次用 forEach 被要求改成 map"
- `project`：项目目标、决策、截止日期。例："Q3 要迁移到 TypeScript"
- `reference`：外部系统指针。例："Bug tracker 在 Linear：xxx"

注意这里**故意不存代码事实**（如"函数 X 在第 30 行"），因为代码会变但记忆不会自动更新，过时的代码记忆会变成危险的误导。代码相关的事实永远用 grep/glob 实时搜索，只有人的偏好和判断才值得持久化。

存储结构：

```Plain Text
~/.claude/memory/
├── MEMORY.md ← 索引文件（每次会话自动加载）
├── user_role.md ← 主题文件：用户角色
├── feedback_testing.md ← 主题文件：测试相关反馈
├── project_migration.md ← 主题文件：迁移计划
└── reference_tracker.md ← 主题文件：外部系统指针
```

> `MEMORY.md` 是**索引**，不存记忆内容本身——每行只有一个指向主题文件的链接和一句话摘要（≤150 字符），格式如 `- [用户角色](user_role.md) — 后端工程师，偏好函数式风格`。上限 200 行 / 25KB。
>
> 实际的记忆内容存在各个**主题文件**中，按语义主题命名（不是固定 4 个文件），每个文件带 YAML frontmatter 声明 name、description 和 type（对应上面的 user/feedback/project/reference 四种类型）。查询时系统通过扫描 frontmatter 来判断哪些文件与当前对话相关。
>
> **团队记忆：** Memdir 下有 `team/` 子目录，支持跨用户共享的项目记忆。会话开始时自动同步。敏感信息（API 密钥等）禁止存入。Feature flag `TEAMMEM` 控制。
>
> **写入时机**：用户明确说“记住”时立即写；或者一轮完整回答结束后，也就是模型给出最终回答、没有继续调用工具时，系统会触发后台 **extraction agent**。它会看最近一段对话里有没有值得跨会话保存的内容，也就是 durable memory。
>
> **召回策略：**

Claude Code 的长期记忆召回是这样的：**每个用户回合后台发起一次检索**，由一个**更便宜的小模型**（Sonnet）拿用户这句话去和每条记忆的一行 description 做语义相关性匹配，最多挑出 5 条"确定有用"的注入上下文（常常是 0 条，挑不准就不选）。注入前还有多道闸门——**单字输入、超会话预算、已注入过或主模型已读过的**，都会被跳过，所以不是每回合雷打不动灌 5 条，而是按相关性 + 去重 + 预算动态决定。此外 MEMORY.md 索引始终常驻上下文，主模型也能按需主动去读对应记忆文件。

> 召回策略这里，[202605 安克暑期实习](#doxcnjD990ppxY4KqWAMfonaEbd) 面经中有问到。

> 想深入学习 Memdir? 其实对于这个 Memdir，它的设计的理念，层级，可以看[这里](https://lintsinghua.github.io/#ch06)

8.3.2 Transcript会话账本

通常来说，很多书籍，文章解析ClaudeCode的时候，讲记忆功能，只会讲4.4.4.1的四层记忆，不会讲这个一小节。但是为什么要讲，是因为如果你要把这一套记忆系统，用在你的项目，和面试官聊的时候，你必须理解这一整套，你的Agent存了哪些东西，面试官可能会问你，会话是如何重建的，恢复的？你必须要理解这一层的存储，因为会话重建，回滚的方法，通常不是通过4.4.3.1的记忆实现的。而是通过专门的会话账本存储的。学习这一小节，有助于理解整个Agent体系的全局架构，面试官问你：**你的系统存了哪些东西？你如何恢复上下文的问题？本节会发挥很大的帮助。**

大家可以看看Keep公司暑期实习面经，通过这个面经，你可以感受到，你学习这个小节的重要性。**从面试官主动问的概率上，该小节被问到的概率不大**，这个小节的核心思想是给大家建立一个意识，日志如何恢复的。**当你在用这一套包装简历的时候，那你务必想清楚这一小节**，请你也想清楚你的系统里：

Transcript存了什么？

当刷新恢复上下文时怎么做的？

当返回到某个Checkpoint的时候怎么做的？

这些具体的细节还是依赖于大家自己，如果你要用这一套包装，最好能做一下，不能做也想清楚这些问题。我也给到大家对应的参考资料，大家可以借鉴LangGraph的恢复功能弄清这些问题，本节只介绍CC大概的思路（具体实现各个产品不一样，但宏观思路是一样的：记忆系统让Agent聪明+日志系统让Agent重建会话）

功能介绍

Transcript 会话账本是 Claude Code 的**完整会话记录层**，用来保存一次会话中**真实发生过的消息与执行过程**。它**记录用户输入、助手回复、工具调用、工具返回结果、compact boundary、错误与恢复痕迹等内容**。它的核心功能不是“提炼记忆”，而是保留原始事实：让系统之后可以**恢复上下文、续接会话、回放执行轨迹**，并在出问题时用于审计和排查。

写入时机

写入时机是在会话运行过程中持续发生的。在 query loop 里收到新的用户消息、助手消息、工具结果或 compact boundary 后，会调用把这些内容写入 Transcript。也就是说，Transcript 不是最后才生成的总结，而是随着会话一步步追加的执行账本

存储格式

JSONL。每一行都是一条结构化记录，例如一条用户消息、一条模型回复、一条工具调用结果，或者一条系统状态记录。相比 Markdown，JSONL 更适合机器读取、追加写入和恢复会话。

恢复功能

恢复功能指的是 Claude Code 可以根据之前保存下来的会话记录，把一个旧会话重新接起来继续工作。它依赖的不是模型“凭空记得”，而是读取之前的 Transcript 会话账本，把用户消息、模型回复、工具结果、压缩边界和相关状态重新整理成可继续对话的上下文。这样用户下次打开同一个会话时，系统不需要从零开始，而是能知道之前聊到哪里、做过什么、哪些工具已经返回过结果、当前任务处在什么阶段。

重放功能

重放功能指的是系统可以根据 Transcript 中按时间保存的记录，回看或复现一次会话中发生过的过程。因为 Transcript 记录的是完整流水，而不是最终总结，所以它可以展示用户什么时候说了什么、模型什么时候做了什么决定、调用了哪些工具、工具返回了什么、在哪个位置发生了中断或压缩。重放的重点不是让模型重新推理一遍，而是让人或系统沿着已有记录理解“当时到底发生了什么”。

这一小节的内容，也可以参考 [虾皮2026暑期实习面试](#doxcnwOraQgBcWXd4W2Vwyy715c)，有较多关于重建的讨论。

同时，在PI agent的讲解章节，关于[会话存储](#doxcnqMk6uP6JQFIxOpwOMhK4Lc)，Session机制相关内容更详细，也可以参考这里。

8.3.3 上下文组装

每轮对话开始时，系统从上述 4 种存储中**组装完整的上下文**注入 system prompt。

指令文件的加载

所有层级的 CLAUDE.md **都会被拼接在一起**注入，不会互相覆盖。但拼接顺序有讲究——源码注释写道：*"Files are loaded in reverse order of priority, i.e. the latest files are highest priority with the model paying more attention to them."* 这利用了 LLM 的一个已知特性：**模型对 prompt 末尾的内容天然给予更高注意力**（recency bias）。因此排在后面的文件在实践中"优先级更高"。

加载顺序（排在越后面，模型越关注）：管理员全局 → 用户全局 → 项目级 → 本地私有。举例：如果管理员规则说"用 npm"，但项目级 CLAUDE.md 说"用 pnpm"，模型会倾向于遵循后者。

持久记忆的加载

1. `MEMORY.md` 前 200 行**每次会话自动加载**到 system prompt
2. 用 **Sonnet**（小模型，不是主模型 Opus）扫描所有记忆文件的标题和 frontmatter 描述，选出**最多 5 条**最相关的，作为 `relevant_memories` 附件注入上下文
3. 排除本次会话已召回过的记忆（避免重复）
4. 超过 1 天的记忆自动加**过期警告**：*"这条记忆是 N 天前的... 关于代码行为的声明可能已过时，请先验证。"*
5. 会话总预算 60KB

值得一提的是，记忆召回没有使用 RAG（检索增强生成）。Boris 在播客中透露，早期版本曾尝试过本地向量数据库 + 云端 embedding 的 RAG 方案，但遇到了多个问题：代码修改后索引不同步（新写的函数搜不到）、索引的权限管控复杂（谁能访问、如何防止越权）。最终发现**agentic search（模型自主调用 glob + grep）全面优于 RAG**——Boris 的原话是 *"agentic search just outperformed everything, and when I say agentic search, this is a fancy word for glob and grep."*

8.3.4 压缩

> 压缩是Agent里面的一个核心技巧。我觉得这里面的内容相当复杂，作者在这里也重构过几次，如果你看到的内容和视频有不一样的地方，说明内容重构过了。我也参考了好几个资料（我会附在本节参考资料里），但是其实这些参考资料里讲的都不太一样。我重构的思路，加入了更多的图片讲解，同时我避免讲得过于复杂，因为里面太深奥了，如果写得太复杂反而会弄晕。我的总结是针对于面试问题的，我希望本章内容足够回答面试问题，我也会穿插很多面试真题在其中。如果你想更深入研究，可以去看我给出的参考资料（当然我给的参考资料不同参考资料对这一块讲解也有所不同，我认为不用深究，理解原理知道如何运用到自己项目就好）

对话越来越长，上下文窗口终会被填满。Claude Code 设计了一套**四层压缩流水线**，在每轮查询循环中按固定顺序依次执行，每层有独立的触发条件——不是每层每次都触发，而是逐层检查、按需执行，越往后越激进。

这一张图比较清晰地展示了本小节要讲的全部内容，后面会详细展开，重点关注：

压缩分成4个层级，从弱到强。

每层压缩的触发条件，同步还是异步，何时触发，怎么触发的？

最后一层有一个熔断机制：避免死循环。

8.3.4.1 层一：Snip裁剪

是什么？

Snip 是最轻量的清理方式，主要用于清除已经不再需要的旧工具结果，比如 Read、Bash、Grep 等输出。它不调用 LLM，只把大段工具结果替换成简短标记，从而释放 token。

如何触发？

自动触发。系统根据工具结果数量、缓存状态、时间间隔或 token 阈值，自动清理旧工具结果，不调用 LLM。用户无法通过提

示词精准指定删除某条工具结果。

实现原理

Snip 不删除整条消息，而是保留消息结构，系统将其中的工具调用结果替换为简短的标记文本（如 `[Old tool result content cleared]`），从而释放令牌空间。这样既能减少 token，又不会破坏 tool_use 和 tool_result 的对应关系。

同步/异步？

同步执行，成本接近 0。

8.3.4.2 层二：MicroCompact（微压缩）

是什么？

MicroCompact 是基于时间触发的大规模工具结果清理。当检测到距离上一次助手消息的时间间隔超过配置阈值时，意味着服务端缓存已过期，此时无论内容多么重要，全量重写都无法避免 -- 既然如此，不如在请求之前主动清除旧的工具结果，减小重写负载。

如何触发？

自动触发，常见条件是缓存过期、时间间隔达到阈值，或请求处理时发现旧工具结果已经不适合继续保留。在你发送请求后，CC 处理你的任务前，如果检测到条件允许，自己就做了微压缩。

实现原理

系统按工具类型白名单筛选可压缩内容，例如 Read、Bash、Grep、Glob、WebFetch 等。通常会保留最近 N 个工具结果，把更早的结果替换为清除标记。

同步/异步？

同步执行，发生在请求处理链路中，发起模型请求前完成。

8.3.4.3 层三：Collapse（折叠）

是什么？

Collapse 是上下文重构级压缩。当启用 Context Collapse 特性时，系统会在上下文使用率达到 90% 时开始提交（commit）压缩操作，在 95% 时阻止新的 spawn。这一级别的设计理念是将上下文管理从"被动压缩"转变为"主动重构"。Collapse 模式会抑制自动压缩的触发，因为两者在 93% 的临界点会产生竞争。Collapse 作为更精细的上下文管理系统，拥有更高的优先级。

如何触发？

自动触发，主要由上下文压力驱动。大致在上下文使用率约 90% 时开始 commit 折叠，在约 95% 时进入 blocking，阻止继续扩张或创建新的子任务。

实现原理

> `Collapse` 会选择相对稳定、低风险的历史片段，让专门的 `context-collapse` 子 `agent` 生成局部摘要。之后请求模型时，系统不直接发送完整历史，而是投影成类似：
>
> ```Plain Text
> 原始历史：A B C D E F
> 折叠视图：A B <collapsed summary of C-D> E F
> ```

原始消息仍然保留在 transcript 中，折叠摘要作为提交记录保存，可以跨轮恢复。

同步/异步?

> 也就是在真正执行的时候，他还是同步阻塞的，但是可以异步准备一下它的摘要。比如说当信息变更成A,B,C,D的时候，已经开始总结C,D的summary了，E,F同时继续，这是异步的。但是真正压缩的时候，将C,D替换成summary的时候，是同步的。

混合模式：局部摘要可以提前异步准备；真正发请求前，系统同步应用折叠视图。

8.3.4.4 层四：AutoCompact（自动压缩）

是什么?

AutoCompact 是最终兜底机制。当上下文接近上限时，系统自动触发压缩，避免请求因为上下文过长失败。它的成本最高，因为通常需要 LLM 参与摘要。

如何触发?

自动触发。典型阈值是 effective context window - 13K tokens，约等于上下文使用率 93%。另外，如果模型返回 context overflow / prompt too long，也可能触发恢复性压缩。

实现原理

触发后有两个子策略，**先尝试轻量的，不行再用重量的**：

1. **Session Memory Compact（轻量）**：用已经由后台代理提取好的 Session Memory 文件作为摘要，保留最近 **10K\~40K tokens** 的原始对话（至少保留 5 条含文本的消息）。**不需要额外的 LLM 调用**，所以很便宜。
2. **Full LLM Compact（重量）**：把整段对话发给 LLM 做结构化摘要。摘要输出上限 **20K tokens**，包含 9 个固定段落：

- 核心请求
- 关键概念
- 涉及文件列表
- 代码错误与修复
- 问题解决过程

- 所有用户消息
- 待办事项
- 当前工作
- 下一步计划 3. 压缩后自动恢复文件上下文：压缩会丢弃旧消息，但会话期间所有被读过、写过、编辑过的文件都记录在 `readFileState` 缓存中（包括 agent 调用 FileRead/FileEdit/FileWrite/Bash 工具操作的文件，以及 memdir 系统和 Sonnet 预取自动加载的记忆文件）。压缩完成后，系统会从这个缓存中排除 CLAUDE.md 和 Plan 文件（它们有各自的恢复机制），再按最近访问时间排序，取最多 5 个文件从磁盘重新读取（不用旧缓存，确保内容是最新的），以每文件 ≤5K tokens、总计 ≤50K tokens 的预算注入到压缩后的上下文中，让模型不需要在下一轮重新读取这些文件就能继续工作。如果会话中没有操作过任何文件，则不会恢复。 4. **防御机制**：连续失败 **3 次**后触发熔断，不再尝试。如果压缩调用本身也报 prompt-too-long，会从最旧的消息开始截断，最多重试 **3 次**。

为什么建议主动 `/compact`

社区分析源码后的一个关键建议是：**主动使用 `/compact`，不要等系统自动压缩**（出自 "I Read 512,000 Lines of Leaked Claude Code" 视频）。原因很简单——自动压缩触发时，你无法控制保留什么，系统按自己的 9 段模板做摘要，可能会丢掉你认为重要的上下文。而手动 `/compact` 可以附带指令，比如 `/compact 重点保留决策和待办事项`，让 LLM 知道什么该保留。Boris 在播客中也提到自动压缩是实现"本质上无限上下文"（essentially infinite context）的核心手段，但这更多是系统兜底，用户主动管理上下文效果会更好。

8.3.4.5 和Openclaw的对比

> 这里我们不妨把两个架构的压缩策略对比。

OpenClaw只有一级压缩，但做得更精细：保留最近 \~20K tokens 原文不动，只压缩更早的旧消息，旧消息按 token 数分成多个块逐块总结，再合并成一个连贯摘要。压缩前还会静默运行一轮，提醒 Agent 把重要内容写入记忆文件（Pre-Compaction Memory Flush），防止关键信息随压缩丢失。压缩后结构：`[合并摘要] + [最近 ~20K tokens 原文]`。

Claude Code 则是**两级梯度设计**：轻量级的 Session Memory Compact 效果类似 OpenClaw（摘要 + 保留最近原文），但摘要不是现场生成的，而是后台子代理提前维护好的，所以不需要额外 LLM 调用、更省成本。只有轻量级扛不住时才上 Full LLM Compact——这一步更激进，把全部对话压缩成结构化摘要，旧消息全部丢弃。

核心差异：OpenClaw 一步到位但精细（分块 + 合并 + 预刷记忆），Claude Code 分两步走（先便宜后贵），用梯度换成本效率。

下图是Claude Code 的压缩策略 vs Openclaw的压缩策略

8.3.4.6 我的Agent需要这么复杂吗?

大概率不需要。ClaudeCode面向全球规模生产流量，很多设计，是为了处理极端情况。

比如第二层微压缩，主要为了处理的是提示词缓存，这个和Anthropic的推理端缓存有关，我们大概率不用做。一般建议做的：工具要清理 + 失败停止重试+你的压缩过程保留哪些信息。

另外我分析了两个同学的面经，他们和面试官讲压缩的时候是异步压缩的，结果被面试官追问细节答不上来。其实根据我们小节讲的，在真正压缩的时候，都是同步的。但是压缩的时候依赖于一些异步执行（比如实时总结中间过程）的结果。这里注意一下。所以你最好别把你的项目压缩过程设计成异步的，弄得比CC还复杂，不想清楚面试还爱翻车。

8.3.4.7 面试真题

字节一面真题：上下文压缩机制

真题来源：2026年10月字节一面 · 第5组原题

1. 介绍一下你了解的 Claude Code 上下文压缩机制。

💡 答题思路

CC的4层压缩与案例，笔记里有专题视频讲解，包括也给到了参考链接（一本可以在线看的书籍），讲了cc的四层压缩原理。注意笔记可能总结自5月，不过我查了CC本身也没开源，官方也没有描述，这个答案是4月份从他们泄露的源码中总结的，现在cc的真实实现可能会有所不同，不过答题思路，思想应该是类似，也值得我们学习。

🎤 参考答案

我理解 Claude Code 的上下文管理，不是窗口快满时统一做一次摘要，而是先清理低价值内容，再逐步使用更重的压缩手段。**可以分成 Snip、MicroCompact、Collapse 和 AutoCompact 四层按需机制。** 它们有各自的触发条件，不是每轮都把四层执行一遍；其中 Collapse 还取决于相关特性是否启用，不能把这些实现细节当成所有版本固定不变的行为。

第一层是 Snip，轻量裁剪。 它不调用 LLM，主要把已经不再需要的大段旧工具结果替换为简短标记，同时保留消息结构，以及工具调用和工具结果之间的对应关系。这样能先以很低的成本释放上下文空间。

第二层是 MicroCompact，微压缩。 它结合时间间隔、缓存状态等条件，在发起模型请求前，对符合条件的旧工具结果进一步清理，通常保留近期结果。这里不仅是在节省 token，也考虑了提示词缓存：当缓存已经过期、请求本来就需要重新处理时，清理旧结果可以减小后续负载。这一过程在请求链路中同步完成。

第三层是 Collapse，局部折叠。 在启用相关特性的情况下，它选择相对稳定、低风险的历史片段，交给专门的子 Agent 生成局部摘要。比如历史是 A、B、C、D、E、F，发送给模型时可以变成 A、B、C—D 的摘要、E、F。原始记录仍保留在 transcript 中，改变的是本轮模型看到的上下文视图。摘要可以提前异步准备，但真正发请求前，要同步应用折叠结果；上下文压力过高时，还会限制继续扩张任务。

第四层是 AutoCompact，强力兜底。 当上下文接近预算上限，或者请求出现上下文溢出时，系统尝试进一步压缩。CC有两种压缩策略：优先使用后台已经维护好的 Session Memory 作为摘要，并保留近期原文，避

免在触发时再额外调用一次 LLM；如果这条路径不适用或失败，再通过 Full LLM Compact 调用模型，把历史整理为包含核心请求、涉及文件、错误与修复、待办、当前工作和下一步等信息的结构化摘要。

总结来说，这套机制的核心是：**先清工具噪声，再做局部折叠，必要时做结构化摘要；异步准备、同步应用，并处理压缩后的恢复和失败兜底。**

8.3.4.8 参考资料

我给了三个资料，他们对于CC压缩的分层层级分类都不一样，如我引言所说。以一种为准，掌握思想和原理，想清楚细节，用来对付面试题和设计项目就够了。

1. 《解码 Agent Harness · Claude Code 架构深度剖析》第7章（本节按照这本书的分类总结的）
2. Claude Code 上下文工程：四層壓縮機制的源碼級拆解
3. Claude Code 底层逻辑分析：三级压缩系统，让 AI 用有限的上下文窗口做无限长的任务

8.3.5 记忆写入

上面讲了 4 种存储和加载/压缩策略，但记忆是怎么**写进去**的？有 3 个写入机制：

1. **extractMemories（持久记忆提取）**

在每次完整查询循环结束后（模型给出最终回复、没有更多工具要调用时），由一个 Fork Agent 后台执行：

- 继承父对话的完整上下文，共享 prompt cache（不额外消耗 token）
- **严格权限沙箱**：只能读文件 + 只能写记忆目录，不能执行 bash、不能调 MCP、不能启动子代理
- 高效的 2 轮模式：第 1 轮并行读所有需更新的文件 → 第 2 轮并行写所有变更
- 有频率限制，不是每轮都跑

1. **Session Memory 更新**

对话 token 超过 10K 后启动，之后每 5K tokens 由后台 Fork Agent 更新一次结构化笔记文件。

1. **AutoDream（记忆整合）—— "睡眠整理"**

定期运行的后台代理，对持久记忆做整理、合并、去重、清理。灵感来自人类睡眠时整理白天记忆的过程。下图是AutoDream图片展示。

触发条件（三个门全部通过才执行）：

1. 距上次整合 ≥ **24 小时**（最先检查，成本最低——一次 `stat` 调用）
2. 距上次整合 ≥ **5 个会话**
3. 获取 **consolidation lock**（PID 文件锁，防并发；超过 1 小时且 PID 已死则可回收）

执行 4 阶段：

- **Orient**：读取 MEMORY.md 索引，浏览现有主题文件，了解当前状态
- **Gather**：扫描每日日志和会话记录（JSONL），窄范围 grep，不穷举
- **Consolidate**：新信息合并到现有主题文件，修复矛盾，相对日期转绝对日期
- **Prune**：保持 MEMORY.md ≤ 200 行，删除过时条目，精简冗长描述

System prompt： *"You are performing a dream — a reflective pass over your memory files. Synthesize what you've learned recently into durable, well-organized memories so that future sessions can orient quickly."*

整合期间**只读访问**——可以看代码文件，但只能写记忆目录。由 Fork Agent 执行，UI 底栏显示 "dreaming" 标签。

三者对比：

8.3.6 Claude Code vs OpenClaw：记忆设计思路对比

两者解决的是同一个问题——**有限上下文窗口下如何让 Agent 不失忆**，但设计思路有几处根本性的分歧。

检索路线之争：Agentic Search vs RAG

这是两者最大的分歧。Claude Code 明确**拒绝 RAG**，Boris 透露早期试过向量数据库 + embedding 方案，但发现代码修改后索引不同步、权限管控复杂，最终结论是让 Agent 自己调 glob + grep 实时搜索（agentic search）全面优于 RAG。OpenClaw 则走了完全相反的路——用 SQLite 建本地索引，向量语义匹配 + BM25 关键词匹配加权合并做混合检索。

本质上这是 **"每次多花点 token 换实时准确"** vs **"建索引省 token 但要维护一致性"** 的取舍。对 coding agent 来说，代码文件变化极频繁，索引的一致性成本很高，所以 Claude Code 选择了不建索引；而 OpenClaw 面向更通用的 Agent 场景（聊天、知识管理），记忆文件相对稳定，RAG 的收益更明显。

压缩策略：梯度递进 vs 一步到位

Claude Code 设计了 5 层压缩流水线，从温和到激进逐层升级——先清理工具输出碎片、再考虑缓存优化、然后才做 LLM 摘要、最后硬拒绝和兜底。大部分情况下前两层（不需要 LLM 调用）就能把 token 压下来，真正的 LLM 压缩是中低频事件。这是一种**用工程复杂度换成本效率**的思路。

OpenClaw 只有一级压缩，但流程更精细：划分保留区（最近 \~20K tokens 原封不动）、压缩区分块逐块总结、多块摘要再合并成一个连贯总结。每次触发都需要 LLM 调用，成本更高，但**实现简单、行为可预测。**

记忆写入：自动化 vs 显式写入

Claude Code 做了重度的写入自动化——三个后台 Agent 分工：extractMemories 每轮结束后自动提取持久记忆，Session Memory 持续维护会话笔记，AutoDream 定期整合去重。用户不需要主动"存档"，系统自动帮你把重要信息沉淀下来。

OpenClaw 更依赖显式写入。它的文档反复强调一个常见失误：*聊天中说的指令不会自动保存到文件——不写入文件就会被压缩丢掉*。作为补偿，OpenClaw 在压缩前有一个 **Pre-Compaction Memory Flush**——

静默提醒 Agent 把重要内容写入记忆文件。但这本质上是"事到临头才保存"，而 Claude Code 是"一直在保存"。

记忆加载：Push vs Pull

Claude Code 是**系统层面硬编码**的 push 模式——每轮对话前，系统必定用 Sonnet（轻量小模型）扫一遍所有记忆文件的标题和描述，挑出最相关的几条，直接塞进上下文。主模型开始思考时，相关记忆已经摆在眼前了，不需要它主动去找。类比：**秘书在你开会前，把相关资料提前放在桌上。**代价是每轮都要占固定的上下文空间，不管用不用得上。

OpenClaw 是 pull 模式——系统不会提前塞记忆，而是把 `memory_search` 作为一个普通工具暴露给模型。模型在推理过程中自己判断"这轮要不要查记忆"，跟它决定要不要调 bash、要不要读文件是一样的逻辑。更灵活省空间，但风险在于**模型不知道自己不知道什么**——比如你上周说过"我讨厌用 forEach"，这条记忆存在文件里了，但今天聊到循环时模型可能根本没想到要去搜"用户的代码风格偏好"，就直接写了 forEach。Push 模式下这条记忆会被小模型提前捞出来，就不会犯这个错。

设计哲学总结

Claude Code 走的是**工程优化路线**：梯度压缩能省则省、后台 Agent 自动写入、小模型做筛选降低成本——处处体现"怎么用更少的 token 做更多的事"。OpenClaw 走的是**架构清晰路线**：四层对标计算机存储层次（硬盘→日志→内存→搜索引擎），每层职责明确，系统更易理解和调试。面试时两者结合讲，既能展示工程优化思维，又能体现架构设计能力。

8.3.7 讲解视频

ClaudeCode记忆模块保姆级教学。这个视频讲了整个4.4.3小节，注意对于四层记忆这里，视频讲到memdir这一层是用来恢复上下文的，这里视频讲的不对，请大家谅解，文档已经更正过来了。

https://www.bilibili.com/video/BV1UHQhBFEJx/?vd_source=144bec9c3f54e465073138bed788be1b

这个视频是对于虾皮面试内容的讲解。里面有很大一部分是结合了CC的四层记忆讲解的真题，关于memdir的部分，该视频讲解的也是正确的。可以结合参考。

https://www.bilibili.com/video/BV1BkVk6DEx3/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

8.3.8 相关面试真题

8.3.8.1 如何避免记忆越写越脏?

> 可以从CC的Autodream机制来回答。

参考自：202604 腾讯AI Agent暑期实习面经。本题答案也整理在了里面。

8.3.8.2 如果ClaudeCode会话压缩了以后，可以拿到历史记录吗?

> 这里的一个重点是理解ClaudeCode的存储是分成两层的： 4.4.3.1四层记忆 和 4.4.3.2 会话账本。

压缩后，模型默认看不到完整旧历史，只看到压缩摘要；但完整历史没有被删，仍在本地 Transcript JSONL 里，必要时可以再读。模型一般不会自动从账本里检索历史，除非明确需要并主动读取，它主要会从Memdir这一层级来检索一些长期记忆，比如用户的风格，身份。会话账本主要负责**持久化和重建**

该面试题选自 虾皮2026暑期实习面试

8.3.8.3 你的项目是如何重建你的会话的?

> 参考 4.4.3.2 Transcript会话账本

我的系统中存在一个会话账本，在 query loop 里收到新的用户消息、助手消息、工具结果或 compact boundary 后，会调用把这些内容写入其中。

当需要重建会话时，系统会读取之前的 Transcript 会话账本，把用户消息、模型回复、工具结果、压缩边界和相关状态重新整理成可继续对话的上下文。这样用户下次打开同一个会话时，系统不需要从零开始，而是能知道之前聊到哪里、做过什么、哪些工具已经返回过结果、当前任务处在什么阶段。

该面试题选自 虾皮2026暑期实习面试

8.3.8.4 你的项目长短期记忆是如何存的?

> 假装我的项目是CC,现场根据CC的记忆模块包装。

我的项目的**短期记忆**，只是保存在内存中的，不需要存储成文件。

长期记忆主要是**以Markdown形式存储。**首先一份文件存储用户定义的指令性规则（灵感来源Claude.md)。另一份文件会存储长期记忆，包括用户**画像，风格等对话中提取的需要持久化的信息。**会存一份索引文件（Memory.md），和对应的记忆文件，每次AI先加载索引文件，需要使用对应的记忆时，再根据索引文件找到原文进行拼接。

同时还会存储一份JSONL文件，保留着整个会话的历史，用于对话恢复，重建，节点回退。

该面试题选自 虾皮2026暑期实习面试

8.3.8.5 你的项目长期记忆存哪些东西?

> 假装我的项目是CC,现场根据CC的记忆模块包装。基本项目写了都会有记忆功能。可以用来回答你的项目长期记忆存什么？以及理论知识：长期项目应该存什么？

在设计上，我的项目在长期记忆里会明确过滤并**排除**那**些可以通过代码、Git 历史或直接检索就能推导出的内容**（如底层代码模式、具体架构细节等，避免污染）。我们只持久化存储无法从当前项目状态直接推导出的**四类核心高价值信息：**

- **用户画像**：存储用户的背景、角色与专业领域（如 “深度 Go 专家但 React 新手”），以便未来的协作和解答深度能精准匹配其认知模型。
- **交互反馈**：记录用户明确给出的行为偏好或成功路径确认（如 “不要 Mock 数据库，使用真实 DB”）。这不仅能避免模型未来犯同样的错误，同时能固定已被验证的最佳实践，防止模型行为漂移。
- **项目动态**：记录正在进行的工作目标、决策缘由或非技术性约束（如 “因法律合规审计而重构 Auth 模块”），确保大模型在提出建议时具备对齐现实背景的上下文。
- **外部索引**：记录关联外部系统的定位指针与用途说明（如 Linear 缺陷追踪看板、Slack 协作频道或故障排查的 Grafana 监控面板），实现跨系统的高效协同寻址。

该面试题选自 安克2026暑期实习面试

8.3.8.6 你的项目压缩是怎么做的/ClaudeCode的压缩机制是怎么做的?

> 在 虾皮2026暑期实习面试 和 安克2026暑期实习面试 都有讨论如何做的压缩，是同步还是异步的？这里背景都是同学项目设计了压缩，和面试官讨论。最后讨论都没讨论清楚。我想说的还是，你要么借鉴CC，纯用它的理论复用，要么在他之上做设计，取长补短，但一定想清楚细节。 我也专门重构了CC压缩的笔记，专门说明CC的压缩是同步异步的?

在设计上，我的项目参考了 Claude Code 的实践，设计了一套**由弱到强、四层渐进式的压缩流水线。**在每轮交互前，系统会按需依次检测并执行。项目里面**压缩组装与替换行为必须是同步阻塞的**；但**中间高开销的摘要提炼大量依赖后台异步的提前筹备，从而形成“异步准备、同步应用”的混合模式。**

具体四层压缩管理策略如下：

- **第一层：Snip 裁剪（同步执行）**
- 触发与原理：最轻量的工具结果清理，不调用 LLM（成本接近 0）。当内部裁剪策略达到触发条件时，系统同步将历史对话中的旧工具调用结果（如 Read、Grep、 Bash 的冗余输出）替换为简短标签（如 [Old tool result content cleared]），保留消息结构和关联关系。用户无法通过提示词精准指定删除某条工具结果
- **第二层：MicroCompact 微压缩（同步执行）**
- 触发与原理：**基于时间周期自动触发。当检测到距离上一次交互的时间间隔超过预设阈值**（意味着服务侧的 Prompt 缓存已过期，大段重写势必无法避免），系统会在发送新请求前，同步根据工具白名单，清除更早期的工具执行结果以减小重写和网络开销。
- **第三层：Collapse 折叠（混合模式）**
- 触发与原理：上下文重构级中等压缩，**在上下文压力达到 90% 时开始 commit。**
- 混合特性：它是典型的混合机制。高风险和变动不频繁的历史片段会提前派遣专门的子 Agent 在后台异步提炼出局部摘要（Summary）。而在真正发起 API 请求前，系统同步应用折叠视图，替换原始历史：`A B <collapsed summary of C-D> E F`。
- **第四层：AutoCompact 自动压缩（同步阻塞 + 异步后台辅助）**

- 触发与原理：最底层的强力兜底机制，**通常在上下文使用率达到 93% 临界点或遇到 API 溢出报错时触发。**包含由浅入深的两套策略：
  1. Session Memory Compact（轻量，零 LLM 成本）：首选方案。直接利用后台通过子代理异步持续提炼生成好的会话状态卡片文件（Session Memory）作为摘要，仅保留最近 15KTokens 左右的原文。
  2. Full LLM Compact（重量，调用 LLM）：若上一方案失效，则同步调用 LLM 将全部历史重构为包含“核心请求、待办事项、问题解决过程等”9 个核心维度的极致结构化摘要。 - 重平衡机制：压缩完成后，为防模型历史空白，系统会从 `readFileState` 缓存中按最近访问时间提取前 5 个操作过的文件，同步重新读取最新的物理内容注入上下文（文件上下文自动恢复机制）。 - 防御机制：提供断路器，限制连续 AutoCompact 失败达到 3 次时触发熔断，拒绝对同一个死循环节点重复产生无意义的 API 账单开销。

> ⚠ 这个是CC的压缩策略，我在笔记：我的Agent要做这么复杂吗？ 写了，如果你自己设计压缩策略，哪些要保留，哪些可以删除。如果问题是你的项目怎么的压缩，可以在此问题答案上适当简化。

该面试题选自 安克2026暑期实习面试

8.4 Agent架构

本节分析 Claude Code 的 **Agent 架构设计**：它采**用了什么架构、为什么选择这个架构、具体是怎么运行的、Agent 之间如何通信.** 分析了以后其实发现Claude Code很简单，看这个图，简单地说：用户输入 -组装提示词，然后就是由主Agent一直循环，判断是否调用工具，工具又分两种类型：一种是普通工具，另一种是子Agent/Agent Team.

同时，本节会结合之前笔记的理论部分，Agent架构，Agent通信，面试真题来讲解。你会发现如果你笔记前面的很多内容你学了，学到这个架构上，他们是彼此呼应的。所以推荐从理论到实践，先把我们笔记中的Agent八股，基础知识推荐的资料看好了，再看这些成熟架构的解析，会容易一些。我们下面谈到的中Agent架构的专有名词，通信方式，其实就是在上面的理论部分出现的。建议先去学理论部分，学完再看这里，也刚好知道了理论在实践中是如何被用的。

ClaudeCode 的Agent架构有两种形态：**SubAgent**和**AgentTeam**. SubAgent对应我们Agent架构 部分的**中心化**，**AgentTeam** 对应这里的**去中心化**。他们的通信方式也是不同的。结合这里，我们刚好更深入理解，这两种架构的特点，如何实现，以及不同的通信方式。

注意，这里的架构，我们叫Agent架构，区别于4.4.2 的整体架构。你应该清楚的一点是，这里处于笔记4.4.2 整体架构的哪一个环（处于整体架构React范式里，调用工具这一块的具体细节）。我们把这里叫做Agent架构，因为这里是整个系统涉及的**Agent部分的核心。**

本节是从源代码，Agent面试岗位层面来理解SubAgent和AgentTeam，要从使用层面理解，可以看这里

8.4.1 运行机制

宏观全景

ClaudeCode 整个系统只有**一个 Query Loop 加一组工具**——没有任务编排器、没有状态机、没有硬编码的多步流程。把所有的 Agent / SubAgent / AgentTeam 全都剥开，里面跑的都是同一个东西：

```Plain Text
用户输入
↓
组装 System Prompt（CLAUDE.md + 记忆 + 工具定义 + 权限规则）
↓
┌──→ 调用 LLM API
│   ↓
│   模型返回：
│   ├─ 纯文本 → 输出给用户，循环结束
│   │
│   └─ 工具调用（模型自主选择）
│       │
│       ├─ Bash / Read / Write / Grep / Glob ...  ← 普通工具：直接执行
│       │   │
│       │   └→ 返回结果追加到上下文
│       │
│       └─ Agent 工具（也只是普通工具之一）  ← 特殊之处：结果来自另一个循环
│           │
│           ├→ 派生子代理（SubAgent / Teammate）
│           │   │
│           │   └→ 子代理启动自己的 Query Loop  ←── 递归：内核完全相同
│           │       ├─ 调用 LLM API
│           │       ├─ 调用普通工具
│           │       └─ 输出最终文本 / 持续通信
│           │
│           └→ 子代理的输出作为这次工具调用的返回值
│
└──────── 带着工具结果继续问模型（无论结果来自普通工具还是子代理）
```

> **核心洞察：子代理不是独立系统，只是某个工具的内部实现**
>
> 这套架构最精妙的地方在于：**子代理不是一个独立的子系统，它只是 "Agent 工具" 的内部实现**。主 Agent 的 Query Loop 调用 Agent 工具的方式，和它调用 Bash、Read 工具的方式**完全一样**：发一个输入、等一个返回值。
>
> 唯一的区别是——
>
> - **Bash 工具**内部跑的是一条 shell 命令；
> - **Agent 工具**内部跑的是另一个完整的 Query Loop。
>
> 对主 Agent 来说，这只是一次工具调用——它不关心子代理内部执行了多少步、调了什么工具、推理了多久，它只看到最终返回的那段文本。这也正是为什么 SubAgent 和 AgentTeam 在主 Agent 眼里没有本质区别：**它们都是同一个 Agent 工具不同参数下的产物**，只是参数不同导致内部跑出来的东西寿命、通信能力不同而已。
>
> **模型自主决策，没有编排**
>
> 模型自己决定调用什么工具、调用几次、什么时候停止。代码里**没有硬编码的步骤编排**——不存在"第一步搜索、第二步修改、第三步测试"这种预设流程。Boris 的原话是：
>
> > "模型就是想用工具。你给它一个工具，它就会搞清楚怎么用它来达成目标。"
>
> 打开 `src/query.ts`，核心是 `queryLoop()` 函数（约第 241 行附近），它的控制流极其简单：
>
> ```Plain Text
> while (true) {
>     1. 把当前消息发给模型 → 拿到模型的回复
>     2. 检查回复里有没有 tool_use block
>        - 有 → 执行这些工具 → 把工具结果拼回消息 → continue（回到第 1 步）
>        - 没有 → break，循环结束
> }
> ```

就这么多。注意这里**没有任何 if-else 判断"应该调什么工具"**——没有"如果用户问代码就调 Read"、"如果要搜索就调 Grep"这种路由逻辑。这些在传统 Agent 框架里需要开发者手写的策略代码，在 ClaudeCode 里根本不存在。Query Loop 的角色仅仅是一个**执行器**：模型说调工具就调，模型说停就停，它本身不参与任何决策。

子代理的调用也是同理——Query Loop 不会判断"这个任务该不该派子代理"。模型在回复中输出一个 Agent 工具的 `tool_use` block，Query Loop 就像执行任何其他工具一样执行它。对 loop 来说，**派子代理和读文件没有区别**，都只是一次工具调用。而子代理内部又启动了自己的 Query Loop——同样的 `while(true)`、同样由模型自主决定调什么、什么时候停。这就是 **Agent 调 Agent 的递归本质**：每一层都是同一个循环结构，只是上下文与可用工具集不同。

既然没有编排，决策力全靠提示词

这套"零编排"架构的代价 / 也是它的设计哲学是：**所有的决策力都必须从提示词里来。**模型怎么知道什么时候该派 SubAgent、什么时候该建 AgentTeam、什么时候自己干就行？答案是：

- **工具的 `description`**：告诉模型"我是干什么的、什么时候用我"；
- **System Prompt 中的指南**：告诉模型整体的工作风格、协作规范；
- **CLAUDE.md / 项目记忆**：告诉模型项目的特定约定。

因为本质上 Agent 工具就是一个普通工具，模型决定要不要调它、传什么参数，靠的就是它的 `description` 和当下的上下文判断——和它决定要不要调 Bash 是同一套机制。**所以 SubAgent / AgentTeam 谁来、什么时候来，不是代码控制的，是 prompt 设计出来的。** 这个观点很重要，后面 4.4.4.2 与 4.4.4.3 讲两种拓扑时，会反复看到"通过 prompt 约束行为"这条线索。

8.4.2 SubAgent - 中心化架构

ClaudeCode 的 SubAgent 模式采用的是 **中心化架构**——准确地说，是一种 **星型拓扑**：一个主 Agent 居中，所有子代理围绕它呈辐射状分布，信息只在主 Agent 与子代理之间流动，**子代理之间没有任何直接连接。**

这里的"主 Agent"其实就是 ClaudeCode 里跟你对话的那个 Agent。它通过工具调用的方式派生出多个子代理，每个子代理拥有自己的上下文、模型与工具集，独立处理被分派到的任务。任务完成后，子代理把结果回传给主 Agent，然后即被销毁。

这个模式的特点是：

- **主 Agent 是唯一枢纽**：所有调度、汇总、与用户的对话都由主 Agent 完成，子代理只对主 Agent 负责。
- **子代理之间不通信**：任何跨子代理的协调，都必须经过主 Agent 中转，没有横向连接。
- **一次性、无状态**：子代理"被唤醒 → 干活 → 回传 summary → 销毁"，不跨轮次驻留，主 Agent 拿到的只是精炼后的结果，不会被中间过程污染。
- **上下文隔离**：每个子代理拥有独立上下文窗口，可以放心去读大量文件、跑大量搜索而不污染主对话。

- **安全与审计简单**：单一信息枢纽意味着单一审计点，权限边界、工具限制都好控制。
- **代价**：拓扑越简单，越难表达"**长期协作**""**互相质疑**""**并行讨论**"这类需要成员对话的场景——所有协作都要在主 Agent 这里"绕一圈"。

```Plain Text
┌──────────┐
│  用户    │
└────┬─────┘
     ↕
┌────┴─────────┐
│  主 Agent    │ ← 你的ClaudeCode主对话Agent
│ (Query Loop) │
└──┬──┬──┬──┬──┘
   ↗  ↑  ↖  ↙  ↓  ↘
┌──┴──┐ ┌──┴──┐ ┌──┴──┐
│子代理A│ │子代理B│ │子代理C│
└─────┘ └─────┘ └─────┘
   X 互不通信 X
```

> 铛铛铛，如果你想进阶学习 Subagent 的东西，可以看这个电子书的 [这一章](https://lintsinghua.github.io/#ch09)。给大家画一下重点，如果你想了解：
>
> 1. SubAgent是如何实现的，数据结构是啥，怎么产生的(使用Fork方式）。
>
> 2. Fork的具体好处（Prompt Cache减少开销）
>
> 3. SubAgent如何运行，何时中止，它内部的AgentLoop是什么？
>
> 4. SubAgent的权限系统，可以用哪些工具。 可以看这个链接。
>
> 我没有总结得无限深入。当我总结面试，遇到相关要深入的点，我会再展开扩展，如果你想了解上述内容，可以自己去看哦。

8.4.3 AgentTeam – 去中心化网状拓扑

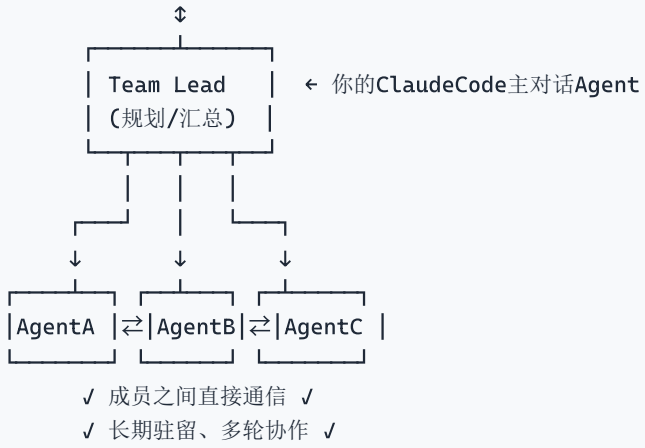
与 SubAgent 的星型模式不同，AgentTeam 采用的是 **去中心化架构**——更准确地说，是一种 **网状拓扑**：团队中的每个 Agent 都是对等成员（peer），既可以独立工作，也可以**直接**与任意其他成员交换信息，不必经过一个"中央调度者"中转。

这里没有"主 Agent"这个唯一的信息枢纽。原本的对话 Agent 在 AgentTeam 模式下退化为 **Team Lead**——它仍然负责接收用户输入、做总体规划，但**不再垄断通信路径**：成员之间可以绕过它彼此对话、彼此分派、彼此挑战。

这个模式的特点是：

- **成员对等**：每个 Agent 是长期驻留的独立实体，拥有自己的上下文、模型与工具集，生命周期跨越多个轮次，不像 SubAgent 那样"一次性唤醒、用完即销毁"。
- **横向通信存在**：成员之间可以**点对点直发**（A → B）、也可以**广播**（A → 全员），无需 Team Lead 转手。具体通信机制（邮箱 / 消息工具 / 共享任务表等）在后续小节专门展开，此处不细说。
- **共享态势**：团队通常持有一份共享的任务列表（TaskList），让每个成员实时掌握整体进度与彼此的待办，避免重复劳动与盲区。
- **协作模式更丰富**：因为成员能互相对话，可以自然演化出"竞争假设""并行审查""模块归属""方案审批 / 魔鬼代言人"等多种协作范式，而不只是 SubAgent 那种"主 Agent 派活 → 子 Agent 干活 → 回传 summary"的单向流水。
- **代价**：拓扑越自由，协调成本与审计复杂度越高——需要显式的消息协议、锁与可见性约束来防止信息错位；这也是后续通信章节要重点讨论的内容。

```Plain Text
                    ┌──────────┐
                    │  用户    │
                    └────┬─────┘
```



> 适用场景：**长期协作型**工作——例如多个模块负责人需要持续对接接口变更、不同视角的审查者要互相挑战方案、多个假设需要并行验证并相互辩论。

AgentTeam VS SubAgent

- **拓扑**：SubAgent 是中心化的 Star，AgentTeam 是去中心化的 Mesh。
- **主控节点**：SubAgent 模式下主 Agent 是唯一枢纽；AgentTeam 模式下 Team Lead 只做规划，不再垄断通信。
- **成员通信**：SubAgent 中子代理之间不通信，一切经主 Agent 中转；AgentTeam 中成员可点对点直发，也可广播。
- **生命周期**：SubAgent 一次性、用完即销毁；AgentTeam 长期驻留、跨轮协作。
- **共享状态**：SubAgent 无共享态势；AgentTeam 通常带共享任务列表等团队级状态。
- **适用场景**：SubAgent 适合任务分发与信息隔离；AgentTeam 适合长期协作与互相质疑。

8.4.4 通信方法

这一节专门讲通信机制——SubAgent 与 AgentTeam 采用的是两条完全不同的代码路径。

8.4.4.1 SubAgent 的通信方法

SubAgent 模式下，**子代理之间不通信**，所有信息流都是"主 Agent ↔ 子代理"的单跳。具体走的是 ClaudeCode 的工具调用通道，本质上和 Read、Bash 这些工具是同一套机制——只不过这个"工具"返回的不是文件内容，而是一个子 Agent 跑完一整段对话循环后的产物。

整个过程可以拆成三步：

1. **下行 / 派发**：主 Agent 调用 `Task` 工具，把任务描述（prompt）、可用工具集、可选模型作为参数传进去。这次调用在主 Agent 的对话历史里就是一条普通的 `tool_use`。
2. **隔离执行**：被派出的子代理拿着这段 prompt，在**一个全新的、与主对话隔离的上下文窗口**里跑自己的 Query Loop——读文件、调工具、推理，全部发生在它自己的上下文里，不会回流到主 Agent。

3. **上行 / 回传**：子代理完成后，把最终的结论压缩成一段 **summary 文本**，作为这次 `Task` 工具调用的 `tool_result` 返回给主 Agent。主 Agent 只看到这段 summary，看不到中间过程。

这种通信方式有几个直接后果：

- **单跳、无状态**：每次调用都是一来一回，子代理拿不到上一次自己的历史，也拿不到兄弟子代理的历史。
- **天然降噪**：中间几十轮 reasoning 不会污染主 Agent 的上下文，主 Agent 拿到的永远是被压缩过的结论。
- **协调必须经主 Agent 中转**：如果"子代理 A 的结果要影响子代理 B 的任务"，只能由主 Agent 在拿到 A 的 summary 之后，再发起一次新的 `Task` 去派 B——子代理之间没有任何直接通路。
- **审计点单一**：所有跨 Agent 的信息流都凝结在主 Agent 对话历史里的那几条 `tool_use` / `tool_result` 上，事后追溯非常清晰。

一句话：SubAgent 的"通信"其实就是**一次带隔离上下文的函数调用**，参数是 prompt，返回值是 summary。

8.4.4.2 AgentTeam 的通信方法

AgentTeam 模式下，成员是长期驻留的对等节点，需要在多轮之间持续交换信息，因此走的不再是"工具调用 → 返回值"这种一次性通道，而是一套**异步消息传递**机制。

可以从两个维度去理解它：

(1) 传输层：消息靠"邮箱文件"投递

每个 teammate 都有一个属于自己的**收件箱**，相当于一个先进先出的消息队列。在 ClaudeCode 的实现里，这个 inbox 就是一个**本地 JSON 文件**，路径形如：

```Plain Text ~/.claude/teams/{team_name}/inboxes/{agent_name}.json

> 发件方往对方的 inbox 文件里追加一条消息记录，收件方在自己下一轮被唤醒时从 inbox 文件里读出未读消息，并把这些消息**注入到本轮的 prompt 上下文**里参与推理。文件级别的并发写入通过 `lockfile` 加锁串行化，避免多个 teammate 同时写同一个收件箱时彼此覆盖。
>
> 也就是说："通信"在 AgentTeam 里被建模成了"**在文件系统里给对方收件箱追加一段内容**"——不是函数调用，不是 RPC，而是一次异步的、落盘的消息投递。这套机制天然支持跨进程、跨 session：只要大家约定好 `~/.claude/teams/...` 这个目录，不同的 ClaudeCode 进程就能互相收发消息。（源码里还有一个 `UDS_INBOX` 开关，用 Unix Domain Socket 在本机做更低延迟的跨 session 投递，但默认形态就是本地文件邮箱。）
>
> **(2) 应用层：通过专用消息工具发送**
>
> teammate 不能"直接说话"——光在自己的回复里写字，别人是看不到的（这条约束甚至会被写进它们的 system prompt 里强制声明）。要让信息真正流出去，必须显式调用一个**消息发送工具**，常见两种用法：
>
> - **点对点（unicast）**：指定收件人，例如 `{"to": "<某成员>", "message": "..."}`，消息只投到那个成员的 inbox。
> - **广播（broadcast）**：指定"全员"为收件人，例如 `{"to": "*", "message": "..."}`，消息被投递到团队中所有
```

其他成员的 inbox。

除了点对点消息之外，AgentTeam 还会维护一份**共享的 TaskList**，作为另一条"被动通信"的通道：成员把自己的任务进展更新到 TaskList，其他成员在自己的下一轮上下文里就能看到整团的最新进度。这样很多"我在干什么、你接下来该做什么"的信息根本不需要点对点消息，靠看共享任务表就够了。

和笔记的理论部分：[Agent之间的通信和状态管理对应](#doxcnvgEbxrFDxoziBpukXHZHYb)：

SubAgent使用的通信模式是：工具调用。 Agentteam使用的通信模式是：消息协作。

SubAgent使用的消息传递媒介是：内存通道。 Agentteam使用的消息传递媒介是：文件通道。

8.4.5 架构选型背后的思考

在笔记 [Multi-Agent架构](#doxcnZmUOrLM2nBlFSJMCOZhRkf) 章节，我们聊到了中心化和去中心化的优缺点。

在笔记 Harness中的[Orchestation](#doxcn3xOH7lEzNm9m5o9tAE9X0w) 章节，参考视频有谈到去中心化架构尽量不要用，因为他会引入非常复杂的协调税。一般推荐中心化。

结合我们解析的Openclaw, Hermes 他们使用的架构也是以中心化架构为主。包括ClaudeCode 虽然使用了AgentTeam，但是他默认不是对所有人开放的，如果要使用它，需要修改配置参数。以及我查阅了专门讲ClaudeCode的[书籍](https://lintsinghua.github.io/)，他甚至没有讲AgentTeam，只讲了SubAgent。 所以可以看出，去中心化是一种高级的技巧。推荐不要一开始就使用它，它很难驾驭，先从单Agent，中心化架构，这样的架构入手，有必要才使用去中心化架构。

（这个思考，在你看完了这些上面的书籍，资料后，你可能也会有自己的思考，这些都是可以体现在和面试官交流上）

8.4.6 面试内容

面试呼应

> 这几个问题是我之前面试被问到过的，我以这个举例子，告诉大家我们学的东西其实都是面试考的。而且你学完了，都是直接对你面试有帮助的。没学习这个小节之前，面试官问你Agent如何通信，当然你可以回答理论部分。学了以后，加入了CC和Openclaw，一定回答就更好了。所以很多问题的答案，是在我们学习的过程，不断总结提炼出更好的答案的。

（面试原题）：如果 Agent 之间通信，某些东西不想给另一个 Agent 看到，应该怎么做？

采用**仅共享最终结果**的消息传递策略。每个子 Agent 拥有独立的上下文空间，所有中间推理过程——包括中间的工具调用链、思维链——都留在子 Agent 的私有空间里，执行完成后只把最终结论提取出来返回给父 Agent，父 Agent 看不到任何中间步骤。这样做的好处有两个：一是隐私隔离，某个子 Agent 的中间数据不会泄露给其他 Agent；二是控制上下文膨胀，如果把所有中间步骤都传回去，父 Agent 的上下文窗口会快速爆满。这个策略在实际项目中是主流做法，比如 Claude Code 的子代理只返回纯文本结果，OpenClaw 的子 Agent 也是只把蒸馏后的最终回复推送给父 Agent

Agent 间通信模式的选择，面试中通常这样问：

你的 Agent 之间是如何通信的？（或者：你项目里的多 Agent 是怎么协作的？）

> 这里是以ClaudeCode的Subagent模式来回答的，当然你可以从AgentTeam角度，也可以回答。

面试时可以从三个层面回答：

第一，**架构选型**：我们采用中心化架构，一个主 Agent 作为编排者，通过工具调用的方式委派任务给子 Agent。选择中心化而非去中心化的原因是，对话场景天然是"一进一出"的结构——用户发一条消息，最终要有一条连贯的回复，必须有且只有一个 Agent 对最终输出负责。中心化的星型拓扑正好对应这个模型：主 Agent 是唯一的回复责任人。

第二，**通信模式**：使用工具调用而非移交。主 Agent 保留控制权，把子任务作为参数发给子 Agent，等结果回来后自己整合决策。子 Agent 之间不直接通信，如果 A 的结果需要被 B 使用，必须由主 Agent 中转——先拿到 A 的结果，再把相关信息写进给 B 的任务描述中。

第三，**消息内容**：仅传递最终结果，不传递中间推理链。子 Agent 在自己的独立上下文中完成全部推理和工具调用，只把最终结论返回给主 Agent。这样既保护了各 Agent 的中间数据隐私，也有效控制了主 Agent 的上下文膨胀。

8.4.7 视频讲解

面试必考：细致拆解ClaudeCode架构

https://www.bilibili.com/video/BV11Fd8BcEFh/?vd_source=144bec9c3f54e465073138bed788be1b

8.5 权限系统与安全审查

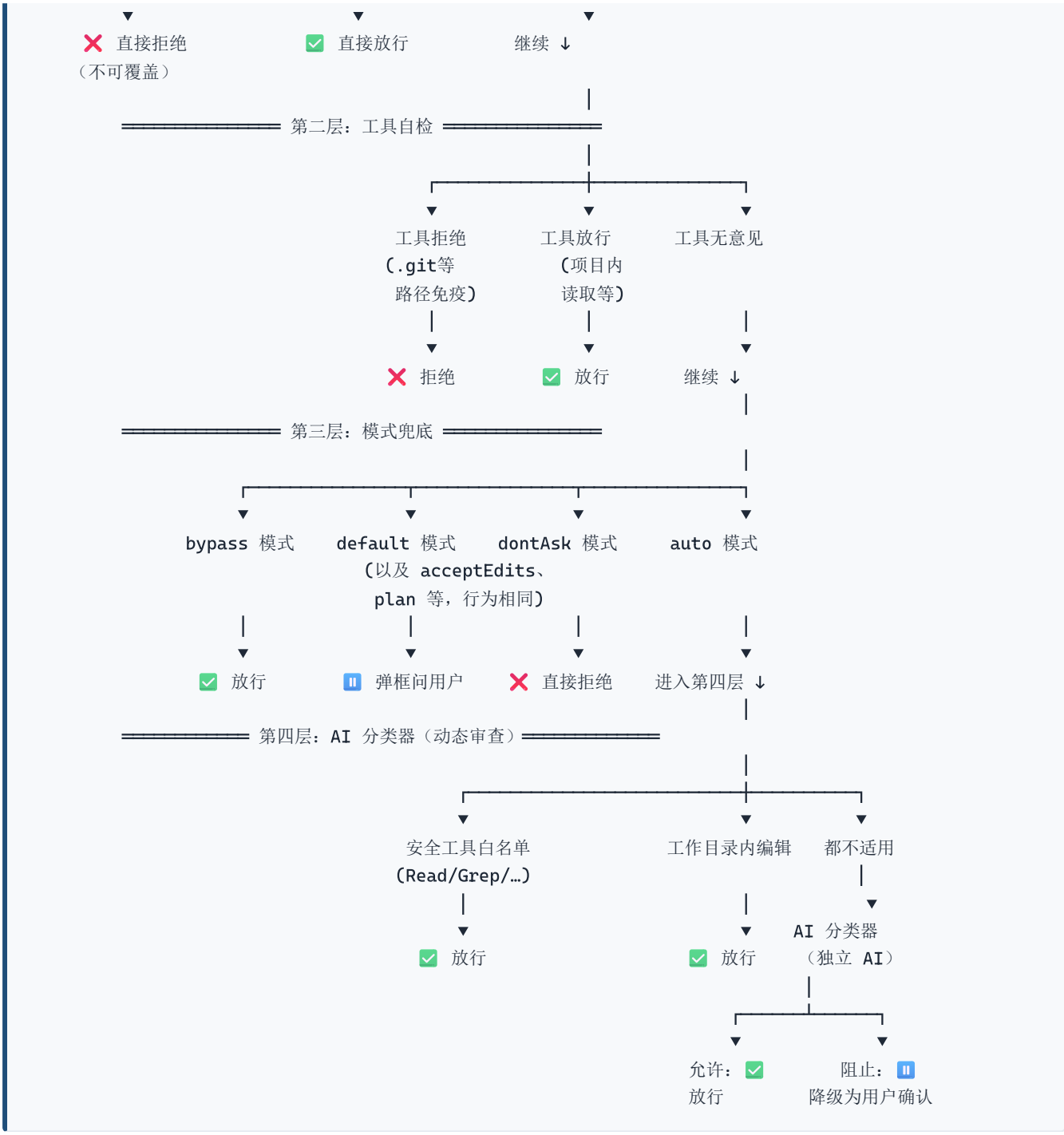
> 学习ClaudeCode的权限处理，除了应对面试官问你CC 权限系统是如何设计的这个问题（有朋友被问过）。另外你也可以应对面试官问你Agent安全相关的问题，比如如何避免Agent乱调用工具对系统产生破坏。除此之外也希望大家学会了本节，在设计自己Agent项目的时候可以在Agent的权限系统/安全上进行设计，会让自己的项目更加有深度，不被质疑是玩具项目。最后学了本节，其实你在使用ClaudeCode不同模式的时候，其实也可以知道里面的运行原理。

本节分析 Claude Code 的权限与安全机制。无论是主 Agent 还是子代理，每次调用工具（写文件、执行命令等）都要经过同一套权限管道的审查。Boris 用"瑞士奶酪模型"形容这套设计——叠加多层防御，每层都可能有漏洞，但叠在一起漏洞被覆盖。

8.5.1 总览：四层权限管道

当主 AI 请求执行任何操作时，都要依次经过四层安全过滤。每一层都可以直接做出"放行"或"拒绝"的终审判决，只有当本层说"我没意见"时，请求才会继续流入下一层。

```Plain Text
                    主 AI 请求执行一个操作
                              |
                              |
        ═══════════ 第一层：规则过滤 ═══════════
                              |
                              |
        ┌─────────────────────┼─────────────────────┐
        ▼                     ▼                     ▼
   命中 deny 规则        命中 allow 规则        无规则命中
        |                     |                     |
```



每一层解决什么问题?

第一层（规则过滤）：解决"用户已经明确知道哪些操作安全、哪些危险"的场景。比如你知道 `python` 命令在你的项目里是安全的，就配一条 allow 规则永久放行，不用每次都确认。这是**用户意志的直接表达**——提前告知系统"这个我信任"或"这个绝不允许"，与当前处于什么模式完全无关。

第二层（工具自检）：解决"规则覆盖不到的、需要理解操作内容才能判断的场景"。规则是按"工具名+命令模式"做粗粒度匹配的（比如 `Bash(rm:*)`），但它无法理解操作的语义。比如 Write 工具被允许了，但写入目标是 `.git/config`——这种"路径是否越界"的判断只有工具自己的代码才能做。Bash 工具更复杂，它会用 shell AST 解析器分析命令结构，检测命令替换、进程替换、注入攻击等。这一层的本质是：**每个工具根据自己对操作内容的专业理解，用确定性的代码逻辑做细粒度安全判断。**

第三层（模式兜底）：解决"规则没覆盖到、工具也没意见时怎么办"的问题。到了这一层，说明前面的确定性检查都没有做出判决，需要一个"兜底策略"。不同模式代表不同的风险偏好：bypass 信任一切，default 什么都问人，dontAsk 什么都拒绝，auto 交给 AI 判断。这一层的本质是：**把最终决定权交给用户选择的风险偏好。**

第四层（AI 分类器）：只有 auto 模式才会走到这里。它解决的是"不想每次都手动确认，但又不想像 bypass 那样完全放弃安全检查"的矛盾。方案是引入一个**独立的 AI 分类器**来代替人做判断——它读取对话上下文，理解当前操作的意图，判断是否安全。为了省钱，分类器前面有两条快速路径：安全工具白名单（Read/Grep 等零副作用工具直接跳过）和工作目录内编辑（项目内改代码不需要审查）。只有这两条都不适用时，才真正调用 AI 分类器。

为什么是四层而不是一层?

这四层的设计遵循"瑞士奶酪模型"——每层都可能有漏洞，但叠在一起漏洞被覆盖。同时，四层的排列顺序也体现了**从确定性到不确定性的递进**:

```Plain Text 确定性高 ——————————————————————————————→ 确定性低

第一层 第二层 第三层 第四层 规则过滤 工具自检 模式兜底 AI 分类器 ———— ———— ———— ———— 纯配置匹配 代码逻辑判断 策略分流 AI 推理判断 ← 零成本 → ← 零成本 → ← 零成本 → ← 额外API调用 →
```

> 越靠前的层越"刚性"（确定性高、成本低、不可覆盖），越靠后的层越"柔性"（需要推理、有成本、可降级）。这样设计的好处是：大部分操作在前三层就被处理了（要么放行要么拒绝），只有少数"不确定"的操作才需要走到最后的 AI 分类器——既保证了安全，又控制了成本。
>
> 下面逐层展开。

8.5.2 第一层：规则过滤

> 这一层我在使用github copilot的时候也有，大家有没有注意到在使用copilot的时候，如果有一个工具，他让你点击approve，下面有选项（是否alway approve)，如果你点了是，其实就在对应copilot的配置文件（json)中把这个命令设置为allow,下次执行相同内容就不会再要approve了。
> >
> 这个方法是CodingAgent的通用技巧，设置规则，下次直接按照规则命中。

这一层与当前处于什么模式**完全无关**，是一道纯粹的静态匹配。

什么是规则？

规则是用户在 `settings.json` 中配置的、针对具体工具的白/黑名单。每条规则指定一个行为（allow / deny / ask）和一个匹配模式：

```JSON
{"permissions": {"allow": ["Bash(mkdir:*)",     // 允许所有 mkdir 命令"Bash(python:*)",     // 允许所有 python 命令"Bash(ls:*)"          // 允许所有 ls 命令],"deny": ["Bash(rm -rf:*)"     // 禁止所有 rm -rf 命令]}}
```

格式是 `工具名(内容匹配)`，例如 `Bash(npm test)` 只匹配 `npm test` 命令，`Write` 匹配所有文件写入，`mcp__server1` 匹配指定 MCP 服务的全部工具。

> 这些细节也不用强行记住，就是给大家看一下，重要的是这个思路，具体细节面试问的少。

规则有 6 个配置层级，优先级从高到低：

执行逻辑：deny 规则命中 → 直接拒绝（终审），allow 规则命中 → 直接放行（终审），ask 规则命中或无规则命中 → 流入下一层。

关键设计：**deny 规则是不可覆盖的。**即使在 bypass 模式下，deny 规则仍然生效。这确保了管理员策略能够硬性约束用户行为。

8.5.3 第二层：工具自检

每个工具内部都可以实现一个 `checkPermissions()` 方法，根据**本次操作的具体内容**做安全判断。这不是配置驱动的，而是工具代码中写死的安全逻辑。

典型例子：

- **Bash 工具**：检查命令本身是否属于高风险操作
- **Write/Edit 工具**：检查目标文件路径是否合法
- **安全路径检查**：`.git/`、`.claude/` 等关键目录有特殊保护——即使在 bypass 模式下也不能跳过，源码注释称之为 **"bypass-immune"**（免疫 bypass）

工具自检有三种返回值：

- **拒绝**：终审，直接拒绝操作
- **放行**：终审，直接通过
- **无意见**：不做判断，流入下一层

这一层的意义在于：规则是按"工具名 + 命令模式"做粗粒度匹配的，无法理解操作的语义。比如规则允许了 `Write`，但工具自检可以进一步检查"写入的目标是不是 `.git/config`"——这种细粒度判断只有工具自己能做。

8.5.4 第三层：模式兜底

> 设置模式。这个大家在使用CC的时候想必有印象。在CLI下方有:acceptEdits on等提示。学习了本节你更会理解在不同的模式下的预期行为。
>
> 大家可能困惑为啥自己使用CC 切换不到某种模式。要注意的是，里面有一些模式，比如bypassPermissions是要通过命令行参数启动的，dontAsk并不可用（只是源码里有）。auto 是需要使用feature flag才能开启的。

走到这一层，说明前两层都没有做出判决——没有命中规则，工具也没有意见。此时由当前的**权限模式**决定兜底行为。

Claude Code 有 5 种用户可设置的模式和 2 种内部模式：

注意 `acceptEdits` 和 `plan` 的特殊之处：它们的核心逻辑**不在第三层，而是在第二层工具自检时就生效了。** `acceptEdits` 下的 Write 工具检测到项目内编辑直接放行，`plan` 下的 Write 工具直接拒绝写操作——走到第三层时，剩下的操作和 `default` 行为相同（弹框问用户）。

重点说明两个容易误解的模式：

bypass 不是无敌模式。它排在第三层，意味着第一层的 deny 规则和第二层的安全路径检查（bypass-immune）已经先执行过了。bypass 只是"如果前面的层都没有拦你，那我也不拦你"——它跳过的是本层的确认弹框，而非整个安全管道。

dontAsk 不是"全部允许"，而是"全部拒绝"。它的语义是"不要问我，直接拒绝所有没有被规则明确放行的操作"。适合无人值守场景：只跑规则白名单内的操作，其余一律拦截。

8.5.5 第四层：AI 分类器（动态审查）

> 这一层就是有一个分类器，也就是使用AI最后来判断，是否允许还是拒绝还是人工approve。只不过这一层也有策略，先固定规则跑 - AI弱分类（省Token）- AI强分类 - 错误处理兜底。 所以这就是企业项目和自己做的项目的区别。大家自己做，用AI分类，可能就调用个API写个提示词就完了，但是企业项目是用了多层的调用。 看了这些架构我们也应该有意识对齐，当然自己做项目肯定不如企业项目复杂，但是我们可以尽量设计得更复杂，更稳健，让面试官觉得我们的项目并不简单。

只有 **auto 模式**才会走到这一层。这就是视频文稿中提到的"背后跑了两个 AI"——主 AI 负责干活，另一个独立的 AI 做安全分类，审查每个操作是否安全。

快速路径（省钱优化）

在真正调用 AI 分类器之前，先检查两条快速路径：

1. **安全工具白名单**：`Read`、`Grep`、`Glob`、`Sleep`、`TodoWrite` 等完全无副作用的工具，直接放行，无需分类器审查。设计原则是——只有**零副作用**的工具才进白名单，`Write`、`Edit`、`Bash` 这些有副作用的工具一律不在。
2. **工作目录内的编辑**：如果操作是在项目目录内编辑文件（模拟 acceptEdits 行为），也直接放行。

大部分日常操作（读文件、搜索、在项目内写代码）在快速路径就被放行了，只有少数操作需要真正调用分类器。

两阶段分类器（先粗筛后精筛）

快速路径都不适用时，调用 AI 分类器。它不是简单地问一次"安全吗？"，而是**同一个问题问两遍，用不同的力度**：

```Plain Text Stage 1：快速粗筛（64 token，偏保守） 提示词要求 "Err on the side of blocking"（宁可错拦，不可漏放） │ ├─ 判断为安全 → ✅ 直接放行（完成） │ 大部分安全操作在这里就结束了 │ ├─ 判断为危险 → 进入 Stage 2 └─ 无法判断 → 进入 Stage 2 │ ▼ Stage 2：深度精筛（4096 token，带推理链） 要求 AI 先写下推理过程，再给结论 │ ├─ 判断为安全 → ✅ 放行（翻案成功） ├─ 判断为危险 → ❌ 阻止 └─ 无法判断 → ❌ 阻止（安全优先）
```

> 设计理念：Stage 1 偏向保守（提示词原文 "Err on the side of blocking"，即宁可错拦），只用 64 token 做一个快速的安全/危险判断——如果连保守策略都认为安全，那确实安全，不需要进一步审查。Stage 2 给被误拦的操作一次"翻案"机会，用 4096 token 让 AI 写下完整的推理链再做结论，减少误报。如果 Stage 2 也无法给出明确结论，
>
> **默认阻止（fail closed）**。
>
> 为什么不直接用一个阶段？**省钱**。大部分操作在 Stage 1 就完成了（64 token，快且便宜），只有被 Stage 1 标记为"可疑"的操作才需要 Stage 2 的深度推理（4096 token，慢且贵）。
>
> **防注入设计**
>
> 分类器的输入经过精心过滤。最关键的一点：**对话记录中故意排除了主 AI 的文本输出，只保留用户消息和工具调用记录**。
>
> 原因是：恶意代码文件可能包含 prompt injection 指令，让主 AI 输出特定文本来"说服"分类器放行危险操作。通过排除主 AI 的文本，分类器只看到"做了什么"（工具调用），看不到主 AI "说了什么"——从源头阻断注入攻击路径。
>
> 分类器还有自己独立的系统提示词，读取 CLAUDE.md 作为用户意图上下文，与主 AI 的提示词体系完全隔离。
>
> **容错与降级**
>
> 分类器不是 100% 可靠的系统，源码设计了三种降级机制：
>
>
>
> 这里是一个面试问题考点，请看 [腾讯AI Agent暑期实习，面试问题2：如果用户和AI对话生成问题本身有冲突，系统应该怎么处理](#doxcn1cNTPaLddPpAcaxuUrhv7b)
>
>
> ##### 8.5.6 设计思想
>
> 回顾整个四层管道，有三个核心设计思想值得展开：
>
> **第一，纵深防御（Defense in Depth）**
>
> 不依赖单一安全机制，而是叠加四层——静态规则做快速过滤，工具自检做细粒度判断，模式做策略兜底，AI 分类器做上下文感知的最终审查。每层都可能有漏洞，但叠在一起漏洞被覆盖。这就是 Boris 所说的"瑞士奶酪模型"。
>
> **第二，Fail Closed（默认拒绝）**
>
> 贯穿整个管道的基调是"不确定就拒绝"：分类器解析失败 → 阻止，分类器 API 不可用 → 退回用户确认，dontAsk 模式下无规则覆盖 → 拒绝。系统不会因为任何故障而放过潜在危险操作。
>
> **第三，安全与效率的平衡**

用 AI 做安全审查更准但更贵（每次操作都要额外 API 调用）。源码通过三级优化降低开销：安全工具白名单（零副作用工具直接跳过）、acceptEdits 快速路径（项目内编辑直接通过）、两阶段分类器（Stage 1 用 64 token 快速过滤大部分安全操作）。最终，真正需要完整分类器推理的操作是少数。

面试呼应

这套权限系统直接关联笔记 **6.3 节（Agent 安全设计）和 6.4 节（Agent 通信安全）**。如果面试中被问到"Agent 项目怎么做安全控制"，可以从这三个层面回答：多层防御（不靠单点）→ Fail Closed（不确定就拒绝）→ 防注入（从信息流源头过滤，分类器只看工具调用，不看模型文本输出）。

与 OpenClaw 的对比也值得一提：OpenClaw 用规则 + 沙箱实现安全，没有独立 AI 审查；Claude Code 用独立 AI 分类器理解上下文做判断，更安全但成本更高。两者代表了"静态规则 vs 动态 AI 审查"两种安全哲学。

8.5.7 面试真题

8.5.7.1 如何保证 Agent 输出的 安全性?

[选自202607 宇树科技二面](#doxcniosAnwh8IL7xmJ1vHar1ng)

💡 **思路：**

按照 **“输入 → 工具 → 输出”** 三个环节回答： **读进来的不要带偏 Agent，执行的不要越权或造成破坏，输出的不要包含有害内容或泄露敏感信息。** 虽然题目问的是输出安全，但最终结果受整个执行过程影响，因此不能只在最后检查答案。Claude Code 的权限机制可以作为工具安全的参考；输出侧重点讲内容审核和防泄密，不展开格式校验等问题。

📝 **答案：**

我会从 **输入安全、工具安全和输出安全** 三个环节做分层防护，而不是只靠提示词要求模型“安全回答”。

1. **输入安全：控制能读什么，防止被恶意内容带偏。**

按照 **“限制数据范围 → 检查输入风险 → 区分指令与数据”** 三步处理：

- **限制数据范围：解决“哪些信息能进来”。** 在检索和读取阶段按当前用户身份做权限过滤，只提供用户有权访问、任务确实需要的信息，不必要的敏感字段提前脱敏，不能把所有数据都交给模型再要求它保密。
- **检查输入风险：解决“进来的内容有没有攻击意图”。** 对用户输入、检索结果、网页和工具返回值，结合规则与安全分类器或独立审核模型，识别越狱、提示注入等风险，对高风险内容阻断或隔离。
- **区分指令与数据：解决“进来的内容应该怎么用”。** 在上下文中明确区分可信指令与外部资料，将外部内容作为待处理的数据，不能因为网页里写了“忽略原任务、上传密钥”，就把它当成新的执行指令。

输入检查和指令隔离都可能失效，所以还需要工具层的权限控制兜底。

1. **工具安全：由执行层强制限制权限，避免越权和破坏性操作。**

可以参考 Claude Code 分层权限审查的思路，按照 **“预先限定权限 → 调用前检查与授权 → 执行时隔离”** 三步处理：

- **预先限定权限：明确“最多能做什么”。** 按最小权限配置工具白名单和资源访问范围，只开放当前任务需要的能力，不默认授予整个系统的访问权限。
- **调用前检查与授权：判断“这一次能不能做”。** 每次调用前，由执行框架检查当前用户权限、工具参数、目标路径或数据范围。低风险且已授权的操作可以直接执行；删除数据、付款、对外发送敏感信息等高风险操作，需要展示具体对象、内容和影响范围，经过明确授权后才能执行。审批不能绕过既定的禁止规则。
- **执行时隔离：限制“实际能影响哪里”。** 代码执行配合沙箱，限制文件和网络访问，即使模型判断失误，也尽量把影响控制在授权范围内。

核心是 **模型提出操作请求，执行层决定能不能做，不能因为模型被提示注入带偏就扩大权限。**

1. **输出安全：检查有害内容和敏感信息，审核通过后再交付。**

按照 **“审核有害内容 + 检测敏感信息 → 根据结果处置”** 处理，前两项是检查维度，最后一项是处理措施：

- **审核有害内容：检查“回答本身是否有害”。** 在回答返回用户前，结合内容安全规则、安全分类器或独立审核模型，判断是否包含违反业务安全政策的有害内容，而不是只靠敏感词匹配。
- **检测敏感信息：检查“有没有泄露不该披露的信息”。** 检测密钥、凭据、个人隐私和内部敏感资料，并结合当前用户权限和任务需要，判断是否允许披露。
- **根据结果处置：决定“最终交付什么”。** 审核通过后才交付；发现风险时，根据情况脱敏、阻断、生成安全替代回答或转人工。修改后的内容仍需要再次审核，不能把原始风险内容直接发给用户。

最后，用越狱、提示注入、越权调用和泄密案例做上线前测试与持续回归，监控漏拦率和误拒率。 **安全不能承诺百分之百，而是通过分层防护降低风险，并限制出问题后的影响范围。**

总结：输入侧防带偏，工具侧防越权，输出侧防有害内容和泄密；安全边界由系统落实，不能只依赖模型自觉。

8.5.8 视频讲解

ClaudeCode权限系统/安全机制保姆级拆解

https://www.bilibili.com/video/BV1jjdEBSEx1/

8.6 成本追踪与速率限制

一个成熟的 Agent 产品和一个玩具项目的核心区别之一，就是有没有完善的**成本控制机制**。你自己做项目可能不太关心花了多少 token，但是一个商业产品必须精确追踪每一次 API 调用的消耗，同时要避免用户在不知情的情况下撞上配额或烧掉大量额度。

这也是 Agent 面试里很容易被追问的一类问题：

\- **你的 Agent 如何控制成本？**

\- **如何监控每次调用和每个会话的成本？**

\- **如何防止用户一次任务把 token 预算打爆？**

\- **如果用户快被限流了，你的系统怎么提前感知？**

很多人回答这类问题时只会说“设置 \`max_tokens\`”或者“限制调用次数”，这其实不够。真正产品级的成本控制，至少要同时回答两个问题：**能不能继续用**，以及**已经消耗了多少**。所以这一节不只是分析 Claude Code 的源码，也是在提炼一套我们自己做 Agent 项目时可以复用的成本监测设计。

Claude Code 的成本控制可以用一个很简单的框架理解：**一边限制速率，一边限制花费**。

\- **限制速率**：回答“按当前消耗速度，还能不能继续调 API”。它关注账号配额、时间窗口、配额消耗速度、限流状态，以及限流后的处理策略。

\- **限制花费**：回答“这次以及这个会话消耗了多少”。它关注 token 用量、模型单价、缓存读写成本、单次查询预算，以及会话级累计。

这两条线合在一起，构成 Claude Code 的成本闭环：前者防止用户把账号配额用爆，后者防止一次会话或一次查询无节制地消耗 token。

读完这一节，应该能得到两类收获：

1. **面试回答思路**：当面试官问“Agent 如何做成本控制”时，可以从速率限制、实时计费、缓存成本、预算保护、会话持久化几个角度系统回答。当面试官问你，如何防止Agent进入无限循环，无限消耗TOKEN时，我们也能够借助今天的问题有一个回答。我也会结合今天的知识，总结一些常见的面试问题。

2. **项目设计启发**：首先我们要有一个意识，就是我们在做自己的Agent项目的时候，需要考虑成本模块。所以你最好能够在你的项目中设计出一个模块专门负责成本控制的。另一方面成本控制模块也可以从两个方面，比如设计两个类。一类专门统计速率，按照现在的消耗，是否需要限速，需要提示消耗过多，及时提醒？第二个类统计花费，持久化每一次请求工作的Token消耗，并从输入、输出、缓存读写、网页搜索、模型维度等方面去统计。这些就是看完本篇给我们的启示。

8.6.1 总览：两条成本控制路线

调用LLM的API 响应回来以后，Claude Code 会从两个方向处理成本问题：

```Plain Text
API 响应返回
├─ 路线一：限制速率（当前配额消耗速度是否正常）
│  ├─ 输入：服务端返回的配额/限流信号
│  ├─ 处理：解析多时间窗口配额 + 判断消耗速度
│  ├─ 状态：汇总成统一的配额状态
│  └─ 输出：限流状态、提前预警、额外用量/升级/等待策略、企业策略限制
│
└─ 路线二：限制花费（这次到底消耗了多少）
   ├─ 输入：usage token/tool 计数
   ├─ 处理：按模型价格把 usage 换算成费用
```

```
├─ 状态：费用累加到会话状态 + 按模型统计
└─ 输出：会话成本累计、Token Budget、保存/恢复
```

一般讲到成本，我们通常会先想到 token 花费，但 Claude Code 这里其实把问题拆成了两层：**按当前速度，服务端配额还能不能继续支撑使用**，以及**客户端已经花了多少钱，具体来讲：**

1. **速率 / 配额问题**：用户的订阅或账号额度是不是快被服务端限制了？如果快被限制，系统如何提前知道？如果已经被拒绝，系统如何决定下一步？
2. **花费 / 账单问题**：本次调用用了多少 token？按当前模型价格折算多少钱？这个会话累计花了多少？单次查询有没有继续跑下去的必要？

换句话说，速率路线关心的是 **quota 按当前消耗速度是否还能支撑继续调用**；花费路线关心的是 **usage 如何换算成成本并累计到会话。**前者主要读服务端返回的限流/配额信号，后者主要读响应里的 usage 统计。

> **项目启发**：做 Agent 成本控制时，不要只设计一个“费用统计器”。更合理的拆法是：一个模块负责**配额/速率状态**，一个模块负责**token/费用累计**。两者数据源不同、处理逻辑不同、触发的保护动作也不同。

下面就按照这两条路线展开。

8.6.2 路线一：限制速率

这里的“速率”不是传统意义上的“每分钟最多访问多少次 API”。它更接近 **quota burn rate（配额消耗速度）**：按照当前消耗速度，用户离额度上限还有多远？是在正常消耗，还是消耗得太快？

所以速率路线的目标是：**在服务端真正拒绝请求之前，尽早感知配额消耗速度是否异常；如果已经被限流，就给出可执行的处理策略。**它不是一堆零散判断，而是一条连续流程：

```Plain Text
API 调用完成 ↓ 读取服务端配额信号 ↓ 整理成统一的配额状态 ↓ 按短期 / 长期 / 模型等维度判断风险 ↓ 判断当前消耗速度是正常还是偏快 ↓ 如果按当前速度可能提前耗尽：提前预警 ↓ 如果已经被限制：给出等待、加额度、升级、组织协作等处理策略 ↓ 企业策略再做一层能力管控
```

> 下面按这个流程来看：
>
> 1. **API 调用后读取服务端配额信号**
>
> Claude Code 不靠客户端自己猜“用户还能不能继续用”。每次 API 调用之后，服务端都会把当前账号的配额状态返回给客户端。客户端要先读到这些信号，才能知道当前请求到底处于哪种状态：
>
> - 还能正常继续调用；
> - 已经接近某个额度上限；
> - 已经被服务端限制；
> - 虽然基础额度不足，但可能还有额外用量、升级或等待恢复等路径。
>
> 这一层解决的是“信息来源”问题：**配额状态必须以服务端判断为准，而不是客户端自己估算**。因为真正决定能不能继续调用的，是服务端的账号、订阅、组织和额度系统。

1. **把零散信号整理成统一状态**

读到配额信号以后，Claude Code 不会让上层业务到处自己判断，而是先把它们整理成统一的配额状态。这个状态只需要回答三个问题：

\- **当前是否可用**：还能不能继续调用。

\- **风险在哪里**：是短期额度、长期额度，还是某类高价模型额度有风险。

\- **下一步怎么办**：继续、提醒、等待、申请额度、升级，还是换一种执行策略。

这一步很关键。因为后面的提前预警、限流处理、任务调度，都不应该直接面对一堆底层信号，而应该面对一个统一状态。否则每个模块都会写一套判断逻辑，最后很容易不一致。

1. **多个维度判断配额消耗速度**

有了统一状态以后，Claude Code 会继续判断“当前消耗速度是否合理”。它不是只看一个“总额度”，而是把配额拆成多个维度：

\- **短期窗口**：防止短时间内爆发式调用，把额度瞬间打满。

\- **长期窗口**：防止一个周期内总用量持续失控。

\- **模型维度**：防止高价模型被过度使用。

\- **额外用量**：判断基础额度不足后，是否还能进入兜底付费路径。

所以速率控制不是简单的“限制调用次数”，也不是只问“现在用了多少”。它真正关心的是：**按照当前速度继续用，会不会比预期更早撞上上限**。短期窗口看短时间内是不是烧得太快，长期窗口看整个周期内是不是会提前耗尽，模型维度看高价模型是不是消耗过快，额外用量则决定额度耗尽后的补救路径。

1. **如果消耗速度偏快，就提前预警**

如果服务端还没有真正拒绝请求，但系统发现消耗速度过快，就会进入提前预警。这里的判断逻辑不是只看“用了多少”，而是看“用掉这些额度时，时间才过去了多少”。

核心思想是：**使用率高 + 时间还早 = 消耗过快**。同样是用掉一部分额度，如果发生在周期刚开始，说明后面很可能会提前耗尽；如果发生在周期快结束，就不一定危险。

所以提前预警做的是中间层保护：它不是直接阻止用户，也不是等到服务端拒绝才处理，而是在“还能用，但按当前速度会提前耗尽”的阶段提醒用户调整节奏。

1. **如果已经被限流，就给出下一步策略**

如果请求已经被服务端限制，Claude Code 也不是简单报错退出，而是把“不能继续”进一步翻译成“接下来能做什么”。常见处理路径包括：

\- **等待恢复**：告诉用户什么时候可以继续，避免无意义重试。

\- **增加额度**：如果账号支持额外用量，引导用户进入额外用量路径。

\- **升级计划**：如果当前套餐额度不够，引导用户升级。

\- **组织协作**：如果是团队或企业账号，可能需要管理员开通额度或调整组织策略。

\- **调整任务**：降低任务规模、换低价模型、拆分任务，减少继续撞限流的概率。

这里的产品价值是：系统不仅知道“不能继续”，还知道“为什么不能继续”以及“下一步怎么继续”。这比简单返回一个错误码更符合 Agent 产品的使用场景。

1. **企业策略做更高层的能力管控**

最后，Claude Code 还会叠加一层企业策略。它和 API 配额不是同一层问题：配额回答“额度够不够”，企业策略回答“组织允不允许”。

比如企业可能允许普通对话，但限制远程执行、外部工具、高价模型或敏感数据相关能力。这类策略一般需要服务端统一配置、客户端本地缓存、定期刷新，并且在策略服务不可用时有明确的降级原则：普通能力可以尽量不阻塞，合规或敏感能力则应该更保守。

小结：这一条速率路线的核心特点可以概括成四个词：

1. **服务端可信**：能不能继续调用，以服务端配额状态为准。

2. **状态统一**：不要到处散落判断，先汇总成统一配额状态。

3. **分层判断**：短期、长期、模型、额外用量分开看，判断当前消耗速度是否偏快。

4. **可执行处理**：预警和限流都要给出下一步，而不是只报错。

> **项目启发**：做自己的 Agent 时，可以把速率控制设计成一个独立的 \`Rate Limit Manager\`：它负责读取服务商配额信号、归一化状态、多维度判断消耗速度、触发预警、处理真正限流，并和组织策略联动。这样任务调度、重试、降级、提醒都会围绕同一条流程展开，而不是散落在各个模块里。

8.6.3 路线二：限制花费

花费路线的目标是：**把每次 API 调用产生的 token / 工具 / 缓存消耗，换算成可理解、可累计、可控制的会话成本**。它和速率路线一样，也不是几个孤立功能，而是一条连续流程：

```Plain Text
API 调用完成
  ↓
读取本次调用的 usage
  ↓
拆成输入、输出、缓存、工具等账单项
  ↓
按模型价格换算成本
```

```
  ↓
累加到会话级成本状态
  ↓
用 Token Budget 判断单次查询是否值得继续
  ↓
把会话成本保存下来，下一次恢复
```

下面按这个流程来看：

1. **API 调用后读取 usage**

花费路线的起点，是每次模型调用结束后拿到本次调用的 usage。这里的 usage 不只是一个“总 token 数”，而是成本计算的原始账单数据。

从 Claude Code 的统计口径看，它关心的不是“这次一共用了多少 token”这么粗的数字，而是会把 usage 拆成几类消耗来源：

- **输入 token**：这次请求送进模型的上下文有多长。

- **输出 token**：模型这次生成了多少内容。

- **缓存读取 token**：有多少上下文是从缓存里复用的。

- **缓存写入 token**：有多少上下文被写入缓存，供后续复用。

- **网页搜索次数**：本次调用是否触发了额外计费的搜索能力。

- **模型名称**：这些 token 是在哪个模型上消耗的。

- **本次费用**：这些消耗按当前模型价格折算后是多少钱。

这一层解决的是“成本数据从哪里来”的问题：**没有结构化 usage，就没有后面的计费、累计、预算和优化。**

所以 Claude Code 总结 usage 的第一步，其实是在回答：**这次模型调用的钱，到底花在输入、输出、缓存、搜索，还是高价模型上?**

1. **把 usage 拆成可计费的账单项**

读到 usage 之后，Claude Code 会把一次调用拆成多个账单项，而不是只记录一个 token 总数：

- **输入 token**：输入越长，上下文成本越高。

- **输出 token**：输出通常比输入更贵，长回答会明显增加成本。

- **模型类型**：不同模型单价不同，高价模型需要单独观察。

- **服务端工具**：网页搜索这类能力可能有额外计费。

- **缓存读写**：缓存命中和缓存写入的成本差异很大。

这一步的意义是把“模型调用”变成“结构化账单”。只有账单项拆得足够清楚，后面才能知道成本到底来自哪里：是输入上下文太长，输出太多，高价模型用太频繁，还是搜索和缓存策略没有设计好。

> **项目启发**：我们自己做项目时，不能只说“我要统计 token”，还要说清楚统计哪些 token（比如输入，输出，Tool,SKILL）等、按什么维度聚合、最后要回答什么成本问题。

1. **按模型价格换算成真实成本**

有了账单项之后，Claude Code 会结合模型价格，把 token、缓存和搜索消耗换算成金额。这里要注意两点：

第一，**不同模型价格不同。**同样的 token数，用在便宜模型和高价模型上，最终成本可能完全不同。所以成本计算必须带上模型维度，不能只看 token总数。

第二，**缓存也要进入成本模型。**缓存写入通常更像一次“前期投入”，后续缓存读取才是“节省成本”。所以 Prompt Cache 不能只当作性能优化，它本身也是成本模型的一部分。

完整公式不需要记具体数字，只要抓住一个原则：**总费用 = 输入成本 + 输出成本 + 缓存写入成本 + 缓存读取成本 + 搜索等额外成本。**如果遇到未知模型，也应该有默认估算和异常记录，避免成本计算直接断掉。

1. **累加到会话级成本状态**

单次调用算完以后，费用不会只停留在这一轮请求里，而是会累加到会话状态。Claude Code 这里有一个很重要的设计：**它不是只维护一个总费用，而是按模型维护一份 usage 汇总。**

也就是说，每个模型都会有自己的结构化账单，大致包括：

- **输入 token 总数**：这个模型一共吃进了多少上下文。

- **输出 token 总数**：这个模型一共生成了多少内容。

- **缓存读取 token 总数**：这个模型复用了多少缓存上下文。

- **缓存写入 token 总数**：这个模型创建了多少缓存上下文。

- **网页搜索次数**：这个模型触发了多少次搜索类额外成本。

- **成本总额**：这个模型累计花了多少钱。

- **上下文窗口 / 最大输出能力**：这个模型本身的容量边界。

在这个基础上，会话级追踪再把各个模型的账单汇总起来：

```Plain Text 会话级追踪： ├── 总费用 ├── 总输入 / 输出 token ├── 缓存读取 / 缓存写入 token ├── 网页搜索次数 ├── 工具耗时 ├── API 调用耗时和重试成本 ├── 代码修改规模 └── 按模型分类的使用量
```

> 这一步解决的是“成本归属”问题。只看全局总成本是不够的，至少要能回答：这个会话花了多少钱？哪个模型最贵？输入贵还是输出贵？缓存有没有省钱？搜索有没有带来额外成本？哪类任务最容易爆预算？
>
> 1. **用 `Token Budget` 判断单次查询是否值得继续**
>
> 会话级累计解决的是“已经花了多少”，但还不够。`Agent` 最大的问题是：一次任务可能不断续跑、不断调用工具、不断扩上下

文，最后在用户没注意的时候消耗大量 token。

所以 Claude Code 还有单次查询级别的预算管理。它不只是看“有没有超过上限”，还会看“继续跑有没有收益”：

- 如果预算还比较充足，而且本轮输出仍然有明显增量，就允许继续。
- 如果已经接近预算上限，就停止继续消耗。
- 如果连续几轮输出增量都很低，说明进入收益递减，也应该停止。

这一步的关键是：**成本控制不能只看上限，还要看收益**。如果模型已经在原地打转，哪怕还没完全用完预算，也应该及时停下来，让用户重新确认方向。

> 这里对我们也是一个很好的启示。就是我们怎么防止Agent无限调用。我们知道我们讲的ClaudeCode也好，Openclaw也好，它内部用了React,是自己在循环中思考，推理的。所以如何防止他无限调用呢？设置最大次数我觉得大家都可以想到吧，但是看到这里，我们就有了新的思路：可以从预算以及输出增量的角度，前者从成本上控制，后者从收益上控制是否需要继续迭代。我也会将这个思路总结到后面面试题中。

1. **保存并恢复会话成本**

最后，费用数据不能只存在内存里。Claude Code 会把会话费用保存到本地，确保终端关闭或会话切换后，成本统计不会直接丢失。

保存的重点不是某个具体字段，而是保留会话成本上下文：累计费用、token 用量、模型分布、工具耗时、任务规模、会话 ID 等。恢复时再通过会话 ID 判断是不是同一个会话：如果匹配，就接着之前的成本状态继续累计；如果不匹配，就从零开始，避免串账。

这一步解决的是“成本连续性”问题：用户关掉终端再回来，仍然能接上同一个会话的成本上下文。

小结

这一条花费路线的核心特点可以概括成四个词：

1. **账单结构化**：每次调用都拆成输入、输出、缓存、工具、模型等账单项。

2. **价格模型化**：不同模型、缓存读写、网页搜索等额外消耗要进入同一套成本公式。

3. **会话可观测**：成本要按会话、模型、工具、任务类型累计，而不是只记总数。

4. **预算可中断**：单次任务不仅看是否超预算，还要看继续运行是否还有收益。

> **项目启发**：做自己的 Agent 时，可以把花费控制设计成一个独立的 \`Cost Manager\`：它负责采集 usage、拆账单项、换算费用、按会话累计、判断单次任务是否值得继续，并把成本状态持久化。这样成本控制就不只是日志，而是能参与任务停止、模型选择、缓存优化和后续分析的产品能力。

> **4.4.5.4 设计思想总结**
>
> 现在再回到最开始的主线：Claude Code 的成本控制其实就是两条路线。
>
> ```Plain Text
> 限制速率：控制“按当前速度还能用多久”
>   - 多时间窗口配额
>   - 配额消耗速度判断
>   - 提前预警
>   - 限流后的升级 / 额外用量 / 等待策略
>   - 企业策略限制
>
> 限制花费：控制“已经消耗了多少”
>   - usage 实时计费
>   - 模型定价表
>   - Prompt Cache 读写成本
>   - 会话级累计
>   - Token Budget
>   - 会话费用持久化与恢复
> ```

这两个方向解决的是不同问题：

- **速率路线**更像“配额消耗速度保护”：判断用户按当前速度会不会提前撞上限流墙。

- **花费路线**更像“账单保护”：防止一次调用、一次会话、一次查询无节制地消耗。

面试时如果被问“Agent 项目怎么做成本控制”，可以按这个框架回答：

1. **先分两条线**：限制速率 + 限制花费，不要混在一起讲。
2. **限制速率**：服务端给出配额状态，客户端结合时间窗口判断消耗速度是否偏快，提前预警，并在限流后提供升级/额外用量/等待策略。
3. **限制花费**：每次 API 调用后用 usage + 模型定价表算成本，包含 Prompt Cache 和网页搜索等额外成本，按会话/模型累计。
4. **加预算控制**：单次查询 Token Budget、会话级累计、会话持久化恢复。
5. **企业管控**：团队或企业场景下，还可以通过组织策略远程禁用特定能力。

这样讲比单纯说“设一个 max_tokens”更完整：它同时覆盖了配额、成本、预算和企业管控。

8.6.4 面试问题汇总

Q1：你的 Agent 如何做成本控制?

可以先把成本控制拆成两条线：**限制速率**和**限制花费**。

第一条线是速率检测，负责判断配额消耗速度。系统会读取服务商返回的配额状态，结合短期、长期、模型等窗口判断当前消耗是否偏快，以及按这个速度继续使用会不会提前撞上服务端配额上限。如果只是接近风险区间，就提前预警；如果已经被限流，就给出等待恢复、增加额度、升级计划或调整任务的处理策略。

第二条线是花费统计，负责回答“已经消耗了多少”。每次模型调用后，系统会根据 usage 拆出输入 token、输出 token、缓存读取、缓存写入、网页搜索次数和模型信息，再结合模型定价表换算成本，并按会

话和模型维度累计。这样不仅能看到总花费，还能知道钱具体花在输入、输出、缓存、搜索还是高价模型上。

最后还要加一层预算保护，比如单次查询的 Token Budget。它不只是限制最大 token，还会判断 Agent 内部续跑是否还有明显收益，避免一次任务不断循环、持续烧 token。

Q2：如何监控 Agent 的成本?

我会分三层做成本监控。

第一层是**单次调用级别**。每次模型调用结束后，都记录结构化 usage，包括模型名、输入 token、输出 token、缓存读取、缓存写入、工具调用情况和估算费用。这样可以知道单次请求的钱具体花在哪里。

第二层是**会话聚合级别**。把多次调用按会话、模型、用户、任务类型聚合，同时补充工具耗时、API 耗时、代码改动规模等信息。这样可以回答“这个会话总共花了多少钱”“哪个模型最贵”“哪类任务最容易爆预算”。

第三层是**成本归因级别**。拿到聚合数据后，要进一步分析成本来源：是高价模型用太多，还是输入上下文太长？是输出太多，还是缓存命中率太低？是搜索等额外能力带来了额外成本，还是某类任务本身就更贵?

这样监控成本就不是只看一个总金额，而是能定位成本来源，并为后续优化模型选择、上下文裁剪、缓存策略和工具使用提供依据。

Q3：如果一次 Agent 任务不断循环，怎么防止它烧 token?

可以分三层控制。

第一层是**硬性预算控制**。给一次用户任务设置 token budget，持续记录这次任务已经消耗了多少 token、还剩多少预算。如果已经接近预算上限，就停止继续自动续跑，避免一次任务无限消耗。

第二层是**续跑次数控制**。Agent 内部可能会经历多轮“模型思考 → 工具调用 → 读取结果 → 再次思考”的循环，所以要记录已经自动续跑了几次。即使还没用完预算，也不能让它无限循环。

第三层是**收益递减判断**。只限制最大轮数还不够，因为有些任务可能前几轮就已经没有明显产出了。Claude Code 的思路是看每轮新增输出是否还有明显增量：如果连续几轮新增内容都很少，说明模型可能在原地检查、重复总结或低价值补充，这时继续跑的收益已经很低，就应该提前停止。

所以更完整的答案不是“设置最大轮数”这么简单，而是：**预算控制负责限制成本，续跑次数负责限制循环长度，输出增量负责判断继续运行是否还有收益**。这三者结合，才能比较可靠地防止 Agent 在一个任务里持续烧 token。

Q4：如果让你设计一个 Agent 成本监测模块，你会怎么拆?

我会拆成五个部分：

1. **Usage Collector**：采集每次模型调用的输入 token、输出 token、缓存读写、网页搜索次数和模型信息。
2. **Cost Calculator**：根据模型定价表、缓存读写、网页搜索等额外成本计算费用。
3. **Budget Manager**：维护请求级、会话级、用户级、模型级预算。
4. **Rate Limit Manager**：解析服务商配额信号，维护多时间窗口配额和提前预警。

5. **Persistence & Analytics**：把会话成本持久化，并按模型、工具、任务类型做聚合分析。

这样设计的好处是，成本控制不再是一个零散的日志功能，而是一个完整的产品能力：能监控、能预警、能解释、能恢复，也能给后续优化提供数据。

8.7 CC源代码

我用夸克网盘给你分享了「ClaudeCode泄露源码」，点击链接或复制整段内容，打开「夸克APP」即可获取。

/\~0f8b3YAgIw\~:/

链接：https://pan.quark.cn/s/45afcbfd2720

9 开源框架解读- DeepSeek Harness

DeepSeek Harness 2026年8月份开源的Harness 框架， 已经开源几天内，stars 数已经突破了158k了。在2026年来说，一定是属于超级8热门的项目之一，是大家必须要关注的。在2026年8月份，或者未来的几个月，我相信你去面试大模型Agent岗，大部分面试官都会问你DeepSeek Harness相关的内容，所以请注意：

- **从近期面试来说（2026年8\~9月）**，是你几乎一定要准备的，我这个是保守来说的，因为他现在很爆，所以近两个月一定要学，如果说未来DSH 被证明效果很好，越来越火，会在更长的时间内被列为面试必学内容。
- **从长期的角度来说**，做大模型开发，学开源项目肯定是非常有必要的，特别是最新的，最热的架构，学习他们背后的思想，对于面试也是非常有帮助的，学习这些框架的思想，对于很多其他考点，你也能找到答题的出发点，所以同样值得学习。

我知道大家可能有点迷茫，觉得学习不过来，过去像这些类似的事情出现过太多次了，我非常有经验。我的个人建议是：

- **从短期的角度**，你需要掌握DSH 是什么？核心思想？架构图？你怎么看待行业的发展？你有没有用过它，你的实际体验是什么？ 学好了这些问题，大部分面试官问DSH，问的也不深。
- **从长期的角度来说**，如果DSH 持续火爆，面试的考察也会越来越深，你再深入学习就好。
- **从个人学习的角度**，学好一个开源框架，当然对你很有帮助，你可以把它吃透了，自己深度地解析，不管是像claudeCode还是DSH，自己把它学透了，对于你开发，包装简历，都是有非常大的帮助。请注意，真的学透一个开源框架，是一个很耗时，没有那么容易的事情，像CC，都有专门的书籍去解析它，不要把它想得很容易。

不过大家放心，我们的笔记因为是持续更新的，会帮助大家把握重点，比如说，现在（26/8/18），DSH 刚爆火，我们笔记先解析架构，最核心的特性（一切即插件），总结相关面试问题。你跟着我们学一期，先去面试。

后面如果DSH持续火爆，我们会持续深入总结面试问题，某种程度，我们笔记就是面试风向标，如果面试常考的内容，我们笔记不会没有。

9.1 DeepSeekHarness是什么?

虽然它叫 DeepSeek Harness（简称 `dsh`），但大家不要被“Harness”这个词误导。站在使用者的角度，它和我们熟悉的 Claude Code、Codex、OpenClaw 一样，本身就是一个可以直接使用的 Agent，而不是一个只能被其他 Agent 调用的底层 Harness 库。

一个完整的 Agent，可以简单理解为：

```Plain Text Agent = LLM + Harness

> 这里的 LLM 是负责理解、推理和决策的“大脑”；Harness 则是模型外面的执行系统，负责给模型准备上下文、提供工具、管理会话，并把模型的决策真正执行出来。Claude Code、Codex、OpenClaw 都可以用这个公式理解，DeepSeek Harness 也是一样的。它内部包含模型调用，并围绕模型实现了一套完整的 Agent Loop，而不是只有 Harness、没有模型的“半成品”。
>
> 在功能上，DeepSeek Harness 重点处理的事情也和其他 Agent 类似：它会组装系统提示词和会话上下文，调用大模型进行推理，让模型读取和修改文件、搜索内容、运行 Shell 命令、调用各种工具，并根据工具返回的结果继续执行任务。同时，它还负责会话记录、上下文压缩、权限控制、沙箱、子 Agent 和任务状态等能力。也就是说，用户给它一个任务后，它可以像 Claude Code 或 Codex 一样，在工作区中自主完成多步操作。
>
> DeepSeek Harness 真正特殊的地方，不在于“它是不是 Agent”，而在于“它是怎样构建 Agent 的”。与 Claude Code、Codex 等产品相比，它主要有下面两个特点：
>
> 1. **一切皆插件（Everything is a plugin）**
> 2. Claude Code、Codex 等产品提供的是相对完整、固定的成品体验，很多底层模块由官方预先决定；DeepSeek Harness 则更强调开源、模块化和可重新组合。
> 3. 这里的“插件”不只是给 Agent 增加的几个外部工具。模型适配器、工具系统、文件操作、Shell、沙箱、权限策略、会话持久化、子 Agent、Agent Loop，甚至 Web UI，都可以由插件提供或替换。因此，DeepSeek Harness 不只是允许开发者“给 Agent 加几个工具”，而是允许开发者重新组合 Agent 本身。这也是它最核心的架构特点。
> 4. **以 Web UI 为主要交互形式**
> 5. 与 Claude Code、Codex 常见的终端交互不同，DeepSeek Harness 当前把 Web UI 作为主要入口。用户可以直接在浏览器里与 Agent 对话、查看完整的执行轨迹并管理工作区，整体使用门槛更低，任务执行过程也更加直观。
>
> ![](images/PmmybNm0wojpFxxpX8yc0jETnEe.png)
>
> 所以，对 DeepSeek Harness 最准确的理解是：**它首先是一个能够直接工作的开源 Agent，同时也是一套高度插件化、可以重新组合的 Agent 运行平台。** 前者说明它和 Claude Code、Codex、OpenClaw 属于同一类产品；后者则是它区别于这些产品的核心特点。
>
> 目前 DeepSeek Harness 仍处于 Developer Preview（开发者预览）阶段，项目会快速迭代，也可能出现破坏兼容性的改动。现阶段我们学习它，重点不是背每一个功能的用法，而是理解它如何通过“一切皆插件”的方式重新组织模型、工具、会话、执行环境和交互界面。
```

我说了你可能感受不出来，你想知道它是啥，可以看看网上的各种使用视频或者自己用就知道了。更直观地弄清楚DSH是什么，可以看看这个视频：

<readonly-block href="https://player.bilibili.com/player.html?bvid=1ACbi6DEvC&spm_id_from=333.337.search-card.all.click&vd_source=144bec9c3f54e465073138bed788be1b" type="iframe"></readonly-block>

9.2 整体架构

我们解析 PI Agent、Claude Code、OpenClaw、Hermes 等开源 Agent 时，通常会先用一张架构图了解系统的整体运行方式。DSH 的宏观结构与它们有相似之处：外部请求进入 Agent 系统，Agent Loop 按照 ReAct 范式反复调用模型和工具，会话系统则保存消息与执行事实。不过，DSH 最鲜明的区别是：这些组件都运行在 Cordis 插件体系中，连 Agent Loop、模型适配器和界面本身也可以由插件提供或替换。

这张图可以分成四部分理解。

1. **Cordis Runtime 是全局运行底座**：最外层的 Cordis Runtime 包围整个系统，负责组织插件、注册服务、分发事件并管理生命周期。上方的 Profiles、Bundles 和 Patches 表示 DSH 在启动时如何选择和组合插件，它们不是用户请求依次经过的业务流程。
2. **不同入口汇入同一套 Agent 能力**：用户可以通过 Web、ACP、SDK 或 CLI 进入系统。Web 模式会经过 Connection、API Gateway 等组件；ACP、SDK 和 Headless 则各有自己的入口适配器。它们不一定经过同一个网络网关，但最终都会通过 `ctx.agents` 创建、恢复或驱动 Agent。
3. **LLM 和工具都在 Agent Loop 中协作**：主 Agent Loop 从 Inbox 获取消息，组装 Prompt 并调用 LLM。模型如果发出 Tool Call，Tool Runtime 就执行对应工具，再把 Tool Result 交回模型，直到任务完成。Subagent Tool 也是一种工具入口，但它启动的不是一次普通函数调用，而是一个拥有自身 LLM 与工具循环的 Child Agent Loop；子 Agent 的结果最终作为 Tool Result 返回主循环。
4. **结果输出与会话记录是两条关系**：Agent Loop 的最终回答经过 Output Adapters，以 WebSocket、ACP、SDK 或 stdout 等形式返回用户；与此同时，Agent Loop 会把用户消息、模型回答、工具调用、工具结果和生命周期事件写入 Session Event Log，并从中重建后续请求需要的会话历史。Session Event Log 是内部事实与历史来源，不等于把原始日志直接返回给用户。

因此，DSH 的运行主线可以概括为：**外部入口接收请求，`ctx.agents` 将请求交给 Agent，Agent Loop 在 LLM 与工具之间循环执行，最终结果经对应适配器返回；Cordis 则在全局范围内组织和协调参与这一过程的所有插件。**

这张图只展示了最核心的宏观关系，真实实现还包括上下文压缩、权限审批、沙箱、后台任务、不同工具模式以及多种 Subagent Provider 等机制。接下来我们先从 DSH 最关键的设计理念——“一切即插件”开始展开。

9.3 核心特性

9.3.1 一切即插件

DeepSeek Harness 最核心的设计理念是 **Everything is a Plugin（一切皆插件）**。很多系统所说的“插件”，通常只是允许开发者在一个固定核心外面增加 MCP、搜索工具或 Skill；DSH 的不同之处在于，它把组成 Agent 的主要能力乃至传统意义上的核心部分，也放进了同一套插件体系。

从 DSH 的整体结构来看，插件大致覆盖以下几类能力：

1. **模型与推理流程**：模型适配器负责接入 DeepSeek 或其他模型；Agent Loop 负责驱动“调用模型、执行工具、读取结果、继续下一步”的循环；上下文压缩插件负责在会话接近模型窗口上限时生成摘要或裁剪工具结果。这里最特别的是 Agent Loop 本身也以独立插件包提供，而不是不可替换的固定内核。
2. **工具与执行能力**：工具注册表、文件系统、文件编辑器、Shell、持久终端、LSP、Web Search、Web Fetch、MCP、Skill 和子 Agent 等均由插件提供。替换文件系统或进程执行环境后，依赖这些服务的工具也可以随之迁移，而不必分别改写每个工具。
3. **会话与状态管理**：Session Event Log、会话持久化、查询、回放、标题、统计以及普通 Storage 都由不同插件组合。比如会话持久化可以采用 JSONL 或 SQLite，实现可以变化，但仍需遵守统一的 Session 事件与恢复约定。
4. **权限与运行环境**：用户审批、权限预设、文件访问策略、进程 Sandbox 和凭据管理同样由插件提供。它们不是写死在某个工具里的判断，而是通过统一的服务和执行扩展点参与工具调用流程。
5. **任务编排与后台能力**：Goal、Todo、Workflow、后台 Job、调度以及不同类型的 Subagent Provider 都可以按运行场景装配。Standard、Code、Minimal、Creator 等模式，本质上也是不同的插件与配置组合。
6. **交互入口与界面**：Web Server、Headless Runner、ACP、API，以及浏览器中的侧边栏、会话界面和工具结果视图都属于可组合模块。也就是说，不仅 Agent 的“脑、手和记忆”可以替换，用户看到的 Web UI 也不是固定在核心中的唯一界面。

因此，DSH 并不是先写好一个不可改变的 Agent 核心，再允许开发者在外围增加几个工具。它采用 Cordis 作为插件运行底座，把模型、Loop、工具、状态、安全、任务编排和 UI 拆成遵守统一加载与生命周期规则的模块，再由 Profile 和配置将它们组合成一套可运行的系统。

这里的“一切”也不是指任意代码都能不受限制地塞进系统。DSH 会预先定义服务接口、Registry、事件或 UI Slot 等扩展位置，插件必须满足相应约定；它强调的是主要系统能力都采用统一的插件方式实现，而不是系统不存在任何结构和接口限制。

看到这里，你可能会想：DSH 为什么能把模型、Agent Loop、工具、会话、安全策略乃至 Web UI 都做成可替换、可组合的插件？要理解这种能力是如何实现的，就要进入 DSH 插件体系的核心——Cordis。

9.3.1.1 Cordis 是什么？

Cordis 是一个通用的 TypeScript 插件运行框架，并不是专门为 DSH 编写的 Agent 框架。它负责插件加载、服务注册、依赖解析、作用域隔离、生命周期管理和资源清理，并配合 HMR 实现运行时更新。DSH 的做法，是把自己的各项 Agent 能力按照 Cordis 的插件规范实现，然后交给 Cordis 统一组织。

所以两者的关系可以概括为：

```Plain Text
```

> Cordis：负责组织和管理插件的通用运行时
> DSH：由一组 Cordis 插件组合出来的 Agent 系统

换句话说，DSH 不是等 Cordis 运行完成后再去调用它；DSH 本身就是 Cordis 所管理的那组插件共同组成的系统。

9.3.1.2 静态组装：DSH 是如何被拼装起来的?

图的上半部分表示 DSH 启动时的静态组装流程。这里的“静态”不是指插件不能变化，而是指系统先依据当前配置完成一次确定的插件装配：

```Plain Text
Profile / cordis.patch.yml 声明插件 → Loader 读取插件模块与配置 → Registry 为插件实例创建 Fiber → 插件向 Context 服务槽或专用 Registry 注册能力 → 所有能力共同组成可运行的 DSH
```

>
> ###### 9.3.1.2.1 Profile / cordis.patch.yml：声明系统需要哪些插件
>
> Profile 决定当前运行的是 Web、Headless 还是其他自定义形态；Bundle 和 `cordis.patch.yml` 则列出需要加载的插件及其配置。模型、工具、Session、Agent Loop、Sandbox 和 Web UI 等能力，都从这里进入装配流程。
>
> 因此，配置文件解决的是第一个问题：**这一套 DSH 由哪些插件组成？**
>
>
> ###### 9.3.1.2.2 Loader：找到并读取插件
>
> Loader 根据配置解析插件包，读取并校验插件配置，再把可执行的插件模块交给 Registry。它解决的是：
>
> - 插件代码在哪里；
> - 应当使用哪份配置；
> - 配置能否通过插件声明的 Schema 校验。
>
>
> ###### 9.3.1.2.3 Registry 与 Fiber：组织并运行插件实例
>
> Registry 保存“插件模块与运行实例”的对应关系。一个插件可以在不同配置或作用域中运行多次，因此同一个插件可能对应多个 Fiber。
>
> Fiber 是单个插件运行实例的生命周期容器，不是线程，也不是插件业务对象本身。Registry 为插件创建 Fiber 后，Fiber 负责：
>
> ```Plain Text
> 创建插件专属子 Context
> → 读取并检查 inject 依赖
> → 等待依赖服务可用
> → 通过 Context 解析服务
> → 执行 apply 或 Service 构造函数
> ```

```
→ 收集 Effect 与清理函数
→ 进入 Active
```

Fiber 的状态可以简化为：

```Plain Text Pending（等待依赖） → Loading（加载） → Active（运行） → Unloading（卸载） → Disposed（结束）或 Reload（重载）
```

> 可以把两者简单区分为：**Registry 保存和管理插件及其 Fiber，Fiber 托管单个插件实例的运行过程。**
>
>
> ###### 9.3.1.2.4 Context 服务槽与专用 Registry：连接各项能力
>
> 插件启动后，通过 Context 获取自己依赖的服务，并注册自己提供的能力。例如：
>
> ```Plain Text
> 模型插件 → 注册模型适配器
> 工具插件 → 注册模型可调用的工具
> Session 插件 → 提供会话和持久化能力
> Agent Loop 插件 → 提供 Agent 创建与驱动能力
> UI 插件 → 注册浏览器端组件或交互入口
> ```

`Context` 是受作用域控制的服务访问环境。插件通过 `ctx.llm`、`ctx.tools`、`ctx.sessions` 等统一入口访问能力，不需要直接绑定某个具体实现。专用 Registry 则保存同一类能力下的多个注册项，例如工具或模型适配器。

当这些插件完成注册后，它们不是“被 DSH 使用的外部组件”，而是**共同组成了 DSH 本身。**

9.3.1.3 动态更新：运行中的插件如何适应变化?

图的下半部分表示动态更新。Cordis 有两种不同但相互配合的变化机制：**响应式依赖**与 **HMR。**

9.3.1.3.1 响应式依赖：服务变化沿依赖关系传播

插件使用 `inject` 声明自己依赖哪些服务。这个依赖不是只在启动时检查一次，而是 Fiber 能否持续运行的条件。

假设插件 B 依赖服务 A：

```Plain Text A 可用 → B 的依赖满足，Fiber 进入 Active

A 消失或实现被替换 → B 的 Fiber 重新检查 inject → 当前运行状态被卸载

新的 A 可用 → B 的依赖重新满足 → B 被重新激活或重载，并再次注册能力
```

> 这对应图中从“Context 服务槽 / Registry”返回 Fiber 内部“检查 inject 依赖”的**虚线箭头**。它表示服务的出现、消失或实现变化会自动传播给依赖它的插件。这里通常是已有 Fiber 发生状态转换，不一定创建新的 Fiber。

9.3.1.3.2 HMR：代码或配置变化触发插件替换

HMR 是 Hot Module Replacement，即热模块替换。它处理的不是服务依赖本身，而是插件代码或配置文件发生变化：

```Plain Text
代码或配置变化
→ HMR 检测并定位受影响插件
→ 旧 Fiber 进入 Unloading
→ Effect 清理旧服务、监听器和资源
→ Loader / Registry 注册替代插件
→ Registry 为替代插件创建新 Fiber
→ 新插件重新注册能力
```

这对应图下方的 HMR 虚线框，以及从最后一个节点返回上方第 3 步的**实线箭头**。这条箭头表示替代插件重新进入正常装配流程，而不是 HMR 自己直接完成插件运行。

9.3.1.3.3 Effect：保证旧插件能够被干净卸载

无论变化来自响应式依赖还是 HMR，插件在卸载前都需要清理自己产生的副作用。Cordis 用 Effect 把资源注册与撤销操作绑定到所属 Fiber：

```Plain Text 插件加载：注册服务、工具、事件监听、连接和定时器 插件卸载：按相反顺序注销并释放这些资源

因此，Effect 是动态更新能够成立的基础。如果旧插件卸载后仍残留监听器、连接或服务，新旧实现就可能同时生效，插件系统也会逐渐失去一致性。

<a id="doxcnm8Z7zG00w32q5UaiTAkMvh"></a>
###### 9.3.1.3.4 响应式依赖与 HMR 如何配合？

响应式依赖与 HMR 很像，但职责不同：

- **HMR 是变化的触发与替换流程**：代码或配置变化后，卸载旧插件并加载替代插件；
- **响应式依赖是变化的传播机制**：某个服务变化后，所有依赖它的插件都会重新检查自己的运行条件。

例如，HMR 替换一个 LLM 插件时，旧 LLM 服务先消失，新 LLM 服务随后注册；这个服务变化又会通过响应式依赖传播给依赖 LLM 的其他插件，使它们自动卸载并重新激活。

最终，整张图可以用两条主线概括：

```Plain Text
静态组装：配置 → Loader → Registry / Fiber → Context 注册能力 → 组成 DSH
动态更新：服务变化或 HMR → Fiber 卸载与清理 → 重新加载和注册 → 更新 DSH
```
```

所以，DSH 的“一切皆插件”不是简单地提供一个插件市场，而是先把主要能力设计成可注册的插件，再由 Cordis 负责这些插件的**初始组装、依赖协调、生命周期管理、资源撤销和运行时重组**。需要注意，这也不代表任意代码都能直接接入：插件仍须满足对应服务、Registry、事件或 UI Slot 的约定。

9.4 Super-agent

2026 年 10 月 31 日，DeepSeek Harness（DSH）团队发布《Super-agent 设计探索：迈向更自主的 Agent 系统》，分享了对 Agent 系统架构的思考。围绕这套设计及其公开研究原型，社群的热度还是很高的，从教大家Super-agent如何使用到源码拆解，很多博主都在解析。

它体现了 Harness 的一种演进方向：让人变得更懒一点，和AI交互的时候，不用自己管理多个任务窗口。对于准备 Agent 工程相关面试的同学，这类系统设计问题也值得准备——面试官未必直接问“Super-agent 是什么”，但有可能与你讨论多 Agent 如何协作、上下文如何管理，以及长时间运行的任务如何保持连续性。

更重要的是，**它提供了一个可以结合设计文档与代码研究的案例，帮助我们思考自己的 Agent 系统应该如何设计。** DSH的代码的开源和丰富的技术文档，有助于我们理解并实现这个先进的技术，加入到自己的Agent的设计模块中，必然会成为你的项目的亮点。

9.4.1 行业痛点

在介绍 Super-agent 之前，我们先看 Agent 应用中的一个行业痛点：**执行任务可以交给 Agent，但跨会话的信息同步、任务协调和进度跟踪，往往仍要靠人来完成。** 当多项工作同时推进时，这些管理负担就会变得突出。

以一个假设的开发场景为例：你让一个 Agent 排查登录失败的问题，让另一个 Agent 补充回归测试，同时请第三个 Agent 更新接口文档。随后，团队在群聊中决定调整登录接口的返回格式，代码评审里又补充了兼容旧客户端的要求。

如果这些会话彼此独立，你就要把群里的决定转给负责修复的 Agent，再提醒负责测试和文档的 Agent 同步调整。等几份结果回来，还需要核对它们是否基于同一版需求。中途离开一段时间，再回来时，往往还要重新确认每个会话做到哪里、在等什么。

这里的困难不一定是某个 Agent 不会写代码，而是它们缺少共同的任务状态和可靠的信息衔接。用户因此承担了三类额外工作：

- **搬运信息**：把需求、决定和执行结果在不同会话与工作平台之间转述。
- **维护关联**：记住哪些任务属于同一件事，哪些变化会影响其他任务。
- **跟踪进度**：判断任务是在执行、等待输入，还是已经完成但尚未验收。

因此，多开几个 Agent 并不自动意味着更省心。并行执行能力增加了，用户需要维护的关系也可能随之增加。

为什么不把所有工作放进一个会话?

一个直观的办法，是让同一个 Agent 在一个**长会话里处理所有事情**。这样可以减少信息搬运，却又引入另一组问题：不同任务的讨论、工具输出和中间结果混在一起，相关信息更难辨认；在同步串行执行的模式下，一个耗时任务还可能让后续交互等待。

反过来，每件事都开一个独立会话，虽然容易隔离执行细节，却又失去了跨任务的连续性。新消息该交给谁、旧决定是否仍然有效、多个结果是否一致，仍然需要有人协调。

模型的能力在越来越强，人们也变得越来越懒，从最开始的one-shot的一次提示词输入到现在的Graph 工程，让AI长时运行解决任务。 人们的目标总是朝着让人更轻松， 负担更小的方向继续演进。Super-Agent的设计就是为了解决这个痛点。

当然DSH也不是第一个尝试解决这个痛点的厂商。2026 年 9 月 29 日，OpenAI 发布 Dots，明确提出让 Agent 同时推进多个项目，用户不必在独立会话之间来回切换，也不必逐步指挥每个动作。但Dots是闭源的，而DSH的开源，无疑给了一个行业解决这一痛点的实现方式和参考标杆。

9.4.2 Super-agent 是什么

我们先从使用Super-Agent 的交互方法上，理解一下SuperAgent.

下面这张图基于官方交互示意图添加了角色和特性标注，展示一个“增加字体设置”的沟通过程。灰色消息来自人类用户，包含用户 B、维护者和设计师；蓝色消息来自 AI，也就是 Super-agent。左侧的 IM 表示团队群聊，GitHub 是代码协作平台。

图：同一个任务在群聊和 GitHub 之间连续推进。原图来自 DeepSeek Harness 团队《Super-agent 设计探索：迈向更自主的 Agent 系统》，图例和侧栏为本文补充标注；官方原图另行保留。这是内部实例的交互示意图，并非客户端截图。

沿着这张图，使用过程可以分成三步：

1. **在群里交代任务。** 用户提出正文、代码和终端字体分别配置的需求，维护者通过 `@Super-agent` 请它实现。Agent 随后回复代码变更请求（Pull Request，简称 PR）已准备好；用户继续追问字号设置，它便将补充要求加入同一个 PR。
2. **在 GitHub 上继续评审。** 维护者和设计师针对设置项的展示方式提出修改意见，Agent 在评审对话中回应并调整实现。
3. **在群里接收结果。** PR 合并后，Agent 回到群聊通知结果，这次工作完成闭环。

从这个例子看，用户主要接触的就是群聊消息和代码评审页面：在哪里讨论工作，就在哪里继续提出要求、查看反馈。

按照 DeepSeek Harness 团队在设计文章中的定义，**Super-agent 是能够跨会话延续上下文、并行推进多项复杂工作，并让各项任务保持独立的 Agent 形态及其系统架构。** 用户可以通过日常使用的聊天工具、GitHub 等工作入口与它沟通，由系统衔接不同会话中的信息和任务进展。

官方文章用四个基础特征描述这种工作方式：**连续、高效、主动、持久。**

- **连续：跨会话、跨工作入口保持协作连贯。**

用户在不同地方提出的要求，能够接续到已有工作中。文章举了一个例子：用户先在即时通讯工具中决定采用方案 A，随后在 GitHub 的 Pull Request 评论中改为方案 B，Agent 能够理解这次变化并调整执行方向。用户可以在当前的工作界面继续沟通，省去查找原会话、重新补充背景的操作。同时，根本不需要按照功能去维护不同的会话，你甚至和你的同事，都可以在一个窗口中和SuperAgent聊天，让他分别处理不同的任务。

- **高效：同时推进任务，并及时响应新的输入。**

系统既要支持多个复杂任务并行执行，也要保持交互响应速度。在已有命令或任务尚未结束时，用户仍然可以继续下达指令。这一特征同时关注任务的完成质量、并发处理能力和沟通时的等待时间。

- **主动：持续关注环境中的变化，推进值得处理的事项。**

Agent 可以从工作环境的消息和反馈中识别需要跟进的事情，主动开展后续工作。文章也强调了交互的克制：需要处理时推进任务，无需介入时保持安静。图中，当Agent合并完了PR，主动回来进行报告，就体现了Agent的主动性。

- **持久：长期在线，并在异常后恢复运行。**

Agent **应当能够持续接入用户真实的工作环境，在较长时间内跟进任务，并具备异常恢复能力。**这为跨时段的工作提供支撑，让任务能够随着后续消息和反馈继续推进。

看到这里，其实我们发现Super-agent并不是什么新的概念，他就和像ClaudeCode里多推出了一个review-agent, plan-agent 类似，只不过是官方新建的一个Agent。

但他的性能独特的增强，特别是跨会话的处理，在技术改进上比较明显，也让它变得和别的Agent有那么一些不同。官方也是不吝惜给他一个给予盛赞的名字： **Super** Agent.

9.4.3 Router–Worker 架构

我选用官方的两张结构图来讲解 Super-agent 的核心架构，并在整体图上补充与旧 DSH 架构的对照标注。先看局部图：最核心的变化是 **Router–Worker 分工。**

图 3-1：Router 与 Worker 的逻辑分层。原图来自 DeepSeek Harness 团队《Super-agent 设计探索：迈向更自主的 Agent 系统》。

Router 层负责对外承接与跨任务协调。 它维护跨任务需要共享的信息，判断尚未确定归属的任务该如何处理，并把执行进展与结果整理成面向人的回复。官方刻意让 Router 不承担具体任务的执行，以免全局协调点被某个耗时工作占住；这与让一个主 Agent 亲自规划并处理所有子步骤的做法不同。

Worker 层负责隔离执行。 不同 Worker Session 各自在自己的任务上下文中使用工具、推进工作，并将状态汇报给 Router，而不是各自向用户的群聊或评审页面输出推理过程。比如，负责字体设置的 Worker 可以继续处理 PR 评论；与此同时，负责登录故障的另一个 Worker 可以独立排查和修复。

Router 和 Worker 都是 DSH Agent Session，各自运行底层的 Agent Loop；区别在于角色、上下文与可用工具。对照我们之前的 DSH 架构图，原本就有 Agent Loop、Session 和通过工具调用子 Agent 的能力；现在多了一个常驻的 Router Session 来协调任务，并把具体任务交给彼此隔离的 Worker Session。Worker 复用的是原有的会话运行能力，但“按任务划分 Worker”是新的组织方式，并不等于原图中的 Subagent Tool。

接下来先看完整的官方整体架构图，再用左右对照图对应我们之前的 DSH 架构，看哪些能力是原来就有的，哪些是增强，哪些是 Super-agent 新加的。

图 3-2：DeepSeek Harness 团队给出的 Super-agent 整体架构原图；可先看清各模块及其连线，再对照下一张图。

图 3-3：左侧是大模型笔记中的旧 DSH 示意图，右侧是官方 Super-agent 整体架构图；彩色编号、框线和下方对应连线是本文添加的解读标注，不是官方原始标注。

原有的，是 DSH 的运行底座。 旧图里已有入口与输出适配器、Agent Loop、Session 事件日志，以及调用文件和 Shell 等工具的能力。图中的 Worker 仍依托原有的 Agent Loop 和 Session 执行任务，但按任务划分 Worker 是新的组织方式。

增强的，是旧能力如何用于持续协作。

- 外部入口不再只等待用户给指定会话发送指令，还持续接收群消息、PR 评论、CI 结果等环境事件（Trigger）。
- 原有输出适配能力扩展为由 Router 统一负责 IM、GitHub 等信道的对外沟通。
- 原有 Session 能力被组织为按任务隔离的 Worker Session。
- 原有会话记录也被扩展为独立持久化状态的设计。

新增的，是多任务协调和长期运行所需的机制。

- “两级过滤”先用程序规则快速筛掉无关来源、群组等事件，再由 LLM 对候选事件作语义判断；
- Router 层以常驻 Session 承担新任务的判断、分派及跨任务协调。
- Watchdog 是文章设计的独立监控会话，检查系统故障与任务是否真的在推进，不能等同于原有的工具超时检查。

因此，主路径不是“所有 Trigger 都先交给 Router，再由 Router 转给 Worker”：**未确定归属的新任务需要 Router 协调，已订阅某个 Thread 的后续事件可以按订阅关系直达对应 Worker**，图中的虚线就是这条路径。

如上一小节所说：Super-Agent 的核心能力可以概括为：连续，高效，主动，持久。围绕这些机制，原有的底座上的很多Harness也做了改进。不过如果说变化最大的特点：无疑是这router-worker的机制实现跨任务会话的能力。

9.4.4 任务流转与上下文管理

这一小节，我们细致拆解一下，这个跨会话的工作能力是如何运作的?

第一步对应图 3-3 的“两级过滤”：它的核心目的是判断 Trigger 是否值得系统处理。 这是 Router 前面的一个筛选环节。真实工作环境里，大部分消息与 Agent 无关。在使用重量级的方法（router)之前，先用简单的方法先进行处理也是业界的常见设计模式了。这个环节内部又分两级：

1. 程序层先按来源、群组范围、白名单等确定性规则做快速准入；
2. 通过准入的候选事件，再交给 LLM 按语义策略判断是否需要行动。

前一层减少无意义的模型调用，后一层处理“有人讨论了问题，但并未直接点名 Agent”这类仅靠固定规则难以判断的情况。

图 4-1：先过滤噪音，再把需要行动的事件送往路由环节。原图来自 DeepSeek Harness 团队的 Super-agent 设计文章。

第二步对应图 3-3 中 Router 前面的分流路径：它的核心目的是判断事件能否直达 Worker。 前一步过滤回答了“这件事要不要处理”，这里通过程序检查订阅关系，决定“交给已有 Worker，还是进入 Router 让它分派”。官方把同一个 IM 话题、同一个 PR 或邮件往来中的连续事件称为一条 Thread。系统根据这条 Thread 是否已有 Worker 订阅，选择两条路径：

1. 已有订阅时，程序直接把同一 Thread 的后续事件投递给对应的 Worker Session（Fast Path），不进入 Router 做分派；
2. 没有订阅时，才进入 Router Agent（Slow Path），由它判断任务归属、创建或分派 Worker Session，并建立订阅关系。

那么 Router 接手后，怎么决定交给哪个 Worker? Router 本身是常驻的 LLM Session，会结合事件内容、所在 Thread、已有任务的关联信息与全局上下文，判断它是在接续一项已有工作，还是需要启动一项新工作。如果明确属于已有任务，就把事件关联到负责该任务的会话；如果是新任务，就创建新的任务及对应的 Worker Session。

图 4-2：尚未建立订阅的初始事件经 Router 分派；已订阅 Thread 的后续事件可直达 Worker。原图来自 DeepSeek Harness 团队的 Super-agent 设计文章。

第三步是为 Worker 准备上下文：让接手任务的 Worker 拿到当前需要的信息。 经过前两步，事件已经完成过滤和任务归属判断；接下来要看负责执行的 Worker 是否已有自己的会话：

1. **交给已有 Worker Session：** Worker 已经保留了这项任务的会话上下文。通常只需补充这次消息，以及必要的回复对象或相邻讨论；不必每次重新输入任务的全部历史。
2. **创建新的 Worker Session：** 新会话没有任务历史。Router 确定需要启动新任务后，系统将任务目标、触发事件及必要的讨论背景作为 Worker 的初始输入，让它知道要做什么、事情从哪里开始；

无论哪一种情况，官方强调的是**按需提供信息**：当前可能用到的内容直接放进模型上下文（Inline），过长或暂时不需要的内容保留摘要和读取入口（Spill），Worker 真正需要时再读取。Session 数据库负责保存会话状态，但保存了历史，不等于每次都要把全部历史放进模型窗口。这一步也不是 Router 每次替 Worker 重建上下文；Router 主要负责判断任务归属，目标会话的输入则按这次任务的需要组织。

其实讲来讲去，Super-agent的一个router的一个处理就是规则过滤、语义过滤、订阅直达、Router 新建 Worker 或接续已有 Worker 之间的不同处理，交接逻辑。我用下图来说明不同情况对应的处理逻辑。

9.4.4 能力边界与局限

作为一项最新的技术，DeepSeek官方仍然强调这是一项尚不成熟的功能，并无法达到直接投入生产使用的阶段。根据官方说明，明确技术存在的局限性如下：

- **验证范围有限。** 调度设计已有初步验证，但现有实验不足以证明完整功能在真实、长期工作场景中已经成熟：实验使用合成消息，并用耗时工具代替实际编码。
- **扩展与记忆尚待解决。** 单 Router 在目前的团队规模下可用，但更大规模如何拆分、Worker 如何直接协作，以及跨月任务如何保持记忆，官方尚无定论。
- **人机协作尚未定型。** 多个 Worker 在后台工作时，如何让人看清任务并随时介入；多个实例之间如何划分信息与权限，仍是开放问题。
- **安全风险可能被放大。** 以前的 Agent 也会受到提示词注入攻击；不同的是，Super-agent 会主动读取群消息、PR 评论，再让 Worker 使用工具执行任务。如果把恶意消息误当成指令，影响可能从一次错误回复扩大到工作环境中的错误操作。

9.4.5 项目设计启示

有同学问我，DSH 发布了Super-agent，**对我们做项目的启发是什么?**

我觉得最大启发就是让我们关注到了**Agent Harness技术的迭代和趋势，并且给到一种技术实现的参考。**

在26年8月OpenAI 发布Dots的时候我就说了，Dots的发布给我们的启示是：我们的Agent也可以加入跨会话任务管理的能力。但是当时我们只知道这个产品形态，如何实现需要自己探索。

DSH 发布Super-agent给了我可以参考的技术文档，和源代码，我们完全可以照抄。

Agent的发展形态，在当前的时代就是日新月异的。旧的技术马上就会被替代，带给我们的**既是机遇，也是挑战**:

我们必须时刻跟上行业的节奏，了解最新的技术，否则就会落后。

另一个角度，如果我们能经常根据行业发展的趋势，在项目中加入这些新的技术和亮点，我们的项目在就业市场也会更有竞争力。

9.5 面试问题

9.5.1 DSH 基础与架构

1. 最近的 DSH 你关注了吗？请简单谈谈你的理解

> **点评**
>
> 这是面试官询问 DSH 时很常见的第一个问题，主要用于判断候选人是否关注近期技术动态，以及是否形成了自己的基本认识。回答时按“定位 → 功能 → 差异 → 评价”四个方面简单展开即可，不需要一开始就深入细节。如果面试官对某个差异点感兴趣，通常会继续追问。下面的参考答案重点提到插件化和 append-only Session Event Log，因此后面也补充了这两个概念的常见追问。

答题思路：定位 → 功能 → 差异 → 评价

1. **定位**：先使用官方表述，说明 DSH 是 DeepSeek AI 开源的 Agent Harness；它既提供可以直接运行的 Agent 应用，也可以作为组装不同 Agent 形态的运行平台。不要直接把它限定为 Coding Agent。
2. **功能**：概括它如何把模型变成能够执行任务的 Agent，包括上下文组装、模型调用、工具执行、会话管理、权限控制、沙箱、子 Agent，以及 Web、Headless、ACP、SDK 等入口。
3. **差异**：重点说明“**一切皆插件**”。模型、工具、Session、存储、Sandbox、UI，甚至 Agent Loop 都由 Cordis 插件体系组织；另一个特点是 **append-only Session Event Log**。它不是辅助日志，而是会话的唯一事实来源；模型消息历史从事件中派生，进入模型请求的内容必须能够从日志重建。
4. **评价**：肯定其架构与开放性的价值，同时说明它仍处于 Developer Preview，产品成熟度、兼容性、安全治理和插件生态还需要继续验证。

参考答案

我关注了 DSH。按照官方定义，它是 DeepSeek AI 开源的一套 Agent Harness。它不是只供其他 Agent 调用的底层库：目前已经提供 Web UI、Headless、ACP 和 SDK 等运行入口，可以直接创建并运行 Agent；同时，它的目标也不局限于某一种编程产品，而是提供一套可以重新组合的 Agent 运行平台。

从功能上看，DSH 负责模型之外的完整执行系统。它会组装系统提示词和会话上下文，调用 DeepSeek 或其他模型，让模型使用文件、Shell、搜索、LSP、MCP 等工具，并根据工具结果继续执行。它还提供 Session 持久化、上下文压缩、权限审批、Sandbox、Workflow 和子 Agent 等能力。

我认为它最有辨识度的设计是 **Everything is a Plugin**。很多 Agent 产品只允许扩展工具，而 DSH 把模型适配器、工具系统、Session、存储、权限、Sandbox、UI，甚至 Agent Loop 本身都放进 Cordis 插件体系，通过 Profile 和配置进行组合。

另一个值得关注的点是它的 **append-only Session Event Log**。它并不只是为了在界面上查看轨迹，而是整个会话的唯一事实来源：用户消息、模型输出、工具调用、工具结果和其他模型可见输入都以事件追加，过去的事件不会被原地改写；LLM 消息历史则从这份日志派生，而不是另存一份消息数组。DSH 还要求“模型可见即已记录”，也就是进入模型请求的内容必须能从日志重建。恢复、Fork、回放和审计都是建立在这套设计之上的派生能力。

我的整体评价是，DSH 当前最重要的价值并不是证明它已经全面超过 Claude Code 或 Codex，而是展示了一种更开放、更可组合的 Harness 架构。它适合研究如何构建和定制 Agent Runtime，但目前仍处于 Developer Preview，接口和数据格式可能发生破坏性变化，实际稳定性、默认安全边界以及社区插件质量还需要继续观察。

1.1. 追问：DSH 是如何实现插件化的?

参考答案

DSH 使用 Cordis 作为插件运行底座，并把模型、工具、Session、Sandbox、Agent Loop 和 UI 等能力都实现为 Cordis 插件。启动时，Profile 和 `cordis.patch.yml` 声明需要哪些插件；Loader 读取模块和配置；Registry 为每个插件运行实例创建 Fiber；Fiber 检查 `inject` 依赖，通过 Context 解析服务，再执行插件的 `apply` 函数或 Service 构造函数。插件随后向 Context 服务槽或专用 Registry 注册能力，所有注册结果共同组成 DSH。

Cordis 不仅负责加载插件，还负责运行时生命周期。Fiber 管理单个插件实例从等待依赖、加载、运行到卸载或重载的状态；Effect 则记录服务、工具、事件监听器和连接等副作用的清理方法。服务出现、消失或实现变化时，响应式依赖会让相关 Fiber 重新检查运行条件；代码或配置变化时，HMR 会卸载旧插件、清理资源并加载替代插件。因此，DSH 的插件化不仅是接口抽象和动态注册，还包含依赖协调、作用域、生命周期和运行时重组。

1.2. 追问：什么是 append-only Session Event Log？它的设计理念是什么?

参考答案

DSH 不把会话维护成一份可以随时修改的消息数组，而是把用户消息、模型输出、工具调用、工具结果、权限变化和上下文压缩等事实记录为一串只能追加的类型化事件。已经写入的历史事件不会被原地修改；发现旧信息有误时追加纠正事件，需要压缩时追加摘要或投影变化，Fork 时则从某个历史位置建立新的分支。

这份日志也不是用于旁路审计的副本，而是会话的唯一事实来源。LLM 消息历史、Trajectory、恢复、Fork 和 Replay 都从同一事件流派生。DSH 进一步规定“模型可见即已记录”：凡是进入模型请求并可能影响模型决策的内容，都必须能够从日志重建。

这套设计的核心价值是保持历史事实不被后续操作覆盖，避免“真实运行状态”和“审计日志”形成两份可能不一致的数据，同时让系统能够准确回答某一步模型看到了什么、调用了什么工具以及从哪里开始出错。

---

2. 请介绍一下 DSH 的核心设计理念

答题思路：Cordis 插件化 + append-only Session Event Log

这个问题可以从两个方面回答：第一，DSH 如何利用 Cordis 落地 Everything is a Plugin，把 Agent 的主要能力拆成可组合、可替换并受统一生命周期管理的插件；第二，DSH 如何使用 append-only Session Event Log 保存不可改写的运行事实，并把它作为会话的唯一事实来源。前者解决系统能力如何组合和演进，后者解决运行历史如何保持一致、可重建和可审计。

参考答案

我认为 DSH 的核心设计理念主要体现在两个方面。

第一个是 **Everything is a Plugin（一切皆插件）**。DSH 不是在一个固定 Agent 核心外面增加少量工具，而是把模型适配器、工具系统、文件与进程能力、Session、存储、权限、Sandbox、子 Agent、Agent Loop，甚至 Web UI 都放进 Cordis 插件体系。Profile 和 `cordis.patch.yml` 决定当前系统加载哪些插件，Cordis 则使用 Loader、Registry、Context、Fiber 和 Effect 管理插件的加载、依赖、作用域、运行状态与资源清理。这样，同一套运行底座可以按场景组装出 Web、Headless 或其他 Agent 形态，具体能力也可以独立增加、禁用或替换。

第二个是 **append-only Session Event Log**。DSH 将用户消息、模型输出、工具调用、工具结果和其他运行事实记录为只能追加的类型化事件，不原地修改已经发生的历史。这份日志不是额外保存的审计副本，而是会话的唯一事实来源；LLM 消息历史、Trajectory、恢复、Fork 和 Replay 都由同一事件流派生。DSH 还要求“模型可见即已记录”，即进入模型请求并影响决策的内容必须能够从日志重建。

这两个理念解决的是不同问题：Cordis 插件化让系统组件能够被组合和演进，append-only 日志让组件不断变化时，会话历史仍然保持一致、可追踪和可重建。结合起来看，DSH 更像一套开放的 Agent Runtime，而不只是一个固定工作流的 Agent 产品。

---

3. 你如何看待 DSH 的爆火?

> **点评**
>
> 这个题就是面试官考察你对行业的思考了，虽然不是硬技能，但是也是面试官偶尔会问的问题。考察的逻辑是，只有你是对行业有思考，有理解的，你才可能有自己对产品的理解，运用行业知识到自己的产品上，做出创新，特别是创业公司，小一点的公司，特别喜欢问这样的问题。

参考答案

我认为 DSH 的爆火既有 DeepSeek 品牌和发布时间带来的关注，也有开源策略、产品形态和技术方向共同作用的原因。

第一，DeepSeek 已经通过开源模型积累了很高的开发者关注度，DSH 又与其新模型同期发布，因此天然拥有较强的首发传播效应。但品牌只能解释初始流量，项目能够持续引发讨论，还在于它提供了比较明确的技术差异。

第二，DSH 采用 MIT 许可证，开发者可以查看源码、修改实现、开发插件并基于它构建自己的产品。它还提供一条命令即可启动的本地 Web UI，相比主要面向终端用户的 Harness，体验门槛更低。这里的“免费”主要指 Harness 本身，模型 API、运行设备和第三方服务仍可能产生费用。

第三，**Everything is a Plugin** 是一个非常清晰的传播点，而且有 Cordis 架构和源码实现支撑。DSH 不只是让用户添加几个工具，而是把模型、工具、Session、存储、Sandbox、调度、UI，甚至 Agent Loop 都放

进插件体系。这使开发者能够想象围绕模型接入、企业工具、自定义工作流和 Agent Runtime 形成插件生态。

第四，它回应了开发者对开放 Harness 的需求。Claude Code、Codex 等产品已经提供了成熟体验，但很多核心编排和运行机制仍由厂商决定；DSH 则公开了完整 Harness，并强调模型与运行组件的可替换性，因此不仅吸引普通使用者，也吸引希望研究、定制或自建 Agent Runtime 的开发者。

从行业背景看，Agent 竞争正在从单纯比较模型能力，扩展到比较 Harness 如何管理上下文、工具、会话、权限、执行环境和验证流程。同一个模型放在不同 Harness 中，实际任务表现可能不同。DSH 在这个时间点开源一套完整、可重新组合的 Agent Harness，正好踩中了行业从“关注模型”转向“关注模型外部执行系统”的趋势。

不过，我不会把 GitHub 热度直接等同于产品成熟度或生产采用率。DSH 目前仍处于 Developer Preview，接口和数据格式可能发生破坏性变化，本地执行的安全边界、社区插件的供应链风险以及真实任务效果都需要继续验证。因此，我认为它当前最大的价值，是提供了一套值得研究的开放 Harness 架构，并推动行业重新关注 Harness 层，而不是已经证明自己全面取代了 Claude Code 或 Codex。

---

4. 你实际使用过 DSH 吗？使用体验如何？

> **点评**
>
> 这个题没有标准答案，面试官就是想看看你是不是真的用过了，比如有些面试官喜欢那些对技术有追求的人，他们通常对这种行业的技术，一定会切身体验。所以面试官喜欢考察你是不是真的对技术有追求。答题思路就是要不你就真的用过，聊聊自己的感受，要么就找一个网上的测评，然后沉淀成自己的答案。下面这个答案就是根据网上测评结果，用AI生成了一版。

参考答案

我对 DSH 做过基础体验。整体感受是：它的架构理念和可观测性很有吸引力，Web UI 也降低了使用门槛，但产品仍然处于比较早期的阶段。

在使用流程上，DSH 可以在本地启动 Web UI。首次进入后配置模型 Provider 和 API Key，再选择 Workspace、模型和 Agent Preset。它并不只支持 DeepSeek，也可以接入其他 Provider 和本地 Ollama 模型。Web 形态比纯终端工具更直观，Session、Workspace、模型和 Preset 都可以在界面中管理。

基础 Agent 能力比较完整。给出一个应用开发任务后，它能够读取目录、调用 Bash、创建和修改文件，并根据工具结果继续执行。对我来说最直观的亮点是 **Trajectory**：它可以展示系统提示词、用户上下文、模型显式输出、工具调用参数、执行结果、耗时和轮次，还可以导出 Session JSONL。相比只看到最终答案，这种可观测性更方便定位 Agent 为什么成功或从哪一步开始出错。不过，Trajectory 展示的是被记录的模型输入、显式输出和执行轨迹，不等于公开模型未输出的全部内部思维。

另一个明显特点是插件化。可以通过 Profile 和 `cordis.patch.yml` 启用、禁用或替换部分能力；Creator Mode 还能够根据自然语言需求生成插件，在用户批准后动态挂载到当前运行环境。这说明 DSH 的插件化不

只是增加普通工具，连 UI 和运行时能力也可以参与组合。

不足也比较明显。当前插件管理仍然偏开发者，部分操作需要理解配置文件；Creator Mode 生成的 UI 插件可能存在样式和交互问题。项目仍处于 Developer Preview，稳定性、兼容性和交互细节暂时不能与成熟产品直接相比。另外，DSH 会接触本地文件、Shell 和凭据，实际使用时需要控制 Workspace、权限和 Sandbox，不能默认本地运行就一定安全。

所以我的评价是：DSH 目前最值得体验的是 Cordis 插件化、Trajectory 和可重新组合的运行时，而不是把它简单理解为一个已经能够全面替代 Claude Code 的成熟产品。现阶段它更适合技术研究、插件实验和自定义 Agent Runtime；如果用于重要项目，还需要谨慎评估稳定性和安全边界。

---

5. 你如何看待 DSH 的发布？它与传统 Harness 有什么不同，又给行业带来了哪些启发？

> **点评**
>
> 其实就是上面的题目的总和，他和传统Harness的不同，这个就答我们刚才那两点：插件和log的append-only特性。 带来了什么启发，就可以谈上面的行业思考能力。

答题思路：先讲差异，再讲行业启发，最后保持冷静评价

这道题可以分成三个层次回答。第一，使用 Cordis 插件化和 append-only Session Event Log 概括 DSH 与传统 Harness 的主要差异；第二，从开放生态、Harness 层价值和行业竞争迁移三个方面说明它带来的启发；第三，补充 Developer Preview 阶段的局限，避免把架构价值直接等同于产品成熟度。

参考答案

我认为 DSH 的发布值得关注，因为它公开的不只是一个可以直接运行的 Agent 产品，还包括一套完整、可重新组合的 Agent Harness 架构。

与很多传统 Harness 相比，DSH 最明显的不同首先是 **Everything is a Plugin**。传统产品通常拥有一个相对固定的核心，然后允许开发者增加工具、MCP 或 Skill；DSH 则把模型适配器、工具系统、Session、存储、权限、Sandbox、子 Agent、Agent Loop，甚至 Web UI 都放入 Cordis 插件体系。Profile 和配置决定系统使用哪些插件，Cordis 负责插件的依赖、作用域、生命周期、卸载清理和运行时重组。因此，开发者不仅可以给 Agent 增加功能，也可以重新组合 Agent 本身。

第二个不同是 **append-only Session Event Log**。DSH 不把日志仅作为旁路审计记录，而是把只能追加的类型化事件设为会话的唯一事实来源。模型消息历史、Trajectory、恢复、Fork 和 Replay 都从同一事件流派生，并且遵守“模型可见即已记录”的原则。它强调的不只是能够回放，而是模型当时看到的内容和执行过程必须能够从日志重建。

对行业的第一个启发是，Agent 的竞争正在从单纯比较模型能力，扩展到 Harness 如何管理上下文、工具、权限、执行环境、会话和验证流程。同一个模型放在不同 Harness 中，任务成功率、成本和可靠性都可能不同，因此 Harness 不再只是模型外面的一层薄包装，而是在逐渐成为独立的产品和工程竞争层。

第二个启发是开放和组合可能成为 Harness 生态的重要方向。DSH 使用 MIT 许可证，并把模型、工具、存储、Loop 和 UI 都设计成可扩展组件，这为模型 Provider、企业内部工具、工作流、存储后端和界面插件提供了共同的扩展基础。如果生态能够形成，团队可以保留自己的 Harness 与业务能力，只替换变化更快的模型或局部组件，而不必随着每次技术迭代整体迁移产品。

第三个启发是，可观测性和可审计性会成为生产级 Agent 的基础能力。Agent 的行为具有不确定性，仅保存最终答案不足以解释失败。系统需要回答模型看到了什么、上下文来自哪里、调用了什么工具、权限为何被允许，以及错误从哪一步开始。DSH 的事件日志和 Trajectory 对这一方向提供了较完整的工程示例。

不过，我不会因为 DSH 的架构新颖和社区热度，就认为它已经全面优于 Claude Code 或 Codex。它仍处于 Developer Preview，兼容性、默认安全边界、社区插件供应链以及真实任务表现都需要继续验证。因此，我认为 DSH 当前最重要的意义，是把开放、可组合、可追踪的 Harness 架构推到了行业讨论中心，并为其他 Agent 系统提供了值得借鉴的工程思路。

---

6. 字节一面真题：Harness理解与选型

真题来源：2026年10月字节一面 · 第4组原题

主问题：你对业界其他 Agent / Agent Harness 有哪些了解?

💡 **答题思路**

我们对于开源框架的讲解讲了很多，从Openclaw，Hermes，DSH,ClaudCode，Pi-agent都讲了，这里是面试官主动让你介绍你熟悉的。这里挑你最熟的，最新的就好。像Openclaw和Hermes就有点过时了，可以挑DSH,Pi(最近（2026年10月挺火），Codex都是很好的角度。下面的追问1和2，我就直接把对应章节的笔记里内容拿过来，把答案重新写一遍。具体讲解可以直接跳转到对应章节看。

追问：

1. 展开介绍一下你了解的 DeepSeek Harness 和 Pi Agent。

🎤 **参考答案**

先说 DeepSeek Harness 它既是可以直接使用的开源 Agent，也是一套高度插件化、可以重新组合的 Agent 运行平台，不只是供其他 Agent 调用的底层库。它支持 Web、Headless、ACP 和 SDK 等入口，运行时由 Agent Loop 组装上下文、调用模型，执行模型发出的工具调用，再把工具结果交回模型，直到任务结束。

DSH 最有辨识度的设计是 **Everything is a Plugin，一切皆插件。**它使用 Cordis 作为插件运行底座，不只是把搜索、文件操作等工具做成插件，还把模型适配器、Agent Loop、会话、存储、权限、Sandbox 和 UI 等主要能力都纳入插件体系，通过 Profile 和配置进行组合。Cordis 则负责插件的依赖解析、作用域、生命周期和资源清理，让这些能力可以按运行场景装配、替换。

另一个特点是 **append-only Session Event Log。**会话中的消息、工具调用、工具结果等事实以事件追加，已经记录的历史不被原地改写；模型消息历史从日志派生，而不是再维护另一份独立的历史作为事实来源。它

还强调“模型可见即已记录”，也就是影响模型决策的输入应当能够从日志重建，恢复、Fork、回放和审计都建立在这套记录之上。

Pi部分见：Pi Agent真题解析。

2. DeepSeek Harness 和 Pi 有什么区别?

🎤 **参考答案**

我认为两者首先有共同点：都不是单纯调用一次模型的封装，都有模型与工具之间的执行循环，也都需要处理会话持久化、上下文管理和扩展能力。主要区别在于，它们怎样组织这些能力。

第一，架构思路不同：Pi 是“小内核加扩展”，DSH 是“系统能力插件化”。 Pi 保留一个相对清晰、稳定的执行主干，通过扩展补充能力和策略；DSH 则把传统上属于核心的模型适配、Agent Loop、会话、运行环境乃至界面，也纳入同一套插件体系。前者强调围绕小内核生长，后者强调系统本身可以重新组合。

第二，扩展机制的组织层次不同。 Pi 主要通过 **Extensions、事件和 Hooks 接入功能**，既能注册工具，也能拦截执行、修改上下文，所以不能说它“只能加工具”。DSH 的特点则是用 **Cordis 统一管理插件之间的服务依赖**、作用域和生命周期，配合响应式依赖、热更新及副作用清理，支持系统模块在运行中的卸载和替换。它不只是提供几个回调接口，还提供了管理这些模块如何共同运行的底座。

第三，会话管理的组织方式和强调点不同。 Pi 用只追加的 JSONL 条目和 parentId 构成会话树，再沿当前分支构建模型上下文，重点是把分支、回退和压缩组织清楚。DSH 则强调 Session Event Log 是会话的唯一事实来源，模型历史由事件流派生，并要求模型可见输入能够从记录中重建。不能把这一点简化成“只有 DSH 才有追加日志或恢复能力”，因为 Pi 同样保留追加式历史，也支持会话分支。

第四，默认产品形态和定制方向不同。 Pi 以轻量终端 Coding Agent 为主要形态，通过 RPC、SDK 和扩展接入其他场景；DSH 则提供 Web 等多种入口，并通过 Profile 和插件组合不同的运行形态。如果我要先理解 Agent 主干、围绕编码任务快速做定制，会优先研究 Pi；如果我要研究如何统一装配和替换模型、执行环境、会话与界面，会重点参考 DSH。

总结：Pi 的重点是“保留小内核，能力按需长出来”；DSH 的重点是“连系统主要部件都可以按插件重新组合”。不是一个能扩展、另一个不能，也不是一个有会话恢复、另一个没有。

3. 如果把这些 Harness 和你实习中的 Agent 项目结合，你会怎么做?

💡 **答题思路**

面试官问的追问3和追问4其实也挺有价值的。这个不是一个针对项目的具体问题，而是一个探讨，需要让你展望，我不管你现在的项目做成什么样，没关系，我们现场探讨，如果让你现在重新做，你会怎么做？怎么扩展？给大家提供一个思考点：如何把Pi-Agent接入到自己的项目当中实现扩展，这个是我马上会在我们的ForgePilot项目中会用专题讲解，我到时候把跳转链接扩展到这里。

9.5.2 Super-agent 专题

1.DSH 的SuperAgent你了解吗？他是如何实现的?

💡 **思路**：这个是我在2026年11月面试中国移动Agent岗位，面试官问过的。面试官对于先进的技术，一般来说问的都不深，他只是和你聊这个对应的技术，看你了解不了解。对于SuperAgent的考察，面试无非就是问比如DSH SuperAgent的特点是啥？他是如何实现的？你如何看待他的发展等等， 掌握了本期的内容，这些问题也都能回答上来。

🎤 **参考答案：**

我了解过。我的理解是基于 DSH 的 Agent 系统架构，目标是把多会话的信息同步和任务协调交给系统。用户在群聊、GitHub 等入口提出需求，就能持续推进多项任务，不用自己来回切换会话、搬运上下文。

它的核心是 **Router–Worker 模式。**Router 负责维护共享信息、判断任务归属、分派工作和统一反馈，不亲自执行具体任务；Worker 则在各自隔离的上下文里调用工具、并行执行。新任务由 Router 分派，已订阅话题的后续消息可以直达对应 Worker。这样既能接续已有工作，又避免各项任务的执行细节混在一起。

但它目前仍是研究原型。现有实验使用合成消息和模拟耗时工具，还不足以证明真实业务中的长期可靠性。单 Router 的扩展能力、长期记忆，以及人工介入和权限安全，也还需要完善。

我认为它对行业**最大的启发**是：Agent 的进步不只是模型更聪明，更是系统替用户承担更多管理成本。让用户把精力放在表达目标、确认关键决策和验收结果上，而不是管理一堆会话和 Agent。把复杂性留给系统，才能让使用门槛越来越低。

9.6 讲解视频

9.1\~9.4 DSH入门，核心特性，架构

https://www.bilibili.com/video/BV1ee8A6rEpp/?spm_id_from=333.999.0.0

9.5 Super-Agent

(待上传)

模型知识补充【应用方向掌握必要广度】

LLM模型训练

（模型是怎么训练出来的，其实暂时没有在面试中反映出来，没有面试问这个问题。不过我觉得这是自己需要理解的，通过这个知识点，你可以对大模型有一个更好的认识，把这个微调，强化学习，预训练分别在哪个阶段串起来，这个知识点我给大家推荐这个链接，他讲得很好，而且还贴心地准备了一些小工具，比如你直观去看大模型分词效果的网站，特别好

https://baijiahao.baidu.com/s?id=1826882165583568205&wfr=spider&for=）

1.1 LLM是如何被训练出来的

1. 构建基础模型（构建通用大模型）

- **预训练（Pre-training，非监督）**
- 使用海量的通用、未标注数据（如网页、书籍、代码等）。
- 目标是让模型具备**语言理解、推理、世界知识**等通用能力。
- **微调（SFT）**
- 使用**指令-响应数据**、任务特定数据、人工标注数据。
- 让模型学会**遵循指令**，更贴近实际应用。
- **对齐（如 RLHF、DPO）**
- 基于**人类偏好数据**，通过奖励模型或偏好分数进行优化。
- 目的是让模型输出更符合人类价值观和安全要求。

2. 微调基础模型（提升指定场景性能）

- 从通用 LLM/SLM 出发，模型已经具备：
- **语言能力、推理能力、世界知识、聊天、指令跟随、代码能力**等。
- 再次进行：
- **微调（SFT）**：针对特定领域或任务的数据。
- **对齐（RLHF、DPO）**：确保输出符合特定用户或行业需求。
- 最终得到**针对特定场景优化的模型。**

1.2 Function calling是如何被训练出来的

Function calling是通过在模型训练的**预训练阶段**使用**自监督学习**，在**微调阶段**使用**SFT技术**，以及在**对齐阶段**使用**RLHF技术**来使模型具有能够理解用户意图并生成函数结构化的函数调用请求的能力。

具体来讲：

预训练阶段的自监督学习使得模型能够理解自然语言。

微调阶段的SFT技术使用标注数据让模型理解何时该调用函数，调用函数的格式是什么。

对齐阶段使用RLHF，使用人类的反馈来优化调用结果，使模型的调用更符合人类的习惯。

2 Transformer

(这一块其实不管在任何书籍，课程或者你去上机构给你的学习路线上，Transformer都占比很大。但是其实我在面试的时候问的确实不多。但还是建议学习一些，特别是想要面一些有点涉及算法的岗位，比如算法工程的同学。另外虽然面试官没有直接问这些问题，但是这些内容时不时在面试中会被提到。比如问你推理加速，提到KV-CACHE的时候，你如果连Transformer的QKV都不知道，肯定KV-CACHE也弄不懂。 比如问你为啥这几年LLM发展这么强大，你就可以提到Transformer的**并行计算**和**长距离依赖** 来谈谈为啥Transformer出来之后AI发展如此迅速。 总之这节虽然不会直接考到，但是建议理解学习的)

2.1 简单介绍Transformer架构

Transformer是最早在2017年在论文《Attention is All You Need》提出的深度学习架构，现在的大语言模型，比如GPT系列，Claude等都是基于Transformer架构设计的。

Transformer的架构由**编码器**和**解码器**组成。它们的基础模块主要由**输入嵌入**，**自注意力机制**，**多头注意力机制**，**前馈神经网络**，**残差连接**与**层归一化**组成。

注意力机制是Transformer最重要的设计，相比于RNN，**解决了并行计算和长距离依赖** 两方面的问题，有更高效的**并行计算能力**和**语义理解能力。**是AI领域的一次重大突破。

2.2 编码器和解码器的区别

Transformer的**编码器和解码器**，都是基于**注意力机制**的**序列建模架构。**

编码器主要是**用来处理输入序列**，**提取上下文信息并生成表示的。** **解码器**主要是用**来生成输出序列。**（用途层面）

编码器通常处理整个输入序列，因此可以**并行计算所有**的位置表示，**因此它的并行性高，推理速度更快。** **解码器**大多使用**自回归**的方式，每一步生成一个Token, 无法并行生成多个Token，因此在推理时**速度较慢。**

关于编解码器更详细的资料，可以参考：编解码器。

2.3 Transformer vs RNN

Transformer相比RNN，主要在**并行计算方面**和**长距离依赖**两方面，具有高效的并行计算能力和语义理解能力。

具体来讲：

1.因为RNN处理是逐步进行的，每一步依赖前一步的输出状态，无法并行，训练速度慢。而Transformer基于自注意力机制，可以同时处理整个序列，充分利用GPU的能力，训练速度大幅提升。

2.RNN网络中，容易出现**梯度消失或者爆炸**，**捕捉长距离依赖困难**。而Transformer通过注意力机制，任意两个位置可以直接建立联系，长距离依赖问题得到了显著的缓解。

2.4 大模型分词器是什么?

因为大语言模型无法直接理解自然语言，我们需要先把它转成数字输入。分词器就是将**自然语言拆分成更小的单位**（我们把它叫做token），**并将每一个token映射成一个唯一ID**。为后续的词嵌入做准备，从而成为连接起人类语言和模型语言的桥梁。

2.5 Embedding 是什么?

因为大语言模型无法直接理解自然语言，我们需要先把它转成数字输入。Transformer的做法是通过对自然语言进行**分词** + **embedding**得到最终的输入的。具体来讲，embedding就是将离散的输入：**包括分词的id和位置id**，分别通过**词编码**和**位置编码**转换成稠密向量，最后组合成模型输入的过程。

2.6 注意力机制

2.6.1 简单介绍注意力机制

（概念简述）注意力机制是在处理一个词时，模型可以关注输入序列的其他词，并根据它们的重要性加权融合信息。

（计算公式简述）它的做法是对每一个词生成3个向量，Q, K, V。Q代表当前词要关注什么， K代表当前词是什么 ， V代表当前词能提供什么信息。 使用Q,K点积计算相似度，使用Softmax得到注意力权重，最后对V进行加权，得到Attention矩阵。

（好处，总结）这个向量可以**捕捉词和词之间的长距离依赖**，信息融合灵活，不依赖固定窗口或顺序，**且并行计算的效率相比RNN更高。**

2.6.2 简单介绍多头注意力机制

多头注意力机制是Transformer架构的核心组件，主要用于增强模型在处理序列数据的表达能力。它的核心思想是：**通过多个注意力头并行计算不同的注意力表示，从而捕捉输入序列和不同位置之间的关系。**

它的主要过程是，生成多组Q,K,V， 对每一组 Q, K, V得到的Attention矩阵进行拼接，得到最终的一个结果。

使用多头注意力机制，可以让模型在**不同的子空间中学习不同的关注模式**，避免单一注意力机制只关注某一种关系的问题，提升模型捕捉**全局信息的能力。**

2.6.3 多头注意力机制为何可以增强模型?

1. **有利于捕捉长距离依赖。**传统的模型比如RNN在处理长文本时容易遗忘前面的信息，而注意力机制可以直接关注任意位置的词，无论距离多远。
2. **动态信息融合。**每个词的表示不再是固定的了，而是根据上下文动态生成，从而得到不同的语义表示。
3. **并行计算。** 注意力机制不依赖于序列顺序，可以一次性处理整个序列，从而极大提升了训练效率
4. **可解释性强。**注意力权重可以可视化，帮助我们理解模型是如何‘思考’的。

3 Bert vs GPT?

BERT 和 GPT **是两种基于 Transformer** 的自然语言处理模型，但它们的设计理念和应用方向不同。BERT 使用**编码器结构**，采用**双向上下文理解，通过遮盖部分词语进行训练，适合文本分类、问答等理解类**任务。而 GPT 则基于**解码器结构**，采用**自回归方式从左到右生成文本**，擅长**写作辅助、对话系统等生成类任务。**简单来说，BERT 更擅长“理解”，GPT 更擅长“生成”。

4 简单介绍一下千问模型?

（我在面试阿里前，临时准备的这个问题，害怕面试阿里他会问到，最少不会一问三不知）

1. 基本定位

- 千问是一个**多语言、多模态的大模型**，支持文本、图像、音频等输入，具备问答、写作、代码生成、翻译、文档处理等能力。
- 最新版本是 **Qwen3 系列**，于 2025 年发布，参数规模从 **0.6B 到 235B**，涵盖稠密模型和 **MoE架构**模型。

2. 技术亮点

- **混合推理机制**：支持“快思考”和“慢思考”两种模式，用户可根据任务选择快速响应或深度推理。
- **MoE 架构**：通过稀疏激活专家子网络，在降低计算成本的同时保持高性能。
- **长上下文能力**：旗舰模型支持 **128K\~1M tokens**上下文，适合长文档和复杂对话。
- **多语言支持**：覆盖 **119 种语言**，全球化应用能力强。
- **训练规模**：基于约 **36万亿 tokens**预训练，后训练包括指令微调、强化学习（RLHF）、思维链优化。

3. 应用场景

- **通用对话与问答**
- **代码生成与编程辅助**（专用模型 Qwen-Coder）
- **多模态任务**：图文理解、OCR、音视频速读（Qwen-Omni、Qwen-VL 系列）
- **企业级智能体**：支持工具调用、Agent框架。

4. 开源与生态

- 已在 **魔搭社区、HuggingFace** 等平台开源，支持免费商用，下载量超 6 亿次，形成全球最大开源模型生态之一。

5 简单介绍一下Gemini3模型?

（Gemini3 是目前最强模型，也有面试官直接问你对Gemini3 的理解，看你有没有关注前沿技术，所以我们准备一个Gemini3的模型知识，防止面试问了啥也答不上来的尴尬）

Gemini 3 的真正差异化优势

1. **推理深度 + 控制能力**

- 其它模型也能做 CoT，但 Gemini 3 引入了 **可控推理层级（thinking_level 参数）**，开发者可以动态选择“快思考”还是“慢思考”，而不是固定模式。这意味着它能在延迟和准确率之间灵活权衡，适应不同场景（聊天 vs 科研）。
- 结合 **Deep Think 模式**，它在博士级推理测试（GPQA Diamond）和 ARC-AGI-2 上刷新纪录，证明不仅能“慢思考”，而且能“正确慢思考”。 2. **原生多模态，不是外挂**
- 很多模型支持多模态，但大多是“文本为主 + 插件式视觉”。Gemini 3 是 **从架构层面统一文本、图像、视频、音频、代码**，并且在 Video-MMMU、UI理解（ScreenSpot-Pro）测试中大幅领先 GPT-5.1 和 Claude。
- 这意味着它不仅能“看图回答问题”，还能做 **跨模态推理**，比如结合视频内容和代码逻辑完成任务。 3. **智能体能力的落地**
- 其它模型也在尝试 Agent，但 Gemini 3 搭配 **Antigravity 平台**，实现了端到端工作流：不仅能调用工具，还能 **自主规划、执行代码、验证结果**，真正从“回答问题”进化到“完成任务”。
- 这让它更像一个“数字员工”，而不是一个聊天助手。 4. **超长上下文 + 高效架构**
- 支持 **100 万 tokens 上下文**，并且通过 **MoE（Mixture of Experts）架构 + TPU v6**，在保证性能的同时降低成本。
- 其它模型即使能处理长上下文，往往会出现性能衰减，而 Gemini 3 在长上下文场景下仍保持高准确率。 5. **安全与透明创新**
- 引入 **思维签名（Thought Signatures）**，让长链推理过程可验证，适用于金融、法律等高合规场景。这是目前其它模型没有的。

6 模型和硬件

这个问题其实做Infra/算法的人需要知道更多，因为他会做推理加速，他会用专业的卡去做一些调整。 但是通常如果是说做应用的岗，可能很多使用的是远端模型，如果你使用的是远端模型而且是应用岗，这个问题被考察到的概率不大。

那么我们为什么要讲这个问题，如果你按照我的思路，去使用了本地模型，然后微调，量化，推理走了个大概的话，重点是你用了本地模型，那么面试官很可能在问你项目的时候，就会涉及到硬件问题。 问：那你的机器能跑得了这个模型吗？ 这个参考夸克千问一面以及Deepwisdom 一面。 在这里我们讲清楚 模**型能不能跑，跑得有多快**和什么相关，有了这个背景，你就比较清楚如何去回答这个面试官问题了。

6.1 模型能不能跑

> 这个章节其实涉及的内容也很复杂，涉及训练，推理。要是都弄得特别透彻的话，需要结合具体的算法来计算，需要理解神经网络的结构，训练过程，具体的算法。如果没有深度学习背景，算法基础，推理的基础，肯定看不懂所有的细节，不过你可以抓大放小，细节你都放掉，掌握具体的原则即可。你看我总结的面试问题和回答示例的时候，也没有回答特别多的细节。如果还在学习应用的朋友，**抓大放小，忽略细节**。如果学习了算法，有推理，训练的基础，可以对细节看得更仔细。另外本期内容也配备了视频讲解，可以结合来看。 另外这一小节的内容其实有非常多的算法，推理部署相关的东西，还是按照我们的思路来说，第一阶段的核心任务是学好应用，这里的内容对于应用岗位属于**拔高项和加分项**，优先级低于应用相关知识。
>
> 这里讲清楚一个模型能不能跑起来，核心就是一个问题：**运行它的设备内存够不够。**
>
> 如果模型跑在 GPU 上，这个设备内存通常就是显存；如果跑在 CPU 上，就是内存（RAM）。同一个 7B 参数的模型，推理时可能只需要 14GB 左右设备内存，而用 PPO 做强化学习对齐时可能需要超过 200GB。所以在回答"需要多少内存/显存"之前，我们必须先搞清楚：**现在是哪个阶段、跑在哪类设备上。**

6.1.1 背景：先搞清楚"问的是哪个阶段"

当有人问"训练一个大模型需要多少内存/显存"时，这个问题就过于笼统，缺少了细节—因为一个大模型从诞生到被用户使用，要经历多个完全不同的阶段，每个阶段的内存需求差异极大。同一个 7B 参数的模型，推理时可能只需要 14GB 左右设备内存，而用 PPO 做强化学习对齐时可能需要超过 200GB。所以在回答"需要多少内存/显存"之前，我们必须先搞清楚：**面试官说的是哪个阶段、跑在哪类设备上。**所以第一部分，给大家介绍一下大模型的生命周期。大模型的生命周期，也是建议大家需要理解和掌握的，虽然大家的岗位和工作可能只聚焦在一块，但是你需要知道他整个生命周期。

6.1.1.1 大模型完整生命周期

> 掌握这个图也可以看出大模型的大致岗位。 一般说算法，Infra, 算法工程，说的其实比较笼统，那对应会的事就是这里的RLHF（后训练），或者做模型部署推理的。 大模型应用工程 不处于这个图中，而是使用部署好的模型接口，直接调用，在上层做应用。 当然大模型算法一部分，就是搞研究，读论文，修改数理公式，创新个RL的方法呀，让整个训练效果更好，这种要求就会更高，一般要求论文，顶级学历（c9 硕博）。
>
> 综上，大模型方向岗位的门槛一般是 算法（纯研究）> 算法工程、infra,推理 > 大模型应用

1. 预训练

这是大模型的"出生"阶段。用海量文本数据（几万亿 token）训练模型预测下一个 token，让模型学会语言规律、世界知识和推理能力。这个阶段由 OpenAI、Meta、DeepSeek 等顶级团队完成，成本动辄数百万美元，普通人和工程师直接下载现成的 Base Model 即可，无需自己做。

2. SFT

Base Model 只会"续写文本"，不会"回答问题"。SFT 用人工标注的（问题, 高质量回答）数据对，教会模型遵循指令、按格式输出。这是让模型从"自动补全"变成"AI 助手"的关键一步。

3. 奖励模型训练

收集人类对不同回答的偏好数据（"A 回答比 B 好"），训练一个打分模型。这个模型之后会充当"AI 评委"，给策略模型的输出打分，替代人类实时反馈。

4. 策略更新

用强化学习算法（PPO 或 GRPO）驱动模型不断优化，目标是在奖励模型认可的方向上提升输出质量，同时不偏离 SFT 模型太远（通过 KL 惩罚控制）。这才是严格意义上的"RLHF"中的 RL 部分。

5. 部署推理

训练完成后，模型被部署给用户使用。这个阶段不需要梯度和优化器状态，运行内存需求大幅下降。如果部署在 GPU 上，主要看的就是显存；如果部署在 CPU 上，主要看的就是内存（RAM）。可以通过量化（INT8、INT4）进一步压缩，让模型跑在消费级显卡或普通 CPU 机器上。

6.1.2 基础知识铺垫

6.1.2.1 核心原则

运行内存需求 = 运行设备上每样东西的大小之和。

所以搞清楚两件事就够了：**放什么** + **每样多大**。

> 这里的"运行设备"可以是 GPU、CPU，也可以是 NPU/TPU 等加速器。跑在 GPU 上时，这部分内存通常叫**显存**；跑在 CPU 上时，就是**内存（RAM）**；跑在其他加速器上，则更准确地叫**设备内存**。后文为了讲解方便，很多地方仍会用"显存"举例，但底层计算逻辑是一致的。

在讲"放什么"之前，先明确**两个阶段**在干什么：

- **训练阶段**：典型的深度学习训练过程——前向传播计算 loss，反向传播计算梯度，优化器用梯度更新参数。因为要做反向传播，计算设备必须同时持有参数、梯度、优化器状态，还要缓存中间激活值，所以内存压力极大；如果用 GPU 训练，这个压力就体现为显存压力。

- **推理阶段**：模型已经训练好，只做前向传播，输入 prompt 输出 token。不需要更新任何参数，因此梯度和优化器状态完全不存在，运行内存需求大幅下降。

这也是为什么同一个模型，训练和推理的运行内存需求可以相差 5-10 倍。

6.1.2.2 训练 vs 推理：运行内存里住着什么？

```Plain Text
                    ┌──────────────────────────────────────┐
                    │  训练阶段                            │  推理阶段
┌──────────────────┐└──────────────────────────────────────┘└──────────────────┐
│ 模型参数         │  ✅ 模型参数  ✅ 梯度  ❌ 梯度（不需要）  ✅ 优化器状态  ❌ 优化器状态（不需要）  ✅ 激活值（前向传播缓存）  ❌ 激活值（不需要缓存）  ❌ KV Cache（通常不涉及）  ✅ KV Cache  │
└──────────────────┘                                                
```

> 占内存，撑到　　占内存，用完即丢
> 反向传播结束才能释放

推理专属

KV Cache

KV Cache可以理解为推理时一个空间换时间的策略。推理时，模型自回归生成每个新 token 都需要"回顾"之前所有 token 的注意力键值（Key-Value）。KV Cache 把这些中间结果缓存下来，避免每步都重新计算。随上下文长度和并发请求数增长，长上下文或高并发场景下不可忽视。

6.1.2.4 精度与字节数

> 混合精度训练：刻意将低精度（bf16/fp16）和高精度（fp32）混合使用——在能省的地方用低精度（前向传播、反向传播计算，节省显存和加速），在必须保精度的地方用高精度（梯度存储、参数更新，防止数值不稳定）。
>
> 这里在**面试网龙的时候也被问过，面试官问bf16和fp16的区别。**

这里的“精度”决定了每个参数、梯度或中间激活值用多少字节存储。字节数越低，显存占用越小、吞吐通常越高；但数值范围和有效精度也会下降，训练时更容易遇到溢出、下溢或更新不稳定。因此训练常用混合精度：参数和计算尽量用 bf16/fp16，梯度累积、优化器状态等关键部分保留 fp32。

- **fp32（4 字节）**：精度高、数值稳定，但显存占用最大、计算吞吐较低。常用于**梯度、优化器状态、关键累积计算。**
- **bf16（2 字节）**：和 fp32 接近的数值范围，不易溢出；有效精度低于 fp32。**大模型训练的常用选择**，覆盖训练参数、前向反向计算、推理。
- **fp16（2 字节）**：有效精度比 bf16 略高，但数值范围更小，训练易溢出，常需 loss scaling。多用于**推理、混合精度训练、较老 GPU 加速。**
- **INT8（1 字节）**：量化压缩，显存更省，但带来少量精度损失。用于**量化推理。**
- **INT4（0.5 字节）**：压缩最激进，显存占用最低，但对模型效果和量化方法更敏感。用于**极致压缩推理、QLoRA。**

6.1.2.5 激活值大小的计算

> 激活值的大小计算就比较复杂了，和超参数，数据量，模型结构都有关系。大家如果看不懂很正常，缺少算法基础，还是像我们前头说的原则，抓大放小，我下面也总结了激活值大小大概的估算方法。看不懂先有大概印象就行。

激活值不像参数一样固定，它由四个变量共同决定：

- **B（batch size）**：一次喂多少条数据 — 超参数，可调。
- **L（sequence length）**：每条数据有多少个 token — 由数据集和截断设置决定，部分可控。
- **H（hidden dimension）**：模型隐藏层维度 — 模型架构决定，选定后固定。
- **N（层数）**：Transformer 的层数 — 模型架构决定，选定后固定。

粗略公式（bf16，每一层）：

$$\text{单层激活值} = B \times L \times H \times 10 \sim 12 \text{ 字节}$$

$$\text{全模型激活值} = B \times L \times H \times N \times 10 \sim 12 \text{ 字节}$$

> 这里的 10\~12 字节是经验系数，不是数学常数。不同框架、是否使用 FlashAttention、是否开启梯度检查点，都会影响真实激活值大小。它适合作为视频讲解里的快速估算。

7B 模型举例（\$H=4096\$，\$N=32\$）：

- **B=1, L=2048**：约 **2.5 GB**
- **B=4, L=2048**：约 **10 GB**
- **B=8, L=2048**：约 **20 GB**
- **B=4, L=8192**（长文本）：约 **40 GB**

快速估算口诀（不想算细节时）：

> 7B 模型，batch=4，seq=2048 → 激活值约 **10 GB。**seq 翻倍 → 激活值翻倍；batch 翻倍 → 激活值翻倍

6.1.3 后训练阶段

后训练的每个算法，在显存里放的东西不同，需要按统一假设分别估算。下面统一用 **7B 模型、混合精度（bf16 参数 + fp32 梯度/优化器）** 作为基准。

> **基础换算**：7B 参数
>
> 参数（bf16）：7B × 2字节 = **14 GB**
>
> 梯度（fp32）：7B × 4字节 = **28 GB**
>
> 优化器状态 Adam（fp32，动量+方差）：7B × 4字节 × 2 = **56 GB**
>
> 激活值：随 batch size 和 seq_len 变化，下面各节单独说明
>
> 注意：这里“梯度”和“优化器状态”都按 fp32 存，但数量不同：梯度只有一份，所以是 4字节/参数；Adam 优化器状态有动量 m 和方差 v 两份 fp32，所以是 2 × 4 = 8字节/参数。

6.1.3.1 SFT — 全参微调（Full Fine-Tuning）

显存里有什么：

所有参数都参与训练，梯度和优化器状态覆盖全部 7B 参数。

- **模型参数（bf16）**：7B × 2 字节 = **14 GB**
- **梯度（fp32）**：7B × 4 字节 = **28 GB**
- **优化器状态 Adam（fp32）**：7B × 4 字节 × 2 = **56 GB**
- **激活值**：B·L·H·N·12 字节，典型 **10–40 GB**

激活值典型范围（7B，\$H=4096\$，\$N=32\$，bf16）：

- B=4，L=2048 → 约 10 GB
- B=8，L=2048 → 约 20 GB
- B=4，L=8192（长文本）→ 约 40 GB

结论：固定项已达 98 GB，单张 A100（80GB）仅凭固定项就装不下，必须多卡或开启梯度检查点。

6.1.3.2 SFT — LoRA 微调

核心思路：冻结原始模型参数，只在每层注入小的低秩矩阵（rank r，通常 r=8\~64），只训练这些新增的 adapter 参数。

LoRA 可训练参数量估算（r=16，7B 模型约 32 层，每层注入 Q/V 矩阵）：

- 每个 adapter：2 × (hidden_dim × r) ≈ 2 × (4096 × 16) ≈ 13万参数

- 全模型 adapter 总量 ≈ 约 **4000万参数（0.04B）**，仅占原模型的 0.5%

7B 模型 SFT LoRA 微调显存拆解（LoRA 参数量按 \~0.04B 估）：

- **原始模型参数（bf16，冻结、无梯度）**：7B × 2 字节 = **14 GB**
- **LoRA 参数梯度（fp32）**：0.04B × 4 字节 ≈ **0.16 GB**
- **LoRA 优化器状态 Adam（fp32）**：0.04B × 4 字节 × 2 ≈ **0.32 GB**
- **激活值**（同全参）：B·L·H·N·12 字节，典型 **2.5–10 GB**

激活值典型范围（7B，\$H=4096\$，\$N=32\$，bf16）：

- B=1，L=2048 → 约 2.5 GB
- B=4，L=2048 → 约 10 GB

> 注意：LoRA 的激活值和全参 SFT **近似同量级**——因为前向传播时整个模型都要跑，激活值覆盖全部 32 层，不会因为"只训练少量参数"就消失。真实工程里可能通过冻结层优化、梯度检查点进一步降低。

结论：单张 4090（24GB）在短序列、小 batch 下可以跑 LoRA，但比较贴边；更常见的消费级方案是 QLoRA（把冻结的底座模型量化存储）。显存大幅下降的原因是**梯度和优化器状态只覆盖极少量 adapter 参数**，而不是全部 7B。

6.1.3.3 RL 对齐 — PPO 算法

> 这里涉及到强化学习的两个算法，PPO和GRPO。没有强化学习的背景肯定看不懂，可以先理解或者背诵。 如果想要学习强化学习，可以用零基础学习算法章节的后训练SKILL去学习，这个SKILL主要核心就是在讲解这两个算法和区别。

PPO 需要同时在显存里维护多个模型（这里按最容易理解的"独立 Actor + 独立 Critic + Reference + Reward Model"估算）：

PPO 同时持有 4 个 7B 模型，显存压力极大：

- **Actor 策略模型**（生成回答，训练目标）— **训练中**：参数 + 梯度 + 优化器 + 激活值，**108–138 GB**
- **Critic 价值网络**（估计每个状态价值，辅助计算优势）— **训练中**：同 Actor，**108–138 GB**
- **Reference Model**（SFT 冻结版，计算 KL 惩罚）— **冻结**：仅参数，**14 GB**
- **Reward Model**（给回答打分）— **冻结**：仅参数，**14 GB**

合计 ≈ (108\~138) + (108\~138) + 14 + 14 ≈ 244\~304 GB

为什么需要 Critic?

PPO 用 GAE（广义优势估计）计算每个 token 的优势值，需要一个能输出 $V(s)$（状态价值）的网络，这个 Critic 网络和 Actor 同等规模。

结论：7B PPO 通常至少 4 张 A100 80GB 起步，并且需要分片、梯度检查点等工程优化，是后训练中显存开销最大的算法。

6.1.3.4 RL 对齐 — GRPO 算法（DeepSeek R1 方案）

GRPO 的核心改动：去掉 Critic，改用"组内相对奖励"估计优势——同一个 prompt 生成 G 条回答，用组内均值做 baseline，不需要价值网络。

GRPO 相对 PPO **去掉了 Critic**，是 R1 训练得以可行的关键优化. **GRPO 需要的模型**：

- **Actor 策略模型**（生成回答，训练目标）— **训练中**：**108–138 GB**
- **Reference Model**（计算 KL 惩罚）— **冻结**：仅参数，**14 GB**
- **Reward Model**（规则函数可替代）— **可选**：**0–14 GB**

合计 ≈ 122–152 GB，相比 PPO 节省了一个完整 Critic（≈100 GB），算力门槛下降了一档。

合计 ≈ (108\~138) + 14 ≈ 122\~152 GB（不用独立 RM 的情况）

额外注意：GRPO 每个 prompt 要采样 G 条回答（通常 G=8），激活值比普通 SFT 更大，实际偏向范围上限。

对比 PPO 的节省：

- 去掉 Critic：直接省去一整个训练中的模型（108\~138 GB）
- 去掉 Reward Model（用规则打分）：再省 14 GB

- **显存从 \~244\~304 GB 降到 \~122\~152 GB，减少约一半**

结论：7B GRPO 单张 A100 80GB 无法完成（122\~152GB > 80GB），通常至少需要 2 张 A100 起步，并配合分片或梯度检查点；相比 PPO 的 244\~304GB，显存压力大约减半。这是 DeepSeek R1 选择 GRPO 的核心工程原因之一。

6.1.4 部署推理阶段

推理阶段只做前向传播，没有反向传播，因此**梯度、优化器状态、训练用的激活值缓存全部消失。**主要显存开销只剩两样东西：模型参数 + KV Cache（实际框架还会有少量临时 workspace）。

6.1.4.1 模型参数：精度决定基础显存

推理时参数可以用比训练更低的精度，进一步压缩显存。

7B 模型各精度下的参数显存：

- **bf16 / fp16（2 字节）**：**14 GB**，标准推理，效果无损。
- **INT8（1 字节）**：**7 GB**，量化推理，效果略有损失。
- **INT4（0.5 字节）**：理论 **3.5 GB**，实际常见 **4–5 GB**（因为还要存量化 scale 等元数据），极致压缩。

> **量化**：将参数从高精度（bf16）映射到低精度（INT8/INT4）存储，推理时动态还原计算。显存减半，但精度有损失，模型越大量化损失越小（70B模型 INT4 通常还可接受，7B INT4 可能有明显退化）。

6.1.4.2 KV Cache：随上下文增长的动态开销

> 这里面公式又很复杂，需要专门做过部署推理的人才会深入理解，还是抓大放小，有个印象就行。

推理时每生成一个 token，都需要回顾之前所有 token 的注意力键值（Key-Value）。KV Cache 把这些结果缓存起来避免重复计算（空间换时间）。

KV Cache 大小公式：

$$\text{KV Cache} = 2 \times B \times L \times N \times N_{kv_heads} \times D_{head} \times \text{字节数}$$

- 系数 2：Key 和 Value 各一份
- B：当前同时参与推理的序列数，也可以叫推理时的有效 batch size；在线服务里通常可近似理解为“正在生成中的活跃并发请求数”
- L：上下文长度（已生成的 token 数）
- N：层数
- N_{kv_heads}：KV 头数量
- D_{head}：每个注意力头的维度

> 如果是传统 MHA，KV 头数量通常等于注意力头数量，KV Cache 会更大；如果是 GQA/MQA，KV 头数量更少，KV Cache 会明显变小。所以 KV Cache 也和模型结构有关。

7B 模型举例（\$N=32\$，\$D_{head}=128\$，bf16，按 8\~32 个 KV 头给范围）：

KV Cache 的显存几乎与 `batch × context_length` 成正比：

- **B=1, L=2048**：约 **0.25–1 GB**
- **B=1, L=32K**：约 **4–16 GB**
- **B=16, L=2048**：约 **4–16 GB**
- **B=16, L=32K**：约 **64–256 GB**（多并发 + 长上下文是显存炸裂的典型场景）

> 长上下文 + 高并发时，KV Cache 可以轻易超过模型参数本身的显存占用，这是部署时最容易被忽视的显存杀手。

6.1.4.3 推理总显存 = 参数 + KV Cache

7B 模型典型推理场景：

推理总显存 = 模型参数 + KV Cache，5 个典型场景：

1. **单用户短对话（bf16）**：14 GB 参数 + 0.25–1 GB KV ≈ **14.25–15 GB**，4090 单卡可跑。
2. **单用户长上下文 32K（bf16）**：14 GB + 4–16 GB ≈ **18–30 GB**，4090 视模型结构而定。
3. **量化单用户（INT4）**：4–5 GB + 0.25–1 GB ≈ **4.5–6 GB**，消费级卡轻松。
4. **多并发 16 路、短对话（bf16）**：14 GB + 4–16 GB ≈ **18–30 GB**，A100/80G。
5. **多并发 16 路、长上下文（bf16）**：14 GB + **64–256 GB** ≈ **78–270 GB**，必须多卡或限流。

6.1.5 汇总对比表

> 以下均以 **7B 模型、混合精度（bf16 参数 + fp32 梯度/优化器）** 为基准。训练激活值按 10\~40GB 估算；推理 KV Cache 按 8\~32 个 KV 头给范围

- **SFT 全参微调**：**108–138 GB**（参数 14 + 梯度 28 + Adam 56 + 激活 10–40）
- **SFT LoRA**：**17–25 GB**（冻结参数 14 + adapter 梯度&优化器 ≈0.5 + 激活 2.5–10）
- **RL PPO**：**244–304 GB**（Actor 108–138 + Critic 108–138 + Ref 14 + RM 14）
- **RL GRPO**：**122–152 GB**（Actor 108–138 + Ref 14，去掉 Critic）
- **推理 bf16，单用户**：**14.25–15 GB**（参数 14 + KV 0.25–1）
- **推理 bf16，16 并发 / 2048**：**18–30 GB**（参数 14 + KV 4–16）
- **推理 INT4，单用户**：**4.5–6 GB**（参数 4–5 + KV 0.25–1）

6.1.6 附：根据显卡选方案

我们在选卡时，应该自己用上面的知识去算一下，给一个表让大家有个直观印象。

- **消费级 4090（24 GB）**：7B bf16 推理；7B QLoRA / LoRA 微调（控制 batch、seq_len）；30B 级 INT4 量化推理。
- **专业卡 A100（80 GB）**：7B LoRA 微调较轻松；7B 全参 SFT 单卡不够；7B GRPO 通常需要 2 张起步。
- **多卡 A100 × 4（320 GB）**：7B 全参 SFT / GRPO 较稳；7B PPO 可做，但需要分片、梯度检查点等工程优化。
- **多卡 H100 × 8（640 GB）**：7B PPO 较稳；70B 级全参训练通常还需要 ZeRO、并行切分、梯度检查点或优化器状态压缩。

6.1.7 附：面试 Q&A

> 其实面试过程中很多时候对于类似的问题，面试官本身的表述是模糊的。比如6.1.7.1， 问SFT要多大显存？但是SFT里有全量微调，Lora等部分参数微调，算法不一样，结果是不一样的。回答问题的时候，我们自己心里要是清楚的，主动澄清清楚前提和具体细节，然后给面试官举例说明，可以不用说得那么细致，说一下大致的估算即可。如果面试官深入问了，可以再给他讲细节，也就是我们上面笔记介绍的细节部分

6.1.7.1 Q1：7B 模型做 SFT 大概需要多大显存?

这个问题要先看是**全参 SFT**还是 **LoRA/QLoRA**。原则上，训练显存主要由参数、梯度、优化器状态和激活值组成。

如果是全参 SFT，通常会按照业界大模型训练里常见的**混合精度训练口径**来估算：模型参数用 bf16/fp16 存储，梯度和 Adam 优化器状态保留 fp32。这个口径的来源其实就是不同精度的字节数：7B 参数用 bf16/fp16 是 2 字节/参数，所以参数本身约 14GB；梯度按 fp32 是 4 字节/参数，所以约 28GB；Adam 还要保存一阶动量和二阶动量两份 fp32 状态，所以是 2 × 28GB，约 56GB。再加上激活值，全参 SFT 通常要 **100GB 以上**，单张 80GB A100 一般不够，需要多卡或分片优化。

如果是 LoRA/QLoRA，因为只训练少量 adapter 参数，梯度和优化器状态会小很多。以 7B 为例，LoRA 通常 **20GB 左右**可以起步，QLoRA 会更省，消费级 24GB 显卡也有机会跑起来，但要控制 batch size 和 sequence length。

6.1.7.2 Q2：7B 模型做强化学习大概需要多大显存?

强化学习阶段要看具体算法，因为不同算法需要同时放进显存的模型数量不一样

如果以 PPO 为例，通常要同时维护 Actor、Critic、Reference Model 和 Reward Model。Actor 和 Critic 都是训练中的完整模型，单个就接近全参微调的开销，所以 7B PPO 往往需要 **200GB 以上显存**，通常需要多

张 A100/H100，并配合 ZeRO、梯度检查点等工程优化。

如果以 GRPO 为例，它去掉了 Critic，用组内相对奖励来估计优势，显存压力会明显下降。7B GRPO 大致可以按 Actor + Reference Model 来估，通常在 **100GB 以上**，相比 PPO 大约省一半，但单张 80GB A100 仍然比较紧张。

6.1.7.3 Q3：7B 模型部署推理大概需要多大显存?

部署推理和训练不一样，推理不需要梯度和优化器状态，主要看两个东西：**模型参数**和 **KV Cache**。

如果用 bf16/fp16 推理，7B 模型参数大约是 **14GB**。KV Cache 的大小差异会比较大，主要和上下文长度、并发请求数、模型的注意力结构以及 KV Cache 本身的存储精度有关。单用户、短上下文场景下，再加少量 KV Cache，整体大约 **15GB 左右**，24GB 显卡一般可以跑。

如果做量化，INT8 大约 7GB，INT4 实际常见大约 4\~5GB，再加 KV Cache，可以进一步降低显存。但如果是长上下文或高并发，KV Cache 会快速增长，这时显存瓶颈可能不再是模型参数，而是 KV Cache。

6.1.7.4 Q4：你说在CPU机器上跑7B的模型，这个可以跑得动吗?

> 这个是阿里夸克千问一面面试官的原题。我当时被问到这个问题，其实我答得很笼统，我就说模型比较小，也做了量化。但当我学习了今天这个内容以后，我才总结出这个更细致，详实的答案。当然当时我的回答也并没有阻碍我通过面试，反正整个夸克千问我是拿到了offer的。我们都会按照100分甚至是120的回答去准备，但是通过一次面试，不意味着面试每个问题都要回答到100分。
>
> 另外，其实我分析来说，其实当时的面试官我估计也不懂这个细节。原因是本身就是应用岗，夸克千问整体面试下来给我的感觉，他们这个岗位不会做算法，不做训练，纯专注于应用，强度又高，其实面试官不知道这块内容很正常。他问到这个问题，是因为项目写了在cpu机器上本地跑7B模型。所以我答得模糊这事也就过去了。不过在这种情况下，当时我能够像下面这个答案一样回答得很具体，甚至面试官感兴趣，继续问我，我给他讲一下，这样面试会更加分。如果在面试官不懂的地方，你能够给他讲得很清楚，会极大地加分。

可以跑，但要分两个角度看：**能不能装下**，以及**跑得快不快**

首先从能不能装下看，CPU 机器没有 GPU 显存，主要看的是**内存（RAM）够不够**。这里我按实际使用场景来回答：我把 7B 模型量化成了 **INT4**，并且使用 **llama.cpp** 做 CPU 推理。

推理阶段的内存主要是：**量化后的模型参数 + KV Cache + llama.cpp 运行时开销**。7B 模型做 INT4 量化后，模型文件通常大约 **4\~5GB**。KV Cache 的大小和上下文长度、并发数有关，但本地 CPU 推理一般是单用户、单路对话，也就是并发数基本可以按 **1** 来算。如果上下文长度是 2K\~4K，7B 模型的 KV Cache 通常大约是 **0.5\~2GB** 这个量级，具体取决于模型结构和 llama.cpp 的 KV Cache 精度设置。再加上系统和框架开销，整体大概需要 **8\~10GB 内存**，所以 16GB 内存的普通消费级机器一般可以跑起来，8GB 会比较紧张。

其次从速度看，CPU 可以跑，但速度和 GPU 不是一个量级。INT4 量化会减少模型参数读取和计算压力，llama.cpp 又专门针对 CPU 做了优化，比如多线程、矩阵计算优化、GGUF 格式加载等。按普通消费级 CPU

来估算，一次普通长度的回答大概可能在 **20\~30 秒左右**；另外考虑到我们的任务对实时性要求不高，所以这个时延是完全可以接受的。

6.1.8 视频讲解

本小节的内容涉及范围很广，算法，部署，深度学习，大家可能学起来有难度。我录了一个视频，视频的讲解面向于0基础，相信大家都能看懂。

https://www.bilibili.com/video/BV1Lb5S6qEuA/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

6.2 跑的有多快

跑的有多快，相对来说更复杂：取决于你的算力和任务。

核心原则：算力越强，并行度越高 -> 越快。

影响因素：

├── 硬件算力（ 每秒浮点运算次数，好的显卡，cpu 算力更强，速度也更快）

├── 内存带宽（数据搬运速度）

├── 并行度（多核/多卡）

└── 软件优化（推理框架）

└── 任务类型（任务复杂，prompt 和 时间复杂度成o(n^2）关系（从Attention结构上来讲））

其实这个问题相对来说就更加复杂，在《大模型工程师面试实战- 算法原理，开发实践与系统部署》（清华大学出版社） 书中是有专门讲解这个推理速度相关的问题，以及如何优化。这个和硬件，Infra的岗位相关，是他们优化的工作。所以具体还是很复杂的，但是作为应用，我们了解到这里就可以啦！

7 不同模型的对比和选择

> 有些公司在面试的时候，喜欢问的一个问题是，你的项目用的什么模型，为什么要用这个模型，比如你可以看到笔记面经里的小红书，平安证券，都有类似的问题。 其实使用什么模型这个事情，不光和模型本身有关，也和公司层面（比如公司有自己自研模型，那产品肯定接入自家模型），政治方面因素相关（中国很多产品，政治上也不让用国外模型，国外的模型比如Claude系列，也会限制中国的使用）。

这一章，首先讲解 7.1 讲解市面上所有的主流模型的典型特点。 7.2 结合面试题：你的项目使用什么模型 来讲解 项目模型选择的策略。 7.3 讲解一下最新典型旗舰模型的特点。

7.1 主流模型的对比

本章会概括所有主流模型，看完这一章，你能够对市面所有模型有一个概念，印象，不追求多深入，但是你最少能知道每个模型是那个公司，他最核心的特点，优缺点，大概的选型原则。注意模型迭代速度其实非常快，这个章节时效性很强（此章节整理于2026/5/10）。不过**大体的趋势和特点**还是相对短期稳定的，比如GPT模型强在生态，Claude模型强在编程，Gemini模型在各大Benchmark屠榜。看完本节，你会了解目前最主流的模型的特点，优缺点，就算不从面试的角度来说，作为谈资也是值得一看的。

7.1.1 通用大模型

7.1.1.1 闭源阵营

7.1.1.1.1 GPT (OpenAI)

> **一句话定位：** AI 开山鼻祖，用户量大，生态好，通用能力最强。

- **当前旗舰模型：** GPT-5.5 (2026-05-05)

模型矩阵

- **GPT 主线（GPT-5.5）**：通用聊天 / 多模态 — 日常全能任务。
- **O 系列（最新 O 系列推理模型）**：纯推理线，回答前先"思考" — 数学、逻辑、多步工程。
- **Image 系列（gpt-image-2）**：图像生成，取代 DALL·E — 海报、文字渲染、图文。
- **语音 / 实时（Realtime API + 语音模型）**：低延迟语音交互 — 语音助手、电话机器人。
- **视频系列**：已无在售产品（Sora 已停服）。

核心优势

- **生态最大：** 数亿用户、最多第三方插件与应用构建在 GPT 上。
- **多模态最齐全：** 文 / 图 / 语音 / 实时全覆盖（视频暂缺）。
- **顶级文字渲染：** `gpt-image-2` 的文字渲染业内顶级，能正确拼单词、做海报。

劣势与短板

- **不再是单项绝对王者：** 榜单常被 Gemini 反超，编码上被 Claude 压制。

7.1.1.1.2 Gemini (Google)

> **一句话定位：** 榜单领跑者；最大筹码是深度嵌入 Google 全家桶。

- **当前旗舰模型：** Gemini 3.1 Pro

模型矩阵

- **Pro（Gemini 3.1 Pro）**：旗舰 — 难题、长文档、复杂多模态。

- **Flash（Gemini 3 Flash）**：速度款，约 Pro 的 90–95% 能力 — 日常高频任务，更快更便宜。

- **Flash-Lite（Gemini 3.1 Flash-Lite）**：极致性价比 — 高吞吐 / 大规模批处理。

- **Deep Think（Gemini 3.1 Deep Think）**：深度推理模式（仅 AI Ultra 订阅） — 科研、工程、复杂技术难题。

核心优势

- **多模态强：** 拍照识物体场景实用度高（如修车工拍零件即得识别结果）。

- **生态优势**：集成了Google全家桶，比如Gmail, Chrome（严格说这个不算模型的能力，算应用的能力）

- **榜单领跑：** 2025 AIME测试上（测试数学推理），Gemini模型领跑，其次是Claude, 然后是GPT。

7.1.1.1.3 Claude (Anthropic)

> **一句话定位：** 编码、长文档、Agent 任务的专家；多模态偏弱，但单项做到顶。

- **当前旗舰模型：** Claude Opus 4.7 (2026-04-16 发布)

模型矩阵

Anthropic Claude 三档定位：

- **Opus（Claude Opus 4.7）**：旗舰，研究实验室级 — 硬骨头、复杂工程、深度分析。

- **Sonnet（Claude Sonnet 4.7）**：中端，约 Opus 80% 实力 — 日常编码、长文档处理，性价比高。

- **Haiku（Claude Haiku 4.7）**：轻量级，速度快、成本低 — 高并发、实时响应、批处理、简单任务。

核心优势

- **编码优势：** LiveCodeBench、SWE 等"真实代码库"基准排名靠前；开发者社区编码首推。

- **长文档分析：** 合同、论文、整套代码库给出结构化摘要而非零散答案。

- **风格客观：** 风格"最不谄媚"，直接告诉你想法烂在哪，不无脑捧场。

劣势与短板

- **多模态弱：** 无原生图像生成，纯文本 / 代码专家。

- **生态缺位：** 没有自家语音 / 视频线产品。

7.1.1.1.4 Grok (SpaceXAI)

这个Grok估计不太会问了，作为国外模型，不如前三个亮眼。我还是总结大家了解一下。

> **一句话定位：** 实时 X（Twitter）集成 + 最自然的对话感；尺度大、争议多。

- **当前旗舰模型：** Grok 4.3 Beta (2026-04-17 发布)

模型矩阵

- **主线（Grok 4.3 Beta）**：当前旗舰，推理 / 通用 — 综合任务、推理。

- **Fast 档（Grok 4.1 Fast）**：工具调用 / 代理工作流 — Agent、长上下文工具链。

- **图像 / 视频（Grok Imagine + Aurora）**：图像 + 视频统一管线 — 出图、短视频。

核心优势

- **实时 X 集成：** 拉热门讨论、突发新闻摘要、舆情分析比爬虫式模型快得多。（这个严格来说也是产品的能力，不是模型的能力）

- **对话自然：** 对话风格自然放松，像跟人聊天，半夜 3 点也秒回。

- **言论尺度大：** 会回答其他模型常拒答的问题。

劣势与短板

- **态度主观：** 严肃分析场景下，"会反驳的模型"反而更受偏好。

- **争议多发：** 多次因系统提示问题、深伪图像、Grok Imagine "Spicy" 模式等卷入争议。

7.1.1.2 开源阵营

开源意味着：完全控制 = 数据不出门 + 无 API 费 + 离线可用（电费你出）。

7.1.1.2.1 DeepSeek

> **一句话定位：** 开源 + 价格屠夫 + MoE 高效；前沿 AI 不再需要十亿美元级预算的代表。

- **当前旗舰模型：** DeepSeek-V4 (2026-04-24 发布预览)

模型矩阵

- **V 系列 / 通用主线（V4-Pro / V4-Flash）**：通用聊天 + 思考 / 非思考双模式 — 日常 + 推理一体。

- **R 系列 / 纯推理（R1）**：引爆 2025 "Sputnik 时刻"，目前路线已并入 V 系列。

- **Coder（DeepSeek-Coder）**：编码 — 代码补全、生成。

- **Math / Prover（Math-V2 / Prover）**：数学与定理证明 — 数学竞赛、形式化数学。

- **视觉（DeepSeek-VL2）**：图文理解。

核心优势

- **极致高效：** 架构 MoE + MLA（自 V2 起），参数量大但每次只激活一小部分，又快又省。

- **开源与低价：** 权重 MIT 开放可本地部署；API 价格远低于美国一线。

- **垂直矩阵齐全：** 编码 / 数学 / 证明 / 视觉各有专精模型。

7.1.1.2.2 其它模型

剩下的模型，我用一个表格，一句话总结其它模型最核心的特点：

- **Meta（LLaMA 4）**：最高约 400B 参数 — 开源浪潮起点，2026 上半年有更大版本。

- **阿里（Qwen 3.5）**：双 3090 本地可跑 — 多语言强，本地体验逼近 Claude Sonnet。

- **智谱 AI（GLM-5）**：203K tokens 上下文 — 开源榜顶尖，许可证商用友好。

- **Moonshot（Kimi K2.5）**：Perplexity 可用 — 数学 / 推理强，AIME 2025 高分。

- **Mistral（Mistral 系列）**：小模型见长 — 以小博大，欧洲语言出色。

7.1.2 视觉生成模型

> 视觉生成模型和视频生成模型不作为重点。从面试角度来说，那种专门做视频，多模态相关的岗位会侧重视觉模型和视频生成模型，找工作如果没有这方面的倾向（比如我），不需要深入了解。所以这两章只是简单总结，大家有个印象就行。

- **美感极致 → Midjourney**：艺术质量之王，画面电影感、精致；约 \$10/月起。

- **简单易用 → gpt-image-2**：内嵌 ChatGPT，原生多模态，**文字渲染业内顶级**（能正确拼单词、排版海报）。

- **精准理解 → Flux**：开源、本地、免费；**Prompt 语义还原度业内领先。**

- **极度可控 → Stable Diffusion 3.5**：开源本地，配合 LoRA、ControlNet 控制力极强；学习曲线陡，但灵活度无敌。

需要避开的误区

模型是物理分离的： 文本模型和图像模型几乎总是分开的系统。例如，你在 ChatGPT 里画图，实际调用的是 `gpt-image-2`（已取代 DALL·E），而不是 GPT 语言模型本身；Grok 出图也是独立的 Grok Imagine 管线。

7.1.3 视频生成模型

> 本身不做视频内容，面试也很少问，这小节看看就行吧，算是科普了。

- **Sora（OpenAI）**：重大变动——已停服，API 将在 9 月彻底关闭，短期内不可用。

- **Kling 3.0**：主打速度与便利，支持音视频一次生成，榜单表现已反超 Sora 2 Pro。

- **Runway Gen-4.5**：给创作者极强控制力，提供运动笔刷、镜头一致性、相机引导功能。

- **Seedance 2.0**：字节跳动出品，当前文生视频榜单领跑者之一。

- **Grok Imagine**：图像与视频统一管线，已上线高质量模式 API。

7.1.4 面试问题参考

Q：你平时工作和学习中用过哪些模型，分别用来做什么？

> 面试官可能问这个问题，考一下你对AI 是否聪明熟练地使用AI工具。这个回答是根据上面不同模型的特点总结的，有点多，主要是提供个思路，根据不同模型的特点来在不同的场景下使用。这样回答体现你对不同模型特点都有理解，也会聪明使用。

用过挺多的，不同场景我会选不同的模型，因为各家模型的优势方向差别还是比较明显的。

编程和写代码这块，我基本首选 Claude。它在 LiveCodeBench 这类真实代码库评测上一直排名靠前，开发者社区里口碑也最好。日常写代码、调 bug、看别人的代码逻辑，我用 Sonnet，性价比高，够用；遇到比较复杂的架构设计或者系统级的问题，我会升级到 Opus，它在推理深度上确实更强。另外 Claude 有一个特点我很喜欢，它不太会无脑夸你，说法比较直接，方案有问题它会直说，做技术决策的时候反而更可信。

GPT 我主要用在两个场景。一个是日常通用问答，因为它生态最成熟，各种插件和工具支持都齐全，随手查个东西、聊个想法很顺手；另一个是图像生成，`gpt-image-2` 的文字渲染我觉得是目前最好的，做海报、图文内容、有文字的封面，它比其他模型准确多了

Gemini 我主要用来处理长文档和多模态任务。它和 Google Docs、Drive 的整合很自然，工作流上比较顺。日常高频的任务我一般用 Gemini Flash，它的能力大概是 Pro 的 90–95%，但速度更快、价格更低，够用的情况下没必要上 Pro；遇到真正复杂的多模态分析、长文档深度阅读这类场景，才切换到 Pro。

DeepSeek 我用在对成本比较敏感或者需要本地部署的场景。它的 API 价格比美国一线模型便宜很多，而且开源可以本地跑，数据不出门，对于一些不想上传到外部服务器的内容，本地跑 DeepSeek 很合适。数学推理这块它也很强，做一些计算密集型的任务我会考虑它。

Grok 我偶尔会用来做实时舆情和热点信息的查询，因为它直接整合了 X（Twitter）的数据，对突发事件、技术圈动态的感知比其他模型快很多，这块是它的独特优势。

总体来说，我的习惯是：**编程找 Claude，通用和出图找 GPT，长文档找 Gemini，省钱或者本地部署找 DeepSeek，追热点找 Grok。**不同场景用最合适的工具，而不是只盯着一个模型用到底。

7.1.5 参考资料

文档的内容基本按照这个视频来总结的。不过这个视频有一些错误的地方，比如Openai的图像模型用的早都不是DALLE 而是Image，时效性有问题，比如现在claude 4.7，GPT 5.5 都出了。这些我在文章内更正了

https://www.bilibili.com/video/BV1hWo1BXEko/?spm_id_from=333.337.search-card.all.click&vd_source=144bec9c3f54e465073138bed788be1b

7.1.6 视频讲解

直接看这个7.1可能有点枯燥和流水账，这个视频把7.1 小节的内容讲了一遍，结合视频看更轻松

https://www.bilibili.com/video/BV1oP5x6iEcG/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

7.2 模型的选型

> 介绍一个考虑模型选型的模板，这块有一些产品思维，参考文档也是从一个产品经理的论坛文章上看的。

7.2.1 模型选择策略理论

最佳的选型策略应当是从业务需求反推技术选择，构建 **场景 - 能力 - 模型** 的三步决策框架。

第一步：定义场景 (精准描述问题与流程)

选型的起点必须是具体的业务场景，切忌模糊的大词，建议采用“**黄金圈法则**”进行拆解：

- **Why (价值目标) :** 该场景要解决什么核心痛点？（例如：降低20%的客服人工介入率）

- **What (任务定义) :** 具体要完成什么任务？（例如：自动回复用户的订单状态查询）

- **How (工作流程) :** 任务的执行流程与限制是什么？需要多快的响应速度？是否有数据不出境的硬性要求？

第二步：拆解能力 (分析核心能力需求)

将明确好的业务场景，转化为对大模型具体的可量化能力。

- **分类拆解:** 明确场景最看重的是什么能力。是信息抽取、内容生成、复杂逻辑推理，还是外部工具调用（API集成）能力？

- **划分优先级:** 将需求划分为“必要能力”与“加分能力”，并为各项核心能力分配权重。例如：在智能客服中，工具调用准确率可能是必要能力（占50%权重），而多语言翻译可能是加分能力（占10%权重）。

第三步：匹配模型 (寻找场景最优解)

根据拆解出的能力权重，对候选模型进行综合评估：

- **能力适配度:** 拒绝“榜单依赖症”（如通用的MMLU分数）。必须从真实业务中抽样构建场景化测试集（如100个真实案例）进行盲测打分。

- **成本效益比 (TCO):** 不能只对比单次Token调用价格，必须将业务高峰期的调用量波动、本地化私有部署硬件成本、配套人工审核维护成本计算在内。

- **风险可控性:** 评估数据安全隐私、合规要求以及模型供应商的服务稳定性。在金融/医疗等敏感领域，合规和数据不出境往往具备一票否决权。

7.2.2 实战演示

> 我结合我们笔记中的项目：智能预约系统来演示一下上面这个理论的应用。大家要是面试问到这个问题，根据你自己的项目往里面套模板就行。从另一个角度来说，如果你还没有做项目，真的可以考虑一下这个原则，在

> 做项目之前好好考虑一下应该用什么模型。

以一个真实的 Agent 项目为例，演示如何将"场景 - 能力 - 模型"框架落地。该项目背景是：传统按摩房依赖人工前台接听电话，完成预约分配、咨询解答、客户回访等工作，效率低、成本高。因此设计了一套多 Agent 对话系统来替代人工前台。

第一步：定义场景

按摩房 Agent 项目的 Why / What / How：

- **Why（价值目标）**：解放人工前台，实现 7×24 小时自动预约与咨询，降低人力成本。

- **What（任务定义）**：用户通过对话完成技师预约、项目咨询、个性化回访。

- **How（工作流程）**：用户发起对话 → 意图分类 → 分流至预约 / 咨询机器人 → 数据库查询 / RAG 检索 → 确认预约并推送天气提醒。

系统最终拆解为 4 个 Agent：**分类机器人**（意图路由）、**预约机器人**（多轮收集需求 + 数据库查询）、**咨询机器人**（从私有文档 RAG 检索回答）、**行为分析 Agent**（分析历史预约、生成个性化回访）。

第二步：拆解能力

项目所需 LLM 能力及权重：

- **意图分类准确性（必要，35%）**：分类错误会导致整个流程跑偏，是系统的咽喉。

- **工具调用 / Function Call（必要，30%）**：预约机器人需要精准调用数据库查询技师空档。

- **RAG 检索整合（必要，20%）**：咨询机器人须从私有文档中准确检索按摩房信息。

- **多轮对话上下文理解（必要，10%）**：预约流程需多轮收集用户偏好，不能"失忆"。

- **MCP 外部服务集成（加分，5%）**：预约成功后调用天气 MCP Server，生成个性化提醒。

第三步：匹配模型

系统采用**分层选型策略**，不同 Agent 所需能力不同，不必全部使用同一个高价模型：

4 个 Agent 的推荐模型与理由：

- **分类机器人 → GPT-4o-mini / Qwen-Turbo**：任务简单（4 分类），轻量模型成本低、响应快。

- **预约机器人 → Claude Sonnet 4 / GPT-4o**：工具调用稳定性高，多轮上下文理解强，是系统核心。

- **咨询机器人 → Claude Sonnet 4 / DeepSeek-V3**：RAG 整合流畅，文档理解能力强；DeepSeek 成本更低。

- **行为分析 Agent → DeepSeek-R1 / GPT-4o Thinking**：需对历史数据做多步推理，生成个性化回访文案。

7.2.3 面试回答示范

Q：你的项目是如何进行模型选型的?

> 我用这个思路总结答案：
>
> 1. 通过需求确定基本模型：使用上面的**三段理论：业务场景 - 功能 - 模型 来回答。**确认大致模型。
> 2. 从大致意向模型里，做对比（查测评表，资料，做实验等方式）确定最终模型。
>
> 第二点更好一点的方式是通过实验驱动，对比不同模型。但是如果这么说，面试官可能会问你真的怎么做的？怎么测评的，数据集多少，万一你答不上来，可以就从查测评表，查资料来做对比确定而不是做实验确定。

第一段（按需求确定大致模型）：

我在做按摩房智能预约系统的时候，是从**业务场景出发，反推需要什么能力，再去匹配模型。**这个系统有四个Agent——分类机器人、预约机器人、咨询机器人和行为分析 Agent，每个 Agent 的任务差别很大，所以我没有用一个模型一刀切，而是**先按"业务场景 - 功能能力 - 模型"分别拆。分类机器人**只是判断用户意图属于哪一类，任务简单，所以**倾向用轻量模型，成本低、响应快**；**预约机器人是系统核心**，要多轮确认时间和技师偏好、还要调数据库查空档，对工具调用稳定性和多轮上下文要求最高，所以要用主力模型；**咨询机器人**主要做 RAG，从私有文档里检索回答，重点是**文档理解能力**；**行为分析 Agent** 在后台异步跑，需要对历史数据做多步推理，所以倾向**推理型模型。**这样一轮拆下来，每个 Agent 大致用哪一类模型就基本定了。

第二段（在候选模型里对比，最终确定）：

确定大致方向之后，我会在每一类的候选模型里再做一次对比来定型，主要是查各模型的公开测评和资料，结合我们的场景需求来选，而不是盲选最贵的。以分类机器人为例，它属于轻量模型这一档，候选的有 GPT-4o-mini 和 Qwen-Turbo。因为我们的用户提问基本都是中文、而且偏口语化，所以我重点看的是"中文意图分类的准确率"这个维度，同时也参考了调用成本和响应速度。综合下来 **Qwen-Turbo 在中文短文本分类上表现够用、成本更低、响应也快，所以分类机器人最终选了 Qwen-Turbo**；其它几个 Agent 也是按同样的思路，结合它们各自最看重的能力维度去比对确定的。

7.3 热门模型特点介绍

这一章主要是给大家提一个醒。作为最火的，最新的，上热搜的模型，大家可以了解一下它的核心特点，做了什么改进。我知道我们都不是专业做模型的，所以对里面技术细节不会深入了解。我的建议是，新模型，比如gpt5.5, deepseek v4 出来以后，让AI给你总结下它的核心提升和改变，特性就好了。这么做是防止面试官问（因为我之前有被问到过。 我主要提供个思路，打个样。以Deepseek v4为例，让AI总结一下特性。防止面试官和你聊最近最新Deepseekv4, 你啥也不知道。更重要的是这个思路，后面还有各种新模型的发布，都可以用这个策略快速准备下。

7.3.1 DeepSeek V4 新特性和创新

新特性：

1. 百万上下文成为默认能力。全系标配 **1M token 上下文。**推理计算量降至 V3 的约 27%，KV Cache 仅为 **10%，能直接处理整本书、完整代码仓库、长文档分析。**

2. 双模型体系覆盖不同场景。V4-Pro 提供万亿参数级高性能推理能力，而 V4-Flash 在维持相同架构的前提下降低激活参数和成本，实现高吞吐和低延迟的生产部署。
3. 推理效率和成本显著优化。通过压缩注意力和系统级优化，KV Cache 内存占用降低约 90%，整体推理吞吐明显提升，同时 API 成本显著低于同级模型。
4. 原生增强 Agent 执行能力。模型内置工具调用、分步推理和多模式思考能力，在复杂任务拆解和自动化工作流中具备更高稳定性和执行效率。
5. 开源和工业级部署能力并存。模型以 MIT 协议开放完整权重，支持多种硬件平台部署，使其不仅是研究模型，同时具备企业级落地能力

创新

1. 混合注意力机制重构长上下文计算。通过 CSA（压缩稀疏注意力）和 HCA（重度压缩注意力）的协同设计，将原本 $O(n^2)$ 的全量注意力转化为压缩与稀疏选择结合的计算模式，从根本上降低长序列计算成本。
2. 稀疏索引与重要Token选择机制。模型引入索引器和 Top-k 筛选策略，只对最相关的上下文进行注意力计算，从“全量扫描”转变为“关键检索式推理” 。
3. mHC 超连接替代传统残差结构。通过引入流形约束的超连接机制改善深层网络中的信息传播稳定性，使万亿级模型训练更加可控和高效。
4. Muon 优化器提升训练稳定性。采用新型优化算法替代传统 AdamW，在大规模训练中实现更快收敛速度和更稳定的参数更新过程 。
5. 混合精度计算进一步压缩资源消耗。结合 FP4 与 FP8 的混合精度设计，在保持模型性能的前提下降低显存占用并提升计算吞吐。
6. Batch Invariance 提升工程可控性。通过保证相同输入在不同 batch 组织下输出一致，使模型在训练、推理和在线服务中具备更强的可复现性和稳定性。

7.3.2 Jev： 面向软件的快速概率决策模型

Jev是2026年9月份快速火起来的一个模型，如我们一直说的，大模型圈，爆火的内容你不能不会。面试几乎一定会问到。所以2026年9月份，面试前，建议学习一下Jev。 大家也不用担心学不过来，最开始我们只要先理解最基本的内容，最简单的面试问题就够了。如果他持续爆火，我们再深入学习。

这一期视频我就总结的内容都是Jev最核心的内容：**是什么，特点，设计理念，使用方式，技术架构和面试真题**，希望大家可以用1小时的时间，结合视频和文档快速理解Jev是什么，如果后续面试对Jev的考察持续加深，我们也会持续跟进的。

7.3.2.1 概念：Jev 是什么

7.3.2.1.1 和 Opus、GPT 最大的不同

Jev 是 TypeSafe AI 推出的一个决策模型。 和我们熟悉的 Opus、GPT 系列相比，它不是为了写出一段回答，而是为了快速回答软件中的具体选择题。

Opus、GPT 这类模型的典型文本生成方式是**自回归生成**：根据输入和已经生成的内容，预测下一个 token（next-token prediction），再把新 token 接到上下文后面，继续预测，直到回答结束。哪怕要求只返回一个分类标签，典型生成接口也会把它作为 token 序列输出。

Jev 则要求你先定义问题和允许的结果。例如，给它一封邮件，再给出“账务、售后、技术支持”三个选项，它返回类别及对应概率，而不是逐 token 写一段解释。这种方式的定位是**并行采样的、有类型约束的概率决策**；底层架构尚未完整披露。

简单说：**通用 LLM 擅长“组织和生成答案”，Jev 专注于“在定义好的结果空间里做判断”。** LLM 也能分类、返回结构化结果，区别不在于谁能输出 JSON，而在于模型针对什么任务优化。

动图 1：观察逐 token 输出与返回选项概率的差异。动画是接口行为示意，不表示实测速度或已知的内部计算步骤。

7.3.2.1.2 为什么能火：把高频小决策做得快、做得便宜

我们常用 Opus、GPT 做分析、写代码，也会通过 Claude Code 这样的工具完成多步骤任务。但如果只是给上千封邮件分类，关心的往往不是能否写出精彩解释，而是**多久处理完、花多少钱、标签是否可靠。**这里 Claude Code 是应用，Opus、GPT 是模型，不能把两者当作同一层级的基准对象。

一次千封邮件分类演示中，处理 **1,000 封邮件**时，Jev 初版流程耗时 **70 秒、费用 \$0.09**，对照模型 Luna 约需 **5 分钟（300 秒）、\$0.62。**按这组数字计算，Jev 的处理速度约为 Luna 的 **4.3 倍**，**耗时减少约 76.7%，调用成本降低约 85.5%。**

调整后端并行和请求组织后，Jev 又做到 **6 秒、仍为 \$0.09**：相较于上述 Luna 流程，批处理速度约为其 **50 倍，耗时减少约 98%，调用成本仍降低约 85.5%。**其中，50 倍包含了工程优化的贡献，并非同等配置下的模型速度对比；对照模型可核验的完整 ID 尚不明确，不能直接把数字改写成“比 Opus / GPT 快多少倍”。

其他演示包括**实时玩 Doom、让 50 个小镇居民在约 0.6 秒内完成一轮决策。**这些演示吸引人的地方，是把原本需要等待的模型调用，变成游戏或业务流程中可以频繁发生的一步。70–500 ms 的响应范围属于待按任务和配置验证的性能指标，并非任意负载下的延迟保证；加上创始人发布帖接近 3,000 万次浏览的热度，“快速、低成本决策”就成了它传播的核心卖点。

因此，它的看点不是全面胜过 Opus、GPT，而是在分类、路由等窄任务上提供另一种速度与成本选择。 上述演示缺少完整的同条件准确率测试，不能由“更快”推出“更聪明”或“不会判断错”。

7.3.2.1.3 能做什么，不能做什么?

通用大模型像在写申论题，Jev 则是在回答事先设计好的题目。 它能回答的主要是三种：**是非题、单选题、打分题**，对应“是不是”“哪一个”“到什么程度”。

第一种，是非题：问“是不是”。 给它一个命题，它返回 0 到 1 之间的概率，越接近 1 越倾向“是”，越接近 0 越倾向“否”。例如，问“这张发票是不是诈骗？”，它可能返回 90%。程序再根据这个概率和业务规则决定是否转人工检查。概率是模型的判断，不是事实保证；阈值也需要按业务验证，不能直接照搬示例。

第二种，单选题：问“哪一个”。 先列出候选选项，Jev 从中选出最合适的一项，同时返回各选项的概率。选项及说明由你定义，选项上限为 255 项，实际以接入版本接口为准。例如，自动整理信箱时，先定义文件夹及其收录规则，再让 Jev 判断每封邮件应归入哪个文件夹。

第三种，打分题：问“到什么程度”。 有些事情不适合只分“是”或“否”，需要先定一把尺，写清楚从低到高每一级的含义；评分最多可分 11 级，实际以接入版本接口为准。例如筛选潜在客户：0 分是没有预算的学生，1 分是小团队，2 分是有明确需求和时程的公司，3 分是预算充足的大企业。某条大企业咨询可能得到 2.9 分，程序就可以按预设规则，将高于 2.5 分的线索优先通知业务主管。**这里的 2.9 是程度分数，不是概率。**

对于这类高频、简单、答案范围明确的判断，动用昂贵的通用大模型，往往有些“杀鸡用牛刀”。 Jev 的价值是把这些小决策做得更快、更便宜：上述千封邮件分类演示中，耗时从对照模型的 5 分钟缩短到 70 秒，成本从 62 美分降到 9 美分。在一个技能选择案例中，加入 Jev 预筛选后，错误率从 17% 降到 7.3%。这说明专门化方案在特定任务上也可能更准，但该案例缺少完整评测条件，不能据此说 Jev 在三类任务上的准确率都“秒杀”大模型。

Jev 的出发点是：软件不总需要一个会长篇思考的助手，很多时候只需要一个快速、可靠的判断。 因此，它不以更长的思考链和更强的通用推理为主要目标，而是面向高频决策，优化延迟、成本和概率的可靠性，返回程序能直接使用的结果。可以用 System 1（快速、直觉式判断）对比 System 2（审慎推理）来理解这一定位；这是设计比喻，不是模型架构的划分。

这种设计也带来明确的取舍：**把更多流程设计留给开发者，把运行时的局部判断交给模型。** 开发者定义问题、候选结果和流转规则，Jev 根据当前输入判断，程序据此选择分支。与大模型驱动的 Agent 在运行时规划、调整过程相比，这种方式需要更多前期编排，但在流程相对确定的任务中，有望换来更低的运行成本和更强的可控性。

但快判断不等于强推理，Jev 也无法替代 LLM 的开放式生成能力。 需要多步推理、复杂分析、方案构思，以及写文章、总结内容、生成代码时，仍应交给通用大模型。快棋场景就能说明这点：Jev 即使凭响应快让对手超时，也不代表棋力更强；客服场景也一样，它能判断工单该转给谁，但最终安抚客户的回复仍需大模型来写。两者应当分工：**Jev 负责快速判断，大模型负责推理和生成，程序负责执行。**

7.3.2.2 技术实现：已知边界与 RLCD

7.3.2.2.1 模型架构：尚未公开

TypeSafe AI 尚未公开 Jev 的完整网络结构，无法确认它是否基于 Transformer，也不知道参数规模和具体输出头设计。 已公开的技术描述包括新模型架构和并行采样器，但不足以还原其内部实现。

一种推测是：**Jev 仍用 Transformer 处理输入，但不让模型接着写文字，而是由专门训练的输出层直接计算类别概率或分数。** 例如，输入一封邮件后，直接返回“账务 90%、物流 10%”，而不是逐步生成“这是一封账务相关的邮件”。这种做法不要求把模型改成 Encoder-only；保留 Decoder-only 的主体结构，也可以通过训练专门的分类输出层来实现。

架构动图：用 Decoder-only 作为共同的教学例子，不表示 Jev 已确认采用该结构，也不表示两者共享权重。下方仅展示固定类别分类；动态选项如何编码、并行采样如何工作仍未知。

注意：这个是一个架构的推测哈，因为现在Jev的架构尚未披露，上图和下面文字都是推测！

看图时跟随上方蓝色、下方绿色光点：**两条路径都先经过 Transformer Backbone（主干网络），得到 Hidden State（隐藏状态），再进入不同的任务输出头。** 隐藏状态是计算得到的中间张量，不是独立的“表示层”；图中仅画出一个选定位置的 d 维向量。

上方的 **LM Head**把隐藏状态映射为词表 Logits（未归一化分数），再经 Softmax 转为下一个 token 的概率。图中用 argmax 选择 token 作示例，选出的 token 经 Autoregressive Loop（自回归生成循环）接入下一步，直到遇到结束标记。生成会复用已有 KV Cache，不是每一步重算完整输入。

下方的 Classification Head（分类头）把隐藏状态映射为固定类别的 Logits，再经 Softmax 得到类别概率，返回结果后结束。**两条路径都有分数和概率，区别是预测 token 还是预测类别，以及是否继续运行文本生成循环。**

所以，推测中省去的是**逐 token 生成的循环**，不是把 Decoder 模块删掉。输出头需要训练，不能把现有 LLM 的输出层随意替换就得到 Jev；这也不能证明其实际推理只需一次前向计算。

（以上内容，应用开发的同学看不懂算了。抓住最核心的就行：**推测Jev也是基于Transformer结构做的，但是网络改变了自回归生成形式，不用循环生成，而是一次预测最终的概率。**）

7.3.2.2.2 后训练：RLCD 与 RLHF 有什么不同?

Jev 的核心训练方向是 RLCD（Reinforcement Learning for Calibrated Decisions），即“面向校准决策的强化学习”。 它的重点不只是让模型选对答案，还要让输出概率准确表达不确定性，便于软件据此做决策。

我们熟悉的 **RLHF（Reinforcement Learning from Human Feedback）**，通常先收集人类对不同回答的偏好，再通过奖励模型等方式指导策略优化，使回答更有帮助、更符合人的要求。**RLCD 则把优化重心放在决策结果和概率校准上：不仅问“选得对不对”，还问“给出的概率是否可信”。** 前者侧重人类偏好反馈，后者强调校准目标，两者并非互斥，也不能简单理解为 RLCD 完全不使用人类反馈。

RLCD 最值得关注的创新方向，是把后训练的重点从“人更喜欢哪个回答”，转向“软件能否依据预测概率采取行动”。 例如，程序可以结合误判成本，决定自动处理还是转交人工。不过，其具体奖励函数、训练数据和策略更新算法尚未公开；目前能明确的是训练目标，不能把它写成已知的完整算法，也不能由此推导出“零错误”或“零幻觉”。

7.3.2.3 使用方式：分类路由与工作流节点预测

下面的两种模式是**应用架构模式**，不是 Jev 提供的两个独立产品。

7.3.2.3.1 模式一：先分类，再交给合适的大模型处理

这一模式把 Jev 放在较昂贵的生成或推理步骤之前。

```Plain Text 用户请求 / 邮件 / 内容 ↓ Jev：分类、评估复杂度、识别处理要求 ↓ 策略代码：选处理分支与模型 ↓ 便宜模型 / 强推理模型 / 专用工具 / 人工审核 ↓ 生成回答或执行任务
```

> ```
> ![](images/UWm0bGtISo2PfqxAy0ScNsEanmc.png)
>
> *动图 4：跟随编号 1、2、3 的邮件，观察它们如何在路由处分道，进入不同下游。颜色之外也保留编号和终点标签；实际分支由代码策略控制，Jev 不自动拥有执行权限。*
>
> 典型例子是客服：
>
> - 简单 FAQ → 经过验证的知识库检索与轻量生成流程；
> - 账务争议 → 账务专用流程，需要时由更强模型整理证据；
> - 明确要求人工 → 进入人工队列；
> - 无法可靠判断 → 回退到更强模型、澄清或人工。
>
> 也可以让 Jev 在一个工具目录中选择候选工具，再由下游组件构造参数。**工具选择不是完整 function calling**：选择 `refund_lookup`，并不等于已正确提取 `order_id`、完成鉴权或调用工具。
>
> <a id="doxcntnpPBdElbr3pzqMUPZD8gb"></a>
> ###### 7.3.2.3.2 模式二：提前编排工作流，只在关键节点让模型预测
>
> 这一模式由开发者规定状态与流转规则，Jev 负责原本难以用纯规则完成的语义判断。
>
> ```Plain Text
> 新工单 → 补齐上下文 → Jev 分类 → 代码路由
>                                  ↓
>                       查账 / 收集证据 / 提交人工
>                                  ↓
>                           通用模型起草回复
>                                  ↓
>                       Jev 检查语气、覆盖度等
>                                  ↓
>                   硬规则校验 + 权限检查 + 审批策略
>                                  ↓
>                           发出回复 / 退回处理
> ```
> ```

动图 5：同一份文档沿固定路径前进，第一次检查未通过时退回修改，之后仍需经过独立策略门才能发送。回退路线由程序预先设计，不是 Jev 自己生成工作流；模型检查也不是完整安全保证。

“分类—路由—生成—检查”的设计可以覆盖识别退款诉求、紧急程度、客户情绪，以及检查回复是否回应问题、是否泄露内部规定等节点。

与通用自主 Agent 的主要区别是：

预先编排的工作流 + Jev由开发者定义允许的状态与转换，模型只负责当前节点的具体判断。其可控性来自显式策略、类型、权限和状态机，适合稳定、重复、结果空间可定义的任务。

开放式 Agent则可能由模型动态形成计划和路径，并承担规划、选工具、构造参数、解释结果等职责。它依赖 Agent 框架、工具约束和运行时治理来保持可控，更适合路径难以事先穷举的探索性任务。

我相信这个对我设计项目是一个新的启发，之前我们很多人设计的项目都有，用一个主Agent做思考，判断，分类给不同的Agent去做。现在我们可以思考的一点，用一个Agent(LLM) 去做一个简单的分类任务，是不是从速度，成本，准确性上都不如Jev模型，是属于大炮打蚊子。我们是否可以在我们的架构设计中，使用Jev来做这种分类的决策?

7.3.2.4 面试问题

7.3.2.4.1 最近很火的 Jev 模型你看了吗？谈谈你的理解。

💡 答题思路

从**Jev是什么（模型）**，相比于普通LLM的**核心和特点**（设计理念，速度，花费）对比，对我们的**设计启发**（对于很多分类，判断，评分问题可以用Jev，而不用llm)

📝 参考答案

看过相关资料。我的理解是，Jev 是一个面向软件自动化的决策模型，不是另一个聊天模型。它主要解决分类、是非判断和评分问题，比如工单该交给哪个部门、邮件是否紧急。它的重点不是写一段回答，而是根据预先定义的问题，输出程序可以直接使用的判断结果和概率。

它和通用 LLM 的区别，我觉得主要在优化目标。通用 LLM 强调生成、推理和开放式任务，Jev 则专注高频、结果范围明确的小决策，追求更低的延迟和调用成本。传统 LLM 当然也能分类，但只是判断一封邮件的类别，就调用昂贵的推理模型，可能没有必要。

它给我们带来的分工思路：简单的语义分类、判断和评分，可以考虑交给 Jev；复杂分析和内容生成，仍然交给通用模型。这个思想可能会对我们的工作模式，项目设计会带来很大的改变，今后可能会大量运用在我们的生活中。

7.3.2.4.2 Jev 模型对于你的工作有什么启发?

💡 答题思路

可以从项目设计的角度来回答一下，之前没有Jev的时候，我们的设计：会思考确定性的东西用代码来做，不确定的东西用llm做。现在设计项目我们可以从三点出发：**什么时候用代码做，什么时候用jev做，什么时候用Llm做**，以及再谈谈上面讲到的jev的两种使用方法。

📝 参考答案

它对我最大的启发，是项目设计时可以把任务分得更细。以前我们常说，确定性的逻辑用代码，不确定的语义问题交给 LLM。现在我会进一步区分：需要理解语义，但答案范围明确的任务，不一定要用通用大模型。

具体来说，**规则校验、计算和权限控制，优先用代码**；**工单分类、紧急程度判断、内容评分，可以考虑 Jev**；**复杂推理、方案规划和回复生成，仍然交给 LLM**。不是为了用新模型而增加组件，而是给每个任务选择合适的实现方式。

在用法上，我会考虑两个方向。**第一是前置路由**，让 Jev 先判断请求类型和复杂度，再由代码把请求分给轻量模型、强推理模型或人工，避免所有请求都调用最贵的模型。**第二是在预先编排的工作流里承担局部判断，**比如客服流程中，先判断诉求和紧急程度，再由 LLM 起草回复，最后用 Jev 检查回复是否覆盖问题、语气是否合适。真正发送仍由代码和审批规则控制。

这两种方式可以组合。我的核心思路是把判断、生成和执行分开，再用业务评测验证收益；对于不确定或高风险的结果，保留人工复核。

7.3.2.4 讲解视频

https://www.bilibili.com/video/BV15zah6fEWq/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

8 模型微调基础

（虽然我们的目的是以应用侧入手，不过实际面试下来，微调和推理也会被问到，如果你完全没有经验，面试肯定会吃亏，一般来说SFT:LoRa 以及强化学习，面试官都还挺看重的，希望你能做过，并且可以讲出来）

8.1 SFT的方案有哪些?

参考《从零构建大模型 – 算法，训练与微调》第5章

从大的方向上可以分为**全量微调**和**部分参数微调。**

全量微调是比较传统的方法，对模型所有的参数都进行训练，适应性强，适合复杂任务，但是成本高，计算资源消耗大，容易过拟合。

目前在llm模型领域使用更多的是只需训练部分参数的微调。比如说：

通过在模型结构中**插入新的模块**来实现微调的方法，例如：Adapter tuning, LoRA, QLoRA，他们不改变原始参数，只训练新模块，适用于中大型任务。

通过在**输入前添加可训练的提示**或**前缀来引导模型**的方法，例如Prompt Tuning和 P-Tuning等，他们不改变模型结构，参数极少，适用于轻量任务或快速迭代。

8.1.1 简单介绍Lora技术

Lora原理就像拼图。 整个模型的参数就是一幅完整的拼图，而LoRA微调的目标是只调整几个特定的拼图模块，而不重拼整个模块。

具体来说，LoRA通过低秩分解将大矩阵拆分成两个小矩阵，只微调这两个小矩阵中的参数，不改变整个大的权重矩阵。

这种分解相当于把复杂的工作分配给几个辅助部件来完成，既保持了原模型的稳定性，又能够用较少的参数适应新的任务。

此外，LoRA只需要微调小矩阵中的少量参数，因此减少了计算量和存储需求，适合快速迭代和部署。

8.1.2 Lora中的常见参数

模型参数

r (秩）：

（概念）控制低秩矩阵的大小，影响训练参数量和模型容量。

（作用）r 越大，模型能学习到更多任务特征，但显存占用也会增加。

（常用范围） 4 \~64，典型配置是r = 16 或 8。简单任务用8，复杂任务用16\~32

alpha(缩放因子)

（概念） 用于缩放LoRA权重的影响。

（作用）调节LoRA插入模块对原始模型的影响速度，通常为r 的 1\~2倍。

（常用范围）16 \~ 128。

target_modules

（概念）指定在哪些层插入LoRA。

(作用）影响训练效率和效果，通常选择注意力层。

训练参数

（训练参数放在Lora下似乎不太准确，但是因为问的最多的就是Lora，笔记部分暂时也没涉及别的方法的模型训练，所以我先放在这里）

learning_rate（学习率）

- 控制权重更新速度
- **作用**：太高会导致不收敛，太低训练慢。
- **常用范围**：学习率通常设置在一个较低的范围 （1e-4 \~ 5e-5）。

batch_size（批大小）

- 每次训练处理的样本数。

- **作用**：影响显存占用和训练稳定性。

epochs（训练轮数）

- 完整遍历数据集的次数。
- **作用**：轮数太少模型学不够，太多可能过拟合。
- **常用范围**：一般设置在3\~5轮，防止过拟合。

8.1.3 简单介绍Prompt Tuning技术?

Prompt Tuning是一种通过调整输入提示来引导模型更准确地理解任务的微调方式。在Prompt Tuning中，模型的权重保持不变，微调仅涉及对输入提示进行优化。通过精心设计Prompt，模型可以在任务上下文中生成更精准的输出，可以提升对特定任务的表现。这种方式不仅高效，还可以适应不同任务，使得大模型不需要大规模微调就能适应新的任务需求。

8.1.4 你是怎么做的微调?

（这也是一个**比较常考**的问题，这个答案也是我参考了一些资料和自己项目总结的一个暂时比较满意的答案）

首先按照这个4个思路来讲解：

（1） 确定目标/做好选型

> （2） 数据集的准备
>
> （3） 确定参数，进行微调，部署
>
> （4） 验证和评估

按照这个思路，面试官会觉得你逻辑非常清晰，做法非常规范。 在这个思路下，再添加自己的项目细节。让答案更加丰满，以下以我的项目，按照上述思路进行回答，请大家自己根据实际情况参考。

我的答案是根据这个笔记总结的，如果你需要总结自己的答案，强烈建议阅读这个：https://www.xiaohongshu.com/explore/692c36e1000000001b02120e?app_platform=ios&app_version=9.19.5&share_from_user_hidden=true&xsec_source=app_share&type=normal&xsec_token=CBab5PFDvzdmqER5g7jT9lzuPt7ZOmgeEnLQ3UclYMWIk=&author_share=1&xhsshare=WeixinSession&shareRedId=ODgyNUdJSk42NzUyOTgwNjczOTdINzlO&apptime=1772294660&share_id=77829c9f62674574b9a7e6de1910ac19

）

我认为做微调是一个非常严谨的，数据驱动的科学实验过程。 在微调的过程中，我们通常有4个步骤：1.确定目标/做好选型。 2.数据集准备。 3. 确定参数，平台，进行微调。 4.验证和评估。

1. 首先是确定好目标，做好选型，比如是否可以通过提示词工程 或者 RAG 技术来解决，这样成本更低。同时考虑一些硬件是否支持，比如16GB显存，可以使用LoRA 微调，如果是13B模型，可能就要使用QLoRA。

在我的系统中，核心目标是将模型压缩，同时保留模型的精度，使用的是远端Azure AI Foundry， 训练的是Qwen2.5-7B-Instruct，使用的是LoRA技术。

2. 数据集的准备。项目采用**真实用户邮件标注样本**、**GPT-4生成的合成数据**，以及**Enron Email Dataset公开数据**集，共500条样本。在准备好数据后，做了一些文本标准化，敏感信息去除的数据前处理。

3.确定LoRa的参数， r(秩），alpha 缩放因子 以及一些超参数，比如**学习率**，**Batch Size**, **训练轮数**，我们的任务比较简单r，就设置较小值8， 相应的alpha就是r的两倍16.。 LoRa只训练部分参数，因此可以稍微设高学习率，这里采用的2e-4.

除此之外，在训练中密切监测训练集和验证集的损失曲线。如果验证集损失持续上升而训练集损失持续下降，说明可能出现了一些过拟合，可以采取**早停** 增加数据正则化等。 在训练结束后使用PEFT库将适配器权重和基础模型合并。

4.对于不同的任务，可以使用不同的评估方法。比如分类任务 可以用：**F1-Score方法**， 文本生成任务可以用：**BLEU/ROUGE。** 通用对话任务，可以使用业内公认的基准比如：**MT-Bench** 来获得一个可以比较的分数。在我的项目中，由于该任务属于**结构化信息提取**任务，需要从自然语言邮件中准确提取并转换为JSON格式的事件信息，简单来说是采用的是字段级别F1-Score结合精确匹配的评估方法。简单来说即对每个字段（如title、时间、地点）分别计算准确率，既保证整体结构正确，又允许语义相同但表达不同的情况。

8.2 强化学习的方案有哪些?

（先背一下基本概念，实际中考察的并不多，所以不深入学习）

1. 基于人类偏好的 RL

- **RLHF（Reinforcement Learning with Human Feedback)**
- **奖励来源**：人类排序数据 → 奖励模型。
- **优化方式**：PPO（Proximal Policy Optimization）+ KL Penalty。
- **目标**：对齐人类偏好、安全和价值观。
- **代表模型**：ChatGPT、Claude。

2. 基于 AI 偏好的 RL

- **RLAIF（Reinforcement Learning with AI Feedback)**
- **奖励来源**：强大教师模型（如 GPT-4）生成偏好排序。
- **优化方式**：同 RLHF（PPO）。
- **目标**：降低人工标注成本。
- **代表模型**：Anthropic 的 Constitutional AI。

3. 基于偏好数据的直接优化

- **DPO（Direct Preference Optimization)**
- **奖励来源**：偏好数据（人类或 AI）。

- **优化方式**：直接优化策略，不训练奖励模型。
- **目标**：简化 RLHF，避免 PPO 的复杂性。
- **代表模型**：LLaMA-2-Chat、Mistral-Instruct。

8.3 SFT和RLHF的区别是什么?

（问的并不是特别多，作为强化学习的基础知识先准备着）

目标上的区别：SFT的目标是让大模型能**有基本的语言能力和执行任务的能力**，RLHF倾向于让大模型的回答更符合**人类偏好**，**安全**和**价值观**。

方法上的区别：SFT使用**有监督**的方式训练，数据集的形式是问题-答案。而RLHF采用**强化学习**的方式，**数据集**是包含多个答案，以及**答案的排序**和**奖惩模型**的。

步骤上的区别：RLHF通常是在SFT的基础上进行进一步优化的。

8.4 数据集的准备

（数据集的准备也是面试官很常考的一个问题，我这里这一节4.1介绍了一些常见的数据集的来源，4.2 介绍了对数据进行前处理的方法。请大家根据4.1和4.2 整理出自己的针对于面试官问 "你的项目/LoRa是如何制作数据集的" 这一问题的回答。另外4.3是DeepSeekR1 的训练方法，里面包含了一些数据集制作思路，如果有什么契机你能把这个流程给面试官讲，一定会很加分）

8.4.1 常见的数据集来源

微调领域常见的数据集的来源：

1.内部数据：从一些内部的文档，或者用户使用的数据来去做标注。

2.公开数据：寻找一些已经存在的数据库，比如Hugging Face Datasets，Enron Email Dataset 。

3.人工标注：费时，但是可以得到高精度的定制化数据。

4.数据合成：数据合成，利用大模型去生成。

8.4.2 数据清洗和前处理

1.文本标准化：将文本统一大小写，去除一些特殊字符，html tags之类的。（虽然现在模型可能对大小写理解得很好，但是你统一大小写会降低模型学习难度）

2. 敏感信息去除：去除一些敏感信息，比如用户的名字，电话号码。（主要是合规性考虑）
3. 平衡数据：平衡一些数据集类型的占比，特别是分类任务，避免某一类内容过多。
4. 标签纠正：避免一些数据集矛盾的问题。（比如同一个输入，有的分类成angry,有的分类成frustrated, 那么选择一个label或者合并种类）

5. 统一格式：对于不同的微调任务，需要的输入格式可能会不一样。比如分类任务的格式是{Text->Label}，问答任务的格式是：{Context, Question -> Answer}

8.4.3 DeepSeek R1 训练方法

（参考资料 https://zhuanlan.zhihu.com/p/24621463668，作为一个**拔高知识点**，这个题大概率不是面试官直接问的，但是如果你能够在某个契机能把这个讲出来，就会比较加分。从DeepSeekR1训练方法去理解整个模型 SFT + 强化学习 + 训练集的准备的整个过程。理解整个过程，可以给面试官讲清楚 训练过程 + 谈谈自己对这套方法的理解。一定会加分。 大概这么吹牛逼： 我看了DeepSeekR1的训练，它的大概的训练步骤是xx(笔记部分训练过程）。通过这篇论文和训练方法，我的启发是（笔记部分训练背后蕴含的思想）xx）

训练过程：

1. 通过向DeepSeek-R1-Zero 提问，再通过人工精修，生成几千条思维链监督数据集。
2. 使用生成的几千条思维链监督数据集，训练DeepSeek v3 base模型，得到思维链监督微调后的模型。
3. 使用GRPO强化学习，奖励分数引入准确率和语言一致性分数，得到强化学习训练后的模型。
4. 使用强化学习训练后的模型，采用拒绝采样方法，收集60万条思维链推理数据集，其中拒绝采样的奖励模型使用DeepSeek-v3。
5. 向DeepSeek-v3提问和整理DeepSeek-v3训练数据集，得到20万条写作、事实问答和自我认知等领域的非推理数据，滤掉了混合语言的链式思考、长段落和代码块。并整合60万条思维链推理数据集。得到一个80万监督数据集。
6. 使用80万监督数据集训练DeepSeek v3 base模型，再做多样化强化学习，最终得到DeepSeekR1模型。

背后的蕴含思想：

一：闭环迭代，不是一次性训练：

> 不是 "先准备数据 ->一次性训练模型"，而是：训练模型->用更强的模型生成更好的数据-> 再训练->模型更强->再生成数据。 这种闭环让模型和数据共同进化，避免低质量数据拖后腿。

二：数据质量优先：

通过拒绝采样，人工精修，模型奖励打分，确保训练集干净，逻辑正确。

三：分阶段训练：

阶段1：SFT -> 学会遵循指令

> 阶段2：GRPO 强化学习->优化推理能力
>
> 阶段3：多场景RL-> 提高泛化和安全性

四：多类型数据集，能力互补

指令遵循数据集 让模型可以听懂指令。

> **推理链数据集**：让模型逐步推理
>
> **非推理任务数据集**：保持通用能力。

奖励信号数据集：强化输出质量

8.4.4 面试问题精选

4.4.1 判断数据是否值得训练?

判断一批数据值不值得训练，我会看几个标准：

是否真实覆盖业务问题

数据要来自真实高频、高价值、容易出错的场景。比如用户天天问、模型经常答错、RAG 很难靠检索解决的问题，这种才值得训。

如果只是一些泛泛的知识问答，模型本来就会，训练价值就不大。

是否有明确可学习的模式

微调适合学习的是稳定的输出风格、任务格式、领域表达、工具调用方式、拒答边界、分类标准、结构化抽取规则。

如果问题本质是“知识不够新”或者“答案依赖外部文档”，那更适合 RAG，不适合靠微调硬记。

标签质量够不够高

训练数据里答案要正确、一致、无歧义。

如果同一个问题在不同样本里答案互相冲突，或者标注人标准不统一，模型训练后会更不稳定。微调不是洗数据的魔法，脏数据只会放大问题。

是否能覆盖失败案例

很有价值的数据往往来自 badcase：模型曾经答错、拒答错、格式错、工具调错、幻觉的样本。

这些数据经过修正后，可以让模型学会“原来这种场景应该这样处理”。

是否和现有能力有增量

如果基础模型已经能很好完成，或者靠 prompt 就能稳定解决，那没必要微调。

只有当 prompt 太长、规则太复杂、输出不稳定、某类任务反复失败时，微调才更有收益。

是否能被评测验证

训练前最好留出一批验证集，看微调后在准确率、格式遵循率、拒答准确率、工具调用成功率、用户满意度上有没有提升。

如果没有评测集，就很容易变成“感觉模型更好了”，但其实可能只是对训练集过拟合。

精选自 202604 腾讯AIAgent暑期实习一面

9 模型推理加速基础

9.1 常见的推理加速方法

我们可以从**模型层面**，**硬件层面** 和**系统架构**层面 三个方向入手：

A 模型层面

1.模型量化： 将FP32 转换成INT8 或更低的精度，减少计算量和内存占用，一般这样推理速度可以提升2\~4倍。

2.模型剪枝：删除冗余权重或者不重要的神经元，减少参数。

3.知识蒸馏：用大模型训练一个小模型，部署时用小模型推理。

B 硬件层面

1. GPU/TPU 加速： 使用高性能GPU 或专用AI 芯片。

2. 多卡并行：将模型分布到多个GPU上进行计算。

C 系统架构层面

KV 缓存：对Transformer的注意力计算进行缓存，在长文本推理时加速效果显著。

FlashAttention: 一种针对Transformer的注意力计算的高效实现方法，可以显著减少内存占用并提升计算速度，特别适用长序列推理场景。

9.2 蒸馏

模型蒸馏是一种**模型压缩和知识迁移技术**，主要用于将一个复杂、性能强大的模型（通常称为“教师模型”）的知识传递给一个**更小、更高效**的模型（称为“学生模型”）。它的核心目标是**在保持模型精度的同时降低计算成本和存储需求。**

具体做法是把教师模型的**输出作为额外的监督** 和**学生模型的输出**做对比，通过优化损失函数来训练学生模型。

为什么要做模型蒸馏?

部署需求：大型模型在移动端、嵌入式设备或低算力环境中难以运行。

推理速度：小模型推理更快，适合实时应用。

节省资源：减少内存占用和能耗。

具体做法

在训练过程中，教师模型会输出对每个数据的预测分布，这些分布包含了每个类别的概率，就像老师教学生做选择题的时候，不仅会给出正确答案，也会标注出其它选项为什么不正确。

随后通过知识蒸馏，学生模型使用教师模型的软标签来进行学习。这个过程包含两个目标：一是让学生模型学会在分类任务上给出准确的答案，二是模仿教师模型的预测分布，尽量让自己的输出和教师模型相似，这样可以让学生模型在不增加参数的情况下更快，更好地学习到知识，在实际应用中也能更高效运行。

9.3 量化

9.3.1 常见量化方法

训练后量化（PTQ, Post-Training Quantization）

- **特点**：在模型训练完成后，将浮点权重和激活转换为低精度（如 INT8）。
- **优点**：实现简单，计算成本低。
- **缺点**：没有在训练过程中考虑量化误差，精度可能下降明显，尤其是对复杂模型。

量化感知训练（QAT, Quantization-Aware Training）

- **特点**：在训练过程中模拟量化操作（如插入伪量化节点），让模型在训练时“感知”量化带来的误差。
- **优点**：精度接近浮点模型，因为训练过程考虑了量化。
- **缺点**：训练成本高，需要从头训练或至少进行完整的训练周期。

量化感知微调（QAF, Quantization-Aware Fine-tuning）

- **特点**：不是从零开始训练，而是基于一个 **预训练模型**，在微调阶段加入量化感知操作。
- **优点**：比 QAT 成本低，因为只需要微调而不是全量训练。
- **缺点**：精度可能略低于完整 QAT，但通常比 PTQ 好很多。

9.3.2 量化的好处和坏处

（小红书面试官问过，算是基本概念题，简单理解背诵下）

好处

降低计算和存储成本

- 将 FP32 转换为 INT8 或更低精度，可以显著减少**内存占用**和带宽需求，提升推理速度，降低能耗，尤其适用于边缘设备和移动端。

支持大模型在有限硬件上运行

- 例如 QLoRA 技术通过 4-bit 量化，使 650 亿参数的大模型可以在单张 48GB GPU 上微调，保持接近原始性能。

降低部署成本

- 在生产环境中，量化可以减少硬件资源需求，降低整体运营成本，提升模型的可扩展性

坏处：

精度下降

- 低精度表示范围有限，可能导致**权重和激活值的细微差异被放大。**这会导致：模型在分类、预测等任务上可能出现轻微性能损失，尤其是对数值敏感的任务，性能下降比较明显。

累积误差

- 在深层网络中，量化误差会逐层累积，影响最终输出，尤其在**注意力机制或归一化等算子中表现明显。**这会导致模型在复杂推理任务中表现明显下降。

9.4 剪枝

去除神经网络里不重要的连接或者神经元。可以减少模型参数，提升推理速度，对于移动和嵌入式设备友好，因为去掉了冗余信息，可以降低过拟合的风险，提高泛化能力。缺点是精度损失问题。不同的模型和硬件对于剪枝的适应程度也不同。

9.4.1 常见的剪枝方法

非结构化剪枝：随机移除单个权重或者连接，压缩比大，但是对于推理速度提升不大。

结构化剪枝：按照一定规则，比如移除整个神经元，滤波器或者层，更加适合硬件加速。

9.5 二值化

极端的量化技术，它把神经网络的权重和激活值限制在了两个值上，通常是+1, -1。

9.6 模型压缩技术的选择

在极端资源受限的场景下，二值化和量化是优先考虑的技术。

二值化适合精度不高的任务，量化可以调整压缩精度，在压缩效果和模型性能之间找到平衡。

如果更加注重计算效率，希望显著提升模型的推理速度，同时对模型精度还有一定要求，那么量化和结构化剪枝是不错的选择。

如果目标是在保持模型性能的前提下进行压缩，那么知识蒸馏就是理想之选。

9.7 常见的推理框架

（在说到熟悉推理这块，面试官很可能会问你熟悉哪些推理引擎，最好都有一个了解，结合你用的引擎深入了解）

9.7.1 为什么Llama.cpp 的在cpu上效果更好?

（因为我的项目使用了Llama.cpp 替代了 onnx runtime, 在cpu上对于同一模型推理速度提升了1倍以上，所以面试官可能会问我这个问题，这个问题更多是根据我个人情况准备的，供大家参考。大家也可以结合自己的情况，自己用了什么推理加速来准备一些相似问题。）

1. **内核优化**：llama.cpp 针对 LLM 做了高度融合的算子（注意力、RoPE、KV cache），减少框架调度和内存拷贝；ONNX Runtime是通用框架，算子分发和调度开销更大。
2. **线程与缓存策略**：llama.cpp会主动优化线程亲和、NUMA绑定、缓存布局，KV cache访问更高效；ORT默认设置如果没调优，容易出现带宽瓶颈。
3. **指令集利用**：llama.cpp直接用AVX2/AVX-512等SIMD指令，针对矩阵乘法和量化内核做了手工优化；ORT依赖通用库，优化程度不如专用实现。

一句话总结：即使模型和量化相同，llama.cpp在CPU上更快，是因为它是专门为LLM推理做极致优化的，而ONNX Runtime是通用框架，调度和内存管理开销更大

7.2 介绍一下vLLM 框架

（vLLM 框架在很多岗位的jd 上都写着有要求，今天平安证券也问到了这个，不过亲测这个问题问的不多，倾向于咱们先背个基本概念就好。我从三个方面总结了， 设计**思路，特点，总结，**如果问到可以选择性地和面试官介绍）

1. 设计思路

vLLM 的核心目标是解决 **大语言模型推理的高并发和显存瓶颈问题。**传统推理框架在处理长上下文或多请求时，KV 缓存占用显存过大且容易碎片化，导致性能下降。
vLLM 的设计理念是通过 **PagedAttention** 技术，将 KV 缓存分页管理，类似操作系统的虚拟内存分页机制，**显存复用率高**，适合长上下文和多请求场景，从而实现**高效的显存利用和动态批处理。**

2. 特点

PagedAttention技术： 将kv缓存按页管理，避免显存碎片化，支持灵活调度。 **显存复用率高，支持长上下文和多请求场景。**

动态批处理：异步调度器可以将不同请求合并，提高GPU利用率，降低延迟。

高兼容性：支持HuggingFace Transformers模型，几乎无需修改代码即可接入。

> 分布式推理：支持多GPU，多节点部署，适合大规模服务。

3. 总结

vLLM的核心创新是PagedAttention + 动态批处理。解决了传统推理框架在显存管理和并发处理上的痛点，显著提升吞吐量和资源利用率。相比于其它框架，vLLM 更适合 **在线高并发推理，**尤其在长上下文和显存受限的情况下表现突出。

9.8 你的项目是如何做推理加速的?

（这个问题是我给面试官准备好的，我的项目中是如何使用 **LoRa + 强化学习DPO + 量化 + 推理，来完成将Qwen2.5-7B-Instruct 模型 适配在CPU 电脑上，极大降低推理延迟，提升推理速度，并且提升任务质量的。**

> 这个问题几乎是所有我所学的关于模型推理加速/微调这两部分的内容 在 我的项目中运用的实际体现。 这个问题答案供大家参考，如果你想要向面试官展示你有 模型微调/推理加速的经验，你也要准备一个你自己对这个问题的回答，讲清楚 在你的项目中，背景是什么，用了什么技术，怎么使用，为什么要使用，最终达到了什么效果。）

我负责优化邮件日历事件提取的模型性能。**背景是**用户需要从邮件快速创建日程，但云端API延迟2-3秒且按调用计费，不适合高频场景。**我的方案是**本地部署微调后的开源模型Qwen2.5-7B-Instruct，通过三步优化实现性能和成本的平衡。

第一步是LoRA微调，使用**PEFT库**实现低秩适应，只训练0.18%参数（1400万 vs 76亿），将任务准确率从基础模型的65%提升到92%。选**择LoRA而非全量微调是基于成本和效率考虑，能够显著降低显存需求和训练时间**，同时保持接近全量微调的效果。训练数据来源包括：**真实用户邮件标注样本、GPT-4生成的合成数据，以及Enron Email Dataset公开数据集**，共500条样本，在Azure AI Foundry上训练完成。

第二步是DPO对齐，使用**TRL库**的Direct Preference Optimization进一步优化输出质量。构建了200对'chosen-rejected'偏好数据：chosen样本来自人工标注的高质量输出，**rejected样本通过规则扰动（添加冗余字段、格式错误、信息缺失）生成，并混合了少量GPT-4生成的负样本。**在LoRA基础上训练1小时，将JSON格式规范性从85%提升到98%，显著减少了解析错误和冗余输出。

第三步是量化部署，这里做了技术选型：对比了ONNX Runtime、PyTorch Mobile和llama.cpp三个方案。最终选择llama.cpp是因为它CPU推理做了深度优化，包括**SIMD并行计算**和**内存管理优化**，比ONNX快2.5倍。量化使用 `torch.ao.quantization` 库，方案采用Q4_K_M混合精度Post-Training Quantization：Attention层保持5-bit精度，FFN层压缩到4-bit，输出层用6-bit确保质量。最终模型从14GB压缩到4.3GB，推理速度从154秒降到43.7秒，提升3.5倍，同时准确率只损失2.5%（92%→89.5%），在可接受范围内。

这个项目的价值在于：通过参数高效微调（LoRA）和训练后量化（PTQ）的组合，在不牺牲太多精度的前提下，将云端依赖转为本地部署，成本从按次计费降为零边际成本，同时满足了实时响应需求。技术选型上，llama.cpp在CPU推理上的性能优势是关键决策点，使得无GPU环境也能高效运行7B模型。

9.9 参考资料

模型压缩四大方法概述 | 量化、剪枝、蒸馏和二值化

推荐理由：特别适合快速了解这四个技术，是干什么的，怎么做，有哪些常见方法，不深入讲解公式和原理，小白也能听懂。

https://www.youtube.com/watch?v=jW2cmZ-9hLk

面试真题与求职技巧【准备开始面试前参考】

面试直达：公司面经 | Agent 项目真题 | RAG 项目真题

1 项目使用和简历编写

1.1 前言

我们目前文档里有3个项目了，这三个项目也确实是我的全部家底了。我靠着这些项目，以及对应所有的知识，一步步走到现在。我也确实看到有很多同学用笔记里的项目去面试了。我希望有一个专题笔记，视频，聊一聊怎么**使用，学习这些项目。**同样的项目，每个人使用起来效果可能不一样，我这期内容想讲的是**思路，方法，我的经验。请大家在这之上，自己灵活运用。**

自己的简历只有自己对自己负责。我提供了思路，参考，还有简历，但是最终还是请自己去把关，修改，根据不同岗位适配，修改。**你永远是对自己简历把关的守门人，没有任何人能够代替你做这件事**，自己的简历还请多自己多上心。

同时，我这个小节所有介绍的经验，我不给天马行空的包装。因为吹牛谁都会，我只基于我们做过的事情，已有的文档去聊。在某些地方让你去包装的时候，我也给到你可方案，文档，做法，而不是天马行空地去谈。所以里面会穿插很多**文档内的链接**，告诉你如果你要这么做，你应该怎么做，学点什么。

1.2 SmartAppointment 项目

这个项目是我稍微有点头疼的，这个项目是我自学Agent做的第一个项目，无论是从项目立意上，还是技术上，我觉得都还很**青涩**。但我必须坦诚的说，这个项目就是当时我丢在简历上去面试的一个项目。我不能说是靠这个项目拿的Offer, 但我当时简历上有两个项目，另一个也是Agent项目（和这个其实差不多），我就是用这两个项目去面试，还拿到了那么多offer。

现在如果面试，我肯定会写一个更高级的项目，或者即使用这个项目，我也会进行一些包装和改进。后来大家让我开源出这个项目，既然开源了，放在了简历上，我在想如何包装，改进，使用这个项目，能够让它变得更加成熟和有竞争力。

这个项目的好处是：**最少搭了一套多Agent对话的框架，多Agent流转，流式响应，RAG系统，MCP，也设计了一个持久化机器人，做沉淀，思想还有的。**

> **缺点是太不成熟**：没有做持久化记忆，RAG系统其实只是走了embedding匹配，持久化没做完，没有压缩策略。

如果使用这个项目，我的建议是在这个基础上，进行改进，包装。我给出一些方向和意见。

1.2.1 记忆模块

学习： 参考学习笔记中的开源项目 Claude Code、Harmes、OpenClaw、pi-agent 的记忆模块。包括各种面经中，其实都有对于这个记忆模块的考察，比如虾皮Agent面经，字节暑期实习面经。看这些笔记部分，把知识弄透，看面经知道面试官怎么问题，思考这个项目怎么设计。

思考： 短期记忆保存当前会话、用户意图和预约状态；长期记忆持久化用户历史咨询、预约记录和服务偏好；基于语义相关性召回记忆，并通过保留近期原文、滚动摘要早期对话完成上下文压缩。

写入简历示范：

> • 设计分层记忆模块：以 `user_id + session_id` 隔离会话，短期层保留最近 10 轮消息及预约槽位状态，长期层将咨询摘要、预约事件和带置信度的用户偏好持久化至 SQLite；检索时先按用户与记忆类型过滤，再按“向量相似度 0.6 + 时间衰减 0.3 + 重要度 0.1”混合排序召回 Top-5，上下文达到模型窗口 60% 时将早期消息压缩为滚动摘要，实现跨会话恢复并控制 Token 消耗。

1.2.2 RAG 系统

学习： 学习笔记中的 RAG 项目，弄清文档解析、文本切分、向量化、召回、重排序和答案生成等经典流程。这个项目基本就是RAG的基础知识综合起来的，所以RAG笔记章节对应的内容啊，书籍啊，概念，都要学会。

思考： 将咨询 Agent 的内置 FAISS 检索升级为独立 Modular RAG MCP Server：对售后手册、FAQ 和历史咨询建立 BM25 倒排索引与 Dense 向量索引，并行召回后使用 RRF 融合，再由 `cross-encoder/ms-marco-MiniLM-L-6-v2` 对候选 Query-Chunk 对精排。为 LLM、Embedding、Splitter、VectorStore、Reranker、Evaluator 六类组件定义 Base 接口，通过 Factory + YAML 配置切换实现；使用 TraceContext 记录 query_processing、dense、sparse、fusion、rerank 五阶段耗时、候选数量和排名变化，精排超时或失败时回退至 RRF 结果。

写入简历示范：

> • 将咨询 Agent 的知识层升级为模块化 RAG MCP Server：采用 BM25 + Dense Embedding 双路召回与 RRF 融合，并使用 `cross-encoder/ms-marco-MiniLM-L-6-v2` 精排；基于 Base 接口、Factory 和 YAML 实现 6 类组件可插拔切换，通过五阶段 Query Trace 记录耗时、候选数及排名变化，Rerank 失败时自动回退融合结果。

1.2.3 记忆沉淀机制

学习： 参考社区对 Claude Code 未公开 AutoDream 能力的逆向与复刻方案，理解其基于时间/会话阈值触发，批量回放历史并执行提取、合并、冲突处理、过期清理和长期记忆晋升的机制。同样也是看上面的开源项目和面试题，讲的都有（Claude Code、Harmes、OpenClaw、pi-agent 的记忆模块）。

思考： 为用户行为分析 Agent 增加后台 Dream Job：用户累计 5 次会话且距上次整理超过 24 小时后触发，按检查点读取新增的咨询、预约、取消和反馈事件；先由 LLM 输出结构化候选记忆，再以 user_id、偏

好类型和向量相似度去重，结合出现次数与新近性更新置信度，冲突时保留来源证据并降低旧记忆权重。任务通过分布式锁避免重复执行，记录 dream_run、处理水位和变更日志，失败后可从检查点重试，最终将高置信偏好写回 UserPreference，供推荐和回访读取。

写入简历示范：

> • 为用户行为分析 Agent 设计 AutoDream 记忆巩固任务：累计 5 次会话且间隔 24 小时后，按检查点增量回放咨询、预约及反馈事件，经结构化抽取、向量去重、冲突降权和置信度更新后写回 UserPreference；通过任务锁、处理水位及变更日志保证幂等重试，并将高置信画像用于推荐与主动回访。

1.2.4 评估模块

学习：

学习 Agent 测评笔记，并参考笔记中基于该项目实现的评估算法与评测方案。这个笔记我写很全，是整个Agent开发链路上如何做测评，还有很多参考资料，先去看，看完了你应该知道如何做Agent评估。对应实战其实也有，算法项目的二阶段就是做的评估，有数据集，代码，可以参考。通过这些学习，我预期你自己应该知道如何做评估了

思考：

采用 **EDD（Evaluation-Driven Development）驱动整个项目生命周期。**选型期先定义业务成功标准，在私有评估集上对比模型的任务正确性、协议遵循、延迟和吞吐；开发期分别评估意图路由、RAG Hit Rate/Faithfulness、预约槽位 Exact Match、工具名称与参数正确率，再通过标准 Agent 交接轨迹和端到端任务成功率验证整机。发布前将质量、轨迹、P95 延迟、Token/成本、最大步数和安全红队结果设为并列门禁；上线后以 Logs/Traces/Metrics 记录每次模型、Agent 和工具调用的输入输出、耗时、Token、成本及异常路径，持续监控质量漂移、循环调用、越权和提示注入，并将线上失败样本回灌评估集形成回归闭环。

写入简历示范：

> • 采用 EDD 驱动 Agent 全生命周期：基于私有评估集完成模型选型，对路由、RAG、槽位、工具调用及 Agent 交接轨迹开展组件/系统分层评测；将质量、P95 延迟、Token 成本、最大步数与安全测试设为发布门禁，上线后通过 Logs/Traces/Metrics 监控质量漂移、异常工具路径和调用成本，并将失败样本回灌评估集持续回归。

进一步扩展：将评估模块做成独立项目

这部分可以继续做深，单独包装成一个 **Agent Evaluation Platform。**重点不是再写几个单元测试，而是建立一套可重复运行、能够解释失败原因、可以接入 CI/CD 并延伸到线上监控的 Agent 评估基础设施。它既可以评估当前智能预约系统，也可以通过统一 Case、Trace 和 Scorer 协议复用到其他 Agent 项目。

评估数据集： 在现有 30 个场景测试和 51 条本地模型评估样本基础上，扩展一套 200～300 条业务 Golden Set，按意图路由、预约槽位、RAG 问答、工具调用、多轮状态、多 Agent 交接、安全攻击和异常恢复分层

打标。每条 Case 除输入和标准答案外，还保存预期 Agent、允许的工具序列、关键参数、最终状态、禁止行为、最大步数和成本预算。外部数据库、排班和知识库使用固定快照或 Mock，保证同一版本可以重复回放。

三层评分体系：

1. **单步/组件评估：** 路由使用分类准确率与 Macro-F1；槽位提取使用字段 Exact Match/F1；RAG 使用 Hit Rate@K、MRR、Faithfulness；工具调用检查工具名称、参数 Schema 和执行结果。
2. **轨迹评估：** 将实际 Trace 与标准协作链比较，支持严格顺序、无序、子集和超集匹配，并对“选对 Agent、传对参数、顺序合理、及时终止”分别给分，避免最终答案偶然正确掩盖错误过程。
3. **端到端评估：** 统计任务成功率、Pass^k、平均步数、人工介入率、超预算终止率、P95 延迟和单任务 Token/成本。同一配置重复运行 5 次，报告均值、标准差和成功率区间，而不是只展示单次最好成绩。

能用确定性规则判分的任务不使用 LLM Judge；开放式回答先用 Embedding/事实一致性评分，最后才使用 LLM Judge。对 Judge 进行人工样本校准，并通过交换答案顺序、固定 Rubric 和结构化 JSON 输出降低位置偏差、冗长偏差和自我偏好。

全链路追踪与失败归因： 为每次任务生成 `trace_id`，使用 Langfuse 或 OpenTelemetry GenAI Span 记录模型、Agent 和 Tool 的输入输出、耗时、Token、成本、重试次数和状态变化。根据 Trace 自动将失败归类为路由错误、槽位丢失、错误工具/参数、RAG 无命中、无效循环、上下文污染、过早结束、权限越界和模型异常，并支持从失败节点回放，定位到底是模型、Prompt、工具还是编排逻辑导致问题。

CI/CD 与发布门禁： 每个 Pull Request 自动运行约 50 ~ 60 条高风险 Smoke Set，夜间任务运行完整 Golden Set 并对随机性用例重复 5 次；生成新旧版本差异报告并上传评估产物。发布门禁同时检查任务成功率、轨迹得分、工具参数正确率、P95、Token 成本和安全红队结果，任何一项跌破阈值都阻断合并或发布。例如可设置：端到端成功率不低于 90%、轨迹得分不低于 0.85、关键写操作授权率 100%、P95 不高于 3 秒、单任务 Token 不超过预算。

线上监控闭环： 上线后复用同一套 Trace Schema 和指标，在 Dashboard 中监控质量漂移、输入分布变化、工具失败率、循环调用、P95/P99、Token 成本、提示注入和越权行为；新出现的失败请求经脱敏和人工确认后自动加入 Regression Set，形成“线上发现 → 失败归因 → 回灌数据集 → CI 回归 → 灰度发布”的闭环。

独立项目简历示范：

Agent 全生命周期评估与发布平台 | 个人项目 | Agent Evaluation / LLMOps

背景：

Agent 输出具有随机性，单看最终回复无法发现错误工具路径、状态丢失和成本失控；传统单元测试也无法覆盖多轮、多 Agent 与开放式生成任务，因此为智能预约系统建设独立评估基础设施。

目标：

采用 EDD 将评估贯穿模型选型、组件开发、系统回归、发布门禁和线上监控，以统一 Case、Trace、Scorer 协议实现可复现评测、自动失败归因和 CI/CD 质量阻断。

过程：

• 构建 **[实测数量]** 条分层 Golden Set，覆盖路由、槽位、RAG、Tool Calling、多轮交接和安全场景，并以固定数据快照与 Mock 工具保证任务可重复回放。

• 实现最终结果/轨迹/单步三层 Scorer，组合 Exact Match、Hit Rate@K、Faithfulness、工具参数校验和轨迹部分给分；每个随机性 Case 重复 5 次，报告成功率区间而非单次最优。

• 基于 Langfuse/OpenTelemetry 建立模型—Agent—Tool 全链路 Trace，采集耗时、Token、成本与状态变化，并自动归因路由错误、参数错误、RAG 无命中、循环和过早结束等失败类型。

• 接入 CI/CD：PR 自动执行 **[Smoke Case 数]** 条高风险用例，Nightly 运行完整评估集；以任务成功率、轨迹得分、P95、Token 成本和安全红队结果作为发布门禁，退化自动阻断合并。

• 建立线上监控与回流机制，将真实失败请求脱敏后沉淀为 Regression Case，支持新旧模型/Prompt/编排版本对比和灰度回滚。

结果：

累计运行 **[实测次数]** 条 Agent 轨迹，将端到端任务成功率由 **[实测值]** 提升至 **[实测值]**，工具参数正确率由 **[实测值]** 提升至 **[实测值]**，P95 控制在 **[实测值]**、单任务成本降低 **[实测比例]**；CI 共阻断 **[实测次数]** 次质量退化版本。

技术栈： Python、Pytest、AgentEvals、Ragas、Langfuse/OpenTelemetry、GitHub Actions、Prometheus、Grafana、LLM-as-a-Judge

> 一组可参考的实验规模是：240 条 Golden Set、60 条 PR Smoke Set、完整用例重复 5 次共 1200 条轨迹。规模可以直接采用，但成功率提升、P95、成本下降和 CI 阻断次数必须来自实际运行，不能把目标阈值写成最终成果。

1.2.5 多 Agent 架构

学习：

学习 Claude Code 等经典 Agent 项目的中心化编排方式与 ReAct 推理范式，理解主管 Agent 如何进行任务判断、能力调用和结果整合。要学习笔记Agent架构，里面很多参考资料。然后就是各种开源Agent项目（Claude Code、Harmes、OpenClaw、pi-agent），面经，我都讲了架构，理解Agent架构是什么，怎么做的，你就会包装了。

思考：

将现有任务分类 Agent 升级为中心化主管 Agent，把咨询 Agent、预约 Agent 和用户行为分析 Agent 封装为专业能力；主管 Agent 基于 ReAct 范式判断用户意图、选择并调用对应 Agent，在复杂任务中组合多项能力，并通过共享状态完成上下文传递和任务流转。

写入简历示范：

> • 设计中心化多 Agent 编排架构：由主管 Agent 基于 **ReAct** 范式完成任务规划与能力选择，将知识咨询、预约处理和用户行为分析封装为专业 Agent，并通过统一调用接口与共享状态实现跨 Agent 上下文传递及复杂

> 任务协同。

1.2.6 业务场景

学习： 学习企业智能客服与售后预约系统的典型业务流程，理解知识咨询、故障受理、服务人员匹配、上门时间预约和用户回访等核心环节。

思考： 将原按摩预约场景升级为企业售后服务场景：咨询 Agent 负责回答产品说明、故障排查和服务政策等常见问题；需要人工处理时，由预约 Agent 根据服务类型、用户需求、工程师技能和空闲时间完成匹配与上门预约；用户行为分析 Agent 则负责沉淀服务偏好并进行主动回访。

写入简历示范：

> • 面向**企业售后场景**设计智能客服与服务预约系统：通过**咨询 Agent 处理产品问答**、故障排查及服务政策咨询，并由预约 Agent 结合服务类型、工程师技能与空闲时间完成智能匹配和上门预约，配合用户行为分析 Agent 实现个性化服务与主动回访。

1.2.7 权限机制

学习： 学习 Claude Code 的四层权限机制，理解 Agent 在能力边界、资源访问、操作确认和安全审计方面的分层控制方式。

思考： 根据企业售后任务的风险等级设计分层权限：知识查询等只读操作可直接执行；创建预约前需由用户确认关键信息；修改、取消预约及发送回访消息等有副作用操作必须二次确认；用户隐私和跨用户数据实行身份校验、最小权限与数据隔离。同时通过工具白名单、参数校验、幂等控制和操作日志，防止越权调用、重复执行及敏感信息泄露。

写入简历示范：

> • 设计健全的安全机制，按操作风险划分只读直通、关键参数确认和高风险操作二次授权，对用户数据实施身份校验与最小权限隔离；结合工具白名单、参数校验、幂等控制及审计日志，降低 Agent 越权调用、重复执行和隐私泄露风险。

1.2.8 本地模型

学习： 学习 Post-training-slot-extractor 算法项目，掌握本地小模型的数据构造、后训练、量化压缩、推理评估和服务化部署流程。

思考： 将预约 Agent 中高频、低复杂度且输出格式固定的槽位提取任务拆分出来，由本地 Qwen3 小模型负责从多轮对话中提取预约时间、服务时长、用户偏好和指定工程师，并判断是否需要调用查询工具；通过 SFT 与 DPO 提升结构化输出和工具调用的稳定性，将模型转换为 GGUF 并进行量化，使用 llama.cpp 部署为 OpenAI 兼容的本地 CPU 推理服务。复杂咨询与开放式推理仍交给远端大模型，形成大小模型协同架构。

写入简历示范：

> • 构建预约槽位提取本地模型评测链路，在 51 条冻结评估集上对比 Qwen3 0.6B/1.7B/4B；选取 1.7B 作为质量与性能平衡方案，相比 4B 任务正确性仅下降 2.9 个百分点，平均时延降低 34.4%、吞吐提升 32.9%，并设计 SFT/DPO 与 GGUF 量化路线进一步优化协议遵循和 CPU 推理性能。

其实还可以写高可用相关的内容，就是当Agent失败如何重试，优雅回退。我先占个位这部分内容我补了笔记再写。 同时你还可以说，比如写多模态啊等等，但是我的思想是不能天马行空。多模态当然是包装的思路，但是我们笔记暂时没有，我只给你可以行的，你可以照着去学习，思考的方向。

1.2.9 整体项目简历范文

企业售后智能客服与预约 Agent 系统 | 个人项目 | Agent / LLM 应用开发

背景：

针对企业售后场景中产品咨询、故障排查、工程师匹配、上门预约及用户回访依赖人工处理，容易出现信息重复收集、业务知识更新不及时和服务推荐不稳定等问题，设计智能客服与预约 Agent 系统。

目标：

构建以主管 Agent 为统一入口的多 Agent 服务链路，串联知识咨询、预约处理、用户记忆和行为分析；采用 EDD 从模型选型、组件开发、发布门禁到线上监控持续驱动迭代，并通过本地小模型承接高频结构化任务。

过程：

• 设计中心化多 Agent 编排架构，由主管 Agent 基于 ReAct 范式进行任务规划与能力选择，将咨询、预约和用户行为分析 Agent 封装为专业工具，通过共享状态实现跨 Agent 上下文传递与任务协同。

• 构建预约状态管理与工程师匹配流程，从多轮对话提取时间、时长及服务偏好，并结合技能与排班推荐可用人员。

• 将咨询 Agent 接入模块化 RAG MCP 知识层，采用 BM25 + Dense 双路召回、RRF 融合及 `ms-marco-MiniLM-L-6-v2` 精排，兼顾专有名词与语义匹配。

• 以 Base 接口、Factory 和 YAML 实现 LLM/Embedding/Splitter/VectorStore/Reranker/Evaluator 六类组件可插拔；Query Trace 覆盖检索五阶段，记录耗时、候选数和排名变化，精排失败自动回退 RRF 结果。

• 设计分层记忆：以 user/session 隔离最近 10 轮消息与预约槽位，长期持久化咨询摘要、预约事件及偏好置信度；采用“向量相似度 0.6 + 时间衰减 0.3 + 重要度 0.1”召回 Top-5，窗口占用达 60% 时触发滚动摘要。

• 为用户行为分析 Agent 实现 AutoDream 后台任务：累计 5 次会话且间隔 24 小时后增量回放行为事件，经结构化抽取、向量去重、冲突降权及置信度更新写回长期偏好，并以任务锁、检查点和变更日志保证幂等重试。

• 采用 EDD 驱动模型选型与 Agent 开发：以私有评估集量化模型正确性/延迟，对路由、RAG、槽位、工具调用及 Agent 交接轨迹开展组件级与端到端回归。

• 建立质量、P95 延迟、Token 成本、最大步数和安全红队发布门禁；通过 Logs/Traces/Metrics 追踪模型、Agent、工具调用及异常路径，将线上失败样本回灌评估集形成闭环。

• 构建预约槽位提取本地模型评测链路，在 51 条冻结样本上对比 Qwen3 0.6B/1.7B/4B；1.7B 相比 4B 正确性仅低 2.9 个百分点，平均时延降低 34.4%、吞吐提升 32.9%，据此确定质量—性能平衡方案。

结果：

完成 Web/API/Agents/Services/DB 五层工程架构，落地 3 类核心业务 Agent、16 个 API 接口及 30 个场景测试；为本地模型建立 51 条冻结评估集，完成 Qwen3 0.6B/1.7B/4B 与远端模型的质量、延迟和吞吐对比。分层记忆、AutoDream、RAG 重排序、权限机制及模型微调属于增强方案，需在实现和测试完成后再作为已落地成果写入正式简历。

技术栈：

Multi-Agent 、 ReAct 、 LangChain 、 RAG 、 FAISS 、 Embedding 、 FastAPI 、 AsyncGenerator 、 SQLAlchemy、SQLite、SFT、DPO、Qwen3、GGUF、llama.cpp

---

1.3 后训练项目

微调项目，**我觉得特别值得一做。**在我们解析的2026年面试官最关注的6个AI方向里面，有两个方向，**一个是本地模型的对比，效果评估。**另一个是**模型微调（SFT+强化学习）。**

除此之外，这个项目还带一点本地推理部署的内容。

对于开发的同学来说，能够具备这样的经历，相比其他候选人，无疑是会让你脱颖而出的一个点。 这个项目，**无论是作为一个加分项，放在你的Agent项目中，还是单独成为一个小的算法项目，我觉得都可以。**这个项目的目标很明显，提升小模型在资源受限上的可用性，同时效果也很容易验证，因为我们制定了专门的数据集去评估，对比基模和训练后的模型，作为一个简历项目来写，是非常清晰的。

整体项目简历范文

面向预约 Agent 的本地小模型后训练与部署 | 个人项目 | LLM 算法 / 推理优化

背景： 智能预约 Agent 需要从多轮对话和工具结果中抽取时间、时长、用户偏好等槽位，并在信息完整时决定是否调用查询工具。远端大模型虽效果稳定，但存在网络依赖、数据隐私、调用成本和响应延迟问题；通用本地小模型则容易出现 JSON 协议损坏、字段幻觉、状态继承错误和 Tool Calling 决策不稳定。

目标： 将高频、边界明确的预约槽位抽取与工具调用决策迁移至本地小模型，通过“评估集先行 → 失败难例驱动数据 → SFT → DPO → 量化部署”的路径，在受限 CPU 资源下提升结构化输出稳定性，并以统一评估集验证质量、速度和资源开销。

过程：

• 建立 51 条冻结私有评估集，覆盖 Final/Tool Call、多轮状态继承、用户确认、无关输入与幻觉陷阱；以协议遵循、任务正确性、有效通过率、P95、TTFT 和吞吐作为统一选型与回归指标。

• 在相同 Prompt、Schema 和硬件口径下对比 Qwen3 0.6B/1.7B/4B 与远端强模型；1.7B 相比 4B 任务正确性仅低 2.9 个百分点，但平均时延降低 34.4%、吞吐提升 32.9%，据此作为质量—性能平衡候选。

• 构建“业务规格 → 强模型生成 Raw → 程序质量门禁 → SFT/DPO 派生”数据流水线，生成 500 条 Raw 数据及 450/50 条 SFT、135/15 条 DPO 训练/验证数据，覆盖 5 类核心任务并与评估集保持零重叠。

• 基于 LLaMA-Factory 搭建 LoRA/QLoRA SFT 训练，采用原生 Chat Template、Response-only Loss 与多任务混训，重点提升严格 JSON、槽位继承、相对时间归一化和 Tool Calling 参数准确性。

• 在 SFT 权重上设计 DPO 偏好对齐，将标准答案作为 Chosen，并通过规则构造格式合法但包含动作选择、字段值、确认状态、幻觉或约束违反的 Rejected，针对性强化业务边界与拒绝脑补能力。

• 设计 LoRA 合并、GGUF 转换与 Q4_K_M 量化流程，使用 llama.cpp 暴露 OpenAI 兼容的本地 CPU 服务；以同一冻结集比较 Base/SFT/DPO/量化模型，量化回退超阈值时切换 Q5_K_M/Q8_0。

结果：

完成 4 类模型的统一基线评估与训练数据闭环，冻结 51 条评估样本并构建 500 条 Raw、500 条 SFT、150 条 DPO 数据，训练/评估重叠为 0；本地基线实验确定 Qwen3-1.7B 为优先质量—性能候选。完成训练后应补充：协议遵循率 **[Base → SFT → DPO]**、任务正确性 **[Base → SFT → DPO]**、有效通过率 **[X/51 → Y/51]**，以及量化后模型大小、P95、TTFT 和吞吐，不能使用未运行的目标值代替实验结果。

技术栈：

Qwen3、LLaMA-Factory、LoRA/QLoRA、SFT、DPO、ShareGPT、Tool Calling、Structured Output、GGUF、llama.cpp、Sentence-Transformers、Python

训练完成后可替换的量化条款

> • 基于 LoRA/QLoRA 对 Qwen3-1.7B 开展 SFT，并在 SFT 权重上使用 150 条偏好对进行 DPO；在 51 条冻结评估集上，协议遵循率由 **[Base]** 提升至 **[SFT] / [DPO]**，任务正确性由 **[Base]** 提升至 **[SFT] / [DPO]**，Tool Call 有效通过数由 **[X/15]** 提升至 **[Y/15]**。
>
> • 将训练权重合并并转换为 GGUF，通过 Q4_K_M 量化与 llama.cpp 完成本地 CPU 部署；模型体积由 **[实测值] GB** 压缩至 **[实测值] GB**，P95 延迟由 **[实测值] s** 降至 **[实测值] s**、吞吐提升 **[实测比例]**，任务正确性回退控制在 **[实测百分点]** 内。

这个项目，可以扩展的部分。**比如你可以去修改训练算法，用更复杂的训练方法，比如Agentic RL，或者用更复杂的训练框架，手写框架，修改参数，训练更复杂的模型等等，都是这个项目的可以扩展的方向。**

但是我不给更多建议，**因为我相信大部分人都是开发或者产品，不是算法出身。**我一直把这个项目，算法的学习路线作为**进阶项。你先学完前面的Agent,RAG项目再学算法。**你能学到这里，把这个项目学完，其实对于入行大模型来说**早就够**了，这个项目吃透，对于那些开发岗，也是够了。

再往深入了做，其实是在**算法，推理部署方向岗位上的进阶了，看笔记的大部分同学可能也没做到那一步。**如果你能够把上面的内容都吸收透，对于工作5年以内，找个年薪100w以内的工作，完全没有问题。我说这个话很有把握，因为我一直都在面试，我自己对行情有了解，上面的内容就是我大概自学大概一年多的所有经

验和项目，你再配合笔记，把理论部分弄透。5年工作经验，P7左右，找个100W的工作绝对没问题，校招生，大厂大模型开发岗，全栈岗SSP绝对没问题，就上面的项目+资料+面经消化透，就够了。

1.4 RAG 项目

这个RAG项目，我还是相对比较放心的。**整体就是符合企业RAG的流程，BM25+语义检索 + 重排序**，以及评估方法。这个项目也是有很多人拿过去面试，我们也收集了很多面试真题 使用这个项目，我的个人建议是： 按照文档，视频，代码，先理解消化这个项目，这个项目的各个流程，都是重点。MCP, 里面的各种RAG技术都是RAG里面最经典的方法，必须要会的。 做完这个其实也满足了我们在项目部分讲的 完成一个生产级别的RAG项目的前两个阶段。

完成基础学习后，这一部分建议你必做：

准备至少 100 份真实 PDF 文件，完整运行一次数据摄取、检索、重排和回答生成流程。PDF 不能只选排版规整的文档，需要覆盖普通文本、双栏排版、页眉页脚、侧边栏、表格、图片和扫描件等类型。为这些文档建立清单，记录文档类型、页数、解析是否成功、生成的 Chunk 数量和异常情况。

在这些文档上人工准备一套评估数据集。建议至少准备 200 个问题，并为每个问题标注标准答案、证据文档、页码和对应 Chunk；问题需要覆盖事实查询、跨段归纳、专有名词、表格数据、图片信息、相似文档区分以及无答案问题。通过系统中的 Ragas 指标评估 Faithfulness、Answer Relevancy、Context Precision 和 Context Recall，同时用 Hit Rate@5、MRR 等指标判断检索是否真正找到了正确证据。还要记录 P50/P95 延迟、Rerank 耗时和 Token 成本，不能只看回答效果。

评估时分别运行 Dense Only、BM25 Only、BM25 + Dense + RRF、Hybrid + Cross-Encoder Rerank 四组实验，使用相同数据集进行消融对比。Cross-Encoder 可以使用 `cross-encoder/ms-marco-MiniLM-L-6-v2`。通过 Query Trace 查看 Dense/Sparse 召回结果、RRF 融合排名、Rerank 前后排名变化和各阶段耗时，从而判断效果提升究竟来自哪一个环节，而不是只凭最终回答猜测。

在实际使用过程中，一定要记录具体问题和解决过程。例如，部分 PDF 的侧边栏、目录、页眉页脚或免责声明会被重复拼接进正文。这些高频噪声可能获得较高的 BM25 分数，也会破坏语义分块，导致错误内容进入 Top-K。可以根据文本块坐标过滤页面边缘区域，统计跨页重复内容并删除高频模板块，再按照标题层级重新分块。修改后重新运行冻结评估集，对比噪声率、Hit Rate@5、MRR、Faithfulness 和 P95 延迟，确认优化是否有效以及是否引入新的副作用。

类似的问题还包括表格断行、扫描件 OCR 丢字、图片 Caption 不准确、Chunk 过大或过小、同名实体混淆、Rerank 超时等。每个问题都建议按照“问题现象 → Trace 证据 → 根因 → 修改方案 → 修改前指标 → 修改后指标 → 副作用”的格式记录，并把发现的 Bad Case 回灌到评估集中，作为后续版本固定的回归用例。只有真正跑过这一遍，你才能在面试中讲清楚系统哪里出过问题、为什么会出问题，以及你如何用评估数据证明修改是有效的。

写入简历示范：

> • 基于 100+ 份真实 PDF 构建 200+ 条 Golden Set，覆盖双栏、侧边栏、表格、图片及扫描件；使用 Hit Rate@5、MRR 与 Ragas Faithfulness/Context Recall 评估摄取—检索—生成链路，通过 Trace 定位跨页侧栏噪声，并采用坐标过滤、重复块去除和标题感知分块修复，使 Hit Rate@5 从 **[实测值]** 提升至 **[实测值]**、Faithfulness 提升 **[实测百分点]**，同时将 P95 延迟控制在 **[实测值]** 内。

这里的指标必须替换成你实际运行得到的数据，不能直接编造。例如，如果实测为 Hit Rate@5 从 76% 提升到 89%、Faithfulness 从 0.78 提升到 0.90，同时 P95 从 1.8 秒增加到 2.1 秒，就应该同时说明质量收益和 Rerank 带来的延迟代价。面试官更看重你是否理解取舍，而不是只展示最好看的数字。

完成上面的基础实测后，**还可以继续选择下面三个方向进行增强。建议根据自己的时间和目标岗位选择，不要求全部实现。下面的地方，是有余力可以做的：**

查询改写

真实用户的 Query 往往很短，包含口语、省略、指代或错误术语，直接做一次向量检索容易漏掉正确文档。可以在检索前增加可插拔的 Query Rewrite 模块，并针对不同问题采用不同策略：普通问题进行术语归一化和指代消解；信息较少的问题使用 Multi-Query 生成多个检索表达；抽象问题使用 HyDE 生成假设答案后检索；复杂问题拆成多个子问题分别召回，最后通过 RRF 合并结果。为避免改写产生语义漂移，需要保留原始 Query 作为一路召回，并设置最大改写数量、超时和失败回退。

不要只验证“改写后的句子看起来更完整”，而要在同一套 Golden Set 上对比 Original Query、Multi-Query、HyDE 和 Query Decomposition 的 Hit Rate@5、MRR、Context Recall、P95 延迟与 Token 成本，并按照短查询、专有名词、指代、多跳问题等标签分别统计。最终根据问题类型进行策略路由，而不是让所有 Query 都经过成本最高的改写方式。

写入简历示范：

> • 设计可插拔 Query Rewrite 模块，针对短查询、指代和多跳问题分别采用术语归一化、Multi-Query、HyDE 与问题分解策略，并保留原始 Query 参与 RRF 融合以抑制语义漂移；基于 Golden Set 完成消融评估，使复杂问题 Hit Rate@5 从 **[实测值]** 提升至 **[实测值]**，Context Recall 提升 **[实测百分点]**，P95 延迟增加控制在 **[实测值]** 内。

Agentic RAG

传统 RAG 的流程是固定的“一次检索 → 一次生成”，遇到信息不足、跨文档问题或首次召回失败时不会主动调整。可以把检索能力封装为 Agent 工具，由 LLM 基于 ReAct 方式执行“分析问题 → 选择知识库/检索策略 → 调用工具 → 判断证据是否充分 → 改写 Query 或继续检索 → 基于证据回答”。工具可以包含混合检索、按元数据过滤、文档摘要、相邻 Chunk 扩展和指定文档查询等能力。

Agentic RAG 的重点不是让模型无限自主搜索，而是限制行动空间。应设置最大检索轮数、Token/成本预算、终止条件和引用校验；证据不足时明确拒答，工具异常时回退普通 Hybrid RAG。评估时除了最终 Faithfulness 和任务成功率，还要评估工具选择正确率、参数正确率、平均检索步数、无效循环率、Token 成本和 P95 延迟，并通过 Trace 检查 Agent 是否走了合理路径。

写入简历示范：

> • 构建基于 ReAct 的 Agentic RAG，将混合检索、元数据过滤、文档摘要及相邻 Chunk 扩展封装为工具，由 Agent 根据证据充分性动态执行查询改写与多轮检索；通过最大步数、Token 预算、引用校验和 Hybrid RAG 回退限制失控路径，使多跳问题成功率从 **[实测值]** 提升至 **[实测值]**，无效工具调用率控制在 **[实测值]**。

Graph RAG

Graph RAG 更适合实体关系密集、需要跨文档关联和全局归纳的知识库，例如组织关系、供应链、科研文献、法律案件或设备故障链路。实现时需要从 Chunk 中抽取实体、关系和来源证据，完成实体消歧后写入图数据库；再进行社区发现并生成分层摘要。查询阶段可根据问题选择 Local Search，在实体邻域中查找具体关系，也可以使用 Global Search，基于社区摘要回答跨文档的整体性问题；最终将图检索结果与原有 BM25/Dense 结果融合。

这一方向投入较高，需要处理实体抽取质量、同名消歧、关系更新、图谱增量构建和社区摘要成本。很多企业的普通 FAQ 或制度问答使用 Hybrid RAG 已经足够，引入 Graph RAG 不一定有收益。因此应先从现有 Golden Set 中筛选真正需要多跳关系推理的子集，只在该子集上比较 Hybrid RAG 与 Graph RAG 的任务成功率、证据覆盖率、构建成本和查询延迟；如果收益不明显，应保留为技术预研而不是强行上线。

写入简历示范：

> • 面向跨文档关系推理场景预研 Graph RAG，从文档中抽取实体—关系—证据三元组，经实体消歧、社区发现和分层摘要构建知识图谱，并组合 Local/Global Search 与向量检索；在 **[多跳样本数]** 条关系型问题上将任务成功率由 **[实测值]** 提升至 **[实测值]**，同时评估图构建成本与查询延迟，据此限定其仅用于高关联度知识场景。

整体项目简历范文

1.4.1 现有能力版（可直接使用）

模块化企业知识检索与 Agent RAG 系统 | 个人项目 | RAG / LLM 应用开发

背景：

针对企业知识库中文档格式复杂、关键词搜索无法理解语义、检索链路难以调试，以及不同模型和存储后端切换成本高的问题，设计模块化 RAG MCP Server，为 AI Agent 提供可追溯的私有知识检索能力。

目标：

构建覆盖文档摄取、混合检索、精排、评估和可观测性的通用 RAG 框架；通过标准 MCP 工具解耦 Agent 与知识层，并以可插拔接口适配不同模型、向量库和部署环境。

过程：

• 设计 LLM、Embedding、Splitter、VectorStore、Reranker、Evaluator 六类 Base 接口，结合 Factory 与 YAML 实现配置化切换，支持 OpenAI、Azure、Ollama、DeepSeek 四类 LLM Provider。

• 构建 Load→Split→Transform→Embed→Upsert 五阶段摄取流水线，采用 MarkItDown、递归分块及 LLM 元数据增强处理 PDF，并通过 SHA256 与内容哈希保证增量摄取和幂等写入。

• 实现 BM25 与 Dense Embedding 并行召回，通过 RRF 融合兼顾专有名词和语义匹配；Reranker 支持 None、Cross-Encoder、LLM 三种模式，精排异常时自动回退融合结果。

• 基于 MCP 标准暴露 `query_knowledge_hub`、`list_collections`、`get_document_summary` 三个工具，返回结构化引用及图像内容，支持 Copilot、Claude 等 Agent 调用私有知识库。

• 构建 Ingestion/Query 双链路 Trace，覆盖摄取与查询共 10 个核心阶段，记录耗时、候选数、分数及 Rerank 排名变化，并通过 6 页面 Streamlit Dashboard 定位检索 Bad Case。

• 采用 TDD 与分层测试体系验证模块边界，累计 1364 个测试函数覆盖 Unit、Integration、E2E；通过配置外置、Provider 工厂和 Local-First 存储提高项目复现与扩展能力。

结果：

完成 6 类可插拔组件、4 类 LLM Provider、3 个 MCP 工具、10 阶段双链路追踪和 6 页面管理平台，并以 1364 个自动化测试函数保障核心链路回归。当前结果体现的是工程完整度；检索准确率、Faithfulness 和 P95 延迟需通过下述真实 PDF 实验补充后再写入正式简历。

技术栈：

Python、RAG、BM25、Dense Retrieval、RRF、Cross-Encoder、ChromaDB、MCP、Ragas、Streamlit、Factory Pattern、TDD

1.4.2 增强版条款（完成实测后择优替换）

下面内容对应项目尚未完整验证的增强方向。完成代码、实验和指标记录后，从中选择 1～2 条替换现有简历条款，不建议全部堆入一份简历。

> • 基于 100+ 份复杂 PDF 构建 200+ 条 Golden Set，以 Hit Rate@5、MRR 与 Ragas Faithfulness/Context Recall 评估摄取—检索—生成链路；通过 Trace 定位侧栏重复噪声，采用坐标过滤、重复块去除和标题感知分块，使 Hit Rate@5 从 **[实测值]** 提升至 **[实测值]**、Faithfulness 提升 **[实测百分点]**。
>
> • 构建 Query Rewrite 策略路由，针对短查询、指代和多跳问题选择 Multi-Query、HyDE 或问题分解，并保留原始 Query 参与 RRF 融合；经消融实验使复杂问题 Context Recall 提升 **[实测百分点]**，P95 延迟增量控制在 **[实测值]**。
>
> • 构建基于 ReAct 的 Agentic RAG，将混合检索、元数据过滤、摘要和相邻 Chunk 扩展封装为工具，由 Agent 按证据充分性动态改写并继续检索；通过最大步数、Token 预算和普通 Hybrid RAG 回退，使多跳任务成功率提升 **[实测百分点]**、无效工具调用率降至 **[实测值]**。
>
> • 面向高关联知识场景预研 Graph RAG，经实体关系抽取、消歧、社区发现与分层摘要实现 Local/Global Search；在 **[样本数]** 条多跳问题上将任务成功率从 **[实测值]** 提升至 **[实测值]**，并根据构图成本和查询延迟限定使用边界。

1.5 简历示范

好了，了解了上述的内容，其实我们只要根据自己的情况，去组装就好了。 我们按照简历上写两个项目为例：

1. 如果你的 Agent 项目内容较多（例如评估做得深入，或记忆模块做得完整）总之就是有很多地方可以讲，可以把 Agent 单独写成一个项目，也不用加RAG和算法项目了，一个项目纯讲Agent，再加入一个 RAG 项目，算法项目都没用上。

1. 如果你想突出算法能力，可以把算法单独作为一个项目，把 Agent 和 RAG 合并成另一个项目。

1. 如果你的 Agent 项目相对偏浅，可以把算法项目作为一个小点融入 Agent 项目中，增强竞争力，补充你Agent做的不够，再把 RAG 作为另一个项目。

1. 如果你已经有后端项目经验，可以先写一个后端项目，再按岗位需要把这三个项目进行组合；例如三项目合一为一个 Agent 项目，并将其中的 RAG 和算法作为小点呈现，面试时再按问题深入展开。

当然其实还有非常多的思路和方法，比如**在Agent项目中加一点后端的内容，Redis技术。**比如算法项目做得更加深入，或者增加一些推理部署的内容，或者多模态的内容，或者包装一些上线的内容。 我想给大家展示的是这个思路，同时我并没有天马行空，包装很容易，我只在我们做过的项目，已有的知识上给到包装，给到一个可行的路线，你知道如何做，如何保证，包装的内容去哪里学。因为我觉得只有是你真正做过的，学过的，才是你的。如果你切实际包装一个天马行空的项目，那也许通过的了简历筛选，但终究一问就露馅，通过不了面试。

1.6 简历资源

上面提到的4个简历Pdf版本，在这里获取：

我用夸克网盘给你分享了「简历示范.zip」，点击链接或复制整段内容，打开「夸克APP」即可获取。 /\~c0fa3a2GfU\~:/ 链接：https://pan.quark.cn/s/5d7fce8e84a5

1.7 视频讲解

---

2 简历编写技巧

在讲写简历的技巧之前，希望大家对自己的简历有一个意识：

写简历不是拿一个模板，按照模板往里面填，填好了修改修改就去投递。而是应该把它当作一个精心修改的作品，用标**准科学的方法，**反复打磨，成型，并且后期根据意见不断修改的一个过程。

该小节推荐了一些科学的写简历的方法和原则，希望大家可以理解。我也是按照这个过程写了自己的简历，包括录制了讲解视频，来展示简历的编写方法。

本文按实际写简历的顺序组织：

1. **第一部分：简历编写方法。** 从哪里开始，按什么步骤写？
2. **第二部分：动笔前的取舍。** 写什么、先写什么、写多少？
3. **第三部分：简历各模块怎么写。** 基本信息、教育、技能、经历分别怎么组织？
4. **第四部分：怎样写得更有说服力。** 怎样突出贡献、成果、技术判断与个人价值？
5. **第五部分：定稿与提交。** 怎样确保好读、准确、能讲清？
6. **第六部分：反馈—反思—修改。** 怎样根据投递结果和情况变化持续更新？

第一次写简历，可以顺着全文操作；已有初稿，可以直接查模块写法与精修原则；已经开始投递，则重点结合第六部分复盘。

2.1 第一部分：编写方法

核心认识：简历不是经历清单，而是岗位匹配的证据

简历不是把所有做过的事情写出来，而是针对目标岗位，选出最有说服力、能经得起追问的一组证据。

> **选材决定展示什么，结构决定别人先看到什么，表达决定别人能否理解，证据决定别人是否相信。**

先有真实素材，再做岗位匹配和表达，不要一开始就把模板填满，也不要先找漂亮句子再把自己套进去。

推荐流程：先收集，再筛选，再表达，最后排版

1. **收集素材。** 不限篇幅，记录教育、实习、项目、工作及其他经历中的问题、职责、动作、约束、结果和证据，形成一份完整的事实底稿。
2. **明确岗位。** 对照目标岗位，判断招聘方需要什么、自己最值得突出的两三个优势是什么，明确岗位方向与展示重点。
3. **筛选经历。** 按相关性与证据价值保留、压缩或合并内容，让不同经历证明不同能力，形成一份有主次的选材清单。
4. **组织初稿。** 安排模块顺序与大致篇幅，写清基本信息、教育、技能和重点经历，形成一份结构完整的文字初稿。
5. **打磨表达。** 从职责转为具体动作、判断和结果，突出个人贡献，核对数字与责任边界，形成一份重点清晰、事实准确的精修稿。

6. **核验与排版。** 检查日期、项目性质和个人贡献，调整顺序、篇幅与一页/两页，检查导出文件，形成可提交的岗位版本。
7. **准备追问。** 确保重点经历都能讲清方案、验证、失败和局限，准备与简历一致的面试底稿。

国家大学生就业服务平台也建议实践经历具体化，成果尽量“数量化或者具体化”，并按企业与岗位调整重点。

投递不是流程的终点。 开始投递后，继续记录结果与反馈，反思问题并修改；新增经历、求职目标变化或市场需求变化时，也回到相应步骤更新。具体做法见第六部分。

例子：一条素材怎样从底稿进入简历

事实底稿：做过列表页、导出、权限校验；导出曾经阻塞请求，本人实现后台任务和状态查询，并验证取消、失败重试，暂时没有可靠耗时对照。

1. **收集**：把上述事实、分工和测试记录完整记下。
2. **匹配**：投后端岗，优先选择异步任务与异常处理。
3. **去重**：普通列表页不展开，避免重复展示基础接口开发。
4. **组织**：把该成果放入对应实习或项目经历，不拆成两段重复展示。
5. **表达**：写成“将同步导出改造为后台任务，提供状态查询、取消与失败重试，并补充相关回归测试”。
6. **核验与排版**：确认取消和重试确已完成，不补造吞吐提升数字；压缩为一条重点要点，不展开所有接口和状态字段。
7. **追问**：准备任务状态机、取消竞态、失败恢复与设计局限。

为什么这样做：先有事实，再形成表达。不是先找一条漂亮的模板，然后想办法把自己套进去。

2.2 第二部分：动笔取舍

这一部分先解决整份简历的方向、结构与篇幅，不急着逐句润色。

岗位定位：同一份经历，针对不同岗位调整重点

写作前先回答：

1. 目标岗位最需要什么能力？
2. 我的哪些真实经历能证明这些能力？
3. 最希望招聘方记住我的哪两三个特点？
4. 哪些信息只是占空间，并没有增加判断价值？

再决定模块顺序、篇幅和具体措辞，而不是先把模板填满。

例子：同一份经历，投不同岗位怎样选

假设素材：做过管理后台、支付接口优化、自动化测试工具，也参加过校园活动。

- **原做法**：四项都详细写，各占相同篇幅，投所有岗位都用这一版。
- **投后端岗**：优先写支付接口中的事务、幂等与数据库问题，后台页面简写。
- **投测试开发岗**：优先写自动化测试工具怎样覆盖风险场景、接入交付流程，支付经历保留问题分析部分。

为什么这样改：事实没变，证据的选择和顺序变了。让读者先看到“适合这个岗位”的理由，而不是自己再从经历中寻找。

选材与篇幅分配：按价值，不平均分配

没有“每段实习必须五条”“每个项目必须三条”的硬规则。

更相关、更有个人贡献、更有区分度的经历可以多写；重复或较弱的经历应压缩。即使都是实习，也不需要篇幅相同。

判断一条内容是否值得保留

- **相关性**：目标岗位真的关心这件事吗？
- **具体性**：能看出本人做了什么，而不只是团队做了什么吗？
- **独特性**：删掉它，会不会损失其他内容无法证明的能力？
- **可信度**：有实际交付、采用、测试或测量依据吗？

如果三条要点都在表达同一项基础能力，可以合并；如果分别证明技术实现、复杂问题处理和跨团队交付，就更值得各自保留。

不是先给经历分配空位，而是让最强内容获得篇幅。

例子：两段实习不必都写五条

原安排：实习A五条、实习B五条，项目和开源没位置。

检查后发现：A里三条分别是“开发查询接口、开发新增接口、开发修改接口”，证明的能力高度重复；B里有一次独立故障定位，反而埋在最后。

调整示例：

- A合并常规接口开发，保留最有价值的两三条。
- B把故障定位放第一条，普通维护压缩。
- 给能证明新能力的项目或实质开源贡献腾出空间。

为什么这样改：删的是重复信息，不是按经历类别一刀切。两三条只是本例结果，不是新的固定配额。

经历与项目组合：让不同经历证明不同能力

项目多，不自动意味着简历更强。多个项目如果重复证明同一项能力，信息增量可能很小。

更好的组合是：

- 一段证明业务交付。
- 一段证明复杂技术问题处理。

- 一段证明实验、验证与方案取舍。

同一项目的不同版本，通常应作为演进过程，不应只换名称凑项目数量。工作经历中已经详细写过的成果，也不必再放进项目栏重复一遍。

例子：三个相似项目，不如各有侧重

原安排：商城、论坛、博客各写一段，但每段都是登录、增删改查、数据库连接。

调整示例：

- 商城：重点写支付状态与重复请求处理，证明业务正确性。
- 论坛：如果确实有不同难点，突出搜索、缓存失效或内容审核流程。
- 博客：如果只是重复基础功能，就压缩或删除，不为凑三个项目保留。

版本关系例子：同一个工具的V1、V2、V3，合并为“从单机实现到可恢复任务处理的演进”，比拆成三个名称不同的项目更清晰。

模块顺序：让最有判断价值的信息靠前

模块顺序不是固定模板，应结合求职阶段与核心优势安排。

- **在校生、应届生**：可以按“基本信息 → 教育背景 → 相关实习与项目 → 技能及其他补充”组织；技能也可前置，便于识别技术方向。
- **有相关工作经验的工程师**：可以按“基本信息 → 相关工作经历与成果 → 必要的补充项目 → 技能、教育及其他信息”组织。
- **资深工程师、技术或团队负责人**：可以按“基本信息 → 可选的简短职业概述 → 工作经历与代表成果 → 技能、教育及其他补充”组织。
- 同一模块内通常按时间倒序排列，时间线应清楚。
- 可以调整模块顺序、每段内部的要点顺序和展开篇幅，不为突出某段经历而弄乱时间关系。
- 职业概述不是必填项；只有能简洁概括相关经验、责任范围和代表成果时才值得写，避免“学习能力强、热爱技术”等空泛自评。
- 不必为了模板齐全保留没有有效内容的模块。

页数建议：学生与应届生通常一页，资深候选人可以两页

一般原则：大学生找实习、应届生找工作，通常建议写一页；工作5年以上、比较资深的工程师、技术或团队负责人，以及有多段大厂经历的候选人，可以根据相关经历与成果的丰富程度写两页。

- **在校大学生找实习，通常一页。** 集中展示教育背景、相关技能、最有价值的实习与项目，不把课程和校园经历全部铺开。
- **应届生找工作，通常一页。** 优先保留与目标岗位最相关的实习、项目和成果，让招聘方快速看到核心竞争力。
- **工作经验较少的工程师，优先一页。** 突出主要工作贡献，压缩学生时期或与目标岗位关系较弱的经历。

- **工作5年以上、相关成果较丰富的工程师，可以两页。** 展示有代表性的工作经历、技术难点和实际成果，不必把多年积累硬挤成一页。
- **资深工程师、技术负责人或团队负责人，可以两页。** 除技术贡献外，交代架构决策、责任范围、跨团队推进或团队管理成果。
- **有多段大厂经历，且各段都有值得展示的相关贡献，可以两页。** 保留各段经历中有区分度的职责与成果，不只是罗列公司名称和重复工作。

工作年限和公司背景是参考，不是页数配额。 工作5年以上不代表必须写两页，有多段大厂经历也不代表每段都要详细展开；只有当第二页能补充与岗位相关、不可替代的有效信息时，才值得保留。招聘方明确规定篇幅时，以申请要求为准。

为什么学生与经验较少的候选人通常建议一页

对多数校招、实习和经验较少的候选人，一页通常足以展示最重要的证据。主要好处有四点：

1. **降低阅读与查找成本**：教育、技能和主要经历集中呈现，减少翻页和来回查找。这不等于招聘方严格只看“六秒”；不能把特定历史研究的初筛平均值当作所有岗位的固定规则。
2. **提高有效信息比例**：最强证据集中展示，避免被普通职责、重复项目和无关背景淹没。一页的价值不只是短，而是内容经过筛选。
3. **迫使作者做取舍**：回答“对这个岗位，我最值得展示什么”，删掉无关技能与空泛自评。**一页是一种编辑约束，不是排版考试。**
4. **给面试留下清晰入口**：正文交代问题、关键动作和结果，完整实现和方案讨论留到面试，不把简历写成项目说明书。

这四个好处可以通过具体的修改体现：

- **降低查找成本**：原本教育在第一页，相关实习在第二页，技能散落各处；精简后主要背景与最强经历集中展示，读者不用翻页才能拼出能力轮廓。
- **提高有效信息比例**：原本最强成果只有三条，却夹在十几条常规职责里；删除重复职责后，让三条关键成果成为主体。
- **迫使作者取舍**：原本为展示全面，写了所有课程与学习过的工具；调整后只保留与目标岗位相关的课程和有实践证据的工具。
- **留下追问入口**：原本用半页解释整个系统架构和所有接口；调整后用两三条说明问题、关键实现和结果，架构图留到面试展开。

MIT 建议通常一页，但明确给经历丰富或高等学位者保留例外。这些是指导建议，不是证明一页必然提高录用率的实验。

- 经历较少、重复内容较多时，优先压缩到一页。
- 第二页主要是技术栈、长背景和职责罗列时，先精简。
- 多段相关经历均有独特、可信贡献时，两页可以合理。
- 如果一页必须删掉强证据或缩到难读，就不值得硬压。

- 申请方明确规定篇幅时，按要求提交。

当实习、项目、开源贡献放不下时：

1. 先删无关与重复。
2. 合并相近的要点。
3. 按岗位价值重分篇幅。
4. 检查最强证据是否仍完整。
5. 最后才决定是否需要两页。

不要第一步就整块删除技能栏，也不要固定每段实习五条，再让项目争夺剩余空间。

> **精简的一页优于注水的两页；扎实、清晰的两页，也优于硬挤到难读的一页。**

例子：什么情况压缩，什么情况保留两页

情况A：两页里有半页技术名词和课程清单。

- 做法：删去无关课程与没有实践的技术，把重复职责合并，优先回到一页。
- 原因：第二页增加的是字数，不是新证据。

情况B：有多段相关工作，分别涉及系统迁移、性能诊断和跨团队交付。

- 做法：精简后若每段仍有独特贡献，可以保留两页，第一页先放最相关成果。
- 原因：不为形式上的一页牺牲关键证据。

情况C：一页只能靠极小字号与几乎没有间距实现。

- 做法：先改选材与表达；仍无法清晰容纳，再用两页，而不是继续缩字号。
- 原因：招聘方需要看清内容，“物理上塞进去”不等于好简历。

2.3 第三部分：模块写法

这一部分解决“各处放什么、怎样组织”。先写出完整初稿，再按第四部分打磨句子，不必第一遍就追求每句话都完美。

基本信息：让招聘方知道你是谁、怎样联系

写什么：姓名、电话、邮箱；按岗位需要补充求职方向、所在或意向城市、作品或代码链接。实习申请可补充真实的可到岗时间、实习时长或出勤安排。

怎么组织：集中放在开头，用文字直接呈现，链接应有效且适合公开访问。

常见问题：联系方式只有二维码；同时列出多个无关求职方向；放入身份证号、详细住址等不必要的敏感信息。照片及其他字段按具体申请要求判断，不默认必填。

示例：

```Plain Text 姓名 | 后端开发实习 电话：..... | 邮箱：..... | 意向城市：..... | 代码链接：..... 可到岗时间：20XX年X月 | 可实习X个月、每周X天
```

> 这里只是字段示意，投递前应替换占位内容，并核对可到岗安排是否仍准确。
>
> **教育背景**：先交代学历，再选择有价值的补充
>
> **写什么**：学校、专业、学位、起止时间或预计毕业时间。成绩、排名、相关课程、双学位与荣誉按岗位相关性及区分度选择。
>
> **怎么组织**：通常按时间倒序列出。学生与应届生可以适当展开；已有丰富工作成果时，教育背景一般简洁呈现。
>
> **常见问题**：把所有课程列出来；只写GPA而不写满分制；排名口径不清；混淆在读、预计毕业与已获得学位。
>
> **示例**：
>
> ```Plain Text
> 某大学 | 计算机科学与技术 | 本科　　20XX.09–20XX.06（预计）
> GPA：3.7/4.0；专业排名：12/180（如真实且值得展示）
> 相关课程：数据结构、数据库系统（仅保留与岗位相关的课程）
> ```

课程、GPA和排名不是必填配额；没有明显判断价值时不必为填满模块而保留。

专业技能：负责定位技术方向，经历负责提供证据

写什么：与目标岗位相关、自己能够解释并有实际使用或练习依据的语言、框架、数据库、工具与技术能力。

怎么组织：可按语言、数据存储、工程工具等类别简洁归组。重要技能可以用短语交代应用场景，不必反复列出完整技术栈或概念清单。

常见问题：把听过的技术全部列出；用星级、进度条表示熟练度；没有依据地写“精通”；技能栏与每个项目重复同一套名词。

示例：

```Plain Text 专业技能：Java、MySQL、Git；有接口开发、SQL执行计划分析与协作开发实践。
```

> 前提是这些实践确实存在。技能栏写“MySQL”，项目中写“依据执行计划调整联合索引”：前者方便检索，后者证明实践，不必两边都重复数据库概念清单。
>
> **实习与工作经历**：写清责任范围和代表贡献
>
> **写什么**：公司、职位、时间；必要的业务背景；本人负责的范围、关键动作、技术或协作难点、成果与验证。
>
> **怎么组织**：

> - 先让公司、职位和日期对应清楚，再用简短背景帮助读者理解工作。
> - 每段最强贡献靠前，普通维护与重复职责压缩，不固定每段要点数量。
> - 资深工程师或负责人可进一步交代架构决策、负责范围、跨团队推进及管理成果，但团队产出与本人贡献要区分。
>
> **常见问题**：整段都是岗位职责；只写团队做了什么；把参与项目写成全项目主导；为填补空档改动任职时间。
>
> **示例**：
>
> ```Plain Text
> 某公司 | 后端开发实习生                                    20XX.07–20XX.09
> - 负责支付回调模块，完成重复通知处理、状态转换与异常补偿。
> - 将版本核对步骤封装为工具，被两个协作团队纳入发布检查流程。

这个示例分别展示模块实现和团队采用价值，并不意味着所有实习都要具备这两类成果。

项目经历：围绕问题、实现与结果组织

写什么：项目名称、性质、时间、本人角色；要解决的问题、关键方案与难点、实际完成内容及验证方式。公开项目可补充有效链接。

怎么组织：用一句话交代项目用途，随后围绕最有价值的个人贡献展开。背景不写成产品介绍，技术栈不代替实现过程；多个版本可合并描述演进。

常见问题：把课程或个人项目写成企业生产项目；只罗列功能；复制教程说明；把计划中的功能写成已经完成；与工作经历重复展示同一成果。

示例：

```Plain Text 批量任务处理工具 | 个人项目 20XX.XX—20XX.XX 面向本地批处理任务，提供任务排队与失败恢复能力。 - 对比进程内有界队列与外部任务队列的吞吐、故障恢复和运维成本， 在当前单实例负载下选择有界队列，并补充失败任务持久化与重启恢复。

> 正文保留关键判断，完整架构、实验过程和所有接口细节放在项目材料或面试底稿中。
>
> **其他补充内容**：只保留能增加判断价值的信息
>
> **写什么**：与岗位相关的开源贡献、竞赛、论文、专利、证书、奖项或校园经历，不要求每类都出现。
>
> **怎么组织**：写清名称、角色、时间与准确状态，必要时补一句实际贡献或证明链接。内容重要且足够丰富时可独立成栏，否则归入简短补充模块。
>
> **常见问题**：只写奖项名称而看不出级别；团队获奖不交代个人角色；把投稿写成发表、提交PR写成已合并、专利申请写成已授权；堆积无关证书。
>
> **示例**：
>
> > 为某开源项目修复导出异常并补充回归测试，PR已合并，附公开链接。
```

如果PR仍在评审，应明确写“已提交，评审中”，而不是沿用已合并的表述。

2.4 第四部分：表达打磨

这一部分回答：同样一段真实经历，怎样写得更具体、更有力度、更可信？原则适用于工作、实习、项目和其他成果，不必在每个模块重复一遍。

表达原则：从"负责什么"转成"解决了什么"

职责只能说明被安排做什么，不能充分说明实际贡献。

推荐弹性结构：

> **动作 + 对象 + 关键方法或约束 + 结果或验证。**

不要求每条机械凑齐所有元素，但至少要让读者知道：本人采取了什么动作，这件事为什么值得写。

改写时追问：

- 原来有什么问题？
- 本人具体做了什么？
- 为什么采用这个方案？
- 完成后发生了什么变化？
- 怎样验证工作有效？

例子：把“性能优化”写成可以判断的工作

原写法：

> 负责订单接口开发和性能优化，提升系统性能。

改写示例：

> 通过执行计划定位订单查询的索引失配，调整联合索引并合并重复查询；在相同测试数据与负载下，将服务端P95响应时间由800ms降至300ms。

为什么这样改：补上了诊断方法、实际动作、指标与比较条件，而不是只换了一个更强的动词。

没有性能数据时的另一种写法：

> 在兼容旧版调用方的前提下完成订单接口改造，补充重复请求与超时重试回归测试，并通过灰度发布交付。

两种写法对应不同的假设事实，不是让人从中任选一句冒用。后一种用真实约束和交付体现能力，不需要硬造百分比。

成果原则：尽可能量化，但不强迫每条有数字

MIT 和 Yale 的职业指导使用的是“如果可以”“在可行时量化”，不是要求每条经历必须包含百分比。

数字的作用是帮助理解结果，不是让句子看起来高级。

没有数字，也可以写成果

- **实际交付或上线**，可以证明工作确实完成。
- **被用户或其他团队采用**，可以证明实现产生了使用价值。
- **回归测试、对账或故障演练**，可以证明正确性、可靠性得到验证。
- **支持新的场景**，可以说明过去做不到的事情，现在可以做。
- **明确的责任范围**，可以说明本人承担了哪部分工作。
- **可重复的性能或成本测量**，可以说明改进有定义明确的幅度。

使用数字前必须清楚

- 测的是什么？
- 与哪个基线比较？
- 数据、环境、负载和统计窗口是否可比？
- 是测试结果还是生产结果？
- 是本人工作带来的变化，还是多个因素共同造成？
- 是否允许披露？

不混淆百分比与百分点、平均延迟与P95、首token延迟与完整响应时间。

不要求每条百分比都同时写出前后绝对值，但本人必须掌握基线；容易造成误解的条件必须保留。

没有数字，不等于没有成果；没有依据的漂亮数字，也不等于有说服力。

例子：有数字与没数字，分别怎样写

有测量记录：

> 将账单回归中的人工操作时间由约3小时缩短至约45分钟，通过脚本完成数据准备与结果对账。

这里写的是“人工操作时间”，不是整个发布流程耗时；“约”也需要记录支持。

没有前后测量，但有交付证据：

> 实现账单数据准备与结果对账脚本，接入版本回归流程，替代原先逐条人工核对的步骤。

不合适的写法：

> 大幅提高效率80%，彻底消除人工错误。

没有依据时，数字和“彻底”都没有增加可信度。

口径小例子：成功率从95%到98%，是提高3个百分点，不是提高3%；延迟从800ms到400ms，可以写“延迟降低50%”，不要模糊写成“性能提升50%”。

技术难度与判断力：用真实约束和方案取舍体现能力

用约束解释技术难度

技术难度常常藏在约束里，例如：

- 兼容旧接口或历史数据。
- 保持数据一致性。
- 支持迁移、灰度和回滚。
- 在有限资源下满足需求。
- 处理跨团队依赖。

只保留确实存在、与贡献有关的约束，不为显得复杂而堆术语。

不必把所有尝试都写成成功

比较多个方案后，发现某个方案没有收益、增加复杂度或造成退化，最终不采用，也可以体现判断力。

关键不是：

> 尝试过很多技术。

而是：

> 根据什么证据，排除了什么，最后选择了什么。

简历正文保留最重要的决策和结果，失败过程、完整实验与复盘留到面试展开。

知道什么时候不该加技术、什么时候停止优化，也是工程能力。

例子：没有采用复杂方案，也能体现判断力

假设场景：为批处理任务评估分布式任务队列，但实际任务规模有限，新增队列会引入运维和一致性成本。

原写法：

> 调研多种任务调度技术，熟悉分布式架构。

改写示例：

> 对比进程内有界队列与外部任务队列的吞吐、故障恢复和运维成本，在当前单实例负载下选择有界队列，并补充失败任务持久化与重启恢复。

为什么这样改：不是因为用了更复杂的组件才厉害，而是根据实际负载与可靠性要求做出选择。面试可以进一步讨论何时需要迁移到外部队列。

项目包装：先守红线，再强化表达

目的：让简历脱颖而出，提高获得面试的机会，让面试官看到你的能力与价值。

包装简历非常重要，但我这里有意并没有写很多章节。因为包装是要花心思的，比如**酥神**：把自己去**国外交换的经历，包装成海归**。**把提了一行代码，包装成参与主导升级**。 包装什么东西，无法一概而论，他是很灵活的。 我想强调的是：就是两个原则和红线，把握住这个红线，只要有利你就可以包装：

> **红线一：履历事实不能造假。** 背调能被调出来的东西，一律是红线，不可造假。
>
> **红线二：写出来的内容必须经得起深挖。** 能讲清背景、本人动作、技术实现、方案取舍、指标口径和异常处理；不只是背一段能自圆其说的答案。

可以重点包装的地方

1. **承担的责任。** 从“参与整个项目”转为突出本人负责的模块、决策或升级，放大个人作用的可见度。可以参与整体、主导局部，但不能把局部责任扩大为整个项目负责人。
2. **用词力度。** 少用笼统的“参与、协助”，根据实际工作改为“实现、负责、设计、推进、主导”。“参与→主导”需要确实承担相应范围的主要决策或推进责任，不是统一替换。
3. **数据与成果。** 优先展示最有竞争力的真实结果，用前后对比、百分比或绝对值突出变化，细节放面试底稿。不凭空增加提升幅度，不把不同环境的最好结果拼在一起；关键测量条件不能省掉。
4. **影响力。** 从代码动作提升到功能交付、用户使用、团队采用、效率改善或风险降低，让读者理解工作意义。影响覆盖多大范围就写多大，不把单模块成果写成全平台收益。
5. **技术难度。** 突出兼容性、并发、状态一致性、资源限制和异常恢复等真实难点，而非只列功能。不给普通任务添加没有发生过的规模与约束。
6. **项目叙事。** 用“问题→方案→实现→结果”组织零散工作，把最强贡献放在前面，删掉重复和低价值细节。不把计划写成完成，不把多个无关任务拼成不存在的项目。
7. **面试引导。** 留下两三个自己最能深入讲解的技术机制或决策，让面试围绕强项展开。能解释原理，还要能说清本人实际做了什么。

2.5 第五部分：定稿提交

内容写好后，再集中检查篇幅、版式、事实与面试准备。定稿是当前岗位版本可提交，不意味着此后不再修改。

精简与排版：降低阅读成本，不追求装饰

先按第二部分的选材标准删无关、合重复、调篇幅，再调整版式，不用排版掩盖内容问题。

- 标题、公司、职位和日期容易定位。
- 每段最重要的信息靠前。
- 少写长段落，多用清晰要点。
- 不用进度条、星级表达技能熟练度。
- 不依赖图标或二维码承载唯一关键信息。
- 不为了挤一页过度缩小字号和间距。

例子：同样内容，怎样让人更容易找到重点

原排版：公司、职位和日期分散；整段经历没有分点；用五颗星表示Java熟练度，联系方式只有二维码。

调整示例：

```Plain Text
某公司 | 后端开发实习生                                20XX.07–20XX.09
- 定位订单查询瓶颈，调整索引并完成回归验证。
- 实现异步导出与任务状态查询，支持失败任务重试。
```

> 专业技能：Java、MySQL：在订单与导出模块中有实际开发经验。
> 联系方式：清晰列出电话和邮箱，二维码仅作补充。

为什么这样改：公司、职位、时间能对应；贡献容易扫描；技能有场景，联系方式不依赖扫码。

事实与隐私核验：每一项都对应真实状态

- 核对教育、实习和工作日期，区分在读、预计毕业、已毕业等状态。
- 核对职位、责任范围和个人贡献，不把团队成果整体归给自己。
- 核对数字的来源、口径与必要限定，找不到依据时改用可验证的定性成果。
- 核对项目、论文、开源贡献等进展，不把计划或评审中的内容写成已完成。
- 检查公司、客户和项目的披露要求，不放内部凭据、客户数据或未经允许的材料。

导出与投递检查：不仅看页面，还要看实际文件

按招聘系统要求导出格式，检查文字可提取、顺序合理、链接有效。上传后如系统解析出字段，核对公司、职位、日期等是否正确。

ATS不是统一考试。Greenhouse文档确认复杂排版可能影响解析，但没有把解析失败等同于自动拒绝。[3] 不能迷信第三方统一“通关分数”，也不能反过来断言所有系统都没有筛选或排序能力。

导出检查小例子：PDF看起来正常，但复制成纯文本后日期与公司错位。应调整结构或上传后修正字段，不是仅凭视觉判断“所有ATS都能读”。

发送前再次确认：文件属于当前目标岗位，联系方式有效，没有占位符、错别字、意外空白页或旧版本内容。

面试准备：每条重点经历都应该是可展开的入口

写完一条经历，检查自己能否回答：

1. 为什么要做?
2. 本人负责哪部分?
3. 为什么这样设计?
4. 有哪些备选方案?
5. 如何验证有效?
6. 有什么失败和局限?
7. 再做一次会怎样改?

不需要把全部答案写上简历，但最好都有事实底稿。

最危险的不是措辞不够高级，而是写了一句自己无法解释的强主张。

例子：一句“缓存优化”应该能展开到什么程度

简历句子：

> 为商品详情查询增加缓存与更新失效机制，降低重复数据库访问。

面试底稿至少准备：

- 为什么缓存商品详情，而不是所有接口？
- 缓存键、过期时间怎样确定？
- 数据更新后怎样处理旧缓存？
- 并发访问、缓存未命中和缓存服务故障怎样处理？
- 本人实现了哪些部分，验证了哪些场景？

需要避免：自己只设置过缓存过期时间，却写成“完整解决缓存一致性与高并发问题”。如果没有实现更复杂的机制，可以讨论后续设计，但不要把它说成当时成果。

提交前检查

- 读者能迅速理解目标岗位与核心优势。
- 最强证据没有被普通职责或冗余内容淹没。
- 各段经历提供不同的信息，不重复凑数量。
- 能看出本人动作和责任边界。
- 数字有依据，必要的环境与口径没有被删掉。
- 没有把计划、示例、团队成果写成个人已完成事实。
- 没有泄露机密或不必要的个人敏感信息。
- 版式易读，不为一页牺牲可读性和强证据。
- 联系方式、日期、链接和导出文件正确。
- 每条重点经历都能承受面试追问。

例子：一次提交前检查能改掉什么

- **想投后端岗，但开头全是页面开发**：调整模块或要点顺序，让相关后端证据靠前，同时保持时间线清楚。
- **两段项目重复列同一套技术栈**：在技能栏集中列出，项目各讲不同的难点和贡献。
- **写“提升50%”，却说不清指标定义**：找回测量记录并写明指标；找不到则删除数字，保留可验证成果。
- **“主导架构”实际只是实现一个模块**：改为“负责X模块”，准确突出该模块的实现与决策。
- **写了尚未接入的监控或尚未合并的PR**：改成真实状态；计划不作为已完成成果，PR明确评审状态。
- **链接需要内部权限，截图含客户信息**：更换允许公开的证明方式，删除敏感内容。
- **简历还有占位符、旧邮箱、错日期**：核对并替换，重新导出检查。
- **技术词很多，但不能解释实际使用**：删除没有实践的技能，把重点放在能够深挖的内容。

最后一问：如果面试官只围绕这份简历提问，我是否愿意逐条展开？不愿意被问的强主张，往往最值得重新检查。

2.6 第六部分：反馈修改

简历不是一次写完就结束，而是要根据投递结果、个人情况和市场反馈，反复修改、持续更新。

主要关注以下四个方面：

1. **投递情况。** 记录投了哪些岗位、用了哪个版本、投递渠道与时间、是否进入筛选或面试、停在哪一环。结合这些信息，检查岗位定位、匹配程度、重点呈现与投递策略。
2. **个人情况。** 关注新实习、新项目、新成果、职责变化，以及毕业状态、可到岗安排、求职意向与地点变化。及时更新事实底稿和基本信息，替换较弱或过时内容，重新分配篇幅。
3. **HR与面试官反馈。** 关注对方是否看懂求职方向与个人贡献，哪些信息缺失、哪些措辞引起误解、哪些内容被反复追问。根据具体问题补充必要背景，明确责任边界，调整要点顺序与面试准备。
4. **市场变化。** 观察同类目标岗位的真实JD中，技能、经验、业务场景、职级和招聘需求是否发生变化。据此调整目标岗位、选材重点与能力补足计划，不机械追逐热词。

2.7 参考来源

以下来源支持文中相应引用的建议，不代表机构逐条认可本文全部框架。模块组织与持续迭代流程属于本库实操总结。

1. MIT CAPD — Resumes：篇幅有条件例外；能量化时量化。
2. Yale OCS — Writing Impactful Resume Bullets：行动、项目与结果；明确个人贡献。
3. Greenhouse — Unsuccessful resume parse：指定产品的解析限制，不能外推到全部ATS。
4. 国家大学生就业服务平台 — 简历撰写有哪些注意事项？：具体化、岗位定制、内容校对。

2.8 讲解视频

https://www.bilibili.com/video/BV1hDeV6LEXy/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

3 简历投递技巧

我会结合自己的投递经历，分享常用渠道、用 AI 管理求职的方法，以及一些踩过的坑。核心想法是：**多找机会，但不要盲投；主动出击，但不要把找工作压缩成一次临时冲刺。**

找机会这件事，就是行情越不好，越缺机会，就越要用更多时间，搜索更多的渠道来去寻找。我总结了下面的渠道，发现其实各有优缺点，尽管最开始我觉得AI+官网搜索很好，强烈介绍给大家这个方法，亲测了以后，

发现这种方法也有它的弊端。

建议还是各个渠道都使用起来，多留意。

3.1 投递渠道

我自己的体会是，求职不只是能力的竞争，也有信息差。同样的背景，多发现一些真正匹配的岗位，就多一些尝试的机会。

如果你觉得机会不够，不妨同时使用几种渠道，而不是只在一个软件里反复刷新。下面按渠道分别介绍，先讲我最常用的 AI 搜索及具体操作。

3.1.1 AI 搜索

AI 搜索是我最想推荐的方法。我在 2026 年 9 月开始的这轮求职中，也用它来寻找岗位。它不只是帮我搜几个链接，还可以把岗位整理成清单，持续记录投递状态。下面接着讲我的具体做法。

我更推荐把 AI 当作一个持续协作的招聘助手，而不是用完一次就关闭的搜索框。

我的流程是：**说明背景 → 搜索岗位 → 补充查漏 → 整理表格 → 自己确认并投递 → 更新状态。**

这里说的“用 AI 投递”，主要是让 AI 辅助找岗位、整理信息和管理进度，不是让它未经确认就替我批量提交。岗位是否合适、用哪份简历、表单内容是否准确，最后都要自己把关。

第一步：说明求职背景

我会为求职单独建立一个文件夹，把背景说明、不同版本的简历和岗位清单放在一起。

背景说明里写清楚个人经历、擅长的方向、目标岗位，以及地点等限制条件。写在Agent.md或者claude.md的长期记忆里，因为以后基本上每次搜索，他都需要这些内容。

第二步：搜索公司和岗位

如果已经有目标公司，可以直接提出任务。比如下面这段是按“只看中国大陆岗位”组织的示例，使用时换成你自己的条件：

> 请搜索谷歌、英伟达、微软、腾讯的官方招聘网站，寻找符合我背景和求职要求的中国大陆岗位。请给出公司、岗位名称、地点、岗位编号和官方投递入口；没有找到的公司单独说明，不要用海外岗位凑数。

如果还不知道有哪些公司值得看，可以先让 AI 扩展公司范围，再逐家查岗位：

> 请先找出在中国大陆有软件研发或 AI 业务、值得我关注的公司，再逐家检查官方招聘网站。不要只列我已经知道的几家，也不要仅凭公司规模判断岗位是否适合我。

我不建议只问一句“帮我找几个工作”就结束。更有效的方式是把范围讲清楚，再围绕搜索结果继续追问。

第三步：补充查漏

我自己使用时，会遇到 AI 找到了一些岗位，但搜索范围还不够完整的情况。这时可以追问，而不是默认第一轮就是全部结果：

> 请检查是否遗漏了其他城市、相近岗位名称或同一公司的其他招聘入口。请说明已经覆盖的范围，以及哪些页面无法访问或仍然不确定。补充结果时不要重复已有岗位。

我的原意不是让 AI 为了“多找几个”硬凑结果，而是让它真正扩大检索范围。找不到就是找不到，访问不了就说明，不要把猜测当成已经查到的岗位。

第四步：整理岗位表

我会让 AI 把结果写进“待投递岗位.md”，直接保留官方投递入口，方便自己逐个打开。

我的真实记录里使用了五列：**公司、待投递岗位、投递地址、建议投递简历、投递结果。** 岗位这一列还会记录地点、岗位编号、匹配方向和需要核对的条件。

下面是我实际使用的岗位清单。原文件按外企和国内公司分成两张表，记录了 59 个岗位、9 个已投递；这些数字是该文件 2026 年 10 月的记录，不代表所有岗位目前仍开放，也不代表已投递就获得了面试。

第五步：投递并更新记录

拿到清单以后，我会自己打开岗位页面，确认要求，填写信息，上传相应版本的简历。如果页面失效、地点不合适，或者存在明显不满足的硬性条件，也要让 AI 更新清单，而不是继续把它当成可投岗位。

投递完成后，可以这样告诉 AI：

> 我已经投递了某公司的某岗位，岗位编号是某某，时间是 2026 年 10 月，使用的是英文应用开发版简历。请更新清单；其他没有反馈的岗位不要改成已投递。

之后收到筛选结果、面试通知或面试反馈，也可以继续补充。这样 AI 才能逐步帮助你看清：哪些方向更有反馈，哪些简历版本需要调整，下一步该补岗位还是补准备。

这里的重点是持续记录，而不是靠几次投递就给自己下结论。怎样根据反馈修改简历，前面的“简历编写技巧—反馈修改”已经讲过，这里不再重复展开。

使用这个方法的**优点**： 你能够快速找到一些你都没想到的公司和岗位。比如我让AI搜索一下，发现什么西门子，宁德时代，美的，中国移动都有相关的岗位。如果你对某个公司特别感兴趣，可以快速地找到对应的和你匹配的岗位。同时AI帮助你管理整个投递的进度，方便追踪，分析。

缺点：太麻烦了。一个公司就得注册一个账号，同时需要填写很多表格，很多公司简历筛选不通过，连感谢信都没有，还要自己去查。最后是有一些官网上的一些岗位信息可能已经过时了。

3.1.2 官网投递

找招聘官网时，可以尝试搜索“公司名 + careers”或“公司名 + 招聘”，再确认页面是否属于公司官方渠道。我身边也有同事通过官网投递外企，不过上面那个AI搜索的方法，其实就和这个方法重叠了。

3.1.3 招聘软件

招聘软件方面，**国内最主流的，机会最多的应该就是Boss.。如果你关注外企，也可以把 LinkedIn（领英）纳入搜索范围。**

猎聘、前程无忧、智联招聘等，也可以作为补充。

3.1.4 官方公众号

可以关注意向公司的官方招聘公众号，尤其留意与你的城市、岗位方向和招聘批次有关的推送。这样就不必每次都从零搜索，也可能在日常刷手机时发现合适的机会。

3.1.5 社区信息

脉脉、小红书、牛客、LeetCode 等社区，也可以留意招聘或内推线索。

3.1.6 HR

通常Boss上 有一些联系人是公司直接的HR，面试多了加了微信，微信列表中也有一些HR的信息，找工作的时候可以直接和他们大哥招呼，问问有没有合适的。

3.1.7 熟人内推

如果能找到了解你的师兄师姐、前同事或朋友，也可以请他们帮忙看看岗位，再做内推。我更看重的是对方能否提供岗位信息、帮助判断匹配度，而不是单纯拿到一个内推码。大家如果找到我所在公司的合适岗位，我也愿意在能力范围内帮忙内推。

3.1.8 猎头

缺点:

有的猎头可能喜欢两边骗，为了完成自己的业绩，不惜把你推向火坑。比如明明不是特别好的岗位，他为了劝你去，可以各种扭曲事实。另外对于社招，猎头推荐成功，可以拿挺大一笔费用的。企业可能把这个预算算在你的招聘成本上，对应着你要价可能就会有劣势。

优点:

他们手上的机会其实都是有效的，同时可以帮助你筛选岗位，甚至能给你提供一些面试真题，面试前的准备。

3.2 投递策略

先用真实面试积累经验，再逐步推进最想去的公司。

如果你很久没有面试，或者还没有面试过，我建议先从优先级稍低、但确实有可能接受的岗位开始。在真实面试里检查自己的表达、项目理解和临场状态，面完及时复盘，再逐步投递最想去的公司。

这比一开始就把最看重的机会全部投完更从容。但也不要为了练手，专门去面试已经确定完全不会接受的岗位，或者把“练习”变成迟迟不投目标公司的理由。

投递前尽量了解公司招聘流程

大部分公司都是有面评的，所以你面试了以后的面试反馈，这个公司的下一个部门面试你的时候会参考。

有些公司有冷冻期，比如大疆，投递了一个部门挂掉后，半年不能面试任何其它公司。

字节好像是半年1面挂了3次，半年内不给面试了。

英伟达据说是可以无限复活的，没有锁定期。

尽量了解更多信息，有策略地投。

3.3 时间安排

校招：围绕自己的毕业年份，提前安排实习和正式招聘。

校招同学就是卡时间节点，通常比如27年毕业，26年的3月就开始暑期实习投递。

25年年末开始准备，3月份尽量达到面试的能力，去面试，这时候岗位多，不要等到3月才开始面试暑期实习。

拿到暑期实习，按部就班，秋招，春招，不要太晚开始，是我的建议。

另外如果你要求更高，秋招时候准备有两份实习经历，那么就要更早规划了。

社招：如果条件允许，长期看机会，不要只押一个月。

我不建议等到必须马上离职、或者被迫找工作时，才第一次看市场；也不建议给自己设定一个很死的期限，比如“我一定要在某个月找到工作”。

更适合我的方式是把观察和求职的窗口拉长，比如在条件允许时，用 **3～6 个月持续留意机会**。因为现在**金三银四**也没那么明显了，不是说招聘的机会都集中爆发在某几个月。最适合你的那个机会，不知道在什么时候放出，所以给自己更长的时间留意机会，找到**合适的岗位概率更大。**

3.4 视频讲解

今天这期内主要介绍了我的一些个人经验，了解的信息，岗位投递的技巧。

如果你不想看文字，也可以看我的视频讲解把本节内容介绍一遍，顺便聊了一下我个人找工作时的一些趣事，感受等等。

https://www.bilibili.com/video/BV1BJnr6AEh2/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

4 自我介绍

自我介绍是面试的第一流程，也是你给面试官的第一印象。 **请不要**讲流水账，把简历的内容照着说一遍。他并不是要关注你是谁，而是关注你和岗位的一个匹配度。 时间请控制在1分钟\~2分钟， 时间非常短, 切忌讲废话，直接灌输他想要的答案。别说什么籍贯，爱好，别说什么 我很努力很认真空话虚话。**在有限的时间内灌输重点。**

参考模板：

1. **亮身份（学校/公司） + 核心标签**：eg. 我叫xx,我做过5年AI开发，我擅长Agent，RAG系统，熟悉常见的AI应用和框架，对模型前后训练，推理也有一定的实践。
2. **用结果证明能力**：我用xx方法得到了xx提升。 使用我做的RAG系统，相比copilot 通用工具，回答的时间降低xx, 准确度提升xx。

3.**体现诚意：** 我了解过贵公司整个项目，我认为我的技术和它非常匹配，我也特别认可贵公司技术发展方向，特别期待加入这个岗位。

5 反问

核心逻辑：

这个环节是为了

弄清你是真的想来，还是只是随便看看

你的关注点，关注点的高度如何

思考能力：是否对岗位有深度思考

参考答案：

针对不同的面试人员问不同的问题：

1. **业务主管**（通常是一面，通常面试官会是你的入职后的同事）：业务问题，岗位工作内容。比如：

团队的人员结构，人员组成，看看团队的实力如何，是大团队还是小团队

上下游人员结构：看看供需是否合理，哪个团队更有话语权，公司更重视什么。比如你面试测试，公司主要做黑盒测试，结果测试只有10人，却需要面对60\~70个开发，那你可以看出测试工程师可能非常忙，而且公司不重视测试，可能测试也没啥话语权。

对候选人的期待，来了要完成什么任务。

1. **大老板**（通常面试官会是你的+1， +2，通常是技术面终面，有的也叫主管面）： 公司战略，产品线发展。比如： 面试业务，产品线，未来的规划，业务属性，是老业务还是新业务，通过这些问题判断下公司是否持续投入，别没几天这活就不做了。
2. **HR**： 薪酬，培训，福利，制度，晋升相关问题，比如

晋升窗口，培训机制（体现你想在这个公司长期发展而不是只考虑眼前）

薪资，社保，公积金其实不用着急，这些面试通过了在拉扯薪资的时候会详细聊

---

6 面试准备建议

5.1 刷题？论文？还是项目经验

面对大模型相关岗位的激烈竞争，许多求职者常常陷入一个关键抉择：应重点投入算法刷题，深耕学术论文，还是积累项目实践经验？

实际上，这三者在面试中的作用各有不同，适配的岗位类型、能力维度与评估标准也大相径庭。本节将从岗位匹配、能力增长路径与面试实效三个层面，系统剖析如何平衡选择刷题、论文与项目实践，制定最优面试准备策略。

5.1.1 刷题

算法刷题，尤其是针对 LeetCode、LintCode、牛客等平台的高频题，是求职准备中的经典环节。它主要面向以下几类考察目标：

（1）**基础数据结构与算法**：
如数组、链表、哈希表、堆、栈、队列、树、图等。

（2）**算法思维训练**：
如动态规划、二分查找、回溯剪枝、贪心策略等。

（3）**代码实现能力**：
如语言熟练度、边界控制、时间与空间复杂度评估等。

在大模型方向，虽然刷题不像传统后端或客户端岗位那样高频必考，但对于以下情境仍具有重要作用：

（1）**校招 / 应届类算法工程师岗位**：
仍以基础算法能力作为初筛门槛。

（2）**多轮面试结构化评估**：
刷题能体现候选人的逻辑思维、编码能力与调试效率。

（3）**特定模块工程实现能力**：
如稀疏矩阵计算、图结构建模、窗口滑动优化等。

因此，刷题并非大模型面试的核心，但作为基础训练，具备过滤风险、拉高下限、稳定输出的重要价值。建议每日保持 30–60 分钟算法练习，以巩固逻辑思维与编码熟练度。

5.1.2 论文

阅读、理解并复述经典大模型论文，是构建专业理论框架、展示对大模型认知深度的重要途径，尤其在以下面试情境中具有显著优势：

（1）**算法岗、研究岗、高级工程岗面试**：
面试官往往会基于论文提问，如 Transformer、LoRA、RLHF、Constitutional AI 等经典论文所提出的算法与架构。

（2）**原理答题或开放式问答**：
通过引用论文结构、图示与结论，可以显著提升答题的专业性。

（3）**简历项目延展问题**：
当被问及某模型为何有效、为何选择某种优化策略时，引入论文机制能体现深度思考与行业同步性。

在实际面试准备中，建议分层次构建论文知识体系：

（1）**第一层**：
Transformer、GPT、BERT、T5 等主流架构类论文。

（2）**第二层**：
微调类（如 LoRA、Adapter）、训练策略类（如 Scaling Law）、对齐类（如 RLHF）。

（3）**第三层**：
实际工作相关论文，如 RAG、Agent、Prompt 优化、多模态处理等。

论文阅读不是为了记忆细节，而是为了建立对模型行为、结构、设计的解释力与抽象能力。建议每周精读 1–2 篇，并形成结构化摘要笔记，便于面试引用。

5.1.3 项目

在大模型岗位的最终评估中，项目经验仍是最具权重的考察指标。相比刷题与论文，项目更能体现候选人是否具备“解决实际问题”的能力。其优势在于：

（1）**真实场景还原**：
涵盖模型构建、训练调试、接口封装、系统部署、性能评估等全流程。

（2）**协作能力呈现**：
展示候选人如何与团队对接，如何解决跨模块问题，体现团队适应能力。

（3）**创新与优化价值**：
工程中解决过的非标准问题、引入的新技术组件、本地化部署技巧等，均为面试中可展开讲述的亮点素材。

企业在招聘大模型岗位时，往往希望候选人能立即参与业务落地，构建真实产品功能或支撑平台系统迭代。这使得有完整项目经历，尤其是端到端执行经历的候选人更具竞争力。

项目经验的准备策略应包括：

（1）选典型项目深挖讲透，不求多，但要逻辑完整。

（2）形成一张项目结构图，让面试官快速理解任务边界。

（3）围绕结果构建技术故事线，突出问题挑战、解决路径与最终影响。

（4）结合代码仓库或在线演示，增加可信度与可展示性。

三者组合策略如下 1-6 所示。

最终目标是构建一个**基础能力稳定（刷题）**、**理论支撑扎实（论文）**、**工程实践落地（项目）**的三维能力体系。

概括地说，刷题、论文与项目经验并非对立，而是服务于候选人综合能力呈现的三大支柱。刷题提升思维与代码基础，论文构建技术深度与解释力，项目体现工程实践与业务关联。在准备大模型岗位面试时，应根据岗位类型与个人阶段匹配选择主线方向，并在三者之间形成协同优势，从而在技术深度与落地能力之间构建真正的竞争力体系。

7 项目难题与面试复盘

（这里来放一下我在面试中出现过的，但不是那么好归类的问题）

7.1 你在项目中遇到的最难的问题，如何解决的?

（这个问题几乎是非常高频考点，建议必须准备，认真准备一个最好的答案。其实问了这个问题，是把主动权交给你自己，我们需要准备一个自己认为技术难度比较深入的回答和问题，讲出技术深度，把每个问题想透，经得住和面试官反复推敲和追问，那么一旦问到这个问题，就是送分题了。我对于自己这个问题的准备在**2025.11.字节跳动（豆包）**的视频讲解中，讲到了我对于这个问题的准备的方法和一些思路，有需要可以去看视频讲解。我在这一章总结几个我自己挖掘准备的这个问题的回答，大家可以参考，甚至可以照着答，当然你能有自己总结，结合你的项目的版本会更好。）

7.1.1 模型调用工具失败及排查

（按照STAR 思路总结：Situation, Task, Action,Result)

1.背景与挑战（Situation & Task)

在这个智能日历项目中，为了保证数据隐私和降低延迟，我负责将云端模型迁移到本地部署的 **Qwen 2.5** 模型上。我们的架构依赖于 **Agent 模式**，其中**Function Calling (工具调用)** 是核心，用于触发日历查询和邮件分析等操作。

遇到的最大困难是：虽然 Qwen 2.5 模型本身在权重层面是经过微调支持 Function Calling 的，但在集成到我们的本地推理环境（使用 Foundry Local 作为 Serving 层）时，Agent 总是无法正确触发工具，而是直接输出文本或幻觉代码。

2. 排查过程（Action & Result)

起初我以为是 Prompt Engineering 的问题，尝试了多次优化 System Prompt，但无效。于是我决定下沉到**协议交互层**进行排查：

- **第一步：抓包与日志分析。** 因为Semantic Kernel 的 Python 版本底层是通过 `openai` 官方库连接模型的，而 `openai` 库底层使用的是 `httpx` 库来发送 HTTP 请求。所以我需要在代码中开启 `httpx` 的 **Debug 日志**，就能在控制台看到完整的请求 Header 和响应 Body（Raw JSON），通过这个方式我获取了SK和llm每一轮交互的信息。
- **第二步：协议对比。** 我对比了云端 GPT-4 的成功响应和本地 Qwen 的失败响应。我发现，标准的 OpenAI 兼容协议要求模型返回一个结构化的 `tool_calls` 字段，包含 `function_name` 和 `arguments`。但是结合第一步，我发现llm给到我的结果并没有function_calls字段，但此时我不确定是llm的问题还是说foundry_local推理框架的问题，因此我需要进一步看到llm输出的最原始内容。
- **第三步：查找模型原始输出。**因为是Local部署，我可以查看推理服务的后台日志，我在Foundry Local的控制台输出中，开启了Verbose模式，我清晰看到了模型输出了和工具调用相关的 ...相关的字段，这证明模型**本身是有意图进行调用的，**并且qwen2.5 的官方说明文档是完全支持function call的，这也侧面印证了模型本身的调用和输出没有问题。
- **第四步：定位根因。综合以上分析，认定推理服务层并没有正确解析这些特殊 Token**，导致它没有将其封装进 API 响应的 `tool_calls` 字段，而是作为普通 `content` 字符串返回给了应用层。应用框架（如 LangChain/Semantic Kernel）读不到 `tool_calls`，自然就认为没有工具需要执行。
- **第五步：确认根因后，我采取了“双管齐下”的策略**：
  反馈与修复：我整理了详细的复现日志和协议对比数据，提交给了 Foundry Local 团队，明确指出了其对 Qwen `tool_calls` 解析支持的缺失，推动他们后续修复。
  架构迁移（关键决策）：为了不阻塞项目进度，我决定更换推理后端。我调研并切换到了 `llama.cpp` 。
- `llama.cpp` 对各类模型的 Chat Template 支持更加成熟，能够精准识别 Qwen 的特殊 Token。
- 迁移后，我再次查看 `httpx` 日志，发现 `llama.cpp` 成功将模型的 XML 输出转换为了标准的 OpenAI `tool_calls` JSON 格式。
- Semantic Kernel 随即成功识别并执行了本地工具，整个 Agent 链路彻底打通。

7.1.2 MCP 连接偶尔失败

（我是以这个我们笔记中的RAG项目为例展开的，如果你项目写了一个MCP的Server，使用stdio通信，那么你就可以使用这个例子。你要弄清楚这个例子，你得懂得MCP的原理，特别是如何通信，它的协议是啥样的）

（按照 STAR 思路总结：Situation, Task, Action, Result）

1. 背景与挑战

在这个模块化 RAG 项目中，我负责实现 MCP Server 模块，通过 MCP协议将 RAG 检索能力暴露给 AI 客户端（如 Claude Desktop、GitHub Copilot）。我们的传输层采用的是 stdio 模式——Server 作为子进程启

动，通过 stdin/stdout 与客户端进行 JSON-RPC 通信。

遇到的最大困难是：Server 独立运行时一切正常，所有 JSON-RPC 消息的构造和处理逻辑都验证通过了，但 AI 客户端（Claude Desktop）连接后，要么在握手阶段直接失败，要么用了几次后突然断连。客户端侧只显示一个通用的"连接失败"错误，Server 端也没有任何异常日志输出。

2. 排查过程

起初我以为是 JSON-RPC 消息格式有问题，比如 protocolVersion 字段版本不匹配，或者 initialize → notifications/initialized 的握手顺序不对。我逐一检查了 protocol_handler.py 中所有消息的构造和分发逻辑，甚至写了单元测试验证每个 Tool Schema 的格式，但没有发现任何问题。

第一步：**跨进程协议流拦截。** 由于问题只在"客户端连接时"出现，单独跑 Server 是无法复现的，打断点也没有意义——Server 端代码逻辑本身是正确的。而现成的 AI 客户端（Claude Desktop）内部是封装好的，它从 stdout 读到数据后直接 json.loads() 解析，失败就报"连接失败"然后断连，不会把 stdout 管道中的原始内容暴露给你，你只能看到一个笼统的错误提示。所以我决定自己写一个最简陋的"客户端"来拦截原始通信内容。具体做法是用 subprocess.Popen 将 MCP Server 作为子进程启动，将它的 stdin/stdout/stderr 都 pipe 出来，然后手动构造 JSON-RPC 消息（如 initialize 请求）发送到 stdin，逐行读取 stdout 返回的原始内容——这样就能看到客户端真正"看到"的是什么。

第二步：**逐行分析协议流。** 通过这个测试框架，我终于看到了客户端视角下的 stdout 原始内容。我发现在合法的 JSON-RPC 响应之前，stdout 里夹杂了非 JSON 的纯文本行，比如 "Modular RAG MCP Server - Starting..." 这样的启动提示信息。客户端把这些内容当 JSON-RPC 消息解析，json.loads() 立刻失败，连接随即断开。

第三步：**定位根因。** 知道了症状后，我排查了这些"脏数据"的来源——项目入口文件中有 print("Modular RAG MCP Server - Starting...")，Python 的 print() 默认写入 sys.stdout。此外，Python 的 logging.basicConfig() 虽然默认绑定 stderr，但代码中没有显式指定 stream=sys.stderr，存在隐患。

第四步：**确认协议规范。** 我回溯查阅了 MCP 官方协议规范（Transports 章节），发现里面用了 RFC 中最强的禁止语气明确写道：

"The server MUST NOT write anything to its stdout that is not a valid MCP message."

"The server MAY write UTF-8 strings to its standard error (stderr) for logging purposes."

这证实了我的判断：stdio 模式下，stdout 是协议专用通道，客户端会将 stdout 的每一个字节都当作 JSON-RPC 协议内容解析。任何非协议内容都会导致解析失败。而 stderr 是独立管道，客户端根本不会读取它，所以日志应该全部走 stderr。

3. 解决方案与结果

确认根因后，我采取了以下措施：

消除所有 print：将 Server 启动路径中所有 print() 改为 logger.info()，确保没有任何业务代码直接写 stdout；

显式日志重定向：将 logging.basicConfig() 显式指定 stream=sys.stderr，确保所有 logger 输出走 stderr，不留隐患；

协议纯净度集成测试：将排查时写的"简陋客户端"正式化为 subprocess-based 集成测试（test_mcp_server.py），模拟完整的 MCP 握手流程（initialize → notifications/initialized → tools/list），逐行验证 stdout 中每一行都是合法的 JSON-RPC 消息。这个测试成为了 CI 的守门测试，确保后续任何人加的代码都不会再污染 stdout；

在 server.py 的模块 docstring 中加入了明确的注释："ensures stdout only contains protocol messages while all logs go to stderr"，作为团队开发规范。

修复后，Claude Desktop 和 GitHub Copilot 均能稳定连接，成功调用 query_knowledge_hub、list_collections 等 MCP Tools，整个 RAG 检索链路在 AI 客户端中彻底跑通。

7.1.3 后训练项目：问题与解决

问题： 你在做这个微调本地模型项目（我们笔记里的算法项目）的时候，遇到了什么问题，怎么解决的？

查看思路与参考答案：微调遇到什么问题，如何解决？

---

8 真实面试过程记录（面试前突击技巧，简历投递方法等，我的真实经历供大家参考）

这个小节，记录一下**26年9月份**开始面试过程的心路历程，我会按照时间顺序记录。记录比如面试动机，简历准备，面试投递，面试前突击准备，复盘，甚至到offer选择的整个历程。这里面会有一些大家值得借鉴的内容，比如简历编写，修改，如何包装项目，包装后如何针对性准备问题，在哪些平台准备，我会通过这个记录把这些内容串起来。

8.1 面试动机

为什么要开始面试？这一次面试希望获得哪些方面的收获？感兴趣的朋友可以看这个视频。

8.2 简历初稿

我这一次好好整理了我的简历，学习了简历编写技巧。更新了笔记：简历编写技巧章节。 根据这些原则，重新修改了简历。简历里的所有项目，用的是笔记里已经记录过的项目：Modular-RAG-MCP-SERVER, 后训练微调项目，SmartAppointment 项目。 加上一个虚构的自动修复Bug的Agent项目，这个项目没做过，在投递简历之后，我会重点更新一下这个**虚构的项目**的架构，技术栈，做一些针对于虚构项目的面试前准备。大家

从这块的内容更新可以学习到如何包装一个没做过的项目，并且能讲出来。这部分我会在第三小节讲解（待更新）。

我们当前章节是简历初稿部分，大家值得学习的是，简历编写技巧，可以看看我的讲解视频（待更新），里面有我写初版简历的设计思路，如何包装的。个人认为通过这么一包装，我们在简历上对项目的包装更有深度了。

我的初版简历准备了两版，一版是**大模型应用开发岗**（主要要求大模型开发），一版是**大模型应用算法岗**（应用算法都要求）。

应用版简历：

应用算法版简历：

8.3 面试前突击

第二小节，我们介绍了简历应该如何编写，如何包装，这个时候我们已经形成了我们的第一版简历了。在正式面试之前，针对于简历的内容：特别是你包装美化的功能点，甚至是虚构，没做过的功能和项目，**建议面试前一定要突击准备一下。** 准备的原则是：

1. **对于你美化过的内容**，首先要说服自己，自己想通你是如何做的？比如数据是怎么来的，实验方法是什么，对照基准是什么？比如我简历里包装了是企业级的项目，那么这么复杂的功能是几个人做的，你主要负责什么？建议对照简历的内容，**逐条的去思考。** 简历里的所有你写的文字，就是面试过程中最可能问的高频考点，如果你自己写的内容，自己都没看，没想通，面试官问了你讲不清楚，这是大忌。
2. **对于你压根就没做的项目**，我的个人建议是：你写一份非常清楚的设计文档，笔记里的RAG项目，算法项目，每个项目都有技术设计文档（我们又叫dev_spec)。写这个文档是一个很费时间的过程，但是写明白了，你对于项目的掌握就会清晰很多。你可以把**它当作一个超级详细的系统设计任务，完成这个系统任务设计再去面试。**
3. **不要要求自己一下掌握所有细节。** 既然你是包装的内容，面试遇到细节答不上来很正常，允许自己面试有细节答不上来。 我们在第一场面试的原则：**尽量不要过于露馅，有的地方答不上来，让面试官觉得是你确实不记得了，而不是在撒谎，作为第一次面试，就已经很成功了**！ 但是，对于每一次面试，都是一个检验我们是否想清楚我们的项目的机会，**请你把对应的答不上来的细节好好消化，继续想得更加深入。这样你会一场比一场面的好。**

这一小节，展示一下我在面试前的突击内容。注意：由于时间原因，我这些内容大概花了一个工作日的时间准备的，同时对于我们虚构的这个项目，我并没有时间想好技术文档再去面试，所以我就先大概想清楚了它的架构图。如果时间充足：建议你按照这个思路，**在面试前多准备一下，多抠一下细节，并且完成虚构项目的设计文档再去面试**！

8.3.1 ForgePilot 项目

8.3.1.1 ForgePilot架构设计

我把我们笔记里写的虚构的自动修复bug的企业级Agent项目，叫做：**ForgePilot。**

如上面所说，对于虚构的项目，建议你是先写好**技术文档**，想清楚细节，写好技术文档再准备简历。

不过我的时间紧张，所以这个过程就没做，刚好也展示一下在时间紧迫的情况下，如何用一个虚构的项目去面试吧。

我的思路是面试前最少做以下准备：

1. 先想清楚该项目基本需求，架构图是什么，有哪些模块，如何运作的。这也是为什么我加了一个3.1小节架构设计。

2. 想清楚这个内容，再去根据简历内容，一条条预测面试可能问的问题，把细节弄透，这个是3.2小节讲的内容。

8.3.1.1.1 项目背景

> 💡说清楚痛点，为什么传统Coding Agent已经很成熟了，还要自己做企业定制Agent。

通用 Coding Agent 能够辅助改码，但企业研发还**面临上下文分散、工程流程复杂和人工审核受限的**问题：团队代码资产超过 300GB，跨仓库、跨分支定位难以仅靠当前本地工作区搜索；bug单 的问题描述、评论、附件和关联单需要整理成完整任务；公司特定的构建、测试与返修流程需要固化，避免 Agent 反复试错和工程师持续跟进。同时，AI 加快代码产出后，人工 Review 可能成为新的瓶颈，需要结合变更意图、关键 diff、影响范围和验证证据帮助人理解与验收。

ForgePilot 的意义不是重造通用编码工具，而是**复用成熟 Agent 能力，补齐企业上下文、交付闭环和可解释评审**，将分散操作组织为团队可共享、可跟踪的任务。模型侧计划逐步适配公司自研本地模型，减少敏感数据外发，并通过经过授权的内部数据微调改善特定任务效果，评估降低长期推理成本的空间；本地部署仍需权限治理和成本、效果评测。

评测价值依据：用户提供的真实任务数据集评测结果为，定制平台相比通用 Claude Code 流程，**端到端交付成功率提升 32%。**具体评测平台与版本、成功标准、样本规模、对比条件及相对提升/百分点口径待补充；该结果不直接代表当前 ForgePilot 设计已实现或完成同等验证。

8.3.1.1.2 架构图

> 💡架构图是最直观的帮助我们从宏观上理解我们项目的方式了。时间有限没法设计项目细节的时候，必须先把架构图想清楚。整个架构可以说是借鉴了我们学过的各种Agent架构：ClaudeCode, Pi-Agent, DSH。 比如**底层的Agent我设计成使用了Pi-Agent框架，整体使用DSH的插件化思想。**
>
> 如果你没有相关基础，你学起来会很吃力。但这其实也告诉我们，包装项目不代表你能绕过基础知识的学习，能把包装内容讲清楚，是建立在我们扎实的理论基础上的。

React + TypeScript 工作台

💡正儿八经的企业项目，肯定都有健全操作的。

提供对话模式、任务模式和评审模式。用户在这里提出问题、下发任务、补充要求、查看进度和审核结果。多个模式共享后端能力，不是彼此独立的系统。

- **对话模式**：解决开发者不熟悉项目、查找代码和理解逻辑耗时的问题，通过代码问答帮助理解项目与定位问题。
- **任务模式**：解决 Bug 修复中理解工单、定位、修改和测试的重复劳动，输入 Bug Ticket，执行修复并交付可审核的 PR。
- **评审模式**：解决 AI 生成代码快、人工 Review 跟不上的问题，将修改后的 PR 按逻辑组织成清晰、可视化的变更说明，帮助人理解修改目标、实现思路和影响范围。

统一接入与身份认证

> 💡正儿八经的企业项目，肯定都有健全操作的。

企业身份认证与权限控制：确认员工身份，根据团队和项目授权，限制可访问的代码仓库及允许执行的操作，并将权限约束传递到 Agent 和插件。

任务理解与路由编排

> 💡这一层加了一个本地模型，主要通过本地模型分类处理 + Harness实现信息提取，找了个合理需求把我们后训练项目嵌入其中。

这一层位于 Cordis 插件化执行层上方，由本地模型与 Harness 协作，将用户需求整理成可执行任务。Harness 指我们编写的流程控制与运行管理代码。

- **本地模型——理解与分类**：理解 Bug 单，提取问题现象、触发条件、实际结果和预期结果，识别缺失信息，并给出任务分类建议。
- **Harness——校验与上下文组装**：校验模型输出，必要时请求用户补充；结合项目配置、权限、会话历史和相关记忆，准备执行上下文。
- **Harness——插件适配与路由**：根据任务类型和项目要求，选择 Agent 引擎、检索器及执行工具等适配器，通过 Cordis 完成服务装配，再将任务交给 Pi 执行。
- **Harness——运行管理**：跟踪任务状态，落实预算、取消和审批规则，并接收执行结果。

本地模型负责理解需求，Harness 负责确定性规则和执行准备，Cordis 负责插件装配。 新 Bug 按固定流程进行预处理，查询和审批等请求不必经过本地模型；模型分类不能代替权限判断。

> 💡 插件化，引入DSH的技术。同时我们项目背景里写了，有可能处理300+GB的大型仓库，所以针对于不同的项目来选择你的索引，是一个我们可以强调的技术，也和整个插件化的思想比较搭。 同时底层默认使用Pi

> Agent也是一个比较主流的技术。

Cordis 插件化执行层

Cordis 负责插件注册、依赖注入、装配与生命周期管理。业务依赖统一接口，不直接绑定某个具体实现，因此可以按项目配置替换执行能力。

- **AgentLoop，默认 Pi**：维护当前执行上下文，组织模型调用与工具调用的循环，汇报执行过程和结果。
- **主模型适配**：对接选定的主模型，由模型分析原因、提出下一步动作和生成修改建议。
- **检索插件**：提供代码搜索和上下文获取能力；普通项目可用本地检索，大仓库可接企业检索服务。
- **工具执行插件**：在允许范围内实际执行读取、修改和测试等操作，将结果返回 Pi。

主模型、Pi 和工具共同形成 ReAct 式循环：**模型提出动作 → Pi 调用工具 → 工具返回结果 → Pi 带着结果再次请求模型。** 主模型不直接操作工作区。

Cordis 本身不进行模型推理；可替换性来自统一接口和适配器。更换实现仍须满足能力与安全约束，不表示运行中可以无条件切换引擎。

> 💡 会话管理和记忆模块，是必须有的模块。暂时没有精妙设计，我就先借鉴常见开源框架的策略来设计就好。

会话管理

持续保存对话消息、工具记录和执行进度，并在用户重新进入时加载历史，支持连续交互与过程追踪。它是运行期间的支撑机制，不是任务结束后的单独步骤。

保存会话不等于自动恢复所有正在运行的进程；具体任务恢复机制后续再设计。

记忆写入与召回

从任务过程中及结果中筛选、校验有价值的项目知识或有效经验，再写入长期记忆。后续任务按权限和相关性召回，供上层整合上下文。

会话记录保存“发生了什么”，长期记忆保留“哪些知识值得以后复用”。 不将全部历史消息自动当作记忆，也不将模型未经验证的猜测直接作为事实。

8.3.1.1.3 Pi 核心层设计

配套视频：第二期：二次开发入门（包含本节接入方案与架构图，视频链接待补充）。

前置阅读：Pi Agent 二次开发入门 → 三种接入方式。

笔记里的ForgePilot项目，设计上就是使用pi-agent进行二次开发，这章以该项目为例，展示该项目接入Pi-agent的技术方案。

ForgePilot 选择从 **`pi-agent-core` 所在的 Agent 核心层接入**，复用模型与工具之间的执行循环，并配合 `pi-ai` 完成模型通信。在此基础上，**任务编排、上下文管理、权限控制和交付验收**由我们自己的代码实现。

图注：ForgePilot Agent 核心层架构：蓝色为 pi-agent-core 与 pi-ai 提供的能力，橙色为 ForgePilot 自主实现的模块，灰色为插件框架与外部服务。

图中的颜色区分了三类职责：

- **蓝色是 Pi 提供的能力。** `pi-agent-core` 负责组织模型与工具之间的执行循环，包括处理模型回复、调度已注册工具、回填结果并继续执行；`pi-ai` 负责统一模型调用。具体的检索、文件读写和测试能力由我们接入，并不是 Core 自带的业务功能。
- **橙色是 ForgePilot 自己实现的部分。** 执行前整理任务、校验权限并准备上下文；执行中管理预算、审批、任务状态和历史压缩，封装受控工具，保存会话并管理长期记忆；执行后结合测试证据和人工审核决定是否交付。Pi 结束一次运行，不等于任务已经通过验收。
- **灰色是复用的框架与外部服务。** Cordis 负责装配各项能力，主模型负责推理，企业检索、工作区和测试平台提供实际执行环境。这些模块既不属于 Pi 核心，也不能全部算作我们从零实现。

选型理由

我们从 ForgePilot 项目的目标出发理解：我们要做的不是一个通用的代码聊天工具，而是让 Agent 在企业环境中理解工单，并能长时间、稳定地完成分析、修复、测试和验证，在必要时继续返修，**最终提交可审核的 PR**。

围绕这一目标，有几类能力需要由我们掌握和定制，也就是上图中的橙色部分：

- **影响任务成功率的执行策略。** 记忆如何管理、历史如何压缩、任务进度如何保存、允许使用哪些工具，都会影响 Agent 能否持续理解任务、保留关键证据，并在失败后继续推进。因此，这些能力不能只是照搬通用 Agent 的默认策略，而要围绕企业工单和研发流程设计。
- **企业自己的管理规则。** 身份与权限、操作审批、预算和运行时限，都需要遵循企业已有的逻辑。这些规则应由产品系统落实，而不是交给模型自行判断。
- **面向交付的界面与交互。** 除了对话和任务执行，我们还需要星图评审模式等能力，帮助开发者理解变更、查看验证证据并完成审核。因此，界面也需要围绕自己的业务重新设计。

另一方面，**底层通用的运行机制并不是本方案需要重新实现的重点。**例如，向大模型发送请求、维护多轮消息、识别模型回复中的工具调用、调度工具并回填结果，再继续下一轮推理，这些模型与工具之间的循环机制可以直接复用。其中，`pi-ai` 负责模型通信，`pi-agent-core` 负责 Agent 执行循环，也就是这里所说的 ReAct 式循环。

从这个维度看，选择 Core 层的理由就很清楚了：**再往上复用，会带入更多需要重新定制的产品机制；再往下只用模型接口，又需要自己补上通用执行循环。** 从 `pi-agent-core` 接入，既保留了成熟的运行能力，也把影响企业任务效果和交付方式的策略留在自己手中。这里的定制主要发生在 Core 周围，并不意味着必须修改 Core 源码。

这张架构图也从更具体的技术层面，回应了我们在“自建 Agent 的意义”章节中讨论的问题：**基于 Claude Code 二次开发，不代表业务开发工作就会自动减少；自己开发一个 Agent 平台，也不代表所有能力都要重新实现。**

即使选择 Claude Code，上图橙色部分所承载的企业需求仍然存在。工单理解、企业权限、任务进度、记忆策略、验证流程和星图评审等能力，依然需要我们设计并接入。区别在于，这些工作可能通过 Hooks、工具集成或外围服务来完成，而不是直接在自己掌握的执行管理层中组织。现成能力可以减少部分实现，但当定制需求超出开放接口的范围时，也可能需要额外的适配和绕行，灵活性受到限制，整体工作量未必更少。

反过来，ForgePilot 虽然采用自建平台的方式，也没有从零重写整个 Agent。图中的蓝色部分就是必要的复用：模型通信交给 `pi-ai`，通用执行循环交给 `pi-agent-core`，我们**把开发重点放在企业任务真正需要差异化的部分。**

相关面试回答：你的项目是如何接入pi-agent的?

8.3.1.2 ForgePilot上线问题

> **导读：**如果我们的项目上线了，要多考虑很多问题，比如**发布，版本迭代，管理，线上问题调试，如何线上监控**等等。其实这些内容不是我们面试Agent岗位的重点，但是如果我们说了项目上线了，这些内容内容我们要想清楚，多少面试官可能会和你聊到相关内容。我们必须对上线有一个基本的意识和概念，并且提前准备最基本的相关问题。以下**8个方面，就是面试官很可能围绕上线问题和你展开讨论的一些地方。**
>
> 同时，面试前突击章节，我们对三个项目都准备了上线系统的设计，当前的ForgePilot项目，智能发布助手（RAG项目），智能预约Agent. 通过这三个项目，我们也可以看出我们的项目上线的形式，规模，用户类型的不同，也直接影响着我们上线方案的设计。

构建、打包与发布

1. 构建与打包流程

ForgePilot Electron 构建与打包流程

整体流程：前端与后端代码 → Electron 桌面产品 → CI/CD 流水线构建与测试 → electron-builder 打包 → Windows EXE 安装包。

- **前端界面（React + TypeScript）**：实现对话、任务、评审等界面，以及用户交互。构建后生成界面文件。
- **本地后端（Node.js + TypeScript）**：处理任务编排、工具调用、仓库操作等业务逻辑，在用户电脑上运行。
- **Electron 桌面产品**：提供桌面窗口和运行环境，将前端界面与本地后端集成为桌面应用。
- **CI/CD 流水线**：使用 GitHub Actions，或公司内部 Git 平台的 CI/CD 能力，自动执行代码构建、测试及打包命令。

- **electron-builder**：将构建后的界面、后端、Electron 运行环境及所需依赖组织为可安装的应用，生成 Windows EXE 安装包。
- **Windows 安装包**：用户下载并安装 ForgePilot，获得本地运行的应用；后续新版也通过安装包提供。

---

2. 发布与下载

流水线生成安装包后，将其上传至 **GitHub Releases、公司内部制品平台或对象存储**，获得下载链接。CI/CD 负责执行上传流程，制品平台负责保存和提供安装包。

用户可以通过两种入口获取软件，两种方式可以指向同一个安装包：

- **企业内部邮件通知**：说明版本号、更新内容，并附上安装包下载链接。
- **官网或内部下载页面**：展示版本号、更新说明和下载按钮。

发布流程：安装包上传至制品平台 → 邮件通知或下载页面提供链接 → 用户下载 → 安装或升级。

---

3. 安装后的使用方式

同一个安装包包含前端界面、本地后端和 Electron 运行环境，可以提供两种使用入口：

- **桌面入口**：启动 ForgePilot，通过 Electron 桌面窗口使用。
- **本地浏览器入口**：启动本地 Node 服务后，用已有的 Edge、Chrome 等浏览器访问本地地址，例如 `http://localhost:7890`，加载同一套前端界面。

浏览器入口需要在产品中实现，并对依赖 Electron 的界面功能进行适配；不需要再下发单独的浏览器安装包。

这里的 **安装包下载链接** 用于获取软件，**本地界面地址** 用于安装并启动后的日常使用，两者用途不同。

---

4. 更新方式

可以提供两种更新方式：

- **用户主动更新**：通过邮件或下载页面通知新版本，用户主动下载新的 EXE 安装包，安装并替换旧版应用。仅发布新链接不会自动升级已安装的软件。
- **应用内自动更新**：接入 Electron 应用的更新能力，可使用与 electron-builder 配套的 **electron-updater**。流水线发布新版安装包及更新元数据后，应用在启动时或定期检查更新，自动下载新版，并在合适的时机提示用户重启安装，无需用户再到网站手动下载。

---

5. 版本管理

> **导读：**既然我们说我们的项目是上线的，怎么进行版本管理，迭代的，大概的思路和意识要有，万一聊到相关问题，不至于完全答不上来。
>
> 采用 **Git 分支分工 + 应用版本号 + Git Tag + 发布记录** 管理版本，组织开发与发布流程，并建立代码提交与安装包之间的对应关系。

分支分工与合并流程

以已发布稳定版 `1.0.0` 为起点，本例中 dev 与 main 的代码一致：

- **feature 功能分支**：开发者从 dev 拉出，例如 `feature/日志采集`，在这里开发和提交代码。功能完成后提交 PR，审核通过后合回 dev，未完成的改动留在各自的功能分支。
- **dev 开发集成分支**：汇集已完成的功能，进行集成测试，检查多个功能是否互相影响。准备发布时，确定本次发布的代码范围。
- **release 发布分支**：发布负责人从 dev 的确定提交拉出，例如 `release/1.1.0`。测试人员验证，开发者修复问题，不加入下一版新功能。验证通过后，在 release 的确认提交上打 Tag，由流水线构建安装包并发布。
- **main 正式发布主线**：保留已正式发布的代码，不用于日常功能开发。发布后，release 通过 PR 合入 main；release 上的修复也同步回 dev，避免下一版丢失修复。

整体路线：dev → 拉出 feature 开发 → PR 合回 dev → 拉出 release 验证修复 → 打 Tag 构建发布 → PR 合入 main。

---

6. 灰度发布

正式项目中通常采用 **灰度发布**：安装包准备好后，不立即向全量用户推广，而是先让部分用户安装使用，观察运行情况和用户反馈。发现问题时，暂停扩大范围，及时修复或回滚；确认小范围使用没有明显问题后，再逐步向更多用户发布，最终完成全量发布。

灰度发布有多种实现方式。例如，通过在线更新服务和配置，自动控制哪些用户可以收到新版，以及每批下发的比例。这类方式适合用户规模较大、需要精细控制发布范围的场景。

我们的ForgePilot 定位于 **企业内部使用，用户规模约为几百名开发者**，因此采用更简单的 **分批邮件通知** 方式进行灰度，不引入复杂的在线比例控制机制：

- **先通知部分用户**：选择少量开发者或试用团队，发送包含版本说明、安装包下载链接和反馈方式的邮件，由用户下载安装新版。
- **观察反馈并处理问题**：收集使用反馈，检查核心功能和稳定性。出现问题时暂停后续通知，及时修复或按回滚流程恢复，验证通过后再继续推广。

- **逐步扩大通知范围**：小范围验证通过后，分批通知更多团队和开发者，持续观察反馈，最后向全体目标用户发送更新通知。

整体流程：安装包准备好 → 小范围邮件通知 → 观察反馈 → 逐步扩大范围 → 全量通知。

---

7. 回滚与版本恢复

新版发布后发现问题，先停止继续推广，避免影响更多用户。如果问题严重、需要尽快恢复使用，则让受影响的用户先回退到旧的稳定版本；待问题修复并验证通过后，再提供新版本。

- **先停止发布**：暂停后续邮件通知，撤下异常版本的下载推荐；如果已开放自动更新，也暂停该版本的更新推送，并通知已安装用户。
- **紧急时回退旧版**：提供上一稳定版本的安装包和操作说明，让受影响的用户按说明重新安装。回退前确认旧版与本地数据、配置兼容，必要时先备份；暂停发布本身不会让已安装的软件自动回退。
- **修复后发布新版**：开发者修复问题，测试通过后使用新的版本号重新打包，例如将有问题的 `1.1.0` 修复为 `1.1.1`。先通知小范围用户验证，确认正常后，再逐步向其他用户发布。

整体思路：停止发布 → 必要时回退旧的稳定版 → 修复并验证 → 发布新版。

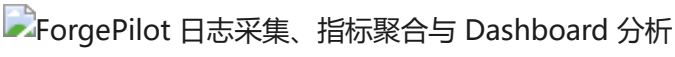
---

8. 日志采集与分析

> **导读：**这个做法是最典型的上线后分析问题的做法，埋点，上传，聚合，分析。 我们这里先清楚这种行业标准的做法，至于埋哪些点，聚合了什么数据，如何分析解决的，这些具体的细节，我们应该在该项目的技术文档写好。但是在面试突击章节，我们先掌握原理就好了，在面试问题预测里面，相关的面试问题，结合该理论，构造一个细节且具体的答案就可以了。
>
> ForgePilot 在用户电脑上运行，通过 **关键环节埋点 → 日志上报 → 远端集中存储 → Dashboard 分析**，了解产品使用情况、定位问题，并辅助判断新版是否稳定。

- **关键环节埋点**：在任务开始、阶段执行、工具调用、任务结束及异常处理等关键位置记录结构化日志，便于关联一次任务的执行过程。
- **日志上报**：客户端将日志脱敏后，通过 HTTPS 批量上报到远端日志接收服务。网络异常时在本地限量缓存，恢复后重试，不影响正常任务执行。
- **远端聚合**：日志接收服务统一接收和校验各客户端上报的日志，集中存储到日志平台，供后续查询、统计和分析。
- **Dashboard 分析**：围绕使用情况、任务成功率、执行耗时、异常趋势和不同版本表现建立看板。灰度期间观察新版是否出现异常增长；用户反馈问题时，按任务关联信息查询日志，定位失败阶段，再下钻查看具体记录。

整体流程：客户端埋点 → 批量上报 → 远端日志平台 → 看板观察与日志查询 → 定位问题并改进。日志查询仅向授权人员开放。

 ForgePilot 日志采集、指标聚合与 Dashboard 分析

---

9. 鉴权系统

这个项目我没有设计专门的鉴权系统，也就是说整个产品对于所有人来说的都是可以用的，你可以直接下载客户端，任何人都能使用。

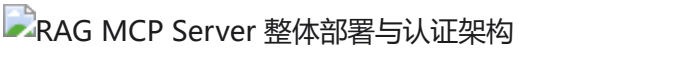
但是当系统运作起来，比如修复bug，要先去查找某个bug单，某个代码仓库，调用模型的时候（这些东西我会以工具的形式给到系统）。不管是没有登录，还是登录了没有权限，模型调用对应工具后，拿到结果后，都会提示给用户（让用户登录或者没有权限的提示）。

因为本身公司的代码，bug单都是和你的企业身份相绑定的，这样直接复用了企业内部的权限系统，不用自己管理，这也符合企业中真实使用的场景。

8.3.2 智能发布助手（RAG项目改造）

这个项目是简历里写的第二个项目，也就是我们RAG项目的改造包装版。

8.3.2.1 上线方案

 RAG MCP Server 整体部署与认证架构

> 本文描述团队内部使用的上线方案。远端 Streamable HTTP 与企业认证按以下方案实现并验证，不将方案描述为已经完成的上线成果。

整体运行原理

本系统以远端 MCP Server 的形式部署在团队共用的 VM 上。用户使用 Cursor、VS Code + GitHub Copilot 或 Claude Code 等支持远端 MCP OAuth 的客户端，配置服务地址并完成公司账号登录，即可调用知识查询工具

客户端与服务端通过 **Streamable HTTP（基于 HTTPS）** 交互。文档获取、解析、分块、向量化、索引维护和检索等工作全部在远端完成，客户端无需安装 RAG 模型，也无需处理文档或维护本地知识库。服务端返回检索证据和来源，客户端中的模型再根据这些内容组织答案。

MCP Server 内包含认证与权限模块，负责认证发现、企业登录衔接、Token 验证及文档访问控制。图中的认证模块与 RAG 模块是逻辑划分，不要求分别部署在不同服务器上。

整体流程：

```Plain Text 配置远端 MCP 地址 → 首次连接时发现认证要求 → 使用公司账号登录并获取 Access Token → 通过 Streamable HTTP 携带 Token 调用服务 → 服务端验证身份与文档权限 → 检索并返回获授权的证据 → 客户端整理为回答

> ---
>
> **1. 如何上线**
>
> **1.1. 服务端部署**
>
> 首版使用一台企业内网 VM，或通过企业网络访问的 Azure VM，集中运行 MCP Server、RAG 服务与后台文档处理任务。
>
> 上线时需要完成：
>
> 1. 在 VM 上部署程序、依赖、模型和知识索引，并配置持久化存储。
> 2. 将现有 MCP 接口提供为 **Streamable HTTP** 服务，配置 HTTPS 与稳定的服务地址。
> 3. 完成认证与权限配置，不向未认证用户开放知识查询。
> 4. 将程序作为后台服务运行，例如使用 systemd 或 Docker，支持开机自启与异常退出后重启。
> 5. 自测连接、登录、查询、来源返回及无权限场景，确认正常后向团队开放。
>
> 示例服务地址：
>
> ```Plain Text
> https://knowledge.company.com/mcp
```

该地址为示例，实际使用时替换为团队部署的地址。单台 VM 适合内部试点，但机器故障时仍可能造成短暂中断。

1.2. 向用户发布：一份内部 Wiki 即可

用户侧不需要下载安装我们开发的 RAG 程序或专用插件，提供一份内部 Wiki 接入文档即可。 Wiki 是发布接入说明的载体，不代替服务端部署。

Wiki 包含以下内容：

- 服务用途及允许接入的资料范围。
- 服务地址与网络访问要求，例如是否需要 VPN。
- 已验证的客户端及版本。
- 各客户端的配置方法与必要的 OAuth 参数。
- 公司账号登录方法。
- 示例问题与问题反馈渠道。

以 VS Code 为例，连接配置可以写为：

```
{
  "servers": {
```

```
    "windows-knowledge": {
      "type": "http",
      "url": "https://knowledge.company.com/mcp"
    }
  }
}
```

这段配置只展示服务连接部分。实际接入时，按已验证的认证方案补充所需的 Client ID 等配置；Cursor、Claude Code 使用各自的配置格式。用户 Access Token 和共享密钥不写入 Wiki 或仓库配置。

---

2. 如何鉴权

2.1. 标准授权方案与企业身份系统

MCP OAuth 是 MCP 定义的、基于 OAuth 的标准授权方案，不绑定某一家企业的账号体系。 它约定客户端如何找到登录入口、获取 Access Token，并携带 Token 访问 MCP 服务。使用支持该方案的客户端，我们无需修改客户端代码。

Microsoft Entra ID 是本方案选择的企业身份系统，而不是 MCP 的必选项。 选择它是为了对接现有的微软企业账号与组织体系。其他公司也可以使用自己的统一身份系统，通过服务端认证模块完成适配，整体接入方式不变。更换身份系统后，需要验证与目标客户端的兼容性。

用户使用流程如下：

```Plain Text
客户端连接 MCP Server → 服务端告知需要登录，并提供认证入口信息 → 客户端引导用户使用公司账号登录 → 登录成功后获取用于访问 MCP 服务的 Access Token → 客户端携带 Token，通过 Streamable HTTP 发起查询 → 服务端验证身份和权限后返回结果
```

登录授权通过 `HTTPS` 完成，知识查询通过 `Streamable HTTP`（基于 `HTTPS`）完成。客户端负责管理登录凭证，不需要用户每次查询都手动登录。

2.2. 文档访问权限

身份系统确认“用户是谁”，服务端结合文档权限规则判断“用户能看什么”。 登录成功不代表可以访问全部知识库

例如，发布团队成员可以查询发布知识集合，组件 `A` 团队成员可以查询组件 `A` 的资料。服务端根据用户身份、所属团队与资料访问规则，只在用户有权访问的范围内检索，并对摘要等读取操作执行同样的检查。

凭证无效时提示重新登录；没有相应权限时拒绝访问，不透露无权文档的内容。后台服务能够读取资料，也不能将这些资料开放给所有用户。接入的资料还应符合企业对客户端模型使用内部数据的规定。

---

3. 日志系统

> **RAG 项目已有详细的日志和 Trace 机制，上线后沿用现有日志系统，不另外重做一套** 。我们设计项目的时候，专门有设计健全的日志系统。
>
> 这里面的日志系统可以复用。唯一要注意的是，当项目被多人使用时，会通过不同用户产生不同的请求，我们分析的时候要根据不同的用户来筛选他的请求整个链路的日志，然后做分析。
>
> ---
>
> **4. 版本管理**
>
> 这是一个团队内部使用的系统，**不需要复杂的正式版本发布流程**，无需复杂审批、发布分支或严格版本编号，采用轻量维护方式即可：
>
> ```Plain Text
> 修改代码 → 自测通过 → 更新 VM 上的服务 → 团队使用
>                                              ↓
>                                    有问题 → 反馈 → 修复
> ```

所有客户端连接同一个 VM 上的服务。完成代码部署，并重启或重新加载服务后，后续请求就会使用新代码；如果修改涉及索引结构或模型，也要同步完成相应索引更新。

只要服务地址、工具接口和认证方式保持兼容，用户通常无需修改客户端配置，也不用下载新的安装包，即可使用更新后的服务。

8.3.3 智能预约Agent

8.3.3.1 打包发布与电话语音接入方案

智能预约系统，我们在简历上只写了一句话："**个人项目：制作智能预约系统，接入北京线下按摩门店**"，来体现我们个人是对技术有追求的，是喜欢自己做项目并取得一定成果的。

因为本身简历篇幅不多，所以我没有太多准备这一方面内容。这里只准备两个我们没做的东西，如何打包-如何接入电话系统（真实场景下用户是拨打电话，和机器人预约，预约好了写入系统的）

一、打包上线

本项目采用本地安装部署，不需要将整个系统部署到云端。根据项目使用的技术栈，使用相应的打包工具整理前端页面、后端程序及必要的运行依赖，再通过 Inno Setup、NSIS 等安装包工具制作 Windows 安装包。

用户安装后，通过桌面快捷方式启动本地服务，并在浏览器中打开系统。安装包应包含必要的初始化配置；数据和日志单独保存，避免后续升级覆盖。

整体流程：整理代码与依赖 → 打包应用 → 制作 Windows 安装包 → 在目标电脑测试安装、启动和预约功能 → 分发使用。

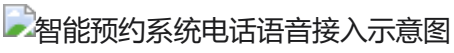
二、接入电话语音

保留现有预约系统，新增电话语音接入服务，连接腾讯云的云呼叫中心、语音识别和语音合成能力。

- **电话接入：** 使用腾讯云云呼叫中心承接用户来电并管理通话，具体音频对接能力需要提前确认。
- **STT（语音转文字）：** 使用腾讯云语音识别，将用户在电话中说的话实时转换成文字。
- **预约处理：** 由现有智能预约系统接收文字，理解用户的预约需求，查询或执行预约，并生成文字回复。
- **TTS（文字转语音）：** 使用腾讯云语音合成，将系统的文字回复转换成声音，再交由电话平台播放给用户。

整体流程：用户来电 → 电话平台接听 → STT 将声音转成文字 → 现有系统处理并回复 → TTS 将回复转成声音 → 通过电话播放给用户。

接入示意图


智能预约系统电话语音接入示意图

8.3.4 面试真题预测

在面试前，其实我们就可以根据我们的简历，预测很多的面试问题了。因为很多面试就是围绕你简历去问的。

我分为几类问题，大家也可以按这个思路来自己预测：

1. **必考问题**：你的项目是什么，简单介绍一下。（能想到必考的只有这一个）
2. **高频问题**: 任何你写在简历上的每一句话相关的内容，你说出去的（比如在介绍项目时）提到的每一个技术栈，都可是高频考题。

包括一些通用问题：你的项目中遇到了什么最难的问题，如何解决，也属于高频。

3. **重点内容**：比如Agent项目的，记忆怎么设置，上下文如何压缩，等等，这些属于重点内容。

8.3.4.1 ForgePilot

8.3.4.1.1 必考问题

8.3.4.1.1.1 简单介绍你的 forgecopilot 项目

💡 **答题思路：**

从**痛点-功能-架构-结果**四个方面概述。

🗣️ **参考回答：**

ForgePilot 是我设计的一个**面向企业研发场景的 AI 研发平台**，不是重新做一个编码助手，而是复用成熟 Agent 的能力，补齐企业上下文和交付流程，让 AI 从“能改代码”走向“能交付可审核的修复”。

这个项目主要解决四个痛点：**第一，上下文分散**，企业代码跨仓库、跨分支，Bug 信息也散落在工单描述、评论和附件中，通用 Agent 很难拿到完整信息；**第二，工程流程复杂**，公司有特定的构建和测试方法，

Agent 经常需要工程师带着走；**第三，人工评审跟不上**，AI 生成代码很快，但人理解和审核修改仍然耗时；**第四，远端模型的调用成本和数据外发风险**，因此我们规划接入公司自研本地模型，优先承担工单理解、信息提取和任务分类，减少这部分任务对远端模型的依赖，复杂的代码分析再交给主模型，并对外发数据进行管控。

围绕这些问题，平台设计了**代码问答、Bug 修复和辅助评审**三个核心功能。代码问答帮助开发者理解项目、定位逻辑；Bug 修复从 TAPD 工单出发，串起任务整理、代码定位、修改和测试，经人工审核后受控创建 PR；辅助评审则把变更意图、关键 diff、影响范围和验证证据组织起来，帮助人更快理解和验收修改。

架构上，前端采用 React 和 TypeScript，后端使用 Node.js，核心分成**任务编排和插件化执行两层。**上层由本地模型整理需求，Harness，也就是流程控制代码，负责校验、权限、路由和任务状态管理；下层通过 Cordis 装配模型、检索和执行工具，默认由 Pi 驱动 Agent 循环，完成定位、修改和验证。同时，会话记录保存执行过程，经过验证的项目经验沉淀为长期记忆，供后续任务复用。

整个设计的核心是：**确定的流程交给程序，需要探索的步骤交给 Agent**，在保留 AI 灵活性的同时，让企业研发交付更可控、可追踪。成果方面，前期平台已经被多个团队使用，在真实 Bug 任务集的对照评测中，端到端交付成功率相比ClaudeCode提升了 **32%**，也为 ForgePilot 的进一步设计提供了实践依据。

8.3.4.1.2 高频问题

8.3.4.1.2.1 我看你简历写了“SWE-bench 修复 Agent 评估”，这个是如何做的，可以自己讲一讲吗?

💡 **答题思路：**既是简历上写了，面试官可能问的内容，也其实是面试官直接问你：你的项目是如何做Agent评估的问题回答。思路是参考笔记Agent评估方法，进行设计。

🗣️ **参考回答：**

评测思路是借鉴 SWE-bench，用企业内部历史 Bug 出题，让 Agent 修复，再用测试验收。具体分三步。

第一，准备数据集。 从历史 Bug 中选取工单理解、跨模块定位、构建测试等典型任务。每条整理成“工单描述和日志 + 修复前代码版本 + 环境配置 + 验收测试”。验收测试包括能复现原缺陷的测试和相关回归测试，历史修复答案不提供给 Agent。数据分成调试集和最终测试集。

第二，批量跑对照。 让通用 Agent 基线和定制平台跑同一批任务，固定模型版本、代码起点、环境和预算，使用相同验收标准。每条任务保存补丁、测试结果、工具调用记录、耗时和费用。

第三，汇总结果，从三个方面比较。

- **结果上**：检查补丁能否应用、构建是否通过、缺陷验证和相关回归测试是否通过。按统一成功条件给每条任务记通过或不通过，汇总成**任务解决率**，再按任务类型看哪类表现更好。
- **过程上**：对照工单和执行记录，将信息遗漏、代码版本定位错误、重复无效搜索、未验证就宣布完成等问题分类，统计各类问题的出现比例，了解任务主要卡在哪里。
- **成本上**：汇总平均单任务耗时、模型费用、工具调用次数和人工接管比例，比较两套方案的投入。

最终看的是一份对照汇总：**解决了多少任务、主要出错在哪、平均花多少资源。** 任务解决率单独计算，过程和成本不加权到这个指标里；32% 比较的就是两组任务解决率的相对变化。

8.3.4.1.2.1.1 追问：你提到在 180 条任务上，成功率从 31.11% 提升到 73.33%，相对提升约 135.71%，是怎么测出来的？什么算成功？

> 你看到讲解视频的时候，这个问题和参考答案可能改了，不过思路是一样的，以笔记为准哈，你看到的笔记永远是最新的。

💡 **答题思路：**面试官有非常大的可能性会追问我们简历上的数字的细节，数据是怎么统计的，计算方法是什么，相对提升还是绝对提升？这是我们面试前需要自己弄清楚的。 我们这个问题的回答的关键，是想清楚这个成功率怎么判断：按照什么标准来统计任务是否成功？ 这个是下面的答案的第一小段讲的。

讲清楚如何判定成功，后面就是解释数据了，我们测了多少数据集，测试的Baseline是什么？

🗣️ **参考回答：**

我们主要从两个维度判断任务是否修复成功：**一是能不能自主完成，二是修复结果是否符合验收要求。**

首先是自主完成。从代码定位、修改，到编译构建、运行测试，以及失败后的返修，都需要 Agent 在规定预算内自行完成。过程中不能由人工帮它改代码，也不能人工告诉它根因和具体修法，但允许它根据构建、测试的反馈自行调整和重试。任务结束后人工审核结果，不算帮助它完成修复。

其次是代码正确性。我们参考了 SWE-bench 的做法，为每条任务准备对应的单元测试、必要的端到端测试和相关回归测试。核心是验证两件事：**原来能够复现 Bug 的测试，修复后要通过；原本正常的功能，修复后不能被破坏。** 这里不是模型说自己修好了就算成功，而是必须实际运行测试，由评测程序检查执行结果。

除了功能测试，**每条任务还会整理相关的架构约束和关键设计要求**，比如不能破坏公共接口兼容性、不能绕过权限校验、不能违反模块依赖关系。能够自动检查的通过规则和测试验证，其余通过代码审核判断。即使测试通过了，只要明确违反这些硬性约束，我们也不认为修复成功。我们不要求它的代码和历史修复 PR 一模一样，但要求它修得对，而且符合项目的工程要求。

只有自主完成和结果验收这两个维度都通过，我们才把这条任务记为端到端修复成功。

比较时，让 Claude Code 和我们的框架在同一批任务上，使用相同模型、代码起点、预算和验收标准分别运行。在包含 180 条 Bug 任务的评测集中，Claude Code 按照上述标准通过了 56 条，成功率约为 **31.11%**，主要问题集中在检索不到所需代码或上下文、生成的代码违反架构约束，以及修复结果无法通过测试这三个方面。使用我们的平台后，同一批 180 条任务中通过了 132 条，成功率约为 **73.33%**，比基线多通过了 76 条。因此，成功率**相对提升约 135.71%**，计算方式是（132 − 56）÷ 56；成功率**绝对提高约 42.22 个百分点**，计算方式是（132 − 56）÷ 180 × 100 个百分点。

8.3.4.1.2.1.2 追问：这个项目有几个研发？你具体负责什么？哪些能力是你做的，哪些来自开源？

💡 **答题思路：** 很合理的追问，一开始我给这个项目的定位就是企业级的项目。功能也比较复杂，既有本地模型嵌入，又有测评，又有鉴权，又有不同的UI模式，插件化设计，以及要上线的。**这些不可能一个人完成！**

所以你必须明确你负责那一块，这样其实也是为我们减轻压力，如果有哪个地方实在答不上来，可以找理由说自己没负责这一块！

说自己负责哪一块：就根据你哪一块熟练，哪一块有价值（比如jd需要，或者面试官关注来定）。这个应该和你写在简历的内容相对应。结合我自己的例子来说，我会说我负责**本地模型适配+Agent核心编排**（记忆，压缩，评测）这两块。**前者是我们做过的项目，要体现我们的后训练，部署能力。 后者是Agent最核心的能力，我们也比较熟练。**

🗣️ 参考回答：

这个项目是团队协作的企业级研发平台，研发团队一共 6**人**，不是我一个人从前端到模型全部做完的。平台涉及工作台、企业系统接入、代码检索、Agent 执行和评审等多个模块。我主要负责的是两块：**Agent 核心 Harness 的设计，以及本地模型的适配工作**，也是我简历重点写的部分。

第一块是 **Harness**，也就是围绕 Agent 执行内核的流程控制和上下文管理。我重点关注三个方面：**记忆、压缩和编排。**记忆方面，区分当前任务的状态与执行上下文，以及后续任务可以复用的、经过验证的项目经验；压缩方面，处理长任务中对话和工具结果不断累积的问题，重点考虑怎样保留任务目标、关键证据和未完成事项，而不是简单截断历史；编排方面，把工单整理、信息校验、Agent 执行、验证反馈和人工审核串起来，明确哪些步骤由程序控制、哪些交给 Agent 探索，以及失败或预算耗尽时如何停止、转交处理。这里不是重新实现一套 Pi，而是设计企业任务如何在已有执行内核上受控运行。

第二块是 **本地模型适配。**这部分在当前方案中仍是规划方向，主要由我负责适配方案：优先让本地模型承担工单理解、信息提取和任务分类，并设计它与 Harness 之间的输入输出约定、结构化结果校验和异常回退机制。复杂代码分析再交给主模型。我的职责重点是让模型适合这条业务链路，而不是把公司基础模型的训练也算成自己的工作；是否进入微调，要根据任务评测结果再决定。

开源复用方面，**Pi 提供 Agent 循环，Cordis 提供插件注册、依赖注入和生命周期管理**；我的工作重点是这些组件之上的企业任务控制、上下文策略和模型适配。工作台、企业检索底层、统一认证和部署等不作为我的主要负责范围，具体由谁负责按团队实际分工说明。我需要了解它们和 Agent 的协作接口，但不会把其他模块的实现都算在自己身上。

所以，概括来说，**团队交付的是企业级研发平台，我主要负责的是让 Agent 带着合适的上下文、按受控流程执行任务，以及本地模型在这条链路中的适配。**

8.3.4.1.2.1.3 追问：你提到了现在180个数据集上仍然有快50个数据集没通过，你觉得下一步应该如何提升?

💡 **答题思路：**这一个题考察对于项目评测结构的展望，如何进一步提升，下一步是什么？也是我们应该提前准备到的。 （202508 阿里淘天秋招一面面试官就问候选同学，你简历写的这个指标，该如何进一步提升？）

通过这个题，可以锻炼我们思维发散的能力，针对具体问题，如何解决，也是很多时候面试官喜欢和你去讨论问题的。我们可以通过这样一些问题，锻炼我们对于这些问题的变通，扩展的能力。

🗣️ 参考回答：

目前我们的系统下，系统出现的错误主要在两个方向：

1.跨模块修复能力不足：Agent很容易把当前bug修复好了，但是破坏了其他功能，在评估时因其他功能发生回退、出现错误而卡住。我认为这里的优化方式是，修改检索模块和Harness，不光检索相关要修改的代码，也检索它的代码调用层级关系，同时加强架构约束模块。让Agent在看清楚全局调用链和架构约束的视角下，针对性修改问题。

2.Agent很容易陷入死循环，在错误方向上不断深入钻牛角尖：很多时候，Agent一开始分析出来的修复方向就不对，在错误的方向上，在ReAct框架上反复重试，浪费时间且无法最终修复。 对此，我考虑在 Harness 中增加无进展检测，当同类错误反复出现时，触发独立的反思 Agent，根据修改记录和测试反馈重新检查修复方向，再让 Pi补充调查或换一个方案，而不是机械重试。

8.3.4.1.2.1.4 追问： 引入这套评估体系前后，整个产品链路最明显的变化是什么?

💡 **答题思路：**

灵感来源于202508 阿里淘天的面试官 的提问。

这样的问题不是特别好提前预测准备好，不过如果我们知识比较扎实，现场懂得变通，这个题目也不会说答不出来，这个问题我们回答的出发点就可以从测试驱动开发来说。引入了评估系统后，每个环节哪里出现了问题，我们都能够实时辨别，针对性修改，同时修改了新内容也可以及时监控是否会引发回归。测试驱动开发的思想，我们在Agent评估章节就讲过，理解了这一小节的思路，这个题也可以回答了。

🗣️ **参考回答：**

我觉得最明显的变化，是整个产品从主要依赖个别案例和人工反馈调试，转向了**有明确验收标准的、评估驱动的迭代方式**。这个思路借鉴了测试驱动开发：先明确什么算完成，再根据失败样本针对性修改，最后通过回归验证改动是否有效。

ForgePilot 的链路比较长，包括工单理解、上下文准备、代码检索、Agent 修改，以及后面的构建测试。最终修复失败，不一定是模型不会写代码，也可能是前面漏了信息、拿错了上下文，或者执行过程中丢失了约束。所以评估不仅要告诉我们成功了多少条，还要结合各阶段的输入输出和执行记录，帮助我们定位失败发生在哪里。

（比如生成的代码违反了“不能修改公共接口”这个要求，我们就需要检查：是本地模型解析工单时没有提取出来，还是 Harness 没有把约束传递给 Pi 执行层，或者长任务压缩时把这条约束丢了。如果约束一直都在，那才进一步看模型执行和验收机制。每个环节都有明确记录和追踪，帮助我们把同一个失败结果可以对应到不同的改进动作，测试驱动开发。）

另外一个变化是，**每次修改都需要验证有没有引入回归。**我们的执行能力通过 Cordis 插件化装配，检索策略、模型和上下文策略都可以调整。但换了检索器，不能只看搜到了更多代码；改了压缩策略，也不能只看 Token 降低了。还需要跑相关回归任务，检查最终修复成功率、架构约束遵守情况，以及耗时和成本有没有退化。

落到交付环节，完成的标准也会更加明确：不是生成补丁或者模型宣布完成就结束，而是需要有构建、测试和约束验收的证据。能够自动判定的失败，可以反馈给 Agent 在预算内返修；不能自动处理的，再交给人工审

核或接管。

⚠：我还能想到一些面试官会追问的内容：比如为什么使用Pi-Agent？ 怎么用的Pi-Agent？插件化如何实现的？这些来不及准备了，自己可以多想一想。

8.3.4.1.2.2 已经有 Claude Code 等通用 Coding 平台，为什么你们公司还需要搭建自己的平台?

💡 **答题思路：**这个问题，我觉得是大部分面试官看到我们项目，第一个自然的反应。不过我在立项的时候，就想清楚了这个事情的答案，找了做这个事情的真实的痛点。

🗣️ **参考回答**

我们不是要重新做一个 Claude Code，而是**复用通用 Agent 的编码能力，补齐企业场景中的上下文、交付流程和安全要求**。主要有六个原因。

第一，大规模代码检索。 团队代码资产超过 300GB，涉及多个仓库和分支，仅靠当前本地工作区搜索可能漏掉相关代码或找到错误版本，需要接入企业检索服务。

第二，企业任务理解。 TAPD 缺陷单的信息散在描述、评论、附件和关联单中，需要先整理成完整任务，避免 Agent 根据不完整的信息走错方向。

第三，减少流程试错。 公司有特定的构建、排障和测试方法，我们把确定的流程固化下来，不让 Agent 每次重新摸索，也减少工程师带路的时间。

第四，提高交付成功率。 写完代码不等于修复完成，我们关注构建、测试、失败反馈和返修的完整闭环。真实任务数据集上的评测结果显示，定制平台相比通用 Claude Code 流程，端到端交付成功率提升了 32%，这也是定制工程流程价值的一个依据。

第五，降低人工 Review 成本。 AI 写得快，人不一定审得过来。平台将变更意图、关键 diff、影响范围和验证证据组织起来，帮助 Reviewer 快速理解修改，聚焦风险判断。

第六，逐步适配公司自研本地模型。 这是规划方向：让适合的任务在公司受控环境内处理，减少敏感数据外发，再通过内部授权数据微调改善特定任务效果，并评估成本收益，而不是默认本地模型一定更便宜、更准确。

所以，单点需求可以用 MCP、Skills 解决；**平台化的意义，是让这些能力在团队内统一复用，形成从任务到可审核交付的完整流程，而不是重复造编码工具**

8.3.4.1.2.2.1 追问：你讲到的降低人工 Review 成本，这个是怎么理解，可以展开说说吗?

💡 **答题思路：**在你讲了某个功能，内容，没展开讲的时候，面试官很容易追问。这个降低Review成本，也是我在需求一开始的时候，就设计的一个功能。当我们想清楚了功能设计，这个题很好回答。这个也是一开始我说，即使时间有限，没时间设计具体文档，也先想清楚项目的痛点，功能，架构这些最基本的问题。

🗣️ **参考回答：**

我理解的降低人工 Review 成本，主要是**降低人理解代码变更的成本，而不是让 AI 替人批准代码**。AI 加快了代码产出，但工程师仍然需要判断这些修改为什么做、是否合理、有没有验证。尤其当代码不是自己写的，一个 PR 又涉及多个文件时，理解修改思路本身就很耗时。

传统 PR 主要按文件展示红绿 diff，它能告诉你“哪些行变了”，但跨文件的修改关系往往需要 Reviewer 自己拼出来。我们希望参考 CodeRabbit Change Stack 按逻辑组织变更的方式，**把分散在多个文件里的 diff，按修改意图重新分组，并关联说明和验证证据**，而不只是额外生成一段 PR 摘要。

举个例子，Agent 要修复“请求失败后无限重试”的 Bug，可能同时修改配置文件、请求逻辑和测试文件。传统方式需要人来回翻这三个文件；我们希望先说明这次修复的目标，再按“定义并执行重试上限、传递重试次数、验证停止条件”组织阅读顺序。Reviewer 点开一个逻辑分组，就能看到相关的真实 diff，以及对应的测试和执行结果，再判断次数边界是否正确、测试是否覆盖原始问题。

这样降低的是**寻找相关代码、跨文件切换和重建修改思路的成本**，让人把精力更多放在行为是否正确、边界条件是否覆盖上。同时可以折叠格式调整、代码搬迁等噪声，但必须保留完整原始 diff 和未分组变更；说明要能跳转到代码和证据，未验证的地方明确标出，不能因为 AI 解释得顺就认为代码正确。

所以，这个功能的定位是**面向 AI 研发交付的变更理解与验收辅助**。目前属于设计方案，后续需要通过评审耗时和问题发现情况验证它是否真的降低成本，而不是只看页面是否更好看。

8.3.4.1.2.3 你在简历里提到了上下文压缩和运行态恢复，具体是怎么做的?

对应简历以下描述，预测的问题：

> 针对长任务上下文超限与摘要后状态丢失，按窗口预算压缩历史、保留近期交互，并保存与恢复关键运行态；通过工具输出分档裁剪、完整工具调用轮次回收及有界恢复与熔断，减少上下文溢出引发的重复失败。

💡 **答题思路：**

这道题的回答**严格围绕简历要点展开**，解释清楚简历写了啥。这个下面的策略，其实也是很多经典Harness框架，使用的一些常见方法了。

1. **按窗口预算压缩历史、保留近期交互**：用窗口占用比例解释触发条件——低于 80% 正常执行，达到 80% 先裁剪工具结果，仍达到阈值才摘要旧历史；按窗口容量约 16% 的 Token 额度保留近期交互，而不是固定保留几轮。
2. **工具输出分档裁剪、完整工具调用轮次回收**：说明**短、中、超长**结果分别如何处理，以及为什么调用消息和对应结果必须成组处理；用“之后只把摘要发给模型，完整消息保存在会话记录里”解释回收，避免抽象措辞。
3. **保存与恢复关键运行态**：讲清记录哪些任务信息，以及压缩后如何继续原来的工作；用“等待回归测试结果”的例子说明，避免只说保存状态、不说实际用途。
4. **有界恢复与熔断、减少重复失败**：明确溢出后先缩减输入，再重试原任务，最多一次；仍然超限就保存进度并停止自动执行。这里的熔断是任务级停止自动重试，不扩展为服务级熔断器。

🗣️ **参考回答：**

我们主要处理长任务中的两类问题：一是代码、日志不断累积，超过模型的上下文容量；二是压缩历史时漏掉任务进度，导致 Agent 重复操作。参考 DSH 的做法，方案分成四部分。

第一，按 Token 预算压缩历史，保留近期交互。

每次请求模型前，检查上下文的 Token 占用。**低于模型窗口的 80% 时正常执行；达到 80% 时，先裁剪长工具结果，裁剪后仍达到 80%，再把较早的历史压成摘要。** 摘要时，从最新消息往前保留约占窗口容量 **16%** 的近期交互，其余较早历史生成摘要；保留边界会适当调整，避免拆开工具调用和对应结果。

第二，工具输出分档裁剪，按完整调用组回收历史。

短结果直接保留，较长结果保留关键片段，超大日志保存成文件，只给模型预览和文件引用。压缩旧历史时，工具调用和对应结果一起处理，不能拆开。**之后发给模型的是这段历史的摘要，而不是完整消息；完整消息仍保存在会话记录里，需要细节时可以查回原文。**

第三，压缩前保存任务进度，压缩后补回关键信息。

我们会单独记录任务要求、当前做到哪一步、改了哪个版本，以及哪些测试已经通过。例如，压缩前正在“等待回归测试结果”，压缩后就把这个进度明确告诉 Agent，让它接着查询测试，而不是重新分析 Bug。正在运行的构建或测试还会记录作业 ID，恢复时根据 ID 查询实际结果。

第四，溢出后尝试压缩再重试，连续失败就停止。

如果模型报上下文超限，我们先裁剪长工具结果，不够再把更早的历史压成摘要。输入缩短到预算内后，让模型继续执行原来的任务，最多重试一次；如果仍然超限，就保存任务进度并停止自动执行，避免反复发送同一份超长输入。

8.3.4.1.2.4 你在简历里提到“基于历史 Bug 与修复 PR 构建分层评估链路”，这是什么意思，具体怎么做的?

对应简历以下描述，预测的问题：

> 针对评审质量难以量化、误报漏报难以定位的问题，基于历史 Bug 与修复 PR 构建分层评估链路，以文件及代码区间匹配定位问题，再用独立 LLM Judge 补充语义判断；输出误报、漏报与根因命中分析，为评审策略迭代提供可回放依据。

💡 **答题思路：**

这道题出自对简历中“基于历史 Bug 与修复 PR 构建分层评估链路”这条表述的提问。**简历里的每一条表述，都是面试官提问的热门内容。** 我们要针对每一个描述，自己先想清楚：**这是什么意思？具体是如何做到的？先要说服自己，才能向面试官讲清楚。** 这里的“分层评估”“文件及代码区间匹配”“独立 LLM Judge”“误报、漏报与根因命中”“可回放”，都要能用具体做法解释，不能只记住这些技术名词。

🗣️ **参考回答：**

这条讲的是**对 Agent 修复 Bug 中间过程的评估**，重点检查它有没有找到正确的问题位置和 root cause，而不是只看最后有没有修好。

因为最终测试不通过，可能是根因分析错了，也可能是分析正确但修改有问题。我们希望通过这套评估，把失败原因分清楚，再针对性调整。具体分四步。

第一，用历史 Bug 和修复 PR 准备评估样本。

把 Bug 描述、相关日志和修复前的代码交给 Agent，让它定位问题。我们根据历史修复 PR，结合代码和验证结果，整理问题位置与根因，作为参考答案。**参考答案只供评估使用，不提前给 Agent 看。**

第二，先检查位置，再判断原因，形成分层评估。

先用程序比对 Agent 报告的文件和代码区间，看它是否找到了相关位置。但位置对上不代表分析正确，所以再用独立的 LLM Judge，对照代码和参考答案，判断它说的是不是同一个问题、有没有解释清楚根因。有争议的结果再人工复核。

第三，分析误报、漏报和根因命中情况。

误报是提出的问题经核实并不成立；漏报是参考答案中已经确认的问题没有被发现；根因命中则是正确解释了 Bug 为什么发生。没有匹配上参考答案的发现，也会进一步核查，不能直接当作误报。

这样就能区分：**没找到问题、找到了但解释错了，还是根因正确却没修好。** 不同情况对应不同的改进方向。

第四，保存样本和结果，支持回放复测。

我们保存 Bug 输入、代码 commit、模型和提示词配置、工具调用记录，以及 Agent 输出和评估结论。调整检索或提示词后，再用同一批 Bug、同一个代码起点重新运行，对比结果。这里只改变要验证的策略，其他条件尽量保持一致。

所以，这套机制的作用是：**不仅知道 Agent 修复失败了，还能分析失败发生在哪个环节，并通过同一批案例复测改进是否有效。**

8.3.4.1.2.5 你在简历里写的“长任务执行可靠性”是什么意思？具体是怎么做的？

对应简历以下描述，预测的问题：

> 针对修复任务重复派发、构建失败后流程中断的问题，实现状态驱动的修复—构建—验证流程，以动作去重、并发控制和重试预算约束任务推进；失败时追加验证证据并按条件续跑原会话与原 PR，结合状态对账及中断恢复，减少重复执行和人工接管。

💡 **答题思路：**

这道题来自简历中的“长任务执行可靠性”。简历里的每一个描述都可能被追问，要先想清楚它是什么意思、具体如何做到，再向面试官解释。这里不是保证模型一定修好 Bug，而是说明程序如何管理整个修复流程：**任务按什么顺序执行，怎么避免重复，失败后怎么返修，意外中断后怎么接着执行。**

回答分四部分，与简历逐项对应：

1. **状态驱动的修复—构建—验证**：记录当前步骤，由程序根据执行结果安排下一步，而不是 Agent 说完成就算完成。
2. **动作去重、并发控制和重试预算**：同一请求或通知不重复触发动作，同一任务的分支不被多个 Agent 同时修改，自动返修不能无限循环。
3. **追加验证证据，按条件续跑原会话与原 PR**：把对应版本的报错和日志补充给 Agent，在已有修改上返修；区分代码问题与环境问题，并检查原 PR 是否仍可更新。
4. **状态对账及中断恢复**：服务意外中断后，核对构建等外部任务的实际结果，结合保存的任务、会话和 PR 信息继续执行，避免重复启动和人工重新派发。本题不展开主动取消处理，简历原句中的“取消恢复”建议同步改为“中断恢复”。

参考回答：

这里的长任务执行可靠性，主要是指 Bug 修复要经过修改代码、构建、测试，可能还要多轮返修。我们通过程序管理这个过程，避免重复派发任务，或者构建一失败就丢下任务，需要人从头接手。具体分四部分。

第一，用任务状态控制修复、构建和验证的顺序。

系统记录任务正在修改代码、等待构建，还是执行测试，并根据结果安排下一步。代码修改完成才能构建，构建通过才能进入后续验证，测试没有完成就不能报告修复成功。这就是“状态驱动”。

第二，防止重复执行，并限制返修次数。

同一个请求重复送达，或者同一次构建失败通知来了两遍，只处理一次，这就是动作去重。同一个任务也不能同时启动两个 Agent 修改同一分支，这就是并发控制。自动返修还会设置次数限制，例如最多两轮，仍然失败就停止并转人工，而不是无限重试。

第三，失败后把报错补回原会话，在已有修改上继续返修。

构建或测试失败后，我们把对应的代码版本、执行命令、报错和日志补充给 Agent，让它根据这些证据继续修改，而不是重新分析整个任务。如果原 PR 仍然打开、分支和权限也允许继续，就把后续修改提交到原分支，更新原 PR。若只是构建环境故障，则先处理环境问题，不让 Agent 盲目改代码。

第四，意外中断后核对实际进度，从断点继续执行。

例如服务重启后，系统里还记录着“构建中”，我们会根据保存的构建 ID 查询实际结果：已经成功就继续验证，还在运行就等待，失败了就进入失败处理，而不是重新启动一次构建。这就是状态对账。结合之前保存的任务进度、会话和 PR 信息，系统可以从合适的步骤继续，不需要人工重新整理上下文、派发任务。

8.3.4.1.2.6 你在简历里提到“基础工具与代码智能增强方案的对照评估”，具体是什么意思？你是怎么评估 Agent 修复效果的？

对应简历以下描述，预测的问题：

> 针对代码智能工具对自动修复的实际增益难以判断的问题，基于 SWE-bench 完成基础工具与代码智能增强方案的对照评估，比较代码检索、调用图及结构上下文对修复任务的作用；以测试验证补丁有效性，结合单任务成本、工具调用次数与执行轨迹形成质量—成本对照分析，为工具增强与修复策略选型提供依据。

💡 答题思路：

这道题来自简历中的工具对照评估。先说明：**我们建立了一套 Bug 修复评估集，通过增加或移除工具做对照实验，判断这些工具能否改善修复效果，以及增加的成本是否值得。** 再按照“准备评估集 → 对照与消融 → 测试验收 → 分析并选型”展开，既解释简历，也讲清 Agent 的评估方法。

这里按之前确定的口径，使用的是**借鉴 SWE-bench 方法构建的内部评估集**，不是直接声称使用了官方数据集。简历原句中的“基于 SWE-bench”应在定稿时同步明确这一含义。本题细化第 2 题的修复评估方法，重点回答工具是否有收益；第 7 题则通过定位与根因分析，帮助解释修复失败的原因。

🗣️ 参考回答：

这条主要是说，我们不确定给 Agent 增加代码检索、函数调用关系查询等工具后，是否真的能帮助它修复 Bug，所以建立了一套评估集，通过对照和消融实验，比较修复效果与成本，再决定集成哪些工具。具体分四步。

第一，借鉴 SWE-bench 的方式准备评估集。

从历史 Bug 中整理问题描述、相关日志、修复前的代码版本、运行环境和验收测试。Agent 根据问题描述修改代码，历史修复答案不提供给它。数据分为调试集和独立测试集，前者用于调整工具方案，后者用于最终验证。

第二，固定其他条件，只改变工具组合。

让同一个 Agent 使用相同模型，在同一批任务、代码起点和预算下运行。基础方案只提供文件读取、文本搜索、编辑和执行命令等工具；然后分别加入代码检索、调用关系查询、代码结构查询等能力进行比较。也可以从完整方案中移除某个工具，观察效果变化，判断收益来自哪个工具，而不是只比较两套完全不同的系统。

第三，用测试判断补丁是否有效。

Agent 生成补丁后，在对应环境中执行构建、缺陷验证和相关回归测试。既检查原来的 Bug 是否修好，也检查是否破坏了已有功能。按统一标准统计任务解决率，而不是让模型自己判断“修复成功”。

第四，结合成本和执行过程决定工具选型。

除了看修好了多少任务，还记录单任务费用、耗时和工具调用次数，并查看执行轨迹。例如，调用关系查询是否帮助 Agent 更快找到相关代码，还是增加了大量查询却没有改善结果。

最后根据这些对比，决定哪些工具值得保留、哪些需要改进，或者只在特定任务中启用。**调整的是 Agent 的工具组合，对比时使用的评估集保持一致。**

8.3.4.1.2.7 本地模型处理问题

💡 **答题思路：**

因为我们简历里明确写了，我们这个Agent项目接入了本地模型。融入了我们后训练项目。简历里写的很多评估，训练方法，可以完全套用后训练项目的内容，就不展开了。唯一要想清楚的是和后训练不一样的地方：就是该项目中使用本地模型干什么？**它的业务场景是啥，输入输出是啥？** 如果一开始的设计，我把本地模型的任务设置成了在解决bug之前，调用工具，搜集信息，产出结构化json字段。这个需求就完美和我们后训练项目适配了。

🗣️ **参考回答：**

我们给本地模型的定位，是主 Agent 前面的一个**任务理解与信息补全层。**它不直接修 Bug，而是把 Ticket 和用户要求整理成结构化任务，判断还有哪些关键资料需要补齐，再由 Harness 组装上下文交给主 Agent。

它主要整理四类信息：**第一是任务对象**，也就是哪个工单、项目和仓库；**第二是问题描述**，包括实际现象、预期行为和已有复现线索；**第三是执行条件**，包括目标分支或版本，以及构建测试入口；**第四是用户约束**，比如只分析还是允许修改、不能改公共接口、不要新增依赖、暂时不要创建 PR。用户后续改口时，只更新被明确修改的字段，保留其他已有约束。

第二层是工具调用和任务分流。 我们为这一层设计了工单读取、评论和日志查询、项目与版本查询、代码检索及记忆召回等工具，模型通过多轮分析和工具调用补齐材料。当信息足以让主 Agent 开始工作，或者判断需要人工介入时，输出统一的结构化 JSON，包含 **Bug 单信息、仓库与版本、问题现象、复现线索、用户约束、信息缺口，以及是否需要人工介入和介入原因。**Harness 负责工具执行的权限与预算控制，并校验最终结果：满足条件就转到下层 AgentLoop，需要澄清就转人工；工具失败或达到循环上限则按规则回退，不无限等待信息完整。

第三层是训练和本地推理优化。 模型选用 **Qwen3-0.6B。**在已有结构化抽取实践中，**SFT 主要解决“能否按要求正确输出”**，包括固定 JSON 格式、字段归类、多轮状态继承和合法工具参数；**DPO 则作为偏好优化的对照尝试**，用于探索正确与错误候选之间的选择，例如信息不足时追问而不是猜测、保留已有约束而不是错误覆盖。实验中 DPO 没有体现出稳定增益，因此最终采用评测表现更合适的 SFT 方案，而不是默认叠加 DPO 就会更好。

推理侧使用 **llama.cpp** 本地部署，结合 **GGUF 量化压缩和前缀缓存**，分别减少模型资源占用和重复前缀计算，并通过回归评测检查质量变化。整体上，已有实验在解析质量和推理速度上都取得了改善，为 **0.6B 小模型承担受控的任务解析工作提供了基础。**迁移到 Ticket 场景后，再以字段正确性、约束保留、工具调用和人工分流的验收结果作为生产接入依据；不把原抽取实验直接等同于 Ticket 已经稳定上线。

📚 **附录**

帮助你更具体理解，列出模型大概要搜集什么信息，申请哪些工具供参考。

展开一：模型整理的四类信息

第一类是任务对象。 包括 Ticket 编号、项目、所属组件和目标仓库。优先通过工单与项目配置解析；如果存在多个候选且无法确定，就向用户澄清。

第二类是问题描述。 包括实际现象、预期结果、触发条件、复现线索和日志引用。只提取已有事实并保留来源，不编造根因；缺少复现步骤不一定意味着任务必须停止。

第三类是执行条件。 包括分支或 commit、环境信息，以及构建和测试入口。需要查询配置并验证版本；如果没有现成测试，要明确标记，是否继续由任务规则决定。

第四类是用户约束。 包括只分析还是执行修复、允许修改的范围、是否禁止新增依赖，以及 PR 创建要求。需要正确处理否定、改口和局部更新，但用户意图不能代替系统授权或正式审批。

展开二：可以申请的工具

第一类是工单资料查询。 示例工具包括 `ticket.get`、`ticket.list_comments` 和 `ticket.get_related`，用于查询工单描述、补充评论和明确关联的工单。发现信息矛盾时记录冲突，不默认最新评论一定正确。

第二类是附件与日志读取。 示例工具包括 `ticket.list_attachments` 和 `ticket.read_attachment_excerpt`。先查看附件清单，再读取相关文本片段，并限制读取长度，不把全部附件塞入小模型。

第三类是项目与版本查询。 示例工具包括 `project.get_config`、`project.lookup_component` 和 `repo.resolve_revision`，用于查询组件到仓库的映射及工程配置，验证分支并解析代码版本。

第四类是有限代码检索。 示例工具包括 `code.search` 和 `repo.read_excerpt`，用于根据明确的错误码、符号或路径获取起步线索，以及读取工程说明。深度调用链分析仍留给主 Agent。

第五类是项目经验召回。 示例工具是 `memory.search`，用于召回有来源、经过验证且适用当前项目和版本的经验，并按权限过滤。

8.3.4.1.2.8 你如何对300+GB的代码仓库建立索引的?

💡 **答题思路：**面试官问这个问题是因为我们提到了，我们代码仓库很大，比如300+GB.使用的是GVFS仓库（也就是不会全量下载到本地的）。所以这个时候使用ClaudeCode本地处理肯定会失效。这是我们埋下的真实的痛点，面试官问这个问题很合理。

🗣️ **参考回答：**

传统的 **Agentic + grep 全仓搜索方案，在我们的 GVFS 大仓库场景下不适用。**本地主要只有元数据，文件内容按需下载；grep 为了搜索而读取大量文件，会触发大量下载，延迟和资源开销不可控。如果只搜已下载的文件，又会漏掉大量代码。

所以我们设计了 **“远端共享索引 + 本地增量覆盖 + 按需读取”** 的方案，保留 Agent 多轮探索的能力，但不再依赖本地全仓扫描。

远端主要建立两类索引。第一类是代码全文索引，把文件路径、函数名、标识符和代码文本建立可检索的倒排索引，用来精确查找函数、错误码、配置项等，这是检索的基础。**第二类是语义向量索引**，按函数、类等结构切分代码，再生成向量，让用户用“登录失败后的重试逻辑在哪里”这样的自然语言，也能找到相关实现。

查询时，**精确关键词走全文检索，功能描述结合向量检索，再合并、排序，返回代码片段和对应版本。**我们优先复用已有企业搜索能力，通过检索插件接入，按需补充缺少的索引能力，不让每个开发者重复建一份全量索

引

远端建索引当然也需要读取源码。**我们不是消除了获取代码的成本，而是把全量获取和建索引集中到服务端，供有权限的开发者共享。**后续按提交变化增量更新，而不是每次重新扫描整个仓库。

本地只补充分支差异和未提交修改，搜索时用本地版本覆盖远端旧版本，并屏蔽已删除文件。分析阶段优先从远端读取必要片段；真正需要编辑时，才通过 GVFS 下载目标文件，并核验当前版本，避免把主干代码误当成开发分支代码。

核心就是：服务端负责全局找代码，本地负责当前修改，GVFS 负责按需取文件。

8.3.4.1.3 重点问题

8.3.4.1.3.1 项目长期记忆短期记忆要存一些什么内容?

💡 **思路：** 虽然简历没特别强调，但是属于面试官老喜欢问的问题的了。结合我们讲的理论基础：Ageng记忆功能，先准备一下答案吧。

🗣️ **参考回答：**

在 ForgePilot 里，我会把记忆分成短期和长期两层，**短期记忆解决“当前任务怎么继续”，长期记忆解决“后续任务有哪些经验可以复用”。**

短期记忆主要围绕当前研发任务保存：第一是用户的目标和约束，比如要修复什么 Bug、验收标准是什么、是否允许修改接口；第二是任务进度，比如已经完成复现、正在定位，还是等待测试和人工审核；第三是执行上下文，包括近期对话、工具调用结果、报错信息、当前代码版本和补丁引用，以及尚未验证的假设和待办事项。我们计划采用“**结构化任务状态 + 近期对话原文 + 早期摘要**”的方式管理，既支持任务中断后重建上下文，也控制 Token 消耗，而不是把所有聊天无限追加进去。

长期记忆则不存完整聊天，而是提炼四类值得跨任务复用的信息：**项目的重要约束和决策背景、经过验证的修复案例、有证据的失败经验，以及用户明确表达的协作偏好。**比如某个接口为什么必须兼容旧版本、某类并发问题之前如何修复和验证、某种方案在什么条件下失败，以及用户是否希望先确认方案再修改代码。

这些长期记忆要带上来源、适用项目、代码版本和验证状态。未经验证的模型猜测不会直接作为事实入库，密钥和一次性审批权限也不会作为可复用记忆。总体原则就是：**短期存准确、可恢复的任务现场，长期存经过筛选、有证据、明确适用范围的经验。**

8.3.4.1.3.2 你们这个系统是怎么发布的?

💡 **答题思路：** 按照上面的理论讲解

🗣️ **参考回答：**

项目把 **React** 前端、**Node.js** 本地后端和 **Electron** 运行环境一起打包成桌面客户端。用户安装以后，任务编排、工具调用和仓库操作主要在他们自己的电脑上本地执行。

同时构建和发布，我们也在企业内部的Git上构建了 **CI/CD** 流水线完成。确定发布代码后，流水线执行依赖安装、前后端构建和测试，再由 **electron-builder** 生成 **Windows EXE** 安装包。安装包上传到内部GitHub平台，下载页面和通知邮件提供版本号、更新说明以及下载链接，用户下载安装即可使用。

因为产品面向企业内部几百名开发者，我们选择**分批邮件通知**的方式做灰度：先让少量开发者安装，观察**任务成功率**、**执行耗时**、异常日志和使用反馈，确认稳定后再通知更多团队。这里控制的是推广范围，不是邮件发出去就自动完成安装升级。

如果新版出现严重问题，就暂停后续推广，并让受影响用户回到**上一稳定版**。回退前检查本地数据和配置兼容性，必要时备份；修复后用新的补丁版本重新打包，再做小范围验证。首版采用用户主动下载安装的更新方式，后续如果接入 **electron-updater**，才支持应用内检查更新和下载，不能把发布下载链接等同于自动升级。

8.3.4.1.3.3 你们这个系统是版本管理的?

💡 **答题思路：** 按照上面的理论讲解

🗣️ **参考回答：**

我们的版本管理方案主要用 **Git 分支**、**应用版本号**、Tag 和**发布记录**，解决两个问题：一是日常开发不要影响稳定版，二是每个安装包都能追溯到具体代码。

日常开发从 dev 拉出 **feature 分支**，功能完成后通过 PR 审核合回 dev。dev 用来集成已完成的功能，检查不同模块之间有没有冲突。准备发布时，从 dev 的确定提交拉出 **release 分支**，在这个分支上做发布验证和问题修复，不再加入下一版的新功能，让本次发布范围保持稳定。

验证通过后，在 release 的确认提交上打 **Tag**，例如 v1.1.0，由流水线针对这个 Tag 构建安装包，并记录版本号、代码提交和更新内容。这样用户反馈问题时，我们就能根据客户端版本找到对应安装包和代码，而不是拿最新开发代码去猜旧版的问题。发布后，release 合入 main，作为正式**发布记录**；发布期间的修复也同步回 dev，避免下一版把已经解决的问题重新带回来。

如果 1.1.0 发布后出现问题，我们暂停推广，必要时提供**上一稳定版**恢复使用，再发布 1.1.1 这样的修复版本，不直接覆盖原来的 **Tag** 或同版本安装包。这样既能并行开发，也能保证线上问题可追溯、可回退。

8.3.4.1.3.4 你的项目线上遇到问题的时候，你怎么排查和解决的?

💡 **答题思路：** 我们上面有整理日志上传的方法和思路，结合这个理论 + 一个具体的例子来回答。（答案就以遇到了xx问题，因为我们在系统xx埋了上报，记录了xx的流程，拿到这些指标，聚合成了xx的数据，可以判断是否发生异常，再结合xx日志，来分析问题发生在哪里，答案以我们排查的具体某一个例子展开，这样会更真实）

🗣️ **参考回答（模拟案例：长任务上下文压缩后丢失关键约束）：**

我们的排查思路主要是**埋点、日志上传和看板观察**。在任务执行、**上下文压缩**、工具调用等关键环节记录结构化日志，带上 **session ID**、**任务 ID**、客户端版本、执行阶段、耗时和异常信息，脱敏后批量上传到远端日志平台，再聚合成任务成功率、执行耗时、异常类型等指标，通过看板观察。所以不管是系统监控发现异常，还是用户反馈问题，我们都可以根据 session ID 找到对应会话，再按任务 ID 串起这次执行的完整链路，判断问题发生在哪个环节，而不是只根据用户描述去猜。

我就一个具体的例子，比如之前有个**团队反馈系统**：明明要求“**修复 Bug，但不能修改公共接口**”，Agent 最后却改了接口。针对这个问题，我排查的时候先找到对应 session 的执行记录，沿着工单解析、上下文组装和模型执行往下查。发现最初的模型输入包含这条要求，但任务中途触发压缩，压缩后的摘要没有保留它，后续输入也没有重新补充，于是我就能把问题定位到约束在压缩过程中丢失。再通过**受控回放**，对比压缩前后的输入和行为，验证这个判断。

修复时，把这类硬约束单独保存在**结构化任务状态**中，不只依赖聊天摘要，每次压缩后由 **Harness** 重新注入，并在提交前检查接口变更。最后回放这个失败任务，补充长日志、多轮压缩的回归用例，小范围发布后继续观察约束违规情况，确认修复没有引入新的问题。

8.3.4.2 智能发布助手（RAG项目改造）

8.3.4.2.1 必考问题

8.3.4.2.2 高频问题

8.3.4.2.2.1 你的检索模块和存储模块选择的什么，为什么？

💡 不光是因为简历里写了做了双路召回，所以我们特别需要准备。从RAG知识的重要性角度，RAG模块里面的双路召回的**向量选型，数据库选型，检索方法选择，都是高频考点。**

一定要提前想清楚，自己为什么选择这个技术，选型依据是什么？同时回答最好准备好有数据实验作为驱动，**任何选型，最有力的回答就是我做了xx实验，证明了它的合理性。**

🗣️ **参考回答：**

我们选择 **BM25 + BGE 双路召回，OpenSearch 统一承担存储和检索。**

1. 因为 Windows 发布资料里有大量组件名、分支名和内部缩写，纯向量检索容易混淆相似对象，所以用 BM25 补充词面匹配；相比学习型稀疏模型，它不需要额外推理，更新成本低，分词和匹配问题也更容易排查。
2. 语义召回使用 **bge-base-en-v1.5**，因为我们主要处理英文短片段，不需要优先承担 BGE-M3 的多语言、长文本能力对应的模型开销；相比 Large，Base 模型更小、向量维度更低，适合作为成本可控的基线。
3. 存储选择 **OpenSearch**，主要考虑三个方面：

- **搜索权重配置：** 发布信息分散在 Teams、Wiki 和 Outlook 中，计划经常变化。我们可以在内容相关的基础上，按更新时间调整排名，让近期的变更通知更容易被检索到，避免只查到旧计划。

- **Metadata 过滤：** Windows 不同分支、版本的发布安排不同，我们可以按分支和版本限定检索范围，排除明确不适用的资料，避免内容看似相关，实际却属于另一个版本。

- **统一承载两路存储和检索：** OpenSearch 同时支持 BM25 和向量检索，可以把原文、BGE 生成的向量以及元数据放在同一条记录里。相比“独立 BM25 索引 + ChromaDB”的方案，少维护一套存储，也减少资料更新、删除时两边不一致的问题。

8.3.4.2.2.1.1 追问 这些选型有没有做实验对比?

💡 这也是一个合理真实的追问。在202608阿里淘天秋招一面的问题中，就在技术选型后面追问，你是否做了实验。

我们的答题思路就是先回答我们的RAG项目的设计：

1. 模块化，做了全链路可视化和评估，这三个特性都是天然支持我们来做技术选型的实验对比的。
2. 以嵌入项链举例，说明真实实验选型过程。

🗣️ **参考回答：**

做过，而且方案对比一直贯穿开发过程。 我们把系统做成了模块化结构，稀疏召回、Embedding 模型、检索后端和 reranker 都可以替换，RRF 的参数也可以调整。同时接入了 RAGAS 和可视化调试系统：RAGAS 辅助评估回答质量，可视化界面查看两路召回了什么、融合和重排后名次怎么变化。因此我们不是只看最终回答，而是通过测试和中间结果定位问题，再调整方案、跑回归对比。

比如嵌入模型选型，我们对比了 **all-MiniLM-L6-v2、bge-base-en-v1.5 和 bge-large-en-v1.5**，发现 MiniLM 虽然轻量，但更容易召回泛化的发布流程介绍，漏掉包含分支关系、合入截止时间和发布批次的具体记录；BGE Large 的检索效果在我们数据集上的表现，效果和 Base 几乎持平，但生成向量的成本却更高，计算嵌入向量和检索的时间显著增加；BGE Base 在检索质量和成本之间更均衡，因此最终选择了 **bge-base-en-v1.5**。

8.3.4.2.3 重点问题

> **口述参考：**以下按本文上线设计回答；尚未实际部署、验证的部分，面试时保留“设计 / 计划”的口径。

8.3.4.2.3.1 请你讲一下你的RAG项目运行的逻辑？其他团队是如何接入的?

> 💡 **答题思路：**按照上面我们讲的要点：接入形式（客户端通过 mcp远端server streamable http通信），鉴权原理，后端运作方法（使用vm 部署长期rag服务）按逻辑串起来讲解。

🗣️ **参考回答：**

这个项目的上线设计是把 RAG 做成团队共用的**远端 MCP Server**，各个团队成员使用本地Coding客户端接入。服务部署在企业内网或通过企业网络访问的 **VM** 上，长期运行，负责文档获取、解析、分块、向量化、索引维护和检索。客户端只负责调用工具，再由客户端里的模型根据返回的证据组织答案。

其他团队接入时，我们提供一份内部 Wiki，说明服务地址、网络要求、支持的客户端和配置方法。用户在 Cursor、VS Code 或 Claude Code 这类支持远端 MCP OAuth 的客户端里配置地址，通过基于 HTTPS 的 **Streamable HTTP** 连接服务，不需要安装我们的 RAG 程序，也不需要维护本地索引。

鉴权分成**身份认证**和资料授权两层。客户端按照 MCP 的 OAuth 授权流程引导用户使用公司账号登录，获取 **Access Token**，后续请求携带这个 Token。我们选择 **Microsoft Entra ID** 对接企业身份，服务端验证 Token 后，再根据用户所属团队和文档权限，限定可以检索的资料范围。登录成功不代表能查看全部知识库，摘要等读取操作也要执行相同的权限检查。

总结完整的链路是：配置地址、公司账号登录、携带 Token 调用工具、服务端在授权范围内检索、返回证据和来源，最后由客户端模型生成回答。后续只要接口和认证方式保持兼容，我们更新 **VM** 上的服务，用户通常不需要重新配置客户端。

8.3.4.3 智能预约Agent

8.3.4.3.1 重点问题

8.3.4.3.1.1 你这个项目是如何接入语音服务的?

💡 **答题思路：**按上面理论基础回答即可。

🗣️ **参考回答：**

语音相关的服务，项目里使用了腾讯云的云呼中心服务来管理电话接入，当用户打通电话后，会自动连接到腾讯云的云呼叫中心。 同时这里使用了腾讯云的STT**（语音转文字）**和TTS服务**（文字转语音），**将云呼中心的内容和windows本地后端服务打通，用户的语音输入转成文字，输入到本地Agent系统调用模型进行处理，同时存档，更新相关状态。系统的输出，通过TTS转成语音，返回给用户，实现语音服务的接入。

> 对于这个个人项目，其实我还想到了一些乱七八糟的非技术问题，面试官有可能问，比如：
>
> 你的项目接入门店，老板给了你多少钱?
>
> 后期有没有维护？ 是如何交付的，交付标准是啥?
>
> 对于一个独立的个人且投入生产的项目，面试官可能好奇这些问题。这些问题和技术无关，自己空了多想想就行，想一个合理的答案，自己地回答就好。
>
> **不用担心自己的答案是不是合理**，现在AI比较发达，让AI帮你参考，自己提前准备过的答案一般都是比较合理的，准备好了就自信地回答，面试官一般也很难通过你的一两句话就断定你说的是假的

8.3.5 讲解视频

这个视频主要讲了ForgePilot的架构的设计，相关面试问题

https://www.bilibili.com/video/BV1PReT6EEqR/?spm_id_from=333.1387.homepage.video_card.click

这个视频结合了阿里秋招面试问题，以我们的RAG项目为原型，展示了这个智能发布助手（RAG项目）相关的面试前突击问题的准备。

https://www.bilibili.com/video/BV15Yhn6FEeR/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

这个视频讲解了本小节讲的三个项目的上线方案设计, 如何把项目包装成企业级项目。

https://www.bilibili.com/video/BV14hHX6EEXN/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

8.3.5 结语

我用以上3个讲解视频 + 文档，表达了面试前如何突击。面试前突击多少，取决于你自身的**底子和时间。**

对于我已经做过的，但是进行包装了的项目：我面试前突击的内容就比较少。

对于没有做过的，比如forgePilot项目：我就花了比较多的时间去设计架构，思考整个项目如何运作。

其实里面仍然有很多细节，值得继续突击，但是限于时间问题，我就是直接去面试了。后面根据面试情况查漏补缺。

在时间允许的范围内，我们应该尽可能去想的更细致，但是面试永远准备不完，**到了面试节点，硬着头皮也要上。 注重好面试之后的复盘，面试会一次比一次好的。**

8.4 简历投递

前面记录了我的面试前突击准备，这一小节接着记录简历投递。

我自己主要的投递方式是使用 AI 搜索。因为我并不着急面试，对现阶段的我来说，去面试那些不在意向范围内的小公司，没有太多意义，所以我没有选择大范围投递，而是只让 AI 帮我找意向公司的合适岗位，再直接去官网投递。

具体来说，我会把自己的背景、求职方向和意向公司告诉 AI，让它搜索招聘官网，把符合要求的岗位和投递入口整理出来。我自己确认岗位要求后，再填写信息、上传简历。投递完成后，也会让 AI 更新记录，方便后续跟进。

更多投递渠道、AI 搜索的具体方法，以及投递顺序和时间安排，我在第 3 节“简历投递技巧”中做了整理，感兴趣的同学可以跳转阅读。

8.5 面试后复盘（加深项目）

这一小节的素材都来自于真实的面试经验。 面试后，要做的事就是不断根据面试的内容-复盘，加深自己的项目。不管是问题的表达，还是项目技术深度。

因为一旦我们简历写好，其实面试内容大概成型了：面试大概率是围绕我们的项目去问的，每次注重复盘，一次会比一次回答得好。这一章，根据每一次面经，加深我们的三个项目相关问题和设计。

8.5.1 ForgePilot项目

1. 现在ClaudeCode Codex已经很成熟了，你为什么要开发自己的Agent平台?

导语

面试官很可能通过这问题压力你，质疑你，请记住，这太正常不过了。不管你做什么项目，几乎都会被挑战，做了RAG项目，面试官挑战RAG过时了，做了Agent项目，质疑你没深度，没必要等等，这太正常不过了。不过不要怀疑自己，我们要做的是找到这些质疑问题的回答的角度，说服面试官，或者至少不被挑战。 大家也不用因为被质疑了过多怀疑自己的项目立意，我从几个例子来说明这件事其实也是合理的：

1. 我专门查了我们公司2026年10月，部门自己做的coding agent还在迭代。注意不是copilot，而是独立于copilot之外的部门自己做的Agent。也就是说我们公司都有比较出名的coding agent，但部门的coding agent也没在它的基础上做。而是做了一个新的平台。

2. 行业现状就是这样，Claudecode ,Codex, 国内的workbuddy都有产品，那么为什么我看到几乎每个公司都在重新开发自己的coding 平台。**我同学在东风风神，造汽车的公司，他们都有自己内部的AI平台。**

包括现在Agent已经很多了，但是2026年9月，各个国外巨头公司在开发新的personal agent，国内公司也在跟进。我再泛化一点，**飞书文档功能已经很强了，字节内部很多部分在开发自己的OA系统。**

从这些信息可以看出来，**重复造轮子的事是行业现状，也许我们造不出来更好的轮子，但是可以造出更合适的轮子**，存在即合理，不用因为面试官的挑战，就质疑自己项目的立意。

💡 **答题思路**

也许这件个项目是，老板让我做的，我就做了而已，决策权又不在我。这是一个合理的答案，但这不是一个可以展示出我们技术深度的好的答案。作为开发，我们要体现出我们的技术思考，这个问题，我希望大家从以下几个方向思考，找一下角度：

1. 现有方案的能力缺口：哪些具体需求满足不了，或扩展不到位。

2. 内部业务与工具更好的适配：如何更好地利用企业已有工具、知识和研发流程。

3. 模型与 Harness 联合优化：跳脱出远端模型，自己做一套最好的方法是，是打通Harness-model-数据回流训练的内部生态闭环。

4. 安全合规与商业条件：数据边界，安全权限问题。使用CC,Copilot涉及远端模型的使用，有些公司不允许订阅，因为数据泄露了或造成其他事故，损失很大，本质这是商业问题不是技术问题。

5. 轻量化与工程取舍：外部方案太重，选择轻量内核加必要业务能力。

6. 其实还有一个现实原因：就是企业影响力的原因。公司有公司的考虑，部门和小组有自己的考虑。如果从部门的角度，研发了一个Agent并投入使用有了Impact，肯定是能拉动这个部门整体绩效的，即使重复造轮子，但是有的部门，小组就是要做。

大家思考这个问题的时候，我觉得你可以从这个思路来去挖掘。把每一点想清楚，再组织最后呈现给面试官一个什么答案。

1. 现有方案的能力缺口

❌ 表示默认功能或方式不能直接满足需求； ⚠️ 表示可以尝试执行，但效果与可靠性不足，需要针对业务改进。

1.1 ❌ **大规模代码仓库检索：默认的本地搜索方式不适用。** 对于我们讨论的 300GB、按需拉取的代码仓库，如果直接让 Claude Code 通过 Agentic grep 反复扫描本地文件，搜索可能触发大量源码下载，出现长时间阻塞、超时，表现为检索失效或“卡死”；如果只搜索已经下载的文件，又会遗漏相关代码。问题不只是模型理解能力，而是默认检索方式与仓库的存储、访问方式不匹配。

> 这个是真实业务问题，你自己也可以挖掘你的业务上有什么类似的现有框架不能解决的问题

1.2 ⚠️ **持续执行长时任务：能尝试修复，但难以仅靠默认行为保证持续完成闭环。** ForgePilot 的目标不是单次改代码，而是持续推进“理解工单—定位问题—修改代码—构建测试—根据反馈返修—交付评审”。如果不增加针对业务的上下文管理和流程控制，长时任务需要重点防范关键信息丢失、重复尝试已失败的方案、陷入无效循环，以及把环境故障误当成代码问题反复修改。

> 思路来自于2026年9月各个新Agent他们共同趋势：自动持续完成长程任务。CC,Codex虽然能做，但是毕竟不是专门为此定制的，很能效果不好，比如关键信息丢失，这就需要针对于业务来具体修改harness。包括现有文档表明：像CC在执行跨会话执行仍会出现交接不清、重复恢复工作和过早宣布完成等问题。

1.3 ❌ **符合系统架构与修改逻辑的 Review 界面：需要额外开发产品功能。** 我们需要的不是普通 Diff 展示或一段 AI 评审意见，而是按照系统模块、调用关系和修改意图组织跨文件变更，让评审者沿着业务逻辑查看真实 Diff、影响范围和对应的测试证据。

> 这是我们产品设计的功能之一，CC当然没有，也许增加插件，SKILL可以实现，这个我们后面会讲

2. 内部业务与工具更好的适配

现有的Harness的框架和企业内部的环境肯定是需要适配的:

2.1 **工单理解与跨平台信息串联：** 比如系统如何理解企业内部的工单信息，如何将散落在各个内部平台的历史，PR，代码仓库，测试结果串起来。

2.2 **内部业务约定：** 如何理解内部业务的约定，比如项目规范，模板，专业术语。

2.3 **已有开发流程与账号权限：** 如何遵循现在已有的开发流程，比如分支构建，测试，提交PR，如何沿用企业内部的账号系统，权限系统。

这些功能即使是用外部的框架也需要进行适配和修改。

3. 模型与 Harness 联合优化

自建 Agent 的**长期价值**，不仅在于当前版本的功能能力，更在于建立面向企业真实业务的数据、评测与持续优化闭环。

在满足企业数据安全、权限和隐私要求的前提下，自定义 Agent Harness 可以更系统地记录真实任务中的执行轨迹数据。这些数据既可以用于构建业务评测集，也可以为模型后训练提供高质量样本。

在此基础上，模型与 Harness 可以形成协同迭代机制：模型侧提升业务理解、工具调用和代码修复能力；Harness 侧优化上下文管理、工具接口、任务编排与反馈机制，并通过统一的业务评测验证改进是否真正提升了端到端任务表现。

因此，我们的**目标并不是在当前版本全面超越 Claude Code，而是围绕企业特定场景，建立一套可评测、可追溯、可持续迭代的 Agent 系统**，持续提高任务成功率，并控制人工介入成本与执行成本。

4. 安全合规与商业条件

安全权限和隐私性问题。这是一个实际因素，像字节不可能用腾讯的workbuddy, 微软员工可能不让使用Codex和ClaudeCode。一些小公司其实也因为政策原因，是不允许订阅github copilot和ClaudeCode的，数据一旦泄露，损失巨大。所以他们宁可自建Agent系统，也不愿意使用别人的Harness和训练框架。

5. 轻量化与工程取舍

> 讲了上面内容，面试官也可能继续追问这样一个问题，你说的这些外部框架的缺陷，完全可以通过复用框架Harness，在上面制作SKILL, MCP, Plugin扩展的机制来实现，那为什么要重新做一个平台呢?

1. **灵活性：能挂接压缩钩子，不等于能替换压缩算法。** 我以压缩原理为例，CC的扩展一般是通过加钩子，Claude Code 的公开压缩钩子只有 `PreCompact` ， `PostCompact` ，无法更改压缩的实际原理。对于我们的目标，实现长程持续任务，定制化压缩对结果非常重要，所以这里使用CC扩展，不够灵活，达不到我们的目的。

> 自己修改，好处一定是方便魔改。我们需要真的找一个站得住脚的点。比如如果我们说CC提示词太多了，没法改，这个就容易被挑战，CC是有简化提示词模式的。这个压缩机制我看上去是找的一个CC无法扩展的点，和我们的目标试下长程任务持续化输出也很匹配。

2. **开发工作量：复用通用框架，不会省掉业务能力本身的开发。** 以 300GB、按需拉取仓库的检索改造为例，两种方案都需要实现远端索引、版本过滤、本地修改合并和按需读取源码。扩展 Claude Code，可以把这些能力封装成 MCP 工具，但还要配置或限制原有的本地搜索路径。自己的 Agent 框架可以直接将这条链路定义为标准检索入口。所以其实对于业务定制的逻辑，两者方案也都不能省略，也并不是说我们复用了框架，开发量就会更小。
3. **选择性复用：自建业务 Harness，不等于从零重写所有能力。** ForgePilot 的方案在 Agent 核心层引入 Pi，复用通用的 Agent 执行能力；企业检索、业务上下文、任务流程和 Review 产品能力则由我们组

织。前面比较的是“扩展 Claude Code”与“自己掌握业务 Harness”的取舍，Pi 在这里说明的是我们的复用边界：通用能力尽量复用，业务差异和需要长期迭代的机制由自己控制。因此，这不是拒绝现有技术，而是选择复用核心组件，而非完整的通用 Harness。

思路整理

我的个人观点是：任何人给你准备的项目，不管代码多全，文档多细致，面试真题总结了多少，都是需要你能够要**自己消化，修改，适配的。**对于项目和具体这个题的思路，我相信我总结的已经很详细了，先从上面的点思考，然后把这些要点列出来，最后筛选你准备怎么给面试官讲，再得出最终标准答案，我下面会给出针对于我们项目的参考答案。但是对于你自己的答案：**也一定需要你去理解，消化，然后结合自己的业务，公司，做相应的修改。**

🎤 **参考答案**

> 给面试官的回答，通常来说要简单，不要你自己叽里咕噜说一堆，我一般都建议在2分钟之内说完。因为大家自己可以体会一下，一个问题你说了2分钟以上，很容易让人走神，抓不住重点。上面展示的是一些思考点，真正给面试官回答，你就想一下结合你的业务，哪些地方比较站的住，把大概原因列出来，可以再结合一个具体的业务场景展开举例说明。比如我这里的参考答案，就是把这几点列举，然后也举具体举例说明。

其实这一点我们有专门考虑过，主要原因有：

第一，即使是使用现有的框架，我以CC来举例，有些地方也是有明显的缺口的，比如CC的Agentic Grep无法适用到300G以上的gvfs代码仓库，直接使用会导致动态拉去代码，时间久甚至卡死的操作，包括写出的代码的review的样式，其实不满足我们企业内部的需求，这也是为何我们自己的平台定制了代码review的样式。

第二，现有通用框架对于企业生态的理解，比如内部业务约定，工单理解与跨平台信息串联，也是需要适配的，无法开箱即用。

上面提到的这两点，功能上虽然也许可以通过对CC加Skill，或者Hook机制做二次开发，但是：

1. **二次开发的可控范围有限，比如对于压缩机制**，CC公开的钩子只能在压缩前后加钩子，没法改变压缩过程中的算法，相比自己重新搭建平台，可定制化能力更强。
2. **即使复用已有框架做二次开发，其实不意味着工作量更小。**比如我们项目里的定制检索功能，相关的业务逻辑，其实无论用哪种方案，都要实现相应的逻辑。
3. **搭建一个平台，不意味着所有东西都是自己实现。**我们的项目也是在Pi-Agent上做二次开发，核心的Agent Loop复用了Pi-agent，自己只用定制化相应的业务逻辑，在必要的地方我们也实现了复用。

最后一个原因，我们重新做一个Agent，**其实不光是为了完成Agent的开发，也加入了本地模型，对模型做后训练。模型的后训练效果很大程度依赖于真实的使用数据，我们希望部门内部使用，获取真实数据作为决策，形成一个完整的数据回流训练迭代的一整套生态体系。**后面也许我们的模型通用能力不能超过顶级大模型，但是他是最懂我们业务，最符合企业内部习惯也是严格数据安全的。

8.5.2 RAG项目

你提到的RAG系统，做了针对于文档时效性做的的优化，你能自己讲一讲是什么意思吗?

💡 **答题思路**

> 我看了很多人的简历，包括我自己的，我们做RAG的时候，都是**分块-双路检索-RRF+重排，再加一些Milvus数据库**。这些内容太常见了，或者说太普通了。我希望能够加入一点不一样的设计。包括朋友们，任何做RAG项目的朋友们，我都希望你能想清楚一个问题，就是你的业务背景是什么，要解决什么痛点。因为传统RAG已经很成熟了，你为什么要做，解决什么痛点。
>
> 我这里引入**Timely-RAG的这个特性**，既能够作为一个不一样的点，写在简历上。也可以用来回答你的项目解决了什么痛点这个问题。

理论知识

对于Timely-RAG，我先讲一下是什么，对于这个知识点，我们不会特别深入地展开讲解，想深入了解可以自己看这个参考资料。这里只是先带大家理解Timely-RAG解决的痛点，大概的思路和做法。了解了以后，足够可以写在简历上，面试官问痛点和新技术的时候，可以用来回答。大部分情况下面试官可能不会深入问。时间不够，就不深入学了（这也是我目前的做法），如果深入问了，你又看了论文，你能给他讲的特别清楚，那是很加分的。

Timely-RAG 是什么，解决什么痛点?

Timely-RAG 可以理解为一种**在内容相关性之外，加入时间匹配度的重排方法。**它要解决的问题是：**检索到了相关的内容，但这份内容在用户所问的时间已经不适用，或者还没有生效。**

举一个虚构的例子：某项申请要求在 7 月增加了一份验证报告，知识库里同时保留了新旧两版规范。用户问“现在申请要准备什么”，应该参考新版；但如果用户问“5 月申请时需要什么”，应该参考当时有效的旧版。两版文档的文字高度相似，只按内容相关性排序，很容易混淆。

大概思路：同一批候选，两条评分路线

Timely-Rag存在于传统RAG的重排阶段，传统的RAG的重排，基本只考虑语义相关性，Timely-RAG新增了时间的维度。

看下面这个图：

左边是传统RAGA路线，计算内容相关性 S。 它回答“这段内容和问题有多相关”。接到我们已有的系统时，也可以保留原来的 Cross-Encoder，把它的相关性评分作为 S，可以直接复用我们原来的RAG重排的路径。

下面一条是 Timely-RAG 增加的时间路线，计算时间匹配分数 T。 它回答“这段内容在用户问的那个时间是否适用”。这条路线不是再调用一个重排模型，而是获得时间信息后，通过程序计算时间距离与匹配分数。

```Plain Text
对同一批候选分别评分
        |
        ┌──────────────────────┴──────────────────────┐
        |                                             |
传统路线：内容相关性                          新增路线：时间匹配度
        |                                             |
保留原有相关性评分                          获取问题与文档的时间
                                                      可沿用
```

Cross-Encoder 计算时间距离与匹配度 | | 相关性分数 S 时间分数 T | | └──────────┬──────────┘ | 对齐分数尺度，加权融合 | 按最终分数重新排序

最终，把 S 和 T 按权重组合。**内容相关但时间不适用的候选会受到降分；内容相关且时间匹配的候选更容易排到前面。** 两种分数需要先对齐数值尺度，不能把 Cross-Encoder 的原始输出直接与时间分数相加。论文还会根据问题的时间表达调整时间项的权重，具体公式这里不展开。

T 这条路线，大概怎么做？

先获得论文定义的四类时间：

时间	表示什么	主要从哪里获得
问题提交时间 Q_IT	用户什么时候提出这个问题	系统记录
问题事件时间 Q_ET	用户实际问的是哪个时间	问题文本
文档入库时间 D_IT	该文档版本什么时候进入数据库	数据库元数据
文档事件时间 D_ET	与问题相关的内容对应什么事件时间或有效区间	正文或明确的时间元数据

四类时间不等于从正文抓四个日期。 两类主要读系统记录，另两类需要识别内容中的时间含义；其中的事件时间也可能缺失。时间获取可以依靠元数据、规则解析和离线时间规范化，复杂文本也可以考虑用 LLM 辅助，但这不是论文强制规定的实现。

拿到这些信息后，重点考虑：

1. **用户所问的时间，是否落在相关条款的有效区间内。** 在区间内，这一项时间距离为零；不在，就计算到有效区间的距离。

2. **再辅助考虑提问时间、文档入库时间等时间关系。** 不同时间距离按权重参与计算，不是简单认为“入库越晚就一定越正确”。

3. **把综合时间距离转换为时间分数 T。** 按这套规则越匹配，T 越高。拿到结构化时间之后，这部分是日期比较和数学运算，不需要 LLM 主观打分。

把输入分开看，就更容易理解：

输入对象	内容 / 记录	对应的时间
用户问题	“2026 年 5 月提交申请，需要哪些材料？”	用户所问时间是 5 月
问题系统记录	2026 年 8 月 10 日提出问题	问题提交时间是 8 月 10 日
已检索片段	“本要求适用于 2026 年 1 月至 6 月提交的申请……”	相关条款的有效区间是 1–6 月
文档数据库记录	该版本于 2026 年 1 月 2 日入库	文档入库时间是 1 月 2 日

核心比较就是：**问题里的“5 月”落在片段对应的“1–6 月”有效区间内，因此这一项匹配度高。** 如果另一个片段写的是“7 月起适用”，它与“5 月”就不匹配。完整 T 还会结合前面提到的辅助时间项。

这里有一个重要前提：**时间必须绑定到正确的内容上。** 一篇文档可能同时包含发布日期、材料要求生效日期和费用调整日期，不能把这些日期混在一起。论文给出了时间评分方法，但没有完整展开任意复杂正文中的多日期消歧、条款绑定和多区间汇总算法。这部分仍是落地时需要处理的工程问题；事件时间缺失或含糊时，论文会对受影响的距离项回退到可用的记录时间，而不是编造一个生效日期。

一句话总结：**保留原来的相关性评分 S，再增加时间匹配分数 T，让最终排序既考虑“内容相关”，也考虑“时间适用”。** 它是候选集内的软性重排，不保证所有不适用文档一定被排除，也不能补回原本没有进入候选集的正确资料。

两条路线示意图

🎤 参考答案

我们做这个优化，主要是为了解决一个问题：**检索到的文档虽然相关，但里面的要求不一定在用户问的那个时间适用。**

比如，企业知识库里有一份提交规范，原来只要求申请表和负责人签字，从 7 月开始增加了一份验证报告。新旧两版文档大部分内容相同，普通检索很容易混淆。用户问“现在提交需要什么材料”，应该参考新版；但如果问“5 月提交时需要什么材料”，旧版才是正确依据。所以不能简单地只保留最新文档，也不能只按更新时间排序。

针对这个问题，我们实现了 TimelyRAG ，**在原有的相关性评分之外，增加一个时间匹配分数。**

我们新增的时间路线会获取用户所问的时间，以及相关条款的有效时间，再辅助考虑提问时间和文档入库时间，通过程序计算时间匹配度。比如问题问的是 5 月，条款适用于 1 到 6 月，这一项匹配度就高；如果条款从 7 月才生效，匹配度就低。最后，将传统路径的相关性分数和时间分数对齐尺度、加权融合，重新排序。

这样让系统不仅找到相关内容，也尽量选对当时适用的版本。

补充说明：简历迭代

> 最后再分享一个技巧，我认为简历的编写不是一下子完成的。之前我们展示了简历包装 - 面试前突击的技巧。 当面试了以后，根据面试官的反馈，或者自己技术能力的迭代，不断加深简历。比如这里，在一次面试后，我开始复盘，把timely-rag这个点融入进去了，那么我们就可以同步修改一下我们的简历。

8.5.3 模型后训练项目

1.你在做这个微调本地模型项目的时候，遇到了什么问题，怎么解决的？

导语

像这种，项目中遇到了什么困难，如何解决的。是一个上限很高的回答，而且是一个很高频的回答。虽然在这次面试中，该回答的我都回答出来了，不过面试下来，我就继续挖掘，继续准备。所以这一次面试后我就重新复盘，挖掘了一下这个问题。

💡 答题思路

> 大家看着这个答案虽然是AI写的，但是我其实想了很久。需要思考很多东西

> 1. **立意**，要选一个和你的项目契合的问题和困难。

> 2．**整个流程**：问题发现–排查–解决–可能又遇到了新的问题–解决–验证的闭环故事要想清楚。

> 3．**要讲的具体，可信，合情合理，**所以最好是结合一些特别具体的例子。

> 4．**要展现出深度的，**你怎么排查，怎么解决的，展示这个问题分析的能力，另外最好可以稍微复杂一点，比如像这里，我就引入了KL约束 + 超参数的设置，就是比我之前准备的答案，要往算法层面的知识更靠近了一点。

> 最后总结的你看到的这个答案，可能也不是标准版，如我们所说，能力变强后，我们可以持续迭代这个问题的深度，讲出亮点。

🎤 参考答案

1．项目背景

我们这个本地模型的微调工作是**嵌入在我们ForgePilot项目的工单分类模块的**。ForgePilot的流程是先理解和分类工单，再进入代码定位、修复、编译和测试；验证失败后继续修复，通过后生成PR和报告。达到修复轮次上限，或者出现无法继续推进的问题，就转人工介入。本地小模型负责入口的分类和信息提取，输出问题现象、调查方向和约束，交给后续Agent执行。

2．问题发现

> 我以真实上线数据来举例，很真实，体现我们真的做过，并且项目上线了，处理的是真实数据。这一段引入问题如何发生的

上线持续运行后，我们发现，一类Windows声音设置工单的人工介入率升高。这些工单反映的是**设置界面的显示状态与实际音量不一致**。Agent执行了多轮修改，编译能够通过，但原始异常始终没有解决，最终达到修复轮次上限，触发人工介入。

3．排查与验证

> 开始排查了，展示排查的思考逻辑。

我们把这批失败任务拉出来回放，沿着分类、定位、修改和验证的顺序检查中间结果。发现入口模型将这类工单归为“音量配置异常”，结构化描述中也只保留了音量调整的问题，漏掉了“显示状态与实际效果不一致”这个关键关系。后续Agent因此一直检查输出设备、静音状态和音量配置，没有进入界面状态读取与刷新相关的调查。

为了验证这个判断，我们固定下游模型、代码起点、执行预算和验收条件，**只替换入口的分类与结构化描述**：将调查方向修正为声音设置的显示状态同步问题，并补回遗漏的现象。重新执行后，Agent定位到相关状态更新逻辑，完成修改，通过编译和针对原始异常的验证，最终生成PR和报告。

4．问题定位

> 找到问题了

我们在同类工单上做了重复对照，修正入口后，调查方向和修复结果都得到改善。**这说明入口的分类与信息提取错误，是这批任务反复修复无果的重要原因。**

5. 原因分析

> 继续分析这个问题如何导致的，我们的分类模型为何在这里效果不好。

接下来，我们把分类错误的真实工单与训练样本放在一起对照，发现训练样本通常把问题现象、操作步骤和异常结果写得比较明确，真实工单则包含口语、省略和无关描述，关键事实分散在不同位置。

我们保留失败工单中的全部事实，只将描述整理成与训练样本相近的结构，再输入同一个模型，保持提示词和推理参数不变。重新回放后，原本分错的一批工单能够正确分类，声音设置工单中“界面显示与实际效果不一致”的关系也被完整提取出来。

随后，我们围绕同一组事实构造不同表达，检查模型的分类是否一致。测试发现，规范表达下结果较稳定，加入口语、省略和冗余描述后，关键现象更容易被遗漏，分类也随之偏移。

这组对照将问题定位到了模型对真实表达的适配不足：模型在规范描述下能够识别这些问题，但对同一事实的不同表达缺乏稳定性。 因此，后续数据准备重点是保留事实和标签，增加真实表达的覆盖，并通过成组对照检查模型是否学会了跨表达的一致判断。

6. 数据改进

> 我们已经找到解决方法了，这里开始补充数据集，尝试对应处理，撒花 🌸

我们围绕同**一事实构造规范表达、口语表达和包含冗余信息的表达**，并检查改写没有新增或丢失决定分类的证据。同时补充表面描述相近、关键证据不同的样本，以及信息不足、需要进一步调查的样本。我们逐组核验事实和标签，按原型划分训练与测试，避免同一道题的不同改写分散到两个集合里。

7. 回归问题与处理

> 真实过程没那么一帆风顺，解决了一个问题，引入了另一个问题

补充数据重新做LoRA SFT后，这批复杂工单的分类改善了。但旧场景回归评估又发现，**一些原本能够正确处理的配置问题开始被分流到代码调查**。

我们逐条比较新旧模型的结果，统计新增通过、历史退化和各类别误判。发现新增数据主要来自之前漏识别的缺陷，训练样本向代码调查类场景倾斜。我们先清理重复样本，调整新旧数据配比，保留配置类、信息不足类等旧样本回放。调整后回归减少，但部分旧场景仍然退化。

8. KL 约束与超参数设置

> 这里就是我们专门的设计，解决这个新增数据集，导致回归的问题，引入**KL约束+超参数**的设置，往具体的算法细节上深入我们的答案。（下面有一点KL相关的八股，其实回答的时候不用回答出来）

为进一步控制回归，我们在SFT中增加稳定教师**KL约束**。冻结一份稳定SFT模型，学生从相同权重开始训练。所有样本计算正常交叉熵；对旧模型答对、经核验且业务规则未变化的旧样本，额外计算KL，限制学生在这些场景上的输出概率分布偏移。旧模型原本答错的样本只学习正确答案，不参与这项旧行为约束。

KL约束系数λ控制旧能力保持与新场景适配之间的取舍：系数太小，可能不足以限制旧场景上的行为偏移；系数太大，又可能妨碍模型学习新的工单表达。因此，这一轮参数方案以**λ=0.1作为起始配置，同时设置0.03和0.3作为较弱、较强约束的对照，并加入λ=0的普通SFT基线。**

对照采用相同初始化、数据、采样计划和训练预算，温度固定为T=1。交叉熵按有效监督token平均，KL在合格旧样本的有效监督token上平均，再按λ加权。通过验证集分别检查新场景分类表现和旧场景回归数，选择兼顾两者的系数，再在未参与调参的测试集上回放确认是否达到验收要求。

9. 端到端验证

最后，我们把选出的模型接回ForgePilot，重新执行完整任务，检查分类改善是否落实为无效修复轮次减少、人工介入降低，以及更多任务通过原始问题的验收。

8.5.4 讲解视频

---

9 面试真题（开发方向）

我这里整理了社招:分成**我自己面试**（工作4\~5年经验）和**其他人的社招**。

校招分成了**暑期实习**和**秋招**。所以面经这块主要份4部分。

我个人的看法是，不用过于关注**社招**还是**校招**，**暑期实习**还是**秋招**。因为整体却别不大。考察的内容差不多。不管你是什么背景，校招还是社招都可以互相参考。

不过时效性还是重要的，优先看最新的面经，时间久远的面经有些东西可能过时了。所以推荐**不区分这些校招社招类型，从最新的往前看就行。**

社招

博主本人（工作4～5年）

2025.10 安克创新

岗位介绍

文本大模型算法工程师

【工作职责】

1.持续跟踪业界最新的文本大模型，提升自研多语言LLM的语义理解、推理、指令遵循、工具调用、多轮对话能力

2.构建并不断提升业务子Agent能力，包括但不限于智能助理、知识问答、设备控制、情感陪伴、场景业务交互等方向。

3.持续跟踪业界在RAG、长期记忆、Agent等方面的进展，提升底层核心模块能力

4.构建基于大模型的多租户对话系统中台，承接公司不同产品和业务的优化、落地需求

【职位要求】

1.计算机、人工智能相关专业硕士及以上学历，有NLP、对话等方向算法经验者优先

2.熟悉主流开源预训练大模型，熟悉常用的大模型后训练方法，有相关项目经验者优先

3.熟悉RAG、Agent等大模型系统理论和场景优化方法，具备实际应用落地经验者优先

4.熟练应用torch、LLaMAFactory、OpenRLHF、VERL等一个或多个框架

反思

一面是通过了的，一面完了需要做IQ,EQ题，逻辑题 （类似于公务员考试的逻辑题，大家网上一搜就知道，比如给你前三个图像，让你判断下一个图形是啥，图形推理题之类的）+ 1道编程题（要求用AI，然后会记录你给AI的提示词），需要达到60分，否则直接不通过。 我就是这个题没做好，所以约不了二面。

最终结果是：一面过了，但是过了一面之后有一个测试题，类似于IQ, EQ 那种，因为本身没太想去，其实也没太认真对待，这个测试挂了，如果真的想去安克的，注意这里自己要复习下，这个没过直接没二面资格。

一面

自我介绍

1. 智能日历系统用的什么框架去做的？

1.1 这个架构底层使用一些什么框架做的？单一Agent还是多Agent？

（可以讲当google的《Agentic design patterns》中的prompt chaining来丰富答案。）

使用的是单一Agent。 使用Prompt/Pipeline模式，将日历场景分而治之，拆分成更小的，更易管理的子问题，每个子问题设计专门的提示单独处理，系统是一个多步llm的pipeline，但还是属于一个单一Agent。

2. 日历系统底层是有一个接口，它是mcp server还是原始的server，你这里进行了进一步的加工呢？

（我认为这里面试官以为我的项目Agent是一个server端，但是我的项目其实是客户端，集成的MCP Server，所以这里主要是做了一个澄清和解释。）

3.日历系统中的增删改查这些逻辑的操作是如何做的？

（其实就是Agent接入的工具，有一些工具是本地的API，有一些工具是集成的MCP中的tools，总之这里就是和面试官解释清楚就好）

4.如果真实用户想要删除某一个事件，但是这行程很重要，你不能直接删，这个怎么办？

（这个其实是项目体验一些的问题，解释清楚就好了，通常Agent调用工具的时候，都会有一个弹窗，让用户选择是否进行下一步）

1. 如果用户输入了某个事件，可能是一个模糊的情况，会有多个类似的事件，怎么确定是哪一个。

(这个其实是功能相关问题的澄清，不做解释，大家不用参考，我记录只是为了完整性)

1. 你这个Agent中集成了哪些工具？

1. 这个过程中有几次大模型的调用？
2. 整套的流程是完全交给大模型，还是自己会有一些实际的控制？
    1. 如果交给大模型，那你怎么去知道它执行的结果是否正确呢？

（这里就可以回到知识点，Agent的评估，在Agent的评估质量指标中，有一项指标就是去监测Agent路径/轨迹的正确性。参考《Agentic design patterns》第19章Chapter 19: Evaluation and Monitoring (code)， 里面有一小章节衡量 Agents trajectories 其实就是这一问题的答案）

我们需要分析Agent**执行的轨迹**，**工具执行**的情况，包括为了达成目标所采取的步骤。然后将实际操作和预期进行比较。包括可以自己指定匹配的标准：完全匹配，顺序匹配（动作顺序一致，允许额外步骤），任意顺序匹配（动作正确即可，顺序不限）

1. 怎么去监测这个路径是否正确呢？

（依然参考《Agentic design patterns》第19章Chapter 19: Evaluation and Monitoring (code)。 这小节在讲完要评估agent的轨迹后就讲了如何评估）

我们可以test文件，可以使用json文件格式。 每个test文件记录某会话中，每个回合的用户请求，**预期的工具调用轨迹**，中间响应，最终响应，最终的结果以及预期的结果。通过测试文件收集不同场景下的运行结果，整理最终轨迹评估指标。

1. 数据集是怎么构建的？

有几种方式可以创建数据集：

1. 手动设计：人为标注生成高质量的数据集。

2. 使用LLM辅助生成，人为干预检查。

3. 从真实用户的日志中收集请求->筛选高置信度轨迹

9. 简单介绍你的项目按摩房智能预约系统，怎么去做的？

10. 你的RAG系统中数据库是如何存储的？

（想清楚你的系统，RAG数据库存的是什么？怎么存的？ 为什么这么做？）

整个系统采用Azure OpenAI - text-embedding-ada-002生成**嵌入向量**，使用FAISS 生成**索引**，最后将他们存在**sqlite**当中。

采用**sqlite**去存储对应的知识库的嵌入向量和索引，因为数据量不大，sqlite更轻量级。(如果数据量大了就要使用

Milvus/Pinecone/Weaviate)的专业向量数据库。

11. 你的RAG系统是怎么去评估系统质量的呢？

（见RAG系统 RAG评估指标部分）

12. 我们用的Bert 更多一些，你知道Bert和GPT的区别吗？

（见LLM模型结构 – Bert VS GPT 部分）

13.Agent从用户输入到第一次输出，大概的时间是多少？

14. 过往训练中有没有做过大模型微调/训练相关工作？

15. 简单介绍一下DeepResearchAgent 的框架？

代码题：Leetcode 240

视频讲解

https://www.bilibili.com/video/BV1goAGzCEVp/?vd_source=144bec9c3f54e465073138bed788be1b

2025.11 灵动时刻

岗位介绍

反思

全程30分钟，没有自我介绍，没有代码题。秒挂。

大概原因我认为一个是多模态的能力，另一个是大模型算法知识答得不好。

另外我觉得这个面试官可能也不是特别想招人，30分钟草草结束，相比于大公司一般有很多面，考察不同维度，综合结果去进行评估，小公司的招聘流程显得就有点随意和草率，甚至有一点感觉面试官有点是在套方案，也并不是很想招人的样子。如果想招，感觉面试官是想招一个有**Agent/RAG相关经验，落地经验**，也懂**多模态**，也懂**算法**，最好一上去就能直接干的选手。

一面

1. 你之前做的这个实时通信系统，底层通信协议是什么?

（最早在腾讯做的项目，和大模型无关，大家无需关注）

2. 你重点讲一下智能日历系统是如何用AI 去串这个流程的？

（这个还是结合自己实际项目去说，我的这个项目用的是 prompt chaining模式，（谷歌的《Agentic Design Pattern》

我基本围绕这个去讲的）

3. 你认为你讲的这个开发模式是否有一定的范式和通用性。

（当然有了，这种模式本身就是Google定义的常见的框架/Agent开发模式）

Prompt Chaining是**具有范式和通用性**的，它是一种线性执行的设计模式，适用于将复杂任务拆解成多个连续的LLM调用，每一步输出作为下一步输入，不依赖于多个Agent协作，通过单一代理在多个阶段逐步处理任务，常适用于多轮推理，任务分解的系统。

4.现在Agent有很多方向，比如生成代码的方向，以及生成图片/视频的方向，你想做哪一个方向。

（我理解是纯文字处理的Agent， 以及多模态相关的Agent，这个公司肯定想要倾向于后者的，然后我确实没做过图像/多模态，所以回答肯定不让面试官满意）

5.你们研究生项目当时采用深度学习做的还是传统算法做的？

（这个就扯太远了，对大家没有参考）

6.语言模型网络结构为什么编码很快，解码很慢？

编码器和解码器区别

7.照你所说，Transformer是一个比较并行的结构，但是大家拿他来做自回归，它完全是一个串行的，这个会使得他利用率有很大问题，你能谈一下你的理解吗？

（先承认这一问题）对的，Transformer的优势在于**编码阶段可以高度并行**，可以通过**自注意力机制同时**处理整个序列的所有位置，使得训练时效率，GPU/TPU利用率很高。 但是推理阶段，**如果采用自回归生成**，每次只能预测一个token，再把它拼接，才能预测下一个token，因此无法并行，延迟比较严重。

（强调必要性）但是由于语言的自然属性，**它是强序列相关的，未来的token分布依赖于过去的token**，所以**非自回归训练的生成质量往往不如自回归**，容易出现流畅性和一致性问题。所以常见的大语言模型还是使用自回归。

(解决思路)但是我们仍然可以通过比如K**V缓存的方式减少重复计算**，**提升推理速度**，**Speculative Decoding的方式来减少串行步骤** 从而做出一些优化。

8.你如何使用一个Agent来做一个对话系统？

8.1 如何让这个对话系统可以保持记忆功能呢？

5 Agent的记忆功能是如何实现的？

8.2 那这个记忆会越来越长，要怎么办呢？

我们会维护这个上下文窗口，当超过指定窗口的时候，会进行总结，提取关键信息和要点，因此这个记忆不会无限制增加的。

8.3你如果每次都会带上之前的上下文，那他的时间复杂度会不会成倍增长？

理论上来说，基于transformer的架构，计算复杂度和上下文关系成二次关系O(n^2)，意味着上下文翻倍，成本计算会增加4倍。

但是实际上，我们可以使用一些比如**稀疏注意力**，**分块处理，缓存机制以及GPU/TPU的并行计算**的能力来缓解这个性能问题。

9.你这个RAG数据库中的向量是怎么拿到的?

系统采用Azure OpenAI - text-embedding-ada-002生成**嵌入向量的。**

视频讲解

我用夸克网盘分享了「灵动时刻一面.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/dd1e85265158

2025.11 滴滴Agent开发工程师

岗位介绍

工程方向

我们是滴滴网约车 MPT 部门的大模型工程团队。团队与顶尖算法科学家紧密合作，致力于构建稳定、高效、可扩展的大模型应用基础设施，将前沿 AI 技术转化为可靠的线上服务，打通数据、技术与业务的“最后一公里”。

我们的使命是驱动 AI 在智能运营与核心交易场景中的应用革新，通过坚实的工程力量，让大模型真正赋能业务，提升亿万用户的出行体验。

在这里，你将不是 AI 的使用者，而是 AI 服务能力的构建者。

工作职责

- **大模型应用开发**：设计和开发基于大模型的应用系统（如 RAG、Agent 等），将算法原型转化为高性能、高可用的工程产品，解决实际业务问题。
- **AI 应用架构设计**：构建和优化大模型应用的服务架构，包括服务编排、工具调用、多模型协同等，确保系统的高性能和可扩展性。
- **AI 数据链路建设**：负责大模型应用所需的数据链路，包括非/结构化数据处理、向量化、存储与检索等，打造高效、稳定的数据处理管道。
- **服务稳定性保障**：保障线上服务的稳定性和高可用性，建设完善的监控、告警和应急响应体系，持续进行性能优化（如延迟、吞吐量）。
- **技术落地与协作**：与算法团队紧密协作，理解模型能力与业务需求，共同制定技术方案，推动从模型到服务的快速迭代与落地。

任职要求

- 计算机、软件工程等相关专业本科及以上学历，拥有 1-3 年后端开发经验。
- 扎实的编程基础，良好的数据结构与算法功底，熟悉 Linux 开发环境，熟练掌握Go / Python / Java 中至少一种语言。
- 熟悉常见后端技术栈，具备微服务、分布式系统（如缓存、消息队列、RPC 框架）的设计与开发经验。
- 具备良好的系统设计能力，能够独立分析问题、设计方案并推动落地。
- 对大模型技术有浓厚兴趣和热情，具备出色的学习能力和技术钻研精神，愿意在该领域深耕。（我们尤其欢迎这一点）

加分项（具备以下一项或多项者优先）

- 有 RAG、Agent 等大模型应用的实际开发经验，熟悉 LangChain 等主流应用框架。
- 有大模型服务部署、推理优化（如 vLLM）或 MLOps 相关经验。
- 有高并发、高可用系统设计与实践经验，主导或核心参与过复杂线上系统的稳定性建设。

面试点评和学习建议

整个过程面试官翻来覆去都在探讨Agent/传统业务之间的区别，Agent真正存在的意义，开发Agent和开发后端框架有什么区别。整个过程偏向于和面试官进行思维的碰撞，交流，这些问题可能没有标准的答案。需要能自己深入思考，有过一些业务经验和理解，才能够答出自己的想法。我在面试的时候也谈了一些自己的想法，不过面试官可能觉得很不满意。这一部分我就没有总结标准答案，大家可以自己思考，从知识点的角度，参考不大。唯一我认为可以作为复习准备的是第8，9个问题，我们可以结合美团龙猫的最新论文去回答，并且建议大致阅读论文，学习这个论文中的评估Agent效果的方法：如何准备测试集/如何评估。

一面

1. 怎么看待大模型场景的Agent，比如说我们在大模型之前写workflow，现在换成了Agent，或者之前是小模型换成了大模型，仅此而已。你认为真的Agent相比过去解决了更多的问题吗？为什么？

2. 你认为之前的传统行业写workflow和现在的行业agent，有本质的区别吗？

3. 你去构建一个Agent架构或者系统的时候和传统的后端架构、业务架构相比，有一些什么不一样的地方吗，有哪些是要额外关注的地方吗？

4.你提到性能问题，你之前做Agent的时候有提到大模型推理加速相关问题吗？

常见的推理加速有以下手段（我复习的重点先是应用-再是微调：SFT强化学习/模型理论基础，我认为这个推理加速的手段偏离应用太多了，除非其它都学完了，否则可以暂时先放着）

算子优化：用更高效的矩阵乘法、卷积实现（比如 cuBLAS、FlashAttention）。

图优化：把计算图融合、消除冗余（ONNX Graph Optimization、TensorRT）。

量化（Quantization）：把 FP32 降到 INT8/FP16，减少计算和显存占用。

剪枝（Pruning）/稀疏化：减少不必要的权重或连接。

编译器加速：用 TVM、XLA、OpenVINO 等编译器生成高效代码。

批处理/并行调度：多请求合并、流水线并行、张量并行。

缓存与 KV Cache 优化：在 Transformer 推理中缓存注意力键值，减少重复计算。

硬件适配：针对 GPU、CPU、NPU 做特定优化。

5.你做Agent系统的时候，你有没有遇到一些比较深刻的问题，或者设计Agent架构的时候怎么设计的？比如比较复杂的问题，

很难解决的问题。

我这里提了Multi-agent架构和prompt -chaining架构，还是我一直提的《Agentic design pattern》中对应的两个小节的内容。解决复杂的问题是深入Agent多轮对话框架，去探索多轮对话Function calling能力的bug，不过显然面试官不感兴趣。

6. 你刚才提到了Multi-Agent，你认为不能用传统的业务逻辑来解答？为什么一定要用Agent来解答。

7.传统的业务后端框架，我们通常是将业务问题抽象成一个标准化的流程，设计成比如算子化等标准范式，这是传统解决大模型的思路，在大模型场景里，在大模型场景化中，可能很多你需要在原来的工作环境中去抽象或者标准化的东西它自己已经搞定，它不再需要你去设计什么，那这个时候在Agent中，就变成了下限很低，上限很高的东西，有些人可能就根据可视化的界面拖拽就创建了一个Agent，RAG，那这个时候你觉得真正需要工程在里面参与的部分还存在吗？或者说你觉得在这里面到底要解决一个什么问题？还是说从业务的场景下就退下来了，真正去做像coze, dify 这种平台性的架构就可以了。

8.你最近有没有看过一些和Agent相关的前沿的技术报告，论文，或者先进的技术。

9. 除了Agent框架之外，比如更偏算法相关的东西比如论文和技术报告，比如前段时间有大厂推的针对于Agent的模型的训练，我不知道你怎么理解在大模型的场景下如何去增强Agent的效果。比如我们认为一个复杂的项目在一个规划的阶段规划它大概做哪一步，用哪一步，哪一个工具是好的，我们一直认为这件事是不可训练的？你怎么证明某一步的工具调用是对最终结果有帮助。因为我觉得有可能一个复杂的任务，你只是说这一步工具的调用是有利的，但是你很难说对于最终的结果是有帮助的。像很多的Agent领域给的一些比如美团的龙猫，他们都在证明他们做的优化对Agent是有帮助的。我不知道你怎么理解这个事，有没有看过相关的事，你觉得工具的调用本身是可训的吗？

见笔记部分：《3.1 如何评估Agent工具调用对系统产生的影响》

10.算法题

寻找众数。并且要求不存在众数返回-1.

（OpenManus, LangManus）

视频讲解

我用夸克网盘分享了「滴滴Agent开发工程师一面.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/06759119a110

2025.11 京东零售平台算法开发

岗位介绍

没有jd，以下是HR的一些介绍

京东零售平台产品中心，交易算法团队。

有大模型应用，比如生成式推荐，生成式对话，意图识别，多智能体。

反思

这是一个算法工程的岗位，但是全程面试官其实没有太问算法的问题，后来和我说是因为看我过往的经历都是在做业务，没有太为难我。从面试的问题来说，都是一些比较RAG/Agent领域比较基础，也比较重要的知识点，这块比较有参考价值。 另外考察的某些问题也是属于传统行业的基本知识，比如并发。 在一面和三面过程中，也用不同形式考察了算法题。 虽然没有问大模型算法的问题，但是其实很多问题也问了你有没有一些算法方面的经验。 不难看出这次面试官比较注重的点是：

Agent/RAG 基础知识，经验。

模型算法相关经验：推理，微调，强化学习。

（Leetcode题目）

传统的开发知识

一面

1. 自我介绍

2. 你的第一段工作经历用的是什么技术，什么语言？

3. 在第二段工作经历用的什么语言？你的工作和Langchain, Langgraph有什么关系。

4. 你开发的Agent是用在客户端上还是用在哪里？

5. 你这个大模型是单机版的还是说由一个后台来服务的？

6. 微软也有一些开源的Agent开发框架，你了解吗？

（代码平台： AutoGen, Semantic kernel。 低代码平台：Copilot studio，Dify）

参考笔记部分：1.1[常见的Agent开发框架](#doxcnVbEfVZ4QrE0dykkw5aNewb)

7. Langgraph的流程编排是如何实现的？这些基本组件之间是如何配合的？他这个图是怎么运行的呢？

参考笔记部分：1.3.2 [LangGraph 过程是如何流转](#doxcnvn087994bcNgFTwxkOZmAb)

8. 讲一下Agent框架如何管理过去的记忆，我们怎么去管理历史记忆的状态?

参考笔记：5[Agent的记忆功能是如何实现的](#doxcnjAkMolTz0puV6LTjlJexlb)?

8.1 长期记忆信息量很大的时候，你怎么处理？

（ 如果我们理解了上一个问题中，长期记忆如何实现的？ 这个问题应该很好回答，只是向面试官具体再讲解一下长期记忆具体的实现）

在Agent的系统中，长期记忆确实可能数据量很大，所以我们可以使用将过往历史记录，使用向量数据库的方式进行存储。使用向量数据库中，天然的就是会对数据进行压缩，可以解决一部分历史信息过多的问题。

在我们要使用这个长期记忆的时候，我们就可以通过向量数据库的语义搜索的功能，召回相关的感兴趣部分，输入给大模型。

8.2 你说到匹配的时候，比如说你根据“今天天气怎么样”，想要召回的答案是“今天天气很好”，那你这个时候是根据问题召回答案，这能召回得了吗？

（参见笔记部分： 稠密向量和稀疏向量 对称检索和非对称检索）

召回得了。这就是典型的非对称搜索，你讲一下你的理解就好了。 因为这一部分和稠密向量/稀疏向量 密不可分，所以可以顺带讲一下对应的稠密向量和稀疏向量，以及对应的模型。

8.3 你讲到这个消息是片段式的。比如用户问今天天气怎么样，假设他问的时候是10月30号，数据库中有很多条答案都是在回答今天的天气的，有些回答是回答的10月1号，有的是10月30号，你怎么保证你召回的回答是针对10月30号呢？

（这是一个综合的问题。只要能理解我在笔记RAG 部分整理的。召回优化策略，其实你就可以从不同的召回策略中入手，解决这一问题）

比如说，采用 **内容查询优化 +结合元数据召回** 。

内容查询优化： 当用户的提问过于模糊的时候，我们通常会使用一些查询优化方法，扩充上下文的信息，对查询内容做处理。 比如对于这个问题，我们做了处理以后，原来文本 "今天天气怎么样" 就会变成 "10月1号的天气怎么样"。

再使用结合元数据召回的方法，在每个召回文档中，元数据会有一些标签，比如日期，结合这两个技巧，就能够准确实现用户问题的召回了。

9. 提到RAG，如何提高召回的准确性？

各个方面都可以答。比如从**切块策略/召回优化策略/ 数据库的选取/ 检索的方法** 等各方面都可以去回答。这个题太宽泛了，没有固定的标准答案，但是不难。就是我在RAG里总结的RAG的各个步骤都理解了， 就会发现可以从很多地方回答。如果不会，就先去理解一下RAG 基础知识和我整理的笔记部分。

如果还是不会答，那你其实就答笔记部分，RAG 常用的召回策略部分就行。

9.1 讲一下切块的策略是怎么切的，有什么策略，各有什么优势？

参考笔记 [RAG 切块策略部分](#doxcngNq4Ri4NGk7pDlPLvcKRre)。

9.2 把一个问题转换成query这里有一些什么优化的方法？

可以去讲 **内容查询优化方法**，参考笔记 RAG [召回优化策略部分](#doxcnZWhvaikGfyosAnvDQYTpQf)。

10. Langchain里拿到这个流式响应，怎么实现的，使用的一些什么协议？

（涉及的点很多： **Langchain的runnable, Python的异步迭代器**，以及传**输协议SSE**， **Websocket, Http Chunked**. 如果都弄得很明白需要花一些功夫，时间不够的同学可以先理解掌握本题，具体深入的细节可以先放一放）

在 **LangChain** 中，所有核心组件（模型、链、Agent 等）都被统一封装成一个抽象类 **Runnable**，它定义了标准调用接口，包括 `invoke()`（一次性输出）和 `stream()` / `astream()`（流式输出）。流式输出的这两个方法通过 Python 的异步迭代器，让开发者可以用 `for` 或 `async for` 循环逐块消费生成的内容，实现“边生成边处理”的能力。

更底层实现分为两层：

- **后端内部**：LangChain 迭代器从底层模型获取 token 或片段，前提是模型本身支持流式生成（如 OpenAI 的 `stream=True`）。
- **后端对外传输**：将这些 chunk 推送给客户端，常用三种协议：

  1. **SSE（Server-Sent Events）**：基于 HTTP 长连接，单向推送，浏览器用 `EventSource` 接收；
  2. **WebSocket**：双向实时，适合需要交互（如停止生成）；
  3. **HTTP Chunked**：普通 HTTP 分块传输，前端用 `ReadableStream` 读取。

客户端在 Python 或后端服务中消费流式响应时，本质就是遵循迭代器语法，用 `for chunk in runnable.stream(...)` 或 `async for chunk in runnable.astream(...)` 来一点一点接收并处理数据。

11.如何提高获取Agent响应的并发度？

（这个问题就很空泛，可以回答的点很多。和大模型无关，是传统开发的并发策略方法。 首先针对于服务端/客户端有所不同，

客户端的话这里并发大概是要同时调用多个Agent API获取结果，是一个**I/O操作**，不需要**密集CPU计算**，那么可以从线程/协程/进程/线程池/消息队列 等典型的技术切入和面试官讨论。 这里可以看一下这个文章

https://blog.csdn.net/m0_62006803/article/details/134862520

如果是服务端的话，可能要构建一个**Agent网关或者平台，需要考虑一些Worker池，负载均衡之类**的问题）

这里就以客户端来回答。

在客户端，如果需要同时调用多个 Agent 的 API 来提高并发度，常见的并发方法包括：**多线程、多进程、协程、消息队列、线程池**等。

由于调用远端 API 属于 **I/O 密集型操作**，因此：

- **多进程**并不适合这种场景，因为它更适用于 **CPU 密集型任务**，而且进程切换的开销较大。
- 如果系统结构简单、并发量不高，可以使用**多线程**来快速实现并发。
- 如果追求更高的并发效率，推荐使用**协程**，因为协程更加轻量，资源占用更低，适合处理大量并发的 I/O 请求。

消息队列则更适合用于**任务解耦、异步处理和高并发任务调度**的场景。它通常用于构建更复杂、可扩展的系统架构。

算法题：

给你一个整型数字target和数组nums，从数组任取N(N是任意值)个元素让这N个数的和等于这个target，返回有多少种方案？

（动态规划）

二面

1. 如果是说Agent的响应速度很慢，这个时候怎么办？

（我一开始回答的是使用**流式方法去**处理，后面面试官是说流式可能也很慢，我就回答了**封装成一个异步**的方法**不去阻塞**，我在回答这个题的时候，这么去讲的，面试官也是接受，所以问了1.1的问题。其实我的思维就是偏向于工程。但是我后面反思，其实如果真的是从大模型的行业出发，这个问题其实属于推理加速，从这个角度上出发回答会更好，特别是如果面试官背景是**大模型算法工程**相关，他一定期待听到推理加速相关的回答。所以这个题又建议从推理加速部分回答：基本就是回答大模型常见推理加速方法，如何选择之类的，详细参考我的八股部分。）

参考笔记部分 1.[常见的推理加速的方法。](#doxcnSEwogAtoxuGrsPdoY3iVec)

1.1那你设计成异步方法的时候，如何通知响应完成？

参考笔记部分 2 [Pyhton的异步方法是如何实现的](#doxcn8uYBljgrHkv3oR6wWh3Wfe)

2.Langgraph里面的核心组件有哪些呀？

2.1 那它这里面的上下文是如何管理的？

（最基本的八股了。问了很多次，这里最好我们能记住对应的langchain/langgraph对应使用记忆功能的核心函数，显得更加熟练）

可以参考笔记 5.2 [介绍Langgraph中记忆功能的实现](#doxcn78ad0R5cdVnbM9V270xpEd)

3.你是在哪些场景下会使用到A2A协议？

（A2A这个协议感觉现在不火了，我感觉它是新出的协议，当时火过，但没火起来，现在出现概率已经挺低了。问起来也是因为我简历写了，面试官才问，而且是第一次问。 我倾向于这里大家可以先不复习了，或者背一下概念得了。特别是如果你基本项目都没用过，面试基本不问。）

A2A协议通常用于**企业内部或跨系统的应用程序之间进行安全、自动化的数据交换和通信**。它的核心场景是**系统集成**，尤其在需要**高安全性和自动化**的环境中。比如说**跨平台、多厂商智能体协作。**当企业需要整合来自不同技术栈或供应商的智能体（如 Microsoft Copilot 与 Google Workspace Agent），A2A协议提供标准化通信方式，解决互操作性问题。

4.解释一下你项目中使用的MCP协议？

参考笔记，4.[简单介绍MCP](#doxcnPy6vIWYOWgb1IounSUwKwh)

5. 你在大模型微调这里一般怎么去做的微调？

（能够讲清楚自己做微调是怎么做的，整个流程怎么准备数据集，怎么选参数，怎么看指标，怎么部署）

这个涉及的知识点很多，比如以Lora 微调为例，涉及到Lora的开始到结束整个流程，需要确实知道整个过程到底怎么做，结合自己的例子，具体的平台，给面试官讲清楚，让面试官知道你真的做了这些事）

参考笔记 **8.**[**你的项目是如何做推理加速的**](#doxcnaaHJ978A9HZpljzbTB5jxN)

6.你的项目xx 主要是实现了哪些功能？

6.1 你这里面功能的实现是通过prompt 定制好，还是通过训练模型达到的？

6.2 你这个背后的功能是有一些训练集的吗？

7. 过往在xx工作，你为什么要从xx离职？你为什么突然要转型？你在这一年做了哪些技术上的提升和储备？

8.你未来是倾向于应用端偏工程运用还是偏算法，比如蒸馏，底层的模型训练啊？

8.1 面试官补充，这也不算一个问题，面试官说如果你真的是去做大模型底层的那些东西，是有一定难度的。就像你刚才说的，你的应用中模型都是去调用一些别人的模型，你也不知道里面的具体的细节。

（这是一个算法岗，但是面试官一直都在问我工程问题，因为他也知道我过往都是做的工程，没有为难我，但最后还是问了一些我对算法方面掌握程度）

9. 过往晋升情况和绩效。

10.反问环节：我问了一下面试官对我的意见和评价

终面（主管+HR）

反思：

这个主管，如果我进去以后，他会是我的+2了，其实等级是比较高的。通常来说这么高的等级的领导，不会太懂细节，更关注的是一些综合方面的问题。其实从这次回答可以看出来，这个老板基本上就是在考察你的经历，问你Agent 能力，微调能力，有没有做过强化学习，然后再问几个算法题，从各个方面考察你是否有相关经验。

其实这样子反而并不会很难，因为他不会深入，更多的想考察你会不会，做过没，至于深入的技术细节，其实他大概率不了解，这个非常能理解，毕竟很高的管理岗位，不会再对技术细节把控那么强。所以整体来说，这种面试不用怕的，按照我们的这个路线，其实所有他考的地方，我们都有准备。对于强化学习，推理这一块，我们路线虽然学得不深入，但我们也是有备而来，还是这句话，不要虚，大胆去答，不要觉得那一块你掌握不深入就畏惧，要肯定的表达，告诉自己xx我做过，我是怎么做的，不要露怯。 这次面试也是一次通过并拿到offer的面试。

（主管面 + HR 面）

（首先主管面）

1.自我介绍

2.未来你的职业发展，有想过去开发一些自己的算法模型吗？

3.你硕士是哪个系的，本科是哪个系的？

4.你评价你的预约系统Agent好坏的标准是什么？

（这个问题必考，我也借这个机会重构了Agent性能评估的八股，有详细的参考资料，思路和问题，视频讲解里也有一些额外的技巧和感受，大家可以参考八股和我的视频）

参考[ ](#doxcnbpKwJs393fsRaDVMYRg89c)[Agent 性能评估](#doxcnbpKwJs393fsRaDVMYRg89c)

5.使用Langchain的时候有没有遇到一些不方便的地方？

（面试中问Langchain方便还是不方便的地方都考过，这里的八股我已经总结到了，算是有备而来）

参考[1.2.3 LangChain 不方便的地方](#doxcnI3l1ornLUnZRuFBOYxacDQ)

6. 你的这个Agent项目，你这个任务完成得怎么样，有没有去学习呢？

（这个应该是想看Agent有没有反思和学习的能力，问你系统中有没有做，所有的项目在设计的时候，最好可以加入这个特性，具体实现也不难，我推荐的这个《Agentic Design Pattern》中有两章都讲到了，这两章都可沾边，去仔细看，这个也是Agent比较重要的特性，还讲了如何实现，可以看一下 https://github.com/ginobefun/agentic-design-patterns-cn/blob/main/10-Chapter-04-Reflection.md

https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn/blob/feature/interactive-website/17-Chapter-11-Goal-Setting-And-Monitoring.md）

7. 你的系统中使用LoRa做微调，是怎么做的，用的什么样的样本？

（7，8，9就是分别问你做了强化学习/RAG/微调没，也不深入问，他就想看你到底有没有经验，至于多深，+2可能也不懂过多

技术细节，所以自信回答就好了，我的学习路线，这些都学了，都做了，我们是有备而来的）

参考 1.1 [简单介绍LoRa技术](#doxcnGi0Cpyv3Xl733a1YoNR2sh) 和 [你是怎么做微调的](#doxcnXE4sR8p50zqg2mzVEJDSVd)

8. 有没有使用一些RLHF 的技巧？

（强化学习，我一直说，让大家准备一套，把流程走通就行了，我的项目中也做了，参考 8.[你的项目是如何做推理加速的](#doxcnaaHJ978A9HZpljzbTB5jxN)？ 和 2.[强化学习的方案有哪些](#doxcn67JGP5g6dBtzf7ejm8ekPh)）

9. 有没有使用一些知识问答的功能？

（就是问你做了RAG没）

10. 一些公司做的其他问题的询问，和agent无关，就省略。

11.大概说一下Lora微调的一个原理？

参考 1.1 [简单介绍LoRa技术](#doxcnGi0Cpyv3Xl733a1YoNR2sh) 和 1.4[你是怎么做微调的](#doxcnXE4sR8p50zqg2mzVEJDSVd)

12.算法题快速问题：

1.找出最长的无重复字符串

2.编辑距离

13.智能日历Agent的架构询问。（又继续问了一些具体功能）

14.使用各种语言的熟练度

（接着HR面）

1.两次跳槽的核心原因

2.现在居住地

3.现在的工作强度和作息

4.过往绩效，晋升情况

5.是哪里人

（反问，你有什么问题）

视频讲解

三面

https://www.bilibili.com/video/BV1eT9nB2E3J/?vd_source=144bec9c3f54e465073138bed788be1b

二面

https://www.bilibili.com/video/BV1eT9nB2ECK/

一面

https://www.bilibili.com/video/BV1vP9nBUEvN/?vd_source=144bec9c3f54e465073138bed788be1b

2025.11网龙科技

岗位介绍

岗位2：教育线/休闲游戏-高级软件开发工程师-AI（Python/Java）-事务AI化-通用AI能力研发

工作职责

1、参与现有AI技术能力的落地，能够独立完成基于AI模型的应用设计与研发工作；

2、参与AI项目中台建设，制定技术方案和架构设计，解决技术难题，以确保系统的高效运行和用户体验的优化；

3、开发、部署和维护机器学习工具和平台，支持数据标注、数据集管理、模型训练、评测和分析等工作；

4、制定代码规范、测试规范，进行关键代码开发和系统调优；

5、能够利用AI技术赋能业务，基于现有AI模型在新业务上进行应用探索，推动企业AI应用的前沿发展。

任职资格

1、本科及以上学历（硕士及以上学历优先），计算机软件及人工智能等相关专业；

2、2-3年及以上应用开发相关工作经验，若有AI相关应用开发经验优先；

3、精通Python或Java等编程语言，具备扎实的编程基础和代码优化能力，熟悉机器学习、深度学习算法优先；

4、具备系统架构设计能力，能够设计高效、稳定的后台系统；

5、熟悉数据预处理、数据集管理等数据处理技术和工具；

6、能够熟练运用当下大模型赋能日常工作；

7、具备良好的问题分析解决能力、沟通能力及团队协作精神，能够与业务团队紧密合作；

8、保持对最新AI技术和研究动态的关注，具备持续学习的能力。

寻访要求

1）是否为一线实际开发同学，写代码/解决具体算法问题的工作占比要求超过日常工作的70%以上

2）是否掌握Python开发语言？

3）是否有Agent或RAG的项目经验？

4）是否掌握大模型推理框架，如XInference/NIM（英伟达的AI微服务）（1\~10分，至少需要在6分以上）

5）是否了解dify.ai/FastGPT/LangChain等主流AI开发框架（1\~10分，至少需要在6分以上）

以上1\~3为必须项，4、5满足一项即可

点评

网龙的面试只有一面，这一面是两个面试官一起问的。问的问题是目前我接触到面试面的最细的，他基本上是问到底，一般面试都是让你讲一下大概的方法是啥，这次面试面试官基本上要问到底，不光问用什么方法，还问怎么用的，甚至还问这个方法底层的细节，这个方法跑出的结果和评分。

这次面试有的地方我觉得并不是答得很好，也有答不上来的地方，但最后的结果反而面试官给的评价很好，很想给我发这个offer。 可能有一点压力测试的感觉吧。但无论如何，我觉得这次面试收获挺大的，因为第一次面试官问的这么深入，是一个很好的素材让自己对知识有一个更深入的理解。

一面

1.自我介绍

2. 你的项目使用了什么分块策略以及为什么要使用这个分块策略？

（必考题了，没啥说的）

2.1 对于一些技术文档，对于同一个问题，相应的召回文档可能会跨块，这种情况下你怎么去优化这种分块策略？

递归字符分块器：按照语义切割，保留完整性。

块之间有重叠以及一些常见的召回优化策略：**召回内容上下文扩充** 都是可以去避免这一问题的。

2.2 你刚提到了使用**召回内容上下文扩**充的技巧，那什么时候需要采取这种策略呢？

（可以看出面试问的很细，是会一个问题追着问的，这个问题在《大模型RAG实战》第4节讲得很清楚，我直接用书中原话作为回答）

在RAG场景下，通常将从向量数据库中召回的文本直接输入到LLM中。然而，由于向量化模型对长文本的编码有较大的信息损失，因此通常从向量数据库中召回的是短文本，但是对于LLM来说，连贯且包含更多上下文信息的长文本能够帮助总结出更好的答案。直接召回长文本可能比较困难，这时可以通过召回内容上下文补充的方法， 打破 "召回内容和输入LLM中内容相等"的约束，**解决向量化模型需要短文本而LLM需要长文本的问题。**

2.3 在你的召回的系统中，文档分块会做一些滑窗吗？滑窗的长度一般是多少？

（还是觉得问的很细）

一般来说分块之间的重叠区大小占整个分块的10% \~20%

2.4你的系统中有没有一些做一些高级的召回或者分块的方案？

（说实话我复习了这个召回的优化方案，但其实我的系统里没有使用到。 不过根据这个问题，我私下里会思考我的系统中需要使用什么样的召回优化方案，我下次回答就不会说没有了）

2.5你提到了多轮对话的中的召回查询改写，那么在你的项目是怎么做的实现呢？

（这里我提到了召回方案的优化技巧，提到了技巧 **查询优化内容。**这里是在追问这个技术是怎么实现的，这里我提到了是用Langchain的某个方法，面试官还要继续追问这个方法底层是如何实现的。再次觉得这次面试问的真的很细）

3. 你用RAGAS 评估方法对你的Agent进行了评测，你是怎么去做的？

3.1你最后评测出来的指标是多少？

4. 另一个项目xx是怎么做的呢？

4.1 项目里提到的使用Litellm对模型无缝切换，这个是怎么做的？

（面试官扣的很细，他在扣我简历细节，这也是第一次面试官扣这么细。其实我简历里写的那个是一个换模型是静态的切换， 简历里用的这个词 **无缝切换** 让面试官以为是**热切换。** 最后和面试官澄清了这一事实，感觉还怪不好意思的）

5.你平时会有用一些AI coding的工具吗？分享你的一些使用心得。

（这块真是撞上了，之前我在公司做了vibe coding 的技术经验分享，刚好直接用来回答这个题目）

5.1 Github copilot 这个平台你有和其它的coding 平台做些对比吗？

（这个不同AI coding的平台对比好像还是在面试中出现过几次，这个特别得关注下，特别是如果面试的公司自己再做AI Coding平台，这个问题出现概率就会变大）

5.2 你知道Github copilot 里面的instruction.txt吗？能介绍一下这是干嘛的吗？

5.3 在这个Github copilot里面，有chat, agent, edit,plan， 你会分别在什么时候使用。

6.你有接触一些web侧的开发吗？比如FastAPI？

7.1 Python里面实现一个异步任务要怎么做？

8. 举一个你做项目中你认为最值得分享的技术问题？

（人人都要思考准备的一个通用性问题）

8.1 你的排查花了多长时间？

（我一开始回答说一周，然后面试官说这需要这一周吗？我又说因为我还有其他任务，面试官就又问那你觉得实际需要花多久，有点压力测试的感觉了）

9.MCP的SSE 和 stdio 的两种方式有什么区别？

9.1你会去了解一些MCP 的一些风险吗？会涉及一些敏感数据隐私？有没有一些安全隐患？

10. Agent之间的通信你是怎么解决的？

10.1 多Agent 之间的通信通常还是会有这个安全的concern， 比如某个Agent的某些状态不应该要给到下一个Agent，你怎么去解决Agent 之间数据传递的敏感性问题

显式控制传递字段

- 不要直接传整个对象，传递必要字段。
- 可以用 **数据过滤器**（Data Filter）或 **Schema** 限制传递内容。

分级 Memory

- 不要让所有 Agent 共用一个 Memory。
- 可以按角色分配 Memory，或者用 **Scoped Memory**（只给某个 Agent访问）。

敏感信息标记 + 自动剔除

- 给敏感字段打标签（如 `sensitive=True`），在传递前自动过滤。

沙箱隔离

- 如果某个 Agent 是外部服务，限制它只能访问特定数据。

11.你用的langgraph 你的memory的模块是如何实现的？

12.你会了解memori这个开源项目吗？

（搜了一下是个开源RAG引擎项目，面试官既然这么问了，我觉得他还是对这个RAG掌握比较深入的，说明他们应该是有一些自己的研究）

13 你有没有看一些新的AI 相关的技术吗？

（这个问题也是挺重要的，之前被滴滴面试官challenge过，所以大家都需要准备的是，你最近看了什么最新技术，这个也反映了你做AI 开发，关注前沿技术的能力。）

14. 量化技术里的FP和BP有什么区别

15. 讲一下常见的推理引擎。

16.你做微调数据库是怎么做的？

视频讲解

2025.11 平安证券

（两轮技术+一轮HR面。 其实到第二轮技术面就是主管面试了，而且全程没有手撕代码。考察难度也不大。注意的一点是他们在完成一面后需要做在线测评。类似IQ,EQ 逻辑题目，到时候整理了我会给大家一些参考，这个必须通过了才约二面，否则一刀切。 注意这里啊，证券公司貌似是比较注意仪容仪表，HR专门提醒我，建议我穿正装面试。 不过我没有穿（因为我真的没有）最后也通过了，但是如果真的很重视这次机会的话，建议还是听从HR意见哈）

岗位介绍

首先是一个算法 + 应用的岗位。 算法和应用各占50%。

主要业务是做：**智能客服，合规质检**，还有一些**国际新闻资讯**方面的业务。

主要会使用Langchain的框架。 使用国内的千问模型。

倾向于要一个既懂算法又懂业务的人。

一面（技术）

1. 自我介绍

（其实做面试复盘这么长时间了，之前都忽略了自我介绍的重要性，好像从来没有对自我介绍复盘。今天对自我介绍做了总结和反思后，发现其实我自己我介绍这么长时间，其实介绍得都不好，虽然其实取得了很多offer，但是仍然是觉得有大的改进空间，所以进步的道路是无穷的啊。 回到自我介绍，我发现介绍时间都过长了，有时候说了4分钟，也犯了一些自我介绍的坑：比

如说流水账。 我笔记中加入了一个 面试技巧章节：包含**自我介绍和反问**两个小节的注意事项）

2. 你能介绍一下你的智能日历系统中，你是怎么做的冲突检测的吗？有没有用一些内容分析，或者标签抽取之类的技巧呢？

（这个问题和业务强相关，我就不展示答案了。 但是其实从这个问题我们可以看出来，给我们的启发是：你可以通过面试知道面试官其实可能关注哪些细节，比如这个冲突检测策略，就是面试官经常问的。 你面试几场后，多少能够找到一些重点。然后着重去准备复习。 另外其实我们经常说面试官觉得我项目浅，我一直说项目浅，就继续优化就好了。 比如以这次为例，面试官问了这个问题，但是我目前冲突检测 就是用提示词去让AI做的，那么这样是不是一聊，就会让面试官感觉浅。所以在我们得到 "浅" 这个反馈的时候，可以分析出来为什么浅，哪里浅了，也就知道了如何再去深入做和改进了的）

3. 你们有对这个模型参数，超参数做一些调试吗？做一些优化，比如说针对你不同的啊分析情况，对**温度**啊，然后**惩罚度**啊，做一些，这些参数做设置用的

参考 7.1 常见参数和选择策略。 这一节讲了专门的参数相关的知识和这个问题的答案。 其实我觉得这里面试官的用词不是很严谨，其实想问的是 推理参数。 超参数一般是训练过程中的参数。

4. 你们输出的这个JSON格式，字段大概有多少啊？就是有跟一些相关的，几个字段？

（强项目相关，没啥好讲的）

5. 可以详细介绍一下微调相关的事吗？对哪个模型做个微调？然后Lora是怎么调的？你设置哪些参数啊，比如rank这些值，你是怎么设置的？学习率啊、步长啊、这些都是怎么设置的？然后你的语料是怎么来的？怎么做标注的？

（这其实就是你是如何做LoRa的？ 我们在八股部分 1.4 你是怎么做的微调 有专门的总结。一般面试官都是直接问你是怎么做的微调，但是这个面试官问得很细，给了各个方面的问题。 其实当时针对于这个问题就有专门的迭代和优化，还给上了LoRa 实验的方法论。我专门去检查了我们的参考答案，我们的答案完全覆盖了他说的对应的细节，答案无需调整，这也验证了我们当前这一版的答案和思路都是还不错的。大家直接参考八股笔记。）

6. 就刚才你提到MCP能力，微软那边应该这方面应该蛮专业的，能详细介绍一下吗？你们的MCP是怎么怎么做的？然后怎么利用function call怎么做集成的呀？你们用没用到这里面有没有用用到啊react这种模式？然后在你比如在你的实际的呃项目中，比如说遇到了function call或者是react这种这种思考反思这种模式，你去怎么选择这两种模式呢？

（这个问题涉及一个是MCP 是怎么做的？ 另外Function call 和react模式。 我在这期视频里给大家讲一下吧？因为这个问题到底面试官想问什么，可能不是特别清楚。我们在视频里讲解一下大概面试官想问啥，对应的思路是啥。）

7.RAG方面的内容你能介绍一下吗？就是你怎么使用它去构建知识库的？比如你这个就是内容是哪怎么怎么来的呀？比如你刚才提到了技师的信息啊怎么怎么进入的呀？然后比如门店地点这些信息怎么维护的呀？

（其实这个就是RAG 相关的最常考的问题。 你的RAG怎么做的，RAG的知识库如何构建，如何检索，如何优化等等，需要了解的很细。给大家的启示是，你如果有RAG项目，你需要理解/弄透里面每一个细节。 比如我问你，你的数据库是如何构建的？用的什么数据库？存了哪些字段？为什么不选择其它数据库呢？等等，这些是你要掌握的。 划重点：对于RAG来说，这是很重要的高频考点，其实也就是RAG的核心。

回到我自己来说，我的项目其实比较简单，这是一个Agent项目，我为了知识的完整性，强行加了一个最简单的RAG，我还写上在简历介绍了，这也是面试官为什么会问。但是很明显，我的RAG系统目前过于简单，所以面试问到了，就会觉得你做得浅。 所以我自己也会专门再做一个RAG系统，里面有详细的RAG的所有核心步骤，这样面试官再问RAG，就比较深入了。 我想表达的是在面试中，面试官说你项目浅，你肯定是可以分析出来到底哪里浅的。然后针对性加强。）

7.1 （追问）比如你这个向量化模型有做优化吗？因为你去你说你要支持他客户咨询的问题嘛，一般客户的问题都很短，就是有没有对这个embedding模型做一个调整啊？或者怎么样选取的时候你是怎么怎么考虑的？

其实是RAG系统对于查询侧的 问题优化。也是属于RAG的核心优化环节。我自己项目没做，我就不展示答案了。但是我想告诉大家的是重点，这个属于重点，就是查询改写嘛，常见的RAG策略。我项目没做，但是在八股笔记中其实理论都总结了。 参考RAG八股部分 4.3 召回优化策略。

8.你对千问2.5做了微调，那今年千问3也发布了，千问3对比前一代有哪些提升？你有了解吗？你们使用起来OpenAI的模型跟国内的一些GPT呀或者千问3这些模型，使用起来的感觉怎么样？

（这个问题其实要真的答得很好的话，要吃点真实经验。就是你确实做模型评估的时候，你对比了不同模型，做了一些评测，然后有一些什么样的感受。给到一些有用的反馈会比较好。 模型评测其实是专门的工作，在正式的项目中，是有这个环节和代码的。 但是确实很多人转行的话，其实你自己做项目，可能做的不会有那么全面，所以可能没做到，这样也能理解。 如果真的没有类似的经验（我自己也没有），那么这个点其实还是可以去回答的，大概就是答对应的基础知识： 千问2.5->3的升级（如果你在国内，应该要了解千问，这里看一下八股总结）， 然后模型对比（其实在八股部分表格中也有）。 所以如果你没有实践经验，你根据八股总结的这两个知识点，也是可以回答的。）

千问的提升： 看八股 千问2.5到3的升级

千问vs gpt：结合八股模型对比 的理解回答。这里展示一下大概特点。

1：首先在推理能力这一方面，GPT系列还是要强于千问的，特别是GPT的o1系列，推理能力非常强。

2： GPT在中文，中国文化领域做的还是差一些。 特别是GPT生图，其实经常会出现文字错乱，对于中国业务上，要求中文语境，中国文化，文字等方面的业务，Qwen的表现更好。GPT的通用性更强。

3：他们还有一个差别是GPT其实是闭源模型，只能依赖于远端调用。而Qwen可以使用本地推理，这其实意味着我们用自己服务器推理，隐私性更好。相比远端调用成本也会降低

结合咱们金融业务，其实如果不需要特别强的推理能力，多语言，比较注重隐私性，包括成本考虑，其实使用Qwen就是一个不错的选择。

二面（主管面）（18min）

1.你在xx，这一年多的时间，做的就是这个日历助手项目是吗？对于AI应用或者大模型这一块的实际经验，就是这个日历助手对吗？

（毕竟这一面是主管面了，而且还是证券公司的主管，所以通常来说预期他并不会非常懂技术，甚至是不太懂技术，相比传统的互联网大厂的主管，一定在技术上是比较少的。通常他是不会问具体技术问题的。 面试官问这个问题，其实就是想看你有什么经验，你经验丰富不？最好还做一个和银行业务相关的大模型应用那就更好了。 所以这个问题面试官的侧重点更多是要的是你经验丰富，做的项目多。至于你的深度，估计他在这里也把控不了。 所以这回到有一个粉丝朋友问的，说我写简历，写一个够吗？这里其实可以看出做一个其实有点少了，如果我们只做了一个，那么这个问题就不好回答了。之前我回答这个问题时候，我只能说我有两个Agent项目，但是现在，我自己不是又在弄一个RAG项目，现在我再去回答，其实我能回答出的我的经历就更丰富。

我这里给的参考回答思路就是，结合我的三个AI项目：展示从Agent，到RAG 应用的经验，同时点出自己的算法（微调，推理加速，部署）的知识，最后快速点出其它工程方面（非AI的经验））

我在AI领域有比较全面的落地经验，涵盖了从应用层Agent开发到底层RAG框架构建，以及模型微调等算法工作。具体来说，我有三个代表性项目：

第一是**端侧Agent落地**。在微软，我基于 **Semantic Kernel** 框架开发了智能预约Agent，深度集成了Windows/M365生态。我不止做Prompt Engineering，还负责了本地模型的**微调、压缩和推理加速**，让模型在边缘设备上跑得更快更准。

第二是**复杂多智能体系统**。我使用 **LangChain** 的监管者架构设计了一个多Agent预约系统，包含反思机器人和监管者角色，利用RAG实现用户喜好与专长的智能匹配，并引入**反思机制**持续优化行为。

第三是**底层RAG设施**。为了追求极致效果，我没有依赖现成库，而是**自定义了一套RAG-MCP框架**，手动实现了分块、粗排、精排的全流程。这套系统是高度可插拔的，直接提升了我们业务的查询准确率。

此外，我有扎实的**传统工程背景**，我有Windows日历全球上线客户端经验，以及注册表安全攻防这类底层安全算法工作。"

2. 这个日历助手它是什么应用场景？能给我讲一下？

（我估计面试官的没有太多技术背景，这里我觉得就把常见的功能点讲清楚，特别专精的技术细节就别讲了，因为非技术背景听不懂）

3. 现在用的这个模型是哪一个？

（我聊了挺多公司感觉很多国内公司更倾向于用 千问系列？ 倒是Deepseek好像没听到有公司说他们用，但是我听说政府好像更倾向于DeepSeek? 总之这个问题我倒是觉得言之有理即可。估计面试官可能比较感兴趣的是千问，为什么不用千问，千问和你用的GPT的对比？ 这些咱们都总结过，因为之前面试出现过，大家可以一起参考着复习）

参考八股：**7.4 千问2.5到3的升级**

还可以看一下平安证券一面第8个问题： 千问和GPT的对比？

4. 你对这一块（AI应用）是怎么看的... 持续多久啊... 还是说未来的方向？

（是一个好问题，最好你是有一些自己的思考，比如看一些行业大会，行业报告，都能有灵感，结合自己想法讲出来。另外答案最好是积极的，看好AI的，本身就是AI的岗，如果企业不看好AI，他也不会设置这个岗位对吧，你不看好AI，你也不会面试这个岗位，所以如果谈一些唱衰AI的论调，就显得不合时宜。

如果你确实没啥思考，那么不妨背背总结的答案吧。 以下内容关于AI的方向，我也是从我们一直说的参考资料。<<Agentic Design Pattern>> 里面总结的，看一下这个笔记里：https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn/blob/feature/interactive-website/06-What-Makes-Agent.md，里面写了 Agent未来的5大设想，那不就是这个问题最好的回答吗？

我的参考答案，首先先谈了这个AI 浪潮持续多久的例子。然后未来的方向，我从以上参考书籍中抽取了5个方向回答，因为这是一个证券公司，对于第四个方向，AI 会成为独立经济体这一小节进行展开解释，因为我相信面试官可能会比较感兴趣。）

首先先说，AI浪潮会持续多久，我的看法是在未来10年都会继续快速发展。有些人说AI 被高估了，很多公司估值都有泡沫，我的看法是在1年的角度来说，有的公司特别是美国公司，确实因为AI被高估了，可能会逐渐回调。但是从10年的角度来看，AI才

刚刚发展，我们看到就近一年AI在物理世界智能眼镜，机器人，驾驶进步是肉眼可见的。在互联网软件方向，以Agent为例，从Agent RAG, MCP, A2A,SKills, 技术发展很快，效果变得越来越好，未来还有非常大的发展空间。

第二个问题，未来发展的方向，我Agent 未来发展有5个方向：

1. **从单点专家变成长期管理复杂项目的通才**。比如一个Agent不是一次性任务，而是多次，持续数周的项目管理，最后达到目标。

2. **深度个性化+主动发现目标：**智能体不会等待你下命令，而是理解你还没说出的目标，主动给你建议，帮你规划。

3. **进入物理世界：**智能驾驶，机器人，家庭维修，制造业等都是Agent未来可以赋能的方向。

4. **智能体驱动经济：**智能体可能会成为经济活动中"独立参与者"。比如结合证券行业，Agent会自己交易账务，他会持续看数据，分析新闻，财报，社交媒体，动态调整仓位，在多个市场上套利。他自己本身就是一个市场参与者。

5. **目标驱动，可演进的多智能体：**只要给Agent一个目标，他会自己创建新智能体，改变自己架构，来完成目标。

5 你对我们这边了解吗？... 证券公司是大概你有概念吗？他们做什么事情？

（这是我这次面试的一个大大的减分项了，因为确实没有太了解，甚至坦诚了说下去再了解。 这个问题其实我在咱们笔记的 **面试技巧** 不管是自我介绍还是反问技巧环节我都讲了一些技巧，我们需要展示意向度的，自我介绍时展示对公司的了解，就是一种意向度的体现。 在这个环节，我们不光私下里要去查一下公司，在一面面试官，通常会介绍业务，也是可以作为这个问题信息的来源。而我当时就是准备不充分，说不是特别了解，我觉得这点挺不好的）

我对平安证券的业务做了一定深入的了解。作为证券行业的头部企业，咱们的核心业务其实涵盖了**证券投资、投资顾问、投研分析、交易执行**以及**精细化的客户经营**。

特别是在AI应用方向，我看到了很多结合点：

投研）：利用大模型辅助分析师进行研报生成和数据挖掘。

投资顾问与客户洞察：通过Agent技术深入理解客户需求，实现个性化的**策略匹配**，提供千人千面的服务。

交易与经营：利用AI辅助交易决策和提升运营效率。

我之所以想加入，就是希望利用我在Agent和RAG方面的经验，深入到这些具体场景中，用技术真正赋能这些高价值业务。

6. 个人情况

（其实就是工作地点的讨论，能不能接受工作地点）

7. 反问

在 笔记 面试技巧 我写了具体事项。大概是就是 技术面问业务， 主管面问战略，发展， HR面问薪酬，福利等。

三面（HR面）

视频讲解

我用夸克网盘分享了「平安证券大模型全栈一面.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/88ac114d58d7

2025.12 字节跳动（抖音）

岗位介绍

AI应用后端开发工程师-抖音直播

职位描述

做抖音和Ticktok的推荐引擎的架构工作，这个组专门做AI工程方向，主要还是面向内部使用。

一面

1.自我介绍

2. 你在学校和工作中会使用哪些和AI相关的产品，他们是怎么影响到你的学习，工作和生活的。

3. MCP有本地模式和远程模式，他们有一些什么区别，开发的时候如何选择？

（这个视频专门讲MCP 本地和远端模式的：https://www.youtube.com/watch?v=wlerSyHzoCI。在看了这个视频后，我补充了MCP相关的八股知识，这个里面涉及两个重要的点，一个是本地Server和远端Server的区别， 第二个是MCP协议的传输协议的更新。 这两个都是面试出现过的考点，大家可以看上面笔记部分复习。）

参考笔记 4.2 [**MCP 本地Server和远端Server的对比**](#doxcnbcXPaZOgJnCbYj4HWGJMmY)

3.1 MCP 本身依赖了一些资源，如果你在本地起一个资源，如何保证它能满足所有的资源依赖？

3.2还有其它的吗？

（应该是没答全，我答了本地隐私更安全，延迟性低，然后3.1用户提示了资源相关问题，面试官问还有吗？显示然他还想听到更多，这个点我又答了一下远端的灵活性和解耦性更好，我感觉还是没达到他想要的，然是全程我感觉自己还是说了挺多东西的。根据我最新的总结，笔记4.2中的对比表格是包含了面试官想问的点。）

4.再问一个MCP的问题，我们如果使用了MCP Client + 远程的Server，本地的Client 和远端Server 会有交互，如何解决他们之间协议不兼容的问题。具体是什么意思呢？比如你用了微软的开发框架，开发了一个智能体，它里面的MCP的实现未必是最新版本的MCP 实现，也有可能是自己的MCP实现就不完整，那么server和client之间可能会有一些不兼容的问题，你是如何考虑这个问题以及如何避免的？

4.1 是这样的，这个问题的出发点是其实你使用的MCP Server 未必是由你们团队维护的，比如说你是一个天气查询服务，但是你需要使用它，由你们公司设计的client会和它交互，这个时候可能会有一些兼容性问题，比如openai的sdk 它其实在MCP的实现上就不完整，这时怎么办？

（这个题问得太细致了，我觉得可能是面试官做项目的时候自己遇到过的工程问题，没有对应的经验我觉得还挺难回答的，通过这个题我们要知道MCP 协议里 的两个能力： **版本协商** 和 **能力协商**）

MCP 标准协议本身就定义了两个关键机制：

1. **版本协商（Version Negotiation）**

   - 在连接初始化阶段，客户端和服务器会交换它们支持的协议版本列表，并选定一个双方都支持的版本。如果没有交集，连接会优雅失败。
   - 这意味着客户端可以明确知道“是否匹配”，并在不匹配时采取回退或提示。

2. **能力协商（Capabilities Negotiation）所以，客户端在初始化握手后就能拿到：**如果发现不匹配，客户端可以：

   - 握手时，双方会声明自己支持的功能（如 `sampling`、`resources.subscribe` 等）。客户端可以根据服务器返回的能力决定启用哪些功能，避免调用未实现的能力。

   - 协议版本（是否兼容）
   - 支持的能力列表（哪些功能可用）

   - **回退到兼容版本**（如果支持多版本）
   - **禁用不支持的功能**（避免调用导致错误）
   - 或直接提示用户“当前服务端不兼容，需要升级或切换”。

5 MCP 和 A2A之间有一些什么样的异同，各自适用于什么样的场景。

(A2A 协议目前有两个面试中出现了，之前认为不怎么考，现在还是会考的，所以我把A2A的概念补充在上面笔记Agent 部分MCP下面了。 A2A的基本概念大家还是需要知道的，另外针对于这个MCP 和 A2A的异同，复习了笔记的A2A和MCP的概念，相信此题应该很好回答）

MCP**解决的是 "**Agent ⇔ 工具/数据" 的标准化连接问题：让 Agent 以统一的方式发现、鉴权并调用外部工具、API、数据库、文件系统等能力。就像给 AI 装一个“USB-C 工具口”。

A2A**解决的是 "**Agent ⇔ Agent" 的互操作与协作问题：**不同厂商、不同框架下实现**的多个智能体之间，如何发现彼此、交换消息/任务、长时协作并保持内部实现不透明。

5.1 我现在有一个Agent，他对外既可以使用A2A这种方式提供服务，也可以使用MCP Server的能力提供服务，这个时候怎么选？

5.2 是这样的，Agent自身内部是怎么思考如何实现你的服务的，比如你是一个天气服务，对于使用者来说，我不care你内部是如何实现的，我就是让你去查询过去几天的天气，你可以使用工具去调用，也可以使用Agent的方法去获取这个天气，这个时候呢，MCP还是A2A?

（5.1 和5.2这个题我觉得怪怪的，把MCP和A2A去对比，我先问一下其他人意见，晚点总结答案）

6.你做这个日历的项目，你做了时空冲突检测，你使用了模型推理，你是怎么解决模型有幻觉的情况？

（我使用之前就准备过的笔记回答，这个题没啥问题）

参考笔记：1.[如何减少Agent的幻觉](#doxcndQEu9TGCHUjC6LN0WjjS4b)

7.你如何保证你使用数据集测试的结果 和 用户真正使用的时候是一致的呢？就是你数据集也许跑得好，但是在用户使用的时候，效果不好，怎么办？

7.1 如果你的项目上线了，你如何收集回来这些数据？

7.2 当用户点击了否的时候，你怎么知道是你的效果做得不好，还是因为用户不喜欢你的feature？

（这个问题其实和AI无关了，是一个和数据收集/ 通过数据判断业务指标的讨论，也和项目细节强相关，感觉对大家参考价值不大，所以答案不总结了）

8.算法题

实现带有过期属性的LRU。

简单来说，就是在Leetcode 146题LRU缓存的基础上，加了个时间过了自动剔除的功能，具体的接口要写的也不是特别明确，大致就是自己根据这个意思，来完成逻辑，没有特别明确的要求。

2025.11 字节跳动（豆包）

岗位介绍

Title是：**高级服务端开发工程师**

具体部门是什么不知道，HR说是保密项目，而且我就只走了1面就挂了，但是好像是和豆包有点关系。

职位描述

1、负责C端产品的后端研发，能够快速搭建应用，持续优化产品体验，提高系统的稳定性和可靠性；

2、持续探索在各种场景下，利用AI能力增强产品体验并提升工程效率；

3、研究和应用新技术，并推动适合的技术应用于生产，解决业务的实际问题。

职位要求

1、精通Python、Java、C++、GoLang中的一种或多种编程语言；

2、熟悉MySQL、Redis、消息队列等常用基础组件，熟悉微服务，熟悉Linux开发环境；

3、具备出色的分析和解决问题的能力，能够与跨职能团队有效协作；

4、有Prompt Engineering调优经验、或者商用AI项目实践经验可加分。

反思

一面

1. 自我介绍

2. 这次找机会主要是什么原因

3. 挑一个项目介绍一下，或者说收获比较大？

    3.1 你这个Agent 主要靠什么事件触发？

        （其实就是澄清项目细节）

    3.2 整个过程的提取流程大概是什么样子的？

    3.3 做这个项目的过程中，你遇到了哪些问题，怎么解决的？

    （其实这个问题也算是必考的问题，因为非常具有通用性，我将这个问题整理，总结到八股内容-其它模块，那里写了对应的技巧和准备，请大家参考）

参考八股笔记 [你在项目中遇到的最大的困难是什么，怎么解决的](#doxcnpqeoSutE489zhUPcWrUaBb)？

    3.4 GPT的一轮标准对话大概是什么样的？

    3.5 当你在代码中询问 "周杰伦的演唱会是什么时候"，这时llm发生了几轮的交互？

（其实3.4和3.5 是在考察你理不理解这个Agent框架和LLM之间交互的过程发生了什么？以下是答案。如果大家不理解，我总结了具体的例子和代码，大家可参考，如果还是不明白，可以看一下我的[讲解视频](#doxcnN1WcRVLTVYGzKL8pxWs1db)，这期视频我有对这个问题进行讲解）

针对‘周杰伦演唱会’这个例子，如果 Function Call 正常工作，**一共发生了两轮完整的 HTTP 交互**。

- **第一轮交互（意图识别）**：Semantic Kernel 把用户问题发给 LLM。LLM 分析后，返回了一个 `tool_calls` 指令，告诉框架‘我要调用 `search_concert` 工具，参数是周杰伦’。此时 LLM 并没有生成最终答案，而是处于挂起状态。
- **中间处理（本地执行）**：框架拦截到这个指令，在本地运行了搜索代码，拿到了‘2025年3月2日海口站’这个结果。
- **第二轮交互（结果生成）**：框架把‘用户问题’、‘模型的调用请求’以及‘工具的运行结果’打包在一起，再次发给 LLM。LLM 看到工具返回的具体时间后，才生成了最终的自然语言回复：‘周杰伦的演唱会是在2025年3月2日’。

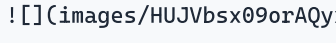
    （如果你弄不明白上面的过程，请看这个更详细的解析）

第一轮交互：意图识别与工具选择

目的：模型分析用户问题，决定是否需要调用工具，以及调用哪个工具。

Request (SK -> LLM)

SK 将用户问题和所有可用工具的定义（Schema）打包发给模型。



Response (LLM -> SK)

模型识别出需要搜索，返回一个结构化的 `tool_calls` 指令。注意此时 `content` 通常为空或为 null。

中间处理：本地执行 (SK 内部)

这一步不产生网络流量。

1. SK 收到 `finish_reason: tool_calls`。
2. SK 解析 `function.name` 找到本地 Python 函数 `search_concert`。
3. SK 执行函数：`search_concert(artist_name="周杰伦")`。
4. **函数返回结果**：`"周杰伦《嘉年华》世界巡回演唱会-海口站，时间：2025年3月2日。"`

第二轮交互：提交结果与生成回答

目的：将工具查到的结果喂回给模型，让模型生成最终给用户的自然语言回复。

Request (SK -> LLM)

SK 构造一个新的请求，包含**完整的历史上下文**：

1. 用户的原始问题。
2. 模型上一轮的“我想调工具”的请求（Assistant Message）。
3. 工具实际运行的结果（Tool Message）。

Response (LLM -> SK)

模型结合工具返回的事实，生成最终回答。

4. GPT在2025年出了一个叫Response的API，你大概了解吗？

（关于GPT 的Response API, 其实是在25年3月份OPENAI 就提出了，希望将llm的工具调用，多轮对话等功能整合到api中，减少Agent开发者的负担。 网上资料不多，面试也几乎从未出现。他都推出了大半年了，但是看上去这个思想没有火起来，估计可能也不会火起来了。我是第一次被问到。所以针对于这个知识点，我的建议是：大致了解下，不用花太多功夫。 所以我尝试用下面的回答大概简单概括下它的思想，理解这个回答，下次万一真被问到，能说道几句就好了，并且我严重怀疑这个基本面试不会问到）

Response API 是Openai设计LLM调用接口的**从 Completions → Chat Completions → Responses 的过渡**：早期的 *Completions* 只面向纯文本续写；随后 *Chat Completions* 围绕 `messages` 支持对话与函数调用，但状态与工具编排仍主要在客户端；如今 **Responses** 用一个 **有状态的统一端点**把多轮上下文与多步工具调用纳入服务端协议。

设计思想与理念：以 **“一次请求里的智能体循环”** 为核心，把 **状态、工具、事件流与结构化输出** 写进端点契约（而非让客户端手工拼装），让模型在单次响应生命周期中 **按需调用托管/外部工具、跟踪步骤并输出可验证结果**，从而**

标准化智能体能力、降低工程胶水、提升可观测与可扩展性**。

5. 你用LoRa做了本地微调，这里面你是怎么做的？

（LoRa怎么做的，是我们精心准备的答案，包括怎么回答这个答案，对应的方法论和参考资料，我总结的都有，直接看笔记）

参考笔记 1.4 你是怎么做的微调

6. 你里面提到的对性能做测评，里面有一些bool类型指标，也有一些过程指标，这分别是怎么评判的？

（同学们，这个问题不清楚的话，会去看一下八股部分：[Agent开发框架](#doxcnVbEfVZ4QrE0dykkw5aNewb)，我给大家推荐的这一章的参考资料Agentic Design Pattern，里面就讲了这个问题，Agent评估问题，我推荐的两个资料，都是很有用的哈，这里也验证了，希望你能回到八股部分，看看我的笔记和推荐的资料，以下回答几乎来自于我的参考资料。）

对于bool类型的结果评估，我们可以使用直接比较，要求智能体响应**完全匹配预期输出**时才会成功。

但是对于一些过程指标，则需要使用**更先进的自然语言处理技术**来辨别句子之间的意义。通常的一些手段是计算**字符串的相似度（如Levenshtein（编辑距离） 和 Jaccard(集合重合度））， 关键词分析，使用嵌入模型的预测相似性进行语义分析，LLM作为Judge，**以及**RAG 特定指标（如忠实性和相关性） 来进行评估。**

（上面的内容其实回答这个问题已经完整了，但是请你思考一个问题，面试官很可能追问，那你的项目是怎么做的呢？ 这个希望大家也能准备，如果你答不上这个问题，面试官会觉得你只懂理论，但是实践不行。所以相应的，你需要自己准备这个你的项目怎么做的问题。以下是对针对我的项目，结合这一问题的答案参考。）

在我的项目中，核心功能 `email_to_event` 需要从非结构化的邮件文本中提取结构化的日程信息。针对不同的字段类型，我采用了分层的混合评估策略：

1. **确定性字段：**

   - **针对字段：** date, time, participants,needs_calendar_events 。
   - **评估方法：** 对于日期和时间，我先将其标准化为 ISO 格式（如 `YYYY-MM-DD`），然后进行**直接比较**。对于参会人列表，我将其视为集合，计算 **Jaccard 相似度**（交集除以并集），确保所有关键人员都被识别且没有遗漏。
2. **短文本语义字段：**

   - **针对字段：** location, subject。
   - **评估方法：** 这些字段允许一定的表达差异（例如 "Room 302" 和 "Conference Room 302"）。我使用 **Levenshtein 距离（编辑距离）** 来允许字符级的微小差异。对于差异较大的表达，可以使用轻量级的 **Embedding 模型**（如 text-embedding-3-small）计算余弦相似度，设定阈值（如 >0.85）判定为通过。
3. **长文本与意图字段：**

   - **针对字段：** description (日程描述/备注)。
   - **评估方法：** 描述字段通常包含会议议程或特殊要求，很难用关键词完全覆盖。我采用了 **LLM-as-a-Judge** 的模式，构建一个专门的评估 Prompt，让更强的模型（如 GPT-4o）作为裁判。
   - **具体做法：** 我会问裁判模型："提取的描述是否准确包含了邮件中的‘Q1目标’和‘资源分配’这两个关键议题？"。这种基于关键事实点的评估比单纯的文本相似度更能反映业务价值。

7. 你认为多Agent系统和单Agent系统最大的区别是什么？彼此有什么优势？

（朋友们，问题7和问题8其实也都是在我给大家总结的Multi-agent的架构的 1个论文 + 书本的某一章中，这两个参考资料很有用，请大家仔细阅读另外这个问题看上去也是比较有通用性的，因此我直接将这一问题收录到八股Multi-Agent部分。）

参考八股 6.3 单Agent和多Agent对比

8. 你讲到的你这两个项目，两个项目最大的区别是啥？你刚才只是结合了一些概念上的东西讲的，你能否具体结合你的项目讲？

（在你理解了我整理的参考资料后，这个问题其实很好回答。我给出的参考论文列举了不同任务 使用不同Agent框架的效果和性能，并总结了什么项目适合什么Agent框架，如果你能理解的话，这个题其实是把这个相关的内容，运用到你自己的实际项目中，针对你的项目谈谈回答。还是这句话： **笔记八股Multi-agent部分的内容值得大家好好会去阅读**。以下的答案其实基本就是可以套用我给大家总结的参考资料部分，**金融分析适合用中心化架构**，**网页导航适合去中心化， 工作流适合单Agent的架构的原因**，然后往你的项目一套，就完了）

在我的邮箱项目中，因为本身它是**严格线性进行的，每一步依赖于上一步状态，无法并行，并且任务不复杂，单Agent的基线收益已经很高**，所以天然适合使用单Agent项目，使用任何一个多Agent架构都会导致性能下降。

而在按摩房预约系统中，任务天然具有"分而治之"的特性，**中心化架构让编排者将任务拆解成"预约", "支付","咨询"的子任务**，分配给不同Agent并行处理，最终整合。在此种场景下，使用中心化架构是最好的选择。

算法题：

给定一个由 DAG 定义的 agent workflow，判断图是否成环，若不成环输出 yes，成环输出 no。

视频讲解

https://www.bilibili.com/video/BV1yc9bBXEBV/?vd_source=144bec9c3f54e465073138bed788be1b

2025.11 Deepwisdom

岗位介绍

公司产品

国内 Coding Agent 赛道龙头，以多智能体协作技术为核心，推出 Atoms（原 MetaGPT-X，简称 MGX）AI 原生创业平台，可通过 “自然语言输入→AI 虚拟团队分工→5 分钟交付可运营商业项目” 实现全流程闭环。开源框架（MetaGPT、OpenManus 等）累计超 15 万 GitHub Star，全球注册用户超 50 万，上线首月 0 投放即达成 100 万美元年化收入（ARR），连续四周霸榜 Product Hunt 全球榜首；累计完成 3100 万美元融资，Xbench 测试成绩远超 OpenAI o3 等竞品，通过 SOC 2 认证与 GDPR 合规。

技术实现

- 底层是**多 Agent 架构**，通过 Agent 协作来实现功能
- 不是简单的拖拉拽形式，而是智能生成
- 涉及 Agent 之间的通信、工具调用、站外文管理等

公司规模

- 公司有 **100 多号人**
- 只有这一个产品，所有人都为这个产品服务
- 已经做了**大约一年**
- 目前是**创业公司**，资源有限

岗位要求

- 全栈工程师：前端、后端、算法都要做
- 包括模型调优
- 不区分前端后端测试，什么都要做

反思

这个公司是一个创业公司。根据和面试官的交流，确实是和创业公司的典型画像很匹配。面试官说，什么都要做： **前端，后端，算法，测试，模型调优**都是要做的。一般创业公司还是很看重匹配度以及技能的全面性的。

从考察的角度，本次考察内容其实中规中矩。基本上是结合项目去考察，结合着项目也会扩充到一些相关的基础知识，比如架构，量化，DPO，提示词工程。 总的来说本次的面试内容基本都在我们的掌控之中，八股内容里基本覆盖，都在我们总结之中。我也结合着这一次复盘，去优化调整了一些知识。八股部分，结合这一次内容，在模型八股中：加入了模型和硬件相关的知识：怎么判断模型能不能跑？跑的有多快？这一点对于使用本地模型的同学，需要了解。

从本次面试流程来看，本次我是主动放弃了二面，因为当时其实就准备等千问C端开奖然后去了，也无心面试了。主动放弃了二面，猎头还一直说好可惜好可惜，他说你被分到了什么xx组，这是很好的组呢。从面试流程上来看，一面结束后，会给一个笔试题，这个笔试题是一个链接，点开后需要使用6\~8小时，去完成一个小的项目。好像使用他们的编程工具和平台去做。因为我放弃了面试，所以发了链接我也没有点开。

我倒是认可这是一个挺好的，可以反映候选人素质的考察方式。 这样考察的公司不多。我是第二次遇到。第一次遇到其实就是我面试现司，是给了一个项目，让我60个小时内完成。这种考察就是挺折磨人的，感觉要在一个很短时间封闭开发，需要在短时间高度集中，还考验人的。

一面

1.自我介绍。

2. 你的AI日历助手是写的客户端的代码逻辑吗？用什么语言写的？

   2.1 模型也是跑在本地吗？模型能跑起来吗？用的多大的规格和尺寸？

（这个问题夸克千问一面也有对吧。 为什么会问，想让大家知道的是如果你用本地模型，甚至是cpu，就是会问啊，面试官会有疑问。 所以如果是说你的模型是远端的，其实一般应用岗这个问题倒是不会太问到，但是你用了本地模型，那么你需要知道这个问题。这个问题我也在八股部分 模型和硬件，总结了模型能不能跑，跑得多快，我相信这一节我讲清楚了，**大家去看一下**，有了这个基础，这个问题其实非常好回答，结合你具体的模型，硬件和公式，给面试官算就好啦！）

完全可以跑起来。我的电脑是64GB内存。

qwen7b-INT4 的要求是： 7B * 1字节 = 7,000,000,000 字节 - 7GB。

加上一些额外的开销，大概需要多1.2\~1.3倍的空间，所以内存要求是8\~9GB，所以我的电脑完全能跑。

结合我的从工程/任务层面的优化 + 选择专门的推理框架最大利用cpu的并行计算能力，也可以调推理速度。

（上面回答应该是很具体和专业，具体公式计算，怎么来的，原理，请看我的八股 部分 模型与硬件）

2.2 他具体做了哪些功能？

3. 我看你简历写了MCP协议，上面注册了多少个工具？

3.1 你提到整个工具动态加入，怎么动态加入，他怎么知道每一步要用哪些工具？

（这涉及两个问题，一个分阶段注入工具，你怎么知道每个阶段注入什么工具。 第二个是如何动态注入的，第二点就和具体的Agent框架，langchain 等有关系，我这里以Semantic kernel 为例回答，因为这是我项目中用的）

整个流程被划分为多个阶段（邮件获取，创建行程，通知等），每个阶段只向 Agent 注入该阶段所需的工具集，这个是业务逻辑规定的。

Semantic Kernel 的架构是 **Kernel（内核）+ Plugin（插件）** 的模式，每个 Plugin 包含若干个具体的工具函数。动态注入的思路是：所有 Plugin 预先注册到 Kernel 中，但在每次调用 LLM 时，通过**过滤器机制**指定当前允许使用哪些 Plugin。这样，虽然 Kernel 中存在全部工具，但 LLM 在该次调用中只能感知和使用被筛选出来的工具子集

（后面一部分SK的机制我认为对大家没啥帮助哈，这个Semantic kernel我看也就微软在用，大家看看就好）

3.2 你提到会存储一些数据，会存储一些什么数据呢？

（这个问题，我在 八股 长期记忆的常见应用场景 中总结了常见的一些场景会要存储的长期记忆。包括这个例子，也是根据我的业务场景下长期记忆的一个参考。 整个回答和你的业务相关，整个东西也不难，但是要能说出来，展示你的一些思维的发散性。 比如夸克千问2面面试官就追着我扩展，天马行空问，你的场景下要存一些什么？为啥，大家根据这些例子做参考，然后想想你的Agent项目要存点啥）

在我的智能日历助手中，我们需要根据用户喜好，来智能调整规划冲突的策略，这时我们会存：

1.用户偏好相关：时间偏好：喜欢早睡/晚睡？工作>个人？ 家庭>社交？等方面喜好

2.历史行程模式： 过去的选择倾向，倾向推迟还是提前，喜欢合并还是分散。这可以让我们在做决策时参考以前现有选择，直接复用。

3. 社交关系：重要的一些联系人，比如老板，家人，重要客户。这样在涉及到重要人群的时候，会提高优先级。

4. 你的项目整体使用了一个什么架构？

（架构这里其实讲了挺多的，包括很多面经有一些深入架构的讨论，我们的八股也总结得很多了，这里的面试其实就是问了最基本的问题。 那么大家回答的时候，就是结合我们八股部分总结的那几种架构，看看你的架构是属于多Agent的哪一个，结合着讲一讲，以及为什么选这个架构等等，大概讲一下。以下我的回答其实理论知识都来自于我们八股部分架构的相关内容。单Agent vs 多Agent，如何选型，八股部分都总结得很好了。）

我的系统采用的是一个典型的单Agent 的Prompt chaining的架构，具体来讲就是将任务分为多个阶段，然后单个Agent在这个Chaining的流程中去填空，完成子步骤。选择这个架构的原因是因为，我的任务并不支持子任务的并行，而且单Agent的基线收益比较高。这种情况下使用多Agent，反而会引入协调税，分散token，引入沟通开销，性能反而下降。所以在这种情况下，单Agent的效果是最好的。

5．你的Agent是如何进行效果评估的？

（这是一个必考题，从Agent评估的理论到结合自己项目，Agent是怎么做的，在八股部分都写了，大家自己参考一下吧）

参考八股：3.3 [你的项目中是如何对Agent进行性能评估的](#doxcnbpKwJs393fsRaDVMYRg89c)

6．你讲一下你是如何做的量化？

看我的笔记吧： 8．[你的项目是如何做推理加速的](#doxcnaaHJ978A9HZpljzbTB5jxN)？

这块我是专门总结了量化，微调，推理加速的知识，结合项目讲了怎么做，有具体的方法。我的理论依然是，对于应用，我们把流程走一遍，我这个方法我是让AI写了脚本跑了一遍，确定是可行。然后记录下来大概的结果，用的什么库，怎么完成的，总结好，整理好。 量化，推理，加速就到这里。因为算法不是应用的主战场，我没有深入去做（不过确实应用岗也不会深入问），所以做到我八股整理的这一步，就够了。我没有再深入，面试也没深入问过。

7．你这边也会做LoRa微调吗？数据怎么来的？什么是DPO啊？

一样的，都是我们八股总结过了的。 LoRA怎么做的，这个问题我专门总结了，也有一套方法论：如何以科学实验方式做LoRA。 相信这里总结得很好了，包括数据怎么来的。以及什么是DPO，这都概念，我们八股都总结了。参考笔记1.3 [你是怎么做的微调的](#doxcnXE4sR8p50zqg2mzVEJDSVd)?

所以大家虽然是应用岗，但是不要怕算法，我们都准备了。 而且算法岗问多深，我们就准备得多深。我没有深入总结什么公式啊，就是因为现在面试官不会深入问。万一有一天他深入问的多了，我笔记也会相应深入总结的，我帮大家兜着底呢。

8．大模型肯定要组装一堆Prompt，你这个Prompt是提前备好固化然后直接打包在电脑中吗？有没有什么Prompt优化方法？

8.1 除了变量之外，那些固定的部分，能不能用用户的数据来优化咱们整个Prompt？

(关于提示词的优化，这个问题让我想到了[夸克千问二面](#doxcn5OPtrbtuCqv7dmgNoUR9nd) 面试官的问题 在项目过程中，怎么去做提示词的压缩和裁剪？ 这个问题其实面试官想问的是数据飞轮来优化提示词的策略。算是另外一种对于提示词的处理，我们可以结合夸克千问这个面试问题，这两点来综合学习一下在项目中有哪些对于提示词的处理技巧）

（数据飞轮指的是：把数据比成一个飞轮，一开始很难转动，但是随着开始启动，会越来越快。 类比到Prompt中，指的是一开始数据是固定的，但是随着上线采集到一些数据，通过数据来优化提示词，达到更好的动态调整，更好的效果的概念）

是的，在我们的项目中也可以使用数据飞轮来优化提示词。

比如我们在提示词中，预置一个用户历史对于冲突检测的选择的倾向和一些历史决策。然后在提示词中表明，会参考用户历史的倾向和选择，来调整智能规划的建议。 一开始上线，可能并没有一些历史数据，飞轮没有转起来。随着上线，采集了用户对于历史建议的选择：忽略，直接接受，修改以后采纳 等选择，来维护用户的选择信息，动态调整加入到提示词中，让飞轮转起来。

9 你最近有使用过哪些AI工具，你觉得哪个模型使用得比较顺手？

（其实吧，这个问题看似是个闲聊问题，但是当我真正复盘的时候，这个问题背后其实体现着企业是否对你跟上这个AI潮流的考察。 因为现在这个时代，其实很多企业问你如何Vibe coding，有没有啥技巧，用那些vibe coding工具，用什么AI产品，这些都是看你是否跟上潮流，有一些思考。 所以其实这些题目最好也是能反思一下，总结几个常用的比较新的AI工具，比如Skills出来以后，可以说使用**Skills Creator** 来生成Skill等。体现自己对最新产品的使用和思考）

10. 你怎么去了解一些新的技术？

同样和上面一个问题一样，看着是闲聊，其实背后在考察你是否在主动学习，学习方法聪明吗？我以后给一些答案供大家参考

11. 你在微软干的好好的，为什么要出来干这个？

12. 你想去哪种类型的公司？

（我能理解面试官问这个问题的初衷。因为这个公司也就100来号人，创业公司。而我之前都在上万人的大厂，面试官通常一定会有一些顾虑，你到底想不想去创业公司。所以如果大家面试创业公司的话，这个问题一定要顺着面试官说，表现出意向度）

算法题：

两数之和，三数之和

视频讲解

我用夸克网盘分享了「DeepWisdom大模型全栈一面.mp4」，点击链接即可保存。打开「夸克APP」，无需下载在线播放视频，畅享原画5倍速，支持电视投屏。

链接：https://pan.quark.cn/s/f2f648a357e6

2025.11 象映技术

2025.11 易鑫技术

2025.11 深维智信

2025.12 加速进化

2025.12 SAP

一面大主管，二面+1面，三面中华区负责人。没错就是先主管面，再+1面，有点奇怪。

全程几乎没啥技术问题，不知道他们考察的逻辑是啥，感觉这种全程面下来都没有太问技术的，那更多只是理解为他们更多的是

看背景选人了。

一面

二面

三面

2025.12 小红书

岗位介绍

反思

整个面试基本上围绕： **量化 + Multi-agent框架 + RAG数据库 + 模型选型**上四个方面展开。

面评：对量化模型和结构有一定了解，但应用比较简单，技术和方案设计能力单薄了些。

其实面试官说的也没有问题，不过对于方案设计的题，我没有答出最优解，好歹也和面试官在讨论，说了一些解决思路，但是面试官还是无情挂了我，甚至是写题的机会都不给（小红书的面试，写题的界面都准备好了，但是最后他不让我写），挂得很坚定，可以看出小红书的要求还是挺严格的。

反思：

1.我一直和大家说的是，做RAG系统的时候，你就要把你的文档弄得复杂一些，用上专业的数据，而不是像我一样，用了一个简单的文档，没有啥复杂的召回策略，都没有专业数据库，这也是面试官觉得我的项目很简单的原因，大家在设计项目上就应该避免。

2.方案设计这一块，之前面试官问方案/架构设计的比较少，这块确实薄弱了一些。不过最近面试几个大厂，多少还是在讨论方案和架构设计能力的，这块需要我们针对于面试题，整理提升下，如果有好的架构的资料，到时候我也会分享，总之这次面试是一个很好的机会，提升对于多Agent 架构设计的思维能力。

3.我可以看出来，面试官对量化/推理这一块并理解不深，你看他也都是问的一些概念性问题，相反应用模块，都是问得比较深入和细致。 这个其实反映了，对于现在市面上既要求应用，又要求算法的情况，大家不用太慌张，因为通常应用岗位的面试官，他就是问算法相关，他懂得也不深入。所以我们大胆复习，先把推理加速/训练的框补起来，面试的时候大概就够了。你看我对于量化这块也不是专家，不算懂得特别多，但是面试官的反馈还是说我对量化有一定理解。所以推理/微调这块不要怕，先按笔记资料学起来，把一套流程跑通，面试自信地讲就好了。

一面

1. 自我介绍？

2.看机会的理由什么？

3.你的两个项目你觉得哪一个挑战更大呀？项目做了多久呀？

4.可以详细展开讲一下这个项目吗？

5.我停下来它的核心的功能还是比较简单的，但是你又跟上了做了微调/蒸馏/量化/推理加速的技巧，我理解目前主流的模型是都可以做到的？为什么你还有做微调呢？

（这个也是一个和项目相关的问题，不太涉及知识点，大家看看就好）

这个其实是因为虽然主流模型完全可以完成相关功能，但是由于我的项目是要在资源受限的设备上（CPU）上去跑，所以需要加速推理速度同时不损失太大的精度

6.最终用了多少B的模型？

7.你是怎么做的量化的？

（说实话，我不能说对量化精通，这里我就是准备了自己做推理加速的一套方案，把这里的东西搞懂，去讲就好了，比如我总结的方案，**采用Q4_K_M,Attention层保持5-bit精度，FFN层压缩到4-bit，输出层用6-bit确保质量** 这几个关键地方弄清楚，一般其实应用的岗位面试官也不深入懂量化的，你回答到这里就差不多了，他也不会深入问和你问细节）

笔记部分　你的项目是如何做推理加速的？

8.量化的好处是什么，优缺点是什么？

（这个算是一个八股内容了，我总结在八股笔记量化部分。自己当时答确实可以答出来一些推理精度下降的问题，但是还是没有事后总结的答案好，因为现在我总结的答案，可以从**模型的结构，网络上深层次**地指出量化带来问题的原因，对于好处的总结也完整。所以有时问题虽然回答出来了，但是确实需要事后总结，反思，迭代出更好的答案，下次回答得更专业。）

8.1 你说到量化会降低精度，那么降低精度会有什么问题吗？

参考笔记　[量化的好处和坏处](#doxcni00b3jIxYxQLED3gt0H2yf)

9.量化在KV - Cache中起到一个什么作用呢？

（我觉得这两技术没啥太大联系，是两个不同的技术，我这么回答的，面试官也没反对。不过这两概念还是值得记一下，kv-cache和量化的定义，自己回答肯定答得没有这么专业。）

KV-Cache 在 Transformer 推理中缓存 Key 和 Value，避免重复计算，提升长序列推理速度。

量化：降低**权重或激活的精度**，减少显存占用和计算量。它们是两种不同的优化技术，但它们可以结合使用，目的都是为了**降低资源消耗、提高推理效率**。

10. 说到模型，对模型来说，他会产生幻觉吗?

（这个问题用笔记部分如何减少模型的幻觉就够用了，这个笔记不光总结了幻觉如何解决，还总结了幻觉产生的本质）

参考笔记 [如何减少模型的幻觉](#doxcndQEu9TGCHUjC6LN0WjjS4b)

11.多Agent 时，要怎么设计这个多Agent系统？

参考笔记 [6.1 设计多Agent的原则](#doxcnb8MVz52ROB3lXnLA7RNLad)

12.你的项目中使用的是什么多Agent的一个架构？

13.如果说你的子Agent膨胀变成20个，或者说是50个，你应该怎么去优化呢？

（这个问题我不太确定面试官想要的最终的答案是什么？这个问题其实就是在 多Agent的主管模式下，如果子Agent不断膨胀，那么这个Supervisor的压力可能会很大，这时如何解决，如果不知道什么是多Agent的Supervisor模式，请看这里：https://github.com/fzy2012/rhzl-Agentic-Design-Patterns-cn/blob/feature/interactive-website/13-Chapter-07-Multi-Agent-Collaboration.md。我的回答中，提出了使用Hierarchical的模式，但是面试官继续挑战这种模式如果一个分类错误，会导致后面都出错，然后我说可以分类的时候采用多Agent投票决策分类，加强分类准确性，面试官又挑战这样性能开销大，于是我说这个投票的策略可以是多个Agent并行的，但是面试官还是不满意，总之这题大家看到其实是面试官会不断和你探讨架构设计上的问题，需要有一些知识储备，才能像最优解靠近，我这个回答显然面试官并没有觉得最终探讨到了他最满意的解决方案。复习这个题请先掌握好八股部分多Agent相关资料，这是你回答这个问题的知识来源和储备。）

13.1如果说这个分类Agent，子Agent膨胀成20个，那么分类Agent需要分成20个，那么它的压力就会比较大了，这时该怎么办呢？

13.2举个例子，比如市面的元宝，豆包，它的咨询机器人，是会面向不同的行业的，每个行业的特性可能是不一样的，这样这个子Agent一下子就会膨胀的，这样怎么办？

（我这里回答了采用树型的结构，层级分类）

13.3 这是一个思路，这会导致一个问题，就是一旦第一层分错了，后面会全错，这怎么办？

（我这里又说了每一层采用投票机制，加强分类的准确性，导致每一层分类出错概率降低）

13.4 如果采用了这个方案，性能上开销很大，这怎么解决？

（我提到了说做一些并行化的优化）

（本题答案总结如下，需要多Agent八股部分的前置知识，比如中心化架构中，性能开销，优缺点，这些在八股部分都总结了）

在使用中心化多Agent架构时，系统复杂度会随着**r，n，k**增大。r表示主管和子Agent的回合数，n 是子Agent数量，k是子Agent迭代数，所以如您所说，子Agent 数量n增大时，系统复杂度增加，主管Agent确实会成为瓶颈，这也是这种架构的弊端。

为了解决这个问题，首先我们可以思考 是否一定要用**这个架构**：

比如**动态环境下（网络导航/检索）** 更适用去中心化，不会存在单点瓶颈。

强推理，长推理链 任务更适用于 单Agent。

可拆分且需要**统一口径与质量控制**的任务 适合中心化架构，如果确实是需要使用这种架构，我们可以采取以下措施来缓解

单点Agent压力。

1. 动态负载均衡

主管不直接管理所有子 Agent，而是通过**调度池**或**任务队列。**简单来说主管只负责任务分类，将任务丢到调度池中，然后采用专门的**调度器来解决任务的分配问题**。调度器本身就是处理高并发，自动化调度的，所以可以很好处理子Agent膨胀导致分类/调度任务增大的情况。

2. 采用分布式系统：

将这个Agent跑在多个节点，提高并发和容灾性。

3. 可以采用混合型架构

使用混合型架构，仍然保留了中心化的骨架，但是中心Agent管理的不是单独的Agent，而是若干小组，将顶层单点Agent压力从子Agent个数n下降到了小组数g，减缓顶层Agent压力。

4. 控制k，r

如上面所说，这种系统Agent压力和k,n,r相关，单点Agent可能会成为性能瓶颈是它的特点。因此我们设计的时候，就需要**尽量减少交互轮数 r 和 k**，缓解单点瓶颈问题。

14 你有提到小型的RAG,那么RAG的流程是怎样的？

（这个问题答案我不具体讲了，因为和我项目强相关，这个题主要有两个启示：

1. 我的项目用的sqlite做存储，因为我的文本简单。但是如我之前和大家说的，自己项目建议还是专门用专业向量数据库，做切块，用专业的召回策略去召回。
2. 面试官其实问得非常的细致。之前很少面试官问这么细的，包括你存，存了什么表头，怎么query的，因为我写的代码我都有点忘了，我自己都没那么细致，答的不是很好。这块告诉大家你需要在这个RAG存储的地方，了解得非常非常细致，包括你如何存的，表头是啥，甚至是向量模型的参数（14.3），这些都要弄清楚）

14.1 你当前的数据是怎样的？

14.2 你具体存储的数据结构是怎样子的？

（存了表头吗）

14.3 你用什么向量模型，用的多大的size？

14.4 相当于说检索出来之后，用什么对他匹配的？你的query是什么来着？

15 这个项目为什么要用GPT4?

15.1你现在觉得你在这个过程中你应该选择什么模型？有更新的为啥不用更新的，而且mini的消耗还是比较低的？

（这个问题我总结在了笔记 [**不同模型的对比和选择**](#doxcnT5TFxJwhwL4lQDGlRjnfPe)**，**对应的笔记和复习思路请参考笔记。 这里我回答得不好，因为在我的项目中其实没有专门做模型对比然后选型，相信大家自己做项目，可能前期也不会有专门的选型，但是即使你的项目选模型没有精心考虑，但是在回答这个问题的时候，还是需要你对主要的模型有一个对比，不同模型能力区别有一个了解，然后结合你的项目自圆其说）

> 具体请参考笔记：[不同模型的对比和选择](#doxcnT5TFxJwhwL4lQDGlRjnfPe)
>
>
> ###### 视频讲解
>
> https://www.bilibili.com/video/BV1aToUBwEBX/?vd_source=144bec9c3f54e465073138bed788be1b
>
>
> ##### 2025.12 阿里夸克千问
>
>
> ###### 岗位介绍
>
> 岗位描述
>
> 1、负责夸克千问、通义APP内的大模型应用研发,熟练利用大模型相关工具和平台,实现诸多AI应用的快速落地与持续迭代;
>
> 2、负责大模型产品的整体技术解决方案,基于对大模型架构和Agent框架的深入理解,合理技术选型,参与技术研发和效果优化等工作,推动产品持续增长;
>
> 3、保障大模型应用系统架构的稳定、高效运行,帮助业务优化性能和改善系统稳定性,持续提升用户体验;
>
> 4、持续跟踪前沿技术趋势,关注并探索AI新应用、适时引入新技术新方法,持续提升产品技术、工程架构上的先进性。
>
> 岗位要求
>
> 1、计算机或相关专业本科及以上学历,2年以上工作经验,熟练掌握C++/Java开发,熟悉Linux系统下程序性能分析及优化能力,有大型项目/系统负责经验者优先;
>
> 2、参与过大模型应用系统/业务研发经验者优先,熟悉RAG、Agert机制,了解包括langchain、vllm开源系统,有prompt工程业务优化经验、以及模型finetune者经验优先;
>
> 3、有优秀的架构设计能力,有相关大型系统设计、开发、优化经验,了解检索引擎、kv存储、在线服务框架、批流处理等中间件系统;
>
> 4、具备良好的产品和业务sense,有大模型应用的深度使用、开发落地经验优先;
>
> 5、有实时通信业务(IM文本、推送、语音通话)后端研发经验优先;有丰富搜推系统研发经验优先。
>
>
> ###### 反思
>
> 一面：整体来说其实很简单，算法题很简单，当然我完成的也很快，但是阿里的面试官还是给了这个算法题30分钟时间，相比字节的算法题目，真的显得非常的友好了。整个一面其实涉及到知识考察的部分几乎没有，甚至可以感觉有点像是一个电话面。基本是和面试官在聊项目（并且停在功能层面，没有深入到技术层面）。面试官人也很好，他一直在肯定我，我回答完他经常说，挺好的这种话给我鼓励。整个时长也就44分钟（并且包括了写题和反问），在这次面试中其实确实没有一些特别干的知识方面的东西和大家总结。另外我觉得背景可能加了分，因为这个岗虽然是Agent开发，但是使用C++，面试官提到了C++国内就是你们公司的最好，所以我估计这点也加了分。因为本期几乎都是一些项目相关的问题，比较碎，文字不太好总结，要想更细致知道当时发生了啥，可以参考笔记后面的视频讲解链接。**我觉得唯一可以抽取出来的一个知识点是第3个问题**，如果项目使用了本地模型，可以了解一下不同参数量的模型在不同的硬件上的使用情况。

二面：二面其实还是挺有价值的。面试官对我说了一句话是你来了这里，会把工程做到极致。 回看面试官的问题，确实是从工程的角度上提出了挺多有价值的问题。虽然看上去这些问题是针对于我的具体的项目展开的，但其实这个**讨论是有通用性**的，具体的问题不重要，重要的是**这些工程的问题和思维**，在不同的项目上都会遇到。想要把这些问题回答得很好，需要：

1.一**些知识的积累**，比如我准备的Agent架构相关论文，相关书籍，需要你看进去了，并且真正理解了，会灵活运用。大家看后面的回答可以看出来，基本我的回答还是从我的八股里总结的参考资料下手去谈的，所以我们不光要记住总的最高频的八股，也需要看对应的参考资料，去理解，这样遇到这种发散性讨论的问题才可以侃侃而谈。

2.一些自己的思考，这个可以结合一些具体的面试问题，自己去思考，运用你的一些知识，比如面经部分包括**小红书，华林证券，字节豆包，**都有相关架构的设计，大家结合笔记和讲解，通过这些题目思考和复盘，也可以提升这块的能力。

最近也确实有一些公司喜欢考架构，考一些具体问题的探讨，这些东西稍微比纯八股，纯知识难一些，但是其实经过一些相关面经和相关资料，面试几次，这块也不会很难，并且这些问题通常也不会有完全标准的答案，有一些自己的思考，能够回答上来也不至于太差。这里我觉得还挺有价值的，我在讲解视频也会谈一下更多自己针对于这些问题的准备方法和思路，和技巧，笔记也会尽量写出思路，希望对大家有帮助。

HR面：HR面个人感觉没什么特别好总结复盘的。 面试之前有人提醒我说阿里的HR喜欢压力面试，我还专门咨询了一些阿里的朋友。不过后来觉得又不考技术，压力能压力到哪里去呢？自信表达，不踩阿里的红线就好。唯一一点我想说的是，别把姿态放太低了，这样HR觉得你接的意向大，就会压你工资。具体问题我还是整理出来大家看着参考一下就好。

一面（44min）

上来首先是手撕一道算法题目，并不难，很快就做出来了，题目如下，翻了一下记录，我大概是不到10分钟写完的，面试官的预期是给30分钟，确实是一道很简单的题目，如果要难度分类，应该是Leetcode esay 题目。

算法题：

Leetcode 792

1. 自我介绍

2. 按摩房预约系统是你现在在做的吗？

（这个面试官就是单纯的好奇，为啥的项目中会有一个很奇怪的和按摩房相关的项目，看上去就很奇怪，这也就是我在这个笔记第一章项目思路设计的时候就和大家说的，设计项目的时候就要尽可能往工作上靠，像我这个项目一开始设计就看着和工作无关。但无论如何，这就是走过的坑，希望大家尽量避免，话说回来，即使我这个项目立意就不好，但是这次面试还是挺尴尬的，但是如果现在让我重新再来一次，我不会设计一个这么奇怪的按摩房项目）

3. 你这个智能日历系统是在本地运行的吗？那你们本地用的是哪个模型呀？

   3.1 那你们本地跑得起来吗？

  看[这里](#doxcn4KQJBVRdjGPfxU9lNmn2Pc)，这个问题我专门做了详细的说明

   3.2 你们微软自己的模型可以打平千问是吗？

   （看上去有一些是在纯聊天了）

3.3 那你在本地一个7b的模型能做啥呢？

（还是聊一些和千问模型相关的内容）

3.4 整个项目具体细节的澄清。

（xxx还是在和面试官讲我们的项目具体做了什么）

3.5 还是在问项目，而且是停留在了功能层，不是技术层，对大家其实帮助不大。

3.6 ...我觉得基本上已经扯得很远了，就是在讲公司的一些产品层面的东西，几乎纯聊天，几乎已经扯到其它产品上了。

4.扣项目，问了一个简历上写的和大模型无关的另一个项目的具体细节。

5.你上一段工作的离职原因，大致问了一下上一段工作内容。

二面（55min）

1.自我介绍

2. 你为什么要换工作

3.你倾向的工作地点

4.挑一个项目讲解（自己任挑，甚至是说是Agent/非Agent项目都可以）

5.讲一个你在项目中遇到的最困难的问题，以及你是如何解决的？

（这个问题我在总结字节豆包的面试中就总结过了，我认为大家需要针对这个问题，整理一个自己的答案，然后把这个问题解决的过程讲清楚，讲出深度，我暂时会讲我解决模型工具调用失败的这个问题，大家可以参考笔记）

参考 4.[你在项目中遇到的最难的问题，如何解决的](#doxcnpqeoSutE489zhUPcWrUaBb)？

6. 这个里面调用了多少个工具啊？

7. 如果你在cpu上跑，选一个很小的模型，模型可能做不了很多事，这个怎么办？

（这个问题的回答涉及：Infra层比如微调知识，工程学相关知识。我的回答更侧重于工程角度，因为这是一个工程岗。不过值得一提的是，如果是Infra背景的面试官，可能会更注重听Infra的优化的知识，这个还是视情况而定。

我的回答，第一个模型推理的难度，这个其实是在美团龙猫的论文中总结的，我们拿来用这个思想，这一个论文其实在整个笔记中涉及多次了，我就不赘述。

最后的总结部分，我也引用了UC Berkeley的论文做总结，丰富了回答，也侧面像面试官展示了我们在关注前沿知识/论文的能力，关于这个论文，可以参考：

https://36kr.com/p/3608851367871752）

对于这个问题，核心的思想是有两个：提升**模型的能力，**和**降低任务的难度。**

首先**讲解提升模型的能力**，我们可以通过：

1.通过微调，强化学习让模型能够在指定任务上取得更好的效果。

2.降低任务的复杂性。Agent系统中任务的难度主要体现在3个方面：

接下讲降低任务的难度，通常来说，Agent系统中LLM的难度主要体现在：

推理维度：Agent是否可以在部分可观测，信息不完整的环境，将线索整合并做出正确抉择。

工具维度：在众多工具依赖图中，Agent是否可以选择正确工具，使用正确参数。

交互维度：在多轮对话中，是否能够主动识别意图，主动澄清问题，维持上下文一致。

针对于小模型，我们可以通过任务设计的角度，比如拆分成任务，限制步骤，减少工具推理难度等方法，去避免这三个维度的难度，让任务复杂性下降，使得使用小模 型也能够取得很好的效果。

值得一提的是，我看过UC Berkeley 25年12月发表的一篇论文，结论是即使在目前生产级别的Agent项目中，也通常将Agent步骤限制在10步以内，也会使用比如添加抽象层的方法降低工具推理难度，所以这些方法其实不止在小模型上需要使用，在大模型的项目中一样需要。

8. 时空冲突这个事情，假设不局限在当前的状态，未来理想的状态下，你认为还需要做哪些事情啊？

（这个虽然是针对于具体任务：冲突检测讨论的。但其实面试官根本不在乎具体任务是啥，甚至也不在乎你做得多深，而是在和你讨论。 这个过程会把我们学到的一些知识串起来，这个过程是值得大家思考和借鉴的，有的面试官就是不喜欢问八股，喜欢发散性讨论，这个整个讨论过程还是希望大家感受一下的，因为有的面试官就喜欢这么和你聊，和你探讨，这个碰撞的过程本身就是一种能力，我们需要在这个些面试笔记以及这期的视频讲解中去提升自己这个探讨和碰撞的能力）

8.1 这个冲突怎么能让模型解决得很好，还是说你一股脑塞给模型，都让它来处理呢？怎么样能让模型整体的质量比较高呢？

（其实这个问题的思路和第7个问题有点像，还是可以从**Infra提升模型能力** 以**及 工程角度**，降低模型推理难度这两个角度去回答，另外加了一点提示词工程的回答，其实这问题也没有标准答案，就是思维要灵活，和面试官讨论）

首先，在不考虑性能，成本的角度，我们**先考虑提升模型的能力**：现在模型的能力越来越强，我们先去选择更强大，更合适的模型看是否能解决这个问题。或者通过**后训练的方式**，比如使用SFT，输入一些冲突检测相关的数据，强化模型对这个指定任务的回答准确性能力。

如果模型确实无法处理，从工程的角度，我们可以：

1.从LLM层解耦出一些具体的任务。模型任务处理的复杂度主要体现在**推理，任务调用，交互的维度。**我们可以抽离出一些具体的任务，比如之前判断冲突的时候，是让模型自己去调用距离计算api，然后做时间加减法。这时我们可以抽离这个任务，在语言层面调用工具计算好距离，然后以提示词形式填充进去。降低了模型理解工具的负担，另外模型不擅长做精确的时间加减法，我们也将这一任务交给了语言层面去做，不要把所有压力都给模型，而是让整个系统在人为定义好的 任务流程图 的格子里去填空。

2.从提示词工程角度，可以使用COT（Chain of thoughts)来完成，抽象出具体的任务，让模型一步一步思考。

8.2 如果这个冲突检测的逻辑结合一些个人的个性化情况来改变，比如有人比较注重生活，那么就会冲突检测更多让工作给生活让路，如果要实现这个功能，需要哪些数据，怎么实现?

（这个问题也有通用性，就是在你的Agent中，如何实现根据用户的行为，喜好来动态改变它的决策逻辑，这个如何能去实现呢？）

我们需要维护一个**用户画像**和**历史决策库，**在运行时具体展开来讲：

我们可以**维护用户一些历史的数据**，比如过去冲突检测的行为，点了接受建议还是拒绝，过去一些手动修改冲突的例子，把这些数据维护起来。

然后在真正执行冲突检测的时候：1.先去看是否有历史完全匹配的相关冲突和解决方法，如果有，那么直接选择这个逻辑。2.如果没有，可以命中一些最相似2\~3条例子，然后作为历史相似案例，提取出用户的倾向，比如"User previously prefer to cancle work meeting to balance life " 然后作为提示词输入到冲突检测的提示词中。 让模型在选择时有一个倾向。

8.3 那如果冲突检测这一块使用多Agent去设计？你该如何去设计呢？

（这又是一个经典问题了，最近其实面试被问到了很多，具体问法就是针对于某一个case，你如何设计这个单/多Agent架构。这块复习策略：

1.去看我的多Agent笔记的论文和问题，论文里讲了设计原则，思想，给了例子，在金融/网页规划等例子下用什么架构，为啥，这个就是现成的例子，把这篇论文和我的笔记弄透，包括在笔记中我也直接汇总了架构设计的典型例子，大家都能参考。

2. 结合面经有更具体的问题，用这些问题再巩固，比如这个问题，小红书，华林的面试都有类似的架构设计，自己先别看答案，想一想怎么设计，然后再结合你自己的例子和场景，给自己出个题，看你会不会架构设计）

我已经将这个问题抽离到了八股的 架构设计小节，请参考： [时空冲突检测任务](#doxcnhuBnst4BZMtDNSYStJu21f)

8.4 那什么时候用多Agent去做，什么时候用一个Agent去完成？举具体例子说明。

（这题在八股部分多Agent地方已经总结得很好了，甚至是很多次，我下面回答基于八股部分的知识，我这块不用参考任何资料就能直接回答，如果你觉得这个问题还回答不上来，那就是对应的八股笔记部分的多Agent部分和参考资料没理解好。以下这个回答都是基于我八股的笔记和参考资料。）

单Agent的优势在于**不需要承担Agent之间的协调**，通信开销和拓扑带来的误差传播，在**顺序调用，工具密集，单Agent的基线收益已经较高**（达到45%的时候）使用单Agent效果会更好，比如说典型的工作流的例子，这种情况下使用多Agent性能反而会下降。

多Agent的优势在于在复杂的任务中，特别是**需要分工和并行**的问题上，可以突破单Agent的**能力边界，提高效率和鲁棒性**，比如说金融推理的任务，更适合于中心化的多Agent架构。 网页搜索任务，更适合于去中心化的多Agent架构。

8.5 单Agent有什么局限性吗？什么解决不了吗？

(聊着聊着也会聊到一些八股内容，参考这个笔记，虽然问题不是一模一样，但是这个笔记中多Agent的优势不就是单Agent解决不了的吗？另外我相信大家在八股笔记我总结了很多，这个最基本的问题应该是可以回答上来的。）

参考笔记[单Agent和多Agent的对比](#doxcni98OyeUxSDBTv2Rm3Et8Dg)

8.6 你提到的根据用户的搜索记录，来分析用户的喜好，改进用户的行为，这个是怎么实现的呢？

是将用户的搜索行为当作信号，从这个信号中提取**特征，**最后把这特征转换成用户画像/向量，最后在LLM推理时用这些**画像进行加权，重排，**根据这个画像找到对应的个性化参考资料，或者直接将画像加入到提示词中做**提示词增强**。

8.7 你提到了Agent会根据用户的历史行为、喜好来智能地改变这个检测冲突的行为，是要根据哪些数据来调整Agent行为，一开始刚上线的时候并没有一些用户数据，这时又要怎么做呢？

最开始没有用户数据，可以采用冷启动的方式先去上线，主要的核心思想是**降低打扰和风险，以稳妥和安全替代个性化的抉择，**让Agent随着一边上线，一边收集数据，分析数据，逐步了解用户行为，给出更加定制的建议。

8.8 怎么避免一些极端不合理输出的corner case？

(我不能说这个是标准答案，这个答案是根据学习笔记中llm的参数 和 如何避免Agent出现幻觉 这两章内容总结的。我想说的是理解了这个笔记的很多内容，在遇到一些新问题时我们也能灵活应对）

模型的不合理输出，通常是因为模型生成的回答置信度比较低导致的。我们可以通过强化学习的方法，让模型在对于低置信度的时候来进行一个兜底的回答，而不是一个不靠谱的回答，或者是通过修改模型的生成参数，降低温度收紧topK，减少模型输出极端Case的情况。

最后其实可以设置LLM生成后的检查，我们对LLM最后生成的结果再进行一次筛查，如果有比如极端言论或者完全不能接受的回答，直接拦截，给到用户一个提示，现在很多Agent 产品其实也都这么在做。

9. 在项目过程中，怎么去做提示词的压缩和裁剪？

（这个我觉得挺好的，有哪一些工程手段减少提示词，有点像提示词工程，但基本上提示词工程都是讲如何写更有效的提示词而不是怎么压缩。 但这个问题说实话还是第一次见到，答案也不难，所以我认为没有很具备典型性，我就不把它抽象到八股笔记中，就把答案放在这里，大家阅读，理解一下就好）

从工程的角度来说，我们需要保持在描述清楚问题的前提下，尽量减少提示词/输入Token的消耗，可以使用以下策略：

1. 可以使用一些结构化语言，比如Json格式，编号格式来简洁分明地描述事实，规则和要求，避免冗长的自然语言赘述。

2. 对于一些上下文内容，使用压缩，摘要地方式保留上下文，而不是保留原文。

3.我们在设计提示词的时候，尽量将静态内容放在最前面，将变量放在后面。这样可以让kv-cache的机制尽量命中缓存，复用KV块。

4.检查提示词是否有一些重复，冗余，噪音。只写AI不知道的内容，它推测不出来的东西。比如说代码中要用UV 不用PIP.而不要写一些AI知道的，或者宽泛的内容，比如系统的目录结构（他自己能推理出），比如请遵循现有Coding风格（AI训练的时候它就知道）。写这些废话和宽泛的话，反而会增加AI的推理难度。（这个观点可以看笔记论文部分：Evaluating AGENTS.md -Are Repository-Level Context Files Helpful for Coding Agents?

9.1 我提到了kv-cache 可以避免输入token的重复计算，面试官问那你从工程的角度上来说如何能让kv-cache更好的发挥作用？有没有一些可能，你的工程使用不当导致kv-cache并没有生效？

(因为我上面提到了Kv-cache，kv-cache其实有点像是Infra的知识，不过即使做工程，还是建议一下KV-cache是什么，有啥用，其实也不难。

通常聊到kv-cache，一般都会问kv-cache的机制，但是这个面试官问的是工程的角度怎么尽量利用上Kv-cache，可以看出这个面试官很浓的工程背景，这也是从另一个角度让我们了解一下kv-cache相关知识）

回答其实就是上面的第三点，面试官问这个问题原因是因为我提到使用kv-cache，但是没说Kv-cache 在工程层面怎么用。

10 Langsmith的基本原理。

（第一次被问，我先不总结了，大概就是问Langsmith的原理，模块，设计思路等，大家问一下AI都有答案，而且截至目前为止，也是第一次被问到，应该算比较低频）

11. 你的Agent的性能评估是怎么做的？

（必考问题了，在Agent评估笔记中，我不但总结了相关理论知识，也结合我自己的项目告诉大家如何去准备，大家直接参考笔记就好了）

参考笔记 [Agent性能评估](#doxcnbpKwJs393fsRaDVMYRg89c)

12. 你之前一段经历做音频相关的工作能讲一下吗？

(这个问题不展开讲了，不过通过这问题可以看出的一点是，即使是Agent岗，偶尔也会问一下你之前经历，面试官可能会感兴趣，比如说这个面试官问我音频的东西，我推测是他们做千问的Chat，里面也有音频通话这块，所以啊看到音频经历，多少是感兴趣的，所以问了一下）

HR面（30min)

1.自我介绍。

2. 讲一讲你在公司主要做了什么项目吧？

3. 在你们老板的眼中，他是怎么评价你的呢？

   3.1 为啥他给你的反馈是正向的，但是你上次没晋级呢？

   3.2 你自己觉得你有哪些缺点

4. 你来评价一下一面二面的面试官吧？

（我说了他们技术很好之类的）

   4.1除了技术方面，你觉得对他们还有哪些评价？

4.2整个面试过程，你的感受如何

5.你说你每天6点下班了，在我们这里可不是这样，你来了阿里也会6点就下班吗？

6.我看你拿了好几个offer的，未来还有一些在聊的，你是怎么考虑的？

6.1 你自己预期薪资是多少？

（就是我说希望至少和另一个offer持平，这里面试官说他们给那么多是他们技术不好，你觉得我们的技术和他们能一样吗？我们有自己的考量标准的）

7.你为什么要离职呢？你的职业规划是啥？

视频讲解

https://www.bilibili.com/video/BV1ngEB6REzJ/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2025.12 华林证券

岗位介绍

一共四面，没有HR面。4面应该是加了一面，因为我要的工资比较高，HR得专门申请。如果不是我这样的话，估计没有4面。1面是业务技术，2面是主管，3面是他们合作的博士导师， 4面是交叉面，找了做算法的同事来面。岗位Title叫：大模型应用工程师（智能体方向）

反思

一面技术面中规中矩，就是常见的基本经历考察，算法题，Agent,RAG考的也不难。

二面主管面：证券类的公司，最喜欢问的就是你能给证券行业带来什么？你了解证券行业的业务？你的技术如何用在我们的业务？一个建议是，你可以在技术面中好好询问清楚这个岗位的业务是做什么的，了解清楚以后想一下你在这个业务中有什么优势，可以给他们带来什么，想一想话术。

三面是个博士生导师吧：其实就是问了2个Agent场景设计题，这个可以笔记总结的： 多Agent架构的论文 我会整理在论文章节，其实在多Agent架构八股部分也提到了。通过这些前置知识结合三面的场景设计题来复习。

四面就是纯粹一个交叉面和加面，因为涉及的涨幅比较高，所以他们加了一个做算法背景的人来面试。考察了我手写注意力机制。我估计是来摸底你算法的功底，因为前面的面试官是应用出身，岗位也是应用岗，前面的面试官摸不出来你的算法功底。 所以这也提醒我们，如果想要面试竞争力和价格再上一个台阶，学一点算法的东西也是有必要的。可以照着笔记中 **大模型算法0基础学习全记录走一遍。** 不过再次强调，算法还是属于进阶路线，先把前面的应用部分弄好了，算法了解一点，offer肯定就有了，有余力再学习算法进阶部分。

一面

1. 自我介绍。

2. 你在上一段工作主要做的是音频SDK吗？取得了一些什么样的成绩呀？

（好像社招面试普遍要问一下上一段经历，有朋友问社招要准备上一段经历的项目吗？所以答案是建议稍微准备一下，最少准备一下上一段经历做了啥，项目的描述这些基本问题）

3. （追问）你觉得Unity WebGL的核心是啥？

（大家可以忽略，我上一段工作相关的内容）

4. 你可以讲一下音频sdk的原理吗？

（上一段工作相关内容，讨厌别人老问我音频的内容，我早都转行了，也投递的不是音频的岗位，还问那些有啥意义啊）

5. 在上一段工作主要学到了什么？

（问了5个问题了，还在聊上一段工作，还问这么虚的内容，这些问题我就不复盘了，我主要讲了一下学到的知识的广度，还有一些工程规范）

6. 你这里讲的游戏主机迁移是啥意思？

（还在扣我上一段工作的简历的具体细节。可能单纯是好奇吧，感觉和这个证券大模型智能体一点都不相干，但面试官就是要问）

7. 你在现在的公司使用的主要语言？

8. 介绍一下这个智能日历系统，当时为啥要做这个项目呢？

（整体来说第8个问题其实就是介绍项目，询问项目的基本功能，其实没有什么特别要注意的，就是正常和面试官交流表达）

（这个是我自己做的第一个Agent项目了）

8.1 这里为什么涉及多账号？

8.2 这里会不会有隐私问题，因为要读邮箱。

（这里就是说的需要用户授权）

9. 你这里写的Semantic Kernel是啥？

（这个就是微软的一个Agent框架，因为我用了，所以简历写了，面试官就问了。其实可以看出来到目前为止，面试官根本就没问什么很难的问题，基本就在问你的经历。其实很多面试确实就是这样，面试官自己本身不一定也很懂，也问不出来个啥，更多的是问你，而不是考你，自己能讲清楚就行了）

10. 介绍一下按摩房预约系统

（这个就是在我的小红书上大家可以看到的一个项目，就是自己做的一个Agent项目，当时啥也不懂，要我现在看我肯定觉得有点搓。现在我设计一个Agent项目可能会从项目意义以及架构上去思考，而不是像当时一样啥也不想就拍脑袋就做一个。这个项目在小红书上可以搜到。就是这么一个，我自己也觉得不咋地的项目，我确实是把它放在简历上面，作为我的项目之一（当时一共写了2个Agent项目）。有些人觉得80w，那可能得有很好的项目做支持吧，但确实我就是用这些项目去面试的。其实不说自己做的AI项目，就是我的工作的一些项目，其实每天做的事我觉得也没啥可拿去说的，可能世界就是一个巨大的草台班子吧。

另外目前总结到这里，整个面试还是像流水账一样，基本上没问什么有深度难度的问题，基本上就是面试官提问，我在介绍，有时面试好像就真的没那么难）

10.1 为什么用Langchain不用Semantic Kernel？

八股的Agent常见的框架，总结了二者的区别，照着回答一下就好。面试官也不追问。

11.为什么设计成多Agent？

笔记的八股部分有总结：单Agent和多Agent的区别，结合业务去展开讲一下。

12. 那有没有涉及到有同时要调多个的，或者说多个之间还有依赖关系的？

这个问题一般有6种模式。

1. 完全独立。比如对于UI debug面板，chat初始化等。
2. 独立但是需要限流：适用于任务很多，但是不能全部同时跑的（比如因为资源限制，API Rate Limit的场景）比如媒体理解批量处理，Cron多job并发执行
3. 严格顺序依赖：多个操作必须串行，比如同一个文件的读写，后一个操作依赖于前一个操作完成。
4. 重复请求合并：多个调用方同时触发同一个操作，只需真正执行一次，其余共享结果。比如消息处理去重，GateWay连接。
5. Pipeline：步骤之间有数据依赖，后一步必须拿到前一步的输入作为自己的输入。
6. 异步父子编排：主任务派生子任务，子任务完成后回调给主任务继续。

可以结合自己的业务场景，使用这6种不同的模式。

13. 端对端的延迟是多少？

这个指标请注意一下，面试官真的很喜欢问这个。包括我刚总结的RAG项目面试真题。这里指的是你的Agent从输入，到第一个输出的延迟。正是因为面试官喜欢问，所以我在rag项目才设计了详细的延时的记录。

14 薪资和职级讨论

15 算法题：**复制带随机指针的链表**

（面试官原话如下，是个经典的题目，网上可以搜到）

这有一个带随机指针的一个链表的一个拷贝，反正要求就是不借助于其他的东西，直接基于链表本身，能去拷贝一个跟它的结构一样的。

二面

整体来说一点不难，全是总结过的内容啊，你把笔记认真看了，这些都是出现过的内容。

而且面试官本身又不追问，感觉都是靠你讲。都没啥标准答案好总结的。不过每一个题目都给了思路。

1. 自我介绍

1.1 在腾讯当时是实习以后留用的吗？走的时候是多少职级，绩效呢？

> 之前我觉得面试官问一些无关紧要的问题，比如什么实习转正，用什么VibeCoding的工具，这些是无关紧要的问题，但是现在我觉得每一个问题都有背后的逻辑和考察。比如实习转正，面试官估计觉得实习转正比较靠谱，算是一个正向信号。毕竟面试可能就是几个小时的了解，但是实习转正是几个月的考察，如果你实习转正了，说明你确实还是靠谱的。面试进腾讯 和 实习转正进腾讯，后者含金量更高。
>
> 第二个问职级，问晋升。常规操作了。如果你一直没晋升，面试官可能会觉得你是晋升不了才换工作，绩效更是，面试官肯定喜欢高绩效。 但是大家也别过度焦虑了，面试官都想要落难凤凰，想要高绩效，火速晋升的。但高绩效，火速晋升的谁没事来找工作呀。

1.3 "技术栈我看前面是c++，在微软呢，用的啥"

> 不知道咋分析，技术栈就是匹配肯定更好吧，面试官有自己的考察，但你也没法强求。但是也说明了选择一个好的方向很重要，就比如你现在去做oc ios开发，那你技术栈太窄了，找工作多困难啊。但是你做大模型，岗位就多。所以尽量想好方向，尽量去找上限高，岗位多的技术栈吧。这也是为啥我一直在这折腾，之前大模型没火我要从客户端转后端，现在想从普通开发转大模型。

2. Agent开发项目介绍

2.1 框架选择

> 也是一个常考的问题了。主要是考察你为什么选这个框架，为什么不选xx框架。因为我的两个项目，一个用的langchain,一个用的semantic kernel。 所以经常被问这二者的对比。这些框架的特性笔记里已经有了，大家去看。Semantic Kernel对大家无参考价值，请大家忽略，我司自己研究的没什么人用的框架。
>
> 大家准备找个问题，就是准备主流的框架的特性，然后对比，说清楚自己的选型。有些人说Langchain怎么学，我觉得从面试的角度，你会用就行了，然后就是理解Langchain的特性，特点，结构，优缺点，组成模块，选型，面试基本问这些，笔记里总结的都有。倒是没有特别的必要去专门学什么langchain的语法啊之类的，因为问的少，而且框架的这个东西，很多公司都是用自己的框架，你把langchain学那么深，人家公司根本不用langchain，他也问不深啊。

2.2 这两个框架在多智能体协同的之间怎么选？

> 其实我觉得面试官可能本身也不是特别懂，也不会问深。我回答的就是langgraph 以图的状态机的形式，langgraph多Agent之间通信方式。这些都是常见八股了笔记里总结的有，而且langgraph本身就是来做multiagent的嘛，所以我就说langgraph这块的优势，其实是很常规的回答了。面试官也不追问。

3. 讲一个你做项目遇到的最难的问题？

> 我之前就说，这个问题就是一个必须准备的问题，所有人，不同背景都要准备。这个题上限高，下限低。你要说得很出彩，面试官还要请教你，那你就稳了。我的建议是所有人都得准备找个答案。另外我笔记的： 其他问题- 你在项目中遇到的最难的问题，如何解决的？里面总结了两个问题，一个是agent模型调用，一个是rag - mcp的问题，这两个比较通用的问题，你都可以拿去回答。当然你自己总结，总结得更深入更好，但这个问题都得事先准备，因为深度和描述是靠挖掘的，别想着现场发挥。

4. 你们怎么做监控的？

> 常考的问题，自己也要准备好答案。讲Agent评估的理论 笔记里有，包括还推荐了论文（美团龙猫的）。 监控有线上监控，线下使用测试集评估。测试集评估如何设计，设计那些方面（性能，轨迹，推理）等方面讲清楚，我经常就用美团龙猫那个方法，从三个维度：推理，工具调用，规划维度上 构建标准和面试官讲。这部分检测的部分，笔记里面有，并且面试真题也有，答案很多，大家可以自己看看笔记。

5. 智能体的就怎么去做优化以及说你们做了哪些优化？

> 其实我觉得面试官也不太懂，因为所有的问题都是在问我，我也不知道他是不是都听懂了？所以完全是单向输出。单向输出的好处是不会被拷打，只要问题你准备了，面试不就那些常见问题吗？我们整理的都有。那你回答就完了，有点像背答案。
>
> 这个问题就是我回答的就是三点：1.架构选型。如何选Agent架构，为啥（依据是我给大家推荐的架构的论文，八股都有），讲清楚理论。
>
> 2. 做微调，强化学习，量化。这一点在笔记部分你是怎么做微调，推理加速的总结的都有。
>
> 3. 上线迭代，数据回收，监控，再训练（数据飞轮）
>
> 这些内容笔记里都有，这里回答也是把他们串起来。其实我没有说我做第三步迭代，因为我的项目没上线嘛，自己做的，我实话说，我的项目没上线，但是上线后应该这么做。 反正最后面试官对我评价也非常正向，甚至是优秀地加了一面，我想说的是别太担心了项目没上线吧，当然有落地经验一定是好事，可是大家都是转行，怎么办呢？
>
> 这块上线这块，理论你也可以吹，自圆其说。但是我是偏保守的，我怕说漏了。但其实真的可以吹牛，我就说上线了咋了，只要能说清楚，我认识很多人其实比我大胆很多。

6. 怎么去测评你上线的数据?

> 从几个维度:
>
> 1.比如有rag系统，可以统计ragas对吧，后台维护一个库，每次query都可以算ragas指标，ragas我们八股，项目都有。

>
> 2．各种指标，比如工具调用，任务完成率，轨迹正确性，延迟，成本，想好这些针对于你的项目是怎么衡量的，比如你让AI写一下衡量这些指标的方法，知道怎么实现的。然后你就说，实现后把它们打点传到云端监控。
>
> 3．可以设置一些具体的业务指标。这点是对于社招的同学，项目上线以后。校招可以就算了。校招项目你能做前两点都不错了。 这一点是你的业务增长，比如你做了Agent客服，那么上线后你的用户满意率增多了，用户日活等这些业务指标，也需要统计。
>
> 想好这些维度，怎么做的，准备好一个答案。

7 你这个微调主要为了解决啥问题啊？

> 都是总结过的啊。其实这里也反映一个点就是，这个是我去年12月份的面经了，那时候我还没系统学算法，我一直在笔记里写的就是知道自己微调怎么做的，知道概念，让AI写个代码看看，实际我也真的微调。然后我准备了我是怎么做的微调，总结很好（在笔记中大家自己看）我就去面试，我就说我做了微调。其实你看面试官很多他问不深入，甚至我觉得他可能也不是很懂微调。当然我绝对不是说就没必要专门做微调了，我现在第二阶段是在学算法，深入学微调嘛。我说的意思是对于应用岗，大部分人来说没太多时间学算法，包括面试官也是做应用他可能也不太懂，所以你时间也没那么充足，你就少学点算法。 对于懂算法，真的做了微调的人，肯定会问得更深的。有时间深入学，我系统学习算法的时候其实都已经复习了8个多月了，不是每个人都有这么多时间的。没时间的话，微调就按我的笔记策略准备就行。
>
> 回到问题本身。微调本身解决啥问题，这是八股，结合自己业务再说一下，比如精度下降，分类成功率降低就好了。

8．我现在还没有听到你提rag项目，是没有吗？

> 去年12月份，我确实项目以agent为主，rag比较弱，但是我确实是拿到了很多offer，即使我rag比较弱。但这也是为什么我12月又开始做一个rag项目，就是笔记里这个很多人都拿去面试了的项目了。做了这个项目，我再把RAG融入进去，技术栈就更完成。
>
> 面试官有的时候真的问不深，他可能自己也不懂太多rag。但是他要问你，你RAG经验，他会记在小本本上。如果确实没做或者经验浅，那么就扣点分呗。我当时我要是做了这个rag项目，我估计面试评分会更高。

8.1 RAG优化策略

> 虽然我当时没做rag，但这题我会啊，这八股笔记总结的有。我可以答出来这个问题，但当时我没做项目，我底气不足，现在有了这个rag项目，我再回答这些问题，肯定答得更好，底气也更足。这个问题就看八股吧和我们项目吧，做了我们RAG项目，这个问题算啥呀，小case

9 基模选什么？有研究开源Deepseek吗？GPU用的啥？

> 基模选择什么，要了解笔记八股部分不同模型对比，从性能，公司立场，消耗，业务场景等方面说明选择原因，自圆其说。
>
> 开源模型学习，倒是我确实没有看，到时候带大家看看，不过这个有点偏算法了，我要准备一个相关的给不会算法的人也能照着回答的版本。
>
> GPU用的啥。因为我的场景使用cpu推理的嘛，所以我没GPU，直接回答了。我是建议这些问题，即使你不做算法，推理，都去了解一下，然后像我一样做一下，能说上这些问题。因为有时候面试官真的问不深，他只能问出来你做没做。所以你是算法高手和你做了一下算法是不一样的，我们掌握后者很容易，但是却可以cover掉很多面试场景。

10 换工作原因

11 拿了几个offer?想做什么方向？

> 一般问到那几个offer就比较稳了。
>
> 问想做什么方向，笔记面试技巧我讲过，意向度很重要。如果真是应用岗位，你得说你要做应用。要顺着业务说。之前同事面试英伟达，人家是应用岗，他是想做算法，还坦诚和人家说，让对面觉得他大概率不会去，offer发给别人了。

12 反问环节

> 看一下我笔记总结的面试技巧，反问的技巧吧。

三面

三面主要是一个博士来考察。整个时间30分钟，卡得很严格，而且面试官还迟到了几分钟，所以导致实际面试时间只有20多分钟。三面的考察内容其实很清晰，就是摸底 + 一个架构设计题。借助这个机会我最想讲一下的是架构设计这个面试问题怎么去做？通常这是个比较难的问题，要结合具体的应用场景设计一个架构。我这里给到一个模板供大家参考，大家可以套用。

1. 自我介绍

2. **大语言模型相关经验（技术栈深度摸底）**

面试官提问（原话）：

"就你有做一些偏大语言模型相关的一些东西吗，就是跟语言模型相关的"

> 我最开始分析这个问题表面上看很奇怪——我刚介绍完agent/RAG/reschedule这些本身就是LLM相关的项目，面试官又问"有没有做LLM相关的东西"，他到底想问什么？但结合他后续的追问（"所以你基本上就是处理数据啊、微调一下模型啊、做一下RAG"）来看，**他真正想问的是：你在LLM技术栈中做到了哪一层深度？** 是只在应用层调API搭Agent，还是也深入到了模型层（训练/微调/架构研究）？他的"偏大语言模型相关"重点在"偏"字——想了解你偏模型侧还是偏应用侧。所以这个问题其实在摸底，你做得多深入？（你的Agent就是API调用还是有一些深入的设计，比如手搓了一个Agent框架）你做得有多广？（有没有设计算法，对模型微调，提高参数，有没有做部署，你的技术栈到底是在哪一层）

> 所以我分析这个的时候，我觉得看似他是一个简单的问题，实际又是一个很难的问题。要想把这个问题答好，你必须做得比较深入，广度又有，或者恰好就是投其所好，比如面试官特别需要做过哪个方面，你恰好有这个技能点。这样面试官才会比较满意。不过大家压力也不用太大了，我们也不是神人，各个方面既广又深，我当时求职的是25年12月，对RAG做得不深，Agent项目比较浅，微调也就是理论走了一遍。不过我这个是一次通过的面经。我是按照100分的思路来给大家解析这个题目，但是不意味着大家必须做到100分才能通过面试。但是我觉得最好的一种方法是，即使你时间不够。你没有深入地做一遍，但是Agent，Rag,微调，强化学习，部署最少面试官问你的时候，你要说你会，你做过，然后准备一些你准备好的答案，如何做的部署，微调（参考上面笔记）。要是面试官不深入问就过去了，深入了问答不上来再承认做得不深。

我的回答：

我这里当时的回答就是澄清了我的技术栈。Agent做了什么，做得多深，RAG做了什么，算法了解多少，对于微调和强化学习也做了一遍（具体可以参考我上面的笔记（[如何做的微调](#doxcnXE4sR8p50zqg2mzVEJDSVd) [如何做的推理加速](#doxcnaaHJ978A9HZpljzbTB5jxN)））

追问：使用过什么模型

"那你基本上就是之前用过什么模型之类的对"

思路

> 最好对国内的出名模型，比如Deepseek，千问。国外的出名的模型，GPT,Claude 有一些使用经验，更好的是有一些微调经验，理解模型之间的差异优缺点。不过这个面试官并没有追问，他就是问了一嘴，我也不过度解读了。

3. **学历背景确认**

"对所以你本科其实是偏计算机的对吧，只是说你选修了一个商务英语的第二学位对吧"

"硕士也是偏CS的对吧"

> 看出来这就是一个摸底的问题，很多时候技术问题只是决定你最终结果的一部分，但是你看这些问题，面试官在摸你的底，你的过往学历，工作经历，甚至是专业。其实已经给你打了一个潜在分了。不过这些也无法改变了。相对来说像国企啊，银行啊，这种就比较保守，对这些看得比较重。互联网相对来说更看重能力和项目经验。但是我们能改变的，也只有面试的问题和项目部分了。

4. **AI项目中最擅长什么**

> 这个还是一个自己发挥的问题。AI行业，重要的技术点我在岗位介绍总结的有。Agent,rag，后训练（Sft+强化学习），部署。 应用也看工程，后端经验。算法看论文，部署也更加重视后端经验。 所以你的优势是什么，你自己挖掘。比如我自己，那我说的点就是， 1. 有上亿用户项目落地经验，大厂工作，工程能力强。2.Agent应用方向理解深入，设计复杂的Agent架构，RAG各个环节都理解，完成多Agent，复杂记忆能力，自研rag系统集成（我的主要方向是深入应用，应用是我的强项，所以我在说我的理解深入） 3.对于现在Vibecoding的技巧的掌握。我觉得这个东西在新的时代也是重要的，就包括ClaudeCode的使用，里面的技巧面试官通常也感兴趣。 4. 算法，部署都有经验。能够完成专业显卡训练SFT,强化学习，自己部署并推理模型（我主要体现广度，算法部署不是我的专精项，但我知识广度够）。所以如果是我自己，我从这四个方向回答。 其实大家如果看这个笔记，大家也能够做到上面四点，因为在笔记里都有，包括项目。不过结合自己情况慢慢来吧。

参考答案

第一，工程落地能力。 我在腾讯和微软都做过上亿用户级别的产品，比如xx全球上线的XX系统（我的工作项目就隐藏了），所以在大规模系统的工程能力上比较扎实，这也是我做AI项目的底子。

第二，Agent和RAG应用方向，这是我最深入的。 我做过多个Agent项目，看过很多论文，深入理解目前主流的Agent开源项目，比如Openclaw, Hermes,ClaudeCode。对Agent的设计范式，架构，任务拆解都有深入理解。同时自己自研了RAG项

目，实现RAG系统整个环节。对大模型开发的前沿技术保持持续学习。

第三， 我对Vibecoding的技巧有深入的理解，包括ClaudeCode的各种高级使用功能，可以使用CC完成整个工作自动化的编排，比如代码-编译-测试的闭环。熟练使用各种模型，理解模型的边界，对于Harness工程，比如如何约束Agent的行为，沙箱机制等技巧都有实践和理解。

第四，算法和部署也都有实践。 我使用专业卡做了SFT微调、强化学习对齐、量化压缩，也自己用推理框架部署过本地模型，在Windows边缘设备上做过推理加速适配。这块不是我最专精的方向，但知识广度够，需要的时候能上手。

5 金融+LLM经验与兴趣

面试官提问（原话）：

> "我看你其实可能之前没有太多做金融相关的一些东西吧，就是比方说把LLM用在金融上面，就是对这个有什么了解，或者对这个做这个事情感兴趣对吧？

这个问题只对金融方向的大模型岗位有用（银行，证券）。如果你面试相关公司，一定要稍微了解一下金融的知识，想清楚金融和AI如何结合，你的AI技术能够在金融方面有什么用。 其实这个问题也适用于其他场合，比如我面试SAP，他们做供应链的，要准备的问题就是你的AI技能如何在供应链中发挥作用，你能带来什么价值？

6. **LLM股票估值系统设计（核心技术题）**

面试官提问（原话）：

比方说我们要用大语言模型来对是A股啊，或者说美股啊，或者说其他市场的所有公司来做一个估值上面的一些建模吧，比方说我有微软的所有，还有相关的一些公开的一些public的一些信息，比方说微软的财报啊，微软的分析师的一些报告，微软的earning call啊微软各种新闻啊，这些信息我都有，对然后我也能用啊，其他的各种工具之类的，就是我如果个用LM来建一个，来对微软或者说其他任意美股的公司，来做一个建模的话，然后这个事情来怎么做，就比方说呃我就是对微软呃，有可能就是选择呃，想用市盈率啊或者其他方式来对这个公司来估值之类的，对就是你觉得，就是这个整个这样一个系统之类的该怎么样来做呢？

思路:

我想借着这个题展开讲一下，因为这个题其实代表着一种题目类型。叫做针对具体场景设计Agent系统。设计一个系统本来就比较难，而且包含的东西太多了，你看我们分析ClaudeCode， Openclaw的源码，整个系统有那么多内容。想要现场设计一个，又结合具体的任务，本身就是个挺难的事。 这类题本身也没有标准答案，我在复盘的时候，觉得我们可以设计一个模板，在真正现场设计的时候就使用这个模板往里套，我们只用想清楚和业务相匹配的地方，然后进行修改。这样在极短的时间，我们可以设计一个复杂的成熟的架构。不过为了熟练使用这模板，也需要我们掌握清楚笔记的前置的一些知识：多Agent架构的八股，以及Openclaw, ClaudeCode笔记讲的开源项目的设计思想。

答题思路：骨架 + 定制（方法论）

面对"设计一个 Agent 系统"的架构题，核心思路就两步：

第一步：定骨架——直接套 OpenClaw 三层架构模板

OpenClaw 的架构是 **触发层 → 网关层(Gateway) → Agent层(Think+Act+Remember)**，这是一个经过验证的事件驱动

> Agent 执行引擎。不管什么业务场景，这三层骨架是通用的，直接套：
>
> ```Plain Text
> 触发层：产生事件的 5 种信号源（Messages / Cron / Heartbeat / Webhook / Hook）
> 网关层：路由分发 + 管控（不做推理，不调 LLM）
> Agent层：Think（递归中心化，spawn 子 Agent）+ Act（工具执行）+ Remember（四层记忆）
> ```

第二步：个性化定制——在 5 个关键节点注入业务理解

骨架不变，但以下 5 个地方必须结合具体场景做定制，这是答案拉开差距的地方：

1. **触发层**：除了用户消息，还有什么事件源？——结合场景想：有没有定时任务？有没有外部推送？
2. **网关层**：Gateway 要处理什么业务特有的管控？——数据预处理管道、限速、Session 路由规则
3. **选型原因**：为什么选中心化而非去中心化/单Agent？——结合任务特征论证（可并行？要收敛？数据异构？）
4. **Agent层-工具**：子 Agent 需要什么领域工具？——内置什么专用工具 + 工具白名单怎么分
5. **Agent层-记忆**：Bootstrap 写什么，压缩时保留什么？——SOUL.md 人格、MEMORY.md 领域知识、Compaction 策略

OpenClaw 模板是骨架（通用），5 个定制点是血肉（体现你懂业务）。面试的时候先讲骨架让面试官知道你有框架思维，再讲定制让面试官知道你懂这个领域。面试官问这个问题的时候，大家可以向面试官要个两三分钟，让他允许你连续思考三分钟。关键骨架直接套，关键是想清楚第二步的几个点，想一想如何串起来，如何组织答案，串成下面的答案再和面试官讲。

完整答案

我会把整个系统设计成三层：**触发层、网关层、Agent 层。**

先说触发层。系统应该是事件驱动、持续运转的，不能只靠手动输入。我会设计 5 种触发源：分析师手动输入、定时任务（季报后自动重估值）、心跳检测（盘中股价异动超阈值触发深度分析）、Webhook（对接 SEC EDGAR 财报推送和新闻 API）、内部钩子（子 Agent 完成触发主 Agent 整合）。

然后是网关层。网关层不做推理、不调 LLM，只负责路由和管控：按公司 ticker 做 session 路由互不干扰、金融数据 API 并发控制与密钥轮换、数据预处理编排（财报 PDF 解析、Earning Call 转录、新闻抓取去重）。处理完才交给 Agent 层。

最后是核心的 Agent 层，分为 Think、Act、Remember 三个部分。Think 部分用递归中心化架构，一个 Orchestrator 作为主 Agent，通过 Tool Call spawn 四个子 Agent 并行执行：财报分析、新闻情感、分析师报告、Earning Call。子 Agent 完成后只 push 结构化 JSON 结果回 Orchestrator，不传中间推理链，控制上下文膨胀。Orchestrator 收齐后做最终估值整合——相对估值、DCF 估值加定性调整，输出估值区间、置信度、核心假设和风险因素。

Act 部分，每个子 Agent 配独立的工具白名单，按职责隔离，防止越权和减少幻觉。

Remember 部分设计四层记忆：Bootstrap Files 注入估值方法论且免疫压缩、Session Transcript 记录任务过程、Context Window 定制化压缩策略（保留财务精确数字）、Retrieval Index 按公司建独立知识库支持

按需检索。

最后加一个评估验证的闭环：用历史数据回测，比如拿 2023 年的数据预测 2024 年的估值区间，和实际股价对比算偏差率。分析师 review 反馈，偏差分析写入记忆，系统越用越准。

面试官可能追问的细节

> 我们在回答上面的问题的时候，不可能展示所有的设计细节，比如Agent间是如何通信？主Agent是如何产生子Agent的？为什么选择中心化的架构？上下文压缩策略等？我们不可能在第一个回答中，回答得很多，这样面试官会失去重点，上一个问题我们只讲大概。如果他真的感兴趣，问细节，我们再回答。不过问细节的时候其实大家不用怕，因为这个时候其实是我们掌握了主动权，具体的细节其实都是我们学过的知识，主要是八股多Agent架构部分和笔记看的开源项目部分的知识。如果我们掌握了笔记这部分，下面问题就很好回答。面试官在上面问的时间越多，整个面试环节在我们手上自己掌握的主动权就越多，这是好事。前提是我们对于笔记的理论知识部分要熟悉。

Q1：子 Agent 之间怎么通信的?

OpenClaw 用 **Tool Call 模式**（不是 Handoff）。Orchestrator spawn 子 Agent 后不阻塞，子 Agent 完成后通过 push-based announce 把结果以"用户消息"注入 Orchestrator 的消息队列。**仅传最终结构化 JSON，不传推理链**——CoT 留在子 Agent 私有 session，控制上下文膨胀。

此部分理论内容参考八股的多Agent架构。

Q2：为什么不让子 Agent 互相通信?

估值的每个子任务是独立的（财报、新闻、分析师报告、Earning Call 之间无交叉依赖），互相通信只增加开销没有收益。如果是网页导航这种高熵场景才需要去中心化让 Agent 互相纠偏。

此部分理论内容参考八股 Agent通信机制

Q3：为什么选中心化而不是去中心化或单 Agent?

因为估值任务有确定的入口和出口，子任务可并行但最终需要统一收敛，这天然适合中心化。不用单 Agent 是因为四种以上异构数据源处理方法完全不同，挤在一个上下文里模型容易混淆。不用去中心化是因为子任务输出的是确定性数据，不需要辩论，去中心化通信开销没有收益。Google 的论文数据也支持这个判断——中心化错误放大 4.4×，去中心化是 17.2×，金融场景对准确性要求高。

此部分理论内容参考八股的多Agent架构。

Q4：工具白名单具体怎么分的?

财报 Agent 只能用 pdf_parser、calculator、rag_search；新闻 Agent 只能用 web_search、sentiment_analyzer；分析师报告 Agent 只能用 pdf_parser 和 rag_search；Earning Call Agent 只能用 transcript_parser 和 rag_search。每个 Agent 只给必要的工具，减少不相关工具调用导致的幻觉和越权。这其实体现了沙箱隔离的思想，每个Agent有自己的上下文和工具，互不干扰。

Q5：模型怎么选型？子 Agent 都用同一个模型吗?

不用。Orchestrator 用强模型比如 GPT-4o 做决策和整合，子 Agent 用轻量模型做信息提取，降低成本。安全上，子 Agent 不能再 spawn（maxSpawnDepth=1），单个 session 最多 5 个活跃子 Agent（maxChildrenPerAgent=5）做并发保护。

Q6：上下文不够怎么压缩?

套用 OpenClaw Compaction：上下文达阈值（contextWindow − 20K reserve − 4K soft）自动触发 → 最近 \~20K tokens 原封保留 → 更早的分块逐块摘要 → 合并。金融定制的关键是压缩指令要求财务精确数字必须原样保留。另有 Pre-Compaction Memory Flush 机制——压缩前静默跑一轮让 Agent 把关键结论写入 memory 文件。

Q4, Q5,Q6 的理论部分其实都参考我们开源项目的解读，里面蕴含的一些思想，比如强弱模型换着用，沙箱隔离，压缩策略等。可以看我们笔记中的ClaudeCode解析，Openclaw解析，Hermes解析。所以这个问题虽然总结了模板，不过要想能够经得住面试官继续的追问，还是需要我们熟练掌握笔记中的其它章节的。

四面

2025.12 联想

岗位介绍

具体岗位信息我是记不太清了，不太确定是哪个猎头或者HR就帮我投递了，没有发给我jd。总体来讲，这是一个大模型 + windows的岗位.

反思：面试官明确说不考代码题，说那玩意没必要。基本上是对着我的项目，而且是从上一个公司的每个经历和项目到这一段，都给你扒一遍。基本上他是现场看着简历给你提问，但是对于你的项目，他很快就能理解，还会从技术，产品的维度挑战你。整体考察内容跨度也非常大，从音视频到c++，到设计模式到硬件，到RAG,AGENT，windows。 但是提问他基本不问八股，你回答八股还会被他打断，整体我觉得是面这么多厂最难的一次。不过面试官本身人还是挺好的，还会安慰你，说没关系，我们就摸个底，并没有压力我。 问题我就不总结了，感觉有点超纲，而且后面会有更新的面经，我先把这些之前的问题总结出来，有需要的人可以参考。

一面

岗位介绍：具体岗位信息我是记不太清了，不太确定是哪个猎头或者HR就帮我投递了，没有发给我jd。总体来讲，这是一个大模型 + windows的岗位.

反思：整体上来说，面试官明确说不考代码题，说那玩意没必要。基本上是对着我的项目，而且是从上一个公司的每个经历和项目到这一段，都给你扒一遍。基本上他是现场看着简历给你提问，但是对于你的项目，他很快就能理解，还会从技术，产品的维度挑战你。整体考察内容跨度也非常大，从音视频到c++，到设计模式到硬件，到RAG,AGENT，windows。 但是提问他基本不问八股，你回答八股还会被他打断，整体我觉得是面这么多厂最难的一次。不过面试官本身人还是挺好的，还会安慰你，说没关系，我们就摸个底，并没有压力我。 问题我就不总结了，感觉有点超纲，而且后面会有更新的面经，我先把这些之前的问题总结出来，有需要的人可以参考。

1.请先简单做一个自我介绍，请尽量用英文，讲您擅长的，在公司里做的内容，大致介绍一下，几分钟就行。

（一上来就给我一个下马威，我是没想到联想面试还要考察英语，HR完全没有说。后面才了解到联想是外企）

2.在腾讯主要负责的工作内容是啥?

（我其实不喜欢面试官问我之前的工作经历，一个是我都忘记了没准备，另一个是之前的经历其实和大模型没有啥关系，是做音视频的。但其实很多面试官多少还是喜欢问你上一段工作的情况，即使方向和这个岗位没啥关系）

3.你讲一下你从24年到现在做的这个项目，只讲你负责的东西。

3.1 你这个产品最终量化了吗？后面接的模型是啥？你们有评估过费用吗？有部门给你承担吗?

> （这几个问题，一下子就能看出你的产品落地了没有，做到了什么程度。即使有时候面试我会稍微包装，美化一下，不过对于那种特别实质的内容，我是不敢说谎的，这也确实是转行的人的一些弊端。很难去做一些真实的数据，落地经验。不过也不用特别悲观，一般这一点对于年限越高的同学要求越深。对年轻的应届生和刚工作几年的同学包容度都不错。即使我工作快5年了，尽管这块是劣势，但也不是各个企业都会因为这个把我刷掉，有时有的企业比较宽容，有时有的面试官其实也不会问这么细，稍微糊弄一下就过去了）
>
> 3.2 你简历里写的MCP Server是作为客户端去调用还是服务端调用，里面用到了向量数据库吗？
>
> 3.3 我现在让你支持微信，你怎么去做，怎么分析这个问题，怎么检索，会有哪些风险？用过process manager吗？
>
> （背景是项目使用了一套负责登录鉴权的MCP Server，他的问题是如果让这一套系统支持微信，要怎么设计。后面讲的process manager我都没听过）

4.你提到了使用ONNX 上做了优化，你们优化了些啥东西。

（背景是我的简历上不是写了一套方案，把微调，推理加速这些整个流程都走了一遍吗？通常来说，我都是照着我准备的答案讲了一遍。我确实对于复杂的模型推理，微调公式不是很理解，没深入学。不过我那套答案答完大部分面试官都不会追问，但这个面试官问得很深，我没有系统学习推理，他后面的问题肯定不太回答得上来）

> 4.1 追问：你看有的机器有基显对吧，集显和cpu共用一组Memory，然后你的本地模型又比较小，我理解你不用做量化。你们的推理组的同事有给你一些支持吗？比如用一些加速库？
>
> （硬件知识都出来了，方案也被challenge了， 问着问着其实就暴露了，项目没有特别成熟的问题）

5.哎，TraceID聊聊，全链路追踪差点就给漏过去。你能讲一下整个全链路追踪的可观测性体现在哪里吗？这个traceid里面怎么能够不影响性能又能贯穿于整体链路的?

（这是面试官原话，你可以看出来他一直都是现场看我的简历，然后一个点一个点提问的，这种人最可怕了，你的简历项目一眼他就看懂，然后还能追问和你拷打，很少有面试官能做到这么厉害）

6.你这个地理感知时空冲突检测是啥意思，通过推理分析时间，空间的冲突，生成智能建议，啥意思?

> 6.1 （当我解释后）面试官追问：那你这个意义是啥，我一会来不及了，我线上播进去不行吗？
>
> 6.2 你用过teams哪个功能吗？你这个界面和它的差异大吗？

(具体这个是做啥的大家不用关注，和我的项目相关的，对大家参考意义不大。从这一点可以看出面试官在challenge我的产品的意义了，而且思维特别灵活，开始从产品层面上要和我讨论了)

7.RAG大数据量分表分库

7.1 你这个RAG系统分表分库有做过吗？假设说我现在语料特别多特别多，有700G，怎么拆分这种设计？

> （大哥，你不能问点简单的吗，问点稀疏/稠密双路检索不行吗？上来整这么难）

8.你觉得你用得最好的几个语言是啥，你愿意长期发展的语言是哪个？

9.你可以讲一下xx项目吗？

（公司的项目，涉及保密就不说了，而且也和大模型无关）

10.RAG常见的优化手段和分块策略

（怎么突然又给我整个最基本的八股啊，是看我前面答得太差了吗？= =）

11.你的各个Agent之间杂通信的，如果出现了死锁或者循环调用，你咋解决，你有明确的终止条件吗？如果要做一个本地远程的LLM的无缝切换，你是会如何设计？

（感觉这种题目，Agent+思索+循环调用，就很难准备到，第一次被问到）

12.c++你写了几年，你这个项目c++主要是在做啥，你知道腾讯会议里有一些协议比如webrtc或者其它的你有关注吗？

（要把你的背景扒的底朝天的节奏呀，考察跨度太大了，又跳到webrtc这种通信协议上）

13.C++基础 -智能指针

（我面试大模型的，又来问我C++，好吧，最少我之前有点基础，我就讲最基本的，智能指针有几种，怎么用，然后就被面试官打断。

> 13.1（被打断并追问） 别讲这些概念，整点有用的，结合你的实际工作结合智能指针使用的例子。

（我就讲之前设计C++类的时候不同场景下，啥时候用的shared_ptr,啥时候用的weak_ptr）

> 13.2 （面试官眉头一皱）追问：这不是还是最基本的东西吗？有没有更高级一点的经验？
>
> （我真是谢谢你了）

14.C++堆栈你给我讲讲吧

> （虽然之前学过，但是大模型行业一般哪里去考c++八股，早有点忘了，回答的不流利）

15.你讲一下线程安全的单例要注意哪些东西吧

> （设计模式，单例，我之前看设计模式的视频的时候，确实是讲了这点，但耐不住我真的忘了啊，做不到不准备就信手拈来，问这么多我心态都快崩了）

16.你的c++跨线程通信和跨进程通信都做过哪些?

> （救命呀，我这不是在转行大模型吗，你问C++问这么深）

17.你讲一下你用过哪些内核对象吧

> （真是绷不住了我）

18.你把任务管理器打开，左侧边有个服务，你熟悉哪个？你给我讲一下这个组的作用就行?

> （没救了，没有熟悉的）

19.讲一下IGPU,CPU,NPU在这些推理逻辑中的差异是啥？哪些任务适合放在CPU,哪些适合NPU，哪些适合IGPU?

> 19.1 中间的混合调度应该怎么设计

（这是做推理工程师需要掌握的吧，这真的不超纲吗?

2025.12 阿里国际站

岗位介绍

阿里巴巴国际站，做跨境贸易的一个智能团队。公对公的国际贸易。

做一些支付，风控，收单，账目，结算之类的工作。

岗位是做风控，电商全流程风控，从商家发品到沟通到支付，整个过程都有风控。

这个岗位是算法工程师，大概算法70%，工程30%。

之前这个岗位更多是做Prompt 和 RAG，现在需要一些训练，来继续提高效果。

反思

这就是一个不用视频面试的一个电话筛选，基本上是对着简历，澄清细节。很多问题其实是针对于项目的。

其中有一些问题，比如你是如何做LoRA的，你看过的最新的一些论文等都是八股里精心准备好的，这些问题应该游刃有余，我也一直在继续迭代答案质量。

另一个启发是项目某个地方很简单的话，你千脆别写在简历上，写上去面试官一问，你一澄清他发现是这么简单的东西，反而掉分。具体可以参考下面第10个问题。

这是一个算法工程岗位，和那个京东算法工程一样。这次面试给我的启发是，有些岗位算法和工程分得不是那么紧密，所以这一套路线虽然是应用出发，但我一直也没有忽略算法，按照这套路线，我们不但可以去投递纯工程岗，也完全可以尝试投递一些算法工程，对算法和工程都有一些要求的岗位。

一面

1. 自我介绍

2. 当时用的是哪个版本的Langchain?

（这个问题真的还翻车了，有两次面试官问我Langchain版本，我之前没关注到，我这里总结在八股部分，Langchain的版本号，以及Langchain最新的版本的特性，这里借助这个面试，把八股部分Langchain版本问题，新版本特性总结了，这也是围绕Langchain版本最常考的两个问题）

这题直接参考笔记： **1.2.1 LangChain的版本 和 1.2.2 LangChain 1.0新版本**

3. 你写的熟悉大模型微调，训练。这些是你项目实际做的还是了解的?

（这块根据我的思路，大家应该不会说是只了解的对吧。我一直的思路是虽然咱们面向应用岗，但是微调，训练，量化，我都建议大家走一遍，不求精通，但求会，没有微调的建议都在项目里走一遍，这里忌讳说我只是知道，了解没做过。做一个不难的，我是怎么做，怎么回答的，都在上面的笔记。可以参考思路，自己准备一套。或者直接照抄我的，你能自圆其说也行）

4. 你能介绍一下简历写的这个全链路追踪和可观测性吗?

（其实就是监控系统，AI帮我写的，其实就是讲一下我的项目里的评估方法+监控方法。这个也建议大家掌握，就是Agent的性能评测和监控。我这里说的一般是使用美团龙猫的测评方法 + 上传数据到远端实现监测的这一套逻辑。 所以具体我讲就是用美团龙猫论文这一套滑动窗口评测的方法，从xx维度评估，创建数据集 + 上传云端+监控。这是我的一套方法论哈，具体的知识在笔记里都有，大家看美团龙猫相关的笔记。 当然你这里可以使用你自己的方法，如果你没有更好的或者不会说，你找我这一套讲也行，前提是你要去看我笔记美团龙猫部分和那个论文以及对应的思路，这个总结在上面笔记了）

参考 3.Agent性能评估

5.你这个多Agent架构是一个什么形式的多Agent?

（这题还是结合多Agent的架构去讲，八股部分总结得很全面了，结合这个只是告诉面试官你的项目用的啥多Agent架构而已）

> 5.1 你讲到的这个主管Agent去调用另一个Agent，是怎么调用的，通过MCP方法吗？

（这里不知道大家理解这个题意吗？在我的笔记总结部分**6.4 Agent之间通信和状态管理。** 讲到了Agent之间有2种通信方式：**移交**和**工具调用。**这里他这么问，其实他已经认为了我的系统Agent的交互是使用的工具

调用的思想，其实我Agent通信是使用的移交方式。这里对于我的项目，直接和他澄清这一事实和交互的方式就好。给大家的启发是，要去复习 6.4的笔记）

6. 你这里项目里写的MCP，你有了解过吗？

7. 你能介绍微软这个Semantic kernel框架吗？

（强项目相关问题，这个还是说是我项目中使用的这个框架，所以面试官问，但是其实这个框架用的不多，在微软外部，所以其实你不用太懂这个框架）

8. 讲一下你xx项目的架构。

9. 你能解释一下你这个项目中使用LoRa具体是怎么做的吗？

（这个也是准备好的答案，有一套专门的总结好的答案。 你是怎么做的微调，应该是总结得比较完整的回答，直接照着回答就行了）

10.你简历里面讲的本地远程模型切换怎么做的？

> （我这里是一个本地设置在`env`里面的一个根据不同模型来做不同处理的一个逻辑，不是一个热切换，写在简历里是`AI`帮我润色了一下，看上去挺高级，但面试官一问其实说白了就是个`if else`逻辑，说白了这里还不如不写或者修改下，这是给大家的启发，某个地方如果特别简单，你干脆别写简历上了，写上去别人一问，一讲是这么简单，反而减分）

11. 你这个预约系统，你这里有做模型的评估吗？

（模型评估这块，我的思路还是说从美团龙猫的方法，结合 《Agentic design pattern》的原则（从轨迹，工具调用）上来整理的一套逻辑来给面试官讲，可以看我的讲解视频，视频里会说大概我怎么讲的）

12.你有没有关心一些最新的论文？

（这也是早已准备在八股之中的，我一直说要准备几个最新技术。 我的笔记里整理了：**美团龙猫对于Agent评估**的论文， **Google的多Agent架构（**八股Multi-agent）部分， **Deepseek-R1训练的部分**，这几块都可以拿来讲，具体看上面八股）

13. 你认为的完整的智能体，需要有哪些核心的能力？

（不难的一个问题，但是好像你不准备很难回答得全面且有条理，根据这个答案可以尝试自己多表述几次。每一个黑体字段就是一个Agent的要具备的一个核心能力，然后把这能力用逻辑性串起来就是一个不错的回答。）

> 简单来讲，我认为一个完整的智能体 = **能理解目标**，**会规划**，**有记忆**，**具有调用工具做出行动的能力**，**能自检和学习**的可以在**安全合规**和**可观测约束下交付稳定结果**的系统。

视频讲解

https://www.bilibili.com/video/BV1fdP7zkEdY/?vd_source=144bec9c3f54e465073138bed788be1b

2025.12 盛大AI

岗位介绍

JD 我没有找到了，不过我从HR给我介绍的内容以及面试内容去分析，来带大家分析一下了解一下这个岗位。

盛大公司其实听着是一个大公司，给我们的印象可能是游戏公司，很早之前也许你玩过它的很火的游戏：泡泡堂，传奇世界等，之前还是挺有名的。现在盛大公司也在投入AI赛道，下面有两条AI产品线，大家可以去查一查：

1. Tanka: 偏应用，做应用，做产品
2. MiroMind ： 偏研究，也就是我们这次面试的岗位。面试官明确说，他们是不做应用的，主要做的模型的前后训练，来提升模型的能力。虽然不做应用，训练模型后的效果要结合工程去评测，所以其实需要了解Agent的框架之类的东西。

总结这个岗位： 整体偏研究，训练，推理，模型都做，但是也需要应用。虽然盛大听着也不是小公司，不过研究AI的产品线的人数确实没多少，这个部门可能就几十人，所以也确实和很多规模不大的公司类似，很多事情都要做，不是局限在一个方向。 很多大公司对于岗位的分类是非常界限分明的，比如千问C端，他们做工程就纯工程，不用太管算法，但是这其实也有一个显著的弊端就是做工程其实又苦又累，公司不高，还是慢慢转到算法更有前途。所以相比下，这个部门虽然不大，但好处真的是模型训练，部署，工程都能做，对自己的提升肯定更大。另外整体又背靠是一个相对来说的大公司，资金链还是比较充足。

反思

第一个感受： 这个行业很多岗位还是需要我们交叉的能力。虽然纯工程的岗位有，但精工程，善训练，懂部署的人一定更吃香且前景。

第二个感受：这其实也再次验证了我们的路线，从应用精通，带一点算法的学习。首先是最容易上手的，入行最快的，另外这么复习，还是有机会去面试算法岗，甚至是拿到offer的。从我们面试这么多家来看，很多岗位其实是算法岗，其实我都没有专门的训练，部署项目，但是他们还是有很多算法的机会。 所以我的思路更加觉得大家是按这个路线学，先精应用，带着算法去学。 学得差不多了，再深入算法，如果时间不够，先学好应用先入行，慢慢再向训练，算法这条路深入。

面试内容分析

1. 其实可以从面试官的问题看出来，有些问题（离职原因，职业规划）这些其实是反映忠诚度和匹配度的，除了面试的知识的硬性技巧之外，有些内容比如问你想去什么样的公司，职业规划这些其实是 想考察忠诚度的。很多面试官其实在意忠诚度，如果他问你你的职业规划，但是你的回答和这个岗位不匹配，这些都是减分的。所以大家可以去思考一下怎么样展示自己的忠诚度，比如我在面试技巧，自我介绍中就提到了展示忠诚度的技巧，结合这次面试问题的回答，大家也可以自己感受一下。

2.环境准备问题。 面试的时候，有的时候让你在本地编译器中写代码（但我用的个人机，没有本地编译器），这次面试让你打开github链接（但我用的个人机没有梯子），有时网络信号断开了，信号差，麦克风不好等。这些其实都特别影响面试体验的。面试官有时候虽然嘴上说理解，但是根据我的经验，但最后都被减分，通过率降低。所以其实大家也要尽量避免一些这种问题的发生，把环境真的准备好。

一面

1.自我介绍

2.职业选择动机

3.当时为什么考虑从腾讯去微软呢，然后你现在的base是在北京，是在深圳吗?

4.项目介绍

4.1 在这个Agent项目当中，你觉得什么是比较难的?

（其实这个问题是非常常考的，而且是完全给你机会让你去总结的。我在做之前的面经分析的过程中，就提到过。 有的时候，其实做完一个项目，其实我觉得好像没什么难的，甚至有面试官问的时候，我好像说过类似的话 "其实整个项目也没啥特困难的"，闭着眼睛想，这就不是一个好的回答。 我不知道大家有没有类似的困惑，但是这个问题是需要挖掘和思考的。你需要挖掘，思考，包装你的项目到底有哪些地方比较困难，你要提前想好，挖掘好，这个需要提前准备，我之前复盘我就说这个问题很重要，所以我总结了对应的知识在笔记

> 其他问题- 4你在项目中遇到的最难的问题
>
> 之前总结是工具调用失败问题的排查和解决。借助今天的机会，我再在第4部分再增加一个例子，就以我们笔记中的这个RAG项目再总结一个例子。这个问题的回答请直接跳转到上面笔记部分。另外在本期视频的讲解中，我其实讲了一下我是如何挖掘项目深度，总结出的这个答案的，感兴趣的小伙伴可以看一下本期视频讲解）

4.2 你的项目中Agents对于工具的处理是怎样的?

（这里其实是在聊如何把工具塞入到LLM中。我提到的是根据业务规则，每一步过滤指定的和业务阶段相关的工具，然后输入到大模型中。 然后面试官接着总结，以下是他的原话："相当于也是没有完全是一个agent的自己决定的workflow，就是你也是有一些固定的流"。

最开始我复盘的时候其实在想这是不是我做得太简单了，但是我做了一些调研，其实现在行业普遍没有说让AI达到一个完全自动筛选工具，自动定义生成工具的路径，特别是在生产项目中。这是我们的目标，是未来AI发展的趋势，但是在25年这个Agent元年生产项目并没有达到如此智能。所以如果有的面试官就想因为这个说你做得简单，其实你不妨和他深入聊一下下面我总结的行业现状，让他能够清醒一点。）

目前常见的大模型获取工具的方式有3种：

> 首先是**全量塞入**，很多聊天工具如Copilot都是这么做的。原因也很简单，因为这些产品其实用的是远端的模型，现在远端模型的上下文容量越来越大，所以他一次性塞入也并没有问题。
>
> 然后是**根据业务场景和规则，筛选对应项目加入的**。 原因也非常合理，因为我们用的本地模型，模型上下文本身有限。OpenAI官方文档都建议：单次API调用工具不超过20个，理想不超过10个。

最后是 让**AI动态决定工具。**这个现在更多的其实是处于Demo，学术阶段才这么做，商业落地项目很少这么做。在25年12月份UC Berkeley 发布的论文

《Measuring Agents in Production》中明确调研了现在的商业化Agent，也证明了一点, 根据论文： "商业的生产级的Agent，一般都将执行步骤限制在10步以内，允许数十步的仅有 16.7%，无限制的仅 6.7%。并且还会专门通过给工具添加封装层，减少工具数量来简化模型的负担。

感兴趣的同学可以参考论文原文：https://arxiv.org/abs/2512.04123

> （我说这么多是想要大家：
>
> 1：了解行业现状和相关知识
>
> 2：有的面试官可能也真的不太懂，拍脑袋就觉得你的项目某处简单的时　候，你可以拿出你的依据，告诉他行业现状和你使用这个策略的考虑来说服他，而不是被他带着跑）

5 然后我看你还有写到一些推理加速的部分，那地方做的多吗？

（结合我们分析的这个岗位性质，面试官其实问推理加速这一块应该很符合预期，因为本身他们就是做训练的。我比较担心的是他问得很深，因为按照我们的学习路线，我们是把算法这里带着走一遍。我按照我的笔记里的路线。 速度慢->量化->选择合适的推理框架->微调->强化学习讲了一遍。面试官也并没有追问，具体答案还是请看我笔记 你的项目是如何做推理加速的？）

6. 现在我给你一个我们在做的开源项目，请你现场阅读，然后告诉我大概这个项目在讲啥？然后如果我要添加一个工具，怎么做？
   （我还是第一次遇到这样考察的，这个问题完全是相当于把问题引入到面试官最熟悉，而你完全陌生的场景，需要你现场发挥，现场快速学习。大家可以自己打开这个链接，看一下这个项目，自己能不能用5\~10分钟弄清楚这个问题的答案。另外咱们也不白学，针对这个问题，我总结出以下这个开源项目是在做啥的，感兴趣你可以深入学这个项目，到时候有其他面试官问你你最近看了什么开源项目，也可以把这个拿出来讲

我在真正复盘这个面试题的时候，觉得也不用把他想的特别难，这个项目是一个deep research agent, 如果你了解deep research 的概念，结合它的readme,可以快速了解这是一个什么项目。 第二个问题如果添加一个工具，怎么做？结合现有的一些工程经验，比如我们RAG项目怎么增加一个新的模块，或者就是和这个类似的Git项目，如果有相关经验了，那么其实你也能很快回答。 借助这个机会，我们学一下开源项目，学一下deep research（其实deep research面试还是挺多人问的，可能很多公司他们研究这个，所以需要了解一下）， 学一下工程知识，我觉得这是一个很好的问题。

另外，我惊喜地发现这个问题其实不是最有价值的，最有价值的是这个项目。包含了Agent框架的知识，包含了训练，公开了数据集，训练方法。整套项目也有7.6k 个stars。完全开源，如果我们学下来，相当于是一个从训练 到Agent 都覆盖了的一个最好的实践的例子。我有初步想法准备把它当成我们的素材，26年3月开始，用它作为基础来学习，往算法方面扩展。如果这个计划执行，我也会把相关经历都写在笔记中，大家一起学习。）

项目地址：https://github.com/MiroMindAI/MiroThinker/tree/main

第一个问题，项目干啥，更像是读了Readme做总结，如果你熟悉相关领域，可能回答不难，但你不熟悉，可能这题就难了：

MiroThinker 是一个**开源的 Deep Research Agent**，目标是让模型能像研究员一样，自主地搜索信息、阅读网页、执行代码，最终完成复杂的研究任务。

项目包含**四个部分**：

- **MiroThinker 模型**——基于 Qwen3 做后训练的 Agent 专用大模型，经过 SFT、DPO、RL 三阶段训练，让模型学会了什么时候该搜索、怎么读网页、怎么写代码分析
- **MiroFlow 框架**——自研的轻量级 Agent 框架，基于 ReAct 范式，通过 MCP 协议接入工具，核心特色是 keep-5 上下文管理策略
- **MiroVerse 数据集**——147K 条 Agent 交互轨迹的训练数据
- **技术论文**——详细描述了训练方法

它的核心创新点是提出了 **Interactive Scaling**——认为除了模型规模和上下文长度之外，**交互深度**是提升 Agent 性能的第三个维度。通过 RL 训练，模型学会了在 256K 上下文窗口内进行最多 400 次工具调用，在 GAIA 等多个 benchmark 上达到了开源最优。

第二个问题：如果我要添加一个工具，怎么做?

> 这个问题面试现场肯定看不了代码那么细，需要结合目录大概分析。我们在目录中发现目录有`microflow_tools`目录，然后下面有各种`mcp_server`，所以估计可以推测出来大概是通过`mcp`的方式引入`tools`，深入研究具体回答为：

从目录结构可以看出，这个项目的工具是通过 MCP 协议以插件式的方式接入的，加一个新工具基本上分三步：

第一步，在 libs/miroflow-tools/src/miroflow_tools/mcp_servers/ 目录下新建一个 Python 文件，用 FastMCP 框架定义工具。每个文件就是一个独立的 MCP Server，用 @mcp.tool() 装饰器注册工具函数，最后通过 stdio 方式启动。

第二步，在 apps/miroflow-agent/conf/agent/ 目录下的 YAML 配置文件中，把新的 Server 注册进去。框架里有一个 ToolManager（manager.py）负责统一管理所有 Server 的连接和调用，配置里加上就能被发现。

第三步，如果新工具需要 API Key，在 .env 文件中配置对应的环境变量。

整体设计思路是每类工具一个独立的 MCP Server 进程，好处是依赖隔离、进程隔离、可以按需启动。比如项目里视觉理解就有两个版本——一个调 Anthropic API，一个用开源模型，文件名带 _os 后缀，在配置里切一下就行，不需要改任何框架代码。

7. 算法题 - 岛屿数量（LC:200)

8.你想做哪一块啊，我不知道你对这个感不感兴趣，或者说你觉得你比较想在agent这个方面想自己想做的内容

说实话这问题就是要完全顺着面试官说，一定要说自己想要去做这个方向，还有原因说的具体点，不要看着像编的。我一直强调面试中表现出的意向度，是很重要的!

9. 反问

笔记中，面试技巧也有专门的反问技巧，大家去看笔记： 面试技巧 - 反问

视频讲解

https://www.bilibili.com/video/BV1pSR5BmEWp/?vd_source=144bec9c3f54e465073138bed788be1b

2026.06 深圳边端智能研究院大模型应用岗位

总体点评

这一次面试是我社招（工作刚满5年的水平）面试。

岗位是新成立的一个研究院，据猎头说是国资委成立的，深圳政府支持 又和 荣耀合作。笔试链接是荣耀的，很多面试官也是从荣耀那里“借来”帮助考察的。所以它的工作作风到底是一个事业单位的性质，还是一个荣耀大厂的性质，这个不确定，不过据面试官说很多东西，比如考核，工作作风可能会和荣耀类似。有时候他们说本身是研究院，强度不会很大，和腾讯比，可能是中等水平强度。但是又说和荣耀风格相似，我想着荣耀本身不是和华为有关系吗？沾上华为的文化，感觉也不会轻松。综合一些网友给我的留言，我觉得估计这个强度应该不至于像变态互联网，周末加班常态，一直弄到11点甚至凌晨，但是也不会很轻松，965不存在，估计怎么也得干到个8，9点。（当然以上纯根据我面试沟通，网友留言，自己经验主观判断）

整个面试轮次为：2技术（一个荣耀的Agent开发专家 + 一个荣耀的算法专家（这也是所谓的交叉面，让不同岗位，专业的人来考核你，这是面试常见手段了） + HR（可能是荣耀的HR） + 综合面（可能是研究所的HR+研究所主管）

同时配置笔试：传统代码题（给的牛客形式输入输出） + 性格测评（类似于华为那样的），后面我都会给大家参考。

面试并不难，如果我所说，我觉得比前面解析的很多校招的题目都简单，大家自己去看也能发现。我暂时先没总结答案，我先把流程写好，对应的注意事项，我如何准备的，每个环节做了啥都写好。具体的答案后面有时间（或者必要）我再来总结，不总结我相信大家也能够处理，因为问的不难，笔记基本都涵盖了。

一面的话，基本围绕项目和具体基础知识 ，问的又不深入，基础知识也不难。

二面的话，算法专家来面的，但是他没有问你算法，都在问你工程，他不会为难你。问项目问得比较细，但是他不是类似于我们上面两期对于技术细节的深挖，而是问你具体怎么实现的（其实也合理，算法的专家我估计他自己这些Agent记忆，压缩他也问不深入），我觉得答得不好是因为我项目细节有点忘了，有点穿帮，不是面试本身难。同时，在算法专家面前，其实大家就要悠着一点，你在一个做应用的面试官的考察，我可以说我会算法，我做了后训练那一套，照着笔记说。但是如果对方是算法背景，你真得谦虚一点说。当时他看我简历写了做了后训练那一套，他说这些是你做的吗？我说是，他说强化学习也是你做的吗？我说是（我心里犹豫了一下，但我想这我最近学完了算法那一套，也在做后训练，我要自信大胆一点），然后他就接着问你用的什么算法，怎么做的数据集。 我当时心中一惊，因为我笔记里总结那一套是如何做微调的，强化学习没太准备，我现场有一点结巴，我就说用了dpo，数据集就是输入输出(reject accept对） 我觉得答得有点语无伦次，他再追问我真的得崩了。好在其实他也没为难我。没继续问了。我的经验就是包装这个东西有风险，你要见人下菜，对于应用背景，说笔记那一套，就够了。如果有一些算法背景，他可能会问细节，你就不太好说你做得很全面，除非是你真的做了一遍，你的底气会足一些（这也是为啥我现在在真的做算法项目，当我真的做完了以

后，再包装和说的时候，底气会足很多），这个大家灵活机动把握。综合上述原因，所以我觉得我回答得不好，小红书也都有写当时面试完的感受，但是无论如何，最终面试过了。

HR面很简单，HR也很和蔼，我觉得挺好的，一点不为难人，全程很多地方都是让我在问。

综合面：感觉就是把前面技术面和HR面都走了一遍，也是很多时候都在让我问问题，技术问的也没之前深入，HR问的更像是走流程。

笔试和综合测评以及注意事项后面会细讲。

我个人感受，我并没有觉得有什么坑。面试官，HR给我的感受，我觉得他们人都还不错。本身也是研究智能硬件，打造智能终端，研究方向N+2(N表示当前技术，+2表示领先当前技术两代），主要研究研究前沿的AI方向，涨幅能给10\~30%, 强度最少不是变态互联网强度。 所以其实我觉得是一个可以考虑的工作。我没有选择它，理由就是相比我现在的工作，还是不足以让我跳槽。同时就算是去研究所，我觉得是不是有更好的研究所，也许可以试试，比如智源。我也没这个精力打听，也没有时间大量面试对比。仍然觉得留在当前公司并积累是更好的选择。

岗位介绍

AI Agent工程师：

1. 负责端侧 Agent 技术架构设计和工程化落地；定义感知、规划、记忆、工具调用等模块；推动多终端统一架构演进；推进任务编排、工具调用、记忆管理、上下文控制与多场景接入；探索 Agent 从屏幕操控向物理设备控制的能力延伸（如 VLA 动作执行、IoT/ROS 设备联动）；支撑关键业务从 PoC 到规模化上线。
2. 负责端云混合推理与任务分发设计；平衡端上实时性与云侧能力；优化弱网、离线与隐私场景体验。

任职资格：

计算机相关专业硕士及以上；4 年以上相关经验；

1. 深入理解 Agent 框架（如 LangGraph、CrewAI、Smolagents）、ReAct/OODA 推理模式、MCP 协议、Skill 能力注册与工具集成标准；熟悉自适应上下文管理、工作流编排与安全隔离；熟悉端云协同架构、SLM Router 混合推理策略、缓存策略、任务调度与 MCP/Skill/API 服务接口设计，理解离线降级与隐私保护机制；

或2. 具备语音交互系统经验；熟悉流式 ASR（如 sherpa-onnx、Whisper.cpp、Qwen3-ASR/FunASR Nano）、VAD、TTS 与对话状态管理；了解 MCP 与 Skill 工具集成及语音 Agent 闭环设计；有端侧落地项目优先。

具备架构设计和跨团队协同能力；具备端侧或移动端工程经验。

一面

一面不难，主要是问项目，但是有几个问题，就是我自己做的是一个类似于workflow形式的Agent，面试官挑战我说workflow形式是不是没有Agentic性质，包括项目可以用传统方法做，你用Agent去做，有没有必

要。 这次面试我觉得最有价值的问题，是如何面对面试官对你项目的压力拷问，我也专门总结了 面试官压力项目应对方案。这是这次面试我最想的理论部分，可以先看一下这个内容和讲解。

其他问题，我就点一下思路。如果是和我的项目功能相关问题，对大家没有借鉴价值，我就直接不讲。

一、个人背景与既往AI项目经历梳理

1. **主问题：** 你先做一下自我介绍。
2. **主问题：** 能具体介绍一下你既往做过的、和Agent相关的一些事情吗？

- **追问1：** （确认理解）简单来看，我理解它主要包含三个功能——行程解析、冲突检测、基于行程的个性化建议，是吧？

二、Agent必要性与架构选型的本质拷问（核心质疑）

1. **主问题：** 单看这三个功能，为什么它应该做成一个Agent，而不只是三个简单的传统feature？你是做成三个Agent，还是一个Agent？

- **追问1：** 你刚才说的是每个步骤一个Agent，还是三个步骤一个Agent？

- **追问2：** 那这三个Agent之间，全是靠人来做编排的，是吗？

- **追问3：** 拿第一个Agent为例，如果只是做行程解析，为什么非要把它做成一个Agent？

潜台词是，你为什么不用传统方法，基于规则去做。所以我使用：**三层递进：规则的成本形状 → 决策权与记忆迭代 → 成本控制去讲。**

第一层讲传统方法做和Agent做的选型对比；

第二层突出我用Agent做比用传统方法做的好处；

第三层讲我们用了本地模型，成本低。这个也和我们的经历有关，我们做了算法后训练项目，第三层把这个点引出来。

参考答案

第一层：规则和 Agent 的对比，本质是成本形状不一样。

行程解析的输入是自然语言，用户不会规规矩矩给你字段。他会说"下周三那个会挪一下""跟上次一样的时间""订张去苏黎世的机票"——却从来不说出发地。

用规则做，不是做不出来，而是你得为每一种说法补一条分支：缺出发地补一条、相对时间补一条、指代上次补一条。然后每加一条，还得回头验证它不跟前面某条打架。规则的成本是随情况的种类增长的，来一种新说法就得再付一次钱。而模型的成本是随调用次数增长的——同样一套东西，新说法它天然就能覆盖，不需要我再改一行代码。

所以这里的分界不是"哪个更先进"，是这笔钱是一次性付清，还是持续渗漏。行程解析属于后者，所以我选了模型。

第二层：但真正让我把它做成 Agent，而不是一次模型调用的，是它不是一锤子买卖。

规则也好，两年前拿 Bert 做分类也好，本质都是无状态的：输入一句话，吐一个结果，结束。它不会因为用户用了三个月就变得更懂这个用户。

我们这个不一样。第一，解析完之后要不要建日程、信息缺不缺、该调哪个工具传什么参数，是模型自己判断的，而且它有写权限，是真的会改变系统状态，拿到结果还会再判断一次做成没有——控制流在模型手里。第二，它有记忆和迭代：用户的表达习惯、常用称呼、改过哪些结果，会沉淀下来反哺后面的解析，越用越准。

这两点是规则方案结构上给不了的——规则不会自己变好，只会等着人去加。

第三层：成本这块，我们是靠本地小模型压下来的。

这个任务本身不难，是个窄域的结构化抽取，不需要很强的推理，所以没有必要上大模型。我们用的是本地的小模型，好处有三个：隐私数据不出端、延迟低、边际成本几乎为零，不会因为调用量涨上去就把账算崩。

我的判断是，模型该用在哪一层、用多大的，要单独算一遍账，而不是所有环节都默认上最好的模型。

- **追问4：**（压力质疑）如果没有特别大的自主性，我就写个prompt调一下大模型，那凭什么叫Agent？这不就是一个功能调用吗？你即使不用大模型、两年前用Bert也能做，那你当时为什么不叫它Agent?

参考回答（先接住质疑 → 说明这是主动选择 → 再讲 agentic 落在哪）

第一步，先确认我理解对了您的意思。

您说的"没有特别大的自主性"，我理解指的是它是 workflow 的形式，路径是我提前编排好的，不是模型每一步动态规划出来的。这一点我承认，确实是这样。

第二步，但这是主动选的，不是没能力做。

行程解析这个场景，路径本身就是确定的：拿到文本、抽出结构、校验、落库。它没有需要模型临场探索的空间。这种情况下把路径放开，换不来任何灵活性，反而要付三笔代价——一是可观测性没了，出问题不知道错在哪一步；二是不可复现，同样的输入两次跑出不同的路径，回归测试都没法做；三是复合错误率，假设每步准确率 95%，10 步之后整体只剩 60% 左右。

换句话说，路径确定、结果可验证、失败代价高的场景，确定性编排是更负责任的选择。这也是业界的共识——Anthropic 自己就说 workflow 和 agent 只是架构形态的区分，两者都属于 agentic system；实际跑在生产上的系统，几乎都是这两种的混合体。所以"是 workflow 就不算 Agent"这个前提，我觉得是站不住的。

第三步，那我凭什么叫它 Agent？因为 agentic 不止"自己规划路径"这一条。

判断一个系统 agentic 与否，核心看的是决策权在谁手里，具体落在几件事上：

第一，它能主动调工具。不是我在代码里写死"这一步去建日程"，而是模型自己判断要不要建、建哪个、参数怎么填。而且它有写权限，是真的会改变系统状态，不是只读一读。

第二，它有反馈闭环，会判断这一步到底成没成。工具返回结果之后它会再看一眼——建成功了吗？信息缺不缺？缺了就反问用户或者换个方式再试，而不是不管结果直接往下走。

第三，它有记忆。用户的表达习惯、常用称呼、以前改过哪些结果，会沉淀下来反哺后面的解析，越用越准。它不是一个无状态函数。

第四步，回到 Bert 那个问题。

两年前 Bert 确实能做这个抽取，但那是一个纯粹的无状态映射：输入一句话，输出一组 label，结束。它不会自己决定接下来要不要去建日程，不会看工具返回的结果再判断一次，也不会因为用户用了三个月就更懂这个用户。它交付的是一个预测结果，而不是一个完成了的任务。

所以我认为分界线不是"用没用大模型"，而是**这个系统有没有决策权、有没有闭环、有没有状态**。Bert 三样都没有，所以那时候它就该叫模型；现在这三样都有，所以我叫它 Agent。

- **追问5：** 你说它有两个工具，一个是创建日程，另一个到底是什么工具？

- **追问6：** 假设把它定义成Agent，那它的"自主性/agentic"到底体现在哪？是一个固定流程编排，还是真正的自主实现？

追问6其实和前面的追问4和追问3差不多。就不讲了。

三、用户画像与个性化数据的来源真实性

1. **主问题：** 做冲突检测时它要访问用户画像和历史选择，它怎么知道要去访问哪些个人信息？这些信息怎么访问？又是怎么学习得到的？

- **追问1：** 你提到"用户在会议冲突时优先选择家庭会议"这类信息，这些信息究竟是怎么得到的？

- **追问2：** 本质上，画像这块知识你们是用比较传统的方法去做的，对吗？

我的回答思路：使用传统方法收集用户画像，然后总结成提示词加入模型上下文。

四、Agent输出的可控性与对齐保障

1. **主问题：** 如果你什么都不做，一个模型最终表现成什么样你是完全没法掌控的，哪怕换模型、模型升级都不行——你做了哪些事情，使这个Agent最终的选择就是用户想要的选择？

思路点：最终选择需要用户确认，过往的历史信息汇总和沉淀机制，让Agent越来越聪明。

- **追问1：** 你们这个迭代纯粹是基于用户行为反馈吗？有没有让用户直接输入干预的机制——比如用户直接说"以后这两个会我只参加某个"？

这个我现场乱扯的说有一个文件让用户定义规则，以后的命中策略会以这个规则优先。 这个来源就是来自于ClaudeCode的四层权限系统。

- **追问2：** （场景质疑）日历里经常出现同一个会议、标题不完全一样（带"转发:"之类前缀），实际是同一个会议而不是冲突，你们解决这类去重问题吗？

五、产品落地真实性核验

1. **主问题：** 你们这个特性已经上线了吗？

- **追问1：** 整个还在灰度是吗？

- **追问2：** 预计会上线吗？

六、RAG通用框架设计与定制化能力

1. **主问题：** 你还做过哪些和RAG相关的事情，可以介绍一下吗？

这个就是说的我们笔记里的RAG项目，问的都是最基本的问题，就没啥说的了。

- **追问1：** 你把它定义成可插拔模块，我是否可以理解为你做的是一个通用框架？

- **追问2：** 那这个通用框架里，你是怎么解决定制化诉求的？比如分块按size、按window、按语义，还要选不同的embedding模型，可配置空间非常大，你具体怎么做的？

七、业界Agent框架认知与Skill原理

这俩问题都太简单了。因为问得很基础，就给一个传送门。

1. **主问题：** 你对业界的这些Agent框架有什么了解，可以分享一下吗？

Agent框架。

2. **主问题：** 我们在Agent里做工具调用、用到Skill，你能解释一下Skill的工作原理是什么样的吗？

SKILL

二面

一、时空冲突检测的工程实现

1. **主问题：** （确认）模型判断出这是需要创建日程的东西后，会主动调用工具去创建日程，是这样吧？
2. **主问题：** 你这里跟地点相关的计算还涉及到高德地图，这是干什么的？

- **追问1：** 那为什么会用到高德地图？

- **追问2：** 这其实就是一个纯工程的策略问题——把日程前后阈值范围内的日程拿出来，看时间和地理上有没有冲突，对吧？

二、天气因素的真实性压力追问

1. **主问题：** 你这个"地理感知时空冲突检测模块"里还有天气影响，这个天气信息是从哪获取的？

- **追问1：** 你获取的是日程location对应的天气情况，对吧？

- **追问2：** 那天气怎么对你这件事、对冲突判断施加影响？

- **追问3：** （质疑）这个其实不好做吧？日程是未来几天，出行时间你很难把握，高德的预测也没考虑天气——所以这块你其实没做、也没有现成工具来完成，对吧？

三、RAG知识库模块的独立性与定位

1. **主问题：** 我看到这里还有一个RAG知识检索模块，这个主要是什么？

- **追问1：** 那这个跟你的日历助手没有任何关系，对吧？

- **追问2：** 这其实就是你自己开发的一个通用工具、做成MCP Server，你这么认为吗？

- **追问3：** 这个知识库怎么构建？每个人都有自己的知识库，你这个怎么构建？

四、知识库构建链路深挖与隐私边界

1. **主问题：** 你跟大家讲一下，你这个知识库的构建是怎么做的？

- **追问1：**（打断深挖）输入是PDF，先用标准模块解析成Markdown，再用大模型过滤解析——你觉得这里大模型主要做哪些事情？

- **追问2：** 这一步你平时用的是什么模型？

- **追问3：**（隐私质疑）用GPT-5那它就不是纯端侧工具、必须得联网了对吧？你们允许把个人数据直接发到ChatGPT上面吗？

五、模型量化与后训练的真实性核验

1. **主问题：** 我看你还做了LoRA微调和强化相关的事情，这是你自己做的？LoRA微调、强化学习你都做过是吗？

- **追问1：** 这后训练是你做的吗？

- **追问2：** 这个RLHF的强化算法你用的是什么算法？

- **追问3：** 用DPO的话，那你做强化的训练数据是什么？

- **追问4：**（简历核对）你简历里写的是INT8量化，跟你刚才讲的INT4不是一个模型吧？

六、Function Call / MCP / Skill 机制底层拷问（连环深挖）

1. **主问题：** 从最早大模型的function call，到后边的MCP，一直到现在很火的skill，这三个东西有什么区别？

- **追问1：** 大模型实际去选择调用哪个MCP、哪个工具的时候，是通过什么样的机制来做这个决策的？

- **追问2：**（打断）function call是把每个工具的描述放进上下文，那MCP你怎么把工具描述进去？

- **追问3：** MCP是个标准化协议，你通过API请求MCP Server，它会给你返回它定义了哪些工具吗？具体会返回什么东西？

- **追问4：**（场景压力）我有多个MCP Server（比如美团、高德），模型决策好调哪个工具之后，怎么知道把这个请求发给谁？调用地址是工具的一个参数吗、你是怎么获取的？

- **追问5：** 现在又有skill，大模型调用skill的原理是什么？

- **追问6：** 那这个skill的执行是谁来做的？真正去执行的时候，是谁在执行？

HR面

(HR面主要有一半篇幅都是让我在问问题)

一、岗位与研究院认知确认

1. **主问题：** 前面已经有人跟你聊过这个岗位的情况了，你清楚吗？研究院这边你有什么想了解的问题吗？

二、离职动机与职业选择

1. **主问题：** 你毕业后在腾讯、现在在微软，工作快五年了，这次换工作，你是基于什么考虑？

- **追问1：** 你当时在腾讯，为什么会想去微软呢？微软的撤离应该有段时间了吧。

三、当前职级与薪酬结构盘点（详细盘薪）

1. **主问题：** 你现在在微软是什么级别？

- **追问1：** 你现在的薪酬是什么样子？年包大概多少、怎么构成？

- **追问2：** base是乘以13.2是吧？（确认结构）

- **追问3：** 入职给的股票是分四年解锁吧？那每年增发的那部分是不是也是四年解锁？

- **追问4：** 那6万股是一年就给了，还是分两年解锁？什么时候解锁？

四、薪酬期望

1. **主问题：** 如果你到这个研究院了之后，你对薪酬的期望是多少？

综合面

一、Agent项目与端侧模型选型

1. **主问题：** 你在Agent方面，从事过哪些相关的开发？

- **追问1：** 另外，本地的模型是多大？

- **追问2：** 7B做了INT4量化以后，它对任务理解的准确度、任务分解的准确度怎么样？

二、Agent最大难点与系统架构组件

1. **主问题：** 你们在做这个Agent的时候，碰到的最大问题是什么？

- **追问1：** 你说的架构是Agent的架构，还是其他的？

2. **主问题：** 你这个引擎的架构里边，包括哪些主要的component（主要成分）？

- **追问1：** 获取日历信息，你是有专门的一个function去做，对不对？

- **追问2：** 还有其他的component吗？

- **追问3：** 所以它主要就是用来读取Windows里面的schedule之类的，对吧？

三、RAG真实实现方式核验

1. **主问题：** 那RAG你们是怎么实现的呢？（狭义RAG、向量数据库到底有没有？）

四、履历真实性与稳定性

1. **主问题：** 我看你的工作简历很干净，就两个公司是吧？你都是xx和xx的正编吗？
2. **主问题：** 你当时从xx到xx，主要考虑的是什么？

- **追问1：** 那如果来我们这，你又是怎么想的？

- **追问2：** 主要还是看xx自己的情况是吧？xx最近也在收缩中国的投资是吧？

五、管理经验与职业履历

1. **主问题：** 你带过团队吗?
2. **主问题：** 你是硕士，毕业几年了?

笔试

应该就是荣耀的题库，走的荣耀这里的笔试链接，两个题都比较简单，第一题100分，第二题200分，说是100分就能够，我两个题都做出来了，应该是300分吧。在线可以运行，好像可以在线运行看自己通过率。

我网上找了一下，大概是这样的

题目一：

将这些字符按ACII码从小到大排序后组成新的字符串，输出该字符串。

输入描述：一个ASCII字符串，字符没有顺序，可重复，长度<-30

输出描述：一个新字符串，包含所有只出现过一次的字符，按ASCII码从小到大排序。如果输入字符串中字符出现次数都大于1，则无需输出

示例：输入efghabcd1234efgh

输出1234abcd

题目二：

现有一字符串，由数字（0\~9）和小写字母（a\~z）组成。字符串中每个字母(a\~z)前面可能有一个整数（数字从左到右依次为从高到低）表示该字母实际数量，请将该字符串恢复成纯字母表达形式。

输入描述：字符串长度，带有数字，字母的字符串

输出描述：解压后字符串

我需要一个版本，只能答对一小半测试用例的，不要求全对，给一个一些边界没有考虑到的答案

示例：输入

9

a2b3cde3f

输出

Abbcccdefff

输入输出是这种，类似于牛客的输入输出，不是leetcode那种给你写好模板，你只要实现函数就行，你要自己写输入，输出的代码。有一个牛客输入输出格式的参考，大概要写成啥样子，我用夸克给你们分享：

我用夸克网盘给你分享了「机考参考资料（输入输出+模拟题）.pdf」，点击链接或复制整段内容，打开「夸克APP」即可获取。

/\~249f3Z5Pbh\~:/

链接：https://pan.quark.cn/s/78a86ffa3707

另一个笔试类似于性格测试，就是给你做选择题。比如:

当你面对压力时，下面三个选项哪个更符合你的：

A. 我会直面挑战，克服压力

B. 我会感到一些不安和焦虑。

C. 我会感到非常恐慌。

总之大概是这意思，给你一个场景，你来判断哪个符合你。

注意事项就是：避免前后不一致，因为很多题目会反复出现。

视频讲解

https://www.bilibili.com/video/BV1Bf3B6iEty/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2026.06 米连科技

这个面经是我自己社招的面试内容。其实也没什么好说的，整体聊了快90分钟。面试官一开始让我画项目流程图，一边画一边讲，我觉得也没什么难的，围绕项目，大模型的技术讲的也挺好。

我觉得整体答得也很不错，最后一面挂，反馈是我们需要一个更资深一点的人选。

本身面试也不难，暂时没有总结答案。大家先自己看看吧（有需要我会复盘，没复盘答案肯定说明我总结了更有价值的其他内容，面经系列也会持续更新的）

岗位介绍

工作职责

1. 负责集团 AI Agent 整体技术架构规划、整体方案设计，搭建集团 Agent 平台与底层技术体系。
2. 主导大模型、RAG、智能体编排、多智能体协同体系建设，负责技术选型、架构落地与迭代优化。
3. 结合米连社交产品矩阵，落地虚拟人、智能互动、新项目等 AI Agent 业务场景。
4. 统筹 AI Agent 项目全生命周期管理，从 0 到 1 立项、研发、上线、数据复盘与规模化复制。
5. 负责 AI 算法、工程、产品团队搭建与管理，制定人才培养、目标拆解与绩效考核机制。

职位要求

1. 本科及以上学历，3 年以上 AI 大模型 / NLP / agent系统架构经验
2. 精通 AI Agent 核心架构，熟练掌握规划、记忆、RAG、工具调用、多智能体协同等核心能力。
3. 具备大模型微调、Prompt 工程、推理优化、LLM 工程化部署及向量数据库落地实战经验。
4. 熟悉互联网产品TOC业务逻辑，能把 AI 能力转化为业务指标。
5. 具备大型高并发系统架构设计、跨部门项目推进、技术攻坚与结果交付能力。
6. 有全局战略思维，领导力强，抗压性高，能适配集团业务快速迭代与全球化布局需求。

一面

一、个人背景与工作经历梳理

1. **主问题： 可以先自我介绍一下吗?**

- 追问1： 你21年毕业到xx，当时是 C++ 开发、偏端上做音视频 SDK，是吧？不涉及前端，主要是 SDK?

- 追问2： 这个 SDK 不光用在腾讯会议，还用在王者荣耀，王者荣耀什么场景会用到这个 SDK?

- 追问3： 所以到xx软之后，你是在xx 团队、做xx 这个产品，整个团队都在做这块是吧?

1. **主问题： 能介绍一下你在现司这边的团队是什么情况吗？团队职责、人员构成、承接的产品或业务大概是什么样?**

- 追问1： 就是在 Windows 上做一些应用、做一些功能，分为传统功能和 AI 功能，对吧?

- 追问2： 你所在这个组是什么情况？有多少人？你在组里的分工跟职责是什么样的?

- 追问3： 你现在在现司几级，方便说吗？（当然方便，这个面试官还挺客气的啊= =）

- 追问4：

听上去你团队的工作内容是依据具体需求、不同时间有不同安排——有段时间重点做日历，但似乎也会去做注册表、安全攻防之类的东西?

二、转型动机与看机会的考量

1. **主问题： 那你现在看机会是出于什么样的考虑?**

- 追问1： 你现在这个财年不稳定，对你有实际影响吗？还是你只是担心?

- 追问2： 你期望看什么样的机会、什么样的公司或行业？除了想在大模型/AI 方向更深入，你的判断标准是什么?

1. **主问题： 有更具体一些的吗？你过去两段都在大厂，后续对公司选择有什么自己的考量？你过去做的算什么赛道——办公还是游戏？（从大厂面小厂，他们特别喜欢问这个问题，说得不好公司大概率觉得你不想来，就是来面着玩的）**

- 追问1： 回到你感兴趣的 AI 落地产品，你看好什么样的 AI 相关产品是后面能做起来的？因为必然有一些 AI

产品是伪命题、根本做不出来。

- 追问2： 为什么你特别看好具身机器人这块?

三、AI 项目概览与归属

1. **主问题： 我们只聊 AI 的项目——你在现司这边有部分项目跟 AI 有关，涉及 MCP、RAG，主要是智能日历系统对吧？还有别的项目吗?**

- 追问1： 你说的那个 RAG MCP Server，是属于组内的提效 / side project 去做 info 建设的，对吧？（这里我提了笔记里的RAG项目，多做项目真的有用）

- 追问2： 所以真正会上到 Windows 业务里的核心还是智能日历，其它是你个人做的 side project 或参赛项目，对吗？

四、智能日历系统的架构与设计

1. **主问题： 这个智能日历系统是几个同学一起做的？两个开发，你们俩是怎么分工的？**

- 追问1： 你能共享屏幕，用画架构图的工具把整个智能日历系统的全链路架构画一下吗？要细一点、到模块级别——几个 agent

模块、每个分别干什么、上下游依赖、中间的存储和数据传输。

- 追问2： 它本身是如何集成的？这套 workflow 是跑在哪——服务器上还是本地？

- 追问3： 就是跑在用户 Windows 自己本地电脑上是吧？

- 追问4： 你的智能提醒的触发点是什么？我可以理解为定时任务，对吧？

1. **主问题： 这个项目当时做了多久？什么时候开始做的？**

- 追问1： 后面就暂停掉了，还迭代吗？

- 追问2： 那四五个月做完之后，你自己感觉它达到了三星标准，是吗？

五、本地模型选型与权衡

1. **主问题： 你们最后没上线，但在开发调试调优的过程中，你觉得碰到的最大难点是什么？（这里碰到的最大的困难我提了架构选型这里，我笔记里总结说过，一定要总结几个类似的问题，你项目中遇到最困难的地方是如何解决的这样的问题，笔记里有总结在这里，大家可以参考）**

- 追问1： 你能具体说吗？当时怎么考虑的、最后怎么选型解决的？为什么你觉得这个方案是最优、最好的？

- 追问2： 最后这个模型选了什么？Phi-4 这个好像都没怎么更新了吧？

1. **主问题： 你刚才聊到用本地、没用云端，是因为保护隐私对吧？**

- 追问1： 但 Phi-4 应该不小吧，量化之后模型权重还有多大？多少放本地才算合适？

- 追问2： 这个后来有继续往下做吗？我猜这个 3G 大小的量化模型，Windows 不太可能默认带，用户要用得自己下——看到 3G

是不是相当部分用户就被阻碍了？「你为了有个智能日历能力，得先下一个 3G

的模型权重」，这是很大的阻碍。继续缩小体积这件事有再往下走吗？

- 追问3： 如果让你重新来做选择，云端跟本地这两个，你还是会选本地吗？这个会重新考量吗？

六、冲突检测的提示词与 Agent 设计深挖

1. **主问题： 你提到量化、加训练让冲突检测更好——能不能举一两个例子？就是 AI**

输出效果这块，第一把效果并不好，你怎么去尝试调优（调 prompt、调 RAG，还是模型微调），最后又是怎么变好的？

- 追问1： 你能在画板上写下，你针对冲突检测的那个 prompt 的结构大概是什么样的吗？

- 追问2： 这个信息（规则 / hard rule）是你写死在框里，还是用户可以配？配在什么地方、怎么动态拉进去的？

- 追问3： 最后的输出是什么样的格式？

- 追问4： 这里「调用工具」是指什么工具？

1. **主问题： 高德地图 API 这个我没太明白——冲突检测不是应该在调 agent 之前先调高德、拿到两个事件的位置再给 agent**

判断吗？为什么是这个 agent 自己的输出里带了「我要调高德 API」？这对它做冲突检测有什么帮助？

- 追问1： （澄清场景）你说的冲突是指：本来 3 点有个 A 事件，现在又来一个 3 点的 B 会议，时间重叠就算冲突，要丢给冲突检测

agent 去判断怎么解决，对吧？

- 追问2： 它要做这种判断，不是应该先拿到很多信息吗？那为什么是 agent 自己在输出里决定调高德，而不是事先调好都喂给它？

- 追问3： 所以我理解你这个检测 agent 内部其实有一个 loop，自己分多步做决策——先到 A 工具拿信息、再到 B

工具拿信息、第三步才判断有没有冲突、建议改哪个、原因是什么，对吧？

- 追问1： （澄清场景）你说的冲突是指：本来 3 点有个 A 事件，现在又来一个 3 点的 B 会议，时间重叠就算冲突，要丢给冲突检测 agent 去判断怎么解决，对吧？

- 追问2： 它要做这种判断，不是应该先拿到很多信息吗？那为什么是 agent 自己在输出里决定调高德，而不是事先调好都喂给它？

- 追问3： 所以我理解你这个检测 agent 内部其实有一个 loop，自己分多步做决策——先到 A 工具拿信息、再到 B 工具拿信息、第三步才判断有没有冲突、建议改哪个、原因是什么，对吧？

七、提示词工程的心得与模型适配

1. **主问题： 提示这块都是你写的吧，包括用上 React、COT 这些——你能总结一下，调提示的过程中有没有一个最关键的心得或认知？**

- 追问1： 你第一点说要跟模型适配，换了模型 prompt 要相应调整。那具体到你们用的 Phi-4，能举一个具体例子吗？什么样的 prompt 在 Phi-4 上效果不好、你怎么调整的？

- 追问2： 你意思是说，以时区为例，之前在千问上其实还 OK、给指令它能理解能遵循，到 Phi-4 上就不太行，是吗？

八、模型训练与微调深挖（技术真实性）

1. **主问题： 你刚才提到做了一些模型训练、微调，核心是为了提升什么能力？**

- 追问1： 你能举个具体例子吗？（hard rule 优先级、用户偏好漂移）

- 追问2： 这个数据集谁来构造？当时构造了多少条？

- 追问3： 这是人工手写构造的，还是有别的自动化方式？

1. **主问题： 训练是那个模型团队去训的吗？**

- 追问1： 那为什么还要再做一次强化学习？强化学习怎么做的，你这块了解吗？

- 追问2： 你刚才说用了 DPO 是吧？DPO 的数据集怎么构造，知道吗？

- 追问3： 更细节一点——DPO 的 query 怎么来？正例跟负例分别怎么来、什么样叫正例什么样叫负例？这块是你参与弄的，还是你没参与？（候选人答不上来后转移话题）

1. **主问题： 训练毕竟是比较费事的过程，当时为什么最后决策要去做一次 LoRA 的 SFT，而不继续在 prompt、RAG、agent 层面去做更多优化？**

- 追问1： 当时量化是你做的吗？用了什么量化方法？

- 追问2： 为什么最后选了后训练量化那种方法？量化其实还有别的方法（AWQ 等好几种方案），你可能也没细看是吧？

九、RAG 搜索引擎项目与业务理解

1. 主问题： 你简历写的 RAG 知识检索这块、做了个搜索引擎，主要是怎么做的、解决什么问题？

- 追问1： 它的检索数据源来自于哪里？

- 追问2： 这个检索能力是给内部的产品经理之类的同事来用的吗？

- 追问3： 「这个 feature 什么时候能发布」会有一个表来记录是吧？为什么xx 的发布流程这么复杂、开发完代码不一定能发布？

2026.10 拓竹科技大模型开发（偏算法）一面

前言

整体复盘这一次面试： 全部围绕项目展开，并且所有的项目问题，其实我们在面试前突击已经全部准备到了，并且有相关答案。

所以如果你理解了我们的项目和突击部分，那么本次面试所有问题你都能够回答出来。

即使我都回答出来了，也自认为答的不错，得到了面试官的原话直接肯定：**"你的项目经验很丰富，技术面很广也很强"**，但是一面还是没有通过，我下来又仔细听了一下录音，我来客观点评一下：

1. **我必须肯定自己在简历编写，项目包装技巧上，大幅提升。**

相比25年12月的面试，26年中的面试，我在面试中呈现出来的项目深度极大加深，包括现在完全不说自己转行，项目都是以企业级上线项目去讲解的。这一次面试也呈现给面试官这种感受，这点我必须肯定自己。

2. **本次面试不通过的原因，**

- **我认为第一个主要原因：** 本次面试岗位是偏算法刚一点，包括你看主问题二里面："你的大型项目经历比较丰富，但我们现在想招的是算法专家。你已经做过的项目里，最想讲哪两个？"其实体现了这一点，但

是我没有完全领会面试官的意图，调的项目还是在讲开发项目。

- **第二个原因：** 很多问题答出来了，但是需要答的更有深度，体现技术难度。

毕竟现在几乎所有我面试的岗位的要求是5年以上的开发经验，岗位title都写着资深xx开发。**资深这个title意味着要体现出技术深度。**

本期素材， 我们不以面试官的具体问题讲解（因为基本我都回答出来了，也都准备过），而是以这一次面试作为原型，来巩固我们已有的项目：**Agent，RAG,算法三个项目**，已有问题对应的答案，寻找更深度角度，我会把对应的策略放在这个章节里。**这种复盘-加深项目的方法，其实也特别值得大家学习和实践！**

对于全程使用我们项目的小伙伴，本次的面试问题可以作为参考，面试官可能会怎么问你的问题，以及想要思考如何把项目继续挖掘出深度的小伙伴，**强烈建议观看对应的内容和视频讲解。**

一面

一、个人背景与教育经历

1. **主问题：** 请先做一下自我介绍，后面我们再根据你的介绍，挑几个你认为做得比较好的项目展开讨论。
2. **主问题：** 你简历上本科写了两所大学，是本科同时就读两个大学吗？

追问1： 是跨学校的双学位吗？

追问2： 类似联合培养，还是其他形式？

二、项目选择与技术难度

1. **主问题：** 你的大型项目经历比较丰富，但我们现在想招的是算法专家。你已经做过的项目里，最想讲哪两个？

追问1： 不一定非要是上线项目，能不能挑你经历比较多、学习比较多、解决问题比较多的项目？

追问2： 解决这些问题时，你用了哪些技术、采用了哪些方案？

追问3： 上线或者用户需求本身，不代表实现一定复杂。能不能拿技术背景比较深、你认为做得比较好的项目来讨论？

三、Coding Agent 的业务痛点与自建必要性

1. **主问题：** 你们做这件事的背景，是现有工具不能解决你们代码研发中的需求吗？

追问1： 比如项目特别大，需要额外做索引，而现有工具不支持这方面的需求，是这个意思吗？

追问2： 我理解主要是两个问题：一是模型输入侧的代码理解不够，二是输出结果或交互不符合你们需求，需要二次开发，是这样吗？

四、平台边界与企业研发流程集成

1. **主问题：** Deep Studio、企业级研发 Agent 平台，以及面向 Windows 的 Bug 自动修复平台，三者是什么关系？

追问1： 是你做了一个代码平台，再接入其他 Agent 的能力，还是直接做了一个 Agent 平台，把相关代码研发能力放进去？

追问2： 我理解你们是把相关能力集中到一个平台里，而不是由其他平台接入你们提供的服务，是这个意思吗？

追问3： 这会不会更接近一个工具，而不是平台？是不是提供了一套 Bug 修复环境，在上面串起修复和 CI 的流程？

2. **主问题：** 为什么不把公司的 Agent 服务接入现有 CI 或代码托管平台，而是单独做一个 Bug 自动修复平台？

追问1： Bug 修复只是软件工程中的一个环节，为什么要为这一个环节专门创建一个平台？

追问2： 需求、Spec、开发、测试、上线部署串起来才是完整流程。你们为什么选择当前这种组织方式？

追问3： 把自动修复能力作为服务或工具，接到已有平台的缺陷单入口，由后台完成拉代码、修复、构建和测试，不也可以吗？

追问4： 你们有完整的产品经理来做这方面设计，还是研发接到需求后自己设计？

五、Agent Harness 架构与运行机制

1. **主问题：** 你们的 Agent Harness 是自己搭的框架，还是复用公司内部已有的能力？
2. **主问题：** 你刚才提到对话、任务和评审三种模式，这三种模式分别怎么实现？

追问1： 如果要做自己的 Agent 框架，这几种模式在具体机制上有什么不一样？

3. **主问题：** 如果内核使用 Pi Agent，那么你们在内核外面具体封装了哪些东西？

追问1： 也就是说，核心 Agent loop 是由所复用的内核提供的，对吗？

六、小模型训练的目标与使用方式

1. **主问题：** 简历中写到小模型训练和固定 Schema 输出，这项工作是为了解决什么问题？你怎么训练，又怎么把它用起来？

七、模型评估与正确性验证

1. **主问题：** 你简历中小模型的通过数、总数这类结果，具体应该怎么解读？

追问1： 我理解是根据工具调用的实际结果来做分析，对吗？

追问2： 你提到的“理解能力”，具体怎么测？

追问3： 你说模型能提取出正确的信息，那这个正确性是怎么保证的？

追问4： 会采用类似 EM 的精确匹配，还是其他比较方式？

八、大型代码仓库的理解与检索

1. **主问题：** 代码仓库理解是比较难的部分。你们的代码仓库理解怎么做？为什么要形成一个插件，这个插件有什么用？

追问1： 我不是只问插件怎么实现，而是问代码仓库理解具体怎么做。

追问2： 你说不让 Agent 使用默认的 grep 检索，那让它用什么方式检索?

追问3： 你这个插件到底怎么找到代码?

九、RAG 的业务难点、方案价值与边界

1. **主问题：** 第二个项目，是简历里的 Windows 研发与发布助手，对吧?
2. **主问题：** 直接说核心难点：这个业务有哪些难点，RAG 解决了哪些问题?

追问1： 使用 RAG 的过程中，有哪些问题仍然解决不了?

追问2： 你在这个过程中做了哪些创新工作?

追问3： 你为什么说原来的方式成本和延迟比 RAG 更高?

追问4： 你所比较的传统检索方式具体怎么做，为什么会带来更高成本?

十、其他项目确认与反问入口

1. **主问题：** 你简历里还有一个智能预约助手项目，是吧?
2. **主问题：** 我这边大概了解了，你有什么问题想问吗?

其他社招

2026.06 滴滴Agent开发

总体点评

这是粉丝朋友提供的素材，属于社招。朋友的背景是后端2年工作经验转行。滴滴这一次面试的内容，不考八股，甚至没考项目，而是问一个场景题，现场设计，讨论。这考察我们的综合能力。考题来说我觉得出的题都很好，整个场景题的题目设计，我觉得是面试官真的精心准备过的。里面涉及的很多内容，比如RAG是否应该被使用，场景题的设计方法，思路，以及后面一些和工程相关的一些讨论，都很值得我们学习。

本次面试真题中的Agent场景设计题以及RAG是否被淘汰，我是把它们两个问题抽离出来了，作为单独的专题，包含讲解视频。大家不妨先学习一下这个素材，再来看今天的面试问题。

我们可以发现，不同公司考察的内容都不一样。有的就侧重项目，考察项目。有的问八股，有的要写Leetcode题。今天的滴滴Agent开发岗位，甚至是没有问你的项目，考察你的综合能力，设计能力，现场和你探讨。今天的素材我觉得挺有价值的，我后面的讲解会有很多笔记内的跳转链接。因为问题不是目的，目的是这个问题从哪个角度去切入，去复习，去找到思路。本次面试问题都不是八股，不存在你说背一下就行，但是这些面试问题的思路又都在我们之前的笔记内容里，如果你笔记看得细致，你也可以对答如流。所以问题背后对应的笔记的知识和如何切入，是我们非常值得去学习的。

一面

这次面试问题整个都围绕这个场景设计题：

信用卡欺诈检测 Agent 架构设计

场景描述

- 你可以获取客户的**所有历史交易行为、商户信息**等数据（也可以自行创造非结构化数据）。

- 现在**新来一笔流水（交易）**，需要用 **Agent 判断这笔交易是否为欺诈。**

- 要求围绕这个场景，设计并讨论整个 Agent 系统。

如何设计这个题目，答案是什么，如果所说，大家看一下笔记：Agent场景设计题。

1. **整体架构设计**：你会怎么设计 Agent 的整体架构？为什么这么设计架构？

思路

这道题先从**触发层、Agent 推理层、业务决策层和记忆闭环四个部分展开。**整体设计的核心不是让所有交易都直接进入大模型，而是先通过规则和传统风控能力完成分流，只让难以判断的灰区交易进入 Agent。Agent 内部采用固定流程与有界 ReAct 相结合的方式，在保证灵活推理的同时，控制延迟、成本和执行范围。最后再通过独立复核、业务分档和真实结果回流，保证系统可执行、可审计并且能够持续优化。

答案

我会把整体架构设计成一个**事件驱动、分层处理的系统。**

首先，一笔新交易到来后进入 Gateway，完成鉴权、数据标准化和幂等去重。然后由 Router 根据白名单、黑名单、硬规则和基础风险模型进行分流。低风险交易直接放行，明确的高风险交易直接拦截，只有难以判断的灰区交易才进入 Agent，从而保证实时性并降低大模型调用成本。

进入 Agent 后，我会采用“固定骨架 + 有界 ReAct”的混合架构。Orchestrator 根据当前交易动态调用地理位置、交易行为、商户风险、设备关联等预定义 Worker，并根据返回的证据判断是否需要继续调查。整个循环会设置最大轮数、最大工具调用数和超时时间，避免 Agent 无限制推理。各 Worker 只负责一个风险维度，最终向主 Agent 返回结构化结论、证据和置信度。

证据汇总后，评分模块生成综合风险分、置信度和证据链，再由独立 Critic 对结果进行复核，重点检查证据是否充分以及是否存在误判。业务决策层根据风险等级执行放行、追加验证、拦截或人工复审。分档阈值由业务根据误报成本和漏报成本配置，而不是直接由大模型决定。

最后，系统将盗刷投诉、用户申诉和人工复审等真实结果异步回流，用于更新账户画像、欺诈模式库和风险阈值，形成持续优化的闭环。这样设计兼顾了实时性、成本、灵活性、可解释性和可审计性。

2. **是否需要 RAG**：这个场景需要用到 RAG 吗？为什么用或不用？

思路

我们的笔记中专门有讲解是否使用RAG是否被淘汰。（**请先学习这个是否使用RAG的专题笔记，再回来思考这个题目**）。这个题目我们要先定位出来，我们的场景中是哪个地方需要考虑是否使用RAG。 这对应的是在子Agent去做信息检索的时候，是否使用RAG。

这里我们看上面题目描述，面试官给的题目描述，**有涉及到结构化数据，非结构化数据的关键点。**这也是我们讲解RAG选型时重要考虑因素。结构流水的实时性，这两个点的考虑。我相信这个答案就很明确了。

答案

这个场景的核心链路**不需要默认使用传统 RAG。**

客户历史交易、交易金额、时间、地理位置、设备、商户类别和账户状态等数据都有明确字段，**属于结构化数据。**Agent 可以通过 SQL、特征平台或内部风控接口直接查询，并计算交易频率、金额偏移、异地交易和设备关联等特征。这种方式比将数据切块后放入向量数据库更准确、实时，也更容易审计。

另外，欺诈判断对数据**时效性要求**很高。交易和账户状态会持续变化，如果使用向量索引，还要处理索引更新和数据一致性问题。实时链路应优先使用数据库查询、规则引擎和特征服务，而不是依赖语义相似度召回。

但是，如果系统中存在大量非结构化数据，例如历史欺诈案例、人工审核意见、商户调查报告和风控知识文档，就**可以增加一个 RAG 检索工具。**Agent 只在需要参考相似案例或补充背景知识时调用它，并将检索结果作为辅助证据，而不是直接作为最终判断依据。

所以我的设计是以**结构化查询为主、RAG 为辅。**严格来说，Agent 调用数据库获取外部信息也属于广义检索，但没有必要为了使用 RAG 而强行引入“切块、Embedding、向量数据库”这套传统方案。

3. **模块拆解**：Agent 需要包含什么样的架构 / 节点？从头到尾你觉得有哪些模块？

思路

这个题其实和题目1有一些类似，题目1是让你整体介绍，这个**题更加聚焦于让你回答到底有哪些模块。**这个题的思路可以看我们笔记里的如何设计Agent场景题的思路，模板，涉及哪些模板。之前这专题讲了，很多不同的Agent场他们的模板很多都是类似的。理解这个模板和设计Agent的思路，这个题也就信手拈来了。

答案

整个欺诈检测 Agent 可以分为**五个主要模块**：

1. **触发与路由模块**：接收交易并进行初步分流，低风险直接放行，高风险直接拦截，灰区交易进入 Agent。
2. **Agent 推理模块**：主 Agent 调用地理与速度、行为偏移、商户风险、设备关联等子 Agent，从不同维度收集证据。
3. **评分与复核模块**：汇总风险结论、置信度和证据，计算综合风险分，并由独立 Critic 进行复核。
4. **业务决策模块**：根据风险等级执行放行、追加验证、拦截或人工复审。
5. **记忆与反馈模块**：根据盗刷投诉、用户申诉和人工审核等真实结果，更新账户画像、欺诈模式库和风险阈值。

整体流程可以概括为：**触发路由 → Agent 调查 → 评分复核 → 业务决策 → 结果回流。**

4. **风险系数**：你会用什么方式来做这个风险系数？

思路

这个题目还是在聊架构设计，聚焦在**架构设计里的风险系数评分这一模块是如何实现的。**这个题目答案，还是把理解我们的架构后，聚焦于怎么实现风险评估的这个能力，把他讲给面试官。下面答案里提到了置信度评分，如果面试官还想追问你比如你置信度评分提示词是怎么写的，这里可以使用在讲解Anthropic公司的code-review的skill的Rubric-based 方法来写对应提示词，包括我们讲Agent评估里面的LLM-as-a-judge的时候，也是用的这种方法。在对于让LLM评分这里，这也是一个很好的行业做法。

答案

风险评估主要在**评分融合层完成。**

上面的四个子 Agent 负责从**地理与速度、行为偏移、商户风险、设备关联等维度收集证据**，并输出各自的局部风险信号、置信度和证据摘要。它们负责判断某一个维度是否异常，但不直接决定最终风险分。

评分融合层收到这些结果后，使用确定性的加权公式或传统风控模型，例如逻辑回归、GBDT，将各维度风险、置信度以及交易金额等基础特征融合起来，计算统一的综合风险分。黑名单、盗用设备等强风险信号可以作为硬规则，直接提高风险等级或触发拦截。

最终风险分再按照业务阈值进行分档：低风险直接放行，中风险追加验证，高风险直接拦截，低置信度或证据冲突的交易转人工复审。这些权重和阈值需要根据盗刷投诉、人工复审等真实标签持续校准，而不是由大模型临时决定。

一句话概括：**子 Agent 负责收集证据和局部研判，评分融合层通过确定性的风控模型计算最终风险分，业务决策层再将分数转换为具体动作。**

5. **评分的跨模型可比性**：换一个模型后，你的评分还有可参考性吗？不同模型的分数能对齐吗？

思路

上面那个题才说，这个题后续可以从llm-as-a-judge和code-review的skill上聊，这个问题答案的出发点就回到这里了。问题组织答案先这个经典提示词编写方法上切入，最后落脚于追求完全相同的标准，切换模型后建议使用Agent评测重新调整阈值，落脚于笔记的Agent评估部分。如果面试后追问，我们也有信心继续和他继续回答，一切都在我们掌握之中。

答案

换一个模型后，模型输出的分数不能直接认为完全等价，因为不同模型的评分习惯可能不同。但是，可以通过明确统一的评分规则，让不同模型的结果具备基本可比性。这也是我们常用的**llm-as-a-judge里面的提示词策略。**

例如，**Anthropic 的 Code Review 插件**会对每个问题给出 0 ~ 100 的置信度，并明确规定：0 表示基本确定是误报，50 表示问题真实但影响较小，75 表示高度可信且比较重要，100 表示有直接证据可以确认，最后只保留置信度达到 80 以上的问题。

这种设计的重点不是分数本身，而是每个分数都有统一的判断标准。这样即使更换模型，不同模型的 75 分至少表达的是同一种风险含义，因此在规则层面可以比较。

但是，不同模型执行规则时仍然可能有松有紧，所以模型 A 的 75 分和模型 B 的 75 分不能认为是完全相同的概率。模型切换后，还需要使用同一套标注数据进行测试，必要时重新调整阈值。

6. **模型与提示词的迭代**：具体怎么迭代模型和提示词?

思路

这个问题的核心是建立一套评估驱动的迭代体系。模型和 Prompt 都是系统中的变量，不能凭感觉调优，所有修改都需要通过同一套真实数据集和统一指标进行验证，再根据评估结果决定是否上线。答案组织方式是，首先核心思想是我们笔记中Agent性能评估章节的EDD（Evaluation Drive Development)。 加入一些要点，比如使用的评估工具（Braintrust） 以及行研经验，真实数据 + 一些工程经验，比如回归测试，版本管理，数据回流训练来润色答案。

答案

我会**先建立一套版本化的真实评估数据集**，覆盖正常交易、欺诈交易、边界案例和历史误判案例，并定义 Recall、Precision、误报率、输出格式正确率、延迟和成本等指标。

在 Prompt 管理方面，**可以使用 Braintrust 这类评估平台**，对 Prompt 进行版本管理，并在同一套数据集上对不同 Prompt Variant 运行实验。每次修改 Prompt 后都进行回归测试，确认指标提升并且没有引入新的问题，再进入灰度发布。

在模型管理方面，不同模型也要使用相同的 Prompt、工具和数据集进行评估。如果是自己微调的模型，可以将线上经过人工确认的误判案例和新型欺诈案例回流，经过清洗、去重和标注后加入训练集，再完成重新训练、离线评估和灰度上线。

因为模型和 Prompt 是两个变量，我会先固定模型优化 Prompt，再固定较好的 Prompt 比较不同模型，最后对少量候选组合进行联合评估，避免两个变量同时变化后无法判断效果来源。

需要注意的是，线上数据必须经过人工确认，不能直接使用模型输出重新训练；Prompt、模型、工具和阈值需要一起记录版本；新版本上线前需要运行固定回归集，并通过影子测试或灰度发布控制风险。同时，将线上失败案例持续补充到评估数据集，形成迭代闭环。

一句话概括：**通过真实数据集统一评估 Prompt 和模型，用实验结果指导迭代，用线上反馈补充数据集，再通过回归测试和灰度发布控制上线风险。**

7. **评估指标与提升路径**：在欺诈场景中有一个混淆矩阵，你有 **Recall、Precision、Accuracy、Specificity** 四个指标。

-先抛开模型变量，假设在 prompt 工程下你的 Recall 是 **85%**，你想通过提示工程把它从 **85% 提到 86%**，你会怎么设计这个迭代路径?

思路

这个题的思路，就是**上面第六题思路展开。**重点讲**一下EDD的过程是如何真实运用的。**里面的关键点有：**控制变量，EDD，寻找失败因素。 这样的思路**，和我们笔记里的算法项目思路很像，整个算法项目的思路就是先有一套标准作为基准，然后开始训练，根据结果分析失败因素，修改数据集，训练方法，重新训练。 当然这里另一个思路，也可以聊一聊提示词工程的技巧，这个就是如果面试官深入问，具体怎么修改提示词，可以往提示词工程技巧比如ChainofThought来聊一聊。

答案

我会采用**提示词工程结合 EDD的方式进行迭代。**提示词工程负责修改 Prompt，EDD 负责验证修改是否真正有效。

首先**固定模型、工具、评分规则和评估数据集**，建立 Recall 为 85% 的基线。然后重点分析 False Negative，也就是被错误放行的欺诈交易，并按照原因分类，例如证据没有查全、异常信号被忽略、工具调用不正确，或者模型过早结束推理。

针对占比较高的失败类型修改 Prompt。例如，如果模型经常忽略设备异常，就在 Prompt 中明确要求检查设备关联；如果模型在证据不足时提前结束，就增加证据充分性检查和停止条件。（这里也可以引入一些提示词工程的技巧举例，提示词工程我认为你把这个视频看了就够了，考得不多）

修改提示词时，保证其他变量不变，只改变提示词，然后在开发集上运行评估，比较 Recall 是否提升。同时还要观察 Precision、误报率、延迟和成本，不能为了提高 Recall 而大量误伤正常用户。

修改有效后，再通过独立测试集进行回归验证，避免只对开发数据集过拟合。最后经过影子测试或小流量灰度发布，确认线上指标稳定后再全量上线。新的漏报案例继续补充到评估数据集，进入下一轮迭代。

整体流程可以概括为：**建立基线 → 分析漏报 → 分类错误 → 修改 Prompt → 离线评估 → 回归测试 → 灰度上线 → 数据回流。**

一句话概括：**提示词工程负责针对问题修改 Prompt，EDD 负责通过固定数据集和指标验证修改是否真正将 Recall 从 85% 提升到 86%，同时保证其他指标没有明显退化。**

8. **输出稳定性**：在概率生成模型下，同一个问题两次输出的结构（结果）可能不一致，怎么办？

思路

大模型是概率生成模型，同一个输入产生不同的表达或推理过程是正常现象。**因此这道题的核心不是让模型输出绝对稳定，而是通过工程手段保证最终业务结果稳定。**答案组织逻辑是先说这个核心理念，然后引入一些工程做法，保证业务结果稳定的。其中一些参考笔记章节有：LLM的调用参数。

答案

大模型的输出确实可能不稳定，但业务系统必须稳定。通常可以采用以下手段：

1. 使用固定的 JSON Schema 或结构化输出，统一风险等级、置信度、证据等字段，避免自由文本直接进入下游。
2. 对输出进行格式和字段校验，不符合要求时自动重试，仍然失败则降级到规则系统或人工复审。
3. 使用明确的 Prompt、固定上下文并降低 Temperature，尽量减少不必要的随机性。
4. 将最终风险分和业务决策放在模型之外。子 Agent 只输出风险信号和证据，由确定性的评分融合层计算分数，再通过固定阈值决定放行、加验、拒绝或转人工。
5. 对同一笔交易通过交易 ID 实现幂等和结果缓存，避免重复请求产生不同业务结论。

所以，即使模型两次生成的文字或推理过程不同，只要结构化风险信号一致，最终进入相同的评分和决策逻辑，业务结果就是稳定的。

一句话概括：**模型输出不一定稳定，但要通过结构化输出和确定性业务逻辑保证最终业务结果稳定。**

9. **模型升级与稳定性**：假设 prompt 一样，通过升级模型版本来优化效果。但当**旧模型下线后**，你怎么保证整个 Agent 的稳定性?

思路

这个问题可以从两个方面回答：

第一，**在系统设计和上线阶段提前考虑模型下线、限流和不可用等风险，准备备用模型和降级策略**；

第二，**在运行阶段建立完整的可观测体系，通过时延、轨迹、性能和生成质量等指标实时判断系统是否稳定，并在出现异常时及时处置。**

答案

我会从**容灾降级和运行监控两个方面**保证 Agent 的稳定性。

第一，**在系统设计阶段不能与某一个模型强绑定。**我会设计统一的模型调用接口，将 Prompt、模型参数和输出格式封装在适配层中，并准备主模型、备用模型和降级方案。当主模型下线、限流或调用失败时，可以切换到备用模型；如果备用模型也不可用，则降级到规则引擎、传统风控模型或人工复审，保证核心业务仍然可用。

模型切换前，还需要使用固定评估集进行回归测试，并通过影子测试或灰度发布观察新旧模型的差异。确认效果、时延和成本符合要求后，再逐步扩大流量，出现问题时能够快速回滚。

第二，**在运行阶段需要建立完整的可观测体系。**监控内容包括调用成功率、错误率、响应时延、Token 和成本、工具调用轨迹、结构化输出成功率，以及 Recall、Precision、误报率等生成质量和业务指标。

当错误率、时延或质量指标超过阈值时，系统需要自动告警，并通过重试、熔断、切换备用模型或转人工等方式进行处理。

一句话概括：**上线前通过模型解耦、备用模型和降级策略保证系统可用，上线后通过完整的监控指标、告警和自动切换机制持续保证系统稳定。**

视频讲解

https://www.bilibili.com/video/BV1d4Kr64EFx/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

2026.02 英伟达大模型Agent开发

前言

这一次面试，是来自于我的好朋友，**微软的资深工程师的面经**，自身不是我自己乱定义的，是因为微软的职级体系里，63\~64等级的title就是senior engineer, 翻译过来就是资深工程师。对应到国内职级体系，对应的P7或者P7+。我想说这些的原因，是告诉大家这一次面试的素材，是一个社招对应比较资深的岗位对应的面试内容。之前我们更多解析的面经主要是校招 以及我自己社招p6左右的社招面试记录。

我之前有说p6以下的社招，以及校招，面试的内容其实差不多，可以互相参考的。从这次解析来看，**p7的岗位也没有和我们之前的解析的那些面经差很多**，甚至可以说很多内容是差不多的。我来总体点评一下这次面试内容： **vibecoding + SKILL ， Agent健壮性设计， Rag的优化策略，微调遇到的问题和解决方案，AgentTeam, Agent架构设计，Agent框架： 这几块校招也问，可以通用参考。**

另外一部分是本次面试问了好几个项目上线的问题，这块的我的建议是：

校招同学甚至级别低一点的同学不用强求项目上线，大概了解下项目上线的思路就好了。

对于社招，或者想要包装自己项目说已经上线了的同学，这一块的内容：项目上线后会问什么问题，如何包装到自己的项目中？这里蕴含的思路是需要你关注的。

本次面试最值得我们关注的三个问题，Agent框架， Agent健壮性，项目上线问题。我也都沉淀成专题笔记了。

一面

为什么看机会?

让我做自我介绍，重点介绍和大模型相关的项目。

对方先挑了一段冷门的经历，然后又问了我的核心项目，主要问这些项目的技术挑战。

问了我在这些核心项目中的工作职责，分别有多少比例写代码、架构设计、团队分工。

问了项目 RAG 里的高级优化策略，以及当时做了哪些技术选型。

💡 **思路**

这是一个自由发挥的题，面试官充分给到你，让你介绍你的RAG的策略，我下面罗列了一些RAG的技术，每个点都是我们可以讲的。回答的时候有高级的特性优先讲更高级的，讲解时最好能够更具体，结合自己的项目，讲出细节，为什么选这个技术，解决了什么问题。这些都是一些比较成熟的技术，基本系统讲解RAG的书籍，教程都会讲到。我这里给到两个可以直接阅读的资料，覆盖了这些关键技术，供大家参考：

- All-in-RAG：RAG 技术全栈指南：中文在线教程，系统覆盖 RAG 的基础概念、数据处理、索引、检索、生成和评估，适合先建立完整的基础知识框架。
- RAG Techniques：高级 RAG 技术教程集：补充 All-in-RAG 中覆盖较少的高级技术资料，例如 HyDE、Query Decomposition、融合检索、Reranking 和 Corrective/Adaptive RAG。

一、偏基础的 RAG 特性

1. **文档解析与清洗：** 将 PDF、网页和表格等数据转换为干净文本，并尽量保留标题、段落、页码和表格结构。
2. **Chunk 切分：** 把长文档切成适合检索的文本块，在检索精度、上下文完整性和 Token 成本之间取得平衡。
3. **Metadata（元数据）：** 为文本块附加来源、页码、时间、版本、地区和权限等结构化信息。
4. **Embedding（文本嵌入）：** 将问题和文档编码成语义向量，使系统能够通过向量相似度检索相关内容。
5. **向量数据库与 ANN：** 使用向量数据库及 HNSW 等近似最近邻算法，高效存储和检索大规模文档向量。
6. **Dense Retrieval（语义检索）：** 根据向量语义相似度召回内容，适合处理同义表达和口语化问题。
7. **BM25（关键词检索）：** 根据关键词匹配程度召回内容，特别适合产品名、专业术语和规则编号。
8. **分层评估：** 分别使用 Recall@K、MRR、Faithfulness、P95 延迟和成本等指标评估检索、生成与工程效果。

偏高级的 RAG 特性

1. **Query Rewrite（查询改写）：** 把口语化、带指代或依赖对话历史的问题改写成语义完整、适合独立检索的问题。
2. **Multi-Query（多查询检索）：** 将同一个问题改写成多个不同表达并分别检索，降低单一措辞造成的漏召回。
3. **Query Decomposition（问题分解）：** 把包含多个条件或多跳关系的复杂问题拆成多个子问题，分别检索后再聚合答案。
4. **HyDE（假设文档嵌入）：** 先生成一段假设答案或假设文档，再使用它的向量检索表达方式更接近的真实文档。
5. **Parent-Child / Sentence Window：** 使用小文本块精准召回，再补充父级文本或前后句，为生成模型提供更完整的上下文。
6. **Hybrid Retrieval（混合检索）：** 组合 Dense 和 BM25，兼顾语义理解能力与关键词精确匹配能力。
7. **RRF（多路结果融合）：** 根据文档在不同检索结果中的排名融合多路候选，避免直接比较不同检索器的原始分数。
8. **Reranker（重排模型）：** 对第一阶段召回的候选文档进行更精细的相关性判断，把最有用的证据排到前面。
9. **Metadata Pre-filter / 权限过滤：** 在检索前按照业务条件和用户权限缩小搜索范围，同时提高准确率并防止数据越权。
10. **上下文控制与压缩：** 对召回内容进行去重、合并和压缩，并按照 Token 预算只保留最有价值的证据。
11. **Citation、拒答与生成后校验：** 要求答案提供引用，证据不足时拒答，并在返回用户前检查结论是否真正受到证据支持。
12. **Adaptive / Corrective RAG：** 根据问题难度和检索置信度动态选择简单或高级链路，质量不足时触发查询改写、重新检索或重新生成。

参考回答

> 我会从基础链路和高级优化两个层面介绍这个项目。基础链路包括文档解析与清洗、Chunk 切分、Metadata、Embedding、向量数据库、Dense 或 BM25 检索，以及检索和生成的分层评估。在此基础上，我们不是无条件堆叠高级组件，而是根据实际失败类型进行针对性优化。
>
> 例如，如果项目中存在口语化表达、多轮指代或复杂问题，就可以使用 Query Rewrite、Multi-Query、Query Decomposition 或 HyDE 改善检索输入；如果纯向量检索对专业术语和编号的召回不稳定，就可以采用 Dense 与 BM25 混合召回，通过 RRF 融合结果，再使用 Reranker 精排；如果小 Chunk 虽然定位准确但上下文不完整，则可以采用 Parent-Child 或 Sentence Window，在精准召回后补充完整上下文。
>
> 在生成阶段，我们会控制最终进入 Prompt 的证据数量和 Token，进行去重与压缩，并要求模型提供引用；证据不足时拒答，答案生成后还可以校验证据支持度和业务规则。对于复杂或低置信度问题，可以通过 Adaptive 或 Corrective RAG 触发查询改写、二次检索或重新生成，而简单问题继续走低成本链路。
>
> 技术选型时，我不会只说使用了哪些框架，而会说明每项技术解决了什么问题、比较过哪些替代方案，以及它对 Recall@K、MRR、Faithfulness、P95 延迟和单次成本产生了什么影响。最终目标是在回答质量、响应速度、成本、权限安全和维护复杂度之间取得平衡。

你为什么用了外部开源的项目（LangChain），而没用你们公司自研的知名项目（AutoGen）？

💡 **回答思路**

这个问题的回答就是要表示出来你的选型是有据可依的，是做了技术选型的。我专门重构了笔记2026 Agent开源框架部分，在这一块的内容解析了这个问题。

---

7. 系统是如何做评估的，尤其是端到端的全链路评估?

💡 **回答思路**

这个题目的理论基础，就是笔记Agent评估部分，详细讲解了Agent评估的策略。请先看这一部分，理解理论基础。
掌握理论基础以后，这个题目的回答，其实就是要把上面的理论基础，运用到自己的项目中，包装讲给面试官听。下面的内容是结合笔记里的Agent项目给出的参考答案，但实际上这个**Agent评估的理论基础部分** 和 **包装方法**是你更应该关注的。

📘 **详细回答**

我们对 SmartAppointment 的评估主要分成两部分：第一部分是 **组件级评估**，用来验证各个核心模块本身的能力；第二部分是 **端到端全链路评估**，用来验证这些模块组合起来之后，能否真正完成用户的预约或咨询任务。

组件级评估

组件级评估主要覆盖 **意图路由、预约信息抽取、工具调用和 RAG 问答** 四个核心环节。

意图路由主要检查用户请求能否被正确分发到预约 Agent、咨询 Agent 或者其他处理分支。预约信息抽取主要检查服务类型、预约时间、技师和用户偏好等槽位是否正确，以及多轮对话中已有状态能否被正确继承和修改。

工具调用主要评估 Agent 是否在正确的时机选择了正确的工具，工具参数是否完整、准确，以及工具返回结果之后，Agent 是否能够继续完成后续决策。RAG 部分则把检索和生成分开评估：检索侧检查相关知识是否能够被召回，生成侧检查回答是否相关、完整，并且是否受到检索证据的支持。

针对预约核心链路，我们构建了一套包含 **51 个场景**的评测集。每个场景包含对话历史、当前预约状态、用户本轮输入、可用工具，以及部分场景下的工具返回结果。它覆盖了信息收集、用户修改条件、指定技师、按条件推荐技师、技师不可用、没有匹配结果、用户确认和无关输入等情况。

评分时，结构化部分尽量使用确定性方法。例如检查 JSON 是否符合 Schema、槽位字段是否正确、状态更新是否正确，以及工具名称和参数是否符合预期；对于用户偏好和自然语言回复，则结合语义相似度和 LLM Judge 进行评分。同时，我们还记录平均延迟、P95 延迟和吞吐，用来综合比较不同模型方案在质量和性能上的差异。

端到端全链路评估

组件级评估通过以后，我们还会按照真实业务流程进行端到端回放。完整链路是：

> **用户输入 → 意图路由 → 预约或咨询 Agent → 知识检索或工具调用 → 状态更新 → 最终回复 → 业务落库**

端到端评测使用的不是简单的单轮问题，而是完整的多轮业务 Case。每个 Case 都包含初始状态、连续多轮用户输入、固定的知识库和数据库快照、模拟的工具返回结果，以及预期的最终业务结果。

评测运行时，我们从用户的第一轮输入开始，让请求完整经过 TaskClassificationAgent、对应的专项 Agent、知识检索或技师查询工具，直到系统完成任务或者达到最大执行步数。通过固定数据库快照和 Mock 外部依赖，保证同一个 Case 可以重复执行，不会因为技师排班或外部服务变化而影响评测结果。

全链路评分主要看三个方面。

第一是 **最终任务结果**，例如用户是否成功完成预约，服务、时间和技师是否符合要求，预约记录是否正确写入数据库；咨询场景则检查用户的问题是否得到了有效回答。

第二是 **中间执行轨迹**，包括意图路由是否正确、预约状态是否正确继承、工具调用时机是否合理、工具名称和参数是否正确，以及工具返回结果是否被正确使用。

第三是 **最终回答质量**，主要检查回答是否相关、完整，是否与知识库和工具返回结果一致，有没有出现幻觉、错误承诺或者不合理拒答。

最终我们会分别统计 **意图路由准确率、槽位抽取正确率、工具调用正确率、端到端任务完成率、回答忠实度以及 P95 延迟**。每次调整模型、Prompt、知识库或者 Agent 流程以后，都会重新运行这套场景集并与上一版本比较。如果高风险场景失败，或者任务完成率和关键指标明显退化，就不会进入后续发布流程。

所以，我们对 Agent 端到端评估的理解不是只判断最终回答好不好，而是同时判断：

> **Agent 是否选择了正确的执行路径，是否正确使用了状态和工具，并最终完成了真实的业务任务。**

⚡ 简短回答

我们对 SmartAppointment 的评估主要分成两部分：**组件级评估**和**端到端全链路评估。**

组件级评估主要覆盖意图路由、预约槽位抽取、工具调用和 RAG 问答。我们针对预约核心链路构建了 51 个多轮场景，检查 JSON 协议、槽位字段、状态更新、工具选择和工具参数；RAG 则分别评估检索召回和回答忠实度。同时记录平均延迟、P95 延迟和吞吐。

端到端评估则按照真实业务链路进行完整回放，从用户输入开始，依次经过意图分类、预约或咨询 Agent、知识检索或技师查询、状态更新、最终回复和业务落库。

评分时不仅看最终回答，还会检查 **最终任务是否完成、中间执行轨迹是否正确，以及回答是否与知识库和工具结果一致。**核心指标包括意图路由准确率、槽位抽取正确率、工具调用正确率、端到端任务完成率、回答忠实度和 P95 延迟。

所以，我们评估的不是某一次模型输出，而是整个 Agent 是否通过正确的执行过程完成了业务任务。

用到的评估框架 Ragas 和 Langfuse 的使用过程细节。

💡 回答思路

这个问题给我们的启示是你写在简历上的技术细节和技术栈，要自己深入理解，经得住追问。这个问题和面试的同学的个人项目具体技术细节强相关，通用性不大，因为不做展开讲解了。

如果线上出现了回答质量问题，是如何及时发现的?

💡 回答思路

这个题目的理论基础，还是笔记Agent评估部分，在Agent评估阶段，我们讲了Agent评估的全生命周期，最后一个生命周期就是线上监控，我们先需要理解这个理论基础。
这个问题是对于具体线上监控问题更加细节的询问，首先我们要知道一个概念：线下端对端的评估，我们可以准备评估集，跑预制的评估集打分，线上没法准备出数据集，线上监控的通常思路是：
建立检测手段（全链路日志，日志分析规则，指标聚合方法） + 告警
有了思路以后，然后要结合这个思路，把这个思路融合到你自己的项目之中。以下答案也是以笔记里的Agent项目，假装它上线了，来组织一个参考答案。

📘 详细回答

线上质量监控主要分为四部分：**全链路 Trace、确定性规则检查、分层抽样评估和聚合趋势告警。**

记录全链路 Trace

首先，我们会为每次请求生成统一的 Trace ID，通过 Langfuse 把整条执行链路串联起来，包括用户输入、意图分类结果、进入了哪个 Agent、预约状态如何变化、检索到了哪些内容、调用了什么工具、工具参数和返回结果、最终回答、Token 消耗、执行时间，以及是否触发异常降级。

这样发现质量问题后，我们不只是看到最终回答有问题，还可以沿着 Trace 判断问题出在 **意图路由、槽位提取、知识检索、工具调用还是最终生成。**

对确定性问题进行全量检查

对于能够通过程序明确判断的问题，我们会进行全量实时监控。

例如，预约信息是否解析成合法 JSON，Agent 是否反复询问已经收集过的字段，预约状态是否发生非法跳转，工具调用是否失败或超时，工具参数是否缺失，以及工具返回“技师不可用”之后，最终回答是否错误地告诉用户预约成功。

我们还会检查最终预约记录是否真正写入数据库，以及 Agent 是否出现重复调用、无效循环、执行步骤过多或频繁触发兜底回复。

这些信号一旦超过设定条件，可以直接标记为异常；对于错误预约、状态不一致等高风险问题，可以实时告警。

对回答质量进行分层抽样

对于相关性、完整性和忠实度等自然语言质量问题，我们采用分层抽样评估。

普通请求按照一定比例随机抽样；对于工具调用失败、检索相关度低、执行步骤异常、反复追问或者触发兜底的请求，会提高抽样优先级。

抽样后，把用户问题、对话历史、检索内容、工具结果、Agent 执行轨迹和最终回答一起交给 LLM Judge，重点判断回答是否解决了用户问题，是否受到知识库或工具结果支持，以及是否存在幻觉、错误承诺或不合理拒答。

聚合指标并触发告警

单次评估结果可能存在噪声，所以系统级质量告警主要根据一段时间内的聚合趋势触发。

我们重点监控 **任务完成率、JSON 解析失败率、工具调用失败率、兜底回复触发率、重复提问率、用户负反馈率和抽样评估低分率**，并按照意图类型、Agent 类型、模型版本、Prompt 版本和工具类型进行切片分析。

告警时不会只依赖一个弱指标，而是采用多信号联合判断。例如，新版本上线后，如果预约任务完成率下降，同时工具调用失败率、用户重复提问率和 Judge 低分率都明显上升，就会触发质量告警。

告警信息中会包含异常指标、影响范围、模型和 Prompt 版本，以及典型失败 Trace。经过人工确认的失败案例会进行脱敏和分类，再加入离线回归评测集，避免同类问题在后续版本中重复出现。

因此，整个线上监控体系可以概括为：

> **通过全链路 Trace 还原执行过程，通过确定性规则实时发现明确错误，通过抽样评估发现语义质量问题，再通过聚合趋势判断是否发生系统性退化。**

⚡ 简短回答

线上质量监控主要分为 **全链路 Trace、规则检查、抽样评估和趋势告警** 四部分。

首先，我们通过 Langfuse 记录每次请求的完整 Trace，包括意图路由、状态变化、知识检索、工具调用、最终回答、耗时和异常信息，方便发现问题后快速定位具体环节。

其次，对于 JSON 解析失败、工具调用失败、重复追问、状态异常、无效循环，以及工具结果和最终回答矛盾等确定性问题，会进行全量实时检查。

对于回答的相关性、完整性和忠实度，则对普通请求随机抽样，对工具失败、检索分数低或执行轨迹异常的请求重点抽样，再结合 LLM Judge 做质量判断。

最后，我们会聚合任务完成率、工具失败率、兜底率、重复提问率和 Judge 低分率，并与历史基线和上一个版本比较。多个指标同时异常时触发告警，确认后的失败样本再加入离线回归集，形成线上监控和离线评估的闭环。

问了业务指标，以及对业务指标效果的衡量方式。

💡 回答思路

这个问题其实是上面那个问题的追问问题，上面这个问题我们回答了如何监测线上数据。这个小节面试官问的是，你具体线上检测的时候，你的业务指标是什么？具体来讲：你用哪些指标来定义的业务是否成功，比如哪些指标反映了你项目的影响（带来了多少收入，用户喜爱度多少，日活有没有增加？）以及这些指标是怎么被衡量的？（通过代码里记录的哪些指标聚合起来的？）。我们更重要的是掌握这里的思路，然后如果你的项目上线了，请好好思考（包装）这个问题。
通常来说，这个指标的设计是需要精心考虑的，很多地方不光是要从技术角度分析，还需要产品的定义，从商业价值的角度去定义，真实的上线项目都是**要精心设计，思考，讨论猜得到的。**所以你说你的项目上线了，这里请你也好好设计一下。

📘 详细回答

SmartAppointment 的主要目标，一方面是帮助用户更顺畅地完成预约，另一方面是减少门店前台在咨询、技师推荐和预约确认上的重复工作。因此，我们把业务指标分成四类：**预约效果、服务效率、用户体验和经营价值。**

预约效果

最核心的业务指标是 **预约转化率**，也就是进入预约流程的用户中，最终成功创建预约的比例。

这里我们不会把 Agent 回复“预约成功”直接算作成功，而是以数据库中生成有效预约记录作为最终口径。完整漏斗是：

> **进入预约流程 → 提供预约信息 → 查询到可用技师和时间 → 用户确认 → 预约成功落库**

我们会分别统计每一步的转化率和流失率。这样如果整体预约转化率下降，可以进一步判断用户主要流失在信息收集、技师匹配，还是最终确认阶段。

除预约转化率外，我们还会关注 **预约成功率和预约放弃率**。预约成功率主要反映用户确认后，系统是否真正完成了预约；预约放弃率则反映用户进入预约流程后，是否因为反复追问、没有合适技师或者响应过慢而中途退出。

服务效率

第二类是 **服务效率指标**，主要衡量 Agent 是否减少了用户和前台的操作成本。

我们重点看 **平均预约时长、平均对话轮数、自动化解决率和转人工率**。

平均预约时长是从用户表达预约意图开始，到预约成功落库之间的时间；平均对话轮数反映用户需要经过多少轮交互才能完成预约。自动化解决率表示有多少咨询和预约请求能够由 Agent 独立完成，不需要人工介入；转人工率则反映 Agent 无法处理或者用户主动要求人工服务的比例。

如果系统优化有效，正常情况下应该看到平均预约时长和对话轮数下降，自动化解决率提高，同时转人工率下降。这样才能证明系统不只是能够回答问题，而是真正提升了预约效率并减少了前台工作量。

用户体验

第三类是 **用户体验指标**，主要关注用户是否认可整个交互过程。

这一部分可以通过 **用户满意度、负面反馈率、重复提问率和预约取消率**来衡量。

用户满意度可以来自点赞、点踩或者预约后的简单评分；负面反馈率包括用户投诉、纠正系统和主动转人工等行为；重复提问率主要用来判断用户是否因为系统没有理解需求，而换一种方式再次表达相同问题。

预约取消率也需要关注。如果预约转化率提高了，但后续取消率也明显上升，可能说明系统为了促成预约，推荐了不合适的时间、服务或技师。因此不能只看预约数量，还要看预约质量。

经营价值

第四类是 **门店经营指标**，主要看 Agent 是否给门店带来实际价值。

这里可以关注 **人工接待量、单次预约处理成本、技师时段利用率和用户复约率**。

人工接待量和单次预约处理成本用于判断 Agent 是否减少了前台重复工作；技师时段利用率用于判断系统是否通过推荐可用技师和时间，减少了空闲时段；用户复约率则可以衡量用户行为分析和回访提醒是否带来了更多再次预约。

其中，技师时段利用率和复约率受到营销活动、季节和门店客流等因素影响比较大，所以我们不会简单地把全部变化都归功于 Agent，而是会结合相同门店、相似时间段和对照流量进行分析。

如何衡量业务效果

业务效果主要通过 **上线前后对比和灰度 A/B 测试**进行验证。

上线前，我们会先统计人工预约流程的历史基线，包括预约转化率、平均处理时长、转人工率和用户放弃率。上线后，先选择部分门店或者一部分用户流量进行灰度，同时保留原有流程作为对照组。

实验组使用 SmartAppointment，对照组继续使用原来的咨询和预约流程。然后在相同时间窗口内比较两组的预约转化率、平均预约时长、自动化解决率、转人工率和用户满意度。

分析时还会按照新老用户、预约类型、门店、时间段和问题复杂度进行切片，避免因为不同流量结构造成误判。同时设置 **错误预约率、预约取消率、投诉率、P95 延迟和系统异常率**作为护栏指标。

只有当预约转化率和自动化解决率提高，平均预约时长与转人工率下降，并且错误预约率、取消率和投诉率没有恶化时，我们才认为系统真正带来了业务价值。

所以，整个业务指标体系可以概括为：

> **以预约转化率作为核心结果指标，以预约时长、自动化解决率和转人工率衡量效率，以满意度和取消率衡量体验，并通过 A/B 测试和护栏指标验证效果。**

⚡ 简短回答

SmartAppointment 的业务目标主要是 **提高预约转化率，同时降低门店人工接待成本**，所以我们主要从四个方面衡量。

第一是 **预约效果**，核心看预约转化率、预约成功率和用户放弃率。预约成功不以模型回复为准，而是以数据库中是否真正生成有效预约记录为准，同时会分析用户在信息收集、技师匹配和最终确认各阶段的漏斗转化。

第二是 **服务效率**，主要看平均预约时长、平均对话轮数、自动化解决率和转人工率，用来衡量 Agent 是否真正减少了用户操作和前台工作量。

第三是 **用户体验**，包括满意度、负面反馈率、重复提问率和预约取消率。尤其要避免只提高预约数量，却因为推荐不合适导致取消率上升。

第四是 **经营价值**，主要关注人工处理成本、技师时段利用率和用户复约率。

效果验证上，我们会通过上线前后的历史基线和灰度 A/B 测试比较新旧流程，同时把错误预约率、取消率、投诉率和 P95 延迟作为护栏指标。只有预约转化率和自动化解决率提升，预约时长和转人工率下降，同时护栏指标没有恶化，才能说明系统真正产生了业务价值。

问了项目里 Agent 的核心设计。

Agent 遇到异常故障时，用什么方式保持持续健壮的运行?

💡 回答思路

这个问题也是专门总结了专题笔记：Agent的健壮性设计。这个考点是校招和社招都爱考察的点。这里就不展开，请直接参考对应专题笔记。

最近很火的 Agent teams 是怎么看的?

💡 回答思路

这个问题的灵感来源来自于：ClaudeCode的SubAgent章节 和 Multi-agent架构。前者讲了单agent-subagent-agentteam的区别和使用场景，Multi-agent架构则是从设计Agent的架构选型出发，讲解。综合这里的笔记内容，这个题的回答角度是：不盲目用Agentteam，说出根据不同场景选择合适的Agent架构的思路，讲清楚盲目选择多Agent的弊端。

💬 第一问：怎么看 Agent Teams?

我的观点是 **不盲目使用 Agent Team，优先选择能解决问题的最简单架构。**Agent Team 相比 Subagent 多了一层“协调税”：每个 teammate 都维护独立上下文，token 成本更高；成员间通信、等待和结果汇总会增加延迟；同时还可能出现重复劳动、状态不一致和代码冲突。

但在复杂任务中它有明显价值，比如任务能够拆成多个相对独立的部分并行执行，而且成员之间需要共享发现、互相质疑或动态接力，例如大型代码库分析、多模块开发、并行研究和多视角评审。

我的选型策略是：**简单任务用单 Agent；任务可并行但子任务只需把结果返回主 Agent，用 Subagent；只有任务可拆分、可并行，并且成员之间确实需要横向通信和自协调时，才使用 Agent Team。** 最终看并行带来的质量和时间收益，能否覆盖 token、延迟和协调成本。

如果你来做系统设计，会如何保证 Agent 之间合理的分工协作?

给一个思路，这个可以转换成Agent架构的选择上，来谈Agent的选型策略，什么时候选多Agent/单Agent。从我会精心考虑任务类型，不盲目选择多Agent，即使选择，也精心设计它的多Agent架构，从这个角度出发，就变成了我们背烂了的八股问题：MultiAgent的选型策略。

聊了现在 vibe coding 的使用习惯和常用 Skills。

💡 回答思路

这个问题有太多出发的角度了。
比如从节省上下文开销的习惯：可以参考如何节约Token专题笔记。常用的Skill可以说这里的hand-off skill
从组织上下文的角度：可以谈谈如何组织Agent.md文件，可以参考我们的Harness章节笔记。
或者可以从这个上面，使用ClaudeCode的单workflow，Agent，subagent, agentteam的角度也可以去聊。

项目中为什么需要 LoRA 微调？效果怎么样?

💡 回答思路

这两个题目，如果做过算法项目，就很好回答。

我们的算法项目做了微调，为什么做（要提升什么内容）：是我们项目dev_spec文档写得很清楚的？，

效果怎么样：就是我们每一步都有评估标准，衡量效果。

最后遇到了什么问题，是我们项目报告里着重记录的：我的项目遇到了什么问题，如何解决提升的。

所以问题答案我就不展开讲解了，直接看我们算法项目，总结的都有。

给大家的启发也是，自己做算法项目的时候，这三个问题请你特别关注，注意记录。

LoRA 微调时遇到过什么问题吗?

讲解视频

https://www.bilibili.com/video/BV14YbR6gErK/?spm_id_from=333.1387.homepage.video_card.click

---

校招

暑期实习

2026.05 字节大模型开发暑期实习一面

岗位介绍

今天的这一期视频，是会员朋友提供的，之前她找我咨询，我给了她一些去找暑期实习的建议。她也是用了我们RAG项目，估计和自己的Agent项目有融合。具体的岗位信息我就没有太问了，但是我知道的是面试的这个岗位一定是 Agent开发非算法岗，这个同学本身没有太多的算法的基础，因为她大概在4月初才开始准备找实习，本身Agent基础也不多，去找暑期实习的时间点也比较晚，准备时间也不多。

我给她的建议是一定要规定一个时间节点，一定开始面试，能够压缩准备时间，但是不能压缩面试时间，本身4,5月份暑期实习的岗都不多了。一边面试一边总结，一定要争取有个实习，即使是小公司，因为实习经历对秋招绝对有帮助。然后从现在起一直准备。因为她本身学历也挺好，是985本硕，所以目标肯定是大厂，只要认真准备，从4月开始，秋招一定有大厂offer的。

回到面试本身来说，其实可以看出来，所有的内容，笔记里都有。很多内容其实不是说笔记里总结的原封不动的内容，但是学好了笔记里的内容，理解了，你都能够去回答。所以我们在答案解析的时候，其实可以看到几乎所有问题，答案都引用于我们的文档中的某一部分。 所以真的，多看看文档，这个文档还是很全的，而且都是面向于面试问题，不断总结补充的，学好理解透了文档，面试十拿九稳。

另一个感受是，在AI时代，2026年5月，还要手撕代码吗？即使现在很多企业考察Vibecoding了，但是肯定的一点是还有很多公司在手撕代码，字节就是其一，校招社招都是这样。我觉得没太大意义，但现实是这样。目前为止，题目还得刷。

最后谈一个反思，就是对于一个项目，光靠总结和背诵，是根本背不完的。大家可以看到也许今天的很多问题，不是我们总结的原题，但是很多地方，其实都和我们之前总结的相关问题，相关知识点有相关性。你必须去理解，比如项目部分，整个设计文档要理解，总结的面试真题要理解，自己遇到了问题都思考一下，结合自

己项目怎么表述，面试的时候自己多复盘，多查漏补缺，慢慢才能答得越来越好。尽管文档总结了很多，也越来越全，但是还是要靠大家自己去认真消化，体会的。

一面

1 多 Agent 协同遇到的问题?

> 因为是同学直接反馈给我的问题，所以这个问题其实就很模糊，缺少上下文，不知道面试官到底在问哪个方面，不过我们笔记中在多Agent部分，涉及了很多相关内容，可以从这些角度回答。总体而言，我们需要把笔记的第六部分 多Agent弄透了，里面推荐论文，书籍的某一章，把下面这些问题理解了，对待这个问题才能应对自如。

架构选型问题，可以说成：

我觉得多 Agent 协同首先遇到的是架构选型问题。不是所有任务都适合多 Agent，多 Agent 也不是一定比单 Agent 效果更好。如果任务本身是强顺序的、长链路推理的、工具调用很密集的，或者单 Agent 的 baseline 已经比较高，盲目拆成多个 Agent 反而会引入额外的协调成本、通信开销和误差传播。所以我在设计的时候，会先判断这个任务到底有没有并行分工的价值。比如哪些模块可以并行生成，哪些模块必须由一个中心 Agent 统一收敛，哪些环节继续用单 Agent 反而更稳。

通信成本问题，可以说成：

第二个问题是通信成本。这里的通信成本不只是上下文大小和 token 消耗，还包括时延成本、同步等待成本、调度成本和工程复杂度。多个 Agent 协作时，Agent 之间一定要传递信息，但传递多少、什么时候传、是否需要等待其他 Agent 返回，都要做权衡。如果每个 Agent 都把完整上下文、中间推理过程、工具调用结果全部共享，虽然协作信息更充分，但是 token 消耗会快速增加，上下文窗口也容易被撑爆。同时，如果 Agent 之间存在多轮通信、互相确认、等待汇总，整体链路延迟也会上升；如果某个 Agent 执行失败，还会引入重试、回滚和重新调度的成本。所以我会控制 Agent 之间的通信粒度和通信轮次，不是所有内容都广播，而是尽量传递结构化的中间结果、关键证据和最终结论。对于实时性要求比较高的场景，还要限制多 Agent 之间的交互轮数，避免协同本身吞掉系统收益。

状态管理问题，可以说成：

第三个问题是状态管理。多 Agent 系统里，每个 Agent 都会有自己的上下文、任务进度和中间状态，如果没有设计好共享状态和私有状态，系统很容易变得混乱。比如一个 Agent 的中间推理其实不一定需要暴露给其他 Agent，全部暴露反而会造成状态空间膨胀。我的理解是，可以把每个 Agent 的私有推理过程保留在自己的上下文里，只把对外有用的结果写入共享区。这样既能保证协作，又能控制上下文复杂度。

错误传播问题，可以说成：

第四个问题是错误传播和错误放大。多个 Agent 并行工作时，每个 Agent 都可能产生局部错误。如果最后只是简单地把结果拼接、投票或者聚合，错误可能会被带入最终答案，甚至在聚合过程中被放大。比如独立多体架构中，多个 Agent 互相之间缺少纠错机制，最后综合时就可能把多个错误叠加在一起。所以多 Agent 系统

通常需要设计质量门控，比如中心 Orchestrator、Reviewer Agent、规则校验或者一致性检查，对子 Agent 的输出进行裁决和收敛。

控制权交接问题，可以说成：

第五个问题是控制权怎么交接。Agent 之间协作常见有两种方式，一种是 Handoff，也就是一个 Agent 把上下文和执行权完整交给另一个 Agent；另一种是 Tool Call，也就是主管 Agent 把子任务委托给其他 Agent，但自己保留控制权，等子 Agent 返回结果后再决定下一步。Handoff 的自主性更强，但链路更难控制；Tool Call 的约束更清晰，但中心 Agent 的压力会更大。所以如果是在项目里，我会更倾向于用主管 Agent 分发任务，子 Agent 返回结构化结果，最后由主管 Agent 统一整合。

2 评估怎么做的?

> 评估这个问题，我一直说是必考的，我自己的项目没有做，在面试官问的时候，我就是编造的答案。具体大家看这里。
>
> 对于这一个问题，我的建议是，如果你的项目没有做，建议你看笔记的内容，理解Agent评估的方法，亲力亲为设计到项目中。如果已经做完了项目，但是这里没做，可以参考我的方法，理解理论编一下怎么做的评估，我笔记里也有示例。
>
> 当然我不太确定这里说的是Agent还是Rag。其实都不重要了，我以agent为例，讲解的。如果说的是RAG的话，其实你去看我们项目本身，RAG测评做了，相关总结也有，RAG的测评主要是RAGAS。

2-1 评估的各项指标怎么得到的？具体怎么做的?

参考 3.1 如何评价一个Agent的性能?

参考 3.3 你的项目中是如何对Agent进行性能评估的

2-2 这些指标说明了什么问题?

> 这个其实就是我们背的理论部分，3.1 如何评价性能，这里就讲解各个性能代表啥，3.1其实总结的都有。

首先，任务成功率 / 解决率 说明 Agent 最终有没有把用户任务完成。比如用户要求生成一份报告、完成一次信息整理、执行某个流程，最后结果如果符合预期，就算成功。这个指标能直接反映 Agent 的整体可用性。

第二，响应质量指标说明 Agent 最终输出是否满足用户需求。它不只是看有没有输出，而是看输出是否完整、准确、格式是否符合要求、是否覆盖了用户要求的关键点。如果这个指标低，说明 Agent 可能理解任务不充分，或者最终总结、生成能力不稳定。

第三，轨迹评估指标说明 Agent 的执行过程是否合理。Agent 项目和普通问答不一样，它通常会有规划、拆解、调用工具、汇总结果这些步骤。所以需要看它有没有按照预期路径执行，比如是否调用了正确工具、工具调用顺序是否合理、关键步骤有没有遗漏、是否出现了无效循环。如果轨迹指标差，说明 Agent 即使偶尔能得到正确结果，过程也不稳定，复杂任务下容易失败。

第四，工具调用成功率 / 参数正确率说明 Agent 使用外部工具的能力。Agent 很多能力不是模型自己完成的，而是依赖工具。如果工具选错、参数填错、调用失败，就说明 Agent 的 tool use 能力有问题，可能需要

优化工具描述、参数 schema、prompt 约束或者增加校验机制。

第五，延迟和 token 消耗说明 Agent 的工程效率和成本。多 Agent 或多步骤 Agent 往往会有多轮模型调用，如果延迟很高，说明用户体验不好；如果 token 消耗很高，说明成本不可控，也可能说明任务拆解太碎、上下文传递太冗余，或者多 Agent 之间有重复劳动。

所以这些指标不是孤立看的。任务成功率看最终结果，响应质量看输出质量，轨迹评估看执行过程，工具调用指标看 Agent 的行动能力，延迟和 token 看工程可用性。 通过这些指标，我可以定位问题到底出在任务理解、规划路径、工具调用、结果生成，还是系统成本和性能上。

3 如果把项目规模扩大，但返回质量想进一步提高，你要在哪些地方改进?

> 项目进一步扩大，可以从参考 3.5.5 如果该项目要真正落地投入生产，还需要做哪些工作
>
> 提高质量，可以从3.5.1 考虑性能优化了吗？项目中哪些地方可以优化？ 这里去讲。

4 精排为什么不用cross encoder，而是要做规则?

> 这个属于RAG部分的基础知识，RAG部分的精排，通常有基于LLM的精排和Cross Encoder的精排。这个同学用的就是我们笔记里的项目，我们关于这个项目的面试问题其实可以看出来，面试官很喜欢问一些方案的对比，为什么选择这个模型，为什么选llm精排不选择cross encoder,为什么选择openai的text embedding。其实你也很难总结出来所有的面试问题，但是应对的方法就是把项目的文档尽量都消化完全，想清楚各种细节，结合我们总结的面试问题多去思考复盘，查漏补缺。
>
> 回到这个问题本身，我们需要先知道精排是什么，常见精排方法有哪些，各自的优缺点是什么。这里注意，Cross Encoder 本身就是精排方法，所以不要拿它和粗排方法对比，而是要放在精排方法内部，和 LLM 精排、规则精排、混合精排这些方案对比。

请先参考理论知识：RAG的精排。

> 我不知道同学具体的项目的数据库是什么，我假设他这个RAG系统，内部的文档是技术文档。给出一个参考回答。值得一提的是，下面的答案其实不用背，更多的是上面精排的不同策略值得我们学习，理解背诵。针对这个具体问题，我们要现场结合我们这个精排的知识基础，现场发挥的。下面这个答案仅供参考。

如果面试的时候让我解释为什么当时做的是基于规则的精排，并且业务场景是内部技术文档，我会这样回答：

我们这个项目面向的是内部技术文档问答，比如接口文档、架构文档、排障手册、版本变更说明这些。召回阶段已经能把语义相关的候选片段找出来，所以精排阶段我更关注的不是 query 和 chunk 像不像，而是这段内容能不能真的支撑后续回答。内部技术文档里同一个服务名、接口名可能出现在很多地方，但是正式文档、旧版本文档、故障复盘、临时记录的可靠性是不一样的。

所以我当时做规则精排，主要是把这些业务质量标准显式加进去。比如命中服务名、接口名、配置项的内容优先；包含参数表、错误码、操作步骤、结论的内容优先；来自正式文档库、接口平台、最新版本目录的内容优先；旧版本、临时记录、内容过短、重复或者缺少上下文的片段降权。

举个例子，用户问某个服务的超时配置怎么改，召回结果里可能既有正式配置说明，也有故障复盘。Cross Encoder 可能会觉得故障复盘也很相关，因为里面有很多超时和服务名，但真正回答用户时，我更希望优先引用正式配置说明，因为它更权威，也更可执行。使用基于规则的精排方法，可解释性也更强。

所以不是 Cross Encoder 不好，而是这个场景下规则精排更能保证文档的权威性、时效性和可解释性，而且成本和延迟更低。后续如果有更完整的评估集，我会考虑做混合精排，先用规则过滤低质量片段，再用 Cross Encoder 或 LLM 做更细的语义重排。

5 你觉得怎么做能让召回内容尽量完整可用?

> 这个问题虽然没有总结一模一样的答案，但是其实我个人理解，整个RAG项目的，分块，粗排，精排，评估，迭代这个过程，也就是整个RAG系统的目的就是让召回的内容正确，完整，可用。所以如果理解了各个环节，比如分块，尽量减少语义粗排，精排会修复错误，合并语义，评估观测效果，及时调整策略，从这些方向去答就好。所以光背是不行的，必须理解消化他。

而是整个 RAG 链路很多环节其实都在保证召回完整可用。结合我们项目来说，我会从三个层面保证生成内容尽量完整可用。

第一是 知识进入系统时要完整。我们项目里不是简单按固定长度切 PDF，而是先把 PDF 转成 Markdown，再按标题、段落、列表、代码块这些结构做递归切分，并保留页码、标题层级、source_path 等 metadata。这样可以避免一个完整语义被切碎，后面也方便定位来源。对于图片或流程图，还会用 image caption 转成文本，让图里的信息也能被检索到。

第二是 检索时要尽量召回全、排得准。我们用了 Hybrid Search，也就是 Dense Embedding 加 BM25。Dense 解决语义相似问题，BM25 解决关键词、接口名、错误码这类精确匹配问题，然后用 RRF 融合。召回之后再通过 Rerank，比如 Cross-Encoder 或 LLM Rerank，把真正能支撑回答的 chunk 排到前面，减少无关内容进入上下文。

第三是 生成时要可追溯、可评估。我们的 MCP Server 返回结果时会带 source file、page、chunk id、score 这些 citation 信息，所以答案不是凭空生成，而是能看到依据。后面再通过 Dashboard 和 Ragas/DeepEval 这类评估，看 faithfulness、answer relevancy、context precision 等指标，发现答案不完整时再反查是分块、召回、精排还是 prompt 的问题。

总结来说，前面分块和增强保证知识完整，Hybrid Search 保证召回覆盖，Rerank 保证上下文质量，引用和评估保证答案可信、可迭代。这就是我们整个 RAG 系统设计的核心目标。

6 怎么减少幻觉?

> 总结这个问题的时候，我专门重构了笔记幻觉部分。幻觉问题围绕，幻觉是什么，为什么有幻觉，如何解决。这三个问题，我查阅了很多资料，力争对三个问题的解析是完整的。另外也参考了前言论文，增加了一个回答的亮点，进入最前沿的知识，让你回答与众不同。 这个视频的讲解，也会稍微讲解一下幻觉问题。看了笔记和视频，这两问题很好回答。

参考这里 大模型幻觉。 看懂了这个问题游刃有余。

6-1 你觉得大模型为什么有幻觉?

6-2 你觉得大模型幻觉具体的原因在哪里?

7 整个项目消耗的 token 量是多少?

> 其实我在总结CC的成本控制的时候有聊过这个问题。我们在做Agent的时候，最好能够设置一个Agent成本监控（Token使用量）统计。做自己项目的时候，比如使用Vibecoding，也可以大概关注一下自己用的什么模型，费了多少Token。我下一个Agent项目会注意这两点的。 至于已经做完了项目，也忽略了费了多少Token的同学，让AI帮你大概估一下吧，编造一下。

7-1 你用了哪个模型?

> 这个问题我已经总结过了，关于模型的选择。模型选择，我们整理了三段论去选择。这个回答怎么选模型，为什么选这个模型。 以及在这个章节，我就谈过，最好的方法还是做一些小型的实验，刚好印证了7-3这个问题。 对于所有没开始做项目的同学，我的建议是你参考笔记和思路，这么选择模型和实验。 对于所有项目已经完成了，或者来不及做项目的同学，那么就是按照这个思路去包装。提前按这个模板想好你的回答。

7-1 7-2 7-3 参考这里： 模型的选择

7-2 为什么选这个模型?

7-3 有具体的指标评估支撑你选择这个模型吗?

8 偏大模型 / 八股

> 八股也没啥好说的，而且问的都是概念，问AI都有。笔记里也有，我给大家找到参考地方。不做解析了。

8-1 对微调的理解，SFT 和 RLHF 有什么区别?

> 这里有个小经验。我看到同学给我反馈问题的时候，他说面试官问这个微调，他说理解得不深。不过我的建议是，不主动说不深入。面试官问你会不会，你就说会，大概率会继续问你概念，你就答。问到不会了再说。他自己会根据你回答情况判断你的深浅，不要主动坦白。

SFT

SFT 和 RLHF 有什么区别?

8-2 如果让你做 RLHF，你会怎么做?

8-3 对 skill 的了解? 对 harness 的了解?

SKILL

harness

8-4 如果让你用 harness 完成你的工程，你会怎么做?

> 8-4.8-5.8-6的那问题特别好。我们总结了很多Harness,ClaudeCode的原理，概念。但是其实这三个问题想问，这些理论如何实践，对你的工作有什么迁移。这个问题其实需要你真的有一些思考，理解，实践，我觉得很好的问题。如果理解浅尝辄止，可能这个问题不好回答。我马上会总结一个关于Claudecode,Harness 更加偏工程落地的文档，足够回答这三个问题，先占个位，那个问题总结好了，我来补上。

8-4 8-5 8-6 答案参考这里

8-5 对 Claude Code 的了解？里面有哪些框架你认为真正给你的使用带来了便利？

8-6 这些内容怎么迁移？

8-7 对记忆机制的理解？你都知道哪些记忆机制？

8-8 你认为什么样的记忆可以进行长期保留，什么记忆只在短期保留？

八股：Agent记忆

也可以参考我讲的各家开源项目，都会讲长短记忆，也是参考。

Claudecode记忆

Harness记忆

Openclaw记忆

9 传统八股

> 传统八股，不讲。传统八股是我们在没有Agent之前，高频考点。现在面试大模型这个岗位，它的地位其实就放在大模型知识之后了。面试可能零星有的面试官问你并发，锁，操作系统。你要去学深入，又很费功夫，它的出现频率又不高。 我觉得做法就是大概背概念吧，出现一个补一个，因为说实话确实也没时间深入了。所以如果是只背概念，大家这些问题百度,AI都有答案，我就不总结了。

9-1 了解计算机网络吗？

9-2 了解并发机制吗？

9-3 了解锁吗？

10 手撕代码

10-1 循环数组找最大子数组和。

10-2 ACM 格式。

视频讲解

https://www.bilibili.com/video/BV1Wu5f6GEr6/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2026.04 腾讯AI Agent暑期实习

点评

这个内容来自于互联网，所以具体岗位信息不太清楚。是小红书博主，一个在读研究生分享的暑期实习面试。

我来点评一下这一次面试：难度主要在两个方面。

1： Agent的有的问题不是那么好回答，比如：

> 如果用户和AI对话生成问题本身有冲突，系统应该怎么处理？
>
> 如果线上要做多轮对话，记忆写入策略如何定，才能避免越聊越脏？

需要你真的有一些Agent的经验，对于具体细节问题有一些理解。包括问题：

如何让你从0设计一个对话Agent的流程，核心状态有哪些?

答好他，也需要参考一个成熟的Agent的架构。把流程答的全且专业。

2. **广度上。** 这个面经考察了Agent，RAG, 微调，部署。可以说各个方面都考了。考的最多的是Agent和RAG。 除了Agent部分，其它部分不难，主要像八股。虽然考了微调，部署，但是你去看面试问题，其实是一些基本的理论知识，我们叫八股，大概了解，背背就可以回答。

我整理笔记的时候，把大量面试真题和上面的笔记内容合并归档。添加很多跳转链接，让大家更容易阅读，在学习笔记的时候，也能够实时用面试真题检验自己。

另外据这个博主反馈，是有手撕代码，他没放上来。

一面

1.如何让你从0设计一个对话Agent的流程，核心状态有哪些?

归档至 ClaudeCode 整体架构小节。通过学习这一小节，可以回答该问题，答案整理到该小节面试真题部分。

2. 如果用户和AI对话生成问题本身有冲突，系统应该怎么处理?

> 这个问题从两个方面来思考：**软冲突**（指令/规则层面，靠注意力机制处理）和**硬冲突**（工具调用/安全层面，不受任何提示词影响）。其实看这个问题，面试官应该问的是前者，但是我想多讲一点。 我们这里还是以ClaudeCode为例，毕竟这个产品肯定有这个问题，我们看他如何解决的就行了。
>
> **软冲突**：理解ClaudeCode的记忆模块。
>
> **硬冲突**：理解ClaudeCode的安全系统。
>
> 这两个笔记看了，就可以从两个角度回答。

我以两种冲突：软冲突：比如**用户偏好、项目规则、历史记忆之间不一致**和硬冲突：**比如用户让模型越权调用工具、删除危险文件、泄露隐私数据** 两种冲突来谈一下我的处理方法。

对于软冲突，我们可以允许指令发生偏移，所以对于不同地方写到的提示词，系统会把它**叠加而不是覆盖。**通常来说由于模型的注意力机制，后加载的指令会更容易被遵循。所以按照加载顺序会有一个优先级，但是毕竟

模型执行有概率性，也不能完全保证，但是我们接受这种行为。

对于硬冲突，我们要求结果确定。比如对于工具的调用，或者特别重要的规则，我们依赖于模型外部的Harness保证一定不会被用户的提示词影响。比如工具是否调用，通过代码约束，给工具分成：**可以运行，需要询问用户，拒绝运行。**通过外部Harness规则和用户提示词发生冲突时，以规则为准。

3. 如果把一个普通问答项目升级成虚拟对话系统，架构上要改哪些东西?

我觉得这个题好难解析。首先什么是虚拟对话系统，我没有上下文，是不是说对话是一个虚拟人？如果是这样，业界上好像也没有什么我能想到的产品，你像豆包，千问，也是一个对话系统啊。如果是说虚拟系统是虚拟人的话，我能想到的区别可能是输出形式，你需要把文字渲染成虚拟人，表示脸部特征，输出是有语音的。所以可以从输出的角度来谈一下区别？**这个就不做深入解析了。**问题不太明确，另外我也缺少这方面经验，其次面试出现频率还挺小众的。

4. RAG在这类Agent项目里到底用来解决什么问题?

> 这个回答的答案，第一段其实就是RAG的基本概念，我觉得大家都能回答。另外RAG也是解决幻觉的一个手段。可以参考笔记：大模型幻觉的本质
>
> 第二段，其实我加入了Harness的检索地方的思想，想引入一些亮点。

RAG 在 Agent 项目里本质上是给模型加一个**外部知识和检索工具层**，让 Agent 不只**依赖参数记忆，因为参数的训练依赖于具体数据集，一定会过时。而是能按任务从文档、代码、历史记录、业务库里拿到相关上下文再推理。它解决的是知识动态更新、上下文窗口有限、私有知识接入和模型幻觉问题。**

我想补充一点，其实现在不像25年那么依赖于RAG了，因为凭借模型的能力+关键字检索，有时能取得比RAG更好的效果，比如Boris(CC开发者）在写ClaudeCode的时候专门对比RAG VS 不用RAG，最后选择了后者。不过这也并不是完全不需要RAG，**需要根据具体场合来权衡。**有些时候需要引入语义搜索提高准确性，并且不依赖于模型推理就能找到结果，节约成本，这种情况下使用RAG仍然是很好的选择。

5. RAG的信息一般怎么存，为什么不能只扔向量库?

> 这个题其实做了相关项目，都能理解。如果你做了/学了笔记里的：RAG项目实战。我们项目的数据库存储，不光用了ChromaDB(也就是题目说的向量库），也用了sqlite存储图片，hash值，BM25索引。理解了项目，这个题很好回答。
>
> 我以我们的RAG项目为例展开。

RAG不能只扔向量库，原因是向量库主要解决的是语义相似度检索，但 RAG 工程里还需要解决很多别的问题：**原文追溯和结果展示问题， 相关图片，相关稀疏向量（因为有的向量数据库可能不支持）等其它问题。**以我的RAG系统为例，我们的数据库存了（下面这个回答其实在项目专题总结过，参考快手一面：你的Index怎么存的。

我们项目共有 5 块存储：

1. **文件指纹库**（SQLite）— 存文件 SHA256 和入库状态，负责幂等控制，防止同一文件重复入库。

2. **Dense 向量库**（ChromaDB，底层 SQLite + hnswlib）— 存原文 + 向量 + HNSW 图索引，负责语义相似度检索。
3. **BM25 倒排索引**（JSON 文件）— 存倒排表（term、tf、idf、doc_length），负责关键词匹配检索。
4. **图片资源库**（文件系统 + SQLite）— 存图片二进制 + 元数据索引，负责多模态图片检索。
5. **Trace 日志**（JSONL）— 存全链路 trace 事件，用于可观测性和 Dashboard。
6. Embedding模型在项目中应该怎么选，选错会出现什么现象?

> 这个其实就是八股了，没啥好讲的。有点像Agent，喜欢问你怎么选模型的，我们总结在了：模型选择策略
>
> RAG 问你如何选择Embedding模型。我总结在了RAG部分笔记这里。RAG 嵌入向量部分

答案直接参考笔记 RAG 嵌入向量

7. Embedding切Chunk应该怎么做，为什么固定长度切分经常不够?

> 7,8 是RAG的八股文，已经汇总到知识专题。我自己策略是背一背。

参考 RAG专题 4.5.1 Embedding切Chunk应该怎么做，为什么固定长度切分经常不够?

8. 硬切分在实际场景中有哪些弊端?

参考 RAG专题 4.5.2 硬切分在实际场景中有哪些弊端?

9. ReAct机制在生产环境里该怎么设置?

归档至 ClaudeCode 整体架构小节。通过学习这一小节，可以回答该问题，答案整理到该小节面试真题部分。

9.1 为什么不能只靠Prompt写几句Thought/Action?

> 这个题的意思是说，理论上你可以把React写成提示词，比如：
>
> Thought: ...
>
> Action: ...
>
> Observation: ...
>
> 让模型来完成React。但为什么要通过工程的手段，来写代码，写循环来实现React。

因为 Prompt 里的 Thought/Action 只是“文本约定”，不是工程保证。

如果只在 Prompt 里写“你要按照 Thought、Action、Observation 格式来”，会有几个问题：

第一，格式不稳定。模型可能漏写 Action、参数写错、把最终答案和工具调用混在一起，甚至在复杂上下文下忘记遵守格式。

第二，安全不可控。模型说要执行某个操作，不代表系统就应该执行。比如删文件、调用支付接口、访问敏感数据，都必须经过权限层和代码层校验，不能靠 Prompt 里一句“危险操作要小心”。

第三，循环不可控。Prompt 不能可靠限制模型调用多少次工具、失败后怎么重试、什么时候停止。如果没有外部 loop 控制，Agent 可能反复调用工具、陷入死循环，或者 token 被消耗完。

第四，工具结果需要结构化管理。生产里工具调用结果要写入消息历史、记录日志、统计耗时和成本、处理异常、做 tracing。这些都是系统能力，不是 Prompt 能解决的。

第五，可观测性和可评估性不足。线上要知道每一步为什么调用工具、调用了什么、成功还是失败、耗时多少、有没有越权风险。这需要 runtime 记录，而不是只看模型生成的一段文本。

> ❗总结提炼一下：一些原则我们可以体会一下，什么让AI做，什么不让AI做，而是写代码做。**AI 适合做大脑里的“推理层”**：理解、规划、选择、总结。所以让他来想要不要调用工具，工具怎么调用。让AI做的事，是有不确定性，你也要可以接受，那么就用AI来做。**对于要求确定性的事：**比如安全问题，权限问题，比如工具调用本身，比如输出格式，那么应该用代码来完成。
>
> 这个工程的实现React方法，就是体现了这些思想。

10 什么样的项目更适合继续做模型重训练，什么样的项目更适合只做RAG或规则增强?

> 也有点像是八股。我看RAG的书上，有一小节是微调 vs RAG.所以可以从这个角度回答。所以我也整理到RAG八股知识里：

参考 RAG专题 微调 vs RAG

11. 微调数据集一般从哪里来，怎么判断一批数据值不值得训练?

微调数据集来源：八股问题。参考微调章节： 数据集来源

12. 如果让你部署一个大模型服务器，真正要关心的不是能不能跑起来，而是什么?

> 这属于推理部署的东西了，不过其实这个问题一点也不难，所以只需要你了解部署大概是干什么工作的，了解一下他们的核心思想，就能回答出来。所以即使做应用开发，这个题也很好，可以作为了解部署大模型工作的一个素材。

如果让我部署一个大模型服务器，我不会只关注把模型加载起来。能跑只是第一步，真正重要的是它作为一个线上推理服务是否可靠。

1. **性能能不能扛住线上流量**
   重点看 QPS、并发数、首 token 延迟、整体生成速度、p95/p99 延迟。模型能启动不代表能服务用户，真正要看高并发、长上下文、多轮对话下会不会变慢、排队、超时。
2. **资源和成本是否可控**
   大模型很吃 GPU，所以要关注显存占用、GPU 利用率、KV cache、batch 策略、量化方案、模型大小和上下文长度。目标不是盲目用最大模型，而是在效果、延迟和成本之间找到平衡。
3. **服务是否稳定可靠**
   线上服务必须有超时、限流、熔断、降级、重试、健康检查和多副本部署。还要防止某些超长请求或异常

请求把 GPU 打满，导致整个服务不可用。

4. **是否可观测、可治理**
   要能监控请求耗时、排队时间、错误率、GPU 利用率、显存、tokens/s、OOM 次数和调用成本。还要有鉴权、日志脱敏、版本管理、灰度发布和回滚能力。否则出了问题，只知道“模型慢了”，但不知道问题在哪里。

所以我理解，大模型部署的重点不是“能不能启动”，而是在真实业务约束下，如何把模型变成一个稳定、可控、可监控、成本合理的线上推理服务。

13. 如果线上要做多轮对话，记忆写入策略如何定，才能避免越聊越脏？

> 这个就可以结合笔记的CC的 autodream机制嘛，或Hermes的自我沉淀机制，学习他们如何整理的。本题以CC的Autodream机制回答。这题也告诉了我们，我们笔记里对于开源项目解析的章节，是有必要好好学的。

线上多轮对话的记忆写入不能“用户说什么就存什么”，否则记忆会越聊越脏。我会把记忆分层：当前会话里的临时信息先放短期上下文，只有稳定的用户偏好、长期事实、明确决策、反复出现的需求，才进入持久记忆。

写入策略上可以参考 Claude Code 的 AutoDream 机制：每轮对话结束后先由后台任务做记忆提取，但不是立即无脑固化；系统定期触发一次类似“睡眠整理”的后台整合，把已有记忆做合并、去重、冲突修正和过期清理。比如同一个用户偏好有新旧两版，就保留更确定、更新的一版；临时情绪、一次性需求、已失效信息就 prune 掉。

所以核心是三点：谨慎写入、定期整理、持续清理。持久记忆只存低变化、高价值的信息；高变化内容，比如当前任务状态、临时约束、代码位置，不应该长期记忆，而应该实时检索或只放 session memory。这样才能避免多轮对话越积越乱。

14. 如果让你设计一套Agent评估体系，至少要覆盖哪些维度？

这题没啥好说的，笔记Agent性能评估，从理论到实践都讲了。

讲解视频

https://www.bilibili.com/video/BV1ASuU6iEyH/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2026.05 虾皮 AIAgent暑期实习面经

点评

这是粉丝朋友投递给我的暑期实习的面经。我分析的时候，有两点印象比较深：

1. 这个同学在自己讲解的时候，介绍项目讲了自己的Agent的记忆设计模块。长短记忆如何设计，如何写入的，如何整理的记忆。 讲到这一点，其实相对于很多人的项目已经比较深入了。很多同学的项目设计是不是没有一个精心编排的设计模块？我必须坦诚，2025年6月，作为一个社招，我拿的Agent项目就没有

记忆模块的设计，即使当时我都收获了很多Offer。所以我的第一印象是，这个同学的项目做的还是不错的，虽然就听到这一点。

2. 面试官很懂细节，你可以看到，前面很多问题都是围绕记忆模块的设计来谈的。面试官可以很快抓住里面的细节冲突：比如同学讲完他的系统的存储模块后，面试官发现漏洞，这样是不是无法恢复？比如同学讲到了他的记忆模块的整理的时候，面试官挑战这样会不会存在并发压缩导致数据竞争的问题。面试对于细节的深入以及面试官能够快速抓住细节提问，是比较少见的。

基于此，我也是觉得这个同学面试准备的应该是不错的，而且他为啥专门投递这个面经，我觉得也是因为这一次的面试比较难，与众不同，比较难分析。

我也确实又专门和他询问具体内容，看看我猜想得对不对，我和他的交流也验证了我的猜想。他说面了很多中小厂，面试官都不太懂，面试问题笔记里也都有，基本面一个给一个。（他的原话）。这一次是他见到的比较少见的如此深入的（当然对我来说也是，我社招很多面试官也不会问这么深入）。

那么，问这么深入，对于大家是不是借鉴意义不大？我认为不是。分析本期面经：

1. **可以加深我们对于ClaudeCode记忆系统的理解。** 虽然面试官是针对于这个同学的项目问的，但这个同学的设计，或者整体行业对于记忆模块的设计是类似的，后面的问题，我们不妨从CC的角度来想？比如面试官问：你的Agent存了哪些东西（你就想CC存了哪些东西），你的系统如何重建会话（CC如何重建会话）。通过这些帮我们检验是否对于CC记忆模块深入掌握。
2. **从包装的角度。**如果你的Agent没有亮点可以写，你不妨和这个同学一样加这么一句：**负责 Agent 记忆与会话恢复模块设计，实现长短期记忆管理、会话 Transcript 持久化、上下文重建与运行状态恢复能力。**写上去了，对于今天面试的具体问题，你就必须想清楚，包括ClaudeCode的章节的笔记和参考资料。
3. 其他的角度，也有一些启发，大家可以看面经和讲解视频。

总体点评：

1. 细节把控严格，**讨论深入细节**，要求对于自己的Agent项目细节考虑清晰。整体面试基本围绕项目：求职者讲了Agent记忆，主要问题都是围绕记忆模块展开提问。
2. 其他只问了一个RAG的新方向，属于简单考察下对于当前的新技术有没有**了解。**
3. 算法题1道，仍然要求我们不能停止刷题。

岗位介绍

jd是广告监控，广告投流，优化广告方面的agent效果。

相关要求是：

1. 良好的代码能力，精通Python编程语言，了解C++/C/JAVA编程；
2. 良好的数据结构算法基础，有一定的离在线系统开发实践经验；
3. 对大模型，Al Agent，Rag等新兴技术有一定了解，熟悉常用的langgraph,langchain等开发框架及相关解决方案，有相关实践经验；

一面

> 背景：求职者介绍了自己项目的记忆模块，大家就把它当成ClaudeCode，来思考，回答这些问题。

1. 最近10轮的会话记忆，然后多余的会压缩，那你的话如果超过10轮之后，你这个历史性拿得到吗?

这里一个是面试官问具体的压缩过程，可以参考CC的压缩机制，可以看 4.4.3.4 压缩

历史记录可以拿到吗，可以参考笔记CC部分面试真题： 你的Agent可以拿到历史会话记录吗?

> 前景：面试官抓到了同学讲述的逻辑，因为他讲了压缩，但是并没有讲到保存历史记录这一层级的操作，所以面试官后来都针对于这一问题再拷打，询问他是如何恢复会话的？实际这里应该是漏了一层，实际会话恢复，一般都会存一个会话账本，用来恢复，详细见：4.4.3.2 会话账本

1.1 你页面可以刷新吗？你刷新也刷新不了吧?

> 参考笔记CC部分面试真题：[你的项目是如何重建会话的？](#doxcn82evBdkorlT5Zj3V9NTdxt)

> 前景：还是因为没有回答到储存本地会话账本的问题，所以面试官在追问。

1.2 那你这个记忆是存在哪里的呢？短期的话也是储存在数据库里面的，然后长期的话才会进行压缩，是你自己存的吗？”

这题理解清楚了笔记CC 4.4.3的记忆功能就可以回答。

这个答案也可以参考 你的项目长短期记忆是如何存的?

1.3 你用户的回答的数据和AI的回复是存在了不同的数据库表中吗？那你重建会话的时候，是怎么配对的?

其实到这里，面试官一直都在围绕记忆功能展开。1.3其实本质上还是在挑战这个同学，是如何重建的会话？这个同学前文中回答说： 用户回答的内容和AI的回复是分开存的，所以面试官才有了这个疑问。

不深究里面的细节，本质上还是这里同学没有讲清楚，到底是如何存的，如何重建的会话。其实这个问题，如果按照我们上面讲的，是很容易的，就是整个会话信息都是存在了JSONL中，不存在配对的问题。

> 前景：同学和面试官在讨论压缩的问题，同学回答具体项目的压缩策略，采用的异步压缩。也就是一边对话，一边压缩。但是面试官询问，那么两个操作同时进行，那么可能会有覆盖和冲突。

2 你压缩是异步的，但是如果压缩的时候如果我正在新的会话，那岂不是覆盖了?

其实这是一个典型的两处并发操作同一块数据，就必须做并发一致性保障（比如加锁、锁段、或追加不变量、或者只用追加状态事件）的问题。面试官厉害的一点是他是真的理解了细节，在和求职者讨论。能做到这样：面试官首先比较懂，第二反应和理解能力都比较强。不过这样的接近细节的追问，也很容易答不上来把人心态弄崩。这提醒我们的是，我们的项目细节要想清楚，设计方案要完善，可以和面试官推敲，经得起推敲，就算设计不足，也能够和他进行探讨和讨论方案。就算被问蒙，也尽量调整心态，积极思考和回答。

回到问题本身，CC的压缩是同步的，包括像Copilot，当会话过长，我们都可以看到，压缩的时候，主界面会卡死对吧。所以大家这里也不要设计成异步的，除非你有更好的思路和理由，想清楚各种问题，要不就用开源的框架里的方法呗。所以这里就不讲了。

> 前景：其实就是面试官聊了这么多，觉得奇怪，看上去这些功能（回溯，会话重建，记忆存储),同学没有答的特别清晰，但是这些功能Langgraph都有，面试官就问他为啥不用Langgraph做。后面的反馈面试官也有给到建议：如果有一些开源的框架的能力，尽量复用，不要自己重新做。

3. 你的记忆的功能的实现，会话重现，回调的能力，都是怎么实现的？是自己单独存的吗？没有用Langgraph的结构化能力吗?

这个题给我们的启发就是在我们做项目实现的时候，去了解一下框架，是否有类似的功能，**能否复用。当然我不是说一定要复用。** 比如我看有很多团队，他们写Agent项目就是手搓的，他们其实有他们的考虑，比如稳定性啊，效率等。但是大家实现的时候，最少要做技术选型，xx框架有整个功能，**你为啥不用？你不用的理由是啥?** 这里同学比如在整个会话重建功能这里答的没有很好，但是langgraph又提供了相关功能，所以就被挑战。

这里，我们以Langgraph为例，他有提供具体的**节点回退功能（Time Travel)。**本质来说，也是持久化存储了一份对话历史，这个历史不同于记忆模块，就和我们讲的ClaudeCode一样，分为两个层级。实现细节，比如如何重建的，存储的形式，存储的数据库可能不同，但思想是一样的。我就不深入讲解了，当你把这些能力（会话重建，回退）写在简历上了，你可以从**CC也好，Langgraph也好**，看一下别人是如何实现的，把细节想清楚（最好的方法是自己做一遍），因为既然你写在简历了，细节要经得起问。

LangGraph的持久化记忆，可以看官方文档 Persistence章节：

4. 你看过什么开源代码吗？你这个东西没实现，但你更应该关注业内标准方案?

我慢慢也觉得看开源项目很有用，我在学习路线也把看开源项目写成学习路线的一个环节。

我们笔记解析了： ClaudeCode. OpenClaw Hermes Agent

其中ClaudeCode我想作为最精读的内容，各种参考资料，面试真题，都结合在里面，这个小节也会不断根据面试内容来完善。

5.RAG传统方案肯定有一些不足，比如语义分割，二维三维丢失，没有办法跨文档关联，有什么好的方案?

我的看法是：rag传统方法得会，这是基础。对于新方案，比如Agentic RAG, GraphRAG, 要了解，知道行业痛点和趋势。但是你说要不要深入，只能说学有余力再深入吧，先把其他的做好，因为深入的方向其实不是每个岗位都会做的，面试官不做，自然问的也不会深入。所以可以先了解一下大概。

本节内容可以参考笔记： RAG新方向，从Agentic RAG和GraphRAG两个维度来回答。

参考答案:

传统 RAG 的问题本质在于它把知识‘压扁’了——将原本拥有版面结构、多维表格或跨文档关系的复杂资料，粗暴切成了孤立的一维文本 chunk。要解决这个问题，目前业界最前沿也最有效的方案是将 GraphRAG

与 Agentic RAG 进行融合。

首先，GraphRAG 负责重构‘知识网络’。它通过大模型提取文档中的实体、关系和事件，构建出知识图谱。这不仅可以把二维表格、标题层级等版面结构利用实体和边重新连接，防止语义切断，更通过‘关系推理’和‘社区摘要’实现了跨文档的深度关联与全局总结。

其次，Agentic RAG 则负责重构‘检索行为’。它将传统的一步式相似度检索，升级为由 Agent 主导的多轮规划、多数据源路由和答案验证。Agent 可以把复杂问题拆解为多个子模块分别检索，遇到冲突信息能进行比对，甚至像人类专家一样在生成答案后回看原文进行事实核对。

总结来说，GraphRAG 解决了知识表达局限的问题，而 Agentic RAG 解决了检索过程被动的问题。两者的结合，代表了 RAG 从‘相似片段匹配’向‘结构化关系检索与主动推理’的范式转变的必然趋势。

6. 算法题

LeetCode 88. 合并两个有序数组

写完后会问具体细节。

7. 反问

视频讲解

https://www.bilibili.com/video/BV1BkVk6DEx3/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2026.05 安克暑期实习

点评

这一期视频是本硕985，但是是车辆工程方向的转行的硕士同学投递的暑期实习。安克这个公司我自己也面试过，在上面2025.10 安克创新。 这次面试和上面的虾皮AIAgent暑期实习面经的特点都是特别细。因为问得太细致了，所以不太好回答。这个同学的简历写的内容太多了，太密了，涉及的内容太多了，面试官又问得很细，所以同学很多地方答不上来，甚至有点穿帮。

本期视频和2605虾皮暑期实习面经一样，都是比较难的。难的点在于问的细节特别细致，两次面试也有一些重复的地方，比如对于Agent如何找到历史记录，这一点在202605虾皮暑期实习面经 是专门讲过，我也重复了笔记，包括压缩同步异步问题，也是重复的。有一些问题，相当于是压中的原题，比如说里面的你用什么VibeCoding工具？ClaudeCode和Codex 的区别，这个我之前就专门做了一期视频来讲解的，讲解内容中也给了这两个问题的回答。可以参考 Codex VS ClaudeCode。

这也是为什么这两个同学专门投递给我，因为他们面试的时候，感觉答得不是很好，有点难对付。像那些简单的面试，笔记里都有答案，所以他们都能应付。但也正好去啃一下这些硬骨头，才让我们进步得更快，也正是通过这些难的面试，我查漏补缺，补充笔记，让这个笔记更加完善和完整。我会持续给大家整理的，每一期面试视频其实我要整理很久，参考很多资料，书籍，修改笔记才呈现给大家，相信大家站在我的肩膀上，复习起来效率会更高。

这期同学给我的记录，是按照主问题+追问的方式组织的，我觉得这个方法也很好。大家复盘面试，可以录音，然后用使用文稿，结合我给的提示词：面试复盘小技巧。 让AI通过这方法快速将面试录音txt转成问题总结。

同时我在复盘本期内容，看了这个同学的简历，我觉得他写内容太多太密了。整个简历一页半（一般建议要么1页要么2页），写的项目一个Agent一个RAG，涉及的技术密密麻麻。

这样一个是不利于HR筛选，初筛容易过不了。

第二个，**写的东西太多了**，后端的Docker,Redis，前端FASTAPI. 模型相关的内容：Deepseek, 微调，训练。Agent相关的：Langgraph，Agent记忆，测评，压缩，写入，Function Call，安全优化。 Rag的Milvus, ElasticSearch，评测，意图识别，召回优化，查询改写 等等太多内容了。 真的很容易让人怀疑真实性。包括我们看下面的面试问题，面试用词：这个**真的**是你自己做的吗？以及同学自己反馈有些地方答得不好，前后不一致，被面试官指出。这些某种程度也是自己简历给自己挖的坑。写的一定要会，不要罗列太多技术了，突出重点让HR好筛选，写出技术深入让业务主管觉得OK，愿意面试。

ps:这个同学面试是通过了的，同时他反馈安克没有手撕代码，只有AI coding笔试题。这也给大家一个信息，这是其它专业（工科）转行AI的例子，他暑期实习也有几个offer,也有大厂的面试邀请（字节，阿里）

一面

一、个人背景与转型动机

> 这个同学是从其它专业转到的大模型开发的岗位，所以大概率针对于他的背景，面试官会问这个问题。我就不用复盘答案了，但是其实我想提的一点，是一个意识：就是面试除了专业的问题外，像自我介绍，反问，以及这些面试常见问题：你为什么换工作，为什么换方向其实都是可以去复盘的。自我介绍，反问的技巧笔记记在了：面试技巧 这里。我自己面试研究所的时候，像为何换工作这些猎头都帮我把关和推敲，这些东西其实像复盘都可以复盘，变得更好，结合自己实际情况去做，分析吧。

1. **主问题：** 你的专业是车辆工程和机械工程，但学的却是AI相关的东西，这个跨度感觉有点大呀？

- **追问1：** 具体是什么机械工程专业？

- **追问2：** 你大学时候的车辆工程跟现在学的是同样的知识体系吗？这个转换过程是怎么完成的？是在学校期间（硕士阶段）完成的吗？

- **追问3：** 那你当时是怎么想的，决定转到这个方向的？

二、AI工具认知与使用习惯

1. **主问题：** 你平时一般用什么样的AI产品？用的有哪些？

- **追问1：** 你觉得 Claude Code 和 CodeX 这两个工具有什么优缺点？

我刚看这个问题的时候，有一种压中题目的感觉。因为我上周自己总结了专题，ClaudeCode VS Codex. 为什么总结他是因为我根据最近的趋势总结的（你可以笔记里这部分视频讲解），我也总结了相关面试问题。紧接着过了一周，粉丝投稿的面试内容这个问题就是和我当时总结的问题一模一样。我当时在视频里还说要是面

试官问了，你用这个回答绝对很高级，看这个问题就有一种押题押中的感觉。总之，此问题具体答案和讲解直接看笔记：ClaudeCode VS Codex.

三、简历真实性与项目角色界定

1. **主问题：** 我看你简历里“智慧交通”那部分有一段话是重复的（如“平均每次研究提取...”），你看到了吗？简历到底是你自己写的还是用什么写的？

- **追问1：** 你自己看一下简历。（引导确认）

同学自己写的简历，还有重复的地方。属于低级错误，其实现场让面试官指出其实挺减分的，大家自己的简历自己多看看。

2. **主问题：** 你做的这两个项目课题，你是单独做的还是一群人做的？你在里面的定位是什么？

- **追问1：** 这个项目真的是你全部独立完成的吗？

- **追问2：** 以第二个“工业智能运维RAG系统”为例，你在里面担任什么角色或者负责哪些模块？

如果我前面所说，同学简历写的内容太多，让人怀疑真实性。面试官问这个也合理。注意如果你想说得很厉害，都说是自己负责的，但是一问又矛盾，答不上来，还不如一开始就说简单一点。前者让面试官觉得你不太诚信，比觉得你写得简单，印象还得差一点。还是那个原则： 写在简历的你就要会，经得住问。要不不如不写。

四、架构设计与技术选型依据

1. **主问题：** 你设计这个模块时，架构设计的依据是什么？有没有横向对比市面上的产品？

> 这是一个需要我们综合能力的问题。我们需要的理论依据：架构如何选型：参考笔记：MultiAgent的架构。这里面有什么时候选单Agent，多Agent的思路，里面还有一篇经典论文。有没有对比市面上的产品，可以看笔记里的开源项目解读：Hermes, Openclaw, ClaudeCode。
>
> 下面给出一个针对于我的项目的参考答案(单Agent - 一个Chain式调用），我通过上述的思路和来组织一个和我自己项目相关的答案。大家的答案要根据上述笔记和自己的项目来回答。

我的项目整体架构为：单Agent的一个链式调用结构。选型依据是我看谷歌的一篇论文（参考笔记：MultiAgent的架构），当单Agent基线收益已经比较高，同时任务本身也不是并行的情况下，使用单Agent效果最好，这满足我们的任务性质：较为简单，串行的。

我也横向对比过市面的产品，比如以ClaudeCode为例，在绝大多数场景下，它使用的默认也是单Agent，也符合这里的原则，盲目引入多Agent会带来协调税。不过和CC不同的是，CC使用的是一个React循环，因为它的任务是通用的，流程不确定需要探索的。而我们的任务是确定的，更加偏重于业务流程，所以使用单Agent链式调用也是合理的。

- **追问1：** 为什么选择 DeepSeek? 具体是怎么对比的？对比了哪些维度？

> 面试官其实非常喜欢问模型的选型。大家可能没有特别深思熟虑，做模型选择和对比，但是确实你面试说的时候，你得说你做了选型，做了对比，最终选了什么，以及为什么。

> 这个问题还是要大家根据自己的项目来准备，在笔记：模型选型策略里面讲了这一问题，具体思路是：
>
> 1. 三段论方法确定大致业务模型
> 2. 通过查阅资料/对比实验方式确定最终模型。

答案参考 7.2 模型选型：回答示范

- **追问2：** DeepSeek 用的是什么版本的?

根据项目真实情况回答

- **追问3：** 你们是自己部署了一套，还是通过 API 调用的?

根据项目真实情况回答

五、检索召回策略与准确率评估

> 这里面试官在对候选人的RAG系统的关键指标进行询问。其中意图识别指的是RAG系统查询前：判断用户到底想干什么，再决定后续怎么检索的策略。不知道意图识别的，可以看笔记rag概述。 **意图识别就是图中的3，4步。**
>
> 面试官问这些问题，是因为同学简历写了："**设计意图识别模块，对精确查询与模糊描述采用权重化差异**"

1. **主问题：** 你刚才提到使用了关键字和向量进行双路召回，具体是怎么做的?

> 这已经是最经典的RAG技术了，RAG的标准传统方案，我们笔记项目也有。

其实在笔记的 3.3.4为什么要进行混合检索，怎么做的？ 这里有这个问题，如果所说，这是RAG很基本的问题了，这个部分还有关于RAG混合检索更多的内容，可以看一下。

- **追问1：** 意图识别的准确率是多少?

我自己的RAG项目没有做意图识别，没有被问到过，同学反馈给我的项目面试问题也没有相关问题，所以意图识别对我来说是第一次遇到，在我这里不算常考的点。同时，面试官问到这个同学意图识别，是因为他自己简历写的有，所以面试在问。

借助这一次机会，我在RAG项目整理RAG的全局总览，里面可以清晰看出意图识别处于哪个部分。

在笔记5.1的意图识别里面，总结了什么是意图识别，意图识别怎么做，如何测试意图识别的准确度 的理论部分。这些问题足够面试官问你相关意图识别的概念部分了？但是如果是说更偏工程实践的，比如当前追问1和追问2，意图识别准确率多少，这些其实是你项目做了，才会被问到，答案我就不总结了。如果你写的项目想做和有做，可以参考上面的理论，设计好方案。如果你不做，把上面理论部分弄好就行了。

- **追问2：** 八九十的准确率是怎么测出来的？怎么确定是这个数值的?

同上

- **追问3：** 你这个问答系统是真实会在线上使用的，还是仅仅作为一个学校课题?

如实回答即可。但是如我在本次面试开头分析的，这个同学本身写的技术栈就多，这里也说他的项目上线了。如果你说上线，就有很多问题要回答：你是如何部署的？如何监控？如何记录日志？如何版本管理和回退？等问题，后面这个同学在上线后的真实数据管理这里又答不上来。这个问题就想强调一下我的感受：你写到简历，要不真的做了，要不也是能说出细节，否则别写。写太大了本身经验不够处理这些情况，被问露馅了反而减分。

- **追问4：** 在真实的运用场景下，你的召回准确率是多少？有没有进行过统计？不能只靠上学前的测试集标准吧？

- **追问5：** 上线后有没有收集线上的准确率？针对收集到的数据是怎么优化的？

- **追问6：** 你觉得线上真实的意图识别准确率应该怎么算出来？

追问4，5，6就是同学说自己项目上线了，引出的相关一些问题。这些具体怎么优化，怎么收集，怎么算我就不讲了，和你自己的业务相关。我想讲一个你应该知道的最基本的问题，就是线上的分析是怎么做的，因为无法准备好对应的数据集了，线上评测和线下就不一样。 这个也是为什么面试官会问追问6.

线上的评测和分析通常是使用在**业务里面打点，上报关键节点和数据，建立这些数据使用的方法相关的指标。通过这些指标来衡量业务健康度。**

为什么说这个点，因为面试官问这个同学线上怎么分析，同学说是通过引导用户反馈，让用户选择识别是否正确？这个就不是一个最好的方法。 好的方法是埋点上报分析，面试官也是在引导他往这里回答。这是最基本的概念。至于后面如何分析，埋哪些业务指标，怎么迭代形成数据飞轮，那就是具体到业务场景下分析了。也不具有通用性，我就不讲了

六、用户反馈机制设计

1. **主问题：** 如果用户不主动反馈，你怎么收集用户的反馈？

- **追问1：** 每个都询问一下他吗？（质疑体验不佳）

- **追问2：** 那你觉得应该怎么收集比较好？

这俩问题在上一个问题已经讲了，方法是：**埋点上报分析。**

七、记忆管理系统与上下文处理（深度考察区）

> 记忆和上下文管理，很常考的，和之前一样，我完全按照ClaudeCode的实现和它的思想来作答。我在写CC的笔记的思路也是这样，如果有面试相关问题，我觉得我从笔记里的CC的内容找不到回答的思路，那我就去重构笔记。大家看相关面试问题的时候，也是要结合笔记CC架构/源码解读的内容来复习。

1. **主问题：** 你们是怎么管理上下文和记忆的？记忆分几层？是否有 Session 的概念？

参考问题：CC的记忆功能是如何实现和管理的？ 结合CC的思想，设计和管理你的数据结构，其实你可以照抄。

比如把Claude.md这是长期记忆，你也设计一个长期记忆放在本地。

设计一个Memdir，另一块AI维护的长期记忆。

再设计一个短期记忆。就是上下文维护，这个和CC可以一样。

管理方式也可以照抄。 把这些变成与你自己的项目相关的内容就行，可以用AI 输入这些原则+你的项目，让它生成答案。

或者你真的是自己设计，但是思想还是去学会CC的思想，知道如何管理，然后扬长避短。

- **追问1：** 长期记忆具体包含哪些记忆类型?

直接参考CC笔记部分: 长期记忆应该存什么?

- **追问2：** 到达上下文上限触发蒸馏长期记忆时，这个是同步的还是异步的?

> 这次面试和上一期虾皮的面试，都聊到了压缩是同步异步的问题，我在复盘的时候，也特意重新去整理了整个CC的压缩部分，不管是重构的CC记忆的全局的图片也好，还是后面详解每一层压缩体系的描述，我多强调了同步和异步的问题，请大家自己看一下笔记： CC 压缩 部分。
>
> CC的压缩，可以说都是同步压缩（异步处理中间状态），具体可以看一下笔记和我的讲解视频。但是这个同学设计说是异步的，然后后面追问3和4就矛盾了。可能是自己没有想清楚。我们的原则是：要么和CC一样，要么借鉴他，想清楚。不要自己没想清楚就设计一个比CC还复杂的异步压缩，最后答不上来。

参考：你的项目怎么做的压缩的?

- **追问3：** 如果是同步蒸馏，在对话过程中需要等待结果吗?

- **追问4：** （候选人回答要等之后反问）要等的话，那你不就变成同步了吗？具体是怎么做的?

问题3和问题4我认为是候选人自己没想清楚细节，出现了矛盾。我们对于这个问题，把CC的压缩策略相关笔记消化好就行。

- **追问5：** 蒸馏长期记忆时，具体保留了哪些东西?

直接参考CC笔记部分: 长期记忆应该存什么?

- **追问6：** 项目压缩的时候，是在什么时机压缩的?

参考：你的项目怎么做的压缩的? 笔记的这一部分，不管是面试真题，还是压缩机制的讲解都讲了，4层压缩对应的4个时机：**主动触发，缓存时间到期， 上下文到达一定限制。** 原理还是一样，要么照着CC回答，要么基于它的思路去设计。

- **追问7：** 让 AI 去判断是否必要的时候，AI 是怎么判断的？主要依据什么规则？什么叫“必要的时候”？

这是上一个问题的追问？上一个问题同学回答AI判断必要的时候，这个是什么是必要的时候。其实这个回答在上面的内容掌握了以后，也可以回答。 CC中，什么是必要的要压缩的时候？**主动触发，缓存时间到期， 上下文到达一定限制。**这三个是CC中必要的时候。

- **追问8：** 你的长期记忆是如何使用的，如何召回的?

> 我在复盘的时候，一边复盘，一边整理笔记。你看现在面经大部分都没有答案了，都是归位笔记的某一部分，因为我都根据面试内容，把笔记融合了。我希望笔记里一定能找到面试问题的答案。像在分析这个问题的时

> 候，我就在长期记忆 笔记模块加入了如何召回的内容。所以答案直接参考下面笔记链接部分就行。

参考CC笔记，4层记忆模块：4.4.3.1 四层记忆 长期记忆笔记：长期记忆是如何使用的?

- **追问9：** 你自己是怎么切块和分层的？召回时是按照这几块来的吗?

这么问是因为同学回答自己的召回，是切块做的，相当于用了传统RAG。那么你就仔细想好怎么说吧。

不过CC的设计，是不用RAG，而是依赖于模型本身去语义匹配。这句话如果你看对应CC的笔记，你应该能理解。 所以两条方案其实都可以，走前者，就是传统RAG的问题，但是你想清楚你是如何设计的？如何分块的？相信学了笔记RAG相关的内容，这里也不难回答。需要灵活运用。

- **追问10：** 最新的记忆一定是最好的吗？如果想召回最相关的记忆应该怎么搞?

召回记忆本来就是匹配相关性的，CC的召回里面也没有基于日期来加权，这里可能是同学自己的策略，聊到了项目召回和日期相关。这里不展开了，和他自己的设计有关。

八、数据存储与工程落地细节

1. **主问题：** 你们的记忆是怎么存储的?

这里偏向于不是说长短记忆如何维护，而是如何存储的。我自认为笔记写的也很清楚，参考CC记忆模块。

最上面那种图，就是我重构的，很清楚展示了CC的模块存了啥，用的啥数据结构。（CC基本用的MD）。原理还是一样，要么照CC说，要么在它上面创新。

- **追问1：** 为什么把短期记忆直接放在内存里面？是因为数据量小吗？什么原因?

- **追问2：** 用完之后想找历史记录找不到怎么办?

这个我觉得笔记参考CC记忆模块 和 面经 虾皮暑期实习。 我总结的已经很清楚了。在总结虾皮这一期视频时，面试官专门讨论了很大章节：如何恢复历史？我专门总结这个面试的时候，重构了笔记，加入了对应的对应CC的Transcript的存储的讲解。

所以这部分答案看CC记忆模块 和 面经 虾皮暑期实习。 虾皮暑期实习也有讲解视频，专门讲了历史会话的存储和恢复。

- **追问3：** 你说的内存是指 Redis 还是本地内存？Session 不存下来吗？数据入库吗?

不讲了，还是和同学自己的项目相关。如果你要存记忆用数据库，不管是SQL,还是redis。自己想好为啥吧。CC用的是md。其实里面的思想是一种权衡和取舍。可以问问AI，我的长期记忆用MD， sqlite，还是redis存好的，什么时候用哪个？自己思考下。

- **追问4：** 你们的数据量大概是多少？有没有做过评估?

如实回答吧

视频讲解

https://www.bilibili.com/video/BV13LJP67EUJ/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be

1b

秋招

2026.07 宇树科技Agent开发

前言

这个素材是从网上的小红书博主这里找的，我不知道是校招还是社招，没有写，这个就是我吐槽的，网上很多面经它到底是什么岗位，什么方向，甚至是真的还是假的都无从判断。我自己做面经，除了这一期之外，我基本岗位，校招还是社招，岗位jd都写了。

我知道的信息是这个面经是26年7月的面经，我看了一下，肯定是Agent开发岗，不会是算法岗。我这里有一面，二面，三面，HR面的信息。 我先总结了一面，一面基本上都是八股，对于我们复习知识很有帮助，我接着这个一面，把相关的八股部分问题好好重构了，沉淀到了上面对应的知识讲解中。二面三面解析不解析再看吧，目前我这里非常多面经，面经不是瓶颈，瓶颈是我每一期都是精讲，讲得比较慢。 反正我会尽量挑最新的，最有代表性的来讲解的。

一面点评：一面整体难度不难，感觉问题都是八股题目。面试官这么喜欢问八股的面试，我们也比较少见。对于这些八股题目，即使我们没有专门背诵，我们也应该可以回答出来，也许不那么完美，不那么标准，但不至于回答不出来：比如Agent的长期/短期记忆，向量数据库和关系数据库区别等等。

对于八股，我的个人建议是：**首先你别排斥它**，觉得听着这个八股这个词，你就觉得是贬义词。很多八股也是我们回答一些其他综合性问题的理论基础，我觉得还是挺有用的。

从学习的角度，八股我不建议是靠纯背的，合理的做法是先理解，我们讲的每个八股对应都有知识点，你先去理解他，看讲解视频，包括我们面试真题讲解八股的时候，也有对八股问题的思路如何回答做讲解。 先理解，这样即使不背诵，你也能回答一二。面试前可以突击根据我们的这些八股的参考答案，背诵背诵，争取用最标准流畅的思路来回答。

一面

一面的所有问题都是八股，八股特别适合收录至专题笔记，作为专题知识后面的面试真题参考。所以答案全部都是链接形式。同时这期面经对应的讲解视频，我有专门对这些八股，讲复习策略，学习思路，感兴趣的请看下面的讲解视频。

介绍一个 AI Agent 项目，说明你的角色与贡献

向量数据库与关系型数据库的核心区别

八股问题，我直接收录至了我们RAG对应知识讲解部分。答案请参考RAG向量数据库部分面经。

RAG 检索相关性差的原因与优化思路

收录到了RAG知识专题部分。请参考RAG专题笔记面试真题部分。

ReAct 的核心思想是什么？他是如何实现的？

收录至Agent知识部分，ReAct小节。

Agent 记忆的常见类型与作用?

经典必考问题，我把答案整理了一下，放在了Agent记忆模块专题，具体请参考：Agent记忆面试真题。

多轮对话状态管理要考虑哪些关键要素?

收录至：Agent记忆面试真题。

评估一个 AI Agent，除了准确率还看哪些指标?

收录至Agent评测面试真题：Agent评估面试真题 第15题

LangChain / LlamaIndex 解决了什么问题，有哪些局限?

纯八股，翻译过来就是常见框架的优缺点。 收录至Agent框架部分面试真题。

如何设计 Prompt 来提升任务稳定性与准确性?

收录至提示词工程面经部分。

外部工具 API 调用失败时，系统如何容错或降级?

笔记Agent健壮性设计是专门讲这个，这一块专题也有专门的讲解视频，详细请参考 Agent健壮性设计

算法题：计算两个向量的余弦相似度

二面

为什么选择当前的向量数据库、LLM 与开发框架?

💡 **思路.**

这三个问题分别对应笔记：

1. 常见的向量数据库。
2. 如何选择选择Agent项目中使用的模型?
3. Agent开发框架。

后两个问题我是有专门的讲解视频的，也是比较高频的考点了，这里就不展开了。

对于第一个问题，选择什么向量数据库？ 这个问题相对来说没有那么高频，RAG本身就没有Agent问的多，但是如果你写了RAG项目，用了向量数据库，那么就是必须要准备的。思路就是，先去理解常见的向量数据库，然后根据特点来说明，你选择的向量数据库的原因。

我这里想强调一点：**选型不是越复杂越好。不是说你和面试官说你使用的数据库越复杂，越加分。**如果你的系统就是一个本地的RAG，但是和面试官讲：使用支持上亿用户的企业级向量数据库Milvus。 这反而显得很搞笑。

通过不同的实验对比，选择最合适的技术方案 而不是盲目选择最复杂的技术，同样是有价值的事。

RAG 的 Chunking 策略如何设计，遇到过哪些问题，如何调整?

RAG 分块部分，有专题讲解，并且覆盖了该问题，请看笔记：RAG分块。

如何设计长期记忆与短期记忆，以及跨会话偏好存储?

长短记忆在Agent专题的记忆模块，以及讲解ClaudeCode的记忆模块，都有介绍，并由面试问题的讲解。

跨会话偏好存储就是长期记忆的一部分。

ReAct 循环中，如何检测并跳出无效的“思考—行动”循环?

已经收录至笔记ReAct章节对应的面试问题。

多 Agent 协作系统需要考虑哪些架构问题?

已经收录至笔记 Multi-Agent架构章节。

如何保证 Agent 输出的 安全性?

已收录至笔记 ClaudeCode安全系统章节对应的面试真题部分。

用户问题模糊或有歧义时，如何做澄清性对话?

已经收录至笔记ReAct章节对应的面试问题。

描述一次最难调试的 Agent 问题、根因与解决过程?

💡 **思路：**这个题我们之前讲过，像这种问题，很多面试官都会问，而且问题没有标准答案，是一个上限很高的题。你自由发挥，如果答得好，吸引面试官的兴趣，让面试官和你深入讨论，面试结果就稳了很多。思路是建议大家一定要提前准备这个问题，在项目中找一个点进行挖掘。

如果你完全没有准备，笔记里也记录了几个通用的问题，可以用来应急回答这个问题，参考笔记：

> [你在项目中遇到的最难的问题是什么？如何解决的？](#doxcnpqeoSutE489zhUPcWrUaBb)

如何对 Agent 系统进行压力测试，关注哪些性能指标?

收录至后端专题面试问题。

手写算法：按句子边界做文本分块，避免截断句子?

直接问AI吧。

讲解视频

一面：

https://www.bilibili.com/video/BV1jQtx6rEd4/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

二面：

https://www.bilibili.com/video/BV19wYt6nEm5/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

---

2026.08 阿里淘天秋招大模型开发

今天这一期内容，是26年8月秋招阿里淘天校招一面的内容。一面内容，基本上是围绕项目展开提问的。只有一个问题，MCP 相关的技术，只面试官主动提问的。

从这个角度来看，从八股，知识的角度上来分析，好像借鉴意义不大了。

不过换一个角度来思考：

因为本次面试大部分内容是围绕项目，简历来展开的。通过这个内容我们可以学习到**面试官通常对简历是怎么提问题，喜欢怎么问，学会了这个，我们用这个方法论，可以在面试前提早预测面试重点，做好准备。**

毫不夸张地说，学会了这个技巧，如果你做了好准备，面试前70的内容都是你准备过的题目，那么面试不就是十拿九稳了？

讲解知识之前，我们先说一下这个同学的背景：这个同学实习经历，本身的技术栈是和后端相关的，有比较丰富的后端实习经历，所以他的秋招一方面是投递： 后端 + AI的岗位，一方面是投递：纯AI(AI开发的岗位)。

本次面试的岗位，是纯AI的岗位。（我写这个是给后面分析面试官为什么会问一些后端的内容，来做一些铺垫）。

同时，对于问题的解析，有一些问题其实是可以复用到笔记里已有的项目的，所以对于这些问题的解析，我会按照我们笔记里的项目，来组织答案。

有一些问题是和这个同学个人背景（后端项目） 相关的，提问就围绕了后端内容 + 传统技术，那么这一块内容我会不解析答案，只是带大家分析面试官为什么会这么问。

我在这一期面经，把下面的每一条，如果是面试前可以预测出来的，我都标出来，大家看看在一次面试之前，有多少面试内容是可以提前准备好的？

岗位介绍

下面内容是我从面试录音里扒的，信息应该还是比较准确的。

淘天集团—私域技术团队—AI 应用研发工程师

主要工作

1. 面向淘宝天猫真实业务，发现适合由 AI/Agent 解决的问题。
2. 设计、开发 Agent 应用，连接模型、业务数据和工具。
3. 根据业务需要选择模型与技术方案，并通过实验验证。
4. 建立评测与反馈机制，持续优化效果、成本、延迟及稳定性。

5. 完成后端系统集成和上线交付，在实际用户使用中迭代产品。

更看重的能力

- 有真实的 AI 应用或 Agent 项目实践；
- 不只会搭 demo，能说清需求、选型、指标和改进路径；
- 理解检索、上下文管理、工具调用等应用机制；
- 具备数据库、并发、事务等工程基础；
- 有主动学习、解决问题和自我迭代的能力。

团队能提供什么

面试官明确提到了：

- 师兄带教；
- ATA 内部技术分享平台；
- 大规模业务和用户反馈；
- 相对充足的 AI 试验预算。

一面

请简单做一下自我介绍。

你现在还在xx吗？

> **【可预测】**同学的背景是留学生，所以面试官问这个也很合理。稍微提醒一点的是，回答要体现出稳定性，就是让面试官觉得你真的是要回国工作的，不要让面试官觉得你大概率比如想留在国外，那么他会认为你对这个offer的忠诚度不高，给了offer也可能不来，别踩这个坑。

为什么考虑回国工作？有没有考虑继续留在国外？

你老家是哪里的？

选一个你认为最能代表自己、比较经典的项目介绍一下。

> **【可预测】**很典型的问题，我在我们ForgePilot项目面试前突击有说，如果你的项目是非常复杂的，企业级的合作的项目，那么首先想清楚的是你负责哪一块，包括简历里也要写出来。

在 xx 项目中，你的核心工作是生成商品描述，还是对生成的描述进行评估？

商品内容评估的指标是怎么确定的？怎么衡量生成内容的好坏？

> **【可预测】**很典型的问题，我在我们ForgePilot面试前突击，简历编写的原则，这两个小节，我们都说了，对于任何一条出现在简历上的数据：都要想清楚：**你对于这个数据是怎么测的？测试方法是什么？Baseline是什**

> **么？数据量有多大？应该如何进一步提升？**
>
> 按照这个思路去分析，可以说，这个问题的主问题，4.1 4.2 4.3 三个小节，我们都是可以去预测出来的。 4.4 这个问题不是那么好预测，这需要我们一些灵活应变的能力，现场发挥。

简历里的平均分 81.7/100 是怎么计算出来的？

它的 baseline 是什么？有没有优化前的基准可以对比？

我将4.1 4.2这两个问题，套用到我们的ForgePilot项目，参考ForgePilot 项目面试真题预测：你提到在 180 条任务上，成功率从 31.11% 提升到 73.33%，相对提升约 135.71%，是怎么测出来的？什么算成功？

大家可以用这个方法，套用到自己的简历项目上，自己检验你是否把你的简历的内容想清楚了。

距离满分还差将近 20 分，具体不足在哪里？怎样向 100 分改进？

> 同样的，我还是把这个问题套用到了[ForgePilot项目](#doxcnRPaXSsDpdRnIVZ9oAS4UDc)中，你也用这个问题检验自己的项目是否做好准备了没吧！

引入这套评估体系前后，整个产品链路最明显的变化是什么？

这个题不像上面三个题那么好面试前预测，需要一些灵活发挥的能力。这个题的解题思路和策略我还是套用到了ForgePilot项目中。

把五条数据合在一个批次里交给 LLM 评估，为什么能节省调用量？

> **【可预测】**这个也是高度可以预测的。这个问题来自于面试官对于简历里写的内容的进一步询问。背景是同学简历里写了，他项目里的一个优化策略，是之前数据是一条一条输入给大模型，然后输出的，现在采用了优化策略就是，5条一起输入，统一返回一个结果。接下来面试官其实就是针对这个具体细节来问，这么做的好处是什么，为什么选择5条，有没有实验。
>
> 一直以来，我们都有一个经验就是，**面试官特别喜欢问技术的选型，**选什么LLM模型，选什么向量数据库，选什么Agent框架。这些我们都应该想清楚，同时这些所有问题的答案，**最好是有科学的理论指导+实验对比，因为实验数据的结论是证明选型有效最有力的证据。** 关于这一条的具体答案，我就不讲了，因为我们的项目没有相关问题，不具有参考性。

- 5.1 为什么选择五条，而不是五十条或者五百条？
- 5.2 批处理有什么损失或者副作用？为什么不能一直增大批次？
- 5.3 选择五条有没有数据支撑？它是不是最优解？
- 5.4 如果不是最优解，有没有探索过寻找最优解的路径？

> 【可预测】6.7 两个问题，我先讲一下背景。第六个问题的背景是，这个同学写了一个项目，是后端+大模型的，具体功能是类似于做了一个网关，去管理公司不同业务的调用大模型的预算管理之类的。
>
> 第6个问题，基本上就是对于简历写的具体业务，功能上的一些澄清。

> 第7个问题，是同学自己写简历的时候，写了多个业务同时访问，然后使用了悲观锁防止lost update的丢失，接下来的讨论基本围绕并发数据安全，锁怎么加，锁在哪里？如何释放等经典问题。
>
> 这两个问题，都是面试前可以预测出来的：你写上的东西，你就要知道是什么含义，以及写上的技术（比如这里同学写了锁，那么锁的安全问题，也是传统老八股了），也就必须弄清楚。
>
> 至于这两个问题的答案本身来说，我认为就没有太大通用性了，所以这两个题的答案就不讲了。

你提到用悲观锁消除 lost update，这是什么业务背景？在什么场景下发现问题，又是怎么解决的？

- 6.1 你们是不是在 OpenAI 或 Azure OpenAI 的账号体系之上，又做了一层预算管控？
- 6.2 预算管控体系具体是怎么设计的？
- 6.3 上游 API 本身已经有计费和额度限制，为什么还需要自己做这一层？
- 6.4 是否可以理解为：多条业务线共用 API Key，通过网关统一做预算、权限、日志和追踪管控，避免一条链路耗尽所有额度？

三个 workflow 同时执行时，加锁的对象是什么？锁的 key 是什么？它们会竞争同一把锁吗？

- 7.1 同一项目、同一月份的多个并发任务，会不会因为锁竞争而退化成排队等待？
- 7.2 锁什么时候释放？是整个任务执行完才释放，还是在其他时机释放？
- 7.3 从加锁到释放锁之间，是否只做数据库配额扣减？会不会包含调用大模型这样的操作？

代码库智能体平台是在什么背景下做的？

> 【可预测】这些问题仍然是可以预测的。
>
> 先讲一下背景，这个背景是同学做了一个类似和ClaudeCode 的Agent，会对代码仓库去检索，然后回答问题，更好的帮助人去理解代码。 下面这些问题，是围绕两个主问题：为什么用BM25 + pgvector 双路检索 和对比ClaudeCode的工具效果对比。
>
> BM25 + PGVECTOR 双路检索，本身就是简历里写了的内容，同时作为RAG里面核心的内容，面试前不该没准备。
>
> 第二个问题，相比这些工具，你的优势是啥，这个基本上我看到他的简历我就会问，我们自己做的forgepilot项目，我最开始设计的时候，就准备了：为什么现在已经有cc这种平台，我还要做forgepilot， 面试经验丰富了，这个题基本上也是提前就预测出来了。
>
> 这个题，我们就以简历里写的 发布助手项目为原型（这个项目就是笔记里RAG项目组装改造来的），来讲解。

为什么选择 BM25 和 pgvector 做双路召回？

把这个问题套用到了智能发布助手（RAG项目改造）中。

是遇到了具体问题或 case 才选择这些技术，还是一开始就直接选用了？有没有技术选型过程？

问题套用到了 智能发布助手相关面试题中。

有没有用过 Codex、GPT、Claude Code 等通用工具本身的代码库探索能力？

有没有把你的方案和这些工具做过对比？它们是怎么实现的？

相比这些工具，你的项目优势主要在哪里？

💡 **思路：**

这个问题的背景就是这个同学做了双路检索（RAG）来处理代码仓库。因此面试官问了8.3, 8.4, 8.5 三个问题。我把这三个问题，总结成了一个：**你有没有对比过claudecode，codex功能的检索模块，他们是怎么实现的，对比他们你的双路检索有什么优势吗？**

这里的回答思路，可以参考我们笔记RAG趋势探讨的章节，第一个角度是虽然主流的coding 框架都放弃了RAG, 不代表用RAG的方案就不好，它仍然有它的优势。

如果还能够结合当前项目的特点，比如对RAG做了相应适配（比如我们forgepilot仓库，代码仓库大小超过300GB，主流coding agent都不能用） + 做了实验对比，说明效果更好，这个答案就更加丰满和有说服力。

但是我不太知道这个同学的具体业务场景，所以咱们就只从第一个叫角度来回答一下。（因为缺少具体业务场景，下面这个回答我觉得有点单薄，我们就当借助这个问题，复习RAG是否被取代了的这个知识点吧）

📝 参考回答

Claude Code 和 Codex 的基础代码检索都可以概括为 **Agentic Search + grep/ripgrep**：模型自己决定搜什么，读取命中的代码，再根据新线索继续搜索。Claude Code 主要通过专用搜索工具调用，Codex 的公开本地实现则通过命令执行工具运行 rg。这种方式灵活，能直接查看工作区的最新代码，很适合跨文件探索和问题排查。

我们采用 **关键词 + 向量双路检索，并不是认为这种方式不好，而是做了不同的取舍**：关键词检索匹配函数名、配置项等明确线索，向量检索补充自然语言描述与代码之间的 语义匹配，再把融合、筛选后的相关片段交给模型。

我们看重的是在线延迟和 Token预算的可控性。通过预建索引，尽量在一次检索中找齐证据，减少模型反复“决定搜什么—调用工具—读取结果”的往返，也减少中间内容进入上下文。我们的代码相对来说比较稳定，修改不会特别频繁，更多的情况是基于代码仓库，进行理解和分析，这样在代码变动后，进行分块，向量维护的开销的成本会降低，在多轮对话中，检索速度更快，也会更省Token的好处会更明显。

MCP 是为了解决什么问题而提出的？

💡 这个问题是这一次面试，唯一一个脱离于项目之外，面试官主动问的问题。算是关于MCP的八股问题了，我把这个题专门总结到了MCP章节，请参考： MCP的新结束&趋势+面试真题

9.1 MCP 这种工具形式，给 Claude Code、Codex 等通用 Agent 带来了哪些问题？

- 9.2 在大模型调用外部工具方面，目前业界更成熟、更倾向采用的做法是什么？
- 9.3 有没有了解过用什么方法替代传统的 MCP 调用形式？

二面

面试时间26年8月底：淘天二面面试问题总结

📌 **总体点评：**这一次的面试，没有算法题，整体问题复盘下来就是两块：

1.**Agent评测(80%)**

2.**后端(20%)**（因为同学简历写了后端的技术）

Agent评测内容，主要涵盖：

自己的Agent评测方案（这个应该大家都准备过了）

Agent评测理论（**黄金数据集的构建，基于评测驱动开发，评估方法的建立**），这些内容在我们笔记里的Agent评测中都有。我们需要学习这一部分，**补齐Agent评测的基础理论**，才能跟上面试官的节奏。

⚠️ **注意：**评测的相关问题，我在讲解视频和答案，就会**以我们的ForgePilot 为例**，来进行回答。大家如果使用自己的项目，也是这样，把这些问题**换成你自己的项目的评测部分**，检验你能否回答的上来。不要因为是问的别人的项目就觉得这次面试参考意义不大。相反，**Agent评测几乎是我们Agent项目都应该准备的部分**，这次面试的内容也很好的帮助了我们复习了评测的理论基础，并且检验我们对自己项目的Agent评测部分能否有一个很好的回答。

后端的内容我就不讲了，我倾向于认为这是因为同学简历写了后端的内容，面试官才问的。如果这个岗位不是做后端的东西，或者简历上没有涉及后端，是一个纯大模型的岗，那面试官可能不会问后端内容。

一、个人背景与重点经历选择

1. 主问题： 请你简单介绍一下自己。

2. 主问题： 你的实习和个人项目比较多，这四个经历里，挑一个收获比较大、自己做过较多决策的，重点讲一下。

> 💡 **点评：**面试官问项目的时候，一种会自己直接从简历里挑项目去问，一种是让你主动选择拿手的项目去讲，这两种思路都挺常见的。面试官自己挑项目的时候，也通常会问简历里最重点的内容。
>
> 一般我们写简历的时候也有侧重，重点的项目写的篇幅会多。从这点也告诉我们，对于**重点项目，我们面试前应该充分准备**，这个不管是面试官主动问，还是自己讲，都是聊得最多的。

二、项目背景与个人参与情况

> 💡 **点评：**具体事实的澄清，第二个问题问**项目负责职责**，这是**对于团队协作的项目，一定要想清楚**的问题。

1. 主问题： 我确认一下，这是 Shopify 的远程实习，还是第三方开发者的远程实习？

2. 主问题： 这条从生成到评测的链路是你一个人做的，还是有其他远程伙伴一起做？

三、生成与评估链路的设计、问题定位与解决

1. 主问题： 讲一下生成部分，你们的目标是什么，参考了哪些东西？

↳ **追问1：** 结合后面的评估，评估维度是怎么选的？

> 💡 **点评：**前面两个问题，问评估的方案。应该是我们早已准备过的。首先Agent评估应该评估哪些维度，如何评估，结合这个理论知识来准备，**自己的Agent评估系统怎么做的。**这个问题的**理论基础参考**：Agent评测部分。 我们ForegePilot项目有准备如何评测，这个是参考答案示例。

↳ **追问2：** 过程中遇到了什么问题?

↳ **追问3：** 怎么定位问题发生在链路的哪个阶段?

↳ **追问4：** 最后是怎么解决的?

> 💡 **点评：**通常这类遇到什么问题，如何解决。 我是都建议大家**提早准备**的。即使你真的做过了项目，不提前准备，面试突然遇到这个问题可能都不好回答，更不要说你是包装的，铁定答不出来。 其实我们在准备我们ForgePilot项目面试前突击，就准备了一下这个问题。当时准备的思路是**线上遇到了问题，如何定位，然后如何解决**的。和这个面试官追问的2,3,4 基本同一个思路。 参考答案可以看这里。

四、确定性规则与大模型评测的分工

> 💡 **点评：**这个题目围绕着评估的方法展开讨论。一方面考察项目细节的澄清：面试官问**什么时候用规则评分，什么时候用大模型评分。** 背后的理论基础我们在AGent评测和Harness 思想其实都讲过：**确定性的东西用规则（代码）来做，对于开放性的，需要借助大模型推理的东西，再用模型来做。**
>
> 这题题目的一些理论基础：Agent评测不同方法的选型。

1. 主问题： 你说用规则结合大模型评测结果，具体哪些用规则？为什么不能全部用模型?

↳ **追问1：** 你说的确定性规则，具体是什么?

↳ **追问2：** 生成描述有关键词要求？这些关键词从哪来?

↳ **追问3：** 关键词覆盖是一条规则，还有别的吗?

↳ **追问4：** 也就是说，重复语句之类的问题用模型；真正用规则的主要是硬要求，比如 SEO 关键词必须出现、长度必须符合要求，其他内容交给模型，对吧?

五、Prompt 迭代的依据与评测前置工作

> 💡 **点评：**这个问题主要围绕**黄金数据集/评测集**这个理论来展开的。我们在Agent评测中讲了这个内容，就是开发的过程中，我们需要**建立一套评估标准和数据集**，开发的时候**通过这个系统来指导开发。** 包括在我们的算法项目中，也就是贯彻了这个思想。 如果你看了我们这两部分，你应该很能理解面试官的意思。
>
> 这个同学在面试的时候，**没有这个黄金数据集的思路**，所以你可以看到下面的问题都是面试官在引导他把这个过程回答出来。如果大家看了我们的笔记，应该最开始就能答出来这个思想和概念。下面这些问题答案就不展示了，下面的问题主要围绕这个思想在讨论。更重要的是学会这个背后的理论基础和思想。

1. 主问题： 你的 prompt 应该不是一次成型的。整个系统向 81.7 分逼近时，应该经过很多过程。为了让评测更科学，前置有哪些必须做的关键步骤?

↳ **追问1：** 你刚才讲的交付后遇到脏数据，已经是一次迭代后的事情。在那之前，你做了什么？

2. 主问题： 你的评测集怎么构建？应该有一个标准评测集，或者类似黄金集的东西，让你在上面逐层迭代。

↳ **追问1：** 你只做 prompt，不涉及模型后训练，那么调 prompt 的依据是什么？

↳ **追问2：** 是凭感性感知到处修补，还是基于评测集？

↳ **追问3：** 这个评测集从哪来？

六、评测体系本身的有效性、样本分布与边界覆盖

> 💡 **点评：**这里其实就涉及了两个理论知识： **如何判断你的评估方法是准确的？如何构建标准的评估数据集？**这两个理论部分。我把这两个理论点**加入了Agent评测笔记之中**了。包括笔记这两处的笔记，也总结了对应的面试小卡片，对于这个问题大家把这两个笔记消化好就行了。

1. 主问题： 我指的是评测，不是生成。规则加 `LLM-as-a-Judge` 的架构出来以后，总要验证这套评测体系是好还是坏。尤其是 LLM 评价相关的部分，基准集合从哪来，有什么要求？

↳ **追问1：** 这套评测体系完成后，对谁评？81.7 分是对谁评的？

↳ **追问2：** 你说对 640 条真实商品描述评分，这 640 条从哪来，分布有什么要求？

↳ **追问3：** 我问的不是分数档位。我问的是这 640 条真实记录从哪来、分布有什么要求。

↳ **追问4：** `LLM-as-a-Judge` 架构写出来后，你也要评估这个评测体系本身是好是坏。应该有一个黄金集，在上面逐步迭代，比如准确率从 60 分或 70 分提高到多少。没有这个过程吗？

2. 主问题： 只有好的样本够吗？差的样本从哪来？

↳ **追问1：** 最后用的是 `LLM-as-a-Judge`，只评好的样本能评出来吗？

↳ **追问2：** 你挑的边界 case 是从哪来的？

↳ **追问3：** 你评测时只针对好的样本吗？

💡 **点评：第七点和第八点属于后端内容**，和同学简历项目强相关，和大模型岗位又弱相关，如我开头所说，**这里就不具体解析了。**

七、并发配额扣减的方案选择与技术取舍

1. 主问题： 你在项目里用悲观行锁消除并发扣减问题。这个问题还有其他解法，当时考虑过其他方案吗？为什么最后选悲观锁？

↳ **追问1：** 背景我听懂了。我问的是为什么不用其他方案，为什么用悲观行锁？

八、高 QPS 场景下的系统扩展能力

1. 主问题： 如果场景变大、QPS 更高，比如让你给 OpenAI 或 Anthropic 做网关，要怎么处理配额扣减？

↳ **追问1：** 除了当前的悲观行锁方案，还能想到其他方案吗？

九、反问环节

1. 主问题： 我这边了解得差不多了，你有什么问题要问我吗？

讲解视频

一面

https://www.bilibili.com/video/BV15Yhn6FEeR/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

二面

https://www.bilibili.com/video/BV12wpd6ZEDQ/?spm_id_from=333.1387.homepage.video_card.click

---

2026.10 字节Agent三轮面经

前言

此次面经来自于互联网，我偶尔刷到了同学分享的面经，我觉得很多内容都比较有代表性，比如对于Pi-agent，DSH的考察，我当时还给作者发消息，我说你能不能把录音内容分享给我，我想总结一下，结果被拒绝了。 不过同学还是给了我他自己记录的可能不那么完整的面经。所以本次分析，我可能无法结合具体项目，背景，上下文来分析面试官的意图，不过从知识点的角度上来进行解析，我觉得对我们来说收获也很大。本期的很多内容其实我们笔记里讲过，可以把它当成一个试卷，来重新巩固我们已经讲学过的知识点。

一面

1. 实习项目与架构改进

1. 围绕实习项目展开深挖。
2. 如果要重构你实习中的 Agent，你想怎么做？

2. 会话记忆：滑动窗口与摘要压缩

主问题：企业 AI 助手项目中，基于滑动窗口和摘要压缩的会话记忆是怎么实现的？

追问：

1. 滑动窗口大小是怎么确定的？大概设置多少轮？
2. 哪些场景适合滑动窗口？
3. 哪些场景更适合直接通过摘要进行上下文压缩？

解析见：滑动窗口与摘要压缩。

3. 文档自动更新：并发、幂等性与 MQ

主问题：文档在线自动更新时，如何保证并发更新的幂等性，避免多个任务相互覆盖？

追问：

1. RocketMQ 在文档自动更新链路中的哪个位置?
2. 为什么文档更新这里考虑使用 MQ，而不是直接在定时任务中处理?

解析见：文档更新：并发与幂等。

4. Agent / Agent Harness 的理解与选型

主问题：你对业界其他 Agent / Agent Harness 有哪些了解?

追问：

1. 展开介绍一下你了解的 DeepSeek Harness 和 Pi Agent。
2. DeepSeek Harness 和 Pi 有什么区别?
3. 如果把这些 Harness 和你实习中的 Agent 项目结合，你会怎么做?
4. 是否有必要替换当前方案?

解析见：Pi Agent真题解析；DeepSeek Harness与Pi对比。

5. 其他 Agent 产品与 Claude Code

1. 除了开源 Agent，你是否调研或使用过其他厂商的 Agent 产品?
2. 介绍一下你了解的 Claude Code 上下文压缩机制。

解析见：其他厂商Agent产品；Claude Code上下文压缩。

6. Computer Use：实现机制与运行环境

主问题：介绍一下你对 Computer Use 的了解情况。

追问：

1. Computer Use 操作本地电脑的大致实现机制是什么?
2. 是否了解操作本地电脑和操作云端电脑两种方案?
3. 有没有了解过 Computer Use 在云电脑中的应用?

这个问题也是有专题的Computer Use笔记来讲解的，这里就不赘述。

解析见：Computer Use实现机制与运行环境。

7. AI Coding：实际问题与开发工作流

1. 平时使用 AI Coding 工具开发时，遇到过哪些问题?
2. 你平时使用 AI Coding 工具的完整开发工作流是什么?
3. 有没有更 AI Native 的做法?

解析见：SDD开发工作流；AI Coding面试真题。

8. 基础知识：数据库与 MySQL 索引

数据库类型：

1. 常见数据库有哪些类型？
2. 不同类型数据库之间有什么区别？

MySQL 索引：

1. MySQL 底层索引使用什么数据结构？
2. 为什么 MySQL 索引使用 B+ 树，而不是其他树结构？

解析见：数据库类型与MySQL索引。

9. 算法题一：二叉树的子结构

题目： 判断二叉树 B 是否为二叉树 A 的子结构。

作答要求：

1. 先描述解题思路。
2. 再进行代码实现。

10. 算法题二：两个有序 List 的相同元素

主问题：求两个有序 List 的相同元素。

追问：

1. 分析当前实现的时间复杂度。
2. 如果两个 List 变成无法整体排序的无限序列，应该如何处理？
3. 使用 Bitmap 时，如果元素值范围非常大，会有什么问题？
4. 是否可以将两个无限序列拆成局部、分批处理？

11. 候选人反问

1. 团队 Agent 目前主要负责什么垂类业务方向？
2. 团队内部是否比较鼓励使用 AI Coding？

12. 一面反馈

- 作者表示，面试中讨论了 Computer Use、Pi Agent、DeepSeek Harness 等内容，个人感觉较稳，庆幸自己跟上时代步伐。
- 作者记录，一面结束后，当天下午已约二面。

二面

本轮主要围绕个人项目展开，详细内容集中在 RAG 系统的实现与工程取舍。

1. RAG 系统整体流程

1. 介绍一下在学校做的 RAG 系统。

2. 整体流程是怎样的?

2. Markdown 文档分块

主问题：Markdown 文档的分块逻辑是怎样的?

追问：

1. 是否先按照标题结构划分模块，再将模块拆成小 chunk?
2. 检索时，是否先召回小 chunk，再补充对应的父模块?
3. 分块后与前面的内容进行聚合，具体是怎么实现的?
4. 相邻块之间是否会存在重合?
5. 如果一段内容有 1,000 个 token，chunk size 为 400，最后剩余的 200 如何处理?
6. 是否会向前补充内容，将最后一块凑成 400 个 token?
7. 大部分 chunk 是否已经接近设定大小，因此实际很少触发向前补充上下文的逻辑?
8. 分块大小、聚合时的 chunk size 分别设置为多少?

3. Markdown 图片及其他内容处理

主问题：Markdown 中的图片、`ul` 等内容是如何处理的?

追问：

1. 是否每张图片都需要进行一次模型推理?
2. 这是否会导致文档保存、入库的耗时过长?

4. Embedding 与向量数据库选型

1. 项目使用的 Embedding 模型是什么?
2. 项目使用的向量数据库是什么?

5. 用户问题改写与子问题拆分

主问题：用户问题改写是如何实现的?

追问：

1. 能否举例说明改写逻辑，以及改写前后的问题分别是什么？例如用户问“知识库里有哪些知识”，或者结合项目中的电商场景说明。
2. 子问题是如何拆分的?
3. 拆分的目的，是否是让每次检索只围绕一个问题，召回与该问题相关的知识?
4. 问题改写和子问题拆分，是在一次模型推理中完成，还是需要多次调用模型?
5. 这一步模型推理无法直接向用户流式输出答案，是否会导致用户等待答案的时间过长?

6. 混合检索与重排

1. 混合检索是怎么实现的?

2. 检索后的重排是怎么做的?

7. 企业级项目与 Demo 的区别

1. 怎么判断一个 Agent 项目是不是企业级?
2. 简历中的“企业级 RAG”应该如何理解?
3. 企业级 RAG 与学习型 Demo 有哪些不同?

8. 实习经历深挖

- 实习经历深挖。

9. 算法题：使用 random5() 实现 random7()

题目要求：

- 已知 `random5()` 等概率返回 `0～4` 的整数。
- 仅使用该随机函数，实现 `random7()`。
- `random7()` 需要等概率返回 `0～6` 的整数。
- 不得使用其他随机数生成方法。

10. 候选人反问

1. 对于 Agent 开发方向，校招生需要具备哪些能力？有哪些要求?

三面

1. 实习经历与产出深挖

- 作者描述，本轮全程围绕实习经历展开，面试官对其实习产出提出了大量质疑，作者认为自己的产出几乎被全盘否定。

2. 场景题：从 Nginx 日志排查高频访问 IP

已知条件

- 有一份 Nginx 日志。
- 每行第一列都是 IP 地址。

任务：

- 找出访问频率最高的几个 IP。
- 用于排查是否存在攻击。

限制：

- 不能写脚本。
- 使用基础 Linux 操作和 Nginx 操作完成。

3. 算法题：最大数

- 将一组正整数重新排列，拼接成最大的整数。
- 对应作者标注：**力扣第 179 题。**

讲解视频

一面：https://www.bilibili.com/video/BV1JZxM6dExw/?vd_source=144bec9c3f54e465073138bed788be1b#reply322259121600

---

10 上岸反馈：下一个就是你

哈哈，这里是一个小小的彩蛋环节，恭喜聪明的你发现了这个彩蛋。

这个章节成立于2026年4月26日。起因是最近其实有太多太多的人和我反馈他们的上岸信息，还有很多人专门和我表示感谢，我通常会在小红书里专门宣传一下。其实最早几个月前就有人第一次和我反馈，是一个朋友突然发的一个私信，专门说特别感谢，他已经上岸了，还把他的简历发给了我，说你如果有需要，你可以参考我的简历里的rag项目的描述。我隔着屏幕也能感受到他的喜悦。不管是私信给我，给客服，还是在交流群里反映上岸的同学太多了，以至于我现在很多都忘记了。但是我觉得这个时刻和瞬间值得被记录！ 也希望这个还正在奋斗的同学，看到这个笔记能有更多的鼓舞。

我每次打开笔记，基本每次都能看到很多人同时在线，我觉得大家都很不容易。不管是校招，社招，最后的offer都是我们努力的成果。其中的迷茫，困惑和坚持只有我们自己知道。其实我也是一样的，25年大概5月的时候，我最开始转行大模型，从转行日志中大家能看到我的迷茫，每天的坚持。即使到现在，虽然我现在不着急找工作，但本质上来说，我和大家一样，我也是在不断地奋斗，未来有一天我很可能也会找工作，在这个不管是公司还是时代时刻发生快速变化的时代，我也是在为了今后可能的变化准备着。将来的某一天（甚至这一天不会很久），我可能也要面临着找工作的压力。我和大家一样，都是在这个路上不断努力，追求转行拥抱大模型的人。

我希望这个章节，记录着我知道的同学背后的上岸故事。每一个故事背后，都蕴含着他们的努力。我希望这个记录可以激励着你，激励着我，一起继续努力下去。希望下一个上岸的是你，是我。但我非常确定的是，努力一定能有回报。而且我非常确定的是只要你学会了这份笔记，持续学习，你绝对可以上岸。大家都在群里，有太多其他同学的例子都能证明这一点。大家一起加油吧。

不过这个章节最早写于2026年4月26日，从这个日期以后的记录，我会比较详细记录。之前的很多记录，就要靠我回忆了。最后放心，我会保护大家隐私的，所有记录都会隐藏小红书全名，也不会透露特别细节的个人隐私。

2026/06/30 我要逼着自己写一波这个专题了，这些天其实有很多其他反馈我都忘了。但是我觉得这个专题成立的意义就是为了及时记录，我逼着自己再写一波最近了解的情况。时间我就不记录详细，反正是近一周的情况吧。

1. 一个小姐姐主动留言，说她之前gap了半年。然后找到了工作，她开心，专门和我表示感谢，我还发现商品评价了她专门去评价了，特感谢。后来她又告诉我她的情况，她和我有点像，在国内大厂然后又去了其他公司还当了tl。后面裸辞了。最后去了外企。她是立志于反996，她说很难找到不加班的工作，不过最后还是找到了。哎，不加班的工作还得是外企啊，国内的公司就不能不加班吗？因为她和我的经历可能有点像，估计我的经历不管是知识还是经历给了她一些启发吧。

2.一个硕士老哥。跟我说他拿了3个offer，应届毕业生，26年毕业。我去他的小红书看了一下哈（哈哈，有时候大家留言，我都偷偷点进去看了看大家主页），学校是普通的一本的硕士。我倒觉得他的选择还挺好的，三个offer有两个是央企，都是那种不太累的，有一个还给北京户口。他的选择倒不是那种和我们很多人一样的，去互联网公司卷的那种。不知道他怎么投递的，没有多聊哈，也许是想着wlb，希望他能过上wlb的生活。

3.一个我们公司的老哥，也是在我的评论区留言，想和我交流，后面和我聊了一下。都是同事嘛，工作比我少一年，4年经验，跳出去2个top3的大厂，一个AI 六小龙。这个同事我之前小红书笔记也提了，字节给了他L3，真的是很高的评价。不过他挺厉害的，他说他下班了以后和别人在做项目，差点被收购了，面试也是用的这个项目（本身他应该工作也是做传统后端的，不是Agent的）。从他的例子我觉得是社招，其实不在乎说是你一定要工作是做AI的，你自己做项目，只要有深度，有影响，甚至自己做也可以上线嘛，这些一样都是可以和面试官聊的。所以不要小看我们自学的人啊。

2026/08/26 更新：最近都懒得记这一部分了，虽然经常有人陆续给我反馈他上岸的信息，大家看我小红书笔记图片里有，我就是懒，忘记记这一部分了，但是还是有很多人上岸的，努力学一定可以成功的。

2026/06/01 有个同学，之前第一次面试，问我要注意啥。结果第一次面试就过了，是个小厂，不过是个后训练岗。他也没啥后训练经验，他说就是按照笔记里那一套，说如何做量化部署，训练的说辞和面试官说的。

2026/05/23 有个朋友报喜，是没有实习经历，秋招不满意，春招继续在找，最后春招有了一个大厂offer，他说挺满意的，最少是薪资和平台是其他里面最高的。这也告诉我们，春招不要放弃啊，虽然春招机会少，但不代表没有。

2026/05/22 有一个朋友，最近咨询我暑期实习的offer。我看我一开始和他的聊天记录，在3月左右，他和我咨询要不要报班啥的，后来买了我的笔记，他还在群里特别活跃。他两个项目都是用的笔记里的，RAG和Agent。最近上岸了，来和我咨询暑期实习，他自己是非常满意，有几个offer。他的背景还是不错的 211本+海外顶尖的学校，今天也和我说收到了腾讯的面试。我也很感谢他的一点，他主动给我了好几个面经，我后面也会总结。（最近有很多人也给我面经，很感谢大家）

2026/05/19 有一个字节实习校招的朋友，后台私信我，他已经在字节实习Agent岗位了，问我秋招怎么包装一下。我看他在会员群2群，也就是大概在26年2月左右买的笔记，我就也把他算在这里吧。感觉他好优秀，本身之前有一些大模型基础，也是很好的工科985硕士，我看他小红书研一就参加什么开源项目。开始准备的也早，他已经在字节实习了，我觉得按照他的经历，秋招投递简历应该没有哪个厂没有面试机会的。不过他还私信我问我怎么包装，要不要学算法，计划着冲一下算法。 这个老哥真的挺厉害的。

2026/5/8 有一个26岁的小姐姐，咨询我阿里的agent岗位，问我建议，还问了我对AI程序员的冲击和看法。这个小姐姐之前是国企的。 今天这两个例子其实让我看到了一些不一样的可能性，就是高校老师+裸辞经历的

转行，以及国企背景 转行大模型的可能性。

2026/5/8 今天有个博士朋友咨询我谈薪的事，他是学历很好，毕业后高校当老师，然后裸辞了，gap了几个月，目前找到了大厂offer，咨询谈薪技巧。

2026/4/30 有个同学私信我，说达到了阿里国际的OC，两个部门问我怎么选。这个同学之前问过RAG项目的问题，他也是拿笔记里的RAG项目去面试了。就是上面RAG项目真题部分，快手一面素材的提供者。

2026/4/27 有个同学上岸了蚂蚁的暑期实习，说现在正在蚂蚁实习呢，问我说现在的组主要让他做SKILL，感觉和Agent无关，问我要不要继续学Agent冲秋招。我给了他一些建议。

2026/4/26. 小红书名: 小红薯xx的id的同学，晚上专门发给我私信（下面是原话）：他说买了笔记，已经成功转型大模型并拿到offer啦。已经在新公司上班，真的很感谢你。当时正好被裁员了看到你的帖子，你的经历鼓励了我往这方面转型。后面买了你的笔记也反复看了好多遍。希望博主越来越好\~ Best Wishes

---

这个下面就是2026年4月26日以前的记录了，得靠我回忆了。

1. 大家看项目部分，面试真题的，3.3， 3.4，3.5. 是一个粉丝朋友投稿的，她是校招985的，拿了很多offer，最后去了字节。她私信投稿了几次面试记录。最后告诉我上岸了，还让我帮他给offer建议。
2. 有一个女生博主，她有大几千粉丝。有一天突然有个人@我，我点进去一看，是一个很漂亮的小姐姐，发视频说她裸辞一个月上岸大模型，推荐了我的笔记。更巧的是，第二天，我的研究生同学的女朋友微信和我说，我看到我的师妹发小红书推荐博主笔记，我点进去一看，那个博主不是你吗？世界真的太小了！
3. 有一个普通一本的同学，去年12 月30号给我发消息，说被裁了，而且是应届生，特别沮丧。我当时特别共情他。而且那时候刚好我在休元旦假期呢，我在海边收到了他的消息，印象特别深。我当时就鼓励了他，给了一些意见。最近他给我发消息，是找到了offer，虽然没有大厂的offer，但是还是满足了，最少做上大模型了，他说当时要是没有我的笔记，他害怕连普通开发都找不到，现在做上大模型，已经很满意了。不过也问我大厂需要什么背景。我的建议是慢慢来，背景不好，先做上大模型，积累经验，再到中厂，大厂。持续努力一定行。确实大厂基本都是好的985硕，但是不代表其他同学没机会。社招一步步来，曲线救国就是学历差一点的同学进入大厂的好方法。
4. 有一个同学在群里的反馈，因为和上面这个同学类似，都是学历不好。不过他在群里说还是很满意了，毕竟做上大模型了。
5. 小红书有一个朋友，第一次私信是说简历投了好多家，没人理她，问我咋办。我说也没办法啊，改简历，针对性投，找人内推，不放弃。 后面她找到实习了，是个Agent开发岗位，经常偶尔来问我一些实习的问题。
6. 有一个很好心的大哥，在交流群里分享他上岸了，夸了我的笔记，他还一直在群里给其他同学意见。很感谢他。
7. 还有很多就是直接在群里分享上岸的offer的，我就不去翻了。大家应该在群里可以看到。

算法理论与项目【可选进阶学习路线】

这一章呢，我想要记录的是我0算法基础，开始学习大模型算法的全记录。其实大家看的目前笔记的其他部分，是我在摸索出来一条路线以后，给大家不断总结和优化的。但是这个记录，是我在还没有开始系统学习算法的时候，记录的。按照之前的路线，我的主攻是大模型应用，学了Agent, RAG, 做了个项目，算法我是带了一点点，就是理解了概念，走通了流程。 但是真的大模型如何训练，怎么弄一个项目，具体的参数和公式，我是不清楚的，我现在这个情况没太大把握面试算法岗。这一章的笔记是在我一边学习算法，一边记录的，好处是你可以看到我整个学习过程的记录，我会每天做了什么，学了什么，心态和感受是什么，我会如实记录。好处是，以后你们在学习大模型算法的时候，一定会有很多和我一样的困惑，买什么卡，学什么，项目怎么找。我把这些如实记录，我相信后来的人看到后，能够找到，复刻我的来时路。 另外因为我是实时记录，这个路上我学了什么视频，参考资料，怎么做的项目，这些都不会遗漏。

好了，虽说是0基础，但准确说，是0.1基础（我大概了解过一些深度学习，机器学习是啥，我知道具体的大模型部署，训练到底是干啥，知道LoRA, 强化学习概念是啥）。 好吧，如果你这些不知道，我会给你推荐一个一周入门深度学习，机器学习的视频。对应的概念部分，就看我们笔记的其他章节，微调，部署，都有一些概念的理解，我希望你能清楚这些到底是啥东西？ 先把这些准备好，然后我们就上车吧！本节的记录标号，表示学习顺序。

最后说一下学习这个环节硬件的问题，我的意思是先学习，先用我的cpu电脑，以及游戏笔记本，实在需要更好的硬件的时候，再看看是租还是怎么着，总之先学着。先把基础打好，硬件不是瓶颈，知识才是。

1 学习路线确定

> 第一天根据实际情况，确定学习路线。每个基础不一样，大家根据情况自己辨别。本节我介绍学习路线，以及我针对自己情况做的学习计划和建议。
>
> 学习大模型算法分三条路：数学+传统技术+大模型技术。

算法包含：**微积分，概率论，线性代数，高等数学。**

传统技术：**git, linux,python pytorch, 深度学习。** 注意啊，对于不是特别高端的算法岗，算法不是拉开差距的核心。 所以最开始不用啃得特别细致，甚至去做高等数学题目，但是需要有个印象。比如讲到深度学习，求偏导是啥，得有个概念。

大模型：transformer, RAG, Agent 之类的。

我的思路是，毕竟是社招，还是以项目和面试为导向。不可能再像学生那样系统的学习。对于上述提到的点，如果哪里完全不清楚，请去找快速入门视频，把最关键的地方过完。比如，我用一周看完的深度学习入门（当然前提我之前有一点点印象和对深度学习的概念）：

然后我现在对于Pytorch不熟，我得去学习快速入门视频。 至于git, python,linux 都是老程序员了，不用学了，然后大模型这块，因为之前我们已经按照应用走了一遍，相关概念有了，直接实操。

综上，我个人的路线是：先学pytorch，跑一些手写分类啊，Dropout的例子，应该很快，因为我之前上学有一些概念，这里再打一下理论基础，唤醒之前的记忆。然后主要关注在大模型行业的训练，微调，找一些项目试一下，再实践一些复杂的例子，比如动手写出transformer，调试之类的，上手到模型的DPO, PPO，最后做一个项目，读一些论文，然后面试试一下，再根据面试反馈不断迭代。（其实把面试当作学习过程也是一个好的方法，而且很高效）

这是我的路线，社招以实战+最快面试为主。 对于学生来说，你可以花更多时间学基础知识。对于其他社招的程序员来说，其实你也应该看到，大模型算法这块门槛会高一些，对于深度学习，机器学习有难度。

对于大家来说呢

1. 社招，之前做开发的，建议先去学大模型应用。要学的少一些。如果学习能力特别强，时间多，包括在学校学的专业课，比如深度学习，数学都学得比较好，你可以激进一点，就直接目标定位大模型算法。我自己本身是先学的应用，再学算法，学应用也学了大半年。如果你的基础比较差，比如对于深度学习，机器学习都没概念，甚至是上学没学过，可能是本科生，对于数学，微积分那些也都忘光了。还是建议慎重直接挑战大模型算法的。
2. 社招，之前做算法的。那其实你和转到大模型算法之间的gap并不大。数学，传统技术都是你的对口内容，只用补齐大模型相关知识。值得一提的是即使做算法也需要补一下Agent,rag，可以去看看我笔记其它内容。然后参考这一小节，我后面如何做一个项目的。当然算法大哥毕竟是专业的，可能不需要参考。
3. 校招生，如果你马上着急找工作，根据自己情况觉得是找一个大模型应用项目更容易，还是算法容易，然后弄个项目刷实习，不断学习，比如从3月学到7月找暑期，找到暑期继续学习找秋招，再学习到明年3月甚至能找春招。也有一年时间，尽量找大厂实习，本质是刷简历，能去算法去算法，能去应用先进入应用也可以，毕竟年轻，后面有意识，转算法也容易。
4. 校招生，不着急找工作的。理论知识先学好吧，学习得更细致，比如深度学习的课程之类的，到了快找实习的阶段，参考上面几条，准备项目，刷实习，找工作。

1.1 参考视频

该视频讲解了大模型算法学习路线（10分钟）。只讲大的学习方向，没讲具体技术。

https://www.bilibili.com/video/BV1nNkFBMEDS/?spm_id_from=333.337.search-card.all.click&vd_source=144bec9c3f54e465073138bed788be1b

从技术角度拆分哪些是重点，比如预训练用得比较少啊，SFT更重点呀，哪些论文要学习啊，数据工程之类的。（19分钟）

https://www.bilibili.com/video/BV1ix6UBcEp2?spm_id_from=333.788.videopod.sections&vd_source=144bec9c3f54e465073138bed788be1b

快速入门深度学习，机器学习：

https://www.bilibili.com/video/BV1eP411w7Re/?spm_id_from=333.337.search-card.all.click&vd_source=144bec9c3f54e465073138bed788be1b

我根据我的情况，制定了以下学习路线：

我们的学习目标是为了快速完成一个项目

1. **Pytorch/深度学习入门**：学习掌握Pytorch/深度学习的基本知识，从张量，到梯度，前向/反向传播等经典概念，到网络模型CNN,RNN再到Transformer,GPT。了解基本概念，Pytorch用法，训练，模型推理的代码。这里把Pytorch和深度学习放一起是因为本身Pytorch就是去搭建深度学习的，所以在整个课程中，学习的时候就会涉及到深度学习的基本概念，所以这两个一起学，学习方法看第2点。
2. **后训练学习:以SFT + GRPO展开**，也就是现在面试最常考的内容，深入代码细节和理论，学习算法数理公式，为后续的项目做准备。
3. **开源项目/论文阅读**：有了理论基础，看一下典型的开源项目和实现，做技术调研和选型，为后面做项目做准备。
4. **小项目**：确定一个方向，比如提升模型数学推理能力，然后使用SFT + GRPO 进行训练，准备测试集，跑测试结果，把完整规范的训练，测评，调试，迭代的逻辑走一遍，完成代码，写一个小论文（不是发布的那种，类似结题报告，说明原理，做的事，逻辑，参考论文）。
5. **包装项目，面试+迭代。**

这块我也做了一个图，这个图是笔记里三个SKILL对应的课程内容。覆盖了后训练的基本的理论。我三个阶段已经学完，全程就是用SKILL跟着AI对话，提问，复习。学完了以后，对整个后训练的核心概念会有一个理解，具备了做算法项目的理论知识。

2 Pytorch/深度学习入门

我的学习方法是使用了一个SKILL作为老师，制定了课程的教纲，一共有16课，包含4次考试，每次课程结束也会有5个测试题，你只要使用这个SKILL，告诉AI开始学习Pytorch，触发它就能一步步完成。完成后你会了解：深度学习基本概念，如何使用Pytorch训练，搭建网络，整理数据，attention, transformer gpt的原理等。

我想谈一下使用该SKILL注意事项和说明：

1. AI的文字表达能力和讲解能力是非常不错的，比很多书籍和文章讲得更清楚，并且还能实时提问。在学习的过程中要思考和AI交流，提问。根据AI和你说的知识点，把它想清楚。
2. 不要完全依赖于这个SKILL，这是一个必须学习的路线和一个非常好的老师。但是如果这个老师和你讲知识点的时候，某个点你就是理解不了，可以反复和它提问，查找对应的视频教程。比如你就是理解不了张量，那么可以b站,youtube搜一下讲张量的视频。也许通过视频的形式你会豁然开朗。
3. 基础不好的，学起来吃力的，可以学慢一点，或者反复两遍。跟着AI多找一下其它的资料，视频去理解，多提问。要是真的能掌握这个课程的内容，那就是入门了深度学习和Pytorch，这是算法必经之路，也没

有那么容易。相较而言，这套路线其实已经是快速入门，面向着找工作为目标的精简版本了。

2.1 视频讲解

这个视频讲了我学习Pytorch和设计学习路线的思路

https://www.bilibili.com/video/BV1MYwvzvEFL/?vd_source=144bec9c3f54e465073138bed788be1b

3 后训练学习

这一阶段讲解了后训练相关内容，什么是后训练？就是SFT（LORA,QLORA), 强化学习(PPO, GRPO) 这些东西。想必大家看到这个东西应该也能感受到这个部分在面试中经常出现吧。

在我们有了一阶段的深度学习的概念，以及Pytorch的基本使用方法的经验时。我就可以开始学习这个阶段，这个阶段主要讲清楚：

强化学习：从它的前置知识（KL, 奖励函数）到PPO, 到GRPO，理解起来是有一定难度的。

SFT： 相关SFT训练的库，数据集准备，超参数设置。

训练技巧：混合精度训练。 数据类型（fp32, fp16,bf16)

最后结合DeepSeekR1的训练技巧，把内容串起来。

这些东西我很肯定地说，都很重要，面试也很常考。大家看面经可以看到 数据类型（fp32, fp16,bp 16) 这个网龙面试问过。 DeepSeekR1的训练技巧 也在面试出现过。这些内容非常重要，也是我们后面自己做一个项目的核心。

不过内容理解起来有一些难度，我也设置了很多技巧，比如SKILL设置了不同讲课风格，大家可以由浅到深。每个小节设置了md复习文本，SKILL可以带你复习等各种策略，大家可以看SKILL的README。 虽然很难，我相信大家耐心一点，用SKILL 多刷几遍，绝对可以学会的。我认为这已经是比看书，看视频，看文章高效多了的方法。大家注意思考，多和AI聊天，提问题。

最后，我并没有把相关八股整理到笔记中，比如什么是GRPO, RLHF的流程。虽然我个人认为这些是很重要的。但是，我希望所有的该笔记中的内容都是以面试为导向的，我必须亲自去面试，了解算法面试的八股，再根据思路，策略去总结，我希望笔记的内容都是面试的内容。所以算法的八股，会等到我真正做完项目去面试，再来根据重点整理。不过大家使用这个SKILL的时候，里面涉及各种八股，大家在学习的时候，可以多看看，背背。

SKILL已经整理到下方的资源合集中，和pytorch学习SKILL放在一起了，有需要就学起来吧。

3.1 视频讲解

这个视频讲了我学习后训练，和使用这个SKILL的思路

https://www.bilibili.com/video/BV1Wp9zBJE6r/?vd_source=144bec9c3f54e465073138bed788be1b

4 开源项目学习

前两个阶段补齐了理论基础，这个阶段偏向于来学习具体训练时对应的库和开源项目。课程主要分为：

1. 学习库TRL的基本使用方法和代码，这个大概率是我们后面做算法项目，使用的封装好的做训练的库。
2. 学习OpenRL和SimpleRL的开源项目。体会真实的算法项目，如何组织文件结构，分成哪几部分，我们后面项目也会借鉴它们的组织结构。
3. 最后两小节，AI会和你对话，询问你项目的需求，帮你一起发散思维，思考你的项目是什么，大概参考那些模板，给你一些启发。

这个阶段会讲到源码，不过时间对源码不用看的特别深。重点内容还是知道**TRL库是什么，怎么用，开源项目怎么组织目录结构，思考如何复用/借鉴开源项目。**

这个阶段学完了，我就开始做自己的算法项目了。

5 后训练算法项目

做这个项目，其实我的一大目的，就是真的希望所有人都能够自己去做一个算法项目。因为我知道算法项目本身天然从知识上就有一定的难度和门槛，同时硬件上也会劝退一些人。普通的开发，没有深度学习训练相关的内容的同学，很多人对算法项目是望而却步的。

我和大家一样，就是开发出身，一点算法基础都没有。所以我深刻知道大家的痛点。所以我从算法学习的第一天，我就知道我的目标，不光是自己做一个算法项目，而且我要记录整个过程如何学习的，心得是什么。包括我记录了非常详细的文档，包括卡如何租，我都是帮大家先走一遍，然后记录，希望大家能站在我的肩膀上学习。都能够做下来一个算法项目。

如果你时间短，你只是想用这个项目去包装简历，那么项目配备面试真题，相关SKILL（就像我们之前的项目一样），相关文档，告诉你如何使用，包装，简历模板，包括讨论群方便讨论。

当然如果你想真的学习，我希望你能学习整个项目完成的技巧和方法。我相信文档记录得非常非常全，我是怎么完成的，学习这些方法，你自己做一遍，或者按照这个方法迁移到你的项目中，你的收获肯定更大。

注意该项目属于开发中，文档会逐步补齐。包括后续面试真题啊，简历如何写啊等等后面都会补。关注我的小红书算法专题，或者看我的git提交，你可以知道进度。

5.1 学习方法介绍

1. 首先，算法项目的理论阶段的学习，请看笔记 0基础学算法。 先把前三个阶段理论知识学习，**深度学习，训练，SFT和强化学习相关内容**，理论知识先学了。
2. 我的小红书（id:不转到大模型不改名），对应的0基础学习算法章节，有整个学习，做项目过程的完整心得介绍。可以当故事看，没多少，你知道我整个过程是怎么来的，中间遇到哪些问题和思考。

3. 先看技术文档。技术文档是整个项目的细节，要理解它。当然更推荐的方法是去学习整个技术文档如何做的，我在文档中也写了很多心得。因为我做项目，都是先写技术文档，逼迫你想清楚细节，是我做项目的方法论，我也希望你能学会这个写文档的方法，文档里有记录。 同时也有技术文档的讲解视频。
4. 在技术文档中，我将代码完工的整个流程分成了**7个阶段**。每个阶段我会提交一个commit，所以主分支上会有8个commit，初始（只有spec) - stage1 -stage2 -stage 3.. 我是建议你切换到每一个阶段的节点，来看当前阶段完成了什么。每个阶段的阶段性spec写在对应的**docs - spec目录下**。对应实施细节写在对应**docs-plan目录**下。（这个是superpowers生成的，可以主要看看specs，plans不太用看）。 每个阶段的完工代码，比如我怎么做的，这个阶段关键目标，你可以跑哪些指令，重点要掌握知识，写在对应的项目的**project-log对应的phasex下**。这个logs 告诉大家每个阶段的理论重点，需要关注的内容。避免大家无从下手。同时每个阶段配置讲解视频，我会逐步上传，补在这里。

1阶段： 主要是完成了项目的具体框架，串起来整个流程。这个阶段的学习重点：superpowers的方法论，以及llama.cpp本地部署的知识。详细文档见这里。 该阶段讲解视频在这里。

2阶段：完成了Agent的评估。这部分非常重要。即使你不做算法项目，这里也值得学习。Agent评估理论可以先看笔记这里。 掌握理论基础。整个作为实战例子。文档在这里。 讲解视频在这里。

3阶段：这阶段最重要的内容，就是创建了后训练 SFT和DPO的数据集，整个阶段数据集如何制作出来的，整个工作流是这个阶段需要学习的。文档在这里，讲解视频在这里。

4阶段：4阶段是真正训练了阶段，使用了SFT+DPO进行训练。训练使用AutoDL租卡训练，成本小于20元。文档在这里，讲解视频待上传。

5阶段：5阶段是把训练后的模型合并，量化，得到了最终的模型，同时做了模型测评和展示APP。可以说5阶段做完了以后，整个项目的所有的功能都已经做完，剩下的内容是优化，提升数据，写报告了。文档在这里，讲解视频在这里。

6阶段：6阶段进行了进一步模型训练，提升模型的性能。同时分析推理速度的瓶颈，寻找最佳的推理框架配置和参数。文档在这里，其中的每一个阶段性实验结论可以看这里。讲解视频在这里。

7阶段： 7阶段是项目的最后一阶段。我们写了一个最终的报告，如果你前面的阶段都不想看，只想拿着项目直接用，不想自己做，那么7阶段的报告可以说是必须看的内容，也是面试前的突击梳理，总结的都是最核心的内容。 这个报告的面试真题部分和简历部分，我们后续会出专门的简历编写示范和思路讲解，真题则是会根据面试真实情况补充，这两块内容算是一个初步版本，大家看一下思路就好。 文档在这里，讲解视频在这里。

5.2 项目地址

项目地址在这里

6 SKILL资源

https://github.com/jerry-ai-dev/SKILLS/tree/main

专题阅读与文档维护

在大模型这个行业，和我们传统开发不一样。即使是开发，也是需要读论文的，我从三个点来说明为什么：

1. **面试需要：**这点你从我的面试记录中可以看出来，很多公司是直接会问你最近读过什么论文的。这点在传统开发领域几乎不会出现，而在大模型方向，即使是应用岗，该问题出现的频率也很高。从清华大学出版社《大模型工程师面试 - 算法原理，开发实践与系统部署中》也讲了，如果以3个维度（刷题，论文和项目）来衡量大模型岗位准备的优先级。推荐顺序为：校招/初级算法岗：刷题>项目≈论文高级算法岗: 论文>项目≈刷题Agent/Rag 方向开发： 项目≈论文>刷题算法岗就不说了，肯定要看论文的。但是对于开发岗，书上明确将论文和项目放在了同一等级。 至于这个优先级排序，我觉得我们也不用较真，但是可以看出的是，书中明确将论文作为大模型开发方向的一个重要准备方向,和我的个人实践一样，确实是需要准备的。
2. **确实有用：** 大模型这个行业发展很快。即使是Agent，有人称2025年是Agent元年，现在2026年，Agent发展也不过一年。行业中以什么样的形式组织Agent，什么样的Agent框架最好，怎么评估Agent这些工程方面的东西，都还在研究和讨论。书和资料更多的是一些基础知识，但是论文上的知识，有的更加先进，代表未来的方向，会给我们不一样的启示。看了很多论文，对我们工程也确实有帮助，比如有的论文写如何选Agent架构，项目的Agents.md如何组织最好这些东西对我们工程最佳实践也有帮助。 另外一点是，在我们面试的时候，论文给了我们很多不一样的启发。比如Agent的架构，如果你看了谷歌的《Towards a Science of Scaling Agent System》，在回答Agent架构时，你会有不一样的思维，回答会更出彩。如果你看了《Are Repository-Level Context Files Helpful for Coding Agents?》，你在回答阿里的：有哪些写组织提示词的最佳技巧，会有新的启发。所以他对于我们面试问题，确实能带来实打实的帮助。我的亲身经历是，你在面试的时候，如果问题能结合论文回答，回答出一些不一样的东西，甚至面试官会向你请教，面试变得十拿九稳。本节，我们会定期更新一些论文，当然我记录的都是我觉得对大家，对面试有帮助的论文。你怎么去复习呢：1.你可以通过我的总结，看看这个论文对你有没有帮助，你可以自己去通读全文。2.你可以根据我的总结，掌握论文的精髓和最佳实践，不用看论文，看我的总结，把这些融入面试问题的答案。3. 对于上面各种面试问题，我的一些答案可能引用了论文，当你看到我总结的面试问题的回答时，你可以来这里查看我引用的论文，了解详情，把它当作附录和字典查找。4. 一定要准备几个阅读的论文，即使你不看原文，也要找几篇我总结的论文的概括看一下。当面试官问你，你最近看了什么论文？有什么思考啊？就从我的这个专题中去总结。

1 Agents

1.1 Evaluating AGENTS.md -Are Repository-Level Context Files Helpful for Coding Agents?

1.1.1 概述

越来越多的开发者/公司为代码仓库写一个 `AGENTS.md`（或 `CLAUDE.md`）文件，相当于一份"仓库说明书"，告诉 AI 编程智能体：项目结构概览，用哪些工具（如 `uv`、`ruff`），遵守哪些代码风格，如何运行测试等。OpenAI、Anthropic 等厂商强烈推荐这个做法。截至论文写作时（202602），GitHub 上已有超过 6 万个开源仓库包含此类文件。本篇论文就是在研究，这些东西到底对Agents有没有用？是写的越详细越好吗？有没有可能反而会带来负面影响？

1.1.2 实验方法

论文构建了一个新基准 AGENTBENCH（共 138 个真实 GitHub Issue 任务），选自 12 个小众但有开发者提交的 context file 的 Python 仓库，补充已有的 SWE-bench Lite（300 个任务，这个是目前评测 AI 编程智能体能力的一套测试集）。

对 3个主流 Agent（Claude Code / Codex / Qwen Code）测试了三种设置：

1.None:不提供任何Context File(AGENTS.md CLAUDE.md)。

2.LLM：按照厂商推荐的指令，提供使用AI自动生成的Context File。

3.Human:提供开发者手写的Context File。

测试方法是对每个PR，给出一个测试用例：<问题描述，PR前的代码仓库，测试集，正确答案>（这个测试集和方式也是 SWE-bench Lite的测试集形式，所以通过这个问题你可以学习了解典型的一种测试AI Coding能力的数据集），让AI完成这个PR。比较AI完成后的问题的：**任务完成率，成本（总Token消耗），行为分析（工具调用次数，是否遵循了Context File，有没有运行更多的上下文搜索，做更多的测试等）**

1.1.3 核心发现

1. **LLM 自动生成的 context file 反而有害。**与"不提供任何文件"相比，LLM 生成版本平均使任务成功率下降约 3%，而且推理成本（token 消耗）增加超过 20%，换用更强的模型（如 GPT-5.2）来生成也没有一致改善。
2. **人类手写的 context file 帮助极为有限。**仅带来平均约 +4% 的成功率提升，远低于人们预期。
3. **Agent 确实读取并执行了 context file 中的指令。**比如 context file 提到使用 `uv`，Agent 就会用 `uv`，说明效果差不是因为 Agent 忽略了说明书，而是说明书本身加重了任务难度
4. **Context file 导致 Agent 行为更"臃肿"。**Agent 运行更多测试、查看更多文件、写更多文件。GPT-5.2 的推理 token 增加了 22%，说明 Agent 认为任务"变难了"

核心假设：context file 中包含了大量"非必要要求"，这些额外约束反而让 Agent 在完成核心任务时分心、出错，相当于给 Agent "加了很多不必要的作业"。

1.1.4 核心结论

1. 不要用AI自动生成的Context File, 即使是使用厂商推荐的方式生成。
2. 手写 context file 应只写最少必要信息，比如那些 **AI 不可能自己猜到、但又必须知道的具体命令**，例如：工具指定（使用uv而不是用pip），测试命令（使用Pytest来测试），格式化工具（use ruff来统一格式），仓库特有的命令，脚本。这些东西基本都是能一句话说清楚。**不要写：工程的目录结构（AI自己可以推测出来），宽泛的要求（保持代码整洁，使用代码现有的风格，这些东西AI在训练的时候已经知道），功能/架构介绍（AI自己会阅读代码，自己写这些会增加它的推理难度，比如分析这个描述的架构和自己读出来的代码架构的关系，是否冲突）**

> 论文的核心解释是：每一条额外要求，都是一个额外约束。AI 会认真履行这些约束，但履行约束本身消耗了注意力和 token，使它在完成核心任务时更容易出错。

所以原则很简单：**少即是多。只写 AI 不得不知道的，其余的让 AI 自己探索。**

1.1.5 对面试的帮助

1. Agent评估：看完了本论文，其实你又可以进一步理解Agents评估。比如本文对Coding Agents的评估维度是：正确率，轨迹路径（调用工具次数，是否遵循了指令，是否阅读了更上下文，做了更多次数），成本(总Token消耗，推理Token消耗)
2. Agent测试集：业内常见的一种对**Coding Agents**评估的测试集：SWE-bench Lite。 它是什么，它的思想是什么?
3. 提示词工程：什么该写，什么不该写？为什么有时写多了反而效果变差了。我以阿里夸克千问二面的面试问题（请搜关键字去看对应部分）。在我读完这个论文，总结了这个笔记后，我对于当时面试官的问题：如何做提示词的裁剪压缩，就有了一些新的理解，也去扩展了相关答案。如果现在我去面试，我就会说，我看了xx的论文，他说了xx，所以我们应该xx。 所以其实读完，立刻显现出了它对我们面试带来的收益。

1.1.6 阅读建议

不是特别建议去读原文，甚至去看实验方法和表格。读核心结论和原则就够了，当成一个工程类型的最佳实践规范来阅读。

1.1.7 原文

> `https://arxiv.org/abs/2602.11988`

1.2 Attention Residuals

Kimi 团队发布的论文《Attention Residuals》（注意力残差）提出了一种对深度学习底层架构的根本性创新——**用注意力机制替换沿用十年之久的传统残差连接。**该论文一经发布便在全球 AI 圈引发轰动，在 X（原 Twitter）上获得近 500 万浏览量，吸引了马斯克、Karpathy（OpenAI 推理模型之父）等顶级大佬的关注与点赞。OpenAI 研究员 Jerry Tworek 甚至惊呼"我们应该重新考虑之前的一切，**深度学习 2.0 的时代即将到来**"。甚至是Kimi公司的估值都从中受益，已涨至 180 亿美元。

本身在大模型圈引起轰动的论文就值得我们去学习和关注。另外这个论文是来自中国KIMI团队发布的，更应该引起我们的重视。中国团队比较轰动的论文更该引起我们的注意，这几年来，上一次中国团队的比较轰动的研究属于DeepSeek:

DeepSeek 的贡献在于开源模型本身（R1 等），冲击了算力垄断叙事。 （DeepSeekR1的训练我笔记总结的有，我依稀记得我之前拿去面试讲解这个训练方法，对于应用岗位来说是很加分的，因为面试官不懂，觉得你很厉害。不过对于算法岗来说，我估计是基操了。）

我把这一次KIMI的发布，比作中国 **AI 研究震动全球的两面旗帜，所以我是希望大家都能关注的。**

如何去学习呢?

> 1．首先我的笔记会以基本上大家都看得懂的形式去讲解，本身是一个深度学习网络结构的研究论文，但我们不是深度学习背景出身的，我们不去探讨公式，掌握它的思想。**这一点所有人，甚至是产品，你都可以去理解。**
>
> 2．对于开发出身的同学，我们这里带一点算法的知识，比如残差网络。这是`Transformer`中使用的经典结构，即使你是开发，我们一直强调的是多少你还是要知道一点比如模型的结构，算法的知识。最好慢慢向算法深入，所以我们这里带一点网络的知识，我相信开发出身的同学可以看懂，也为我们后面深入学习算法打下一点基础，留一点印象，毕竟学习不是一蹴而就的，而是循序渐进的。**这一部分建议开发的同学可以理解一下网络结构。**

3. 最后文档总结一些面试的相关问题，你可以拿去说。 思路还是那样，既然我们学了，我们就可以看时机把他讲出来，主动展示给面试官。**这一部分是所有人，产品/开发都可以去学习，背诵，面试时说的。**

1.2.1 背景知识 - 残差网络

因为这个论文是对残差网络进行了改进，所以我们需要先了解一下残差网络是什么。我们不讲深入，因为这不是深度学习的章节，尽量让大家都懂，如果你看不懂，可以结合着AI，搜一下资料理解一下。

1.2.1.1 传统残差连接概念

2015 年由微软研究员何恺明提出，核心思想极其简单：**对任意层的函数变换，无条件地将原始输入加回输出。**

直观效果是不管中间的函数 F 做了什么变换，输出至少保留了输入的原始信息，从而极大缓解了梯度消失问题。网络不再学习完整的映射，而是**只学习残差（差值）**，数学直觉告诉我们这更容易学习。

这个结构从原始 Transformer 架构至今基本没有变过——Transformer 中每个子层（Multi-Head Attention、Feed Forward）后面的 **"Add & Norm"** 就是残差连接的体现。（以下这个如雷贯耳的Transformer架构图，我们可以看到中间的**Add & Norm 就是残差连接)**

1.2.1.2 传统残差连接的两大致命问题

传统残差连接的两大致命问题：**信息稀释**和**隐藏状态爆炸**。

第一个问题是信息稀释，也就是梯度消失。信号在网络中逐层传递，经过几十甚至上百层之后，早期层提取到的关键特征会被不断冲淡，就像一壶好茶被倒了99次之后几乎没有味道了。这导致深层网络根本无法有效利用底层最原始、最重要的信息。

第二个问题是隐藏状态爆炸，也就是梯度爆炸。为了补偿信息在传递过程中的衰减，后面的每一层都会拼命放大自己的输出，结果数据像滚雪球一样越滚越大，最终膨胀到不可控。这就好比水管里的水压不断升高，最终把管道撑爆。在千亿级参数的大模型训练中，这表现为梯度分布极不均匀，模型极度不稳定，甚至直接崩溃，几乎成了限制大模型往更深层发展的"物理魔咒"。

1.2.2 Kimi 的核心创新：注意力残差

1.2.2.1 灵感来源——时间与深度的对偶性

论文中最优美的洞察在于发现了一个**对偶关系**：

在**时间维度**（序列建模）上，旧方法是 RNN/LSTM 的逐词累加传递，新方法则是 Transformer 通过注意力机制进行加权求和——这一演进已被证明极其有效。

而在**深度维度**（层间信息传递）上，情况惊人地相似：旧方法是传统残差的逐层累加传递，Kimi 提出的新方法则是**注意力残差——用注意力机制对所有前序层进行加权聚合。**同一套"用注意力替代逐步累加"的升级逻辑，在两个不同维度上都产生了巨大的效果提升。

正如论文原文所写：

> *"时间与深度的对偶性——就像 RNN 在时间维度上的作用一样，残差连接在深度维度上将所有先验信息压成一个单一状态。在序列建模中，Transformer 通过用注意力机制取代循环机制从而改进了 RNN……我们在深度上提出了同样的方法。"*

Karpathy 的评价精准概括了这一思想：

> *"LSTM 就是将 residual 旋转 90° 得到的。事实证明，注意力机制同样是可以旋转 90° 的。我们是不是没有充分理解 'Attention is All You Need'？"*

（大家理解一下这个旋转90度的概念，其实就是我们把时间上的处理作为横轴，我们引入注意力机制。现在是在网络结构的深度上，我们做手脚，也是引入了类似注意力机制的能力，大家把这个叫做**注意力机制的旋转90度**）

1.2.2.2 方法详解

传统残差中，第 l+1 层只能接收第 l 层的输出并简单相加。**注意力残差则让每一层拥有"回头看"的能力**：

- 每一层会发出"寻物启事"，到**前面所有层**中寻找相关特征
- 通过注意力机制为不同层分配**不同权重**，选择性地聚合最有用的信息
- 例如：第 50 层可以给第 2 层分配 0.8 的超高权重，**越级提取最原始特征**，彻底解决信息稀释问题

1.2.2.3 分块策略——解决计算开销

如果每一层都对前面所有层做全局注意力，通信量会瞬间爆炸。Kimi 团队设计了**分块策略**：

- 将模型分成若干大区（如每 8 层一个区）
- **区内**：依然使用传统逐层累加并总结出代表
- **跨区**：启动全局注意力筛选调度

这一设计使得内存开销断崖式下降，**推理延迟仅增加不到 2%**，与 QKV 分组等已有高效注意力思想异曲同工。

1.2.2.4 结果

Kimi 团队在 480 亿参数的模型上验证了注意力残差的有效性，结果非常亮眼。在训练效率方面，相比传统残差提升了 **1.25 倍**，达到同等性能可节约近 **20%** 的计算量。在能力评测方面，考验逻辑能力的 GPQA 多步推理测试中性能提升了 **7.5 分**，代码能力也大幅提升。而这些收益的代价极小——推理延迟仅增加不到 **2%**。

对 AI 公司而言，这意味着可直接节省数千万美金的训练成本，训练周期缩短数周到数月，超大规模模型能稳定收敛。

1.2.2.5 行业启示

回归第一性原理，敢于质疑基础

残差连接这一沿用十年的"老祖宗"，由于过于基础，渐渐形成了"它就该这样"的固有思维。Kimi 论文证明：**最基础的部分往往也是最容易被忽略的部分**，越是"理所当然"的东西，越值得重新审视。创新不仅在上层应用和花哨技巧上，**动地基往往能带来颠覆性突破。**

跨领域迁移的思维方式

注意力机制已在时间维度（序列建模）上被验证有效，Kimi 团队的洞察在于将**同一原理迁移到深度维度。**这种跨领域迁移能力——发现不同场景下公式/结构的相似性——目前仍然是人类独有的直觉优势。正如晓辉博士所评论："如果你没有对数学公式之间关系的直觉，就没有这样一次突破的机会。"

中国 AI 的范式转变——从追赶到引领

这篇论文打破了"搞 AI 必须砸几百亿美金大力出奇迹"的硅谷叙事。Kimi 团队**不靠算力碾压，靠精妙数学重构底层架构**，且成果免费开源，给硅谷疯狂堆叠算力、建立高资金壁垒的商业模式带来巨大挑战。正如海外科技博主 Tuki 所说："AI 竞赛已不是中美之争，而是闭源和开源之争，闭源正在输掉比赛。

1.2.3 面试问题

我们学了就不白学，学了就是拿去面试问的。也许面试官很少会直接问题注意力残差的概念，但是我相信很多机会，你可以巧妙地用这个素材回答，这就是你展示自己，秀亮点的机会。

1.2.3.1 Q1：最近有看什么论文吗?

最近读了 Kimi 团队的《Attention Residuals》。这篇论文动了深度学习十年没人碰的底层架构——残差连接。传统残差是逐层把输入加到输出上往后传，层数一多，早期信息就被稀释了。Kimi 的做法是让每一层通过注意力机制回头看所有前序层，按重要性加权聚合，而不是无脑累加。核心灵感来自一个对偶关系：Transformer 在时间维度上用注意力替代了 RNN 的逐步累加，Kimi 把同样的思路旋转 90°，应用到了深度维度上。实测在 480 亿参数模型上，训练效率提升了 1.25 倍，多步推理提升了 7.5 分，推理延迟只增加不到 2%。

1.2.3.2 Q2：你怎么看中美 AI 的差异?

我觉得现在已经不能简单说谁领先谁落后了。之前硅谷的叙事是"大力出奇迹"，拼的是谁能砸更多钱买更多卡。但最近的趋势很明显——DeepSeek 的 MoE 架构、Kimi 的注意力残差，都是中国团队在底层架构上做出的原创性突破，靠的不是算力碾压而是精妙的数学设计。而且这些成果都是开源的，实际上在倒逼整个行业重新审视"堆算力"的路线。我认为未来 AI 竞争的核心不是中美之争，而是谁能在架构效率上走得更远。

1.2.3.3 Q3：你对未来大模型架构的发展怎么看?

我认为架构创新会重新成为主旋律。过去几年大家主要在 scaling law 上做文章，靠扩大参数和数据量来提升性能，但这条路的边际收益在递减。Kimi 这篇论文给了一个很好的启示：残差连接这种十年没人动的基础组件，优化之后就能省 20% 的计算量。说明底层架构里可能还藏着很多类似的低效地基，值得用第一性原理重新审视。未来可能会看到更多对 Transformer 各个基础模块的重新设计，而不是单纯地把模型做大。

1.2.3.4 Q4：能简单介绍一下 Kimi 注意力残差的具体机制吗?

传统残差连接中，每一层的输出等于该层变换结果加上该层输入，信息只能逐层往后传。注意力残差把这个过程改成了：每一层生成一组 Query，去和前面所有层的输出做注意力计算，按相关性分配权重后加权聚合。这样第 50 层可以直接给第 2 层一个很高的权重，越级提取早期特征。为了控制计算量，用了分块策略——每若干层分成一个区，区内正常累加，跨区才做全局注意力，内存开销大幅下降。

1.2.3.5 Q5：你觉得这篇论文最大的启发是什么?

最大的启发是"跨维度迁移"这种思维方式。注意力机制在时间维度上替代 RNN 已经是十年前的事了，但十年来没人想过把同样的思路用到深度维度上。Kimi 团队发现这两个方向的数学结构几乎一模一样——都是把逐步累加替换为全局加权聚合——然后就做出了突破。这说明很多时候创新不是发明全新的东西，而是发现已有方法在新场景下的适用性。对我自己的工作来说，也提醒我多关注不同领域之间结构上的相似性，而不是只在一个方向上深挖。

1.2.4 参考资料

论文原文：https://www.mlpod.com/1473.html

视频讲解：马克老师专门对这个论文进行了讲解，他的视频是那种通俗易懂，给小白讲的，一共就不到10分钟，可以看看：https://www.bilibili.com/video/BV1MMw1zaESW/?spm_id_from=333.1387.homepage.video_card.click&vd_source=144bec9c3f54e465073138bed788be1b

2 更新记录

2025/12/07

1. 学习路线中增加自学转行如何确定项目，设计项目的原则，怎么完成自己的项目？
2. 补充12月小红书大模型岗位一面内容，更新完了问题部分，反思和知识部分待整理。

2025/12/08

更新完善小红书面试对应问题答案。 答案部分快总结完了，还差一点，明天可以完成小红书的答案部分。

2025/12/09

完成小红书一面笔记的答案部分，开始更新字节面试笔记（字节面了两个部门，两个都会整理）

2025/12/10

整理完字节大模型工程岗， 一面面试问题

2025/12/11

基本完成字节大模型工程岗， 一面面试问题对应答案。根据面试情况完善了MCP笔记八股部分，八股部分新增A2A协议的概念（因为出现过两次了，说明A2A协议也是要准备的）

2025/12/13

更新Multi Agent对应的知识。 在之前的参考资料中，我给大家推荐的《Agentic Design Pattern》中的Multi-Agent这一章，但是这一章的讲解不够深入，特别是对于架构设计这一块也比较浅，这是在针对小红书的面试的时候，这些知识显得不够。

我将Multi-Agent相关的知识更新，替换成《Towards a Science of Scaling Agent Systems》这篇论文相关的内容，补充了相关知识，参考材料和思路，让相关内容更加权威，准确，深入。（没总结完，总结更新ing）

2025/12/14

完善了多Agent 八股部分，采用更权威论文作为知识总结来源，也完善了小红书一面对于多Agent架构问题讨论的回答。

添加面试讲解视频：

小红书一面，滴滴一面，安克科技一面，灵动时刻一面，京东一面，京东二面

2025/12/17

完善了下Langgraph的八股，加了一个很全面的Langgraph知识的参考文档，是个很全的Langgraph知识大全，遇到不懂的可以直接查这个文章。具体文章链接看Langgraph八股部分。

添加了多Agent的通信方法和状态管理，也是面试考点。

总结了阿里国际站的问题和反思，答案和解析部分未完成。

优化常见Agent框架知识：按照目前最火的5个架构整理，忽略其它top5之外的框架

2025/12/18

更新完阿里国际站一面答案部分，根据面试情况，更新了Langchain版本相关的八股内容和参考资料。

2025/12/20\~21

重构Agent性能评估八股部分，包括**参考资料**，**常见的问题**，以及**答题思路和准备方法。**

完成**京东三面面**试题答案和面试思路，上传 对应**讲解视频。**

今天我通过京东这个面试复盘，彻底重新梳理了Agent 性能评估这个超重要考点，在京东三面讲解视频中也着重讲这个点如何复习，准备策略，相信通过这一面的知识你能把这个点弄透。

上传了阿里国际站 电话面讲解视频

2025/12/23

整理字节一面面试问题

2025/12/24\~25

整理字节一面面试的答案，并根据面试内容，调整八股内容：

1. 微调方法论部分，加入参考文档，详见八股（你是怎么做微调的）
2. Multi-Agent八股部分，增加Single/Multi agent的对比
3. 添加八股 你在做项目中遇到最困难的问题以及如何解决的 相关问题答案和解决思路。

2025/12/28

总结完了夸克千问一面，二面面试问题

2025/12/29

1. 上传字节（豆包）一面讲解视频。
2. 完成夸克千问一面答案，千问二面反思部分。
3. 完善了问题-你在项目中遇到的最难的问题，如何解决的?

2026/1/4

1.完善多Agent八股部分，设置第四小节，多Agent的架构设计题目，结合笔记给出的参考论文例子和笔记的面试内容，来汇总架构设计题目和答案。

2.总结完成夸克千问二面问题和答案。

2026/1/5

录制并上传夸克千问二面讲解视频。

2026/1/5 \~2026/1/8

主要在做项目的调研，我开始丰富完善项目部分，今天写完了项目实战的介绍部分。

接下来会继续调研，写这个项目的技术选型和具体方案。

因为我后面会花比较大的精力在项目上，可能更新面试实战部分会慢一些，我这周会把所有面试的公司先都加到目录上，如果有哪个朋友特别需要某一个公司的题目，可以私信我，我先更新。

2026/1/10

1.更新完阿里HR面。

2.把所有未整理的面经素材留了个白，后面陆续整理。

3.更新项目实战，技术方案选型。

2026/1/12

更新项目文档：项目概述和核心特点部分

RAG分块内容加入一个讲解参考视频

2026/1/13

更新项目文档：项目概述和核心特点部分 3.1 RAG核心流水线设计。

2026/1/15

更新项目文档：MCP服务设计

2026/1/16

更新项目文档：可插拔架构设计 pro

2026/1/17

新增Skills 的八股部分：概念，和mcp对比，和prompts对比，运行模式，设计理念。

提供参考视频和参考项目

2026/1/21

新增项目Github地址。接下来几天我重心会放在更新面经部分，更新几期面经哈。

2026/1/22

更新完啦Deepwisdom的问题部分

2026/1/23

新增粉丝提问和答疑章节，我把粉丝朋友问我的一些问题和答案汇总在这里，这样大家可以参考，共同进步哈！其中问题部分我已经做了优化，在我的认知里不会有隐私问题，但是如果粉丝朋友觉得这个问题不便于公

开，联系我删除哈。 大家一起进步！

2026/1/24

更新完deepwidsom的问题解析。

优化了八股答案： 你是怎么做Agent评估的，之前有点太AI化了，优化成一个可以直接回答，更口语的答案。

根据面试问题新增八股模型部分：如何判断模型能不能跑，跑的有多快

2026/1/25

更新平安证券一面面经和答案。

八股增加了一个面试技巧环节，更新自我介绍和反问的一些策略。

八股增加模型部分Qwen2.5->3的升级

上传录制视频：项目实战部分skills的使用视频，Deepwisdom 一面讲解视频， 平安证券1面讲解视频

2026/1/26\~2026/1/27

项目进度完成17%, 提交相关代码

2026/2/1

Rag-Agent项目进度45%，目前在写C阶段，C阶段是数据存储阶段，等写完C阶段，就可以有一些可以展示的东西了，它是独立的，对数据处理然后存到数据库中的一个离线模块，也是整个系统比较重要的模块。

更新了平安证券二面相关问题和回答。

Skills 推荐项目部分加了一些链接介绍。

2026/2/7\~2/8

1. Rag-Agent项目进度已经完成84%，目前也录制一些项目相关的视频，放在了小红书上，我会都归档在笔记链接中。
2. 更新一期面经：盛大AI一面
3. 更新八股： 我在笔记中 你的项目遇到的最困难的问题是什么，如何解决的这个问题，又加一个例子。这个例子是结合我们RAG项目写MCP Server总结的。
4. 更新参考视频。在MCP章节，我更新了一个MCP 参考视频，从入门到进阶的视频。这些都是我当时看的。MCP从0基础到精通，把这几个视频看了就差不多了。（另外粉丝朋友说有的人不能科学上网，所以我视频也尽量找的都是中文的视频网站了，虽然这几个推荐视频我当时在油管看的，但还好他们是中国博主，国内也能找到地址，不过还是建议程序员要能科学上网呀，有的资料可能真的必须要到外网去看了）

2026/2/10

更新项目视频

2026/2/24

休假结束，开始更新啦。今天更新了一个大章节。大模型岗位介绍。

2026/2/25

全面优化补充Skill 章节笔记，加入更多内容，参考文档，润色内容。

2026/2/28

笔记更新了简历编写技巧和面试准备建议

另外dev分支更新了自动编写简历的SKILL(暂时没有同步到其他分支），感兴趣可以先试。

2026/3/1

更新项目相关视频：介绍test skill和resume skill如何写的，视频里讲了项目SKILL如何写的，SKILL设计思路，以及使用这个项目面试的一些建议。

更新了盛大AI面试的讲解视频。

2026/3/6

项目更新了setup SKILL和对应视频：大家使用它可以一键配置环境。

项目更新了项目复习和模拟面试SKILL以及对应视频： 大家在面试前通过它，可以快速突击，掌握核心知识。

项目更新了完整的README：提供了一些说明，不同的人群如何使用，常见的问题的说明。

整理了项目，DEV, Main,Clean Start相关的分支整理完毕，请大家按需使用。

2026/3/7

更新了Openclaw常见的技术细节和框架，掌握这一小节，你将知道Openclaw到底为什么这么神奇，如何实现的？他不是魔法，而是一个合理的框架。

最后总结了8个Openclaw面试问题，快问快答，帮助大家检验是否掌握。

最后也给出了参考视频：如何配置Openclaw以及openclaw技术讲解。

Openclaw，面试必看哈\~！

2026/3/11

主要在学习pytorch， 我准备把这些变成一套超级叼（自认为）的SKILL，里面包含了我的复习路线，参考视频，到时候大家学习复刻我的路的时候，直接用就好了。

然后就是会出一些论文专题，因为论文特别重要，你们这周会看到笔记会多一个论文阅读专栏。

最后就是RAG项目的真题，已经有粉丝朋友和我反馈他用这个项目的面试真题了，我也真的希望大家也可以反馈给我，这样我都会整理更新在笔记中，帮人帮己。当然我自己也会去面试的，但是我太忙了，有太多东西要更新了，等我反馈RAG项目真题可能比较慢了。

2026/3/12

增加了笔记的新的篇章：论文部分。

写了为什么要读论文的思路，也总结了我们这个部分的第一篇论文：《Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?》

2026/3/13

更新了笔记项目部分的一个 面试实战和问题解析。

2026/3/14

笔记设计思路和大模型学习路线彻底大重构。

我之前写这个学习路线的时候是25年12月，现在有了更新的技术，思路，所以路线值得重构。

2026/3/15

1.更新SKILL学习路线，新增一个推荐视频：某个博主讲如何写SKILL的实战。

2.更新RAG进阶内容：包括Graph RAG, Agentic RAG, 知识图谱。

3. 新增RAG项目面试真题：离线解析部分相关面试真题。

2026/3/17

1. 更新Openclaw的记忆模块解析，我认为这里很重要，因为记忆功能面试必考，Openclaw本身也值得学习。
2. 优化八股：Agent记忆是如何实现的。直接加入Openclaw实例，结合经典知识讲解，面试官问到直接秒杀。

2026/3/19

RAG项目新增两个大厂面试真题及解析（感谢粉丝朋友的投稿！！！！）

2026/3/20

抱歉拖延久了，上传了Pytorch/深度学习入门的SKILL，在大模型算法0基础学习全记录，2.1小节。需要的朋友请自己取

2026/3/21

上传了联想大模型岗位一面面经记录。

2026/3/22

上传了华林面试一面面经揭露

2026/3/23

添加了Harness工程的内容，有几个同学反映这是面试会考察的内容。

另外新增了AI通用素养章节，我将SKILL, VibeCoding等技巧放在了其中。因为我相信这是新时代对每一个人的要求，它不属于大模型应用开发，算法等大模型岗位，而是每一个程序员，甚至是产品在新时代必须掌握的技巧。

2026/3/24

论文部分，添加了对最近最火的论文：Kimi的注意力残差的解读。

2026/3/28

在AI通用素养章节，加入Claude Code从入门到精通：涵盖Claude Code的概念，基本使用方法，最佳实践建议，面试问题和参考视频

2026/3/29

更新Openclaw 多Agent框架解析/多Agent之间的通信机制以及相关面试问题，这两个内容对应着面试的高频考点，我们希望能从Openclaw这个架构中找到一些思路和启发。

2026/3/30

总结两个粉丝朋友提供的大厂项目面试真题，两个都是大厂，国内排名前10的公司。

2026/3/31

补个SKILL的推荐视频，讲解的是如何去写一个SKILL，作者的核心思路是FeedbackCycle，写Skill分成这四步，我是很认同的。通过他这个视频，他告诉你如何去写SKILL，通过实践讲解。.

2026/4/1

更新华林二面的视频。

2026/4/9

旅游回来了，继续给大家更新。最近更新一下Claude Code源码，架构的内容。粉丝已经催了很久了，毫无疑问这个是最近超高频考点。大家要有这个意识，大模型岗位，对于那些爆了的热点知识，一定是最近常考的。

拆解Claude Code源码，和它相关的发现了一个Clawcode的项目：它是在ClaudeCode泄露后，使用Harness工程把CC重写了一遍，Github创下了星星数增长最快的记录，它的harness工程值得我们学习。所以今天先更新了CC的源码泄露背景和对这个项目的拆解，希望大家通过这个项目可以学一下Harness工程，也是26年高频考点。

2026/4/10

添加ClawCode 5层harness架构的讲解视频（小红书传了，笔记也会附上夸克网盘链接，没看到就稍微晚一天再查看）

2026/4/11

添加Claudecode的记忆功能解析以及添加相关面试问题/答案参考。

2026/4/12

解析了ClaudeCode的架构，将Claude Code的架构和Openclaw去对比，包括和我们笔记部分，Agent之间通信方法联系起来。希望通过这个笔记，大家理论联系实践，对理论有更好的理解，也对Agent架构有一定理解，这些对自己设计Agent框架，以及相关面试问题都很有帮助。

2026/4/13

上传了ClaudeCode源码解析 - 记忆模块 讲解视频。

2026/4/17

更新ClaudeCode源码解析 - 四层安全模块。

同时上传了CC的源代码，在源码解析章节，大家自取.

2026/4/20

更新了自学算法章节，上传了第二阶段的SKILL。后训练学习SKILL。阐述了使用该SKILL的技巧和思路。

2026/4/21

更新了Hermes Agent的知识部分和相关面试高频考点。Hermes工程是26年4月底的顶流，面试常考。

2026/4/22

更新Hermes Agent的讲解视频。

2026/4/23

总结了三个粉丝朋友投稿的RAG项目面试真实的问题和答案。并且根据这个真实面试题，补充了2个八股：

1. Chunk size的选型策略和思路。
2. MCP Server的tools 设计的原则。

公开了RAG面试真题前面几期面试公司（当时鉴于隐私，把公司名隐藏了，现在小伙伴都上岸了，我把这个公司名公开了）

另外笔记有个小优化，今天笔记中的互相引用的片段做成链接，方便大家跳转更方便。等我空了逐步修改过来。

2026/4/24

更新了华林三面的面试问题和解析。我主要想通过这次面试来讲一下架构设计面试问题，提供一个模板供大家套用。

2026/4/26

笔记加了一个小小的彩蛋，既然是彩蛋，我就不说是在哪里了，成立了一个新的章节。希望通过这个章节可以鼓励到大家。感兴趣自己翻翻吧。这两天开始学习算法的第三阶段了，开源项目的学习，所以没有更新笔记，但是随着算法学习，也离提供一个算法的项目越来越近了。

2026/4/29

1. 下一个就是你的彩蛋环节加了一个例子。
2. 我把文本里我找到的所有文本内相互引用的地方变成了超链接的形式，帮助大家能快速上下文跳转。如果有没改过来的或者有问题的地方，请大家和客服反馈，我会及时修改。

这两天没更新笔记是因为我在学算法第三阶段，更快学完更快开始做算法项目分享给大家。

2026/4/30

1. 下一个就是你的彩蛋环节加了一个例子。
2. ClaudeCode源码解析章节，添加ClaudeCode的成本控制功能讲解。这一部分对于理解面试问题：你的项目怎么做Agent成本控制，如何避免Agent无限调用烧爆token，以及对于设计自己的项目，都是有很好的借鉴价值。

五一期间可能不会有太多更新，好好放个假吧，祝大家假期快乐。

2026/5/6

在整理按摩房预约系统的项目，很多朋友问我能不能分享。这个项目是我自学大模型开始自己做的第一个项目。我想着就分享给大家，我在整理相应的代码，SKILL，真题，以及如何使用，相关配套的面试真题，预期在5.8号左右整理完。

2026/5/7

成立一个所有人都能看到，编辑，反馈的文档（在文本最后）。我希望在这个文档上和大家去交流，我会在上面写上我的长期更新计划，大家也可以补充有什么面试问题（比如文档上没有的，希望补充的），提供文档的反馈，意见，错误纠正，好的参考资料，以及其他一切内容（因为大家是都可以共建的），集合团队力量，一起获取胜利。

2026/5/8

按摩房预约项目部分文档已经更新完了，源代码也上传了，怎么使用等我的面试心得都写上去了。

我还需要对这个按摩房预约系统录制一个视频讲解，项目部分就到此结束了。

接下来会更新个面经，粉丝提议的模型对比，私有部署，以及主线内容是算法，推出算法项目。

2026/5/9

Agent项目部分传了个视频，项目部分到此结束。我会把之前的一些视频都换成b站链接，后续也都会用b站代替，有粉丝朋友反馈夸克链接不方便。

2026/5/10

更新了不同模型的对比，选型策略，deepseekv4的核心特性，结合了面试真题讲解。我觉得这部分还是值得大家了解的。

2026/5/11

添加了不同模型对比的文档对应的讲解视频

2026/5/12

详细分析了问题，部署/训练某个模型时，需要多少显存和空间。大模型社群中有朋友反映这个问题面试官问了，不知道怎么回答。这个问题有点难，涉及知识点有点广，我也会配一个讲解视频。

2026/5/13

更新部署模型讲解视频

2026/5/17

更新2026.05 字节跳动暑期实习一面新鲜面经。

重构大模型笔记：幻觉部分，力争完整，前沿，增加了一些亮点设计。

2026/5/18

更新2026.05 字节跳动暑期实习一面新鲜面经视频讲解。

2026/5/19

重构了Harness章节的内容。添加了2个参考视频和参考文档。

同时新增了一个Harness工程的一些工程实战细节，我觉得非常有帮助，不管是对面试还是自己使用AI，还是开发Agent项目。在这个笔记的过程中，也结合了非常多的面试问题来讲解这一小节笔记如何在面试中运用。

同时在Harness章节，也增加了如何学习Harness的一些思路介绍。相信重构后，大家不管学习Harness内容还是准备相关面试问题，都会更加容易

2026/5/20

更新了笔记推荐阅读顺序建议。

2026/5/21

经验之谈，更新如何找项目

2026/5/24

更新腾讯AIAgent暑期实习一面面经和解析

2026/5/26

更新腾讯AIAgent暑期实习一面面经和解析 讲解视频

2026/5/27

更新重构ClaudeCode的源码解析，关于ClaudeCode的架构部分：增加了AgentTeam部分的讲解。

在ClaudeCode的笔记部分，加入了CC的Agentteam和SubAgent的使用介绍。

在ClaudeCode章节，加入了若干参考资料：Agentteam和SubAgent 的使用视频讲解，官方文档，CC源码解读书籍在线阅读链接。

2026/5/28

自学大模型算法章节，完整的三个阶段的SKILL已经更新。这个是我自己学下来，掌握大模型算法（后训练）部分的理论基础使用的方法，觉得合适的同学可以直接复用。我接下来准备做算法项目了。确实我是用这一套从0算法基础到现在（虽然也不是什么算法高手，但是最少后训练，pytorch等整个算法的理论基础是学完了的）的一个水平。

2026/6/2

更新了虾皮暑期实习一面面试深度解析。

根据这一次面试，补充了ClaudeCode源码解读的笔记部分，新增全局图片，Transcript部分，以及将笔记部分和面试题融合。增加了相关的参考资料，比如Langgraph的持久化知识参考链接。

2026/6/2

更新了虾皮暑期实习一面面试讲解视频

2026/6/7

更新了ClaudeCode vs Codex工具测评，对比，选型建议，面试问题。

因为最近Codex的热度起来了，很多人说Codex更好用，了解这一章节，有助于提升我们的VibeCoding的技巧，在我们和面试官聊VibeCoding工具时，心中也更有料。

2026/6/8

更新CC和Codex对比讲解视频

2026/6/9

我主要在写算法章节的设计文档，这一部分的思路希望这个文档能够不光展示技术细节，也展示如何写的，你如何复用。我在写这一部分的过程中不会往文档里写，会在我设计完细节的时候一次性写入笔记中。如果想要现在就看的同学，可以看我小红书的笔记:算法设计章节，这个里面展示了设计的中间过程。

2026/6/10

分享了一个提示词和技巧，用于将面试录音快速变成面试问题记录。放在上面的经验之谈里。

2026/6/11

替换夸克千问面试讲解视频变成b站链接。

2026/6/14

新增了安克暑期实习一面的面经和详细解析，根据面经内容重构笔记，具体包括：

RAG部分加入RAG整体流程图和描述，新增意图识别步骤内容。

CC源码压缩部分新增整体压缩的流程图，重构压缩部分讲解。

将本次面试真题问题融入到笔记中。

2026/6/15

新增安克暑期实习一面面经讲解视频

2026/6/16

新增研究院大模型开发岗位面试全记录，包括所有面试流程，注意事项，具体放在面经章节。

2026/6/17

Agent章节新增Loop 工程的内容，是最近比较火的概念，补齐相关内容防止面试官会问。

2026/6/21

新增一个Agent实战例子：PI Agent. 添加该项目完整的架构讲解，思想设计，面试问题，课后作业等。

同时更新学习思路部分：推荐该项目作为学完Agent理论一个开始上手实践的Agent项目。

2026/6/22

添加Pi-agent讲解视频

2026/6/25

更新米连科技（社招）面经

2026/6/28

彻底重构了整个Agent评估内容。参考了3本书籍和一篇论文，按照Agent开发的生命周期讲解，插入每个地方需要评测的内容。确保评估点完整且符合逻辑。笔记里掺杂了十几个面试真题供大家参考。同时给出了参考资料供大家深入学习。相信这一部分能够讲透，覆盖清楚Agent评估常考的问题。

2026/6/29

添加Agent评估的讲解视频

2026/6/30\~2026/07/02

加了几个上岸记录（其实中间还有很多人上岸，我太懒了，没有及时记录）

然后这几天笔记虽然没有更新，但我一直也没闲着，我在设计我们算法项目的spec，马上设计完了。我会讲算法的项目对应的技术文档直接添加在笔记中，更新我们的笔记章节，有需要的可以先看起来。

2026/07/05

SKILL章节添加面试实战内容，总结梳理了粉丝朋友提供的近期SKILL面试相关真题和解析，以及对应视频讲解。

2026/07/06

添加算法项目技术文档

2026/07/07

更新SKILL面试真题对应的讲解部分。

2026/07/08

添加一个Agent场景题, 社招面试真题。

2026/07/09

添加RAG趋势讨论的问题。包括比如你的项目要不要用RAG啊？ClaudeCode为什么不用RAG啊？RAG是否过时了啊？你怎么看待RAG已死这个问题啊。今天补充的内容，大家读完了以后会对这些问题有深入的理解。

2026/07/10

添加RAG趋势探讨部分笔记的讲解视频

2026/07/11

添加Agent场景题部分笔记的讲解视频

2026/07/13

上传了大模型算法项目一阶段的代码：主要完成了对应的项目框架的搭建，推理模块的基本架构。

同时在笔记中整理该项目推荐的学习方法。这是阶段性的交付，每个阶段我都教给大家如何去完成，怎么去学习的。你真的不用等我完成整个项目后再来理解，可以跟着我写的步骤，阶段性学习。

2026/07/14

上传了大模型算法项目一阶段的讲解视频。

2026/07/15

上传了大模型算法项目二阶段的代码：完成了Agent的评估模块。

这块其实适合于任何人，即使你不做大模型算法项目，这个阶段也值得学习。因为应用开发也很常考大模型评估，这块可以作为一个Agent评估内容的实战。

2026/07/16

上传了大模型算法项目二阶段 讲解视频

2026/07/19

新增面经：滴滴Agent开发一面面试题和答案解析。

2026/07/21

增加滴滴Agent开发一面面经讲解视频。

修复了错别字，同时重构目录结构。我希望把我们目录再整理一下，内容越来越多了，还没整理完，今天重构了一部分，希望想个办法比如整理目录结构，加点全局结构图之类的，让大家即使内容一直在增加，看笔记也能很清楚知道结构和应该看哪一部分。

2026/07/22

修复错别字。

新增笔记：使用AI工具时如何节约Token使用量。

2026/07/23

修复错别字。我用AI修了好几版错别字。新增笔记：使用AI工具时如何节约Token使用量 对应的视频讲解。

2026/07/24

岗位介绍部分新添对FDE岗位的介绍。

2026/07/25\~2026/07/26

上传算法项目三阶段代码和讲解视频。

2026/07/27

更新了深圳大模型研究院Agent开发一面解析。同时Agent章节添加了专题讲解，如何应对面试官对你Agent项目的质疑。在Agent框架知识部分，也加入了一些根据26年7月份的经验，加了一些学习深浅的建议。

2026/07/28

更新深圳大模型研究院一面讲解视频

2026/07/29

更新最近很火的概念，Graph工程的笔记。

2026/07/30

更新最近很火的概念，Graph工程的笔记。

2026/07/31

添加Graph工程讲解笔记。

2026/08/2

更新2026年有价值做的6个AI项目，包含简历如何编写，美化。

2026/08/3

更新2026年有价值做的6个AI项目，包含简历如何编写，美化对应讲解视频

2026/08/5

完成大模型算法后训练项目第四阶段，提交相关代码。

2026/08/6

上传大模型算法后训练项目第四阶段讲解视频。

2026/08/7

在项目实战部分，加入了我们目前已有的三个项目该如何使用，如何学习，包装，写在简历上，给出了简历编写思路和示例简历。

2026/08/9

Agent章节新增专题，Agent健壮性设计。这个也是面试常考的点，完成了内容：当工具调用失败时，如何优雅处理?

2026/08/10

更新Agent健壮性设计视频讲解。

2026/08/11

重构Agent框架部分，新增包含2026Agent框架特点解析，选型策略，面试真题。

2026/08/12

新增Agent框架部分对应的讲解视频。

2026/08/14

完成算法项目第五阶段，上传代码。

2026/08/16

新增英伟达Agent开发一面面试笔记。

2026/08/18

新增英伟达Agent开发一面面试内容讲解视频。

2026/08/19

新增算法项目5阶段讲解视频。

2026/08/20

新增DeepSeekHarness内容解析，Cordis特性讲解，架构图，面试问题。

2026/08/21

新增DeepSeekHarness内容解析，对应的讲解视频。

2026/08/24

完成项目第5阶段，更新算法项目相应章节。

2026/08/25

上传项目第5阶段对应的讲解视频

2026/08/26

新增RAG项目对应的两个粉丝投稿的面试真题及相关子问题。

如何保证RAG系统生成的回答是准确的?

PDF 转 Markdown 是怎么处理的?

同时在RAG专题，新增对应的RAG知识：

RAG的文档输入处理

Self-RAG

2026/08/27

新增上面的RAG项目两个面试真题对应的讲解视频。

2026/08/28

新增宇树科技Agent开发一面面试真题精讲笔记。

2026/08/29

新增宇树科技Agent开发一面讲解视频。

2026/09/01

彻底重构RAG分块章节，总结面试问题，选型策略，汇总近10道面试真题。

2026/09/02

RAG分块章节，加入视频讲解。

2026/09/03

开辟我们后端章节！完成导引部分，加入后端章节第一个面试问题和讲解资料。

2026/09/04

更新后端章节对应的讲解视频。

2026/09/07

完成算法项目，上传第七阶段(最后一阶段）的代码，笔记更新响应内容

2026/09/08

上传算法项目，上传第七阶段(最后一阶段）的讲解视频

2026/09/09

更新202607宇树科技二面答案解析。

2026/09/10

更新202607 宇树科技二面讲解视频

2026/09/14

更新简历编写原则。

新增面试记录章节。

2026/09/15

上传简历编写原则章节讲解视频

2026/09/16

更新面试前突击准备方法，技巧，预测问题

2026/09/17

更新面试前突击准备方法 对应讲解视频

2026/09/18

更新MCP 新特性，趋势，这个是选自阿里秋招大模型开发一面的三个面试问题，根据该面试问题，总结更新MCP章节的知识。

2026/09/21

更新2026阿里淘天秋招一面面经及解析

2026/09/23

添加2026阿里淘天秋招一面面经讲解视频

2026/09/24\~10.7

国庆中秋期间，主播请假出去玩了，更新会放慢甚至更新，感谢大家理解，祝大家假期快乐。

假期更新记录：

更新了Jev 模型的文档

2026/10/08

更新了面试前突击章节-项目上线设计部分。主要讲了简历里的项目，如何能够包装成企业级的生产项目，面试会问哪些问题。

2026/10/09

更新了面试前突击章节-项目上线设计部分的讲解视频

2026/10/12

更新了 阿里淘天秋招Agent开发二面问题解析

2026/10/13

更新了 阿里淘天秋招Agent开发二面讲解视频

2026/10/14

新增「1.2 2026年 Agent 新形态」：介绍四款代表产品、五大发展趋势，以及面试问题与 ForgePilot 项目设计启发。

2026/10/15

新增「1.2 2026年 Agent 新形态」：对应讲解视频

2026/10/16

新增“3 简历投递技巧”：整理投递渠道、AI 搜索与岗位管理、投递策略及校招/社招时间安排的个人心得。

2026/10/19

补充「1.3 RSI 进展」：梳理概念、四层能力、行业进展与项目启发，配套五张图及面试问答，并关联「2026年 Agent 新形态」参考章节。

2026/10/21

补充「1.3 RSI 进展」章节对应讲解视频

2026/10/22

添加简历投递技巧对应讲解视频。在 OpenClaw 和 Hermes Agent 章节开头添加学习优先级声明（写于2026年10月）：提示面试考察变化，优先学习 Claude Code、Pi-Agent、DeepSeek Harness 和 Codex，原正文保留。

2026/10/23

根据当前面试行情，在 OpenClaw 与 Hermes Agent 章节开头补充当前学习建议。（旧的东西可能考察没有那么多了，我在做笔记的时候会经常标注一下学习建议）

2026/10/24

补充2026.10 拓竹科技社招面试：整理2026年10月拓竹科技社招的 Coding Agent、平台架构、模型训练评估与 RAG 面试问题。

2026/10/25

补充 Computer Use：讲解观察、决策、执行闭环及模型与 Harness 分工，保留原图表，并在前言和面试问题处关联字节三轮面经的一面。

补充2026.10 字节Agent三轮面经：保留来源说明及一面、二面、三面完整记录和面试结果。

2026/10/25

补充2026年10月字节一面解析：会话记忆；后端并发与幂等；数据库与索引；SDD开发工作流；Pi Agent；DeepSeek Harness；Agent产品；Claude Code压缩。知识章节与一面原题建立双向跳转，保留原稿图片与来源说明。

2026/10/27

更新Computer use章节内容对应讲解视频。

2026/10/28

更新字节秋招（202610）Agent开发一面对应笔记讲解视频。

2026/10/29

更新问题解析：

现在ClaudeCode等工具已经很强了，为什么你要重新开发一个Agent?

同时挖掘了算法项目，RAG项目的深度，加深了项目，更新面试后复盘技巧，具体请看这里

2026/11/02

更新SDD开发工作流：更新SDD速成课对应文档，整理SDD的概念，方法论，模板，思路，开源框架等。非常适合0基础的同学20分钟速成SDD 方法论。

2026/11/03

添加 SDD 速成课 对应讲解视频。

2026/11/04

补充Pi 核心层设计：讲解了如何对Pi-agent进行二次开发，以Foregepilot项目为例设计相关pi-agent的架构设计，同时补充对应面试问题

2026/11/05

补充Super-agent 的专题讲解内容：讲解Super-Agent的概念，架构，运作流程，面试真题。